

INTERNATIONAL STANDARD

IEC
61158-6

Third edition
2003-05

**Digital data communications
for measurement and control –
Fieldbus for use in industrial
control systems –**

**Part 6:
Application layer protocol specification**



Reference number
IEC 61158-6:2003(E)

Publication numbering

As from 1 January 1997 all IEC publications are issued with a designation in the 60000 series. For example, IEC 34-1 is now referred to as IEC 60034-1.

Consolidated editions

The IEC is now publishing consolidated versions of its publications. For example, edition numbers 1.0, 1.1 and 1.2 refer, respectively, to the base publication, the base publication incorporating amendment 1 and the base publication incorporating amendments 1 and 2.

Further information on IEC publications

The technical content of IEC publications is kept under constant review by the IEC, thus ensuring that the content reflects current technology. Information relating to this publication, including its validity, is available in the IEC Catalogue of publications (see below) in addition to new editions, amendments and corrigenda. Information on the subjects under consideration and work in progress undertaken by the technical committee which has prepared this publication, as well as the list of publications issued, is also available from the following:

- **IEC Web Site** (www.iec.ch)

- **Catalogue of IEC publications**

The on-line catalogue on the IEC web site (http://www.iec.ch/searchpub/cur_fut.htm) enables you to search by a variety of criteria including text searches, technical committees and date of publication. On-line information is also available on recently issued publications, withdrawn and replaced publications, as well as corrigenda.

- **IEC Just Published**

This summary of recently issued publications (http://www.iec.ch/online_news/justpub/jp_entry.htm) is also available by email. Please contact the Customer Service Centre (see below) for further information.

- **Customer Service Centre**

If you have any questions regarding this publication or need further assistance, please contact the Customer Service Centre:

Email: custserv@iec.ch
Tel: +41 22 919 02 11
Fax: +41 22 919 03 00

INTERNATIONAL STANDARD

IEC
61158-6

Third edition
2003-05

Digital data communications for measurement and control – Fieldbus for use in industrial control systems –

Part 6: Application layer protocol specification

© IEC 2003 — Copyright - all rights reserved

No part of this publication may be reproduced or utilized in any form or by any means, electronic or mechanical, including photocopying and microfilm, without permission in writing from the publisher.

International Electrotechnical Commission, 3, rue de Varembé, PO Box 131, CH-1211 Geneva 20, Switzerland
Telephone: +41 22 919 02 11 Telefax: +41 22 919 03 00 E-mail: inmail@iec.ch Web: www.iec.ch



Commission Electrotechnique Internationale
International Electrotechnical Commission
Международная Электротехническая Комиссия

PRICE CODE **XW**

For price, see current catalogue

CONTENTS

FOREWORD.....	32
0 Introduction.....	34
1 Scope	36
2 Normative references.....	37
3 Terms and definitions.....	38
3.1 Summary.....	38
3.2 Terms and definitions from other ISO/IEC standards	38
3.3 Terms and definitions from IEC 61158-5.....	39
3.4 Other terms and definitions.....	39
3.5 Abbreviations and symbols.....	49
3.6 Conventions	53
3.7 Conventions used in state machines.....	61
4 Type 1	68
4.1 FAL syntax description	68
4.2 Transfer syntaxes.....	107
4.3 FAL protocol state machines structure.....	132
4.4 AP-Context state machine	134
4.5 FAL Service Protocol Machine (FSPM).....	151
4.6 Application Relationship Protocol Machines (ARPMs).....	157
4.7 DLL Mapping Protocol Machine (DMPM)	298
4.8 Protocol options	312
5 Type 2	316
5.1 Abstract syntax.....	316
5.2 Transfer syntax	390
5.3 Structure of FAL protocol state machines	402
5.4 Context state machine	402
5.5 FAL Service Protocol Machine (FSPM).....	402
5.6 Application Relationship Protocol Machines (ARPMs).....	408
5.7 DLL Mapping Protocol Machine (DMPM)	492
5.8 Alternate DLL mapping protocol machine.....	496
6 Type 3	502
6.1 FAL syntax description	502
6.2 Transfer syntax	506
6.3 FAL protocol state machines	553
6.4 AP-Context state machine	567
6.5 FAL Service Protocol Machines (FSPMs)	567
6.6 Application Relationship Protocol Machines (ARPMs).....	650
6.7 DLL Mapping Protocol Machines (DMPMs).....	791
6.8 Parameters for a DP-slave.....	827
7 Type 4	828
7.1 FAL syntax description	828
7.2 Transfer syntaxes.....	830
7.3 FAL Protocol State Machines.....	837
7.4 AP-Context State Machine.....	838
7.5 FAL Service Protocol Machine (FSPM).....	838

7.6	Application Relationship Protocol Machine (ARPM)	843
7.7	DLL Mapping Protocol Machine (DMPM)	846
7.8	Protocol options	849
8	Type 5	850
8.1	Overview	850
8.2	FAL Syntax Description	850
8.3	Transfer syntax	850
8.4	FAL Protocol State Machine Structure	899
8.5	SMK State Machine	899
8.6	VCR State Machine	914
8.7	FAL Service Protocol Machine (FSPM)	915
8.8	Application Relationship Protocol Machines (ARPMs)	916
8.9	DLL Mapping Protocol Machine (DMPM)	930
9	Type 6	936
9.1	Reference	936
9.2	Usage of Parameters	936
9.3	AP Context State Tables	936
9.4	Application Relationship protocol machines (ARPMs)	936
9.5	DLL Mapping protocol machine (DMPM)	952
10	Type 7	962
10.1	Abstract syntax of data type	962
10.2	Transfer Syntaxes	967
10.3	Structure of Protocol Machines	1027
10.4	AP-Context state machine	1028
10.5	Sub-MMS FAL Service Protocol Machine (FSPM)	1028
10.6	DLL Mapping Protocol Machine (DMPM) and Association Relationship Protocol Machine (ARPM)	1033
10.7	Protocol options	1081
11	Type 8	1101
11.1	FAL Syntax Description	1101
11.2	Transfer Syntax	1110
11.3	Protocol Machine Overview	1118
11.4	AP-Context State Machine	1119
11.5	FAL Service Protocol Machine (FSPM)	1119
11.6	Application Relationship Protocol Machines (ARPMs)	1119
11.7	DLL Mapping Protocol Machine	1128
12	Type 9	1139
12.1	FAL syntax description	1139
12.2	Transfer Syntax	1158
12.3	FAL Protocol State Machines Structure	1167
12.4	VCR PM State Machine	1170
12.5	AREP State Machines	1182
13	Type 10	1230
13.1	FAL Syntax Description	1230
13.2	Transfer Syntax	1247
13.3	FAL Protocol State Machines	1247

13.4 AP Context State Machine	1248
13.5 FAL Service Protocol Machines (FSPM)	1248
13.6 Application Relationship Protocol Machine (ARPM)	1303
13.7 DLL Mapping Protocol Machine (DMPM)	1306
13.8 Protocol Options	1309
Figure 1 – Relationship of IEC 61158-6 to other fieldbus layers and to users of the fieldbus Application service.....	34
Figure 2 – Attribute table format and terms	53
Figure 3 – Service request/response parameter	54
Figure 4 – Common structure of specific fields.....	58
Figure 5 – Example of an STD	63
Figure 6 – Example of an evaluation net	67
Figure 7 – APDU overview	108
Figure 8 – Identification information (format 1).....	108
Figure 9 – Identification information (format 2).....	109
Figure 10 – Identification information (format 3).....	109
Figure 11 – Identification information (format 4).....	109
Figure 12 – Coding of the data type BinaryDate	111
Figure 13 – Encoding of Time of Day value	111
Figure 14 – Encoding of Time Difference value	112
Figure 15 – Encoding Time value	112
Figure 16 – Encoding of data of data type DL—Time-offset.....	113
Figure 17 – Structure of an object definition.....	114
Figure 18 – APDU overview	117
Figure 19 – Type field	117
Figure 20 – Identifier octet (context-specific)	118
Figure 21 – Identifier octet (FAL-specific)	118
Figure 22 – Length octet (one-octet format)	119
Figure 23 – Length octet (three-octet format).....	119
Figure 24 – APDU overview	127
Figure 25 – Relationships among protocol machines and adjacent layers	133
Figure 26 – AP to AP-Context initiation state machine	135
Figure 27 – State transition diagram of FSPM.....	153
Figure 28 – State transition diagram of the QUU ARPM	160
Figure 29 – State transition diagram of QUB-CO ARPM.....	169
Figure 30 – State transition diagram of the QUB-CL ARPM.....	181
Figure 31 – State transition diagram of QUB-seg ARPM	191
Figure 32 – State transition diagram of QUB-FC ARPM	211
Figure 33 – State transition diagram of BUB ARPM.....	238
Figure 34 – State transition diagram of BNB ARPM (basic state machine)	254
Figure 35 – State transition diagram of BNB ARPM (confirmed service sending and receiving – client)	255
Figure 36 – State transition diagram of BNB ARPM (confirmed service receiving and responding – server).....	255
Figure 37 – State transition diagram of BNB ARPM (unconfirmed service sending- client) ..	255
Figure 38 – State transition diagram of the BNU ARPM	279

Figure 39 – State transition diagram of the BNU-MP ARPM	291
Figure 40 – State transition diagram of DMPM	304
Figure 41 – Primitives exchanged between protocol machines	313
Figure 42 – Primitives exchanged between protocol machines	314
Figure 43 – Primitives exchanged between protocol machines	315
Figure 44 – Network connection parameters	333
Figure 45 – Time tick	335
Figure 46 – Connection establishment time-out.....	337
Figure 47 – Segment type	352
Figure 48 – Port segment.....	353
Figure 49 – Encapsulation message	377
Figure 50 – FixedLengthBitString compact encoding bit placement rules	394
Figure 51 – Example compact encoding of a OCTET FixedLengthBitString	394
Figure 52 – Example compact encoding of a WORD FixedLengthBitString	395
Figure 53 – Example compact encoding of a DWORD FixedLengthBitString	395
Figure 54 – Example compact encoding of a LWORD FixedLengthBitString	395
Figure 55 – Example 2 of formal encoding of a structure type specification.....	399
Figure 56 – Example of abbreviated encoding of a structure type specification	399
Figure 57 – Example 1 of formal encoding of an array type specification.....	400
Figure 58 – Example 2 of formal encoding of an array type specification.....	401
Figure 59 – Example 1 of abbreviated encoding of an array type specification	401
Figure 60 – Example 2 of abbreviated encoding of an array type specification	402
Figure 61 – State transition diagram of UCMM client.....	411
Figure 62 – State transition diagram of high–end UCMM server	413
Figure 63 – State transition diagram of low–end UCMM server	415
Figure 64 – Sequence diagram for a UCMM with one outstanding message.....	416
Figure 65 – Sequence diagram for a UCMM with multiple outstanding messages.....	417
Figure 66 – TPDU buffer	418
Figure 67 – Data flow diagram using a client transport class 0 and server transport class 0	420
Figure 68 – Sequence diagram of data transfer using transport class 0.....	421
Figure 69 – Class 0 client STD.....	422
Figure 70 – Class 0 server STD	423
Figure 71 – Data flow diagram using client transport class 1 and server transport class 1...	424
Figure 72 – Sequence diagram of data transfer using client transport class 1 and server transport class 1	425
Figure 73 – Class 1 client STD.....	427
Figure 74 – Class 1 server STD	428
Figure 75 – Data flow diagram using client transport class 2 and server transport class 2...	430
Figure 76 – Diagram of data transfer using client transport class 2 and server transport class 2 <i>without</i> returned data	431
Figure 77 – Sequence diagram of data transfer using client transport class 2 and server transport class 2 <i>with</i> returned data	432
Figure 78 – Class 2 client STD.....	434
Figure 79 – Class 2 server STD	436
Figure 80 – Data flow diagram using client transport class 3 and server transport class 3...	439
Figure 81 – Sequence diagram of data transfer using client transport class 3 and server transport class 3 <i>without</i> returned data	440

Figure 82 – Sequence diagram of data transfer using client transport class 3 and server transport class 3 <i>with</i> returned data	441
Figure 83 – Class 3 client STD.....	443
Figure 84 – Class 3 server STD	445
Figure 85 – Data flow diagram using transport classes 4 and 5.....	447
Figure 86 – Sequence diagram of message exchange using transport classes 4 and 5	448
Figure 87 – Sequence diagram of messages overwriting each other	449
Figure 88 – Sequence diagram of queued message exchange using transport classes 4 and 5	450
Figure 89 – Sequence diagram of retries using transport classes 4 and 5	451
Figure 90 – Sequence diagram of idle traffic using transport classes 4 and 5.....	452
Figure 91 – Classes 4 and 5 basic structure.....	453
Figure 92 – Class 6 basic structure	454
Figure 93 – Classes 4 to 6 general STD.....	455
Figure 94 – Class 4 sender STD	457
Figure 95 – Class 4 receiver STD	460
Figure 96 – Sequence diagram of three fragments using transport class 5	463
Figure 97 – Sequence diagram of fragmentation with retries using transport class 5	464
Figure 98 – Sequence diagram of two fragments using transport class 5	465
Figure 99 – Sequence diagram of aborted message using transport class 5.....	465
Figure 100 – Class 5 sender STD	467
Figure 101 – Class 5 receiver STD	470
Figure 102 – Data flow diagram for transport class 6	474
Figure 103 – Sequence diagram of message exchange using transport class 6	476
Figure 104 – Sequence diagram of retries using transport class 6.....	476
Figure 105 – Sequence diagram of idle traffic using transport class 6	477
Figure 106 – Sequence diagram of request overwriting null	478
Figure 107 – Sequence diagram of response overwriting ACK of null.....	479
Figure 108 – Sequence diagram of three fragments using transport class 6	480
Figure 109 – Sequence diagram of fragmentation with retries using transport class 6	481
Figure 110 – Sequence diagram of two fragments using transport class 6.....	482
Figure 111 – Sequence diagram of aborted fragmented sequence using transport class 6 ..	483
Figure 112 – Class 6 client STD.....	484
Figure 113 – Class 6 server STD	487
Figure 114 – Data flow diagram for a link producer and consumer	492
Figure 115 – State transition diagram for a link producer	495
Figure 116 – State transition diagram for a link consumer	496
Figure 117 – Coding of the data type BinaryDate	508
Figure 118 – Encoding of Time Of Day value	508
Figure 119 – Encoding of Time Difference value	508
Figure 120 – Encoding of Network Time value	509
Figure 121 – Encoding of Network Time Difference value	509
Figure 122 – Example Modul_Status_Array.....	514
Figure 123 – Example of Ext_Diag_Data in case of DPV1 diagnosis format with alarm and status PDU.....	549
Figure 124 – Example of Ext_Diag_Data in case of the basic diagnosis format	550
Figure 125 – Example of a special identifier format.....	550

Figure 126 – Example of a special identifier format with data types.....	551
Figure 127 – Example of a special identifier format with data types.....	551
Figure 128 – Example of a empty slot with data types.....	551
Figure 129 – Example for multi-variable device with AI and DO function blocks	552
Figure 130 – Identifiers (ID)	552
Figure 131 – Identifier list	552
Figure 132 – Structure of the Data_Unit for the request- and response-DLPDU	553
Figure 133 – Structuring of the protocol machines and adjacent layers in a DP-slave	556
Figure 134 – Structuring of the protocol machines and adjacent layers in a DP-master (class 1)	557
Figure 135 – Structuring of the protocol machines and adjacent layers in a DP-master (class 2)	558
Figure 136 – Sequence of the communication between DP-master and DP-slave	560
Figure 137 – Sequence of communication between DP-master (class 2) and DP-master (class 1)	562
Figure 138 – Sequence of acyclic communication between DP-master (class 1) and DP-slave	563
Figure 139 – Example for connection establishment on MS2.....	565
Figure 140 – Idle at master-side on MS2.....	566
Figure 141 – Idle at slave-side on MS2	567
Figure 142 – Example for connection establishment on MS2(server-side)	696
Figure 143 – Structure of RM entries in the RM_Registry.....	697
Figure 144 – APDU Header structure	830
Figure 145 – Instruction subfield of ControlStatus	830
Figure 146 – Errorcode subfield of ControlStatus	831
Figure 147 – Remaining subfields of ControlStatus	831
Figure 148 – DataFieldFormat encoding	832
Figure 149 – Structure of request APDU Body	832
Figure 150 – Structure of response APDU Body.....	832
Figure 151 – Variable Identifier	832
Figure 152 – Code subfield of Variable Identifier	833
Figure 153 – Summary of FAL Architecture.....	837
Figure 154 – FSPM proxy object state machine	839
Figure 155 – FSPM real object state machine	843
Figure 156 – ARPM State Machine	844
Figure 157 – DLPM State Machine.....	847
Figure 158 – State Transition Diagram for SMK	901
Figure 159 – State Transition Diagram of Client / Server ARPM.....	919
Figure 160 – State Transition diagram of the publisher / subscriber ARPM	926
Figure 161 – State transition diagram of DMPM	932
Figure 162 – State transition diagram of QUB-PC ARPM	940
Figure 163 – State transition diagram of the BNU-PC ARPM.....	950
Figure 164 – State transition diagram of DMPM	955
Figure 165 – Encoding of a CompactValue	967
Figure 166 – Organisation of the bits and octets within a PDU	968
Figure 167 – Encoding of a Bitstring	972
Figure 168 – Encoding of a Floating point.....	973

Figure 169 – Encoding of a structure	974
Figure 170 – Encoding of a boolean array.....	975
Figure 171 – Representation of a MCS PDU	981
Figure 172 – Relationships among Protocol Machines and Adjacent Layers.....	1027
Figure 173 – A_Readloc service evaluation net.....	1033
Figure 174 – A_Writeloc service evaluation net.....	1034
Figure 175 – A_Update service evaluation net	1035
Figure 176 – A_Readfar service evaluation net	1037
Figure 177 – A_writefar service evaluation net.....	1039
Figure 178 – A_Sent service evaluation net	1040
Figure 179 – A_Received service evaluation net.....	1040
Figure 180 – Association establishment: Requester element state machine	1047
Figure 181 – Association establishment: Responder element state machine	1048
Figure 182 – Association termination: Requester element state machine	1050
Figure 183 – Association termination: Responder element state machine	1052
Figure 184 – Association revocation: Requester element state machine	1053
Figure 185 – Association revocation: Acceptor element state machine.....	1054
Figure 186 – Interactions between state machine in an associated mode data transfer	1056
Figure 187 – Transfer service – Requester element state machine	1060
Figure 188 – Transfer service: Acceptor element state machine.....	1061
Figure 189 – Unacknowledged transfer: Requester element state machine	1062
Figure 190 – Unacknowledged transfer: Acceptor element state machine	1062
Figure 191 – Acknowledged transfer: Requester element state machine	1064
Figure 192 – Acknowledged transfer: Acceptor element state machine	1065
Figure 193 – Numbering mechanism state machine	1066
Figure 194 – Retry mechanism state machine.....	1068
Figure 195 – Anticipation mechanism state machine	1071
Figure 196 – Segmentation mechanism state machine.....	1073
Figure 197 – Reassembly mechanism state machine	1075
Figure 198 – Interaction of state machine in a non associated data transfer.....	1077
Figure 199 – Unacknowledged transfer: Requester element state machine	1078
Figure 200 – Unacknowledged transfer: Acceptor element state machine	1078
Figure 201 – Acknowledged transfer: Requester element state machine	1080
Figure 202 – Acknowledged transfer: Acceptor element state machine	1081
Figure 203 – APDU overview	1111
Figure 204 – APDU Header.....	1111
Figure 205 – PDU withType Extension	1111
Figure 206 – PDU with Address Extension	1111
Figure 207 – PDU with Type and Length Extension.....	1111
Figure 208 – Example of an Establish-Request PDU	1112
Figure 209 – Encoding of a PRIVATE tagged value	1112
Figure 210 – Encoding of a context specific tagged value	1113
Figure 211 – Identification information fields.....	1113
Figure 212 – ID-info for Tag 0 .. 14 , Length entry 0 .. 6.....	1113
Figure 214 – ID-info for Tag 15 .. 255 , Length entry 0 .. 6.....	1113
Figure 215 – ID-info for Tag 0 .. 14 , Length entry 7 .. 255.....	1113

Figure 216 – ID-info for Tag 15 .. 255 , Length entry 7 .. 255	1114
Figure 217 – Encoding of Boolean value TRUE.....	1114
Figure 218 – Encoding of Boolean value FALSE	1114
Figure 219 – Encoding of Strings	1114
Figure 220 – Encoding of BinaryDate Value	1115
Figure 221 – Encoding of BinaryDate2000 Value	1115
Figure 222 – Encoding of Time of Day value	1116
Figure 223 – Encoding of Time Difference Value.....	1116
Figure 224 – Encoding of Time Value	1117
Figure 225 – Example for an Object Definition	1118
Figure 226 – Primitives Exchanged between Protocol Machines	1119
Figure 227 – State Transition Diagram of QUB-TM AREP	1126
Figure 228 – State transition diagram of Type-8 DMPM	1131
Figure 229 – Structure of an object description in the OD	1154
Figure 230 – Structure of the OD object description	1154
Figure 231 – Structure of a domain entry in the S-OD	1154
Figure 232 – Object description of program invocation in the DP-OD	1154
Figure 233 – Object description of simple variable in the S-OD	1154
Figure 234 – Object description of array in the S-OD	1154
Figure 235 – Object description of record in the S-OD	1155
Figure 236 – Object description of variable list in the DV-OD	1155
Figure 237 – Object description of data type in the ST-OD.....	1155
Figure 238 – Object description of data type structure description in the ST-OD	1155
Figure 239 – Object description of event in the S-OD.....	1155
Figure 240 – Insertion of identification information in the FMS PDU	1158
Figure 241 – Coding of the ID info with and without extension	1160
Figure 242 – Coding with identification	1160
Figure 243 – Coding without identification.....	1160
Figure 244 – Representation of the value true.....	1160
Figure 245 – Representation of the value false	1160
Figure 246 – Coding of data of data type Integer16.....	1161
Figure 247 – Coding of data of data type Unsigned16	1161
Figure 248 – Coding of data of data type Floating Point.....	1162
Figure 249 – Coding of data of data type Visible String.....	1162
Figure 250 – Coding of data of data type Octet String	1162
Figure 251 – Coding of data of data type Date	1163
Figure 252 – Coding of data of data type Time Of Day.....	1164
Figure 253 – Coding of data of data type Time Difference.....	1164
Figure 254 – Coding of data of data type Bit String	1165
Figure 255 – Coding of data of data type Time Value.....	1165
Figure 256 – Coding of data of user data definitions with identifier.....	1165
Figure 257 – Coding of data of user data definitions without identifier.....	1166
Figure 258 – Coding of ID info for a SEQUENCE	1166
Figure 259 – Relationships among protocol machines and adjacent layers	1168
Figure 260 – VCR state machine	1172
Figure 261 – State transition diagram of FSPM	1193

Figure 262 – State transition diagram of the QUU ARPM	1196
Figure 263 – State transition diagram of QUB ARPM	1199
Figure 264 – State transition diagram of the BNU ARPM.....	1208
Figure 265 – State transition diagram of DMPM	1213
Figure 266 - Error Message structure.....	1230
Figure 267 – Coding scheme of ITEMQUALITYDEF	1233
Figure 268 – Relationship among Protocol Machines	1248
Figure 269 – State Transition Diagram of FSPM	1258
Figure 270 – State Transition Diagram of ARPM	1304
Figure 271 – State Transition Diagram of DMPM	1308
Table 1– Get_Attribute_All response service rules	55
Table 2 – Example class level object/service specific reply data of Get_Attribute_All.....	55
Table 3 – Example Get_Attribute_All data array method	55
Table 4 – Set_Attribute_All request service rules	56
Table 5 – Example Set_Attribute_All attribute ordering method	56
Table 6 – Example Set_Attribute_All data array method.....	57
Table 7 – Conventions used for state machines	61
Table 8 – State event matrix format	63
Table 9 – Example state event matrix	64
Table 10 – State machine description elements	64
Table 11 – Description of state machine elements	65
Table 12 – Conventions used in state machines.....	65
Table 13 – Reason codes	106
Table 14 – FAL header	107
Table 15 – Identification octet classes	128
Table 16 – Unit of measurement of contents length octets	129
Table 17 – Primitives issued by FAL-user to AP-Context.....	134
Table 18 – Primitives issued by AP-Context to FAL-user.....	134
Table 19 – AP-Context state machine sender transactions.....	136
Table 20 – AP-Context state machine receiver transactions.....	140
Table 21 – Function ResetArep.....	147
Table 22 – Function ApContextTest	148
Table 23 – Function ServicesSupportedTest.....	148
Table 24 – Function ApExplicitConnection	148
Table 25 – Function ImmediateAcknowledge	148
Table 26 – Function ConfirmedServiceCheck.....	148
Table 27 – Function UnconfirmedServiceCheck	148
Table 28 – Function ArServiceCheck	148
Table 29 – Function ArFspmService	149
Table 30 – Function ArAcceeSupported	149
Table 31 – Function MaxFalPduLengthTest	149
Table 32 – Function NegotiateOutstandingServices	149
Table 33 – Function RequestedServicesSupportedTest	149
Table 34 – Function IndicatedServicesSupportedTest.....	150
Table 35 – Function InvokeldExistent	150

Table 36 – Function SameService.....	150
Table 37 – Primitives issued by AP-Context to FSPM	151
Table 38 – Primitives issued by FSPM to AP-Context	152
Table 39 – FSPM state table – Sender transactions.....	153
Table 40 – FSPM state table – receiver transactions.....	155
Table 41 – Function SelectArep	156
Table 42– Primitives issued by FSPM to ARPM	157
Table 43 – Primitives issued by ARPM to FSPM	157
Table 44 – Parameters used with primitives exchanged between FSPM and ARPM	157
Table 45 – QUU ARPM states.....	160
Table 46 – QUU ARPM state table – sender transactions.....	161
Table 47 – QUU ARPM state table – receiver transactions.....	162
Table 48 – Function GetArepId ().....	163
Table 49 – Function BuildFAL-PDU.....	163
Table 50 – Function FAL_Pdu_Type	163
Table 51 – Primitives issued by FSPM to ARPM	163
Table 52 – Primitives issued by ARPM to FSPM	164
Table 53 – Parameters used with primitives exchanged between FSPM and ARPM	164
Table 54 – QUB-CO ARPM states.....	169
Table 55 – QUB-CO ARPM state table – sender transactions	169
Table 56 – QUB-CO ARPM state table – receiver transactions.....	171
Table 57 – Function GetArepId ().....	177
Table 58 – Function BuildFAL-PDU.....	177
Table 59 – Function FAL_Pdu_Type	177
Table 60 – Function AbortIdentifier	177
Table 61 – Function AbortReason.....	177
Table 62 – Function AbortDetail.....	178
Table 63 – Primitives issued by FSPM to ARPM	178
Table 64 – Primitives issued by ARPM to FSPM	178
Table 65 – Parameters used with primitives exchanged between FSPM and ARPM	179
Table 66 – QUB-CL ARPM states	181
Table 67 – QUB-CL ARPM state table – sender transactions	182
Table 68 – QUB-CL ARPM state table – receiver transactions	183
Table 69 – Function GetArepId ().....	184
Table 70 – Function BuildFAL-PDU.....	185
Table 71 – Function FAL-Pdu_Type.....	185
Table 72 – Primitives issued by FSPM to ARPM	185
Table 73 – Primitives issued by ARPM to FSPM	186
Table 74 – Parameters used with primitives exchanged between FSPM and ARPM	186
Table 75 – QUB-seg ARPM states	191
Table 76 – QUB-seg ARPM state table – sender transactions	192
Table 77 – ARPM state table – receiver transactions	195
Table 78 – Function GetArepId	203
Table 79 – Function BuildFAL-PDU.....	203
Table 80 – Function FAL_Pdu_Type	204
Table 81 – Function AbortIdentifier	204

Table 82 – Function AbortReason	204
Table 83 – Function AbortDetail	204
Table 84 – Function BuildFAL-segment	204
Table 85 – Function MoreFollows	204
Table 86 – Function ResetIntermediatePDU	205
Table 87 – Function AddSegment	205
Table 88 – Function GetIntermediatePDU	205
Table 89 – Primitives issued by FSPM to ARPM	205
Table 90 – Primitives issued by ARPM to FSPM	206
Table 91 – Parameters used with primitives exchanged between FSPM and ARPM	206
Table 92 – QUB-FC ARPM states	210
Table 93 – QUB-FC ARPM state table – sender transactions	211
Table 94 – QUB-FC ARPM state table – receiver transactions	217
Table 95 – Function GetArepld ()	229
Table 96 – Function BuildFAL-PDU	229
Table 97 – Function FAL_Pdu_Type	229
Table 98 – Function AREPContextCheck()	230
Table 99 – Function AbortIdentifier	230
Table 100 – Function AbortReason	230
Table 101 – Function AbortDetail	230
Table 102 – Function StartTimer	230
Table 103 – Function StopTimer	231
Table 104 – Function ResetCounters	231
Table 105 – Function IncrementCounter	231
Table 106 – Function DecrementCounter	231
Table 107 – Function GetCounterValue	231
Table 108 – Function Xon	231
Table 109 – Function SetXonOff	231
Table 110 – Primitives issued by FSPM to ARPM	232
Table 111 – Primitives issued by ARPM to FSPM	233
Table 112 – Parameters used with primitives exchanged between FSPM and ARPM	233
Table 113 – BUB ARPM states	237
Table 114 – BUB ARPM state table – sender transactions	239
Table 115 – BUB state table – receiver transactions	241
Table 116 – Function GetArepld ()	248
Table 117 – Function BuildFAL-PDU	248
Table 118 – Function FAL_Pdu_Type	248
Table 119 – Function AbortIdentifier	248
Table 120 – Function AbortReason	248
Table 121 – Function AbortDetail	248
Table 122 – Primitives issued by FSPM to ARPM	249
Table 123 – Primitives issued by ARPM to FSPM	249
Table 124 – Parameters used with primitives exchanged between FSPM and ARPM	250
Table 125 – BNB ARPM states	254
Table 126 – BNB ARPM state table – sender transactions (basic state machine)	256
Table 127 – BNB state table – receiver transactions (basic state machine)	259

Table 128 – BNB ARPM state table – sender transactions (confirmed service sending and receiving – client).....	265
Table 129 – BNB state table – receiver transactions (confirmed service sending and receiving – client).....	267
Table 130 – BNB ARPM state table – sender transactions (confirmed service receiving and responding – server)	269
Table 131 – BNB state table – receiver transactions (confirmed service receiving and responding – server)	270
Table 132 – BNB ARPM state table – sender transactions (unconfirmed service sending – client)	271
Table 133 – BNB state table – receiver transactions (unconfirmed service sending – client)	271
Table 134 – Function GetArepld ()	271
Table 135 – Function dmpm_duplicate_dlsdu ()	272
Table 136 – Function BuildFAL-PDU ()	272
Table 137 – Function FAL_Pdu_Type ()	272
Table 138 – Function AbortIdentifier ()	272
Table 139 – Function AbortReason ()	272
Table 140 – Function AbortDetail ()	272
Table 141 – Function StartTimer ()	273
Table 142 – Function StopTimer ()	273
Table 143 – Function StopService ()	273
Table 144 – Function RestoreService ()	273
Table 145 – Function Flag ()	273
Table 146 – Function SetFlag ()	273
Table 147 – Function StoreIndex ()	274
Table 148 – Function TestIndex ()	274
Table 149 – Primitives issued by FSPM to ARPM	274
Table 150 – Primitives issued by ARPM to FSPM	275
Table 151 – Parameters used with primitives exchanged between FSPM and ARPM	275
Table 152 – BNU ARPM states	279
Table 153 – BNU ARPM state table – sender transactions	280
Table 154 – BNU ARPM state table – receiver transactions	281
Table 155 – Function GetArepld ()	284
Table 156 – Function BuildFAL-PDU	284
Table 157 – Function FAL_Pdu_Type	284
Table 158 – Function AbortIdentifier	284
Table 159 – Function AbortReason	284
Table 160 – Function AbortDetail	285
Table 161 – Primitives issued by FSPM to ARPM	285
Table 162 – Primitives issued by ARPM to FSPM	286
Table 163 – Parameters used with primitives exchanged between FSPM and ARPM	286
Table 164 – BNU-MP ARPM states	290
Table 165 – BNU-MP ARPM state table – sender transactions	291
Table 166 – BNU-MP ARPM state table – receiver transactions	293
Table 167 – Function GetArepld ()	297
Table 168 – Function BuildFAL-PDU	297

Table 169 – Function FAL_Pdu_Type	297
Table 170 – Function AbortIdentifier	297
Table 171 – Function AbortReason	298
Table 172 – Function AbortDetail	298
Table 173 – Function dedicatedArep ()	298
Table 174 – Primitives issued by ARPM to DMPM	299
Table 175 – Primitives issued by DMPM to ARPM	300
Table 176 – Parameters used with primitives exchanged between ARPM and DMPM	301
Table 177 – Primitives exchanged between Data Link Layer and DMPM	302
Table 178 – DMPM state descriptions	304
Table 179 – DMPM state table – sender transactions	304
Table 180 – DMPM state table – receiver transactions	307
Table 181 – Function PickArep	311
Table 182 – Function FindAREP	311
Table 183 – Function LocateArep	311
Table 184 – UCMM_PDU header format	318
Table 185 – UCMM command codes	318
Table 186 – Transport class 0 header	319
Table 187 – Transport class 1 header	319
Table 188 – Transport class 2 header	320
Table 189 – Transport class 3 header	320
Table 190 – Classes 4 to 6 header format	320
Table 191 – Real-time data header – exclusive owner	321
Table 192 – Real-time data header– redundant owner	322
Table 193 – Forward_Open request format	324
Table 194 – Forward_Open_Good response format	324
Table 195 – Forward_Open_Bad response format	325
Table 196 – Ex_Forward_Open request format	326
Table 197 – Ex_Forward_Open_Good response format	326
Table 198 – Ex_Forward_Open_Bad response format	327
Table 199 – Forward_Close request format	327
Table 200 – Forward_Close_Good response format	328
Table 201 – Forward_Close_Bad response format	328
Table 202 – Unconnected_Send request format	329
Table 203 – Unconnected_Send_Good response format	329
Table 204 – Unconnected_Send_Bad response format	330
Table 205 – Get_Connection_Data request format	330
Table 206 – Get_Connection_Data response format	331
Table 207 – Search_Connection_Data request format	331
Table 208 – Get_Object_Owner request format	332
Table 209 – Forward_Open_Good response format	332
Table 210 – Time-out multiplier	335
Table 211 – Time tick units	336
Table 212 – Selection of connection ID	339
Table 213 – Transport class, trigger and Is_Server format	340
Table 214 – MR_Request_Header format	340

Table 215 – MR_Response_Header format.....	340
Table 216 – Structure of Get_Attribute_All_ResponsePDU body.....	341
Table 217 – Structure of Set_Attribute_All_RequestPDU body.....	341
Table 218 – Structure of Get_Attribute_List_RequestPDU body.....	341
Table 219 – Structure of Get_Attribute_List_ResponsePDU body.....	341
Table 220 – Structure of Set_Attribute_List_RequestPDU body.....	342
Table 221 – Structure of Set_Attribute_List_ResponsePDU body.....	342
Table 222 – Structure of Reset_RequestPDU body.....	342
Table 223 – Structure of Reset_ResponsePDU body.....	342
Table 224 – Structure of Start_RequestPDU body.....	342
Table 225 – Structure of Start_ResponsePDU body.....	342
Table 226 – Structure of Stop_RequestPDU body.....	343
Table 227 – Structure of Stop_ResponsePDU body.....	343
Table 228 – Structure of Create_RequestPDU body.....	343
Table 229 – Structure of Create_ResponsePDU body.....	343
Table 230 – Structure of Delete_RequestPDU body.....	343
Table 231 – Structure of Delete_ResponsePDU body.....	343
Table 232 – Structure of Get_Attribute_Single_ResponsePDU body.....	344
Table 233 – Structure of Set_Attribute_Single_RequestPDU body.....	344
Table 234 – Structure of Set_Attribute_Single_ResponsePDU body.....	344
Table 235 – Structure of Find_Next_Object_Instance_RequestPDU body.....	344
Table 236 – Structure of Find_Next_Object_Instance_ResponsePDU body.....	344
Table 237 – Structure of Apply_Attributes_RequestPDU body.....	345
Table 238 – Structure of Apply_Attributes_ResponsePDU body.....	345
Table 239 – Structure of Save_RequestPDU body.....	345
Table 240 – Structure of Save_ResponsePDU body.....	345
Table 241 – Structure of Restore_RequestPDU body.....	345
Table 242 – Structure of Restore_ResponsePDU body.....	345
Table 243 – Identity object class attributes.....	346
Table 244 – Identity object instance attributes.....	346
Table 245 – Identity object bit definitions for status instance attribute.....	346
Table 246 – Bits 4 – 7 of status instance attribute.....	347
Table 247 – Class level object/service specific response data of Get_Attribute_All.....	347
Table 248 – Instance level object/service specific response data of Get_Attribute_All.....	348
Table 249 – Modified instance level object/service specific response data of Get_Attribute_All.....	348
Table 250 – Object-specific parameter for Reset.....	348
Table 251 – Message router object class attributes.....	349
Table 252 – Message router object instance attributes.....	349
Table 253 – Class level object/service specific response data of Get_Attribute_All.....	349
Table 254 – Instance level object/service specific response data of Get_Attribute_All.....	349
Table 255 – Assembly object class attributes.....	350
Table 256 – Assembly object instance attributes.....	350
Table 257 – Connection manager object class attributes.....	350
Table 258 – Connection manager object instance attributes.....	351
Table 259 – Possible port segment examples.....	353

Table 260 – TCP/IP link address examples	354
Table 261 – Logical segments	355
Table 262 – Electronic key segment format.....	356
Table 263 – Network segments.....	357
Table 264 – Data segment.....	358
Table 265 – ANSI_Extended_Symbol segment	358
Table 266 – Addressing categories	359
Table 267 – Class code ID ranges	359
Table 268 – Attribute ID ranges	359
Table 269 – Service code ranges.....	360
Table 270 – objects	360
Table 271 – Reserved class attributes for all object class definitions	361
Table 272 – Common services list	361
Table 273 – Services specific to Connection Manager	362
Table 274 – Device type numbering	362
Table 275 – Connection manager service request error codes	363
Table 276 – General status codes.....	371
Table 277 – Extended status codes	373
Table 278 – Encapsulation header	378
Table 279 – Encapsulation command codes	378
Table 280 – Encapsulation status codes	379
Table 281 – Options flags	379
Table 282 – Nop request encapsulation header	380
Table 283 – RegisterSession request encapsulation header	380
Table 284 – RegisterSession request data portion	380
Table 285 – Options flags	380
Table 286 – RegisterSession reply encapsulation header	381
Table 287 – RegisterSession reply data portion	381
Table 288 – UnRegisterSession request encapsulation header	382
Table 289 – ListServices request encapsulation header.....	382
Table 290 – ListServices reply encapsulation header.....	382
Table 291 – ListServices reply data portion.....	383
Table 292 – Service type codes	383
Table 293 – Communications capability flags.....	383
Table 294 – ListIdentity request encapsulation header.....	384
Table 295 – ListIdentity reply encapsulation header.....	384
Table 296 – ListIdentity reply data portion.....	385
Table 297 – ListInterfaces request encapsulation header.....	385
Table 298 – ListInterfaces reply encapsulation header.....	385
Table 299 – SendRRData request encapsulation header	386
Table 300 – SendRRData request data portion	386
Table 301 – SendRRData reply encapsulation header	387
Table 302 – SendUnitData request encapsulation header	387
Table 303 – SendUnitData request data portion.....	387
Table 304 – Common packet format.....	388
Table 305 – Address and data item structure	388

Table 306 – Address type ID's	388
Table 307 – Data type ID's.....	388
Table 308 – Null address type.....	388
Table 309 – Connected address type	389
Table 310 – Sequenced address type	389
Table 311 – UCMM data type.....	389
Table 312 – Connected data type	389
Table 313 – Sockaddr info items	390
Table 314 – BOOLEAN encoding	391
Table 315 – Example compact encoding of a BOOL value	391
Table 316 – Encoding of SignedInteger values	391
Table 317 – Example compact encoding of a SignedInteger value	391
Table 318 – UnsignedInteger values	392
Table 319 – Example compact encoding of an UnsignedInteger	392
Table 320 – FixedLengthReal values	392
Table 321 – Example compact encoding of a REAL value	392
Table 322 – Example compact encoding of a LREAL value	392
Table 323 – FixedLengthReal values	393
Table 324 – STRING value	393
Table 325 – STRING2 value	393
Table 326 – STRINGN value.....	393
Table 327 – SHORT_STRING value	393
Table 328 – Example compact encoding of a STRING value	394
Table 329 – Example compact encoding of STRING2 value	394
Table 330 – SHORT_STRING type	394
Table 331 – Example compact encoding of a single dimensional ARRAY.....	395
Table 332 – Example compact encoding of a multi-dimensional ARRAY	396
Table 333 – Example compact encoding of a STRUCTURE	397
Table 334 – Identification codes and descriptions of elementary data types	398
Table 335 – Example 1 of formal encoding of a structure type specification	399
Table 336 – Primitives issued by FAL user to FSPM	403
Table 337 – Primitives issued by FAL user to FSPM	405
Table 338 – Primitives issued by FSPM to FAL user	407
Table 339 – Parameters used with primitives exchanged between FAL user and FSPM	408
Table 340 – Primitives issued by FSPM to ARPM	409
Table 341 – Primitives issued by ARPM to FSPM	410
Table 342 – Parameters used with primitives exchanged between FSPM and ARPM	410
Table 343 – UCMM client states	411
Table 344 – State event matrix of UCMM client.....	412
Table 345 – High-end UCMM server states.....	413
Table 346 – State event matrix of high-end UCMM server.....	414
Table 347 – Low-end UCMM server states.....	415
Table 348 – State event matrix of low-end UCMM server	415
Table 349 – Notification	418
Table 350 – Transport classes	419
Table 351 – Primitives issued by FSPM to ARPM	419

Table 352 – Primitives issued by ARPM to FSPM	419
Table 353 – Parameters used with primitives exchanged between FSPM and ARPM	420
Table 354 – Class 0 transport client states	421
Table 355 – Class 0 client SEM	422
Table 356 – Class 0 transport server states	423
Table 357 – Class 0 server SEM	423
Table 358 – Class 1 transport client states	426
Table 359 – Class 1 client SEM	427
Table 360 – Class 1 transport server states	428
Table 361 – Class 1 server SEM	429
Table 362 – Class 2 transport client states	433
Table 363 – Class 2 client SEM	434
Table 364 – Class 2 transport server states	435
Table 365 – Class 2 server SEM	437
Table 366 – Class 3 transport client states	442
Table 367 – Class 3 client SEM	443
Table 368 – Class 3 transport server states	444
Table 369 – Class 3 server SEM	446
Table 370 – Write and trigger events in class 4 and 5 transport	448
Table 371 – Common states for transport classes 4 to 6	454
Table 372 – Classes 4 to 6 general SEM	455
Table 373 – Class 4 transport sender states	457
Table 374 – Class 4 sender SEM	458
Table 375 – Class 4 transport receiver states	459
Table 376 – Class 4 receiver SEM	461
Table 377 – Class 5 transport sender states	466
Table 378 – Class 5 sender SEM	468
Table 379 – Class 5 transport receiver states	470
Table 380 – Class 5 receiver SEM	471
Table 381 – Class 6 transport client states	484
Table 382 – Class 6 client state event matrix	485
Table 383 – Class 6 transport server states	486
Table 384 – Class 6 server SEM	489
Table 385 – Primitives issued by ARPM to DMPM	493
Table 386 – Primitives issued by DMPM to ARPM	493
Table 387 – Parameters used with primitives exchanged between ARPM and DMPM	493
Table 388 – Primitives exchanged between Data Link Layer and DMPM	494
Table 389 – Parameters used with primitives exchanged between DMPM and Data Link	494
Table 390 – Link producer states	495
Table 391 – State event matrix of link producer	495
Table 392 – Link consumer states	495
Table 393 – State event matrix of link consumer	496
Table 394 – UDP data format for class 0 and class 1	499
Table 395 – APDU syntax	502
Table 396 – Substitutions	504
Table 397 – Alarm_Type range	512

Table 398 – Status_Type value range	512
Table 399 – Alarm_Specifier	513
Table 400 – Range of Modul_Status_Entry (1-4)	514
Table 401 – Error type	516
Table 402 – Specification of the bits Lock_Req and Unlock_Req	519
Table 403 – Range of Length_of_Manufacturer_Specific_Data if used in Chk_Cfg-REQ-PDU	525
Table 404 – Range of Length_of_Manufacturer_Specific_Data if used in Get_Cfg-RES-PDU	526
Table 405 – Values (codes) for data types	527
Table 406 – Specification of the bits for Un-/Sync and Un-/Freeze	528
Table 407 – Coding of the Function_Code/ Function_Num	530
Table 408 – Coding of the Error_Code / Function_Num	531
Table 409 – Values of Error_Decode	531
Table 410 – Coding of Error_Code_1 at DPV1	532
Table 411 – Values of MDiag_Identifier	533
Table 412 – Values for Area_Code_UpDownload	535
Table 413 – Values for Area_CodeActBrct	535
Table 414 – Values for Area_CodeAct	536
Table 415 – Values for Data_rate	537
Table 416 – Values for Slave_Type	539
Table 417 – Values for Alarm_Mode	540
Table 418 – Values for Subnet	545
Table 419 – Values of reason code if instance is DLL	545
Table 420 – Values of reason code if instance is MS2	546
Table 421 – Values of Extended_Function_Num	547
Table 422 – Values of FI_State	548
Table 423 – Assignment of state machines	555
Table 424 – Primitives issued by AP-Context to FSPMS	568
Table 425 – Primitives issued by FSPMS to AP-Context	570
Table 426 – FSPMS state table	576
Table 427 – Functions used by the FSPMS	598
Table 428 – Primitives issued by AP-Context to FSPMM1	599
Table 429 – Primitives issued by FSPMM1 to AP-Context	602
Table 430 – FSPMM1 state table	608
Table 431 – Functions used by the FSPMM1	633
Table 432 – Primitives issued by AP-Context to FSPMM2	633
Table 433 – Primitives issued by FSPMM2 to AP-Context	635
Table 434 – FSPMM2 state table	638
Table 435 – Functions used by the FSPMM2	649
Table 436 – Primitives issued by FSPMS to MSCY1S	650
Table 437 – Primitives issued by MSCY1S to FSPMS	651
Table 438 – Rules for DPV1_Status_1, DPV1_Status_2 and DPV1_Status_3 check	652
Table 439 – MSCY1S state table	657
Table 440 – Functions used by the MSCY1S	677
Table 441 – Primitives issued by FSPMS to MSAC1S	679

Table 442 – Primitives issued by MSAC1S to FSPMS	680
Table 443 – Primitives issued by MSCY1S to MSAC1S	680
Table 444 – Primitives issued by MSAC1S to MSCY1S	680
Table 445 – Parameter used with primitives exchanged between MSAC1S and MSCY1S ...	680
Table 446 – MSAC1S state table	682
Table 447 – Functions used by the MSAC1S	691
Table 448 – Primitives issued by FSPMS to SSCY1S	692
Table 449 – Primitives issued by SSCY1S to FSPMS	692
Table 450 – SSCY1S state table	693
Table 451 – Functions used by the SSCY1S	694
Table 452 – Primitives issued by FSPMS to MSRM2S	695
Table 453 – Primitives issued by MSRM2S to FSPMS	695
Table 454 – MSRM2S state table	698
Table 455 – Primitives issued by FSPMS to MSAC2S	700
Table 456 – Primitives issued by MSAC2S to FSPMS	701
Table 457 – Primitives issued by MSRM2S to MSAC2S	701
Table 458 – Primitives issued by MSAC2S to MSRM2S	702
Table 459 – Parameter used with primitives exchanged with MSAC2S	702
Table 460 – MSAC2S state table	704
Table 461 – Primitives issued by MSCS1S to FSPMS	716
Table 462 – MSCS1S state table	717
Table 463 – Primitives issued by FSPMM1 to MSCY1M	718
Table 464 – Primitives issued by MSCY1M to FSPMM1	718
Table 465 – Parameters used with primitives exchanged between FSPMM1 and MSCY1M	719
Table 466 – MSCY1M state table	722
Table 467 – Primitives issued by FSPMM1 to MSAL1M	736
Table 468 – Primitives issued by MSAL1M to FSPMM1	736
Table 469 – Primitives issued by MSCY1M to MSAL1M	736
Table 470 – Primitives issued by MSAL1M to MSCY1M	736
Table 471 – Parameter used with primitives exchanged between MSAL1M and MSCY1M	737
Table 472 – Possible values in the Alarm_State_Table	738
Table 473 – MSAL1M state table	740
Table 474 – Primitives issued by FSPMM1 to MSAC1M	744
Table 475 – Primitives issued by MSAC1M to FSPMM1	745
Table 476 – Primitives issued by MSAL1M to MSAC1M	745
Table 477 – Primitives issued by MSAC1M to MSAL1M	745
Table 478 – Parameter used with primitives exchanged between MSAL1M and MSCY1M	746
Table 479 – MSAC1M state table	751
Table 480 – Primitives issued by FSPMM1 to MMAC1	757
Table 481 – Primitives issued by MMAC1 to FSPMM1	757
Table 482 – MMAC1 state table	759
Table 483 – Primitives issued by FSPMM1 to MSCS1M	763
Table 484 – Primitives issued by MSCS1M to FSPMM1	764
Table 485 – MSCS1M state table	765

Table 486 – Primitives issued by FSPMM2 to MSAC2M	768
Table 487 – Primitives issued by MSAC2M to FSPMM2	769
Table 488 – Parameters used with primitives exchanged with MSAC2M	769
Table 489 – MSAC2M state table	774
Table 490 – Primitives issued by FSPMM2 to MMAC2	784
Table 491 – Primitives issued by MMAC2 to FSPMM2	785
Table 492 – Parameters used with primitives exchanged with MMAC2	785
Table 493 – MMAC2 state table	787
Table 494 – Primitives issued by FSPMS to DMPMS	791
Table 495 – Primitives issued by DMPMS to FSPMS	791
Table 496 – Primitives issued by MSCY1S to DMPMS	792
Table 497 – Primitives issued by DMPMS to MSCY1S	792
Table 498 – Primitives issued by DMPMS to SSCY1S	793
Table 499 – Primitives issued by MSAC1S, MSRM2S, MSAC2S to DMPMS	793
Table 500 – Primitives issued by DMPMS to MSAC1S, MSRM2S, MSAC2S	793
Table 501 – Primitives issued by DMPMS to MSCS1S	793
Table 502 – Primitives issued by DMPMS to DL	794
Table 503 – Primitives issued by DL to DMPMS	794
Table 504 – Parameters used with primitives exchanged with DMPMS	795
Table 505 – DMPMS state table	796
Table 506 – Functions used by the DMPMS	802
Table 507 – Primitives issued by FSPMM1 to DMPMM1	803
Table 508 – Primitives issued by DMPMM1 to FSPMM1	803
Table 509 – Primitives issued by MSCY1M to DMPMM1	804
Table 510 – Primitives issued by DMPMM1 to MSCY1M	804
Table 511 – Primitives issued by MSAL1M, MSAC1M to DMPMM1	805
Table 512 – Primitives issued by DMPMM1 to MSAL1M, MSAC1M	805
Table 513 – Primitives issued by MMAC1 to DMPMM1	805
Table 514 – Primitives issued by DMPMM1 to MMAC1	805
Table 515 – Primitives issued by MSCS1M to DMPMM1	806
Table 516 – Primitives issued by DMPMM1 to MSCS1M	806
Table 517 – Primitives issued by DMPMM1 to DL	807
Table 518 – Primitives issued by DL to DMPMM1	808
Table 519 – Parameters used with primitives exchanged with DMPMM1	809
Table 520 – Possible values of status	809
Table 521 – DMPMM1 state table	811
Table 522 – Functions used by the DMPMM1	819
Table 523 – Primitives issued by FSPMM2 to DMPMM2	820
Table 524 – Primitives issued by DMPMM2 to FSPMM2	820
Table 525 – Primitives issued by MSAC2M to DMPMM2	821
Table 526 – Primitives issued by DMPMM2 to MSAC2M	821
Table 527 – Primitives issued by MMAC2 to DMPMM2	821
Table 528 – Primitives issued by DMPMM2 to MMAC2	821
Table 529 – Primitives issued by DMPMM2 to DL	822
Table 530 – Primitives issued by DL to DMPMM2	822
Table 531 – Parameters used with primitives exchanged with DMPMM2	823

Table 532 – DMPMM2 state table	823
Table 533 – Functions used by DMPMM2	827
Table 534 – Bus parameter/reaction times for a DP-slave	827
Table 535 – APDU Header	828
Table 536 – APDU Body	829
Table 537 – Transfer syntax for Array	835
Table 538 – Transfer Syntax for Structure	835
Table 539 – Common Variable Object Attributes	835
Table 540 – Variable Type Identifiers	836
Table 541 – FIFO Variable Object Attributes	836
Table 542 – Error Codes	837
Table 543 – Primitives exchanged between FAL-User and FSPM	838
Table 544 – REQUEST.req FSPM constraints	840
Table 545 – REQUEST.req FSPM actions	840
Table 546 – RESPONSE.cnf FSPM constraints	841
Table 547 – RESPONSE.cnf FSPM actions	842
Table 548 – AR Send.ind proxy FSPM constraints	842
Table 549 – AR Send.ind proxy FSPM actions	842
Table 550 – AR Send.ind real FSPM constraints	843
Table 551 – AR Send.ind real FSPM Actions	843
Table 552 – Primitives issued by FSPM to ARPM	844
Table 553 – Primitives issued by ARPM to FSPM	844
Table 554 – Primitives issued by ARPM to ARPM	844
Table 555 – AR Send.req ARPM Constraints	844
Table 556 – AR Send.req ARPM Actions	845
Table 557 – AR Acknowledge.req ARPM Constraints	845
Table 558 – AR Acknowledge.req ARPM Actions	845
Table 559 – AR Send.ind ARPM Constraints	845
Table 560 – AR Send.req ARPM Actions	846
Table 561 – Primitives issued by ARPM to DLPM	846
Table 562 – Primitives issued by DLPM to ARPM	847
Table 563 – Primitives issued by DLPM to Data Link Layer	847
Table 564 – Primitives issued by Data Link Layer to DLPM	847
Table 565 – AR Send.req DLPM Constraints	848
Table 566 – AR Send.req DLPM Actions	848
Table 567 – AR Acknowledge.req DLPM Constraints	848
Table 568 – AR Acknowledge.req DLPM Actions	848
Table 569 – DL-UNITDATA.ind DLPM Constraints	849
Table 570 – DL-UNITDATA.ind DLPM Actions	849
Table 571 – Data Types	851
Table 572 – Data Types	851
Table 573 – Type 5 APDU Header Format	852
Table 574 – FDA Address Use	853
Table 575 – FDA Address header field APDUs sent by a client VCR endpoint	854
Table 576 – FDA Address header field APDUs sent by a server VCR endpoint	854
Table 577 – FDA Address header field APDUs sent by a publisher VCR endpoint	855

Table 578 – FDA Address header field APDUs sent by a report source VCR endpoint	855
Table 579 – APDU Trailer Fields	855
Table 580 – Request APDU Parameters	857
Table 581 – SMK FDA Address Values	859
Table 582 – SMK FDA Address Values	860
Table 583 – Request APDU Parameters	861
Table 584 – SMK FDA Address Values For SM Identify	862
Table 585 – SMK FDA Address Values For SMK Set Assignment Info Request APDUs	862
Table 586 – SMK Clear Address Request APDU Parameters	863
Table 587 – SMK FDA Address Values For SMK Set Assignment Info Request APDUs	863
Table 588 – SMK Set Assignment Info Request APDU Parameters	864
Table 589 – SMK Set Assignment Info Response APDU Parameters	865
Table 590 – SMK FDA Address Values For SMK Device Clear Assignment Info APDUs	865
Table 591 – SMK Clear Assignment Info Request APDU Parameters	866
Table 592 – SMK FDA Address Values For SMK Device Annunciation Request APDUs	866
Table 593 – SMK Device Annunciation Request APDU Parameters	867
Table 594 – Initiate Request APDU Parameters	869
Table 595 – Initiate Response APDU Parameters	869
Table 596 – Abort Request APDU Parameters	869
Table 597 – Get Response APDU Parameters	870
Table 598 – Identify Response APDU Parameters	870
Table 599 – Get OD Request APDU Parameters	870
Table 600 – Get OD Response APDU Parameters	871
Table 601 – Initiate Put OD Request APDU Parameters	871
Table 602 – Put OD Request APDU Parameters	871
Table 603 – Generic Initiate Download Sequence Request APDU Parameters	872
Table 604 – Generic Download Segment Request APDU Parameters	873
Table 605 – Generic Terminate Download Sequence Request APDU Parameters	873
Table 606 – Response APDU Parameters	873
Table 607 – Initiate Download Sequence Request APDU Parameters	874
Table 608 – Download Segment Request APDU Parameters	874
Table 609 – Download Segment Response APDU Parameters	874
Table 610 – Terminate Download Sequence Request APDU Parameters	875
Table 611 – Initiate Upload Sequence Request APDU Parameters	875
Table 612 – Upload Segment Request APDU Parameters	876
Table 613 – Upload Segment Response APDU Parameters	876
Table 614 – Terminate Upload Sequence Request APDU Parameters	876
Table 615 – Request Domain Download Request APDU Parameters	877
Table 616 – Request Domain Upload Request APDU Parameters	877
Table 617 – Create Program Invocation Request APDU Parameters	878
Table 618 – Create Program Invocation Response APDU Parameters	878
Table 619 – Delete Program Invocation Request APDU Parameters	878
Table 620 – Start Request APDU Parameters	879
Table 621 – Stop Request APDU Parameters	879
Table 622 – Resume Request APDU Parameters	879
Table 623 – Reset Request APDU Parameters	880

Table 624 – Kill Request APDU Parameters.....	880
Table 625 – Read Request APDU Parameters	880
Table 626 – Read Response APDU Parameters.....	880
Table 627 – Read with Subindex Request APDU Parameters	881
Table 628 – Read with Subindex Response APDU Parameters	881
Table 629 – Write Request APDU Parameters	881
Table 630 – Write with Subindex Request APDU Parameters.....	882
Table 631 – Define Variable List Request APDU Parameters	882
Table 632 – Define Variable List Response APDU Parameters	882
Table 633 – Delete Variable List Request APDU Parameters	883
Table 634 – Information Report Request APDU Parameters	883
Table 635 – Information Report with Subindex Request APDU Parameters	883
Table 636 – Information Report On Change Request APDU Parameters	884
Table 637 – Information Report On Change with Subindex Request APDU Parameters	884
Table 638 – Event Notification Request APDU Parameters	884
Table 639 – Alter Event Condition Monitoring Request APDU Parameters	884
Table 640 – Acknowledge Event Notification Request APDU Parameters.....	885
Table 641 – LAN Redundancy Diagnostic Message Request APDU Parameters	886
Table 642 – LAN Redundancy Get Information Response APDU Parameters.....	887
Table 643 – LAN Redundancy Get Statistics Request APDU Parameters.....	889
Table 644 – Object Description Header.....	891
Table 645 – Null Object	891
Table 646 – Structure of the List of Object Descriptions.....	891
Table 647 – Structure of a Load Region in the S-OD.....	892
Table 648 – Structure of a Function Invocation in the DP-OD.....	893
Table 649 – Structure of an Event in the S-OD	893
Table 650 – Structure of a Data Type in the ST-OD	894
Table 651 – Structure of a Data Type Structure Description in the ST-OD.....	894
Table 652 – Structure of a Simple Variable in the S-OD	895
Table 653 – Structure of an Array in the S-OD	895
Table 654 – Structure of a Record in the S-OD	896
Table 655 – Structure of a Variable List in the DV-OD.....	896
Table 656 – Common Error Parameters	897
Table 657 – PI Error Parameters.....	897
Table 658 – OD Error Parameters.....	897
Table 659 – Error Class and Error Code Values.....	898
Table 660 – SMKPM Service Primitives	900
Table 661 – SMKPM states.....	901
Table 662 – SMKPM state table – Initialization	902
Table 663 – SMKPM state table – Receive Transitions	902
Table 664 – SMKPM state table – Internal Events.....	907
Table 665 – HseRepeatTimerExpires ()	908
Table 666 – RcvNewNetworkAddress (interface, address)	908
Table 667 – RcvMsg ().....	908
Table 668 – SntpSyncLost ().....	908
Table 669 – AddressToClear (sm_svc)	909

Table 670 – AssignmentInfo_Set ()	909
Table 671 – ConfigurationSessionActive ()	909
Table 672 – DeviceRedundancyState ()	909
Table 673 – DevId_Match (sm_svc)	909
Table 674 – DuplicateQueryIdMatch (sm_svc)	909
Table 675 – DuplicatePdTagDetected ()	910
Table 676 – FdaAddressType (sm_svc)	910
Table 677 – IsValid (sm_svc)	910
Table 678 – NetworkAddressChange (interface, address)	910
Table 679 – NumberOfAssignedAddresses ()	910
Table 680 – OperationalRestore ()	911
Table 681 – PdTag_Match (sm_svc)	911
Table 682 – PdTagDeviceIndex_Check (sm_svc)	911
Table 683 – Query_Match (sm_svc)	911
Table 684 – QueryType (sm_svc)	911
Table 685 – SmCacheEntry (sm_svc)	911
Table 686 – Clear_Address (interface_to_clear)	912
Table 687 – Clear_DuplicatePdTagFlag ()	912
Table 688 – Get_AddlCode ()	912
Table 689 – New_Address (interface, address)	912
Table 690 – Restart_HseRepeatTimer ()	913
Table 691 – Restore_Defaults ()	913
Table 692 – Send_SM_CommonErrorRsp (sm_service_type, svc_spec_params)	913
Table 693 – Send_SM_ReqRspMessage (sm_svc)	913
Table 694 – Set_Assignment_Data (sm_svc)	913
Table 695 – Set_DuplicatePdTagFlag ()	913
Table 696 – SvcType (sm_svc)	914
Table 697 – Additional Code Used by Error Class and Code	914
Table 698 – Additional Code Parameter Ids	914
Table 699 – Primitives issued by FSPM to ARPM	916
Table 700 – Primitives issued by ARPM to FSPM	916
Table 701 – Parameters used with primitives exchanged between FSPM and ARPM	917
Table 702 – Client / Server ARPM states	919
Table 703 – Client / Server ARPM state table – Sender transitions	920
Table 704 – Client / Server ARPM state table – Receiver transitions	921
Table 705 – Primitives issued by FSPM to ARPM	923
Table 706 – Primitives issued by ARPM to FSPM	924
Table 707 – Parameters used with primitives exchanged between FSPM and ARPM	924
Table 708 – Publisher / Subscriber ARPM States	925
Table 709 – MulticastARPM state table – sender transitions	926
Table 710 – MulticastARPM state table – receiver transitions	927
Table 711 – BuildFAL-ErrPDU()	927
Table 712 – BuildFAL-ReqRspPDU()	928
Table 713 – GetArepld()	928
Table 714 – ConfigurationArCheckOK()	928
Table 715 – FAL_Pdu_BufferSize()	928

Table 716 – FAL_Pdu_Confirmed()	928
Table 717 – FAL_Pdu_DuplicateMsg ()	928
Table 718 – FAL_Pdu_GetVcrlId()	929
Table 719 – FAL_Pdu_InactivityCloseTime()	929
Table 720 – FAL_Pdu_TransmitDelayTime()	929
Table 721 – FAL_Pdu_SvcType()	929
Table 722 – FAL_Pdu_RemoteAddress()	929
Table 723 – FAL_Pdu_TrailerFields()	929
Table 724 – FAL_Pdu_ServiceSpecificParameters()	930
Table 725 – FAL_Pdu_Valid()	930
Table 726 – MaxOutstandingReached()	930
Table 727 – StartInactivityCloseTimer()	930
Table 728 – Primitives issued by ARPM to DMPM	931
Table 729 – Primitives issued by DMPM to ARPM	931
Table 730 – Parameters used with primitives exchanged between ARPM and DMPM	931
Table 731 – Primitives exchanged between the Socket model and DMPM	931
Table 732 – Parameters of DMPM/Socket model primitives	932
Table 733 – DMPM state descriptions	932
Table 734 – DMPM state table – sender transitions	933
Table 735 – DMPM state table – receiver transitions	934
Table 736 – ConnectionOriented	934
Table 737 – GetBufferedData	934
Table 738 – GetConnectionId	934
Table 739 – LoadBuffer	934
Table 740 – RemainingBufferSizeCheck	935
Table 741 – StartTransmitDelayTimer	935
Table 742 – Type 6 specific transition	936
Table 743 – Primitives issued by FSPM to ARPM	937
Table 744 – Primitives issued by ARPM to FSPM	937
Table 745 – Parameters used with primitives exchanged between FSPM and ARPM	937
Table 746 – QUB ARPM states	939
Table 747 – QUB-PC ARPM state table – sender transactions	940
Table 748 – QUB-PC ARPM state table – receiver transactions	942
Table 749 – Function GetArepld ()	945
Table 750 – Function BuildFAL-PDU	945
Table 751 – Function FAL_Pdu_Type	946
Table 752 – Function AbortIdentifier	946
Table 753 – Function AbortReason	946
Table 754 – Function AbortDetail	946
Table 755 – Function ApExplicitConnection	946
Table 756 – Function ImmediateAcknowledge	946
Table 757 – Function ArFspmService	947
Table 758 – Function ArAcceeSupported	947
Table 759 – Function MaxFalPduLengthTest	947
Table 760 – Primitives issued by FSPM to ARPM	947
Table 761 – Primitives issued by ARPM to FSPM	948

Table 762 – Parameters used with primitives exchanged between FSPM and ARPM	948
Table 763 – BNU ARPM states	949
Table 764 – BNU-PC ARPM state table - sender transactions	950
Table 765 – BNU-PC ARPM state table - receiver transactions	951
Table 766 – Function GetArepld ()	951
Table 767 – Function BuildFAL-PDU	952
Table 768 – Function FAL_Pdu_Type	952
Table 769 – Function MaxFalPduLengthTest	952
Table 770 – Primitives issued by ARPM to DMPM	953
Table 771 – Primitives issued by DMPM to ARPM	954
Table 772 – Parameters used with primitives exchanged between ARPM and DMPM	954
Table 773 – Primitives exchanged between Data Link Layer and DMPM	955
Table 774 – DMPM state descriptions	955
Table 775 – DMPM state table – sender transactions	956
Table 776 – DMPM state table – receiver transactions	958
Table 777 – Function PickArep	960
Table 778 – Function FindAREP	960
Table 779 – Function LocateArep	960
Table 780 – Example of encoding of a SEQUENCE	963
Table 781 – Example of encoding of a SEQUENCE OF	964
Table 782 – Example of encoding of a CHOICE	964
Table 783 – Example of encoding of an object identifier	965
Table 784 – Example of encoding of a PDU	966
Table 785 – MPS PDU types	969
Table 786 – Fields of a CompactValuePDU	970
Table 787 – Fields of a VariableDescriptionPDU	976
Table 788 – Fields of AccessDescriptionPDU	978
Table 789 – Fields of TypeDescriptionPDU	979
Table 790 – Fields of a ListDescriptionPDU	979
Table 791 – Coding of the different MCS PDU types	981
Table 792 – Coding of the variable part of the PDU	981
Table 793 – Structure of Association Establishment Request	982
Table 794 – Structure of an associated establishment response	986
Table 795 – Structure of an association termination request	988
Table 796 – Structure of an association termination response	988
Table 797 – Structure of an association revocation request	989
Table 798 – Structure of an associated transfer request	990
Table 799 – Structure of an associated transfer acknowledgement	990
Table 800 – Structure of a non-associated transfer request	991
Table 801 – Structure of a non-associated transfer acknowledgement	992
Table 802 – Definitions of object classes	994
Table 803 – Definition of Sub-MMS Services	995
Table 804 – Structure of the antiduplication list	1069
Table 805 – Structure of the reassembly list	1074
Table 806 – PV_R/W parameter values	1082
Table 807 – PV_IND parameter values	1082

Table 808 – PV_LIS parameter values	1083
Table 809 – Constraints on PV_LIS parameter.....	1083
Table 810 – PV_AT parameter values	1083
Table 811 – PV_RE parameter values	1084
Table 812 – PV_UT parameter values.....	1084
Table 813 – Constraints on PV_RE parameter	1084
Table 814 – PH_R_A parameter values	1084
Table 815 – PH_R_S parameter values	1084
Table 816 – PH_R_P parameter values	1085
Table 817 – PH_P_A parameter values.....	1085
Table 818 – PH_P_S parameter values.....	1085
Table 819 – PH_P_P parameter values.....	1086
Table 820 – PH_COH parameter values.....	1086
Table 821 – PH_FIA parameter values.....	1086
Table 822 – PH_SPF parameter values	1086
Table 823 – PH_SPM parameter values.....	1087
Table 824 – PH_ACC parameter values.....	1087
Table 825 – PH_RES parameter values	1087
Table 826 – PH_AK parameter values	1087
Table 827 – PH_RA parameter values	1088
Table 828 – PH_SR parameter values	1088
Table 829 – PH_CF parameter values	1088
Table 830 – Constraints on PH_RA parameter	1088
Table 831 – Constraints on PH_SR parameter.....	1088
Table 832 – PT_OCT parameter values	1089
Table 833 – PT_BIN parameter values.....	1089
Table 834 – PT_VIS parameter values.....	1089
Table 835 – PT_BOO parameter values.....	1089
Table 836 – PT_BCD parameter values	1090
Table 837 – PT_BTM parameter values	1090
Table 838 – PT_INT parameter values.....	1090
Table 839 – PT_UNO parameter values	1090
Table 840 – PT_FPT parameter values	1091
Table 841 – PT_GTM parameter values.....	1091
Table 842 – PT_TAB parameter values.....	1091
Table 843 – PT_STR parameter values.....	1092
Table 844 – Constraints on PT_TAB parameter	1092
Table 845 – Constraints on PT_STR parameter	1092
Table 846 – Conformance classes for environment management.....	1095
Table 847 – Conformance classes for VMD management.....	1096
Table 848 – Conformance classes for PI management.....	1097
Table 849 – Conformance classes for domain management.....	1098
Table 850 – Conformance classes for variable/variable list management.....	1099
Table 851 – Conformance classes for event management.....	1100
Table 852 – Primitives issued by FAL to ARPM.....	1124
Table 853 – Primitives issued by ARPM to FAL.....	1124

Table 854 – Parameters used with Primitives Exchanged between FAL and ARPM.....	1124
Table 855 – QUB-TM ARPM States	1126
Table 856 – QUB-TM State Table - Sender Transactions	1126
Table 857 – QUB-TM State Table - Receiver Transactions	1127
Table 858 – Function GetArepld ()	1127
Table 859 – Function BuildFAL-PDU	1127
Table 860 – Function FAL_Pdu_Type	1127
Table 861 – Function ResetCounters	1128
Table 862 – Function IncrementCounter	1128
Table 863 – Function DecrementCounter	1128
Table 864 – Function GetCounterValue	1128
Table 865 – Primitives issued by ARPM to DMPM.....	1129
Table 866 – Primitives issued by DMPM to ARPM.....	1130
Table 867 – Parameters used with Primitives Exchanged between ARPM and DMPM.....	1130
Table 868 – Primitives exchanged between Type8 Data Link Layer and DMPM	1131
Table 869 – Type-8 DMPM state descriptions	1131
Table 870 – Type-8 DMPM state table – sender transactions	1132
Table 871 – Type-8 DMPM state table – receiver transactions	1136
Table 872 – Function PickArep	1137
Table 873 – Function FindAREP	1137
Table 874 – Function SelectNextArep	1137
Table 875 – Function ArepRole.....	1138
Table 876 – Function FalArHeader.....	1138
Table 877 – Function AddUcsPduHeader.....	1138
Table 878 – Function RemoveUcsPduHeader	1138
Table 879 – Function DILinkStatus.....	1138
Table 880 – Field with Fixed Format	1141
Table 881 – Object codes	1157
Table 882 – Standard data types	1157
Table 883 – Reason codes	1157
Table 884 – Coding for Date Type	1163
Table 885 – Primitives issued by VCR PM to FSPM	1169
Table 886 – Primitives issued by FSPM to VCR PM.....	1170
Table 887 – Primitives issued by FAL-User to VCR PM.....	1170
Table 888 – Primitives issued by VCR PM to FAL-User.....	1171
Table 889 – AP-VCR state machine transactions	1173
Table 890 – FSPM state table – sender transactions	1194
Table 891 – FSPM state table – receiver transactions.....	1195
Table 892 – QUU ARPM states	1196
Table 893 – QUU ARPM state table – sender transactions.....	1197
Table 894 – QUU ARPM state table – receiver transactions.....	1198
Table 895 – QUB ARPM states	1199
Table 896 – QUB ARPM state table – sender transactions.....	1200
Table 897 – QUB ARPM state table – receiver transactions	1202
Table 898 – BNU ARPM states	1207
Table 899 – BNU ARPM state table – sender transactions	1208

Table 900 – BNU ARPM state table – receiver transactions	1210
Table 901 – DMPM state descriptions	1212
Table 902 – DMPM state table – sender transactions.....	1213
Table 903 – DMPM state table – receiver transactions.....	1216
Table 904 – Primitives issued by FAS User to FSPM	1220
Table 905 – Primitives issued by FSPM to FAS User	1220
Table 906 – Primitives issued by FSPM to ARPM	1221
Table 907 – Primitives issued by ARPM to FSPM	1222
Table 908 – Parameters used with primitives exchanged between FSPM and ARPM	1222
Table 909 – Primitives issued from ARPM to DMPM	1223
Table 910 – Primitives issued by DMPM to ARPM.....	1223
Table 911 – Parameters used with primitives exchanged between ARPM and DMPM	1224
Table 912 – Primitives exchanged between Data Link Layer and DMPM.....	1225
Table 913 – Function SelectArep().....	1227
Table 914 – Function GetArepId()	1227
Table 915 – Function BuildFAS-PDU	1227
Table 916 – Function FAS_Pdu_Type	1227
Table 917 – Function AbortIdentifier	1228
Table 918 – Function AbortReason	1228
Table 919 – Function AbortDetail.....	1228
Table 920 – Function PickArep	1228
Table 921 – Function FindAREP	1228
Table 922 – Function LocateQubArep	1229
Table 923 – Function SetIdentifier()	1229
Table 924 – Error Messages	1231
Table 925 – VARTYPE.....	1233
Table 926 – ITEMQUALITYDEF values.....	1234
Table 927 – STATEDEF values.....	1235
Table 928 – GROUPEXCEPTIONDEF values	1235
Table 929 – ACCESSRIGHTSDEF values.....	1235
Table 930 – PERSISTDEF values	1236
Table 931 – UUID values	1238
Table 932 – Data format for serialized connection data.....	1247
Table 933 – Primitives issued by FAL User to FSPM.....	1249
Table 934 – Primitives issued by FSPM to FAL User.....	1254
Table 935 – FSPM state descriptions.....	1258
Table 936 – FSPM state table.....	1259
Table 937 – Primitives issued by FSPM to ARPM	1303
Table 938 – Primitives issued by ARPM to FSPM	1304
Table 939 – Parameters used with primitives exchanged between FSPM and ARPM	1304
Table 940 – ARPM state descriptions	1304
Table 941 – ARPM state table	1305
Table 942 – Primitives issued by ARPM to DMPM.....	1306
Table 943 – Primitives issued by DMPM to ARPM.....	1306
Table 944 – Parameters used with primitives exchanged between ARPM and DMPM	1307
Table 945 – Primitives issued by DMPM to ORPC model.....	1307

Table 946 – Primitives issued by ORPC model to DMPM	1308
Table 947 – Parameters used with primitives exchanged between DMPM and ORPC model	1308
Table 948 – DMPM state descriptions	1308
Table 949 – DMPM state table	1309

INTERNATIONAL ELECTROTECHNICAL COMMISSION

DIGITAL DATA COMMUNICATIONS FOR MEASUREMENT AND CONTROL – FIELD BUS FOR USE IN INDUSTRIAL CONTROL SYSTEMS –

Part 6: Application Layer protocol specification

FOREWORD

- 1) The IEC (International Electrotechnical Commission) is a worldwide organization for standardization comprising all national electrotechnical committees (IEC National Committees). The object of the IEC is to promote international co-operation on all questions concerning standardization in the electrical and electronic fields. To this end and in addition to other activities, the IEC publishes International Standards. Their preparation is entrusted to technical committees; any IEC National Committee interested in the subject dealt with may participate in this preparatory work. International, governmental and non-governmental organizations liaising with the IEC also participate in this preparation. The IEC collaborates closely with the International Organization for Standardization (ISO) in accordance with conditions determined by agreement between the two organizations.
- 2) The formal decisions or agreements of the IEC on technical matters express, as nearly as possible, an international consensus of opinion on the relevant subjects since each technical committee has representation from all interested National Committees.
- 3) The documents produced have the form of recommendations for international use and are published in the form of standards, technical specifications, technical reports or guides and they are accepted by the National Committees in that sense.
- 4) In order to promote international unification, IEC National Committees undertake to apply IEC International Standards transparently to the maximum extent possible in their national and regional standards. Any divergence between the IEC Standard and the corresponding national or regional standard shall be clearly indicated in the latter.
- 5) The IEC provides no marking procedure to indicate its approval and cannot be rendered responsible for any equipment declared to be in conformity with one of its standards.

The International Electrotechnical Commission (IEC) draws attention to the fact that it is claimed that compliance with this International Standard may involve the use of a patent as follows, where the [xx] notation indicates the holder of the patent right:

Type 8 and possibly other types:

DE 197 39 297 A1 [PxC] "Automatisierungssystem und Steuervorrichtung zur transparenten Kommunikation zwischen verschiedenen Netzwerken."

US Application number 09/145,848 Filing Date 09/02/98
[PxC] "Automation System and connecting Apparatus for the Transparent Communication between two Networks."

The IEC takes no position concerning the evidence, validity and scope of these patent rights.:

The holders of these patent rights have assured the IEC that they are willing to negotiate licences under reasonable and non-discriminatory terms and conditions with applicants throughout the world. In this respect, the statement of the holders of these patent rights are registered with the IEC. Information may be obtained from:

[PxC]: Phoenix Contact GmbH & Co. KG
Referat Patente / Patent Department
Postfach 1341
D-32819 Blomberg
Germany

Attention is drawn to the possibility that some of the elements of this International Standard may be the subject of patent rights other than those identified above. IEC shall not be held responsible for identifying any or all such patent rights.

International standard IEC 61158-6 has been prepared by subcommittee 65C: Digital communications, of IEC technical committee 65: Industrial-process measurement and control.

This third edition cancels and replaces the second edition published in 2000. This edition constitutes a technical revision.

The text of this standard is based on the prior edition and the following documents:

FDIS	Report on voting
65C/293/FDIS	65C/301/RVD

Full information on the voting for the approval of this standard can be found in the report on voting indicated in the above table.

This edition includes the following significant changes from the prior edition:

- a) updates to Types 1, 2, 3, 4, 5, 7 and 8;
- b) addition of Types 6, 9 and 10.

This publication has been drafted in accordance with the ISO/IEC Directives, Part 2.

The committee has decided that the contents of this publication will remain unchanged until 2007. At this date, the publication will be

- reconfirmed;
- withdrawn;
- replaced by a revised edition, or
- amended.

IEC 61158 consists of the following parts, under the general title *Digital data communications for measurement and control — Fieldbus for use in industrial control systems*:

- Part 1: Overview and guidance for the IEC 61158 series
- Part 2: Physical Layer specification and service definition
- Part 3: Data Link Service definition
- Part 4: Data Link Protocol specification
- Part 5: Application Layer Service definition
- Part 6: Application Layer protocol specification

The contents of the corrigendum of July 2004 have been included in this copy.

0 Introduction

This part of IEC 61158 is one of a series produced to facilitate the interconnection of automation system components. It is related to other standards in the set as defined by the “three-layer” Fieldbus Reference Model, which is based in part on the Basic Reference Model for Open Systems Interconnection. Both Reference Models subdivide the area of standardisation for interconnection into a series of layers of specification, each of manageable size.

This standard describes the Fieldbus Application Layer (FAL) protocol that defines the information interchange and the interactions between Application Entity invocations (AE-Is) to support the Application Services.

The Application Protocol provides the Application Service by making use of the services available from the Data Link Layer or other immediately lower layer. The relationship between the International Standards for Fieldbus Application Service, Fieldbus Application Protocol, Fieldbus Data Link Service and System Management is illustrated in Figure 1.

NOTE Systems Management, as used in this standard, is a local mechanism for managing the layer protocols

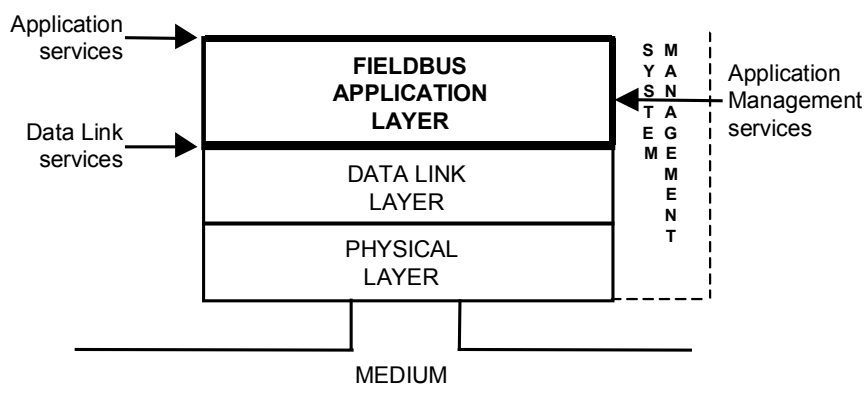


Figure 1 – Relationship of IEC 61158-6 to other fieldbus layers and to users of the fieldbus Application service

An Application Process uses the Fieldbus Application Layer services to exchange information with other Application Processes. The services define the abstract interface between the application process and the Application Layer.

The Application Layer protocol is the set of rules that governs the format and meaning of the information exchange between the Application Layers in various devices. The Application Layer uses the protocol to implement the Application Layer services definitions.

The protocol machine defines the various states of an Application Layer and the valid transitions between the states. It may be considered as a finite state machine. The protocol machine is described using state tables. The information is exchanged between the application process and the protocol machine through application service data units. The protocol machine exchanges information with other protocol machines through application protocol data units (APDU).

This set of Application Layer standards does not specify individual implementations or products, nor does it constrain the implementations of Application Entities (AEs) and interfaces within the industrial automation system.

This set of Application Layer standards does not contain test procedures to ensure compliance with such requirements.

The primary aim of this standard is to provide a set of rules for communication expressed in terms of the procedures to be carried out by peer Data Link entities at the time of communication. These rules for communication are intended to provide a sound basis for development in order to serve a variety of purposes:

- a) as a guide for implementors and designers;
- b) for use in the testing and procurement of equipment;
- c) as part of an agreement for the admittance of systems into the open systems environment;
- d) as a refinement to the understanding of time-critical communications within OSI.

This standard is concerned, in particular, with the communication and interworking of sensors, effectors and other automation devices. By using this standard together with other standards positioned within the OSI or Fieldbus Reference Models, otherwise incompatible systems may work together in any combination.

DIGITAL DATA COMMUNICATIONS FOR MEASUREMENT AND CONTROL – FIELDBUS FOR USE IN INDUSTRIAL CONTROL SYSTEMS –

Part 6: Application Layer protocol specification

1 Scope

The Fieldbus Application Layer (FAL) is an Application Layer communication standard designed to support the conveyance of time-critical application requests and responses among devices in an automation environment. The term “time-critical” is used to represent the presence of a time-window, within which one or more specified actions are required to be completed with some defined level of certainty. Failure to complete specified actions within the time window risks failure of the applications requesting the actions, with attendant risk to equipment, plant and possibly human life.

This part of this standard specifies interactions between remote applications in terms of

- the encoding rules that are applied to all the Application Layer Protocol Data Units (APDUs);
- the formal Abstract Syntax definitions of such APDUs;
- the protocol state machine descriptions that handle the APDUs and the primitives in the correct sequences;
- the mappings of the APDUs to and from the Data Link Layer services defined in IEC 61158-3.

The FAL encoding rules are designed assuming that both the encoder (sender) and the decoder (receiver) have the common knowledge of the abstract syntax. Wherever possible, data types identifiers are not encoded and transferred over the network.

NOTE This is why the Abstract Syntax Notation One / Basic Encoding Rule is not practical for the FAL.

The purpose of this part of this standard is to define the protocol provided

- to the Fieldbus Data Link Layer at the boundary between the Application and Data Link Layers of the Fieldbus Reference Model, and
- to the System Management at the boundary between the System Management and Application Layers of the Fieldbus Reference Model.

This standard defines 10 different Application Layer protocols; each corresponds to the Application Layer service definitions specified in the IEC 61158-5 standard. They are identified in the 61158 series of specifications as Type 1, Type 2, Type 3, Type 4, Type 5, Type 6, Type 7, Type 8, Type 9, and Type 10. Although it may be possible to use different types of protocols on the same network provided that the underlying lower layers are compatible, this specification does not assure such interoperability among different types.

2 Normative references

The following referenced documents are indispensable for the application of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

IEC 60559:1989, *Binary Floating-point Arithmetic for Microprocessor Systems*

IEC 61131-3:1993, *Programmable controllers – Part 3: Programming languages*

IEC 61158, *Digital data communications for measurement and control — Fieldbus for use in industrial control systems*

IEC 61158-3:2003, *Digital data communications for measurement and control — Fieldbus for use in industrial control systems — Data Link service definition*

IEC 61158-4:2003, *Digital data communications for measurement and control — Fieldbus for use in industrial control systems — Data Link protocol specification*

IEC 61158-5:2003, *Digital data communications for measurement and control — Fieldbus for use in industrial control systems — Application layer service definition*

ISO/IEC 7498 (all parts), *Information technology – Open Systems Interconnection – Basic Reference Model*

ISO/IEC 8822:1994, *Information technology – Open Systems Interconnection – Presentation service definition*

ISO/IEC 8824:1990, *Information technology – Open Systems Interconnection – Specification of Abstract Syntax Notation One (ASN.1)*

ISO/IEC 8825:1990, *Information technology – Open Systems Interconnection – Specification of Basic Encoding Rules for Abstract Syntax Notation One (ASN.1)*

ISO/IEC 9506-2:1990, *Industrial automation systems – Manufacturing Message Specification – Part 2: Protocol specification*

ISO/IEC 9545:1994, *Information technology – Open Systems Interconnection – Application Layer structure*

ISO/IEC 10646 (all parts), *Information technology – Universal Multiple-Octet Coded Character Set (UCS)*

ISO/IEC 10731:1994, *Information technology – Open Systems Interconnection – Basic Reference Model – Conventions for the definition of OSI services*

ISO 8649:1996, *Information technology – Open Systems Interconnection – Service definition for the Association Control Service Element Protocol specification*

ISO 8650 (all parts), *Information technology – Open Systems Interconnection – Connection-oriented protocol for the Association Control Service Element Protocol specification*

3 Terms and definitions

3.1 Summary

For the purposes of this document, the following definitions apply.

3.2 Terms and definitions from other ISO/IEC standards

3.2.1 Terms and definitions from ISO/IEC 7498-1

- a) application entity;
- b) application process;
- c) application protocol data unit;
- d) application service element;
- e) application entity invocation;
- f) application process invocation;
- g) application transaction;
- h) real open system;
- i) transfer syntax.

3.2.2 Terms and definitions from ISO/IEC 8822

- a) abstract syntax;
- b) presentation context.

3.2.3 Terms and definitions from ISO/IEC 9545

- a) application-association;
- b) application-context;
- c) application context name;
- d) application-entity-invocation;
- e) application-entity-type;
- f) application-process-invocation;
- g) application-process-type;
- h) application-service-element;
- i) application control service element.

3.2.4 Terms and definitions from ISO/IEC 8824

- a) object identifier;
- b) type;
- c) value;
- d) simple type;
- e) structured type;
- f) component type;
- g) tag;
- h) Boolean type;
- i) true;
- j) false;
- k) integer type;
- l) bitstring type;
- m) octetstring type;
- n) null type;
- o) sequence type;
- p) sequence of type;
- q) choice type;
- r) tagged type;
- s) any type;
- t) module;
- u) production.

3.2.5 Terms and definitions from ISO/IEC 8825

- a) encoding (of a data value);
- b) data value;
- c) Identifier Octets (the singular form is used in this standard);
- d) Length Octet(s) (both singular and plural forms are used in this standard);
- e) Contents Octets.

3.3 Terms and definitions from IEC 61158-5

- a) application relationship;
- b) conveyance path;
- c) client;
- d) dedicated AR;
- e) dynamic AR;
- f) error class;
- g) error code;
- h) name;
- i) numeric identifier;
- j) peer;
- k) pre-defined AR endpoint;
- l) pre-established AR endpoint;
- m) publisher;
- n) subscriber;
- o) server.

3.4 Other terms and definitions

3.4.1 Summary

NOTE Type 1 definitions may apply to all the types except if the same item is used with different definitions in other type. In this case the latter has precedence.

The following terms and definitions are used in this standard.

3.4.2 Type 1 definitions

3.4.2.1 called

service user or a service provider that receives an indication primitive or a request APDU

3.4.2.2 calling

service user or a service provider that initiates a request primitive or a request APDU

3.4.2.3 interoperability

capability of User Layer entities to perform coordinated and cooperative operations using the services of the FAL

3.4.2.4 management information

network accessible information that supports the management of the Fieldbus environment

3.4.2.5 receiving

service user that receives a confirmed primitive or an unconfirmed primitive, or a service provider that receives a confirmed APDU or an unconfirmed APDU

3.4.2.6 resource

resource is a processing or information capability of a subsystem

3.4.2.7 sending

service user that sends a confirmed primitive or an unconfirmed primitive, or a service provider that sends a confirmed APDU or an unconfirmed APDU

3.4.3 Type 2 definitions

3.4.3.1 allocate

take a resource from a common area and assign that resource for the exclusive use of a specific entity

3.4.3.2 application

function or data structure for which data is consumed or produced

3.4.3.3 application objects

multiple object classes that manage and provide a run time exchange of messages across the network and within the network device

3.4.3.4 attribute

description of an externally visible characteristic or feature of an object

NOTE The attributes of an object contain information about variable portions of an object. Typically, they provide status information or govern the operation of an object. Attributes may also affect the behaviour of an object. Attributes are divided into class attributes and instance attributes.

3.4.3.5 behaviour

indication of how an object responds to particular events

3.4.3.6 class

set of objects, all of which represent the same kind of system component

NOTE A class is a generalisation of an object; a template for defining variables and methods. All objects in a class are identical in form and behaviour, but usually contain different data in their attributes.

3.4.3.7 class attributes

attribute that is shared by all objects within the same class

3.4.3.8 class code

unique identifier assigned to each object class

3.4.3.9 class specific service

service defined by a particular object class to perform a required function which is not performed by a common service

NOTE A class specific object is unique to the object class which defines it

3.4.3.10 client

- a) object which uses the services of another (server) object to perform a task
- b) initiator of a message to which a server reacts

3.4.3.11 communication objects

components that manage and provide a run time exchange of messages across the network

EXAMPLES Connection Manager object, Unconnected Message Manager (UCMM) object, Message Router object.

3.4.3.12 connection

logical binding between application objects that may be within the same or different devices

NOTE Connections may be either point-to-point or multipoint.

3.4.3.13 connection ID (CID)

identifier assigned to a transmission that is associated with a particular connection between producers and consumers, providing a name for a specific piece of application information

3.4.3.14 connection path

an octet stream which defines the application object to which a connection instance applies

3.4.3.15 connection point

buffer which is represented as a subinstance of an Assembly object

3.4.3.16 consume

act of receiving data from a producer

3.4.3.17 consumer

node or sink that is receiving data from a producer

3.4.3.18 consuming application

application that consumes data

3.4.3.19 cyclic

repetitive in a regular manner

3.4.3.20 device

physical hardware connected to the link

NOTE: A device may contain more than one node.

3.4.3.21 device profile

collection of device dependent information and functionality providing consistency between similar devices of the same device type

3.4.3.22 end node

producing or consuming node

3.4.3.23 end point

one of the communicating entities involved in a connection

3.4.3.24 error

discrepancy between a computed, observed or measured value or condition and the specified or theoretically correct value or condition

3.4.3.25 frame

denigrated synonym for DLPDU

3.4.3.26 instance

the actual physical occurrence of an object within a class, identifying one of many objects within the same object class.

EXAMPLE California is an instance of the object class state.

NOTE The terms object, instance, and object instance are used to refer to a specific instance.

3.4.3.27 instance attribute

attribute that is unique to an object instance and not shared by the object class

3.4.3.28 instantiated

object that has been created in a device

3.4.3.29 keeper

object responsible for distributing link configuration data to all nodes on the link

3.4.3.30 little endian

Describes a model of memory organisation which stores the least significant octet at the lowest address, or for transfer, which transfers the lowest order octet first

3.4.3.31 Lpacket

a piece of application information that contains a size, control octet, tag, and link data.

NOTE Peer Data Link Layers use Lpackets to send and receive service data units from higher layers in the OSI stack.

3.4.3.32 member

piece of an attribute that is structured as an element of an array

3.4.3.33 message router

object within a node that distributes messaging requests to appropriate application objects

3.4.3.34 multipoint connection

connection from one node to many

NOTE Multipoint connections allow messages from a single producer to be received by many consumer nodes.

3.4.3.35 network

a set of nodes connected by some type of communication medium, including any intervening repeaters, bridges, routers and lower-layer gateways

3.4.3.36 object

abstract representation of a particular component within a device, usually a collection of related data (in the form of variables) and methods (procedures) for operating on that data that have clearly defined interface and behaviour

3.4.3.37 object specific service

service unique to the object class which defines it

3.4.3.38 originator

client responsible for establishing a connection path to the target

3.4.3.39 point-to-point connection

connection that exists between exactly two application objects

3.4.3.40 produce

act of sending data to be received by a consumer

3.4.3.41 producer

node that is responsible for sending data

3.4.3.42 server

- a) role of an AREP in which it returns a confirmed service response APDU to the client that initiated the request
- b) object which provides services to another (client) object

3.4.3.43 service

operation or function than an object and/or object class performs upon request from another object and/or object class

3.4.3.44 target

end-node to which a connection is established

3.4.3.45 temporary node

transient node

3.4.3.46 transaction id

field within a UCMM header that matches a response with the associated request

3.4.3.47 unconnected message manager (UCMM)

component within a node that transmits and receives unconnected explicit messages and sends them directly to the Message Router object

3.4.3.48 unconnected service

messaging service which does not rely on the set up of a connection between devices before allowing information exchanges

3.4.4 Type 3 definitions

3.4.4.1 access protection

limitation of the usage of an application object to one client

3.4.4.2 address-assignment-table

mapping of the client's internal I/O-Data object storage to the decentralised input and output data objects

3.4.4.3 channel

single physical or logical link of an input or output application object of a server to the process

3.4.4.4 channel related diagnosis

information concerning a specific element of an input or output application object, provided for maintenance purposes

EXAMPLE: validity of data

3.4.4.5 configuration check

comparison of the expected I/O-Data object structuring of the client with the real I/O-Data object structuring to the server in the start-up phase

3.4.4.6 configuration fault

an unacceptable difference between the expected I/O-Data object structuring and the real I/O-Data object structuring, as detected by the server

3.4.4.7 configuration identifier

representation of a portion of I/O Data of a single input- and/or output-module of a server.

3.4.4.8 configuration identifier related diagnosis

compromises all module specific data available at the server for maintenance purposes

3.4.4.9 control commands

action invocations transferred from client to server to clear outputs, freeze inputs and/or synchronise outputs

3.4.4.10 data consistency

means for coherent transmission and access of the input- or output-data object between and within client and server

3.4.4.11 Data-eXchange-Broadcast (DXB)

service for conveyance of information between the DP-master (Class 1) and the assigned DP-slaves initiating the Publisher and Subscriber functionality in the DP-slaves

3.4.4.12 default DL-address

value 126 as an initial value for DL-address, which has to be changed (e.g. by assignment of an DL-address via the fieldbus) before operation with a DP-master (class 1)

3.4.4.13 device related diagnosis

compromises all data related to the whole DP-slave available at the server for maintenance purposes

3.4.4.14 diagnosis information

all data available at the server for maintenance purposes

3.4.4.15 diagnosis information collection

system diagnosis information that is assembled at the client side

3.4.4.16 DP-master (class 1)

a controlling device which controls several DP-slaves (field devices)

NOTE: This is usually a programmable controller or a distributed control system.

3.4.4.17 DP-master (class 2)

controlling device which manages configuration data (parameter sets) and diagnosis data of a DP-master (Class 1), and that additionally performs all communication capabilities of a DP-master (Class 1)

3.4.4.18 DP-slave

field device that can be assigned to one DP-master (Class 1) as a provider for cyclic I/O data exchange, in addition acyclic functions and alarms could be provided

3.4.4.19 DXB request

cyclic data exchange service request with a specific DL service between the DP-master (Class 1) and the assigned DP-slaves initiating the Publisher functionality in the DP-slaves

3.4.4.20 DXB response

cyclic data exchange service response not to the individual address of the requester (DP-master (Class 1)) but to the broadcast address (=127)

3.4.4.21 freeze

function at the DP-slaves for simultaneous data transfer between the input data object and the process

3.4.4.22 group

set of DP-slaves which perform a Freeze or Sync function

3.4.4.23 I/O data

object designated to be transferred cyclically for the purpose of processing.

3.4.4.24 ident number

DP-master (Class 1) or DP-slave device type

3.4.4.25 index

address of an object within an application process

3.4.4.26 isochronous mode

special operation mode of a DP system that implies both a constant DP cycle with a fixed schedule of the cyclic and acyclic DP services, and the synchronisation of the application in the DP-master (Class 1) and the DP-slaves with this constant DP cycle

3.4.4.27 master parameter set

the configuration and parameterisation data of all DP-slaves that are assigned to the corresponding DP-master and the bus parameters

3.4.4.28 module

addressable unit inside the DP-slave

3.4.4.29 process data

object(s) which are already pre-processed and transferred acyclically for the purpose of information or further processing

3.4.4.30 publisher

role of a CR endpoint that transmits APDUs onto the Fieldbus for consumption by one or more subscribers

NOTE: A publisher may not be aware of the identity or the number of subscribers and it may publish its APDUs using a dedicated CR.

3.4.4.31 real configuration

input and output data structure of a DP-slave, including definition of data consistency

3.4.4.32 subscriber

role of a CREP in which it receives APDUs produced by a publisher

3.4.4.33 slot

address of a module within a DP-slave

3.4.4.34 sync

function at the DP-slaves for simultaneous data transfer between the output data object and the process

3.4.5 Type 4 definitions

None.

3.4.6 Type 5 definitions

None.

3.4.7 Type 6 definitions

3.4.7.1 called

service user or a service provider that receives an indication primitive or a request APDU

3.4.7.2 calling

service user or a service provider that initiates a request primitive or a request APDU

3.4.7.3 master

AREP which can initiate a connection on the associated QUB-PC AR

3.4.7.4 receiving

service user that receives a confirmed primitive or an unconfirmed primitive, or a service provider that receives a confirmed APDU or an unconfirmed APDU

3.4.7.5 sending

service user that sends a confirmed primitive or an unconfirmed primitive, or a service provider that sends a confirmed APDU or an unconfirmed APDU

3.4.7.6 slave

AREP which can not initiate a connection on the associated QUB-PC AR

3.4.8 Type 7 definitions

None.

3.4.9 Type 8 definitions

None.

3.4.10 Type 9 definitions

Type 9 uses the same definitions as Type 1.

3.4.11 Type 10 definitions**3.4.11.1 active connection control object**

instance of a certain FAL class that abstracts the interconnection facility (as Consumer and Provider) of an automation device

3.4.11.2 configuration data base

interconnection information maintained by the ACCO ASE

3.4.11.3 connection

the logical link between sink and source of attributes and services at different custom interfaces of RT-Auto ASEs is referred to as connection

NOTE There is a distinction between data and event connections. The logical link and the data flow between sink and source of automation data items is referred to as data connection. The logical link and the data flow between sink (method) and source (event) of operational services is referred to as event connection

3.4.11.4 connection channel

description of a connection between a sink and a source of data items

3.4.11.5 consumer

node or sink that is receiving data from a producer

3.4.11.6 consumerID

unambiguous identifier within the scope of the ACCO assigned by the consumer to recognize the internal data of a configured interconnection sink

3.4.11.7 data marshaling

the encoding of parameters of the FAL service primitives with respect to their interface definition

NOTE This is part of the abstract ORPC model.

3.4.11.8 engineering

abstract term that characterizes the client application or device responsible for configuring an automation system via interconnecting data items

3.4.11.9 event

an instance of a change of conditions

3.4.11.10 interface

collection of FAL class attributes and services that represents a specific view on the FAL class

3.4.11.11 interface definition language

syntax and semantics of describing service parameters in a formal way

NOTE This description is the input for the ORPC model, especially for the ORPC wire protocol.

3.4.11.12 interface pointer

key attribute that unambiguously addresses an object interface instance

3.4.11.13 logical device

a certain FAL class that abstracts a software component or a firmware component as an autonomous self-contained facility of an automation device

3.4.11.14 method

<object> a synonym for an operational service which is provided by the server ASE and invoked by a client

3.4.11.15 object remote procedure call

model for object oriented or component based remote method invocation

3.4.11.16 physical device

a certain FAL class that abstracts the hardware facilities of an automation device

3.4.11.17 property

a synonym for ASE attributes which are readable or writeable via operational ASE services

NOTE These services are generally named "get_<Attribute Name>" or "set_<Attribute Name>" and correspond with the IDL keywords "propget" and "propput" used in Type 10 clauses of IEC 61158-6.

3.4.11.18 provider

source of a data connection

3.4.11.19 providerID

an unambiguous identifier within the scope of the ACCO assigned by the provider to recognize the internal data of a configured connection source

3.4.11.20 quality code aware

attribute of the RT-Auto class that indicates that an RT-Auto object uses a status code for its data items

3.4.11.21 quality code

additional status information of a data item

3.4.11.22 quality code unaware

opposite of quality code aware

3.4.11.23 RT-auto

an FAL class that abstracts the automation function as a process-related component of an automation device

3.4.11.24 runtime object model

objects that exist in a device together with their interfaces and methods that are accessible

3.5 Abbreviations and symbols**3.5.1 Common note**

NOTE Type 1 abbreviations and symbols may apply to all the types except where the same abbreviation or symbols are used in different ways with other types. In this case the latter has precedence.

3.5.2 Type 1 Abbreviations and symbols

AE	Application Entity
AE-I	Application Entity Invocation
AL	Application Layer
AP	Application Process
Ap_	Prefix for Data types defined for AP ASE
Ar_	Prefix for Data types defined for AR ASE
APDU	Application Protocol Data Unit
AR	Application Relationship
AREP	Application Relationship End Point
ASE	Application Service Element
ASN.1	Abstract Syntax Notation One
BCD	Binary Coded Decimal
BER	Basic Encoding Rule
cnf	confirmation primitive
DI_	Prefix for Data types defined for Data Link Layer types
DL	Data Link
DLC	Data Link Connection
DLCEP	Data Link Connection End Point
DLPDU	Data Link Protocol Data Unit
DLSAP	Data Link Service Access Point
DLSDU	Data Link Service Data Unit
Dt_	Prefix for Data types defined for Data Type ASE

Err	Error (used to indicate an APDU type)
Er_	Prefix for Error types defined
Ev_	Prefix for Data types defined for Event ASE
FAL	Fieldbus Application Layer
Fi_	Prefix for Data types defined for Function Invocation ASE
FIFO	First In First Out
Gn_	Prefix for Data Types defined for general use
ID	Identifier
IEC	International Electrotechnical Commission
in	input primitive
ind	indication primitive
ISO	International Organization for Standardization
LAS	Link Active Scheduler
Lr_	Prefix for Data types defined for Load Region ASE
lsb	least significant bit
Mn_	Prefix for Data Types defined for Management ASE
msb	most significant bit
out	output primitive
OSI	Open Systems Interconnection
PDU	Protocol Data Unit
PICS	Protocol Implementation Conformance Statement
QoS	Quality Of Service
Req	Request (used to indicate an APDU type)
req	request primitive
Rsp	Response (used to indicate an APDU type)
rsp	response primitive
SAP	Service Access Point
SDU	Service Data Unit
ToS	Type Of Service
Vr_	Prefix for Data types defined for Variable ASE

3.5.3 Type 2 Abbreviations and symbols

ASCII	American Standard Code for Information Interchange
CID	connection ID
CIP	The control and information protocol; includes both connected and unconnected messaging.
DLL	Data Link Layer
PDU	protocol data unit
OSI	open systems interconnection (see ISO 7498)
Rcv	receive
Rx	receive
SDU	service data unit
SEM	state event matrix
STD	state transition diagram, used to describe object behaviour
Tx	transmit
Xmit	transmit
CM_API	actual packet interval
O2T or O⇒T	originator to target (connection parameters)
CM_RPI	requested packet interval
TUI	table unique identifier
T2O or T⇒O	target to originator (connection parameters)

3.5.4 Type 3 Abbreviations and symbols

Ack	Acknowledgement
Add	Address
AP	Application Process
APDU	Application Protocol Data Unit
API	Application Process Identifier
AR	Application Relationship
AREP	Application Relationship End Point
ARL	Application Relationship List
ASE	Application Service Element
Cfg	Configuration
cnf	confirmation primitive
CREP	Communication Relationship End Point
CRL	Communication Relationship List
D_	Destination
DLL	Data Link Layer
DLM	Data Link-management
DLSAP	Data Link Service Access Point
DLSDU	Data Link Service Data Unit
DMPM	Data Link Mapping Protocol Machine
DMPMM1	Data Link Mapping Protocol Machine DP-master (Class 1)
DMPMM2	Data Link Mapping Protocol Machine DP-master (Class 2)
DMPMS	Data Link Mapping Protocol Machine DP-slave
DSAP	Destination Service Access Point
DXB	Data eXchange Broadcast
FAL	Fieldbus Application Layer
FIFO	First In First Out
FSPMM1	FAL Service Protocol Machine DP-master (Class 1)
FSPMM2	FAL Service Protocol Machine DP-master (Class 2)
FSPMS	FAL Service Protocol Machine DP-slave
HSA	Highest Station Address
ID	Identifier
IEC	International Electrotechnical Commission
ind	indication primitive
Inp	Input
ISO	International Organization For Standardization
IsoM	Isochronous Mode
L_sdu	Link Service Data Unit
lsb	least significant bit
msb	most significant bit
OSI	Open Systems Interconnection
Outp	Output
Param	Parameter
PDU	Protocol Data Unit
RD_	Read
Rem	Remote
Req	Request (used to indicate an APDU type)
req	request primitive
REQ	Request to indicate a Request PDU
RES	Response to indicate a Response PDU
RM	Resource Manager
RPL_UPD	Reply Update
Rsp	Response (used to indicate an APDU type)
rsp	response primitive
S_	Source
SAP	Service Access Point
SCL	Security Level
SD	Start Delimiter
SDN	Send Data with no Acknowledge
SDR	Station Delay Responder

Seq	Sequence
Src	Source
SRD	Send and Request Data with Reply
SSAP	Source Service Access Point
TR	Technical Report
USIF	User Interface
WD	Watchdog

3.5.5 Type 4 Abbreviations and symbols

None.

3.5.6 Type 5 Abbreviations and symbols

None

3.5.7 Type 6 Abbreviations and symbols

Type 6 uses the Abbreviations and symbols of Type 1.

3.5.8 Type 7 Abbreviations and symbols

A	Actuator
ACSE	Standard ISO 8649/ISO8650
AEI	Application Entity Implementation
API	Invocation of the Application Process
BER	ISO8825 Basic Encoding Rules
BT	Buffer Transfer - Layer 2
C	Controller
CC	Cell Controller
CADM	Computer Assisted Design & Manufacturing
CAPM	Computer Assisted Product Management
Cnf	Confirm
CS	Control System
CSD	Control System Device
DLSE	Data Link Service Element
Ind	Service Indication
MCS	Messaging Common Services
MMS	Standard ISO9506
MPS	Periodic/Aperiodic Manufacturing Services
MSG	Messaging Services - Layer 2
MAP3.0	ISO functional Profile
OS	Operator station
Rp	Service Response
Rq	Request
S	Sensor

3.5.9 Type 8 Abbreviations and symbols

None.

3.5.10 Type 9 Abbreviations and symbols

None.

3.5.11 Type 10 Abbreviations and symbols

ACCO	Active Connection Control Object
IDL	Interface Definition Language
IP	Internet Protocol
DNS	Domain Name Service
LDev	Logical Device
ORPC	Object Remote Procedure Call
PDev	Physical Device
QoS	Quality of Service
QC	Quality Code
RT	Runtime
TCP	Transmission Control Protocol

3.6 Conventions

3.6.1 General concept

The FAL is defined as a set of object-oriented ASEs. Each ASE is specified in a separate subclause. Each ASE specification is composed of three parts: its class definitions, its services, and its protocol specification. The first two are contained in IEC 61158-5. The protocol specification for each of the ASEs is defined in this standard.

The class definitions define the attributes of the classes supported by each ASE. The attributes are accessible from instances of the class using the Management ASE services specified in IEC 61158-5 standard. The service specification defines the services that are provided by the ASE.

This standard uses the descriptive conventions given in ISO/IEC 10731.

3.6.2 Conventions for Type 1

3.6.2.1 Conventions for class definitions

The Data Link Layer mapping definitions are described using templates. Each template consists of a list of attributes for the class. The general form of the template is defined in IEC 61158-5.

3.6.2.2 Abstract syntax conventions

When the "optionalParametersMap" parameter is used, a bit number which corresponds to each OPTIONAL or DEFAULT production is given as a comment.

3.6.3 Conventions for Type 2

3.6.3.1 Attribute specification

Attributes are defined in an Attribute Table using the format and terms defined in Figure 2.

Attribute ID	Name	Data type	Semantics of values
--------------	------	-----------	---------------------

Figure 2 – Attribute table format and terms

The **Attribute ID** shall be a unique integer identification value assigned to an attribute. The valid ranges for Attribute IDs shall be as specified in 5.1.1.10.1.3 The Attribute ID shall identify the particular attribute being accessed.

Name shall refer to the name assigned to the attribute. An attribute may contain sub-elements, which may also have names, as is the case with the **STRUCT** of data type. The attribute name shall be the name which appears first, or at the top row in the name column (the row that contains the Attribute ID).

Every attribute shall be assigned a **Data type** which shall be either an elementary or a derived data type. The data types specified for the defined attributes shall be used in all implementations.

NOTE The elementary data types are defined in IEC 61158-5.

Semantics of values shall specify the meaning of the value(s) of the attribute. If this information needs more room than can fit in the table it will immediately follow the Class Attribute table. Included in the Class Attribute table will be an appropriate reference to this information.

If a Class Attribute is *optional*, then a default value or a special case processing method shall be defined such that the Client (Requester) can process the error message that occurs when accessing those objects that choose not to implement the class attribute.

3.6.3.2 Common services

3.6.3.2.1 Service_PDU definitions

Each service has unique parameters for request and response. The service request and response parameters shall be defined using service Request/Response parameter tables as defined in Figure 3.

Name	Type	Semantics of values
------	------	---------------------

Figure 3 – Service request/response parameter

Name shall refer to the name given to the service request/response parameter.

Type shall specify the data type of the service request/response parameter.

Semantics of values shall specify the meaning of the values of the service request/response parameter, e.g. "the value is counts of microseconds."

3.6.3.2.2 Get_Attribute_All response

3.6.3.2.2.1 General definition

When the Get_Attribute_All common service is included in the list of supported System/Object Management services for an object class, then the **Get_Attribute_All** response shall be detailed for this class : the sequence or order of the data returned in the Service_ResponsePDU shall be specified. There are three ways in which the Get_Attribute_All Service_ResponsePDU may be specified:

- list the ordering of the attributes in the response message;
- specify the actual data array of the response message;
- combine the above two methods such that the actual data array also contains the attribute numbers.

Whichever method is used, the rules specified in Table 1 shall be adhered to when specifying the Get_Attribute_All Service_ResponsePDU of an object class for both the Class Attributes and the Instance Attributes.

Table 1– Get_Attribute_All response service rules

Rule number	Rule
1	If the definition of the Get_Attribute_All response includes optional attributes, then default values shall be specified in the response description. These defaults shall be used if an implementation does not support the optional attributes. If any of the following optional attributes are included in an object specification, but not supported in the implementation of the object, then the following shall be adhered to: – if the class attribute “Optional attribute list” is not supported, the default value of zero shall be inserted into the response array and no optional attribute numbers shall follow; – if the class attribute “optional service list” is not supported, the default value of zero shall be inserted into the response array and no optional service numbers shall follow.
2	If new attributes are added to an existing object, those attributes shall be added to the end of the response attribute list or data array to ensure compatibility with different object revisions.
3	Whichever method is used to specify the response, it shall be done in such a way as to be unambiguous, including rules to deal with variable length fields and padding.
4	The Get_Attribute_All response shall include only the open attributes; it shall not include any vendor specific attributes.

Table 2 is an example of the attribute ordering method of specifying the service data portion of a Get_Attribute_All response for class level attributes of an object which supports **optional gettable** class attributes 1,2,3 and 4.

Table 2 – Example class level object/service specific reply data of Get_Attribute_All

Class attribute ID	Attribute name and default value
1	Revision, default = 0x0001
2	Max Instance, default = 0x0000
3	Number of Instances, default = 0x0000
4	Attribute list, number of attributes, default = 0x0000

Table 3 is an example of the data array method of specifying the service data portion of a Get_Attribute_All response for class level attributes of an object which supports **optional gettable** class attributes 1,2,3 and 4.

Table 3 – Example Get_Attribute_All data array method

Octet	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
0	Revision (low octet) Default = 1							
1	Revision (high octet) Default = 0							
2	Max Instance (low octet) Default = 0							
3	Max Instance (high octet) Default = 0							
4	Number of Instances (low octet) Default = 0							
5	Number of Instances (high octet) Default = 0							
6	Optional Attribute List : number of attributes (low octet) Default = 0							
7	Optional Attribute List : number of attributes (high octet) Default = 0							
8	Optional Attribute List : optional attribute #1 (low octet)							
9	Optional Attribute List : optional attribute #1 (high octet)							
2n + 6	Optional Attribute List : optional attribute #n (low octet)							
2n + 7	Optional Attribute List : optional attribute #n (high octet)							

3.6.3.2.2.2 Revisions

The defined Get_Attribute_All response for an object may increase in size with each revision of the object; however, to insure interoperability, the format of the first part of the response shall remain parseable by a client of the older revision of the object. Clients (Requesters) need not make use of this compatibility requirement.

NOTE The Revision class attribute is not the revision of an implementation (which is reflected in the Identity Object, Minor/Major revision status bits), but the revision of the *class definition*.

3.6.3.2.3 Set_Attribute_All request

3.6.3.2.3.1 General definition

When the Set_Attribute_All common service is included in the list of supported common services, then the format of the **Set_Attribute_All request** shall be included in the object specification. The sequence or order of the data supplied in the Service Data portion of the request shall be specified. There are three ways in which the Set_Attribute_All request may be specified:

- list the order of the attributes in the request message;
- specify the actual data array of the request message;
- combine the above two methods such that the actual data array also contains the attribute numbers.

Whichever method is used, the rules specified in Table 4 shall be adhered to when specifying an object's Set_Attribute_All request in the object specification for both the Class Attributes and the Instance Attributes.

Table 4 – Set_Attribute_All request service rules

Rule number	Rule
1	An object shall support the Set_Attribute_All service only if all settable attributes shown in the Set_Attribute_All request are implemented as settable.
2	If new settable attributes are added to an existing object, those attributes shall be added to the end of the request attribute list or data array.
3	Whichever method is used to specify the response, it shall be done in such a way as to be unambiguous, including rules to deal with variable length fields and padding.
4	The Set_Attribute_All request shall include only the open attributes. It shall not include any vendor specific attributes.

Table 5 is an example of the attribute ordering method of specifying the service data portion of a Set_Attribute_All request for instance level attributes of an object which supports required settable instance attributes 7,8,9,10,11 and 12.

Table 5 – Example Set_Attribute_All attribute ordering method

Class Attribute ID	Attribute name
7	Output Range
8	Value Data Type
9	Fault State
10	Idle State
11	Fault Value
12	Idle Value

Table 6 is an example of the data array method of specifying the service data portion of a Set_Attribute_All request for instance level attributes of an object which supports required settable instance attributes 7,8,9,10,11 and 12.

Table 6 – Example Set_Attribute_All data array method

Octet	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
0	Output Range							
1	Value Data Type							
2	Fault State							
3	Idle State							
4	Fault Value (low octet)							
5	Fault Value (high octet)							
6	Idle Value (low octet)							
7	Idle Value (high octet)							

3.6.3.2.3.2 Revisions

A server processing a Set_Attribute_All request may need to process more data than is expected by the server if the client has implemented a newer revision of this object while the server an older revision. As a result, the server object may have to process a Set_Attribute_All request that contains more data because the additional attributes were not recognised. If the server receives more data in a Set_Attribute_All request than it expects the server shall respond with a general status code equal to *Too much data* (0x15).

NOTE The Revision class attribute is not the revision of an implementation (which is reflected in the Identity Object, Minor/Major revision status bits), but the revision of the *class definition*.

3.6.4 Conventions for Type 3

3.6.4.1 Abstract syntax conventions

The following conventions and rules are used:

3.6.4.1.1 PDUs are described as octets or groups of octets

- 1) Groups of octets separated by a comma appear in the order they are transferred

If optional octets are not present the following octets appear without a gap

- 2) If octets or groups of octets are grouped within "{" }" the order is arbitrary
- 3) If octets or groups of octets are marked with "*" they may appear more than once. If it is used within a "{" }" section they may appear mixed with other octets or group of octets of this section.
- 4) Octets can be grouped or values can be assigned within "()"
- 5) If octets or groups of octets are grouped within "["]" the group can be omitted
- 6) Complex APDUs may be built by means of substitutions (sub-structures)
- 7) Exclusive selections of octets or groups of octets are separated by "^"

NOTE 1 The formal PDU example

AP_PDU=Octet1,OctetGroup1,[Octet2],[Octet3],[OctGroup2*],OctetGroup3^Octet4

According to this the following variants are valid on the wire (non exhaustive):

Variant 1: Octet1, OctetGroup1, Octet2, Octet3, OctetGroup2,OctetGroup3

Variant 2: Octet1, OctetGroup1, Octet2, Octet3, OctetGroup2, OctetGroup2, OctetGroup2,OctetGroup3

Variant 3: Octet1, OctetGroup1, OctetGroup2, OctetGroup2, OctetGroup2,OctetGroup3, OctetGroup2

Variant4: Octet1, OctetGroup1, OctetGroup2, OctetGroup3, OctetGroup2, OctetGroup2, OctetGroup2, OctetGroup2

Variant 5: Octet1, OctetGroup1, Octet3, Octet4

NOTE 2 The arbitrary order implies that groups of octets are characterised by a special header that is described within the coding rules.

NOTE 3 The APDU syntax implies that according to the maximum DLSDU a APDU shall not exceed 244 octets in total.

3.6.4.2 Convention for the encoding of reserved bits and octets

The term "reserved" may be used to describe bits in octets or whole octets. All bits or octets that are reserved should be set to zero at the sending side and shall not be tested at the receiving side except it is explicitly stated or if the reserved bits or octets are checked by a state machine.

The term "reserved" may also be used to indicate that certain values within the range of a parameter are reserved for future extensions. In this case the reserved values should not be used at the sending side and shall not be tested at the receiving side except it is explicitly stated or if the reserved values are checked by a state machine.

3.6.4.3 Conventions for the common coding s of specific field octets

APDUs may contain specific fields that carry information in a primitive and condensed way. These fields shall be coded in the order according to Figure 4.

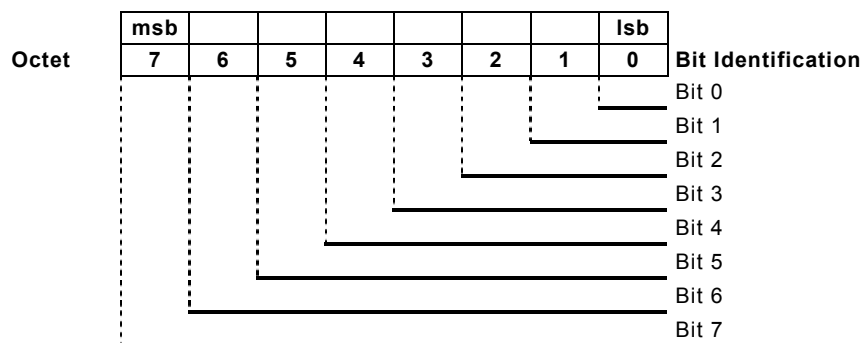


Figure 4 – Common structure of specific fields

- Bits may be grouped as group of bits. Each bit or group of bits shall be addressed by its Bit Identification (e.g. Bit 0, Bit 1 to 4). The position within the octet shall be according to the figure above. Alias names may be used for each bit or group of bits or they may be marked as reserved. The grouping of individual bits shall be in ascending order without gaps. The values for a group of bits may be represented as binary, decimal or hexadecimal values. This value shall only be valid for the grouped bits and can only represent the whole octet if all 8 bits are grouped. Decimal or hexadecimal values shall be transferred in binary values so that the bit with the highest number of the group represents the msb concerning the grouped bits.

EXAMPLE 1: Description and relation for the specific field octet

Bit 0: reserved.

Bit 1-3: Reason_Code The decimal value 2 for the Reason_Code means general error.

Bit 4-7: shall always set to one.

The octet that is constructed according to the description above looks as follows:

(msb) Bit 7 = 1,

Bit 6 = 1,

Bit 5 = 1,

Bit 4 = 1,

Bit 3 = 0,

Bit 2 = 1,

Bit 1 = 0,

(lsb) Bit 0 = 0.

The bit combination "0-1-0" for Bit 1-3 equals the decimal value 2.

3.6.5 Conventions for Type 4

None.

3.6.6 Conventions for Type 5

None.

3.6.7 Conventions for Type 6

None.

3.6.8 Conventions for Type 7

None.

3.6.9 Conventions for Type 8

A part of Type 1 conventions apply.

3.6.10 Conventions for Type 9

A part of Type 1 conventions apply.

3.6.11 Conventions for Type 10**3.6.11.1 IDL - Interface Definition Language****3.6.11.1.1 Basic Information**

The Interface Definition Language (IDL) is a language for specifying operations (procedures or functions), parameters to these operations, and data types.

Notation

The syntax of IDL is described using an extended BNF (Backus-Naur Form) notation. The meaning of the BNF notation is as follows:

- Brackets ([]) enclose an optional part of the syntax.
- Ellipsis points (...) indicate that the left clause can be repeated either zero or more times if it is optional or one or more times if it is required. The vertical bar (|) indicates alternative productions; it is read as "or."
- Language punctuation that does not conflict with punctuation characters used in the BNF notation appears in a production in the appropriate position. Language punctuation that does conflict with punctuation characters used in the BNF notation is enclosed in less-than and greater-than symbols; for example, <[>. Note particularly that when ' (single quotation) or " (double quotation) appear in a production, they are a part of the language and shall appear in IDL source.

Elements in the grammar that are capitalized are terminals of the grammar. For example, <Identifier> is not further expanded. Also, keywords of the language are terminals of the grammar. For example, the keyword boolean is not further expanded.

3.6.11.1.2 Keywords and Reserved Words

Some keywords are reserved words, and shall not be used as identifiers. Keywords that are not reserved may be used as identifiers, except when used as attributes (that is, within [] (brackets)). Reserved words are keywords of the language that shall not be used as user identifiers.

IDL reserved words are given in the following list:

boolean	byte	case	char
const	default	double	enum
FALSE	float	handle_t	hyper
import	int	interface	long
NULL	pipe	short	small
struct	switch	TRUE	typedef
union	unsigned	void	

The following list gives the IDL keywords that are reserved when in the context of an attribute: that is, between [](brackets):

align	broadcast	broadcast	comm_status
context_handle	endpoint	first_is	handle
idempotent	ignore	implicit_handle	in
last_is	length_is	local	max_is
maybe	out	ptr	ref
size_is	string	switch_is	switch_type
transmit_as	uuid	version	

3.6.11.1.3 Identifiers

Each object is named with a unique identifier. The maximum length of an identifier is 31 characters.

Some identifiers are used as a base from which the compiler constructs other identifiers. These identifiers have further restrictions on their length. The character set for identifiers contains the alphabetic characters A to Z and a to z, the digits 0 to 9, and the _ (underbar) character. An identifier shall start with an alphabetic character or the _ (underbar) character.

3.6.11.1.4 The IDL is a case-sensitive language.IDL Punctuation

The punctuation used in IDL consists of the following characters:

The . (dot)

The , (comma)

The pair ()(parentheses)

The pair [](brackets)

The pair {}(braces)

The ; (semicolon)

The : (colon)

The * (asterisk)

The ' (single quote)

The " (double quote)

The = (equal sign)

3.6.11.1.5 Interface Definition Structure

An interface definition written in IDL has the following structure:

```
<interface> ::= <interface_header> { <interface_body> }
```

3.6.11.1.6 Interface Header

The structure of the interface header is as follows:

```
<interface_header> ::= [ < <interface_attributes> > ] interface <Identifier>
```

where:

```
<interface_attributes>::=<interface_attribute>[,<interface_attribute>]...
<interface_attribute>::=uuid (<Uuid_rep>)
    |version (<Integer_literal>[.<Integer_literal>])
    |endpoint (<port_spec>[,<port_spec>]...)
    |local
    |pointer_default (<ptr_attr>)
<port_spec>::=<Family_string>:<[><Port_string><]>
```

If an interface defines any operations, exactly one of the uuid attribute or the local attribute shall be specified. Whichever is specified shall appear exactly once.

It is permissible to have neither the uuid nor the local attribute if the interface defines no operations. The version attribute may occur at most once.

3.6.11.1.7 Interface Body

The structure of the interface body is as follows:

```
<interface_body>::=[<import>...]<interface_component>
[<interface_component>...]
```

where:

```
<import>::=import <import_list>;
<interface_component>::=<export>
    |<op_declarator>;
<export>::=<type_declarator>;
    |<const_declarator>;
    |<tagged_declarator>;
```

3.6.11.1.8 Further Possibilities

It is possible to use comments. The IDL allows to declare constants. Type declarations are subdivided into base types and the fundamental integer, char, boolean, byte, void, handle_t types, constructed types, structures, unions, enumerated types, pipes and arrays.

3.7 Conventions used in state machines

3.7.1 State machine conventions for Type 1

The state machines are described in Table 7:

Table 7 – Conventions used for state machines

#	Current state	Event / condition => action	Next state
Name of this transition.	The current state to which this state transition applies.	Events or conditions that trigger this state transaction. => The actions that are taken when the above events or conditions are met. The actions are always indented below events or conditions.	The next state after the actions in this transition is taken.

The conventions used in the state machines are as follows:

:= Value of an item on the left is replaced by value of an item on the right. If an item on the right is a parameter, it comes from the primitive shown as an input event.

xxx A parameter name.

Example:

Identifier := reason
means value of a 'reason' parameter is assigned to a parameter called 'Identifier.'

"xxx" Indicates fixed value.

Example:

Identifier := "abc"
means value "abc" is assigned to a parameter named 'Identifier.'

- = A logical condition to indicate an item on the left is equal to an item on the right.
- < A logical condition to indicate an item on the left is less than the item on the right.
- > A logical condition to indicate an item on the left is greater than the item on the right.
- <> A logical condition to indicate an item on the left is not equal to an item on the right.
- && Logical "AND"
- || Logical "OR"

This construct allows the execution of a sequence of actions in a loop within one transition. The loop is executed for all values from start_value to end_value.

Example:

```
for (Identifier := start_value to end_value)
    actions
endfor
```

This construct allows the execution of alternative actions depending on some condition (which might be the value of some identifier or the outcome of a previous action) within one transition.

Example:

```
If (condition)
    actions
else
    actions
endif
```

Readers are strongly recommended to refer to the subclauses for the AREP attribute definitions, the local functions, and the FAL-PDU definitions to understand protocol machines. It is assumed that readers have sufficient knowledge of these definitions, and they are used without further explanations.

3.7.2 State machine conventions for Type 2

3.7.2.1 General

State changes may be triggered by events (internal or external) or service invocations. Reaction to service invocations may depend on the value(s) of the attribute(s) accessed by the service.

Behaviour of the state machine shall be defined for combinations of :

- the *event* the machine receives;
- the *state* the machine is in when it receives notification of a state-changing event.

To define behaviour in these terms, a State Transition Diagram (STD) and a State Event Matrix (SEM) are used when applicable in the state machine specification.

3.7.2.2 State Transition Diagram (STD)

An **event** is an external stimulus that may cause a state transition. An STD graphically illustrates the states of an object and includes events, service calls and changes of attributes that cause it to transition to another state. Figure 5 shows an example of an STD.

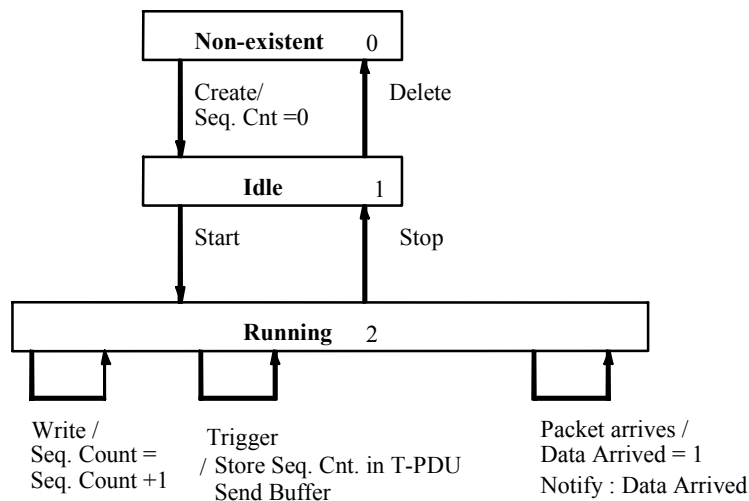


Figure 5 – Example of an STD

NOTE Event that is the primary cause of the State Transition is followed by "/" mark. Notification to upper layer is marked by "Notify :"

3.7.2.3 State Event Matrix

A **state** is the current active mode of operation of the state machine object (e.g. Running, Idle).

A state event matrix is a table that lists all possible events, services and changes in attribute values that initiate a state change, and indicates the response by the object to the event based on the state of the object when it receives notification of that event.

State event matrix format is as shown in Table 8:

Table 8 – State event matrix format

Event	State		
	State 1		State n
Event A description	Function triggered by event A in State 1 (if any) Notification to FAL user (if any) Transition to another state (if any)		Function triggered by event A in State n (if any) Notification to FAL user (if any) Transition to another state (if any)
.....			
Event X description	Function triggered by event X in State 1 (if any) Notification to FAL user (if any) Transition to another state (if any)		Function triggered by event X in State n (if any) Notification to FAL user (if any) Transition to another state (if any)
In absence of function, notification or transition, the corresponding entry shall not be made in the table.			

3.7.2.4 Example state event matrix

Table 9 shows an example of a state machine with three states :

- *Non-existent*: the object has not yet been created; objects transition to the existent state via the create service (if the object may be dynamically created) or at power-up (if the object is fixed by design/implementation);
- *Idle*: the object accepts services (e.g. Get_Attribute_Single), but does not produce or consume data onto or from the link;
- *Running*: the object is performing all its specified functions.

Table 9 – Example state event matrix

Event	State		
	Non-existent	Idle	Running
Create	Transition to Idle	Error: Object already exists.	Error: Object already exists.
Delete	Error: Object does not exist. (General Error Code 0x16)	Transition to Non-existent	Error: Object State Conflict (General Error Code 0x0C)
Start	Error: Object does not exist. (General Error Code 0x16)	Transition to Running	Error: Object State Conflict (General Error Code 0x0C)
Stop	Error: Object does not exist. (General Error Code 0x16)	Error: Object State Conflict (General Error Code 0x0C)	Transition to Idle
Get_Attribute_Single	Error: Object does not exist. (General Error Code 0x16)	Validate/service the request. Return response	Validate/service the request. Return response
Set_Attribute_Single	Error: Object does not exist. (General Error Code 0x16)	Validate/service the request. Return response	Error: Object State Conflict (General Error Code 0x0C)

3.7.3 State machine conventions for Type 3

The protocol sequences are described by means of State Machines.

In state diagrams states are represented as boxes state transitions are shown as arrows. Names of states and transitions of the state diagram correspond to the names in the textual listing of the state transitions.

The textual listing of the state transitions is structured as follows, see also Table 10.

The first row contains the name of the transition.

The second row in define the current state.

The third row contains an optional event followed by Conditions starting with a “/” as first line character and finally followed by the Actions starting with a “=>” as first line character.

The last row contains the next state.

If the event occurs and the conditions are fulfilled the transition fires, i.e. the actions are executed and the next state is entered.

The layout of a Machine description is shown in Table 10. The meaning of the elements of a State Machine Description are shown in Table 11.

Table 10 – State machine description elements

#	Current state	Event or condition => action	Next state

Table 11 – Description of state machine elements

Description element	Meaning
Current state	Name of the given states.
Next state	
#	Name or number of the state transition.
Event	Name or description of the event.
/Condition	Boolean expression. The preceding “\” is not part of the condition.
=> Action	List of assignments and service or function invocations. The preceding “=>” is not part of the action.

The conventions used in the state machines are shown in Table 12.

Table 12 – Conventions used in state machines

Convention	Meaning
:=	Value of an item on the left is replaced by value of an item on the right. If an item on the right is a parameter, it comes from the primitive shown as an input event.
xxx	A parameter name. Example: Identifier := reason means value of a 'reason' parameter is assigned to a parameter called 'Identifier.'
"xxx"	Indicates fixed value. Example: Identifier := "abc" means value "abc" is assigned to a parameter named 'Identifier.'
=	A logical condition to indicate an item on the left is equal to an item on the right.
<	A logical condition to indicate an item on the left is less than the item on the right.
>	A logical condition to indicate an item on the left is greater than the item on the right.
<>	A logical condition to indicate an item on the left is not equal to an item on the right.
&&	Logical "AND"
	Logical "OR"
for (Identifier := start_value to end_value) actions endfor	This construct allows the execution of a sequence of actions in a loop within one transition. The loop is executed for all values from start_value to end_value.
If (condition) actions else actions	This construct allows the execution of alternative actions depending on some condition (which might be the value of some identifier or the outcome of a previous action) within one transition. The parts beginning with “else” can be omitted if there is no action if the condition is not fulfilled.

Readers are strongly recommended to refer to the subclauses for the AREP and CREP attribute definitions, the local functions, and the FAL-PDU definitions to understand protocol machines. It is assumed that readers have sufficient knowledge of these definitions and they are used without further explanations.

In addition the following description elements are used:

Wildcard in names

name_XXX: “XXX” is used as wildcard string for all names beginning with “name”.

The typical use of a wildcard is an Event. In this context there are as many state transitions as possible events for this wildcard exists.

Conditional Macro

<CONDITIONAL -MACRO-NAME>

<

CONDITION1: macrobody1

CONDITION2: macrobody2

...

>

CONDITION1, CONDITION2, etc. define all possible cases for the conditional macro. The "CONDITIONAL-MACRO-NAME" acts as place holder for the "macrocode" depending on the result of the condition.

Replacement Macro

<REPLACEMENT-MACRO-NAME>

<

XXX= Name1 : macrobody1

XXX= Name2 : macrobody2

...

>

Name1, Name2, etc. define all possible cases for the replacement macro. The "REPLACEMENT-MACRO-NAME" acts as place holder for the "macrocode" depending on the value of the current use of the wildcard XXX.

Example:

<SERVICE_REQ_PARA>

<

XXX=Read: Para1, Para2

XXX=Write: Para1, Para2, Para3

>

when called in

MSAB_XXX.req(<SERVICE_REQ_PARA>)

will result in

MSAB_Read.req(Para1, Para2) or MSAB_Write.req(Para1, Para2, Para3)

3.7.4 State machine conventions for Type 4

None.

3.7.5 State machine conventions for Type 5

The same conventions as Type 1 apply.

3.7.6 State machine conventions for Type 6

None.

3.7.7 State machine conventions for Type 7

3.7.7.1 General

This subclause recalls the main definitions associated with the three specification techniques used : 'Evaluation Networks', 'Abstract objects', 'Service Conventions'.

3.7.7.2 Evaluation networks

To be helpful for the reader some mechanisms are explained by a graphical representation as shown in Figure 6. These representations are based on evaluation network built on an oriented graph with three types of nodes:

- The states, represented by a disk,
- The requests represented by a rectangle,
- The transitions, represented by a horizontal line.

The evaluation network is built according to the following principle:

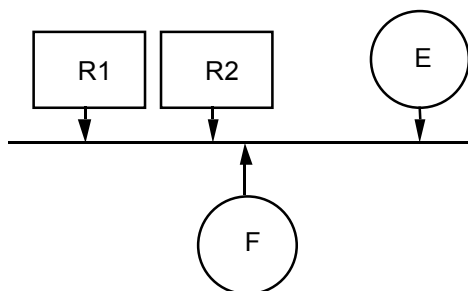


Figure 6 – Example of an evaluation net

When the described system is in state E, the arrival of request R1 or R2 results in a transition which switches it state F.

On the other hand, crossing a transition takes place, by convention, in zero time.

3.7.7.3 Simple actions

Simple actions can be associated with each transition, corresponding either to the transmission of requests or to the execution of local actions.

They are materially represented by a triangle, labelled with the name of the request or of the description of the executed action.

3.7.7.4 Conditions

The crossing of a transition can concern a condition. In this case, the oriented graph associated with the representation, shows a condition associated with the arc which precedes the transition.

A corollary of this representation allows defining choices for crossing a transition from the same initial state.

In the same way as for a simple transition, the transmission of an action can be associated with the crossing of a conditional transition.

3.7.8 State machine conventions for Type 8

The same conventions as Type 1 apply.

3.7.9 State machine conventions Type 9

The same conventions as Type 1 apply.

3.7.10 State machine conventions Type 10

The conventions used for state machines are according to conventions used for Type 3 (see 3.7.3).

4 Type 1

4.1 FAL syntax description

4.1.1 General

FalArHeader

```
FalArHeader ::= Unsigned8 {
  -- bit 8      FAL Protocol Specifier (Always 0 for the protocol defined by this Clause of IEC 61158-6)
  -- bit 7 - 4  Protocol Identifier    (Identifies abstract syntax, revision, and encoding rules)
  -- bit 3      Protocol Specific bit  (Reserved for each protocol to use)
  -- bit 2-1    PDU Identifier         (Identifies a PDU type within a Protocol Identifier)
}
```

The FalArHeader shall be the first octet of all the FAL PDUs. It enables multiple protocols as well as encoding rules to coexist on the same network. 4.2.1 defines the code points for the currently identified combinations of protocol and encoding rules.

4.1.2 FAL-AR PDU abstract syntax 1

4.1.2.1 Top level definition

The productions defined here shall be used with the Compact Encoding Rules, and it is strongly recommended that they be used with the messaging and buffered encoding rules to produce the shortest PDUs that are suitable for a limited bandwidth underlying layer.

```
FalArPDU ::= ConfirmedSend-CommandPDU
           || ConfirmedSend-AffirmativePDU
           || ConfirmedSend-NegativePDU
           || UnconfirmedSend-CommandPDU
           || UnconfirmedAcknowledgedSend-CommandPDU
           || UnconfirmedAcknowledgedSend-AffirmativePDU
           || Idle-CommandPDU
           || AR-XON-OFF-CommandPDU
           || Establish-CommandPDU
           || Establish-AffirmativePDU
           || Establish-NegativePDU
           || Abort-PDU
```

4.1.2.2 Confirmed send service

```
ConfirmedSend-CommandPDU ::= SEQUENCE {
  FalArHeader,
  InvokeID,
  ConfirmedServiceRequest
}
```

```
ConfirmedSend-AffirmativePDU ::= SEQUENCE {
  FalArHeader,
  InvokeID,
  ConfirmedServiceResponse
}
```

```
ConfirmedSend-NegativePDU ::= SEQUENCE {
  FalArHeader,
  InvokeID,
  ConfirmedServiceError
}
```

4.1.2.3 Unconfirmed send service

```
UnconfirmedSend-CommandPDU ::= SEQUENCE {
  FalArHeader,
  InvokeID,
  UnconfirmedServiceRequest
}
```

4.1.2.4 Unconfirmed acknowledged send service

```

UnconfirmedAcknowledgedSend-CommandPDU ::= SEQUENCE {
    FalArHeader,
    unconfirmed-PDU [4] IMPLICIT SEQUENCE {
        InvokeID,
        UnconfirmedServiceRequest
    }
}

UnconfirmedAcknowledgedSend-AffirmativePDU ::= {
    FalArHeader
}

```

4.1.2.5 Idle send service

```

IdleSend-CommandPDU ::= {
    FalArHeader
}

```

4.1.2.6 AR-XON-OFF send service

```

AR-XON-OFF-CommandPDU ::= SEQUENCE {
    FalArHeader,
    AR-XON-OFFPDU [7] IMPLICIT SEQUENCE {
        Invoke ID,
        XON-OFF
    }
}

```

4.1.2.7 Establish service

```

Establish-CommandPDU ::= SEQUENCE {
    FalArHeader,
    RespondingAREP,
    RequestingAREP,
    MAXOSCC,
    MAXOSCS,
    MAXUCSC,
    MAXUCSS,
    CIU,
    InvokeID,
    Initiate-RequestPDU
}

Establish-AffirmativePDU ::= SEQUENCE {
    FalArHeader,
    InvokeID,
    Initiate-ResponsePDU
}

Establish-NegativePDU ::= SEQUENCE {
    FalArHeader,
    InvokeID,
    Initiate-ErrorPDU
}

```

4.1.3 FAL-AR PDU abstract syntax 2

4.1.3.1 Top level definition

The productions defined here shall be used with the Traditional Encoding Rules to maintain compatibility with prior standards.

```

FalArPDU ::= ConfirmedSend-RequestPDU
|| ConfirmedSend-ResponsePDU
|| UnconfirmedSend-PDU
|| UnconfirmedAcknowledgedSend-RequestPDU
|| UnconfirmedAcknowledgedSend-ResponsePDU
|| Idle-CommandPDU
|| AR-XON-OFF-CommandPDU
|| Establish-RequestPDU
|| Establish-Request2PDU
|| Establish-ResponsePDU
|| Establish-ErrorPDU
|| Abort-PDU

```

4.1.3.2 Confirmed send service

```
ConfirmedSend-RequestPDU ::= SEQUENCE {
    FalArHeader,
    confirmed-RequestPDU          [1] IMPLICIT SEQUENCE {
        InvokeID,
        ConfirmedServiceRequest
    }
}

ConfirmedSend-ResponsePDU ::= SEQUENCE {
    FalArHeader,
    pduBody CHOICE {
        confirmed-ResponsePDU    [2] IMPLICIT SEQUENCE {
            InvokeID,
            ConfirmedServiceResponse
        },
        confirmed-ErrorPDU       [3] IMPLICIT SEQUENCE {
            InvokeID,
            ConfirmedServiceError
        }
    }
}
```

4.1.3.3 Unconfirmed send service

```
UnconfirmedSend-PDU ::= SEQUENCE {
    FalArHeader,
    unconfirmed-PDU              [4] IMPLICIT SEQUENCE {
        InvokeID,
        UnconfirmedServiceRequest
    }
}
```

4.1.3.4 Unconfirmed acknowledged send service

```
UnconfirmedAcknowledgedSend-RequestPDU ::= SEQUENCE {
    FalArHeader,
    unconfirmed-PDU              [4] IMPLICIT SEQUENCE {
        InvokeID,
        UnconfirmedServiceRequest
    }
}

UnconfirmedAcknowledgedSend-ResponsePDU ::= FalArHeader
```

4.1.3.5 Idle send service

```
IdleSend-CommandPDU ::= FalArHeader
```

4.1.3.6 AR-XON-OFF send service

```
AR-XON-OFF-CommandPDU ::= SEQUENCE {
    FalArHeader,
    AR-XON-OFFPDU               [7] IMPLICIT SEQUENCE {
        Invoke ID,
        XON-OFF
    }
}
```

4.1.3.7 Establish service

```
Establish-RequestPDU ::= SEQUENCE {
    FalArHeader,
    RespondingAREP,
    RequestingAREP,
    userData                  [6] IMPLICIT SEQUENCE {
        InvokeID,
        InitiateRequest        [0] IMPLICIT Initiate-RequestPDU
    }
}
```

```

Establish-Request2PDU ::= SEQUENCE {
    FalArHeader,
    RespondingAREP,
    RequestingAREP,
    AREPContext
    [5] IMPLICIT SEQUENCE {
        MaxOSCC,
        MaxOSCS,
        MAXUCSC,
        MAXUCSS,
        CIU
    }
}

Establish-ResponsePDU ::= SEQUENCE {
    FalArHeader,
    userData
    [6] IMPLICIT SEQUENCE {
        InvokeID,
        InitiateResponse
    }
    [1] IMPLICIT Initiate-ResponsePDU
}

Establish-ErrorPDU ::= SEQUENCE {
    FalArHeader,
    userData
    [6] IMPLICIT SEQUENCE {
        InvokeID,
        initiateError
    }
    [2] IMPLICIT Initiate-ErrorPDU
}

```

4.1.3.8 MaxOSCC

MaxOSCC ::= Unsigned8

4.1.3.9 MaxOSCS

MaxOSCS ::= Unsigned8

4.1.3.10 MaxUCSC

MaxUCC ::= Unsigned8

4.1.3.11 MaxUCSS

MaxUCS ::= Unsigned8

4.1.3.12 XON_OFF

XON_OFF ::= Unsigned8

4.1.3.13 CIU

CIU ::= Unsigned32

4.1.4 Abstract syntax of PDUBody

4.1.4.1 Abort service

```

Abort-PDU ::= SEQUENCE {
    FalArHeader,
    Identifier,
    ReasonCode,
    AdditionalDetail
}

```

4.1.4.2 InvokeID

InvokeID ::= Unsigned8

4.1.4.3 ConfirmedServiceRequest

```
ConfirmedServiceRequest ::= CHOICE {
    status-Request          [0] IMPLICIT Status-RequestPDU,
    identify-Request        [1] IMPLICIT Identify-RequestPDU,
    read2-Request           [2] IMPLICIT Read2-RequestPDU,
    write-Request           [3] IMPLICIT Write-RequestPDU,
    getAttributes2-Request  [4] IMPLICIT GetAttributes2-RequestPDU,
    readWithType-Request    [5] IMPLICIT ReadWithType-RequestPDU,
    writeWithType-Request   [6] IMPLICIT WriteWithType-RequestPDU,
    defineVariableList-Request [7] IMPLICIT DefineVariableList-RequestPDU,
    deleteVariableList-Request [8] IMPLICIT DeleteVariableList-RequestPDU,
    initiateClientPullDownloadStatic-Request [9] IMPLICIT InitiateClientPullDownloadStatic-RequestPDU,
    pullDownloadSegment-Request [10] IMPLICIT PullDownloadSegment-RequestPDU,
    terminatePullDownload-Request [11] IMPLICIT TerminatePullDownload-RequestPDU,
    initiateClientPullUploadStatic-Request [12] IMPLICIT InitiateClientPullUploadStatic-RequestPDU,
    pullUploadSegment-Request [13] IMPLICIT PullUploadSegment-RequestPDU,
    terminatePullUpload-Request [14] IMPLICIT TerminatePullUpload-RequestPDU,
    initiateServerPullDownload-Request [15] IMPLICIT InitiateServerPullDownload-RequestPDU,
    initiateServerPullUpload-Request [16] IMPLICIT InitiateServerPullUpload-RequestPDU,
    createFunctionInvocation-Request [17] IMPLICIT CreateFunctionInvocation-RequestPDU,
    deleteFunctionInvocation-Request [18] IMPLICIT DeleteFunctionInvocation-RequestPDU,
    start-Request           [19] IMPLICIT Start-RequestPDU,
    stop-Request            [20] IMPLICIT Stop-RequestPDU,
    resume-Request          [21] IMPLICIT Resume-RequestPDU,
    reset-Request           [22] IMPLICIT Reset-RequestPDU,
    kill-Request            [23] IMPLICIT Kill-RequestPDU,
    enableEvent-Request     [24] IMPLICIT EnableEvent-RequestPDU,
    acknowledgeEventNotification-Request [25] IMPLICIT AcknowledgeEventNotification-RequestPDU,
    physicalRead-Request    [26] IMPLICIT PhysicalRead-RequestPDU,
    physicalWrite-Request   [27] IMPLICIT PhysicalWrite-RequestPDU,
    beginSetAttributes-Request [28] IMPLICIT BeginSetAttributes-RequestPDU,
    setAttributes2-Request  [29] IMPLICIT SetAttributes2-RequestPDU,
    endSetAttributes-Request [30] IMPLICIT EndSetAttributes-RequestPDU,
    initiateClientPushDownload-Request [31] IMPLICIT InitiateClientPushDownload-RequestPDU,
    pushDownloadSegment-Request [32] IMPLICIT PushDownloadSegment-RequestPDU,
    terminatePushDownload-Request [33] IMPLICIT TerminatePushDownload-RequestPDU,
    discard-Request         [34] IMPLICIT Discard-RequestPDU,
    readList-Request        [35] IMPLICIT ReadList-RequestPDU,
    writeList-Request       [36] IMPLICIT WriteList-RequestPDU,
    exchange-Request        [37] IMPLICIT Exchange-RequestPDU,
    getEventSummary-Request [38] IMPLICIT GetEventSummary-RequestPDU,
    getEventSummaryList-Request [39] IMPLICIT GetEventSummaryList-RequestPDU,
    setAttributes1-Request  [40] IMPLICIT SetAttributes1-RequestPDU,
    getAttributes1-Request  [41] IMPLICIT GetAttributes1-RequestPDU,
    read1-Request          [42] IMPLICIT Read1-RequestPDU,
    conclude-Request        [43] IMPLICIT Conclude-RequestPDU,
    subscribe-Request       [44] IMPLICIT Subscribe-RequestPDU,
    initiateClientPullDownloadDynamic-Request [45] IMPLICIT InitiateClientPullDownloadDynamic-RequestPDU,
    initiateClientPullUploadDynamic-Request [46] IMPLICIT InitiateClientPullUploadDynamic-RequestPDU,
    exchangeList-Request    [47] IMPLICIT ExchangeList-RequestPDU,
    enableEventList-Request [48] IMPLICIT EnableEventList-RequestPDU,
    confirmedAcknowledgeEventList-Request [49] IMPLICIT ConfirmedAcknowledgeEventList-RequestPDU,
    getAttributes3-Request  [50] IMPLICIT GetAttributes3-RequestPDU,
    queryEventSummaryList-Request [51] IMPLICIT QueryEventSummaryList-RequestPDU
}
```

4.1.4.4 ConfirmedServiceResponse

```

ConfirmedServiceResponse ::= CHOICE {
  status-Response           [0] IMPLICIT Status-ResponsePDU,
  identify-Response         [1] IMPLICIT Identify-ResponsePDU,
  read2-Response           [2] IMPLICIT Read2-ResponsePDU,
  write-Response            [3] IMPLICIT Write-ResponsePDU,
  getAttributes2-Response   [4] IMPLICIT GetAttributes2-ResponsePDU,
  readWithType-Response     [5] IMPLICIT ReadWithType-ResponsePDU,
  writeWithType-Response    [6] IMPLICIT WriteWithType-ResponsePDU,
  defineVariableList-Response [7] IMPLICIT DefineVariableList-ResponsePDU,
  deleteVariableList-Response [8] IMPLICIT DeleteVariableList-ResponsePDU,
  initiateClientPullDownloadStatic-Response [9] IMPLICIT InitiateClientPullDownloadStatic-ResponsePDU,
  pullDownloadSegment-Response [10] IMPLICIT PullDownloadSegment-ResponsePDU,
  terminatePullDownload-Response [11] IMPLICIT TerminatePullDownload-ResponsePDU,
  initiateClientPullUploadStatic-Response [12] IMPLICIT InitiateClientPullUploadStatic-ResponsePDU,
  pullUploadSegment-Response [13] IMPLICIT PullUploadSegment-ResponsePDU,
  terminatePullUpload-Response [14] IMPLICIT TerminatePullUpload-ResponsePDU,
  initiateServerPullDownload-Response [15] IMPLICIT InitiateServerPullDownload-ResponsePDU,
  initiateServerPullUpload-Response [16] IMPLICIT InitiateServerPullUpload-ResponsePDU,
  createFunctionInvocation-Response [17] IMPLICIT CreateFunctionInvocation-ResponsePDU,
  deleteFunctionInvocation-Response [18] IMPLICIT DeleteFunctionInvocation-ResponsePDU,
  start-Response           [19] IMPLICIT Start-ResponsePDU,
  stop-Response            [20] IMPLICIT Stop-ResponsePDU,
  resume-Response          [21] IMPLICIT Resume-ResponsePDU,
  reset-Response           [22] IMPLICIT Reset-ResponsePDU,
  kill-Response            [23] IMPLICIT Kill-ResponsePDU,
  enableEvent-Response     [24] IMPLICIT EnableEvent-ResponsePDU,
  acknowledgeEventNotification-Response [25] IMPLICIT AcknowledgeEventNotification-ResponsePDU,
  physicalRead-Response    [26] IMPLICIT PhysicalRead-ResponsePDU,
  physicalWrite-Response   [27] IMPLICIT PhysicalWrite-ResponsePDU,
  beginSetAttributes-Response [28] IMPLICIT BeginSetAttributes-ResponsePDU,
  setAttributes2-Response   [29] IMPLICIT SetAttributes2-ResponsePDU,
  endSetAttributes-Response [30] IMPLICIT EndSetAttributes-ResponsePDU,
  initiateClientPushDownload-Response [31] IMPLICIT InitiateClientPushDownload-ResponsePDU,
  pushDownloadSegment-Response [32] IMPLICIT PushDownloadSegment-ResponsePDU,
  terminatePushDownload-Response [33] IMPLICIT TerminatePushDownload-ResponsePDU,
  discard-Response         [34] IMPLICIT Discard-ResponsePDU,
  readList-Response        [35] IMPLICIT ReadList-ResponsePDU,
  writeList-Response       [36] IMPLICIT WriteList-ResponsePDU,
  exchange-Response        [37] IMPLICIT Exchange-ResponsePDU,
  getEventSummary-Response [38] IMPLICIT GetEventSummary-ResponsePDU,
  getEventSummaryList-Response [39] IMPLICIT GetEventSummaryList-ResponsePDU,
  setAttributes1-Response   [40] IMPLICIT SetAttributes1-ResponsePDU,
  getAttributes1-Response   [41] IMPLICIT GetAttributes1-ResponsePDU,
  read1-Response           [42] IMPLICIT Read1-ResponsePDU,
  conclude-Response        [43] IMPLICIT Conclude-ResponsePDU,
  subscribe-Response       [44] IMPLICIT Subscribe-ResponsePDU,
  initiateClientPullDownloadDynamic-Response [45] IMPLICIT InitiateClientPullDownloadDynamic-ResponsePDU,
  initiateClientPullUploadDynamic-Response [46] IMPLICIT InitiateClientPullUploadDynamic-ResponsePDU,
  exchangeList-Response    [47] IMPLICIT ExchangeList-ResponsePDU,
  enableEventList-Response [48] IMPLICIT EnableEventList-ResponsePDU,
  confirmedAcknowledgeEventList-Response [49] IMPLICIT ConfirmedAcknowledgeEventList-ResponsePDU,
  getAttributes3-Response   [50] IMPLICIT GetAttributes3-ResponsePDU,
  queryEventSummaryList-Response [51] IMPLICIT QueryEventSummaryList-ResponsePDU
}

```

4.1.4.5 ConfirmedServiceError

```
ConfirmedServiceError ::= CHOICE {
    status-Error           [0] IMPLICIT ErrorType,
    identify-Error         [1] IMPLICIT ErrorType,
    read2-Error            [2] IMPLICIT ErrorType,
    write-Error            [3] IMPLICIT ErrorType,
    getAttributes2-Error   [4] IMPLICIT ErrorType,
    readWithType-Error     [5] IMPLICIT ErrorType,
    writeWithType-Error    [6] IMPLICIT ErrorType,
    defineVariableList-Error [7] IMPLICIT ErrorType,
    deleteVariableList-Error [8] IMPLICIT ErrorType,
    initiateClientPullDownloadStatic-Error [9] IMPLICIT ErrorType,
    pullDownloadSegment-Error [10] IMPLICIT ErrorType,
    terminatePullDownload-Error [11] IMPLICIT ErrorType,
    initiateClientPullUploadStatic-Error [12] IMPLICIT ErrorType,
    pullUploadSegment-Error [13] IMPLICIT ErrorType,
    terminatePullUpload-Error [14] IMPLICIT ErrorType,
    initiateServerPullDownload-Error [15] IMPLICIT ErrorType,
    initiateServerPullUpload-Error [16] IMPLICIT ErrorType,
    createFunctionInvocation-Error [17] IMPLICIT ErrorType,
    deleteFunctionInvocation-Error [18] IMPLICIT ErrorType,
    start-Error            [19] IMPLICIT ErrorType,
    stop-Error             [20] IMPLICIT ErrorType,
    resume-Error           [21] IMPLICIT ErrorType,
    reset-Error            [22] IMPLICIT ErrorType,
    kill-Error             [23] IMPLICIT ErrorType,
    enableEvent-Error      [24] IMPLICIT ErrorType,
    acknowledgeEventNotification-Error [25] IMPLICIT ErrorType,
    physicalRead-Error     [26] IMPLICIT ErrorType,
    physicalWrite-Error    [27] IMPLICIT ErrorType,
    beginSetAttributes-Error [28] IMPLICIT ErrorType,
    setAttributes2-Error   [29] IMPLICIT ErrorType,
    endSetAttributes-Error [30] IMPLICIT ErrorType,
    initiateClientPushDownload-Error [31] IMPLICIT ErrorType,
    pushDownloadSegment-Error [32] IMPLICIT ErrorType,
    terminatePushDownload-Error [33] IMPLICIT ErrorType,
    discard-Error          [34] IMPLICIT ErrorType,
    readList-Error         [35] IMPLICIT ErrorType,
    writeList-Error        [36] IMPLICIT ErrorType,
    exchange-Error         [37] IMPLICIT ErrorType,
    getEventSummary-Error  [38] IMPLICIT ErrorType,
    getEventSummaryList-Error [39] IMPLICIT ErrorType,
    setAttributes1-Error   [40] IMPLICIT ErrorType,
    getAttributes1-Error   [41] IMPLICIT ErrorType,
    read1-Error            [42] IMPLICIT ErrorType,
    conclude-Error         [43] IMPLICIT ErrorType,
    subscribe-Error        [44] IMPLICIT ErrorType,
    initiateClientPullDownloadDynamic-Error [45] IMPLICIT ErrorType,
    initiateClientPullDownloadDynamic-Error [46] IMPLICIT ErrorType,
    exchangeList-Error     [47] IMPLICIT ErrorType,
    enableEventList-Error  [48] IMPLICIT ErrorType,
    confirmedAcknowledgeEventList-Error [49] IMPLICIT ErrorType,
    getAttributes3-Error   [50] IMPLICIT ErrorType,
    queryEventSummaryList-Error [51] IMPLICIT ErrorType,
    xon-off-Error          [52] IMPLICIT ErrorType
}
```

4.1.4.6 Error type

```
ErrorType ::= SEQUENCE {
    errorClass               [0] IMPLICIT ErrorClass,
    optionalParametersMap    [10] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    additionalCode           [1] IMPLICIT Integer16 OPTIONAL,
    additionalDescription     [2] IMPLICIT VisibleString OPTIONAL,
    serviceSpecificInfo      [3] IMPLICIT ANY OPTIONAL
}
```


4.1.4.7 Error class

```

ErrorClass ::= CHOICE {
    vfdState          [1] IMPLICIT Integer8 {
        other (0)
    },
    applicationReference [2] IMPLICIT Integer8 {
        other (0),
        application-unreachable (1),
        application-reference-invalid (2),
        context-unsupported (3)
    },
    definition [3] IMPLICIT Integer8 {
        other (0),
        object-undefined (1),
        object-attributes-inconsistent (2),
        name-already-exists (3),
        type-unsupported (4),
        type-inconsistent (5)
    },
    resource [4] IMPLICIT Integer8 {
        other (0),
        memory-unavailable (1)
    },
    service [5] IMPLICIT Integer8 {
        other (0),
        object-state-conflict (1),
        pdu-size (2),
        object-constraint-conflict (3),
        parameter-inconsistent (4),
        illegal-parameter (5)
    },
    access [6] IMPLICIT Integer8 {
        other (0),
        object-invalidated (1),
        hardware-fault (2),
        object-access-denied (3),
        invalid-address (4),
        object-attribute-inconsistent (5),
        object-access-unsupported (6),
        object-non-existent (7),
        type-conflict (8),
        named-access-unsupported (9),
        access-to-element-unsupported (10)
    },
    objectDescription [7] IMPLICIT Integer8 {
        other (0),
        name-length-overflow (1),
        od-overflow (2),
        od-write-protected (3),
        extension-length-overflow (4),
        od-description-length-overflow (5),
        operational-problem (6)
    },
    conclude [9] IMPLICIT Integer8 {
        other (0),
        further-communication-required (1)
    },
    other [8] IMPLICIT Integer8 {
        other (0)
    }
}

```

4.1.4.8 Unconfirmed PDUs

```
UnconfirmedServiceRequest ::= CHOICE {
    informationReport-Request      [0]    IMPLICIT InformationReport-RequestPDU,
    unsolicitedStatus-Request      [1]    IMPLICIT UnsolicitedStatus-RequestPDU,
    eventNotification-Request      [2]    IMPLICIT EventNotification-RequestPDU,
    informationReportWithType      [3]    InformationReportWithType-RequestPDU,
    eventNotificationWithType      [4]    EventNotificationWithType-RequestPDU,
    actionInvoke-Request          [5]    IMPLICIT ActionInvoke-RequestPDU,
    actionReturn-Request          [6]    IMPLICIT ActionReturn-RequestPDU,
    informationReportList-Request  [7]    IMPLICIT InformationReportList-RequestPDU,
    reject-Request                [8]    IMPLICIT Reject-RequestPDU,
    notificationRecovery-Request  [9]    IMPLICIT NotificationRecovery-RequestPDU,
    eventNotificationList-Request [10]   IMPLICIT EventNotificationList-RequestPDU
}
```

4.1.4.9 Management ASE

4.1.4.9.1 Begin set attributes service PDUs

```
BeginSetAttributes-RequestPDU ::= Integer8 {
    loadingNonInteracting      (0),
    appendingInteracting       (1),
    freshLoadingInteracting    (2)
}
```

BeginSetAttributes -ResponsePDU ::= NULL

4.1.4.9.2 Create function invocation service PDUs

```
CreateFunctionInvocation-RequestPDU ::= SEQUENCE {
    listOfRelatedObjects      [0]    IMPLICIT SEQUENCE OF Gn_AccessSpecification,
    accessPrivilege           [1]    IMPLICIT Fi_AccessPrivilege,
    optionalParametersMap     [10]   IMPLICIT Gn_OptionalParametersMap16 OPTIONAL,
    fiName                    [2]    IMPLICIT Gn_Name OPTIONAL,
    fiIndex                   [12]   IMPLICIT Gn_NumericID OPTIONAL,
    userDescription           [3]    IMPLICIT ANY OPTIONAL,
    reusable                  [4]    IMPLICIT Gn_Reusable DEFAULT TRUE,
    deletable                 [5]    IMPLICIT Gn_Deletable OPTIONAL,
    startServiceArgumentType  [6]    IMPLICIT Gn_NumericID OPTIONAL,
    stopServiceArgumentType   [7]    IMPLICIT Gn_NumericID OPTIONAL,
    resumeServiceArgumentType [8]    IMPLICIT Gn_NumericID OPTIONAL,
    resetServiceArgumentType  [9]    IMPLICIT Gn_NumericID OPTIONAL,
    killServiceArgumentType   [11]   IMPLICIT Gn_NumericID OPTIONAL
}
```

CreateFunctionInvocation-ResponsePDU ::= Gn_NumericID

4.1.4.9.3 Define variable list service PDUs

```
DefineVariableList-RequestPDU ::= SEQUENCE {
    listOfVariables           [0]    IMPLICIT SEQUENCE OF Gn_AccessSpecification,
    accessPrivilege           [1]    IMPLICIT Vr_AccessPrivilege,
    optionalParametersMap     [10]   IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    variableListName          [2]    IMPLICIT Gn_Name OPTIONAL,
    variableListIndex         [5]    IMPLICIT Gn_NumericID OPTIONAL,
    userDescription           [3]    IMPLICIT ANY OPTIONAL,
    deletable                 [4]    IMPLICIT Gn_Deletable OPTIONAL
}
```

DefineVariableList-ResponsePDU ::= Gn_NumericID

4.1.4.9.4 Delete function invocation service PDUs

DeleteFunctionInvocation-RequestPDU ::= Gn_AccessSpecification

DeleteFunctionInvocation-ResponsePDU ::= NULL

4.1.4.9.5 Delete variable list service PDUs

DeleteVariableList-RequestPDU ::= Gn_AccessSpecification

DeleteVariableList-ResponsePDU ::= NULL

4.1.4.9.6 End set attributes service PDUs

EndSetAttributes-RequestPDU ::= NULL

EndSetAttributes-ResponsePDU ::= NULL

4.1.4.9.7 Get attributes list service PDUs

```

GetAttributes1-RequestPDU ::= CHOICE {
    listOfObjectsAndAttributes      [0] IMPLICIT SEQUENCE OF SEQUENCE {
        falClass                    [6] IMPLICIT Gn_NumericID,
        keyAttribute                 Gn_KeyAttribute,
        listOfRequestedAttributes    [7] IMPLICIT Mn_AttributesMap
    },
    rangeOfObjectsAndAttributes     [1] IMPLICIT SEQUENCE {
        startingNumericId           [0] IMPLICIT Gn_NumericID,
        listOfRequestedAttributes    [1] IMPLICIT Mn_AttributesMap
    }
}

GetAttributes2-RequestPDU ::= SEQUENCE {
    listOfAttributes                [0] IMPLICIT Mn_SelectedAttributes,
    accessSpecification CHOICE {
        numericID                   [1] IMPLICIT Gn_NumericID,
        variableName                 [2] IMPLICIT Gn_Name,
        variableListName             [3] IMPLICIT Gn_Name,
        domainName                   [4] IMPLICIT Gn_Name,
        fiName                       [5] IMPLICIT Gn_Name,
        eventName                    [6] IMPLICIT Gn_Name,
        startIndex                   [7] IMPLICIT Gn_NumericID
    }
}

GetAttributes3-RequestPDU ::= SEQUENCE {
    objectClass                     [2] IMPLICIT Gn_ObjectClass,
    start objectID                  Gn_AccessSpecification
}

GetAttributes1-ResponsePDU ::= SEQUENCE {
    listOfAttributeIdAndValues       [0] IMPLICIT Mn_AttributeValues,
    more                             [1] IMPLICIT Gn_MoreFollows
}

GetAttributes2-ResponsePDU ::= SEQUENCE {
    listOfObjectDefinition           [0] IMPLICIT SEQUENCE OF Gn_ObjectDefinition,
    more                             [1] IMPLICIT Gn_MoreFollows
}

GetAttributes3-ResponsePDU ::= CHOICE {
    listofobjectID                  [0] IMPLICIT SEQUENCE {
        listofID                    [2] IMPLICIT SEQUENCE OF Gn_AccessSpecification,
        morefollows                  [3] IMPLICIT Gn_MoreFollows
    },
    --The object class value in the request is <> 0
    listofobjectDefinition           [1] IMPLICIT SEQUENCE {
        listofDefinition             [4] IMPLICIT SEQUENCE OF Mn_AttributesValues
        morefollows                  [5] IMPLICIT Gn_MoreFollows
    },
    --The object classvalue in the request is = 0
}

```

4.1.4.9.8 Set attributes service PDUs

```

SetAttributes1-RequestPDU ::= SEQUENCE OF SEQUENCE {
    falClass                        [6] IMPLICIT Gn_NumericID,
    keyAttributes                   Gn_KeyAttributes,
    listOfAttributeUpdates          [7] IMPLICIT Mn_AttributeValues
}

SetAttributes2-RequestPDU ::= SEQUENCE OF {
    objectDefinition                [0] IMPLICIT Gn_ObjectDefinition
}

SetAttributes1-ResponsePDU ::= NULL
SetAttributes2-ResponsePDU ::= NULL

```

4.1.4.10 Application process ASE

4.1.4.10.1 Get status service PDUs

Status-RequestPDU ::= NULL

```
Status-ResponsePDU ::= SEQUENCE {
    logicalStatus          [0] IMPLICIT Unsigned8 {
        state-changes-allowed      (0),
        no-state-changes-allowed   (1),
        limited-services-permitted  (2),
        od-loading-non-interacting  (4),
        od-loading-interacting     (5)
    },
    physicalStatus         [1] IMPLICIT Unsigned8 {
        operational                (0),
        partially-operational      (1),
        inoperable                 (2),
        needs-commissioning        (3)
    },
    optionalParametersMap  [10] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    localDetail            [2] IMPLICIT BitString OPTIONAL -- bit 1
}
```

4.1.4.10.2 Identify service PDUs

Identify-RequestPDU ::= NULL

```
Identify-ResponsePDU ::= SEQUENCE {
    vendorName             [0] IMPLICIT VisibleString,
    modelIdentifier [1]    IMPLICIT VisibleString,
    vendorRevision         [2] IMPLICIT VisibleString,
    optionalParametersMap  [10] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    serialNumber           [3] IMPLICIT OctetString OPTIONAL -- bit 1
}
```

4.1.4.10.3 Initiate service PDUs

```
Initiate-RequestPDU ::= SEQUENCE {
    versionObjectDefinitionsCalling [0] IMPLICIT Integer16,
    apDescriptorCalling             [1] IMPLICIT OctetString,
    accessProtectionSupportedCalling [2] IMPLICIT Ap_AccessProtectionSupported,
    passwordAndAccessGroupsCalling  [3] IMPLICIT Ap_AccessControl,
    configuredMaxPduSizeSending     [4] IMPLICIT Unsigned8, -- See NOTE
    configuredMaxPduSizeReceiving   [5] IMPLICIT Unsigned8, -- See NOTE
    listOfSupportedServicesCalling  [6] IMPLICIT Mn_PduSupportedMap,
    configuredMaxOutstandingServicesRequesting [7] IMPLICIT Unsigned8,
    configuredMaxOutstandingServicesResponding [8] IMPLICIT Unsigned8,
    optionalParametersMap          [10] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    userDetails                     [11] IMPLICIT OctetString OPTIONAL, -- bit 1
    configuredNestingLevel          [12] IMPLICIT Unsigned8 DEFAULT 1 -- bit 2
}
```

```
Initiate-ResponsePDU ::= SEQUENCE {
    versionObjectDefinitionsCalled [0] IMPLICIT Integer16,
    profileNumberCalled            [1] IMPLICIT OctetString,
    accessPrivilegeSupportedCalled [2] IMPLICIT Ap_AccessProtectionSupported,
    passwordAndAccessGroupsCalled  [3] IMPLICIT Ap_AccessControl,
    negotiatedMaxOutstandingServicesRequesting [4] IMPLICIT Unsigned8,
    negotiatedMaxOutstandingServicesResponding [5] IMPLICIT Unsigned8,
    optionalParametersMap          [10] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    userDetails                     [11] IMPLICIT OctetString OPTIONAL, -- bit 1
    negotiatedNestingLevel          [12] IMPLICIT Unsigned8 DEFAULT 1, -- bit 2
    negotiatedListOfServices        [13] IMPLICIT Mn_PduSupportedMap OPTIONAL -- bit 3
}
```

```

Initiate-ErrorPDU ::= SEQUENCE {
    errorCode
        [0] IMPLICIT Integer8 {
            other (0),
            max-fal-pdu-size-insufficient (1),
            service-not-supported (2),
            version-obj-def-incompatible (3),
            user-initiate-denied (4),
            password-error (5),
            profile-number-incompatible (6)
        },
    maxPduLengthSendingCalled [1] IMPLICIT Unsigned8, -- See NOTE
    maxPduLengthReceivingCalled [2] IMPLICIT Unsigned8, -- See NOTE
    listOfSupportedServicesCalled [3] IMPLICIT Mn_PduSupportedMap
}

```

NOTE For QUB-Seg the PDU length is given in multiples of 256 octets, i.e. value 4 means $4 \times 256 = 1024$ octets

4.1.4.10.4 Status notification service PDU

UnsolicitedStatus-RequestPDU ::= Status-ResponsePDU

4.1.4.10.5 Subscribe service PDUs

```

Subscribe-RequestPDU ::= CHOICE {
    joining
        [0] IMPLICIT SEQUENCE {
            publishedObject Gn_KeyAttributes,
            scheduleInterval [11] IMPLICIT DI_CyclicInterval,
            optionalParametersMap [12] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
            maximumARs [13] IMPLICIT Unsigned8 DEFAULT 1 OPTIONAL, -- bit 1
            maximumPermissibleJitter [14] IMPLICIT DI_PermissibleJitter OPTIONAL, -- bit 2
            desiredPublishingStartTime [15] IMPLICIT FieldbusTime OPTIONAL, -- bit 3
            earliestPublishingStartTime [16] IMPLICIT FieldbusTime OPTIONAL, -- bit 4
            latestPublishingStartTime [17] IMPLICIT FieldbusTime OPTIONAL -- bit 5
        },
    listOfLeavingAREPs [1] IMPLICIT SEQUENCE OF Gn_NumericID
}

Subscribe-ResponsePDU ::= SEQUENCE {
    optionalParametersMap [0] Gn_OptionalParametersMap8 OPTIONAL,
    dataType [1] Gn_NumericID OPTIONAL, -- bit 1
    dataLength [2] DI_SduSize OPTIONAL, -- bit 2
    dedicated [3] Ar_Dedicated DEFAULT TRUE, -- bit 3
    listOfArInformation [4] IMPLICIT SEQUENCE OF SEQUENCE {
        publishingAREPid [0] Gn_NumericID,
        publishingDIMapping [1] Gn_NumericID,
        scheduledStartTime [2] FieldbusTime
    } OPTIONAL -- bit 4
}

```

4.1.4.10.6 Reject service PDU

```

RejectPDU ::= SEQUENCE {
    original-invokeID [0] IMPLICIT Integer8,
    reject-code [1] IMPLICIT Integer8 {
        other (0),
        illegal-confirmed-service-request (1),
        pdu-error (2),
        pdu-size (5)
    }
}

```

4.1.4.10.7 Conclude service PDUs

Conclude-Request ::= Null

Conclude-Response ::= Null

4.1.4.11 Load region ASE

4.1.4.11.1 Discard service PDUs

Discard-RequestPDU ::= Gn_KeyAttribute -- Load Region LocalID

Discard-ResponsePDU ::= NULL

4.1.4.11.2 InitiateClientPullDownloadStatic service PDUs

InitiateClientPullDownloadStatic-RequestPDU ::= Gn_AccessSpecification

InitiateClientPullDownloadStatic-ResponsePDU ::= NULL

4.1.4.11.3 InitiateClientPullDownloadDynamic service PDUs

```
InitiateClientPullDownloadDynamic-RequestPDU ::= SEQUENCE {
    loadRegionKeyAttribute      Gn_AccessSpecification,
    userData                    [0] OctetString,
    sharable                    [1] BOOLEAN,
    accessProtection            [2] Fi-AccessPrivilege
}
```

InitiateClientPullDownloadDynamic-ResponsePDU ::= NULL

4.1.4.11.4 InitiateClientPullUploadStatic service PDUs

InitiateClientPullUploadStatic-RequestPDU ::= Gn_AccessSpecification

InitiateClientPullUploadStatic-ResponsePDU ::= NULL

4.1.4.11.5 InitiateClientPullUploadDynamic service PDUs

InitiateClientPullUploadDynamic-RequestPDU ::= Gn_AccessSpecification

```
InitiateClientPullUploadDynamic-ResponsePDU ::= SEQUENCE {
    ulsmNumericID              [0] Gn_NumericID,
    userData                    [1] OctetString
}
```

4.1.4.11.6 InitiateServerPullDownload service PDUs

```
InitiateServerPullDownload-RequestPDU ::= SEQUENCE {
    loadRegionKeyAttribute      Gn_AccessSpecification,
    optionalParametersMap       [10] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    additionalInformation        [11] IMPLICIT VisibleString OPTIONAL
}                                     -- bit 1
```

InitiateServerPullDownload-ResponsePDU ::= NULL

4.1.4.11.7 InitiateServerPullUpload service PDUs

```
InitiateServerPullUpload-RequestPDU ::= SEQUENCE {
    loadRegionKeyAttribute      Gn_AccessSpecification,
    optionalParametersMap       [10] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    additionalInformation        [2] IMPLICIT VisibleString OPTIONAL
}                                     -- bit 1
```

InitiateServerPullUpload-ResponsePDU ::= NULL

4.1.4.11.8 InitiateClientPushDownload service PDUs

InitiateClientPushDownload-RequestPDU ::= Gn_AccessSpecification

InitiateClientPushDownload-ResponsePDU ::= NULL

4.1.4.11.9 PullDownloadSegment service PDUs

```
PullDownloadSegment-RequestPDU ::= SEQUENCE {
    loadRegionKeyAttribute      Gn_AccessSpecification,
    optionalParametersMap       [10] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    segmentNumber               [11] IMPLICIT Fi_SegmentNumber OPTIONAL
}                                     -- bit 1
```

```
PullDownloadSegment-ResponsePDU ::= SEQUENCE {
    loadData                    [0] IMPLICIT OctetString,
    moreFollows                 [1] IMPLICIT Gn_MoreFollows
}
```

4.1.4.11.10 PullUploadSegment service PDUs

PullUploadSegment-RequestPDU ::= Gn_AccessSpecification

```
PullUploadSegment-ResponsePDU ::= SEQUENCE {
    loadData                    [0] IMPLICIT OctetString,
    moreFollows                 [1] IMPLICIT Gn_MoreFollows
}
```

4.1.4.11.11 PushDownloadSegment service PDUs

```

PushDownloadSegment-RequestPDU ::= SEQUENCE {
    loadRegionKeyAttribute      Gn_AccessSpecification,
    loadData                    [2] IMPLICIT OctetString,
    moreFollows                  [3] IMPLICIT Gn_MoreFollows,
    optionalParametersMap        [10] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    segmentNumber                [11] IMPLICIT Fi_SegmentNumber OPTIONAL
}
-- bit 1

```

PushDownloadSegment-ResponsePDU ::= NULL

4.1.4.11.12 TerminatePullDownload service PDUs

```

TerminatePullDownload-RequestPDU ::= SEQUENCE {
    loadRegionKeyAttribute      Gn_AccessSpecification,
    terminateReason              [2] IMPLICIT Lr_TerminateReason
}

```

TerminatePullDownload-ResponsePDU ::= NULL

4.1.4.11.13 TerminatePushDownload service PDUs

TerminatePushDownload-RequestPDU ::= Gn_AccessSpecification

TerminatePushDownload-ResponsePDU ::= Fi_TerminateReason

4.1.4.11.14 TerminatePullUpload service PDUs

TerminatePullUpload-RequestPDU ::= Gn_AccessSpecification

TerminatePullUpload-ResponsePDU ::= NULL

4.1.4.12 Function invocation ASE**4.1.4.12.1 ActionInvoke service PDU**

```

ActionInvoke-RequestPDU ::= SEQUENCE {
    keyAttribute                Gn_KeyAttribute,
    actionInvocationID           [6] IMPLICIT Fi_ActionInvokeID,
    optionalParametersMap        [10] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    executionArgument            [7] IMPLICIT OctetString OPTIONAL
}
-- bit 1

```

4.1.4.12.2 ActionReturn service PDU

```

ActionReturn-RequestPDU ::= SEQUENCE {
    actionInvocationID           [0] IMPLICIT Fi_ActionInvokeID,
    actionResults CHOICE {
        actionSuccess              [1] IMPLICIT ANY,
        actionFailure              [2] IMPLICIT ErrorType
    }
}

```

4.1.4.12.3 Kill service PDUs

```

Kill-RequestPDU ::= SEQUENCE {
    keyAttribute                Gn_KeyAttribute,
    optionalParametersMap        [10] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    executionArgument            [7] IMPLICIT OctetString OPTIONAL
}
-- bit 1

```

Kill-ResponsePDU ::= NULL

4.1.4.12.4 Reset service PDUs

```

Reset-RequestPDU ::= SEQUENCE {
    keyAttribute                Gn_KeyAttribute,
    optionalParametersMap        [10] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    executionArgument            [7] IMPLICIT OctetString OPTIONAL
}
-- bit 1

```

Reset-ResponsePDU ::= NULL

4.1.4.12.5 Resume service PDUs

```
Resume-RequestPDU ::= SEQUENCE {
    keyAttribute          Gn_KeyAttribute,
    optionalParametersMap [10] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    executionArgument     [7] IMPLICIT OctetString OPTIONAL
}
-- bit 1
```

Resume-ResponsePDU ::= NULL

4.1.4.12.6 Start service PDUs

```
Start-RequestPDU ::= SEQUENCE {
    keyAttribute          Gn_KeyAttribute,
    optionalParametersMap [10] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    executionArgument     [7] IMPLICIT OctetString OPTIONAL
}
-- bit 1
```

Start-ResponsePDU ::= NULL

4.1.4.12.7 Stop service PDUs

```
Stop-RequestPDU ::= SEQUENCE {
    keyAttribute          Gn_KeyAttribute,
    optionalParametersMap [10] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    executionArgument     [7] IMPLICIT OctetString OPTIONAL
}
-- bit 1
```

Stop-ResponsePDU ::= NULL

4.1.4.13 Variable access ASE

4.1.4.13.1 Exchange service PDUs

```
Exchange-RequestPDU ::= SEQUENCE {
    specifierForVariableToWrite [0] Gn_KeyAttribute,
    valueToWrite                [1] IMPLICIT ANY,
    specifierForVariableToRead  [2] Gn_KeyAttribute,
    optionalParametersMap       [2] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    objectRevisionForVariableWriting [3] IMPLICIT Gn_ObjectRevision OPTIONAL,
    objectRevisionRequested     [4] IMPLICIT Gn_RequestFlag DEFAULT FALSE
}
-- bit 1
-- bit 2
```

```
Exchange-ResponsePDU ::= SEQUENCE {
    writeResult [0] IMPLICIT Vr_WriteResult,
    readResult CHOICE {
        status [1] IMPLICIT Er_Access,
        value [2] IMPLICIT ANY,
        valueWithRevision [3] IMPLICIT SEQUENCE {
            value [4] IMPLICIT ANY,
            objectRevision [5] IMPLICIT Gn_ObjectRevision
        }
    }
}
}
```

4.1.4.13.2 Exchange list service PDUs

```
ExchangeList-RequestPDU ::= SEQUENCE {
    writeList-Request [0] IMPLICIT WriteList-RequestPDU,
    readList-Request [1] IMPLICIT ReadList-RequestPDU
}
```

```
ExchangeList-Response ::= SEQUENCE {
    writeList-Response [0] IMPLICIT WriteList-ResponsePDU,
    readList-Response [1] IMPLICIT ReadList-ResponsePDU
}
```

4.1.4.13.3 Information report service PDU

```
InformationReport-RequestPDU ::= SEQUENCE {
    value [4] IMPLICIT ANY,
    optionalParametersMap [10] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    variableSpecifier      Gn_KeyAttribute OPTIONAL,
    subIndex [3] IMPLICIT Gn_SubIndex OPTIONAL,
    objectRevision [11] IMPLICIT Gn_ObjectRevision OPTIONAL
}
-- bit 1
-- bit 2
-- bit 3
```


4.1.4.13.4 Information report with type service PDU

```

InformationReportWithType-RequestPDU ::= SEQUENCE{
    TypeDescription          [4] Gn_TypeDescription,
    value                    [5] IMPLICIT ANY,
    optionalParametersMap    [10] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    variableSpecifier        Gn_KeyAttribute OPTIONAL,           -- bit 1
    subIndex                 [3] IMPLICIT Gn_SubIndex OPTIONAL,  -- bit 2
    objectRevision           [11] IMPLICIT Gn_ObjectRevision OPTIONAL -- bit 3
}

```

4.1.4.13.5 Information list report service PDU

```

InformationReportList-RequestPDU ::= SEQUENCE {
    ListOfVariableSpecifier CHOICE {
        variableList          [0] IMPLICIT Gn_KeyAttribute,
        listOfVariable        [1] IMPLICIT Gn_ListOfVariableAccess
    },
    listOfDataType            [7] IMPLICIT SEQUENCE OF Gn_FullyNestedTypeDescription,
    listOfData                [8] IMPLICIT SEQUENCE OF ANY,
    optionalParametersMap     [10] IMPLICIT Gn_OptionalParametersMap8,
    userDetail                [11] IMPLICIT OctetString OPTIONAL,      -- bit 1
    listOfObjectRevision      [12] SEQUENCE OF Gn_ObjectRevision OPTIONAL -- bit 2
}

```

- The listOfDataType should either contain one element per variable or should be empty. The listOfObjectRevision should contain one element per variable or should be absent.

4.1.4.13.6 Read service PDUs

```

Read1-RequestPDU ::= SEQUENCE {
    variableSpecifier        Gn_KeyAttribute,
    optionalParametersMap    [10] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    objectRevisionRequested  [11] IMPLICIT Gn_RequestFlag DEFAULT FALSE OPTIONAL -- bit 1
}

Read2-RequestPDU ::= SEQUENCE {
    variableSpecifier        Gn_KeyAttribute,
    optionalParametersMap    [10] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    subIndex                 [3] IMPLICIT Gn_SubIndex OPTIONAL -- bit 1
}

Read1-ResponsePDU ::= SEQUENCE {
    value                    [0] IMPLICIT ANY,
    optionalParametersMap    [1] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    objectRevision           [2] IMPLICIT Gn_ObjectRevision OPTIONAL -- bit 1
}

Read2-ResponsePDU ::= ANY

```

4.1.4.13.7 PhysicalRead service PDUs

```

PhysicalRead-RequestPDU ::= SEQUENCE {
    local-address            [0] IMPLICIT Gn_PhysicalAddress,
    length                   [1] IMPLICIT Gn_Length
}

```

PhysicalRead-ResponsePDU ::= ANY

4.1.4.13.8 ReadWithType service PDUs

```

ReadWithType-RequestPDU ::= SEQUENCE{
    variableSpecifier        Gn_KeyAttribute,
    optionalParametersMap    [10] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    subIndex                 [3] IMPLICIT Gn_SubIndex OPTIONAL -- bit 1
}

ReadWithType-ResponsePDU ::= SEQUENCE{
    typeDescription          [0] Gn_TypeDescription,
    value                    [1] IMPLICIT ANY,
    optionalParametersMap    [10] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    objectRevision           [11] IMPLICIT Gn_ObjectRevision OPTIONAL -- bit 1
}

```

4.1.4.13.9 Read list service PDUs

```

ReadList-RequestPDU ::= SEQUENCE {
    ListOfVariableSpecifier CHOICE {
        variableList                [0] IMPLICIT Gn_KeyAttributes,
        listOfVariable                [1] IMPLICIT Gn_ListOfVariableAccess,
        listOfNumericAddress          [2] IMPLICIT SEQUENCE OF Gn_NumericAddress
    },
    additionalInfoRequested          BitString8 {
        typeDescriptionRequested      (1),
        objectRevisionRequested       (2)
    }
}

ReadList-ResponsePDU ::= SEQUENCE {
    ListOfAccessResult               [0] IMPLICIT BitString,
    listOfDataType                   [1] IMPLICIT SEQUENCE OF Gn_FullyNestedTypeDescription,
    listOfData                       [2] IMPLICIT SEQUENCE OF ANY,
    listOfObjectRevision             [3] IMPLICIT SEQUENCE OF Gn_ObjectRevision
}

```

- The listOfAccessResult parameter specifies for each requested variable if the access was successful (bit = 1) or not (bit = 0).
- The listOfDataType should either contain one element per variable or should be empty. The same applies for the listOfObjectRevision.
- The listOfData parameter specifies for each requested variable the value of the variable if the access succeeds or an ErrorClass if it fails. The ErrorClass is coded as an unsigned16 where the most significant octet contains the “error class” and the least significant octet contains the “error code” (see ErrorClass definition).

4.1.4.13.10 Write service PDUs

```

Write-RequestPDU ::= SEQUENCE {
    variableSpecifier                Gn_KeyAttribute,
    value                            [4] IMPLICIT ANY,
    optionalParametersMap            [10] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    subIndex                         [3] IMPLICIT Gn_SubIndex OPTIONAL,
    objectRevision                   [11] IMPLICIT Gn_ObjectRevision OPTIONAL
}

```

-- bit 1
-- bit 2

Write-ResponsePDU ::= NULL

4.1.4.13.11 PhysicalWrite service PDUs

```

PhysicalWrite-RequestPDU ::= SEQUENCE {
    local-address                    [0] IMPLICIT Gn_PhysicalAddress,
    data                             [1] IMPLICIT ANY
}

```

PhysicalWrite-ResponsePDU ::= NULL

4.1.4.13.12 WriteWithType service PDUs

```

WriteWithType-RequestPDU ::= SEQUENCE{
    VariableSpecifier                Gn_KeyAttribute,
    TypeDescription                  [4] Gn_TypeDescription,
    value                            [5] IMPLICIT ANY,
    optionalParametersMap            [10] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    subIndex                         [3] IMPLICIT Gn_SubIndex OPTIONAL,
    objectRevision                   [11] IMPLICIT Gn_ObjectRevision OPTIONAL
}

```

- bit 1
- bit 2

WriteWithType-ResponsePDU ::= NULL

4.1.4.13.13 Write list service PDUs

```

WriteList-RequestPDU ::= SEQUENCE {
    ListOfVariableSpecifier CHOICE {
        variableList                [0] IMPLICIT Gn_KeyAttributes,
        listOfVariable                [1] IMPLICIT Gn_ListOfVariableAccess,
        listOfNumericAddress          [2] IMPLICIT SEQUENCE OF Gn_NumericAddress
    },
    listOfDataType                   [3] IMPLICIT SEQUENCE OF Gn_FullyNestedTypeDescription,
    listOfValue                       [4] IMPLICIT SEQUENCE OF ANY,
    listOfObjectRevision             [5] IMPLICIT SEQUENCE OF Gn_ObjectRevision
}

```

- The listOfDataType should either contain one element per variable or should be empty. The same applies for the listOfObjectRevision.

```
WriteList-ResponsePDU ::= SEQUENCE {
    ListOfAccessResult          [0] IMPLICIT BitString,
    listOfAccessError          [1] IMPLICIT SEQUENCE OF ErrorClass
}
```

- The listOfAccessResult parameter specifies for each requested variable if the access was successful (bit = 1) or not (bit = 0).
- The listOfAccessError contains one error code for each failed access. If a variable access was successful, there is no corresponding error code in this list.
- The ErrorClass is coded as an unsigned16 where the most significant octet contains the “error class” and the least significant octet contains the “error code” (see ErrorClass definition).

4.1.4.14 Event management ASE

4.1.4.14.1 Confirmed acknowledge event service PDUs

```
AcknowledgeEventNotification-RequestPDU ::= SEQUENCE {
    keyAttribute                Gn_AccessSpecification,
    reportedEventCount          [2] IMPLICIT Ev_EventCount
}
```

```
AcknowledgeEventNotification-ResponsePDU ::= NULL
```

4.1.4.14.2 EnableEvent service PDUs

```
EnableEvent-RequestPDU ::= SEQUENCE {
    keyAttribute                Gn_AccessSpecification,
    enabled                     [2] IMPLICIT Gn_Enabled
}
```

```
EnableEvent-ResponsePDU ::= NULL
```

4.1.4.14.3 EnableEventList service PDUs

```
EnableEventList-RequestPDU ::= SEQUENCE {
    listOfKeyAttribute          [0] IMPLICIT SEQUENCE OF Gn_AccessSpecification,
    listOfEnabledFlags          [1] IMPLICIT BitString
}
```

```
EnableEventList-ResponsePDU ::= SEQUENCE {
    listOfEnableResult          [0] IMPLICIT BitString,
    listOfError                 [1] IMPLICIT ErrorClass
}
```

4.1.4.14.4 Event notification service PDU

```
EventNotificationList-RequestPDU ::= SEQUENCE {
    eventNotifierID             IMPLICIT Gn_NumericID,
    eventDataContained          Boolean,
    notificationTypeInfo CHOICE {
        basic                    [0] NULL,
        sequenced                [1] Ev_SequenceNumber,
        time-of-notification     [2] Ev_TimeTag,
        compound                  [3] IMPLICIT SEQUENCE {
            sequence-number      Ev_SequenceNumber,
            time-tag              Ev_TimeTag
        }
    },
    listOfEventID               SEQUENCE OF Gn_AccessSpecification,
    listOfEv_EventDataType      SEQUENCE OF Gn_FullyNestedTypeDescription,
    listOfEventContents CHOICE {
        simple                    [0] SEQUENCE OF ANY, -- Null, Ev_EventData,
        reportedEventCount       [1] SEQUENCE OF ANY, -- Ev_EventCount, Ev_EventData
        eventDetectionTime       [2] SEQUENCE OF ANY, -- Ev_TimeTag, Ev_EventData
        composite                 [3] SEQUENCE OF ANY -- Ev_EventCount, Ev_TimeTag, Ev_EventData
    }
}
```

- If the eventDataContained parameter is TRUE, the Ev_EventData value shall be present.
- The components of event contents shall be appended together without any gap.

```

EventNotification-RequestPDU ::= SEQUENCE {
    KeyAttribute          Gn_KeyAttribute,
    eventNumber           [2] IMPLICIT Unsigned8,
    optionalParametersMap [10] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    eventData             [3] IMPLICIT ANY OPTIONAL,
    eventNotifierID       [11] IMPLICIT Gn_NumericID OPTIONAL,
    sequenceNumber        [12] IMPLICIT Ev_SequenceNumber OPTIONAL,
    notificationTime      [13] IMPLICIT Ev_TimeTag OPTIONAL
}

```

4.1.4.14.5 Event notification with type service PDU

```

EventNotificationWithType-RequestPDU ::= SEQUENCE{
    KeyAttribute          Gn_KeyAttribute,
    eventNumber           [2] IMPLICIT Unsigned8,
    typeDescription       [3] Gn_TypeDescription,
    optionalParametersMap [10] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    value                [4] IMPLICIT ANY OPTIONAL,
    eventNotifierID       [11] IMPLICIT Gn_NumericID OPTIONAL,
    sequenceNumber        [12] IMPLICIT Ev_SequenceNumber OPTIONAL,
    notificationTime      [13] IMPLICIT Ev_TimeTag OPTIONAL
}

```

4.1.4.14.6 Get event summary service PDUs

```

GetEventSummary-RequestPDU ::= SEQUENCE {
    eventID               [0] IMPLICIT Gn_NumericID,
    optionalParametersMap [1] IMPLICIT OptionalParametersMap8 OPTIONAL,
    recoveredMessageRequested [2] IMPLICIT Gn_RequestFlag OPTIONAL,
    objectRevisionRequested [3] IMPLICIT Gn_RequestFlag OPTIONAL
}

```

```

GetEventSummary-ResponsePDU ::= SEQUENCE {
    eventStatus           [0] IMPLICIT Ev_EventStatus,
    optionalParametersMap [1] IMPLICIT OptionalParametersMap8 OPTIONAL,
    objectRevision        [2] IMPLICIT Gn_ObjectRevision OPTIONAL,
    recoveredMessage       [3] IMPLICIT Ev_EventMessage OPTIONAL
}

```

4.1.4.14.7 Get event summary list service PDUs

```

GetEventSummaryList-RequestPDU ::= SEQUENCE {
    request events        CHOICE {
        allEvents         [1] IMPLICIT Null,
        listOfEvents      [2] IMPLICIT SEQUENCE OF Gn_AccessSpecification
    },
    additionalInfoRequest [3] IMPLICIT BitString8 {
        recoveredMessageRequested (1), --true for requested
        objectRevisionRequested   (2) --true for requested
    }
}

```

```

GetEventSummaryList-ResponsePDU ::= CHOICE {
    listOfEvents [1] IMPLICIT SEQUENCE {
        listOfAccessResult [0] IMPLICIT BitString,
        listOfEventMessage Ev_ListOfEventResponse,
        listOfObjectRevision [5] IMPLICIT SEQUENCE OF Gn_ObjectRevision
    },
    allEvents [2] IMPLICIT SEQUENCE {
        listOfEventID [0] IMPLICIT SEQUENCE OF Gn_AccessSpecification,
        listOfEventMessage Ev_ListOfEventResponse,
        listOfObjectRevision [5] IMPLICIT SEQUENCE OF Gn_ObjectRevision
    }
}

```

— The listOfAccessResult parameter specifies for each requested event, if the access was successful (bit=1) or not (bit=0).

4.1.4.14.8 Notification recovery service PDU

```

NotificationRecovery-RequestPDU ::= SEQUENCE {
    notifierID           [0] IMPLICIT Gn_NumericID,
    optionalParametersMap [1] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    numberToRecover       [2] IMPLICIT Ev_NumberToRecover DEFAULT 1 OPTIONAL,
    sequenceNumber        [3] IMPLICIT Ev_SequenceNumber OPTIONAL
}

```

4.1.4.14.9 Confirmed acknowledge event list service PDUs

```
ConfirmedAcknowledgedEventList-RequestPDU ::= SEQUENCE {
    acknowledgmentPolicy          [0] IMPLICIT Ev_AcknowledgmentPolicy,
    listOfEventID                  [1] IMPLICIT SEQUENCE OF Gn_AccessSpecification,
    listOfMessageSpecifier CHOICE {
        reportedEventCount          [2] IMPLICIT SEQUENCE OF Ev_EventCount,
        eventDetectionTime          [3] IMPLICIT SEQUENCE OF Ev_TimeTag,
        eventCountAndDetectionTime [4] IMPLICIT SEQUENCE OF ANY -- Ev_EventCount, Ev_TimeTag
    },
    listOfAcknowledgmentData       [5] IMPLICIT SEQUENCE OF OctetString
}

ConfirmedAcknowledgedEventList-ResponsePDU ::= SEQUENCE {
    listOfAccessResult             [0] IMPLICIT BitString,
    listOfError                    [1] IMPLICIT ErrorClass
}
```

4.1.4.14.10 Query event summary list service PDUs

```
QueryEventSummaryList-RequestPDU ::= SEQUENCE {
    eventselectionCriterion        [0] IMPLICIT BitString8,
    requestedEvent CHOICE {
        allEvents                  [1] IMPLICIT Null,
        listOfEvents               [2] IMPLICIT SEQUENCE OF Gn_AccessSpecification
    },
    additionalInfoRequested       [3] IMPLICIT BitString8 {
        recoveredMessageRequested (1), --true for requested
        objectRevisionRequested   (2) --true for requested
    }
}

QueryEventSummaryList-ResponsePDU ::= CHOICE {
    listOfEvents                  [1] IMPLICIT SEQUENCE {
        listOfAccessResult        [0] IMPLICIT BitString,
        listOfEventMessage        Ev_ListOfEventResponse,
        listOfObjectRevision      [5] IMPLICIT SEQUENCE OF Gn_ObjectRevision
    },
    allEvents                    [2] IMPLICIT SEQUENCE {
        listOfEventsID            [0] IMPLICIT SEQUENCE OF Gn_AccessSpecification,
        listOfEventMessage        Ev_ListOfEventResponse,
        listOfObjectRevision      [5] IMPLICIT SEQUENCE OF Gn_ObjectRevision
    }
}
```

- The listOfAccessResult parameter specifies for each event requested if the access was successful (bit=1) or not (bit=0).

4.1.4.15 Type definitions

4.1.4.15.1 AP ASE types

4.1.4.15.1.1 Ap_AccessProtectionSupported

```
Ap_AccessProtectionSupported ::= Boolean
-- True means Access Protection is supported.
-- False means Access Protection is not supported.
```

4.1.4.15.1.2 Ap_AccessControl

```

Ap_AccessControl ::= BitString {
    password_Bit1      (8),
    password_Bit2      (7),
    password_Bit3      (6),
    password_Bit4      (5),
    password_Bit5      (4),
    password_Bit6      (3),
    password_Bit7      (2),
    password_Bit8      (1),
    access_Groups-1    (16),
    access_Groups-2    (15),
    access_Groups-3    (14),
    access_Groups-4    (13),
    access_Groups-5    (12),
    access_Groups-6    (11),
    access_Groups-7    (10),
    access_Groups-8    (9)
}

```

-- The Password (Unsigned8) is encoded as a bit string.

4.1.4.15.2 AR ASE types

4.1.4.15.2.1 Ar_Dedicated

```

Ar_Dedicated ::= Boolean

```

-- The value of TRUE means the AREP is dedicated.
 -- The value of FALSE means the AREP is not dedicated.

4.1.4.15.2.2 AREP

```

RespondingAREP ::= Unsigned32
RequestingAREP ::= Unsigned32

```

-- Responding DLCEP address
 -- Requesting DLCEP address

4.1.4.15.2.3 Abort types

4.1.4.15.2.3.1 Identifier

```

Identifier ::= Unsigned8 {
    user      (1),
    nma      (5)
}

```

4.1.4.15.2.3.2 Reason code

```

ReasonCode ::= Unsigned8

```

4.1.4.15.2.3.3 Additional detail

```

AdditionalDetail ::= OctetString

```

4.1.4.15.2.4 Ar_ServiceTimeOut

```

Ar_ServiceTimeOut ::= Unsigned8

```

4.1.4.15.3 Data Link Layer types

4.1.4.15.3.1 General

The data types listed here are defined in IEC 61158-3 and IEC 61158-4.

4.1.4.15.3.2 DI_CyclicInterval

```

DI_CyclicInterval ::= Unsigned32

```

4.1.4.15.3.3 DI_DIsapAddress

```

DI_DIsapAddress ::= Unsigned32

```

4.1.4.15.3.4 DI_PermissibleJitter

```

DI_PermissibleJitter ::= Unsigned16

```

4.1.4.15.3.5 DI_SduSize

DI_SduSize ::= Unsigned16

4.1.4.15.4 Data type ASE types**4.1.4.15.4.1 Dt_ListOfFields**

```

Dt_ListOfFields ::= SEQUENCE {
    fieldDataType                [0] IMPLICIT Gn_NumericID,
    optionalParametersMap       [2] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    fieldName                    [0] IMPLICIT Gn_Name OPTIONAL
}
-- bit 1

```

4.1.4.15.5 Event ASE types**4.1.4.15.5.1 Ev_AcknowledgmentDataSpecifier**

Ev_AcknowledgmentDataSpecifier ::= Ev_DataSpecifier

4.1.4.15.5.2 Ev_AcknowledgementPolicy

```

Ev_AcknowledgementPolicy ::= Boolean
-- True means all messages are to be acknowledged.
-- False means only the specified message is to be acknowledged.

```

4.1.4.15.5.3 Ev_ContainedEventData

```

Ev_ContainedEventData ::= Boolean
-- True means the event message contains event data.
-- False means the event message does not contain event data.

```

4.1.4.15.5.4 Ev_ContainedMessageType

```

Ev_ContainedMessageType ::= Unsigned8 {
    simple                (1),
    reportedEventCount    (2),
    eventDetectionTime     (3),
    composite              (4),
    any                    (5)
}

```

4.1.4.15.5.5 Ev_DataSpecifier

```

Ev_DataSpecifier ::= CHOICE {
    variableID            [0] IMPLICIT Gn_NumericID,
    dataTypeID            [1] IMPLICIT Gn_NumericID,
    dataLength            [2] IMPLICIT Unsigned8
}

```

4.1.4.15.5.6 Ev_EventCount

En_EventCount ::= Unsigned8

4.1.4.15.5.7 Ev_EventData

Ev_EventData ::= ANY

— Ev_EventData contains the data associated with the respective event. If the data is not to be included in the notification, its length is 0.

4.1.4.15.5.8 Ev_EventMessage

```

Ev_EventMessage ::= CHOICE {
    eventKeyAttributeOnly      [0] IMPLICIT Gn_KeyAttribute,
    eventKeyAttributeWithCount [1] IMPLICIT Ev_KeyAttributeWithMessageCount,
    eventKeyAttributeWithTime  [2] IMPLICIT Ev_KeyAttributeWithData,
    eventKeyAttributeWithData  [3] IMPLICIT Ev_KeyAttributeWithData
}

Ev_ListOfEventResponse ::= CHOICE {
    simple
        recoveredCount      [1] IMPLICIT SEQUENCE OF Ev_Status,
        recoveredDetectionTime [2] IMPLICIT SEQUENCE OF ANY, --Ev_Status, RecoveredCount
    composite
        recoveredDetectionTime [3] IMPLICIT SEQUENCE OF ANY,
        --Ev_tStatus, RecoveredDetectionTime
        recoveredCount          [4] IMPLICIT SEQUENCE OF ANY
        --Ev_Status, RecoveredCount, RecoveredDetectionTime
}

```

- The components of Ev_Status & RecoveredCount shall be appended together without any gap.
- The components of Ev_Status & RecoveredDetectionTime shall be appended together without any gap.
- The components of Ev_Status & RecoveredCount & RecoveredDetectionTime shall be appended together without any gap.

```
Ev_Status ::= BitString8 {
    active          (1), --true for active
    detected        (2), --true for detected
    enabled         (3), --true for enabled
    acknowledged    (4) --true for acked
}
```

RecoveredCount ::= Unsigned8

RecoveredDetectionTime ::= Ev_TimeTag

4.1.4.15.5.9 Ev_EventStatus

```
Ev_EventStatus ::= SEQUENCE {
    active          [0] IMPLICIT Boolean,          -- True means active
    detected        [1] IMPLICIT Boolean,          -- True means detected
    enabled         [2] IMPLICIT Boolean,          -- True means enabled
    acknowledgmentSupported [3] IMPLICIT Boolean,  -- True means supported
    acknowledged    [4] IMPLICIT Boolean          -- True means acknowledged
}
```

4.1.4.15.5.10 Ev_ErrorStatus

```
Ev_ErrorStatus ::= Boolean
-- True means the underlying condition is active.
-- False means the underlying condition is not active.
```

4.1.4.15.5.11 Ev_LastReportedEventInfo

```
Ev_LastReportedEventInfo ::= SEQUENCE {
    optionalParametersMap [0] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    count                 [1] IMPLICIT Unsigned8 OPTIONAL,          -- bit 1
    detectionTime         [2] IMPLICIT Ev_TimeTag OPTIONAL          -- bit 2
}
```

4.1.4.15.5.12 Ev_MessageType

```
Ev_MessageType ::= Unsigned8 {
    simpleMessage          (1),
    reportedEventCount     (2),
    eventDetectionTime     (3),
    composite              (4),
    simpleMessageWithData  (5),
    reportedEventCountWithData (6),
    eventDetectionTimeWithData (7),
    compositeWithData      (8)
}
```

4.1.4.15.5.13 Ev_NotificationType

```
Ev_NotificationType ::= Unsigned8 {
    basic          (1),
    sequenced     (2),
    timeOfNotification (3),
    compound       (4)
}
```

4.1.4.15.5.14 Ev_NumberToRecover

Ev_NumberToRecover ::= Unsigned8

4.1.4.15.5.15 Ev_SequenceNumber

Ev_SequenceNumber ::= Unsigned8

4.1.4.15.5.16 Ev_SelectionCriterion

```
Ev_SelectionCriterion ::= SEQUENCE {
    enabled                [0] IMPLICIT Boolean,
    detected               [1] IMPLICIT Boolean,
    acked                  [2] IMPLICIT Boolean,
    active                  [3] IMPLICIT Boolean
}
```

4.1.4.15.5.17 Ev_TimeTag

```
Ev_TimeTag ::= Unsigned16
```

4.1.4.15.6 Function invocation ASE types**4.1.4.15.6.1 Fi_ActionInvokeID**

```
Fi_ActionInvokeID ::= Unsigned8
```

4.1.4.15.6.2 Fi_AccessPrivilege

```
Fi_AccessPrivilege ::= BitString {
    rightToStartPassword      (24),
    rightToStopPassword       (23),
    rightToDeletePassword     (22),
    rightToStartAccessGroup   (20),
    rightToStopAccessGroup    (19),
    rightToDeleteAccessGroup   (18),
    rightToStartAllPartner     (32),
    rightToStopAllPartner     (31),
    rightToDeleteAllPartner    (30),
    password_Bit1             (8), -- The Password (Unsigned8) is encoded as a bit string.
    password_Bit2             (7),
    password_Bit3             (6),
    password_Bit4             (5),
    password_Bit5             (4),
    password_Bit6             (3),
    password_Bit7             (2),
    password_Bit8             (1),
    access_Groups-1           (16),
    access_Groups-2           (15),
    access_Groups-3           (14),
    access_Groups-4           (13),
    access_Groups-5           (12),
    access_Groups-6           (11),
    access_Groups-7           (10),
    access_Groups-8           (9)
}
```

4.1.4.15.6.3 Fi_SegmentNumber

```
Fi_SegmentNumber ::= Unsigned32 -- The first segment number shall be one(1).
```

4.1.4.15.6.4 Fi_State

```
Fi_State ::= Unsigned8 {
    unrunnable               (1),
    idle                     (2),
    running                  (3),
    stopped                  (4),
    starting                 (5),
    stopping                 (6),
    resuming                 (7),
    resetting                (8)
}
```

4.1.4.15.7 General types**4.1.4.15.7.1 Gn_AccessSpecification**

```
Gn_AccessSpecification ::= CHOICE {
    index                    [0] IMPLICIT Index,
    name                     [1] IMPLICIT Name
}
```

4.1.4.15.7.2 Gn_Deletable

Gn_Deletable ::= Boolean
 -- True means deletable.
 -- False means not deletable.

4.1.4.15.7.3 Gn_Enabled

Gn_Enabled ::= Boolean
 -- True means enabled.
 -- False means disabled.

4.1.4.15.7.4 Gn_FullyNestedTypeDescription

Gn_FullyNestedTypeDescription ::= CHOICE {
 Boolean [1] Gn_Length,
 integer8 [2] Gn_Length,
 integer16 [3] Gn_Length,
 integer32 [4] Gn_Length,
 unsigned8 [5] Gn_Length,
 unsigned16 [6] Gn_Length,
 unsigned32 [7] Gn_Length,
 float32 [8] Gn_Length,
 float64 [15] Gn_Length,
 binaryDate [11] Gn_Length,
 timeOfDay [12] Gn_Length,
 timeDifference [13] Gn_Length,
 universalTime [16] Gn_Length,
 fieldbusTime [17] Gn_Length,
 time [21] Gn_Length,
 bitstring8 [22] Gn_Length,
 bitstring16 [23] Gn_Length,
 bitstring32 [24] Gn_Length,
 visiblestring1 [25] Gn_Length,
 visiblestring2 [26] Gn_Length,
 visiblestring4 [27] Gn_Length,
 visiblestring8 [28] Gn_Length,
 visiblestring16 [29] Gn_Length,
 octetstring1 [30] Gn_Length,
 octetstring2 [31] Gn_Length,
 octetstring4 [32] Gn_Length,
 octetstring8 [33] Gn_Length,
 octetstring16 [34] Gn_Length,
 bcd [35] Gn_Length,
 iso10646char [36] Gn_Length,
 binarytime0 [40] Gn_Length,
 binarytime1 [41] Gn_Length,
 binarytime2 [42] Gn_Length,
 binarytime3 [43] Gn_Length,
 binarytime4 [44] Gn_Length,
 binarytime5 [45] Gn_Length,
 binarytime6 [46] Gn_Length,
 binarytime7 [47] Gn_Length,
 binarytime8 [48] Gn_Length,
 binarytime9 [49] Gn_Length,
 visiblestring [9] Gn_Length,
 octetstring [10] Gn_Length,
 bitstring [14] Gn_Length,
 compactBooleanArray [37] Gn_Length,
 compactBCDArray [38] Gn_Length,
 iso646string [39] Gn_Length,
 array [55] IMPLICIT SEQUENCE {
 number-of-elements [0] IMPLICIT Unsigned8,
 element-type [1] Gn_FullyNestedTypeDescription
 },
 structure [56] IMPLICIT SEQUENCE OF Gn_FullyNestedTypeDescription
}

4.1.4.15.7.5 Gn_KeyAttribute

Gn_KeyAttribute ::= CHOICE {
 -- When this type is specified, only the key attributes of the class referenced are valid.
 numericID [0] IMPLICIT Gn_NumericID,
 name [1] IMPLICIT Gn_Name,
 listName [2] IMPLICIT Gn_Name,
 numericAddress [4] IMPLICIT Gn_NumericAddress,
 symbolicAddress [5] IMPLICIT Gn_SymbolicAddress
}

4.1.4.15.7.6 Gn_Length

Gn_Length ::= Unsigned8

4.1.4.15.7.7 Gn_ListOfVariableAccess

```
Gn_ListOfVariableAccess ::= SEQUENCE OF CHOICE {
    numericID          [0] IMPLICIT Gn_NumericID,
    name               [1] IMPLICIT Gn_Name,
    listName           [2] IMPLICIT Gn_Name,
    symbolicAddress    [5] IMPLICIT Gn_SymbolicAddress,
    partialNumericID   [6] IMPLICIT SEQUENCE {
        numericID      [0] IMPLICIT Gn_NumericID,
        partialAccess
    },
    partialName        [7] IMPLICIT SEQUENCE {
        name           [0] IMPLICIT Gn_Name,
        partialAccess  Gn_PartialAccess
    },
    partialListName    [8] IMPLICIT SEQUENCE {
        listName       [0] IMPLICIT Gn_Name,
        partialAccess  Gn_PartialAccess
    }
}
```

4.1.4.15.7.8 Gn_MoreFollows

Gn_MoreFollows ::= Boolean

4.1.4.15.7.9 Gn_Name

Gn_Name ::= OctetString

4.1.4.15.7.10 Gn_NameWithComponent

```
Gn_NameWithComponent ::= SEQUENCE {
    name          [0] IMPLICIT Gn_Name,
    component     [1] IMPLICIT Unsigned16
}
```

4.1.4.15.7.11 Gn_NumericAddress

```
Gn_NumericAddress ::= SEQUENCE {
    startAddress [0] IMPLICIT Unsigned32, -- physical address of the starting location
    length       [1] IMPLICIT Unsigned16 -- octet length of a memory block
}
```

4.1.4.15.7.12 Gn_NumericID

Gn_NumericID ::= Unsigned16 -- The values of this parameter are unique within an AP.

4.1.4.15.7.13 Gn_NumericIDWithComponent

```
Gn_NumericIDWithComponent ::= SEQUENCE {
    numericID [0] IMPLICIT Gn_NumericID,
    component [1] IMPLICIT Unsigned16
}
```

4.1.4.15.7.14 Gn_ObjectDefinition

Gn_ObjectDefinition ::= OctetString -- The semantics of this parameter are application specific.

4.1.4.15.7.15 Gn_ObjectRevision

Gn_ObjectRevision ::= Unsigned8
 -- bits 8 - 5 are used as the Major Revision
 -- bits 4 - 1 are used as the Minor Revision
 -- each may have a value between zero and 15 inclusive

4.1.4.15.7.16 Gn_OptionalParametersMaps

Gn_OptionalParametersMap8 ::= BitString8

Gn_OptionalParametersMap16 ::= BitString16

Gn_OptionalParametersMap32 ::= BitString32

- The above types have special semantics to represent which optional parameters are included
- in the transfer syntax. Bit 1 corresponds to the first optional parameter of a given primitive,
- bit 2 corresponds to the second optional parameter, up to bit "n" corresponds to the "n-th"
- optional parameters. The value of one indicates that the corresponding optional parameter is
- present in the transfer syntax, and the value of zero indicates that it is not present.

4.1.4.15.7.17 Gn_PartialAccess

```
Gn_PartialAccess ::= CHOICE {
    componentID          [3] IMPLICIT Gn_SubIndex,
    componentList        [20] IMPLICIT SEQUENCE {
        nestingLevel     [1] Unsigned8,
        accessedElements [2] SEQUENCE OF Gn_Sub-Index
    }
}
```

- The componentID of the partial access selects a single element on the first level of nesting for access. The componentList specifies one possibly complex base element on a specified level of nesting and, if necessary, selects a number of subelements from this base element. The first elements (their number is given by nestingLevel) of the accessedElements list define the path to the base element. The rest of the list selects a number of subelements to be accessed. If the rest of the list is empty, the complete base element is accessed.

4.1.4.15.7.18 Gn_PhysicalAddress

Gn_PhysicalAddress ::= Unsigned32

4.1.4.15.7.19 Gn_RequestFlag

Gn_RequestFlag ::= Boolean

- TRUE means the specified parameter is requested in the response.
- FALSE means the specified parameter is not requested in the response.

4.1.4.15.7.20 Gn_Reusable

Gn_Reusable ::= Boolean

- True means reusable.
- False means not reusable.

4.1.4.15.7.21 Gn_SubIndex

Gn_SubIndex ::= Unsigned8

4.1.4.15.7.22 Gn_SymbolicAddress

Gn_SymbolicAddress ::= VisibleString

4.1.4.15.7.23 Gn_TypeDescription

```
Gn_TypeDescription ::= SEQUENCE OF CHOICE {
    simple [1] IMPLICIT ANY,
    -- Octet1: High octet Data Type Gn_NumericID
    -- Octet2: Low octet Data Type Gn_NumericID
    -- Octet3: Length
    array [2] IMPLICIT ANY,
    -- Octet1: High octet Data Type Gn_NumericID
    -- Octet2: Low octet Data Type Gn_NumericID
    -- Octet3: Length
    -- Octet4: Number of Elements
    structure [3] IMPLICIT ANY
    -- Octet 1 : High octet Data Type Index (Element 1)
    -- Octet 2 : Low octet Data Type Index (Element 1)
    -- Octet 3 : Length (Element 1)
    -- Octet 3n-2 : High octet Data Type Index (Element n)
    -- Octet 3n-1 : Low octet Data Type Index (Element n)
    -- Octet 3n-0 : Length (Element n)
}
```

4.1.4.15.7.24 Gn_ObjectClass

```
Gn_ObjectClass ::= Unsigned8 {
    anyObject                (0),
    loadRegion               (2),
    functionInvocation        (3),
    event                    (4),
    fixedLength&StringDataType (5),
    structuredDataType        (6),
    fixedLength&StringVariable (7),
    arrayVariable             (8),
    dataStructureVariable     (9),
    variableList              (10),
    arrayDataType             (12),
    eventList                 (17),
    notifier                  (18),
    action                    (19)
}
```

4.1.4.15.8 Load region ASE types

4.1.4.15.8.1 Lr_FixedLengthSegment

```
Lr_FixedLengthSegment ::= Boolean
-- True means LR supports fixed-length segments only.
-- False means LR supports variable-length segments.
```

4.1.4.15.8.2 Lr_LocalAddress

```
Lr_LocalAddress ::= Unsigned32
```

4.1.4.15.8.3 Lr_Size

```
Lr_Size ::= Unsigned32
```

4.1.4.15.8.4 Lr_State

```
Lr_State ::= Unsigned8 {
    NotDownloadable          (1),
    downloadable              (2),
    downloading               (3),
    downloadFailure           (4),
    downloadSuccess           (5),
    loaded                    (6),
    inUse                     (7)
}
```

4.1.4.15.8.5 Lr_TerminateReason

```
Lr_TerminateReason ::= Boolean
-- True means the load process was successfully completed.
-- False means the load process was completed with failure.
```

4.1.4.15.8.6 Lr_UploadState

```
Lr_UploadState ::= Unsigned8 {
    notUploadable            (1),
    uploadable                (2),
    uploading                 (3),
    completingUpload          (4)
}
```

4.1.4.15.9 Management ASE types

4.1.4.15.9.1 Attribute maps

4.1.4.15.9.1.1 Mn_ActionAttrMap

```
Mn_ActionAttrMap ::= Bitstring16 {
    commonAttrValues          (1),
    listOfSupportedAttributes (2),
    listOfOperationalServices (3),
    reusable                  (4),
    listOfRelatedObjects       (5),
    actionInvokeServiceArgumentDataType (6),
    actionReturnServiceArgumentDataType (7)
}
```

4.1.4.15.9.1.2 Mn_ApAttrMap

```
Mn_ApAttrMap ::= BitString16 {
    commonAttrValues          (1),
    listOfSupportedAttributes (2),
    listOfSupportedServices   (3),
    manufacturerIdentifier    (4),
    ApdescriptorReference      (5),
    networkAccessState        (6),
    listOfArEndpointInfo      (7),
    listOfInuseArEndpointInfo (8),
    listOfSupportedApoClassAndObjects (9),
    listOfDisapAddresses      (10),
    listOfEventSummarySelectionCriteria (11)
}
```

4.1.4.15.9.1.3 Mn_AttributeMap

```
Mn_AttributeMap ::= SEQUENCE {
    Mn_CommonAttrMap,
    attributeMap CHOICE {
        apAttrMap          [1] IMPLICIT Mn_ApAttrMap,
        dataTypeAttrMap    [2] IMPLICIT Mn_DataTypeAttrMap,
        variableAttrMap     [3] IMPLICIT Mn_VariableAttrMap,
        variableListAttrMap [4] IMPLICIT Mn_VariableListAttrMap,
        eventAttrMap        [5] IMPLICIT Mn_EventAttrMap,
        eventListAttrMap    [6] IMPLICIT Mn_EventListAttrMap,
        notifierAttrMap     [7] IMPLICIT Mn_NotifierAttrMap,
        loadRegionAttrMap   [8] IMPLICIT Mn_LoadRegionAttrMap,
        functionInvocationAttrMap [9] IMPLICIT Mn_FunctionInvocationAttrMap,
        actionAttrMap       [10] IMPLICIT Mn_ActionAttrMap
    }
}
```

4.1.4.15.9.1.4 Mn_CommonAttrMap

```
Mn_CommonAttrMap ::= BitString8 {
    numericID      (1),
    name           (2),
    instanceOf     (3),
    parentClass    (4),
    userDescription (5),
    objectRevision (6)
}
```

4.1.4.15.9.1.5 Mn_DataTypeAttrMap

```
Mn_DataTypeAttrMap ::= BitString16 {
    commonAttrValues          (1),
    listOfSupportedAttributes (2),
    dataTypeNumericId         (3),
    dataTypeName              (4),
    format                    (5),
    octetLength               (6),
    numberOfFields            (7),
    listOfFields              (8),
    numberOfArrayElements     (9),
    arrayElementType          (10)
}
```

4.1.4.15.9.1.6 Mn_EventAttrMap

```
Mn_EventAttrMap ::= BitString16 {
    commonAttrValues          (1),
    listOfSupportedAttributes (2),
    listOfOperationalServices (3),
    eventStatus               (4),
    messageType               (5),
    accessPrivilege           (6),
    lastReportedEventInfo     (7),
    eventDataSpecifier        (8),
    acknowledgmentDataSpecifier (9)
}
```

4.1.4.15.9.1.7 Mn_EventListAttrMap

```

Mn_EventListAttrMap ::= BitString8 {
    commonAttrValues          (1),
    listOfSupportedAttributes  (2),
    listOfOperationalServices  (3),
    numberOfEntries           (4),
    listOfEvents               (5)
}

```

4.1.4.15.9.1.8 Mn_FunctionInvocationAttrMap

```

Mn_FunctionInvocationAttrMap ::= BitString16 {
    commonAttrValues          (1),
    listOfSupportedAttributes  (2),
    listOfOperationalServices  (3),
    accessPrivilege           (4),
    deletable                  (5),
    reusable                   (6),
    functionInvocationState     (7),
    numberOfRelatedObjectsInUse (8),
    listOfRelatedObjects       (9),
    startArgumentDataTypes     (10),
    stopServiceArgumentDataTypes (11),
    resumeServiceArgumentDataTypes (12),
    resetServiceArgumentDataTypes (13),
    killServiceArgumentDataTypes (14),
    executionArgument          (15)
}

```

4.1.4.15.9.1.9 Mn_LoadRegionAttrMap

```

Mn_LoadRegionAttrMap ::= BitString16 {
    commonAttrValues          (1),
    listOfSupportedAttributes  (2),
    listOfOperationalServices  (3),
    loadRegionSize            (4),
    accessPrivilege           (5),
    localAddress               (6),
    loadRegionState           (7),
    uploadState                (8),
    numberOfRelatedObjectsInUse (9),
    listOfRelatedObjects       (10),
    contentsSize               (11),
    loadImageName              (12),
    fixedLengthSegment         (13),
    fixedSegmentLength         (14),
    intersegmentRequestTimeout (15),
    loadRegionSharable         (16)
}

```

4.1.4.15.9.1.10 Mn_NotifierAttrMap

```

Mn_NotifierAttrMap ::= BitString16 {
    commonAttrValues          (1),
    listOfSupportedAttributes  (2),
    listOfOperationalServices  (3),
    enabled                    (4),
    notificationArepClass      (5),
    notificationArep           (6),
    notificationType           (7),
    containedMessageType       (8),
    containedEventData          (9),
    lastNotificationSequenceNumber (10),
    listOfEvents               (11)
}

```

4.1.4.15.9.1.11 Mn_VariableAttrMap

```
Mn_VariableAttrMap ::= BitString16 {
    commonAttrValues          (1),
    listOfSupportedAttributes  (2),
    listOfOperationalServices (3),
    symbolicAddress           (4),
    dataType                   (5),
    length                     (6),
    variableLengthConveyance   (7),
    numberOfArrayElements      (8),
    accessPrivilege            (9),
    localDetail                 (10),
    fullyNestedTypeDescription (11)
}
```

4.1.4.15.9.1.12 Mn_VariableListAttrMap

```
Mn_VariableListAttrMap ::= BitString8 {
    commonAttrValues          (1),
    listOfSupportedAttributes  (2),
    listOfOperationalServices (3),
    numberOfEntries           (4),
    listOfVariables            (5),
    deletable                  (6),
    accessPrivilege            (7)
}
```

4.1.4.15.9.2 Attribute values

4.1.4.15.9.2.1 Mn_ActionAttrValues

```
Mn_ActionAttrValues ::= SEQUENCE {
    optionalParametersMap [0] IMPLICIT Gn_OptionalParametersMap16 OPTIONAL,
    commonAttrValues      [1] IMPLICIT Mn_CommonAttrValues OPTIONAL,      -- bit 1
    listOfSupportedAttributes [2] IMPLICIT Mn_ActionAttrMap OPTIONAL,      -- bit 2
    listOfOperationalServices [3] IMPLICIT Mn_ActionServiceMap OPTIONAL,   -- bit 3
    reusable               [4] IMPLICIT Gn_Reusable DEFAULT TRUE,          -- bit 4
    listOfRelatedObjects   [5] IMPLICIT SEQUENCE OF Gn_KeyAttribute OPTIONAL, -- bit 5
    actionInvokeServiceArgumentDataType [6] IMPLICIT SEQUENCE OF Gn_NumericID OPTIONAL, -- bit 6
    actionActionServiceArgumentDataType [7] IMPLICIT SEQUENCE OF Gn_NumericID OPTIONAL -- bit 7
}
```

4.1.4.15.9.2.2 Mn_ApAttrValues

```
Mn_ApAttrValues ::= SEQUENCE {
    optionalParametersMap [0] IMPLICIT Gn_OptionalParametersMap16 OPTIONAL,
    commonAttrValues      [1] IMPLICIT Mn_CommonAttrValues OPTIONAL,      -- bit 1
    listOfSupportedAttributes [2] IMPLICIT Mn_ApAttrMap OPTIONAL,          -- bit 2
    listOfSupportedServices [3] IMPLICIT Mn_ApServiceMap OPTIONAL,         -- bit 3
    manufacturerIdentifier [4] IMPLICIT Mn_ManufacturerIdentifier OPTIONAL, -- bit 4
    ApdescriptorReference   [5] IMPLICIT OctetString OPTIONAL,              -- bit 5
    networkAccessState      [6] IMPLICIT Mn_NetworkAccessState OPTIONAL,    -- bit 6
    listOfArEndpointInfo    [7] IMPLICIT SEQUENCE OF Mn_ArEndpointInfo OPTIONAL, -- bit 7
    listOfInuseArEndpointInfo [8] IMPLICIT SEQUENCE OF Mn_InuseArEndpointInfo OPTIONAL, -- bit 8
    listOfSupportedApoClassAndObjects [9] IMPLICIT SEQUENCE OF Mn_ApoClassAndObjects OPTIONAL, -- bit 9
    listOfDisapAddresses    [10] IMPLICIT SEQUENCE OF Dl_DisapAddress OPTIONAL, -- bit 10
    listOfEventSummarySelectionCriteria [11] IMPLICIT SEQUENCE OF Ev_SelectionCriterion OPTIONAL -- bit 11
}
```

4.1.4.15.9.2.3 Mn_AttributeValues

```
Mn_AttributeValues ::= CHOICE {
    ApAttrValues [1] IMPLICIT Mn_ApAttrValues,
    dataTypeAttrValues [2] IMPLICIT Mn_DataTypeAttrValues,
    variableAttrValues [3] IMPLICIT Mn_VariableAttrValues,
    variableListAttrValues [4] IMPLICIT Mn_VariableListAttrValues,
    eventAttrValues [5] IMPLICIT Mn_EventAttrValues,
    eventListAttrValues [6] IMPLICIT Mn_EventListAttrValues,
    notifierAttrValues [7] IMPLICIT Mn_NotifierAttrValues,
    loadRegionAttrValues [8] IMPLICIT Mn_LoadRegionAttrValues,
    functionInvocationAttrValues [9] IMPLICIT Mn_FunctionInvocationAttrValues,
    actionAttrValues [10] IMPLICIT Mn_ActionAttrValues
}
```


4.1.4.15.9.2.4 Mn_CommonAttrValues

```

Mn_CommonAttrValues ::= SEQUENCE {
    optionalParametersMap      [0] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    numericID                  [1] IMPLICIT Gn_NumericID,                      -- bit 1
    name                       [2] IMPLICIT Gn_Name OPTIONAL,                  -- bit 2
    instanceOf                 [3] IMPLICIT Gn_NumericID OPTIONAL,             -- bit 3
    parentClass                [4] IMPLICIT Gn_NumericID OPTIONAL,             -- bit 4
    userDescription            [5] IMPLICIT OctetString OPTIONAL,              -- bit 5
    objectRevision             [6] IMPLICIT Gn_ObjectRevision OPTIONAL          -- bit 6
}

```

4.1.4.15.9.2.5 Mn_DataTypeAttrValues

```

Mn_DataTypeAttrValues ::= SEQUENCE {
    optionalParametersMap      [0] IMPLICIT Gn_OptionalParametersMap16 OPTIONAL,
    commonAttrValues           [1] IMPLICIT Mn_CommonAttrValues OPTIONAL,      -- bit 1
    listOfSupportedAttributes   [2] IMPLICIT Mn_DataTypeAttrMap OPTIONAL,      -- bit 2
    dataTypeNumericId          [3] IMPLICIT Gn_NumericID OPTIONAL,             -- bit 3
    dataTypeName               [4] IMPLICIT Gn_Name OPTIONAL,                 -- bit 4
    format                    [5] IMPLICIT Gn_NumericID OPTIONAL,             -- bit 5
    octetLength                [6] IMPLICIT Unsigned8 OPTIONAL,              -- bit 6
    numberOfFields             [7] IMPLICIT Unsigned8 OPTIONAL,              -- bit 7
    listOfFields               [8] IMPLICIT SEQUENCE OF Dt_ListOfFields OPTIONAL -- bit 8
    numberOfArrayElements      [9] IMPLICIT Gn_NumericID OPTIONAL,             -- bit 9
    arrayElementType          [10] IMPLICIT Gn_NumericID OPTIONAL -- bit 10
}

```

4.1.4.15.9.2.6 Mn_EventAttrValues

```

Mn_EventAttrValues ::= SEQUENCE {
    optionalParametersMap      [0] IMPLICIT Gn_OptionalParametersMap16 OPTIONAL,
    commonAttrValues           [1] IMPLICIT Mn_CommonAttrValues OPTIONAL,      -- bit 1
    listOfSupportedAttributes   [2] IMPLICIT Mn_EventAttrMap OPTIONAL,         -- bit 2
    listOfOperationalServices   [3] IMPLICIT Mn_EventServiceMap OPTIONAL,      -- bit 3
    eventStatus                [4] IMPLICIT Ev_EventStatus OPTIONAL,          -- bit 4
    messageType               [5] IMPLICIT Ev_MessageType OPTIONAL,          -- bit 5
    accessPrivilege            [6] IMPLICIT Vr_VariablelistAccessPrivilege OPTIONAL, -- bit 6
    lastReportedEventInfo      [7] IMPLICIT Ev_LastReportedEventInfo OPTIONAL, -- bit 7
    eventDataSpecifier         [8] IMPLICIT Ev_DataSpecifier OPTIONAL,         -- bit 8
    acknowledgmentDataSpecifier [9] IMPLICIT EV_AcknowledgementDataSpecifier OPTIONAL -- bit 9
}

```

4.1.4.15.9.2.7 Mn_EventListAttrValues

```

Mn_EventListAttrValues ::= SEQUENCE {
    optionalParametersMap      [0] IMPLICIT Gn_OptionalParametersMap8 OPTIONAL,
    commonAttrValues           [1] IMPLICIT Mn_CommonAttrValues OPTIONAL,      -- bit 1
    listOfSupportedAttributes   [2] IMPLICIT Mn_EventListAttrMap OPTIONAL,     -- bit 2
    listOfOperationalServices   [3] IMPLICIT Mn_EventListServiceMap OPTIONAL, -- bit 3
    numberOfEntries            [4] IMPLICIT Unsigned16 OPTIONAL,              -- bit 4
    listOfEvents               [5] IMPLICIT SEQUENCE OF Gn_NumericID OPTIONAL -- bit 5
}

```

4.1.4.15.9.2.8 Mn_FunctionInvocationAttrValues

```

Mn_FunctionInvocationAttrValues ::= SEQUENCE {
    optionalParametersMap      [0] IMPLICIT Gn_OptionalParametersMap16 OPTIONAL,
    commonAttrValues           [1] IMPLICIT Mn_CommonAttrValues OPTIONAL,      -- bit 1
    listOfSupportedAttributes   [2] IMPLICIT Mn_FunctionInvocationAttrMap OPTIONAL, -- bit 2
    listOfOperationalServices   [3] IMPLICIT Mn_FunctionInvocationServiceMap OPTIONAL, -- bit 3
    accessPrivilege            [4] IMPLICIT Vr_AccessPrivilege OPTIONAL,      -- bit 4
    deletable                  [5] IMPLICIT Mn_Deletable OPTIONAL,            -- bit 5
    reusable                   [6] IMPLICIT Mn_Reusable DEFAULT TRUE OPTIONAL, -- bit 6
    functionInvocationState     [7] IMPLICIT Fi_State OPTIONAL,              -- bit 7
    numberOfRelatedObjectsInUse [8] IMPLICIT Unsigned8 OPTIONAL,              -- bit 8
    listOfRelatedObjects       [9] IMPLICIT SEQUENCE OF Gn_KeyAttribute OPTIONAL, -- bit 9
    startArgumentDataTypes     [10] IMPLICIT Gn_NumericID OPTIONAL,           -- bit 10
    stopServiceArgumentDataTypes [11] IMPLICIT Gn_NumericID OPTIONAL,         -- bit 11
    resumeServiceArgumentDataTypes [12] IMPLICIT Gn_NumericID OPTIONAL,       -- bit 12
    resetServiceArgumentDataTypes [13] IMPLICIT Gn_NumericID OPTIONAL,        -- bit 13
    killServiceArgumentDataTypes [14] IMPLICIT Gn_NumericID OPTIONAL,         -- bit 14
    executionArgument          [15] IMPLICIT OctetString OPTIONAL             -- bit 15
}

```

4.1.4.15.9.2.9 Mn_LoadRegionAttrValues

```

Mn_LoadRegionAttrValues ::= SEQUENCE {
    optionalParametersMap      [0] IMPLICIT Gn_OptionalParametersMap16 OPTIONAL,
    commonAttrValues           [1] IMPLICIT Mn_CommonAttrValues OPTIONAL,      -- bit 1
    listOfSupportedAttributes   [2] IMPLICIT Mn_LoadRegionAttrValues OPTIONAL,  -- bit 2
    listOfOperationalServices   [3] IMPLICIT Mn_LoadRegionServiceMap OPTIONAL,  -- bit 3
    loadRegionSize              [4] IMPLICIT Lr_Size OPTIONAL,                  -- bit 4
    accessPrivilege             [5] IMPLICIT Vr_AccessPrivilege OPTIONAL,       -- bit 5
    localAddress                [6] IMPLICIT Lr_LocalAddress OPTIONAL,          -- bit 6
    loadRegionState             [7] IMPLICIT Lr_State OPTIONAL,                 -- bit 7
    uploadState                 [8] IMPLICIT Lr_UploadState OPTIONAL,           -- bit 8
    numberOfRelatedObjectsInUse [9] IMPLICIT Unsigned8 OPTIONAL,                -- bit 9
    listOfRelatedObjects        [10] IMPLICIT SEQUENCE OF Gn_AccessSpecification OPTIONAL, -- bit 10
    contentsSize                [11] IMPLICIT Lr_Size OPTIONAL,                 -- bit 11
    loadImageName               [12] IMPLICIT Gn_Name OPTIONAL,                 -- bit 12
    fixedLengthSegment          [13] IMPLICIT Lr_FixedLengthSegment OPTIONAL,   -- bit 13
    fixedSegmentLength          [14] IMPLICIT Lr_Size OPTIONAL,                 -- bit 14
    intersegmentRequestTimeout  [15] IMPLICIT Ar_ServiceTimeOut OPTIONAL        -- bit 15
    loadRegionSharable          [16] IMPLICIT Boolean OPTIONAL                  -- bit 16
}

```

4.1.4.15.9.2.10 Mn_NotifierAttrValues

```

Mn_NotifierAttrValues ::= SEQUENCE {
    optionalParametersMap      [0] IMPLICIT Gn_OptionalParametersMap16 OPTIONAL,
    commonAttrValues           [1] IMPLICIT Mn_CommonAttrValues OPTIONAL,      -- bit 1
    listOfSupportedAttributes   [2] IMPLICIT Mn_NotifierAttrMap OPTIONAL,       -- bit 2
    listOfOperationalServices   [3] IMPLICIT Mn_NotifierServiceMap OPTIONAL,    -- bit 3
    enabled                    [4] IMPLICIT Gn_Enabled OPTIONAL,                -- bit 4
    notificationArepClass       [5] IMPLICIT Gn_NumericID OPTIONAL,             -- bit 5
    notificationArep            [6] IMPLICIT Gn_NumericID OPTIONAL,             -- bit 6
    notificationType            [7] IMPLICIT Ev_NotificationType OPTIONAL,       -- bit 7
    containedMessageType        [8] IMPLICIT Ev_ContainedMessageType OPTIONAL,   -- bit 8
    containedEventData          [9] IMPLICIT Ev_ContainedEventData OPTIONAL,     -- bit 9
    lastNotificationSequenceNumber [10] IMPLICIT Ev_SequenceNumber OPTIONAL,    -- bit 10
    listOfEvents                [11] IMPLICIT Gn_NumericID OPTIONAL              -- bit 11
}

```

4.1.4.15.9.2.11 Mn_VariableAttrValues

```

Mn_VariableAttrValues ::= SEQUENCE {
    optionalParametersMap      [0] IMPLICIT Gn_OptionalParametersMap16 OPTIONAL,
    commonAttrValues           [1] IMPLICIT Mn_CommonAttrValues OPTIONAL,      -- bit 1
    listOfSupportedAttributes   [2] IMPLICIT Mn_VariableAttrMap OPTIONAL,       -- bit 2
    listOfOperationalServices   [5] IMPLICIT Mn_VariableServiceMap OPTIONAL,    -- bit 3
    symbolicAddress             [4] IMPLICIT Gn_SymbolicAddress OPTIONAL,       -- bit 4
    dataType                   [5] IMPLICIT Gn_NumericID OPTIONAL,              -- bit 5
    length                      [6] IMPLICIT Unsigned8 OPTIONAL,                -- bit 6
    variableLengthConveyance     [7] IMPLICIT Vr_variableLengthConveyance OPTIONAL, -- bit 7
    numberOfArrayElements       [8] IMPLICIT Unsigned8 OPTIONAL,                -- bit 8
    accessPrivilege             [9] IMPLICIT Vr_AccessPrivilege OPTIONAL,       -- bit 9
    localAddress                [10] IMPLICIT Gn_NumericAddress OPTIONAL        -- bit 10
    fullyNestedTypeDescription  [11] IMPLICIT Gn_FullyNestedTypeDescription OPTIONAL -- bit 11
}

```

4.1.4.15.9.2.12 Mn_VariableListAttrValues

```

Mn_VariableListAttrValues ::= SEQUENCE {
    optionalParametersMap      [0] IMPLICIT Gn_OptionalParametersMap8,
    commonAttrValues           [1] IMPLICIT Mn_CommonAttrValues OPTIONAL,      -- bit 1
    listOfSupportedAttributes   [2] IMPLICIT Mn_VariableListAttrMap OPTIONAL,   -- bit 2
    listOfOperationalServices   [3] IMPLICIT Mn_VariableListServiceMap OPTIONAL, -- bit 3
    numberOfEntries             [4] IMPLICIT Unsigned8 OPTIONAL,                -- bit 4
    listOfVariables             [5] IMPLICIT SEQUENCE OF Gn_AccessSpecification OPTIONAL, -- bit 5
    deletable                   [6] IMPLICIT Gn_Deletable OPTIONAL,             -- bit 6
    accessPrivilege             [7] IMPLICIT Vr_AccessPrivilege OPTIONAL        -- bit 7
}

```

4.1.4.15.9.3 Mn_ApoClassAndObjects

```

Mn_ApoClassAndObjects ::= SEQUENCE {
    numericIdentifier          [0] IMPLICIT Gn_NumericID,
    romRamFlag                [1] IMPLICIT Gn_RomRamFlag,
    maxNameLength              [2] IMPLICIT Unsigned8,
    accessProtectionSupported [3] IMPLICIT Gn_AccessProtection,
    versionOfList              [4] IMPLICIT Mn_Version,
    localReference             [5] IMPLICIT Unsigned32,
    optionalParametersMap      [6] IMPLICIT Gn_OptionalParametersMap16 OPTIONAL,
    numericIdentifier          [7] IMPLICIT Gn_NumericID OPTIONAL,      -- bit 1
    numberOfObjects            [8] IMPLICIT Unsigned8 OPTIONAL,        -- bit 2
    localReference             [9] IMPLICIT Unsigned32 OPTIONAL,        -- bit 3
    numericIdentifier          [10] IMPLICIT Gn_NumericID OPTIONAL,     -- bit 4
    numberOfObjects            [11] IMPLICIT Unsigned8 OPTIONAL,        -- bit 5
    localReference             [12] IMPLICIT Unsigned32 OPTIONAL,       -- bit 6
    numericIdentifier          [13] IMPLICIT Gn_NumericID OPTIONAL,     -- bit 7
    numberOfObjects            [14] IMPLICIT Unsigned8 OPTIONAL,        -- bit 8
    localReference             [15] IMPLICIT Unsigned32 OPTIONAL,       -- bit 9
    numericIdentifier          [16] IMPLICIT Gn_NumericID OPTIONAL,     -- bit 10
    numberOfObjects            [17] IMPLICIT Unsigned8 OPTIONAL,        -- bit 11
    localReference             [18] IMPLICIT Unsigned32 OPTIONAL        -- bit 12
}

```

4.1.4.15.9.4 Mn_ArEndpointInfo

```

Mn_ArEndpointInfo ::= SEQUENCE {
    configuredMaxPduSizeSending          [0] IMPLICIT Unsigned8,
    configuredMaxPduSizeReceiving        [1] IMPLICIT Unsigned8,
    configuredMaxOutstandingServicesRequesting [2] IMPLICIT Unsigned8,
    configuredMaxOutstandingServicesResponding [3] IMPLICIT Unsigned8,
    listOfSupportedServices               [4] IMPLICIT Mn_ServiceMap,
    configuredMaxDataStructureNestingLevel [5] IMPLICIT Unsigned8
}

```

4.1.4.15.9.5 Mn_Deletable

Mn_Deletable ::= Boolean

-- The value of TRUE means the object is remotely deletable.
 -- The value of FALSE means the object is not remotely deletable.

4.1.4.15.9.6 Mn_InuseArEndpointInfo

```

Mn_InuseArEndpointInfo ::= SEQUENCE {
    negotiatedMaxPduSizeSending          [0] IMPLICIT Unsigned8,
    negotiatedMaxPduSizeReceiving        [1] IMPLICIT Unsigned8,
    negotiatedMaxOutstandingServicesRequesting [2] IMPLICIT Unsigned8,
    negotiatedMaxOutstandingServicesResponding [3] IMPLICIT Unsigned8,
    outstandingServicesRequestingCounter [4] IMPLICIT Unsigned8,
    outstandingServicesRespondingCounter [5] IMPLICIT Unsigned8,
    negotiatedMaxDataStructureNestingLevel [6] IMPLICIT Unsigned8
}

```

4.1.4.15.9.7 Mn_ManufacturerIdentifier

```

Mn_ManufacturerIdentifier ::= SEQUENCE {
    vendorName          [0] IMPLICIT OctetString,
    modelIdentifier      [1] IMPLICIT OctetString,
    vendorRevision       [2] IMPLICIT OctetString,
    serialNumber         [3] IMPLICIT OctetString
}

```

4.1.4.15.9.8 Mn_NetworkAccessState

```

Mn_NetworkAccessState ::= SEQUENCE {
    serviceAccessStatus [0] IMPLICIT Unsigned8 {
        readyForCommunication (1),
        limitedNumberOfServices (2),
        loadingNonInteracting (3),
        loadingInteracting (4)
    },
    operationalStatus [1] IMPLICIT Unsigned8 {
        operational (1),
        partiallyOperational (2),
        notOperational (3),
        needsMaintenance (4)
    }
}

```

4.1.4.15.9.9 Mn_PduSupportedMap

```

Mn_PduSupportedMap ::= BitString {
    getAttributes2PDU          (1),    --All mandatory
    unsolicitedStatusPDU      (2),
    setAttributes2PDU          (3),
    clientPulledDownloadPDU    (4),
    clientPullUploadPDU        (5),
    serverPullDownloadPDU      (6),
    serverPulledUploadPDU      (7),
    createFunctionInvocationPDU (8),
    -- deleteFunctionInvocationPDU (8),
    startPDU                   (9),
    -- stopPDU                   (9),
    -- resumePDU                 (9),
    -- resetPDU                  (9),
    kill PDU                   (10),
    read2PDU                   (11),
    writePDU                   (12),
    physicalReadPDU            (15),
    physicalWritePDU           (16),
    informationReportPDU       (17),
    createVariableListPDU      (19),
    -- deleteVariableListPDU     (19),
    eventNotificationPDU       (20),
    confirmedAcknowledgeEventPDU (22),
    enableEventPDU             (23),
    clientPushDownloadPDU      (25),
    getAttributesPDU           (26),    -- Selective
    getAttributesListPDU       (27),
    subscribePDU               (28),
    readListPDU                (29),
    writeListPDU               (30),
    informationReportListPDU    (31),
    exchangePDU                (32),
    unconfirmedAcknowledgeEventsPDU (33),
    getEventSummaryPDU         (34),
    getEventSummaryListPDU     (35),
    queryEventSummaryPDU       (36),
    notificationRecoveryPDU     (37),
    discardPDU                 (38),
    actionInvokePDU            (39),
    actionReturnPDU            (40),
    setAttributes1PDU          (41),
    read1PDU                   (42),
    getAttributes1PDU          (43),
    concludePDU                (44)
}

```

4.1.4.15.9.10 Mn_SelectedAttributes

```

Mn_SelectedAttributes ::= Unsigned8 {
    allMandatory          (0),    -- All mandatory attributes
    userSpecified         (128),  -- Specified by the AttributeMap
    minimumMandatory      (255)   -- Minimum mandatory attributes
}

```

4.1.4.15.9.11 Mn_ServiceMap

4.1.4.15.9.11.1 Mn_ActionServiceMap

```

Mn_ActionServiceMap ::= BitString8 {
    actionInvoke          (1),
    actionReturn          (2)
}

```

4.1.4.15.9.11.2 Mn_ApServiceMap

```

Mn_ApServiceMap ::= BitString8 {
    subscribe                (1),
    identify                  (2),
    getStatus                 (3),
    statusNotification        (4),
    initiate                  (5),
    terminate                 (6),
    conclude                  (7)
}

```

4.1.4.15.9.11.3 Mn_EventServiceMap

```

Mn_EventServiceMap ::= BitString8 {
    unconfirmedAcknowledgeEvents (1),
    confirmedAcknowledgeEvents  (2),
    enableEvents                 (3),
    getEventSummary              (4),
    getEventSummaryList          (5),
    queryEventSummary            (6)
}

```

4.1.4.15.9.11.4 Mn_EventListServiceMap

```

Mn_EventListServiceMap ::= BitString8 {
    getEventSummary            (1),
    queryEventSummary           (2)
}

```

4.1.4.15.9.11.5 Mn_FunctionInvocationServiceMap

```

Mn_FunctionInvocationServiceMap ::= BitString8 {
    start                      (1),
    stop                       (2),
    resume                     (3),
    reset                      (4),
    kill                       (5)
}

```

4.1.4.15.9.11.6 Mn_LoadRegionServiceMap

```

Mn_LoadRegionServiceMap ::= BitString8 {
    initiateLoad               (1),
    pushSegment                 (2),
    pullSegment                 (3),
    terminateLoad               (4),
    discard                     (5)
}

```

4.1.4.15.9.11.7 Mn_NotifierServiceMap

```

Mn_NotifierServiceMap ::= BitString8 {
    EventNotification           (1),
    notificationRecovery         (2)
}

```

4.1.4.15.9.11.8 Mn_VariableListServiceMap

```

Mn_VariableListServiceMap ::= BitString8 {
    read                       (1),
    write                      (2),
    readList                   (3),
    writeList                   (4),
    exchange                   (5),
    informationReport           (6),
    informationReportList       (7)
}

```

4.1.4.15.9.11.9 Mn_VariableServiceMap

```
Mn_VariableListServiceMap ::= BitString8 {
    read                (1),
    write               (2),
    readList            (3),
    writeList           (4),
    exchange            (5),
    informationReport    (6),
    informationReportList (7)
}
```

4.1.4.15.10 Variable ASE types

4.1.4.15.10.1 Vr_AccessPrivilege

```
Vr_AccessPrivilege ::= BitString {
    rightToReadPassword      (24),
    rightToWritePassword     (23),
    rightToDeletePassword    (22),
    rightToReadAccessGroup   (20),
    rightToWriteAccessGroup  (19),
    rightToDeleteAccessGroup (18),
    rightToReadAllPartners   (32),
    rightToWriteAllPartners  (31),
    rightToDeleteAllPartners (30),
    password_Bit1            (8),      -- The Password (Unsigned8) is encoded as a bit string.
    password_Bit2            (7),
    password_Bit3            (6),
    password_Bit4            (5),
    password_Bit5            (4),
    password_Bit6            (3),
    password_Bit7            (2),
    password_Bit8            (1),
    access_Groups-1          (16),
    access_Groups-2          (15),
    access_Groups-3          (14),
    access_Groups-4          (13),
    access_Groups-5          (12),
    access_Groups-6          (11),
    access_Groups-7          (10),
    access_Groups-8          (9)
}
```

4.1.4.15.10.2 Vr_VariableLengthConveyance

```
Vr_VariableLengthConveyance ::= Boolean -- True means only the octets that are in use in a string are
conveyed.
-- False means all the octets in a string are conveyed.
```

4.1.4.15.10.3 Vr_WriteResult

```
Vr_WriteResult ::= CHOICE {
    success                [0] IMPLICIT NULL,
    service                [5] IMPLICIT Er_Service,
    access                 [6] IMPLICIT Er_Access,
    others                 [8] IMPLICIT Er_Other
}
```

4.1.5 Data types

4.1.5.1 Notation for the Boolean type

```
Boolean ::= BOOLEAN
-- TRUE if the value is non-zero.
-- FALSE if the value is zero.
```

4.1.5.2 Notation for the Integer type

```
Integer ::= INTEGER -- any integer
Integer8 ::= INTEGER (-128..+127) -- range  $-2^7 \leq i \leq 2^7-1$ 
Integer16 ::= INTEGER (-32768..+32767) -- range  $-2^{15} \leq i \leq 2^{15}-1$ 
Integer32 ::= INTEGER -- range  $-2^{31} \leq i \leq 2^{31}-1$ 
```

4.1.5.3 Notation for the Unsigned type

Unsigned ::= INTEGER -- any non-negative integer
 Unsigned8 ::= INTEGER (0..255) -- range $0 \leq i \leq 2^8-1$
 Unsigned16 ::= INTEGER (0..65535) -- range $0 \leq i \leq 2^{16}-1$
 Unsigned32 ::= INTEGER -- range $0 \leq i \leq 2^{32}-1$

4.1.5.4 Notation for the Floating Point type

Floating32 ::= BIT STRING SIZE (4) -- IEC-60559Single precision
 Floating64 ::= BIT STRING SIZE (8) -- IEC-60559Double precision

4.1.5.5 Notation for the BitString type

BitString ::= BIT STRING -- For generic use
 BitString8 ::= BIT STRING SIZE (8) -- Fixed eight bits bitstring
 BitString16 ::= BIT STRING SIZE (16) -- Fixed 16 bits bitstring
 BitString32 ::= BIT STRING SIZE (32) -- Fixed 32 two bits bitstring

4.1.5.6 Notation for the OctetString type

OctetString ::= OCTET STRING -- For generic use
 OctetString2 ::= OCTET STRING SIZE (2) -- Fixed two-octet octet string
 OctetString4 ::= OCTET STRING SIZE (4) -- Fixed four-octet octet string
 OctetString6 ::= OCTET STRING SIZE (6) -- Fixed six-octet octet string
 OctetString7 ::= OCTET STRING SIZE (7) -- Fixed seven-octet octet string
 OctetString8 ::= OCTET STRING SIZE (8) -- Fixed eight-octet octet string
 OctetString16 ::= OCTET STRING SIZE (16) -- Fixed 16 octet octet string

4.1.5.7 Notation for VisibleString type

VisibleString2 ::= VisibleString SIZE (2) -- Fixed two-octet visible string
 VisibleString4 ::= VisibleString SIZE (4) -- Fixed four-octet visible string
 VisibleString8 ::= VisibleString SIZE (8) -- Fixed eight-octet visible string
 VisibleString16 ::= VisibleString SIZE (16) -- Fixed 16 octet visible string

4.1.5.8 Notation for the UNICODEString type

UNICODEString ::= UNICODEString -- 16-bit character code set defined in ISO 10646.

4.1.5.9 Notation for the FieldbusTime type

FieldbusTime -- Format and granularity are defined in IEC 61158-3 and IEC 61158-4.

4.1.5.10 Notation for the Universal Time type

UniversalTime ::= VisibleString SIZE (12) -- YYMMDDhhmmss

4.1.5.11 Notation for Binary Time type

BinaryTime0 ::= BIT STRING SIZE (16) -- 10 μ s resolution
 BinaryTime1 ::= BIT STRING SIZE (16) -- 0.1 ms resolution
 BinaryTime2 ::= BIT STRING SIZE (16) -- 1 ms resolution
 BinaryTime3 ::= BIT STRING SIZE (16) -- 10 ms resolution
 BinaryTime4 ::= BIT STRING SIZE (32) -- 10 μ s resolution
 BinaryTime5 ::= BIT STRING SIZE (32) -- 0.1 ms resolution
 BinaryTime6 ::= BIT STRING SIZE (32) -- 1 ms resolution
 BinaryTime7 ::= BIT STRING SIZE (32) -- 10 ms resolution
 BinaryTime8 ::= BIT STRING SIZE (48) -- 10 μ s resolution
 BinaryTime9 ::= BIT STRING SIZE (48) -- 0.1 ms resolution

4.1.5.12 Notation for BCD type

BCD ::= Unsigned8 (0..9) -- Lower four bits are used to express one BCD value.

4.1.5.13 Notation for Compact Boolean Array type

CompactBooleanArray ::= BitString -- Each zero bit representing Boolean value FALSE.
 -- Each one bit representing Boolean value TRUE.
 -- Unused bits, if any, shall be placed in bits 7-1 of the last octet.

4.1.5.14 Notation for Compact BCD Array type

CompactBCDArray ::= OctetString -- One BCD value is represented by four bits, an unused
 -- nibble, if any, shall be placed in bits 4-1 of the last octet,
 -- and shall be set to 1111_F.

4.1.5.15 Notation for BinaryDate type

BinaryDate ::= OctetString7

4.1.5.16 Notation for TimeOfDay type

TimeOfDay ::= OctetString6

4.1.5.17 Notation for TimeDifference type

TimeDifference ::= OctetString6

4.1.5.18 Notation for TimeValue type

TimeValue ::= OctetString8

4.1.5.19 Notation for DL—Time-offset type

DL-Time-offsetType ::= OctetString

4.1.6 Reason codes

4.1.6.1 General

Table 13 specifies the values of the Reason parameter of the Data Link Layer DL-Disconnect service that are supplied by the FAL.

Table 13 – Reason codes

FAL reason parameter	Value
Invalid FAL-PDU	31 (Hex)
Remote Address Mismatch	32 (Hex)
Multiple Initiators	33 (Hex)
Invalid DL-Event	34 (Hex)
AREP Busy	35 (Hex)
AREP Not Found	36 (Hex)
Invalid AREP Type	37 (Hex)
DLCEP Not Found	38 (Hex)
Invalid DI PDU	39 (Hex)
Not allowed AREP primitive in connection establishment	3A (Hex)
Number of parallel services exceeded	3B (Hex)
Not allowed service as server	3C (Hex)
Not allowed service as client	3D (Hex)
Context Check Negative	3E (Hex)
Invalid FAL-PDU	3F (Hex)
Invalid Event for Role	40 (Hex)
UCA_AckPDU received and UCC=0	41 (Hex)
RCTimer expired	42 (Hex)
RSTimer expired	43 (Hex)
MCTimer expired	44 (Hex)
MSTimer expired	45 (Hex)
Execution error in cyclic data transfer	46 (Hex)

4.2 Transfer syntaxes

4.2.1 Overview and FAL header

All the FAL PDUs shall have the common PDU-header called *FalArHeader*. The *FalArHeader* identifies abstract syntax, transfer syntax, and each of the PDUs. Table 14 defines how this header shall be used.

NOTE The structure of the *FalArHeader* is defined in the main text of this standard.

FAL Protocol Specifier (Bit 8) is always zero (0) for the protocol defined by this standard.

Table 14 – FAL header

Bit position of the <i>FalArHeader</i>			Abstract syntax	Encoding rule	PDU type	Revision
8	7 6 5 4 3	2 1				
0	0 0 0 0 0	0 0	FAL AS2	TER	Establish-RequestPDU	Revision1
0	0 0 0 0 0	0 1	FAL AS2	TER	Establish-ResponsePDU	Revision1
0	0 0 0 0 0	1 0	FAL AS2	TER	Establish-ErrorPDU	Revision1
0	0 0 0 0 0	1 1	FAL AS2	TER	AbortPDU	Revision1
0	0 0 0 0 1	0 0	FAL AS2	TER	ConfirmedSend-RequestPDU	Revision1
0	0 0 0 0 1	0 1	FAL AS2	TER	ConfirmedSend-ResponsePDU	Revision1
0	0 0 0 0 1	1 0	FAL AS2	TER	UnconfirmedSendPDU	Revision1
0	1 0 0 0 0	0 0	FAL AS1	CER	Establish-CommandPDU	Revision1
0	1 0 0 0 0	0 1	FAL AS1	CER	Establish-AffirmativePDU	Revision1
0	1 0 0 0 0	1 0	FAL AS1	CER	Establish-NegativePDU	Revision1
0	1 0 0 0 0	1 1	FAL AS1	CER	AbortPDU	Revision1
0	1 0 0 0 1	0 0	FAL AS1	CER	ConfirmedSend-CommandPDU	Revision1
0	1 0 0 0 1	0 1	FAL AS1	CER	ConfirmedSend-AffirmativePDU	Revision1
0	1 0 0 0 1	1 0	FAL AS1	CER	ConfirmedSend-NegativePDU	Revision1
0	1 0 0 0 1	1 1	FAL AS1	CER	UnconfirmedSend-CommandPDU	Revision1
0	1 0 0 1 0	0 0	FAL AS1	CER	UnconfirmedAcknowledgedSend-CommandPDU	Revision1
0	1 0 0 1 0	0 1	FAL AS1	CER	UnconfirmedAcknowledgedSend-AffirmativePDU	Revision1
0	1 0 0 1 0	1 0	FAL AS1	CER	IdleSend-CommandPDU	Revision1
0	1 0 0 1 0	1 1	FAL AS1	CER	AR-XON-OFF-CommandPDU	Revision1
0	0 1 0 S 0	0 0	FAL AS1	MER	Establish-CommandPDU	Revision1
0	0 1 0 S 0	0 1	FAL AS1	MER	Establish-AffirmativePDU	Revision1
0	0 1 0 S 0	1 0	FAL AS1	MER	Establish-NegativePDU	Revision1
0	0 1 0 S 0	1 1	FAL AS1	MER	AbortPDU	Revision1
0	0 1 0 S 1	0 0	FAL AS1	MER	ConfirmedSend-CommandPDU	Revision1
0	0 1 0 S 1	0 1	FAL AS1	MER	ConfirmedSend-AffirmativePDU	Revision1
0	0 1 0 S 1	1 0	FAL AS1	MER	ConfirmedSend-NegativePDU	Revision1
0	0 1 0 S 1	1 1	FAL AS1	MER	UnconfirmedSend-CommandPDU	Revision1
0	1 1 0 S 0	0 0	FAL AS1	BER	BufferedDataTransferPDU	Revision1
0	0 0 1 0 0	0 0	FAL AS2	TER	UnconfirmedAcknowledgedSend-CommandPDU	Revision1
0	0 0 1 0 0	0 1	FAL AS2	TER	UnconfirmedAcknowledgedSend-AffirmativePDU	Revision1
0	0 0 1 0 0	1 0	FAL AS2	TER	IdleSend-CommandPDU	Revision1
0	0 0 1 0 0	1 1	FAL AS2	TER	AR-XON-OFF-CommandPDU	Revision1
0	0 0 1 0 1	0 0	FAL AS2	TER	Establish-Request2PDU	Revision1
1	X X X X X	X X			Reserved	

- All other code points are reserved for additional protocols and future revisions.
- S-bit (bit4) is used for segmentation purposes only. It indicates to the receiver of an FAL PDU, if this PDU is the last segment (S = 0) or not (S = 1).

4.2.2 Traditional Encoding Rule (TER)

4.2.2.1 General

The Traditional Encoding Rule (TER) is a preferable encoding rule that is compatible with existing standards.

4.2.2.2 Overview of encoding

The FAL-PDUs encoded with the TER shall have a uniform format. The FAL-PDUs shall consist of two major parts, the “APDU Header” part and the “APDU Body” part as shown in Figure 7.

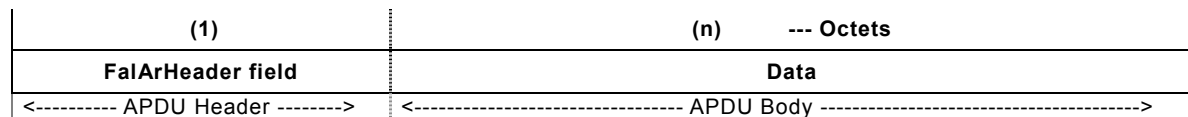


Figure 7 – APDU overview

4.2.2.3 APDU header encoding

The APDU Header part is always present in all APDUs which conform to this standard. It consists of one field: the FalArHeader Field. Refer to 4.2.1 for the encoding rule of the FalArHeader field.

4.2.2.4 APDU body encoding

4.2.2.4.1 General

The FAL-PDUs are encoded with the TER in the following manner:

```

FAL-PDU ::= {Components}
Components ::= UserData ||
               IdentificationInformation ContentsOctets
IdentificationInformation ::= P/C Flag Tag Length
ContentsOctets ::= OCTETSTRING
    
```

4.2.2.4.2 Structure of the identification information

The Identification Information consists of the P/C flag, the Tag field, and the Length field as shown in Figure 8.

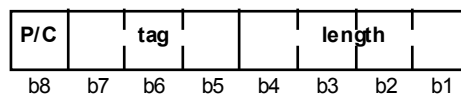


Figure 8 – Identification information (format 1)

The P/C flag indicates either the ContentsOctets is a simple component (primitive types, such as Integer8), or a structured component (constructed, such as SEQUENCE, SEQUENCE OF types).

P/C Flag =0 means the ContentsOctets is a simple component.
 P/C Flag =1 means the ContentsOctets is a structured component.

The Tag field identifies the semantics of the ContentsOctets.

The Length field indicates the length of the ContentsOctets in octets if it is a primitive type and the number of contained components if it is a structured type.

The three components of the Identification Information are allotted in one octet as follows:

P/C flag	bit8
Tag field	bit7 - bit5, where bit7 is the msb and bit5 is the lsb.
Length field	bit3 - bit1, where bit4 is the msb and bit1 is the lsb.

If the number of the Tag is greater than six, or the number of the Length is greater than 14, the extension octet for each of the fields shall be used.

If the extension octet for the Tag is used, the Tag field in the Identification Information shall be set to seven. The Tag Extension octet that immediately follows the Identification Information shall contain the number of the Tag, with bit8 of the extension octet as the msb and bit1 the lsb as shown in Figure 9 through Figure 11.



Figure 9 – Identification information (format 2)

If the extension octet for the Length is used, the Length field in the Identification Information shall be set to 15. The Length Extension octet that immediately follows the Identification Information shall contain the number of the Length, with bit8 of the extension octet as the msb and bit1 the lsb as shown below:



Figure 10 – Identification information (format 3)

If both the Tag and the Length fields are extended, the Tag Extension octet follows the Identification Information and is followed by the Length Extension octet as shown below:

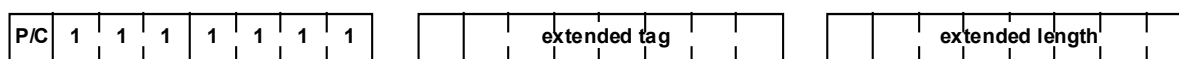


Figure 11 – Identification information (format 4)

If the abstract syntax does not contain a tag for a data item, then no Identification Information is encoded. The semantics and length of this data item are implicitly known by the abstract syntax.

4.2.2.5 Encoding of simple variables

4.2.2.5.1 Encoding of a Boolean value

- The encoding of a Boolean value shall be primitive. The ContentsOctets shall consist of a single octet.
- If the Boolean value is FALSE, the ContentsOctets shall be 0 (zero). If the Boolean value is TRUE, the ContentsOctets shall be FFh.

4.2.2.5.2 Encoding of an Integer value

- The encoding of a fixed-length Integer value of Integer8, Integer16, and Integer32 types shall be primitive, and the ContentsOctets shall consist of exactly one, two, or four octets, respectively.
- The ContentsOctets shall be a two's complement binary number equal to the integer value, and consist of bits 8 to 1 of the first octet, followed by bits 8 to 1 of the second octet, followed by bits 8 to 1 of each octet in turn up to and including the last octet of the ContentsOctets.

NOTE The value of a two's complement binary number is derived by numbering the bits in the ContentsOctets, starting with bit 1 of the last octet as bit zero and ending the numbering with bit 8 of the first octet. Each bit is assigned a numerical value of 2^{N-1} , where N is its position in the above numbering sequence. The value of the two's complement binary number is obtained by adding the numerical values assigned to each bit for those bits which are set to one, excluding bit 8 of the first octet, and then reducing this value by the numerical value assigned to bit 8 of the first octet if that bit is set to one.

4.2.2.5.3 Encoding of an Unsigned value

- a) The encoding of a fixed-length Unsigned value of Unsigned8, Unsigned16, and Unsigned32 types shall be primitive, and the ContentsOctets shall consist of exactly one, two, or four octets, respectively.
- b) The ContentsOctets shall be a binary number equal to the Unsigned value, and consist of bits 8 to 1 of the first octet, followed by bits 8 to 1 of the second octet, followed by bits 8 to 1 of each octet in turn up to and including the last octet of the ContentsOctets.

NOTE The value of a binary number is derived by numbering the bits in the ContentsOctets, starting with bit 1 of the last octet as bit zero and ending the numbering with bit 8 of the first octet. Each bit is assigned a numerical value of 2^{N-1} , where N is its position in the above numbering sequence. The value of the binary number is obtained by adding the numerical values assigned to each bit for those bits which are set to one.

4.2.2.5.4 Encoding of a Floating-Point value

- a) The encoding of a Floating32 or Floating64 value shall be primitive, and the ContentsOctets shall consist of exactly four or eight octets, respectively.
- b) The ContentsOctets shall contain floating-point values defined in conformance with IEC-60559 Standard. The sign is encoded in bit 8 of the first octet. It is followed by the exponent starting from bit 7 of the first octet, and then the mantissa starting from bit 7 of the second octet for Floating32 and from bit 4 of the second octet for Floating64.

4.2.2.5.5 Encoding of a Visible String value

- a) The encoding of a variable length VisibleString value shall be primitive.
- b) The Length field shall indicate as a binary number the number of octets in the VisibleString value.
If the VisibleString is of zero length, there shall be no ContentsOctets, and the Length shall be zero.
- c) If the VisibleString is not of zero length, the ContentsOctets shall be equal in value to the octets in the data value, in the order they appear in the data value, and with the most significant bit of an octet of the data value aligned with the most significant bit of an octet of the ContentsOctets.

4.2.2.5.6 Encoding of an Octet String value

- a) The encoding of a variable length OctetString value shall be primitive.
- b) The Length field shall indicate as a binary number the number of octets in the OctetString value.
If the OctetString is of zero length, there shall be no ContentsOctets, and the Length shall be zero.
- c) If the OctetString is not of zero length, the ContentsOctets shall be equal in value to the octets in the data value, in the order they appear in the data value, and with the most significant bit of an octet of the data value aligned with the most significant bit of an octet of the ContentsOctets.

4.2.2.5.7 Encoding of a BinaryDate value

- a) The encoding of a BinaryDate value shall be primitive.
- b) The Length field shall indicate as a binary number the number of octets in the BinaryDate value.
- c) The ContentsOctets shall be equal in value to the octets in the data value, as shown in Figure 12.

Bit-No	msb				lsb				
	8	7	6	5	4	3	2	1	
Octet									
1	2 ¹⁵	2 ¹⁴	2 ¹³	2 ¹²	2 ¹¹	2 ¹⁰	2 ⁹	2 ⁸	0...59 999 ms
2	2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	
3	RSV	RSV	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	0...59 min
4	SU	RSV	RSV	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	0...23 hours
5	day of week			day of month				2 ⁰	1...7 day of week
	2 ²	2 ¹	2 ⁰	2 ⁴	2 ³	2 ²	2 ¹		1...31 day of month
6	RSV	RSV	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	1...12 months
7	RSV	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	0 ... 50 years 2000 to 2050 51 ... 99 years 1951 to 1999

Figure 12 – Coding of the data type BinaryDate

NOTE To avoid the Y2K problem the values of the year are from 0 to 50 for the years 2000 to 2050 and from 51 to 99 for the years 1951 to 1999. This is according to the British Standards Institution (BSi) DISC PD2000-1:1998 "A Definition of Year 2000 Conformance Requirements" Rule 3 (b).

4.2.2.5.8 Encoding of a Time Of Day value

- a) The encoding of a Time Of Day value shall be primitive.
- b) The Length field shall indicate as a binary number the number of octets in the Time Of Day value.
- c) The ContentsOctets shall be equal in value to the octets in the data value, as shown in Figure 13.

bits	8	7	6	5	4	3	2	1	
octets									
1	0	0	0	0	2 ²⁷	2 ²⁶	2 ²⁵	2 ²⁴	number of milliseconds since midnight
2	2 ²³	2 ²²	2 ²¹	2 ²⁰	2 ¹⁹	2 ¹⁸	2 ¹⁷	2 ¹⁶	
3	2 ¹⁵	2 ¹⁴	2 ¹³	2 ¹²	2 ¹¹	2 ¹⁰	2 ⁹	2 ⁸	
4	2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	
5	2 ¹⁵	2 ¹⁴	2 ¹³	2 ¹²	2 ¹¹	2 ¹⁰	2 ⁹	2 ⁸	number of days since 01.01.84 (optional)
6	2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	
MSB									

Figure 13 – Encoding of Time of Day value

4.2.2.5.9 Encoding of a Time Difference value

- a) The encoding of a Time Difference value shall be primitive.
- b) The Length field shall indicate as a binary number the number of octets in the Time Difference value.
- c) The ContentsOctets shall be equal in value to the octets in the data value, as shown in Figure 14.

bits	8	7	6	5	4	3	2	1	
octets									number of milliseconds
1	0	0	0	0	2^{27}	2^{26}	2^{25}	2^{24}	
2	2^{23}	2^{22}	2^{21}	2^{20}	2^{19}	2^{18}	2^{17}	2^{16}	
3	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	
4	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	number of days (optional)
5	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	
6	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
MSB									

Figure 14 – Encoding of Time Difference value

4.2.2.5.10 Encoding of a Bit String value

- The encoding of a variable length BitString value shall be primitive.
- The Length field shall indicate the number of octets in the BitString encoding, which is the number of octets used to encode a BitString.

If the BitString is empty (no BitString value at all), there shall be no ContentsOctets, and the Length field shall be zero.

- If the BitString is not empty, the ContentsOctets shall encode the BitString value as follows:

The value of the BitString, commencing with the first bit and proceeding to the trailing bit, shall be placed in bits 8 to 1 of the first octet, followed by bits 8 to 1 of the second octet, followed by bits 8 to 1 of each octet up to and including the last octet of the ContentsOctets.

4.2.2.5.11 Encoding of a Time value

- The encoding of a Time value shall be primitive.
- The Length field shall indicate as a binary number the number of octets in the Time value.
- The ContentsOctets shall be equal in value to the octets in the data value, as shown in Figure 15.

bits	8	7	6	5	4	3	2	1	
octets	SN	2 ⁶²	2 ⁶¹	2 ⁶⁰	2 ⁵⁹	2 ⁵⁸	2 ⁵⁷	2 ⁵⁶	signed integer of 8 octets length of 1/32ms unit
1									
2	2 ⁵⁵	2 ⁵⁴	2 ⁵³	2 ⁵²	2 ⁵¹	2 ⁵⁰	2 ⁴⁹	2 ⁴⁸	
3	2 ⁴⁷	2 ⁴⁶	2 ⁴⁵	2 ⁴⁴	2 ⁴³	2 ⁴²	2 ⁴¹	2 ⁴⁰	
4	2 ³⁹	2 ³⁸	2 ³⁷	2 ³⁶	2 ³⁵	2 ³⁴	2 ³³	2 ³²	
5	2 ³¹	2 ³⁰	2 ²⁹	2 ²⁸	2 ²⁷	2 ²⁶	2 ²⁵	2 ²⁴	
6	2 ²³	2 ²²	2 ²¹	2 ²⁰	2 ¹⁹	2 ¹⁸	2 ¹⁷	2 ¹⁶	
7	2 ¹⁵	2 ¹⁴	2 ¹³	2 ¹²	2 ¹¹	2 ¹⁰	2 ⁹	2 ⁸	
8	2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	
MSB									

Figure 15 – Encoding Time value

4.2.2.5.12 Encoding of a Null value

- The encoding of a NULL value shall be primitive.
- The ContentsOctet shall be omitted.

4.2.2.5.13 Encoding of an ANY value

The Data Type ANY contains one or more data elements of the Data Types which are chained together without any gap. The composition is known implicitly.

4.2.2.5.14 Encoding of DL-Time-offset vlaue

This data type is used to represent DL—Time-offset in the required units and precision for Event Time stamp reporting. as defined in 61158-4, Type 6. See Figure 16.

bits	8	7	6	5	4	3	2	1	
octet 1	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	slot clock / 2 unsigned integer
2	2^{55}	2^{54}	2^{53}	2^{52}	2^{51}	2^{50}	2^{49}	2^{48}	
3	2^{47}	2^{46}	2^{45}	2^{44}	2^{43}	2^{42}	2^{41}	2^{40}	
4	2^{39}	2^{38}	2^{37}	2^{36}	2^{35}	2^{34}	2^{33}	2^{32}	
5	2^{31}	2^{30}	2^{29}	2^{28}	2^{27}	2^{26}	2^{25}	2^{24}	
7	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	bus cycle counter unsigned integer of 7 octets length
8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
9	0	0	0	0	0	0	0	2^0	NcycleLO
msb									
Parameter		Range of values		Meaning of parameter		Coding of parameter			
slot clock/2		0...161		bit-times		unsigned integer			
bus cycle counter		$0 \dots 2^{56} - 1$		slot-times		unsigned integer			
NcycleLO		0, 1		0, 1		unsigned integer			
RSV				reserved					

Figure 16 – Encoding of data of data type DL—Time-offset

4.2.2.6 Encoding of structured types

4.2.2.6.1 General

When a structured type is encoded, the P/C flag of the Identification Information shall be set to one. The Length field, whether or not it is extended, shall contain the number of components of the structured type being encoded.

4.2.2.6.2 Encoding of a SEQUENCE value

The SEQUENCE type is comparable to a record. It represents a collection of user data of the same or of different Data Types.

A SEQUENCE type value may contain a simple variable or a further structured variable as its components. If a SEQUENCE type contains another structured type value, it shall be counted as a single component even if it contains several components.

4.2.2.6.3 Encoding of a SEQUENCE OF value

The SEQUENCE OF type represents a succession of components. It is comparable to an array.

A SEQUENCE OF type value may contain one or more simple or constructed variables. If a SEQUENCE OF type contains another structured type value, it shall be counted as a single component even if it contains several components.

The encoding is as for the structure SEQUENCE. For the statement of the number of components the number of repetitions shall be taken into account.

4.2.2.6.4 Encoding of a CHOICE value

A CHOICE type represents a selection from a set of predefined values. The components of a CHOICE construct shall have different tags to allow proper identification. Instead of the CHOICE construct, the actually selected component is encoded. Only one component shall be encoded for a CHOICE.

4.2.2.6.5 Object definition parameter

The mapping of the individual object definitions onto the parameters "List Of Objects and Attributes" (data type Gn_ObjectDefinition) of the GetAttributes2 and SetAttributes2 PDUs is shown in Figure 17.

NOTE The semantics of the Object Definition Parameter of type Gn_ObjectDefinition is application specific and the FAL does not encode or decode it. However, since prior standards defined standard semantics of this parameter, and it is used with the TER, its definition is placed here. Readers are advised that this subclause is not for the FAL, but for the FAL user.

The object class is the identifier of the object and indicates the class to which this object belongs. The other object attributes are object specific and shall be coded as a string of octets. In addition to the object attributes, the object class shall be transmitted in the GetAttributes and SetAttributes services, except List Header, whose numeric ID is zero.

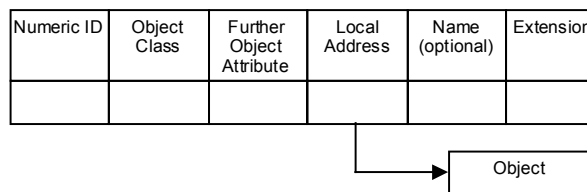


Figure 17 – Structure of an object definition

4.2.2.7 Object definition

4.2.2.7.1 General

```
Object-Definition ::= ListHeader
    || DataTypeList
    || StaticList
    || VariableListDefinition
    || FunctionInvocationDefinition
```

4.2.2.7.2 ListHeader

```
ListHeader ::= ANY {
    Unsigned16,          -- numericId
    Boolean,             -- romRamFlag
    Unsigned8,           -- maxNameLength
    Boolean,             -- accessProtectionSupported
    Integer16,           -- versionOfObjectDefinition
    Unsigned32,          -- localReferenceOfListHeader
    Unsigned16,          -- numberOfEntriesInDataTypeList
    Unsigned32,          -- localReferenceOfDataTypeList
    Unsigned16,          -- firstNumericIdOfStaticList
    Unsigned16,          -- numberOfEntriesInStaticList
    Unsigned32,          -- localReferenceOfStaticList
    Unsigned16,          -- firstNumericIdOfVariableListDefinition
    Unsigned16,          -- numberOfEntriesInVariableListDefinition
    Unsigned32,          -- localReferenceOfVariableListDefinition
    Unsigned16,          -- firstNumericIdOfFunctionInvocationDefinition
    Unsigned16,          -- numberOfEntriesInFunctionInvocationDefinition
    Unsigned32,          -- localReferenceOfFunctionInvocationDefinition
}
```


4.2.2.7.3 DataTypeList

```

DataTypeList ::= DataTypeDefinition
                ||StructuredDataTypeDefinition
DataTypeDefinition ::= ANY {
    Unsigned16,                -- numericId
    Integer8,                  -- objectClass
    Unsigned8,                  -- dataTypeNameLength
    VisibleString               -- dataTypeName
}
StructuredDataTypeDefinition ::= ANY {
    Unsigned16,                -- numericId
    Integer8,                  -- objectClass
    Unsigned8,                  -- numberOfElements
    recordList SEQUENCE OF {
        Unsigned16,            -- numericIdOfDataTypeDefinition
        Unsigned8,              -- dataLength
    }
}

```

4.2.2.7.4 StaticList

```

StaticList ::= VariableDefinition
                || ArrayDefinition
                || StructureDefinition
                || LoadRegionDefinition
                || EventDefinition
VariableDefinition ::= ANY {
    Unsigned16,                -- numericId
    Integer8,                  -- objectClass
    Unsigned16,                -- numericIdOfDataTypeDefinition
    Unsigned8,                  -- dataLength
    Vr_AccessPrivilege,        -- accessPrivilege
    Unsigned32,                 -- localReferenceOfVariable
    VisibleString,              -- variableName
    Unsigned8,                  -- Length of the extension field. If the extension field is not present, the
                                -- value of this field shall be zero.
    OctetString                 -- extension
}
ArrayDefinition ::= ANY {
    Unsigned16,                -- numericId
    Integer8,                  -- objectClass
    Unsigned16,                -- numericIdOfDataTypeDefinition
    Unsigned8,                  -- dataLength
    Unsigned8,                  -- numberOfElements
    Vr_AccessPrivilege,        -- accessPrivilege
    Unsigned32,                 -- localReferenceOfArray
    VisibleString,              -- arrayName
    Unsigned8,                  -- Length of the extension field. If the extension field is not present, the
                                -- value of this field shall be zero.
    OctetString                 -- extension
}
StructureDefinition ::= ANY {
    Unsigned16,                -- numericId
    Integer8,                  -- objectClass
    Unsigned16,                -- numericIdOfDataTypeDefinition
    Vr_AccessPrivilege,        -- accessPrivilege
    VisibleString,              -- structureName
    Unsigned8,                  -- Length of the extension field. If the extension field is not present, the
                                -- value of this field shall be zero.
    OctetString,               -- extension
    SEQUENCE OF Unsigned32      -- localReferenceOfElement
}

```

```

LoadRegionDefinition ::= ANY {
    Unsigned16,          -- numericId
    Integer8,            -- objectClass
    Unsigned16,          -- loadRegionSize
    Vr_AccessPrivilege,  -- accessPrivilege
    Unsigned32,          -- localReferenceOfLoadRegion
    Unsigned8,           -- loadRegionState
    Unsigned8,           -- uploadState
    Integer16,           -- numberOfRelatedObjectsInUse
    VisibleString,       -- loadRegionName
    Unsigned8,           -- Length of the extension field. If the extension field is not present, the
                        -- value of this field shall be zero.
    OctetString          -- extension
}

EventDefinition ::= ANY {
    Unsigned16,          -- numericId
    Integer8,            -- objectClass
    Unsigned16,          -- numericIdOfEventData
    Unsigned8,           -- eventDataLength
    Vr_AccessPrivilege,  -- accessPrivilege
    Boolean,             -- enabled
    VisibleString,       -- eventName
    Unsigned8,           -- Length of the extension field. If the extension field is not present, the
                        -- value of this field shall be zero.
    OctetString          -- extension
}

```

4.2.2.7.5 VariableListDefinition

```

VariableListDefinition ::= ANY {
    Unsigned16,          -- numericId
    Integer8,            -- objectClass
    Unsigned16,          -- numericIdOfElements
    Vr_AccessPrivilege,  -- accessPrivilege
    Boolean,             -- deletable
    SEQUENCE OF Unsigned32,
    VisibleString,       -- variableListName
    Unsigned8,           -- Length of the extension field. If the extension field is not present, the
                        -- value of this field shall be zero.
    OctetString          -- extension
}

```

4.2.2.7.6 FunctionInvocationDefinition

```

FunctionInvocationDefinition ::= ANY {
    Unsigned16,          -- numericId
    Integer8,            -- objectClass
    Unsigned8,           -- numberOfRelatedObjects
    Fi_AccessPrivilege,  -- accessPrivilege
    Boolean,             -- deletable
    Boolean,             -- reusable
    Unsigned8,           -- functionInvocationState
    SEQUENCE OF Unsigned16,
    VisibleString,       -- NumericIdOfLoadRegion
    Unsigned8,           -- functionInvocationName
    Unsigned8,           -- Length of the extension field. If the extension field is not present,
                        -- the value of this field shall be zero.
    OctetString          -- extension
}

```

4.2.3 Compact Encoding Rule (CER)

4.2.3.1 General

The APDUs that conform to this standard shall be encoded in a uniform format as shown in Figure 18. The APDUs consist of two major parts: the "APDU Header" part and the "APDU Body" part.

(1)	(1 or two)	(1)	(n) --- Octets
FalArHeader Field	Type Field	(InvokeID)*	Service Specific Parameters
<----- APDU Header ----->			<----- APDU Body ----->

NOTE The presence of the InvokeID Field depends on the APDU type.

Figure 18 – APDU overview

In order to realize an efficient APDU while maintaining flexible encoding, different encoding rules are used for the APDU Header part and the APDU Body part.

NOTE The IEC Data Link layer service provides a DLSDU parameter which implies the length of the APDU. Hence, the APDU length information is not included in the APDU.

4.2.3.2 APDU header encoding

4.2.3.2.1 General

The APDU Header part is always present in all APDUs which conform to this standard. It consists of three fields: the AR Control Field, the Type Field, and the optional InvokeID Field. They are shown in Figure 18.

4.2.3.2.2 FalArHeader field encoding

Refer to 4.2.1 for the encoding rule of the FalArHeader field.

4.2.3.2.3 Encoding of Type field

- Both the APDU type and the service type are encoded in the Type Field that is always the second of the APDUs.
- All bits of the Type Field are used to encode the service type.
 - The service types are encoded in bits 8 to 1 of the Type Field, with bit 8 the most significant bit and bit 1 the least significant bit. The range of the service type shall be between 0 (zero) and 254, inclusive.
 - The value of 255 is reserved for future extensions to this standard.
 - The service types are specified in the abstract syntax as a positive integer value. This positive integer value shall be encoded in the bits described in b.1), where the most significant bit of a positive integer value shall be aligned to bit 8 and its least significant bit to bit 1.
- Figure 19 illustrates the encoding of the Type Field.

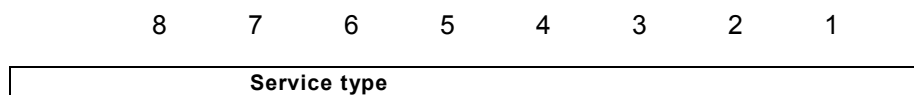


Figure 19 – Type field

4.2.3.2.4 Encoding of InvokeID field

The InvokeID Field shall be present if it is indicated in the abstract syntax. Otherwise, this field shall not be present. If present, the Invoke ID parameter supplied by a service primitive shall be placed in this field.

4.2.3.2.5 APDU body encoding

4.2.3.2.5.1 General

The FAL encoding rules are based on terms and conventions defined in ISO/IEC 8825. The encoding consists of three components in the following order:

- An Identifier Octet

- Length Octet(s)
- The Contents Octets

The Identifier Octet or Length Octet(s), or both, may be removed from the encoding, depending data type of the field of the APDUs being encoded. The rules for removal are defined later.

4.2.3.2.5.2 Identifier octet

- a) The Identifier Octet shall encode the tag defined in the FAL Abstract Syntax and shall consist of one octet.

The Identifier Octet comprises a class and a number. The class encodes the class of the FAL tag, whereas a number encodes a number associated to each tag.

- b) Two classes are defined for the Identifier Octet: The Context-Specific class and the FAL-Specific class.
- 1) The scope of a Context-Specific class is limited to the construction in which it is used. The same tag may be used to represent different types in different productions.
 - 2) The scope of an FAL-Specific identifier is global to the module definitions of the Abstract Syntax definition. The same tag shall designate the same type within the same module.
 - 3) Bit 8 of the Identifier Octets shall indicate its class. If it is set to zero (0), the class shall be Context-Specific. If it is set to one (1), the class shall be FAL-Specific.
- c) The bits 7 to 1 of the Identifier Octet shall represent the value of the tag, with bit 7 being the most significant bit and bit 1 the least significant bit. This gives tags with the possible values of 0 (zero) through 126 inclusive, for each of the classes.
- d) The value of 127 is reserved for future extensions this standard.
- e) The convention for expressing an FAL-specific identifier with a tagged value of two shall be FAL-Specific 2.
- f) When necessary, the abstract syntax shall explicitly specify which class shall be used. The default class shall be "Context Specific."
- g) Figure 20 and Figure 21 depict encoding examples of Identifier Octets.

8	7	6	5	4	3	2	1
0	Tag Number						

Figure 20 – Identifier octet (context-specific)

8	7	6	5	4	3	2	1
1	Tag Number						

Figure 21 – Identifier octet (FAL-specific)

4.2.3.2.5.3 Length octet(s)

- a) The Length Octet(s) shall consist of one octet or three octets.
- 1) If the value of the first Length Octet is other than 255, there shall be no subsequent Length Octets and the first octet shall contain the value for the Length Octet defined later.
 - 2) If the value of the first Length Octet is 255, there shall be two subsequent Length Octets that shall contain the value for the Length Octets defined later. In this case, the length information of the Contents Octets shall be represented by the last two octets of the Length Octets, where the most significant bit of the second of three Length Octet(s) shall be the most significant bit of the length value and the least significant bit of the third of the three Length Octet(s) shall be the least significant bit of the length value.
- b) It shall be the sender's option to use either of the Length Octet(s) formats. The three octets format may be used to convey a length value of one, for instance.

- c) The meaning of the Length Octet(s) depends on the type of the value being encoded. If the encoding of the Contents Octets is primitive, the Length Octet(s) shall contain the number of octets in the Contents Octets. If the encoding of the Contents Octets is constructed, the Length Octet(s) shall contain the number of the first level components of the Contents Octets.
- d) Figure 22 and Figure 23 depict encoding examples of the Length Octet(s):

8	7	6	5	4	3	2	1
(msb)	value of the Length Octet as defined in (3) above						(lsb)

Figure 22 – Length octet (one-octet format)

first octet	15	second and third octets	1
1 1 1 1 1 1 1 1	(msb)	value of the Length Octets as defined in (3) above	(lsb)

Figure 23 – Length octet (three-octet format)

4.2.3.2.5.4 Contents octets

- a) The Contents Octets shall encode the data value according to the encoding rule defined for its type.
- b) The Contents Octets shall have either of the following two forms: a primitive encoding or a constructed encoding.
- 1) If the Contents Octets contain a primitive encoding, they represent an encoding of one value.
 - 2) If the Contents Octets contain a constructed encoding, they represent an enumerated encoding of more than one value.

4.2.3.3 Data type encoding rules (base encoding)

4.2.3.3.1 General

The base encoding rules of the following primitive data types defined in IEC 61158-5 are specified here.

- Boolean
- NULL
- Integer, Integer8, Integer16, Integer32
- Unsigned, Unsigned8, Unsigned16, Unsigned32
- Floating32, Floating64
- BitString, BitString8, BitString16, BitString32
- OctetString, OctetString2, OctetString4, OctetString8, OctetString16
- VisibleString, VisibleString2, VisibleString4, VisibleString8, VisibleString16
- ISO10646String
- UniversalTime
- BinaryTime0, BinaryTime1, BinaryTime2, BinaryTime3, BinaryTime4
- BinaryTime5, BinaryTime6, BinaryTime7, BinaryTime8, BinaryTime9
- CompactBCDArray
- BCD
- FieldbusTime
- CompactBooleanArray
- ANY

The base encoding rules of the following constructed data types are also provided:

- Array
- Structure

4.2.3.3.2 Encoding of a NULL value

- a) The encoding of a NULL value shall be primitive.
- b) The Identifier Octet shall be present, and the class number of the NULL value shall be FAL-Specific 2.
- c) The Length Octet(s) shall be omitted.
- d) The Contents Octet shall be omitted.
- e) The NULL type shall not be used to represent that there is no data value present. It may be used in the following two cases:
 - 1) An APDU which has no associated parameters:

Example: NoEffectReqPDU ::= NULL

- 2) In the CHOICE type

4.2.3.3.3 Encoding of a Boolean value

- a) The encoding of a Boolean value shall be primitive.
- b) The Identifier Octet and Length Octet(s) shall not be present.
- c) If a Boolean parameter is mandatory in an abstract syntax, then a fixed-length BitString value shall be placed where the Boolean parameter is first present in the abstract syntax and the value of the Boolean parameter and all the subsequent mandatory Boolean parameters shall be encoded in the BitString value in the order they appear. The first Boolean value shall be placed in the most significant bit of the BitString value and the following Boolean values shall be placed in the descending bits, consequently.
- d) If a Boolean parameter is optional in an abstract syntax, then its value shall be encoded in the next lower order bit of the OptionalParametersMap parameter whose higher order bit is used to indicate the presence of this parameter.
- e) If the Boolean value is FALSE, the corresponding bit shall be 0 (zero). If the Boolean value is TRUE, the bit shall be 1 (one).

4.2.3.3.4 Encoding of a variable length Integer value

- a) The encoding of a variable-length Integer value shall be primitive.
- b) The Identifier Octet shall not be present.
- c) The Length Octet(s) shall indicate the number of octets in the Contents Octets that are actually encoded.
- d) The Contents Octets shall be a two's complement binary number equal to the integer value, and consist of bits 8 to 1 of the first octet, followed by bits 8 to 1 of the second octet, followed by bits 8 to 1 of each octet in turn up to and including the last octet of the Contents Octets.

NOTE The value of a two's complement binary number is derived by numbering the bits in the Contents Octets, starting with bit 1 of the last octet and ending the numbering with bit 8 of the first octet. Each bit is assigned a numerical value of 2^{N-1} , where N is its position in the above numbering sequence. The value of the two's complement binary number is obtained by adding the numerical values assigned to each bit for those bits which are set to one, excluding bit 8 of the first octet, and then reducing this value by the numerical value assigned to bit 8 of the first octet if that bit is set to one.

- e) If the Contents Octets of a variable length Integer value encoding consist of more than one octet, and if the bits of the first octet and bit 8 of the second octet are all ones or zeros, it shall be the sender's option to remove the first octet or not. This process may be repeated to eliminate all such octets. The receivers shall be able to decode the value whether or not such an octet(s) is removed.

4.2.3.3.5 Encoding of a fixed length Integer value

- a) The encoding of a fixed-length Integer value of Integer8, Integer16, and Integer32 types shall be primitive, and the Contents Octets shall consist of exactly one, two, or four octets, respectively.
- b) The Identifier Octet and Length Octet(s) shall not be present.
- c) The Contents Octets shall be a two's complement binary number equal to the integer value, and consist of bits 8 to 1 of the first octet, followed by bits 8 to 1 of the second octet, followed by bits 8 to 1 of each octet in turn up to and including the last octet of the Contents Octets.

NOTE The value of a two's complement binary number is derived by numbering the bits in the Contents Octets, starting with bit 1 of the last octet and ending the numbering with bit 8 of the first octet. Each bit is assigned a numerical value of 2^{N-1} , where N is its position in the above numbering sequence. The value of the two's complement binary number is obtained by adding the numerical values assigned to each bit for those bits which are set to one, excluding bit 8 of the first octet, and then reducing this value by the numerical value assigned to bit 8 of the first octet if that bit is set to one.

4.2.3.3.6 Encoding of a variable length Unsigned value

- a) The encoding of a variable-length Unsigned value shall be primitive.
- b) The Identifier Octet shall not be present.
- c) The Length Octet(s) shall indicate the number of octets in the Contents Octets that are actually encoded.
- d) The Contents Octets shall be a binary number equal to the Unsigned value, and consisting of bits 8 to 1 of the first octet, followed by bits 8 to 1 of the second octet, followed by bits 8 to 1 of each octet in turn up to and including the last octet of the Contents Octets.

NOTE The value of a binary number is derived by numbering the bits in the Contents Octets, starting with bit 1 of the last octet as bit zero and ending the numbering with bit 8 of the first octet. Each bit is assigned a numerical value of 2^N , where N is its position in the above numbering sequence. The value of the binary number is obtained by adding the numerical values assigned to each bit for those bits which are set to one.

- e) If the Contents Octets of an Unsigned value encoding consist of more than one octet, and if the bits of the first octet are all zeros, it shall be the sender's option to remove the first octet or not. This process may be repeated to eliminate such octets. The receivers shall be able to decode the value whether or not such an octet(s) is removed.

4.2.3.3.7 Encoding of a fixed length Unsigned value

- a) The encoding of a fixed-length Unsigned value of Unsigned8, Unsigned16, and Unsigned32 types shall be primitive, and the Contents Octets shall consist of exactly one, two, or four octets, respectively.
- b) The Identifier Octet and Length Octet(s) shall not be present.
- c) The Contents Octets shall be a binary number equal to the Unsigned value, and consist of bits 8 to 1 of the first octet, followed by bits 8 to 1 of the second octet, followed by bits 8 to 1 of each octet in turn up to and including the last octet of the Contents Octets.

NOTE The value of a binary number is derived by numbering the bits in the Contents Octets, starting with bit 1 of the last octet and ending the numbering with bit 8 of the first octet. Each bit is assigned a numerical value of 2^{N-1} , where N is its position in the above numbering sequence. The value of the binary number is obtained by adding the numerical values assigned to each bit for those bits which are set to one.

4.2.3.3.8 Encoding of a Floating Point value

- a) The encoding of a Floating32 or Floating64 value shall be primitive, and the Contents Octets shall consist of exactly four or eight octets, respectively.
- b) The Identifier Octet and Length Octet(s) shall not be present.
- c) The Contents Octets shall contain floating point values defined in conformance with IEC-60559 Standard. The sign is encoded by using bit 8 of the first octet. It is followed by the exponent starting from bit 7 of the first octet, and then the mantissa starting from bit 7 of the second octet for Floating32 and from bit 4 of the second octet for Floating64.

4.2.3.3.9 Encoding of a variable length BitString value

- a) The encoding of a variable-length BitString value shall be primitive.
- b) The Identifier Octet shall not be present.
- c) The Length Octet(s) shall indicate the number of octets in the BitString encoding, which is the number of octets used to encode a BitString value plus one for the "Number of Unused Bits" field (see (4)).
- d) If the BitString is empty (no BitString value at all), there shall be no subsequent octets, and the Length Octet(s) shall be zero.
- e) If the BitString is not empty, the Contents Octets shall encode the Number of Unused Bits and the BitString value as follows:
- f) The Number of Unused Bits field shall follow the Length Octet(s) and shall always be one octet. It shall encode as an Unsigned8 type the number of unused bits in the final octet of the Contents Octets. This number shall be in the range zero to seven, inclusive.

NOTE 1 The Number of Unused Bits field is provided for the service user.

NOTE 2 The Number of Unused Bits is selected in accordance with ISO/IEC 8825.

- g) The value of the BitString, commencing with the first bit and proceeding to the trailing bit, shall be placed in bits 8 to 1 of the first octet which follows the Number of Unused Bits field, followed by bits 8 to 1 of the second octet, followed by bits 8 to 1 of each octet up to and including the last octet of the Contents Octets. The unused bits shall be placed in bits 7 to 1 of the last octet, and their values are not specified.

4.2.3.3.10 Encoding of a fixed length BitString value

- a) The encoding of a fixed-length Bitstring value of BitString8, BitString16, and BitString32 types shall be primitive, and the Contents Octets shall consist of exactly one, two, or four octets, respectively.
- b) The Identifier Octet and Length Octet(s) shall not be present.
- c) A BitString value, commencing with the first bit and proceeding to the trailing bit, shall be placed in bits 8 to 1 of the first octet, followed by bits 8 to 1 of the second octet, followed by bits 8 to 1 of each octet up to and including the last octet of the Contents Octets.

4.2.3.3.11 Encoding of a variable length OctetString value

- a) The encoding of a variable-length OctetString value shall be primitive.
- b) The Identifier Octet shall not be present.
- c) The Length Octet(s) shall indicate as a binary number the number of octets in the OctetString value.
- d) If the OctetString is of zero length, there shall be no subsequent octets, and the Length Octet(s) shall be zero.
- e) If the OctetString is not of zero length, the Contents Octets shall be equal in value to the octets in the data value, in the order they appear in the data value, and with the most significant bit of an octet of the data value aligned with the most significant bit of an octet of the Contents Octets.

4.2.3.3.12 Encoding of a fixed length Octet String value

- a) The encoding of a fixed-length OctetString value of OctetString2, OctetString4, OctetString8, and OctetString16 types shall be primitive, and the Contents Octets shall consist of exactly two, four, eight, or 16 octets, respectively.
- b) The Identifier Octet and Length Octet(s) shall not be present.
- c) The Contents Octets shall be equal in value to the octets in the data value, in the order they appear in the data value, and with the most significant bit of an octet of the data value aligned with the most significant bit of an octet of the Contents Octets.

4.2.3.3.13 Encoding of a variable length VisibleString value

- a) The encoding of a VisibleString value shall be primitive.

- b) The Identifier Octet shall not be present.
- c) The Length Octet(s) shall indicate as a binary number the number of octets in the VisibleString value.
- d) If the VisibleString is of zero length, there shall be no subsequent octets, and the Length Octet(s) shall be zero.
- e) If the VisibleString is not of zero length, the Contents Octets shall be equal in value to the octets in the data value, in the order they appear in the data value, and with the most significant bit of an octet of the data value aligned with the most significant bit of an octet of the Contents Octets.

4.2.3.3.14 Encoding of a fixed length VisibleString value

- a) The encoding of a fixed-length VisibleString value of VisibleString2, VisibleString4, VisibleString8, and VisibleString16 types shall be primitive, and the Contents Octets shall consist of exactly two, four, eight, or 16 octets, respectively.
- b) The Identifier Octet and Length Octet(s) shall not be present.
- c) The Contents Octets shall be equal in value to the octets in the data value, in the order they appear in the data value, and with the most significant bit of an octet of the data value aligned with the most significant bit of an octet of the Contents Octets.

4.2.3.3.15 Encoding of an ISO10646String value

- a) The encoding of an ISO10646String value shall be primitive, and the Contents Octets shall consist of zero or an even number of octets.
- b) The Identifier Octet shall not be present.
- c) The Length Octet(s) shall indicate as a binary number the number of octets in the Contents Octets.
- d) If the ISO10646String is of zero length, there shall be no subsequent octets, and the Length Octet(s) shall be zero.
- e) If an ISO10646String value is not of zero length, the Contents Octets shall be equal in value to the octets in the data value.
- f) Each ISO10646 character shall be placed in two octets in the Contents Octets, with the high order octet placed in the first octet and the low-order octet in the subsequent octet, and with the most significant bit of an octet of the data value aligned with the most significant bit of an octet of the Contents Octets.

4.2.3.3.16 Encoding of a UniversalTime value

- a) The encoding of a UniversalTime value shall be primitive, and the Contents Octets shall consist of 12 octets.
- b) The Identifier Octet and Length Octet(s) shall not be present.
- c) The Contents Octets shall be equal in value to the octets in the data value, and with the most significant bit of an octet of the data value aligned with the most significant bit of an octet of the Contents Octets.
 - 1) The first “D” of a UniversalTime value shall be placed in the first octet of the Contents Octets, followed by the second “D” in the second octet, up to the second “s” in the 12th octet of the Contents Octets.
 - 2) A UniversalTime value consists of two sets, each of six characters. The first set is YYMMDD, where YY is the two low-order digits of the Gregorian calendar, MM is the month (counting January as 01 and December as 12), and DD is the day of the month (01 to 31). The second set is hhmmss, where hh is hour (00 to 23), mm is minutes (00 to 59), and ss is seconds (00 to 59).
 - 3) In countries where the Gregorian calendar is not adapted, it is possible to substitute the Gregorian YYMM with their local calendar convention. If a local calendar system is adopted, YY shall be the two low-order digits of that calendar system and MM shall be the month counting the first month of the calendar system as 01.

4.2.3.3.17 Encoding of BinaryTime value

- a) The encoding of a BinaryTime0, BinaryTime1, BinaryTime2, BinaryTime3, BinaryTime4, BinaryTime5, BinaryTime6, BinaryTime7, BinaryTime8 and BinaryTime9 value shall be primitive.
- b) The Identifier Octet and Length Octet(s) shall not be present.
- c) The Contents Octets shall be a binary number equal to the binary time value, and consisting of bits 8 to 1 of the first octet, followed by bits 8 to 1 of the second octet, followed by bits 8 to 1 of each octet in turn up to and including the last octet of the Contents Octets.
 - 1) The Contents Octets of a BinaryTime0, BinaryTime1, BinaryTime2, and BinaryTime3 value shall consist of two octets.
 - 2) The Contents Octets of a BinaryTime4, BinaryTime5, BinaryTime6, and BinaryTime7 value shall consist of four octets.
 - 3) The Contents Octets of a BinaryTime8 and BinaryTime9 value shall consist of six octets.

NOTE The value of the granularity of each BinaryTime type is defined in IEC 61158-5.

4.2.3.3.18 Encoding of a CompactBCDArray value

- a) The encoding of a CompactBCDArray value shall be primitive.
- b) The Identifier Octet shall not be present.
- c) The Length Octet(s) shall indicate as a binary number the number of octets in the array.
- d) If the number of BCD values is zero, there shall be no subsequent octets, and the Length Octet(s) shall be zero.
- e) The Contents Octets shall encode the BCD values as the first BCD value shall be placed as a binary number in bits 8 to 5 of the first Contents Octets, the second BCD value shall be placed in bits 4 to 1 of the first Contents Octets. This will be repeated for the remaining BCD values and Contents Octets up to and including the last octet of the Contents Octets. The values of any unused bits in the last Contents Octets shall be set to F16.

4.2.3.3.19 Encoding of an Array value

An Array value shall be encoded as a SEQUENCE OF value.

4.2.3.3.20 Encoding of a Structure value

A Structure value shall be encoded as a SEQUENCE value.

4.2.3.3.21 Encoding of the ANY type

NOTE 1 The ANY type is used to convey a value of an APO, such as a data value to be written to a variable object. Since an APO is not managed by the FAL, it does not provide any encoding means for this type of value. It is assumed that the service users know the data types of such data values.

- a) The encoding of the ANY type shall be primitive or constructed.
- b) The Length Octet(s) shall be present and shall indicate the number of octets in the Contents Octets.

NOTE 2 This ensures that the first octet in the ANY type always indicates the total octets of the ANY type.

- c) The Contents Octets shall be the encoding of a data value.
 - 1) The data value shall be encoded by the service user so that the resultant encoding is identical to what is defined in this standard.

NOTE 3 If a data to be encoded as ANY is of type Integer, for example, the service user is responsible for providing octets that represent the value of the Integer value based on the Integer encoding rule defined in this standard.

- 2) If the encoding of a data value is primitive, the Length Octet(s) of the data value, if it is present, shall be omitted.

- 3) If an Integer value is encoded as ANY, the Contents Octets of ANY include only the Contents Octets of the Integer value. Its Length Octet(s) shall be omitted.
- 4) If the encoding of a data value is constructed, the Length Octet(s) of the data value, if it is present, shall be present in the Contents Octets of ANY.

4.2.3.3.22 Encoding of a BCD value

- a) A BCD value shall be encoded as an Unsigned8 value.
- b) A BCD value shall be placed in bits 4 to 1 of the Contents Octets of an Unsigned8 value. The values of the bits 8 to 5 shall be zero (0).

4.2.3.3.23 Encoding of a FieldbusTime value

A FieldbusTime value shall be encoded as a BitString32 value.

4.2.3.3.24 Encoding of a Compact Boolean Array value

A Compact Boolean Array value shall be encoded as a BitString value.

4.2.3.4 Key words encoding rules

4.2.3.4.1 General

The encoding rules for the following key words are defined:

- Sequence
- Sequence of
- Choice
- TaggedType
- IMPLICIT
- OPTIONAL
- DEFAULT

4.2.3.4.2 Encoding of a SEQUENCE value

- a) The encoding of a SEQUENCE value shall be constructed.
- b) The Identifier Octet, if it is present, shall have the class number of FAL -Specific 1.
- c) The Identifier Octet shall be omitted from the encoding of the SEQUENCE value used in the FAL syntax descriptions specified in clause 4.
- d) The Length Octet(s), if it is present, shall indicate, as a binary number, the number of productions (not the number of octets) in the SEQUENCE value to be encoded.
 - 1) If there are zero data values, there shall be no subsequent octets, and the Length Octet(s) shall be zero.
 - 2) The Length Octet(s) shall be omitted from the encoding of the SEQUENCE value used in the FAL syntax descriptions specified in clause 4.

NOTE Rules 2.1) and 3.2), and the fact that all the APDU abstract syntax definitions are referenced with the IMPLICIT key word, ensure that the encoding of the APDU body starts with the encoding of the first component of each APDU productions, not with the encoding of SEQUENCE or SEQUENCE OF key words that envelop the productions.

- e) The Contents Octets shall consist of the complete encoding of one data value from each of the types listed in the SEQUENCE type, in the order of their appearance in the definition, unless the type was referenced with the keyword "OPTIONAL" or the keyword "DEFAULT."

4.2.3.4.3 Encoding of a SEQUENCE OF value

- a) The encoding of a SEQUENCE OF value shall be constructed.

- b) The Identifier Octet shall be present. The class number of the SEQUENCE OF value shall be FAL-Specific 1.
- c) The Length Octet(s) shall indicate as a binary number the number of productions (not the number of octets) in the SEQUENCE OF value to be encoded.
- d) If there are zero productions, there shall be no subsequent octets, and the Length Octet(s) shall be zero.
- e) The Contents Octets shall consist of the complete encoding of the data values from the type listed in the SEQUENCE OF value. The order of the encoding of the data values shall be the same as the order of the data values in the SEQUENCE OF value to be encoded.

4.2.3.4.4 Encoding of productions with OPTIONAL and DEFAULT key words

- a) The encoding of a data value may, but need not, be present for a type which was referenced with the keyword "OPTIONAL" or the keyword "DEFAULT."
- b) If a SEQUENCE contains productions with the OPTIONAL or DEFAULT key word, those productions shall not be tagged and, in the abstract syntax constructions to be used with this encoding rule, shall be placed at the end of the SEQUENCE type. Then a production

optionalParametersMap		Gn_OptionalParametersMap8
	or	
optionalParametersMap		Gn_OptionalParametersMap16
	or	
optionalParametersMap		Gn_OptionalParametersMap32

shall be placed just before the first OPTIONAL or DEFAULT production.

- c) The "optionalParametersMap" parameter, of type Gn_OptionalParametersMap8, Gn_OptionalParametersMap16, or Gn_OptionalParametersMap32 shall indicate only the OPTIONAL or DEFAULT productions which are defined in the abstract syntax.
- d) The "optionalParametersMap" parameter is one of the fixed-length BitString types, and the value of one in each of the bits indicates that the corresponding OPTIONAL or DEFAULT production is present in the encoding, and the value of zero indicates that it is not present.
- e) The least significant bit of the optionalParametersMap parameter corresponds to the first OPTIONAL or DEFAULT production in the SEQUENCE, the second least significant bit corresponds to the second production, and the n-th bit corresponds to the n-th production.
- f) If a SEQUENCE value contains more than 32 OPTIONAL or DEFAULT productions, the BitString type shall be used as the type of the optionalParametersMap parameter.
- g) The Number of Unused Bits field of this BitString value shall be zero since the bits are used beginning from the lsb as described above.
- h) If it is possible that all the optional parameters are not present in an encoding, the optionalParametersMap parameter itself may become OPTIONAL.
- i) When the optionalParametersMap parameter is OPTIONAL, there shall be no additional optionalParametersMap parameters present to indicate that the optionalParametersMap parameter is OPTIONAL.

4.2.3.4.5 Encoding of a CHOICE value

The encoding of a CHOICE value shall be the same as the encoding of a value of the chosen type.

NOTE In the abstract syntax constructions to be used with this encoding rule, each type listed as a CHOICE shall be an unambiguously tagged type, since the Identifier Octet is frequently not encoded for the type specified.

4.2.3.4.6 Encoding of a Tagged value

- a) The encoding of a Tagged value shall be primitive if the referenced encoding is primitive and constructed if it is constructed.
- b) The Identifier Octet shall contain the value of the tag and its class.
- c) The Identifier Octet for the tagged value shall not be encoded unless those used with the CHOICE type.
- d) The Length Octet(s) may or may not be present.

NOTE These rules ensure that the Length Octet(s) of the Tagged value always indicates the number of octets of the Contents Octet(s) of the Tagged value.

- 1) If the referenced encoding is primitive and does not have the Length Octet(s), the Length Octet(s) of the Tagged value shall indicate the number of octets of the Contents Octet(s) of the referenced encoding.
 - 2) If the referenced encoding is primitive and has the Length Octet(s), the Length Octet(s) of the referenced encoding shall be placed in the Length Octet(s) of the Tagged value.
 - 3) If the referenced encoding is constructed, the Length Octet(s) of the Tagged value shall indicate the number of components of the referenced encoding.
- e) The Contents Octet(s) shall be those of the referenced encoding.

4.2.3.4.7 Encoding of an IMPLICIT value

- a) The IMPLICIT keyword shall only be used with a Tagged type value, or one of the APDU class specifiers defined in the "Encoding of Type Field" subclause.
- b) If the IMPLICIT keyword is used, the Identifier Octet of the type which immediately follows the keyword shall not be present.

NOTE If a base encoding does not have the Identifier Octet and/or Length Octet(s), the IMPLICIT keyword does not affect the encoding.

4.2.4 Buffer-oriented Encoding Rules (BER) and Messaging Encoding Rule (MER)

4.2.4.1 Overview

4.2.4.1.1 General

The Encoding Rule for Buffer Services (BER) is an encoding rule for new implementation where Time critical transfer is selected.

4.2.4.1.2 Overview of encoding

The PDUs that conform to this standard shall be encoded in a uniform format as shown in Figure 24. The PDUs consist of two major parts: the "APDU Header" part and the "APDU Body" part.

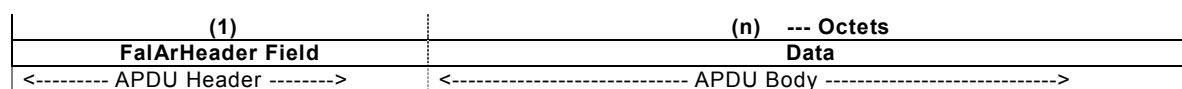


Figure 24 – APDU overview

4.2.4.1.3 APDU header encoding

The APDU Header part is always present in all APDUs which conform to this Clause of IEC 61158-6. It consists of one field: the FalArHeader Field. Refer to 4.2.1 for the encoding rule of the FalArHeader field.

4.2.4.1.4 APDU body encoding

The FAL encoding rules are based on terms and conventions defined in ISO/IEC standards. The encoding consists of three components in the following order:

- An Identifier Octet

- Length Octet(s)
- The ContentsOctets

4.2.4.1.5 Identifier octet

- a) The Identifier Octet shall encode the type and shall consist of one octet.
- b) The value of this octet is 40 h. Other values are reserved for management.

4.2.4.1.6 Length octet

The Length Octet shall consist of one octet.

4.2.4.1.7 Contents octets

The Contents Octets shall encode the data value as a string of octet.

4.2.4.2 Encoding Rule for Messaging services (MER)

4.2.4.2.1 General

The encoding of an ASN.1 type value may comprise the following components:

- Identification Octet
- Content Length Octets
- Content Octets

4.2.4.2.2 Identification octet

The Identification octet is used for the encoding of the type tag associated with the value. It contains the class and number of the type tag as shown in Table 15.

The Identification Octet only exists in the encoding of tagged types appearing in a CHOICE. If present, the Identification Octet is structured as follows:

Bit 8 defines the value type class:

Table 15 – Identification octet classes

Class	Bit 8 value
FAL specific	0
Context-specific	1

Bits 7 to 1 are used for encoding the identifier numbers in the range 0 to 127, in the form of an unsigned binary number, whose most and least significant bits are respectively bits 7 and 1.

4.2.4.2.3 Contents length octets

If present, the encoding of the contents length of the type value always consists of two octets representing a number of units of measurement (unsigned binary number).

The unit of measurement is type-dependent, as shown in Table 16.

Table 16 – Unit of measurement of contents length octets

Type	Unit of measurement
SEQUENCE OF	octet
SEQUENCE	octet
INTEGER	octet
BOOLEAN	octet
BIT STRING	bit
OCTET STRING	octet

4.2.4.2.4 Contents octets

The ContentsOctets shall encode the data value according to the encoding rules defined for the respective type as specified in the following subclauses. The ContentsOctets may consist of zero, one or more octets.

4.2.4.3 Type encoding rules

4.2.4.3.1 Boolean

A Boolean value shall be encoded as follows:

- The Identifier Octet and the Length Octets shall not be present.
- The ContentsOctets always consist of one octet. If the Boolean value equals FALSE, all bits of the octet are 0. If the Boolean value equals TRUE, the octet can contain any combination of bits other than the encoding for FALSE.

4.2.4.3.2 Integer

An integer value shall be encoded as follows:

- The Identifier Octet shall not be present.
- The Length Octets shall not be present, if the size of the integer type is invariable. An integer with invariable size is created by constraining the possible value. The length octets shall be present, if the size of the integer value is variable.
- The ContentsOctets shall contain the two complement binary number equal to the integer value. The most significant bits of the integer value are encoded in bit8 to bit1 of the first octet, the next in bit8 to bit1 of the next octet and so on. If the values of an integer type are restricted to negative and non-negative numbers, bit8 of the first octet gives the sign of the value, if the values are restricted to non-negative numbers only, no sign bit is needed (see examples below)

Examples:

- INTEGER (-128..128) is an integer with invariable octet length of one.
- INTEGER (0..255) is also an integer with invariable octet length of one.
- INTEGER is an integer with variable octet length.

4.2.4.3.3 Bit string

A bitstring value shall be encoded as follows:

- The Identifier Octet shall not be present
- The Length Octets shall not be present, if the size of the bitstring type is invariable. A bitstring with invariable size is created by applying a size constraint containing only one value on the bitstring type. The length octets shall be present, if the size of the bitstring value is variable.
- The ContentsOctets comprise as many octets as necessary to contain all bits of the actual value: $N_Octets = (N_Bits - 1) \div 8 + 1$. The bitstring value commencing with the

first bit and proceeding to the trailing bit, shall be placed in bits 8 to 1 of the first octet, followed by bits 8 to 1 of the second octet and so on. If the number of bits is not a multiple of 8 there are so-called unused bits, which are located in the least significant bits of the last octet. The value of the unused bits may be zero or one and carry no meaning.

Examples:

- BIT STRING SIZE (30) is a bitstring with invariable octet length of four and two unused bits.
- BIT STRING SIZE (1..32) is a bitstring with variable octet length.
- BIT STRING is a bitstring with variable octet length.

4.2.4.3.4 Octet string

An octetstring value shall be encoded as follows:

- a) The Identifier Octet shall not be present.
- b) The Length Octets shall not be present, if the size of the octetstring type is invariable. An octetstring with invariable size is created by applying a size constraint containing only one value on the octetstring type. The length octets shall be present, if the size of the octetstring value is variable.
- c) The ContentsOctets shall be equal in value to the octets in the data value.

Examples:

- OCTET STRING SIZE (30) is an octetstring with invariable octet length of 30.
- OCTET STRING SIZE (1..32) is an octetstring with variable octet length.
- OCTET STRING is an octetstring with variable octet length.

4.2.4.3.5 Visible string

A visiblestring value shall be encoded as follows:

- a) The Identifier Octet shall not be present.
- b) The Length Octets shall not be present, if the size of the visiblestring type is invariable. A visiblestring with invariable size is created by applying a size constraint containing only one value on the visiblestring type. The length octets shall be present, if the size of the visiblestring value is variable.
- c) The ContentsOctets shall be equal in value to the octets in the data value.

Examples:

- VisibleString SIZE (30) is a visiblestring with invariable octet length of 30.
- VisibleString SIZE (1..32) is a visiblestring with variable octet length.
- VisibleString is a visiblestring with variable octet length.

4.2.4.3.6 SEQUENCE types

A value of a SEQUENCE type shall be encoded as follows:

- a) The Identifier Octet shall not be present.
- b) The Length Octets shall be there and specify the number of octets being used for the ContentsOctets. However, for the first Keyword "SEQUENCE" of FaIArPDU, this length shall not be encoded at all.
- c) The ContentsOctets shall consist of the encoding of all the element types in the same order as they are specified in the ASN.1 description of the SEQUENCE type.

4.2.4.3.7 SEQUENCE OF types

A value of a SEQUENCE OF type shall be encoded as follows:

- a) The Identifier Octet shall not be present.
- b) The Length Octets shall be there and specify the number of octets being used for the ContentsOctets.
- c) The ContentsOctets shall consist of the encoding of all the element types in the same order as they are specified in the ASN.1 description of the SEQUENCE OF type.

4.2.4.3.8 CHOICE types

A value of a CHOICE type shall be encoded as follows:

- a) The Identifier Octet shall not be present.
- b) The Length Octets shall not be present.
- c) The ContentsOctets shall consist of the encoding of the selected type of the alternative type list.

4.2.4.3.9 Null

A value of a NULL type shall be encoded as follows:

- a) The Identifier Octet shall not be present.
- b) The Length Octets shall not be present.
- c) The ContentsOctets shall not be present.

4.2.4.3.10 Tagged types

A value of a tagged type shall be encoded as follows:

- a) The Identifier Octet shall only be present, if the tagged type is part of an Alternative Type List in a CHOICE construct.
- b) The Length Octets shall not be present.
- c) The ContentsOctets shall consist of the encoding of the Type which was tagged.

4.2.4.3.11 IMPLICIT types

A value of an IMPLICIT type shall be encoded as follows:

- a) The Identifier Octet shall not be present.
- b) The Length Octets shall not be present.
- c) The ContentsOctets shall consist of the encoding of the Type being referenced by the IMPLICIT construct, except for the case when the referenced Type is a SEQUENCE type. In this case, the ContentsOctets consist only of the ContentsOctets of the referenced SEQUENCE type, the length information of this SEQUENCE type should not occur in the encoding.

4.2.4.3.12 OPTIONAL and DEFAULT types

For the encoding of a value of an OPTIONAL or DEFAULT type there are two possibilities:

- a) If the OPTIONAL/DEFAULT type is a subtype of a SEQUENCE type containing a Gn_OptionalParametersMap element, which is used to determine the existence or non-existence of all optional subtypes within this structure. The least significant bit of this Gn8OptionalParametersMap type corresponds to the first OPTIONAL or DEFAULT subtype, the second least significant bit corresponds to the second subtype and so on. In this case, a value of an OPTIONAL or DEFAULT type shall be encoded as follows:

- 1) If the corresponding bit of the Gn_OptionalParametersMap is zero or the Gn_OptionalParametersMap is missing altogether (it may be an OPTIONAL itself), the OPTIONAL/DEFAULT value does not exist and should not be encoded at all. If the corresponding bit of the Gn_OptionalParametersMap is one, the value exists, and its encoding should be as follows:
 - 2) The Identifier Octet shall not be present.
 - 3) The Length Octets shall not be present.
 - 4) The ContentsOctets shall consist of the encoding of the Type being referenced by the OPTIONAL/DEFAULT construct.
- b) Otherwise, a value of an OPTIONAL or DEFAULT type shall be encoded as follows:
 - 1) The Identifier Octet shall not be present.
 - 2) The Length Octets shall be present. If there is no value for this type, the Length Octets contain the value 0.
 - 3) The ContentsOctets shall consist of the encoding of the referenced Type, if there is a value for this type, otherwise no ContentsOctets exist.
 - 4) If a Gn_OptionalParametersMap type is defined as OPTIONAL itself, it is always encoded in the second way.

4.2.4.3.13 ANY types

An ANY type is used for the definition of complex types, whose structure is described informally rather than in ASN.1, generally because its definition in ASN.1 would not result in an optimal encoding. Usually the structuring information (Identifier Octets, Length Octets) in the encoding of such types can be omitted, since the receiver of the encoded type has additional knowledge, which enables him to encode such an optimized PDU.

- a) A value of an ANY type shall be encoded as follows:
 - b) The Identifier Octet shall not be present.
 - c) The Length Octets shall not be present.
 - d) The ContentsOctets shall consist of the encoding of all implicit types which constitute the ANY type.

4.2.4.3.14 Encoding of APDU header

The APDU Header always begins with the FALArHeader. This implies that the Contents Length of the surrounding SEQUENCE is not encoded.

4.3 FAL protocol state machines structure

Interface to FAL services and protocol machines are specified in this subclause.

NOTE The state machines specified in this subclause and ARPMs defined in the following sections only define the valid events for each. It is a local matter to handle these invalid events.

The behavior of the FAL is described by three integrated protocol machines. Specific sets of these protocol machines are defined for different AREP types. The three protocol machines are: FAL Service Protocol Machine (FSPM), the Application Relationship Protocol Machine (ARPM), and the Data Link Layer Mapping Protocol Machine (DMPM). The relationship among these protocol machines as well as primitives exchanged among them are depicted in Figure 25.

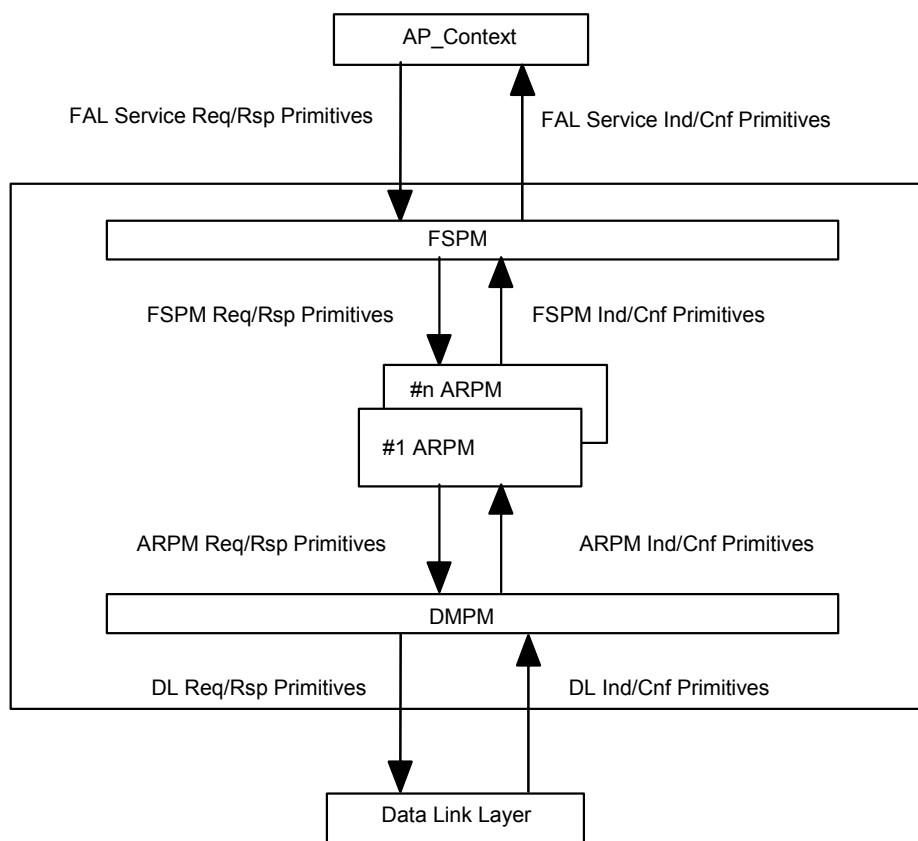


Figure 25 – Relationships among protocol machines and adjacent layers

The FSPM describes the service interface between the AP-Context and a particular AREP. The FSPM is common to all the AREP classes and does not have any state changes. The FSPM is responsible for the following activities:

- a) to accept service primitives from the FAL service user and convert them into FAL internal primitives;
- b) to select an appropriate ARPM state machine based on the AREP Identifier parameter supplied by the AP-Context and send FAL internal primitives to the selected ARPM;
- c) to accept FAL internal primitives from the ARPM and convert them into service primitives for the AP-Context;
- d) to deliver the FAL service primitives to the AP-Context based on the AREP Identifier parameter associated with the primitives.

The ARPM describes the establishment and release of an AR and exchange of FAL-PDUs with a remote ARPM(s). The ARPM is responsible for the following activities:

- e) to accept FAL internal primitives from the FSPM and create and send other FAL internal primitives to either the FSPM or the DMPM, based on the AREP and primitive types;
- f) to accept FAL internal primitives from the DMPM and send them to the FSPM as a form of FAL internal primitives;
- g) if the primitives are for the Establish or Abort service, it shall try to establish or release the specified AR.

The DMPM describes the mapping between the FAL and the DLL. It is common to all the AREP types and does not have any state changes. The DMPM is responsible for the following activities:

- h) to accept FAL internal primitives from the ARPM, prepare DLL service primitives, and send them to the DLL;

- i) to receive DLL indication or confirmation primitives from the DLL and send them to the ARPM in a form of FAL internal primitives.

4.4 AP-Context state machine

4.4.1 Primitive definitions

4.4.1.1 Primitives exchanged between FAL-user and AP-Context

Primitives exchanged between FAL-user and AP-Context are shown in Table 17 and 18.

Table 17 – Primitives issued by FAL-user to AP-Context

Primitive name	Source	Associated parameters and functions
Terminate.req	FAL-User	Refer to FAL Service Definition (IEC 61158-5)
Initiate.req	FAL-User	
Initiate.rsp(+)	FAL-User	
Initiate.rsp(-)	FAL-User	
ConfirmedService.req	FAL-User	
ConfirmedService.rsp	FAL-User	
UnconfirmedService.req	FAL-User	

Table 18 – Primitives issued by AP-Context to FAL-user

Primitive name	Source	Associated parameters and functions
Terminate.ind	AP-Context	Refer to FAL Service Definition (IEC 61158-5)
Initiate.ind	AP-Context	
Initiate.cnf(+)	AP-Context	
Initiate.cnf(-)	AP-Context	
ConfirmedService.ind	AP-Context	
ConfirmedService.cnf	AP-Context	
UnconfirmedService.ind	AP-Context	

4.4.2 State machine description

The attributes used in this state machine are defined as elements of the AP attribute *List Of In-Use AR Endpoint Info*. See Figure 26 for the state machine.

CLOSED

The AP-Context for an AR is not open. Only the service primitives Initiate.req, Establish.ind, Terminate.req and Abort.ind are allowed. All other service primitives shall be rejected with the Terminate service.

OPENING-REQUESTING (REQ)

The local FAL user wishes to open the AP-Context for an AR. Only the service primitives Establish.cnf(+), Establish.cnf(-), Terminate.req and Abort.ind are allowed. All other services shall be rejected with the Terminate service.

OPENING-RESPONDING (RSP)

The remote AP-Context wishes to open the AP-Context for an AR. Only the service primitives Initiate.rsp(+), Initiate.rsp(-), Terminate.req and Abort.ind are allowed. All other service primitives shall be rejected with the Terminate service.

OPEN

The AP-Context for an AR is open. The service primitives *Initiate.req*, *Initiate.rsp(+)*, *Initiate.rsp(-)* are not allowed and shall be rejected with the *Terminate* service. The following actions shall be taken to reset the AP-Context for an AR.

Set the *Outstanding Services Requesting Counter* and *Outstanding Services Responding Counter* to 0.

Set the *Context State* to "CLOSED"

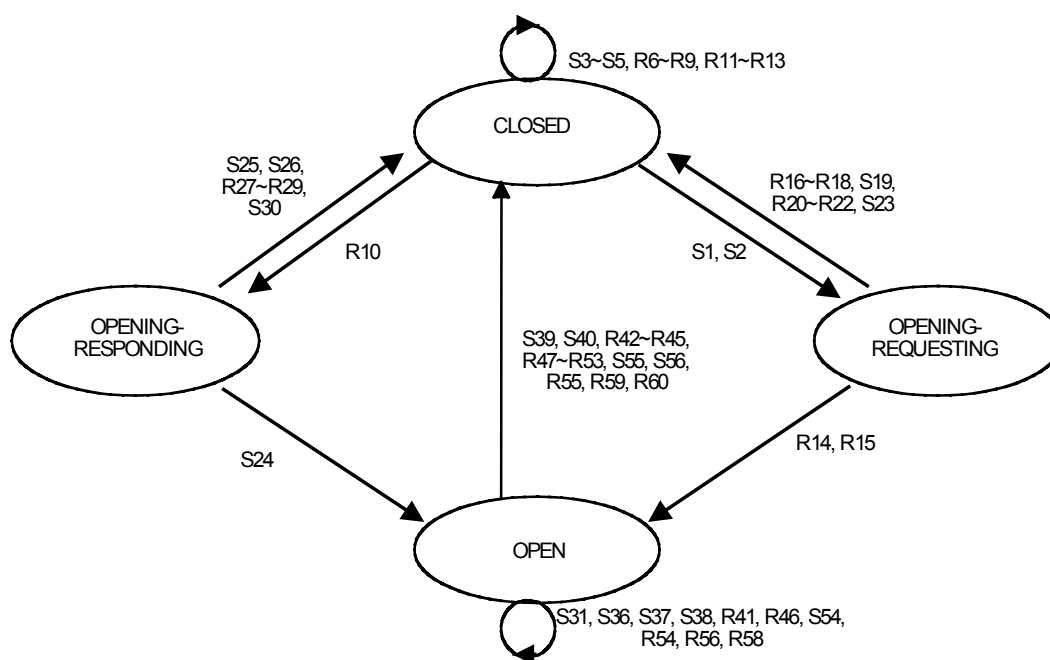


Figure 26 – AP to AP-Context initiation state machine

4.4.3 AP to AP-Context initiation state transitions

The AP to AP-Context initiation state transitions are defined in Table 19 and Table 20. In the tables of this subclause "Data" parameter for FAL Service Protocol Machine is expressed by "FalApuBody" to emphasize its semantics.

NOTE Since Type 1 defines different abstract syntax for the same service, multiple PDU forms are used in the following state tables. It is assumed and advised that only one selection that uses the same abstract syntax will be used in any instances of these state machines.

Table 19 – AP-Context state machine sender transactions

#	Current state	Event or condition => action	Next State
S1	CLOSED	Initiate.req && ApContextTest = "True" && ApExplicitConnection = "True" => EST.req { FalApuBody := "Establish-CommandPDU" } or EST.req { FalApuBody := "Establish-RequestPDU" } or EST.req { FalApuBody := "Establish-Request2PDU" }	REQ
S2	CLOSED	Initiate.req && ApContextTest = "True" && ApExplicitConnection = "False" => EST.req { FalApuBody := "NULL" }	REQ
S3	CLOSED	Initiate.req && ApContextTest = "False" => Terminate.ind { LocallyGenerated := "True", TerminatIdentifier := "FAL", ReasonCode := "Context error" }	CLOSED
S4	CLOSED	Terminate.req => (no actions taken)	CLOSED
S5	CLOSED	FAL service.primitive <> "Initiate" && FAL service.primitive <> "Terminate" => Terminate.ind { LocallyGenerated := "True", TerminatIdentifier := "FAL", ReasonCode := "User error" }	CLOSED
S19	REQ	Terminate.req => Abort.req { Originator:= "TerminatIdentifier of Terminate.req", ReasonCode := "ReasonCode of Terminate.req" }, ResetArep	CLOSED
S23	REQ	FAL service.primitive <> "Initiate.rsp" && FAL service.primitive <> "Terminate.req" => Abort.req { Originator := "FAL", ReasonCode := "User error" }, Terminate.ind { LocallyGenerated := "True", TerminatIdentifier := "FAL", ReasonCode := "User error" }, ResetArep	CLOSED

Table 19 (continued)

#	Current state	Event or condition => action	Next state
S24	RSP	Initiate.rsp(+) => EST.rsp(+){ FalApduBody := "Establish-AffirmativePDU" } or EST.rsp(+){ FalApduBody = "Establsih-ResponsePDU" } }	OPEN
S25	RSP	Initiate.rsp(-) => EST.rsp(-){ FalApduBody := "Establish-NegativePDU" }, or EST.rsp(-){ FalApduBody = "Establsih-ErrorPDU" }, } ResetArep	CLOSED
S26	RSP	Terminate.req => Abort.req { Originator := "TerminatIdentifier of Terminate.req", ReasonCode := "ReasonCode of Terminate.req" }, } ResetArep	CLOSED
S30	RSP	FAL service.primitive <> "Initiate.rsp" && FAL service.primitive <> "Terminate.req" => Abort.req { Originator := "FAL", ReasonCode := "User error" }, } Terminate.ind { LocallyGenerated := "True", TerminatIdentifier := "FAL", ReasonCode := "User error" }, } ResetArep	CLOSED
S31	OPEN	ConfirmedService.req && ConfirmedServiceCheck = "True" && OutstandingServicesRequestingCounter < NegotiatedMaxOutstanding- ServicesRequesting && InvokeldExistent = "False" && PDU length ≤ NegotiatedMaxPduSizeSending && RequestedServiceSupportedTest = "True" => CS.req { FalApduBody := "ConfirmedSend-CommandPDU" }, or CS.req { FalApduBody := "ConfirmedSend-RequestPDU" }, } OutstandingServicesRequestingCounter := OutstandingServicesRequestingCounter+1	OPEN

Table 19 (continued)

#	Current state	Event or condition => action	Next state
S36	OPEN	UnconfirmedService.req && UnconfirmedServiceCheck = "True" && PDU length ≤ NegotiatedMaxPduSizeSending && RequestedServiceSupportedTest = "True" && ImmediateAcknowledge = "False" => UCS.req { FalApuBody := "UnconfirmedSend-CommandPDU" } or UCS.req { FalApuBody := "UnconfirmedSend-PDU" }	OPEN
S37	OPEN	UnconfirmedService.req && UnconfirmedServiceCheck = "True" && PDU length ≤ NegotiatedMaxPduSizeSending && RequestedServiceSupportedTest = "True" && ImmediateAcknowledge = "True" => UCA.req { FalApuBody := "UnconfirmedAcknowledgedSend-CommandPDU" } or UCA.req { FalApuBody := "UnconfirmedAcknowledgedSend-RequestPDU" }	OPEN
S38	OPEN	AR_ASE service request && ArServiceCheck = "True" && RequestedServiceSupportedTest = "True" => ArFspmService NOTE This state is provided to access the FSPM direct from the FAL User. The function ArFspmService generates an FSPM primitive as defined later in this subclause.	OPEN
S39	OPEN	Terminate.req => Abort.req { Originator := "TerminatIdentifier of Terminate.req", ReasonCode := "ReasonCode of Terminate.req" }, ResetArep	CLOSED
S40	OPEN	Faulty, unknown or not-allowed FAL service.primitive => Abort.req { AbortIdentifier := "FAL", ReasonCode := "User error" }, Terminate.ind { LocallyGenerated := "True", TerminatIdentifier := "FAL", ReasonCode := "User error" }, ResetArep	CLOSED

Table 19 (continued)

#	Current state	Event or condition => action	Next state
S54	OPEN	ConfirmedService.rsp && ConfirmedServiceCheck = "True" && InvokeldExistent = "True" && SameService = "True" && PDU length ≤ NegotiatedMaxPduSizeSending => CS.rsp { FalApduBody := "ConfirmedSend-AffirmativePDU" }, or CS.rsp { FalApduBody := "ConfirmedSend-NegativePDU" }, or CS.rsp { FalApduBody := "ConfirmedSend-ResponsePDU" }. OutstandingServicesRespondingCounter := OutstandingServicesRespondingCounter-1 NOTE An extra protocol means shall be provided to determine whether ConfirmedSend-AffirmativePDU or ConfirmedSend-NegativePDU is used.	OPEN
S55	OPEN	ConfirmedService.rsp && ConfirmedServiceCheck = "True" && InvokeldExistent = "False" => Abort.req { Originator := "FAL", ReasonCode := "Invokeld-error-response" }, Terminate.ind { LocallyGenerated := "True", Terminateldentifier := "FAL", ReasonCode := "Invokeld-error-response" }, ResetArep	CLOSED
S56	OPEN	ConfirmedService.rsp && ConfirmedServiceCheck = "True" && InvokeldExistent = "True" && SameService = "False" => Abort.req { Originator := "FAL", ReasonCode := "Service-error" }, Terminate.ind { LocallyGenerated := "True", Terminateldentifier := "FAL", ReasonCode := "Service-error" }, ResetArep	CLOSED

Table 20 – AP-Context state machine receiver transactions

#	Current state	Event or condition => action	Next state
R6	CLOSED	Abort.ind => (no actions taken)	CLOSED
R7	CLOSED	Faulty or unknown AR_FSPM service primitive => Abort.req { Originator := "FAL", ReasonCode := "AR_ASE error" }	CLOSED
R8	CLOSED	AR_FSPM service primitive <> "EST.ind" && AR_FSPM service.primitive <> "Abort.ind" => Abort.req { Originator:= "FAL", ReasonCode := "Connection State Conflict" }	CLOSED
R9	CLOSED	EST.ind && FalApduBody = not allowed, unknown or faulty FAL PDU => Abort.req { Originator := "FAL", ReasonCode := "FAL PDU error" }	CLOSED
R10	CLOSED	EST.ind && (FalApduBody = "Establish-CommandPDU" FalApduBody = "Establish-RequestPDU" FalApduBody = "Establish-Request2PDU") && ApContextTest = "True" && MaxFalPduLengthTest = "True" && ServicesSupportedTest = "True" => NegotiateOutstandingServices, Initiate.ind { }	RSP
R11	CLOSED	EST.ind && (FalApduBody = "Establish-CommandPDU" FalApduBody = "Establish-RequestPDU" FalApduBody = "Establish-Request2PDU") && ApContextTest = "False" => Abort.req { Originator := "FAL", ReasonCode := "AR error" }	CLOSED
R12	CLOSED	EST.ind && (FalApduBody = "Establish-CommandPDU" FalApduBody = "Establish-RequestPDU" FalApduBody = "Establish-Request2PDU") && ApContextTest = "True" && MaxFalPduLengthTest = "False" => EST.rsp(-) { FalApduBody := "Establish-NegativePDU", ErrorCode := "Max-Fal-Fdu-Size-Insufficient" } or EST.rsp(-) { FalApduBody := "Establish-ErrorPDU", ErrorCode := "Max-Fal-Fdu-Size-Insufficient" } }	CLOSED

Table 20 (continued)

#	Current state	Event or condition => action	Next state
R13	CLOSED	EST.ind && (FalApuBody = "Establish-CommandPDU" FalApuBody = "Establish-RequestPDU" FalApuBody = "Establish-Request2PDU") && ApContextTest = "True" && MaxFalPduLengthTest = "True" && ServicesSupportedTest = "False" => EST.rsp(-) { FalApuBody := "Establish-NegativePDU", ErrorCode := "Service-Not-Supported" } or EST.rsp(-) { FalApuBody := "Establish-ErrorPDU", ErrorCode := "Service-Not-Supported" }	CLOSED
R14	REQ	EST.cnf(+) && (FalApuBody = "Establish-AffirmativePDU" FalApuBody = "Establish-ResponsePDU") && ApExplicitConnection = "True" => Initiate.cnf(+) {	OPEN
R15	REQ	EST.cnf(+) && FalApuBody = "NULL" && ApExplicitConnection = "False" => Initiate.cnf(+) {	OPEN
R16	REQ	EST.cnf(-) && (FalApuBody = "Establish-NegativePDU" FalApuBody = "Establish-ErrorPDU") && ApExplicitConnection = "True" => "Initiate.cnf(-) { ErrorCode := "Errorcode of EST.cnf(-)" }, ResetArep	CLOSED
R17	REQ	EST.cnf(-) && FalApuBody = "NULL" && ApExplicitConnection = "False" => Initiate.cnf(-) { ErrorCode := "Errorcode in EST.cnf" }, ResetArep	CLOSED
R18	REQ	Abort.ind => Terminate.ind { LocallyGenerated := "LocallyGenerated of Abort.ind", Terminateldentifier := "Originator of Abort.ind", ReasonCode := "ReasonCode of Abort.ind" }, ResetArep	CLOSED

Table 20 (continued)

#	Current state	Event or condition => action	Next state
R20	REQ	Faulty or unknown AR_FSPM service primitive => Abort.req { Originator := "FAL", ReasonCode := "AR_ASE error" }, => Terminate.ind { LocallyGenerated := "True", Terminateldentifier := "FAL", ReasonCode := "AR_ASE error" }, ResetArep	CLOSED
R21	REQ	AR_FSPM service.primitive <> "EST.cnf" && AR_FSPM service.primitive <> "Abort.ind" => Abort.req { Originator := "FAL", ReasonCode := "Connection State Conflict" }, Terminate.ind { LocallyGenerated := "True", Terminateldentifier := "FAL", ReasonCode := "Connection State Conflict" }, ResetArep	CLOSED
R22	REQ	EST.cnf && FalA pduBody = "not allowed, unknown or faulty FAL PDU" => Abort.req { Abortldentifier := "FAL", ReasonCode := "FAL PDU error" }, Terminate.ind { LocallyGenerated := "True", Terminateldentifier := "FAL", ReasonCode := "FAL PDU error" }, ResetArep	CLOSED
R27	RSP	Abort.ind => Terminate.ind { LocallyGenerated := "LocallyGenerated of Abort.ind", Terminateldentifier := "Originator of Abort.ind", ReasonCode := "ReasonCode of Abort.ind" }, ResetArep	CLOSED
R28	RSP	Faulty or unknown AR_FSPM service primitive => Abort.req { Abortldentifier := "FAL", ReasonCode := "AR_ASE error" }, Terminate.ind { LocallyGenerated := "True", Terminateldentifier := "FAL", ReasonCode := "AR_ASE error" }, ResetArep	CLOSED

Table 20 (continued)

#	Current state	Event or condition => action	Next state
R29	RSP	AR_FSPM service primitive <> "Abort.ind" => Abort.req { Originator := "FAL", ReasonCode := "State Conflict with AR_ASE" }, Terminate.ind { LocallyGenerated := "True", Terminateldentifier := "AP_ASE", ReasonCode := "Connection State Conflict with AR_ASE" }, ResetArep	CLOSED
R41	OPEN	CS.ind && (FalAduBody = "ConfirmedSend-CommandPDU" FalAduBody = "ConfirmedSend-RequestPDU") && PDU length ≤ NegotiatedMaxPduSizeReceiving && OutstandingServicesRespondingCounter < NegotiatedMaxOutstanding- ServicesResponding && InvokeldExistent = "False" && IndicatedServiceSupportedTest = "True" => ConfirmedService.ind { }, OutstandingServicesRespondingCounter := OutstandingServicesRespondingCounter+1	OPEN
R42	OPEN	CS.ind && (FalAduBody = "ConfirmedSend-CommandPDU" FalAduBody = "ConfirmedSend-RequestPDU") && PDU length > NegotiatedMaxPduSizeReceiving => Abort.req { Originator := "FAL", ReasonCode := "PDU-size" }, Terminate.ind { LocallyGenerated := "True", Terminateldentifier := "FAL", ReasonCode := "PDU-size" }, ResetArep	CLOSED
R43	OPEN	CS.ind && (FalAduBody = "ConfirmedSend-CommandPDU" FalAduBody = "ConfirmedSend-RequestPDU") && PDU length ≤ NegotiatedMaxPduSizeReceiving && OutstandingServicesResponding ≥ NegotiatedMaxOutstanding- ServicesResponding => Abort.req { Originator := "FAL", ReasonCode := "Max-services-overflow" }, Terminate.ind { LocallyGenerated := "True", Terminateldentifier := "FAL", ReasonCode := "Max-Services-Overflow" }, ResetArep	CLOSED

Table 20 (continued)

#	Current state	Event or condition => action	Next state
R44	OPEN	CS.ind && (FalApduBody = "ConfirmedSend-CommandPDU" FalApduBody = "ConfirmedSend-RequestPDU") && PDU length ≤ NegotiatedMaxPduSizeReceiving && OutstandingServicesRespondingCounter < NegotiatedMaxOutstandingServicesResponding && InvokeldExistent = "True" => Abort.req { Originator := "FAL", ReasonCode := "Invokeld-error-request" }, Terminate.ind { LocallyGenerated := "True", Terminateldentifier := "FAL", ReasonCode := "Invokeld-error-request" }, ResetArep	CLOSED
R45	OPEN	CS.ind && (FalApduBody = "ConfirmedSend-CommandPDU" FalApduBody = "ConfirmedSend-RequestPDU") && PDU length ≤ NegotiatedMaxPduSizeReceiving && OutstandingServicesRespondingCounter < NegotiatedMaxOutstandingServicesResponding && InvokeldExistent = "False" && IndicatedServiceSupportedTest = "False" => Abort.req { Originator := "FAL", ReasonCode := "Feature-not-supported" }, Terminate.ind { LocallyGenerated := "True", Terminateldentifier := "FAL", ReasonCode := "Feature-not-supported" }, ResetArep	CLOSED
R46	OPEN	UCS.ind && (FalApduBody = "UnconfirmedSend-CommandPDU" FalApduBody = "UnconfirmedSend-PDU") && PDU length ≤ NegotiatedMaxPduSizeReceiving && IndicatedServiceSupportedTest = "True" => UnconfirmedService.ind { } }	OPEN
R47	OPEN	UCS.ind && (FalApduBody = "UnconfirmedSend-CommandPDU" FalApduBody = "UnconfirmedSend-PDU") && PDU length > NegotiatedMaxPduSizeReceiving => Abort.req { Originator := "FAL", ReasonCode := "PDU-size" }, Terminate.ind { LocallyGenerated := "True", Terminateldentifier := "FAL", ReasonCode := "PDU-size" }, ResetArep	CLOSED

Table 20 (continued)

#	Current state	Event or condition => action	Next state
R48	OPEN	UCS.ind && (FalAduBody = "UnconfirmedSend-CommandPDU" FalAduBody = "UnconfirmedSend-PDU") && PDU length ≤ NegotiatedMaxPduSizeReceiving && IndicatedServiceSupportedTest = "False" => Abort.req { Originator := "FAL", ReasonCode := "Feature-not-supported" }, Terminate.ind { LocallyGenerated := "True", Terminateldentifier := "FAL", ReasonCode := "Feature-not-supported" }, ResetArep	CLOSED
R49	OPEN	Abort.ind => Terminate.ind { LocallyGenerated := "LocallyGenerated of Abort.ind", Terminateldentifier := "Originator of Abort.ind", ReasonCode := "ReasonCode of Abort.ind" }, ResetArep	CLOSED
R50	OPEN	EST.ind && (FalAduBody = "Establish-CommandPDU" FalAduBody = "Establish-RequestPDU" FalAduBody = "Establish-Request2PDU") => Abort.req { Originator := "FAL", ReasonCode := "Connection-State-Conflict" }, Terminate.ind { LocallyGenerated := "True", Terminateldentifier := "FAL", ReasonCode := "Connection-State-Conflict" }, ResetArep	CLOSED
R51	OPEN	Faulty or unknown AR_FSPM service primitive => Abort.req { Originator := "FAL", ReasonCode := "AR_ASE error" }, Terminate.ind { LocallyGenerated := "True", Terminateldentifier := "FAL", ReasonCode := "AR_ASE error" }, ResetArep	CLOSED

Table 20 (continued)

#	Current state	Event or condition => action	Next state
R52	OPEN	Not-allowed AR_FSPM service primitive => Abort.req { Originator := "FAL", ReasonCode := "Connection-State-Conflict" }, Terminate.ind { LocallyGenerated := "True", Terminateldentifier := "FAL", ReasonCode := "Connection-State-Conflict" }, ResetArep	CLOSED
R53	OPEN	valid AR_FSPM Send Service primitive (one of CS, US, UCA) && FalApduBody = not-allowed, unknown or faulty FAL PDU && ArAccessSupported = "False" => Abort.req { Originator := "AP_ASE", ReasonCode := "FAL-PDU-error" }, Terminate.ind { LocallyGenerated := "True", Terminateldentifier := "FAL", ReasonCode := "FAL –PDU-error" }, ResetArep	CLOSED
R54	OPEN	UCA.ind && ArAccessSupported = "True" && (FalApduBody = "UnconfirmedAcknowledgedSend-CommandPDU" FalApduBody = "UnconfirmedAcknowledgedSend-RequestPDU") && PDU length ≤ NegotiatedMaxPduSizeReceiving => UnconfirmedService.ind { }	OPEN
R55	OPEN	UCA.ind && ArAccessSupported = "True" && (FalApduBody = "UnconfirmedAcknowledgedSend-CommandPDU" FalApduBody = "UnconfirmedAcknowledgedSend-RequestPDU") && PDU length > NegotiatedMaxPduSizeReceiving => Abort.req { Originator := "FAL", ReasonCode := "PDU-size" }, Terminate.ind { LocallyGenerated := "True", Terminateldentifier := "FAL", ReasonCode := "PDU-size" }, ResetArep	CLOSED
R56	OPEN	AR_ASE service indication => ArFspmService NOTE This state is provided to access the FSPM direct from the FAL User. The function ArFspmService generates an FSPM primitive as defined later in this subclause.	OPEN

Table 20 (continued)

#	Current state	Event or condition => action	Next state
R58	OPEN	CS.cnf && (FalAduBody = "ConfirmedSend-AffirmativePDU" FalAduBody = "ConfirmedSend-NegativePDU" FalAduBody = "ConfirmedSend-ResponsePDU") && PDU length ≤ NegotiatedMaxPduSizeReceiving && InvokeldExistent = "True" && SameService = "True" => ConfirmedService.cnf { }, OutstandingServicesRequestingCounter := OutstandingServicesRequestingCounter-1	OPEN
R59	OPEN	CS.cnf && (FalAduBody = "ConfirmedSend-AffirmativePDU" FalAduBody = "ConfirmedSend-NegativePDU" FalAduBody = "ConfirmedSend-ResponsePDU") && PDU length > NegotiatedMaxPduSizeReceiving => Abort.req { AbortIdentifier := "FAL", ReasonCode := "PDU-size" }, Terminate.ind { LocallyGenerated := "True", Terminateldentifier := "FAL", ReasonCode := "PDU-size" }, ResetArep	CLOSED
R60	OPEN	CS.cnf && (FalAduBody = "ConfirmedSend-AffirmativePDU" FalAduBody = "ConfirmedSend-NegativePDU" FalAduBody = "ConfirmedSend-ResponsePDU") && PDU length ≤ NegotiatedMaxPduSizeReceiving && InvokeldExistent = "False" => Abort.req { AbortIdentifier := "FAL", ReasonCode := "Invokeld-error-responding" }, Terminate.ind { LocallyGenerated := "True", Terminateldentifier := "FAL", ReasonCode := "Invokeld-error-responding" }, ResetArep	CLOSED

4.4.4 Functions

Table 21 through 36 define internal functions that are used by the AP-Context state machine.

Table 21 – Function ResetArep

Name	ResetArep		
Input	Arep_Id	Output	True or False
Function	Closes the AP AR Context and initializes all elements of the AP attribute <i>List Of In-Use AR Endpoint Info</i> to zero (0).		

Table 22 – Function ApContextTest

Name	ApContextTest		
Input	Arep_Id	Output	True or False
Function	Locates the entry for the Selected AREP in the AP attribute <i>List Of In-Use AR Endpoint Info</i> , verifies its contents, and ensures that there is a matching AREP defined for the AR_ASE. The way in which the verification is carried out is dependent on implementation.		

Table 23 – Function ServicesSupportedTest

Name	ServicesSupportedTest		
Input	Arep_Id	Output	True or False
Function	Upon receipt of an Initiate-RequestPDU the called AP_ASE shall compare the Local List of Supported Services against those contained in the Initiate PDU. The ServicesSupportedTest fails if either the Local FAL does not support all the services as a responder that the remote FAL supports as a requestor, or the remote FAL does not support all the services as a responder that the local FAL supports as a requestor.		

Table 24 – Function ApExplicitConnection

Name	ApExplicitConnection		
Input	Arep_Id	Output	True or False
Function	Refer to AREP to determine if this AP_ASE supports explicit connection between APs.		

Table 25 – Function ImmediateAcknowledge

Name	ImmediateAcknowledge		
Input	Arep_Id	Output	True or False
Function	Refer to AREP to determine if this AP_ASE requires immediate acknowledge in the underlying layer.		

Table 26 – Function ConfirmedServiceCheck

Name	ConfirmedServiceCheck		
Input	Arep_Id	Output	True or False
Function	Determines if the called FAL service is a confirmed service except for AR ASE services.		

Table 27 – Function UnconfirmedServiceCheck

Name	UnconfirmedServiceCheck		
Input	Arep_Id	Output	True or False
Function	Determines if the called FAL service is an unconfirmed service except AR ASE services.		

Table 28 – Function ArServiceCheck

Name	ArServiceCheck		
Input	Arep_Id	Output	True or False
Function	Determines if the called service is an AR ASE service (FAL service or AR FSPM service).		

Table 29 – Function ArFspmService

Name	ArFspmService		
Input	Arep_Id	Output	FAL Service or AR FSPM Service
Function	Generate an AR ASE service primitive. Relationship between FAL Services and AR FSPM Services shall be as follows:		
	AR-Unconfirmed Send	UCS	
	AR-Confirmed Send	CS	
	AR-Establish	EST	
	AR-Abort Abort		
	AR-Compel	FCMP	
	AR-Get Buffered Message	GBM	
	AR-Status	FSTS	
	AR-XON-OFF	AR_XON_OFF	
	AR-RemoteRead	RR	
	AR-RemoteWrite	RW	

Table 30 – Function ArAcceeSupported

Name	ArAcceeSupported		
Input	Arep_Id	Output	True or False
Function	Determines if the AP accepts AR ASE Send (ConfirmedSend or UnconfirmedSend) services.		

Table 31 – Function MaxFalPduLengthTest

Name	MaxFalPduLengthTest		
Input	Arep_Id	Output	True or False
Function	Upon receipt of an Initiate-RequestPDU, the called AP_ASE shall check the requested maximum PDU length against its defined context. The following tests are performed:		
	1. If (ConfiguredMaxPDUSizeSending > MaxPDUSizeReceiving of the Initiate PDU), this test fails;		
	2. If (ConfiguredMaxPDUSizeReceiving < MaxPDUSizeSending of the Initiate PDU), this test fails.		

Table 32 – Function NegotiateOutstandingServices

Name	NegotiateOutstandingServices		
Input	Arep_Id	Output	True or False
Function	Upon receipt of an Initiate-RequestPDU, the called AP_ASE shall perform the following negotiation:		
	1. If ConfiguredMaxOutstandingServicesRequesting > MaxOutstandingServicesResponding of the Initiate-RequestPDU then NegotiatedMaxOutstandingServicesRequesting = MaxOutstandingServicesResponding of the Initiate-RequestPDU else NegotiatedMaxOutstandingServicesRequesting = ConfiguredMaxOutstandingServicesRequesting;		
	2. If ConfiguredMaxOutstandingServicesResponding < MaxOutstandingServicesRequesting of the Initiate PDU then NegotiatedMaxOutstandingServicesResponding = ConfiguredMaxOutstandingServicesResponding else NegotiatedMaxOutstandingServicesResponding = MaxOutstandingServicesRequesting of the Initiate PDU;		

Table 33 – Function RequestedServicesSupportedTest

Name	RequestedServicesSupportedTest		
Input	Arep_Id, Service.primitive	Output	True or False
Function	Upon receipt of a service request from FAL user, the called AP_ASE shall compare the Local List of Supported Services against the requested service primitive. This test fails if the service primitive is not contained in the Local List of Supported Services.		

Table 34 – Function IndicatedServicesSupportedTest

Name	IndicatedServicesSupportedTest		
Input	Arep_Id, service.primitive	Output	True or False
Function	Upon receipt of a service indication from AR_ASE, the called AP_ASE shall compare the Local List of Supported Services against the indicated service primitive. This test fails if the service primitive is not contained in the Local List of Supported Services.		

Table 35 – Function InvokeIdExistent

Name	InvokeIdExistent		
Input	Arep_Id, Invoke_Id	Output	True or False
Function	Upon receipt of a service primitive, the called AP_ASE shall - compare the local list of invoke IDs in use against the requested Invoke ID if the service primitive is request; - compare the local list of invoke IDs in use against the responded invoke ID if the service primitive is response. This test fails if the invoke ID does not exist in the local list of Invoke IDs in use.		

Table 36 – Function SameService

Name	SameService		
Input	Arep_Id, Invoke_Id	Output	True or False
Function	Upon receipt of a service primitive the called AP_ASE shall - compare the local list of outstanding services (responding) against the service if the service primitive is response; - compare the local list of outstanding services (requesting) against the service if the service primitive is confirmation. This test fails if the service is not the same as that in the local list of outstanding services.		

4.5 FAL Service Protocol Machine (FSPM)

4.5.1 Summary

The FAL Service Protocol Machine is common to all the AREP types. Only applicable primitives are different among different AREP types. It has one state called "ACTIVE."

4.5.2 Primitive definitions

4.5.2.1 Primitives exchanged between AP-Context and FSPM

The primitives exchanged between the AP-Context and the FSPM are described in Table 37 and 38.

Table 37 – Primitives issued by AP-Context to FSPM

Primitive name	Source	Associated parameters	Functions
EST.req	AP-Context	Arep_Id, Data, Remote_DLCEP_Address	This primitive is used to convey an Establish request primitive from the AP-Context to the FSPM.
EST.rsp(+)	AP-Context	Arep_Id, Data	This primitive is used to convey an Establish response(+) primitive from the AP-Context to the FSPM.
EST.rsp(-)	AP-Context	Arep_Id, Data	This primitive is used to convey an Establish response(-) primitive from the AP-Context to the FSPM.
Abort.req	AP-Context	Arep_Id, Identifier, Reason_Code, Additional_Detail	This primitive is used to convey an Abort request primitive from the AP-Context to the FSPM.
CS.req	AP-Context	Arep_Id, Data	This primitive is used to convey a Confirmed Send (CS) request primitive from the AP-Context to the FSPM.
CS.rsp	AP-Context	Arep_Id, Data	This primitive is used to convey a Confirmed Send (CS) response primitive from the AP-Context to the FSPM.
UCS.req	AP-Context	Arep_Id, Remote_DLSAP_Address, Data	This primitive is used to convey an Unconfirmed Send (UCS) request primitive from the AP-Context to the FSPM.
FCMP.req	AP-Context	Arep_Id	This primitive is used to convey an FAL-Compel (FCMP) request primitive from the AP-Context to the FSPM.
GBM.req	AP-Context	Arep_Id	This primitive is used to convey a Get-Buffered-Message (GBM) request primitive from the AP-Context to the FSPM.
RW.req	AP-Context	Arep_Id, Data, Priority	This primitive is used to remotely write the contents of a buffer.
RR.req	AP-Context	Arep_Id, Priority	This primitive is used to get the contents of a remote buffer.
UCA.req	AP-Context	Arep_Id, Remote_Dlsap_Address, Data	This primitive is used to send an unconfirmed service request.
AR_XON_OFF.req	AP-Context	Arep_Id, Remote_Dlsap_Address, Data	This primitive is used to perform flow control.

Table 38 – Primitives issued by FSPM to AP-Context

Primitive name	Source	Associated parameters	Functions
EST.ind	FSPM	Arep_Id, Data	This primitive is used to convey an Establish indication primitive from the FSPM to the AP-Context.
EST.cnf(+)	FSPM	Arep_Id, Data	This primitive is used to convey an Establish result(+) primitive from the FSPM to the AP-Context.
EST.cnf(-)	FSPM	Arep_Id, Data	This primitive is used to convey an Establish result(-) primitive from the FSPM to the AP-Context.
Abort.ind	FSPM	Arep_Id, Locally_Generated_Identifier, Reason_Code, Additional_Detail	This primitive is used to convey an Abort indication primitive from the FSPM to the AP-Context.
CS.ind	FSPM	Arep_Id, Data	This primitive is used to convey a Confirmed Send (CS) indication primitive from the FSPM to the AP-Context.
CS.cnf	FSPM	Arep_Id, Data	This primitive is used to convey a Confirmed Send (CS) confirmation primitive from the FSPM to the AP-Context.
UCS.ind	FSPM	Arep_Id, Remote_DLSAP_Address, Duplicate_FAL-SDU, Data, Local_Timeliness, Remote_Timeliness	This primitive is used to convey an Unconfirmed Send (UCS) indication primitive from the FSPM to the AP-Context.
FCMP.cnf	FSPM	Arep_Id, Status	This primitive is used to convey an FAL-Compel (FCMP) confirmation primitive from the FSPM to the AP-Context.
GBM.cnf(+)	FSPM	Arep_Id, Duplicate_FAL-SDU, Data, Local_Timeliness, Remote_Timeliness	This primitive is used to convey a Get-Buffered-Message (GBM) positive confirmation primitive from the FSPM to the AP-Context.
GBM.cnf(-)	FSPM	Arep_Id	This primitive is used to convey a Get-Buffered-Message (GBM) negative confirmation primitive from the FSPM to the AP-Context.
FSTS.ind	FSPM	Arep_Id, Reported_Status	This primitive is used to convey an FAL-Status (FSTS) indication primitive from the FSPM to the AP-Context.
UCA.ind	FSPM	Arep_Id, Remote_DLSAP_Address, Duplicate_FAL-SDU, Data	This primitive is used to convey an unconfirmed service indication.
AR_XON_OFF.ind	FSPM	Arep_Id, Remote_DLSAP_Address, Data	This primitive is used to convey a flow control indication.
RW.cnf(+)	FSPM	Arep_Id, Status	This primitive is used to convey a Remote_Write (RW) positive confirmation primitive.
RW.cnf(-)	FSPM	Arep_Id, Status	This primitive is used to convey a Remote_Write (RW) negative confirmation primitive.
RR.cnf(+)	FSPM	Arep_Id, Data, Local_Timeliness, Remote_Timeliness	This primitive is used to convey a Remote_Read (RR) positive confirmation primitive.
RR.cnf(-)	FSPM	Arep_Id, Status	This primitive is used to convey a Remote_Read (RR) negative confirmation primitive.

4.5.2.2 Parameters of AP-Context /FSPM primitives

All the parameters used in the primitives exchanged between the AP-Context and the FSPM are identical to those defined in the “Operational Services” subclause.

4.5.3 FSPM state tables

4.5.3.1 General

The FSPM state machines are described in Figure 27, Table 39 and Table 40.

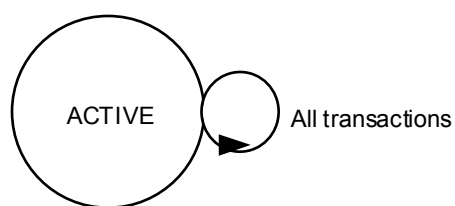


Figure 27 – State transition diagram of FSPM

Table 39 – FSPM state table – Sender transactions

#	Current state	Event or condition => action	Next state
S1	ACTIVE	EST.req && SelectArep (Arep_Id) = "True" => EST_req { user_data := Data, remote_dlcep_address := Remote_DLCEP_Address }	ACTIVE
S2	ACTIVE	EST.rsp(+) && SelectArep (Arep_Id) = "True" => EST_rsp(+) { user_data := Data }	ACTIVE
S3	ACTIVE	EST.rsp(-) && SelectArep (Arep_Id) = "True" => EST_rsp(-) { user_data := Data }	ACTIVE
S4	ACTIVE	UCS.req && SelectArep (Arep_Id) = "True" => UCS_req { remote_dlsap_address := Remote_DLSAP_Address, user_data := Data }	ACTIVE
S5	ACTIVE	CS.req && SelectArep (Arep_Id) = "True" => CS_req { user_data := Data }	ACTIVE
S6	ACTIVE	CS.rsp && SelectArep (Arep_Id) = "True" => CS_rsp { user_data := Data }	ACTIVE
S7	ACTIVE	Abort.req && SelectArep (Arep_Id) = "True" => Abort_req { identifier := Identifier, reason_code := Reason_Code, additional_detail := Additional_Detail }	ACTIVE
S8	ACTIVE	FCMP.req && SelectArep (Arep_Id) = "True" => FCMP_req { }	ACTIVE
S9	ACTIVE	GBM.req && SelectArep (Arep_Id) = "True" => GBM_req { }	ACTIVE

Table 39 (continued)

#	Current state	Event or condition => action	Next state
S10	ACTIVE	<pre> RW.req && SelectArep(Arep_Id) = "True" => RW_req{ user_data := Data, priority := Priority } </pre>	ACTIVE
S11	ACTIVE	<pre> RR.req && SelectArep(Arep_Id) = "True" => RR_req { priority := Priority } </pre>	ACTIVE
S12	ACTIVE	<pre> UCA.req && SelectArep (Arep_Id) = "True" => UCA_req { remote_dlsap_address := Remote_DLSAP_Address, user_data := Data } </pre>	ACTIVE
S13	ACTIVE	<pre> AR_XON_OFF.req && SelectArep (Arep_Id) = "True" => AR_XON_OFF_req { remote_dlsap_address := Remote_DLSAP_Address, user_data := Data } </pre>	ACTIVE
S14	ACTIVE	<pre> Any FSPM.req && SelectArep (Arep_Id) = "False" => (no actions defined by the protocol, see notes 1 and 2.) </pre>	ACTIVE
NOTE 1 A primitive generated in the FSPM sender state machine is sent to an appropriate ARPM that is selected by the FSPM using the SelectArep function. The Arep_Id parameter supplied by the AP-Context is the argument of this function. NOTE 2 If the SelectArep function returns the value of False, it is a local matter to report such instance and the FSPM does not generate any primitives for the ARPM.			

Table 40 – FSPM state table – receiver transactions

#	Current state	Event or condition => action	Next state
R1	ACTIVE	EST_ind => EST.ind { Arep_Id := arep_id, Data := user_data }	ACTIVE
R2	ACTIVE	EST_cnf(+) => EST.cnf(+) { Arep_Id := arep_id, Data := user_data }	ACTIVE
R3	ACTIVE	EST_cnf(-) => EST.cnf(-) { Arep_Id := arep_id, Data := user_data }	ACTIVE
R4	ACTIVE	UCS_ind => UCS.ind { Arep_Id := arep_id, Remote_DLSAP_Address := remote_dlsap_address, Duplicate_FAL-SDU := duplicate_fal_sdu, Data := user_data, Local_Timeliness := local_timeliness, Remote_Timeliness := remote_timeliness }	ACTIVE
R5	ACTIVE	CS_ind => CS.ind { Arep_Id := arep_id, Data := user_data }	ACTIVE
R6	ACTIVE	CS_cnf => CS.cnf { Arep_Id := arep_id, Data := user_data }	ACTIVE
R7	ACTIVE	Abort_ind => Abort.ind { Arep_Id := arep_id, Locally_Generated := locally_generated, Identifier := identifier, Reason_Code := reason_code, Additional_Detail := additional_detail }	ACTIVE
R8	ACTIVE	FCMP_cnf => FCMP.cnf { Arep_Id := arep_id, Status := status }	ACTIVE
R9	ACTIVE	GBM_cnf(+) => GBM.cnf(+) { Arep_Id := arep_id, Duplicate_FAL-SDU := duplicate_fal_sdu, Data := user_data, Local_Timeliness := local_timeliness, Remote_Timeliness := remote_timeliness }	ACTIVE

Table 40 (continued)

#	Current state	Event or condition => action	Next state
R10	ACTIVE	GBM_cnf(-) => GBM.cnf(-) { Arep_Id := arep_id, }	ACTIVE
R11	ACTIVE	FSTS_ind => FSTS.ind { Arep_Id := arep_id, Reported_Status := reported_status }	ACTIVE
R12	ACTIVE	RW_cnf(-) => RW.cnf(-) { Arep_Id := arep_id, Status := status }	ACTIVE
R13	ACTIVE	RW_cnf(+) => RW.cnf(+) { Arep_Id := arep_id, Status := status }	ACTIVE
R14	ACTIVE	RR_cnf(-) => RR.cnf(-) { Arep_Id := arep_id, Status := status }	ACTIVE
R15	ACTIVE	RR_cnf(+) => RR.cnf(+) { Arep_Id := arep_id, Data := user_data, Local_Timeliness := local_timeliness, Remote_Timeliness := remote_timeliness }	ACTIVE
R16	ACTIVE	UCA_ind => UCA.ind { Arep_Id := arep_id, Remote_DLSAP_Address := remote_dlsap_address, Duplicate_FAL-SDU := duplicate_fal_sdu, Data := user_data, }	ACTIVE
R17	ACTIVE	AR_XON_OFF_ind => AR_XON_OFF.ind { Arep_Id := arep_id, Remote_DLSAP_Address := remote_dlsap_address, Data := user_data, }	ACTIVE

4.5.3.2 Functions

The function used in this state machine is as shown in Table 41.

Table 41 – Function SelectArep

Name	SelectArep	Used in	FSPM
Input		Output	
Arep_Id		True False	
Function	Looks for the AREP entry that is specified by the Arep_Id parameter. The Arep_Id parameter is provided with the AP-Context service primitives.		

4.6 Application Relationship Protocol Machines (ARPMs)

4.6.1 Queued User-triggered Unidirectional (QUU) ARPM

4.6.1.1 Primitive definitions

4.6.1.1.1 Primitives exchanged between ARPM and FSPM

The primitives exchanged between the ARPM and the FSPM are listed in Table 42 and 43.

Table 42– Primitives issued by FSPM to ARPM

Primitive name	Source	Associated parameters	Functions
EST_req	FSPM	user_data	This is an FAL internal primitive used to convey an Establish request primitive from the FSPM to the ARPM.
Abort_req	FSPM	identifier, reason_code, additional_detail	This is an FAL internal primitive used to convey an Abort request primitive from the FSPM to the ARPM.
UCS_req	FSPM	remote_dlsap_address, user_data	This is an FAL internal primitive used to convey an Unconfirmed Send (UCS) request primitive from the FSPM to the ARPM.

Table 43 – Primitives issued by ARPM to FSPM

Primitive name	Source	Associated parameters	Functions
EST_cnf(+)	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish response(+) primitive from the ARPM to the FSPM.
Abort_ind	ARPM	arep_id, locally_generated, identifier, reason_code, additional_detail	This is an FAL internal primitive used to convey an Abort primitive from the ARPM to the FSPM.
UCS_ind	ARPM	arep_id, remote_dlsap_address, user_data	This is an FAL internal primitive used to convey an Unconfirmed Send (UCS) indication primitive from the ARPM to the FSPM.
FSTS_ind	ARPM	arep_id, reported_status	This is an FAL internal primitive used to convey confirmation status.

4.6.1.1.2 Parameters of FSPM/ARPM primitives

The parameters used with the primitives exchanged between the FSPM and the ARPM are described in Table 44.

Table 44 – Parameters used with primitives exchanged between FSPM and ARPM

Parameter name	Description
Arep_id	This parameter is used to unambiguously identify an instance of the AREP that has issued a primitive. A means for such identification is not specified by this standard.
User_data	This parameter conveys FAL-User data.
Locally_generated	This parameter conveys value that is used for the Locally_Generated parameter.
Identifier	This parameter conveys value that is used for the Identifier parameter.
Reason_code	This parameter conveys value that is used for the Reason_Code parameter.
Additional_detail	This parameter conveys value that is used for the Additional_Detail parameter.
Remote_dlsap_address	This parameter conveys value that is used for the Remote_DLSAP_Address parameter.

4.6.1.2 DLL mapping of QUU AREP class

4.6.1.2.1 QUU AREP class formal model

This subclause describes the mapping of the QUU AREP Class to the Fieldbus Data Link Layer. It does not redefine the DLSAP attributes or DLME attributes that are or will be defined in the Data link layer standard; rather, it defines how they are used by this AR class.

The DLL mapping attributes and their permitted values and the DLL services used with the QUU AREP class are defined in this subclause.

CLASS:		QuuCI
PARENT CLASS:		QueuedUser-TriggeredUnidirectionalAREP
ATTRIBUTES:		
1	(m) KeyAttribute:	LocalDisapAddress
2	(c) Constraint:	RemoteAddressConfigurationType = Linked
2.1	(m) Attribute:	RemoteDisapAddress
3	(m) Attribute:	LocalDisapRole (Basic, Group)
4	(c) Constraint:	Role = ReportSource
4.1	(m) Attribute:	DefaultQosAsSender
4.1.1	(m) Attribute:	DIIPriority (Urgent, Normal, TimeAvailable)
4.1.2	(m) Attribute:	MaxConfirmDelayOnUnitdata
4.1.3	(m) Attribute:	DlpduAuthentication (Ordinary, Source, Maximal)
4.1.4	(m) Attribute:	DISchedulingPolicy (Implicit)
4.1.5	(o) Attribute:	DleRemoteConf (True, False)
5	(m) Attribute:	ExplicitQueue (True, False)
6	(c) Constraint:	ExplicitQueue = True
6.1	(m) Attribute:	QueueBindings—either for a sender or a receiver
6.1.1	(m) Attribute:	MaxQueueDepth
6.1.2	(m) Attribute:	MaxDisduSize
DLL SERVICES:		
1	(m) OpsService:	DL-Unitdata
2	(c) Constraint:	ExplicitQueue = True
3	(m) OpsService:	DL-Get

4.6.1.2.2 Attributes

4.6.1.2.2.1 LocalDisapAddress

This attribute specifies the DLSAP address to which this AREP is attached. This attribute is a DLSAP-address if the Role attribute has the value of ReportSource, and either a group DL-address or a DLSAP-address if the Role attribute has the value of ReportSink.

This attribute supplies the value for the “DL(SAP)-address” parameter specified in the DLL.

This attribute contains the following three subattributes: Link Address, Node Address, and Selector.

4.6.1.2.2.2 RemoteAddressConfigurationType

This attribute specifies how this AREP is used with the remote AREP. The value of “Free” means that this AREP can communicate with any remote AREP. The value of “Linked” means that this AREP can only communicate with the AREP specified by the RemoteDisapAddress attribute.

4.6.1.2.2.3 RemoteDisapAddress

This attribute specifies the remote address to which FAL-PDUs are sent (for ReportSource AREPs), or from which they are received (for ReportSink AREPs).

If the RemoteAddressConfigurationType attribute is Linked, the value of this attribute has been configured. If it is Free, the value of this attribute is provided with a service request.

This attribute contains the following three subattributes: Link Address, Node Address, and Selector.

4.6.1.2.2.4 LocalDisapRole

This attribute specifies the behavior of the local DLSAP to be used. If the Role is ReportSource, this attribute has the value of Basic. If the Role is ReportSink, it has the value of either Basic or Group.

This attribute supplies the value for the “DL(SAP)-role” parameter specified by the DLL.

4.6.1.2.2.5 DefaultQosAsSender

The DefaultQosAsSender attributes specify the DLL quality of service that is used by the sending AREP. The receiving DLL shall support the quality of service specified by these attributes.

4.6.1.2.2.6 DllPriority

This attribute defines the DLL priority, and thus restricts the maximum length of an FAL-PDU, of the conveyance path of an AR.

This attribute supplies the value for the “DLL priority” parameter of the DLL. The values Urgent, Normal, and Time-Available correspond to URGENT, NORMAL, and TIME-AVAILABLE as defined in the Fieldbus Data Link Layer standard, respectively.

NOTE It is not possible to use different priorities for each FAL-PDU sent from the same QUU AREP.

4.6.1.2.2.7 MaxConfirmDelayOnUnitdata

This attribute specifies the maximum confirmation delay for a local confirmation from a DL-Unitdata request primitive.

This attribute supplies the value for the “Max confirm delay on locally-confirmed DL-Unitdata” parameter of the DLL.

4.6.1.2.2.8 DlpduAuthentication

This attribute specifies the lower bound of the length of the DL-addresses to be used by the DLL.

This attribute supplies the value for the “DLPDU authentication” parameter of the DLL. The values Ordinary, Source, and Maximal correspond to ORDINARY, SOURCE, and MAXIMAL as defined in the Fieldbus Data link layer standard, respectively.

4.6.1.2.2.9 DISchedulingPolicy

This attribute provides the guidance to the DLL on the scheduling needed by an AR. For this AREP, the DLL tries to transmit the FAL-PDU as soon as it is passed from the FAL.

This attribute supplies the value for DL-Scheduling-policy attribute. Only the value Implicit is used. It corresponds to IMPLICIT as defined in the Fieldbus Data link layer standard.

4.6.1.2.2.10 DleRemoteConf

This optional attribute specifies, when it is present and true, that the remote immediate DLL acknowledgement shall be used. If it is not present, the remote immediate DLL acknowledgement shall not be used.

4.6.1.2.2.11 ExplicitQueue

This attribute specifies, when True, that the characteristics of the associated sending and receiving queues are explicitly configured and managed through the Network Management. The value of False means that queues with implementation specific depth and length are provided by the DLL.

4.6.1.2.2.12 QueueBindings

The following attributes specify the explicit queue that is bound to this DLSAP. For a sender, the queue is for sending. For a receiver, it is for receiving.

4.6.1.2.2.13 MaxQueueDepth

This attribute specifies the maximum number of FAL-PDUs that can be queued for sending or receiving.

This attribute supplies the value for the “Maximum queue depth” parameter of the DLL.

4.6.1.2.2.14 MaxDisduSize

This attribute specifies the maximum length of an FAL-PDU that can be sent or received by the DLL.

This attribute supplies the value for the “Maximum DLSDU size” parameter of the DLL.

4.6.1.2.3 DLL services

Refer to 4.7.4, Data Link Layer Service Selection, for DLL service descriptions.

4.6.1.3 QUU ARPM state machine

4.6.1.3.1 QUU ARPM states

The defined states and their descriptions of the QUU ARPM are shown in Table 45 and Figure 28.

Table 45 – QUU ARPM states

State	Description
CLOSED	The AREP is defined, but not capable of sending or receiving FAL-PDUs. It may send or receive Establish service FAL-PDUs while in this state.
OPEN	The AREP is defined and capable of sending or receiving FAL-PDUs.

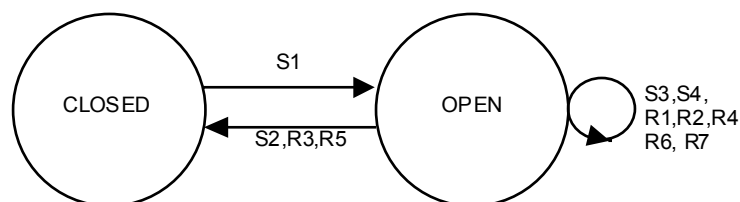


Figure 28 – State transition diagram of the QUU ARPM

4.6.1.3.2 QUU ARPM state table

Table 46 and Table 47 define the QUU ARPM state machine.

Table 46 – QUU ARPM state table – sender transactions

#	Current state	Event or condition => action	Next state
S1	CLOSED	EST_req => EST_cnf(+) { arep_id := GetArepld (), user_data := "null" }	OPEN
S2	OPEN	Abort_req => (no actions taken)	CLOSED
S3	OPEN	UCS_req && RemoteAddressConfigurationType = "Linked" && Role = "ReportSource" => FAL-PDU_req { dmpm_service_name := "DMPM_Unitdata_req", arep_id := GetArepld (), dlsdu := BuildFAL-PDU (fal_pdu_name := "UCS_PDU", fal_data := user_data) } }	OPEN
S4	OPEN	UCS_req && RemoteAddressConfigurationType = "Free" && Role = "ReportSource" => RemoteDisapAddress := remote_dlsap_address, FAL-PDU_req { dmpm_service_name := "DMPM_Unitdata_req", arep_id := GetArepld (), dlsdu := BuildFAL-PDU (fal_pdu_name := "UCS_PDU", fal_data := user_data) } }	OPEN

Table 47 – QUU ARPM state table – receiver transactions

#	Current state	Event or condition => action	Next state
R1	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Unitdata_ind" && RemoteAddressConfigurationType = "Linked" && RemoteDisapAddress = calling_address && Role = "ReportSink" && FAL_Pdu_Type (fal_pdu) = "UCS_PDU" => UCS_ind { arep_id := GetArepId (), user_data := fal_pdu }	OPEN
R2	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Unitdata_ind" && RemoteAddressConfigurationType = "Free" && Role = "ReportSink" && FAL_Pdu_Type (fal_pdu) = "UCS_PDU" => UCS_ind { arep_id := GetArepId (), remote_disap_address := calling_address, user_data := fal_pdu }	OPEN
R3	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Unitdata_ind" && Role = "ReportSink" && FAL_Pdu_Type (fal_pdu) <> "UCS_PDU" => Abort_ind { arep_id := GetArepId (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU", additional_detail := "null" }	CLOSED
R4	OPEN	ErrorToARPM => (no actions taken)	OPEN
R5	OPEN	FAL-PDU_ind && Role = "ReportSource" => Abort_ind { arep_id := GetArepId (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid Event for Role", additional_detail := "null" }	CLOSED
R6	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_unidata_cnf" && Role = "ReportSource" && Fal_Pdu_Type(fal_pdu) = "UCS_PDU" && reason = "success" => FSTS_Ind{ arep_id := GetArep(), reported_status := "dl_unidata_cnf" }	OPEN
R7	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_unidata_cnf" && Role = "ReportSource" && Fal_Pdu_Type(fal_pdu) = "UCS_PDU" && reason <> "success" => FSTS_Ind{ arep_id := GetArep(), reported_status := "unidata_cnf NOACK" }	OPEN

4.6.1.3.3 Functions used by QUU ARPM

Table 48 through Table 50 define the functions used by this state machine.

Table 48 – Function GetArepld ()

Name	GetArepld ()	Used in	ARPM
Input		Output	
(none)		AREP Identifier	
Function	Returns a value that can unambiguously identify the current AREP.		

Table 49 – Function BuildFAL-PDU

Name	BuildFAL-PDU	Used in	ARPM
Input		Output	
fal_pdu_name, fal_data		dlsdu	
Function	Builds an FAL-PDU out of the parameters given as input variables.		

Table 50 – Function FAL_Pdu_Type

Name	FAL_Pdu_Type	Used in	ARPM
Input		Output	
fal_pdu		One of the FAL-PDU types defined in the FAL-PDUs subclause.	
Function	This function decodes the FAL-PDU that is conveyed in the fal_pdu parameter and retrieves one of the FAL-PDU types.		

4.6.2 Queued User-triggered Bidirectional-Connection Oriented (QUB-CO) ARPM

4.6.2.1 Primitive definitions

4.6.2.1.1 Primitives exchanged between ARPM and FSPM

The primitives exchanged between the ARPM and the FSPM are listed in Table 51 and Table 52.

Table 51 – Primitives issued by FSPM to ARPM

Primitive name	Source	Associated parameters	Functions
EST_req	FSPM	user_data, remote_dlcep_address	This is an FAL internal primitive used to convey an Establish request primitive from the FSPM to the ARPM.
EST_rsp(+)	FSPM	user_data	This is an FAL internal primitive used to convey an Establish response(+) primitive from the FSPM to the ARPM.
EST_rsp(-)	FSPM	user_data	This is an FAL internal primitive used to convey an Establish response(-) primitive from the FSPM to the ARPM.
Abort_req	FSPM	identifier, reason_code, additional_detail	This is an FAL internal primitive used to convey an Abort request primitive from the FSPM to the ARPM.
CS_req	FSPM	user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) request primitive from the FSPM to the ARPM.
CS_rsp	FSPM	user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) response primitive from the FSPM to the ARPM.

Table 52 – Primitives issued by ARPM to FSPM

Primitive name	Source	Associated parameters	Functions
EST_ind	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish indication primitive from the ARPM to the FSPM.
EST_cnf(+)	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish response(+) primitive from the ARPM to the FSPM.
EST_cnf(-)	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish response(-) primitive from the ARPM to the FSPM.
Abort_ind	ARPM	arep_id, locally_generated, identifier, reason_code, additional_detail	This is an FAL internal primitive used to convey an Abort primitive from the ARPM to the FSPM.
CS_ind	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) indication primitive from the ARPM to the FSPM.
CS_cnf	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) confirmation primitive from the ARPM to the FSPM.

4.6.2.1.2 Parameters of FSPM/ARPM primitives

The parameters used with the primitives exchanged between the FSPM and the ARPM are described in Table 53.

Table 53 – Parameters used with primitives exchanged between FSPM and ARPM

Parameter name	Description
arep_id	This parameter is used to unambiguously identify an instance of the AREP that has issued a primitive. A means for such identification is not specified by this standard.
user_data	This parameter conveys FAL-User data.
locally_generated	This parameter conveys value that is used for the Locally_Generated parameter.
identifier	This parameter conveys value that is used for the Identifier parameter.
reason_code	This parameter conveys value that is used for the Reason_Code parameter.
additional_detail	This parameter conveys value that is used for the Additional_Detail parameter.
reported_status	This parameter conveys a Data Link Layer event status.
Remote_dlcep_address	This parameter conveys value of the remote dlcep address.

4.6.2.2 DLL mapping of QUB AREP class

4.6.2.2.1 QUU-CO AREP class formal model

This subclause describes the mapping of the QUB-CO AREP Class to the Fieldbus Data Link Layer. It does not redefine the DLSAP attributes or DLME attributes that are or will be defined in the Data link layer standard; rather, it defines how they are used by this AR class.

The DLL Mapping attributes and their permitted values and the DLL services used with the QUB-CO AREP class are defined in this subclause.

CLASS: QubCo

PARENT CLASS: QueuedUser-TriggeredBidirectionalAREP

ATTRIBUTES:

- 1 (m) KeyAttribute: LocalDlcepAddress
- 2 (c) Constraint: RemoteAddressConfigurationType = Linked
- 2.1 (m) Attribute: RemoteDlcepAddress
- 3 (c) Constraint: Role = Client || Peer
- 3.1 (m) Attribute: Initiator (True, False)
- 4 (m) Attribute: DisapRole (Basic)
- 5 (m) Attribute: QosParameterSet
- 5.1 (m) Attribute: DlcepClass (Peer)
- 5.2 (m) Attribute: DlcepDataDeliveryFeatures
- 5.2.1 (m) Attribute: FromRequestorToResponder (Classical, Disordered)

5.2.2	(m) Attribute:	FromResponderToRequestor (Classical, Disordered)
5.3	(m) Attribute:	Priority
5.3.1	(m) Attribute:	DIIPriority (Urgent, Normal, TimeAvailable)
5.3.2	(m) Attribute:	DIIPriorityNegotiated (Urgent, Normal, TimeAvailable)
5.4	(m) Attribute:	DlpduAuthentication (Ordinary, Source, Maximal)
5.5	(m) Attribute:	ResidualActivity
5.5.1	(m) Attribute:	ResidualActivityAsSender (True, False)
5.5.2	(m) Attribute:	ResidualActivityAsReceiver (True, False)
5.6	(m) Attribute:	MaxConfirmDelay
5.6.1	(m) Attribute:	MaxConfirmDelayOnDIConnect
5.6.2	(m) Attribute:	MaxConfirmDelayOnDIData
5.7	(m) Attribute:	DISchedulingPolicy (Implicit)
5.8	(m) Attribute:	ExplicitQueue (True, False)
5.9	(c) Constraint:	ExplicitQueue = True
5.9.1	(m) Attribute:	MaxDlsduSizes
5.9.1.1	(m) Attribute:	MaxDlsduSizeFromRequestor
5.9.1.2	(m) Attribute:	MaxDlsduSizeFromResponder
5.9.1.3	(m) Attribute:	MaxDlsduSizeFromRequestorNegotiated
5.9.1.4	(m) Attribute:	MaxDlsduSizeFromResponderNegotiated
5.9.2	(m) Attribute:	MaxQueueDepth
5.9.2.1	(m) Attribute:	MaxSendingQueueDepth
5.9.2.2	(m) Attribute:	MaxReceivingQueueDepth

DLL SERVICES:

1	(m) OpsService:	DL-Data
2	(c) Constraint:	ExplicitQueue = True
2.1	(m) OpsService:	DL-Get
3	(m) OpsService:	DL-Connect
4	(m) OpsService:	DL-Connection-Established
5	(m) OpsService:	DL-Disconnect

4.6.2.2.2 Attributes**4.6.2.2.2.1 LocalDlcepAddress**

This attribute specifies the local DLCEP address and identifies the DLCEP.

The value of this attribute is used as the “DLCEP-address” parameter of the DLL.

NOTE 1 The value of this attribute is also carried in the Establish Request PDU.

This attribute contains the following three subattributes: Link Address, Node Address, and Selector.

NOTE 2 Since the local Link and Node addresses are set by the Network Management, only the Selector portion of the LocalDlcepAddress attribute is a configurable attribute of the FAL.

4.6.2.2.2.2 RemoteAddressConfigurationType

This attribute specifies how this AREP is used with the remote AREP. The value of “Free” means that this AREP can communicate with any remote AREP. The value of “Linked” means that this AREP can only communicate with the AREP specified by the RemoteDlcepAddress attribute.

4.6.2.2.2.3 RemoteDlcepAddress

This attribute specifies the remote DLCEP address and identifies the DLCEP.

This attribute supplies the value for called DLCEP-address of the DL-Connect service.

NOTE The value of this attribute is also carried in the header part of the Establish Request PDU.

This attribute contains the following three subattributes: Link Address, Node Address, and Selector.

4.6.2.2.2.4 Role

This attribute specifies possible roles of this AREP. The value of “Client” means that this AREP issues FAL request PDUs to a remote AREP whose Role is either “Server” or “Peer.” An AREP whose Role is “Server” is a mirror of “Client.” An AREP whose Role is “Peer” can act as a “Client” and a “Peer” simultaneously.

4.6.2.2.2.5 Initiator

This attribute specifies whether this AREP is capable of issuing a DL-Connect request or not. The value of “True” means it is capable of doing it. If the value of this attribute is “False,” it can only accept a DL-Connect indication from a remote AREP.

4.6.2.2.2.6 DisapRole

This attribute specifies the behavior of the DLSAP that is used by the AREP.

This attribute supplies the value for the “DL(SAP)-role” parameter. The possible value Basic corresponds to BASIC as defined in the Data link layer standard.

4.6.2.2.2.7 QosParameterSet

The QosParameterSet attributes specify the DL quality of service that is used by this AREP. These attribute values may be negotiated with the remote AREP.

4.6.2.2.2.8 DlcepClass

This attribute specifies the behavior of the DLCEP which is attached to the AREP.

This attribute supplies the value for the “DLCEP class” parameter of the DLL. The possible value of this attribute is Peer and corresponds to Peer defined by the DLL.

4.6.2.2.2.9 DlcepDataDeliveryFeatures

These two attributes specify data delivery features of the DLL.

The permitted values Classical and Disordered correspond respectively to CLASSICAL and DISORDERED defined by the Data link layer standard.

The FromRequestorToResponder and FromResponderToRequestor attributes shall have the same value.

4.6.2.2.2.10 FromRequestorToResponder

This attribute specifies the DLL data delivery feature of the DLPDUs sent from the AREP whose Initiator attribute has a value of “True” to the remote AREP. It supplies the value for the “DLCEP data delivery features from requestor to responder(s)” parameter defined in the DLL.

4.6.2.2.2.11 FromResponderToRequestor

This attribute specifies the DLL data delivery feature of the DLPDUs sent from the AREP whose Initiator attribute has a value of “False” to the remote AREP. It supplies the value for the “DLCEP data delivery features from responder(s) to requestor” parameter defined in the DLL.

4.6.2.2.2.12 DllPriority

This attribute specifies the configured value of the DLL priority.

NOTE It is not possible to use different priorities for each FAL-PDU sent from the same QUB-CO AREP. Also, it is not permitted to use different priorities for the send and receive conveyance paths.

4.6.2.2.2.13 DllPriorityNegotiated

This attribute specifies the negotiated value of the DLL priority.

4.6.2.2.2.14 DlpduAuthentication

This attribute specifies the lower bound of the length of DL-addressing to be used by the DLL.

This attribute supplies the value for the “DLPDU-authentication” parameter of the DLL. The permitted value Ordinary, Source, and Maximal correspond to ORDINARY, Source, and MAXIMAL, respectively, as defined in the Fieldbus Data link layer standard.

4.6.2.2.2.15 ResidualActivityAsSender

This attribute specifies sender’s DLC residual activity. It supplies the value for the “Residual activity as sender” parameter defined in the DLL. The possible values are “True” and “False.”

4.6.2.2.2.16 ResidualActivityAsReceiver

This attribute specifies receiver’s DLC residual activity. It supplies the value for the “Residual activity as receiver” parameter defined in the DLL. The possible values are “True” and “False.”

4.6.2.2.2.17 MaxConfirmDelay

This attribute specifies the maximum confirmation delay of certain DLL connection-oriented services.

4.6.2.2.2.18 MaxConfirmDelayOnDIConnect

This attribute specifies the maximum confirmation delay for a confirmation from a DL-Connect service.

This attribute supplies the value for the “Maximum confirmation delay on DL-Connect, DL-Reset and DL-Subscriber-Query” parameter specified in the Data link layer standard.

4.6.2.2.2.19 MaxConfirmDelayOnDIData

This attribute specifies the maximum confirmation delay for a confirmation from a DL-Data service.

This attribute supplies the value for the “Maximum confirmation delay on DL-Data” parameter specified in the Data link layer standard (IEC 61158-3).

4.6.2.2.2.20 DISchedulingPolicy

This attribute provides the guidance to the DLL on the scheduling needed by an AR. For this AREP, the DLL tries to transmit the FAL-PDU as soon as possible.

This attribute supplies the value for the “DL-Scheduling-policy” parameter of the DLL. The permitted value Implicit corresponds to IMPLICIT defined in the Fieldbus Data link layer standard.

4.6.2.2.2.21 MaxDisduSizeFromRequestor

This attribute specifies the configured value of the maximum length of an FAL-PDU that can be sent from the AREP whose Initiator attribute has a value of “True” to the remote AREP.

This attribute supplies the value for the “Maximum DLSDU sizes from requestor” parameter of the DLL.

4.6.2.2.2.22 MaxDisduSizeFromResponder

This attribute specifies the configured value of the maximum length of an FAL-PDU that can be sent from the AREP whose Initiator attribute has a value of “False” to the remote AREP.

This attribute supplies the value for the “Maximum DLSDU sizes from responder” parameter of the DLL.

4.6.2.2.2.23 MaxDisduSizeFromRequestorNegotiated

This attribute specifies the negotiated value of the maximum length of an FAL-PDU that can be sent from the AREP whose Initiator attribute has a value of “True” to the remote AREP.

4.6.2.2.2.24 MaxDisduSizeFromResponderNegotiated

This attribute specifies the negotiated value of the maximum length of an FAL-PDU that can be sent from the AREP whose Initiator attribute has a value of “False” to the remote AREP.

4.6.2.2.2.25 MaxSendingQueueDepth

This attribute specifies the maximum number of FAL-PDUs that can be queued for transmission.

This attribute supplies the value for the “Maximum queue depth” parameter of the DLL for Send queues.

4.6.2.2.2.26 MaxReceivingQueueDepth

This attribute specifies the maximum number of FAL-PDUs that can be queued at reception.

This attribute supplies the value for the “Maximum queue depth” parameter of the DLL for Receive queues.

4.6.2.2.3 DLL services

Refer to 4.7.4, Data Link Layer Service Selection, for DLL service descriptions.

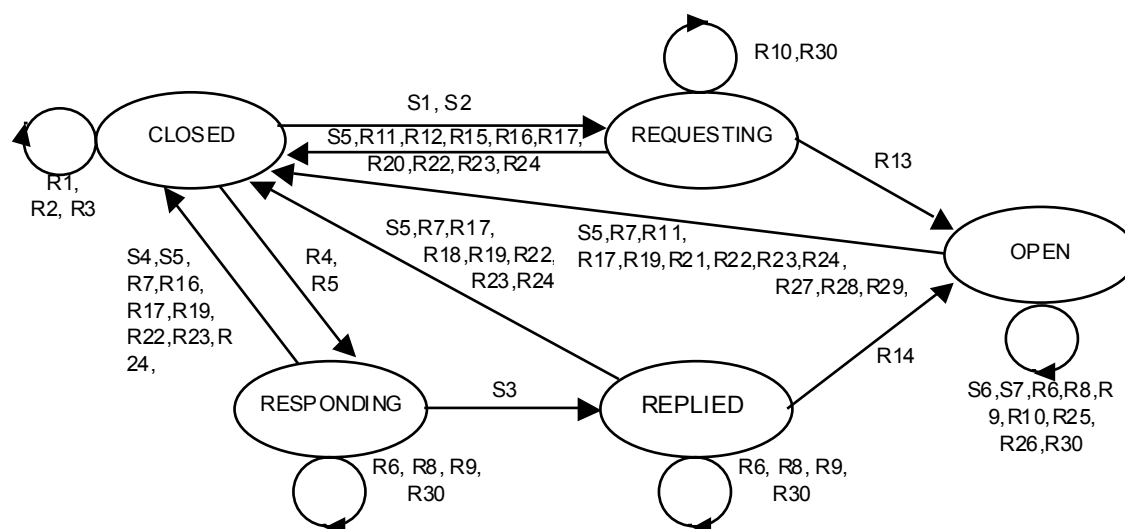
4.6.2.3 QUB-CO AREP state machine

4.6.2.3.1 QUB-CO ARPM states

The defined states and their descriptions of the QUB-CO ARPM are shown in Table 54 and Figure 29.

Table 54 – QUB-CO ARPM states

State	Description
CLOSED	The AREP is defined, but not capable of sending or receiving FAL-PDUs. It may send or receive Establish service FAL-PDUs while in this state.
OPEN	The AREP is defined and capable of sending or receiving FAL-PDUs.
REQUESTING (REQ)	The AREP has sent an Establish Request FAL-PDU and is waiting for a response from the remote AREP.
RESPONDING (RSP)	The AREP has received an Establish Request FAL-PDU, delivered an Establish.ind primitive and is waiting for a response from its user.
REPLIED (REPL)	The Server AREP has issued an EST_rsp(+) primitive and is waiting for receiving a "connection-established" indication from the DLL.
SAME	Indicates that the next state is the same as the current state.

**Figure 29 – State transition diagram of QUB-CO ARPM**

4.6.2.3.2 QUB-CO ARPM state table

Table 55 and Table 56 define the QUB-CO ARPM state machine.

Table 55 – QUB-CO ARPM state table – sender transactions

#	Current state	Event or condition => action	Next state
S1	CLOSED	EST_req && Initiator = "True" && RemoteAddressConfigurationType := "Free" => RemoteDlcepAddress := remote_dlcep_address, FAL-PDU_req { dmpm_service_name := "DMPM_Connect_req", arep_id := GetArepld (), called_address := "default dlsap address", calling_address := "default dlsap address", local_dlcep_address := LocalDlcep, dlsdu := BuildFAL-PDU (fal_pdu_name := "EST_ReqPDU", calling_dlcep_address := LocalDlcepAddress, called_dlcep_address := RemoteDlcepAddress, fal_data := user_data) }	REQ

#	Current state	Event or condition => action	Next state
S2	CLOSED	<pre> EST_req && Initiator = "True" && RemoteAddressConfigurationType := "Linked" => FAL-PDU_req { dmpm_service_name := "DMPM_Connect_req", arep_id := GetArepld (), called_address := "default dlsap address", calling_address := "default dlsap address", local_dlcep_address := LocalDlcep, dlsdu := BuildFAL-PDU (fal_pdu_name := "EST_ReqPDU", calling_dlcep_address := LocalDlcepAddress, called_dlcep_address := RemoteDlcepAddress, fal_data := user_data) } </pre>	REQ
S3	RSP	<pre> EST_rsp(+) => FAL-PDU_req { dmpm_service_name := "DMPM_Connect_rsp", arep_id := GetArepld (), responding_address := "default dlsap address", local_dlcep_address := LocalDlcepAddress, dlsdu := BuildFAL-PDU (fal_pdu_name := "EST_RspPDU", fal_data := user_data) } </pre>	REPL
S4	RSP	<pre> EST_rsp(-) => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "connection rejection-transient condition", dlsdu := BuildFAL-PDU (fal_pdu_name := "EST_ErrPDU", fal_data := user_data) } </pre>	CLOSED
S5	NOT CLOSED	<pre> Abort_req => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "disconnection-normal condition", dlsdu := BuildFAL-PDU (fal_pdu_name := "Abort_PDU", fal_id := identifier, fal_reason_code := reason_code, fal_additional_detail := additional_detail) } </pre>	CLOSED
S6	OPEN	<pre> CS_req && Role = "Client" "Peer" => FAL-PDU_req { dmpm_service_name := "DMPM_Data_req", arep_id := GetArepld (), dlsdu := BuildFAL-PDU (fal_pdu_name := "CS_ReqPDU", fal_data := user_data) } </pre>	OPEN
S7	OPEN	<pre> CS_rsp && Role = "Server" "Peer" => FAL-PDU_req { dmpm_service_name := "DMPM_Data_req", arep_id := GetArepld (), dlsdu := BuildFAL-PDU (fal_pdu_name := "CS_RspPDU", fal_data := user_data) } </pre>	OPEN

Table 56 – QUB-CO ARPM state table – receiver transactions

#	Current state	Event or condition => action	Next state
R1	CLOSED	Connect_ind && Initiator = "True" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Multiple Initiators", dlsdu := "null" }	CLOSED
R2	CLOSED	Connect_ind && Initiator = "False" && FAL_Pdu_Type (dls_user_data) <> "EST_ReqPDU" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }	CLOSED
R3	CLOSED	Connect_ind && Initiator = "False" && FAL_Pdu_Type (dls_user_data) = "EST_ReqPDU" && RemoteAddressConfigurationType = "Linked" && RemoteDlcepAddress <> calling_dlcep_address => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Remote Address Mismatch", dlsdu := "null" }	CLOSED
R4	CLOSED	Connect_ind && Initiator = "False" && FAL_Pdu_Type (dls_user_data) = "EST_ReqPDU" && RemoteAddressConfigurationType = "Free" => RemoteDlcepAddress := calling_dlcep_address, MaxDlsduSizeFromRequestorNegotiated := dlsdu_size_from_requestor, MaxDlsduSizeFromResponderNegotiated := dlsdu_size_from_responder, EST_ind { arep_id := GetArepld (), user_data := dls_user_data }	RSP
R5	CLOSED	Connect_ind && Initiator = "False" && FAL_Pdu_Type (dls_user_data) = "EST_ReqPDU" && RemoteAddressConfigurationType = "Linked" && RemoteDlcepAddress = calling_dlcep_address => MaxDlsduSizeFromRequestorNegotiated := dlsdu_size_from_requestor, MaxDlsduSizeFromResponderNegotiated := dlsdu_size_from_responder, EST_ind { arep_id := GetArepld (), user_data := dls_user_data }	RSP

TABLE 56 (continued)

#	Current state	Event or condition => action	Next state
R6	RSP REPL OPEN	Connect_ind && Initiator = "False" && RemoteAddressConfigurationType = "Linked" && RemoteDlcepAddress <> calling_dlcep_address => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Remote Address Mismatch", dlsdu := "null" }	SAME
R7	RSP REPL OPEN	Connect_ind && Initiator = "False" && RemoteDlcepAddress = calling_dlcep_address => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU" }	CLOSED
R8	RSP REPL OPEN	Connect_ind && Initiator = "False" && FAL_Pdu_Type (dls_user_data) = "EST_ReqPDU" && RemoteAddressConfigurationType = "Free" && RemoteDlcepAddress <> calling_dlcep_address => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "AREP Busy", dlsdu := "null" }	SAME
R9	RSP REPL OPEN	Connect_ind && Initiator = "False" && FAL_Pdu_Type (dls_user_data) <> "EST_ReqPDU" && RemoteAddressConfigurationType = "Free" && RemoteDlcepAddress <> calling_dlcep_address => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }	SAME
R10	REQ OPEN	Connect_ind && Initiator = "True" && RemoteDlcepAddress <> calling_dlcep_address => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Multiple Initiators", dlsdu := "null" }	SAME

TABLE 56 (continued)

#	Current state	Event or condition => action	Next state
R11	REQ OPEN	Connect_ind && Initiator = "True" && RemoteDlcepAddress = calling_dlcep_address => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Multiple Initiators", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Multiple Initiators" }	CLOSED
R12	REQ	Connect_cnf && FAL_Pdu_Type (dls_user_data) <> "EST_RspPDU" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU" }	CLOSED
R13	REQ	Connect_cnf && FAL_Pdu_Type (dls_user_data) = "EST_RspPDU" => MaxDlsduSizeFromRequestorNegotiated := dlsdu_size_from_requestor, MaxDlsduSizeFromResponderNegotiated := dlsdu_size_from_responder, DllPriorityNegotiated := dll_priority, EST_cnf(+) { arep_id := GetArepld (), user_data := dls_user_data }	OPEN
R14	REPL	FAL-PDU_ind && dmpm_service_name = "DMPM_Connection_Established_ind" => (no actions taken)	OPEN
R15	REQ	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && FAL_Pdu_Type (fal_pdu) = "EST_ErrPDU" => EST_cnf(-) { arep_id := GetArepld (), user_data := fal_pdu }	CLOSED

TABLE 56 (continued)

#	Current state	Event or condition => action	Next state
R16	REQ RSP	<pre> FAL-PDU_ind && dmpm_service_name <> "DMPM_Disconnect_ind" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid DL-Event", disdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid DL-Event" } </pre>	CLOSED
R17	NOT CLOSED	<pre> FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu <> "null" && FAL_Pdu_Type (fal_pdu) = "Abort_PDU" => Abort_ind{ arep_id := GetArepld (), locally_generated := "False", identifier := AbortIdentifier (fal_pdu), reason_code := AbortReason (fal_pdu), additional_detail := AbortDetail (fal_pdu) } </pre>	CLOSED
R18	REPL	<pre> FAL-PDU_ind && ((dmpm_service_name <> "DMPM_Disconnect_ind") && (dmpm_service_name <> "DM_Connection_Established_ind")) => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid DL-Event", disdu := "null" }, Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid DL-Event", additional_detail := "null" } </pre>	CLOSED
R19	REPL RSP OPEN	<pre> FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu <> "null" && FAL_Pdu_Type (fal_pdu) <> "Abort_PDU" => Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU", additional_detail := "null" } </pre>	CLOSED
R20	REQ	<pre> FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu <> "null" && ((FAL_Pdu_Type (fal_pdu) <> "Abort_PDU") && (FAL_Pdu_Type (fal_pdu) <> "EST_ErrPDU")) => Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU", additional_detail := "null" } </pre>	CLOSED

TABLE 56 (continued)

#	Current state	Event or condition => action	Next state
R21	OPEN	FAL-PDU_ind && ((dmpm_service_name <> "DMPM_Disconnect_ind") && (dmpm_service_name <> "DMPM_Data_ind")) => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid DL-Event", disdu := "null" }, Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid DL-Event", additional_detail := "null" }	CLOSED
R22	NOT CLOSED	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu = "null" && originator = "remote_dls_provider" => Abort_ind{ arep_id := GetArepld (), locally_generated := "False", identifier := "Data Link Layer", reason_code := reason, additional_detail := "null" }	CLOSED
R23	NOT CLOSED	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu = "null" && originator = "remote_dls_user" => Abort_ind{ arep_id := GetArepld (), locally_generated := "False", identifier := "FAL", reason_code := reason, additional_detail := "null" }	CLOSED
R24	NOT CLOSED	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu = "null" && originator = "local_dls_provider" => Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "Data Link Layer", reason_code := reason, additional_detail := "null" }	CLOSED
R25	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Peer" "Server" && FAL_Pdu_Type (fal_pdu) = "CS_ReqPDU" => CS_ind { arep_id := GetArepld (), user_data := fal_pdu }	OPEN

TABLE 56 (continued)

#	Current state	Event or condition => action	Next state
R26	OPEN	<pre> FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Client" "Peer" && FAL_Pdu_Type (fal_pdu) = "CS_RspPDU" => CS_cnf { arep_id := GetArepld (), user_data := fal_pdu } </pre>	OPEN
R27	OPEN	<pre> FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Server" && FAL_Pdu_Type (fal_pdu) <> "CS_ReqPDU" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU", additional_detail := "null" } </pre>	CLOSED
R28	OPEN	<pre> FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Client" && FAL_Pdu_Type (fal_pdu) <> "CS_RspPDU" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU", additional_detail := "null" } </pre>	CLOSED
R29	OPEN	<pre> FAL-PDU_ind && Role = "Peer" && dmpm_service_name = "DMPM_Data_ind" && ((FAL_Pdu_Type (fal_pdu) <> "CS_ReqPDU") && (FAL_Pdu_Type (fal_pdu) <> "CS_RspPDU")) => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU", additional_detail := "null" } </pre>	CLOSED
R30	NOT CLOSED	ErrorToARPM (No actions taken. See note.)	SAME

TABLE 56 (continued)

#	Current state	Event or condition => action	Next state
NOTE It is a local matter to report an error status to network management entities. The ARPM does not abort the existing connections. The FAL user may issue an Abort request to disconnect the current connection, depending on the type of status information conveyed by the ErrorToARPM primitive.			

4.6.2.3.3 Functions used by QUB-CO ARPM

Table 57 through Table 62 define functions used by this state machine.

Table 57 – Function GetArepld ()

Name	GetArepld ()	Used in	ARPM
Input		Output	
(none)		AREP Identifier	
Function	Returns a value that can unambiguously identify the current AREP.		

Table 58 – Function BuildFAL-PDU

Name	BuildFAL-PDU	Used in	ARPM
Input		Output	
fal_pdu_name, calling_dlcep_address, called_dlcep_address, fal_data, fal_id, fal_reason_code, fal_additional_detail		dlsdu	
Function	Builds an FAL-PDU out of the parameters given as input variables.		

Table 59 – Function FAL_Pdu_Type

Name	FAL_Pdu_Type	Used in	ARPM
Input		Output	
dls_user_data fal_pdu		One of the FAL-PDU types defined in clause 4.	
Function	This function decodes the FAL-PDU that is conveyed in the dls_user_data or fal_pdu parameter and retrieves one of the FAL-PDU types.		

Table 60 – Function AbortIdentifier

Name	AbortIdentifier	Used in	ARPM
Input		Output	
fal_pdu		The Identifier parameter of the Abort service.	
Function	This function decodes the Abort_PDU that is conveyed in the fal_pdu parameter and extracts the Identifier parameter.		

Table 61 – Function AbortReason

Name	AbortReason	Used in	ARPM
Input		Output	
fal_pdu		The Reason Code parameter of the Abort service.	
Function	This function decodes the Abort_PDU that is conveyed in the fal_pdu parameter and extracts the Reason Code parameter.		

Table 62 – Function AbortDetail

Name	AbortDetail	Used in	ARPM
Input		Output	
fal_pdu		The Additional Detail parameter of the Abort service.	
Function	This function decodes the Abort_PDU that is conveyed in the fal_pdu parameter and extracts the Additional Detail parameter.		

4.6.3 Queued User-triggered Bidirectional-Connectionless (QUB-CI) ARPM

4.6.3.1 Primitive definitions

4.6.3.1.1 Primitives exchanged between ARPM and FSPM

The primitives exchanged between the ARPM and the FSPM are listed in Table 63 and Table 64.

Table 63 – Primitives issued by FSPM to ARPM

Primitive name	Source	Associated parameters	Functions
EST_req	FSPM	user_data	This is an FAL internal primitive used to convey an Establish request primitive from the FSPM to the ARPM.
Abort_req	FSPM	identifier, reason_code, additional_detail	This is an FAL internal primitive used to convey an Abort request primitive from the FSPM to the ARPM.
CS_req	FSPM	user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) request primitive from the FSPM to the ARPM.
CS_rsp	FSPM	user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) response primitive from the FSPM to the ARPM.
UCS_req	FSPM	remote_dlsap_address, user_data	This is an FAL internal primitive used to convey an Unconfirmed Send (UCS) request primitive from the FSPM to the ARPM.

Table 64 – Primitives issued by ARPM to FSPM

Primitive name	Source	Associated parameters	Functions
EST_cnf(+)	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish response(+) primitive from the ARPM to the FSPM.
EST_cnf(-)	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish response(-) primitive from the ARPM to the FSPM.
CS_ind	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) indication primitive from the ARPM to the FSPM.
CS_cnf	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) confirmation primitive from the ARPM to the FSPM.
UCS_ind	ARPM	arep_id, remote_dlsap_address, duplicate_fal_sdu, user_data, local_timeliness, remote_timeliness	This is an FAL internal primitive used to convey an Unconfirmed Send (UCS) indication primitive from the ARPM to the FSPM.
FSTS_ind	ARPM	arep_id, reported_status	This is an FAL internal primitive used to convey a FAL-Status (FSTS) indication primitive from the ARPM to the FSPM.

4.6.3.1.2 Parameters of FSPM/ARPM primitives

The parameters used with the primitives exchanged between the FSPM and the ARPM are described in Table 65.

Table 65 – Parameters used with primitives exchanged between FSPM and ARPM

Parameter name	Description
arep_id	This parameter is used to unambiguously identify an instance of the AREP that has issued a primitive. A means for such identification is not specified by this standard.
user_data	This parameter conveys FAL-User data.
locally_generated	This parameter conveys value that is used for the Locally_Generated parameter.
identifier	This parameter conveys value that is used for the Identifier parameter.
reason_code	This parameter conveys value that is used for the Reason_Code parameter.
additional_detail	This parameter conveys value that is used for the Additional_Detail parameter.
duplicate_fal_sdu	This parameter conveys value that is used for the Duplicate_FAL-SDU parameter.
remote_dlsap_address	This parameter conveys value that is used for the Remote_DLSAP_Address parameter.
status	This parameter conveys value that is used for the Status parameter.
reported_status	This parameter conveys a Data Link Layer event status.
local_timeliness	This parameter conveys value that is used for the Local_Timeliness parameter.
remote_timeliness	This parameter conveys value that is used for the Remote_Timeliness parameter.

4.6.3.2 DLL mapping of QUB-CL AREP class

4.6.3.2.1 QUB-CL AREP class formal model

This subclause describes the mapping of the QUB_CL AREP Class to the Fieldbus Data Link Layer. It does not redefine the DLSAP attributes or DLME attributes that are or will be defined in the Data link layer standard; rather, it defines how they are used by this AR class.

The DLL Mapping attributes and their permitted values and the DLL services used with the QUB_CL AREP class are defined in this subclause.

CLASS: QubCI

PARENT CLASS: QueuedUser-TriggeredBidirectionalConnectionlessAREP

ATTRIBUTES:

- 1 (m) KeyAttribute: LocalDisapAddress
- 2 (m) Attribute: RemoteDisapAddress
- 3 (m) Attribute: LocalDisapRole (Basic)
- 4 (m) Attribute: DefaultQosAsSender
- 4.1 (m) Attribute: DllPriority (Urgent, Normal, TimeAvailable)
- 4.2 (m) Attribute: MaxConfirmDelayOnUnitdata
- 4.3 (m) Attribute: DlpduAuthentication (Ordinary, Source, Maximal)
- 4.4 (m) Attribute: DlschedulingPolicy (Implicit)
- 4.5 (m) Attribute: DleRemoteConf (True, False)
- 5 (m) Attribute: ExplicitQueue (True, False)
- 6 (c) Constraint: ExplicitQueue = True
- 6.1 (m) Attribute: SendingQueueBindings
- 6.1.1 (m) Attribute: MaxQueueDepth
- 6.1.2 (m) Attribute: MaxDisduSize
- 6.2 (m) Attribute: ReceivingQueueBindings
- 6.2.1 (m) Attribute: MaxQueueDepth
- 6.2.2 (m) Attribute: MaxDisduSize

DLL SERVICES:

- 1 (m) OpsService: DL-Unitdata
- 2 (c) Constraint: ExplicitQueue = True
- 2.1 (m) OpsService: DL-Get

4.6.3.2.2 Attributes

4.6.3.2.2.1 LocalDisapAddress

This attribute specifies the DLSAP address to which this AREP is attached. It supplies the value for the “DL(SAP)-address” parameter specified in the DLL.

This attribute contains the following three subattributes: Link Address, Node Address, and Selector.

4.6.3.2.2.2 RemoteDisapAddress

This attribute specifies the remote address to which FAL-PDUs are sent (for Source AREPs), or from which they are received (for Sink AREPs).

If the RemoteAddressConfigurationType attribute is Linked, the value of this attribute has been configured. If it is Free, the value of this attribute is provided with a service request.

This attribute contains the following three subattributes: Link Address, Node Address, and Selector.

4.6.3.2.2.3 LocalDisapRole

This attribute specifies the behavior of the local DLSAP to be used. No DLCEPs are needed on this Basic DLSAPRole.

This attribute supplies the value for the “DL(SAP)-role” parameter specified by the DLL.

4.6.3.2.2.4 DefaultQosAsSender

The DefaultQosAsSender attributes specify the DLL quality of service that is used by the sending AREP. The receiving DLL shall support the quality of service specified by these attributes.

4.6.3.2.2.5 DllPriority

This attribute defines the DLL priority, and thus restricts the maximum length of an FAL-PDU, of the conveyance path of an AR.

This attribute supplies the value for the “DLL priority” parameter of the DLL. The values Urgent, Normal, and Time-Available correspond to URGENT, NORMAL, and TIME-AVAILABLE as defined in Part-3 and –4 of this International Standard, respectively.

NOTE It is not possible to use different priorities for each FAL-PDU sent from the same QUB_CL AREP. The priority is identical on the sending way and on the receiving way.

4.6.3.2.2.6 MaxConfirmDelayOnUnitdata

This attribute specifies the maximum confirmation delay for a local confirmation from a DL-Unitdata request primitive.

This attribute supplies the value for the “Max confirm delay on locally-confirmed DL-Unitdata” parameter of the DLL.

4.6.3.2.2.7 DlpduAuthentication

This attribute specifies the lower bound of the length of the DL-addresses to be used by the DLL.

This attribute supplies the value for the “DLPDU authentication” parameter of the DLL. The values Ordinary, Source, and Maximal correspond to ORDINARY, SOURCE, and MAXIMAL as defined in Part-3 and –4 of this International Standard, respectively.

4.6.3.2.2.8 DISchedulingPolicy

This attribute provides the guidance to the DLL on the scheduling needed by an AR. For this AREP, the DLL tries to transmit the FAL-PDU as soon as it is passed from the FAL.

This attribute supplies the value for DL-Scheduling-policy attribute. Only the value Implicit is used. It corresponds to IMPLICIT as defined in Part-3 and –4 of this International Standard.

4.6.3.2.2.9 DleRemoteConf

This attribute specifies, when true, that the remote DLL immediate Ack shall be used.

4.6.3.2.2.10 ExplicitQueue

This attribute specifies, when True, that the characteristics of the associated sending and receiving queues are explicitly configured and managed through the Network Management. The value of False means that queues with implementation specific depth and length are provided by the DLL.

4.6.3.2.2.11 QueueBindings

The following attributes specify the explicit queue that is bound to this DLSAP. For the sending part, the queue is for sending. For the receiving part, it is for receiving.

4.6.3.2.2.12 MaxQueueDepth

This attribute specifies the maximum number of FAL-PDUs that can be queued for sending or receiving.

This attribute supplies the value for the “Maximum queue depth” parameter of the DLL.

4.6.3.2.2.13 MaxDisduSize

This attribute specifies the maximum length of an FAL-PDU that can be sent or received by the DLL.

This attribute supplies the value for the “Maximum DLSDU size” parameter of the DLL.

4.6.3.2.3 DLL services

Refer to 4.7.4, Data Link Layer Service Selection, for DLL service descriptions.

4.6.3.3 QUB-CL ARPM state machine

4.6.3.3.1 QUB-CL ARPM states

The defined states and their descriptions of the QUB-CL ARPM are shown in Table 66 and Figure 30.

Table 66 – QUB-CL ARPM states

State	Description
CLOSED	The AREP is defined, but not capable of sending or receiving FAL-PDUs. It may send or receive Establish service FAL-PDUs while in this state.
OPEN	The AREP is defined and capable of sending or receiving FAL-PDUs.

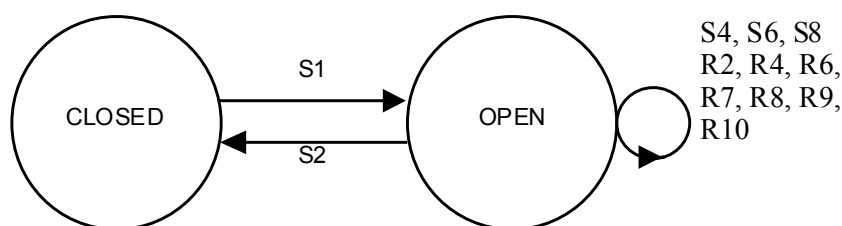


Figure 30 – State transition diagram of the QUB-CL ARPM

4.6.3.3.2 QUB-CL ARPM state table

Table 67 and Table 68 define the QUB-CL state machine.

Table 67 – QUB-CL ARPM state table – sender transactions

#	Current state	Event or condition => action	Next state
S1	CLOSED	EST_req => EST_cnf(+) { arep_id := GetArepld (), user_data := "null" }	OPEN
S2	OPEN	Abort_req => (no action taken)	CLOSED
S3	OPEN	CS_req && ConfigurationType = "Linked" && Role = "Client" "Peer" => FAL-PDU_req { dmpm_service_name := "DMPM_Unitdata_req", arep_id := GetArepld (), called_address := Remote_dlsap_address, dlsdu := BuildFAL-PDU (fal_pdu_name := "CS_ReqPDU", fal_data := user_data) } }	OPEN
S4	OPEN	CS_req && RemoteAddressConfigurationType = "Free" && Role = "Client" "Peer" => FAL-PDU_req { dmpm_service_name := "DMPM_Unitdata_req", arep_id := GetArepld (), called_address := remote_dlsap_address, dlsdu := BuildFAL-PDU (fal_pdu_name := "CS_ReqPDU", fal_data := user_data) } }	OPEN
S6	OPEN	CS_rsp && RemoteAddressConfigurationType = "Free" && Role = "Server" "Peer" => FAL-PDU_req { dmpm_service_name := "DMPM_Unitdata_req", arep_id := GetArepld (), called_address := remote_dlsap_address, dlsdu := BuildFAL-PDU (fal_pdu_name := "CS_RspPDU", fal_data := user_data) } }	OPEN
S7	OPEN	UCS_req && ConfigurationType = "Linked" && Role = "Server" "Peer" => FAL-PDU_req { dmpm_service_name := "DMPM_Unitdata_req", arep_id := GetArepld (), called_address := Remote_dlsap_address, dlsdu := BuildFAL-PDU (fal_pdu_name := "UCS_PDU", fal_data := user_data) } }	OPEN

#	Current state	Event or condition => action	Next state
S8	OPEN	<pre> UCS_req && RemoteAddressConfigurationType = "Free" && Role = "Server" "Peer" => FAL-PDU_req { dmpm_service_name := "DMPM_Unitdata_req", arep_id := GetArepld (), called_address := remote_dlsap_address, dlsdu := BuildFAL-PDU (fal_pdu_name := "UCS_PDU", fal_data := user_data) } </pre>	OPEN

Table 68 – QUB-CL ARPM state table – receiver transactions

#	Current state	Event or condition => action	Next state
R1	OPEN	<pre> FAL-PDU_ind && dmpm_service_name = "DMPM_Unitdata_ind" && ConfigurationType = "Linked" && RemoteDlsapAddress = calling_address && FAL_Pdu_Type (fal_pdu) = "CS_ReqPDU" && Role = "Server" "Peer" => CS_ind { arep_id := GetArepld (), remote_dlsap_address := "null", user_data := fal_pdu } </pre>	OPEN
R2	OPEN	<pre> FAL-PDU_ind && dmpm_service_name = "DMPM_Unitdata_ind" && RemoteAddressConfigurationType = "Free" && FAL_Pdu_Type (fal_pdu) = "CS_ReqPDU" && Role = "Server" "Peer" => CS_ind { arep_id := GetArepld (), remote_dlsap_address := calling_address, user_data := fal_pdu } </pre>	OPEN
R3	OPEN	<pre> FAL-PDU_ind && dmpm_service_name = "DMPM_Unitdata_ind" && ConfigurationType = "Linked" && RemoteDlsapAddress = calling_address && FAL_Pdu_Type (fal_pdu) = "CS_RspPDU" && Role = "Client" "Peer" => CS_cnf { arep_id := GetArepld (), remote_dlsap_address := "null", user_data := fal_pdu } </pre>	OPEN
R4	OPEN	<pre> FAL-PDU_ind && dmpm_service_name = "DMPM_Unitdata_ind" && RemoteAddressConfigurationType = "Free" && FAL_Pdu_Type (fal_pdu) = "CS_RspPDU" && Role = "Client" "Peer" => CS_cnf { arep_id := GetArepld (), remote_dlsap_address := calling_address, user_data := fal_pdu } </pre>	OPEN

#	Current state	Event or condition => action	Next state
R5	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Unitdata_ind" && ConfigurationType = "Linked" && RemoteDlsapAddress = calling_address && FAL_Pdu_Type (fal_pdu) = "UCS_PDU" && Role = "Server" "Peer" => UCS_ind { arep_id := GetArepId () remote_dlsap_address := "null", user_data := fal_pdu }	OPEN
R6	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Unitdata_ind" && RemoteAddressConfigurationType = "Free" && FAL_Pdu_Type (fal_pdu) = "UCS_PDU" && Role = "Client" "Peer" => UCS_ind { arep_id := GetArepId (), remote_dlsap_address := calling_address, user_data := fal_pdu }	OPEN
R7	OPEN	FAL-PDU_Ind && dmpm_service_name = "DMPM_Unidata_cnf" && FALPdu_Type(fal-pdu) = "CS_Req PDU" && Role = "Client Peer" && dl-status <> success => CS_Cnf { arep_id := GetArepId(), remote_dlsap_address := null if Linked otherwise calling_address, user_data := null, result := dl_status }	OPEN
R8	OPEN	FAL-PDU_Ind && dmpm_service_name = "DMPM_Unidata_cnf" && FALPdu_Type(fal-pdu) = "CS_Req PDU" && Role = "Client Peer" && dl-status = "success" => (no action)	OPEN
R9	OPEN	FAL-PDU_Ind && dmpm_service_name = "DMPM_Unidata_cnf" && FALPdu_Type(fal-pdu) = "UCS_Req PDU" && Role = "Client Peer" => FSTS_Ind{ arep_Id := GetArepId() reported_status := dl_status }	OPEN
R10	OPEN	FAL-PDU_Ind && dmpm_service_name = "DMPM_Unidata_cnf" && FALPdu_Type(fal-pdu) = "CS_Rsp PDU" && Role = "Server Peer" => (no action)	OPEN

4.6.3.3.3 Functions used by QUB-CL ARPM

Table 69 through Table 71 define the functions used by QUB-CL.

Table 69 – Function GetArepId ()

Name	GetArepId ()	Used in	ARPM
Input		Output	
(none)		AREP Identifier	
Function	Returns a value that can unambiguously identify the current AREP.		

Table 70 – Function BuildFAL-PDU

Name	BuildFAL-PDU	Used in	ARPM
Input	fal_pdu_name, calling_dlcep_address, called_dlcep_address, fal_data, fal_id, fal_reason_code, fal_additional_detail	Output	dlsdu
Function	Builds an FAL-PDU out of the parameters given as input variables.		

Table 71 – Function FAL-Pdu_Type

Name	FAL_Pdu_Type	Used in	ARPM
Input	dls_user_data	Output	One of the FAL-PDU types defined in clause 4.
Function	This function decodes the FAL-PDU that is conveyed in the dls_user_data parameter and retrieves one of the FAL-PDU types.		

4.6.4 Queued User-triggered Bidirectional-Segmentation (QUB-Seg) ARPM

4.6.4.1 Primitive definitions

4.6.4.1.1 Primitives exchanged between ARPM and FSPM

Table 72 and Table 73 define the primitives exchanged between ARPM and FSPM.

Table 72 – Primitives issued by FSPM to ARPM

Primitive name	Source	Associated parameters	Functions
EST_req	FSPM	user_data	This is an FAL internal primitive used to convey an Establish request primitive from the FSPM to the ARPM.
EST_rsp(+)	FSPM	user_data	This is an FAL internal primitive used to convey an Establish response(+) primitive from the FSPM to the ARPM.
EST_rsp(-)	FSPM	user_data	This is an FAL internal primitive used to convey an Establish response(-) primitive from the FSPM to the ARPM.
Abort_req	FSPM	identifier, reason_code, additional_detail	This an FAL internal primitive used to convey an Abort request primitive from the FSPM to the ARPM.
CS_req	FSPM	user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) request primitive from the FSPM to the ARPM.
CS_rsp	FSPM	user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) response primitive from the FSPM to the ARPM.
UCS_req	FSPM	remote_dlsap_address, user_data	This is an FAL internal primitive used to convey an Unconfirmed Send (UCS) request primitive from the FSPM to the ARPM.

Table 73 – Primitives issued by ARPM to FSPM

Primitive name	Source	Associated parameters	Functions
EST_ind	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish indication primitive from the ARPM to the FSPM.
EST_cnf(+)	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish response(+) primitive from the ARPM to the FSPM.
EST_cnf(-)	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish response(-) primitive from the ARPM to the FSPM.
Abort_ind	ARPM	arep_id, locally_generated, identifier, reason_code, additional_detail	This is an FAL internal primitive used to convey an Abort primitive from the ARPM to the FSPM.
CS_ind	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) indication primitive from the ARPM to the FSPM.
CS_cnf	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) confirmation primitive from the ARPM to the FSPM.
UCS_ind	ARPM	arep_id, remote_dlsap_address, duplicate_fal_sdu, user_data, local_timeliness, remote_timeliness	This is an FAL internal primitive used to convey an Unconfirmed Send (UCS) indication primitive from the ARPM to the FSPM.

4.6.4.1.2 Parameters of FSPM/ARPM primitives

The parameters used with the primitives exchanged between the FSPM and the ARPM are described in Table 74.

Table 74 – Parameters used with primitives exchanged between FSPM and ARPM

Parameter name	Description
arep_id	This parameter is used to unambiguously identify an instance of the AREP that has issued a primitive. A means for such identification is not specified by this standard.
user_data	This parameter conveys FAL-User data.
locally_generated	This parameter conveys value that is used for the Locally_Generated parameter.
identifier	This parameter conveys value that is used for the Identifier parameter.
reason_code	This parameter conveys value that is used for the Reason_Code parameter.
additional_detail	This parameter conveys value that is used for the Additional_Detail parameter.
duplicate_fal_sdu	This parameter conveys value that is used for the Duplicate_FAL-SDU parameter.
remote_dlsap_address	This parameter conveys value that is used for the Remote_DLSAP_Address parameter.
status	This parameter conveys value that is used for the Status parameter.
reported_status	This parameter conveys a Data Link Layer event status.
local_timeliness	This parameter conveys value that is used for the Local_Timeliness parameter.
remote_timeliness	This parameter conveys value that is used for the Remote_Timeliness parameter.

4.6.4.2 DLL mapping of QUB-seg AREP class

4.6.4.2.1 QUB-seg AREP class formal definition

This subclause describes the mapping of the QUB-Seg AREP class to the Fieldbus Data Link Layer. It does not redefine the DLSAP attributes or DLME attributes that are or will be defined in the Data link layer standard; rather, it defines how they are used by this AREP class.

The DLL Mapping attributes and their permitted values and the DLL services used with the QUB-Seg AREP class are defined in this subclause.

This class is defined to support the concurrent use of confirmed and unconfirmed services on the same AR as well as the conveyance of services with a maximum data size of 64 koctets by providing a segmentation functionality, which is invisible to the FAL user. This class uses the DDL features of CLASSICAL connections to ensure the correct segmentation and reassembling of FAL services.

CLASS:	QubSeg
PARENT CLASS:	QueuedUser-TriggeredBidirectionalSegmentationAREP
ATTRIBUTES:	
1	(m) KeyAttribute: LocalDlcepAddress
2	(m) Attribute: RemoteDlcepAddress
3	(m) Attribute: DlsepRole (Basic)
4	(m) Attribute: QosParameterSet
4.1	(m) Attribute: DlcepClass (Peer)
4.2	(m) Attribute: DlcepDataDeliveryFeatures
4.2.1	(m) Attribute: FromRequestorToResponder (Classical)
4.2.2	(m) Attribute: FromResponderToRequestor (Classical)
4.3	(m) Attribute: Priority
4.3.1	(m) Attribute: DIIPriority (Urgent, Normal, TimeAvailable)
4.3.2	(m) Attribute: DIIPriorityNegotiated (Urgent, Normal, TimeAvailable)
4.4	(m) Attribute: Dl pduAuthentication (Ordinary, Source, Maximal)
4.5	(m) Attribute: ResidualActivity
4.5.1	(m) Attribute: ResidualActivityAsSender (True, False)
4.5.2	(m) Attribute: ResidualActivityAsReceiver (True, False)
4.6	(m) Attribute: MaxConfirmDelay
4.6.1	(m) Attribute: MaxConfirmDelayOnDIConnect
4.6.2	(m) Attribute: MaxConfirmDelayOnDIData
4.7	(m) Attribute: DISchedulingPolicy (Implicit)
4.8	(m) Attribute: ExplicitQueue (True, False)
4.9	(c) Constraint: ExplicitQueue = True
4.9.1	(m) Attribute: MaxDlsduSizes
4.9.1.1	(m) Attribute: MaxDlsduSizeFromRequestor
4.9.1.2	(m) Attribute: MaxDlsduSizeFromResponder
4.9.1.3	(m) Attribute: MaxDlsduSizeFromRequestorNegotiated
4.9.1.4	(m) Attribute: MaxDlsduSizeFromResponderNegotiated
4.9.2	(m) Attribute: QueueBindings
4.9.2.1	(m) Attribute: SendingBufferOrQueueIdentifier
4.9.2.2	(m) Attribute: ReceivingBufferOrQueueIdentifier
4.9.3	(m) Attribute: MaxQueueDepth
4.9.3.1	(m) Attribute: MaxSendingQueueDepth
4.9.3.2	(m) Attribute: MaximumReceivingQueueDepth
DLL SERVICES:	
1	(m) OpsService: DL-Data
2	(c) Constraint: ExplicitQueue = True
2.1	(m) OpsService: DL-Get
3	(m) OpsService: DL-Connect
4	(m) OpsService: DL-Connection-Established
5	(m) OpsService: DL-Disconnect

4.6.4.2.2 Attributes

4.6.4.2.2.1 LocalDlcepAddress

This attribute specifies the local DLCEP address and identifies the DLCEP.

The value of this attribute is used as the "DLCEP-address" parameter of the DLL.

NOTE 1 The value of this attribute is also carried in the Establish Request PDU.

This attribute contains the following three subattributes: Link Address, Node Address, and Selector.

NOTE 2 Since the local Link and Node addresses are set by the Network Management, only the Selector portion of the LocalDlsepAddress attribute is a configurable attribute of the FAL.

4.6.4.2.2.2 RemoteDlcepAddress

This attribute specifies the remote DLCEP address and identifies the DLCEP.

This attribute supplies the value for called DLCEP-address of the DL-Connect service.

NOTE The value of this attribute is also carried in the header part of the Establish Request PDU.

This attribute contains the following three subattributes: Link Address, Node Address, and Selector.

4.6.4.2.2.3 DisapRole

This attribute specifies the behavior of the DLSAP that is used by the AREP.

This attribute supplies the value for the “DL(SAP)-role” parameter. The possible value Basic corresponds to BASIC as defined in the Data link layer standard.

4.6.4.2.2.4 QosParameterSet

The QosParameterSet attributes specify the DL quality of service that is used by this AREP. These attribute values may be negotiated with the remote AREP.

4.6.4.2.2.5 DlcepClass

This attribute specifies the behavior of the DLCEP which is attached to the AREP.

This attribute supplies the value for the “DLCEP class” parameter of the DLL. The possible value of this attribute is Peer and corresponds to Peer defined by the DLL.

4.6.4.2.2.6 DlcepDataDeliveryFeatures

These two attributes specify data delivery features of the DLL.

The permitted values Classical and Disordered correspond, respectively to CLASSICAL and DISORDERED defined by the Data link layer standard.

The FromRequestorToResponder and FromResponderToRequestor attributes shall have the same value.

4.6.4.2.2.7 FromRequestorToResponder

This attribute specifies the DLL data delivery feature of the DLPDUs sent from the AREP whose Initiator attribute has a value of “True” to the remote AREP. It supplies the value for the “DLCEP data delivery features from requestor to responder(s)” parameter defined in the DLL. Only the value “CLASSICAL” is allowed since disordered segments would corrupt the FAL PDUs.

4.6.4.2.2.8 FromResponderToRequestor

This attribute specifies the DLL data delivery feature of the DLPDUs sent from the AREP whose Initiator attribute has a value of “False” to the remote AREP. It supplies the value for the “DLCEP data delivery features from responder(s) to requestor” parameter defined in the DLL. Only the value “CLASSICAL” is allowed since disordered segments would corrupt the FAL PDUs.

4.6.4.2.2.9 DllPriority

This attribute specifies the configured value of the DLL priority.

NOTE It is not possible to use different priorities for each FAL-PDU sent from the same QUB AREP. Also, it is not permitted to use different priorities for the send and receive conveyance paths.

4.6.4.2.2.10 DllPriorityNegotiated

This attribute specifies the negotiated value of the DLL priority.

4.6.4.2.2.11 DlpduAuthentication

This attribute specifies the lower bound of the length of DL-addressing to be used by the DLL.

This attribute supplies the value for the “DLPDU-authentication” parameter of the DLL. The permitted value Ordinary, Source, and Maximal correspond to ORDINARY, Source, and MAXIMAL, respectively, as defined in Part-3 and –4 of this International Standard.

4.6.4.2.2.12 ResidualActivityAsSender

This attribute specifies sender’s DLC residual activity. It supplies the value for the “Residual activity as sender” parameter defined in the DLL. The possible values are “True” and “False.”

4.6.4.2.2.13 ResidualActivityAsReceiver

This attribute specifies receiver’s DLC residual activity. It supplies the value for the “Residual activity as receiver” parameter defined in the DLL. The possible values are “True” and “False.”

4.6.4.2.2.14 MaxConfirmDelay

This attribute specifies the maximum confirmation delay of certain DLL connection-oriented services.

4.6.4.2.2.15 MaxConfirmDelayOnDIConnect

This attribute specifies the maximum confirmation delay for a confirmation from a DL-Connect service.

This attribute supplies the value for the “Maximum confirmation delay on DL-Connect, DL-Reset and DL-Subscriber-Query” parameter specified in the Data link layer standard.

4.6.4.2.2.16 MaxConfirmDelayOnDIData

This attribute specifies the maximum confirmation delay for a confirmation from a DL-Data service.

This attribute supplies the value for the “Maximum confirmation delay on DL-Data” parameter specified in the Data link layer standard (IEC 61158-3).

4.6.4.2.2.17 DISchedulingPolicy

This attribute provides the guidance to the DLL on the scheduling needed by an AR. For this AREP, the DLL tries to transmit the FAL-PDU as soon as possible.

This attribute supplies the value for the “DL-Scheduling-policy” parameter of the DLL. The permitted value Implicit corresponds to IMPLICIT defined in Part-3 and –4 of this International Standard.

4.6.4.2.2.18 MaxDisduSizeFromRequestor

This attribute specifies the configured value of the maximum length of an FAL-PDU that can be sent from the AREP whose Initiator attribute has a value of “True” to the remote AREP.

This attribute supplies the value for the “Maximum DLSDU sizes from requestor” parameter of the DLL.

4.6.4.2.2.19 MaxDisduSizeFromResponder

This attribute specifies the configured value of the maximum length of an FAL-PDU that can be sent from the AREP whose Initiator attribute has a value of “False” to the remote AREP.

This attribute supplies the value for the “Maximum DLSDU sizes from responder” parameter of the DLL.

4.6.4.2.2.20 MaxDisduSizeFromRequestorNegotiated

This attribute specifies the negotiated value of the maximum length of an FAL-PDU that can be sent from the AREP whose Initiator attribute has a value of “True” to the remote AREP.

4.6.4.2.2.21 MaxDisduSizeFromResponderNegotiated

This attribute specifies the negotiated value of the maximum length of an FAL-PDU that can be sent from the AREP whose Initiator attribute has a value of “False” to the remote AREP.

4.6.4.2.2.22 SendingBufferOrQueueIdentifier

This attribute provides a local means to identify a queue that is used to store sending FAL-PDUs.

This attribute supplies the value for the “Buffer-and-queue bindings as sender” parameter of the DLL.

4.6.4.2.2.23 ReceivingBufferOrQueueIdentifier

This attribute provides a local means to identify a queue that is used to store receiving FAL-PDUs.

This attribute supplies the value for the “Buffer-and-queue bindings as receiver” parameter of the DLL.

4.6.4.2.2.24 MaxSendingQueueDepth

This attribute specifies the maximum number of FAL-PDUs that can be queued for transmission.

This attribute supplies the value for the “Maximum queue depth” parameter of the DLL for Send queues.

4.6.4.2.2.25 MaxReceivingQueueDepth

This attribute specifies the maximum number of FAL-PDUs that can be queued at reception.

This attribute supplies the value for the “Maximum queue depth” parameter of the DLL for Receive queues.

4.6.4.2.3 DLL services

Refer to 4.7.4, Data Link Layer Service Selection, for DLL service descriptions.

4.6.4.3 QUB-seg ARPM state machine

4.6.4.3.1 QUB-seg ARPM states

The defined states and their descriptions of the QUB-Seg ARPM are shown in Table 75 and Figure 31.

Table 75 – QUB-seg ARPM states

State	Description
CLOSED	The AREP is defined, but not capable of sending or receiving FAL-PDUs. It may send or receive Establish service FAL-PDUs while in this state.
OPEN	The AREP is defined and capable of sending or receiving FAL-PDUs.
REQUESTING (REQ)	The AREP has sent an Establish Request FAL-PDU and is waiting for a response from the remote AREP.
RESPONDING (RSP)	The AREP has received an Establish Request FAL-PDU, delivered an Establish.indication primitive and is waiting for a response from its user.
REPLIED (REPL)	The Server AREP has issued an EST_rsp(+) primitive and is waiting for receiving a "connection-established" indication from the DLL.
SAME	Indicates that the next state is the same as the current state.

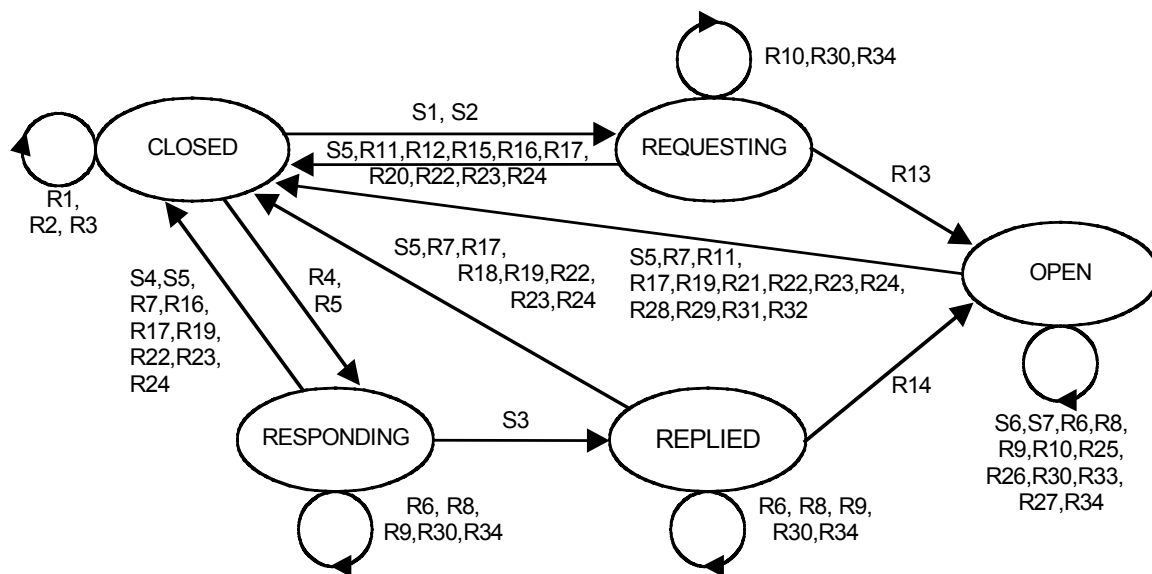


Figure 31 – State transition diagram of QUB-seg ARPM

4.6.4.3.2 QUB-seg ARPM state table

Table 76 and Table 77 define the QUB-seg ARPM state machine.

Table 76 – QUB-seg ARPM state table – sender transactions

#	Current state	Event or condition => action	Next state
S1	CLOSED	<pre> EST_req && Initiator = "True" && RemoteAddressConfigurationType = "Free" => RemoteDlcepAddress := remote_dlcep_address, FAL-PDU_req { dmpm_service_name := "DMPM_Connect_req", arep_id := GetArepld (), called_address := "default dlsap address", calling_address := "default dlsap address", local_dlcep_address := LocalDlcep, dlsdu := BuildFAL-PDU (fal_pdu_name := "EST_ReqPDU", calling_dlcep_address := LocalDlcepAddress, called_dlcep_address := RemoteDlcepAddress, fal_data := user_data) } </pre>	REQ
S2	CLOSED	<pre> EST_req && Initiator = "True" && RemoteAddressConfigurationType = "Linked" => FAL-PDU_req { dmpm_service_name := "DMPM_Connect_req", arep_id := GetArepld (), called_address := "default dlsap address", calling_address := "default dlsap address", local_dlcep_address := LocalDlcep, dlsdu := BuildFAL-PDU (fal_pdu_name := "EST_ReqPDU", calling_dlcep_address := LocalDlcepAddress, called_dlcep_address := RemoteDlcepAddress, fal_data := user_data) } </pre>	REQ
S3	RSP	<pre> EST_rsp(+) => FAL-PDU_req { dmpm_service_name := "DMPM_Connect_rsp", arep_id := GetArepld (), responding_address := "default dlsap address", local_dlcep_address := LocalDlcep, dlsdu := BuildFAL-PDU (fal_pdu_name := "EST_RspPDU", fal_data := user_data) } </pre>	REPL
S4	RSP	<pre> EST_rsp(-) => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "connection rejection–transient condition", dlsdu := BuildFAL-PDU (fal_pdu_name := "EST_ErrPDU", fal_data := user_data) } </pre>	CLOSED
S5	NOT CLOSED	<pre> Abort_req => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "disconnection–normal condition", dlsdu := BuildFAL-PDU (fal_pdu_name := "Abort_PDU", fal_id := identifier, fal_reason_code := reason_code, fal_additional_detail := additional_detail) } </pre>	CLOSED

Table 76 (continued)

#	Current state	Event or condition => action	Next state
S6	OPEN	<pre> CS_req && Role = "Client" "Peer" && PDU length < MaxDisduSizeFromRequestorNegotiated => FAL-PDU_req { dmpm_service_name := "DMPM_Data_req", arep_id := GetArepld (), dlsdu := BuildFAL-PDU (fal_pdu_name := "CS_ReqPDU", fal_data := user_data) } </pre>	OPEN
S7	OPEN	<pre> CS_rsp && Role = "Server" "Peer" && PDU length < MaxDisduSizeFromRequestorNegotiated => FAL-PDU_req { dmpm_service_name := "DMPM_Data_req", arep_id := GetArepld (), dlsdu := BuildFAL-PDU (fal_pdu_name := "CS_RspPDU", fal_data := user_data) } </pre>	OPEN
S8	OPEN	<pre> UCS_req && Role = "Server" "Peer" && PDU length < MaxDisduSizeFromRequestorNegotiated => FAL-PDU_req { dmpm_service_name := "DMPM_Data_req", arep_id := GetArepld (), dlsdu := BuildFAL-PDU (fal_pdu_name := "UCS_ReqPDU", fal_data := user_data) } </pre>	OPEN
S9	OPEN	<pre> CS_req && Role = "Client" "Peer" && PDU length >= MaxDisduSizeFromRequestorNegotiated => for (n := 1 to (PDU length - 1) div (MaxDisduSizeFromRequestorNegotiated-1)+1) FAL-PDU_req { dmpm_service_name := "DMPM_Data_req", arep_id := GetArepld (), dlsdu := BuildFAL-Segment (fal_pdu_name := "CS_ReqPDU", fal_data := Segment_data, n) } endfor </pre> NOTE 1 When the length of the FAL-PDU is greater or equal to max size Disdu negotiated, the QUB_Seg protocol splits the FAL-PDU into N ordered segment_data numbered from 1 to N. The first Segment_data contain the FAL_Header. NOTE 2 For each Segment_data, the function "Build-FAL-Segment" builds an APDU-Segment := FAL Header followed with Segment_data without any gap. The header shall have its bit 4 := 1 for the Segment_data 1 to N-1, and 0 for the Segment_data N. NOTE 3 The first APDU-Segment to be sent through the the network shall contain the Segment_data 1 and so on.	OPEN

Table 76 (continued)

#	Current state	Event or condition => action	Next state
S10	OPEN	CS_rsp && Role = "Server" "Peer" && PDU length >= MaxDlsduSizeFromRequestorNegotiated => for (n := 1 to (PDU length - 1) div (MaxDlsduSizeFromRequestorNegotiated-1)+1) FAL-PDU_req { dmpm_service_name := "DMPM_Data_req", arep_id := GetArepld (), dlsdu := BuildFAL-Segment (fal_pdu_name := "CS_RspPDU", fal_data := Segment_data, n) } endfor NOTE 1 When the length of the FAL-PDU is greater or equal to max size Dlsdu negotiated, the QUB_Seg protocol splits the FAL-PDU into N ordered segment_data numbered from 1 to N. The first Segment_data shall contain the FAL_Header. NOTE 2 For each segment_data, the function "Build-FAL-Segment" builds an APDU-Segment := FAL Header followed with Segment_data without any gap. The header shall have its bit 4 := 1 for the segment_data 1 to N-1, and 0 for the segment_data N. NOTE 3 The first APDU-Segment to be sent through the network shall contain the Segment_data 1 and so on.	OPEN
S11	OPEN	UCS_req && Role = "Server" "Peer" && PDU length >= MaxDlsduSizeFromRequestorNegotiated => for (n := 1 to (PDU length - 1) div (MaxDlsduSizeFromRequestorNegotiated-1)+1) FAL-PDU_req { dmpm_service_name := "DMPM_Data_req", arep_id := GetArepld (), dlsdu := BuildFAL-Segment (fal_pdu_name := "UCS_ReqPDU", fal_data := SEgment_data, n) } endfor NOTE 1 When the length of the FAL-PDU is greater or equal to max size Dlsdu negotiated, the QUB_Seg protocol splits the FAL-PDU into N ordered Segment_data numbered from 1 to N. The first Segment_data shall contain the FAL_Header. NOTE 2 For each Segment_data, the function "Build-FAL-Segment" builds an APDU-Segment := FAL Header followed with Segment_data without any gap. The header shall have its bit 4 := 1 for the Segment_data 1 to N-1, and 0 for the Segment_data N. NOTE 3 The first APDU-Segment to be sent through the the network contains the Segment_data 1 and so on.	OPEN

Table 77 – ARPM state table – receiver transactions

#	Current state	Event or condition => action	Next state
R1	CLOSED	Connect_ind && Initiator = "True" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Multiple Initiators", dlsdu := "null" }	CLOSED
R2	CLOSED	Connect_ind && Initiator = "False" && FAL_Pdu_Type (dls_user_data) <> "EST_ReqPDU" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }	CLOSED
R3	CLOSED	Connect_ind && Initiator = "False" && FAL_Pdu_Type (dls_user_data) = "EST_ReqPDU" && RemoteAddressConfigurationType = "Linked" && RemoteDlcepAddress <> calling_dlcep_address => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Remote Address Mismatch", dlsdu := "null" }	CLOSED
R4	CLOSED	Connect_ind && Initiator = "False" && FAL_Pdu_Type (dls_user_data) = "EST_ReqPDU" && RemoteAddressConfigurationType = "Free" => RemoteDlcepAddress := calling_dlcep_address, MaxDlsduSizeFromRequestorNegotiated := dlsdu_size_from_requestor, MaxDlsduSizeFromResponderNegotiated := dlsdu_size_from_responder, EST_ind { arep_id := GetArepld (), user_data := dls_user_data }	RSP
R5	CLOSED	Connect_ind && Initiator = "False" && FAL_Pdu_Type (dls_user_data) = "EST_ReqPDU" && RemoteAddressConfigurationType = "Linked" && RemoteDlcepAddress = calling_dlcep_address => MaxDlsduSizeFromRequestorNegotiated := dlsdu_size_from_requestor, MaxDlsduSizeFromResponderNegotiated := dlsdu_size_from_responder, EST_ind { arep_id := GetArepld (), user_data := dls_user_data }	RSP

Table 77 (continued)

#	Current state	Event or condition => action	Next state
R6	RSP REPL OPEN	Connect_ind && Initiator = "False" && RemoteAddressConfigurationType = "Linked" && RemoteDlcepAddress <> calling_dlcep_address => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Remote Address Mismatch", dlsdu := "null" }	SAME
R7	RSP REPL OPEN	Connect_ind && Initiator = "False" && RemoteDlcepAddress = calling_dlcep_address => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU" }	CLOSED
R8	RSP REPL OPEN	Connect_ind && Initiator = "False" && FAL_Pdu_Type (dls_user_data) = "EST_ReqPDU" && RemoteAddressConfigurationType = "Free" && RemoteDlcepAddress <> calling_dlcep_address => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "AREP Busy", dlsdu := "null" }	SAME
R9	RSP REPL OPEN	Connect_ind && Initiator = "False" && FAL_Pdu_Type (dls_user_data) <> "EST_ReqPDU" && RemoteAddressConfigurationType = "Free" && RemoteDlcepAddress <> calling_dlcep_address => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }	SAME
R10	REQ OPEN	Connect_ind && Initiator = "True" && RemoteDlcepAddress <> calling_dlcep_address => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Multiple Initiators", dlsdu := "null" }	SAME

Table 77 (continued)

#	Current state	Event or condition => action	Next state
R11	REQ OPEN	Connect_ind && Initiator = "True" && RemoteDlcepAddress = calling_dlcep_address => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Multiple Initiators", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Multiple Initiators" }	CLOSED
R12	REQ	Connect_cnf && FAL_Pdu_Type (dls_user_data) <> "EST_RspPDU" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU" }	CLOSED
R13	REQ	Connect_cnf && FAL_Pdu_Type (dls_user_data) = "EST_RspPDU" => ResetIntermediatePDU () MaxDlsduSizeFromRequestorNegotiated := dlsdu_size_from_requestor, MaxDlsduSizeFromResponderNegotiated := dlsdu_size_from_responder, DIIPriorityNegotiated := dll_priority, EST_cnf(+) { arep_id := GetArepld (), user_data := dls_user_data }	OPEN
R14	REPL	FAL-PDU_ind && dmpm_service_name = "DMPM_Connection_Established_ind" => ResetIntermediatePDU ()	OPEN
R15	REQ	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && FAL_Pdu_Type (fal_pdu) = "EST_ErrPDU" => EST_cnf(-) { arep_id := GetArepld (), user_data := dls_user_data }	CLOSED

Table 77 (continued)

#	Current state	Event or condition => action	Next state
R16	REQ RSP	FAL-PDU_ind && dmpm_service_name <> "DMPM_Disconnect_ind" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid DL-Event", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid DL-Event" }	CLOSED
R17	NOT CLOSED	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu <> "null" && FAL_Pdu_Type (fal_pdu) = "Abort_PDU" => Abort_ind{ arep_id := GetArepld (), locally_generated := "False", identifier := AbortIdentifier (fal_pdu), reason_code := AbortReason (fal_pdu), additional_detail := AbortDetail (fal_pdu) }	CLOSED
R18	REPL	FAL-PDU_ind && ((dmpm_service_name <> "DMPM_Disconnect_ind") && (dmpm_service_name <> "DM_Connection_Established_ind")) => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid DL-Event", dlsdu := "null" }, Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid DL-Event", additional_detail := "null" }	CLOSED
R19	REPL RSP OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu <> "null" && FAL_Pdu_Type (fal_pdu) <> "Abort_PDU" => Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU", additional_detail := "null" }	CLOSED
R20	REQ	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu <> "null" && ((FAL_Pdu_Type (fal_pdu) <> "Abort_PDU") && (FAL_Pdu_Type (fal_pdu) <> "EST_ErrPDU")) => Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU", additional_detail := "null" }	CLOSED

Table 77 (continued)

#	Current state	Event or condition => action	Next state
R21	OPEN	FAL-PDU_ind && ((dmpm_service_name <> "DMPM_Disconnect_ind") && (dmpm_service_name <> "DMPM_Data_ind")) => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid DL-Event", dlsdu := "null" }, Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid DL-Event", additional_detail := "null" }	CLOSED
R22	NOT CLOSED	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu = "null" && originator = "remote_dls_provider" => Abort_ind{ arep_id := GetArepld (), locally_generated := "False", identifier := "Data Link Layer", reason_code := reason, additional_detail := "null" }	CLOSED
R23	NOT CLOSED	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu = "null" && originator = "remote_dls_user" => Abort_ind{ arep_id := GetArepld (), locally_generated := "False", identifier := "FAL", reason_code := reason, additional_detail := "null" }	CLOSED
R24	NOT CLOSED	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu = "null" && originator = "local_dls_provider" => Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "Data Link Layer", reason_code := reason, additional_detail := "null" }	CLOSED

Table 77 (continued)

#	Current state	Event or condition => action	Next state
R25	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Peer" "Server" && FAL_Pdu_Type (fal_pdu) = "CS_ReqPDU" && AddSegment (fal_pdu) = "OK" => if (MoreFollows(fal_pdu) = "False") CS_ind { arep_id := GetArepld (), user_data := GetIntermediatePDU () } endif NOTE When the length of an FAL-PDU is greater or equal to the max size Disdu negotiated, the DLL delivers the received APDU segments, starting with the APDU segment 1 and ending with the APDU segment N (the same order as they were sent by the sender). Each APDU_Segment contains an FAL header and a Segment_data. The DLL disconnects the channel if a DLL DLPDU (segment) is missing, the QUB_Seg protocol reassembles the Segment_data of the N received APDU segments. The N-1 APDU_Segments shall have a Header with bit 4 = 1, and the last APDU_Segment shall have a header with bit 4 = 0. The function "AddSegment" removes the header of the APDU_Segment and appends the Segment_data to the previous received Segment_data. Once the N Segment_data are appended together without any gap, the function "GetintermediatePDU" gives the original FAL-APDU.	OPEN
R26	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Client" "Peer" && FAL_Pdu_Type (fal_pdu) = "CS_RspPDU" && AddSegment (fal_pdu) = "OK" => if (MoreFollows(fal_pdu) = "False") CS_cnf { arep_id := GetArepld (), user_data := fal_pdu } endif NOTE When the length of an FAL-PDU is greater or equal to the max size Disdu negotiated, the DLL delivers the received APDU segments, starting with the APDU segment 1 and ending with the APDU segment N (the same order as they were sent by the sender). Each APDU_Segment contains a FAL header and a Segment_data. The DLL disconnects the channel if a DLL DLPDU (segment) is missing, the QUB_Seg protocol reassembles the Segment_data of the N received APDU segments. The N-1 APDU_segments shall have a Header with bit 4 = 1, and the last APDU_Segment shall have a header with bit 4 = 0. The function "AddSegment" removes the header of the APDU_Segment and appends the Segment_data to the previous received Segment_data. Once the N Segment_data are appended together without any gap, the function "GetintermediatePDU" gives the original FAL-APDU.	OPEN

Table 77 (continued)

#	Current state	Event or condition => action	Next state
R27	OPEN	<pre> FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Server" "Peer" && FAL_Pdu_Type (fal_pdu) = "UCS_ReqPDU" && AddSegment (fal_pdu) = "OK" => if (MoreFollows(fal_pdu) = "False") UCS_ind { arep_id := GetArepld (), remote_dlsap_address = "null", user_data := fal_pdu } endif NOTE When the length of an FAL-PDU is greater or equal to the max size Dlsdu negotiated, the DLL delivers the received APDU segments, starting with the APDU segment 1 and ending with the APDU segment N (the same order as they were sent by the sender). Each APDU_Segment contains a FAL header and a Segment_data. The DLL disconnects the channel if a DLL DLPDU (segment) is missing, the QUB_Seg protocol reassembles the Segment_data of the N received APDU segments. The N-1 APDU_Segments shall have a Header with bit 4 = 1, and the last APDU_Segment shall have a header with bit 4 = 0. The function "AddSegment" removes the header of the APDU_Segment and appends the Segment_data to the previous received Segment_data. Once the N Segment_data are appended together without any gap, the function "GetintermediatePDU" gives the original FAL-APDU. </pre>	OPEN
R28	OPEN	<pre> FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Peer" "Server" && FAL_Pdu_Type (fal_pdu) = "CS_ReqPDU UCS_ReqPDU" && AddSegment (fal_pdu) <> "OK" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU", additional_detail := "null" } </pre>	CLOSED
R29	OPEN	<pre> FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Peer" "Client" && (FAL_Pdu_Type (fal_pdu) = "CS_RspPDU") && AddSegment (fal_pdu) <> "OK" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU", additional_detail := "null" } </pre>	CLOSED

Table 77 (continued)

#	Current state	Event or condition => action	Next state
R30	OPEN	<pre> FAL-PDU_ind && dmpm_service_name = "DMPM_Data_Cnf(-)" && Role = "Server" && FAL_Pdu_Type (fal_pdu) <> "CS_RspPDU" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU", additional_detail := "null" } </pre>	CLOSED
R31	OPEN	<pre> FAL-PDU_ind && dmpm_service_name = "DMPM_Data_Cnf" && Role = "Client" && FAL_Pdu_Type (fal_pdu) <> "CS_RspPDU" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU", additional_detail := "null" } </pre>	CLOSED
R32	OPEN	<pre> FAL-PDU_ind && Role = "Peer" && dmpm_service_name = "DMPM_Data_ind" && ((FAL_Pdu_Type (fal_pdu) <> "CS_ReqPDU") && (FAL_Pdu_Type (fal_pdu) <> "CS_RspPDU") && (FAL_Pdu_Type (fal_pdu) <> "UCS_ReqPDU")) => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU", additional_detail := "null" } </pre>	CLOSED

Table 77 (continued)

#	Current state	Event or condition => action	Next state
R33	OPEN	FAL-PDU_Ind && dmpm-service-name = "DMPM_Data_Cnf(-)" && Role = "Client Peer" &&FAL_Pdu_Type(fal_pdu) = "UCS_ReqPDU" => <pre> FAL_PDU_Req{ dmpm-service-name := "DMPM_Disconnect-req" arep-Id := GetArepId(), reason := "dl-status", dl-sddu := "null" } Abort_Ind{ arep_Id := GetArepId() locally-generated := "true" Identifier := "Data Link Layer" reason code := reason additional_detail := "null" } </pre>	OPEN
R34	NOT CLOSED	ErrorToARPM => (no actions taken) NOTE 1 It is a local matter to report this error status to network management entities. The ARPM does not abort the existing connections. The FAL user may issue an Abort request to disconnect the current connection, depending on the type of status information conveyed by the ErrorToARPM primitive. NOTE 2 This event mainly occurs when the DLL has not been able to convey a DL service to the remote partner within the specified time interval. In this case an implementation may also decide to pass negative responses for all confirmed FAL service requests that are affected by this failure to the respective FAL user.	SAME

4.6.4.3.3 Functions used by QUB-seg ARPM

Table 78 through Table 88 define the functions used by this state machine.

Table 78 – Function GetArepId

Name	GetArepId	Used in	ARPM
Input		Output	
(none)		AREP Identifier	
Function	Returns a value that can unambiguously identify the current AREP.		

Table 79 – Function BuildFAL-PDU

Name	BuildFAL-PDU	Used in	ARPM
Input		Output	
fal_pdu_name, calling_dlcep_address, called_dlcep_address, fal_data, fal_id, fal_reason_code, fal_additional_detail		dlIsdu	
Function	Builds an FAL-PDU out of the parameters given as input variables.		

Table 80 – Function FAL_Pdu_Type

Name	FAL_Pdu_Type	Used in	ARPM
Input		Output	
dls_user_data		One of the FAL-PDU types defined in the FAL-PDUs subclause.	
Function	This function decodes the FAL-PDU that is conveyed in the dls_user_data parameter and retrieves one of the FAL-PDU types.		

Table 81 – Function AbortIdentifier

Name	AbortIdentifier	Used in	ARPM
Input		Output	
fal_pdu		The Identifier parameter of the Abort service.	
Function	This function decodes the Abort_PDU that is conveyed in the fal_pdu parameter and extracts the Identifier parameter.		

Table 82 – Function AbortReason

Name	AbortReason	Used in	ARPM
Input		Output	
fal_pdu		The Reason Code parameter of the Abort service.	
Function	This function decodes the Abort_PDU that is conveyed in the fal_pdu parameter and extracts the Reason Code parameter.		

Table 83 – Function AbortDetail

Name	AbortDetail	Used in	ARPM
Input		Output	
fal_pdu		The Additional Detail parameter of the Abort service.	
Function	This function decodes the Abort_PDU that is conveyed in the fal_pdu parameter and extracts the Additional Detail parameter.		

Table 84 – Function BuildFAL-segment

Name	BuildFAL-Segment	Used in	ARPM
Input		Output	
fal_pdu_name, fal_data, segment_number		dlsdu	
Function	Builds an FAL-Segment out of the parameters given as input variables. The segment_number shall be >= 1 and defines, which part of the fal_pdu is to be put in the returned segment. This function adds the FAL-Header to each segment. It sets bit 4 of the FAL-Header to 1, indicating that there are more segments to be sent, when it is not the last segment. Otherwise bit 4 is set to 0. Except for this bit the FAL-Header is identical in all segments of one FAL-PDU.		

Table 85 – Function MoreFollows

Name	MoreFollows	Used in	ARPM
Input		Output	
dls_user_data		Boolean value	
Function			
This function inspects the FAL-PDU that is conveyed in the dls_user_data parameter and returns "True", if bit5 of the FAL-Header is set. Otherwise it returns "False".			

NOTE The following three functions make use of a persistent variable IntermediatePDU, in which all segments of the current FAL User Data are stored by the receiver of this data.

Table 86 – Function ResetIntermediatePDU

Name	ResetIntermediatePDU	Used in	ARPM
Input		Output	
(none)		(none)	
Function	This function removes all segments which have so far been collected from the IntermediatePDU.		

Table 87 – Function AddSegment

Name	AddSegment	Used in	ARPM
Input		Output	
dls_user_data		Error Code	
Function	This function adds the received dls_user_data as a new segment to the IntermediatePDU. It checks segments reassembling protocol error. If no error was detected during these checks, the error code "OK" is returned.		

Table 88 – Function GetIntermediatePDU

Name	GetIntermediatePDU	Used in	ARPM
Input		Output	
(none)		fal_user_data	
Function	This function returns the complete FAL user data, which was received in multiple segments. After this function call the variable IntermediatePDU does not contain any segments.		

4.6.5 Queued User-triggered Bidirectional-Flow Control (QUB-FC) ARPM

4.6.5.1 QUB-FC Primitive definitions

4.6.5.1.1 Primitives exchanged between ARPM and FSPM

Table 89 and Table 90 list the primitives exchanged between the ARPM and the FSPM.

Table 89 – Primitives issued by FSPM to ARPM

Primitive name	Source	Associated parameters	Functions
EST_req	FSPM	user_data	This is an FAL internal primitive used to convey an Establish request primitive from the FSPM to the ARPM.
EST_rsp(+)	FSPM	user_data	This is an FAL internal primitive used to convey an Establish response(+) primitive from the FSPM to the ARPM.
EST_rsp(-)	FSPM	user_data	This is an FAL internal primitive used to convey an Establish response(-) primitive from the FSPM to the ARPM.
Abort_req	FSPM	identifier, reason_code, additional_detail	This is an FAL internal primitive used to convey an Abort request primitive from the FSPM to the ARPM.
CS_req	FSPM	user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) request primitive from the FSPM to the ARPM.
CS_rsp	FSPM	user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) response primitive from the FSPM to the ARPM.
UCA_req	FSPM	remote_dlsap_address, user_data	This is an FAL internal primitive used to convey an Unconfirmed Acknowledged Send (UCA) request primitive from the FSPM to the ARPM.
AR_XON_OFF_req	FSPM	remote_dlsap_address, user_data	This is an FAL internal primitive used to convey AR-XON-OFF request primitive from the FSPM to the ARPM.

Table 90 – Primitives issued by ARPM to FSPM

Primitive name	Source	Associated parameters	Functions
EST_ind	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish indication primitive from the ARPM to the FSPM.
EST_cnf(+)	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish response(+) primitive from the ARPM to the FSPM.
EST_cnf(-)	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish response(-) primitive from the ARPM to the FSPM.
Abort_ind	ARPM	arep_id, locally_generated, identifier, reason_code, additional_detail	This is an FAL internal primitive used to convey an Abort primitive from the ARPM to the FSPM.
CS_ind	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) indication primitive from the ARPM to the FSPM.
CS_cnf	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) confirmation primitive from the ARPM to the FSPM.
UCA_ind	ARPM	arep_id, remote_dlsap_address, duplicate_fal_sdu, user_data	This is an FAL internal primitive used to convey an Unconfirmed Acknowledged Send (UCA) indication primitive from the ARPM to the FSPM.
AR_XON_OFF_ind	ARPM	arep_id, remote_dlsap_address, user_data	This is an FAL internal primitive used to convey a AR-XON-OFF indication primitive from the ARPM to the FSPM.

4.6.5.2 Parameters of FSPM/ARPM primitives

The parameters used with the primitives exchanged between the FSPM and the ARPM are described in Table 91.

Table 91 – Parameters used with primitives exchanged between FSPM and ARPM

Parameter name	Description
arep_id	This parameter is used to unambiguously identify an instance of the AREP that has issued a primitive. A means for such identification is not specified by this standard.
User_data	This parameter conveys FAL-User data.
Locally_generated	This parameter conveys value that is used for the Locally_Generated parameter.
Identifier	This parameter conveys value that is used for the Identifier parameter.
Reason_code	This parameter conveys value that is used for the Reason_Code parameter.
Additional_detail	This parameter conveys value that is used for the Additional_Detail parameter.
Duplicate_fal_sdu	This parameter conveys value that is used for the Duplicate_FAL-SDU parameter.
Remote_dlsap_address	This parameter conveys value that is used for the Remote_DLSAP_Address parameter.
Status	This parameter conveys value that is used for the Status parameter.
Reported_status	This parameter conveys a Data Link Layer event status.
Local_timeliness	This parameter conveys value that is used for the Local_Timeliness parameter.
Remote_timeliness	This parameter conveys value that is used for the Remote_Timeliness parameter.

4.6.5.3 DLL mapping of QUB-FC AREP class

4.6.5.3.1 QUB-FC AREP class formal model

This subclause describes the mapping of the QUB-FC AREP class to the Fieldbus Data Link Layer. It does not redefine the DLSAP attributes or DLME attributes that are or will be defined in the Data link layer standard; rather, it defines how they are used by this AR class.

The DLL Mapping attributes and their permitted values and the DLL services used with the QUB-FC AREP class are defined in this subclause.

CLASS: QubFC

PARENT CLASS: QueuedUser-TriggeredBidirectional-FlowControlAREP

ATTRIBUTES:

- | | | | |
|---------|-----|---------------|---|
| 1 | (m) | KeyAttribute: | LocalDlcepAddress |
| 2 | (m) | Attribute: | RemoteDlcepAddress |
| 3 | (m) | Attribute: | DlsapRole (Basic) |
| 4 | (m) | Attribute: | QosParameterSet |
| 4.1 | (m) | Attribute: | DlcepClass (Peer) |
| 4.2 | (m) | Attribute: | DlcepDataDeliveryFeatures |
| 4.2.1 | (m) | Attribute: | FromRequestorToResponder (Classical, Disordered) |
| 4.2.2 | (m) | Attribute: | FromResponderToRequestor (Classical, Disordered) |
| 4.3 | (m) | Attribute: | Priority |
| 4.3.1 | (m) | Attribute: | DlIPriority (Urgent, Normal, TimeAvailable) |
| 4.3.2 | (m) | Attribute: | DlIPriorityNegotiated (Urgent, Normal, TimeAvailable) |
| 4.4 | (m) | Attribute: | DlpduAuthentication (Ordinary, Source, Maximal) |
| 4.5 | (m) | Attribute: | ResidualActivity |
| 4.5.1 | (m) | Attribute: | ResidualActivityAsSender (True, False) |
| 4.5.2 | (m) | Attribute: | ResidualActivityAsReceiver (True, False) |
| 4.6 | (m) | Attribute: | MaxConfirmDelay |
| 4.6.1 | (m) | Attribute: | MaxConfirmDelayOnDIConnect |
| 4.6.2 | (m) | Attribute: | MaxConfirmDelayOnDIData |
| 4.7 | (m) | Attribute: | DISchedulingPolicy (Implicit) |
| 4.8 | (m) | Attribute: | ExplicitQueue (True, False) |
| 4.9 | (c) | Constraint: | ExplicitQueue = True |
| 4.9.1 | (m) | Attribute: | MaxDlsduSizes |
| 4.9.1.1 | (m) | Attribute: | MaxDlsduSizeFromRequestor |
| 4.9.1.2 | (m) | Attribute: | MaxDlsduSizeFromResponder |
| 4.9.1.3 | (m) | Attribute: | MaxDlsduSizeFromRequestorNegotiated |
| 4.9.1.4 | (m) | Attribute: | MaxDlsduSizeFromResponderNegotiated |
| 4.9.2 | (m) | Attribute: | MaxQueueDepth |
| 4.9.2.1 | (m) | Attribute: | MaxSendingQueueDepth |
| 4.9.2.2 | (m) | Attribute: | MaximimReceivingQueueDepth |

DLL SERVICES:

- | | | | |
|-----|-----|-------------|---------------------------|
| 1 | (m) | OpsService: | DL-Data |
| 2 | (c) | Constraint: | ExplicitQueue = True |
| 2.1 | (m) | OpsService: | DL-Get |
| 3 | (m) | OpsService: | DL-Connect |
| 4 | (m) | OpsService: | DL-Connection-Established |
| 5 | (m) | OpsService: | DL-Disconnect |

4.6.5.3.2 Attributes

4.6.5.3.2.1 LocalDlcepAddress

This attribute specifies the local DLCEP address and identifies the DLCEP.

The value of this attribute is used as the "DLCEP-address" parameter of the DLL.

NOTE 1 The value of this attribute is also carried in the Establish Request PDU.

This attribute contains the following three subattributes: Link Address, Node Address, and Selector.

NOTE 2 Since the local Link and Node addresses are set by the Network Management, only the Selector portion of the LocalDlsapAddress attribute is a configurable attribute of the FAL.

4.6.5.3.2.2 RemoteDlcepAddress

This attribute specifies the remote DLCEP address and identifies the DLCEP. This attribute supplies the value for called DLCEP-address of the DL-Connect service.

NOTE The value of this attribute is also carried in the header part of the Establish Request PDU.

This attribute contains the following three subattributes: Link Address, Node Address, and Selector.

4.6.5.3.2.3 DisapRole

This attribute specifies the behavior of the DLSAP that is used by the AREP. This attribute supplies the value for the "DL(SAP)-role" parameter. The possible value Basic corresponds to BASIC as defined in the Data link layer standard.

4.6.5.3.2.4 QosParameterSet

The QosParameterSet attributes specify the DL quality of service that is used by this AREP. These attribute values may be negotiated with the remote AREP.

4.6.5.3.2.5 DlcepClass

This attribute specifies the behavior of the DLCEP which is attached to the AREP. This attribute supplies the value for the "DLCEP class" parameter of the DLL. The possible value of this attribute is Peer and corresponds to Peer defined by the DLL.

4.6.5.3.2.6 DlcepDataDeliveryFeatures

These two attributes specify data delivery features of the DLL. The permitted values Classical and Disordered correspond, respectively to CLASSICAL and DISORDERED defined by the Data link layer standard.

The FromRequestorToResponder and FromResponderToRequestor attributes shall have the same value.

4.6.5.3.2.7 FromRequestorToResponder

This attribute specifies the DLL data delivery feature of the DLPDUs sent from the AREP whose Initiator attribute has a value of "True" to the remote AREP. It supplies the value for the "DLCEP data delivery features from requestor to responder(s)" parameter defined in the DLL.

4.6.5.3.2.8 FromResponderToRequestor

This attribute specifies the DLL data delivery feature of the DLPDUs sent from the AREP whose Initiator attribute has a value of "False" to the remote AREP. It supplies the value for the "DLCEP data delivery features from responder(s) to requestor" parameter defined in the DLL.

4.6.5.3.2.9 DIIPriority

This attribute specifies the configured value of the DLL priority.

NOTE It is not possible to use different priorities for each FAL-PDU sent from the same QUB AREP. Also, it is not permitted to use different priorities for the send and receive conveyance paths.

4.6.5.3.2.10 DIIPriorityNegotiated

This attribute specifies the negotiated value of the DLL priority.

4.6.5.3.2.11 DlpduAuthentication

This attribute specifies the lower bound of the length of DL-addressing to be used by the DLL.

This attribute supplies the value for the “DLPDU-authentication” parameter of the DLL. The permitted value Ordinary, Source, and Maximal correspond to ORDINARY, Source, and MAXIMAL, respectively, as defined in Part-3 and –4 of this International Standard.

4.6.5.3.2.12 ResidualActivityAsSender

This attribute specifies the sender’s DLC residual activity. It supplies the value for the “Residual activity as sender” parameter defined in the DLL. The possible values are “True” and “False.”

4.6.5.3.2.13 ResidualActivityAsReceiver

This attribute specifies the receiver’s DLC residual activity. It supplies the value for the “Residual activity as receiver” parameter defined in the DLL. The possible values are “True” and “False.”

4.6.5.3.2.14 MaxConfirmDelay

This attribute specifies the maximum confirmation delay of certain DLL connection-oriented services.

4.6.5.3.2.15 MaxConfirmDelayOnDIConnect

This attribute specifies the maximum confirmation delay for a confirmation from a DL-Connect service.

This attribute supplies the value for the “Maximum confirmation delay on DL-Connect, DL-Reset and DL-Subscriber-Query” parameter specified in the Data link layer standard.

4.6.5.3.2.16 MaxConfirmDelayOnDIData

This attribute specifies the maximum confirmation delay for a confirmation from a DL-Data service.

This attribute supplies the value for the “Maximum confirmation delay on DL-Data” parameter specified in the Data Link Layer standard.

4.6.5.3.2.17 DISchedulingPolicy

This attribute provides the guidance to the DLL on the scheduling needed by an AR. For this AREP, the DLL tries to transmit the FAL-PDU as soon as possible.

This attribute supplies the value for the “DL-Scheduling-policy” parameter of the DLL. The permitted value Implicit corresponds to IMPLICIT defined in the Data Link Layer standard.

4.6.5.3.2.18 MaxDlsduSizeFromRequestor

This attribute specifies the configured value of the maximum length of an FAL-PDU that can be sent from the AREP whose Initiator attribute has a value of “True” to the remote AREP.

This attribute supplies the value for the “Maximum DLSDU sizes from requestor” parameter of the DLL.

4.6.5.3.2.19 MaxDisduSizeFromResponder

This attribute specifies the configured value of the maximum length of an FAL-PDU that can be sent from the AREP whose Initiator attribute has a value of “False” to the remote AREP.

This attribute supplies the value for the “Maximum DLSDU sizes from responder” parameter of the DLL.

4.6.5.3.2.20 MaxDisduSizeFromRequestorNegotiated

This attribute specifies the negotiated value of the maximum length of an FAL-PDU that can be sent from the AREP whose Initiator attribute has a value of “True” to the remote AREP.

4.6.5.3.2.21 MaxDisduSizeFromResponderNegotiated

This attribute specifies the negotiated value of the maximum length of an FAL-PDU that can be sent from the AREP whose Initiator attribute has a value of “False” to the remote AREP.

4.6.5.3.2.22 MaxSendingQueueDepth

This attribute specifies the maximum number of FAL-PDUs that can be queued for transmission.

This attribute supplies the value for the “Maximum queue depth” parameter of the DLL for Send queues.

4.6.5.3.2.23 MaxReceivingQueueDepth

This attribute specifies the maximum number of FAL-PDUs that can be queued at reception.

This attribute supplies the value for the “Maximum queue depth” parameter of the DLL for Receive queues.

4.6.5.3.3 DLL services

Refer to 4.7.4, Data Link Layer Service Selection, for DLL service descriptions.

4.6.5.4 QUB-FC ARPM state machine

4.6.5.4.1 QUB-FC ARPM states

The defined states and their descriptions of the QUB-FC ARPM are shown in Table 92 and Figure 32.

Table 92 – QUB-FC ARPM states

State	Description
CLOSED	The AREP is defined, but not capable of sending or receiving FAL-PDUs. It may send or receive Establish service FAL-PDUs while in this state.
OPEN	The AREP is defined and capable of sending or receiving FAL-PDUs.
REQUESTING (REQ)	The AREP has sent an Establish Request FAL-PDU and is waiting for a response from the remote AREP.
RESPONDING (RSP)	The AREP has received an Establish Request FAL-PDU, delivered an Establish.indication primitive and is waiting for a response from its user.
REPLIED (REPL)	The Server AREP has issued an EST_rsp(+) primitive and is waiting for receiving a "connection-established" indication from the DLL.
SAME	Indicates that the next state is the same as the current state.

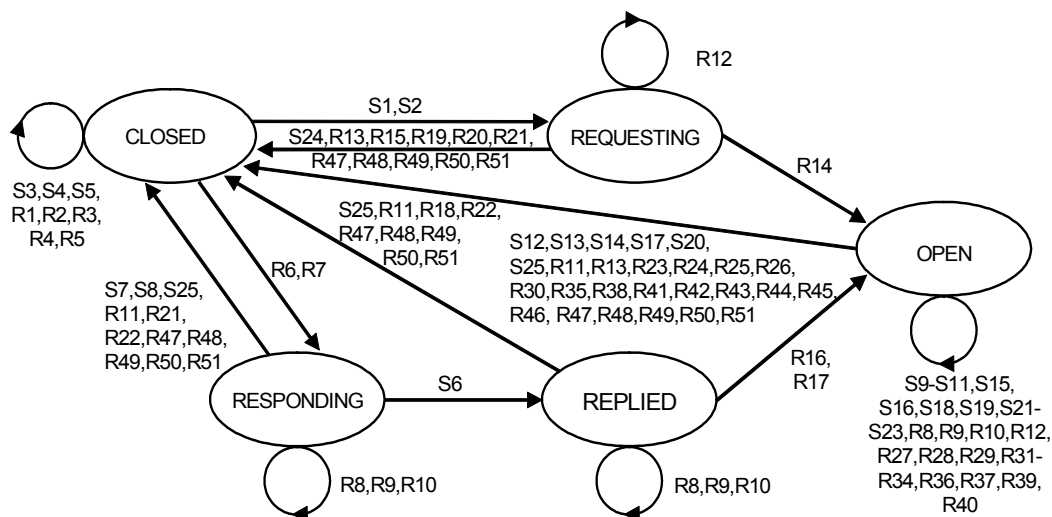


Figure 32 – State transition diagram of QUB-FC ARPM

4.6.5.4.2 QUB-FC ARPM state table

Table 93 and Table 94 define the state machine of the QUB-FC ARPM.

Table 93 – QUB-FC ARPM state table – sender transactions

#	Current state	Event or condition => action	Next state
S1	CLOSED	EST_req && Initiator = "True" && RemoteAddressConfigurationType := "Free" => RemoteDlcepAddress := remote_dlcep_address, FAL-PDU_req { dmpm_service_name := "DMPM_Connect_req", arep_id := GetArepld (), called_address := "default dlsep address", calling_address := "default dlsep address", local_dlcep_address := LocalDlcep, dlsep := BuildFAL-PDU (fal_pdu_name := "EST_ReqPDU", calling_dlcep_address := LocalDlcepAddress, called_dlcep_address := RemoteDlcepAddress, fal_data := user_data) } }	REQ
S2	CLOSED	EST_req && Initiator = "True" && RemoteAddressConfigurationType := "Linked" => FAL-PDU_req { dmpm_service_name := "DMPM_Connect_req", arep_id := GetArepld (), called_address := "default dlsep address", calling_address := "default dlsep address", local_dlcep_address := LocalDlcep, dlsep := BuildFAL-PDU (fal_pdu_name := "EST_ReqPDU", calling_dlcep_address := LocalDlcepAddress, called_dlcep_address := RemoteDlcepAddress, fal_data := user_data) } }	REQ

Table 93 (continued)

#	Current state	Event or condition => action	Next state
S3	CLOSED	EST_req && Initiator = "False" => Abort_ind { arep_id := GetArepId (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid Event for Role" } }	CLOSED
S4	CLOSED	unallowed primitive => Abort_ind { arep_id := GetArepId (), locally_generated := "True", identifier := "FAL", reason_code := "Unallowed AREP primitive in connection establishment " } }	CLOSED
S5	CLOSED	Abort_req => ignore	CLOSED
S6	RSP	EST_rsp(+) => FAL-PDU_req { dmpm_service_name := "DMPM_Connect_rsp", arep_id := GetArepId (), responding_address := "default dlsap address", local_dlcep_address := LocalDlcep, dlsdu := BuildFAL-PDU (fal_pdu_name := "EST_RspPDU", fal_data := user_data) } }	REPL
S7	RSP	EST_rsp(-) => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepId (), reason := "connection rejection-transient condition", dlsdu := BuildFAL-PDU (fal_pdu_name := "EST_ErrPDU", fal_data := user_data) } }	CLOSED
S8	RSP	unallowed primitive => Abort_ind { arep_id := GetArepId (), locally_generated := "True", identifier := "FAL", reason_code := "Unallowed AREP primitive in connection establishment " }, FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepId (), reason := " Unallowed AREP primitive in connection establishment ", dlsdu := "null" } }	CLOSED
S9	OPEN	CS_req && Role = "Client" "Peer" && Xon="True" && GetCounterValue(OSCC) < maxOSCC && CIU = 0 => FAL-PDU_req { dmpm_service_name := "DMPM_Data_req", arep_id := GetArepId (), dlsdu := BuildFAL-PDU (fal_pdu_name := "CS_ReqPDU", fal_data := user_data) }, IncrementCounter(OSCC)	OPEN

Table 93 (continued)

#	Current state	Event or condition => action	Next state
S10	OPEN	CS_req && Role = "Client" "Peer" && Xon="True" && GetCounterValue(OSCC) < maxOSCC && CIU > 0 => FAL-PDU_req { dmpm_service_name := "DMPM_Data_req", arep_id := GetArepld (), dlsdu := BuildFAL-PDU (fal_pdu_name := "CS_ReqPDU", fal_data := user_data) }, StartTimer(TC) IncrementCounter(OSCC)	OPEN
S11	OPEN	CS_req && Role = "Client" "Peer" && Xon="False" => CS_cnf (-) { arep_id := GetArepld (), user_data := "null" }	OPEN
S12	OPEN	CS_req && Role = "Client" "Peer" && Xon="True" && GetCounterValue(OSCC) ≥ maxOSCC => Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Number of parallel services exceeded" }, FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := " Number of parallel services exceeded", dlsdu := "null" }, StopTimer ResetCounters	CLOSED
S13	OPEN	CS_req && Role = "Server" => Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Unallowed service as server" }, FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Unallowed service as server", dlsdu := "null" }, StopTimer ResetCounters	CLOSED

Table 93 (continued)

#	Current state	Event or condition => action	Next state
S14	OPEN	<pre> CS_rsp && Role = "Server" "Peer" && CIU = 0 => FAL-PDU_req { dmpm_service_name := "DMPM_Data_req", arep_id := GetArepld (), dlsdu := BuildFAL-PDU (fal_pdu_name := "CS_RspPDU", fal_data := user_data) }, DecrementCounter(OSCS) </pre>	OPEN
S15	OPEN	<pre> CS_rsp && Role = "Server" "Peer" && CIU > 0 => FAL-PDU_req { dmpm_service_name := "DMPM_Data_req", arep_id := GetArepld (), dlsdu := BuildFAL-PDU (fal_pdu_name := "CS_RspPDU", fal_data := user_data) }, StartTimer(TS) DecrementCounter(OSCS) </pre>	OPEN
S16	OPEN	<pre> AR_XON_OFF_req && Role = "Server" "Peer" => FAL-PDU_req { dmpm_service_name := "DMPM_Data_req", arep_id := GetArepld (), dlsdu := BuildFAL-PDU (fal_pdu_name := "AR_XON_OFF_ReqPDU", fal_data := user_data) } </pre>	OPEN
S17	OPEN	<pre> CS_rsp && Role = "Client" "Peer" => Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Unallowed service as client" }, FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := " Unallowed service as client", dlsdu := "null" }, StopTimer ResetCounters </pre>	CLOSED
S18	OPEN	<pre> UCA_req && Xon = "True" && Role = "Client" "Peer" && GetCounterValue(UCC) < maxUCC && CIU = 0 => FAL-PDU_req { dmpm_service_name := "DMPM_Data_req", arep_id := GetArepld (), dlsdu := BuildFAL-PDU (fal_pdu_name := "UCA_PDU", fal_data := user_data) }, IncrementCounter(UCC) </pre>	OPEN

Table 93 (continued)

#	Current state	Event or condition => action	Next state
S19	OPEN	<pre> UCA_req && Xon = "True" && Role = "Client" "Peer" && GetCounterValue(UCC) < maxUCC && CIU > 0 => FAL-PDU_req { dmpm_service_name := "DMPM_Data_req", arep_id := GetArepld (), dlsdu := BuildFAL-PDU (fal_pdu_name := "UCA_PDU", fal_data := user_data) }, StartTimer(TC) IncrementCounter(UCC) </pre>	OPEN
S20	OPEN	<pre> UCA_req && Xon = "True" && Role = "Server" "Peer" => Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Unallowed service as server" }, FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Unallowed service as server", dlsdu := "null" }, StopTimer ResetCounters </pre>	CLOSED
S21	OPEN	<pre> UCA_req && Xon = "False" => (no actions taken) </pre>	OPEN
S22	OPEN	<pre> TCTimer expired => StartTimer(TC) FAL-PDU_req { dmpm_service_name := "DMPM_Data_req", arep_id := GetArepld (), dlsdu := BuildFAL-PDU (fal_pdu_name := "IdlePDU", fal_data := "null") } </pre>	OPEN
S23	OPEN	<pre> TSTimer expired => StartTimer(TS) FAL-PDU_req { dmpm_service_name := "DMPM_Data_req", arep_id := GetArepld (), dlsdu := BuildFAL-PDU (fal_pdu_name := "IdlePDU", fal_data := "null") } </pre>	OPEN

Table 93 (continued)

#	Current state	Event or condition => action	Next state
S24	NOT CLOSED	Abort_req => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "disconnection-normal condition", dlsdu := BuildFAL-PDU (fal_pdu_name := "Abort_PDU", fal_id := identifier, fal_reason_code := reason_code, fal_additional_detail := additional_detail) }, StopTimer, ResetCounters	CLOSED

Table 94 – QUB-FC ARPM state table – receiver transactions

#	Current state	Event or condition => action	Next state
R1	CLOSED	Connect_ind && Initiator = "True" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := dlcep_dl_id (from Connect_ind), reason := "Multiple Initiators", dlsdu := "null" }	CLOSED
R2	CLOSED	Connect_ind && Initiator = "False" && FAL_Pdu_Type (dls_user_data) <> "EST_ReqPDU" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := dlcep_dl_id (from Connect_ind), reason := "Invalid FAL-PDU", dlsdu := "null" }	CLOSED
R3	CLOSED	Connect_ind && Initiator = "False" && FAL_Pdu_Type (dls_user_data) = "EST_ReqPDU" && AREPContextCheck (dlsdu) = "False" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := dlcep_dl_id (from Connect_ind), reason := "Context Check Negative", dlsdu := BuildFAL-PDU (fal_pdu_name := "Abort_PDU", fal_id := identifier, fal_reason_code := "Context Check Negative", fal_additional_detail := maxOSCC, maxOSCS, maxUCSC, maxUCSS, CI) }	CLOSED
R4	CLOSED	unallowed primitive => (no actions taken)	CLOSED
R5	CLOSED	Connect_ind && Initiator = "False" && FAL_Pdu_Type (dls_user_data) = "EST_ReqPDU" && RemoteAddressConfigurationType = "Linked" && RemoteDlcepAddress <> calling_dlcep_address => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := dlcep_dl_id (from Connect_ind), reason := "Remote Address Mismatch", dlsdu := "null" }	CLOSED

Table 94 (continued)

#	Current state	Event or condition => action	Next state
R6	CLOSED	Connect_ind && Initiator = "False" && FAL_Pdu_Type (dls_user_data) = "EST_ReqPDU" && AREPContextCheck (dlsdu) = "True" && RemoteAddressConfigurationType = "Linked" && RemoteDlcepAddress = calling_dlcep_address => MaxDlsduSizeFromRequestorNegotiated := dlsdu_size_from_requestor, MaxDlsduSizeFromResponderNegotiated := dlsdu_size_from_responder, EST_ind { arep_id := GetArepld (), user_data := dls_user_data } }	RSP
R7	CLOSED	Connect_ind && Initiator = "False" && FAL_Pdu_Type (dls_user_data) = "EST_ReqPDU" && AREPContextCheck (dlsdu) = "True" && RemoteAddressConfigurationType = "Free" => RemoteDlcepAddress := calling_dlcep_address, MaxDlsduSizeFromRequestorNegotiated := dlsdu_size_from_requestor, MaxDlsduSizeFromResponderNegotiated := dlsdu_size_from_responder, EST_ind { arep_id := GetArepld (), user_data := dls_user_data } }	RSP
R8	RSP REPL OPEN	Connect_ind && Initiator = "False" && RemoteAddressConfigurationType = "Linked" && RemoteDlcepAddress <> calling_dlcep_address => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := dlcep_dl_id (from Connect_ind), reason := "Remote Address Mismatch", dlsdu := "null" } }	SAME
R9	RSP REPL OPEN	Connect_ind && Initiator = "False" && FAL_Pdu_Type (dls_user_data) = "EST_ReqPDU" && RemoteAddressConfigurationType = "Free" && RemoteDlcepAddress <> calling_dlcep_address => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := dlcep_dl_id (from Connect_ind), reason := "AREP Busy", dlsdu := "null" } }	SAME
R10	RSP REPL OPEN	Connect_ind && Initiator = "False" && FAL_Pdu_Type (dls_user_data) <> "EST_ReqPDU" && RemoteAddressConfigurationType = "Free" && RemoteDlcepAddress <> calling_dlcep_address => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := dlcep_dl_id (from Connect_ind), reason := "Invalid FAL-PDU", dlsdu := "null" } }	SAME

Table 94 (continued)

#	Current state	Event or condition => action	Next state
R11	RSP REPL OPEN	Connect_ind && Initiator = "False" && FAL_Pdu_Type (dls_user_data) = "EST_ReqPDU" && RemoteDlcepAddress = calling_dlcep_address => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := dlcep_dl_id (from Connect_ind), reason := "Invalid FAL-PDU", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU" } 	CLOSED
R12	REQ OPEN	Connect_ind && Initiator = "True" && RemoteDlcepAddress <> calling_dlcep_address => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := dlcep_dl_id (from Connect_ind), reason := "Multiple Initiators", dlsdu := "null" } 	SAME
R13	REQ OPEN	Connect_ind && Initiator = "True" && RemoteDlcepAddress = calling_dlcep_address => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := dlcep_dl_id (from Connect_ind), reason := "Multiple Initiators", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Multiple Initiators" } 	CLOSED
R14	REQ	Connect_cnf && FAL_Pdu_Type (dls_user_data) = "EST_RspPDU" => MaxDlsduSizeFromRequestorNegotiated := dlsdu_size_from_requestor, MaxDlsduSizeFromResponderNegotiated := dlsdu_size_from_responder, DllPriorityNegotiated := dll_priority, EST_cnf(+) { arep_id := GetArepld (), user_data := dls_user_data } 	OPEN

Table 94 (continued)

#	Current state	Event or condition => action	Next state
R15	REQ	Connect_cnf && FAL_Pdu_Type (dls_user_data) <> "EST_RspPDU" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU" }	CLOSED
R16	REPL	FAL-PDU_ind && dmpm_service_name = "DMPM_Connection_Established_ind" && CIU=0 => (no actions taken)	OPEN
R17	REPL	FAL-PDU_ind && dmpm_service_name = "DMPM_Connection_Established_ind" && CIU>0 => StartTimer(RS)	OPEN
R18	REPL	FAL-PDU_ind && ((dmpm_service_name <> "DMPM_Disconnect_ind") && (dmpm_service_name <> "DM_Connection_Established_ind")) => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid DL-Event", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid DL-Event", additional_detail := "null" }	CLOSED
R19	REQ	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && FAL_Pdu_Type (dls_user_data) = "EST_ErrPDU" => EST_cnf(-) { arep_id := GetArepld (), user_data := dls_user_data }	CLOSED
R20	REQ	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu <> "null" && ((FAL_Pdu_Type (dls_user_data) <> "Abort_PDU") && (FAL_Pdu_Type (dls_user_data) <> "EST_ErrPDU")) => Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU", additional_detail := "null" }	CLOSED

Table 94 (continued)

#	Current state	Event or condition => action	Next state
R21	REQ RSP	FAL-PDU_ind && dmpm_service_name <> "DMPM_Disconnect_ind" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid DL-Event", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid DL-Event" }	CLOSED
R22	REPL RSP	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu <> "null" && FAL_Pdu_Type (dls_user_data) <> "Abort_PDU" => Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU", additional_detail := "null" }, StopTimer, ResetCounters	CLOSED
R23	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu <> "null" && FAL_Pdu_Type (dls_user_data) <> "Abort_PDU" && Role = "Client" "Peer" => Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU", additional_detail := "null" }, StopTimer, ResetCounters	CLOSED
R24	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu <> "null" && FAL_Pdu_Type (dls_user_data) <> "Abort_PDU" && Role = "Server" "Peer" => Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU", additional_detail := "null" }, StopTimer, ResetCounters	CLOSED

Table 94 (continued)

#	Current state	Event or condition => action	Next state
R25	OPEN	<pre> FAL-PDU_ind && ((dmpm_service_name <> "DMPM_Disconnect_ind") (dmpm_service_name <> "DMPM_Data_ind")) && Role = "Client" "Peer" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid DL-Event", dlsdu := "null" }, Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid DL-Event", additional_detail := "null" }, StopTimer, ResetCounters </pre>	CLOSED
R26	OPEN	<pre> FAL-PDU_ind && ((dmpm_service_name <> "DMPM_Disconnect_ind") (dmpm_service_name <> "DMPM_Data_ind")) && Role = "Server" "Peer" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid DL-Event", dlsdu := "null" }, Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid DL-Event", additional_detail := "null" }, StopTimer, ResetCounters </pre>	CLOSED
R27	OPEN	<pre> FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Peer" "Server" && FAL_Pdu_Type (dls_user_data) = "CS_ReqPDU" && GetCounterValue(OSCS) < maxOSCS && CIU = 0 => CS_ind { arep_id := GetArepld (), user_data := fal_pdu }, IncrementCounter(OSCS) </pre>	OPEN
R28	OPEN	<pre> FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Peer" "Server" && FAL_Pdu_Type (dls_user_data) = "CS_ReqPDU" && GetCounterValue(OSCS) < maxOSCS && CIU > 0 => CS_ind { arep_id := GetArepld (), user_data := fal_pdu }, IncrementCounter(OSCS) , StartTimer(RS) </pre>	OPEN

Table 94 (continued)

#	Current state	Event or condition => action	Next state
R29	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Client" "Peer" && FAL_Pdu_Type (dls_user_data) = "AR_XON_OFF_ReqPDU" => SetXon (user_data), AR_XON_OFF_ind { arep_id := GetArepld (), user_data := fal_pdu } 	OPEN
R30	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Server" "Peer" && FAL_Pdu_Type (dls_user_data) = "CS_ReqPDU" && GetCounterValue(OSCS) ≥ maxOSCS => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Number of parallel services exceeded", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Number of parallel services exceeded", additional_detail := "null" }, StopTimer, ResetCounters	CLOSED
R31	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Client" "Peer" && FAL_Pdu_Type (dls_user_data) = "CS_RspPDU" && CIU = 0 => CS_cnf { arep_id := GetArepld (), user_data := fal_pdu }, DecrementCounter(OSCC)	OPEN
R32	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Client" "Peer" && FAL_Pdu_Type (dls_user_data) = "CS_RspPDU" && CIU > 0 => CS_cnf { arep_id := GetArepld (), user_data := fal_pdu }, DecrementCounter(OSCC) , StartTimer(RC)	OPEN

Table 94 (continued)

#	Current state	Event or condition => action	Next state
R33	OPEN	<pre> FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && FAL_Pdu_Type (dls_user_data) = "UCA_ReqPDU" && Role = "Server" "Peer" && CIU = 0 => UCA_ind { arep_id := GetArepld (), user_data := fal_pdu }, FAL-PDU_req { dmpm_service_name := "DMPM_Data_req", arep_id := GetArepld (), dlsdu := BuildFAL-PDU (fal_pdu_name := "UCA_AckPDU", fal_data := "null") } </pre>	OPEN
R34	OPEN	<pre> FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && FAL_Pdu_Type (dls_user_data) = "UCA_ReqPDU" && Role = "Server" "Peer" && CIU > 0 => UCA_ind { arep_id := GetArepld (), user_data := fal_pdu }, FAL-PDU_req { dmpm_service_name := "DMPM_Data_req", arep_id := GetArepld (), dlsdu := BuildFAL-PDU (fal_pdu_name := "UCA_AckPDU", fal_data := "null") }, StartTimer(RS) </pre>	OPEN
R35	OPEN	<pre> FAL-PDU_ind && dmpm_service_name = DMPM_Data_ind" && FAL_Pdu_Type (dls_user_data) = "UCA_ReqPDU" && Role = "Client" "Peer" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid Event for Role", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid Event for Role", additional_detail := "null" }, StopTimer, ResetCounters </pre>	CLOSED
R36	OPEN	<pre> FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && FAL_Pdu_Type (dls_user_data) = "UCA_AckPDU" && Role = "Client" "Peer" && GetCounterValue(UCC) > 0 && CIU = 0 => DecrementCounter(UCC) </pre>	OPEN

Table 94 (continued)

#	Current state	Event or condition => action	Next state
R37	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && FAL_Pdu_Type (dls_user_data) = "UCA_AckPDU" && Role = "Client" II "Peer" && GetCounterValue(UCC) > 0 && CIU > 0 => DecrementCounter(UCC) , StartTimer(RC)	OPEN
R38	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && FAL_Pdu_Type (dls_user_data) = "UCA_AckPDU" && Role = "Client" II "Peer" && GetCounterValue(UCC) = 0 => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "UCA_AckPDU received and UCC=0", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "UCA_AckPDU received and UCC=0", additional_detail := "null" }, StopTimer, ResetCounters	CLOSED
R39	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && FAL_Pdu_Type (dls_user_data) = "IdlePDU" && CIU > 0 && Role = "Client" II "Peer" => StartTimer(RC)	OPEN
R40	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && FAL_Pdu_Type (dls_user_data) = "IdlePDU" && CIU > 0 && Role = "Server" II "Peer" => StartTimer(RS)	OPEN
R41	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && FAL_Pdu_Type (dls_user_data) = "IdlePDU" && CIU = 0 => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU", additional_detail := "null" }, StopTimer, ResetCounters	CLOSED

Table 94 (continued)

#	Current state	Event or condition => action	Next state
R42	OPEN	<pre> FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Client" && ((FAL_Pdu_Type (dls_user_data) <> "CS_RspPDU") (FAL_Pdu_Type (dls_user_data) <> "UCA_AckPDU") (FAL_Pdu_Type (dls_user_data) <> "IdlePDU")) => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU", additional_detail := "null" }, StopTimer, ResetCounters </pre>	CLOSED
R43	OPEN	<pre> FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Server" && ((FAL_Pdu_Type (dls_user_data) <> "CS_ReqPDU") (FAL_Pdu_Type (dls_user_data) <> "UCA_ReqPDU") (FAL_Pdu_Type (dls_user_data) <> "IdlePDU")) => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU", additional_detail := "null" }, StopTimer, ResetCounters </pre>	CLOSED

Table 94 (continued)

#	Current state	Event or condition => action	Next state
R44	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Client" && ((FAL_Pdu_Type (dls_user_data) <> "CS_ReqPDU") (FAL_Pdu_Type (dls_user_data) <> "CS_RspPDU") (FAL_Pdu_Type (dls_user_data) <> "UCA_ReqPDU") (FAL_Pdu_Type (dls_user_data) <> "UCA_AckPDU") (FAL_Pdu_Type (dls_user_data) <> "IdlePDU")) => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU", additional_detail := "null" }, StopTimer, ResetCounters	CLOSED
R45	OPEN	RCTimer expired => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "RCTimer expired", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "RCTimer expired", additional_detail := "null" }, StopTimer, ResetCounters	CLOSED
R46	OPEN	RSTimer expired => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "RSTimer expired", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "RSTimer expired", additional_detail := "null" }, StopTimer, ResetCounters	CLOSED

Table 94 (continued)

#	Current state	Event or condition => action	Next state
R47	NOT CLOSED	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu <> "null" && FAL_Pdu_Type (dls_user_data) = "Abort_PDU" => Abort_ind{ arep_id := GetArepld (), locally_generated := "False", identifier := AbortIdentifier (fal_pdu), reason_code := AbortReason (fal_pdu), additional_detail := AbortDetail (fal_pdu) }, StopTimer, ResetCounters	CLOSED
R48	NOT CLOSED	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu = "null" && originator = "remote_dls_provider" => Abort_ind{ arep_id := GetArepld (), locally_generated := "False", identifier := "Data Link Layer", reason_code := reason, additional_detail := "null" }, StopTimer, ResetCounters	CLOSED
R49	NOT CLOSED	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu = "null" && originator = "remote_dls_user" => Abort_ind{ arep_id := GetArepld (), locally_generated := "False", identifier := "FAL", reason_code := reason, additional_detail := "null" }, StopTimer, ResetCounters	CLOSED
R50	NOT CLOSED	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu = "null" && originator = "local_dls_provider" => Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "Data Link Layer", reason_code := reason, additional_detail := "null" }, StopTimer, ResetCounters	CLOSED

Table 94 (continued)

#	Current state	Event or condition => action	Next state
R51	NOT CLOSED	ErrorToARPM => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepId (), reason := reason, dlsdu := "null" }, Abort_ind { arep_id := GetArepId (), locally_generated := "True", identifier := "FAL", reason_code := reason, additional_detail := "null" }, StopTimer, ResetCounters	CLOSED

4.6.5.4.3 Functions used by QUB-FC ARPM

Table 95 through Table 109 define the functions used by this state machine.

Table 95 – Function GetArepId ()

Name	GetArepId ()	Used in	ARPM
Input		Output	
(none)		AREP Identifier	
Function	Returns a value that can unambiguously identify the current AREP.		

Table 96 – Function BuildFAL-PDU

Name	BuildFAL-PDU	Used in	ARPM
Input		Output	
fal_pdu_name, calling_dlcep_address, called_dlcep_address, fal_data, fal_id, fal_reason_code, fal_additional_detail		dlsdu	
Function	Builds an FAL-PDU out of the parameters given as input variables.		

Table 97 – Function FAL_Pdu_Type

Name	FAL_Pdu_Type	Used in	ARPM
Input		Output	
dls_user_data		One of the FAL-PDU types defined in the FAL-PDUs subclause.	
Function	This function decodes the FAL-PDU that is conveyed in the dls_user_data parameter and retrieves one of the FAL-PDU types.		

Table 98 – Function AREPContextCheck()

Name	AREPContextCheck()	Used in	ARPM			
Input		Output				
dl_sdu		Boolean value.				
Function						
This function checks the AREP context parameters that are received with establish service. The compatibility of the remote to the local context is shown in the following matrix:						
Local Context						
Remote Context	Type	maxOSCC	maxOSCS	maxUCSC	maxUCSS	CI
Type	=					
maxOSCC				≥		
mmaxOSCS		≤				
maxUCSC				≥		
maxUCSS			≤			
CIU				=		
Explanation:						
≤: local value smaller than or equal to remote value						
≥: local value larger than or equal to remote value						
=: local value equal to remote value						

Table 99 – Function AbortIdentifier

Name	AbortIdentifier	Used in	ARPM
Input		Output	
fal_pdu		The Identifier parameter of the Abort service.	
Function	This function decodes the Abort_PDU that is conveyed in the fal_pdu parameter and extracts the Identifier parameter.		

Table 100 – Function AbortReason

Name	AbortReason	Used in	ARPM
Input		Output	
fal_pdu		The Reason Code parameter of the Abort service.	
Function	This function decodes the Abort_PDU that is conveyed in the fal_pdu parameter and extracts the Reason Code parameter		

Table 101 – Function AbortDetail

Name	AbortDetail	Used in	ARPM
Input		Output	
fal_pdu		The Additional Detail parameter of the Abort service.	
Function	This function decodes the Abort_PDU that is conveyed in the fal_pdu parameter and extracts the Additional Detail parameter.		

NOTE The following two functions make use of persistent protocol timers that are able to issue the local events 'TSTimer expired', 'TCTimer expired', 'RCTimer expired' and 'RSTimer expired' to notify the expiration of the appropriate time interval to the ARPM.

Table 102 – Function StartTimer

Name	StartTimer	Used in	ARPM
Input		Output	
identifier			
Function	This function starts the selected persistent protocol timer as follows: If identifier is TS then function starts the TSTimer with the value of CIU/3. If identifier is TC then function starts the TCTimer with the value of CIU/3. If identifier is RS then function starts the RSTimer with the value of CIU. If identifier is RC then function starts the RCTimer with the value of CIU.		

NOTE The appropriate timer is restarted if it was still running.

Table 103 – Function StopTimer

Name	StopTimer	Used in	ARPM
Input		Output	
Function	This function stops all local persistent protocol timers of the ARPM.		

NOTE The following functions make use of local persistent variables OSCC (current value of outstanding services at client), UCC (current value of unconfirmed services at client) and OSCI (current value of outstanding services at server) that supports counting of outstanding services. The initial values of OSCC, UCC, OSCI are 0.

Table 104 – Function ResetCounters

Name	ResetCounters	Used in	ARPM
Input		Output	
Function	This function sets OSCC, UCC, OSCI to zero.		

Table 105 – Function IncrementCounter

Name	IncrementCounter	Used in	ARPM
Input		Output	
identifier			
Function	This function increments the selected counter.		

Table 106 – Function DecrementCounter

Name	DecrementCounter	Used in	ARPM
Input		Output	
identifier			
Function	This function decrements the selected counter.		

Table 107 – Function GetCounterValue

Name	GetCounterValue	Used in	ARPM
Input		Output	
identifier		value	
Function	This function returns the current value of the selected counter.		

NOTE The following function makes use of local persistent variable XON_OFF_Flag that stores the current value of flow control from the AR_XON_OFF_ReqPDU. The initial value of XON_OFF_Flag is True.

Table 108 – Function Xon

Name	Xon	Used in	ARPM
Input		Output	
(none)		Boolean (True/False)	
Function	This function returns True if XON_OFF_Flag was True otherwise it returns False.		

Table 109 – Function SetXonOff

Name	SetXonOff	Used in	ARPM
Input		Output	
user_data			
Function	This function extracts the XON_OFF octet from user_data. It sets XON_XOFF_Flag True if XON_OFF is 'ON' within the PDU otherwise it sets the flag False.		

4.6.6 Buffered User-triggered Bidirectional (BUB) ARPM

4.6.6.1 Primitive definitions

4.6.6.1.1 Primitives exchanged between ARPM and FSPM

Table 110 and Table 111 list the primitives exchanged between the ARPM and the FSPM.

Table 110 – Primitives issued by FSPM to ARPM

Primitive name	Source	Associated parameters	Functions
EST_req	FSPM	user_data	This is an FAL internal primitive used to convey an Establish request primitive from the FSPM to the ARPM.
EST_rsp(+)	FSPM	user_data	This is an FAL internal primitive used to convey an Establish response(+) primitive from the FSPM to the ARPM.
EST_rsp(-)	FSPM	user_data	This is an FAL internal primitive used to convey an Establish response(-) primitive from the FSPM to the ARPM.
Abort_req	FSPM	identifier, reason_code, additional_detail	This an FAL internal primitive used to convey an Abort request primitive from the FSPM to the ARPM.
CS_req	FSPM	user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) request primitive from the FSPM to the ARPM.
CS_rsp	FSPM	user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) response primitive from the FSPM to the ARPM.
UCS_req	FSPM	remote_dlsap_address, user_data	This is an FAL internal primitive used to convey an Unconfirmed Send (UCS) request primitive from the FSPM to the ARPM.
FCMP_req	FSPM	(none)	This is an FAL internal primitive used to convey a FAL-Compel (FCMP) request primitive from the FSPM to the ARPM.
GBM_req	FSPM	(none)	This is an FAL internal primitive used to convey a Get-Buffered-Message (GBM) request primitive from the FSPM to the ARPM.

Table 111 – Primitives issued by ARPM to FSPM

Primitive name	Source	Associated parameters	Functions
EST_ind	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish indication primitive from the ARPM to the FSPM.
EST_cnf(+)	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish response(+) primitive from the ARPM to the FSPM.
EST_cnf(-)	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish response(-) primitive from the ARPM to the FSPM.
Abort_ind	ARPM	arep_id, locally_generated, identifier, reason_code, additional_detail	This is an FAL internal primitive used to convey an Abort primitive from the ARPM to the FSPM.
CS_ind	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) indication primitive from the ARPM to the FSPM.
CS_cnf	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) confirmation primitive from the ARPM to the FSPM.
UCS_ind	ARPM	arep_id, remote_dlsap_address, duplicate_fal_sdu, user_data, local_timeliness, remote_timeliness	This is an FAL internal primitive used to convey an Unconfirmed Send (UCS) indication primitive from the ARPM to the FSPM.
FCMP_cnf	ARPM	arep_id, status	This is an FAL internal primitive used to convey a FAL-Compel (FCMP) confirmation primitive from the ARPM to the FSPM.
GBM_cnf(+)	ARPM	arep_id, duplicate_fal_sdu, user_data, local_timeliness, remote_timeliness	This is an FAL internal primitive used to convey a Get-Buffered-Message (GBM) positive confirmation primitive from the ARPM to the FSPM.
GBM_cnf(-)	ARPM	arep_id	This is an FAL internal primitive used to convey a Get-Buffered-Message (GBM) negative confirmation primitive from the ARPM to the FSPM.
FSTS_ind	ARPM	arep_id, reported_status	This is an FAL internal primitive used to convey a FAL-Status (FSTS) indication primitive from the ARPM to the FSPM.

4.6.6.1.2 Parameters of FSPM/ARPM primitives

The parameters used with the primitives exchanged between the FSPM and the ARPM are described in Table 112.

Table 112 – Parameters used with primitives exchanged between FSPM and ARPM

Parameter name	Description
arep_id	This parameter is used to unambiguously identify an instance of the AREP that has issued a primitive. A means for such identification is not specified by this standard.
user_data	This parameter conveys FAL-User data.
locally_generated	This parameter conveys value that is used for the Locally_Generated parameter.
identifier	This parameter conveys value that is used for the Identifier parameter.
reason_code	This parameter conveys value that is used for the Reason_Code parameter.
additional_detail	This parameter conveys value that is used for the Additional_Detail parameter.
duplicate_fal_sdu	This parameter conveys value that is used for the Duplicate_FAL-SDU parameter.
remote_dlsap_address	This parameter conveys value that is used for the Remote_DLSAP_Address parameter.
status	This parameter conveys value that is used for the Status parameter.
reported_status	This parameter conveys a Data Link Layer event status.
local_timeliness	This parameter conveys value that is used for the Local_Timeliness parameter.
remote_timeliness	This parameter conveys value that is used for the Remote_Timeliness parameter.

4.6.6.2 DLL mapping of BUB AREP class

4.6.6.2.1 BUB AREP class formal model

This subclause describes the mapping of the BUB AREP class to the Fieldbus Data Link Layer. It does not redefine the DLSAP attributes or DLME attributes that are or will be defined in the Data Link Layer standard; rather, it defines how they are used by this AR class.

The DLL Mapping attributes and their permitted values and the DLL services used with the BUB AREP class are defined in this subclause.

CLASS:		BubCo
PARENT CLASS:		BufferedUser-TriggeredBidirectionalAREP
ATTRIBUTES:		
1	(m) KeyAttribute:	LocalDlcepAddress
2	(m) Attribute:	RemoteDlcepAddress
3	(m) Attribute:	DlsapRole (Initiator, Constrained)
4	(m) Attribute:	QosParameterSet
4.1	(m) Attribute:	DlcepClass (Peer)
4.2	(m) Attribute:	DlcepDataDeliveryFeatures
4.2.1	(m) Attribute:	FromRequestorToResponder (Ordered, Unordered)
4.2.2	(m) Attribute:	FromResponderToRequestor (Ordered, Unordered)
4.3	(m) Attribute:	Priority
4.3.1	(m) Attribute:	DllPriority (Urgent, Normal, TimeAvailable)
4.3.2	(m) Attribute:	DllPriorityNegotiated (Urgent, Normal, TimeAvailable)
4.4	(m) Attribute:	DlpduAuthentication (Ordinary, Source, Maximal)
4.5	(m) Attribute:	ResidualActivity
4.5.1	(m) Attribute:	ResidualActivityAsSender (True, False)
4.5.2	(m) Attribute:	ResidualActivityAsReceiver (True, False)
4.6	(m) Attribute:	MaxConfirmDelay
4.7	(c) Constraint:	DlsapRole = Initiator
4.7.1	(m) Attribute:	MaxConfirmDelayOnDIConnect
4.7.2	(m) Attribute:	MaxConfirmDelayOnDIData
4.8	(m) Attribute:	DISchedulingPolicy (Explicit)
4.9	(m) Attribute:	MaxDlsduSizes
4.9.1	(m) Attribute:	MaxDlsduSizeFromRequestor
4.9.2	(m) Attribute:	MaxDlsduSizeFromResponder
4.9.3	(m) Attribute:	MaxDlsduSizeFromRequestorNegotiated
4.9.4	(m) Attribute:	MaxDlsduSizeFromResponderNegotiated
DLL SERVICES:		
1	(m) OpsService:	DL-Put
2	(m) OpsService:	DL-Get
3	(m) OpsService:	DL-Connect
4	(m) OpsService:	DL-Connection-Established
5	(m) OpsService:	DL-Disconnect
6	(m) OpsService:	DL-Buffer-Received
7	(m) OpsService:	DL-Buffer-Sent
8	(m) OpsService:	DL-Compel-Service

4.6.6.2.2 Attributes

4.6.6.2.2.1 LocalDlcepAddress

This attribute specifies the local DLCEP address and identifies the DLCEP. The value of this attribute is used as the “DLCEP-address” parameter of the DLL.

NOTE 1 The value of this attribute is also carried in the Establish Request PDU.

This attribute contains the following three subattributes: Link Address, Node Address, and Selector.

NOTE 2 Since the local Link and Node addresses are set by the Network Management, only the Selector portion of the LocalDisapAddress attribute is a configurable attribute of the FAL.

4.6.6.2.2.2 RemoteDlcepAddress

This attribute specifies the remote DLCEP address and identifies the DLCEP.

This attribute supplies the value for called DLCEP-address of the DL-Connect service.

NOTE 1 The value of this attribute is also carried in the header part of the Establish Request PDU.

This attribute contains the following three subattributes: Link Address, Node Address, and Selector.

NOTE 2 Since the local Link and Node addresses are set by the Network Management, only the Selector portion of the LocalDisapAddress attribute is a configurable attribute of the FAL.

4.6.6.2.2.3 DisapRole

This attribute specifies the behavior of the DLSAP that is used by the AREP.

This attribute supplies the value for the “DL(SAP)-role” parameter. The permitted values Initiator, Constrained Responder and Unconstrained Responder, correspond respectively to INITIATOR, CONSTRAINED, and UNCONSTRAINED as defined in the Data Link Layer standard.

4.6.6.2.2.4 QosParameterSet

The QosParameterSet attributes specify the DL quality of service that is used by the AREP. These attribute values may be negotiated with the remote AREP.

4.6.6.2.2.5 DlcepClass

This attribute specifies the behavior of the DLCEP which is attached to the AREP.

This attribute supplies the value for the “DLCEP class” parameter of the DLL. The possible value of this attribute is Peer and corresponds to PEER defined by the DLL.

4.6.6.2.2.6 DlcepDataDeliveryFeatures

These two attributes specify the data delivery features of the DLL.

The permitted values Ordered and Unordered correspond respectively to ORDERED and UNORDERED defined by the Data Link Layer standard.

The FromRequestorToResponder and FromResponderToRequestor attributes shall have the same value.

4.6.6.2.2.7 FromRequestorToResponder

This attribute specifies the DLL data delivery feature of the AREP as a sender. It supplies the value for the “DLCEP data delivery features from requestor to responder(s)” parameter defined in the DLL.

4.6.6.2.2.8 FromResponderToRequestor

This attribute specifies the DLL data delivery feature of the AREP as a receiver. It supplies the value for the “DLCEP data delivery features from responder(s) to requestor” parameter defined in the DLL.

4.6.6.2.2.9 DllPriority

This attribute specifies the DLL priority before negotiation.

NOTE It is not possible to use different priorities for each FAL-PDU sent from the same BUB AREP. Also it is not permitted to use different priorities for the send and the receive conveyance paths.

4.6.6.2.2.10 DllPriorityNegotiated

This attribute specifies the DLL priority after negotiation.

4.6.6.2.2.11 DlpduAuthentication

This attribute specifies the lower bound of the length of DL-addressing to be used by the DLL.

This attribute supplies the value for the “DLPDU-authentication” parameter of the DLL. The permitted value Ordinary, Source and Maximal correspond to ORDINARY, SOURCE and MAXIMAL respectively, as defined in Part-3 and –4 of this International Standard.

4.6.6.2.2.12 ResidualActivityAsSender

This attribute specifies sender’s DLC residual activity. It supplies the value for the “Residual activity as sender” parameter as defined in the DLL. The possible values are “True” and “False”.

4.6.6.2.2.13 ResidualActivityAsReceiver

This attribute specifies receiver’s DLC residual activity. It supplies the value for the “Residual activity as receiver” parameter defined in the DLL. The possible values are “True” and “False”.

4.6.6.2.2.14 MaxConfirmDelay

This attribute specifies the maximum confirmation delay of certain DLL connection-oriented services.

4.6.6.2.2.15 MaxConfirmDelayOnDIConnect

This attribute specifies the maximum confirmation delay for a confirmation from a DL-Connect service.

This attribute supplies the value for the “Maximum confirmation delay on DL-Connect, DL-Reset and DL-Subscriber-Query” parameter specified in the Data Link Layer standard.

4.6.6.2.2.16 MaxConfirmDelayOnDIData

This attribute specifies the maximum confirmation delay for a confirmation from a DL-Data service.

This attribute supplies the value for the “Maximum confirmation delay on DL-Data” parameter specified in Part-3 and –4 of this international standard.

4.6.6.2.2.17 DISchedulingPolicy

This attribute provides the guidance to the DLL on the scheduling needed by an AR. For this AREP, the DLL tries to transmit the FAL-PDU as soon as possible.

This attribute supplies the value for the “DL-Scheduling-policy” parameter of the DLL. The permitted value Explicit corresponds to EXPLICIT defined in Part-3 and –4 of this International Standard.

4.6.6.2.2.18 MaxDisduSizeFromRequestor

This attribute specifies the maximum length of an FAL-PDU that can be sent from this AREP before negotiation.

This attribute supplies the value for the “Maximum DLSDU sizes from requestor” parameter of the DLL.

4.6.6.2.2.19 MaxDisduSizeFromResponder

This attribute specifies the maximum length of an FAL-PDU that can be received by this AREP before negotiation.

This attribute supplies the value for the “Maximum DLSDU sizes from responder” parameter of the DLL.

4.6.6.2.2.20 MaxDisduSizeFromRequestorNegotiated

This attribute specifies the maximum length of an FAL-PDU that can be sent from this AREP after negotiation.

4.6.6.2.2.21 MaxDisduSizeFromResponderNegotiated

This attribute specifies the maximum length of an FAL-PDU that can be received by this AREP after negotiation.

4.6.6.3 BUB ARPM state machine**4.6.6.3.1 BUB ARPM states**

The defined states and their descriptions of the BUB ARPM are shown in Table 113 and Figure 33.

Table 113 – BUB ARPM states

State	Description
CLOSED	The AREP is defined, but not capable of sending or receiving FAL-PDUs. It may send or receive Establish service FAL-PDUs while in this state.
OPEN	The AREP is defined and capable of sending or receiving FAL-PDUs.
REQUESTING (REQ)	The AREP has sent an Establish Request FAL-PDU and is waiting for a response from the remote AREP.
RESPONDING (RSP)	The AREP has received an Establish Request FAL-PDU, delivered an Establish.indication primitive and is waiting for a response from its user.
REPLIED (REPL)	The Server AREP has issued an EST_rsp(+) primitive and is waiting for a “connection established” indication from the DLL.
SAME	Indicates that the next state is the same as the current state.

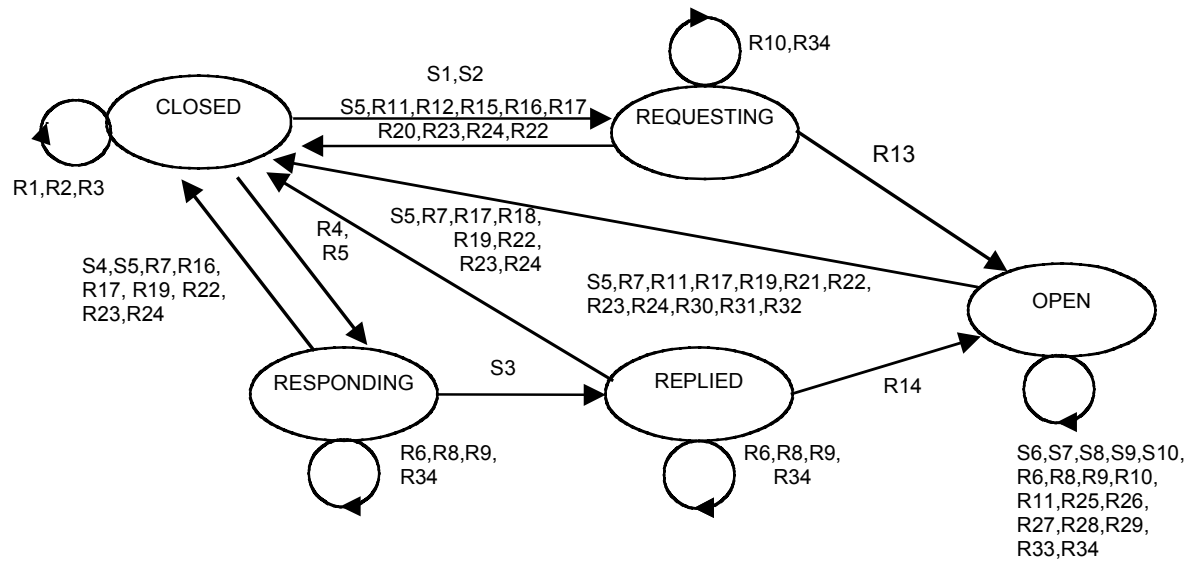


Figure 33 – State transition diagram of BUB ARPM

4.6.6.3.2 BUB ARPM state table

Table 114 and Table 115 define the BUB ARPM state machine.

Table 114 – BUB ARPM state table – sender transactions

#	Current state	Event or condition => action	Next state
S1	CLOSED	<pre> EST_req && Initiator = "True" && RemoteAddressConfigurationType:= "Free" => RemoteDlcepAddress:= remote_dlcep_address, FAL-PDU_req { dmpm_service_name:= "DMPM_Connect_req", arep_id:= GetArepld(), called_address:= "default dlsap address", calling_address:= "default dlsap address", local_dlcep_address:= LocalDlcep, dlsdu:= BuildFAL-PDU (fal_pdu_name:= "EST_ReqPDU", calling_dlcep_address:= LocalDlcepAddress, called_dlcep_address:= RemoteDlcepAddress, fal_data:= user_data) } </pre>	REQ
S2	CLOSED	<pre> EST_req && Initiator = "True" && RemoteAddressConfigurationType:= "Linked" => FAL-PDU_req { dmpm_service_name:= "DMPM_Connect_req", arep_id:= GetArepld(), called_address:= "default dlsap address", calling_address:= "default dlsap address", local_dlcep_address:= LocalDlcep, dlsdu:= BuildFAL-PDU (fal_pdu_name:= "EST_ReqPDU", calling_dlcep_address:= LocalDlcepAddress, called_dlcep_address:= RemoteDlcepAddress, fal_data:= user_data) } </pre>	REQ
S3	RSP	<pre> EST_rsp(+) => FAL-PDU_req { dmpm_service_name:= "DMPM_Connect_rsp", arep_id:= GetArepld(), responding_address:= "default dlsap address", local_dlcep_address:= LocalDlcep, dlcep_dl_id:= DlcepDIIdentifier, dlsdu:= BuildFAL-PDU (fal_pdu_name:= "EST_RspPDU", fal_data:= user_data) } </pre>	REPL
S4	RSP	<pre> EST_rsp(-) => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), dlcep_dl_id:= DlcepDIIdentifier, reason:= "connection rejection-transient condition", dlsdu:= BuildFAL-PDU (fal_pdu_name:= "EST_ErrPDU", fal_data:= user_data) } </pre>	CLOSED

Table 114 (continued)

#	Current state	Event or condition => action	Next state
S5	NOT CLOSED	Abort_req => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), dlcep_dl_id:= DlcepDlIdentifier, reason:= "disconnection-normal condition", dlsdu:= BuildFAL-PDU (fal_pdu_name:= "Abort_PDU", fal_id:= identifier, fal_reason_code:= reason_code, fal_additional_detail:= additional_detail) } }	CLOSED
S6	OPEN	CS_req && Role = "Client" "Peer" => FAL-PDU_req { dmpm_service_name:= "DMPM_Put_req", arep_id:= GetArepld(), dlsdu:= BuildFAL-PDU (fal_pdu_name:= "CS_ReqPDU", fal_data:= user_data) } }	OPEN
S7	OPEN	CS_rsp && Role = "Server" "Peer" => FAL-PDU_req { dmpm_service_name:= "DMPM_Put_req", arep_id:= GetArepld(), dlsdu:= BuildFAL-PDU (fal_pdu_name:= "CS_RspPDU", fal_data:= user_data) } }	OPEN
S8	OPEN	FCMP_req && Role = "Client" "Server" "Peer" => FAL-PDU_req { dmpm_service_name:= "DMPM_Compel_Service_req", arep_id:= GetArepld(), action_class:= "Local" }	OPEN
S9	OPEN	GBM_req && Role = "Client" "Server" "Peer" => FAL-PDU_req { dmpm_service_name:= "DMPM_Get_req", arep_id:= GetArepld() }	OPEN
S10	OPEN	UCS_req => FAL-PDU_req { dmpm_service_name:= "DMPM_Put_req", arep_id:= GetArepld(), dlsdu:= BuildFAL-PDU (fal_pdu_name:= "UCS_ReqPDU", fal_data:= user_data) } }	OPEN

Table 115 – BUB state table – receiver transactions

#	Current state	Event or condition => action	Next state
R1	CLOSED	Connect_ind && Initiator = "True" => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), dlcep_dl_id:= dlcep_dl_id (from Connect_ind), reason:= "Multiple Initiators", dlsdu:= "null" }	CLOSED
R2	CLOSED	Connect_ind && Initiator = "False" && FAL_Pdu_type (dls_user_data) <> "EST_ReqPDU" => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), dlcep_dl_id:= dlcep_dl_id (from Connect_ind), reason:= "Invalid FAL-PDU", dlsdu:= "null" }	CLOSED
R3	CLOSED	Connect_ind && Initiator = "False" && FAL_Pdu_type (dls_user_data) = "EST_ReqPDU" && RemoteAddressConfigurationType = "Linked" && RemoteDlcepAddress <> calling_dlcep_address => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), dlcep_dl_id:= dlcep_dl_id (from Connect_ind), reason:= "Remote Address Mismatch", dlsdu:= "null" }	CLOSED
R4	CLOSED	Connect_ind && Initiator = "False" && FAL_Pdu_type (dls_user_data) = "EST_ReqPDU" && RemoteAddressConfigurationType = "Free" => RemoteDlcepAddress:= calling_dlcep_address, DlcepDllIdentifier:= dlcep_dl_id, MaxDlsduSizeFromRequestorNegotiated := dlsdu_size_from_requestor, MaxDlsduSizeFromResponderNegotiated := dlsdu_size_from_responder, EST_ind { arep_id:= GetArepld (), user_data:= dls_user_data }	RSP
R5	CLOSED	Connect_ind && Initiator = "False" && FAL_Pdu_type (dls_user_data) = "EST_ReqPDU" && RemoteAddressConfigurationType = "Linked" && RemoteDlcepAddress = calling_dlcep_address => DlcepDllIdentifier:= dlcep_dl_id, MaxDlsduSizeFromRequestorNegotiated := dlsdu_size_from_requestor, MaxDlsduSizeFromResponderNegotiated := dlsdu_size_from_responder, EST_ind { arep_id:= GetArepld (), user_data:= dls_user_data }	RSP

Table 115 (continued)

#	Current state	Event or condition => action	Next state
R6	RSP REPL OPEN	Connect_ind && Initiator = "False" && RemoteAddressConfigurationType = "Linked" && RemoteDlcepAddress <> calling_dlcep_address => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), dlcep_dl_id:= dlcep_dl_id (from Connect_ind), reason:= "Remote Address Mismatch", dlsdu:= "null" }	SAME
R7	RSP REPL OPEN	Connect_ind && Initiator = "False" && RemoteDlcepAddress = calling_dlcep_address => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), dlcep_dl_id:= dlcep_dl_id (from Connect_ind), reason:= "Invalid FAL-PDU", dlsdu:= "null" }, Abort_ind { arep_id:= GetArepld (), locally_generated:= "True", identifier:= "FAL", reason_code:= "Invalid FAL-PDU" }	CLOSED
R8	RSP REPL OPEN	Connect_ind && Initiator = "False" && FAL_Pdu_type (dls_user_data) = "EST_ReqPDU" && RemoteAddressConfigurationType = "Free" && RemoteDlcepAddress <> calling_dlcep_address => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), dlcep_dl_id:= dlcep_dl_id (from Connect_ind), reason:= "AREP Busy", dlsdu:= "null" }	SAME
R9	RSP REPL OPEN	Connect_ind && Initiator = "False" && FAL_Pdu_type (dls_user_data) <> "EST_ReqPDU" && RemoteAddressConfigurationType = "Free" && RemoteDlcepAddress <> calling_dlcep_address => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), dlcep_dl_id:= dlcep_dl_id (from Connect_ind), reason:= "Invalid FAL-PDU", dlsdu:= "null" }	SAME
R10	REQ OPEN	Connect_ind && Initiator = "True" && RemoteDlcepAddress <> calling_dlcep_address => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), dlcep_dl_id:= dlcep_dl_id (from Connect_ind), reason:= "Multiple Initiators", dlsdu:= "null" }	SAME

Table 115 (continued)

#	Current state	Event or condition => action	Next state
R11	REQ OPEN	Connect_ind && Initiator = "True" && RemoteDlcepAddress = calling_dlcep_address => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), dlcep_dl_id:= dlcep_dl_id (from Connect_ind), reason:= "Multiple Initiators", dlsdu:= "null" }, Abort_ind { arep_id:= GetArepld (), locally_generated:= "True", identifier:= "FAL", reason_code:= "Multiple Initiators" }	CLOSED
R12	REQ	Connect_cnf && FAL_Pdu_Type (dls_user_data) <> "EST-RspPDU" => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), dlcep_dl_id:= DlcepDllIdentifier, reason:= "Invalid FAL-PDU", dlsdu:= "null" }, Abort_ind { arep_id:= GetArepld (), locally_generated:= "True", identifier:= "FAL", reason_code:= "Invalid FAL-PDU" }	CLOSED
R13	REQ	Connect_cnf && FAL_Pdu_Type (dls_user_data) = "EST-RspPDU" => MaxDlsduSizeFromRequestorNegotiated := dlsdu_size_from_requestor, MaxDlsduSizeFromResponderNegotiated := dlsdu_size_from_responder, DllPriorityNegotiated:= dll_priority, EST_cnf(+) { arep_id:= GetArepld (), user_data:= dls_user_data }	OPEN
R14	REPL	FAL-PDU_ind && dmpm_service_name = "DMPM_Connection_Established_ind" => (no actions taken)	OPEN
R15	REQ	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && FAL_Pdu_Type (fal_pdu) = "EST-ErrPDU" => EST_cnf(-) { arep_id:= GetArepld (), user_data:= dls_user_data }	CLOSED

Table 115 (continued)

#	Current state	Event or condition => action	Next state
R16	REQ RSP	<pre> FAL-PDU_ind && dmpm_service_name <> "DMPM_Disconnect_ind" => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), dlcep_dl_id:= DlcepDlIdentifier, reason:= "Invalid DI PDU", dlsdu:= "null" }, Abort_ind { arep_id:= GetArepld (), locally_generated:= "True", identifier:= "FAL", reason_code:= "Invalid DI PDU" } </pre>	CLOSED
R17	NOT CLOSED	<pre> FAL-PDU_ind && dmpm_service_name <> "DMPM_Disconnect_ind" && fal_pdu <> "null" && FAL_Pdu_Type (fal_pdu) = "Abort_PDU" => Abort_ind { arep_id:= GetArepld (), locally_generated:= "False", identifier:= AbortIdentifier (fal_pdu), reason_code:= AbortReason (fal_pdu), additional_detail:= AbortDetail (fal_pdu) } </pre>	CLOSED
R18	REPL	<pre> FAL-PDU_ind && ((dmpm_service_name <> "DMPM_Disconnect_ind") && (dmpm_service_name <> "DM_Connection_Established_ind")) => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), dlcep_dl_id:= DlcepDlIdentifier, reason:= "Invalid DI PDU", dlsdu:= "null" }, Abort_ind { arep_id:= GetArepld (), locally_generated:= "True", identifier:= "FAL", reason_code:= "Invalid DI PDU" additional_detail:= "null" } </pre>	CLOSED
R19	REPL RSP OPEN	<pre> FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu <> "null" && FAL_Pdu_Type (fal_pdu) <> "Abort_PDU" => Abort_ind { arep_id:= GetArepld (), locally_generated:= "True", identifier:= "FAL", reason_code:= "Invalid FAL-PDU" additional_detail:= "null" } </pre>	CLOSED

Table 115 (continued)

#	Current state	Event or condition => action	Next state
R20	REQ	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu <> "null" && ((FAL_Pdu_Type (fal_pdu) <> "Abort_PDU") && (FAL_Pdu_Type (fal_pdu) <> "EST_ErrPDU") => Abort_ind { arep_id:= GetArepld (), locally_generated:= "True", identifier:= "FAL", reason_code:= "Invalid FAL-PDU" additional_detail:= "null" }	CLOSED
R21	OPEN	FAL-PDU_ind && ((FAL_Pdu_Type (fal_pdu) <> "DMPM_Disconnect_ind") && (FAL_Pdu_Type (fal_pdu) <> "DMPM_Put_Out") => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), dlcep_dl_id:= DlcepDlIdentifier, reason:= "Invalid DI PDU", dlsdu:= "null" }, Abort_ind { arep_id:= GetArepld (), locally_generated:= "True", identifier:= "FAL", reason_code:= "Invalid DI PDU" additional_detail:= "null" }	CLOSED
R22	NOT CLOSED	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu = "null" && originator = "remote_dls_provider" => Abort_ind { arep_id:= GetArepld (), locally_generated:= "False", identifier:= "Data Link Layer", reason_code:= reason additional_detail:= "null" }	CLOSED
R23	NOT CLOSED	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu = "null" && originator = "remote_dls_user" => Abort_ind { arep_id:= GetArepld (), locally_generated:= "False", identifier:= "FAL", reason_code:= reason additional_detail:= "null" }	CLOSED
R24	NOT CLOSED	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu = "null" && originator = "local_dls_provider" => Abort_ind { arep_id:= GetArepld (), locally_generated:= "True", identifier:= "Data Link Layer", reason_code:= reason additional_detail:= "null" }	CLOSED

Table 115 (continued)

#	Current state	Event or condition => action	Next state
R25	OPEN	FAL-PDU_ind && Role = "Peer" "Server" && dmpm_service_name = "DMPM_Buffer_Received_ind" && FAL_Pdu_Type (fal_pdu) = "CS_ReqPDU" => CS_ind { arep_id:= GetArepld (), user_data:= fal_pdu } 	OPEN
R26	OPEN	FAL-PDU_ind && Role = "Client" "Peer" && dmpm_service_name = "DMPM_Buffer_Received_ind" && FAL_Pdu_Type (fal_pdu) = "CS_RspPDU" => CS_cnf { Arepl_id:= GetArepld (), user_data:= fal_pdu } 	OPEN
R27	OPEN	FAL-PDU_ind && Role = "Peer" "Client" "Server" && dmpm_service_name = "DMPM_Buffer_Sent_ind" => FSTS_ind { arep_id:= GetArepld(), reported_status:="Buffer_Sent" } 	OPEN
R28	OPEN	FAL-PDU_ind && Role = "Peer" "Client" "Server" && dmpm_service_name = "DMPM_Get_cnf" && status = "success" => GMB_cnf(+) { arep_id:= GetArepld(), duplicate_fal_sdu=duplicate_dlsdu, user_data:=fal_pdu } 	OPEN
R29	OPEN	FAL-PDU_ind && Role = "Peer" "Client" "Server" && dmpm_service_name = "DMPM_Get_cnf" && status <> "success" => GMB_cnf(-) { arep_id:= GetArepld() } 	OPEN
R30	OPEN	FAL-PDU_ind && Role = "Server" && dmpm_service_name = "DMPM_Buffer_Received_ind" && FAL_Pdu_Type (fal_pdu) <> "CS_ReqPDU" && FAL_Pdu_Type (fal_pdu) <> "UCS_ReqPDU" => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), dlcep_dl_id:= DlcepDllIdentifier, reason:= "Invalid FAL-PDU", dlsdu:= "null" }, Abort_ind { arep_id:= GetArepld (), locally_generated:= "True", identifier:= "FAL", reason_code:= "Invalid FAL-PDU" additional_detail:= "null" } 	CLOSED

Table 115 (continued)

#	Current state	Event or condition => action	Next state
R31	OPEN	<pre> FAL-PDU_ind && Role = "Client" && dmpm_service_name = "DMPM_Buffer_Received_ind" && FAL_Pdu_Type (fal_pdu) <> "CS_RspPDU" && FAL_Pdu_Type (fal_pdu) <> "UCS_ReqPDU" => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), dlcep_dl_id:= DlcepDllIdentifier, reason:= "Invalid FAL-PDU", dlsdu:= "null" }, Abort_ind { arep_id:= GetArepld (), locally_generated:= "True", identifier:= "FAL", reason_code:= "Invalid FAL-PDU" additional_detail:= "null" } </pre>	CLOSED
R32	OPEN	<pre> FAL-PDU_ind && Role = "Peer" && dmpm_service_name = "DMPM_Buffer_Received_ind" && ((FAL_Pdu_Type (fal_pdu) <> "CS_ReqPDU") && (FAL_Pdu_Type (fal_pdu) <> "CS_RspPDU") && (FAL_Pdu_Type (fal_pdu) <> "UCS_ReqPDU") => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), dlcep_dl_id:= DlcepDllIdentifier, reason:= "Invalid FAL-PDU", dlsdu:= "null" }, Abort_ind { arep_id:= GetArepld (), locally_generated:= "True", identifier:= "FAL", reason_code:= "Invalid FAL-PDU" additional_detail:= "null" } </pre>	CLOSED
R33	OPEN	<pre> FAL-PDU_ind && dmpm_service_name = "DMPM_Buffer_Received_ind" && FAL_Pdu_Type (fal_pdu) = "UCS_ReqPDU" => UCS_ind { arep_id:= GetArepld (), user_data:= fal_pdu } </pre>	OPEN
R34	NOT CLOSED	<pre> ErrorToARPM => (no action taken) NOTE It is a local matter to report this error status to network management entities. The ARPM does not abort the existing connections. The FAL user may issue an Abort request to disconnect the current connection, depending on the type of status information conveyed by the ErrorToARPM primitive. </pre>	SAME

4.6.6.3.3 Functions used by BUB ARPM

Table 116 through Table 121 define the functions used by this state machine.

Table 116 – Function GetAreId ()

Name	GetAreId ()	Used in	ARPM
Input		Output	
(none)		AREP Identifier	
Function	Returns a value that can unambiguously identify the current AREP.		

Table 117 – Function BuildFAL-PDU

Name	BuildFAL-PDU	Used in	ARPM
Input		Output	
fal_pdu_name, calling_dlcep_address, called_dlcep_address, fal_data, fal_id, fal_reason_code, fal_additional_detail		Disdu	
Function	Builds an FAL-PDU out of the parameters given as input variables.		

Table 118 – Function FAL_Pdu_Type

Name	FAL_Pdu_Type	Used in	ARPM
Input		Output	
dls_user_data		One of the FAL-PDU types defined in the FAL-PDUs subclause.	
Function	This function decodes the FAL-PDU that is conveyed in the dls_user_data parameter and retrieves one of the FAL-PDU types.		

Table 119 – Function AbortIdentifier

Name	AbortIdentifier	Used in	ARPM
Input		Output	
fal_pdu		The Identifier parameter of the Abort service.	
Function	This function decodes the Abort_PDU that is conveyed in the fal_pdu parameter and extracts the Identifier parameter.		

Table 120 – Function AbortReason

Name	AbortReason	Used in	ARPM
Input		Output	
fal_pdu		The Reason Code parameter of the Abort service.	
Function	This function decodes the Abort_PDU that is conveyed in the fal_pdu parameter and extracts the Reason Code parameter.		

Table 121 – Function AbortDetail

Name	AbortDetail	Used in	ARPM
Input		Output	
fal_pdu		The Additional Detail parameter of the Abort service.	
Function	This function decodes the Abort_PDU that is conveyed in the fal_pdu parameter and extracts the Additional Detail parameter.		

4.6.7 Buffered Network-scheduled Bidirectional (BNB) ARPM

4.6.7.1 Primitive definitions

4.6.7.1.1 Primitives exchanged between ARPM and FSPM

Table 122 and Table 123 define the primitives exchanged between the FSPM and the ARPM.

Table 122 – Primitives issued by FSPM to ARPM

Primitive name	Source	Associated parameters	Functions
EST_req	FSPM	user_data	This is an FAL internal primitive used to convey an Establish request primitive from the FSPM to the ARPM.
EST_rsp(+)	FSPM	user_data	This is an FAL internal primitive used to convey an Establish response(+) primitive from the FSPM to the ARPM.
EST_rsp(-)	FSPM	user_data	This is an FAL internal primitive used to convey an Establish response(-) primitive from the FSPM to the ARPM.
Abort_req	FSPM	identifier, reason_code, additional_detail	This is an FAL internal primitive used to convey an Abort request primitive from the FSPM to the ARPM.
CS_req	FSPM	user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) request primitive from the FSPM to the ARPM.
CS_rsp	FSPM	user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) response primitive from the FSPM to the ARPM.
UCS_req	FSPM	remote_dlsap_address, user_data	This is an FAL internal primitive used to convey an Unconfirmed Send (UCS) request primitive from the FSPM to the ARPM.

Table 123 – Primitives issued by ARPM to FSPM

Primitive name	Source	Associated parameters	Functions
EST_ind	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish indication primitive from the ARPM to the FSPM.
EST_cnf(+)	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish response(+) primitive from the ARPM to the FSPM.
EST_cnf(-)	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish response(-) primitive from the ARPM to the FSPM.
Abort_ind	ARPM	arep_id, locally_generated, identifier, reason_code, additional_detail	This is an FAL internal primitive used to convey an Abort primitive from the ARPM to the FSPM.
CS_ind	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) indication primitive from the ARPM to the FSPM.
CS_cnf	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) confirmation primitive from the ARPM to the FSPM.
UCS_ind	ARPM	arep_id, remote_dlsap_address, duplicate_fal_sdu, user_data, local_timeliness, remote_timeliness	This is an FAL internal primitive used to convey an Unconfirmed Send (UCS) indication primitive from the ARPM to the FSPM.

4.6.7.1.2 Primitives issued by ARPM to FSPM parameters of FSPM/ARPM primitives

The parameters used with the primitives exchanged between the FSPM and the ARPM are described in Table 124.

Table 124 – Parameters used with primitives exchanged between FSPM and ARPM

Parameter name	Description
arep_id	This parameter is used to unambiguously identify an instance of the AREP that has issued a primitive. A means for such identification is not specified by this standard.
user_data	This parameter conveys FAL-User data.
locally_generated	This parameter conveys value that is used for the Locally_Generated parameter.
identifier	This parameter conveys value that is used for the Identifier parameter.
reason_code	This parameter conveys value that is used for the Reason_Code parameter.
additional_detail	This parameter conveys value that is used for the Additional_Detail parameter.
duplicate_fal_sdu	This parameter conveys value that is used for the Duplicate_FAL-SDU parameter.
remote_dlsap_address	This parameter conveys value that is used for the Remote_DLSAP_Address parameter.
status	This parameter conveys value that is used for the Status parameter.
reported_status	This parameter conveys a Data Link Layer event status.
local_timeliness	This parameter conveys value that is used for the Local_Timeliness parameter.
remote_timeliness	This parameter conveys value that is used for the Remote_Timeliness parameter.

4.6.7.2 DLL mapping of BNB AREP class

4.6.7.2.1 BNB AREP class formal model

This subclause describes the mapping of the BNB AREP Class to the Fieldbus Data Link Layer. It does not redefine the DLSAP attributes or DLME attributes that are or will be defined in the Data Link Layer standard; rather, it defines how they are used by this AR class.

The DLL Mapping attributes and their permitted values and the DLL services used with the BNB AREP class are defined in this subclause.

CLASS: Bnb
PARENT CLASS: BufferedNetwork-TriggeredBidirectionalAREP
ATTRIBUTES:

- 1 (m) KeyAttribute: LocalDlcepAddress
- 2 (m) Attribute: RemoteDlcepAddress
- 3 (m) Attribute: DlcepRole (Basic)
- 4 (m) Attribute: QosParameterSet
- 4.1 (m) Attribute: DlcepClass (Peer)
- 4.2 (m) Attribute: DlcepDataDeliveryFeatures
- 4.2.1 (m) Attribute: FromRequestorToResponder (Ordered)
- 4.2.2 (m) Attribute: FromResponderToRequestor (Ordered)
- 4.3 (m) Attribute: Priority
- 4.3.1 (m) Attribute: DlIIPriority (Urgent, Normal, TimeAvailable)
- 4.3.2 (m) Attribute: DlIIPriorityNegotiated (Urgent, Normal, TimeAvailable)
- 4.4 (m) Attribute: DlIpduAuthentication (Ordinary, Source, Maximal)
- 4.5 (m) Attribute: ResidualActivity
- 4.5.1 (m) Attribute: ResidualActivityAsSender (True, False)
- 4.5.2 (m) Attribute: ResidualActivityAsReceiver (True, False)
- 4.6 (m) Attribute: MaxConfirmDelay
- 4.6.1 (m) Attribute: MaxConfirmDelayOnDIConnect
- 4.6.2 (m) Attribute: MaxConfirmDelayOnDIData
- 4.7 (m) Attribute: DISchedulingPolicy (Explicit)
- 4.8 (m) Attribute: MaxDlIsduSizes
- 4.8.1 (m) Attribute: MaxDlIsduSizeFromRequestor
- 4.8.2 (m) Attribute: MaxDlIsduSizeFromResponder
- 4.8.3 (m) Attribute: MaxDlIsduSizeFromRequestorNegotiated
- 4.8.4 (m) Attribute: MaxDlIsduSizeFromResponderNegotiated

DLL SERVICES:

- 1 (m) OpsService: DL-Put
- 2 (m) OpsService: DL-Get
- 3 (m) OpsService: DL-Connect
- 4 (m) OpsService: DL-Connection-Established
- 5 (m) OpsService: DL-Disconnect
- 6 (m) OpsService: DL-Buffer-Received
- 7 (m) OpsService: DL-Buffer-Sent

4.6.7.2.2 Attributes

4.6.7.2.2.1 LocalDlcepAddress

This attribute specifies the local DLCEP address and identifies the DLCEP. The value of this attribute is used as the “DLCEP-address” parameter of the DLL.

NOTE 1 The value of this attribute is also carried in the Establish Request PDU.

This attribute contains the following three subattributes: Link Address, Node Address, and Selector.

NOTE 2 Since the local Link and Node addresses are set by the Network Management, only the Selector portion of the LocalDlcepAddress attribute is a configurable attribute of the FAL.

4.6.7.2.2.2 RemoteDlcepAddress

This attribute specifies the remote DLCEP address and identifies the DLCEP.

This attribute supplies the value for called DLCEP-address of the DL-Connect service.

NOTE 1 The value of this attribute is also carried in the header part of the Establish Request PDU.

This attribute contains the following three subattributes: Link Address, Node Address, and Selector.

NOTE 2 Since the local Link and Node addresses are set by the Network Management, only the Selector portion of the LocalDlcepAddress attribute is a configurable attribute of the FAL.

4.6.7.2.2.3 DlcepRole

This attribute specifies the behavior of the DLSAP that is used by the AREP. This attribute supplies the value for the “DL(SAP)-role” parameter.

4.6.7.2.2.4 QosParameterSet

The QosParameterSet attributes specify the DL quality of service that is used by the AREP. These attribute values may be negotiated with the remote AREP.

4.6.7.2.2.5 DlcepClass

This attribute specifies the behavior of the DLCEP which is attached to the AREP.

This attribute supplies the value for the “DLCEP class” parameter of the DLL. The possible value of this attribute is Peer and corresponds to PEER defined by the DLL.

4.6.7.2.2.6 DlcepDataDeliveryFeatures

These attributes specify the data delivery features of the DLL.

The permitted value Ordered corresponds to ORDERED defined by the Data Link Layer standard.

NOTE The FromRequestorToResponder and FromResponderToRequestor attributes shall have the same value.

4.6.7.2.2.7 FromRequestorToResponder

This attribute specifies the DLL data delivery feature of the AREP as a sender. It supplies the value for the “DLCEP data delivery features from requestor to responder(s)” parameter defined in the DLL.

4.6.7.2.2.8 FromResponderToRequestor

This attribute specifies the DLL data delivery feature of the AREP as a receiver. It supplies the value for the “DLCEP data delivery features from responder(s) to requestor” parameter defined in the DLL.

4.6.7.2.2.9 DllPriority

This attribute specifies the DLL priority before negotiation.

NOTE It is not possible to use different priorities for each FAL-PDU sent from the same BNB AREP. Also, it is not permitted to use different priorities for the send and the receive conveyance paths.

4.6.7.2.2.10 DllPriorityNegotiated

This attribute specifies the DLL priority after negotiation.

4.6.7.2.2.11 DlpduAuthentication

This attribute specifies the lower bound of the length of DL-addressing to be used by the DLL.

This attribute supplies the value for the “DLPDU-authentication” parameter of the DLL. The permitted value Ordinary, Source and Maximal correspond to ORDINARY, SOURCE and MAXIMAL respectively, as defined in Part-3 and –4 of this International Standard.

4.6.7.2.2.12 ResidualActivityAsSender

This attribute specifies sender’s DLC residual activity. It supplies the value for the “Residual activity as sender” parameter as defined in the DLL. The possible values are “True” and “False”.

4.6.7.2.2.13 ResidualActivityAsReceiver

This attribute specifies receiver’s DLC residual activity. It supplies the value for the “Residual activity as receiver” parameter defined in the DLL. The possible values are “True” and “False”.

4.6.7.2.2.14 MaxConfirmDelay

This attribute specifies the maximum confirmation delay of certain DLL connection-oriented services.

4.6.7.2.2.15 MaxConfirmDelayOnDIConnect

This attribute specifies the maximum confirmation delay for a confirmation from a DL-Connect service.

This attribute supplies the value for the “Maximum confirmation delay on DL-Connect, DL-Reset and DL-Subscriber-Query” parameter specified in the Data Link Layer standard.

4.6.7.2.2.16 MaxConfirmDelayOnDIData

This attribute specifies the maximum confirmation delay for a confirmation from a DL-Data service.

This attribute supplies the value for the “Maximum confirmation delay on DL-Data” parameter specified in the Data Link Layer standard (IEC 61158-3).

4.6.7.2.2.17 DISchedulingPolicy

This attribute provides the guidance to the DLL on the scheduling needed by an AR. For this AREP, the DLL tries to transmit the FAL-PDU as soon as possible.

This attribute supplies the value for the “DL-Scheduling-policy” parameter of the DLL. The permitted value Explicit corresponds to EXPLICIT defined in Part-3 and –4 of this International Standard.

4.6.7.2.2.18 MaxDisduSizeFromRequestor

This attribute specifies the maximum length of an FAL-PDU that can be sent from this AREP before negotiation.

This attribute supplies the value for the “Maximum DLSDU sizes from requestor” parameter of the DLL.

4.6.7.2.2.19 MaxDisduSizeFromResponder

This attribute specifies the maximum length of an FAL-PDU that can be received by this AREP before negotiation.

This attribute supplies the value for the “Maximum DLSDU sizes from responder” parameter of the DLL.

4.6.7.2.2.20 MaxDisduSizeFromRequestorNegotiated

This attribute specifies the maximum length of an FAL-PDU that can be sent from this AREP after negotiation.

4.6.7.2.2.21 MaxDisduSizeFromResponderNegotiated

This attribute specifies the maximum length of an FAL-PDU that can be received by this AREP after negotiation.

4.6.7.2.3 DLL services

Refer to 4.7.4, Data Link Layer Service Selection, for DLL service descriptions.

4.6.7.3 BNB ARPM state machine

4.6.7.3.1 BNB ARPM states

The defined states and their descriptions of the BNB ARPM are shown in Table 125 and Figure 34 through 37.

Table 125 – BNB ARPM states

State	Description
CLOSED	The AREP is defined, but not capable of sending or receiving FAL-PDUs. It may send or receive Establish service FAL-PDUs while in this state.
OPEN	The AREP is defined and capable of sending or receiving FAL-PDUs.
OPEN_CSC	The AREP is defined and capable of sending Confirmed Request FAL-PDUs as client.
OPEN_CSS	The AREP is defined and capable of sending Confirmed Response FAL-PDUs as server.
OPEN_UCSC	The AREP is defined and capable of sending Unconfirmed Request FAL-PDUs as client.
REQUESTING (REQ)	The AREP has sent an Establish Request FAL-PDU and is waiting for a response from the remote AREP.
RESPONDING (RSP)	The AREP has received an Establish Request FAL-PDU, delivered an Establish.indication primitive and is waiting for a response from its user.
REPLIED (REPL)	The Server AREP has issued an EST_rsp(+) primitive and is waiting for receiving a "connection established" indication from the DLL.
WAIT_SENT_CSC	The AREP has put a Confirmed Request FAL-PDU into the buffer and is waiting for a "DMPM_Buffer_Sent" indication from the DLL.
WAIT_SENT_CSS	The AREP has put a Confirmed Response FAL-PDU into the buffer and is waiting for a "DMPM_Buffer_Sent" indication from the DLL.
WAIT_SENT_UCSC	The AREP has put an Unconfirmed Request FAL-PDU into the buffer as client and is waiting for a "DMPM_Buffer_Sent" indication from the DLL.
WAIT_RESOURCE_CSS	The AREP is waiting for a free resource (buffer) to send a Confirmed Response FAL-PDU.
WAIT_RESOURCE_CSC	The AREP is waiting for a free resource (buffer) to send a Confirmed Request FAL-PDU.
WAIT_CYC_RESOURCE	The AREP is waiting for a free resource (buffer) to send a Confirmed Request FAL-PDU.
WAIT_RESOURCE_UCSC	The AREP is waiting for a free resource (buffer) to send an Unconfirmed Request FAL-PDU.
WAIT_RES	The AREP is capable of receiving a Confirmed Response FAL-PDU.
WAIT_CON	The AREP is capable of getting a Confirmed Response FAL-PDU from the buffer.
WAIT_CYC_RES	The AREP is capable of receiving a Confirmed Response FAL-PDU and sending a Confirmed Request FAL-PDU cyclic.
WAIT_CYC_IND	The AREP is capable of receiving a Confirmed Request FAL-PDU and sending a Confirmed Response FAL-PDU cyclic.
WAITING_CS	The AREP is waiting for a Confirmed Response FAL-PDU after receiving an Unconfirmed Request FAL-PDU.
SAME	Indicates that the next state is the same as the current state.

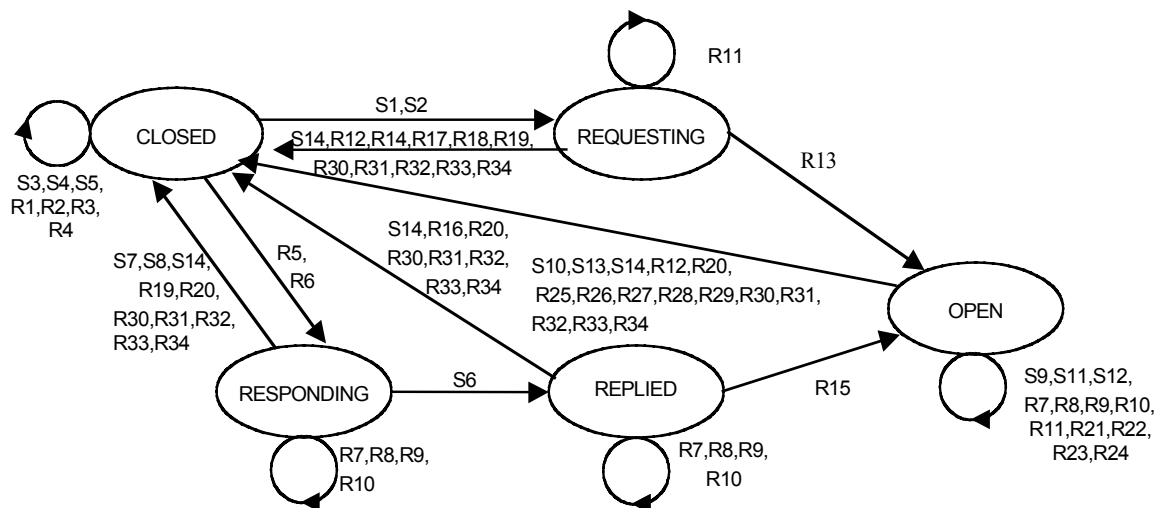


Figure 34 – State transition diagram of BNB ARPM (basic state machine)

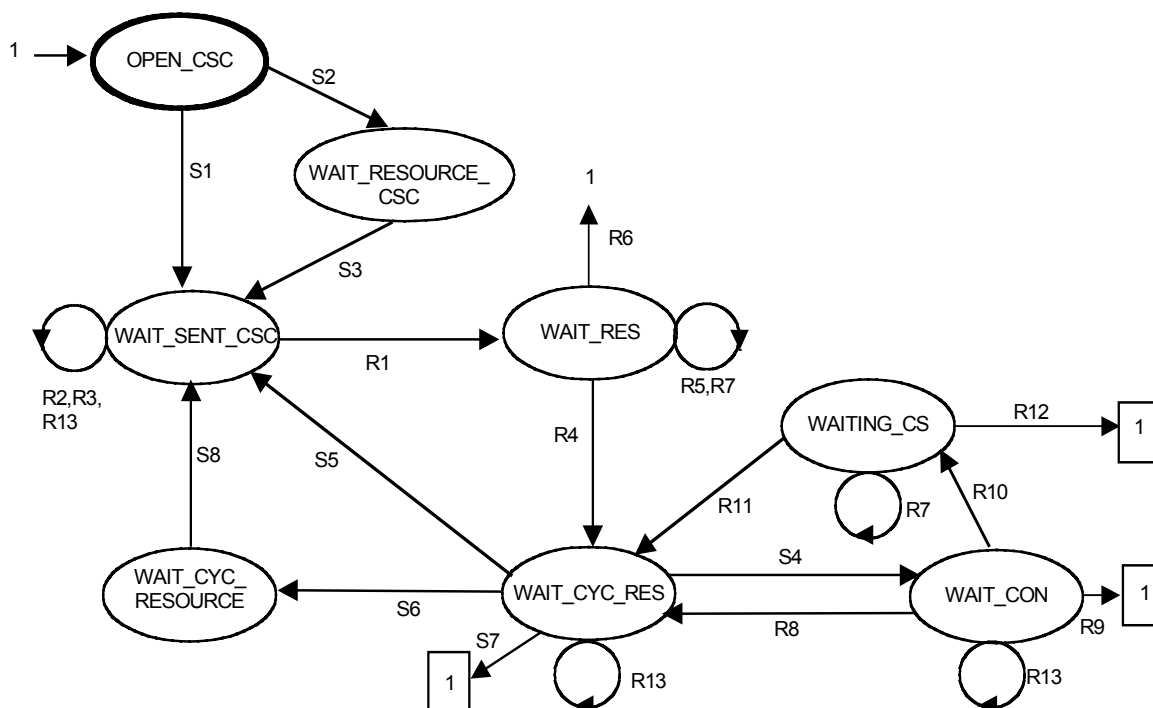


Figure 35 – State transition diagram of BNB ARPM (confirmed service sending and receiving – client)

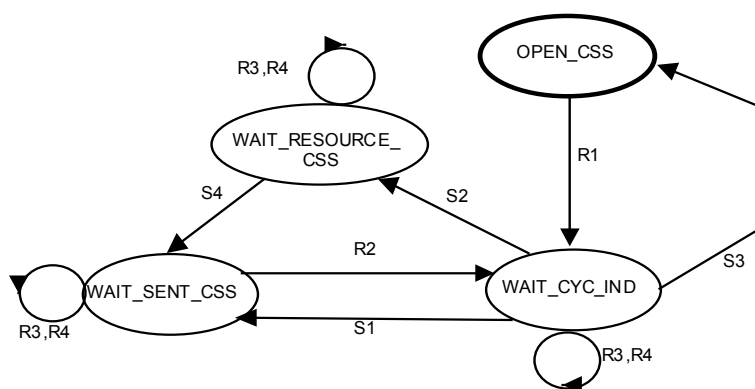


Figure 36 – State transition diagram of BNB ARPM (confirmed service receiving and responding – server)

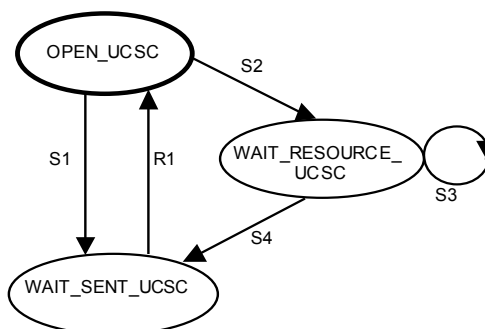


Figure 37 – State transition diagram of BNB ARPM (unconfirmed service sending- client)

4.6.7.3.2 BNB ARPM state table (basic state machine)

Table 126 and Table 127 define the BNB ARPM state machine.

Table 126 – BNB ARPM state table – sender transactions (basic state machine)

#	Current state	Event or condition => action	Next state
S1	CLOSED	<pre> EST_req && Initiator = "True" && RemoteAddressConfigurationType = "Free" => RemoteDlcepAddress := remote_dlcep_address, FAL-PDU_req { dmpm_service_name := "DMPM_Connect_req", arep_id := GetArepld (), called_address := "default dlsap address", calling_address := "default dlsap address", local_dlcep_address := LocalDlcep, dlsdu := BuildFAL-PDU (fal_pdu_name := "EST_ReqPDU", calling_dlcep_address := LocalDlcepAddress, called_dlcep_address := RemoteDlcepAddress, fal_data := user_data) } </pre>	REQ
S2	CLOSED	<pre> EST_req && Initiator = "True" && RemoteAddressConfigurationType = "Linked" => FAL-PDU_req { dmpm_service_name := "DMPM_Connect_req", arep_id := GetArepld (), called_address := "default dlsap address", calling_address := "default dlsap address", local_dlcep_address := LocalDlcep, dlsdu := BuildFAL-PDU (fal_pdu_name := "EST_ReqPDU", calling_dlcep_address := LocalDlcepAddress, called_dlcep_address := RemoteDlcepAddress, fal_data := user_data) } </pre>	REQ
S3	CLOSED	<pre> EST_req && Initiator = "False" => Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid Event for Role" } </pre>	CLOSED
S4	CLOSED	<pre> unallowed primitive => Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Unallowed AREP primitive in connection establishment" } </pre>	CLOSED
S5	CLOSED	<pre> Abort_req => (ignore) </pre>	CLOSED
S6	RSP	<pre> EST_rsp(+) => FAL-PDU_req { dmpm_service_name := "DMPM_Connect_rsp", arep_id := GetArepld (), responding_address := "default dlsap address", local_dlcep_address := LocalDlcep, dlsdu := BuildFAL-PDU (fal_pdu_name := "EST_RspPDU", fal_data := user_data) } </pre>	REPL

Table 126 (continued)

#	Current state	Event or condition => action	Next state
S7	RSP	EST_rsp(-) => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "connection rejection-transient condition", dlsdu := BuildFAL-PDU (fal_pdu_name := "EST_ErrPDU", fal_data := user_data) } }	CLOSED
S8	RSP	unallowed primitive => Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Unallowed AREP primitive in connection establishment " }, FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := " Unallowed AREP primitive in connection ", dlsdu := "null" } }	CLOSED
S9	OPEN	CS_req && VA_ASE_Service = "Read.req" && status CS Req. = OPEN_CSC => start CS Req.	OPEN
S10	OPEN	CS_req && VA_ASE_Service <> "Read.req" => Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Unallowed AREP primitive " }, FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := " Unallowed AREP primitive ", dlsdu := "null" }, reset all machines, StopTimer	CLOSED
S11	OPEN	UCS_req && status UCSC Req. = OPEN_UCSC => start UCSC Req.	OPEN
S12	OPEN	UCS_req && status UCSC Req. <> OPEN_UCSC => (no actions taken)	OPEN

Table 126 (continued)

#	Current state	Event or condition => action	Next state
S13	OPEN	unallowed Services => <pre> Abort_ind { arep_id := GetArepId (), locally_generated := "True", identifier := "FAL", reason_code := "Unallowed AREP primitive " }, FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepId (), reason := "Error of Remote Station", dlsdu := BuildFAL-PDU (fal_pdu_name := "Abort_PDU", fal_id := identifier, fal_reason_code := reason_code, fal_additional_detail := additional_detail) }, reset all machines, StopTimer </pre>	CLOSED
S14	NOT CLOSED	Abort_req => <pre> FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepId (), reason := "disconnection-normal condition", dlsdu := BuildFAL-PDU (fal_pdu_name := "Abort_PDU", fal_id := identifier, fal_reason_code := reason_code, fal_additional_detail := additional_detail) }, reset all machines, StopTimer </pre>	CLOSED

Table 127 – BNB state table – receiver transactions (basic state machine)

#	Current state	Event or condition => action	Next state
R1	CLOSED	Connect_ind && Initiator = "True" => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), reason:= "Multiple Initiators", dlsdu:= "null" }	CLOSED
R2	CLOSED	Connect_ind && Initiator = "False" && FAL_Pdu_type (dls_user_data) <> "EST_ReqPDU" => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), reason:= "Invalid FAL-PDU", dlsdu:= "null" }	CLOSED
R3	CLOSED	Connect_ind && Initiator = "False" && FAL_Pdu_type (dls_user_data) = "EST_ReqPDU" && RemoteAddressConfigurationType = "Linked" && RemoteDlcepAddress <> calling_dlcep_address => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), reason:= "Remote Address Mismatch", dlsdu:= "null" }	CLOSED
R4	CLOSED	unallowed primitive => (no actions taken)	CLOSED
R5	CLOSED	Connect_ind && Initiator = "False" && FAL_Pdu_type (dls_user_data) = "EST_ReqPDU" && RemoteAddressConfigurationType = "Linked" && RemoteDlcepAddress = calling_dlcep_address => MaxDlsduSizeFromRequestorNegotiated := dlsdu_size_from_requestor, MaxDlsduSizeFromResponderNegotiated := dlsdu_size_from_responder, EST_ind { arep_id:= GetArepld (), user_data:= dls_user_data }	RSP
R6	CLOSED	Connect_ind && Initiator = "False" && FAL_Pdu_type (dls_user_data) = "EST_ReqPDU" && RemoteAddressConfigurationType = "Free" => RemoteDlcepAddress:= calling_dlcep_address, MaxDlsduSizeFromRequestorNegotiated := dlsdu_size_from_requestor, MaxDlsduSizeFromResponderNegotiated := dlsdu_size_from_responder, EST_ind { arep_id:= GetArepld (), user_data:= dls_user_data }	RSP

Table 127 (continued)

#	Current state	Event or condition => action	Next state
R7	RSP REPL OPEN	Connect_ind && Initiator = "False" && FAL_Pdu_type (dls_user_data) = "EST_ReqPDU" && RemoteAddressConfigurationType = "Linked" && RemoteDlcepAddress <> calling_dlcep_address => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), reason:= "Remote Address Mismatch", dlsdu:= "null" } 	SAME
R8	RSP REPL OPEN	Connect_ind && Initiator = "False" && FAL_Pdu_type (dls_user_data) = "EST_ReqPDU" && RemoteDlcepAddress = calling_dlcep_address => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), reason:= "Invalid FAL-PDU", dlsdu:= "null" }, Abort_ind { arep_id:= GetArepld (), locally_generated:= "True", identifier:= "FAL", reason_code:= "Invalid FAL-PDU" } reset all machines, StopTimer	CLOSED
R9	RSP REPL OPEN	Connect_ind && Initiator = "False" && FAL_Pdu_type (dls_user_data) = "EST_ReqPDU" && RemoteAddressConfigurationType = "Free" && RemoteDlcepAddress <> calling_dlcep_address => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), reason:= "AREP Busy", dlsdu:= "null" } 	SAME
R10	RSP REPL OPEN	Connect_ind && Initiator = "False" && FAL_Pdu_type (dls_user_data) <> "EST_ReqPDU" && RemoteAddressConfigurationType = "Free" && RemoteDlcepAddress <> calling_dlcep_address => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), reason:= "Invalid FAL-PDU", dlsdu:= "null" } 	SAME
R11	REQ OPEN	Connect_ind && Initiator = "True" && RemoteDlcepAddress <> calling_dlcep_address => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), reason:= "Multiple Initiators", dlsdu:= "null" } 	SAME

Table 127 (continued)

#	Current state	Event or condition => action	Next state
R12	REQ OPEN	Connect_ind && Initiator = "True" && RemoteDlcepAddress = calling_dlcep_address => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), reason:= "Multiple Initiators", dlsdu:= "null" }, Abort_ind { arep_id:= GetArepld (), locally_generated:= "True", identifier:= "FAL", reason_code:= "Multiple Initiators" } reset all machines, StopTimer	CLOSED
R13	REQ	Connect_cnf && FAL_Pdu_Type (dls_user_data) = "EST-RspPDU" => MaxDlsduSizeFromRequestorNegotiated := dlsdu_size_from_requestor, MaxDlsduSizeFromResponderNegotiated := dlsdu_size_from_responder, DllPriorityNegotiated:= dll_priority, EST_cnf(+) { arep_id:= GetArepld (), user_data:= dls_user_data }	OPEN
R14	REQ	Connect_cnf && FAL_Pdu_Type (dls_user_data) <> "EST-RspPDU" => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), reason:= "Invalid FAL-PDU", dlsdu:= "null" }, Abort_ind { arep_id:= GetArepld (), locally_generated:= "True", identifier:= "FAL", reason_code:= "Invalid FAL-PDU" }	CLOSED
R15	REPL	FAL-PDU_ind && dmpm_service_name = "DMPM_Connection_Established_ind" => (no actions taken)	OPEN
R16	REPL	FAL-PDU_ind && ((dmpm_service_name <> "DMPM_Disconnect_ind") && (dmpm_service_name <> "DM_Connection_Established_ind")) => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), reason:= "Invalid DI PDU", dlsdu:= "null" }, Abort_ind { arep_id:= GetArepld (), locally_generated:= "True", identifier:= "FAL", reason_code:= "Invalid DI PDU" additional_detail:= "null" }	CLOSED

Table 127 (continued)

#	Current state	Event or condition => action	Next state
R17	REQ	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && FAL_Pdu_Type (dls_user_data) = "EST-ErrPDU" => EST_cnf(-) { arep_id:= GetArepld (), user_data:= dls_user_data }	CLOSED
R18	REQ	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu <> "null" && ((FAL_Pdu_Type (dls_user_data) <> "Abort_PDU") && (FAL_Pdu_Type (dls_user_data) <> "EST-ErrPDU") => Abort_ind { arep_id:= GetArepld (), locally_generated:= "True", identifier:= "FAL", reason_code:= "Invalid FAL-PDU" additional_detail:= "null" }	CLOSED
R19	REQ RSP	FAL-PDU_ind && dmpm_service_name <> "DMPM_Disconnect_ind" => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), reason:= "Invalid DI PDU", dlsdu:= "null" }, Abort_ind { arep_id:= GetArepld (), locally_generated:= "True", identifier:= "FAL", reason_code:= "Invalid DI PDU" }	CLOSED
R20	REPL RSP OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu <> "null" && FAL_Pdu_Type (dls_user_data) <> "Abort_PDU" => Abort_ind { arep_id:= GetArepld (), locally_generated:= "True", identifier:= "FAL", reason_code:= "Invalid FAL-PDU" additional_detail:= "null" } reset all machines, StopTimer	CLOSED
R21	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Buffer_Received_ind" && FAL_Pdu_Type (dls_user_data) = "CS_ReqPDU" && status CS Res. = OPEN_CSS => start CS Rsp.	OPEN
R22	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Buffer_Received_ind" && dmpm_duplicate_dlsdu () = "False" && FAL_Pdu_Type (dls_user_data) = "UCS_ReqPDU" => UCS_ind { arep_id := GetArepld (), user_data := fal_pdu }	OPEN
R23	OPEN	UCSC Req. finished	OPEN

Table 127 (continued)

#	Current state	Event or condition => action	Next state
R24	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Buffer_Received_ind" && dmpm_duplicate_dlsdu () = "True" => (no actions taken)	OPEN
R25	OPEN	FAL-PDU_ind && ((FAL_Pdu_Type (dls_user_data) <> "AbortPDU") (FAL_Pdu_Type (dls_user_data) <> "CS_RspPDU") (FAL_Pdu_Type (dls_user_data) <> "CS_ReqPDU") (FAL_Pdu_Type (dls_user_data) <> "UCS_ReqPDU")) => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), reason:= "Invalid FAL-PDU", dlsdu:= "null" }, Abort_ind { arep_id:= GetArepld (), locally_generated:= "True", identifier:= "FAL", reason_code:= "Invalid FAL-PDU" additional_detail:= "null" } reset all machines, StopTimer	CLOSED
R26	OPEN	MCTimer expired => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), reason:= "MCTimer expired", dlsdu:= "null" }, Abort_ind { arep_id:= GetArepld (), locally_generated:= "True", identifier:= "FAL", reason_code:= "MCTimer expired" additional_detail:= "null" }, reset all machines, StopTimer	CLOSED
R27	OPEN	MSTimer expired => FAL-PDU_req { dmpm_service_name:= "DMPM_Disconnect_req", arep_id:= GetArepld(), reason:= "MSTimer expired", dlsdu:= "null" }, Abort_ind { arep_id:= GetArepld (), locally_generated:= "True", identifier:= "FAL", reason_code:= "MSTimer expired" additional_detail:= "null" }, reset all machines, StopTimer	CLOSED

Table 127 (continued)

#	Current state	Event or condition => action	Next state
R28	OPEN	CS Res. finished => <pre> Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Execution error in cyclic data transfer" }, FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Execution error in cyclic data transfer", dlsdu := "null" }, reset all machines, StopTimer </pre>	CLOSED
R29	OPEN	CS Rsp. finished => <pre> Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Execution error in cyclic data transfer" }, FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Execution error in cyclic data transfer", dlsdu := "null" }, reset all machines, StopTimer </pre>	CLOSED
R30	NOT CLOSED	FAL-PDU_ind && dmpm_service_name <> "DMPM_Disconnect_ind" && fal_pdu <> "null" && FAL_Pdu_Type (dls_user_data) = "Abort_PDU" => <pre> Abort_ind { arep_id:= GetArepld (), locally_generated:= "False", identifier:= AbortIdentifier (fal_pdu), reason_code:= AbortReason (fal_pdu), additional_detail:= AbortDetail (fal_pdu) } reset all machines, StopTimer </pre>	CLOSED
R31	NOT CLOSED	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu = "null" && originator = "remote_dls_provider" => <pre> Abort_ind { arep_id:= GetArepld (), locally_generated:= "False", identifier:= "Data Link Layer", reason_code:= reason additional_detail:= "null" } reset all machines, StopTimer </pre>	CLOSED

Table 127 (continued)

#	Current state	Event or condition => action	Next state
R32	NOT CLOSED	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu = "null" && originator = "remote_dls_user" => Abort_ind { arep_id:= GetArepld (), locally_generated:= "False", identifier:= "FAL", reason_code:= reason additional_detail:= "null" } reset all machines, StopTimer	CLOSED
R33	NOT CLOSED	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu = "null" && originator = "local_dls_provider" => Abort_ind { arep_id:= GetArepld (), locally_generated:= "True", identifier:= "Data Link Layer", reason_code:= reason, additional_detail:= "null" } reset all machines, StopTimer	CLOSED
R34	NOT CLOSED	ErrorToARPM => (no actions taken) NOTE It is a local matter to report this error status to network management entities. The ARPM does not abort the existing connections. The FAL user may issue an Abort request to disconnect the current connection, depending on the type of status information conveyed by the ErrorToARPM primitive.	SAME

4.6.7.3.3 BNB ARPM state table (confirmed service sending and receiving – client)

Table 128 and Table 129 define the BNB ARPM state machine.

Table 128 – BNB ARPM state table – sender transactions (confirmed service sending and receiving – client)

#	Current state	Event or condition => action	Next state
S1	OPEN_CSC	CS Req. started && Flag (Buffer_sent_Flag) = "True" => SetFlag (Buffer_sent_Flag,False), FAL-PDU_req { dmpm_service_name:= "DMPM_Put_req", arep_id:= GetArepld(), dlsdu:= BuildFAL-PDU(fal_pdu_name:= "CS_ReqPDU", fal_data:= user_data) }, StoreIndex (index), SetFlag (Change_Flag,True), StartTimer(MC)	WAIT_SENT_CSC
S2	OPEN_CSC	CS Req. started && Flag (Buffer_sent_Flag) = "False" StoreService(CS_req)	WAIT_RESOURCE

#	Current state	Event or condition => action	Next state
S3	WAIT_RESOURCE_CSC	status UCS Req. <> WAIT_SENT_UCS && status CS Rsp. <> WAIT_SENT_CSS => SetFlag (Buffer_sent_Flag,False), RestoreService(CS_req), FAL-PDU_req { dmpm_service_name:= "DMPM_Put_req", arep_id:= GetArepld(), dlsdu:= BuildFAL-PDU(fal_pdu_name:= "CS_ReqPDU", fal_data:= user_data) }, StoreIndex (index), SetFlag (Change_Flag,True), StartTimer(MC)	WAIT_SENT_CSC
S4	WAIT_CYC_RES	CS_req && VA_ASE_Service = "Read.req" && TestIndex (index) = "True" => FAL-PDU_req { dmpm_service_name:= "DMPM_Get_req", arep_id:= GetArepld(), }	WAIT_CON
S5	WAIT_CYC_RES	CS_req && VA_ASE_Service = "Read.req" && TestIndex (index) = "False" && Flag (Buffer_sent_Flag) = "True" => SetFlag (Buffer_sent_Flag,False) FAL-PDU_req { dmpm_service_name:= "DMPM_Put_req", arep_id:= GetArepld(), dlsdu:= BuildFAL-PDU (fal_pdu_name:= "CS_ReqPDU", fal_data:= user_data) }, StoreIndex (index), SetFlag (Change_Flag,True)	WAIT_SENT_CSC
S6	WAIT_CYC_RES	CS_req && VA_ASE_Service = "Read.req" && TestIndex (index) = "False" && Flag (Buffer_sent_Flag) = "False" => StoreService(CS_req)	WAIT_CYC_RESOURCE
S7	WAIT_CYC_RES	CS_req && VA_ASE_Service <> "Read.req" => CS Req. finish	OPEN_CSC
S8	WAIT_CYC_RESOURCE	status UCS Req. <> WAIT_SENT_UCS && status CS Rsp. <> WAIT_SENT_CSS => SetFlag (Buffer_sent_Flag,False), RestoreService(CS_req), FAL-PDU_req { dmpm_service_name:= "DMPM_Put_req", arep_id:= GetArepld(), dlsdu:= BuildFAL-PDU(fal_pdu_name:= "CS_ReqPDU", fal_data:= user_data) }, StoreIndex (index), SetFlag (Change_Flag,True)	WAIT_SENT_CSC

Table 129 – BNB state table – receiver transactions (confirmed service sending and receiving – client)

#	Current state	Event or condition => action	Next state
R1	WAIT_SENT_CSC	FAL-PDU_ind && dmpm_service_name = "DMPM_Buffer_Sent_ind" && Flag (Buffer_sent_Flag) = "False" => SetFlag (Buffer_sent_Flag,True)	WAIT_RES
R2	WAIT_SENT_CSC	FAL-PDU_ind && dmpm_service_name = "DMPM_Buffer_Received_ind" && dmpm_duplicate_dlsdu = "False" && FAL_Pdu_Type (dls_user_data) = "CS_RspPDU" && TestIndex (index) = "True" => StartTimer(MC)	WAIT_SENT_CSC
R3	WAIT_SENT_CSC	FAL-PDU_ind && dmpm_service_name = "DMPM_Buffer_Received_ind" && dmpm_duplicate_dlsdu = "False" && FAL_Pdu_Type (dls_user_data) <> "CS_RspPDU" => (no actions taken)	WAIT_SENT_CSC
R4	WAIT_RES	FAL-PDU_ind && dmpm_service_name = "DMPM_Buffer_Received_ind" && dmpm_duplicate_dlsdu () = "False" && FAL_Pdu_Type (dls_user_data) = "CS_RspPDU" && TestIndex (index) = "True" => CS_con { arep_id := GetArepld (), user_data := fal_pdu }, SetFlag (Change_Flag,False), StartTimer(MC)	WAIT_CYC_RES
R5	WAIT_RES	FAL-PDU_ind && dmpm_service_name = "DMPM_Buffer_Received_ind" && dmpm_duplicate_dlsdu () = "False" && FAL_Pdu_Type (dls_user_data) = "CS_RspPDU" && Flag (Change_Flag) = "True" && IDM_REQ-Index <> index => (no actions taken)	WAIT_RES
R6	WAIT_RES	FAL-PDU_ind && dmpm_service_name = "DMPM_Buffer_Received_ind" && dmpm_duplicate_dlsdu () = "False" && FAL_Pdu_Type (dls_user_data) = "CS_RspPDU" && TestIndex (index) = "False" && Flag (Change_Flag) = "False" => CS Req. finish	OPEN_CSC
R7	WAIT_RES WAITING_CS	FAL-PDU_ind && dmpm_service_name = "DMPM_Buffer_Received_ind" && dmpm_duplicate_dlsdu () = "False" && FAL_Pdu_Type (dls_user_data) <> "CS_RspPDU" => (no actions taken)	WAIT_RES
R8	WAIT_CON	FAL-PDU_ind && dmpm_service_name = "DMPM_Get_cnf" && FAL_Pdu_Type (dls_user_data) = "CS_RspPDU" && TestIndex (index) = "True" => CS_con { arep_id := GetArepld (), user_data := fal_pdu }	WAIT_CYC_RES
R9	WAIT_CON	FAL-PDU_ind && dmpm_service_name = "DMPM_Get_cnf" && FAL_Pdu_Type (dls_user_data) = "CS_RspPDU" && TestIndex (index) = "False" => CS Req. finish	OPEN_CSC

#	Current state	Event or condition => action	Next state
R10	WAIT_CON	FAL-PDU_ind && dmpm_service_name = "DMPM_Get_cnf" && FAL_Pdu_Type (dls_user_data) <> "CS_ResPDU" => (no actions taken)	WAITING_CS
R11	WAITING_CS	FAL-PDU_ind && dmpm_service_name = "DMPM_Buffer_Received_ind" && dmpm_duplicate_dlsdu = "False" && FAL_Pdu_Type (dls_user_data) = "CS_RspPDU" && TestIndex (index) = "True" => CS_con { arep_id := GetArepId (), user_data := fal_pdu }, StartTimer(MC)	WAIT_CYC_RES
R12	WAITING_CS	FAL-PDU_ind && dmpm_service_name = "DMPM_Buffer_Received_ind" && dmpm_duplicate_dlsdu = "False" && FAL_Pdu_Type (dls_user_data) = "CS_RspPDU" && TestIndex (index) = "False" => CS Req. finish	OPEN_CSC
R13	WAIT_CYC_RES WAIT_SENT_CSC WAIT_CON WAIT_CYC_RESOURCE	FAL-PDU_ind && dmpm_service_name = "DMPM_Buffer_Received_ind" && dmpm_duplicate_dlsdu = "False" && FAL_Pdu_Type (dls_user_data) = "CS_RspPDU" => StartTimer(MC)	SAME

4.6.7.3.4 BNB ARPM state table (confirmed service receiving and responding – server)

Table 130 and Table 131 define the BNB ARPM state machine.

Table 130 – BNB ARPM state table – sender transactions (confirmed service receiving and responding – server)

#	Current state	Event or condition => action	Next state
S1	WAIT_CYC_IND	CS_rsp && VA_ASE_Service = "Read.rsp" && Flag (Buffer_sent_Flag) = "True" => SetFlag (Buffer_sent_Flag,False), FAL-PDU_req { dmpm_service_name:= "DMPM_Put_req", arep_id:= GetArepld (), dlsdu:= BuildFAL-PDU (fal_pdu_name:= "CS_RspPDU", fal_data:= user_data) } 	WAIT_SENT_CSS
S2	WAIT_CYC_IND	CS_rsp && VA_ASE_Service = "Read.rsp" && Flag (Buffer_sent_Flag) = "False" => StoreService(CS_req) 	WAIT_RESOURCE_CSS
S3	WAIT_CYC_IND	CS_rsp && VA_ASE_Service <> "Read.rsp" => CS Rsp. finish 	OPEN_CSS
S4	WAIT_RESOURCE_CSS	Closed UCS && ((state CS Req. = OPEN_CSC) (state CS Req. = WAIT_CYC_RES)) => SetFlag (Buffer_sent_Flag,False), RestoreService(CS_req) FAL-PDU_req { dmpm_service_name:= "DMPM_Put_req", arep_id:= GetArepld(), dlsdu:= BuildFAL-PDU(fal_pdu_name:= "CS_RspPDU", fal_data:= user_data) }, StartTimer(MS) 	WAIT_SENT_CSS

Table 131 – BNB state table – receiver transactions (confirmed service receiving and responding – server)

#	Current state	Event or condition => action	Next state
R1	OPEN_CSS	CS Rsp. Started => StoreService(FAL-PDU_ind) CS_ind { arep_id:= GetArepld (), user_data:= fal_pdu }, StartTimer(MS)	WAIT_CYC_IND
R2	WAIT_SENT_CSS	FAL-PDU_ind && dmpm_service_name = "DMPM_Buffer_Sent_ind" && Flag (Buffer_sent_Flag) = "False" RestoreService(FAL-PDU_ind), CS_ind { arep_id:= GetArepld (), user_data:= fal_pdu }, SetFlag (Buffer_sent_Flag,True)	WAIT_CYC_IND
R3	WAIT_CYC_IND WAIT_SENT_CSS WAIT_RESOURCE_CSS	FAL-PDU_ind && dmpm_service_name = "DMPM_Buffer_Received_ind" && dmpm_duplicate_dlsdu = "False" && FAL-PDU_Type (dls_user_data) = "CS_ReqPDU" => StoreService(FAL-PDU_ind), CS_ind { arep_id:= GetArepld (), user_data:= fal_pdu }, StartTimer(MS)	SAME
R4	WAIT_CYC_IND WAIT_SENT_CSS WAIT_RESOURCE_CSS	FAL-PDU_ind && dmpm_service_name = "DMPM_Buffer_Received_ind" && dmpm_duplicate_dlsdu = "False" && FAL-PDU_Type (dls_user_data) <> "CS_ReqPDU" (no actions taken)	SAME

4.6.7.3.5 BNB ARPM state table (unconfirmed service sending – client)

Table 132 and Table 133 define the BNB ARPM state machine.

**Table 132 – BNB ARPM state table – sender transactions
(unconfirmed service sending – client)**

#	Current state	Event or condition => action	Next state
S1	OPEN_UCSC	UCSC Req. started && Flag (Buffer_sent_Flag) = "True" => SetFlag (Buffer_sent_Flag,False), FAL-PDU_req { dmpm_service_name:= "DMPM_Put_req", arep_id:= GetArepId(), dlsdu:= BuildFAL-PDU (fal_pdu_name:= "UCS_ReqPDU", fal_data:= user_data) } }	WAIT_SENT_UCSC
S2	OPEN_UCSC	UCSC Req. started && Flag (Buffer_sent_Flag) = "False" => StoreService(UCS_req)	WAIT_RESOURCE_UCSC
S3	WAIT_RESOURCE_UCSC	state CS Req. = WAIT_SENT_CSC state CS Rsp. = WAIT_SENT_CSS (no actions taken)	WAIT_RESOURCE_UCSC
S4	WAIT_RESOURCE_UCSC	state CS Req. <> WAIT_SENT_CSC && state CS Rsp. <> WAIT_SENT_CSS => SetFlag (Buffer_sent_Flag,False), RestoreService(UCS_req), FAL-PDU_req { dmpm_service_name:= "DMPM_Put_req", arep_id:= GetArepId(), dlsdu:= BuildFAL-PDU (fal_pdu_name:= "UCS_ReqPDU", fal_data:= user_data) } }	WAIT_SENT_UCSC

Table 133 – BNB state table – receiver transactions (unconfirmed service sending – client)

#	Current state	Event or condition => action	Next state
R1	WAIT_SENT_UCSC	FAL-PDU_ind && dmpm_service_name = "DMPM_Buffer_Sent_ind" && Flag (Buffer_sent_Flag) = "False" => SetFlag (Buffer_sent_Flag,True), finish UCSC Req.	OPEN_UCSC

4.6.7.3.6 Functions used by BNB ARPM

Table 134 through Table 148 define the functions used by this state machine.

Table 134 – Function GetArepId ()

Name	GetArepId ()	Used in	ARPM
Input		Output	
(none)		AREP Identifier	
Function	Returns a value that can unambiguously identify the current AREP.		

Table 135 – Function dmpm_duplicate_dlsdu ()

Name	dmpm_duplicate_dlsdu ()	Used in	ARPM
Input		Output	
(none)		Boolean	
Function	Returns a value that identifies the duplicated receiving.		

Table 136 – Function BuildFAL-PDU ()

Name	BuildFAL-PDU ()	Used in	ARPM
Input		Output	
fal_pdu_name, calling_dlcep_address, called_dlcep_address, fal_data, fal_id, fal_reason_code, fal_additional_detail		dlsdu	
Function	Builds an FAL-PDU out of the parameters given as input variables.		

Table 137 – Function FAL_Pdu_Type ()

Name	FAL_Pdu_Type ()	Used in	ARPM
Input		Output	
dls_user_data		One of the FAL-PDU types defined in the clause 4.	
Function	This function decodes the FAL-PDU that is conveyed in the dls_user_data parameter and retrieves one of the FAL-PDU types.		

Table 138 – Function AbortIdentifier ()

Name	AbortIdentifier ()	Used in	ARPM
Input		Output	
fal_pdu		The Identifier parameter of the Abort service.	
Function	This function decodes the Abort_PDU that is conveyed in the fal_pdu parameter and extracts the Identifier parameter.		

Table 139 – Function AbortReason ()

Name	AbortReason ()	Used in	ARPM
Input		Output	
fal_pdu		The Reason Code parameter of the Abort service.	
Function	This function decodes the Abort_PDU that is conveyed in the fal_pdu parameter and extracts the Reason Code parameter.		

Table 140 – Function AbortDetail ()

Name	AbortDetail ()	Used in	ARPM
Input		Output	
fal_pdu		The Additional Detail parameter of the Abort service.	
Function	This function decodes the Abort_PDU that is conveyed in the fal_pdu parameter and extracts the Additional Detail parameter.		

NOTE The following two functions make use of persistent protocol timers that are able to issue the local events “MSTimer expired” and “MCTimer expired” to notify the expiration of the appropriate time interval to the ARPM.

Table 141 – Function StartTimer ()

Name	StartTimer ()	Used in	ARPM
Input		Output	
identifier			
Function	This function starts the selected persistent protocol timer as follows: If identifier is MS then function starts the MTimer with the value of CIN. If identifier is MC then function starts the MCTimer with the value of CIN. NOTE The appropriate timer is restarted if it was still running.		

Table 142 – Function StopTimer ()

Name	StopTimer ()	Used in	ARPM
Input		Output	
Function	This function stops all local persistent protocol timers of the ARPM.		

NOTE The following two functions make use of local persistent variables to store/restore all parameters of the service request.

Table 143 – Function StopService ()

Name	StopService ()	Used in	ARPM
Input		Output	
service			
Function	This function stores the service data unit.		

Table 144 – Function RestoreService ()

Name	RestoreService ()	Used in	ARPM
Input		Output	
service			
Function	This function restores all service parameters. That means the calling ARPM has got access to the service data unit.		

NOTE The following two functions make use of local persistent variables Change_Flag and Buffer_sent_Flag that store the result of checks. The initial value of Change_Flag is False and of Buffer_sent_Flag is True.

Table 145 – Function Flag ()

Name	Flag ()	Used in	ARPM
Input		Output	
identifier		Boolean (True/False)	
Function	This function returns the contents of the identified Flag.		

Table 146 – Function SetFlag ()

Name	SetFlag ()	Used in	ARPM
Input		Output	
identifier			
Boolean (True/False)			
Function	This function sets the identified flag corresponding to the input value.		

NOTE The following two functions make use of local persistent variables that store the passed value.

Table 147 – Function StoreIndex ()

Name	StoreIndex ()	Used in	ARPM
Input		Output	
index			
Function	This function stores the current index.		

Table 148 – Function TestIndex ()

Name	TestIndex ()	Used in	ARPM
Input		Output	
index		Boolean (True/False)	
Function	This function returns the result of the index check.		

4.6.8 Buffered Network-scheduled Unidirectional (BNU) ARPM

4.6.8.1 Primitive definitions

4.6.8.1.1 Primitives exchanged between ARPM and FSPM

Table 149 and Table 150 list the primitives exchanged between the FSPM and the ARPM.

Table 149 – Primitives issued by FSPM to ARPM

Primitive name	Source	Associated parameters	Functions
EST_req	FSPM	user_data	This is an FAL internal primitive used to convey an Establish request primitive from the FSPM to the ARPM.
Abort_req	FSPM	identifier, reason_code, additional_detail	This an FAL internal primitive used to convey an Abort request primitive from the FSPM to the ARPM.
UCS_req	FSPM	user_data	This is an FAL internal primitive used to convey an Unconfirmed Send (UCS) request primitive from the FSPM to the ARPM.
FCMP_req	FSPM	(none)	This is an FAL internal primitive used to convey an FAL-Compel (FCMP) request primitive from the FSPM to the ARPM.
GBM_req	FSPM	(none)	This is an FAL internal primitive used to convey a Get-Buffered-Message (GBM) request primitive from the FSPM to the ARPM.

Table 150 – Primitives issued by ARPM to FSPM

Primitive name	Source	Associated parameters	Functions
EST_cnf(+)	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish response(+) primitive from the ARPM to the FSPM.
EST_cnf(-)	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish response(-) primitive from the ARPM to the FSPM.
Abort_ind	ARPM	arep_id, locally_generated, identifier, reason_code, additional_detail	This is an FAL internal primitive used to convey an Abort primitive from the ARPM to the FSPM.
UCS_ind	ARPM	arep_id, duplicate_fal_sdu, user_data, local_timeliness, remote_timeliness	This is an FAL internal primitive used to convey an Unconfirmed Send (UCS) indication primitive from the ARPM to the FSPM.
FCMP_cnf	ARPM	arep_id, status	This is an FAL internal primitive used to convey an FAL-Compel (FCMP) confirmation primitive from the ARPM to the FSPM.
GBM_cnf(+)	ARPM	arep_id, duplicate_fal_sdu, user_data, local_timeliness, remote_timeliness	This is an FAL internal primitive used to convey a Get-Buffered-Message (GBM) positive confirmation primitive from the ARPM to the FSPM.
GBM_cnf(-)	ARPM	arep_id	This is an FAL internal primitive used to convey a Get-Buffered-Message (GBM) negative confirmation primitive from the ARPM to the FSPM.
FSTS_ind	ARPM	arep_id, reported_status	This is an FAL internal primitive used to convey a FAL-Status (FSTS) indication primitive from the ARPM to the FSPM.

4.6.8.1.2 Parameters of FSPM/ARPM primitives

The parameters used with the primitives exchanged between the FSPM and the ARPM are described in Table 151.

Table 151 – Parameters used with primitives exchanged between FSPM and ARPM

Parameter name	Description
arep_id	This parameter is used to unambiguously identify an instance of the AREP that has issued a primitive. A means for such identification is not specified by this standard.
user_data	This parameter conveys FAL-User data.
locally_generated	This parameter conveys value that is used for the Locally_Generated parameter.
identifier	This parameter conveys value that is used for the Identifier parameter.
reason_code	This parameter conveys value that is used for the Reason_Code parameter.
additional_detail	This parameter conveys value that is used for the Additional_Detail parameter.
duplicate_fal_sdu	This parameter conveys value that is used for the Duplicate_FAL-SDU parameter.
status	This parameter conveys value that is used for the Status parameter.
reported_status	This parameter conveys a Data Link Layer event status.
local_timeliness	This parameter conveys value that is used for the Local_Timeliness parameter.
remote_timeliness	This parameter conveys value that is used for the Remote_Timeliness parameter.

4.6.8.2 DLL mapping of BNU AREP class

4.6.8.2.1 BNU AREP class formal model

This subclause describes the mapping of the BNU AREP class to the Fieldbus Data Link Layer. It does not redefine the DLSAP attributes or DLME attributes that are or will be defined in the Data Link Layer standard; rather, it defines how they are used by this AR class.

The DLL mapping attributes and their permitted values and the DLL services used with the BNU AREP class are defined in this subclause.

CLASS:		Bnu
PARENT CLASS:		BufferedNetworkScheduledUnidirectionalAREP
ATTRIBUTES:		
1	(m)	KeyAttribute: PublisherDlcepAddress
2	(m)	Attribute: DlcapRole (Basic)
3	(m)	Attribute: QosParameterSet
3.1	(m)	Attribute: DlcepClass (Publisher, Subscriber)
3.2	(m)	Attribute: DlPriority (Urgent, Normal, TimeAvailable)
3.3	(m)	Attribute: Dl pduAuthentication (Ordinary, Source)
3.4	(m)	Attribute: ResidualActivity
3.4.1	(m)	Attribute: AsSender (False)
3.5	(m)	Attribute: MaxConfirmDelayOnDIConnect
3.6	(m)	Attribute: DISchedulingPolicy (Explicit)
3.7	(m)	Constraint: DlcepClass = Publisher
3.7.1	(m)	Attribute: FromRequestorToResponder (Ordered, Unordered)
3.7.2	(m)	Attribute: MaxDlsduSizeFromRequestor
3.7.3	(o)	Attribute: SenderTimeliness
3.7.3.1	(o)	Attribute: PublisherDITimelinessClass (Residence, Update, Synchronized, None)
3.7.3.2	(o)	Attribute: PublisherTimeWindowSize
3.7.3.3	(o)	Constraint: PublisherDITimelinessClass == Update Synchronized
3.7.3.3.1	(o)	Attribute: PublisherSynchronizingDlcep
3.8	(m)	Constraint: DlcepClass = Subscriber
3.8.1	(m)	Attribute: FromResponderToRequestor (Ordered, Unordered)
3.8.2	(m)	Attribute: MaxDlsduSizeFromResponder
3.8.3	(o)	Attribute: ReceiverTimeliness
3.8.3.1	(o)	Attribute: PublisherDITimelinessClass (Residence, Update, Synchronized, None)
3.8.3.2	(o)	Attribute: PublisherTimeWindowSize
3.8.3.3	(o)	Attribute: SubscriberDITimelinessClass (Residence, Update, Synchronized, None)
3.8.3.4	(o)	Attribute: SubscriberTimeWindowSize
3.8.3.5	(o)	Constraint: SubscriberDITimelinessClass == Update Synchronized
3.8.3.5.1	(o)	Attribute: SubscriberSynchronizingDlcep
3.9	(m)	Attribute: LasScheduled (True, False)
DLL SERVICES:		
1	(m)	OpsService: DL-Put
2	(m)	OpsService: DL-Get
3	(m)	OpsService: DL-Connect
4	(m)	OpsService: DL-Connection-Established
5	(m)	OpsService: DL-Disconnect
6	(m)	OpsService: DL-Buffer-Received
7	(m)	OpsService: DL-Buffer-Sent
8	©	Constraint: LasScheduled == False
8.1	(m)	OpsService: DL-Compel-Service

4.6.8.2.2 Attributes

4.6.8.2.2.1 PublisherDlcepAddress

This attribute specifies the Publisher's DLCEP address and identifies the DLCEP.

The value of this attribute is used as the "DLCEP-address" parameter of the DLL.

This attribute contains the following three subattributes: Link Address, Node Address, and Selector.

NOTE Since the local Link and Node addresses are set by the Network Management, only the Selector portion of the LocalDlcapAddress attribute is a configurable attribute of the FAL.

4.6.8.2.2.2 Role

This attribute specifies the role of this AREP. The value of "PushPublisher" indicates that this AREP is used as a publisher. The value of "PushSubscriber" means that it is used as a subscriber.

4.6.8.2.2.3 DlsapRole

This attribute specifies the behavior of the DLSAP that is used by the AREP.

This attribute supplies the value for the “DL(SAP)-role” parameter. The possible value Basic corresponds to BASIC as defined in the Data Link Layer standard.

4.6.8.2.2.4 DlcepClass

This attribute specifies the behavior of the DLCEP which is attached to the AREP.

This attribute supplies the value for the “DLCEP class” parameter of the DLL. The possible value of this attribute is either Publisher or Subscriber and corresponds to PUBLISHER and SUBSCRIBER defined by the DLL, respectively.

4.6.8.2.2.5 DIIPriority

NOTE It is not possible to use different priorities for each FAL-PDU sent from the same BNU AREP.

4.6.8.2.2.6 DlpduAuthentication

This attribute specifies the lower bound of the length of DL-addressing to be used by the DLL.

This attribute supplies the value for the “DLPDU-authentication” parameter of the DLL. The permitted value Ordinary, and Source correspond to ORDINARY and SOURCE, respectively, as defined in Part-3 and –4 of this International Standard.

4.6.8.2.2.7 ResidualActivity

This attribute specifies receiver’s DLC residual activity. It supplies the value for the “Residual activity as receiver” parameter defined in the DLL. The possible values are “True” and “False.”

4.6.8.2.2.8 MaxConfirmDelayOnDIConnect

This attribute specifies the maximum confirmation delay for a confirmation from a DL-Connect service.

This attribute supplies the value for the “Maximum confirmation delay on DL-Connect, DL-Reset and DL-Subscriber-Query” parameter specified in the Data Link Layer standard.

4.6.8.2.2.9 DISchedulingPolicy

This attribute provides the guidance to the DLL on the scheduling needed by an AR. For this AREP, the DLL tries to transmit the FAL-PDU when it is instructed to do so by a stimulus from the network.

This attribute supplies the value for the “DL-Scheduling-policy” parameter of the DLL. The permitted value Explicit corresponds to EXPLICIT defined in the Data Link Layer standard (IEC 61158-3).

4.6.8.2.2.10 FromRequestorToResponder

This attribute is used if the Role attribute has a value of “PushPublisher” and specifies the DLL data delivery feature of the AREP.

It supplies the value for the “DLCEP data delivery features from requestor to responder(s)” parameter of the DLL. The possible values Ordered and Unordered correspond to ORDERED and UNORDERED defined in the Data Link Layer standard.

If the DuplicatePduDetectionSupported attribute is True, this attribute has the value of “Ordered.”

4.6.8.2.2.11 MaxDlsduSizeFromRequestor

This attribute is used if the Role attribute has a value of “PushPublisher” and specifies the maximum length of an FAL-PDU that can be sent from this AREP.

This attribute supplies the value for the “Maximum DLSDU sizes from requestor” parameter of the DLL.

4.6.8.2.2.12 FromResponderToRequestor

This attribute is used if the Role attribute has a value of “PushSubscriber” and specifies the DLL data delivery feature of the AREP.

It supplies the value for the “DLCEP data delivery features from responder(s) to requestor” parameter of the DLL. The possible values Ordered and Unordered correspond to ORDERED and UNORDERED defined in the Data Link Layer standard.

If the DuplicatePduDetectionSupported attribute is True, this attribute has the value of “Ordered.”

4.6.8.2.2.13 MaxDlsduSizeFromResponder

This attribute is used if the Role attribute has a value of “PushSubscriber” and specifies the maximum length of an FAL-PDU that can be received by this AREP.

This attribute supplies the value for the “Maximum DLSDU size from responder” parameter of the DLL.

4.6.8.2.2.14 PublisherDITimelinessClass

This optional attribute provides the timeliness class of a Publisher provided by the DLL. This attribute supplies the value for the “DL-timeliness-class” parameter of the DLL. The permitted values Residence, Update, Synchronized, and None correspond to RESIDENCE, UPDATE, SYNCHRONIZED, and NONE defined by the DLL.

4.6.8.2.2.15 PublisherTimeWindowSize

This optional attribute provides time window of a Publisher provided by the DLL. This attribute supplies the value for the “Time window size” parameter of the DLL.

4.6.8.2.2.16 PublisherSynchronizingDlcep

This optional attribute is present when the PublisherDITimelinessClass attribute has the value of Update or Synchronized and provides a DLCEP that is used to generate synchronizing events by the DLL. The FAL user may derive the AREP from which the synchronizing events are delivered by this attribute value. This attribute supplies the value for the “synchronizing DLCEP” parameter of the DLL.

4.6.8.2.2.17 SubscriberDITimelinessClass

This optional attribute provides the timeliness class of a Subscriber provided by the DLL. This attribute supplies the value for the “DL-timeliness-class” parameter of the DLL. The permitted values Residence, Update, Synchronized, and None correspond to RESIDENCE, UPDATE, SYNCHRONIZED, and NONE defined by the DLL.

4.6.8.2.2.18 SubscriberTimeWindowSize

This optional attribute provides time window of a Subscriber provided by the DLL. This attribute supplies the value for the “Time window size” parameter of the DLL.

4.6.8.2.2.19 SubscriberSynchronizingDlcep

This optional attribute is present when the SubscriberDITimelinessClass attribute has the value of Update or Synchronized and provides a DLCEP that is used to generate synchronizing events by the DLL. The FAL user may derive the AREP from which the synchronizing events are delivered by this attribute value. This attribute supplies the value for the “synchronizing DLCEP” parameter of the DLL.

4.6.8.2.2.20 LasScheduled

This attribute specifies whether this AREP is LAS-scheduled or not. The value of True means that this AREP is LAS-Scheduled. The value of False means that it is not LAS-Scheduled.

NOTE This attribute does not affect the operation of the FAL and the DLL. It is used by the FAL users only.

4.6.8.2.3 DLL services

Refer to 4.7.4, Data Link Layer Service Selection, for DLL service descriptions.

4.6.8.3 BNU AREP state machine

4.6.8.3.1 BNU ARPM states

The defined states together with their descriptions of the BNU ARPM are listed in Table 152 and Figure 38.

Table 152 – BNU ARPM states

State	Description
CLOSED	The AREP is defined, but not capable of sending or receiving FAL-PDUs.
REQUESTING	The AREP has issued an EST_req and waiting for an EST_cnf primitive.
OPEN	The AREP is defined and capable of sending or receiving FAL-PDUs.

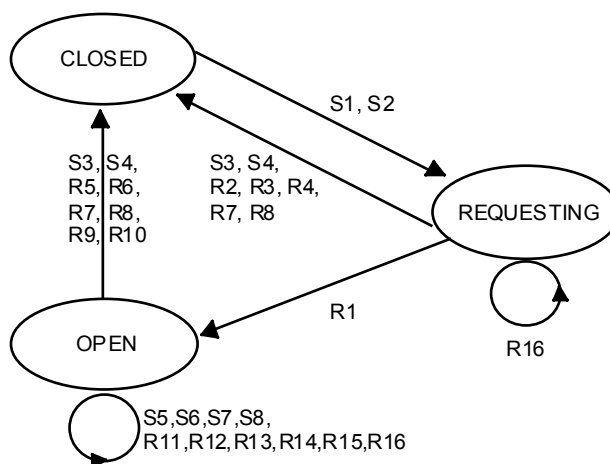


Figure 38 – State transition diagram of the BNU ARPM

4.6.8.3.2 BNU ARPM state table

Table 153 and Table 154 define the BNU ARPM state machine.

Table 153 – BNU ARPM state table – sender transactions

#	Current state	Event or condition => action	Next state
S1	CLOSED	EST_req && Role = "PushPublisher" => FAL-PDU_req { dmpm_service_name := "DMPM_Connect_req", arep_id := GetArepld (), called_address := "null", calling_address := "default dlsap address", local_dlcep_address := PublisherDlcepAddress, dlsdu := "null" }	REQ
S2	CLOSED	EST_req && Role = "PushSubscriber" => FAL-PDU_req { dmpm_service_name := "DMPM_Connect_req", arep_id := GetArepld (), called_address := PublisherDlcepAddress, calling_address := "default dlsap address", local_dlcep_address := "default subscriber dlcep address", dlsdu := "null" }	REQ
S3	NOT CLOSED	Abort_req && Role = "PushPublisher" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "disconnection-normal condition", dlsdu := BuildFAL-PDU (fal_pdu_name := "Abort_PDU", fal_id := identifier, fal_reason_code := reason_code, fal_additional_detail := additional_detail) }	CLOSED
S4	NOT CLOSED	Abort_req && Role = "PushSubscriber" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "disconnection-normal condition", dlsdu := "null" }	CLOSED
S5	OPEN	UCS_req && Role = "PushPublisher" => FAL-PDU_req { dmpm_service_name := "DMPM_Put_req", arep_id := GetArepld (), dlsdu := BuildFAL-PDU (fal_pdu_name := "UCS_PDU", fal_data := user_data) }	OPEN
S6	OPEN	FCMP_req && Role = "PushPublisher" => FAL-PDU_req { dmpm_service_name := "DMPM_Compel_Service_req", arep_id := GetArepld (), action_class := "Local" }	OPEN

#	Current state	Event or condition => action	Next state
S7	OPEN	FCMP_req && Role = "PushSubscriber" => FAL-PDU_req { dmpm_service_name := "DMPM_Compel_Service_req", arep_id := GetArepld (), action_class := "Remote" }	OPEN
S8	OPEN	GBM_req && Role = "PushSubscriber" => FAL-PDU_req { dmpm_service_name := "DMPM_Get_req", arep_id := GetArepld () }	OPEN

Table 154 – BNU ARPM state table – receiver transactions

#	Current state	Event or condition => action	Next state
R1	REQ	Connect_cnf => EST_cnf(+) { arep_id := GetArepld (), user_data := "null" }	OPEN
R2	REQ	FAL-PDU_ind && Role = "PushSubscriber" && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu = "null" && originator = "local_dls_provider" => EST_cnf(-) { arep_id := GetArepld (), user_data := "null" }	CLOSED
R3	REQ	FAL-PDU_ind && Role = "PushSubscriber" && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu = "null" && originator = "remote_dls_provider" => EST_cnf(-) { arep_id := GetArepld (), user_data := "null" }	CLOSED
R4	REQ	FAL-PDU_ind && Role = "PushSubscriber" && dmpm_service_name <> "DMPM_Disconnect_ind" => Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid DI Event", additional_detail := "null" }	CLOSED
R5	OPEN	FAL-PDU_ind && Role = "PushSubscriber" && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu = "null" && originator = "local_dls_provider" => Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "Data Link Layer", reason_code := reason, additional_detail := "null" }	CLOSED

Table 154 (continued)

#	Current state	Event or condition => action	Next state
R6	OPEN	<pre> FAL-PDU_ind && Role = "PushSubscriber" && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu = "null" && originator = "remote_dls_provider" => Abort_ind { arep_id := GetArepld (), locally_generated := "False", identifier := "Data Link Layer", reason_code := reason, additional_detail := "null" } </pre>	CLOSED
R7	NOT CLOSED	<pre> FAL-PDU_ind && Role = "PushSubscriber" && dmpm_service_name = "DMPM_Disconnect_ind" && Fal_Pdu_Type (fal_pdu) = "Abort_PDU" => Abort_ind { arep_id := GetArepld (), locally_generated := "False", identifier := AbortIdentifier (fal_pdu), reason_code := AbortReason (fal_pdu), additional_detail := AbortDetail (fal_pdu) } </pre>	CLOSED
R8	NOT CLOSED	<pre> FAL-PDU_ind && Role = "PushSubscriber" && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu <> "null" && Fal_Pdu_Type (fal_pdu) <> "Abort_PDU" => Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU", additional_detail := "null" } </pre>	CLOSED
R9	OPEN	<pre> FAL-PDU_ind && Role = "PushSubscriber" && ((dmpm_service_name <> "DMPM_Buffer_Received_ind") && (dmpm_service_name <> "DMPM_Disconnect_ind") && (dmpm_service_name <> "DMPM_Compel_Service_cnf") && (dmpm_service_name <> "DMPM_Get_cnf") && (dmpm_service_name <> "DMPM_Buffer_Sent_ind")) => Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid DI Event", additional_detail := "null" }, FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid DL-Event", dlsdu := "null" } </pre>	CLOSED

Table 154 (continued)

#	Current state	Event or condition => action	Next state
R10	OPEN	FAL-PDU_ind && Role = "PushSubscriber" && dmpm_service_name = "DMPM_Buffer_Received_ind" && FAL_pdu_type <> "UCS_PDU" => Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU", additional_detail := "null" }, FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }	CLOSED
R11	OPEN	FAL-PDU_ind && Role = "PushSubscriber" && dmpm_service_name = "DMPM_Buffer_Received_ind" && FAL_Pdu_Type (fal_pdu) = "UCS_PDU" => UCS_ind { arep_id := GetArepld (), duplicate_fal_sdu := duplicate_dlsdu, user_data := fal_pdu, local_timeliness := local_dle_timeliness, remote_timeliness := remote_dle_timeliness }	OPEN
R12	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Compel_Service_cnf" => FCMP_cnf { status := reason }	OPEN
R13	OPEN	FAL-PDU_ind && Role = "PushSubscriber" && dmpm_service_name = "DMPM_Get_cnf" && reason = "success" => GBM_cnf(+) { arep_id := GetArepld (), duplicate_fal_sdu := duplicate_dlsdu, user_data := fal_pdu, local_timeliness := local_dle_timeliness, remote_timeliness := remote_dle_timeliness }	OPEN
R14	OPEN	FAL-PDU_ind && Role = "PushSubscriber" && dmpm_service_name = "DMPM_Get_cnf" && reason <> "success" => GBM_cnf(-) { arep_id := GetArepld () }	OPEN
R15	OPEN	FAL-PDU_ind && Role = "PushPublisher" && dmpm_service_name = "DMPM_Buffer_Sent_ind" => FSTS_ind { arep_id := GetArepld (), reported_status := "Buffer-Sent" }	OPEN

Table 154 (continued)

#	Current state	Event or condition => action	Next state
R16	NOT CLOSED	ErrorToARPM => (no actions taken) NOTE It is a local matter to report this error status to network management entities. The ARPM does not abort the existing connections. The FAL user may issue an Abort request to disconnect the current connection, depending on the type of status information conveyed by the ErrorToARPM primitive.	SAME

4.6.8.3.3 Functions used by BNU ARPM

Table 155 through Table 160 define the functions used by this state machine.

Table 155 – Function GetArepld ()

Name	GetArepld ()	Used in	ARPM
Input		Output	
(none)		AREP Identifier	
Function	Returns a value that can unambiguously identify the current AREP.		

Table 156 – Function BuildFAL-PDU

Name	BuildFAL-PDU	Used in	ARPM
Input		Output	
fal_pdu_name, calling_dlcep_address, called_dlcep_address, fal_data, fal_id, fal_reason_code, fal_additional_detail		dlidu	
Function	Builds an FAL-PDU out of the parameters given as input variables.		

Table 157 – Function FAL_Pdu_Type

Name	FAL_Pdu_Type	Used in	ARPM
Input		Output	
fal_pdu		One of the FAL-PDU types defined in the clause 4.	
Function	This function decodes the FAL-PDU that is conveyed in the fal_pdu parameter and retrieves one of the FAL-PDU types.		

Table 158 – Function AbortIdentifier

Name	AbortIdentifier	Used in	ARPM
Input		Output	
fal_pdu		The Identifier parameter of the Abort service.	
Function	This function decodes the Abort_PDU that is conveyed in the fal_pdu parameter and extracts the Identifier parameter.		

Table 159 – Function AbortReason

Name	AbortReason	Used in	ARPM
Input		Output	
fal_pdu		The Reason Code parameter of the Abort service.	
Function	This function decodes the Abort_PDU that is conveyed in the fal_pdu parameter and extracts the Reason Code parameter.		

Table 160 – Function AbortDetail

Name	AbortDetail	Used in	ARPM
Input		Output	
fal_pdu		The Additional Detail parameter of the Abort service.	
Function			
This function decodes the Abort_PDU that is conveyed in the fal_pdu parameter and extracts the Additional Detail parameter.			

4.6.9 Buffered Network-scheduled Unidirectional-MP (BNU-MP) ARPM

4.6.9.1 Primitive definitions

4.6.9.1.1 Primitives exchanged between ARPM and FSPM

Table 161 and Table 162 list the primitives exchanged between the FSPM and the ARPM.

Table 161 – Primitives issued by FSPM to ARPM

Primitive name	Source	Associated parameters	Functions
EST_req	FSPM	user_data	This is an FAL internal primitive used to convey an Establish request primitive from the FSPM to the ARPM.
Abort_req	FSPM	identifier, reason_code, additional_detail	This an FAL internal primitive used to convey an Abort request primitive from the FSPM to the ARPM.
RR . req	FSPM	priority	This is an FAL internal primitive used to convey a Remote Read request primitive from the FSPM to the ARPM
RW . req	FSPM	priority, decoded_buffer_data	This is an FAL internal primitive used to convey a Remote Write request primitive from the FSPM to the ARPM
UCS_req	FSPM	remote_dlsap_address, user_data	This is an FAL internal primitive used to convey an Unconfirmed Send (UCS) request primitive from the FSPM to the ARPM.
FCMP_req	FSPM	(none)	This is an FAL internal primitive used to convey an FAL-Compel (FCMP) request primitive from the FSPM to the ARPM.
GBM_req	FSPM	(none)	This is an FAL internal primitive used to convey a Get-Buffered-Message (GBM) request primitive from the FSPM to the ARPM.

Table 162 – Primitives issued by ARPM to FSPM

Primitive name	Source	Associated parameters	Functions
EST_cnf(+)	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish response(+) primitive from the ARPM to the FSPM.
EST_cnf(-)	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish response(-) primitive from the ARPM to the FSPM.
Abort_ind	ARPM	arep_id, locally_generated, identifier, reason_code, additional_detail	This is an FAL internal primitive used to convey an Abort primitive from the ARPM to the FSPM.
UCS_ind	ARPM	arep_id, remote_dlsap_address, duplicate_fal_sdu, user_data, local_timeliness, remote_timeliness	This is an FAL internal primitive used to convey an Unconfirmed Send (UCS) indication primitive from the ARPM to the FSPM.
RR . cnf(+)	ARPM	arep_id, decoded_data, local_timeliness, remote_timeliness	This is an FAL internal primitive used to convey a Remote Read response(+) primitive from the ARPM to the FSPM
RR . cnf(-)	ARPM	arep_id	This is an FAL internal primitive used to convey a Remote Read response(-) primitive from the ARPM to the FSPM
RW . cnf(+)	ARPM	arep_id	This is an FAL internal primitive used to convey a Remote Write response(+) primitive from the ARPM to the FSPM
RW . cnf(-)	ARPM	arep_id	This is an FAL internal primitive used to convey a Remote Write response primitive from the ARPM to the FSPM
FCMP_cnf	ARPM	arep_id, status	This is an FAL internal primitive used to convey an FAL-Compel (FCMP) confirmation primitive from the ARPM to the FSPM.
GBM_cnf(+)	ARPM	arep_id, duplicate_fal_sdu, user_data, local_timeliness, remote_timeliness	This is an FAL internal primitive used to convey a Get-Buffered-Message (GBM) positive confirmation primitive from the ARPM to the FSPM.
GBM_cnf(-)	ARPM	arep_id	This is an FAL internal primitive used to convey a Get-Buffered-Message (GBM) negative confirmation primitive from the ARPM to the FSPM.
FSTS_ind	ARPM	arep_id, reported_status	This is an FAL internal primitive used to convey an FAL-Status (FSTS) indication primitive from the ARPM to the FSPM.

4.6.9.1.2 Parameters of FSPM/ARPM primitives

The parameters used with the primitives exchanged between the FSPM and the ARPM are described in Table 163.

Table 163 – Parameters used with primitives exchanged between FSPM and ARPM

Parameter name	Description
arep_id	This parameter is used to unambiguously identify an instance of the AREP that has issued a primitive. A means for such identification is not specified by this standard.
user_data	This parameter conveys FAL-User data.
locally_generated	This parameter conveys value that is used for the Locally_Generated parameter.
identifier	This parameter conveys value that is used for the Identifier parameter.
reason_code	This parameter conveys value that is used for the Reason_Code parameter.
additional_detail	This parameter conveys value that is used for the Additional_Detail parameter.
duplicate_fal_sdu	This parameter conveys value that is used for the Duplicate_FAL-SDU parameter.
status	This parameter conveys value that is used for the Status parameter.
reported_status	This parameter conveys a Data Link Layer event status.
local_timeliness	This parameter conveys value that is used for the Local_Timeliness parameter.
remote_timeliness	This parameter conveys value that is used for the Remote_Timeliness parameter.

4.6.9.2 DLL mapping of BNU-MP AREP class

4.6.9.2.1 BNU-MP AREP class formal model

This subclause describes the mapping of the BNU-MP AREP class to the Fieldbus Data Link Layer. It does not redefine the DLSAP attributes or DLME attributes that are or will be defined in the Data Link Layer standard; rather, it defines how they are used by this AR class.

The DLL Mapping attributes and their permitted values and the DLL services used with the BNU-MP AREP class are defined in this subclause.

CLASS:		BnuMp
PARENT CLASS:		BufferedNetworkScheduledUnidirectionalAREP
ATTRIBUTES:		
1	(m) KeyAttribute:	PublisherDlcepAddress
2	(m) Attribute:	DisapRole (Basic)
3	(m) Attribute:	QosParameterSet
3.1	(m) Attribute:	DlcepClass (Push Publish, Push Subscriber)
3.2	(m) Attribute:	DllPriority (Urgent, Normal, TimeAvailable)
3.3	(m) Attribute:	DlpduAuthentication (Ordinary, Source)
3.4	(m) Attribute:	ResidualActivity
3.4.1	(m) Attribute:	AsSender (False)
3.5	(m) Attribute:	MaxConfirmDelayOnDIConnect
3.6	(m) Attribute:	DISchedulingPolicy (Explicit)
3.7	(m) Constraint:	DlcepClass = Publisher
3.7.1	(m) Attribute:	FromRequestorToResponder (Ordered, Unordered)
3.7.2	(m) Attribute:	MaxDlsduSizeFromRequestor
3.7.3	(o) Attribute:	SenderTimeliness
3.7.3.1	(o) Attribute:	PublisherDITimelinessClass (Residence, Update, Synchronized, None)
3.7.3.2	(o) Attribute:	PublisherTimeWindowSize
3.7.3.3	(o) Constraint:	PublisherDITimelinessClass == Update Synchronized
3.7.3.3.1	(o) Attribute:	PublisherSynchronizingDlcep
3.8	(m) Constraint:	DlcepClass = Subscriber
3.8.1	(m) Attribute:	FromResponderToRequestor (Ordered, Unordered)
3.8.2	(m) Attribute:	MaxDlsduSizeFromResponder
3.8.3	(o) Attribute:	ReceiverTimeliness
3.8.3.1	(o) Attribute:	PublisherDITimelinessClass (Residence, Update, Synchronized, None)
3.8.3.2	(o) Attribute:	PublisherTimeWindowSize
3.8.3.3	(o) Attribute:	SubscriberDITimelinessClass (Residence, Update, Synchronized, None)
3.8.3.4	(o) Attribute:	SubscriberTimeWindowSize
3.8.3.5	(o) Constraint:	SubscriberDITimelinessClass == Update Synchronized
3.8.3.5.1	(o) Attribute:	SubscriberSynchronizingDlcep
3.9	(m) Attribute:	LasScheduled (True, False)
DLL SERVICES:		
1	(m) OpService:	DL-Put
2	(m) OpService:	DL-Get
3	(m) OpService:	DL-Connect
4	(m) OpService:	DL-Connection-Established
5	(m) OpService:	DL-Disconnect
6	(m) OpService:	DL-Buffer-Received
7	(m) OpService:	DL-Buffer-Sent
8	(m) Constraint:	LasScheduled == False
8.1	(m) OpService:	DL-Compel-Service

4.6.9.2.2 Attributes

4.6.9.2.2.1 PublisherDlcepAddress

This attribute specifies the Publisher's DLCEP address and identifies the DLCEP. The value of this attribute is used as the "DLCEP-address" parameter of the DLL. This attribute contains a flat addressing.

4.6.9.2.2.2 DisapRole

This attribute specifies the behavior of the DLSAP that is used by the AREP. This attribute supplies the value for the “DL(SAP)-role” parameter.

4.6.9.2.2.3 QosParameterSet

The QosParameterSet attributes specify the DL quality of service that is used by the AREP. These attribute values may be negotiated with the remote AREP.

4.6.9.2.2.4 DIIPriority

NOTE It is not possible to use different priorities for each FAL-PDU sent from the same BNU-MP AREP.

4.6.9.2.2.5 DIpduAuthentication

This attribute specifies the lower bound of the length of DL-addressing to be used by the DLL.

This attribute supplies the value for the “DLPDU-authentication” parameter of the DLL. The permitted value Ordinary, Source and Maximal correspond to ORDINARY, SOURCE and MAXIMAL respectively, as defined in Part-3 and –4 of this International Standard.

4.6.9.2.2.6 ResidualActivity

This attribute specifies receiver’s DLC residual activity. It supplies the value for the “Residual activity as receiver” parameter defined in the DLL. The possible values are “True” and “False”.

4.6.9.2.2.7 MaxConfirmDelayOnDIConnect

This attribute specifies the maximum confirmation delay for a confirmation from a DL-Connect service.

This attribute supplies the value for the “Maximum confirmation delay on DL-Connect, DL-Reset and DL-Subscriber-Query” parameter specified in the Data Link Layer standard.

4.6.9.2.2.8 DISchedulingPolicy

This attribute provides the guidance to the DLL on the scheduling needed by an AR. For this AREP, the DLL tries to transmit the FAL-PDU when it is instructed to do so by a stimulus from the network.

This attribute supplies the value for the “DL-Scheduling-policy” parameter of the DLL. The permitted value Explicit corresponds to EXPLICIT defined in the Data Link Layer standard (IEC 61158-3).

This attribute provides the guidance to the DLL on the scheduling needed by an AR. For this AREP, the DLL tries to transmit the FAL-PDU when it is instructed to do so by a stimulus from the network.

This attribute supplies the value for the “DL-Scheduling-policy” parameter of the DLL. The permitted value Explicit corresponds to EXPLICIT defined in the Data Link Layer standard (IEC 61158-3).

4.6.9.2.2.9 FromRequestorToResponder

This attribute is used if the Role attribute has a value of “Push Publish” and specifies the DLL data delivery feature of the AREP.

It supplies the value for the “DLCEP data delivery features from requestor to responder(s)” parameter of the DLL. The possible values Ordered and Unordered correspond to ORDERED and UNORDERED defined in the Data Link Layer standard.

If the DuplicatePduDetectionSupported attribute is True, this attribute has the value of “Ordered.”

4.6.9.2.2.10 MaxDisduSizeFromRequestor

This attribute is used if the Role attribute has a value of “Push Publish” and specifies the maximum length of an FAL-PDU that can be sent from this AREP.

This attribute supplies the value for the “Maximum DLSDU sizes from requestor” parameter of the DLL.

4.6.9.2.2.11 FromResponderToRequestor

This attribute is used if the Role attribute has a value of “Push Subscriber” and specifies the DLL data delivery feature of the AREP.

It supplies the value for the “DLCEP data delivery features from responder(s) to requestor” parameter of the DLL. The possible values Ordered and Unordered correspond to ORDERED and UNORDERED defined in the Data Link Layer standard.

If the DuplicatePduDetectionSupported attribute is True, this attribute has the value of “Ordered.”

4.6.9.2.2.12 MaxDisduSizeFromResponder

This attribute is used if the Role attribute has a value of “Push Subscriber” and specifies the maximum length of an FAL-PDU that can be received by this AREP.

This attribute supplies the value for the “Maximum DLSDU size from responder” parameter of the DLL.

4.6.9.2.2.13 SenderTimeliness

This optional attribute is derived from that of the DLL.

4.6.9.2.2.14 PublisherDITimelinessClass

This optional attribute provides the timeliness class of a Publisher provided by the DLL. This attribute supplies the value for the “DL-timeliness-class” parameter of the DLL. The permitted values Residence, Update, Synchronized, and None correspond to RESIDENCE, UPDATE, SYNCHRONIZED, and NONE defined by the DLL.

4.6.9.2.2.15 PublisherTimeWindowSize

This optional attribute provides time window of a Publisher provided by the DLL. This attribute supplies the value for the “Time window size” parameter of the DLL.

4.6.9.2.2.16 PublisherSynchronizingDlcep

This optional attribute is present when the PublisherDITimelinessClass attribute has the value of Update or Synchronized and provides a DLCEP that is used to generate synchronizing events by the DLL. The FAL user may derive the AREP from which the synchronizing events are delivered by this attribute value. This attribute supplies the value for the “synchronizing DLCEP” parameter of the DLL.

4.6.9.2.2.17 SubscriberDITimelinessClass

This optional attribute provides the timeliness class of a Subscriber provided by the DLL. This attribute supplies the value for the “DL-timeliness-class” parameter of the DLL. The permitted values Residence, Update, Synchronized, and None correspond to RESIDENCE, UPDATE, SYNCHRONIZED, and NONE defined by the DLL.

4.6.9.2.2.18 SubscriberTimeWindowSize

This optional attribute provides a time window of a Subscriber provided by the DLL. This attribute supplies the value for the “Time window size” parameter of the DLL.

4.6.9.2.2.19 SubscriberSynchronizingDlcep

This optional attribute is present when the SubscriberDITimelinessClass attribute has the value of Update or Synchronized and provides a DLCEP that is used to generate synchronizing events by the DLL. The FAL user may derive the AREP from which the synchronizing events are delivered by this attribute value. This attribute supplies the value for the “synchronizing DLCEP” parameter of the DLL.

4.6.9.2.2.20 LasScheduled

This attribute specifies whether this AREP is LAS-scheduled or not. The value of True means that this AREP is LAS-Scheduled. The value of False means that it is not LAS-Scheduled.

NOTE This attribute does not affect the operation of the FAL and the DLL. It is used by the FAL users only.

4.6.9.2.3 DLL services

Refer to 4.7.4, Data Link Layer Service Selection, for DLL service descriptions.

4.6.9.3 BNU-MP ARPM state machine

4.6.9.3.1 BNU-MP ARPM states

The defined states together with their descriptions of the BNU-MP ARPM are listed in Table 164 and Figure 39.

Table 164 – BNU-MP ARPM states

State	Description
CLOSED	The AREP is defined, but not capable of sending or receiving FAL-PDUs.
REQUESTING	The AREP has issued an EST_req and waiting for an EST_cnf primitive.
ASKING	Transient state before the completion of RR.rq or RW.rq primitive.
GETTING	Transient state before the completion of RR.rq or RW.rq primitive.
OPEN	The AREP is defined and capable of sending or receiving FAL-PDUs.

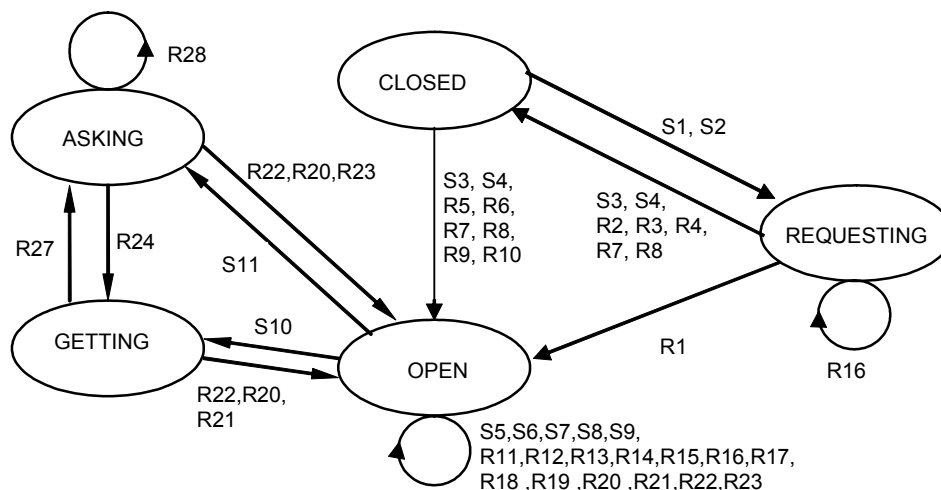


Figure 39 – State transition diagram of the BNU-MP ARPM

4.6.9.3.2 BNU-MP ARPM state table

Table 165 and Table 166 define the BNU-MP state machine.

Table 165 – BNU-MP ARPM state table – sender transactions

#	Current state	Event or condition => action	Next state
S1	CLOSED	EST_req && Role = "Push Publish" => FAL-PDU_req { dmpm_service_name := "DMPM_Connect_req", arep_id := GetArepld (), called_address := "null", calling_address := "default dlsap address", local_dlcep_address := Push DlcepAddress, dlsdu := "null" }	REQ
S2	CLOSED	EST_req && Role = "Push Subscriber" => FAL-PDU_req { dmpm_service_name := "DMPM_Connect_req", arep_id := GetArepld (), called_address := PublisherDlcepAddress, calling_address := "default dlsap address", local_dlcep_address := "default subscriber dlcep address", dlsdu := "null" }	REQ
S3	NOT CLOSED	Abort_req && Role = "Push Publish" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "disconnection-normal condition", dlsdu := BuildFAL-PDU (fal_pdu_name := "Abort_PDU", fal_id := identifier, fal_reason_code := reason_code, fal_additional_detail := additional_detail) }	CLOSED

#	Current state	Event or condition => action	Next state
S4	NOT CLOSED	Abort_req && Role = "Push Subscriber" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "disconnection-normal condition", dlsdu := "null" }	CLOSED
S5	OPEN	UCS_req && Role = "Push Publish" && dedicatedArep() = "False" => FAL-PDU_req { dmpm_service_name := "DMPM_Put_req", arep_id := GetArepld (), dlsdu := BuildFAL-PDU (fal_pdu_name := "UCS_PDU", fal_sdu := user_data) }	OPEN
S6	OPEN	FCMP_req && Role = "Push Publish" => FAL-PDU_req { dmpm_service_name := "DMPM_Compel_Service_req", arep_id := GetArepld (), action_class := "Local" }	OPEN
S7	OPEN	FCMP_req && Role = "Push Subscriber" => FAL-PDU_req { dmpm_service_name := "DMPM_Compel_Service_req", arep_id := GetArepld (), action_class := "Remote" }	OPEN
S8	OPEN	GBM_req && Role = "Push Subscriber" => FAL-PDU_req { dmpm_service_name := "DMPM_Get_req", arep_id := GetArepld () }	OPEN
S9	OPEN	UCS_req && Role = "Push Publish" && dedicatedArep() = "True" => FAL-PDU_req { dmpm_service_name := "DMPM_Put_req", arep_id := GetArepld (), dlsdu := BuildFAL-PDU (fal_pdu_name := "BufferDataTransfer_PDU", fal_sdu := user_data) }	OPEN
S10	OPEN	RW_req && Role = "Push Publish" && dedicatedArep() = "True" => FAL-PDU_req { dmpm_service_name := "DMPM_Put_req", arep_id := GetArepld (), action_class := "local" dlsdu := BuildFAL-PDU (fal_pdu_name := "BufferDataTransfer_PDU", fal_sdu := user_data) }	GETTING

#	Current state	Event or condition => action	Next state
S11	OPEN	RR_req && Role = "Push Subscriber" && dedicatedArep() = "True" action_class := "Remote" => FAL-PDU_req { dmpm_service_name := "DMPM_Compel_req", arep_id := GetArepId (), }	ASKING

Table 166 – BNU-MP ARPM state table – receiver transactions

#	Current state	Event or condition => action	Next state
R1	REQ	Connect_cnf => EST_cnf(+) { arep_id := GetArepId (), user_data := "null" }	OPEN
R2	REQ	FAL-PDU_ind && Role = "Push Subscriber" && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu = "null" && originator = "local_dls_provider" => EST_cnf(-) { arep_id := GetArepId (), user_data := "null" }	CLOSED
R3	REQ	FAL-PDU_ind && Role = "Push Subscriber" && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu = "null" && originator = "remote_dls_provider" => EST_cnf(-) { arep_id := GetArepId (), user_data := "null" }	CLOSED
R4	REQ	FAL-PDU_ind && Role = "Push Subscriber" && dmpm_service_name <> "DMPM_Disconnect_ind" => Abort_ind { arep_id := GetArepId (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid DI Event", additional_detail := "null" }	CLOSED
R5	OPEN	FAL-PDU_ind && Role = "Push Subscriber" && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu = "null" && originator = "local_dls_provider" => Abort_ind { arep_id := GetArepId (), locally_generated := "True", identifier := "Data Link Layer", reason_code := reason, additional_detail := "null" }	CLOSED

Table 166 (continued)

#	Current state	Event or condition => action	Next state
R6	OPEN	<pre> FAL-PDU_ind && Role = "Push Subscriber" && dmpm_service_name = "DMPM_Disconnect_ind" && dlsdu = "null" && originator = "remote_dls_provider" => Abort_ind { arep_id := GetArepld (), locally_generated := "False", identifier := "Data Link Layer," reason_code := reason, additional_detail := "null" } </pre>	CLOSED
R7	NOT CLOSED	<pre> FAL-PDU_ind && Role = "Push Subscriber" && dmpm_service_name = "DMPM_Disconnect_ind" && Fal_Pdu_Type (fal_pdu) = "Abort_PDU" => Abort_ind { arep_id := GetArepld (), locally_generated := "False", identifier := AbortIdentifier (fal_pdu), reason_code := AbortReason (fal_pdu), additional_detail := AbortDetail (fal_pdu) } </pre>	CLOSED
R8	NOT CLOSED	<pre> FAL-PDU_ind && Role = "Push Subscriber" && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu <> "null" && Fal_Pdu_Type (fal_pdu) <> "Abort_PDU" => Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU", additional_detail := "null" } </pre>	CLOSED
R9	OPEN	<pre> FAL-PDU_ind && Role = "Push Subscriber" && ((dmpm_service_name <> "DMPM_Buffer_Received_ind") && (dmpm_service_name <> "DMPM_Disconnect_ind") && (dmpm_service_name <> "DMPM_Compel_Service_cnf") && (dmpm_service_name <> "DMPM_Get_cnf") && (dmpm_service_name <> "DMPM_Buffer_Sent_ind")) => Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid DI Event", additional_detail := "null" }, FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid DL-Event", dlsdu := "null" } </pre>	CLOSED

Table 166 (continued)

#	Current state	Event or condition => action	Next state
R10	OPEN	FAL-PDU_ind && Role = "Push Subscriber" && dmpm_service_name = "DMPM_Buffer_Received_ind" && FAL_pdu_type <> "UCS_PDU" && FAL_pdu_type <> "BufferDataTransfer_PDU", => Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU", additional_detail := "null" }, FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }	CLOSED
R11	OPEN	FAL-PDU_ind && Role = "Push Subscriber" && dmpm_service_name = "DMPM_Buffer_Received_ind" && FAL_Pdu_Type (fal_pdu) = "UCS_PDU BufferDataTransfer_PDU" => UCS_ind { arep_id := GetArepld (), duplicate_fal_sdu := duplicate_dlsdu, user_data := fal_pdu, local_timeliness := local_dle_timeliness, remote_timeliness := remote_dle_timeliness }	OPEN
R12	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Compel_Service_cnf" => FCPM_cnf { status := dl_status }	OPEN
R13	OPEN	FAL-PDU_ind && Role = "Push Subscriber" && dmpm_service_name = "DMPM_Get_cnf" && status = "success" => GBM_cnf(+) { arep_id := GetArepld (), duplicate_fal_sdu := duplicate_dlsdu, user_data := fal_pdu, local_timeliness := local_dle_timeliness, remote_timeliness := remote_dle_timeliness }	OPEN
R14	OPEN	FAL-PDU_ind && Role = "Push Subscriber" && dmpm_service_name = "DMPM_Get_cnf" && status <> "success" => GBM_cnf(-) { arep_id := GetArepld () }	OPEN
R15	OPEN	FAL-PDU_ind && Role = "Push Publish" && dmpm_service_name = "DMPM_Buffer_Sent_ind" => FSTS_ind { arep_id := GetArepld (), reported_status := "Buffer-Sent" }	OPEN

Table 166 (continued)

#	Current state	Event or condition => action	Next state
R18	OPEN	FAL-PDU_ind && Role = "Push Publish" && dmpm_service_name = "DMPM_Put.cnf" && dl_status = "success" => FSTS_ind { arep_id := GetArepld (), reported_status := "Put Ok" }	OPEN
R19	OPEN	FAL-PDU_ind && Role = "Push Publish" && dmpm_service_name = "DMPM_Put.cnf" && dl_status<> "success" => FSTS_ind { arep_id := GetArepld (), reported_status := dl_status }	OPEN
R20	GETTING , ASKING	FAL-PDU_ind && Role = "Push Subscriber" && (dmpm_service_name = "DMPM_Getcnf" dmpm_service_name = "DMPM_Compel_Service_cnf") && dl_status<> "success" => RR cnf(-) { arep_id := GetArepld (), reported_status := dl_status }	OPEN
R21	GETTING	FAL-PDU_ind && Role = "Push Subscriber" && dmpm_service_name = "DMPM_Buffer_Received_ind" && dl_status= "success" => RR cnf(+) { arep_id := GetArepld (), user_data := fal_pdu, local_timeliness := local_dle_timeliness, remote_timeliness := remote_dle_timeliness }	OPEN
R22	GETTING , ASKING	FAL-PDU_ind && Role = "Push Publish" && (dmpm_service_name = "DMPM_Putcnf" dmpm_service_name = "DMPM_Compel_Service_cnf") && dl_status<> "success" => RW cnf(-) { arep_id := GetArepld (), reported_status := dl_status }	OPEN
R23	ASKING	FAL-PDU_ind && Role = "Push Publish" && dmpm_service_name = "DMPM_Buffer_Sent_ind" && dl_status = "success" => RW cnf(+) { arep_id := GetArepld (), reported_status := dl_status }	OPEN
R24	ASKING	FAL-PDU_ind && Role = "Push Subscriber" && dmpm_service_name = "DMPM_Compel_Service_cnf" && dl_status = "success" => (no action)	GETTING

Table 166 (continued)

#	Current state	Event or condition => action	Next state
R27	GETTING	FAL-PDU_ind && Role = "Push Publish" && dmpm_service_name = "DMPM_Put_cnf" && dl_status = "success" => FAL_PDU_req{ dmpm-service-name := "DMPM_compel_service_req" arep_id := GetArepId() action_class := "local" } }	ASKING
R28	ASKING	FAL_PDU_Ind && Role = "Push Publish" && dmpm-service-name = "DMPM_Compel_Service_Cnf" && dl_Status = "success" => (no action)	Asking

4.6.9.3.3 Functions used by BNU-MP ARPM

Table 167 through Table 173 define the functions used by this state machine.

Table 167 – Function GetArepId ()

Name	GetArepId ()	Used in	ARPM
Input		Output	
(none)		AREP Identifier	
Function	Returns a value that can unambiguously identify the current AREP.		

Table 168 – Function BuildFAL-PDU

Name	BuildFAL-PDU	Used in	ARPM
Input		Output	
fal_pdu_name, calling_dlcep_address, called_dlcep_address, fal_data, fal_id, fal_reason_code, fal_additional_detail		dlsdu	
Function	Builds an FAL-PDU out of the parameters given as input variables.		

Table 169 – Function FAL_Pdu_Type

Name	FAL_Pdu_Type	Used in	ARPM
Input		Output	
dls_user_data		One of the FAL-PDU types defined in the clause 4.	
Function	This function decodes the FAL-PDU that is conveyed in the dls_user_data parameter and retrieves one of the FAL-PDU types.		

Table 170 – Function AbortIdentifier

Name	AbortIdentifier	Used in	ARPM
Input		Output	
fal_pdu		The Identifier parameter of the Abort service.	
Function	This function decodes the Abort_PDU that is conveyed in the fal_pdu parameter and extracts the Identifier parameter.		

Table 171 – Function AbortReason

Name	AbortReason	Used in	ARPM
Input		Output	
fal_pdu		The Reason Code parameter of the Abort service.	
Function	This function decodes the Abort_PDU that is conveyed in the fal_pdu parameter and extracts the Reason Code parameter.		

Table 172 – Function AbortDetail

Name	AbortDetail	Used in	ARPM
Input		Output	
fal_pdu		The Additional Detail parameter of the Abort service.	
Function	This function decodes the Abort_PDU that is conveyed in the fal_pdu parameter and extracts the Additional Detail parameter.		

Table 173 – Function dedicatedArep ()

Name	dedicatedArep()	Used in	DMPM
Input		Output	
(none)		True False	
Function	Checks if the AREP is dedicated or not, True means the AREP is dedicated		

4.7 DLL Mapping Protocol Machine (DMPM)

4.7.1 General

The DLL Mapping Protocol Machine is common to all the AREP types. Only applicable primitives are different among different AREP types.

Remarks about dl identifiers:

The IEC Data Link Layer standard defines two types of identifiers to distinguish each DL-primitive or to match one DL outgoing primitive with its mate incoming primitive. They are suffixed as dl-identifier or DLS-user-identifier. In a real implementation of an FAL-DL interface, these identifications may be achieved by means of a pointer to memory location or a return value of a function call, or something else. Since they are purely a local matter, it is not testable over the Fieldbus network. For this reason, they are not included as parameters of the DMPM primitives.

“dl-identifiers” and “dls-user-identifiers” are mandatory in the DL-services. The FAL assumes that the values of these parameters are provided from DLSAPs or DLCEPs by a local means.

A remark about DLS-user identification:

It is assumed that a connection between one ARPM instance and one DMPM instance is established locally other than a protocol means. Therefore, DLS-user_identification parameters are not used in the DMPM primitives.

A remark about buffer or queue identifiers:

The IEC Data Link Layer standard uses parameters to identify a queue or a buffer that are shared between the Data Link Layer and the DLS-user. Although they are useful to clarify the operations of the Data Link Layer, none of them affects protocol behavior of the FAL and DL. In a real implementation, these parameters are implementation dependent. Therefore, this standard does not include parameters that directly correspond to these buffer or queue identifiers. A means for identifying the buffers and queues between the FAL and the DL is a local matter.

A remark about initialization of the Data Link Layer:

The IEC Data Link Layer standard defines services to setup resources within the layer, such as DL-Create or DL-Bind services. Although they are useful to clarify the operations of the Data Link Layer, none of them affects protocol behavior of the FAL and DL. Therefore, the FAL assumes that such initialization procedures have been executed prior to the operations of the FAL state machines. For the same reason, these DL-services are not listed in 4.7.4, Data Link Layer Service.

4.7.2 Primitive definitions**4.7.2.1 Primitives exchanged between DMPM and ARPM**

Table 174 and Table 175 list the primitives exchanged between the DMPM and the ARPM.

Table 174 – Primitives issued by ARPM to DMPM

Primitive names	Source	Associated parameters	Functions
FAL-PDU_req	ARPM	dmpm_service_name, arep_id, called_address, calling_address, responding_address, local_dlcep_address, reason, action_class, remote_dlsap_address, dll_priority, dls_user_data_timeliness, dlsdu	This primitive is used to request the DMPM to transfer an FAL-PDU, or to request an abort without transferring an FAL-PDU. It passes the FAL-PDU to the DMPM as a DLSDU. It also carries some of the Data Link Layer parameters that are referenced there.

Table 175 – Primitives issued by DMPM to ARPM

Primitive names	Source	Associated parameters	Functions
FAL-PDU_ind	DMPM	dmpm_service_name, originator, reason, duplicate_dlsdu, calling_address, calling_dlsap_address, dll_priority, fal_pdu, local_dle_timeliness, remote_dle_timeliness, sender_time_of_production	This primitive is used to pass an FAL-PDU received as a Data Link Layer service data unit to a designated ARPM. It also carries some of the Data Link Layer parameters that are referenced in the ARPM.
Connect_ind	DMPM	calling_dlcep_address, called_dlcep_address, called_address, calling_address, dlcep_class, delivery_from_responder, delivery_from_requestor, dll_priority, max_confirm_delay_on_connect, max_confirm_delay_on_data, dlpdu_authentication, residual_activity_as_sender, residual_activity_as_receiver, dlsdu_size_from_requestor, dlsdu_size_from_responder, sender_dl_timeliness_class, sender_time_of_production, receiver_dl_timeliness_class, dls_user_data	This primitive is used to convey a DL_Connect.ind primitive to the ARPM to process connection establishment. All the parameters that are associated with the DL_Connect.ind primitive are carried with this primitive.
Connect_cnf	DMPM	responding_address, dlcep_class, delivery_from_responder, delivery_from_requestor, dll_priority, max_confirm_delay_on_connect, max_confirm_delay_on_data, dlpdu_authentication, residual_activity_as_sender, residual_activity_as_receiver, dlsdu_size_from_requestor, dlsdu_size_from_responder, sender_dl_timeliness_class, receiver_dl_timeliness_class, sender_time_of_production, dls_user_data	This primitive is used to convey a DL_Connect.cnf primitive to the ARPM. All the parameters that are associated with the DL_Connect.cnf are carried with this primitive.
ErrorToARPM	DMPM	originator, reason	This primitive is used to convey selected communication errors reported by the Data Link Layer to a designated ARPM.

4.7.2.2 Parameters of ARPM/DMPM primitives

The parameters used with the primitives exchanged between the ARPM and the DMPM are described in Table 176.

Table 176 – Parameters used with primitives exchanged between ARPM and DMPM

Parameter name	Description
arep_id	This parameter carries a local identifier to specify the associated AR instance.
action_class	This parameter conveys the value of the dl_action_class parameter.
calling_address	This parameter conveys the value of the dl_calling_address.
calling_dlcep_address	This parameter conveys the value of the RequestingAREP parameter supplied with the EST_ReqPDU.
called_dlcep_address	This parameter conveys the value of the RespondingAREP parameter supplied with the EST_ReqPDU.
called_address	This parameter conveys the value of the dl_called_address parameter.
dmpm_service_name	This parameter conveys a Data Link Layer service name. Possible values are all the DL-XXXX.yyy primitives defined in this subclause and are represented as DMPM_XXXX.yyy.
dlcep_address	This parameter conveys the value of the dl_dlcep_address parameter.
duplicate_dlsdu	This parameter conveys the value of the dl_duplicate_dlsdu parameter of the received primitive.
dlcep_class	This parameter conveys the value of the dl_dlcep_class parameter.
dll_priority	This parameter conveys the value of the dl_dll_priority parameter.
delivery_from_responder	This parameter conveys the value of the dl_delivery_from_responder parameter.
delivery_from_requestor	This parameter conveys the value of the dl_delivery_from_requestor parameter.
dlsdu_size_from_requestor	This parameter conveys the value of the dl_dlsdu_size_from_requestor parameter.
dlsdu_size_from_responder	This parameter conveys the value of the dl_dlsdu_size_from_responder parameter.
dls_user_data	This parameter conveys the value of the dl_dls_user_data parameter.
dlsdu	This parameter conveys the value of the dl_dls_user_data parameter.
dlpdu_authentication	This parameter conveys the value of the dl_dlpdu_authentication parameter.
fal_pdu	This parameter conveys the value of the dl_dls_user_data parameter.
local_dlsap_address	This parameter conveys the value of the dl_local_dlsap_address parameter.
remote_dle_timeliness	This parameter conveys the value of the dl_sender_and_remote_dle_timeliness parameter.
local_dle_timeliness	This parameter conveys the value of the dl_dle_timeliness parameter.
max_confirm_delay_on_connect	This parameter conveys the value of the dl_max_confirm_delay_on_connect parameter.
max_confirm_delay_on_data	This parameter conveys the value of the dl_max_confirm_delay_on_data parameter.
originator	This parameter conveys the value of the dl_originator parameter.
reason	This parameter conveys the value of the dl_reason parameter.
responding_address	This parameter conveys the value of the dl_responding_address parameter.
residual_activity_as_sender	This parameter conveys the value of the dl_residual_activity_as_sender parameter.
residual_activity_as_receiver	This parameter conveys the value of the dl_residual_activity_as_receiver parameter.
sender_dl_timeliness_class	This parameter conveys the value of the dl_sender_dl_timeliness_class parameter.
receiver_dl_timeliness_class	This parameter conveys the value of the dl_receiver_dl_timeliness_class parameter.
sender_time_of_production	This parameter conveys the value of the dl_time_of_production parameter.
remote_dlsap_address	This parameter conveys the value of the dl_remote_dlsap_address parameter.

4.7.2.3 Primitives exchanged between Data Link Layer and DMPM

Table 177 lists the primitives exchanged between the Data Link Layer and the DMPM.

NOTE The following primitives and their parameters are defined in IEC 61158-3.

Table 177 – Primitives exchanged between Data Link Layer and DMPM

Primitive names	Source	Associated parameters
DL_Connect.ind	Data Link Layer	dl_called_address, dl_calling_address, dl_dlcep_class, dl_delivery_from_requestor, dl_delivery_from_responder, dl_dll_priority, dl_max_confirm_delay_on_connect, dl_max_confirm_delay_on_data, dl_dlpdu_authentication, dl_residual_activity_as_sender, dl_residual_activity_as_receiver, dl_dlsdu_size_from_requestor, dl_dlsdu_size_from_responder, dl_sender_dl_timeliness_class, dl_sender_time_of_production, dl_receiver_dl_timeliness_class, dl_dls_user_data
DL_Connect.req(out)	Data Link Layer	(none)
DL_Connect.cnf		dl_responding_address, dl_dlcep_class, dl_delivery_from_requestor, dl_delivery_from_responder, dl_dll_priority, dl_max_confirm_delay_on_connect, dl_max_confirm_delay_on_data, dl_dlpdu_authentication, dl_residual_activity_as_sender, dl_residual_activity_as_receiver, dl_dlsdu_size_from_requestor, dl_dlsdu_size_from_responder, dl_sender_dl_timeliness_class, dl_receiver_dl_timeliness_class, dl_sender_dl_timeliness_class, dl_sender_time_of_production, dl_receiver_dl_timeliness_class, dl_dls_user_data
DL_Connection_Established.ind	Data Link Layer	(none)
DL_Disconnect.ind	Data Link Layer	dl_originator, dl_reason, dl_dls_user_data
DL_Data.ind	Data Link Layer	dl_dls_user_data
DL_Data.cnf	Data Link Layer	dl_status
DL_Buffer_Received.ind	Data Link Layer	dl_duplicate_dlsdu
DL_Unitdata.ind	Data Link Layer	dl_called_address, dl_calling_address, dl_dll_priority, dl_dls_user_data
DL_Unitdata.cnf	Data Link Layer	dl_status
DL_Buffer_Sent.ind	Data Link Layer	(none)
DL_Get.req(out)	Data Link Layer	dl_status, dl_reported_service_identification_class, dl_calling_dlsap_address, dl_dll_priority, dl_dls_user_data, dl_local_dle_timeliness, dl_sender_and_remote_dle_timeliness, dl_sender_time_of_production
DL_Put.req(out)	Data Link Layer	dl_status
DL_Compel_Service.req(out)	Data Link Layer	dl_status

Primitive names	Source	Associated parameters
DL_Connect.req(in)	DMPM	dl_called_address, dl_calling_address, dl_dlcep_address, dl_dlcep_class, dl_delivery_from_requestor, dl_delivery_from_responder, dl_dll_priority, dl_max_confirm_delay_on_connect, dl_max_confirm_delay_on_data, dl_dlpdu_authentication, dl_residual_activity_as_sender, dl_residual_activity_as_receiver, dl_scheduling_policy, dl_dlsdu_size_from_requestor, dl_dlsdu_size_from_responder, dl_sender_dl_timeliness_class, dl_sender_time_window_size, dl_sender_synchronizing_dlcep, dl_sender_time_of_production, dl_receiver_dl_timeliness_class, dl_receiver_time_window_size, dl_receiver_synchronizing_dlcep, dl_dls_user_data
DL_Connect.rsp	DMPM	dl_responding_address, dl_dlcep_address, dl_dlcep_class, dl_delivery_from_requestor, dl_delivery_from_responder, dl_dll_priority, dl_max_confirm_delay_on_connect, dl_max_confirm_delay_on_data, dl_dlpdu_authentication, dl_residual_activity_as_sender, dl_residual_activity_as_receiver, dl_scheduling_policy, dl_dlsdu_size_from_requestor, dl_dlsdu_size_from_responder, dl_sender_dl_timeliness_class, dl_sender_time_window_size, dl_sender_synchronizing_dlcep, dl_sender_time_of_production, dl_receiver_dl_timeliness_class, dl_receiver_time_window_size, dl_receiver_synchronizing_dlcep, dl_dls_user_data
DL_Unitdata.req	DMPM	dl_called_address, dl_calling_address, dl_dll_priority, dl_max_confirm_delay, dl_remote_dle_confirmed, dl_dls_user_data
DL_Disconnect.req	DMPM	dl_reason dl_dls_user_data
DL_Data.req	DMPM	dl_dls_user_data
DL_Get.req(in)	DMPM	(none)
DL_Put.req(in)	DMPM	dl_dls_user_data, dl_dls_user_data_timeliness
DL_Compel_Service.req(in)	DMPM	dl_action_class, dl_remote_dlsap_address, dl_priority

4.7.2.4 Parameters of DMPM/Data Link Layer primitives

The parameters used with the primitives exchanged between the DMPM and the Data Link Layer are identical to those defined in the Data Link Layer Service Definition (IEC 61158-3). They are prefixed by “dl_” to indicate that they are used by the FAL.

4.7.3 DMPM state machine

4.7.3.1 DMPM states

The defined state of the DMPM together with its description are listed in Table 178 and Figure 40.

Table 178 – DMPM state descriptions

State name	Description
ACTIVE	The DMPM in the ACTIVE state is ready to transmit or receive primitives to or from the Data Link Layer and the ARPM.

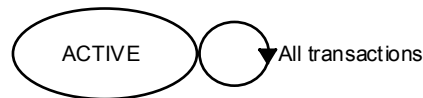


Figure 40 – State transition diagram of DMPM

4.7.3.2 DMPM state table

Table 179 and Table 180 define the DMPM state machine.

NOTE 1 Although each primitive contains all the available parameters, only those applicable to particular ARPM are relevant.

NOTE 2 Parameters starting with a capital letter, "DlcepClass" for instance, refer to those defined in the attribute list of each ARPM. Therefore, they are not conveyed by the service primitives defined here.

Table 179 – DMPM state table – sender transactions

#	Current state	Event or condition => action	Next state
S1	ACTIVE	FAL-PDU_req && dmpm_service_name = "DMPM_Connect_req" => PickArep (arep_id), DL_Connect.req(in) { dl_called_address := called_address, dl_calling_address := calling_address, dl_dlcep_address := local_dlcep_address, dl_dlcep_class := DlcepClass, dl_delivery_from_requestor := FromRequestorToResponder, dl_delivery_from_responder := FromResponderToRequestor, dl_dll_priority := DIIPriority, dl_max_confirm_delay_on_connect := MaxConfirmDelayOnDIConnect, dl_max_confirm_delay_on_data := MaxConfirmDelayOnDIData, dl_dlpdu_authentication := DlpduAuthentication, dl_residual_activity_as_sender := ResidualActivityAsSender, dl_residual_activity_as_receiver := ResidualActivityAsReceiver, dl_scheduling_policy := DISchedulingPolicy, dl_dlsdu_size_from_requestor := MaxDlsduSizeFromRequestor, dl_dlsdu_size_from_responder := MaxDlsduSizeFromResponder, dl_sender_dl_timeliness_class := PublisherDITimelinessClass, dl_sender_time_window_size := PublisherTimeWindowSize, dl_sender_synchronizing_dlcep := PublisherSynchronizingDlcep, dl_time_of_production := TimeOfProduction, dl_receiver_dl_timeliness_class := SubscriberDITimelinessClass, dl_receiver_time_window_size := SubscriberTimeWindowSize, dl_receiver_synchronizing_dlcep := SubscriberSynchronizingDlcep, dl_dls_user_data := dlsdu }	ACTIVE

Table 179 (continued)

#	Current state	Event or condition => action	Next state
S2	ACTIVE	FAL-PDU_req && dmpm_service_name = "DMPM_Connect_rsp" => PickArep (arep_id), DL_Connect.rsp { dl_responding_address := responding_address, dl_dlcep_address := local_dlcep_address, dl_dlcep_class := DlcepClass, dl_delivery_from_requestor := FromRequestorToResponder, dl_delivery_from_responder := FromResponderToRequestor, dl_dll_priority := DIIPriority, dl_max_confirm_delay_on_connect := MaxConfirmDelayOnDIConnect, dl_max_confirm_delay_on_data := MaxConfirmDelayOnDIData, dl_dpdu_authentication := DlpduAuthentication, dl_residual_activity_as_sender := ResidualActivityAsSender, dl_residual_activity_as_receiver := ResidualActivityAsReceiver, dl_scheduling_policy := DISchedulingPolicy, dl_dlsdu_size_from_requestor := MaxDlsduSizeFromRequestorNegotiated, dl_dlsdu_size_from_responder := MaxDlsduSizeFromResponderNegotiated, dl_sender_dl_timeliness_class := PublisherDITimelinessClass, dl_sender_time_window_size := PublisherTimeWindowSize, dl_sender_synchronizing_dlcep := PublisherSynchronizingDlcep, dl_sender_time_of_production := TimeOfProduction, dl_receiver_dl_timeliness_class := SubscriberDITimelinessClass, dl_receiver_time_window_size := SubscriberTimeWindowSize, dl_receiver_synchronizing_dlcep := SubscriberSynchronizingDlcep, dl_dls_user_data := dlsdu } }	ACTIVE
S3	ACTIVE	FAL-PDU_req && dmpm_service_name = "DMPM_Disconnect_req" => PickArep (arep_id), DL_Disconnect.req { dl_reason := reason, dl_dls_user_data := dlsdu } }	ACTIVE
S4	ACTIVE	FAL-PDU_req && dmpm_service_name = "DMPM_Data_req" => PickArep (arep_id), DL_Data.req { dl_dls_user_data := dlsdu } }	ACTIVE
S5	ACTIVE	FAL-PDU_req && dmpm_service_name = "DMPM_Unitdata_req" => PickArep (arep_id), DL_Unitdata.req { dl_called_address := RemoteDisapAddress, dl_calling_address := LocalDisapAddress, dl_dll_priority := DIIPriority, dl_max_confirm_delay := MaxConfirmDelayOnUnitdata, dl_remote_dle_confirmed := DleRemoteConf, *1 dl_dls_user_data := dlsdu } } *1: Default "False"	ACTIVE

Table 179 (continued)

#	Current state	Event or condition => action	Next state
S6	ACTIVE	<pre> FAL-PDU_req && dmpm_service_name = "DMPM_Put_req" => PickArep (arep_id), DL_Put.req(in) { dl_dls_user_data := dlsdu, dl_dls_user_data_timeliness := dls_user_data_timeliness } DL_Put.req(out) -- immediate response && dl_status = "success" => FAL-PDU_ind { dmpm_service_name := "DMPM_Put_cnf", reason := dl_status } DL_Put.req(out) -- immediate response && dl_status <> "success" => FAL-PDU_ind { dmpm_service_name := "DMPM_Put_cnf", reason := dl_status } ErrorToARPM { originator := "local_dls", reason := dl_status } } </pre>	ACTIVE
S7	ACTIVE	<pre> FAL-PDU_req && dmpm_service_name = "DMPM_Get_req" => PickArep (arep_id), DL_Get.req(in) { } DL_Get.req(out) -- immediate response && dl_reported_service_identification_class = "NONE" => FAL-PDU_ind { dmpm_service_name := "DMPM_Get_cnf", reason := dl_status, calling_dlsap_address := dl_calling_dlsap_address, fal_pdu := dl_, local_dle_timeliness := dl_local_dle_timeliness, remote_dle_timeliness := dl_sender_and_remote_dle_timeliness, sender_time_of_production := dl_sender_time_of_production } </pre>	ACTIVE
S8	ACTIVE	<pre> FAL-PDU_req && dmpm_service_name = "DMPM_Compel_req" => PickArep (arep_id), DL_Compel_Service.req(in) { dl_action_class := action_class dl_remote_dlsap_address := remote_dlsap_address, dl_priority := dll_priority } DL_Compel_Service.req(out) -- immediate response => FAL-PDU_ind { dmpm_service_name := "DMPM_Compel_Service_cnf", reason := dl_status } </pre>	ACTIVE

Table 180 – DMPM state table – receiver transactions

#	Current state	Event or condition => action	Next state
R1	ACTIVE	DL_Connect.ind && LocateArep (dl_dls_user_data) = "NOT FOUND" => DL_Disconnect.req { dl_reason := "AREP Not Found", dl_dls_user_data := "null" }	ACTIVE
R2	ACTIVE	DL_Connect.ind && LocateArep (dl_dls_user_data) = "NOT Relevant" => DL_Disconnect.req { dl_reason := "Invalid AREP Type", dl_dls_user_data := "null" }	ACTIVE
R3	ACTIVE	DL_Connect.ind && LocateArep (dl_dls_user_data) = "True" => Connect_ind { calling_dlcep_address := RequestingAREP, ; from dlsdu of DL_Connect.ind called_dlcep_address := RespondingAREP, ; from dlsdu of DL_Connect.ind called_address := dl_called_address, calling_address := dl_calling_address, dlcep_class := dl_dlcep_class, delivery_from_requestor := dl_delivery_from_requestor, delivery_from_responder := dl_delivery_from_responder, dll_priority := dl_dll_priority, max_confirm_delay_on_connect := dl_max_confirm_delay_on_connect, max_confirm_delay_on_data := dl_max_confirm_delay_on_data, dlpdu_authentication := dl_dlpdu_authentication, residual_activity_as_sender := dl_residual_activity_as_sender, residual_activity_as_receiver := dl_residual_activity_as_receiver, dlsdu_size_from_requestor := dl_dlsdu_size_from_requestor, dlsdu_size_from_responder := dl_dlsdu_size_from_responder, sender_dl_timeliness_class := dl_sender_dl_timeliness_class, sender_time_of_production := dl_sender_time_of_production, receiver_dl_timeliness_class := dl_receiver_dl_timeliness_class, dls_user_data := dl_dls_user_data }	ACTIVE
R4	ACTIVE	DL_Connect.cnf && FindAREP () = "False" => DL_Disconnect.req { dl_reason := "DLCEP Not Found", dl_dls_user_data := "null" }	ACTIVE
R5	ACTIVE	DL_Connect.cnf && FindAREP () = "True" => Connect_cnf { responding_address := dl_responding_address, dlcep_class := dl_dlcep_class, delivery_from_requestor := dl_delivery_from_requestor, delivery_from_responder := dl_delivery_from_responder, dll_priority := dl_dll_priority, max_confirm_delay_on_connect := dl_max_confirm_delay_on_connect, max_confirm_delay_on_data := dl_max_confirm_delay_on_data, dlpdu_authentication := dl_dlpdu_authentication, residual_activity_as_sender := dl_residual_activity_as_sender, residual_activity_as_receiver := dl_residual_activity_as_receiver, dlsdu_size_from_requestor := dl_dlsdu_size_from_requestor, dlsdu_size_from_responder := dl_dlsdu_size_from_responder, sender_dl_timeliness_class := dl_sender_dl_timeliness_class, sender_time_of_production := dl_sender_time_of_production, receiver_dl_timeliness_class := dl_receiver_dl_timeliness_class, dls_user_data := dl_dls_user_data }	ACTIVE

Table 180 (continued)

#	Current state	Event or condition => action	Next state
R6	ACTIVE	DL_Connection_Established.ind && FindAREP () = "False" => (no actions taken)	ACTIVE
R7	ACTIVE	DL_Connection_Established.ind && FindAREP () = "True" => FAL-PDU_ind { dmpm_service_name := "DMPM_Connection_Established_ind", fal_pdu := "null" }	ACTIVE
R8	ACTIVE	DL_Disconnect.ind && FindAREP () = "False" => (no actions taken)	ACTIVE
R9	ACTIVE	DL_Disconnect.ind && FindAREP () = "True" => FAL-PDU_ind { dmpm_service_name := "DMPM_Disconnect_ind", originator := dl_originator, reason := dl_reason, fal_pdu := dl_dls_user_data }	ACTIVE
R10	ACTIVE	DL_Unitdata.ind && FindAREP (dl_called_address) = "False" => (no actions taken)	ACTIVE
R11	ACTIVE	DL_Unitdata.ind && FindAREP (dl_called_address) = "True" && ExplicitQueue = "False" => FAL-PDU_ind { dmpm_service_name := "DMPM_Unitdata_ind", calling_address := dl_calling_address, dll_priority := dl_dll_priority, fal_pdu := dl_dls_user_data }	ACTIVE
R12	ACTIVE	DL_Unitdata.ind && FindAREP (dl_called_address) = "True" && ExplicitQueue = "True" => DL_Get.req(in) { } DL_Get.req(out) -- immediate response && dl_status = "success" && dl_reported_service_identification_class = "DL(SAP)-ADDRESS" => FAL-PDU_ind { dmpm_service_name := "DMPM_Unitdata_ind", calling_address := dl_calling_dlsap_address, dll_priority := dl_dll_priority, fal_pdu := dl_dls_user_data, local_dle_timeliness := dl_local_dle_timeliness, remote_dle_timeliness := dl_sender_and_remote_dle_timeliness, sender_time_of_production := dl_sender_time_of_production } DL_Get.req(out) -- immediate response && dl_status <> "success" => FAL-PDU_ind { dmpm_service_name := "DMPM_Unitdata_ind", reason := dl_status } ErrorToARPM { originator := "local_dls", reason := dl_status }	ACTIVE

Table 180 (continued)

#	Current state	Event or condition => action	Next state
R13	ACTIVE	DL_Unitdata.cnf && dl_status <> "success" => FAL-PDU_ind { dmpm_service_name := "DMPM_Unitdata_cnf", reason := dl_status } ErrorToARPM { originator := "local_dls", reason := dl_status }	ACTIVE
R14	ACTIVE	DL_Unitdata.cnf && dl_status = "success" => FAL-PDU ind { dmpm service name := "DMPM_Unidata_cnf", reason := dl_status }	ACTIVE
R15	ACTIVE	DL_Data.ind && FindAREP () = "False" => (no actions taken)	ACTIVE
R16	ACTIVE	DL_Data.ind && FindAREP () = "True" && ExplicitQueue = "False" => FAL-PDU_ind { dmpm_service_name := "DMPM_Data_ind", fal_pdu := dl_dls_user_data }	ACTIVE
R17	ACTIVE	DL_Data.ind && FindAREP () = "True" && ExplicitQueue = "True" => DL_Get.req(in) { } DL_Get.req(out) -- immediate response && dl_status = "success" && dl_reported_service_identification_class = "DLCEP" => FAL-PDU_ind { dmpm_service_name := "DMPM_Data_ind", fal_pdu := dl_dls_user_data } DL_Get.req(out) -- immediate response && dl_status <> "success" => FAL-PDU_ind { dmpm_service_name := "DMPM_Get_cnf", reason := dl_status } ErrorToARPM { originator := "local_dls", reason := dl_status }	ACTIVE
R18	ACTIVE	DL_Data.cnf && dl_status = "success" => (no actions taken)	ACTIVE

Table 180 (continued)

#	Current state	Event or condition => action	Next state
R19	ACTIVE	DL_Data.cnf && dl_status <> "success" => FAL-PDU_ind { dmpm_service_name := "DMPM_Data_cnf", reason := dl_status } ErrorToARPM { originator := "local_dls", reason := dl_status }	ACTIVE
R20	ACTIVE	DL_Buffer_Received.ind && FindAREP () = "False" => (no actions taken)	ACTIVE
R21	ACTIVE	DL_Buffer_Received.ind && FindAREP () = "True" => DL_Get.req(in) { } DL_Get.req(out) -- immediate response && dl_status = "success" && dl_reported_service_identification_class = "NONE" => FAL-PDU_ind { dmpm_service_name := "DMPM_Buffer_Received_ind", duplicate_dlsdu := dl_duplicate_dlsdu, ; from DL_Buffer_Received.ind fal_pdu := dl_dls_user_data, local_dle_timeliness := dl_local_dle_timeliness, remote_dle_timeliness := dl_sender_and_remote_dle_timeliness, sender_time_of_production := dl_sender_time_of_production } DL_Get.req(out) -- immediate response && dl_status <> "success" => FAL-PDU_ind { dmpm_service_name := "DMPM_Get_cnf", reason := dl_status } ErrorToARPM { originator := "local_dls", reason := dl_status }	ACTIVE
R22	ACTIVE	DL_Buffer_Sent.ind && FindAREP () = "False" => (no actions taken)	ACTIVE
R23	ACTIVE	DL_Buffer_Sent.ind && FindAREP () = "True" => FAL-PDU_ind { dmpm_service_name := "DMPM_Buffer_Sent_ind" }	ACTIVE

4.7.3.3 Functions used by DMPM

Table 181 through Table 183 define the functions used by the DMPM.

Table 181 – Function PickArep

Name	PickArep	Used in	DMPM
Input		Output	
arep_id		(all the attributes of the specified AREP)	
Function			
Selects the attributes for the AREP specified by the arep_id parameter. After this function is executed, the attributes of the selected AREP are available to the state machine.			

Table 182 – Function FindAREP

Name	FindAREP	Used in	DMPM
Input		Output	
dl_called_address (local mapping)		True False	
Function	This function identifies the AREP that shall be bound with an active DMPM. True means the AREP exists. If it does, this function also returns a means to send a DMPM primitive to that AREP		

Table 183 – Function LocateArep

Name	LocateArep	Used in	DMPM
Input		Output	
dl_dls_user_data (of DL_Connect.ind)		True Not Found Not Relevant Access to attributes of the located AREP.	
Function	This function returns the value of True and a means to get access to the attributes of the located AREP if all of the following conditions are met. Otherwise, it returns the value of either “Not Found” or “Not Relevant.” Decodes the dlsdu that is conveyed by the dl_dls_user_data argument and checks if the FAL PDU type is “EST_ReqPDU.” Table 432 Decodes the FAL-PDU and extracts the RequestingAREP and RespondingAREP parameters. c) Looks for the AREPs that can accept a peer-to-peer connection, such as a QUB AREP, whose LocalDicepAddress attribute value is equal to the RespondingAREP.		

4.7.4 Data Link Layer service selection

4.7.4.1 General

This subclause briefly describes the Data Link Layer services utilized by the FAL. These Data Link Layer services are fully defined in the Data Link Layer standard (IEC 61158-3).

NOTE The FAL assumes that resources, such as buffers or queues, are set up prior to any operations of FAL protocol machines by a local means. DL services such as Create or Bind may be used for this purpose, the use of these services is not required by the FAL. Therefore, these local services are not listed in this subclause.

4.7.4.2 DL-CONNECT

This service is used to establish a new connection or join in an existing one, and refers to the Connect service.

4.7.4.3 DL-CONNECTION-ESTABLISHED

The DL-Connection-Established service is used to inform the FAL which has sent an ASC.rsp(+) primitive that a Data Link Layer connection is ready to use.

4.7.4.4 DL-DISCONNECT

This service is used to release an existing connection or leave from it and refers to the Disconnect service.

4.7.4.5 DL-UNITDATA

This service is used for the connectionless data transfer mode and refers to the Unitdata service. It is used to transmit an FAL-PDU from one AREP.

4.7.4.6 DL-DATA

This service is used for connection-oriented data transfer mode and refers to the Data service. This service is used to transfer an FAL-PDU from one AREP.

4.7.4.7 DL-PUT

This service is used to copy an FAL-PDU to a buffer. It refers to the Put service.

4.7.4.8 DL-GET

This service is used to read an FAL-PDU from a buffer, or to attempt to remove an FAL-PDU from a FIFO queue, which has been bound to a DLSAP. It refers to the Get service.

4.7.4.9 DL-BUFFER-RECEIVED

The DL-Buffer-Received service is used to inform the FAL of a new update on the specified receive buffer.

4.7.4.10 DL-BUFFER-SENT

The DL-Buffer-Sent service is used to inform the FAL that the specified buffer content has just been sent.

4.7.4.11 DL-COMPEL-DATA

The DL-Compel-Data service is used to compel transmission of the content of the buffer of the specified DLCEP.

4.8 Protocol options**4.8.1 Protocol option 1****4.8.1.1 General**

This subclause provides a guideline for migration from a previous national standard to the IEC FAL. Although some protocol machines as well as encoding rules are selected by this option, this standard does not intend to endorse such selection or restrict other possible selections.

4.8.1.2 Abstract syntax selection

This profile uses the following abstract syntax:

- FAL-AR PDU Abstract Syntax 2

4.8.1.3 Protocol machine overview

The overview of protocol machines of Protocol option 1 is shown Figure 41.

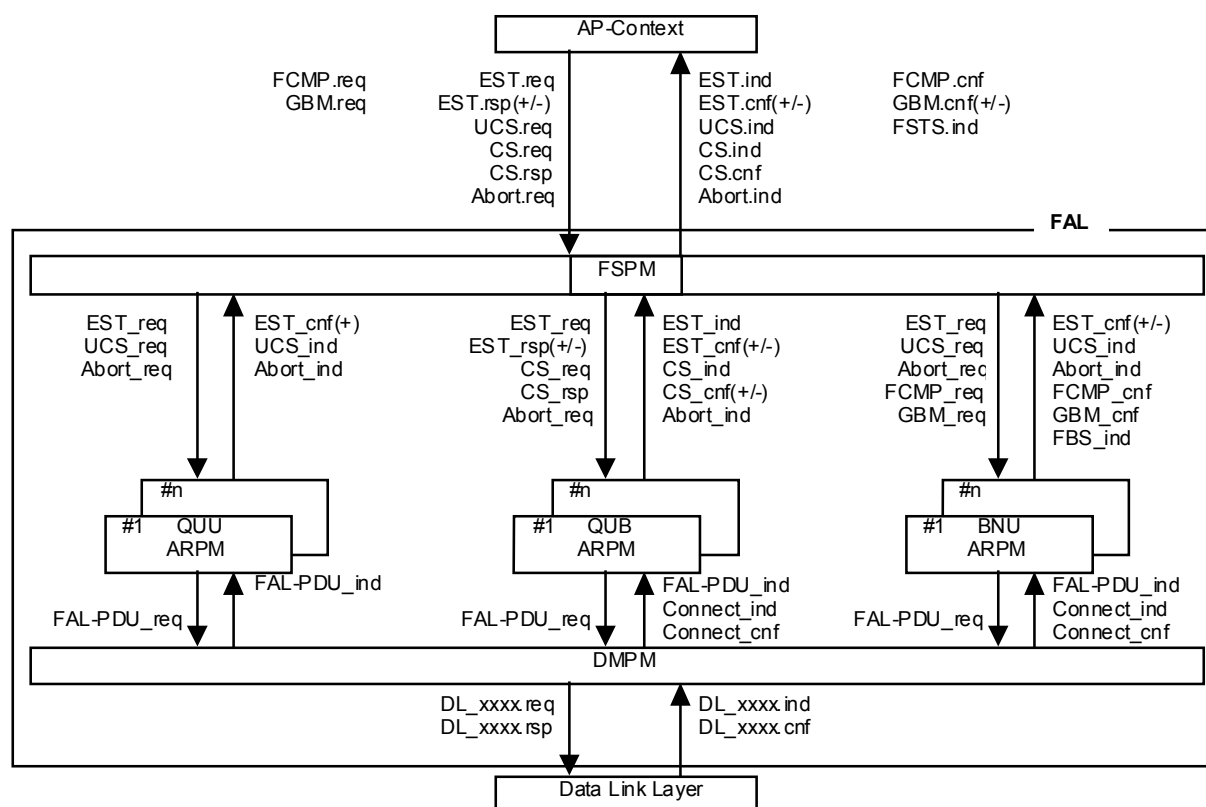


Figure 41 – Primitives exchanged between protocol machines

4.8.1.4 Application Relationship Protocol Machine (ARPM) selection

The following ARPMs are selected for this profile:

- Queued User-triggered Unidirectional ARPM (QUU)
- Queued User-triggered Bidirectional ARPM (QUB)
- Buffered Network-scheduled Unidirectional (BNU)

4.8.1.5 Encoding rule selection

This profile uses the following encoding rule:

- Traditional Encoding Rule (TER)

4.8.2 Protocol option 2

4.8.2.1 General

This subclause provides a guideline for migration from a previous national standard to the IEC FAL. Although some protocol machines as well as encoding rules are selected by this option, this standard does not intend to endorse such selection or restrict other possible selections.

4.8.2.2 Abstract syntax

This profile uses the following abstract syntax:

- FAL-AR PDU Abstract Syntax 1

4.8.2.3 Protocol machine overview

The overview of protocol machines of Protocol option 2 is shown in Figure 42.

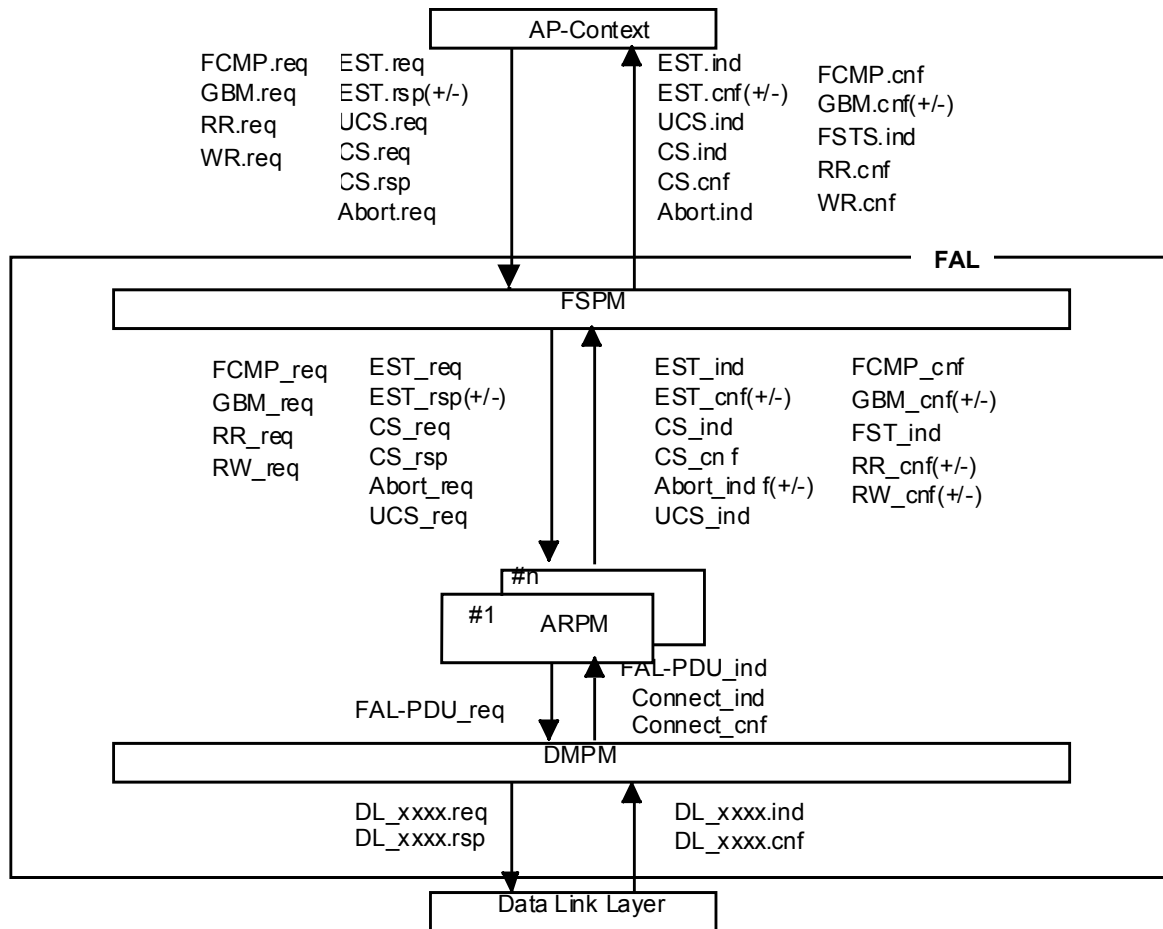


Figure 42 – Primitives exchanged between protocol machines

4.8.2.4 Application Relationship Protocol Machine (ARPM) selections

The following ARPMs are used for this profile:

- Queued User-triggered Bidirectional-Connectionless (QUB-CL)
- Queued User-triggered Bidirectional-Segmentation (QUB-SEG)
- Buffered Network-scheduled Unidirectional-MP (BNU-MP)
- Buffered User-triggered Bidirectional (BUB)
- The other ARPMs defined in this standard can be used.

4.8.2.5 Encoding rule selection

This profile uses the following encoding rules:

- Buffer Encoding Rule (BER) and Messaging Encoding Rule (MER)

4.8.3 Protocol option 3

4.8.3.1 General

This subclause provides a guideline for migration from a previous national standard to the IEC FAL. Although some protocol machines as well as encoding rules are selected by this option, this standard does not intend to endorse such selection or restrict other possible selections.

4.8.3.2 Abstract syntax

This profile uses the following abstract syntax:

- FAL-AR PDU Abstract Syntax 2

4.8.3.3 Protocol machine overview

The overview of protocol machines of Protocol option 3 is shown in Figure 43.

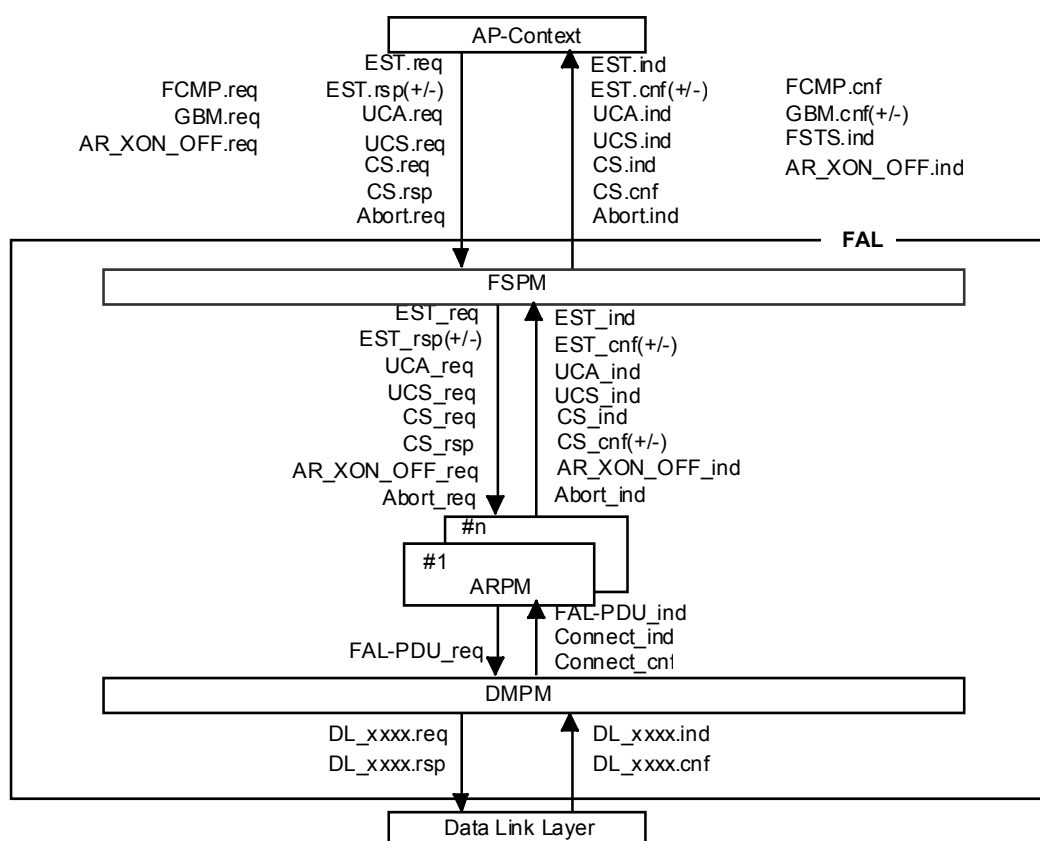


Figure 43 – Primitives exchanged between protocol machines

4.8.3.4 Application Relationship Protocol Machine (ARPM) selection

The following ARPMs are selected for this profile:

- Queued User-triggered Bidirectional-Flow Control (QUB-FC)
- Buffered Network-scheduled Bidirectional (BNB)

4.8.3.5 Encoding rule selection

This profile uses the following encoding rule:

- Traditional Encoding Rule (TER)

5 Type 2

5.1 Abstract syntax

5.1.1 FAL PDU abstract syntax

5.1.1.1 General

FAL_PDU shall be either a UCMM_PDU or a Transport_PDU.

UCMM_PDU is used to convey information for connection-less services.

Transport_PDU is used to convey information for connection-oriented services.

A connected message assumes previously negotiated resources and parameters at its source, its destination(s), and any intermediate transit points. These resources are referenced by a unique connection identifier and do not need to be contained in each message, only the connection ID is needed to identify the message and refer to its related parameters, thus giving significant savings in message efficiency.

An unconnected message provides a means to communicate on the local link without previously negotiated resources at the destination so it shall carry full destination ID details, internal data descriptors and full source ID details if a reply is requested. Unconnected messages are used mainly to create connections.

The unconnected service is provided by the Unconnected Message Manager (UCMM). Messages received through the UCMM are forwarded to the Message Router (MR), which direct them to the appropriate internal object for execution. Connections are established by specific unconnected messages sent through the Connection Manager (CM) using UCMM services. Connections may be established either to the Message Router (for messaging purpose), or directly to an application object. Connection target is specified using a connection path, and may be either on the local or a remote link, through several intermediate router nodes. Once a connection is established with an application object, the UCMM, MR and CM are no longer required, since data will be exchanged directly with the connected objet, based on the corresponding connection ID. Connected messages sent to the Message Router will be forwarded to the appropriate internal object for execution.

5.1.1.2 PDU structure

5.1.1.2.1 UCMM_PDU structure

UCMM_PDU is a sequence of a UCMM_Header, followed by either OM_Service (Object Management ASE) or CM_Service (Connection Manager ASE).

OM_Service shall be either OM_Request or OM_Response.

OM_Request shall consist of MR_Request_Header and a Service_RequestPDU.

Service_RequestPDU shall be one of the following :

- Get_Attribute_Single_RequestPDU
- Get_Attribute_All_RequestPDU
- Get_Attribute_List_RequestPDU
- Set_Attribute_Single_RequestPDU
- Set_Attribute_All_RequestPDU
- Set_Attribute_List_RequestPDU
- Reset_RequestPDU

- Create_RequestPDU
- Delete_RequestPDU
- Start_RequestPDU
- Stop_RequestPDU
- Find_Next_Object_Instance_RequestPDU
- NOP_RequestPDU
- Apply_Attributes_RequestPDU
- Save_RequestPDU
- Restore_RequestPDU

OM_Response shall consist of MR_Response_Header and a Service_ResponsePDU.

Service_ResponsePDU shall be one of the following :

- Get_Attribute_Single_ResponsePDU
- Get_Attribute_All_ResponsePDU
- Get_Attribute_List_ResponsePDU
- Set_Attribute_Single_ResponsePDU
- Set_Attribute_All_ResponsePDU
- Set_Attribute_List_ResponsePDU
- Reset_ResponsePDU
- Create_ResponsePDU
- Delete_ResponsePDU
- Start_ResponsePDU
- Stop_ResponsePDU
- Find_Next_Object_Instance_ResponsePDU
- NOP_ResponsePDU
- Apply_Attributes_ResponsePDU
- Save_ResponsePDU
- Restore_ResponsePDU

CM_Service shall be either CM_Request or CM_Response.

CM_Request shall consist of MR_Request_Header and a CM_RequestPDU.

CM_RequestPDU shall be one of the following :

- Forward_Open_RequestPDU
- Forward_Close_RequestPDU
- Ex_Forward_Open_RequestPDU
- Unconnected_Send_RequestPDU
- Get_Connection_Data_RequestPDU
- Search_Connection_Data_RequestPDU
- Get_Object_Owner_RequestPDU

CM_Response consists of MR_Response_Header and a CM_ResponsePDU.

CM_ResponsePDU shall be one of the following :

- Forward_Open_ResponsePDU
- Forward_Close_ResponsePDU
- Ex_Forward_Open_ResponsePDU
- Unconnected_Send_ResponsePDU
- Get_Connection_Data_ResponsePDU
- Search_Connection_Data_ResponsePDU
- Get_Object_Owner_ResponsePDU

5.1.1.2.2 Transport_PDU structure

Transport_PDU is a sequence of a Transport_Header, followed by either OM_Service (Object Management ASE) or Application Data.

OM_Service is defined in 5.1.1.2.1 above.

Application Data comes from the source to which the connection has been made.

5.1.1.3 UCMM_PDUs

All UCMM_PDUs shall be sent and received using fixed tag 0x83 or Management tag 0x88. When a device becomes powered, the UCMM shall invoke the DLL_enable_fixed_request service of the Data Link Layer to request that fixed tag 0x83 or 0x88 Lpackets be routed to the UCMM. All sending and receiving of TPDUs shall use the DLL_fixed_request.req and DLL_fixed_request.ind service primitives of the Data Link Layer, respectively. The packet parameter of these services shall be prefixed with the following header to form the Transport PDU before sending to the Data Link Layer. The format of the UCMM_PDU Header is shown in Table 184.

Table 184 – UCMM_PDU header format

Parameter name	Format
command_code	USINT
Timeout	USINT
TransactionID	UINT

The command_code shall specify the type of UCMM_PDU as shown in Table 185. UCMM packets received with a reserved command_code shall be discarded without acknowledgement.

Table 185 – UCMM command codes

Command_code	Description
0	Reserved
1	acknowledge a request
2	request with retry until acknowledged
3	response with retry until acknowledged
4	request with no acknowledge and no response
5	acknowledge a response
6	response which shall not retry (no acknowledge)
7	request with retry until response (no acknowledge)
8	Request which shall not retry and shall cause a code 6 response
9 – 255	Reserved

The timeout field shall specify the duration of the transaction in an 8-bit floating-point format. The most significant 5 bits of the field shall be an unsigned exponent that is not biased. The least significant 3 bits shall be the least significant 3 bits of a 4 bit unsigned mantissa. The most significant bit of the mantissa shall be 1 and shall not appear explicitly in the 8-bit representation. The binary point shall be positioned between the implied 1 and the rest of the mantissa. The units of the transaction duration computed using the timeout field shall be in milliseconds.

NOTE Using ANSI C precedence rules, the number of 0,125ms ticks is $(8 \mid \text{timeout} \& 7) \ll (\text{timeout} \gg 3 \& 31)$.

The transactionID field shall be composed of two sub-fields:

- the least significant 10 bits, RECORD, shall identify a specific transaction;
- the most significant 6 bits, SEQUENCE, shall be used for duplicate detection. It shall be incremented for every unique message on this RECORD; however it shall not be incremented on a retry.

Only one message shall be outstanding for a given RECORD.

If multiple outstanding messages are required then multiple RECORDs are required.

When an I'm alive fixed tag packet is received from a node (see IEC 61158-4, Type 2, Station Management), all UCMM transactions with that node shall be aborted.

5.1.1.4 Transport_Headers

5.1.1.4.1 General

Contents of Transport headers varies depending on the class of transport selected during the connection establishment.

5.1.1.4.2 Class 0 (null or base)

The class 0 transport header shall be an unsigned 16-bit number. This header shall be written to the TPDU buffer by the transport and shall be simply discarded by the transport from the packet coming from the consumer. The format of the Header is shown in Table 186.

Table 186 – Transport class 0 header

Parameter name	Format
don't_care	UINT

5.1.1.4.3 Class 1 (duplicate detection)

The class 1 transport header shall be an unsigned 16-bit sequence count. This header shall be written to the TPDU buffer by the transport and shall be read by the transport from the packet coming from the consumer. The format of the Header is shown in Table 187.

Table 187 – Transport class 1 header

Parameter name	Format
sequence_count	UINT

5.1.1.4.4 Class 2 (acknowledged)

Like the class 1 transport header, the class 2 transport header shall be a 16-bit sequence count. This header shall be written to the TPDU buffer by the transport and shall be read by the transport from the packet coming from the consumer. The format of the Header is shown in Table 188.

Table 188 – Transport class 2 header

Parameter name	Format
sequence_count	UINT

5.1.1.4.5 Class 3 (verified)

Like the class 1 transport header, the class 3 transport header shall be a 16-bit sequence count. This header shall be written to the TPDU buffer by the transport and shall be read by the transport from the packet coming from the consumer. The format of the Header is shown in Table 189.

Table 189 – Transport class 3 header

Parameter name	Format
sequence_count	UINT

5.1.1.4.6 Class 4 (non-blocking), class 5 (non-blocking, fragmenting) and class 6 (multipoint connection, fragmenting)

These classes of transports all use the header format shown in Table 190. Classes 4 and 5 have two headers in each TPDU, one Sender and one Receiver. Class 6 uses a Sender header in the Client to Server direction and a Receiver header in the Server to Client direction.

Table 190 – Classes 4 to 6 header format

Command			Sequence number	Data	
MSB	Bits 15 – 12		Bits 11 – 8	Bits 7 – 0	LSB
Value:	Meaning in Sender Header:	Meaning in Receiver Header:	Increments from 0 to 15, then rolls over to zero again. Used in Sender header to distinguish a retry from a new transmission. Used in Receiver header to indicate which sender TPDU is being ACK'd or NAK'd.	For "First": Number of 256 octet blocks (minus 1) to allocate to hold complete message. For "NAK": Reason code. For others: Set to zero when generating; ignore when receiving.	
0.	Null	ACK			
1.	Only	NAK			
2.	First	Not Initialised			
3.	Middle	Undefined			
4.	Last	Undefined			
5. to 15 are undefined					

The Command identifies the use of the TPDU by the transport. The Command field is defined differently for the Sender and the Receiver header.

Sender header commands :

Null = no data in this Transport PDU

Only = this is the Transport PDU conveying all of the Application data

First / Middle / Last = position of this Transport PDU within the segmented Application data

The Sequence Number is used in the Sender header to distinguish a retry from a new transmission. It is used in Receiver header to indicate which sender TPDU is being ACK'd or NAK'd. This is **NOT** used for applications request/response matching. This is only used by the transport state machines.

The Data is used by one Sender command and one Receiver command. It is used in the *First* Sender command to indicate the number of 256 octet blocks (minus 1) the Receiver shall allocate for the complete application message. That is, application messages of size 0 to 256 octets shall have the *First* Data field set to 0; 257 to 512, to 1; and so on up to 65536 octets. It is used in the NAK Receiver command to provide a reason code:

Class 4 (the Point-to-Point non-Blocking transport) uses two headers in each TPDU, followed by the application data. The Sender Header is first in the TPDU, the Receiver Header is second, followed by the application data. Class 4 only uses the *Null* and *Only* Sender Commands (numbered 0 and 1).

Class 5 (the Point-to-Point Fragmenting non-Blocking transport) uses two headers in each TPDU, followed by the application data. The Sender Header is first in the TPDU, the Receiver Header is second, followed by the application data. Class 5 uses all of the Sender Commands.

Class 6 (the Multipoint connection Fragmenting transport) uses one header in each TPDU, followed by the application data. Class 6 uses all of the Sender Commands. The Sender Header is used in the Client-to-Server TPDU and the Receiver Header is used in the Server to Client TPDU.

5.1.1.4.7 Exclusive owner O⇒T data format

Exclusive owner connections shall have either of two real-time transfer formats:

- 32-bit header, fixed size;
- no header, variable size.

The 32-bit header prefixed to the real-time data shall be in the form shown in Table 191.

Table 191 – Real-time data header – exclusive owner

Parameter	Format	Size (bits)
Run_idle	RUN_IDLE	1
Reserved	UDINT	31

The *run_idle* flag (bit 0) shall be set (1 = RUN) to indicate that the following data shall be sent to the target application. It shall be clear (0 = IDLE) to indicate that the idle event shall be sent to the target application. The reserved field (bits 1-31) shall be reserved and set to 0.

If the *no_header* transfer format is used, the reception of a packet with data beyond the transport header shall indicate the RUN mode. If the packet is truncated after the transport header, the target shall be sent an idle event.

5.1.1.4.8 Redundant owner O⇒T data format

5.1.1.4.8.1 General format

Redundant owner shall have a 32-bit header prefixed to the real-time data. This header shall be in the form shown in Table 192.

Table 192 – Real-time data header– redundant owner

Parameter	Format	Size (bits)
Run_idle	RUN_IDLE	1
COO	BOOLEAN	1
ROO	USINT	2
Reserved	UDINT	28

5.1.1.4.8.2 Run_idle flag

The run_idle flag (bit 0) shall have the same meaning as it does in an exclusive owner connection and shall be one of RUN or IDLE. The reserved field (bits 4-31) shall be reserved and set to 0.

5.1.1.4.8.3 Claim output ownership (COO) flag

The COO flag shall be set (1) when an originator application wants its connection to be the owning connection of the target application. The COO flag shall be reset (0) when an originator application does not want its connection to be the owning connection of the target application. When the owning connection resets (0) its COO flag, its sibling connections shall be checked for a set (1) COO flag. The new owner shall be any of the connections that have their COO flag set.

NOTE This results in undefined behaviour if more than one other connection has its COO flag set.

5.1.1.4.8.4 Ready for ownership of outputs (ROO) priority value

The ROO priority value shall be non-zero when an originator application does not want to force its connection to be the owning connection of the target application, but is ready to be the owning connection should there be no originator applications claiming to be the owning connection. The ROO priority value shall be zero when the originator application does not want to be the owning connection of the target connection and is not to be the owning connection should there be no originator application claiming to be the owning connection. The ROO priority value shall be used only when the COO flag is reset.

The value of the ROO field can range from 0 to 3. The originator applications shall each determine a unique non-zero ROO value.

5.1.1.4.8.5 Determining the owning connection

The originator applications shall determine among themselves which originator application has the owning connection. The owning connection shall be determined by the originator application that sets its COO flag. In situations where multiple originator applications have their COO flag set or where no originator connections have their COO flag set, the following rules shall be applied by the target transport to determine the owning connection.

- There shall be no owning connection until an originator application sends a real-time packet with the COO flag set;
- If there is only one originator application which had the COO flag set in its last real-time packet, that originator application shall have the owning connection;
- If there are multiple originator applications which had the COO flag set in its last real-time packet, the last originator application that transitioned its COO flag from reset to set shall have the owning connection.

- If the originator application with the owning connection resets its COO flag, closes its connection, or if that connection times out, and no other originator applications have their COO flags set, the originator application with the highest non-zero ROO priority value shall have the owning connection.
- If all of the originator applications have their COO flags reset and ROO priority values set to zero, there shall be no owning connection.
- When the first real-time packet containing a set COO flag is received by the target transport, the originator application that sent the real-time packet shall have the owning connection.

5.1.1.4.8.6 Transporting events and data to a target application

The connection related events from each of the redundant owner connections shall be combined using the following rules such that the target application sees only a single exclusive owner connection:

- Until an owning connection is initially determined, the transport shall not indicate real-time data reception to the target application;
- If an owning connection is determined, the transport shall indicate the event consistent with the owning connections real-time data to the target application. If the Run/Idle flag is reset in the real-time data for the owning connection, the transport shall indicate the idle state to the target application. If the Run/Idle flag is set in the real-time data for the owning connection, the transport shall indicate the run state and the real-time data to the target application;
- If an owning connection had been previously determined, but no originator is currently claiming or ready for ownership, the transport shall indicate idle state to the target application;
- If all the redundant connections are closed or have been timed out, the target transport shall indicate the event consistent with the last connection to have been closed or timed out to the target application.

5.1.1.5 CM_PDUs

5.1.1.5.1 General

Connection establishment and maintenance services are provided by the Connection Manager. They are :

CM_Forward_Open (corresponds to Forward_Open and Ex_Forward_Open PDUs)

CM_Forward_Close (corresponds to Forward_Close PDUs)

CM_Unconnected_Send (corresponds to Unconnected_Send PDUs)

CM_Get_Connection_Data (corresponds to Get_Connection_Data PDUs)

CM_Search_Connection_Data (corresponds to Search_Connection_Data PDUs)

CM_Get_Object_Owner (corresponds to Get_Object_Owner PDUs)

5.1.1.5.2 Connection manager

The Connection Manager Object is used to manage the establishment and maintenance of communication connections.

The Connection Manager objects at different nodes shall communicate using the UCMM services described in IEC 61158-5. The TPDUs with which peer Connection Managers communicate shall be derived from the services of the Connection Manager.

5.1.1.5.3 Forward_Open

5.1.1.5.3.1 General

The response to the Forward_Open_Request_PDU is a Forward_Open_ResponsePDU. The Forward_Open response shall contain the original connection serial number and owner vendor and serial number and the connection IDs (CID) necessary to complete the connection if successful and error information if the connection failed.

NOTE The Forward_Open request sets up network, transport, and application connections. An application connection consists of a single transport connection, and one or two network connections that are in turn comprised of multiple link connections. Each port segment in the connection path uses a link connection. The Forward_Open service between two devices builds one or two link connections as specified by the network connection parameter and the requested packet intervals (CM_RPI). Since up to two network connections can be required for a single transport connection, they are differentiated by the O⇒T and T⇒O designations; O⇒T means originator to target, and T⇒O means target to originator.

5.1.1.5.3.2 Format of Forward_Open request

The format of the Forward_Open request TPDU shall be of the form shown in Table 193:

Table 193 – Forward_Open request format

Parameter name	Format
priority_and_tick	USINT
connection time-out ticks	USINT
O2T_CID	UDINT
T2O_CID	UDINT
connection_serial_number	UINT
vendor_ID	UINT
originator_serial_number	UDINT
connection timeout multiplier	USINT
reserved[3]	USINT
O2T_CM_RPI	UDINT
O2T_connection_parameters	UINT
T2O_CM_RPI	UDINT
T2O_connection_parameters	UINT
xport_type_and_trigger	USINT
connection_path_size	USINT
connection_path[]	UINT

5.1.1.5.3.3 Format of Forward_Open response if success

If the status parameter of the CM_open_response is zero (no error), the format of the Forward_Open response TPDU shall be of the form shown in Table 194.

Table 194 – Forward_Open_Good response format

Parameter name	Format
O2T_CID	UDINT
T2O_CID	UDINT
connection_serial_number	UINT
vendor_ID	UINT
originator_serial_number	UDINT
O2T_CM_API	UDINT
T2O_CM_API	UDINT
Application_reply_size; (in 16-bit words)	USINT
Reserved	USINT
application_reply[]	UINT

Success shall be returned when the connection requested has been established from this point forward in the path. This response also shall indicate the connection serial number and the actual packet rate of the connection. Once the successful response has been received, the connection shall be open from this point forward in the path. Targets shall wait at least 10 seconds after sending the CM_Forward_Open_Good_Response for the first packet on a connection.

5.1.1.5.3.4 Format of Forward_Open response if failure

If the status parameter of the CM_open_response is not zero (error), the format of the Forward_Open response TPDU shall be of the form shown in Table 195.

Table 195 – Forward_Open_Bad response format

Parameter name	Format
connection_serial_number	UINT
vendor_ID	UINT
originator_serial_number	UDINT
remaining_path_size; (in 16-bit words)	USINT
Reserved	USINT

This format shall be used for all failures of the connection system. The requested connection shall not be established, and the object specific status words shall contain information about the reason for the failure. The remaining_path_size shall contain the length of the path at the point the connection failed. This information can be used to debug the problem.

In the failure response, the remaining remaining_path_size shall be the “pre-stripped” size. This shall be the size of the path when the node first receives the request and has not yet started processing it. A target node may return either the “pre-stripped” size or 0 for the remaining remaining_path_size.

A duplicate Forward_Open service shall be defined as a Forward_Open service whose vendor_ID, connection_serial_number, and originator_serial_number match an existing connection's parameters. If the duplicate Forward_Open service is a null Forward_Open service (defined as the connection type in both the O2T and T2O network connection parameter fields are NULL), then the Forward_Open service shall be forwarded to the application for further processing. Null Forward_Open requests may be used to reconfigure the connection. The Connection Manager in the intermediate nodes need not allocate additional resources for a duplicate Forward_Open request since the resources have already been allocated. If the duplicate Forward_Open request is not NULL, then an general status = 0x01, extended status = 0x0100 shall be returned.

5.1.1.5.4 Ex_Forward_Open

5.1.1.5.4.1 General

The Ex_Forward_Open shall have the same function as the Forward_Open except that it shall allow the establishment of connections larger than 511 octets.

The response to the Ex_Forward_Open_Request_PDU is an Ex_Forward_Open_ResponsePDU. The Ex_Forward_Open response shall contain the original connection serial number and owner vendor and serial number and the connection IDs (CID) necessary to complete the connection if successful and error information if the connection failed.

NOTE The Ex_Forward_Open request is very similar to the Forward_Open request.

5.1.1.5.4.2 Format of Ex_Forward_Open request

The format of the Ex_Forward_Open request TPDU shall be of the form shown in Table 196:

Table 196 – Ex_Forward_Open request format

Parameter name	Format
priority_and_tick	USINT
connection time-out ticks	USINT
O2T_CID	UDINT
T2O_CID	UDINT
connection_serial_number	UINT
vendor_ID	UINT
originator_serial_number	UDINT
connection timeout multiplier	USINT
reserved[3]	USINT
O2T_CM_RPI	UDINT
O2T_ex_connection_size	UINT
O2T_connection_parameters	UINT
T2O_CM_RPI	UDINT
T2O_ex_connection_size	UINT
T2O_connection_parameters	UINT
xport_type_and_trigger	USINT
connection_path_size	USINT
connection_path[]	UINT

The connection size field in the two connection_parameters shall be reserved in this case and set to zero. The ex_connection_size parameters shall be used instead to specify the maximum size of the connection (up to 65535).

5.1.1.5.4.3 Format of Ex_Forward_Open response if success

If the status parameter of the CM_open_response is zero (no error), the format of the Ex_Forward_Open response TPDU shall be of the form shown in Table 197.

Table 197 – Ex_Forward_Open_Good response format

Parameter name	Format
O2T_CID	UDINT
T2O_CID	UDINT
connection_serial_number	UINT
vendor_ID	UINT
originator_serial_number	UDINT
O2T_CM_API	UDINT
T2O_CM_API	UDINT
Application_reply_size; (in 16-bit words)	USINT
Reserved	USINT
application_reply[]	UINT

Success shall be returned when the connection requested has been established from this point forward in the path. This response also shall indicate the connection serial number and the actual packet rate of the connection. Once the successful response has been received, the connection shall be open from this point forward in the path.

5.1.1.5.4.4 Format of Ex_Forward_Open response if failure

If the status parameter of the CM_open_response is not zero (error), the format of the Ex_Forward_Open response TPDU shall be of the form shown in Table 198.

Table 198 – Ex_Forward_Open_Bad response format

Parameter name	Format
connection_serial_number	UINT
vendor_ID	UINT
originator_serial_number	UDINT
remaining_path_size; (in 16-bit words)	USINT
Reserved	USINT

This format shall be used for all failures of the connection system. The requested connection shall not be established, and the object specific status words shall contain information about the reason for the failure. The remaining_path_size shall contain the length of the path at the point the connection failed. This information can be used to debug the problem.

In the failure response, the remaining remaining_path_size shall be the “pre-stripped” size. This shall be the size of the path when the node first receives the request and has not yet started processing it. A target node may return either the “pre-stripped” size or 0 for the remaining remaining_path_size.

5.1.1.5.5 Forward_Close

5.1.1.5.5.1 General

The Forward_Close request shall remove a connection from all the nodes participating in the original connection. The Forward_Close shall be sent between Connection Managers as specified in the connection_path. The Forward_Close request shall cause all resources in all nodes participating in the connection to be deallocated, including connection IDs, link transmit time, and internal memory buffers.

If an intermediate node cannot find the connection that is to be closed (it may have timed out at the node), the Forward_Close request shall still be forwarded to downstream nodes or the target application (via the CM_close_indication).

NOTE The Forward_Close is always forwarded to allow downstream nodes or the target application to release any resources that were allocated for the connection.

5.1.1.5.5.2 Format of Forward_Close request

The format of the Forward_Close request shall be of the form shown in Table 199.

Table 199 – Forward_Close request format

Parameter name	Format
connection_priority/tick time	USINT
connection_timeout	USINT
connection_serial_number	UINT
vendor_ID	UINT
originator_serial_number	UDINT
connection_path_size	USINT
Reserved	USINT
connection_path[]	UINT

5.1.1.5.5.3 Format of Forward_Close response if success

The Forward_Close response shall be sent between Connection Managers in response to a Forward_Close request, and shall contain the status of the close request. If the status parameter of the CM_close_response is zero (no error), the format of the Forward_Close response TPDU shall be of the form shown in Table 200.

Table 200 – Forward_Close_Good response format

Parameter name	Format
connection serial number	UINT
vendor ID	UINT
originator serial number	UDINT
application_reply_size; in 16-bit words	USINT
Reserved	USINT
application_reply[]	UINT

A successful Forward_Close response shall be returned when the close has been acknowledged by all the nodes from this point in the path to the end of the path. The response shall indicate the connection_serial_number, vendor_ID, and originator_serial_number of the closed connection. Once the response indicating success has been received, the connection shall be closed from this point in the path to the end. The target application may optionally pass back application specific information in the successful close response. If no application_reply information is contained in the service the application_reply_size shall be zero.

5.1.1.5.5.4 Format of Forward_Close response if failure

If the status parameter of the CM_close_response is not zero (error), the format of the Forward_Close response shall be of the form shown in Table 201.

Table 201 – Forward_Close_Bad response format

Parameter name	Format
connection_serial_number	UINT
vendor_ID	UINT
originator_serial_number	UDINT
remaining_path_size	USINT
reserved	USINT

The object specific status words shall contain information about the reason for the failure. The remaining_path_size shall contain the path size from the point at which the close connection failed.

In the failure response, the remaining_path_size shall be the “pre-stripped” size. This shall be the size of the path when the node first receives the request and has not yet started processing it. A target node shall return either the “pre-stripped” size or 0 for the remaining_path_size.

5.1.1.5.6 Unconnected_Send

5.1.1.5.6.1 Format of Unconnected_Send request

The Unconnected_Send service shall allow an application to send a message to a device without first setting up a connection. The Unconnected_Send service shall use the Connection Manager object in each intermediate node to forward the message and to remember the return path. The UCMM of each link shall be used to forward the request from Connection Manager to Connection Manager just as it is for the Forward_Open service; however, no connection shall be built. The Unconnected_Send service shall be sent to the local Connection Manager and shall be sent between intermediate nodes. When an intermediate node removes the last port segment, the message shall be formatted as a UCMM message and sent to the port and link address of the last segment.

NOTE The target node never sees the Unconnected_Send service but only a standard message arriving via the UCMM.

If the message contains an odd number of octets, a pad octet shall be appended to the end of the message allowing the remaining fields to be aligned on a 16-bit boundary. The format of the Unconnected_Send request shall be of the form of the form shown in Table 202.

Table 202 – Unconnected_Send request format

Parameter name	Format
priority/tick time	USINT
conn time-out ticks	USINT
Message_size; in octets, not including pad	UINT
Message Router PDU ; if message_router_PDU is odd size, include pad octet	MR_Request
pad	USINT
path_size	USINT
Reserved	USINT
path[]	UINT

The message_size shall be the size of the message_router_PDU in octets without the pad. The reserved field shall be set to zero. The path_size shall be the number of 16-bit words in the path field.

5.1.1.5.6.2 Unconnected_Send response

The Unconnected_Send response shall be generated by the last intermediate node from the UCMM response generated by the target node or by an intermediate node as the result of a UCMM time-out, a problem with the embedded message, or a problem with the Unconnected Service Request itself. The packet shall be routed from intermediate node to intermediate node using the information stored when the Unconnected_Send request was processed. The response shall contain a header with status information about the request and a variable length response generated by the target node.

5.1.1.5.6.3 Format of Unconnected_Send response if success

This format shall be used when an unconnected message response is received. The application response shall be the format of the Message response from the target and shall contain error codes that resulted from the execution of the message by the application at the target node. The structure of the response is shown in Table 203.

Table 203 – Unconnected_Send_Good response format

Parameter name	Format
application_reply []	USINT

5.1.1.5.6.4 Format of Unconnected_Send response if failure

This format shall be used if the originating node or any intermediate node found a problem in the message format, message parameters, or a UCMM timed-out due to no acknowledge or no response received. A UCMM time-out may be the result of

- The target device not being connected or powered up. (link specific time-out);
- An intermediate device not being connected or powered up. (link specific time-out);
- A target device being out of UCMM buffers or overloaded (Flow Control). (link specific time-out);
- An intermediate device being out of UCMM buffers or overloaded (Flow Control). (link specific time-out);
- An invalid path segment type (logical or network segment);
- An invalid port segment address (link or port address invalid).

Since the Unconnected message system does not implement any flow control, the failure of an Unconnected_Send message need not mean a device is not present in the system. The format of the Unconnected_Send response service for a failure shall be of the form shown in Table 204.

Table 204 – Unconnected_Send_Bad response format

Parameter name	Format
remaining_path_size	USINT
pad	USINT

In the failure response, the remaining Connection Path Size shall be the “pre-stripped” size. This shall be the size of the path when the node first receives the request and has not yet started processing it. A target node may return either the “pre-stripped” size or 0 for the remaining Connection Path Size.

5.1.1.5.7 Get_Connection_Data

5.1.1.5.7.1 General

This service shall return the parameters associated with a specified connection number. The connection number may be different from device to device even for the same connection. The connection number corresponds to the offset into the Connection Manager attribute that enumerates the status of the connections.

NOTE This service may be used for network diagnostics.

5.1.1.5.7.2 Format of Get_Connection_Data request

The format of the Get_Connection_Data request shall be of the form shown in Table 205.

Table 205 – Get_Connection_Data request format

Parameter name	Format
connection_number	UINT

5.1.1.5.7.3 Format of Get_Connection_Data response

The format of the Get_Connection_Data response TPDU shall be of the form shown in Table 206.

Table 206 – Get_Connection_Data response format

Parameter name	Format
connection_number	UINT
connection_state	UINT
originator_port	UINT
target_port	UINT
connection_serial_number	UINT
vendor_ID	UINT
serial_number	UDINT
originator_O2T_CID	UDINT
target_O2T_CID	UDINT
O2T_connection timeout multiplier	USINT
reserved1[3]	USINT
originator_O2T_CM_RPI	UDINT
originator_O2T_CM_API	UDINT
originator_T2O_CID	UDINT
target_T2O_CID	UDINT
T2O_connection timeout multiplier	USINT
reserved2[3]	USINT
originator_T2O_CM_RPI	UDINT
originator_T2O_CM_API	UDINT

5.1.1.5.8 Search_Connection_Data

5.1.1.5.8.1 General

This service shall return the parameters associated with a specified connection within a device identified by vendor and serial number .

NOTE This service may be used for network diagnostics.

5.1.1.5.8.2 Format of Search_Connection_Data request

The format of the Search_Connection_Data request shall be of the form shown in Table 207.

Table 207 – Search_Connection_Data request format

Parameter name	Format
connection_serial_number	UINT
vendor_ID	UINT
serial_number	UDINT

5.1.1.5.8.3 Format of Search_Connection_Data response

The format of the Search_Connection_Data response shall be the same as the response from the Get_Connection_Data service (see 5.1.1.5.7).

5.1.1.5.9 Get_Object_Owner

5.1.1.5.9.1 General

The Get_Object_Owner service of the Connection Manager shall return data about the connection(s) that own(s) a particular object. It shall be implemented in any device that accepts redundant connections.

5.1.1.5.9.2 Format of Get_Object_Owner request

The format of the Get_Object_Owner request TPDU shall be of the form shown in Table 208:

Table 208 – Get_Object_Owner request format

Parameter name	Format
reserved	USINT
path_size	USINT
path[]	UINT

The reserved field shall be set to zero.

5.1.1.5.9.3 Format of Get_Object_Owner response

The format of the Get_Object_Owner response TPDU shall be of the form shown in Table 209.

Table 209 – Forward_Open_Good response format

Parameter name	Format
number_of_connections	USINT
number_claiming_ownership	USINT
number_ready_for_ownership	USINT
last_action	USINT
connection_serial_number	UINT
originator_vendor_ID	UINT
originator_serial_number	UDINT

5.1.1.6 CM PDU components

5.1.1.6.1 Network connection parameters

5.1.1.6.1.1 Format

Network connection parameters shall be provided as a single 16-bit word that contains five fields, shown in Figure 44. The fields within the 16-bit word shall indicate

- the type of network connection desired;
- whether the buffer size is variable or fixed;
- the priority of the connection;
- the size of the connection buffer required.

The size of the connection buffer shall include any transport header.

Network Connection Parameters

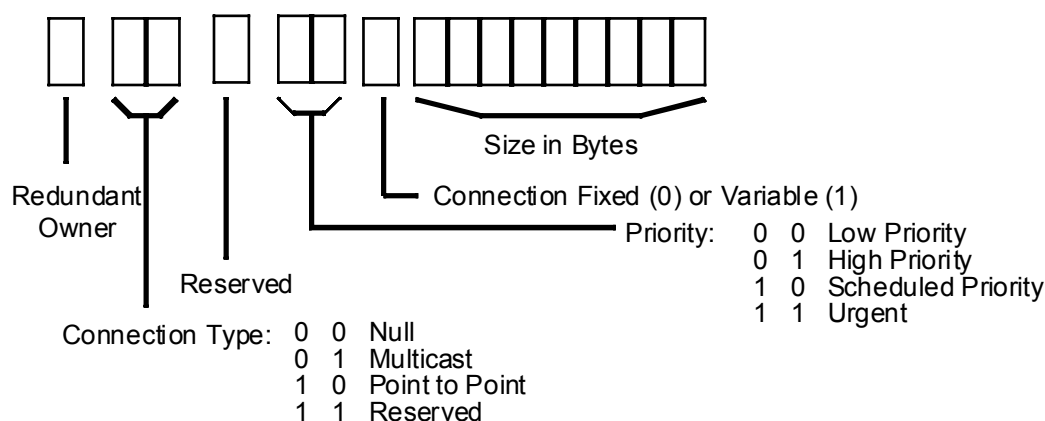


Figure 44– Network connection parameters

5.1.1.6.1.2 Connection type

The connection type shall be one of NULL, MULTIPOINT, or POINT2POINT. The NULL type shall indicate that no network connection is required, while MULTIPOINT and POINT2POINT refer to the network connection types as defined in IEC 61158-5. The NULL connection type may be used to reconfigure a connection.

5.1.1.6.1.3 Priority

Priority shall be one of LOW, HIGH, and SCHEDULED as specified in IEC 61158-4, Type 2.

NOTE SCHEDULED data is prioritised on a link wide basis; however, the two unscheduled priorities (LOW and HIGH) are arbitrated independently at each node. One node may transmit a LOW priority packet while another node has HIGH priority data to send.

5.1.1.6.1.4 Connection fixed/variable

The connection can be set up to use fixed or variable sized connections. With a fixed size connection, each transmission on the connection shall use the same size buffer. Otherwise, either a buffer overrun or underrun condition shall occur. With a variable sized connection, each transmission on the connection may send a variable amount of data up to the maximum size, which shall be specified when the connection is opened. Buffer underruns shall not be reported, but a buffer overrun can still occur if too much data is sent.

5.1.1.6.1.5 Connection size

The network connection size shall be the size of the buffer required for the connection, which includes the data and any transport header. For a variable sized connection, the size shall be the maximum size of the buffer for any transfer. The actual size of the transfer for a variable connection shall be equal to or less than the size specified for the network connection. The maximum buffer size shall be dependent on the links that the connection traverses.

5.1.1.6.1.6 Redundant owner

The redundant owner bit in the O⇒T direction shall be set (= 1) to indicate that more than one owner may be permitted to make a connection simultaneously. The bit shall be clear (= 0) to indicate an exclusive-owner, input only or listen-only connection.

5.1.1.6.1.7 Reserved parameters

Reserved bits shall be clear (= 0).

5.1.1.6.2 Packet interval

5.1.1.6.2.1 Requested packet interval (CM_RPI)

The requested packet interval shall be the requested time between packets in microseconds. The format of the CM_RPI shall be a 32-bit integer in microseconds.

5.1.1.6.2.2 Actual packet interval (CM_API)

The actual packet interval shall be the actual time between packets in microseconds. The format of the CM_API shall be a 32-bit integer in microseconds.

5.1.1.6.2.3 Usage

The requested packet interval shall be the time between packets requested by the receiving device. The value shall be used to allocate link transmit time at each of the producing nodes. The allocation of link transmit time may have to be adjusted when the actual packet rate or actual packet interval is returned, since it is possible for the two values to differ. The path time-out value at each of the intermediate and target nodes shall also be set to the connection time-out multiplier times the CM_API. The CM_RPI is therefore required for all connections.

5.1.1.6.2.4 Function of priorities

Scheduled Priority

For scheduled priority, the CM_RPI shall be the packet rate of the repetitive data. On links that support allocation of link transmit time, link transmit time shall be reserved for this packet. For scheduled priority, the data shall also be restricted to the specified packet rate, which means that if data arrives at an intermediate node faster than the specified packet rate, the node shall filter the packets to the specified rate. Since each node's scheduled priority update rate is in discrete quanta, the Actual Packet Interval (CM_API) may be smaller (more rapid) than the CM_RPI. The path time-out value shall be set to the Connection Time-out multiplier times the CM_API.

High Priority

For high priority, the CM_RPI shall be used to set the path time-out in the intermediate and target nodes. The CM_RPI shall therefore be set to the slowest packet rate expected, which shall preclude having the connection close due to a path time-out. The longer the path time-out, the longer the time required to reclaim resources in the intermediate nodes as a result of faults in the network. Since the high priority is not quantised at any of the nodes, the CM_API shall equal the CM_RPI. To maintain consistency, however, the time-out value shall again be set to the Connection Time-out multiplier times the CM_API.

Low Priority

For low priority, the CM_RPI shall be used to set the path time-out in the intermediate and target nodes. The CM_RPI shall therefore be set to the slowest packet rate expected, which shall preclude having the connection close due to a path time-out. The longer the path time-out, the longer the time required to reclaim resources in the intermediate nodes as a result of faults in the network. Since the low priority is not quantised at any of the nodes, the CM_API shall equal the CM_RPI. To maintain consistency, however, the time-out value shall again be set to the Connection Time-out multiplier times the CM_API.

5.1.1.6.3 Connection time-out multiplier

The Connection Time-out Multiplier specifies the multiplier applied to the CM_RPI to obtain the connection time-out value. Device shall stop transmitting on a connection whenever the connection times out even if the pending close has been sent. The multiplier shall be as represented by Table 210.

Table 210 – Time-out multiplier

Value	Multiplier	Notes
0	X 4	Default
1	X 8	
2	X 16	
3	X 32	
4	X 64	
5	X 128	
6	X 256	
7	X 512	
8 – 255	reserved	

5.1.1.6.4 Connection request priority and tick time

Two values are packed into this field. The connection request priority determines the priority of the UCMM messages used to establish the connection and has no correlation with the priority of the connection being made. Although Low priority is used for most connection establishments, High priority may be used for those special circumstances which require a connection be made quickly. The use of low priority was chosen to avoid congestion with time-critical messaging. UCMM messages can not use scheduled priority.

The tick time shall be used in conjunction with the connection request time-out ticks value. This value determines the time between "ticks" in the latter field. Thus, if the tick time is 1ms., then a connection request time-out value of 5 would translate into 5 ms. If the tick time was 2 ms, then a connection request time-out value of 5 would translate into 10 ms.

The format of the Connection Request Priority/Tick Time shall be as shown in Figure 45.

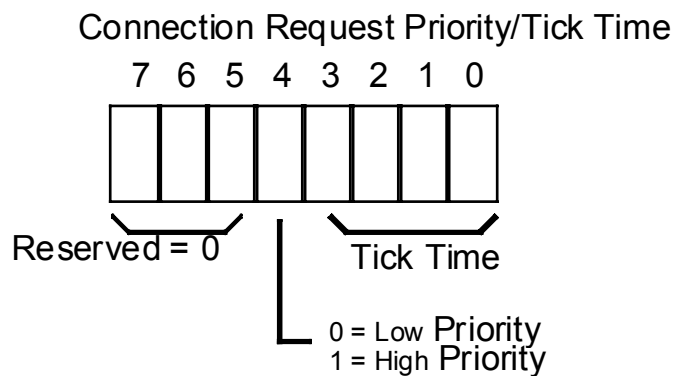
**Figure 45 – Time tick**

Table 211 shall determine the time between ticks:

Table 211 – Time tick units

Tick time		
Tick time (binary)	Time per tick	Max time
0000	1 ms	255 ms
0001	2 ms	510 ms
0010	4 ms	1 020 ms
0011	8 ms	2 040 ms
0100	16 ms	4 080 ms
0101	32 ms	8 160 ms
0110	64 ms	16 320 ms
0111	128 ms	32 640 ms
1000	256 ms	65 280 ms
1001	512 ms	130 560 ms
1010	1 024 ms	261 120 ms
1011	2 048 ms	522 240 ms
1100	4 096 ms	1 044 480 ms
1101	8 192 ms	2 088 960 ms
1110	16 384 ms	4 177 920 ms
1111	32 768 ms	8 355 840 ms

5.1.1.6.5 Connection request time-out ticks

The connection request time-out ticks shall be used to specify the amount of time the originating application shall wait for the connection to be established. Each hop of the connection has a link specific time-out value implemented by the UCMM, and after the automatic retries have been exhausted without an acknowledge the UCMM shall generate a time-out error. Therefore if a device in the path shall be either not present or too busy to acknowledge a UCMM request, a UCMM time-out shall occur as fast as would be possible. The only time the connection request time-out period would be encountered shall be if the target node acknowledges the connection request but fails to respond or if the connection request shall be delivered but a failure in the network prevents the response from being returned. The connection request time-out shall be not the path time-out value derived from the CM_RPI and used to close a connection after it shall be established. The connection request time-out shall be specified by a combination of the connection request time-out ticks multiplied by the time per tick which shall be determined by the tick time value.

Since a particular connection request time-out can be expressed in several different formats, the rule shall be to **always pack it such that the tick time value shall be minimised**. This allows the best granularity for decreasing the time-out as the message is passed through hops. This also means that the tick time value may change as the message is passed through multiple hops. As an example, 300 ms may be encoded as:

1. 0001 10010110 -> Time per Tick = 2 ms, Ticks = 150 (right)

2. 0010 01001011 -> Time per Tick = 4 ms, Ticks = 75 (wrong)

The connection request time-out value shall be dependent on the number of port segments in the connection path, the processing time at the target node, and the congestion of the intermediate networks. The connection request time-out can be a relatively large value since the full connection request time-out shall only occur in a few rare cases. The only time the full connection request time-out would occur would be if:

- the connection request was successfully delivered to the target node and the node failed to generate a response. This would usually be caused by a firmware error in the device.
- the connection request was successfully delivered to the target node and the response was discarded due to congestion at an intermediate node.

- the connection request was successfully delivered to the target node and the target node or one of the intermediate nodes was removed or powered down before the reply was returned.

The UCMM shall be used to forward the connection request to the next node in the path. The UCMM sends the request and then waits a link specific time for an acknowledgement or reply, if no acknowledgement or reply shall be received before the link specific time-out, the request shall be retried. After a link specific number of retries (typically three), the UCMM shall signal a time-out on the connection request. The Connection manager which generated the request shall return an error reply to the connection request. Therefore the connection request shall time-out as soon as possible for the most common system faults

- missing or powered down devices in the path;
- congestion at a device in the path.

To aid in debugging systems, each hop in the path shall decrement the time-out by about 80 ms before sending it out in the Forward_Open to the next device in the path. Each intermediate device (router) shall round the connection time-out value that shall be decremented such that a minimum of 1 tick shall be subtracted. This shall enable the last device before the failure to time-out first and return information about how far the connection was made before the failure. This information can be used to help in debugging problems in a system. A typical system function is shown in Figure 46.

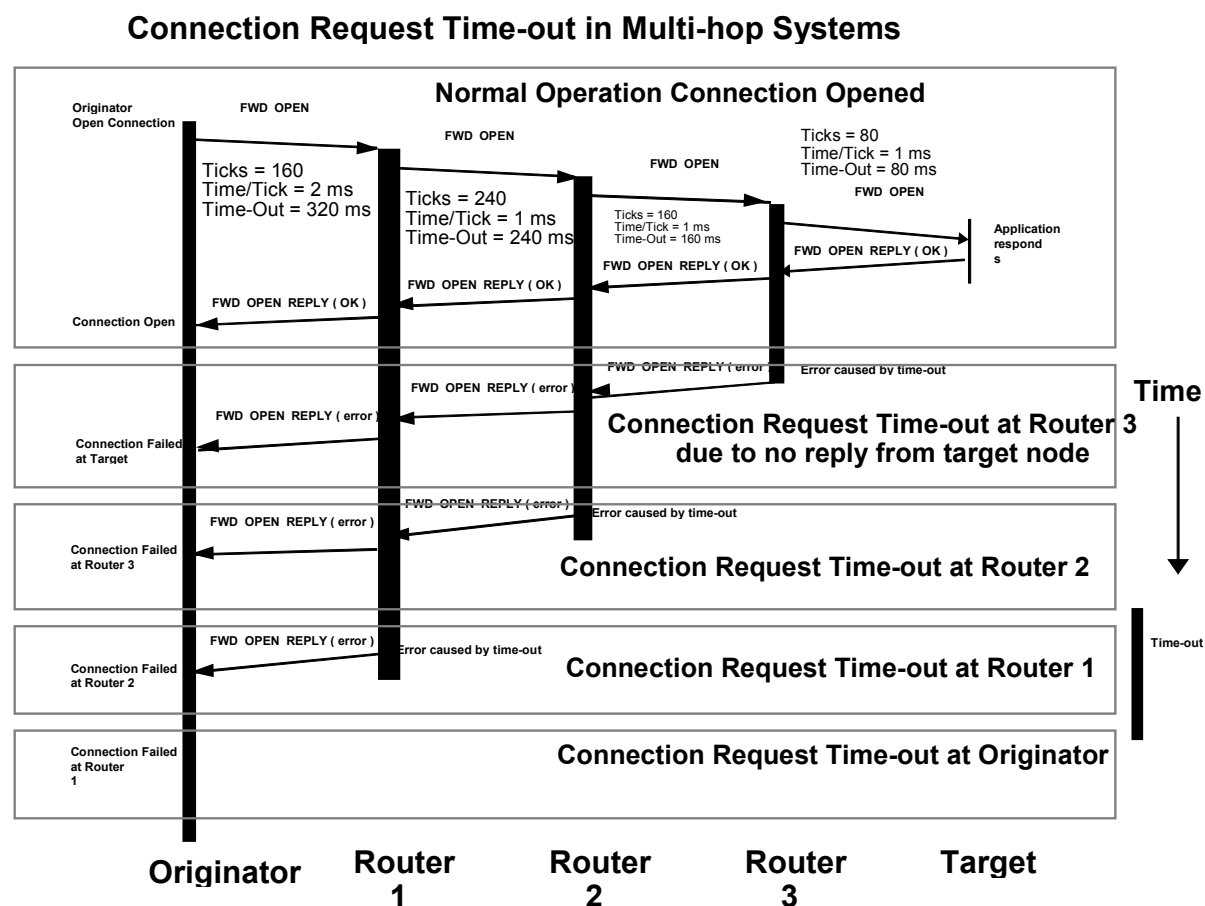


Figure 46 – Connection establishment time-out

In Figure 46 a connection is requested over a path containing 4 hops. The Connection request Time-out is specified as 320 ms by the originator of the connection. In the first case the connection is completed and the calculated time-out values are shown at each hop in the path. The second case is where the last device receives an acknowledgement but does not receive a response and times out. It returns a time-out error to the originator. Since the time-out value decreases for each hop, the time-out error shall be returned to the originator before the originator times out.

The other cases show time-outs at the other devices, and in each case the time-out error shall be returned to the originator before the originator itself times out. The last case is where no response is received by the originator and the connection open process is timed out. The reason to decrease the time-out at each node is so additional error information indicating where in the path the time-out occurred can be sent. Remember that if any of the devices do not receive an acknowledgement, the connection request shall immediately be timed out and an error reply shall be generated.

5.1.1.6.6 Connection serial number

The connection serial number shall be a unique 16-bit value selected by the Connection Manager at the originator of the connection. The originator shall make sure that the 16-bit value is unique for the device. There shall be no other significance placed on the number by any other nodes in the connection path. The connection serial numbers shall be unique but do not have to be sequential. For example, an operator interface may have a large number of connections open at the same time, each with a unique number. The same values could be repeated at other operator interface stations. A possible implementation would be to have a connection list which points to the descriptor for each connection, and the connection serial number could be the index into the table.

5.1.1.6.7 Vendor ID

The vendor number shall be a unique number assigned to the various vendors of products. Each vendor has a unique number assigned.

5.1.1.6.8 Originator serial number

The originator serial number shall be a unique 32-bit value that is assigned to a device at the time of manufacture. This value shall be guaranteed to be unique for all devices manufactured by the same vendor. No significance shall be attached to the number. The combination of Vendor ID and Originator Serial Number shall be unique throughout the entire system.

5.1.1.6.9 Connection number

The connection number shall be a 16-bit value that is assigned by the Connection Manager when a connection is opened. This value allows other nodes to obtain connection data from the Connection Manager. This number shall not be confused with the Connection Serial Number.

5.1.1.6.10 Connection path size

The connection path size shall be the length of the connection path in 16-bit words. The length of the connection path varies during the connection process, since each node in the connection path removes the current port segment and forwards only the remaining path segments to the next node.

5.1.1.6.11 Connection path

Refer to Path description in 5.1.1.9.

5.1.1.6.12 Connection remaining path

Refer to Path description in 5.1.1.9.

5.1.1.6.13 Network connection ID

The Network Connection ID shall be a 32-bit value built as follows :

- Bits 31-24 shall be reserved and shall be set to zero;
- Bits 23-16 shall contain a node MAC ID depending on selected connection type
- Bits 15-0 shall contain the 16-bit connection number.

Table 212 shows options for selection of Connection ID.

Table 212 – Selection of connection ID

Connection type	MAC ID	Connection number
Multipoint	MAC of Producer	Unique by Device
Point to Point	MAC of Consumer	Unique by Device

The Network Connection ID shall be link specific and shall not be related to the connection serial number, which is connection specific and the same over all the links. The fields of the Network Connection ID shall be used to set the screening mechanism for the specified link.

The generic tag for the Lpackets exchanged over a connection and described in IEC 61158-4, Type 2, shall be built as follows, based on the contents of the CID fields :

- the first transmitted octet of the generic tag shall contain the MAC ID
- the second transmitted octet of the generic tag shall contain the least significant octet of the connection number
- the third transmitted octet of the generic tag shall contain the most significant octet of the connection number.

To guarantee that connection IDs are unique on a link the following rules shall be used when choosing CIDs:

- the producer shall choose the CID for a multipoint transport;
- the consumer shall choose the CID for a point-to-point transport;
- on a ControlNet link, the node responsible for the choice of the CID (either producer or consumer) shall insert its MAC ID into bits 23-16 of the CID in the Forward_Open;
- a multipoint CID shall not be reused until all connections associated with the CID have been closed or timed out;
- receiving an I'm alive fixed tag packet from a node shall immediately close any connections with that device.

5.1.1.6.14 Transport class and trigger

The transport class and trigger specify the type of transport required for the connection. This information shall not be used by the Connection Manager but passed on to the application. The application shall determine if the transport type is supported and if an instance of the required transport is available. The octet that encode the transport class and trigger shall be of the following form shown in Table 213.

Table 213 – Transport class, trigger and Is_Server format

TransportClass (4 bits)	TriggerMode (3 bits)	Is_Server (1 bit)
NULL = 0 DUPLICATE_DETECT = 1 ACKNOWLEDGED = 2 VERIFIED = 3 NONBLOCKING = 4 FRAGMENTING = 5 MULTIPOINT_FRAG = 6	CYCLIC = 0 CHANGE_OF_STATE = 1 APPLICATION = 2	IS_NOT_SERVER = 0 IS_SERVER = 1

The least significant 4 bits, class, shall determine which transport type is specified and shall be in the range 0 to 6. The class values 7 through 15 shall be reserved. Bits 4-6, called trigger, shall determine what event causes the production of data on the connection and shall be in the range 0 to 2. The trigger values 3 through 7 shall be reserved. For transports that need to specify the target as a server, the is_server field shall be 1; otherwise, it shall be 0. For transport classes 0 and 1, the is_server field shall be ignored. Bit 7 shall be the is_server field;

See IEC 61158-5 for details on the behaviour for the different combinations of is_server, class, and trigger, CM_RPI, and CM_API.

5.1.1.7 MR headers

All request packets delivered to the Message Router, either from a connection or from the UCMM, shall have a header of the form shown in Table 214.

Table 214 – MR_Request_Header format

Parameter	Format
Service	USINT
path_size	USINT
path[]	UINT

The first octet, service, shall be directly delivered to the application object and shall be in the range 1 through 127. The path_size shall determine the number of 16-bit words in the path array. The path shall contain addressing information for the packet and shall be padded.

Response packets from the Message Router shall have a header of the form shown in Table 215.

Table 215 – MR_Response_Header format

Parameter	Format
Service	USINT
Reserved	USINT
general_status	USINT
Extended_status_size	USINT
Extended_status[]	UINT

The service field shall be the value of the MR_Request_Header.service field with its most significant bit set. If MR_Request_Header.service equals 0x05, the MR_Response_Header.service shall equal 0x85. The general_status field shall be filled using the Status Code parameter of the service response. Likewise the extended_status, extended_status_size, and response shall be filled using the Extended Status and other response parameters from the service response, respectively. The response_size shall not be explicitly included in the MR_Response_Header; it shall be implied by number of octets in the MR_Response_Header.

5.1.1.8 OM_Service_PDU**5.1.1.8.1 General syntax****5.1.1.8.1.1 Get_Attribute_All**

The Get_Attribute_All_RequestPDU body is empty.

The Get_Attribute_All_ResponsePDU body shall be as specified in Table 216.

Table 216 – Structure of Get_Attribute_All_ResponsePDU body

Name	Data type
Attribute Data	Object/class attribute-specific

5.1.1.8.1.2 Set_Attribute_All

The Set_Attribute_All_RequestPDU body shall be as specified in Table 217.

Table 217 – Structure of Set_Attribute_All_RequestPDU body

Name	Data type
Attribute Data	Object/class attribute-specific

The Get_Attribute_All_ResponsePDU body is empty.

5.1.1.8.1.3 Get_Attribute_List

The Get_Attribute_List_RequestPDU body shall be as specified in Table 218.

Table 218 – Structure of Get_Attribute_List_RequestPDU body

Name	Data type
Attribute Count	UINT
Attribute List	Array of UINT's

The Get_Attribute_List_ResponsePDU body shall be as specified in Table 219.

Table 219 – Structure of Get_Attribute_List_ResponsePDU body

Name	Data type	Semantics of values
Attribute Count	UINT	
Attribute Data	Array of STRUCT	
Attribute Identifier	UINT	
Attribute Status	UINT	Values 0-255 are reserved to mirror common service status codes. Values 256-65535 are available for object/class attribute-specific errors.
Attribute Value	Object/class attribute- specific	

5.1.1.8.1.4 Set_Attribute_List

The Set_Attribute_List_RequestPDU body shall be as specified in Table 220.

Table 220 – Structure of Set_Attribute_List_RequestPDU body

Name	Data type
Attribute Count	UINT
Attribute Data	Array of STRUCT
Attribute Identifier	UINT
Attribute Value	Object/class attribute- specific

The Set_Attribute_List_ResponsePDU body shall be as specified in Table 221.

Table 221 – Structure of Set_Attribute_List_ResponsePDU body

Name	Data type	Semantics of values
Attribute Count	UINT	
Attribute Status List	Array of STRUCT	
Attribute Identifier	UINT	
Attribute Status	UINT	Values 0-255 are reserved to mirror common service status codes. Values 256-65535 are available for object/class attribute-specific errors.

5.1.1.8.1.5 Reset

The Reset_RequestPDU body shall be as specified in Table 222, if the **optional** “Object Specific Data” service parameter of the Reset request is specified. Else it shall be empty.

Table 222 – Structure of Reset_RequestPDU body

Name	Data type
Object Specific Data	Object/class service- specific

The Reset_ResponsePDU body shall be as specified in Table 223, if the **optional** “Object Specific Data” service parameter of the Reset response is specified. Else it shall be empty.

Table 223 – Structure of Reset_ResponsePDU body

Name	Data type
Object Specific Data	Object/class service- specific

5.1.1.8.1.6 Start

The Start_RequestPDU body shall be as specified in Table 224, if the **optional** “Object Specific Data” service parameter of the Start request is specified. Else it shall be empty.

Table 224 – Structure of Start_RequestPDU body

Name	Data type
Object Specific Data	Object/class service- specific

The Start_ResponsePDU body shall be as specified in Table 225, if the **optional** “Object Specific Data” service parameter of the Start response is specified. Else it shall be empty.

Table 225 – Structure of Start_ResponsePDU body

Name	Data type
Object Specific Data	Object/class service- specific

5.1.1.8.1.7 Stop

The Stop_RequestPDU body shall be as specified in Table 226, if the **optional** “Object Specific Data” service parameter of the Stop request is specified. Else it shall be empty.

Table 226 – Structure of Stop_RequestPDU body

Name	Data type
Object Specific Data	Object/class service- specific

The Stop_ResponsePDU body shall be as specified in Table 227, if the **optional** “Object Specific Data” service parameter of the Stop response is specified. Else it shall be empty.

Table 227 – Structure of Stop_ResponsePDU body

Name	Data type
Object Specific Data	Object/class service- specific

5.1.1.8.1.8 Create

The Create_RequestPDU body shall be as specified in Table 228, if the **optional** “Object Specific Data” service parameter of the Create request is specified. Else it shall be empty.

Table 228 – Structure of Create_RequestPDU body

Name	Data type
Object Specific Data	Object/class service- specific

The Create_ResponsePDU body shall be as specified in Table 229, if the **optional** “Object Specific Data” service parameter of the Create response is specified. Else it shall be empty.

Table 229 – Structure of Create_ResponsePDU body

Name	Data type
Instance ID	UINT
Object Specific Data	Object/class service- specific

5.1.1.8.1.9 Delete

The Delete_RequestPDU body shall be as specified in Table 230, if the **optional** “Object Specific Data” service parameter of the Delete request is specified. Else it shall be empty.

Table 230 – Structure of Delete_RequestPDU body

Name	Data type
Object Specific Data	Object/class service- specific

The Delete_ResponsePDU body shall be as specified in Table 231, if the **optional** “Object Specific Data” service parameter of the Delete response is specified. Else it shall be empty.

Table 231 – Structure of Delete_ResponsePDU body

Name	Data type
Object Specific Data	Object/class service- specific

5.1.1.8.1.10 Get_Attribute_Single

The Get_Attribute_Single_RequestPDU body is empty.

The Get_Attribute_Single_ResponsePDU body shall be as specified in Table 232.

Table 232 – Structure of Get_Attribute_Single_ResponsePDU body

Name	Data type
Attribute Data	Object/class attribute-specific

5.1.1.8.1.11 Set_Attribute single

The Set_Attribute_Single_RequestPDU body shall be as specified in Table 233.

Table 233 – Structure of Set_Attribute_Single_RequestPDU body

Name	Data type
Attribute Data	Object/class attribute-specific

The Set_Attribute_Single_ResponsePDU body shall be as specified in Table 234, if the **optional** “Object Specific Data” service parameter of the Set_Attribute_Single response is specified. Else it shall be empty.

Table 234 – Structure of Set_Attribute_Single_ResponsePDU body

Name	Data type
Object Specific Data	Object/class service- specific

5.1.1.8.1.12 Find_Next_Object_Instance

The Find_Next_Object_Instance_RequestPDU body shall be as specified in Table 235.

Table 235 – Structure of Find_Next_Object_Instance_RequestPDU body

Name	Data type
Maximum Returned Values	USINT

The Find_Next_Object_Instance_ResponsePDU body shall be as specified in Table 236.

Table 236 – Structure of Find_Next_Object_Instance_ResponsePDU body

Name	Data type
Number Of List Members	USINT
Instance ID List	Array of UINT

5.1.1.8.1.13 NOP

The NOP_RequestPDU body is empty.

The NOP_ResponsePDU body is empty.

5.1.1.8.1.14 Apply_Attributes

The Apply_Attributes_RequestPDU body shall be as specified in Table 237, if the **optional** “Object Specific Data” service parameter of the Apply_Attributes request is specified. Else it shall be empty.

Table 237 – Structure of Apply_Attributes_RequestPDU body

Name	Data type
Object Specific Data	Object/class service- specific

The Apply_Attributes_ResponsePDU body shall be as specified in Table 238, if the **optional** “Object Specific Data” service parameter of the Apply_Attributes response is specified. Else it shall be empty.

Table 238 – Structure of Apply_Attributes_ResponsePDU body

Name	Data type
Object Specific Data	Object/class service- specific

5.1.1.8.1.15 Save

The Save_RequestPDU body shall be as specified in Table 239, if the **optional** “Object Specific Data” service parameter of the Save request is specified. Else it shall be empty.

Table 239 – Structure of Save_RequestPDU body

Name	Data type
Object Specific Data	Object/class service- specific

The Save_ResponsePDU body shall be as specified in Table 240, if the **optional** “Object Specific Data” service parameter of the Save response is specified. Else it shall be empty.

Table 240 – Structure of Save_ResponsePDU body

Name	Data type
Object Specific Data	Object/class service- specific

5.1.1.8.1.16 Restore

The Restore_RequestPDU body shall be as specified in Table 241, if the **optional** “Object Specific Data” service parameter of the Restore request is specified. Else it shall be empty.

Table 241 – Structure of Restore_RequestPDU body

Name	Data type
Object Specific Data	Object/class service- specific

The Restore_ResponsePDU body shall be as specified in Table 242, if the **optional** “Object Specific Data” service parameter of the Restore response is specified. Else it shall be empty.

Table 242 – Structure of Restore_ResponsePDU body

Name	Data type
Object Specific Data	Object/class service- specific

5.1.1.8.2 Identity object specific syntax elements

5.1.1.8.2.1 Attributes

Class attributes

The format of the Identity Object class attributes shall be as specified Table 243.

Table 243 – Identity object class attributes

Attribute ID	Name	Data type
1	Revision	UINT
2	Max Instance	UINT
6	Max ID Number of Class Attributes	UINT
7	Max ID Number of Instance Attributes	UINT

Instance attributes

The format of the Identity Object instance attributes is as specified in Table 244.

Table 244 – Identity object instance attributes

Attribute ID	Name	Data type	Semantics of values
1	Vendor ID	UINT	The value zero shall not be used.
2	Device Type	UINT	A listing of the Device Type ranges can be found in 5.1.1.10.3.
3	Product Code	UINT	The value zero is not valid.
4	Revision	STRUCT of	The value zero shall not be valid for either the Major and Minor Revision fields of a compliant product. A damaged product may return zero to indicate unknown revision if its storage of revision become corrupt.
	Major Revision	USINT	The Major Revision attribute shall be limited to 7 bits. The eighth bit is reserved and shall be zero.
	Minor Revision	USINT	
5	Status	WORD	Bit definitions are described in Table 245.
6	Serial Number	UDINT	
7	Product Name	SHORT_STRING	The maximum number of 8-bit characters in this string shall be 32. Each of the characters shall be in the range 0x20 to 0x7E.

Table 245 – Identity object bit definitions for status instance attribute

Bit(s):	Called:	Definition
0	Owned	1 for TRUE, 0 for FALSE
1		Reserved, set to 0
2	Configured	1 for TRUE, 0 for FALSE
3		Reserved, set to 0
4,5,6,7		See Table 246.
8	Minor Recoverable Fault	1 for TRUE, 0 for FALSE
9	Minor Unrecoverable Fault	1 for TRUE, 0 for FALSE
10	Major Recoverable Fault	1 for TRUE, 0 for FALSE
11	Major Unrecoverable Fault	1 for TRUE, 0 for FALSE
12, 13		Reserved, shall be set to 0
14, 15		Reserved, shall be set to 0

The values of the status bits 4 to 7 shall be as shown in Table 246.

Table 246 – Bits 4 – 7 of status instance attribute

Bits 4, 5, 6, 7:	Meaning
0 0 0 0	Self-Testing (power-up) or State Unknown
0 0 0 1	Firmware Update in progress
0 0 1 0	Communication Fault (lost connection)
0 0 1 1	Unkeyed, Awaiting Connection
0 1 0 0	Non-Volatile Configuration Bad
0 1 0 1	Major Fault (see bits 10 and 11 for fault classification and LED states) Minor fault is not a state change
0 1 1 0	Connected, Active
0 1 1 1	Idle (program mode type)
1 0 0 0	reserved, shall not be used
1 0 0 1	
1 0 1 0	reserved for product specific states
1 0 1 1	
1 1 0 0	
1 1 0 1	
1 1 1 0	
1 1 1 1	

5.1.1.8.2.2 Common services

Get_Attribute_All Response

At the **class level**, the order of the attributes returned in the “Attribute Data” parameter of the Get_Attribute_All response shall be as defined in Table 247.

Table 247 – Class level object/service specific response data of Get_Attribute_All

Attribute ID	Octet	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
1	0	Revision (low octet) Default = 1							
	1	Revision (high octet) Default = 0							
2	2	Max Instance (low octet) Default = 1							
	3	Max Instance (high octet) Default = 0							
6	4	Max ID Number of Class Attributes (low octet) Default = 0							
	5	Max ID Number of Class Attributes (high octet) Default = 0							
7	6	Max ID Number of Instance Attributes (low octet) Default = 0							
	7	Max ID Number of Instance Attributes (high octet) Default = 0							

At the **instance level**, the order of the attributes returned in the “Attribute Data” parameter of the Get_Attribute_All response shall be as defined in Table 248.

Table 248 – Instance level object/service specific response data of Get_Attribute_All

Attribute ID	Octet	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
1	0	Vendor (low octet)							
	1	Vendor (high octet)							
2	2	Device Type (low octet)							
	3	Device Type (high octet)							
3	4	Product Code (low octet)							
	5	Product Code (high octet)							
4	6	Major Revision							
	7	Minor Revision							
5	8	Status (low octet)							
	9	Status (high octet)							
6	10	Serial Number (low octet)							
	11	Serial Number							
	12	Serial Number							
	13	Serial Number (high octet)							
7	14	Product Name length							
	15	Product Name (1 st character)							
	16	Product Name (2 nd character)							
	n	Product Name (last character)							

Because the length of the name is not known before issuing the Get_Attribute_All service request, implementors shall allow enough memory space to store a response up to 32 characters in length.

To maintain consistency, if any instance attributes are added at some later date, the instance Get_Attribute_All response shall be modified to include the attributes as defined in Table 249.

Table 249 – Modified instance level object/service specific response data of Get_Attribute_All

Attribute ID	Octet	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
8	n+1	State Default = 255							
9	n+2	Space reserved for attribute 9 low octet, default=0							
	n+3	Space reserved for attribute 9 high octet, default=0							
10	n+4	Heartbeat Interval, Default = 0							

Reset service

The Reset common service shall have the object-specific parameter specified in Table 250.

Table 250 – Object-specific parameter for Reset

Name	Type	Semantics of values
Type	USINT	Value 0 shall emulate as closely as possible cycling power on the item the <i>Identity Object</i> represents. This value shall be the default if this parameter is omitted. Value 1 shall return as closely as possible to the out-of-box configuration, then shall emulate cycling power as closely as possible. Values 2-255 are reserved.

5.1.1.8.3 Message router object specific syntax elements

5.1.1.8.3.1 Attributes

Class attributes

The Message Router Object class attributes shall be as specified in Table 251.

Table 251 – Message router object class attributes

Attribute ID	Name	Data type
1	Revision	UINT
4	Optional attribute list	STRUCT of
	Number of attributes	UINT
	Optional attributes	ARRAY of UINT
5	Optional service list	STRUCT of
	Number services	UINT
	Optional services	ARRAY of UINT
6	Max ID Number of Class Attributes	UINT
7	Max ID Number of Instance Attributes	UINT

Instance attributes

The format of the Message Router Object instance attributes is as specified in Table 252.

Table 252 – Message router object instance attributes

Attribute ID	Name	Data type
1	Object_List	STRUCT of
	Number	UINT
	Classes	ARRAY of UINT
2	Number available	UINT

5.1.1.8.3.2 Common services**Get_Attribute_All Response**

At the **class level**, the order of the attributes returned in the “Attribute Data” parameter of the Get_Attribute_All response shall be as specified in Table 253. Default values for all unsupported class attributes shall be as shown in that table.

Table 253 – Class level object/service specific response data of Get_Attribute_All

Class attribute ID	Attribute name, description and default value
1	Revision default = 1
4	Optional attribute list, number of attributes default = 0
5	Optional service list, number of services default = 0
6	Max ID number of class attributes default = 0
7	Max ID number of instance attributes default = 0

Default values for all unsupported instance attributes shall be as shown in Table 254.

Table 254 – Instance level object/service specific response data of Get_Attribute_All

Instance attribute ID	Attribute name, description and default value
1	Object list number, number of supported classes default = 0
2	Number available, maximum number of connections default = 0
3	Always = 0x0000

5.1.1.8.4 Assembly object specific syntax elements**5.1.1.8.4.1 Attributes****Class attributes**

The format of the Assembly Object class attributes shall be as specified in Table 255.

Table 255 – Assembly object class attributes

Attribute ID	Name	Data type
1	Revision	UINT
2	Max Instance	UINT

Instance attributes

The format of the Assembly Object instance attributes is as specified in Table 256.

Table 256 – Assembly object instance attributes

Attribute ID	Name	Data type	Semantics of values
1	Number of Members in List	UINT	
2	Member List	ARRAY of STRUCT	This attribute has a complex data type.
	Member Data Size	UINT	Size in bits
	Member Path Size	UINT	Size in octets (0 = Empty Path)
	Member Path	ARRAY of OCTET	Packed path to the member data. See 5.1.1.9 for definition of Path.
3	Data	ARRAY of OCTETS	Contain all of the member data packed into one array. This data may contain many different data types. For efficiency it is best to keep this data word aligned by packing it on word boundaries and adding padding as needed. This can be accomplished by using “empty paths” (Member Path Size = 0).
4	Size	UINT	

5.1.1.8.5 Connection manager object specific syntax elements**5.1.1.8.5.1 Attributes****Class attributes**

The format of the Connection Manager Object class attributes shall be as specified in Table 257.

Table 257 – Connection manager object class attributes

Attribute ID	Name	Data type
1	Revision	UINT
2	Max Instance	UDINT
4	Optional Attribute List	STRUCT of
	Number attributes	UINT
	Optional attributes	ARRAY of UINT

Instance attributes

The format of the Connection Manager Object instance attributes is as specified in Table 258

Table 258 – Connection manager object instance attributes

Attribute ID	Name	Data type	Semantics of values
1	OpenReqs	UINT	
2	OpenFormat Rejects	UINT	
3	OpenResource Rejects	UINT	
4	OpenOther Rejects	UINT	
5	CloseReqs	UINT	
6	CloseFormat Rejects	UINT	
7	CloseOther Rejects	UINT	
8	ConnTimeouts	UINT	
9	Connection Entry List	STRUCT of	
	NumConnEntries	UINT	Number of bits used in the ConnOpenBits element. This attribute, divided by 8 and incremented for any remainder, gives the length in OCTETS of the array in the ConnOpenBits field.
	ConnOpenBits	ARRAY of OCTET	Bit field. List of connection data. Each bit represents a possible connection. 0 = No Connection. 1 = Connection Established.
11	CpuUtilization	UINT	CPU Utilization in tenths of a percent. Range of 0 – 1000 representing 0 to 100%.
12	MaxBuffSize	UDINT	Amount of buffer space is in Octets
13	BufSize Remaining	UDINT	Amount of buffer space is in Octets

5.1.1.9 Message and connection paths

5.1.1.9.1 Contents

The path shall specify the object element that is either the target of a connection request, or the destination of a message request. The path shall contain multiple segments which can indicate the route to the next node (in the case of multiple links), what to connect to or where to send a message in the target device. Each segment shall be comprised of a segment descriptor octet which specifies the segment type and segment information whose size is dependent on the segment type. The two types of path shall be

- padded paths;
- packed paths.

These two path formats (padded and packed) shall not be interchangeable. Usage of padded versus packed shall be dependent on the context of use. If no format is explicitly specified for a path, then padded format shall be used.

Each segment of a padded path shall be 16 bit word aligned. If a pad octet is required to achieve the alignment, then the segment shall specify the location of the pad octet. A packed path shall not contain the pad octets.

The possible segment types shall be as follows:

- Port segment (where);
- Logical segment (what);
- Network segment (when or how);
- Symbolic segment;
- Data segment.

5.1.1.9.2 Segment type

Each segment of the path shall contain a segment type to indicate how the segment is to be interpreted. The segment type is contained in the first octet of the segment. The bits of the first octet of a path segment shall be numbered 0 to 7, where bit 0 shall be the least significant bit of the octet. Bits of the segment type are defined in Figure 47.

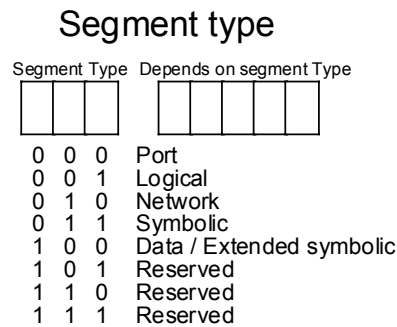


Figure 47 – Segment type

5.1.1.9.3 Port segment

The port segment shall indicate

- communication port through which to leave the node (expressed as a 16-bit number);
- link address of the next device in the path.

Figure 48 shows the overall structure of the port segment.

The bits 5-7 of the first octet shall be zero to indicate the port segment type. Bit 4, the node/slot address size bit, shall be set to 0 if the link address is one octet. Bit 4 shall be set to 1 if the link address is larger than one octet. If the link address is larger than one octet, its size in octets shall be in the second octet of the port segment.

Bits 0 – 3, the port identifier, shall indicate through which port to leave the node. The port identifier shall specify a port number or an escape to an extended port identifier when the module can support more than 14 ports. Port number 0 shall be reserved. Port number 1 shall only be used to represent a back-plane port. If the port identifier is 15, then a 16-bit field, called the extended port identifier, shall be the next part of the port segment; otherwise, the value of the port identifier shall be the port number.

The port number shall be followed by a link address whose format depends on the port type. If the link address is greater than one octet, then it shall be padded so that the entire port segment is an even number of octets. The octet specifying the size of the link address shall not include the pad octet. The pad octet shall be set to 0.

NOTE 1 For the common port types, the link address is a single octet. Other port types, such as TCP/IP encapsulation, may require a larger link address (see 5.1.3 and 5.8).

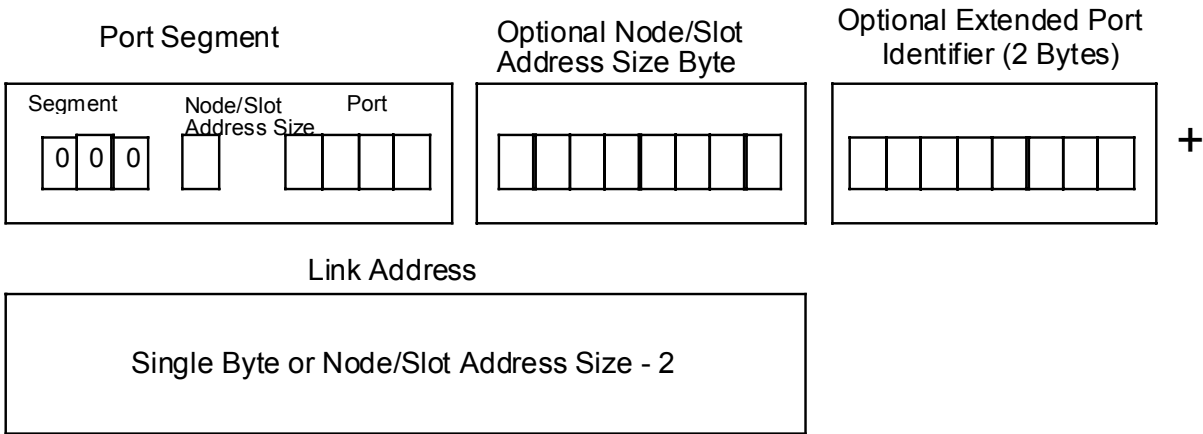


Figure 48 – Port segment

The Extended Port Identifier format of the Port Segment shall only be used when there are more than 14 ports possible on the device. The Port Segment shall always be packed into the smallest Port Segment format possible. Examples of possible port segments are as shown in Table 259.

Table 259 – Possible port segment examples

Port segment	Notes
[02][06]	Port 2, MAC ID 6
[15][0E][31][33][31][2E][31][35][31][2E][31][33][32][2E][35][35]	Multi-Octet Address for Ethernet Port 5, Link Address 131.151.132.55 (IP Address). The Address is defined as a character array.
[0F][12][00][01]	Port 18, MAC ID 1

The link address portion of a TCP/IP connection path segment shall be encoded within the port segment as a string of ASCII characters. The string shall be one of the following forms:

- IP address in dot notation, for example “130.151.132.55” (see RFC 1166 for the format of IP addresses);
- IP address in dot notation, followed by a “.” separator, followed by the TCP port number to be used at the specified IP address;
- Host name, for example “plc.type2.org”. The host name shall be resolved via a DNS request to a name server (see RFC 1035 for information on host names and name resolution);
- Host name, followed by a “.” separator, followed by the TCP port number to be used at the specified host.

The port number shall be represented in either hex or decimal. Hex shall be indicated by a leading “0x”. When a port number is specified, it shall be used rather than the TCP port number reserved for the encapsulation protocol.

Since port segments shall be word-aligned, a pad octet may be required at the end of the string. The pad octet shall be 0x00, and shall not be counted in the Optional Address Size field of the port segment.

NOTE 2 Examples of port segments for TCP/IP are shown in Table 260 below.

Table 260 – TCP/IP link address examples

Port segment	IP address	Notes
[12][0D] [31][33][30][2E] [31][35][31][2E] [31][33][32][2E][31][00]	130.151.132.1	Multi-octet address for port 2, 13 octet string plus a pad octet
[13][12] [70][6C][63][2E] [63][6F][6E][74][72][6F][6C][6E][65][74][2E] [6F][72][67]	plc. type2.org	Multi-octet address for port 3, 18 octet string, no pad octet
[16][15] [31][33][30][2E] [31][35][31][2E] [31][33][32][2E] [35][35][3A] [30][78][33][32][31][30][00]	130.151.132.55:0x3210	Multi-octet address for port 6, 21 octet string plus a pad octet
[15][17] [70][6C][63][2E] [63][6F][6E][74][72][6F][6C][6E][65][74][2E] [6F][72][67][3A] [39][38][37][36][00]	plc. type2.org:9876	Multi-octet address port 5, 32 octet string plus a pad octet

5.1.1.9.4 Logical segment

The segment type (first octet) of a logical segment shall be in the range 0x20 through 0x3F as shown in Table 261 (the most significant bits shall be 001). The logical segment shall select a particular addressable entity within a device. The segment type shall indicate the format of the logical segment. A packed logical segment shall append a 8-bit, 16-bit or 32-bit integer to the segment type to specify the addressed entity, depending on selected format. A padded 8-bit logical segment shall be identical to a packed 8-bit logical segment. The padded 16-bit logical segments shall insert a pad octet between the segment type and the 16-bit integer, as shown in Table 261. Likewise, the padded 32-bit logical segments shall insert a pad octet between the segment type and the 32-bit integer, as shown in Table 261. The pad octet shall be set to zero.

Table 261 – Logical segments

First octet	Logical segment name	Padded format examples (as transmitted)	Packed format examples (as transmitted)	Notes
0x20	8-bit class	[20][04]	[20][04]	class 0x04 (Assembly object)
0x21	16-bit class	[21][00][34][12]	[21][34][12]	class 0x1234
0x22	32-bit class	[22][00][78][56][34][12]	[22][78][56][34][12]	class 0x12345678
0x24	8-bit instance	[24][04]	[24][04]	instance 0x04
0x25	16-bit instance	[25][00][34][12]	[25][34][12]	instance 0x1234
0x26	32-bit instance	[26][00][78][56][34][12]	[26][78][56][34][12]	instance 0x12345678
0x28	8-bit member	[28][04]	[28][04]	member 0x04
0x29	16-bit member	[29][00][34][12]	[29][34][12]	member 0x1234
0x2A	32-bit member	[2A][00][78][56][34][12]	[2A][78][56][34][12]	member 0x12345678
0x2C	8-bit connection point	[2C][04]	[2C][04]	connection point 0x04
0x2D	16-bit connection point	[2D][00][34][12]	[2D][34][12]	connection point 0x1234
0x2E	32-bit connection point	[2E][00][78][56][34][12]	[2E][78][56][34][12]	connection point 0x12345678
0x30	8-bit attribute	[30][04]	[30][04]	attribute 0x04
0x31	16-bit attribute	[31][00][34][12]	[31][34][12]	attribute 0x1234
0x32	32-bit attribute	[32][00][78][56][34][12]	[32][78][56][34][12]	attribute 0x12345678
0x34	electronic key	[34][04] [12][00] [EF][BE] [0E][0B] [83] [01]	[34][04] [12][00] [EF][BE] [0E][0B] [83] [01]	vendor ID = 0x0012 product type = 0xBEEF product code = 0x0B0E major revision = 0x03 minor revision = 0x01 accept compatible keys = yes
0x23, 0x27, 0x2B, 0x2F, 0x33, 0x35 – 0x3F	Reserved			

Class Segment

The class segment shall specify a type of object within a device. For example, the class could specify the Connection Manager (0x06), Message Router (0x02), Assembly (0x04) or Identity object (0x01).

NOTE 1 IEC 61158-5 describes the object model including the definition of class, instance, attribute, connection point, and members of a library of common objects.

Instance Segment

The instance shall be an integer assigned to an object when it is created. The instance segment shall specify the particular instance of an object. Instance 0, called the class instance, shall specify the class itself.

Connection Point Segment

A connection point segment shall be shorthand for a sub-instance of an Assembly object. The connection point segment shall specify the particular connection point.

Attribute Segment

The attributes of an object shall be an enumerated list of externally visible values. An attribute segment shall specify which attribute is selected. These attributes can be a simple data type such as an integer or the attribute may be a complex structure.

Member Segment

Complex attributes shall be composed of members. A member segment shall specify which member of the attribute. For attributes that are arrays, such as instance attribute 0x03 of the Assembly object, the member shall indicate the one-based offset within the array.

Electronic Key Segment

The electronic key segment shall be used to verify the type of target or router device. The key may be included as the first logical segment in a connection path and shall be checked by the Message Router of the receiving node against the values contained in instance 1 of its Identity object before any additional address checks are made. On a multi-hop message, sent to a remote link via intermediate routers, multiple key segments may appear. The format of the electronic key segment shall be as shown in Table 262.

Table 262 – Electronic key segment format

Parameter	Format	Value
segment_type	USINT	always 0x34
electronic_key_type	USINT	always 0x04
vendor_ID	UINT	these correspond to the values in instance 1 of the identity object
product_type	UINT	
product_code	UINT	
major_revision : 7	USINT	
compatible_match : 1	USINT	
minor_revision	USINT	

The segment_type for a key segment shall be 0x34. The electronic_key_type shall determine the format of a specific key segment and shall be set to 0x04. All other values for the electronic_key_type shall be reserved. The vendor_ID shall specify the device vendor, or zero if no specific vendor is required.

The product_type shall specify a class of products such as digital input or analogue outputs. The product_type shall be ignored when it is set to zero. The product_code shall be vendor specific with each vendor having a unique code. The product_code shall be ignored when set to zero.

The major_revision and compatible_match fields shall be contained in the least significant 7 bits and most significant bit of the same octet, respectively. The major_revision shall specify the functionality level of a device. The major_revision shall be ignored when set to zero. The minor_revision shall specify the revision of a device that does not effect network-visible behaviour. A value of zero shall specify that the originator of the request accepts any minor revision.

The compatible_match shall modify the key matching function. If the bit is clear (= 0), then any non-zero vendor_ID, product_type, product_code, major_revision, and minor_revision shall match exactly. If the bit is set (= 1), the device may accept the key for any device which it can emulate. The level of emulation shall be product specific.

NOTE 2 IEC 61158-5 describes the Identity object and the rules for updating the revision fields.

5.1.1.9.5 Network segment

The segment type (first octet) of a network segment shall be in the range 0x40 through 0x5F as shown in Table 263 (the most significant bits shall be 010). The network segment shall be used to specify network parameters which may be required by a node to transmit a message across a network. The network segment shall immediately precede the port segment of the device to which it applies. In other words, the network segment shall be the first item in the path that the device receives.

Table 263 – Network segments

First octet	Network segment name
0x40	reserved
0x41	schedule segment
0x42	fixed tag segment
0x43	production inhibit time segment
0x44 – 0x5F	reserved

Schedule Segment

The segment type of the schedule network segment shall be 0x41. The schedule network segment shall specify

- a multiplier that, when multiplied by the NUT, gives the CM_API (actual packet interval) of the scheduled transport;
- a phase that determines on which values of the Moderator.interval_count to transmit.

NOTE 1 The Data Link Layer defines the Moderator.interval_count, see IEC 61158-4, Type 2.

This segment shall be included in the path to a device so that each intermediary node can schedule link transmit time for subsequent scheduled traffic. The multiplier shall be one of 1, 2, 4, 8, 16, 32, 64 or 128. The phase shall be in the range 0 through (multiplier – 1). The second octet of the schedule network segment shall encode both the multiple and phase by adding them together.

NOTE 2 If a transport produces every 64th NUT starting on interval_count = 17, then the encoded value is 17 + 64 = 81.

An encoded value of zero shall specify that the transport has not yet been given permission to use the scheduled priority. If a node receives a request for a scheduled connection in which either no schedule network segment exists, or the schedule network segment exists but the encoded value is 0, the node shall return general status = 0x01, and extended status = 0x0317.

NOTE 3 A connection originator is given permission to use the scheduled priority through the Scheduling object (IEC 61158-5).

Fixed Tag Segment

The segment type of the fixed tag network segment shall be 0x42. The fixed tag network segment shall specify the fixed tag which is to be used when sending an unconnected message. This segment subtype shall precede the port segment within the path. If the fixed tag segment is not present, then the default fixed tag shall be used.

NOTE 4 The fixed tag segment can be used within the path of the Unconnected_Send service (see 5.1.1.5.6) of the Connection Manager to send requests to the Keeper object via the Management UCMM (see IEC 61158-5).

Production inhibit time segment

The segment type of the production inhibit time segment shall be 0x43. The second octet shall specify the production inhibit time, which is the minimum time, in milliseconds, between successive transmissions of connected data for the specified connection.

NOTE 5 For example, if a production inhibit time of 10 milliseconds is specified, new data shall be sent no sooner than 10 milliseconds after the previous data.

The production inhibit time segment shall be used only for change-of-state transport class 1 connections over TCP/IP links, to place a limit on the packet rate. The production inhibit time segment may be sent to an end or intermediate device.

When the production inhibit time segment is used, the RPI shall determine when to re-send data. The RPI shall be larger than the production inhibit time. If the RPI is smaller than the production inhibit time, the Forward_Open reply shall be returned with error (status 0x01,

extended status 0x11B). If the production inhibit time segment is omitted, the producer shall use a default production inhibit time of 1/4 the RPI. If the production inhibit time is 0, there shall be no limit on how fast data may be sent on the connection.

NOTE 6 The method for transmitting APDUs over Ethernet-TCP/IP is specified in 5.1.3 and 5.8.

5.1.1.9.6 Symbolic segment

The range of segment types reserved for symbolic segments shall be 0x60 through 0x7F.

5.1.1.9.7 Data segment

The segment type (first octet) of a data segment shall be in the range 0x80 through 0x8F (the most significant bits shall be 1000). This IEC 61158-6 only defines the format of the simple data segment with a segment type of 0x80; all other data segment types shall be reserved (0x81 through 0x8F).

Simple Data Segment

The segment type of the simple data segment shall be 0x80. The second octet shall represent the number of 16-bit words of variable length data. Following the second octet shall be the variable length data. A path may contain more than one simple data segment as shown in Table 264.

Table 264 – Data segment

Parameter	Format	Value
segment_type	USINT	always 0x80
segment_size	USINT	size of the data[] array in 16-bit words
data[]	UINT	

The data can be configuration information for an object, additional parameters necessary for the connection, or any other information. The data segment shall not be interpreted by any other device in the path, so only the originating and target applications need agree on its contents.

5.1.1.9.8 Extended symbol segment

The segment type (first octet) of an extended symbol segment shall be in the range 0x90 through 0x9F (the most significant bits shall be 1001). This IEC 61158-6 only defines the format of the extended symbol segments 0x91. All other extended symbol segment types shall be reserved (0x90 and 0x92 through 0x9F).

ANSI Extended Symbol Segment

The segment type of the ANSI extended symbol segment shall be 0x91. The second octet shall represent the number of characters (8-bit) in the symbol. Following the second octet shall be the variable length symbol as shown in Table 265.

Table 265 – ANSI_Extended_Symbol segment

Parameter	Format	Value
segment_type	USINT	always 0x91
symbol_size	USINT	size of the symbol[] array
symbol[]	USINT	
pad	USINT	only present if symbol_size is odd, always set to zero

The symbol_size shall be the size of the symbol[] array in octets and shall not be zero. The symbol_size shall not count the pad octet if it is included.

5.1.1.10 Class, attribute and service codes

5.1.1.10.1 Code ranges

5.1.1.10.1.1 Defined ranges

There shall be three categories for address ranges of Class IDs, Attribute IDs and Service Codes as specified in Table 266.

Table 266 – Addressing categories

This category	Refers to
Open	A value which has the same meaning for all implementers. All Object Classes and services defined in IEC 61158-5 fall under this category.
Vendor-specific	A range of values specific to the vendor of a device. These are used by vendors to extend their devices beyond the available <i>Open</i> options. A vendor internally manages the use of values within this range. Applies to object classes, attributes, and services.
Object-class specific	A range of values whose meaning is defined by an Object Class. This range applies to Service Code definitions.

The Class ID, Attribute ID and Service code values shall be as defined in Table 267 through Table 269.

5.1.1.10.1.2 Class code ID ranges

The Class ID values shall be as shown in Table 267. Class Code = 0x00 is reserved and shall not be used.

Table 267 – Class code ID ranges

Range (hex)	Range (decimal)	Meaning	Quantity
00	00	Reserved	1
01 - 63	01 - 99	Open	99
64 - C7	100 - 199	Vendor Specific	100
C8 - EF	200 - 239	Reserved	40
F0 - 2FF	240 - 767	Open	528
300 - 4FF	768 - 1279	Vendor Specific	512
500 - FFFF	1280 - 65535	Reserved	64 256

5.1.1.10.1.3 Attribute ID ranges

The Attribute ID values shall be as shown in Table 268. Attribute ID = 0x00 is reserved and shall not be used.

Table 268 – Attribute ID ranges

Range (hex)	Range (decimal)	Meaning	Quantity
00	00	Reserved	1
01 - 63	01 - 99	Open	99
64 - C7	100 - 199	Vendor Specific	100
C8 - EF	200 - 239	Reserved	40
F0 - 2FF	240 - 767	Open	528
300 - 4FF	768 - 1279	Vendor Specific	512
500 - FFFF	1280 - 65535	Reserved	64 256

5.1.1.10.1.4 Service code ranges

The **Service code** shall be a unique hexadecimal value assigned to each service. The Service Code values shall be as shown in Table 269. Service Code = 0x00 is reserved and shall not be used. The object-specific service codes shall be unique only within the class which define them.

Table 269 – Service code ranges

Range (hex)	Range (decimal)	Meaning	Quantity
00	00	Reserved	1
01 - 31	01 - 49	Open. These are referred to as <i>Common Services</i> . They are defined in Type 2 clauses in IEC 61158-5.	49
32 - 4A	50 - 74	Vendor Specific	25
4B - 63	75 - 99	Object Class Specific	25
64 - 7F	100 - 127	Reserved for future use	28
80 - FF	128 - 255	Reserved for response messages	128

5.1.1.10.2 Code definitions

5.1.1.10.2.1 Communication object classes

Table 270 defines the class codes for the object classes. All other class codes listed in within the “open” range and not listed in Table 270 shall be reserved. The requirements for these objects are defined in IEC 61158-5.

Table 270 – objects

Class code (hex)	Class code (decimal)	Object name
0x01	1	Identity object
0x02	2	Message Router object
0x04	4	Assembly object
0x06	6	Connection Manager object
0xF0	240	ControlNet object
0xF1	241	Keeper object
0xF2	242	Scheduling object
0xF5	245	TCP/IP interface object
0xF6	246	Ethernet link object

NOTE The ControlNet, Keeper, Scheduling, TCP/IP interface and Ethernet link object classes are part of System Management.

5.1.1.10.2.2 Predefined class attributes

Seven predefined Class Attribute IDs shall be reserved, and these predefined/reserved class attributes shall have the definitions listed in Table 271. Because these attributes are reserved, class Attribute ID numbers 1 through 7 shall always be reserved. Therefore, if a class attribute is added to an object class specification, it shall start with Attribute ID #8.

Table 271 – Reserved class attributes for all object class definitions

Attribute ID	Name	Data type	Semantics of values
1	Revision	UINT	The starting value assigned to this attribute is one (1). If updates that require an increase in this value are made, then the value of this attribute increases by one (1).
2	Max Instance	UINT or UDINT	The class definition shall define the data type used.
3	Number of Instances	UINT or UDINT	The class definition shall define the data type used.
4	Optional attribute list	STRUCT of	
	Number of attributes	UINT	
	Optional attributes	ARRAY of UINT	
5	Optional service list	STRUCT of	
	Number services	UINT	
	Optional services	ARRAY of UINT	
6	Maximum ID Number Class Attributes	UINT	
7	Maximum ID Number Instance Attributes	UINT	

5.1.1.10.2.3 OM_Service codes

Codes of the common services for the deterministic control network objects shall be as shown in Table 272.

NOTE Table 272 lists the codes and names of the common services while IEC 61158-5 provides a general description of each service.

Table 272 – Common services list

Common service code	Common service name
0x00	reserved
0x01	Get_Attribute_All
0x02	Set_Attribute_All
0x03	Get_Attribute_List
0x04	Set_Attribute_List
0x05	Reset
0x06	Start
0x07	Stop
0x08	Create
0x09	Delete
0x0A – 0x0C	reserved
0x0D	Apply_Attributes
0x0E	Get_Attribute_Single
0x0F	reserved
0x10	Set_Attribute_Single
0x11	Find_Next_Object_Instance
0x12 – 0x14	reserved
0x15	Restore
0x16	Save
0x17	NOP (No operation)
0x18 – 0x1B	reserved

5.1.1.10.2.4 CM_Service codes

Codes of the services specific to the Connection Manager shall be as shown in Table 273.

NOTE Table 273 lists the codes and names of the services while IEC 61158-5 provides a general description of each service.

Table 273 – Services specific to Connection Manager

Service code	Service name
0x4E	Forward_Close
0x52	Unconnected_Send
0x54	Forward_Open
0x56	Get_Connection_Data
0x57	Search_Connection_Data
0x59	Ex_Forward_Open
0x5A	Get_Object_Owner

5.1.1.10.3 Device types

In order to allow interoperability and interchangeability, a device type shall be used to identify similar devices which

- exhibit the same behaviour;
- produce and/or consume the same set of data;
- contain the same set of configurable attributes.

The formal definition of this information is known as a Device Profile : all devices with the same device type number shall meet the requirements specified in the Device Profile for that device type.

“Device Type” is a required instance attribute of the Identity object as described in IEC 61158-5.

Device types shall be either publicly defined or vendor specific. Table 274 reveals the numbering scheme to be used for device type numbering.

Table 274 – Device type numbering

Range (hex)	Type	Quantity
00 - 63	Publicly Defined – Reserved	100
64 - C7	Vendor Specific	100
C8 - FF	Reserved	56
100 - 2FF	Publicly Defined – Reserved	512
300 - 4FF	Vendor Specific	512
500 - FFFF	Reserved	64256

All publicly defined device types shall have the same meaning for all implementers and are reserved. The Generic Device type (type number = 0x00) shall define a device that does not fit into any of the defined device types.

NOTE The actual definition of publicly defined device types and corresponding Device Profiles is outside the scope of this standard.

Vendor specific device types may be developed and vendors need not publish them. Each vendor shall maintain their own vendor-specific device types and corresponding number allocation.

5.1.1.11 Error codes

5.1.1.11.1 CM errors

5.1.1.11.1.1 Error code listing

The error codes are returned with the reply to a Connection Manager Service Request which resulted in an error. These error codes shall be used to help diagnose the problem with a Service Request. The error code shall be split into an 8 bit general status and one or more 16 bit words of extended status. Unless specified otherwise, only the first word of extended status shall be required. Additional words of extended status may be used to specify additional module specific debug information. All devices which originate messages shall be able to handle multiple words of extended status.

Table 275 provides a summary of the available error codes.

Table 275 – Connection manager service request error codes

General Status	Extended Status	Explanation
0x00		Service completed successfully.
0x01	0x0100	Connection in Use or Duplicate Forward Open.
0x01	0x0103	Transport Class and Trigger combination not supported
0x01	0x0106	Ownership Conflict
0x01	0x0107	Connection not found at target application.
0x01	0x0108	Invalid Connection Type. Indicates a problem with either the Connection Type or Priority of the Connection.
0x01	0x0109	Invalid Connection Size
0x01	0x0110	Device not configured
0x01	0x0111	CM_RPI not supported. May also indicate problem with connection time-out multiplier.
0x01	0x0113	Connection Manager cannot support any more connections
0x01	0x0114	Either the Vendor Id or the Product Code in the key segment did not match the device
0x01	0x0115	Product Type in the key segment did not match the device
0x01	0x0116	Major or Minor Revision information in the key segment did not match the device
0x01	0x0117	Invalid Connection Point
0x01	0x0118	Invalid Configuration Format
0x01	0x0119	Connection request fails since there is no controlling connection currently open.
0x01	0x011A	Target Application cannot support any more connections
0x01	0x011B	RPI is smaller than the production inhibit time
0x01	0x0203	Connection cannot be closed since the connection has timed out
0x01	0x0204	Unconnected Send timed out waiting for a response.
0x01	0x0205	Parameter Error in Unconnected Send Service
0x01	0x0206	Message too large for Unconnected message service
0x01	0x0207	Unconnected acknowledge without reply
0x01	0x0301	No buffer memory available
0x01	0x0302	Network link transmit time not available for data
0x01	0x0303	No Tag filters available
0x01	0x0304	not Configured to send realtime data
0x01	0x0311	Port specified in Port Segment Not Available
0x01	0x0312	Link Address specified in Port Segment Not Available
0x01	0x0315	Invalid Segment Type or Segment Value in Path
0x01	0x0316	Path and Connection not equal in close
0x01	0x0317	Either Segment not present or Encoded Value in Network Segment is invalid.
0x01	0x0318	Link Address to Self Invalid
0x01	0x0319	Resources on Secondary Unavailable
0x01	0x031A	Connection already established.
0x01	0x031B	Direct connection already established.

General Status	Extended Status	Explanation
0x01	0x031C	Miscellaneous
0x01	0x031D	Redundant connection mismatch
0x02	n/a	Connection Manager resources are unavailable to handle service request
0x03	n/a	Invalid connection number specified by the Get_Connection_Data service. This is also returned by the Search_Connection_Data service if the specified connection is not found.
0x04	Zero Based Word Offset	Segment Type in path is invalid. The Extended Status shall be the word offset (0 based) to the word in the path where the error occurred. The offset starts at the first word after the path size. This error shall not be returned if an error occurs when parsing the Connection Path.
0x05	Zero Based Word Offset	Destination in path is invalid. The Extended Status shall be the word offset (0 based) to the word in the path where the error occurred. The offset starts at the first word after the path size. This error shall not be returned if an error occurs when parsing the Connection Path.
0x07	n/a	Connection has been lost. This is used by the Get/Set Services when they are made through a connection.
0x08	n/a	Connection Manager does not support the requested Service.
0x09	Index to Element	Error in Data Segment. Extended Status shall be index to where the error was encountered in the Data Segment. The Configuration Revision Number if present in the Data Segment shall always be index 1. If the error occurs with the Get/Set Services, then the extended status indicates the attribute number which failed.
0x0C	Optional	Service cannot be performed while Object is in current state. The 1 st word of Extended Status may optionally contain the object's current state.
0x10	Optional	Service cannot be performed while Device is in current state. The 1 st word of Extended Status may optionally contain the device's current state.
0x11	n/a	Response data too large. This is used by the get services to indicate the amount of data requested was too large to fit into the response buffer.
0x13	n/a	Not enough data was received.
0x14	Attribute Id	Attribute specified in FIND service is not supported by Connection Manager
0x15	n/a	Too much data was received.
0x25	0x0114	Either the Vendor Id or the Product Code in the key segment did not match the device. Used if the Key Segment was contained in the path.
0x25	0x0115	Product Type in the key segment did not match the device. Used if the Key Segment was contained in the path.
0x25	0x0116	Major or Minor Revision information in the key segment did not match the device. Used if the Key Segment was contained in the path.
0x26	n/a	Invalid path size
NOTE 1 The word "n/a" in the Extended Status Column is used to signify that there is no additional Extended Status which is required to be returned for the particular General Status Code.		
NOTE 2 The word "optional" in the Extended Status Column is used to signify that if Extended Status information is used, then the first word of that extended status is already defined and user defined extended status shall begin with the second word of extended status.		

5.1.1.11.1.2 General status code 0x01

Usage

This general status code shall be returned if there is a problem with a parameter in a service request to the Connection Manager. The following sections shall provide details of the specific extended status codes which are to be returned with the general status.

Connection in Use (Extended Status = 0x0100)

This extended status code shall be returned when an originator is trying to make a connection to a connection point with which the originator may have already established a connection (duplicate Forward_Open). This may result from the originator establishing a connection with a connection point and the success response being lost in the return path. The originator shall then time-out on the request and attempt to establish the connection again even though the target actually established the connection.

The suggested response from the originator in this case shall be to either close and establish the connection again or wait for the connection to time-out and then establish the connection again. It shall be recognised that in the latter case, it shall take the connection 60 seconds (time-out for the first data to be received across a connection) to time-out before the connection can be re-established.

A duplicate Forward_Open shall be defined as an open request which has the same connection serial number, vendor ID, and originator serial number as a connection on the target which is already open. If an intermediate node receives a duplicate Forward_Open request, then the intermediate node shall pass on the request. However, the intermediate node shall not need to allocate transport resources for a new connection.

Transport Trigger Not Supported (Extended Status = 0x0103)

A transport class and trigger combination has been specified which is not supported by the application. Although routers may use the transport/trigger field, they shall not fail the connection based on its contents. Only targets shall return this extended status code.

Ownership Conflict (Extended Status = 0x0106)

The connection cannot be established since another connection already “owns” some of the resources required for this connection. An example of this would be that only one connection can control an output point on an I/O Module. If a second controlling connection is attempted, this error shall be returned. This extended status code shall only be returned by a target node.

Connection Not Found at Target Application (Extended Status = 0x0107)

This extended status code shall be returned by the close connection request, where the connection which is to be closed is not active at the target node. This extended status code shall only be returned by a target node. An intermediate node shall not generate this extended status code. If the specified connection is not found at the intermediate node, the close request shall still be forwarded using the path specified in the Forward_Close request.

Invalid Connection Type (Extended Status = 0x0108)

This extended status code shall be returned as the result of specifying a connection type (including fixed/variable sized connection) or connection priority which is not supported by the target application. Only a target node shall return this extended status code.

Invalid Connection Size (Extended Status = 0x0109)

This extended status code is returned when the target or router does not support the specified connection size. This could occur at a target because the size does not match the required size for a fixed size connection. It could occur at an intermediate device if the requested size is too large for the specified network.

Device Not Configured (Extended Status = 0x0110)

This extended status code shall be returned when a connection is requested from a target application which has not been configured and the connection request does not contain a data segment for configuration. Only a target node shall return this extended status code.

CM_RPI Not Supported (Extended Status = 0x0111)

This extended status code shall be returned if the device can not support the requested O⇒T or T⇒O CM_RPI. This extended status code shall also be used if the connection time-out multiplier produces a time-out value which is not supported by the device.

Connection Manager Out of Connections (Extended Status = 0x0113)

The maximum number of connections allowed by this instance of the Connection Manager has been exceeded.

Vendor ID or Product Code Error (Extended Status = 0x0114)

The Product Code or Vendor Id specified in the electronic key logical segment does not match the Product Code or Vendor Id in the target device.

Product Type Error (Extended Status = 0x0115)

The Product Type specified in the electronic key logical segment does not match the Product Type in the target device.

Revision Mismatch (Extended Status = 0x0116)

The major and minor revision information in conjunction with the exact match bit specified in the electronic key logical segment does not correspond to a valid revision for this device.

Invalid Connection Point (Extended Status = 0x0117)

The connection point specified in the connection path does not correspond to a valid connection point for the target application. This error could also be returned if a connection point was required, but not provided by a connection request.

Invalid Configuration Format (Extended Status = 0x0118)

An instance number specified for the configuration data does not correspond to a configuration instance. For example the connection path specifies a float configuration when the connection points specify a raw connection. This extended status code shall be used when the logical instance segment in the connection path is used to indicate a configuration type instead of an instance.

Controlling Connection Not Open (Extended Status = 0x0119)

The extended status code shall be returned when an attempt is made to establish an echo (i.e. listen only) connection to a connection which has no controlling connection (i.e. owner).

Application Out of Connections (Extended Status = 0x011A)

The maximum number of connections supported by this instance of the Target Application has been exceeded.

For example, the Connection Manager could support 20 connections while the Target Application can only support 10 connections (there may be other Target Applications which an originator can connect to). On the 11th Connection Request to the Target Application, this extended status code would be used to signify that the Target Application is out of Connections.

RPI too small (Extended Status = 0x011B)

Specified RPI is smaller than the production inhibit time.

Connection Timed Out (Extended Status = 0x0203)

This extended status code shall occur when a client tries to send a connected message over a connection that has been timed-out. This extended status code shall only occur at the originating node.

Unconnected_Send Timed Out (Extended Status = 0x0204)

The Unconnected Send Timed Out shall occur when the UCMM times out before a reply is received. This may occur for an Unconnected_Send, Forward_Open, or Forward_Close service. This means that the UCMM has tried a link specific number of times using a link specific retry timer and has not received an acknowledgement or reply. This may be the result of congestion at the destination node or may be the result of a node not being powered up or present. This extended status code shall be returned by the originating node or any intermediate node.

Parameter Error in Unconnected_Send Service (Extended Status = 0x0205)

One of the parameters in the unconnected send service was in error.

This shall be caused by an invalid Connection Tick Time and Connection time-out combination when a intermediate node attempts to decrement this value before passing the Unconnected_Send, Forward_Open, or Forward_Close service on to the next node in the path.

This shall also be caused when the Unconnected_Send is too large to be sent out on a network connection.

Message Too Large for Unconnected Message Service (Extended Status = 0x0206)

The message to be sent via the unconnected message service was larger than fits in the buffers allocated for the unconnected send service. This extended status code can only be detected by the originating node.

Unconnected Acknowledge Without Reply (Extended Status = 0x0207)

The message to be sent via the unconnected message service was acknowledged but not responded to.

No Buffer Memory Available (Extended Status = 0x0301)

The extended status code shall occur when insufficient memory is available to allocate a connection buffer. Routers and target nodes shall detect this error.

Link Transmit Time Not Available (Extended Status = 0x0302)

This extended status code shall be returned by any device in the path which is a producer and can not allocate sufficient link transmit time for the connection on its link. This can occur at any node. This can only occur for connections which are specified as scheduled priority.

No Screeners Available (Extended Status = 0x0303)

Any device in the path which is a consumer and does not have a screener available shall return this extended status code.

Not Configured to Send Real-time Data (Extended Status = 0x0304)

If requested to make a scheduled connection, any device which is unable to send scheduled packets shall return this extended status code. For example, this code shall be returned by a node whose MAC ID is greater than SMAX.

Port Not Available (Extended Status = 0x0311)

This extended status code is the result of specifying a non existent port in a port segment.

Link Address Not Available (Extended Status = 0x0312)

This extended status code is the result of a port segment which specifies an invalid link address. This extended status code shall not be used for valid link addresses which do not respond.

Invalid Segment Type in Path (Extended Status = 0x0315)

This extended status code is the result of a device being unable to decode the connection path. This could be caused by an unrecognised path type, a segment type occurring unexpectedly, or a myriad of other problems.

Error in Close Path (Extended Status = 0x0316)

The path in the close service does not match the connection being closed. This means the connection points to a different module or application than is specified in the path. The connection is deleted but the error message shall be returned.

Scheduling Not Specified (Extended Status = 0x0317)

Either a Schedule Segment was expected in the path and not found, or the Encoded Value in the Schedule Segment was zero (i.e. invalid).

Link Address to Self Invalid (Extended Status = 0x0318)

Under some conditions (depends on the device), a link address in the Port Segment which points to the same device (loopback to yourself) is invalid.

Secondary Resources Unavailable (Extended Status = 0x0319)

In a dual chassis redundant system, a connection request which is made to the primary system shall be duplicated on the secondary system. If the secondary system is unable to duplicate the connection request, then this extended status code shall be returned.

Connection Already Established (Extended Status = 0x031A)

A request for a direct connection has been refused because the corresponding data is already included in a larger block of data being sent via another connection.

Direct Connection Already Established (Extended Status = 0x031B)

A request for a connection has been refused because a member of the corresponding data is already being sent via a direct connection.

Miscellaneous (Extended Status = 0x031C)

Essentially, if no other extended status code applies, then return this one.

Redundant connection mismatch (Extended Status = 0x031D)

This extended status code shall be returned when establishing a redundant owner connection without matching the following fields in the Forward_Open to the same fields in the other redundant owner connections to the same target path :
O2T_RPI, O2T_connection_parameters, T2O_RPI, T2O_connection_parameters,
xport_type_and_trigger.

5.1.1.11.1.3 General status code 0x02

This general status code shall be returned when the Connection Manager object lacks the resources to handle a connection request.

5.1.1.11.1.4 General status code 0x03

This general status code shall be returned by the Get_Connection_Data request when the connection number supplied with the service is invalid (i.e. no connection is present). This general status code is also returned by the Search_Connection_Data service when the connection specified by the Connection Serial Number, Vendor Id, and Originator Serial Number is not found at the target.

5.1.1.11.1.5 General status code 0x04

This general status code shall be returned when there is a problem with the Segment Type in the path of a service request. The extended status provides a zero based 16 bit word offset to the place where the error occurred. The offset begins with the first word after the path size in the message.

5.1.1.11.1.6 General status code 0x05

This general status code shall be returned when there is a problem with the Destination in the path of a service request. The extended status provides a zero based 16 bit word offset to the place where the error occurred. The offset begins with the first word after the path size in the message.

5.1.1.11.1.7 General status code 0x07

The general status code shall be returned by the following services if the requests were made via a Message Router connection and the connection had been closed:

Get_Attribute_All;
Set_Attribute_All;
Get_Attribute_Single;
Set_Attribute_Single;
Get_Attribute_List;
Set_Attribute_List.

5.1.1.11.1.8 General status code 0x08

This general status code shall be returned when the Connection Manager object on a particular device does not support the requested service.

5.1.1.11.1.9 General status code 0x09

This general status code shall be returned when there is an error in the data segment. The extended status shall provide an indication of the problem. When this general status code is returned in response to a Set_Attributes_All request, the extended status shall indicate the attribute number of the first attribute which caused the error.

5.1.1.11.1.10 General status code 0x0C

This general status code shall be returned when the state of the target application prevents the service request from being handled. The first word of Extended Status can report the object's current state. For the Connection Manager object, the extended status is considered optional and is not required.

For example, an object may need to be in an edit mode before attributes can be set. This is different from a service being rejected due to the state of the device, as in 5.1.1.11.1.11.

5.1.1.11.1.11 General status code 0x10

This general status code shall be returned when the state of the device prevents the service request from being handled. The first word of Extended Status can report the device's current state. For the Connection Manager object, the extended status is considered optional and is not required.

For example, a controller may have a key switch which when set to the "hard run" state causes Service Requests to several different objects to fail (i.e. program edits). This general status code would then be returned.

5.1.1.11.1.12 General status code 0x11

This general status code shall be returned by services to indicate that the response buffer is not large enough to hold the data, which was requested by the service. This applies in particular to Get_Connection_Data, Search_Connection_Data, Get_Attribute_All, Get_Attribute_List, and Get_Attribute_Single services.

5.1.1.11.1.13 General status code 0x13

This general status code shall be returned when not enough data was provided to perform a service request. This could mean that there wasn't enough data in the Connection Path or the size of the message was too small to even route the message to the appropriate object.

5.1.1.11.1.14 General status code 0x14

This general status code shall be returned by the find service when an attribute included in the service is not defined for the Connection Manager. The extended status shall indicate the attribute number of the first attribute which caused the error.

5.1.1.11.1.15 General status code 0x15

This general status code shall be returned when too much data was provided to perform a service request. This could mean that there was too much data in the Connection Path or the size of the message was greater than expected for a given service request.

5.1.1.11.1.16 General status code 0x25

Usage

This general status code shall be returned if the electronic key logical segment is included as part of the path and a problem was detected with the data contained in the electronic key logical segment. The extended status shall provide details of the error. This general status code shall not be returned if the electronic key logical segment was part of the connection path. In that case, general status 0x01 (Connection Failure) would be returned. This subclause provides details of the specific extended status codes which are to be returned.

Vendor ID or Product Code Error 0x0114

The Product Code or Vendor Id specified in the electronic key logical segment does not match the Product Code or Vendor Id in the target device.

Product Type Error 0x0115

The Product Type specified in the electronic key logical segment does not match the Product Type in the target device.

Revision Mismatch 0x0116

The major and minor revision information in conjunction with the exact match bit specified in the electronic key logical segment does not correspond to a valid revision for this device.

5.1.1.11.1.17 General status code 0x26

This general status code shall be returned when the path size is invalid. This could mean that not enough or too much data is contained in the path to correctly specify the destination of a service request.

5.1.1.11.2 OM errors

5.1.1.11.2.1 General status format

General status used in the service primitives is specified in Table 276.

Table 276 – General status codes

Status code	Name	Description and meaning of status code
0x00	Success	Service was successfully performed by the object specified.
0x01	Connection failure	A connection related service failed along the connection path.
0x02	Resource unavailable	Resources needed for the object to perform the requested behaviour were unavailable. Further object specific information shall be supplied in the object specific status field of the response.
0x03	Invalid value in object specific data parameter of a service request	A portion of the data supplied as a object specific data parameter of a service was invalid. The verification of the data shall be specified in the object specification of the object reporting the error.
0x04	Path segment error	The path segment identifier or the segment syntax was not understood by the processing node. The word offset to the first segment of the path that is not understood shall be supplied in the first word of the object specific status field of the response. The offset shall be zero based and calculated from the first word following the path size in the message. Path processing shall stop when a path segment error is encountered.
0x05	Path destination unknown	The path is referencing an object class, instance or structure element that is not known or is not contained in the processing node. The word offset to the first segment component that references something that is unknown or not present in the processing node shall be supplied in the first word of the object specific status field of the response. The offset shall be zero based and calculated from the first word following the path size in the message. Path processing shall stop when a path destination unknown error is encountered.
0x06	Partial transfer	Only part of the expected data was transferred.
0x07	Connection lost	The messaging connection was lost.
0x08	Unimplemented service	The service requested was not implemented or defined for this class or object instance.
0x09	Invalid attribute value	The value of an attribute of the object or class is invalid. The object specific status shall report the attribute number and the status code of the first attribute refusing data.
0x0A	Attribute list error	An attribute in the Get_Attribute_List or Set_Attribute_List response has a non zero status.
0x0B	Already in requested mode/state	The object is already in the mode/state being requested by the service. The object specific status shall report the current status of the object.
0x0C	Object is not able to perform service in its current mode/state	The object can not perform the requested service in its current mode/state. The object specific status shall report the current status of the object.
0x0D	Object already exists	The requested instance of object to be created already exists.
0x0E	Attribute value not settable	The object attribute is not a settable attribute. The object specific status shall report the number of the attribute refusing data.
0x0F	Access permission does not allow service	The access permissions do not allow the object to perform the service. The access permissions available to the object shall be reported in the extended status. See Table 277 for details concerning extended status.
0x10	Device's mode/state does not allow object to perform service	The device containing the object does not allow the object to perform the service in the current mode/state of the device. The object specific status shall report the device current status, e.g. a controller may have a key switch which when set to the 'hard run' state causes Service Requests to several different objects to fail (i.e. program edits) causing this status code to be returned.
0x11	Reply data too large	The data to be transmitted in the response buffer is larger than the allocated response buffer; therefore, no data was transferred.
0x12	Fragmentation of a primitive value	The service specified an operation that is going to fragment a primitive data value, i.e. half a REAL data type.
0x13	Not enough data	The service did not supply enough data to perform the specified operation.
0x14	Undefined attribute	The attribute specified is not defined for the class or object.

Table 276 (continued)

Status code	Name	Description and meaning of status code
0x15	Too much data	The service supplied more data than was expected (depending on the service and the object, the service might still be processed).
0x16	Object does not exist	The object specified does not exist in the device.
0x17	Service fragmentation sequence not currently in progress	The fragmentation sequence for this service is not currently active for this data.
0x18	No stored attribute data	The attribute data of this object was not saved prior to the requested service.
0x19	Store operation failure	The attribute data of this object was not saved due to some failure during the attempt.
0x1A	Routing failure, request packet too large for network	The service request packet was too large for transmission on a network in the path to the destination. The routing device was forced to abort the service.
0x1B	Routing failure, response packet too large for network	The service response packet was too large for transmission on a network in the path from the destination. The routing device was forced to abort the service.
0x1C	Missing attribute list entry data	The service did not supply an attribute in a list of attributes that was needed by the service to perform the requested behaviour.
0x1D	Invalid attribute value list	The service is returning the list of attributes supplied with status information for those attributes that were invalid.
0x1E	Embedded service error	An embedded service resulted in an error.
0x1F	Connection Related Failure	A service failed because of an error condition related to the processing of a connection related service. This might occur during connected and unconnected messaging. The same extended status codes used for General Status Code 0x01 are returned for this error's extended status.
0x20	reserved	reserved
0x21	Write once value or medium already written	An attempt was made to write to a write once medium (e.g. WORM drive, PROM) that has already been written, or to modify a value that cannot be changed once established.
0x22	Invalid Reply Received	An invalid reply is received (e.g. reply service code does not match the request service code, or reply message is shorter than the minimum expected reply size). This status code can serve for other causes of invalid replies.
0x23	reserved	reserved
0x24	reserved	reserved
0x25	Key Failure in path	The Key Segment which was included as the first segment in the path does not match the destination module. The object specific status shall indicate which part of the key check failed.
0x26	Path Size Invalid	The size of the path which was sent with the Service Request is either not large enough to allow the Request to be routed to an object or too much routing data was included.
0x27	Unexpected attribute in list	An attempt was made to set an attribute that cannot be set at this time.
0x28	Invalid Member ID	The Member ID specified in the request does not exist in the specified Class/Instance/Attribute.
0x29	Member not settable	A request to modify a non-modifiable member was received.
0x2A – 0xCF	Reserved	Reserved.
0xD0 – 0xFF	Reserved object/class specific	This range of status codes has been reserved for use by object and class specific services.

NOTE IEC 61158-5 contains more detail on the service response general status codes for each common service.

5.1.1.11.2.2 Extended status format

The MR_response_Header contains a parameter called ext_stat[] which is labelled extended status data.

Extended status codes used in the service primitives are specified in Table 277.

Table 277 – Extended status codes

Code	Format	Meaning	Description
0x04	UINT	path_segment_offset	word offset from the beginning of the path of the incorrect segment
0x05	UINT	path_segment_offset	word offset from the beginning of the path of the first segment that is referencing a non-existent object or class
0x09	UINT	attribute_number	number of the attribute refusing the data value
	UINT	attribute_status	status code explaining reason for invalid attribute value
0x0B	UDINT	object_status	status/state/mode information of the object
0x0C	UDINT	object_status	status/state/mode information of the object
0x10	UDINT	device_status	status/state/mode information of the device
0x19	DWORD	storage_status	status/state/mode information of the storage media
0x1D	UINT	attribute_count	number of attribute values being returned
	STRUCT of	status_of_attributes []	array of status information
	UINT	attribute_number	number of attribute
	UINT	attribute_status	status of attribute

5.1.2 Data abstract syntax specification

5.1.2.1 Transport format specification

The lower layers of open system architectures are concerned with the transport of user data among distributed functional units. In these layers, the user data may be regarded simply as a sequence of octets. However, application layer entities may manipulate the values of quite complex data types. To achieve independence between the application layer and lower layers, data types may be specified in an abstract syntax notation.

Supplementing the abstract syntax with one or more algorithms (called encoding rules) may determine the values of the lower layer octets which carry the application layer values. The combination of the abstract syntax with a single set of transfer rules produces a specific transfer syntax.

5.1.2.2 Abstract syntax notation

The data type definitions provided in this IEC 61158-6 shall be written in Abstract Syntax Notation One (ASN.1), as defined in ISO/IEC 8824. These type definitions shall be a part of the ASN.1 module “Network DataTypes.” The beginning ASN.1 statement indicating that these definitions are in this module is:

```
control network DataTypes DEFINITIONS ::= BEGIN
```

and the closing ASN.1 statement shall be the keyword “END”.

The abstract definitions that follow shall comprise the set of control network data types. In addition, provision is made to extend or derive new data types based on existing defined types, and to include those in a “type dictionary.”

5.1.2.3 Control network data specification

The notation [typeld] for directly derived, enumerated, subrange and structured bit string data shall mean that the tag shall be taken from the “type” field in the corresponding VariableDictionaryEntry.

```
Network Data ::= CHOICE{ElementaryData, DerivedData}
```

```

ElementaryData ::= CHOICE {
    BOOL,
    FixedLengthInteger,
    FixedLengthReal,
    AnyTime,
    AnyDate,
    AnyString,
    FixedLengthBitString}

DerivedData ::= CHOICE {
    DirectlyDerivedData,
    EnumeratedData,
    SubrangeData,
    StructuredBitStringData,
    ARRAY,
    STRUCT,
    FunctionBlockData}

DirectlyDerivedData ::= [typeId] NetworkData
EnumeratedData ::= [typeId] USINT
SubrangeData ::= [typeId] FixedLengthInteger
StructuredBitStringData ::= [typeId] FixedLengthBitString
FixedLengthInteger ::= CHOICE {SignedInteger, UnsignedInteger}
SignedInteger ::= CHOICE {SINT, INT, DINT, LINT}
UnsignedInteger ::= CHOICE {USINT, UINT, UDINT, ULINT}
FixedLengthReal ::= CHOICE {REAL, LREAL}
AnyTime ::= CHOICE {ITIME, TIME, FTIME, LTIME}
AnyDate ::= CHOICE {DATE, TIME_OF_DAY, DATE_AND_TIME}
AnyString ::= CHOICE {STRING, STRING2}
FixedLengthBitString ::= CHOICE {OCTET, WORD, DWORD, LWORD}
BOOL ::= [PRIVATE 1] IMPLICIT BOOLEAN
SINT ::= [PRIVATE 2] IMPLICIT OCTET STRING-- 1 octet
INT ::= [PRIVATE 3] IMPLICIT OCTET STRING-- 2 octets
DINT ::= [PRIVATE 4] IMPLICIT OCTET STRING-- 4 octets
LINT ::= [PRIVATE 5] IMPLICIT OCTET STRING-- 8 octets
USINT ::= [PRIVATE 6] IMPLICIT OCTET STRING-- 1 octet
UINT ::= [PRIVATE 7] IMPLICIT OCTET STRING-- 2 octets
UDINT ::= [PRIVATE 8] IMPLICIT OCTET STRING-- 4 octets
ULINT ::= [PRIVATE 9] IMPLICIT OCTET STRING-- 8 octets
REAL ::= [PRIVATE 10] IMPLICIT OCTET STRING-- 4 octets
LREAL ::= [PRIVATE 11] IMPLICIT OCTET STRING-- 8 octets
STIME ::= [PRIVATE 12] IMPLICIT DINT
DATE ::= [PRIVATE 13] IMPLICIT UINT
TIME_OF_DAY ::= [PRIVATE 14] IMPLICIT UDINT
DATE_AND_TIME ::= [PRIVATE 15] IMPLICIT SEQUENCE {
    time_of_day UDINT,
    date UINT }
STRING ::= [PRIVATE 16] IMPLICIT SEQUENCE {
    charcount UINT,
    stringcontents OCTET STRING} -- one octet per character
OCTET ::= [PRIVATE 17] IMPLICIT OCTET STRING-- 1 octet

```

```

WORD ::= [PRIVATE 18] IMPLICIT OCTET STRING-- 2 octets
DWORD ::= [PRIVATE 19] IMPLICIT OCTET STRING-- 4 octets
LWORD ::= [PRIVATE 20] IMPLICIT OCTET STRING-- 8 octets
STRING2 ::= [PRIVATE 21] IMPLICIT SEQUENCE {
    charcount      UINT,
    string2contents OCTET STRING} -- 2 octets/ character
FTIME ::= [PRIVATE 22] IMPLICIT DINT
LTIME ::= [PRIVATE 23] IMPLICIT LINT
ITIME ::= [PRIVATE 24] IMPLICIT INT
STRINGN ::= [PRIVATE 25] IMPLICIT SEQUENCE {
    charsize      UINT,
    charcount     UINT,
    stringNcontents OCTET STRING} -- N octets/ character
SHORT_STRING ::= [PRIVATE 26] IMPLICIT SEQUENCE {
    charcount     USINT,
    stringcontents OCTET STRING} -- one octet per character
ARRAY ::= SEQUENCE OF NetworkData -- All of same base type
STRUCT ::= SEQUENCE OF NetworkData -- May be different types
FunctionBlockData ::= SET{
    inputs  [0] IMPLICIT STRUCT OPTIONAL,
    outputs [1] IMPLICIT STRUCT OPTIONAL,
    controlInputs [2] IMPLICIT STRUCT OPTIONAL,
    controlOutputs [3] IMPLICIT STRUCT OPTIONAL}

```

5.1.2.4 Data type specification / dictionaries

The definition of an object may include text that defines attributes. Attributes shall be assigned a *Data Type* in an object specification. The Data Type may be one of those defined in this standard or may be an object specific extension to this standard. The following definition shall provide a *Type Specification* for data and shall provide a structure for extending or deriving new data types based on existing defined types.

```

Dictionary ::= CHOICE {VariableDictionary, TypeDictionary}
VariableDictionary ::= SEQUENCE OF VariableDictionaryEntry
VariableDictionaryEntry ::= SEQUENCE{
    name AnyString,
    id FixedLengthInteger,
    type TypeID,
    ranges SEQUENCE OF Subrange,-- for arrays
    accessPrivilege  BOOL {READ_ONLY(0), READ_WRITE(1)}
TypeID ::= OCTET STRING -- ASN.1 encoded tag value of the
-- DataTypeSpecification module
Subrange ::= SEQUENCE {
    minValue FixedLengthInteger,
    maxValue FixedLengthInteger}
TypeDictionary ::= SEQUENCE OF TypeDictionaryEntry
TypeDictionaryEntry ::= SEQUENCE {
    name AnyString,
    type TypeID,
    spec DataTypeSpecification}

```

```

DataTypeSpecification ::= CHOICE {
    alt      [PRIVATE 0] IMPLICIT AlternateTypeSpec,
    bool     [PRIVATE 1] IMPLICIT NULL, -- BOOL
    sint     [PRIVATE 2] IMPLICIT NULL, -- SINT
    int      [PRIVATE 3] IMPLICIT NULL, -- INT
    dint     [PRIVATE 4] IMPLICIT NULL, -- DINT
    lint     [PRIVATE 5] IMPLICIT NULL, -- LINT
    usint    [PRIVATE 6] IMPLICIT NULL, -- USINT
    uint     [PRIVATE 7] IMPLICIT NULL, -- UINT
    udint    [PRIVATE 8] IMPLICIT NULL, -- UDINT
    ulint    [PRIVATE 9] IMPLICIT NULL, -- ULINT
    real     [PRIVATE 10] IMPLICIT NULL, -- REAL
    lreal    [PRIVATE 11] IMPLICIT NULL, -- LREAL
    stime    [PRIVATE 12] IMPLICIT NULL, -- STIME
    date     [PRIVATE 13] IMPLICIT NULL, -- DATE
    tod      [PRIVATE 14] IMPLICIT NULL, -- TIME_OF_DAY
    dat      [PRIVATE 15] IMPLICIT NULL, -- DATE_AND_TIME
    str1     [PRIVATE 16] IMPLICIT NULL, -- STRING
    octet    [PRIVATE 17] IMPLICIT NULL, -- OCTET
    word     [PRIVATE 18] IMPLICIT NULL, -- WORD
    dword    [PRIVATE 19] IMPLICIT NULL, -- DWORD
    lword    [PRIVATE 20] IMPLICIT NULL, -- LWORD
    str2     [PRIVATE 21] IMPLICIT NULL, -- STRING2
    ftime    [PRIVATE 22] IMPLICIT NULL, -- FTIME
    ltime    [PRIVATE 23] IMPLICIT NULL, -- LTIME
    itime    [PRIVATE 24] IMPLICIT NULL, -- ITIME
    strN     [PRIVATE 25] IMPLICIT NULL, -- STRINGN
    shstr    [PRIVATE 26] IMPLICIT NULL, -- SHORT_STRING
    constructedData CHOICE {
        abbrevStruc [0] IMPLICIT AbbreviatedStrucTypeSpec,
        abbrevArr   [1] IMPLICIT AbbreviatedArrayTypeSpec,
        frmlStruc   [2] IMPLICIT FormalStrucTypeSpec,
        frmlArr     [3] IMPLICIT FormalArrayTypeSpec,
        expBitStr   [4] IMPLICIT ExpandedFixedLenBitStrTypeSpec,
        expStr1     [5] IMPLICIT ExpandedStringTypeSpec,
        expStr2     [6] IMPLICIT ExpandedString2TypeSpec
    }
}

AbbreviatedStrucTypeSpec ::= UINT
AbbreviatedArrayTypeSpec ::= DataTypeSpecification
FormalStrucTypeSpec ::= SEQUENCE OF DataTypeSpecification
FormalArrayTypeSpec ::= SEQUENCE {
    lowBound [0] IMPLICIT FixedLengthInteger, -- Array Lower Bound
    highBound [1] IMPLICIT FixedLengthInteger, -- Array Upper Bound
    data     [2] IMPLICIT DataTypeSpecification
}

ExpandedFixedLenBitStrTypeSpec ::= SEQUENCE {
    bitStrType DataTypeSpecification -- OCTET, WORD, DWORD, or LWORD
    bitFields [7] IMPLICIT BitFieldDef
}

BitFieldDef ::= SEQUENCE OF {
    bitDef [2] IMPLICIT OCTET STRING} -- Length is always 2 octets.

-- First octet contains starting
-- Bit Position. Trailing octet
-- contains the number of bits.

ExpandedStringTypeSpec ::= UINT-- String Length In Octets
ExpandedString2TypeSpec ::= UINT-- String Length In Octets

```

```

AlternateTypeSpec ::= CHOICE {
    directlyDerivedTypeSpec [0] IMPLICIT TypeID,
    subrangeTypeSpec        [1] IMPLICIT SubrangeTypeSpec ,
    enumeratedTypeSpec       [2] IMPLICIT EnumeratedTypeSpec,
    fbTypeSpec               [3] IMPLICIT FBTypeSpec}

SubrangeTypeSpec ::= SEQUENCE{
    baseType      TypeID,      -- NOTE  minValue and maxValue
    minValue      FixedLengthInteger, -- shall be within the range
    maxValue      FixedLengthInteger} -- of baseType values

EnumeratedTypeSpec ::= SEQUENCE OF AnyString

BitNameDefintion ::= SEQUENCE {
    bitName  AnyString,
    bitNumber USINT}

FBTypeSpec ::= SET{
    inputs          [0] IMPLICIT FbtElementSpec OPTIONAL,
    outputs         [1] IMPLICIT FbtElementTypeSpec OPTIONAL,
    controlInputs   [2] IMPLICIT FbtElementTypeSpec OPTIONAL,
    controlOutputs  [3] IMPLICIT FbtElementTypeSpec OPTIONAL}

FbtElementTypeSpec ::= SEQUENCE OF ElementSpec

ElementSpec ::= SEQUENCE {
    name      AnyString,
    typespec  ElementTypeSpec}

ElementTypeSpec ::= CHOICE {
    [0] IMPLICIT TypeID,
    [1] IMPLICIT SubrangeTypeSpec,
    [2] IMPLICIT EnumeratedTypeSpec,
    [3] IMPLICIT FormalArrayTypeSpec,
    [4] IMPLICIT ExpandedStringTypeSpec,
    [5] IMPLICIT ExpandedString2TypeSpec}

```

The following END statement shall terminate the ASN.1 module opened in 5.1.1.

END.

5.1.3 Encapsulation abstract syntax

5.1.3.1 Encapsulation protocol

5.1.3.1.1 Common note

NOTE In order to send TPDU's over TCP/IP, an encapsulation protocol is required. This subclause defines the encapsulation protocol requirements. The encapsulation protocol is a generic protocol that may be used for transporting data other than Type 2 TPDU's.

5.1.3.1.2 Encapsulation messages

All encapsulation messages shall be composed of a fixed-length header of 24 octets followed by an optional data portion. The total encapsulation message length (including header) shall be limited to 65535 octets. The encapsulation message length shall not override length restrictions imposed by the encapsulated protocol. Its structure shall be as shown in Figure 49, where the top of the diagram indicates the portion of the message sent first on the wire.

NOTE The encapsulation data portion of the message is optional.

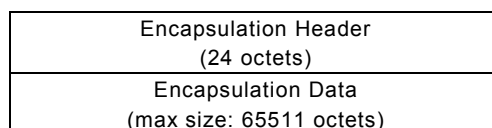


Figure 49 – Encapsulation message

5.1.3.1.3 Encapsulation header

The encapsulation header shall be as shown in Table 278.

Table 278 – Encapsulation header

Field name	Format	Description
Command	UINT	Encapsulation command
Length	UINT	Length, in octets, of the data portion of the message, i.e., the number of octets following the header
Session handle	UDINT	Session identification (application dependent)
Status	UDINT	Status code
Sender context	ARRAY of 8 USHORT	Information pertinent only to the sender of an encapsulation command
Options	UDINT	Options flags

Multi-octet integer fields in encapsulation messages shall be transmitted as specified in 5.2.

Although the header contains no explicit information to distinguish between a request and a reply, this information shall be determined in either of two ways:

- implicitly, by the command and the context in which the message is generated. (For example, in the case of the RegisterSession command, the request is generated by an originator and the target generates the reply) ;
- explicitly, by the contents of an encapsulated protocol packet in the data part of the message.

5.1.3.1.4 Command field

The allocation of command codes shall be as shown in Table 279.

Table 279 – Encapsulation command codes

Command code	Command
0x00	Nop
0x01 – 0x03	Reserved
0x04	ListServices
0x05 – 0x64	Reserved
0x65	ResgisterSession
0x66	UnRegisterSession
0x67 – 0x6E	Reserved
0x6F	SendRRData
0x70	SendUnitData
0x71 – 0xFFFF	Reserved

A device shall accept commands that it does not support without breaking the session or underlying TCP connection. A status code indicating that an unsupported command was received shall be returned to the sender of the message (see 5.1.3.1.7).

5.1.3.1.5 Length field

The length field in the header shall specify the size in octets of the data portion of the message. The field shall contain zero for messages that contain no data. The total length of a message shall be the sum of the number contained in the length field plus the 24-octet size of the encapsulation header.

The entire encapsulation message shall be read even if the length may be invalid for a particular command or exceeds the target's internal buffers.

NOTE Failure to read the entire message can result in losing track of the message boundaries in the TCP octet stream.

5.1.3.1.6 Session handle field

The Session Handle shall be generated by the target and returned to the originator in response to a RegisterSession request. The originator shall insert it in all subsequent session requests (requests using session commands) to that particular target. In the case where the target initiates and sends a command to the originator, the target shall include this field in the request that it sends to the originator.

NOTE Some commands (i.e., Nop) do not require a session handle, even if a session has been established. Whether or not a particular command requires a session is noted in the description of that command.

5.1.3.1.7 Status field

The value in the Status field shall indicate whether or not the receiver was able to execute the requested encapsulation command. A value of zero in a reply shall indicate successful execution of the command. In all requests issued by the sender, the Status field shall contain zero. If the receiver receives a request with a non-zero Status field, the request shall be ignored and no reply shall be generated.

NOTE This field does not reflect errors that are generated by an encapsulated protocol packet contained within the data portion of the message. For example, an error encountered during an end node's processing of a Set Attributes service.

The status codes shall be as shown in Table 280.

Table 280 – Encapsulation status codes

Status code	Description
0x00	Success
0x01	The sender issued an invalid or unsupported encapsulation command.
0x02	Insufficient memory resources in the receiver to handle the command.
0x03	Poorly formed or incorrect data in the data portion of the encapsulation message.
0x04 – 0x63	Reserved
0x64	An originator used an invalid session handle when sending an encapsulation message to the target.
0x65	The target received a message of invalid length (see 5.1.3.1.4).
0x66 – 0x68	Reserved
0x69	Unsupported protocol revision.
0x6A – 0xFFFF	Reserved

5.1.3.1.8 Sender context field

The sender of the command shall assign the value in the Sender Context field of the header. The receiver shall return this value in its reply.

5.1.3.1.9 Options field

To encapsulate PDUs from the Network and Transport Layer, the Options flags shall be set to zero. The allocation of Option flags shall be as shown in Table 281.

Table 281 – Options flags

Options flags	Definition
Bits 0 – 15	Allocated for compatibility with existing protocols
Bits 16 – 31	Area for future expansion

5.1.3.2 Command descriptions

5.1.3.2.1 Nop

Either an originator or a target may send a Nop request. No reply shall be generated. The data portion of the request shall be from 0 to 65511 octets long. The receiver shall ignore any data that is contained in the message.

NOTE A Nop provides a way for either an originator or target to determine if the TCP connection is still open.

The Nop request encapsulation header shall be as shown in Table 282.

Table 282 – Nop request encapsulation header

Field name	Field value
Command	Nop (0)
Length	Length of data portion
Session handle	Ignored
Status	0
Sender context	Ignored
Options	0

5.1.3.2.2 RegisterSession

An originator shall send a RegisterSession request to a target to initiate a session.

NOTE See 5.8.5, for detailed information on establishing and maintaining a session.

The RegisterSession request encapsulation header shall be as shown in Table 283.

Table 283 – RegisterSession request encapsulation header

Field name	Field value
Command	RegisterSession (0x65)
Length	4
Session handle	0
Status	0
Sender context	Sender context
Options	0

The parameters in the data portion shall determine the version of the protocol and any session options as shown in Table 284. The protocol version shall be set to 1. The session options shall be set to 0.

Table 284 – RegisterSession request data portion

Field	Type	Description
Protocol version	UINT	Requested protocol version (1)
Options flags	UINT	Session options (0)

Table 285 below shows the organisation of the Options flags:

Table 285 – Options flags

Options flags	Definition
Bits 0 – 7	Allocated for compatibility with existing protocols
Bits 8 – 15	Reserved for future expansion

The target shall send a RegisterSession reply to indicate that it has registered the originator. The reply shall have the same format as the request as shown in Table 286.

Table 286 – RegisterSession reply encapsulation header

Field name	Field value
Command	RegisterSession (0x65)
Length	4
Session handle	Handle returned by target
Status	0
Sender context	Context preserved from request
Options	0

The Session Handle field of the header shall contain a target-generated identifier that the originator shall save and insert in the Session Handle field of the header for all subsequent requests to that target. This field shall be valid only if the Status field is zero (0).

The Sender Context field of the header shall contain the same values present in the original sender request. If the originator has been registered with the target, the Status field shall be zero (0). If the target was unable to register, the Status field shall be non-zero.

The data portion of the reply shall have the same format as the request as shown in Table 287.

Table 287 – RegisterSession reply data portion

Field	Type	Description
Protocol Version	UINT	Protocol version supported
Options	UINT	Session options preserved from request

The Protocol Version field shall equal the requested version if the originator was successfully registered. If the target does not support the requested version of the protocol,

- the session shall not be created;
- the Status field shall be set;
- the target shall return the highest supported version in the Protocol Version field.

If all requested options are supported, the Options field shall return the originator's value. This value shall be zero.

5.1.3.2.3 UnRegisterSession

Either an originator or a target may send this request to terminate the session. The receiver shall initiate a close of the underlying TCP/IP connection when it receives this request. The session shall also be terminated when the transport connection between the originator and target is terminated. The receiver shall perform any other associated cleanup required on its end. There shall be no reply to this command.

The UnregisterSession request format shall be as shown in Table 288.

Table 288 – UnRegisterSession request encapsulation header

Field name	Field value
Command	UnRegisterSession (0x65)
Length	4
Session Handle	Handle from RegisterSession
Status	0
Sender Context	Sender context
Options	0

The Session Handle shall be set to the value obtained by the original RegisterSession reply. The handle shall become invalid after this request has been received and processed by the target.

5.1.3.2.4 ListServices

The ListServices request shall be sent to determine which encapsulation service classes the target device supports.

NOTE Each service class has a unique type code, and an optional ASCII name.

One service class is defined, with type code 0x100 and name “Communications”. This service class shall indicate that the device supports encapsulation of Type 2 PDU's. All devices that support Type 2 PDU encapsulation shall support the ListServices command and Communications service class.

The ListServices request encapsulation header shall be as shown in Table 289.

Table 289 – ListServices request encapsulation header

Field name	Field value
Command	ListServices (0x04)
Length	0
Session handle	Ignored
Status	0
Sender context	Chosen by sender
Options	0

The receiver shall reply with an encapsulation message consisting of the header and data, as shown in Table 290 and Table 291. The data portion of the reply shall provide the information on the services supported.

Table 290 – ListServices reply encapsulation header

Field name	Field value
Command	ListServices (0x04)
Length	0
Session handle	Ignored
Status	0
Sender context	Preserved from request
Options	0

The data portion of the reply shall contain a 2-octet object count followed by an array of objects describing the service(s) provided, as shown in Table 291.

Table 291 – ListServices reply data portion

Field name	Type	Description
Object count	UINT	Number of services objects to follow
Type code	UINT	Service type code
Object length	UINT	Remaining length in octets
Version	UINT	Encapsulation protocol version
Flags	UINT	Capability Flags
Name	ARRAY of 16 USINT	Name of service

The Type Code shall identify the service class as shown in Table 292.

Table 292 – Service type codes

Service type code	Description
0x00 – 0xFF	Reserved
0x100	Communications service
0x101 – 0xFFFF	Reserved

The Version field shall indicate the version of the service supported by the target to help maintain compatibility between applications.

Each service shall have a different set of capability flags. Unused flags shall be set to zero.

The Capability Flags, defined for the Communications service, shall be as shown in Table 293.

Table 293 – Communications capability flags

Flag value	Description
Bits 0 – 4	Reserved
Bit 5	If the device supports Type 2 PDU encapsulation this bit shall be set (= 1) ; otherwise, it shall be clear (= 0)
Bits 6 – 7	Reserved
Bit 8	Supports Class 0 and 1 UDP-based connections
Bits 9 – 15	Reserved

The Name field shall allow up to a 16-octet, NULL-terminated ASCII string for descriptive purposes only. The 16-octet limit shall include the NULL character.

5.1.3.2.5 ListIdentity

A connection originator may use the ListIdentity request to locate and identify potential targets. This request shall be sent as a broadcast message using UDP and does not require that a session be established.

The ListIdentity request encapsulation header shall be as shown in Table 294.

Table 294 – ListIdentity request encapsulation header

Field name	Field value
Command	ListIdentity (0x63)
Length	0
Session handle	This field is ignored since a session need not be established before sending the ListIdentity request.
Status	0
Sender context	0
Options	0

One reply item is defined for this command, Target Identity, with item type code 0x0C. This item shall be supported (returned) by all CIP capable devices.

Each receiver of the ListIdentity command shall reply with an encapsulation message consisting of the header and data, as shown in Table 295 and Table 296. The data portion of the message shall provide the information on the targets identity. The reply shall be sent to the IP address from which the broadcast request was received.

Table 295 – ListIdentity reply encapsulation header

Field name	Field value
Command	List Identity (0x63)
Length	0
Session handle	Ignored
Status	0
Sender context	0
Options	0

The data portion of the reply is structured as a Common Packet Format that contains a 2-octet object count followed by an array of objects providing the target identity, as shown in Table 296.

At a minimum, the CIP Identity item shall be returned and has the format as defined in Table 296. Part of this item definition follows the Get Attribute All service response definition of the Identity Object (data returned based on instance one of this object), and may adopt new members if and when new members are added to that service response. Unlike most fields in the Common Packet Format, the Socket Address field shall be sent in big endian order.

Table 296 – ListIdentity reply data portion

Field name	Type	Description
Object count	UINT	Number of identity objects to follow
Type code	UINT	Code indicating object type of CIP Identity (0x0C)
Object length	UINT	Remaining length in octets
Version	UINT	Encapsulation protocol version
Socket Address	ARRAY of :	Socket Address
	INT	sin_family (big-endian)
	UINT	sin_port (big-endian)
	UDINT	sin_addr (big-endian)
	ARRAY of USINT	sin_zero (length of 8) (big-endian)
Vendor ID ^a	UINT	Device manufacturers Vendor ID
Device Type ^a	UINT	Device Type of product
Product Code ^a	UINT	Product Code assigned with respect to device type
Revision ^a	USINT[2]	Device revision
Status ^a	WORD	Current status of device
Serial Number ^a	UDINT	Serial number of device
Product Name ^a	SHORT_STRING	Human readable description of device
State ^a	USINT	Current state of device
NOTE ^a These parameters are further defined by the corresponding instance attribute of the Identity Object.		

5.1.3.2.6 ListInterfaces

The optional ListInterfaces request shall be used by a connection originator to identify potential non-Type 2 communication interfaces associated with the target. A session need not be established to send this command.

The ListInterfaces request encapsulation header shall be as shown in Table 297.

Table 297 – ListInterfaces request encapsulation header

Field name	Field value
Command	List Interfaces (0x64)
Length	0
Session handle	Ignored
Status	0
Sender context	0
Options	0

If supported, the receiver of a ListInterfaces request command shall reply an encapsulation message consisting of the header and data, as shown below in Table 298.

Table 298 – ListInterfaces reply encapsulation header

Field name	Field value
Command	List Interfaces (0x64)
Length	0
Session handle	Ignored
Status	0
Sender context	0
Options	0

The data portion of the message is structured as a Common Packet Format, which contains a 2-octet object count followed by an array of objects providing information on the non-Type 2 communication interfaces associated with the target.

There are no publicly defined items returned with this reply. The vendor-specific item(s) which is/are returned shall, at a minimum, return a 32 bit Interface Handle which is used by other encapsulation commands, for example, the SendRRData command.

5.1.3.2.7 SendRRData

A SendRRData request shall transfer an encapsulated request/reply packet between the originator and target, where the originator initiates the command. The actual request/reply packets shall be encapsulated in the data portion of the message and shall be the responsibility of the target and originator. The SendRRData command shall be used to send encapsulated UCMM messages.

The SendRRData request encapsulation header shall be as shown in Table 299.

Table 299 – SendRRData request encapsulation header

Field name	Field value
Command	SendRRData (0x6F)
Length	Length of data portion
Session handle	Handle returned by RegisterSession
Status	0
Sender context	Sender context
Options	0

The data portion of the message shall contain request parameters and the encapsulated protocol packet as shown in Table 300.

Table 300 – SendRRData request data portion

Field name	Type	Description
Interface handle	UDINT	Shall be 0
Timeout	UINT	Operation Timeout
Encapsulated protocol packet (see Common Packet Format specification in 5.1.3.3)		

The Interface handle shall identify the Communications Interface to which the request is directed. This handle shall be 0 for encapsulating Type 2 PDUs.

The target shall abort the requested operation after the timeout expires. Timeout values shall be expressed in seconds. The minimum timeout value shall be one (1) second; the maximum value shall be 65535 seconds (about 18 hours). A value of zero shall indicate that the operation shall only be aborted when the encapsulated protocol times-out. Since the Network and Transport Layer already defines a timeout mechanisms for both connected and unconnected messages, the Timeout field shall be zero.

The encapsulated protocol packet shall be encoded in a Common Packet Format as shown in 5.1.3.3.

The SendRRData reply, as shown in Table 301, shall contain data in response to the SendRRData request. The reply to the original encapsulated protocol request shall be contained in the data portion of the SendRRData reply.

Table 301 – SendRRData reply encapsulation header

Field name	Field value
Command	SendRRData (0x6F)
Length	Length of data portion
Session handle	Handle returned by RegisterSession
Status	0
Sender context	Preserved sender context
Options	0

The format of the data portion of the reply message shall be the same as that of the SendRRData request message.

5.1.3.2.8 SendUnitData

The SendUnitData request shall send encapsulated connected messages. A reply need not be returned. This command may be used when the encapsulated protocol has its own underlying end-to-end transport mechanism.

The format of the SendUnitData request shall be as shown in Table 302.

Table 302 – SendUnitData request encapsulation header

Field name	Field value
Command	SendUnitData (0x70)
Length	Length of data portion
Session handle	Handle returned by RegisterSession
Status	0
Sender context	Sender context
Options	0

The data portion of the message shall contain request parameters as well as the encapsulated protocol packet as shown in Table 303.

Table 303 – SendUnitData request data portion

Field name	Type	Description
Interface handle	UDINT	Shall be 0
Timeout	UINT	Operation timeout
Encapsulated protocol packet (see Common Packet Format specification in 5.1.3.3)		

Interface handle and Timeout shall be the same as in the SendRRData request.

5.1.3.3 Common packet format

5.1.3.3.1 General

The common packet format shall consist of an item count, followed by an address item, then a data item (in that order) as shown in Table 304. Additional optional items may follow.

NOTE The common packet format defines a standard format for protocol packets that are transported with the encapsulation protocol. The common packet format is a general-purpose mechanism designed to accommodate future packet or address types.

Table 304 – Common packet format

Field name	Type	Description
Item count	UINT	Number of items to follow (shall be at least 2)
Address item	Item Struct (see below)	Addressing information for encapsulated packet
Data item	Item Struct (see below)	The encapsulated data packet
Optional additional items		

The address and data item structure shall be as shown in Table 305.

Table 305 – Address and data item structure

Field name	Type	Description
Type ID	UINT	Type of item encapsulated
Length	UINT	Length in octets of data to follow
Data	Variable	The data (if length >0)

The address type ID's shall be as shown in Table 306.

Table 306 – Address type ID's

Address type ID	Description
0x00	Null (used for UCMM messages)
0x01 – 0xA0	Reserved
0xA1	Connection-based (used for connected messages)
0xA2 – 0x8001	Reserved
0x8002	Sequenced Address Type
0x8003 – 0xFFFF	Reserved

The data type ID's shall be as shown in Table 307.

Table 307 – Data type ID's

Data type ID	Description
0x00 – 0xB0	Reserved
0xB1	Connected Transport PDU
0xB2	Unconnected message
0xB3 – 0x7FFF	Reserved
0x8000	Sockaddr Info, originator-to-target
0x8001	Sockaddr Info, target-to-originator
0x8002 – 0xFFFF	Reserved

5.1.3.3.2 Address types

5.1.3.3.2.1 Null

The null address type shall contain only the type id and the length as shown in Table 308. The length shall be zero. No data shall follow the length. Since the null address type contains no routing information, it shall be used when the protocol packet itself contains any necessary routing information. The null type shall be used for Unconnected Messages.

Table 308 – Null address type

Field name	Type	Field value
Type ID	UINT	0
Length	UINT	0

5.1.3.3.2.2 Connected

This address type shall be used when the encapsulated protocol is connection-oriented. The data shall contain a connection identifier, as shown in Table 309.

NOTE Connection identifiers are exchanged in the Forward_Open service of the Connection Manager.

Table 309 – Connected address type

Field name	Type	Field value
Type ID	UINT	0xA1
Length	UINT	4
Data	UDINT	Connection Identifier

5.1.3.3.2.3 Sequenced address type

This address type shall be used for class 0 and class 1 connected data. The data shall contain a connection identifier and a sequence number, as shown in Table 310. Usage is further described in 5.8.3.2.

Table 310 – Sequenced address type

Field name	Type	Field value
Type ID	UINT	0x8002
Length	UINT	8
Data	UDINT	Connection Identifier
	UDINT	Sequence Number

5.1.3.3.3 Data types

5.1.3.3.3.1 Unconnected

The data type that encapsulates an unconnected message shall be as shown in Table 311.

Table 311 – UCMM data type

Field name	Type	Field value
Type ID	UINT	0xB2
Length	UINT	Length, in octets, of the unconnected message
Data	Variable	The unconnected message

The format of the unconnected message shall be the same as PDU destined for the Message Router. The UCMM header, defined in 5.1.1.3, shall not be present. The context field in the encapsulation header shall be used for unconnected request/reply matching.

5.1.3.3.3.2 Connected

The data type that encapsulates a connected transport PDU shall be as shown in Table 312.

Table 312 – Connected data type

Field name	Type	Field value
Type ID	UINT	0xB1
Length	UINT	Length, in octets, of the transport PDU
Data	Variable	The transport PDU

5.1.3.3.3 Sockaddr info

The Sockaddr Info items shall be used to encapsulate socket address information necessary to send datagrams (the connected data) between the target and originator. There are separate items for originator-to-target and target-to-originator socket information.

The Sockaddr Info items shall have the structure shown in Table 313.

Table 313 – Sockaddr info items

Field name	Type	Field value
Type ID	UINT	0x8000, 0x8001
Length	UINT	16 (octets)
Data	ARRAY	See below

The data portion of the Sockaddr Info type shall follow the format of the sockaddr_in structure as defined in the Winsock specification, version 1.1. The sockaddr_in structure is shown below:

```
struct sockaddr_in {
    short    sin_family;
    u_short  sin_port;
    struct    in_addr sin_addr;
    char     sin_zero[8];
};
```

The multi-octet fields in the sockaddr_in structure shall be returned in TCP/IP octet order, since this is the octet ordering used by the Winsock API.

5.2 Transfer syntax

5.2.1 Compact encoding

5.2.1.1 Encoding rules

This subclause describes the means by which the data types defined in this standard shall be encoded/transferred across the control network. The abstract syntax definition along with a particular set of encoding rules shall result in the transfer syntax. For application user data, a single set of encoding rules shall be defined (Compact Encoding), resulting in the Compact transfer syntax.

Compact Encoding rules shall start with the encoding rules defined in ISO/IEC 8825. Compact Encoding then applies optimisation rules, starting with the outer most Service Data Unit (SDU) and progressing to each successive encapsulated SDU. Compact Encoding shall define a more efficient encoding mechanism by reducing the amount of information (overhead) transferred between devices.

The difference between a Compact encoded value and an ASN.1 encoded value shall be the removal of the fields describing the type and length of the information. The TAG and LENGTH components of an ASN.1-encoded value shall not be transmitted on the control network. In addition, the Compact Encoding rules shall indicate that octet ordering rules are the reverse of those seen in ASN.1.

Given the conditions listed in 5.2.1.2, general rules shall be applied to an ASN.1 encoded value to generate a Compact encoded value. The general rules shall be as follows:

- remove the Identifier Octets;
- remove the “TAG” octets specified by ASN.1;
- remove the Length Octets;

- remove the “LENGTH” octets specified by ASN.1;
- reverse the octet ordering for multiple content octets.

5.2.1.2 Encoding constraints

The representation of a variable value using Compact Encoding shall be possible with the following restrictions:

- the variable type shall be fixed length and shall have no conditional or optional fields;
- the encoding of a given variable shall be represented with a constant number of octets derived from the type specification of this variable.

5.2.1.3 Examples

5.2.1.3.1 BOOL encoding

The BOOLEAN encoding shall be performed on a single OCTET as described in Table 314.

Table 314 – BOOLEAN encoding

If the value is:	Then:
FALSE	bit 0 of the octet is 0 ('00'H)
TRUE	bit 0 of the octet is 1 ('01'H)

A FALSE BOOL shall be represented as shown in Table 315.

Table 315 – Example compact encoding of a BOOL value

Octet number	1 st
BOOL	00

5.2.1.3.2 SignedInteger encoding

The SignedInteger encoding shall be performed as described in Table 316.

Table 316 – Encoding of SignedInteger values

Octet number	1 st	2 nd	3 rd	4 th	5 th	6 th	7 th	8 th
SINT	0LSB							
INT	0LSB	1LSB						
DINT	0LSB	1LSB	2LSB	3LSB				
LINT	0LSB	1LSB	2LSB	3LSB	4LSB	5LSB	6LSB	7LSB
NOTE The example in Table 317 illustrates the encoding of a variable of type DINT whose value is 0x12345678.								

Table 317 – Example compact encoding of a SignedInteger value

Octet number	1 st	2 nd	3 rd	4 th
DINT	78	56	34	12

5.2.1.3.3 UnsignedInteger encoding

The UnsignedInteger encoding shall be performed as described in Table 318.

Table 318 – UnsignedInteger values

Octet number	1 st	2 nd	3 rd	4 th	5 th	6 th	7 th	8 th
USINT	0LSB							
UINT	0LSB	1LSB						
UDINT	0LSB	1LSB	2LSB	3LSB				
ULINT	0LSB	1LSB	2LSB	3LSB	4LSB	5LSB	6LSB	7LSB
NOTE Table 319 illustrates the encoding of a variable of type UDINT whose value is 0xAABBCCDD.								

Table 319 – Example compact encoding of an UnsignedInteger

Octet number	1 st	2 nd	3 rd	4 th
UDINT	DD	CC	BB	AA

5.2.1.3.4 FixedLengthReal encoding

The FixedLengthReal encoding shall be performed as described in Table 320.

Table 320 – FixedLengthReal values

Octet number	1 st	2 nd	3 rd	4 th	5 th	6 th	7 th	8 th
REAL	0LSB	1LSB	2LSB	3LSB				
LREAL	0LSB	1LSB	2LSB	3LSB	4LSB	5LSB	6LSB	7LSB

Table 321 illustrates the encoding of a variable (Float1) whose type is REAL and whose value is Float1: = 10,0.

NOTE 1 The assignment of the value is using the IEC 61131-3 notation. The ASN.1 value is {‘41200000’H} in IEEE format : 1,25*23 , exponent is 130 (bias 127), fraction is 25.

Table 321 – Example compact encoding of a REAL value

Octet contents	0LSB	1LSB	2LSB	3LSB
REAL	00	00	20	41

Table 322 illustrates the encoding of a variable (Float2) whose type is LREAL and whose value is Float2: = –100,0.

NOTE 2 The ASN.1 value is {‘C059000000000000’H} in IEEE format : 1,5625*26, exponent is 1029 (bias 1023), fraction is .5625.

Table 322 – Example compact encoding of a LREAL value

Octet contents	0LSB	1LSB	2LSB	3LSB	4LSB	5LSB	6LSB	7LSB
LREAL	00	00	00	00	00	00	59	C0

5.2.1.3.5 Time encodings

Table 323 illustrates the encoding of Real numbers with fixedlengths.

Table 323 – FixedLengthReal values

Octet number	1 st	2 nd	3 rd	4 th	5 th	6 th	7 th	8 th
TIME	0LSB	1LSB	2LSB	3LSB				
DATE	0LSB	1LSB						
TIME_OF_DAY	0LSB	1LSB	2LSB	3LSB				
DATE_AND_TIME	0LSB-Time	1LSB-Time	2LSB-Time	3LSB-Time	0LSB-Date	1LSB-Date		
FTIME	0LSB	1LSB	2LSB	3LSB				
LTIME	0LSB	1LSB	2LSB	3LSB	4LSB	5LSB	6LSB	7LSB

5.2.1.3.6 String encodings

This subclause gives examples of the Compact Encoding of STRING, STRING2, STRINGN, and SHORT_STRING data values.

NOTE The preferred string type for user supplied string data is STRING2 due to international character string requirements.

Table 324 provides a generic illustration of the encoding of a STRING value.

Table 324 – STRING value

	Contents (charcount)		Contents (string contents)
STRING	0LSB	1LSB	0LSB

1 octet characters

Table 325 provides a generic illustration of the encoding of a STRING2 value.

Table 325 – STRING2 value

	Contents (charcount)		Contents (string2contents)	
STRING2	0LSB	1LSB	0LSB	1LSB

2 octet characters

Table 326 provides a generic illustration of the encoding of a STRINGN value.

Table 326 – STRINGN value

	Contents (charsize)		Contents (charcount)		Contents (stringNcontents)	
STRINGN	0LSB	1LSB	0LSB	1LSB	0LSB	NLSB

N octet characters

Table 327 provides a generic illustration of the encoding of a SHORT_STRING value.

Table 327 – SHORT_STRING value

	Contents (charcount)	Contents (short_string)
SHORT_STRING	0LSB	0LSB

1 octet characters

Table 328 illustrates the encoding of a string variable whose contents equal "Mill". Encoding examples of all string types are presented. Character coding is specified in ISO/IEC 10646-1. The hexadecimal equivalent is: {4D696C6C'H} for 8 bit encoding.

Table 328 encodes "Mill" as a STRING type.

Table 328 – Example compact encoding of a STRING value

	Contents (charcount)		Contents (string contents)			
STRING	04	00	4D	69	6C	6C

Table 329 encodes "Mill" as a STRING2 type.

Table 329 – Example compact encoding of STRING2 value

	Contents (charcount)		Contents (string2 contents)							
STRING2	04	00	4D	00	69	00	6C	00	6C	00

Table 330 encodes "Mill" as a SHORT_STRING type.

Table 330 – SHORT_STRING type

	Contents (charcount)		Contents (short_string contents)			
SHORT_STRING	04		4D	69	6C	6C

5.2.1.3.7 FixedLengthBitString encoding

This subclause provides examples of the Compact Encoding of OCTET, WORD, DWORD, LWORD data values. Figure 50 illustrates the bit placement rules associated with the Compact Encoding of a FixedLengthBitString.

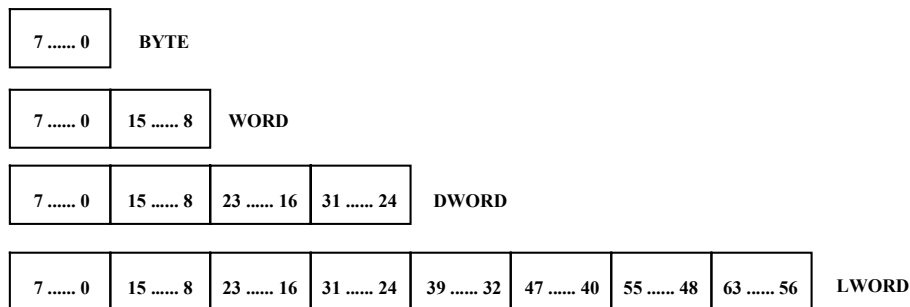


Figure 50 – FixedLengthBitString compact encoding bit placement rules

Figure 51 through Figure 54 illustrate the encoding of OCTET, WORD, DWORD, and LWORD.

Bits In Memory: 7 0
00001111

Compact Encoded BYTE
00001111 or 0x0F

Figure 51 – Example compact encoding of a OCTET FixedLengthBitString

Bits In Memory: 15 0
 00001111 11001111

Compact Encoded WORD
 11001111 00001111 or 0xCF0F

Figure 52 – Example compact encoding of a WORD FixedLengthBitString

Bits In Memory: 31 0
 00001111 11001111 10110110 00111110

Compact Encoded DWORD
 00111110 10110110 11001111 00001111 or 0x3EB6CF0F

Figure 53 – Example compact encoding of a DWORD FixedLengthBitString

Bits In Memory: 63 0
 11110000 11001111 10110110 00111110 11110000 00101101 00011110 00001111

Compact Encoded LWORD
 00001111 00011110 00101101 11110000 00111110 10110110 11001111 11110000 or 0xF1E2DF03EB6CFF0

Figure 54 – Example compact encoding of a LWORD FixedLengthBitString**5.2.1.3.8 Array encoding****5.2.1.3.8.1 One-dimensional array encoding**

The Array encoding shall use the encoding rules for the data types for each element and concatenates the elements which compose the array. The encoded values of the array elements shall be encoded in the same order as they are declared in the corresponding ASN.1 type or variable specification. These elements may be of any data type.

The ASN.1–style definition of a single–dimensional array in the control network shall be:

ARRAY ::= SEQUENCE OF { array_dimension_low_bound,
 array_dimension_high_bound, NetworkData }

Assume the following array definition::

ARRAY1 ::= SEQUENCE OF {array_dimension_low_bound := 0,
 array_dimension_high_bound := 1, UINT}

Plugging the UINT values {1,2} into this array definition yields the encoding specified in Table 331.

Table 331 – Example compact encoding of a single dimensional ARRAY

Octet number	1 st	2 nd	3 rd	4 th
ARRAY	01	00	02	00

5.2.1.3.8.2 Two-dimensional array encoding

ARRAY ::= SEQUENCE OF { array_dimension_low_bound,
 array_dimension_high_bound,
 SEQUENCE OF { array_dimension_low_bound,
 array_dimension_high_bound, NetworkData } }

5.2.1.3.8.3 Three-dimensional array encoding

```
ARRAY ::= SEQUENCE OF { array_dimension_low_bound,
    array_dimension_high_bound,
    SEQUENCE OF { array_dimension_low_bound,
        array_dimension_high_bound,
        SEQUENCE OF { array_dimension_low_bound,
            array_dimension_high_bound, NetworkData } } }
```

Since control network data may comprise either ElementaryData or DerivedData, a new type or variable specification may be required before transmitting the values for the ARRAY.

A multi-dimensional array shall be seen on the wire as a single-dimensional array. The order of the array elements shall be maintained via the packing/unpacking sequence followed by the end nodes. The sequence followed shall be to access the Nth dimension of the array for all values of the other dimensions.

This shall be achieved by first accessing the array with all dimensions set to their initial index values. After this the Nth dimension is incremented through all of its index values. When the end of the index range for the Nth dimension is reached, the (N-1)th dimension is incremented, and the Nth dimension is set to its initial index value. This process is repeated until all of the array's dimensions have reached the ends of their index ranges, and results in the array being packed into the message buffer as a single-dimensional array. The same procedure is followed to unpack the single-dimensional array into a multi-dimensional array.

The two-dimensional array

```
ARRAY1 [0..1 , 0..2] of UINT := { { 1, 2, 3 },
    { 4, 5, 6 } }
```

results in the data stream shown in Table 332 when it is packed into a single-dimensional array following the compact encoding rules:

Table 332 – Example compact encoding of a multi-dimensional ARRAY

Octet number	1 st	2 nd	3 rd	4 th	5 th	6 th	7 th	8 th	9 th	10 th	11 th	12 th
ARRAY	01	00	02	00	03	00	04	00	05	00	06	00

5.2.1.3.9 Structure encoding

The structure encoding shall use the encoding rules for the data types for each element and concatenates the elements which compose the structure.

The encoded values of the structure elements shall be encoded in the same order as they are declared in the corresponding ASN.1 type or variable specification. These elements may be of any Data type.

```
STRUCT ::= SEQUENCE OF NetworkData -- May be different types
```

Since NetworkData may be comprised of either ElementaryData or DerivedData, a new type or variable specification may be required before transmitting the values for the STRUCT.

Assume the following structure definition:

```
newStruct ::= SEQUENCE { BOOL,UINT,DINT}
```

Plugging the values {TRUE,'1234'H,'56789ABC'H} into the structure results in the encoding as specified in Table 333.

Table 333 – Example compact encoding of a STRUCTURE

Octet number	1 st	2 nd	3 rd	4 th	5 th	6 th	7 th
newStruct	01	34	12	BC	9A	78	56

5.2.1.3.10 Complex encoded data format examples

5.2.1.3.10.1 General

The examples 5.2.1.3.10.2 and 5.2.1.3.10.3 show how more complex data formats shall be packed. Example 5.2.1.3.10.2 shows the packing of an array of structures. Example 5.2.1.3.10.3 shows how a structure with an array element is packed.

5.2.1.3.10.2 Example 1: encoding an array of structures:

```

STRUCT1 ::= SEQUENCE OF {
    UINT      ele1;
    USINT     ele2;
    USINT     ele3;
    USINT     ele4;
    UINT      ele5
}

ARRAY1 [ 0..1 , 0..2 ] of STRUCT1 := {
    { { 1, 2, 3, 4, 5 }, { 6, 7, 8, 9, 10 },
      { 11, 12, 13, 14, 15 } },
    { { 15, 14, 13, 12, 11 }, { 10, 9, 8, 7, 6 },
      { 5, 4, 3, 2, 1 } } }

```

results in the following data stream:

```

[01][00][02][03][04][05][00] [06][00][07][08][09][0A][00]
[0B][00][0C][0D][0E][0F][00] [0F][00][0E][0D][0C][0B][00]
[0A][00][09][08][07][06][00] [05][00][04][03][02][01][00]

```

5.2.1.3.10.3 Example 2: encoding a structure with an array element

```

STRUCT2 ::= SEQUENCE OF {
    UINT      ele1;
    ARRAY [ 0..2 ] of USINT array2;
    UINT      ele5;
}

STRUCT2 := { 1, { 2, 3, 4 }, 5 }

```

results in the following data stream:

```

[01][00] [02][03][04] [05][00]

```

5.2.2 Data type reporting

5.2.2.1 Object data representation

Objects may choose to implement a mechanism for reporting Data Type of a particular Attribute or transmitting type information along with the actual data. This subclause defines the means by which Data Typing information is conveyed.

The specification of Data Type information utilises the ASN.1 methodology specified in ISO/IEC 8824 and ISO/IEC 8825 with the control network defined optimisations to encode the **DataTypeSpecification** production defined in 5.1.2.4.

The control network defined optimisation is that the Length Octet of a NULL type is not encoded.

NOTE For example, the encoding of 'abc [PRIVATE 1] IMPLICIT NULL' would be: 0xC1 (a tag with no length octet).

5.2.2.2 Elementary data type reporting

Elementary data types shall be identified using the identification codes defined in Table 334. These codes shall define the encoding of the primitive members of the DataTypeSpecification production. The control network specifies that ASN.1 NULL types shall not report the Length Octet of zero (0).

Table 334 – Identification codes and descriptions of elementary data types

Data type name	Data type code (in hex)	Data type description
BOOL	C1	Logical Boolean with values TRUE and FALSE
SINT	C2	Signed 8-bit integer value
INT	C3	Signed 16-bit integer value
DINT	C4	Signed 32-bit integer value
LINT	C5	Signed 64-bit integer value
USINT	C6	Unsigned 8-bit integer value
UINT	C7	Unsigned 16-bit integer value
UDINT	C8	Unsigned 32-bit integer value
ULINT	C9	Unsigned 64-bit integer value
REAL	CA	32-bit floating point value
LREAL	CB	64-bit floating point value
STIME	CC	Synchronous time information
DATE	CD	Date information
TIME_OF_DAY	CE	Time of day
DATE_AND_TIME	CF	Date and time of day
STRING	D0	character string (1 octet per character)
OCTET	D1	bit string - 8-bits
WORD	D2	bit string - 16-bits
DWORD	D3	bit string - 32-bits
LWORD	D4	bit string - 64-bits
STRING2	D5	character string (2 octets per character)
FTIME	D6	Duration (high resolution)
LTIME	D7	Duration (long)
ITIME	D8	Duration (short)
STRINGN	D9	character string (N octets per character)
SHORT_STRING	DA	character sting (1 octet per character, 1 octet length indicator)

5.2.2.3 Constructed data type reporting

5.2.2.3.1 Structure type definition

5.2.2.3.1.1 General

The control network defines two different methods for reporting Structure type definitions:

- Formal Encoding (FormalStrucTypeSpec);
- Abbreviated Encoding (AbbreviatedStrucTypeSpec).

Formal encoding shall be used to provide a detailed report of the complete structure definition, including the complete definition of all component data types. Abbreviated encoding shall be used to specify a *shorter* form of the structure definition. This *shorter* form shall not include the data types associated with the structure's components.

5.2.2.3.1.2 Formal encoding for structure type information

The examples 5.2.2.3.1.3 and 5.2.2.3.1.4 illustrate formal encoding for structure type specifications. This is actually an example of the encoding of the FormalStrucTypeSpec production defined in 5.1.2.4.

5.2.2.3.1.3 Example 1

Table 335 illustrates the encoding of the following structure definition.

STRUCT ::= SEQUENCE OF { BOOL, UINT, DINT }

Table 335 – Example 1 of formal encoding of a structure type specification

STRUCT type	Type length	Component types	Component types	Component types
		BOOL	UINT	DINT
A2	03	C1	C7	C4

NOTE The IMPLICIT NULL types from the DataTypeSpecification production are not followed by a Length Octet of zero (0).

5.2.2.3.1.4 Example 2

Figure 55 illustrates the encoding of the following structure definition.

STRUCT_MAIN ::= SEQUENCE OF { UINT, STRUCT_SUB, INT }

with subelement STRUCT_SUB defined as:

STRUCT_SUB ::= SEQUENCE OF { UINT, SINT, INT }

Struct Type	Type Length	Component Types						
		UINT	Struct Type	Type Length	Nested Structure Component Types			
					UINT	SINT	INT	INT
A2	07	C7	A2	03	C7	C2	C3	C3

Figure 55 – Example 2 of formal encoding of a structure type specification

5.2.2.3.1.5 Abbreviated encoding for structure type information

The example 5.2.2.3.1.6 illustrates the abbreviated encoding for structure type specifications. This is actually an example of the encoding of the AbbreviatedStrucTypeSpec production defined in 5.1.2.4.

The UINT defined within the AbbreviatedStrucTypeSpec production shall be initialised with a 16 bit Cyclic Redundancy Check (CRC) value (see IEC 61158-4, Type 2). This can be used by application logic to determine whether or not the format of the structure has changed. The octet stream used to produce the CRC is the actual formally encoded (FormalStrucTypeSpec) structure type specification.

5.2.2.3.1.6 Example

Figure 56 shows the abbreviated encoding of the structure definition presented in 5.2.2.3.1.5:

Struct Type	Type Length	UINT containing CRC	
A0	02	C7	26

Figure 56 – Example of abbreviated encoding of a structure type specification

NOTE The algorithms presented in this IEC 61158-6 are used to generate the CRC value of 0x26C7 from the Formally Encoded type specification: [A2][07][C7][A2][03] [C7][C2][C3][C3].

5.2.2.3.2 Array type definition

Two different methods for reporting array type definitions shall be:

- Formal Encoding (FormalArrayTypeSpec);
- Abbreviated Encoding (AbbreviatedArrayTypeSpec).

Formal encoding shall be used to provide a detailed report of the complete array definition, including the data content and the array's dimensions. Abbreviated encoding shall be used to specify a *shorter* form of the array definition. This *shorter* form shall not include information specifying the array's dimensions.

5.2.2.3.3 Formal encoding for array type information

5.2.2.3.3.1 Common notes

NOTE 1 This subclause contains no normative requirements.

NOTE 2 The examples 5.2.2.3.3.2 and 5.2.2.3.3.3 illustrate formal encoding for structure type specifications. This is actually an example of the encoding of the FormalArrayTypeSpec production defined in 5.1.2.4.

5.2.2.3.3.2 Example 1

Figure 57 shows the formal encoding of the following array definition

```
ARRAY1 ::= SEQUENCE OF { array_dimension_low_bound := 0,
                        array_dimension_high_bound := 9,
                        UINT
```

Array Type	Type Length	Lower Bound Tag	Lower Bound Length	Lower Bound	Upper Bound Tag	Upper Bound Length	Upper Bound	UINT
A3	07	80	01	00	81	01	09	C7

Figure 57 – Example 1 of formal encoding of an array type specification

NOTE The IMPLICIT NULL types from the DataTypeSpecification production are not followed by a Length Octet of zero (0).

5.2.2.3.3.3 Example 2

Figure 58 shows the encoding of the following array definition.

```
ARRAY1 ::= SEQUENCE OF { array_dimension_low_bound := 0,
                        array_dimension_high_bound := 19,
                        SEQUENCE OF { array_dimension_low_bound := 0,
                        array_dimension_high_bound := 255,
                        STRUCT_ELE } }
```

in which STRUCT_ELE is defined as:

```
STRUCT_ELE ::= SEQUENCE OF { UINT, SINT, INT }
```

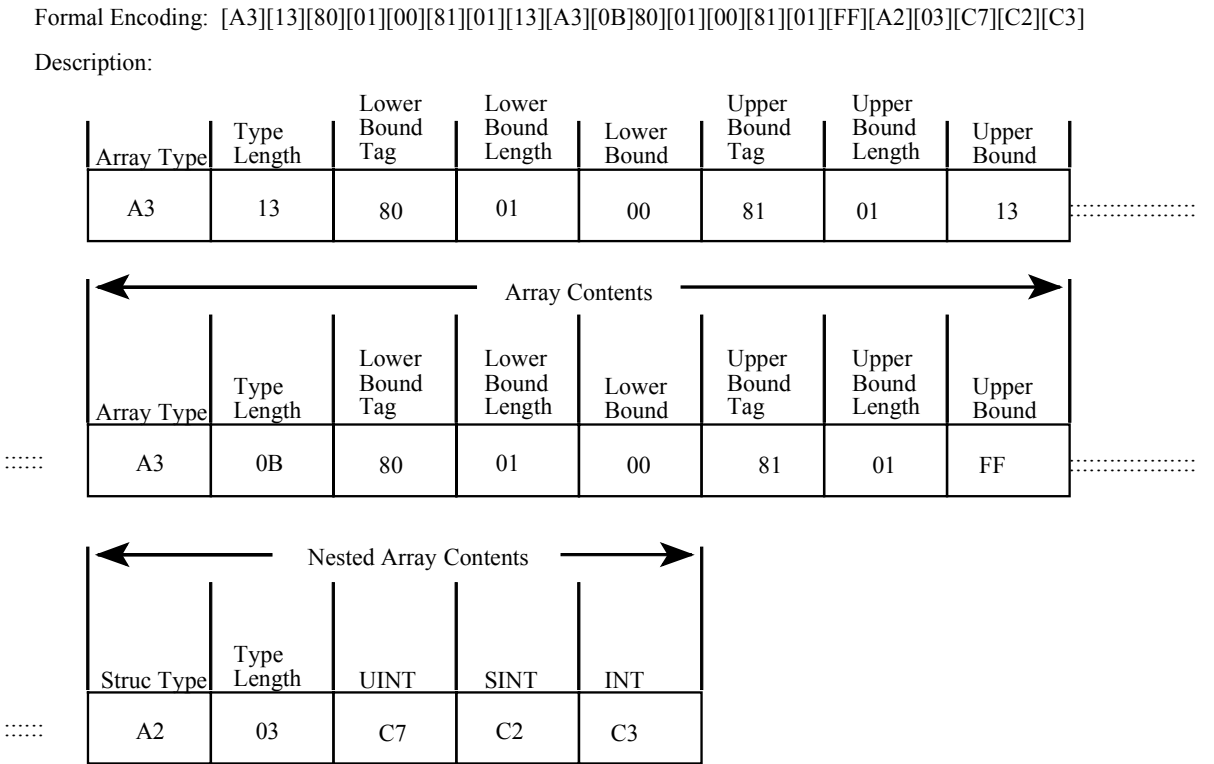


Figure 58 – Example 2 of formal encoding of an array type specification

NOTE The IMPLICIT NULL types from the DataTypeSpecification production are not followed by a Length Octet of zero (0).

5.2.2.3.4 Abbreviated tag encoding for array type information

5.2.2.3.4.1 Common notes

NOTE 1 This subclause contains no normative references.

NOTE 2 The abbreviated encoding of an Array type shall not include the information specifying the Array's dimensions. This is actually an example of the encoding of the AbbreviatedArrayTypeSpec production defined in 5.1.2.4.

5.2.2.3.4.2 Example 1

Figure 59 shows the abbreviated encoding of the following array definition:

```
ARRAY2 ::= SEQUENCE OF { array_dimension_low_bound := 0,
                          array_dimension_high_bound := 9,
                          UINT }
```

Array Type	Type Length	UINT
A1	01	C7

Figure 59 – Example 1 of abbreviated encoding of an array type specification

NOTE The IMPLICIT NULL types from the DataTypeSpecification production are not followed by a Length Octet of zero (0).

5.2.2.3.4.3 Example 2

Figure 60 shows the abbreviated encoding of the following array definition:

```

ARRAY ::= SEQUENCE OF { array_dimension_low_bound := 0,
                        array_dimension_high_bound := 19,
                        SEQUENCE OF { array_dimension_low_bound := 0,
                        array_dimension_high_bound := 899,
STRUCT_ELE } }
    
```

in which STRUCT_ELE is defined as:

```

STRUCT_ELE ::= SEQUENCE OF { UINT, SINT, INT }
    
```

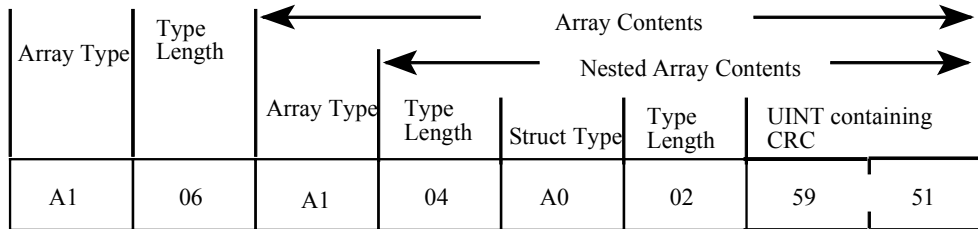


Figure 60 – Example 2 of abbreviated encoding of an array type specification

5.3 Structure of FAL protocol state machines

Interface to FAL services and protocol machines are specified in this subclause. Conventions used for the descriptions are given in the generic part of this IEC Standard.

This fieldbus follows the structure outlined for Type 1 fieldbus with the following specific features :

1. There is no formal definition of AP-Context Machine
2. There is a formally-defined FSPM Machine serving as an interface between FAL User and ARPM.
3. There are ARPM Machines of two different types :
 - a) one ARPM machine for connection-less application relationships
 - b) seven ARPM machines for connection-oriented application relationships
4. DMPM Machine is defined at the interface to the Type 2 Data Link layer

5.4 Context state machine

This fieldbus has no formal AP-Context State Machine.

5.5 FAL Service Protocol Machine (FSPM)

5.5.1 General

This fieldbus FAL Service Protocol State Machine maps FAL User services onto services of communication objects internal to FAL.

5.5.2 Primitive definitions

The Objects within FSPM shall provide the following services :

Get_Attribute_All	Reads values of all attributes of the specified object class or instance
Set_Attribute_All	Writes specified values to all attributes of the specified object class or instance
Get_Attribute_List	Reads values of the specified list of attributes of the specified object class or instance
Set_Attribute_List	Writes specified values to the specified list of attributes of the specified object class or instance
Get_Attribute_Single	Reads value of the specified attribute of the specified object class or instance
Set_Attribute_Single	Writes specified value to the specified attribute of the specified object class or instance
Reset	Resets the specified object class or instance
Create	Creates an instance of the specified object class
Delete	Deletes an instance of the specified object class
Start	Starts execution of the specified object.
Stop	Stops execution of the specified object.
Find_Next_Object_Instance	Finds the identifier of the next unused instance of the specified object class.
NOP	Triggers corresponding NOP response from the specified object.
Apply_Attributes	Causes pending attribute values to become active in the specified object.
Save	Saves attributes contents of the specified object.
Restore	Restores attributes contents of the specified object.

Refer to IEC 1158-5 for detailed definition of these services.

Primitives of Common Services exchanged between UCMM and FAL User are shown in Table 337 and Table 338.

All services have the following common parameters, as shown in Table 336 :

Table 336 – Primitives issued by FAL user to FSPM

Common parameters	Req	Ind	Rsp	Cnf
Argument	M			
AREP	M			
Local	S			
UCMM Record identifier	S			
Transport identifier	S			
Receiver/Server Local ID		M		
Path	M	C		
Routing Path	M			
Add. Path	U	U(=)		
Port ID		M		
Result (+)			S	S(=)
AREP				M=
Receiver/Server Local ID			M=	
Service status			M	M(=)
Result (-)			S	S(=)
AREP				M=
Receiver/Server Local ID			M=	
Service status			M	M(=)

Only those argument parameters additional to the common ones are shown in Table 337 and Table 338.

Additional parameters of indication service primitives are the same as those of the corresponding request primitive.

Additional parameters of confirmation service primitives are the same as those of the corresponding response primitive.

Table 337 – Primitives issued by FAL user to FSPM

Primitive name	Source	Additional parameters	Functions
Get_attribute_all.req	FAL User	None	Conveys a request from FAL User to a target object to supply values of all attributes of the specified object class or instance
Set_attribute_all.req	FAL User	Attribute Data	Conveys a request from FAL User to a target object to write specified values of all attributes of the specified object class or instance
Get_attribute_list.req	FAL User	Attribute Count Attribute List	Conveys a request from FAL User to a target object to supply values of specified list of attributes of the specified object class or instance
Set_attribute_list.req	FAL User	Attribute Count Attribute Data	Conveys a request from FAL User to a target object to write specified values of specified list of attributes of the specified object class or instance
Get_attribute_single.req	FAL User	None	Conveys a request from FAL User to a target object to supply values of the specified attribute of the specified object class or instance
Set_attribute_single.req	FAL User	Attribute Data	Conveys a request from FAL User to a target object to write the specified values into specified attribute of the specified object class or instance
Reset.req	FAL User	Object Specific Data (optional)	Conveys a request from FAL User to reset specified target object class or instance
Create.req	FAL User	Object Specific Data (optional)	Conveys a request from FAL User to a target object to create an instance of the specified object class
Delete.req	FAL User	Object Specific Data (optional)	Conveys a request from FAL User to a target object to delete an instance of the specified object class
Start.req	FAL User	Object Specific Data (optional)	Conveys a request from FAL User to a target object to start an instance of the specified object class
Stop.req	FAL User	Object Specific Data (optional)	Conveys a request from FAL User to a target object to stop an instance of the specified object class
Find_next_object_instance.req	FAL User	Maximum Returned Values	Conveys a request from FAL User to a target device to find the identifier of the next unused instance of the specified object class
NOP.req	FAL User	None	Conveys a request from FAL User to a target object to send back a corresponding NOP response
Apply_Attributes.req	FAL User	Object Specific Data (optional)	Conveys a request from FAL User to a target object to activate pending attribute values
Save.req	FAL User	Object Specific Data (optional)	Conveys a request from FAL User to a target object to save its attributes contents
Restore.req	FAL User	Object Specific Data (optional)	Conveys a request from FAL User to a target object to restore its attributes contents
Get_attribute_all.rsp	FAL User	(+) Attribute Data (-) Specific Status Code	Returns a response from FAL User to a target object to supply values of all attributes of the specified object class or instance
Set_attribute_all.rsp	FAL User	(+) None (-) Specific Status Code	Returns a response from FAL User to a target object to write specified values of all attributes of the specified object class or instance
Get_attribute_list.rsp	FAL User	(+) Attribute Count Attribute Data (-) Specific Status Code	Returns a response from FAL User to a target object to supply values of specified list of attributes of the specified object class or instance
Set_attribute_list.rsp	FAL User	(+) Attribute_count Attribute Status List (-) Specific Status Code	Returns a response from FAL User to a target object to write specified values of specified list of attributes of the specified object class or instance
Get_attribute_single.rsp	FAL User	(+) Attribute Data (-) Specific Status Code	Returns a response from FAL User to a target object to supply values of the specified attribute of the specified object class or instance
Set_attribute_single.rsp	FAL User	(+) Object Specific Data (optional) (-) Specific Status Code	Returns a response from FAL User to a target object to write the specified values into specified attribute of the specified object class or instance
Reset.rsp	FAL User	(+) Object Specific Data (optional) (-) Specific Status Code	Returns a response from FAL User to a target object to confirm that the instance of the specified object class has been reset

Primitive name	Source	Additional parameters	Functions
Create.rsp	FAL User	(+) Object Specific Data (optional) (-) Specific Status Code	Returns a response from FAL User to a target object to confirm that the instance of the specified object class has been created
Delete.rsp	FAL User	(+) Object Specific Data (optional) (-) Specific Status Code	Returns a response from FAL User to a target object to confirm that the instance of the specified object class has been deleted
Start.rsp	FAL User	(+) Object Specific Data (optional) (-) Specific Status Code	Returns a response from FAL User to a target object to confirm that the instance of the specified object class has started
Stop.rsp	FAL User	(+) Object Specific Data (optional) (-) Specific Status Code	Returns a response from FAL User to a target object to confirm that the instance of the specified object class has stopped
Find_next_object_instance.rsp	FAL User	(+) Number Of List Members Instance ID List (-) Specific Status Code	Returns a response from FAL User to a target device to find the identifier of the next unused instance of the specified object class
NOP.rsp	FAL User	None	Returns a response from FAL User to a target object to confirm that the NOP has been received
Apply_Attributes.rsp	FAL User	Object Specific Data (optional)	Returns a response from FAL User to a target object to confirm that pending attribute values have been activated
Save.rsp	FAL User	Object Specific Data (optional)	Returns a response from FAL User to a target object to confirm that its attributes contents have been saved
Restore.rsp	FAL User	Object Specific Data (optional)	Returns a response from FAL User to a target object to confirm that its attributes contents have been restored

Table 338 – Primitives issued by FSPM to FAL user

Primitive name	Source	Additional parameters	Functions
Get_attribute_all.ind	FSPM	same as in .req primitive	Indicates reception of Get_attribute_all.req
Set_attribute_all.ind	FSPM	same as in .req primitive	Indicates reception of Set_attribute_all.req
Get_attribute_list.ind	FSPM	same as in .req primitive	Indicates reception of Get_attribute_list.req
Set_attribute_list.ind	FSPM	same as in .req primitive	Indicates reception of Set_attribute_list.req
Get_attribute_single.ind	FSPM	same as in .req primitive	Indicates reception of Get_attribute_single.req
Set_attribute_single.ind	FSPM	same as in .req primitive	Indicates reception of Set_attribute_single.req
Reset.ind	FSPM	same as in .req primitive	Indicates reception of Reset.req
Create.ind	FSPM	same as in .req primitive	Indicates reception of Create.req
Delete.ind	FSPM	same as in .req primitive	Indicates reception of Delete.req
Start.ind	FSPM	same as in .req primitive	Indicates reception of Start.req
Stop.ind	FSPM	same as in .req primitive	Indicates reception of Stop.req
Find_next_object_instance.ind	FSPM	same as in .req primitive	Indicates reception of Find_next_object_instance.req
NOP.ind	FSPM	same as in .req primitive	Indicates reception of NOP.req
Apply_Attributes.ind	FSPM	same as in .req primitive	Indicates reception of Apply_Attributes.req
Save.ind	FSPM	same as in .req primitive	Indicates reception of Save.req
Restore.ind	FSPM	same as in .req primitive	Indicates reception of Restore.req
Get_attribute_all.cnf	FSPM	same as in .rsp primitive	Indicates reception of Get_attribute_all.rsp
Set_attribute_all.cnf	FSPM	same as in .rsp primitive	Indicates reception of Set_attribute_all.rsp
Get_attribute_list.cnf	FSPM	same as in .rsp primitive	Indicates reception of Get_attribute_list.rsp
Set_attribute_list.cnf	FSPM	same as in .rsp primitive	Indicates reception of Set_attribute_list.rsp
Get_attribute_single.cnf	FSPM	same as in .rsp primitive	Indicates reception of Get_attribute_single.rsp
Set_attribute_single.cnf	FSPM	same as in .rsp primitive	Indicates reception of Set_attribute_single.rsp
Reset.cnf	FSPM	same as in .rsp primitive	Indicates reception of Reset.rsp
Create.cnf	FSPM	same as in .rsp primitive	Indicates reception of Create.rsp
Delete.cnf	FSPM	same as in .rsp primitive	Indicates reception of Delete.rsp
Start.cnf	FSPM	same as in .rsp primitive	Indicates reception of Start.rsp
Stop.cnf	FSPM	same as in .rsp primitive	Indicates reception of Stop.rsp
Find_next_object_instance.cnf	FSPM	same as in .rsp primitive	Indicates reception of Find_next_object_instance.rsp
NOP.cnf	FSPM	same as in .rsp primitive	Indicates reception of NOP.rsp
Apply_Attributes.cnf	FSPM	same as in .rsp primitive	Indicates reception of Apply_Attributes.rsp
Save.cnf	FSPM	same as in .rsp primitive	Indicates reception of Save.rsp
Restore.cnf	FSPM	same as in .rsp primitive	Indicates reception of Restore.rsp

5.5.3 Parameters of primitives

The parameters used with the primitives exchanged between the FSPM and the ARPM are described in Table 339.

Table 339 – Parameters used with primitives exchanged between FAL user and FSPM

Parameter name	Description
AREP : Local identifier UCMM Record identifier Transport identifier	Identifies the entity to be used to convey the service. This parameter may use a dedicated identifier associated with a local entity, the identifier of a UCMM transaction record previously created, or the transport identifier returned by the connection establishment process and associated with the selected AR.
Receiver/Server Local ID	Generated by the Message Router ASE of the responding node. Identifies locally this invocation of the service. It is used to associate service responses with indications.
Path : Routing Path Additional path	In the request, it specifies the FAL APO or FAL APO element that is the destination of the service request. In the indication, it contains only those segments beyond the logical class segment from the service request, i.e. the optional additional information for the target APO (Add.Path).
Port ID	Indicates on which port of the device the service indication arrived.
Service status : Status Code (mandatory) Extended Status (conditional)	Provides information on the result of service execution. It is returned in all confirmed service response primitives (+ and -). Status code indicates the type of error (see IEC 61158-5 for details). Extended status code gives details of the status (see IEC 61158-5 for details).
Attribute Data	1) A stream of information containing all of the attributes. Classes/Objects which support this service shall define the format of this parameter. 2) An array of structures, predefined by the system for the given service
Attribute Count	Number of attribute numbers in the attribute list
Attribute list []	List (array) of the attribute numbers of the attributes to get from the class or object
Attribute Number	Number representing the attribute value
Attribute Status	Status information of attribute
Attribute Value	Sequence of data specific to the attribute of the object or class
Object Specific Data	Class/Instance specific parameters. Class/Instance specification shall specify the format.
Instance ID	Value assigned to identify the newly created object
Maximum Returned Values	Maximum number of Instance ID values to be returned in the response message
Number Of List Members	Number of Instance IDs specified in response message
Instance ID List	Returned Instance ID List. The Instance IDs are returned in 16 bit integer fields.

5.5.4 FSPM state machines

FSPM State Machine has got only one possible state : Running.

5.6 Application Relationship Protocol Machines (ARPMs)

5.6.1 General

This fieldbus has Application Relationship Protocol Machines for :

- connection-less application relationships
- connection-oriented application relationships

Connection-less relationship is of one type only.

Connection-oriented relationships fit into one of 7 transport classes :

- 0 Null (or Base)
- 1 Duplicate Detection
- 2 Acknowledged
- 3 Verified
- 4 Non-blocking
- 5 Non-blocking, Fragmenting
- 6 Multipoint, Fragmenting

Although similar, each of these transport classes has its own ARPM.

5.6.2 Connection-less ARPM (UCMM)

5.6.2.1 General

Functions of the connection-less ARPM are provided by the Unconnected Message Manager (UCMM) object. UCMM shall provide an unconnected request/response message services, limited to a single link that supports multiple outstanding messages. The required number of outstanding messages shall be implementation specific. The UCMM shall be present in all nodes, and shall support at least one outstanding message.

5.6.2.2 Primitive definitions

The UCMM Object shall provide the following services :

UCMM_Create	Creates an instance of transaction record. Puts UCMM into Running state. This service is local, it does not result in a PDU being sent.
UCMM_Delete	Deletes existing transaction record. Puts UCMM into Inactive state. This service is local, it does not result in a PDU being sent. This service is local, it does not result in a PDU being sent.
UCMM_Write	Writes an item of application data into Transport PDU buffer ; this results in sending the data to a specified target.
UCMM_Abort	Aborts existing transaction.

Refer to IEC 1158-5 for detailed definition of these services.

Primitives exchanged between UCMM and FSPM are shown in the following Table 340 and Table 341.

Table 340 – Primitives issued by FSPM to ARPM

Primitive name	Source	Associated parameters	Functions
UCMM_Create_req	FSPM	fixed tag max retries,	Conveys a request from the FSPM to the ARPM to create a transaction record.
UCMM_Delete_req	FSPM	record	Conveys a request from the FSPM to the ARPM to delete previously established transaction record.
UCMM_Write_req	FSPM	record destination ID UCMM service response timeout transaction priority application data	Conveys a request from the FSPM to the ARPM to send an item of application data using a previously created transaction record.
UCMM_Write_rsp	FSPM	record application data	Conveys a response from the FSPM, via Message Router to the ARPM to send a response to UCMM_Send_req using a previously created transaction record.
UCMM_Abort_req	FSPM	record	Aborts the transaction corresponding to the specified transaction record ; removes the packet from the local DLL if it has not yet been transmitted.

Table 341 – Primitives issued by ARPM to FSPM

Primitive name	Source	Associated parameters	Functions
UCMM_Create_cnf	ARPM	record service_status	Conveys a confirmation from the ARPM to the FSPM that transaction record has been created.
UCMM_Write_ind	ARPM	record source ID application data	Conveys an indication from the ARPM to the Message Router that data has arrived on the previously created transaction record. The Message Router then passes the indication to the appropriate application object.
UCMM_Write_cnf	ARPM	record service status application data	Conveys a confirmation from the ARPM to the Message Router of the execution of FSPM UCMM_Send_req on a previously created transaction record.

5.6.2.3 Parameters of primitives

The parameters used with the primitives exchanged between the FSPM and the ARPM are described in Table 342 .

Table 342 – Parameters used with primitives exchanged between FSPM and ARPM

Parameter name	Description
record	Identifies unambiguously the instance of the temporary connection along which the FSPM has issued a UCMM_Send request primitive. A means for such identification is not specified in this document.
application data	Conveys FAL-User data.
fixed tag	Set to value of 0x83 for UCMM and 0x88 for Management UCMM
destination ID	Unambiguous identifier (MAC ID) of the source node of UCMM_write_req.
source ID	Unambiguous identifier (MAC ID) of the destination node of UCMM_write_req.
service	The service parameter shall specify the delivery mechanism to use for this message and shall be one of : Request_With_Acknowledge = 2 (retries untill acknowledged) Request_With_No_Response = 4 (no acknowledgement) Request_With_No_ACK = 7 (retries untill response) Request_With_No_Retry = 8 (with response) NOTE These values correspond to command codes contained in the UCMM header
timeout	Duration of the transaction in microseconds. If no UCMM_Send_cnf is received before the time-out expires, the transaction is aborted and the send_status parameter shall be TIMEOUT.
retries	Conveys the maximum allowable number of retries.
create_status	Status returned with UCMM_Create_conf = - success - cannot create
send_status	Status returned with UCMM_Send_conf = - success - record_not_created - packet_too_big - no_acknowledge - aborted - timeout

5.6.2.4 UCMM state machines

5.6.2.4.1 UCMM client

5.6.2.4.1.1 States

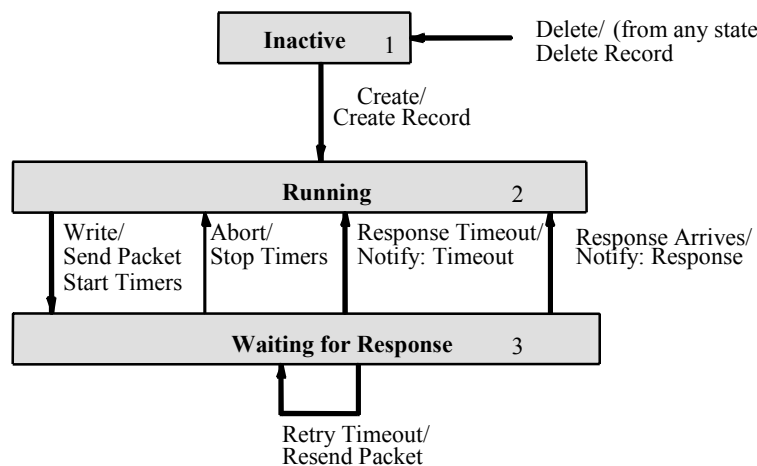
The defined states and their descriptions of the UCMM Client are listed in Table 343 below :

Table 343 – UCMM client states

State	Description
Inactive	The record for UCMM transfer does not exist.
Running	The record for UCMM transfer has been created.
Waiting for response	Client sent an item of application data and is waiting for a response.

5.6.2.4.1.2 State transition diagram

Figure 61 shows the state transition diagram of UCMM client.

**Figure 61 – State transition diagram of UCMM client**

5.6.2.4.1.3 State event matrix

The state transitions for the UCMM Client shall be as specified in Table 344.

Table 344 – State event matrix of UCMM client

Event	State		
	Inactive	Running	Waiting for response
Create.req	1) create record 2) transition to Running		
Delete.req	no action	1) delete record 2) increment SEQUENCE 3) transition to Inactive	1) Delete record 2) increment SEQUENCE 3) transition to Inactive
Write.req	confirm status = INVALID_RECORD	1) send packet 2) start Response Timer 3) initialise Retry Count 4) start Retry Timer 5) transition to Waiting	confirm status = BUSY
Abort.req	confirm status = INVALID_RECORD	no action	1) stop timers 2) increment SEQUENCE 3) transition to Running
Retry Timeout Request packet available			1) Decrement Retry Count IF Retry Count expired 2) Notify: Timeout-No ACK 3) Free request buffer 4) Increment Sequence # 5) Stop Response Timer 6) Transition to Running ELSE. 1) Resend packet 2) Restart Retry Timer
Response Timeout			1) confirm with status = TIMEOUT-No Response 2) increment SEQUENCE 3) transition to Running
Response Arrives		no action	1) confirm with status = SUCCESS 2) Send ACK_RESP, unless response indicates no ACK_RESP desired 3) increment SEQUENCE 4) transition to Running
ACK_REQ Arrives, sequence number matches stored value		No Action	1) Free request buffer 2) stop retry timer
ACK_REQ Arrives, sequence number not equal to stored value		No Action	No Action

The sequence number shall be set to zero at initialisation. The sequence number value shall be maintained in the inactive state, to be used when the record transitions to running. The retry timeout value shall be fixed for the link at a value which guarantees that both the client and server nodes have an opportunity to transmit an unscheduled packet.

The response time-out is the response time provided when the record is created.

5.6.2.4.2 High-end UCMM server

5.6.2.4.2.1 Functions

When a packet arrives at the server an instance of a transaction shall be created if resources are available. If resources are not available the packet shall be dropped. When the instance is created, a transaction record shall be created for that instance. This record shall be active for the life of this transaction. The transaction shall end :

- when an ACK_RESP is received after a response was sent;
- when the response time timer expires;
- or, when a new packet is received after a response was sent.

5.6.2.4.2.2 States

The defined states and their descriptions of the UCMM High-end Server are listed in Table 345 below :

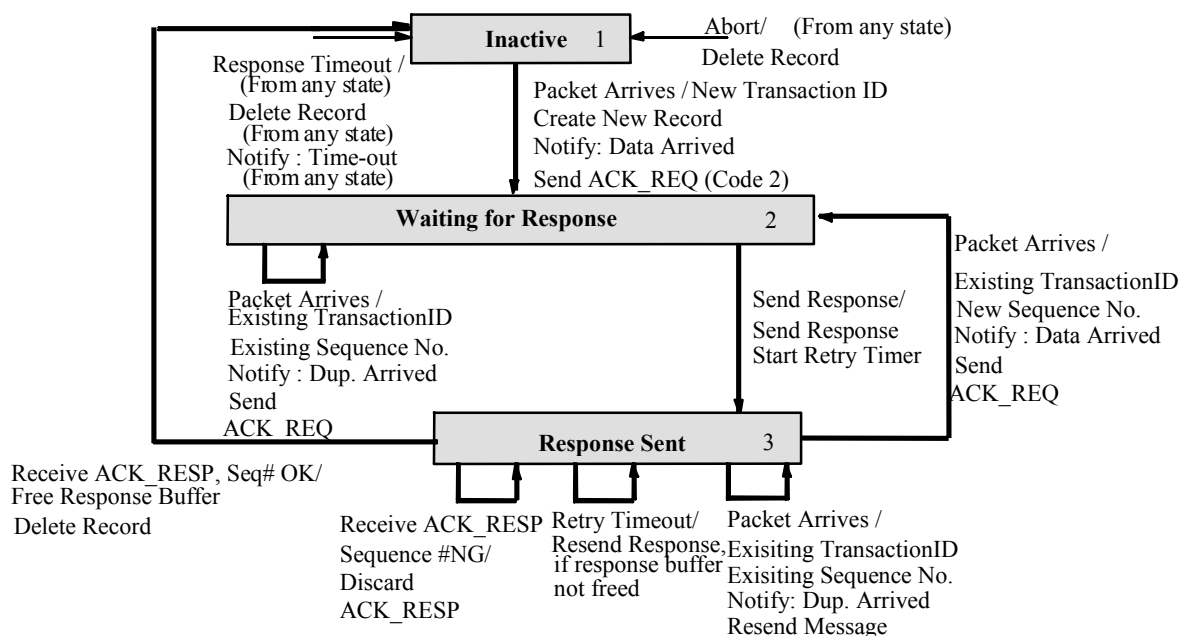
Table 345 – High-end UCMM server states

State	Description
Inactive	The record for UCMM transfer does not exist.
Waiting for Response	Data packet arrived with new Transaction ID. New record is created. The Server is waiting for response to be sent by the Server application.
Response sent	Server application sent a response.

5.6.2.4.2.3 State transitions

Figure 62 shows the high-end UCMM Server state transition diagram.

Figure 62 – State transition diagram of high-end UCMM server



5.6.2.4.2.4 State event matrix

The state transitions for the high-end UCMM server shall be as specified in Table 346.

The sequence number shall be set to zero at initialisation. The sequence number value shall be maintained in the Inactive state, to be used when the record transitions to Running. The retry time-out value shall be fixed for the local link.

The response time-out shall be the response time provided when the record is created.

Table 346 – State event matrix of high-end UCMM server

Event	State		
	Inactive	Waiting for response	Response sent
Packet arrives request (code 2) New Transaction ID and source address	1) Create new record 2) Notify: Data Arrived 3) Send ACK_REQ 4) Start Response Timer 5) Transition to Waiting for App Response	Not Applicable	Not Applicable
Packet arrives request (code 7) New Transaction ID and source address	1) Create new record 2) Notify: Data Arrived 3) Start Response Timer 4) Transition to Waiting for App Response	Not Applicable	Not Applicable
Packet arrives Existing Transaction ID and source address New Sequence #	Not Applicable	No Action	1) Update Record 2) Notify: Data Arrived 3) Send ACK_REQ 4) Transition to Waiting for App Response
Packet arrives Existing Transaction ID Existing Sequence #	Not Applicable	1) Notify: Dup. Arrived (Optional) 2) Send ACK_REQ	1) Notify: Dup. Arrived (Optional) 2) Resend Response Message
Response Timer Expires	Not Applicable	1) Notify: Timeout 2) Delete record 3) Transition to Inactive	1) Notify: Timeout 2) Delete record 3) Transition to Inactive
Abort	Error	1) Delete record 2) Transition to Inactive	1) Delete record 2) Transition to Inactive
Send Response	Error	1) Send Response 2) Start Retry Timer 3) Transition to Response Sent	Error
Retry Timeout	Not Applicable	Not Applicable	1) Decrement Retry Count IF Retry Count expired 2) Free Response buffer 3) Stop Response Timer 4) Transition to Inactive ELSE 1) Resend Response message 2) Start Retry Timer
ACK_RESP arrives, sequence number matches stored value	No action	No action	1) Delete record 2) Transition to Inactive
ACK_RESP arrives, sequence number not equal to stored value	No action	No action	No action

The response time is the response time received in the message header.

5.6.2.4.3 Low-end UCMM server

5.6.2.4.3.1 Functions

The low-end server cannot support the following features:

- perform response message retries
- detect duplicate messages

5.6.2.4.3.2 States

The defined states and their descriptions of the UCMM Low-end Server are listed in Table 347 below :

Table 347 – Low-end UCMM server states

State	Description
Inactive	The record for UCMM transfer does not exist.
Waiting for Response	Data packet arrived with new Transaction ID. New record is created. The Server is waiting for response to be sent by the Server application.

5.6.2.4.3.3 State transitions

Figure 63 shows the low-end UCMM server state transition diagram.

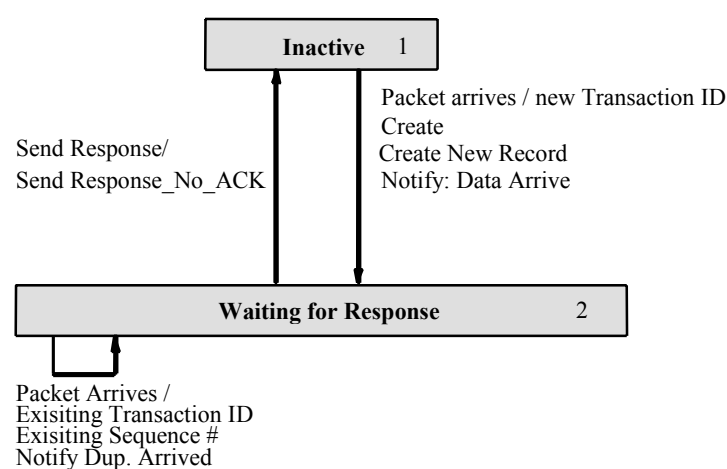


Figure 63 – State transition diagram of low-end UCMM server

5.6.2.4.3.4 State event matrix

The state transitions for the Low-end UCMM server shall be as specified in in Table 348.

Table 348 – State event matrix of low-end UCMM server

Event	State	
	Inactive	Waiting for response
Packet arrives New Transaction ID and source address	1) Create new record 2) Notify: Data Arrived 3) IF Response not immediately available, Send ACK_REQ 4) Transition to Waiting for App Response	Not Applicable
Packet arrives Existing Transaction ID and source address New Sequence #	No action	No action
Packet arrives Existing Transaction ID Existing Sequence #	No action	1) Notify: Dup. Arrived (Optional)
Send Response	Error	1) Send Response_No_ACK 2) Transition to Inactive
ACK_RESP arrives	No action	No Action

5.6.2.5 Examples of UCMM sequences

Figure 64 shows a sequence diagram for a UCMM with one outstanding message and Figure 65 shows a sequence diagram for a UCMM with multiple outstanding messages.

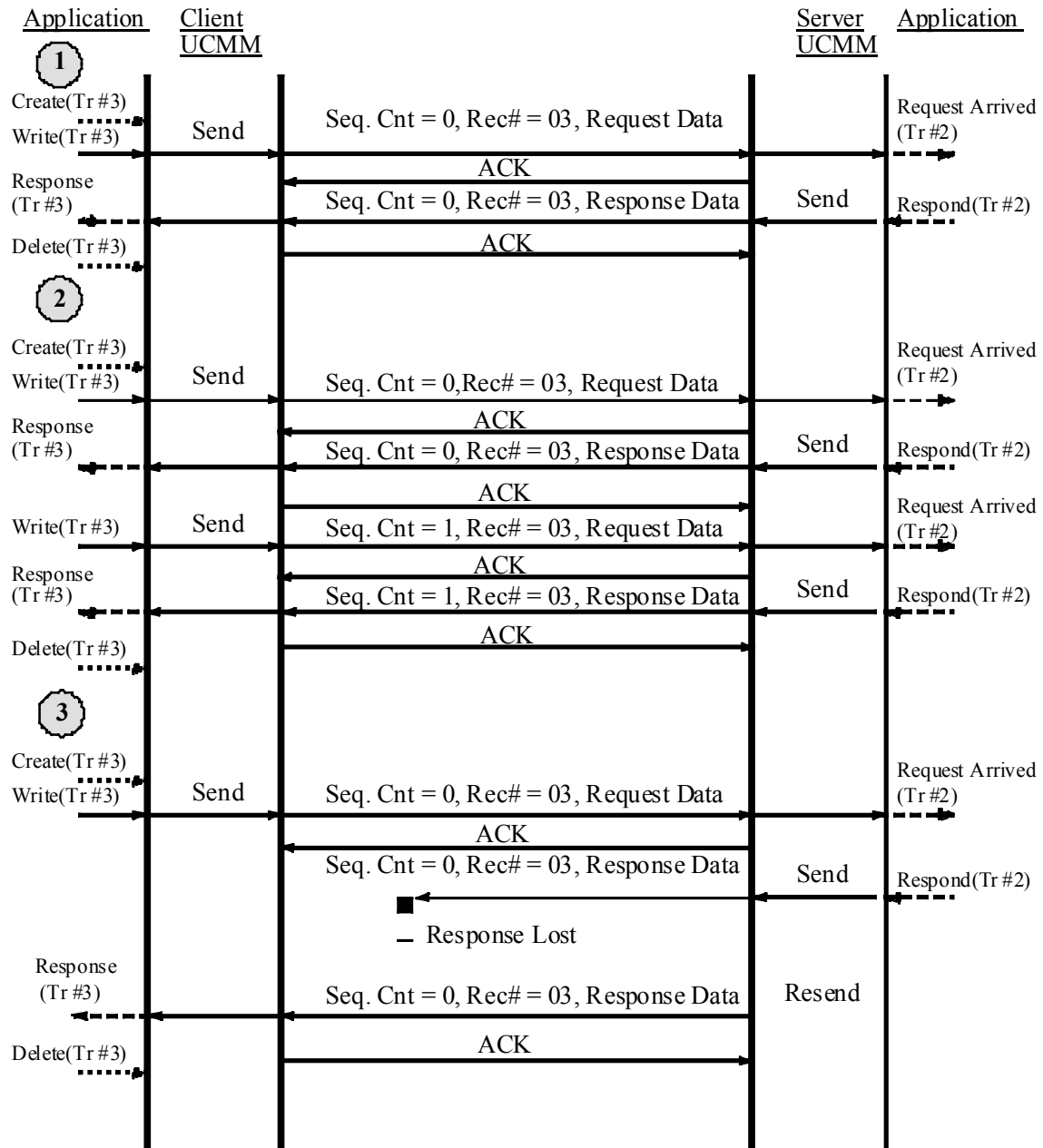


Figure 64 – Sequence diagram for a UCMM with one outstanding message

Example 1. Typical transaction: Instance is created, request is written, response is returned, and instance is deleted.

Example 2. Two transactions/same instance: In this case the instance is opened and two requests are sent. Note that the client waits for the first response to return before issuing a second request using the same transaction number.

Example 3. Lost response: In this case the response is lost and the server issues a retry.

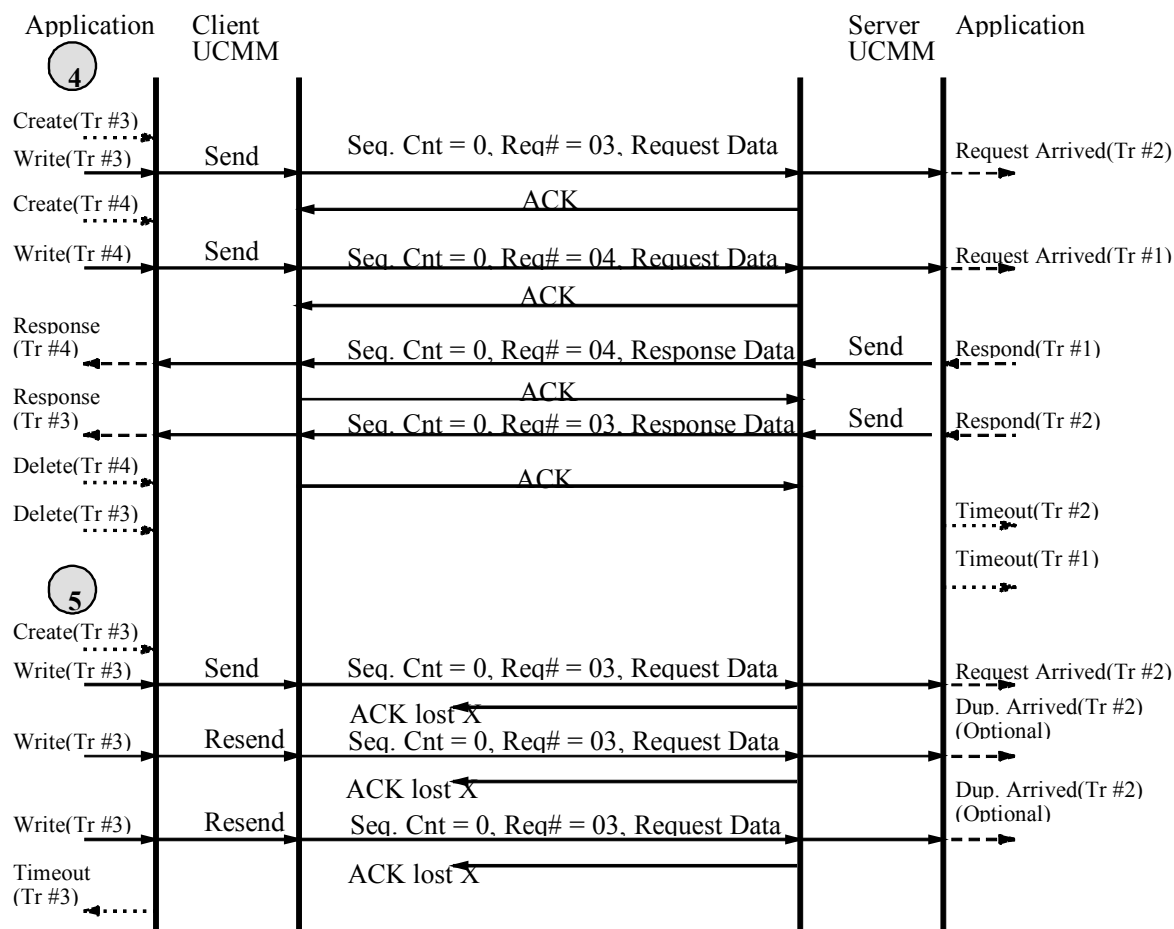


Figure 65 – Sequence diagram for a UCMM with multiple outstanding messages

Example 4. Multiple outstanding: In this case the client creates two instances and sends a second request before the first request has completed. Note that the client uses a different transaction number for the second request.

Example 5. Timeout: In this case the server application does not respond and the client issues a timeout. The timeout causes an error status to be returned to the application. Transaction instance shall be deleted.

5.6.2.6 Management UCMM

Any device that has implemented the Keeper object shall also implement the Management UCMM server. With two exceptions, the Management UCMM server shall be identical to the fixed tag 0x83 UCMM server:

- transactions for the Management UCMM server shall be transmitted on fixed tag 0x88;
- the Management UCMM server shall not transmit any packets unless its node contains a Keeper object in the master state.

Any device may implement the Management UCMM client. The Management UCMM client is identical to the fixed tag 0x83 UCMM client except that transactions for the Management UCMM client shall be transmitted on fixed tag 0x88.

NOTE Messages to the Keeper object are broadcast since all devices are permitted to implement this object. To facilitate communication to the Keeper object without impacting non-Keeper devices, these messages are transmitted on a different fixed tag.

5.6.3 Connection-oriented ARPMs (transports)

5.6.3.1 Transport PDU buffer

5.6.3.1.1 Transport PDU format

The TPDU buffer contains a TPDU packet. The TPDU packet consists of a transport header and data. Applications shall write data to and read data from the TPDU buffers directly. Instances of transports shall write and read the transport header in the TPDU buffer while consumers shall write data to the TPDU buffer and producers shall read data from the TPDU buffer. The TPDU buffers and buffer management are not part of the communication services. This process is shown in Figure 66.

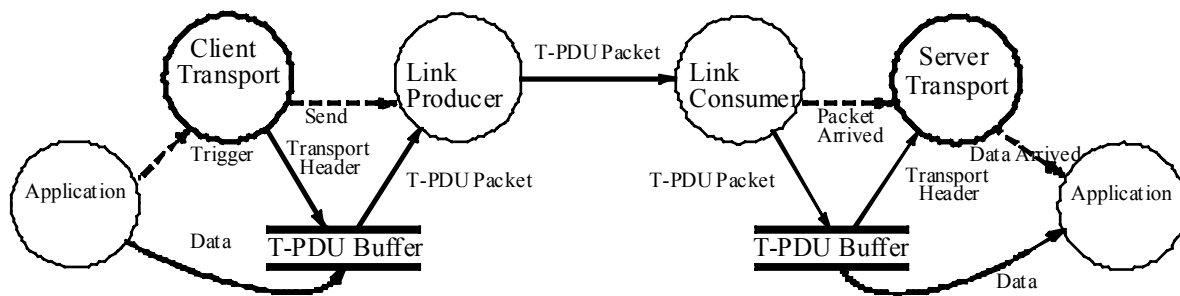


Figure 66 – TPDU buffer

5.6.3.1.2 Transport PDU buffer management

TPDU buffer management is implementation specific. Implementations that use single buffering, double buffering, overwrite or queuing to manage TPDU buffers are all allowed.

Implementers are responsible for ensuring buffer integrity (atomic access).

Applications are responsible for ensuring that the buffer has been initialised before a trigger is issued. Failure to initialise the buffer shall allow unknown data to be sent. This is true for producers and the client and server sides.

5.6.3.1.3 Notification

Transport classes can signal the application through events. The valid events are as specified in Table 349 :

Table 349 – Notification

Event	Client				Server			
	Class 0	Class 1	Class 2	Class 3	Class 0	Class 1	Class 2	Class 3
Data Arrived					n	n	n	n
Duplicate Arrived						n	n	n
Acknowledgement			n					
Verification				n				

NOTE A *Duplicate Arrived* event is normally not of interest to the application; however, it might be useful in triggering a watchdog timer to indicate that the client's application is alive and triggering.

5.6.3.2 Transport classes

Applications interface to transport services through the supported transport classes. Table 350 lists the general-purpose transport classes that have been defined for the systems. Each transport class provides a different level of functionality. Transport class 0 provides the minimum functionality required of a transport class, enabling data transfer between applications.

Table 350 – Transport classes

Class number	Class name
0	Null (or Base)
1	Duplicate Detection
2	Acknowledged
3	Verified
4	Non-blocking
5	Non-blocking, Fragmenting
6	Multipoint, Fragmenting

5.6.3.3 Common primitive definitions

Transport classes shall provide the following services :

TR_Write	Writes an item of application data into sending Transport PDU buffer, ready to be sent through the pre-established connection.
TR_Trigger	Causes an item of application data or response data previously placed in sending Transport PDU buffer, to be sent through the pre-established connection.
TR_Packet_arrived	Indicates to the Transport instance that a data packet arrived in the receiving Transport PDU buffer. This service is initiated by local action.
TR_Ack_received	Indicates to the user that a data packet arrived in the receiving Transport PDU buffer indicating an acknowledgement of arrival of previously sent data in the receiving Transport PDU buffer. This service is initiated by local action.
TR_Verify	Indicates to the user that a data packet arrived in the receiving Transport PDU buffer indicating a verification of arrival of previously sent data in the receiving user object. This service is initiated by local action.
TR_Status_Update	Indicates to the user that an updated status is present in the Transport PDU buffer.

Refer to IEC 1158-5 for detailed definition of these services.

Primitives exchanged between Transport instances and FSPM are shown in the following Table 351 and Table 352.

Table 351 – Primitives issued by FSPM to ARPM

Primitive name	Source	Associated parameters	Functions
TR_Write_req	FSPM	Transport identifier Application data	Conveys a request from the FSPM to the ARPM to write application data into Transport PDU buffer.
TR_Trigger.req	FSPM	Transport identifier	Conveys a request from the FSPM to the ARPM to trigger transfer of application data from Transport PDU buffer to the Link Producer.
TR_Verify.req	FSPM	Transport identifier	Conveys a request from the FSPM to the ARPM to trigger transfer of write verification data into Transport PDU buffer.

Table 352 – Primitives issued by ARPM to FSPM

Primitive name	Source	Associated parameters	Functions
TR_Packet_arrived.ind	ARPM	Transport identifier Data arrived Duplicate arrived	Conveys an indication to the FSPM that an item of data has arrived at the Link Consumer.
TR_Ack_arrived.ind	ARPM	Transport identifier	Conveys an indication to the FSPM that an acknowledgement has arrived at the Link Consumer.
TR_Status_Update.ind	ARPM	Transport identifier	Conveys an indication to the FSPM that a new status has arrived at the Link Consumer.
TR_Verify.cnf	ARPM	Transport identifier	Conveys a confirmation from the ARPM to FSPM to indicate reception of verification DLPDU.

5.6.3.4 Parameters of common primitives

The parameters used with the primitives exchanged between the FSPM and the ARPM are described in Table 353 .

Table 353 – Parameters used with primitives exchanged between FSPM and ARPM

Parameter name	Description
Transport identifier	Identifier of the instance of the transport invoked in the transaction.
Application data	Contains application data (service request/response parameters or user formatted data).
Data arrived Duplicate arrived	The Data_Arrived and Duplicate_Arrived parameters indicate whether the new packet contains new data, or whether it is just a duplicate from a previously received packet.

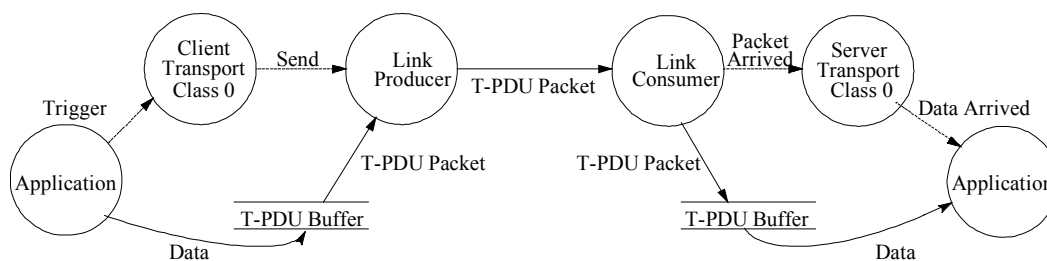
5.6.3.5 Transport state machines – class 0

5.6.3.5.1 Functions

Transport class 0 is the simplest transport class, and can use either a point-to-point or multi-point connection. Class 0 is typically used to transmit or receive inputs on a multipoint connection or outputs on a point-to-point connection.

Possible uses of transport class 0 include sampled inputs, standard outputs, cyclic block transfers, diagnostic events, and communication between controllers and operator interface devices.

Since transport class 0 does not detect duplicate data packets, the target application is responsible for detecting them. A duplicate data packet is a retransmission of the last data packet sent. Figure 67 shows the actions which take place during a data transfer using client transport class 0 and server transport class 0.



NOTE: The client transport instance does not write a sequence count to the client-side T-PDU buffer, and the server transport instance does not read a sequence count from the server-side T-PDU buffer. The transport header is defined as a don't care.

Figure 67 – Data flow diagram using a client transport class 0 and server transport class 0

Figure 68 shows the sequence in which actions take place when transferring data using client transport class 0 and server transport class 0.

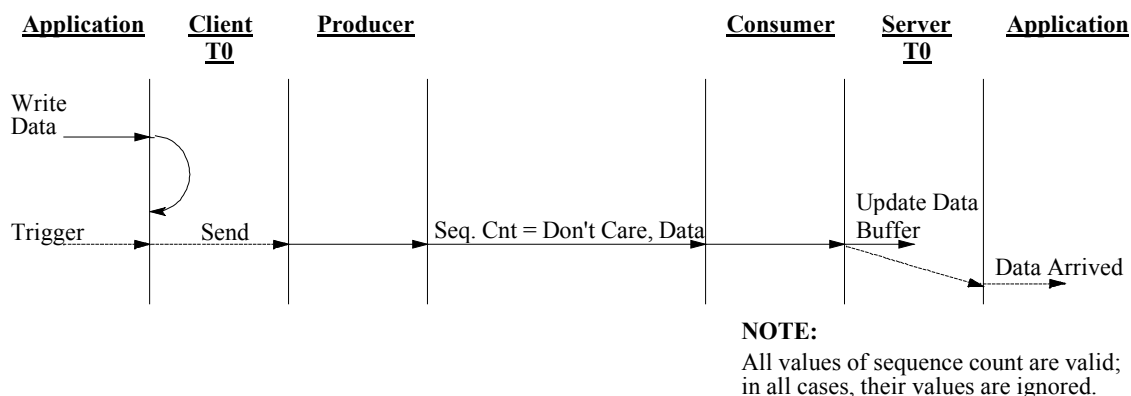


Figure 68 – Sequence diagram of data transfer using transport class 0

5.6.3.5.2 Transport class 0 client

5.6.3.5.2.1 Functions

The function of client transport class 0 is just to pass the *trigger* service from the application to the producer as a *send* service. It is useful for the client transport class 0 to be shown from a high-level design view. In actual implementations, the application may trigger the producer directly. An idle state is provided for consistency with other transport classes. In the *idle* state all events, except *delete* and *start*, are ignored.

A class 0 client transport is responsible for:

- Accepting the client application's trigger that it has written a data packet to the client-side T-PDU buffer
- Writing a transport header to the T-PDU buffer for each packet that the client application writes to the client-side T-PDU buffer (optional)
- Triggering the producing node of the network connection to produce the T-PDU packet on the link and send it to the consuming node of the network connection

5.6.3.5.2.2 States

The defined states and their descriptions of the Class 0 Transport Client are listed in Table 354 below.

Table 354 – Class 0 transport client states

State	Description
Non-existent	The instance of Transport Class 0 does not exist.
Idle	The instance of Transport Class 0 has been created but not yet started.
Running	The instance of Transport Class 0 has been created and started.

5.6.3.5.2.3 State transitions

State transition diagram of the class 0 client transport is shown in Figure 69.

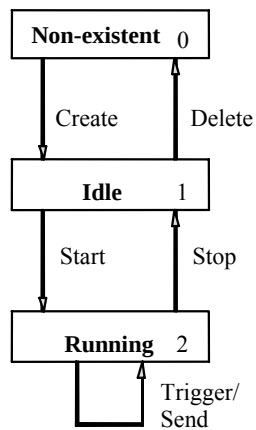


Figure 69 – Class 0 client STD

5.6.3.5.2.4 State event matrix

State event matrix of the class 0 client transport is shown in Table 355.

Table 355 – Class 0 client SEM

Event	State		
	Non-existent	Idle	Running
Create	Transition to Idle	Error	Error
Delete	Error	Transition to Non-existent	Error
Start	Error	Transition to Running	Error
Stop	Error	No Action	Transition to Idle
Trigger	Error	No Action	Send

5.6.3.5.3 Transport class 0 server

5.6.3.5.3.1 Function

The function of server transport class 0 is to pass the *packet arrived* event from the consumer to the producer as a *data arrived* event. It is useful for the server transport class 0 to be shown from a high-level design view. In actual implementations, the application may receive the event directly from the consumer. In the *idle* state all packet arrivals are ignored.

A class 0 server transport is responsible for:

- Accepting the notification from the consuming node of the network connection that it has written a data packet to the server-side T-PDU buffer
- Locating and reading the transport header of each packet that the consuming node of the network connection writes to the server-side T-PDU buffer
- Notifying the server application that data has been written to the server-side T-PDU buffer

5.6.3.5.3.2 States

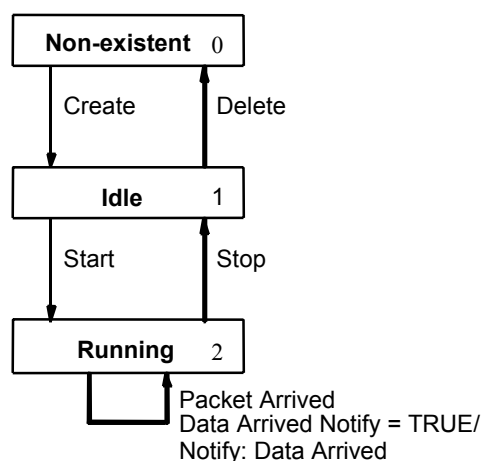
The defined states and their descriptions of the Class 0 Transport Server are listed in Table 356 below:

Table 356 – Class 0 transport server states

State	Description
Non-existent	The instance of Transport Class 0 does not exist.
Idle	The instance of Transport Class 0 has been created but not yet started.
Running	The instance of Transport Class 0 has been created and started.

5.6.3.5.3.3 State transitions

State transitions of the class 0 server transport are shown in Figure 70

**Figure 70 – Class 0 server STD**

5.6.3.5.3.4 State event matrix

State event matrix of the class 0 server transport is shown in Table 357.

Table 357 – Class 0 server SEM

Event	State		
	Non-existent	Idle	Running
Create	Transition to Idle	Error	Error
Delete	Error	Transition to Non-existent	Error
Start	Error	Transition to Running	Error
Stop	Error	No action	Transition to Idle
Packet Arrived & Data Arrived Notify = TRUE	No action	No action	Notify: Data Arrived

5.6.3.6 Transport state machines – class 1

5.6.3.6.1 Functions

Transport class 1, like class 0, shall allow either a point-to-point or multipoint connection. Unlike class 0, which has no transport header, the class 1 transport shall include a sequence number, which is used to detect the delivery of duplicate data packets.

NOTE 1 Class 1 is preferred to class 0 for targets because the target application does not have to perform duplicate detection, which reduces the overhead required in those end nodes that support the change of state transport trigger. Class 1 permits an efficient change of state trigger, since was designed to allow change of state data transfers.

Figure 71 shows the actions which take place during data transfer using a class 1 client transport instance and a class 1 server transport instance.

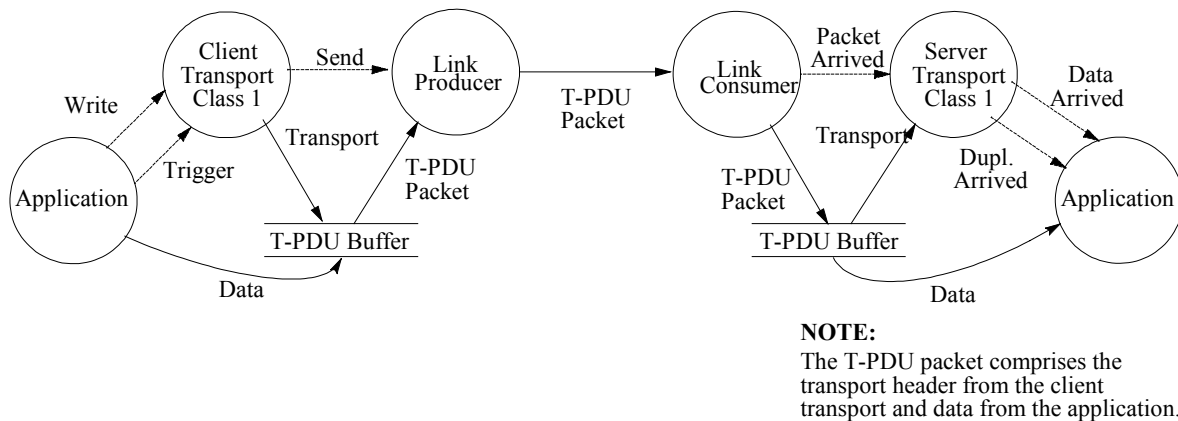
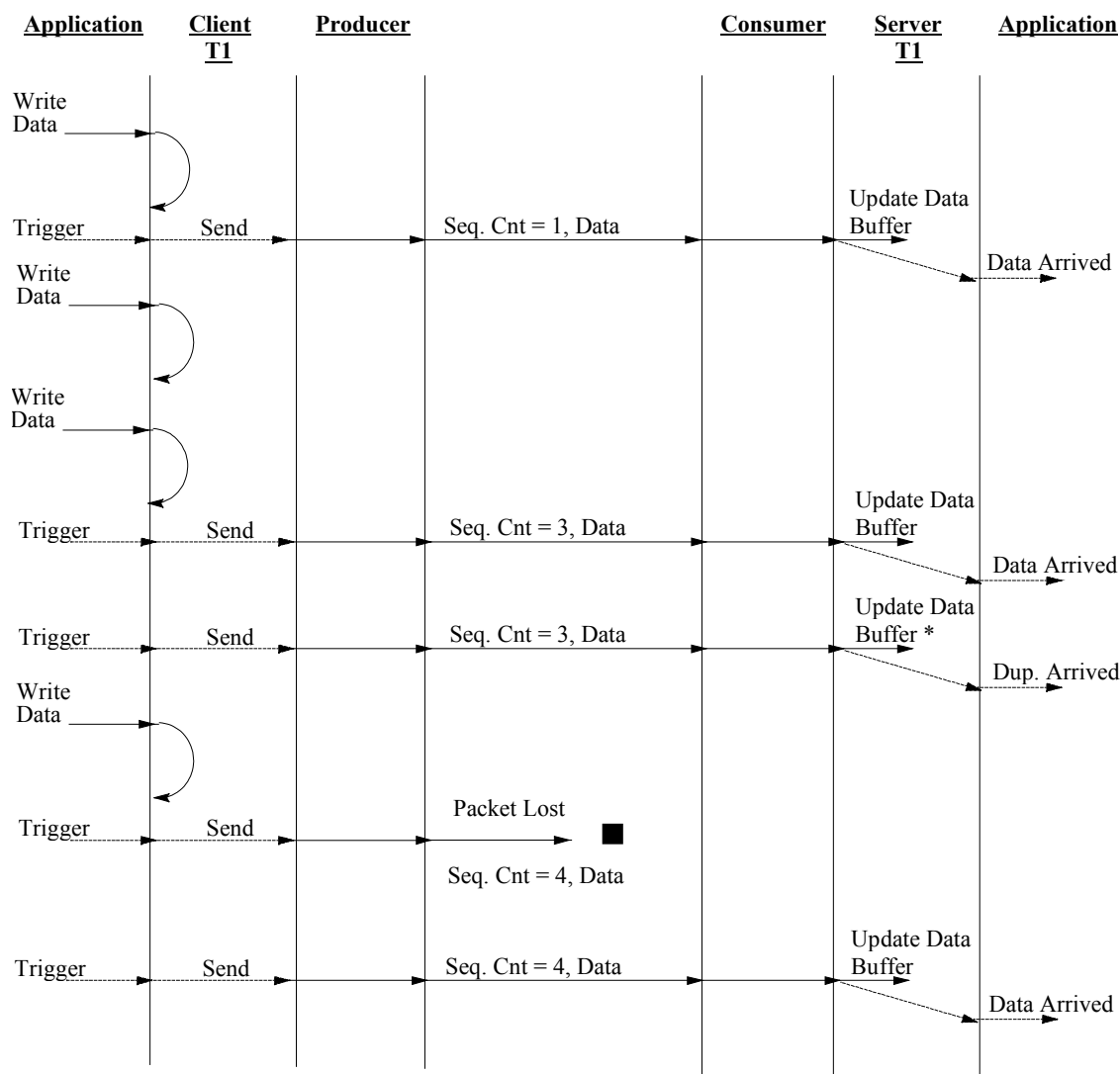


Figure 71 – Data flow diagram using client transport class 1 and server transport class 1

Figure 72 shows the sequence in which actions take place when transferring data using client transport class 1 and server transport class 1. Note that the sequence count is incremented with every write. Also note that in the case where the packet is lost on a new data sample, and the packet is later triggered and sent, the data received by the client is treated as new data. This is because this is the first time the server has received this sequence count. This mechanism provides fault tolerance for those samples that change infrequently.

NOTE 2 It is an implementation choice to decide whether to update the data buffer upon receipt of duplicate data or not, based on the potential impact of the additional processing.



**Figure 72 – Sequence diagram of data transfer
using client transport class 1 and server transport class 1**

Transport class 1 can be used in conjunction with a heartbeat timer to keep a connection open when there are periods when no data is transmitted. The heartbeat timer shall be initialised every time the producing node of the network connection transmits a packet, and shall be set so that its maximum value, or the maximum time between data transmissions, is less than the path time-out value for the network connection. When the heartbeat timer reaches its maximum value, it triggers the client transport instance so that the producing node produces and retransmits the packet in the client-side T-PDU buffer before the connection times out. The consuming node of the network connection, recognising that the packet has the same sequence count as the last packet it received, may or may not overwrite the packet in the server-side T-PDU buffer; that implementation decision is left to the product developers, since they can best assess the impact of additional processing.

Applications could use this transport class to distinguish between new samples and old samples that were sent to maintain the connection. To maintain the connection, a timer may be used to trigger the production of the current data in the TPDU buffer. To transmit new data the application would first write to the TPDU buffer and then trigger the client transport.

Applications could use this transport class to distinguish between new samples and old samples that were sent to maintain the connection. To maintain the connection, the data arrived and duplicated arrived events may be ORed together to reset a timer. If this timer were to expire, the application would know that no data was received within the designated time. The data arrived event would identify new samples of data. This may allow applications to reduce their overhead by only processing new data samples.

Possible uses of transport class 1 include sampled inputs, standard outputs, cyclic block transfers, diagnostic events, and communication between controllers and operator interface devices.

5.6.3.6.2 Transport class 1 client

5.6.3.6.2.1 Functions

Transport class 1 starts with the behaviour in class 0 and adds a sequence count. This sequence count is incremented by the client transport class when data is written to the data buffer. When the transport receives a *write* event, it shall increment the sequence count. When a *trigger* event is received the transport shall update the sequence count in the TPDU buffer and invoke the *send* service.

Applications could use this transport class to distinguish between new samples and old samples that were sent to maintain the connection. To maintain the connection, a timer may be used to trigger the production of the current data in the TPDU buffer. To transmit new data the application would first write to the TPDU buffer and then trigger the client transport.

A class 1 client transport is responsible for:

- Accepting the client application's trigger that it has written a data packet to the client-side T-PDU buffer
- Writing a transport header to the T-PDU buffer for each packet that the client application writes to the client-side T-PDU buffer.
- Initialising the sequence count in the transport header (for the first packet transmitted) or incrementing the sequence count (for subsequent packets of the same transmission)
- Triggering the producing node of the network connection to produce the T-PDU packet on the link and send it to the consuming node of the network connection

5.6.3.6.2.2 States

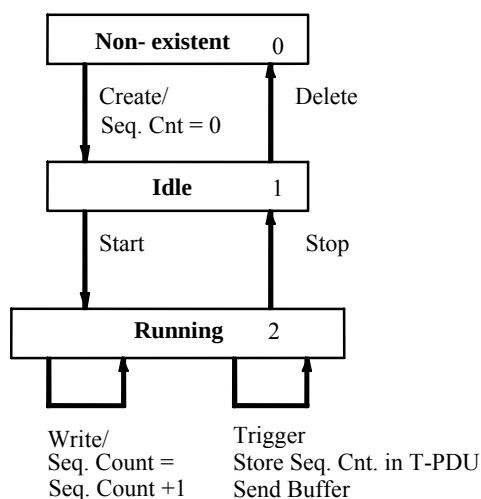
The defined states and their descriptions of the Class 1 Transport Client are listed in Table 358 below :

Table 358 – Class 1 transport client states

State	Description
Non-existent	The instance of Transport Class 1 does not exist.
Idle	The instance of Transport Class 1 has been created but not yet started.
Running	The instance of Transport Class 1 has been created and started.

5.6.3.6.2.3 State transitions

State transitions of the class 1 client transport are shown in Figure 73.



NOTE The sequence count is set to 0 by the create service. It is then incremented after every *write* event until it reaches a maximum value of $2^{16}-1$. After that it rolls over to 0.

Figure 73 – Class 1 client STD

5.6.3.6.2.4 State event matrix

State transitions of the class 1 client transport are shown in Table 359.

Table 359 – Class 1 client SEM

Event	State		
	Non-existent	Idle	Running
Create	1) Transition to Idle 2) Set seq. count = 0	Error	Error
Delete	Error	Transition to Non-existent	Error
Start	Error	Transition to Running	Error
Stop	Error	No Action	Transition to Idle
Write	Error	No Action	1) Seq. Cnt = Seq. Cnt + 1
Trigger	Error	No Action	1) Store Seq. Cnt. in TPDU Buffer 2) Send

5.6.3.6.3 Transport class 1 server

5.6.3.6.3.1 Functions

Transport class 1 starts with the behaviour in class 0 and adds a sequence count. The received sequence count is compared with the previous sequence count. If they are equal, the received packet is considered to be a duplicate packet. If they are not equal, the received data is considered to be a new sample. There is no attempt to count how many sequence counts have changed between samples.

Applications could use this transport class to distinguish between new samples and old samples that were sent to maintain the connection. To maintain the connection, the data arrived and duplicated arrived events may be ORed together to reset a timer. If this timer were to expire, the application would know that no data was received within the designated time. The data arrived event would identify new samples of data. This may allow applications to reduce their overhead by only processing new data samples.

A class 1 server transport is responsible for

- Accepting the notification from the consuming node of the network connection that it has written a data packet to the server-side T-PDU buffer;
- Locating and reading the transport header of each packet that the consuming node of the network connection writes to the server-side T-PDU buffer;
- Triggering the server application that data has been written to the server-side T-PDU buffer;
- Comparing the sequence count of each packet with that of the previous packet;
- Notifying the server application that the most recently arrived packet is a duplicate of the previous one when their sequence counts match.

5.6.3.6.3.2 States

The defined states and their descriptions of the Class 1 Transport Server are listed in Table 360 below :

Table 360 – Class 1 transport server states

State	Description
Non-existent	The instance of Transport Class 1 does not exist.
Idle	The instance of Transport Class 1 has been created but not yet started.
Ready to run	The instance of Transport Class 1 has been created and started, waiting for a first data packet to arrive.
Running	The instance of Transport Class 1 has been created and started, first data packet has arrived.

5.6.3.6.3.3 State transitions

State transitions of the class 1 server transport are shown in Figure 74.

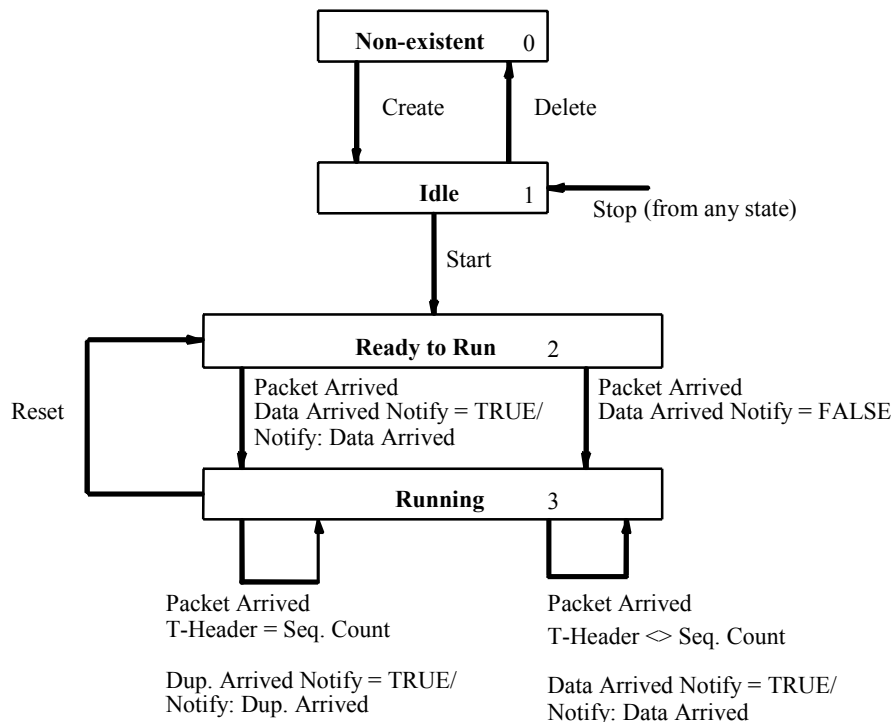


Figure 74 – Class 1 server STD

5.6.3.6.3.4 State event matrix

State transitions of the class 1 server transport are shown in Table 361.

Table 361 – Class 1 server SEM

Event	State/			
	Non-existent	Idle	Ready to run	Running
Create	Transition to Idle	Error	Error	Error
Delete	Error	Transition to Non-existent	Error	Error
Start	Error	Transition to Ready to Run	Error	Error
Stop	Error	No action	Transition to Idle	Transition to Idle
Reset	Error	Error	No Action	Transition to Ready to Run
Packet Arrived Transport Header <> Seq. Count Data Arrived Notify = TRUE Duplicate Arrived Notify = TRUE	No action	No action	Transition to Running Notify: Data Arrived	Notify: Data Arrived
Packet Arrived Transport Header = Seq. Count Data Arrived Notify = TRUE Duplicate Arrived Notify = TRUE	No action	No action	Transition to Running Notify: Data Arrived	Notify: Duplicate Arrived
Packet Arrived Transport Header <> Seq. Count Data Arrived Notify = TRUE Duplicate Arrived Notify = FALSE	No action	No action	Transition to Running Notify: Data Arrived	Notify: Data Arrived
Packet Arrived Transport Header = Seq. Count Data Arrived Notify = TRUE Duplicate Arrived Notify = FALSE	No action	No action	Transition to Running Notify: Data Arrived	No action
Packet Arrived Transport Header <> Seq. Count Data Arrived Notify = FALSE Duplicate Arrived Notify = TRUE	No action	No action	Transition to Running	No action
Packet Arrived Transport Header = Seq. Count Data Arrived Notify = FALSE Duplicate Arrived Notify = TRUE	No action	No action	Transition to Running	Notify: Duplicate Arrived
Packet Arrived Transport Header <> Seq. Count Data Arrived Notify = FALSE Duplicate Arrived Notify = FALSE	No action	No action	Transition to Running	No action
Packet Arrived Transport Header = Seq. Count Data Arrived Notify = FALSE Duplicate Arrived Notify = FALSE	No action	No action	Transition to Running	No action

5.6.3.7 Transport state machines – class 2

5.6.3.7.1 Functions

Transport class 2 starts with the behaviour in class 1 and adds a return connection that acknowledges the delivery of a packet. The producer connection from the client node can be point to point or multipoint. The connection from the server to the client is point to point.

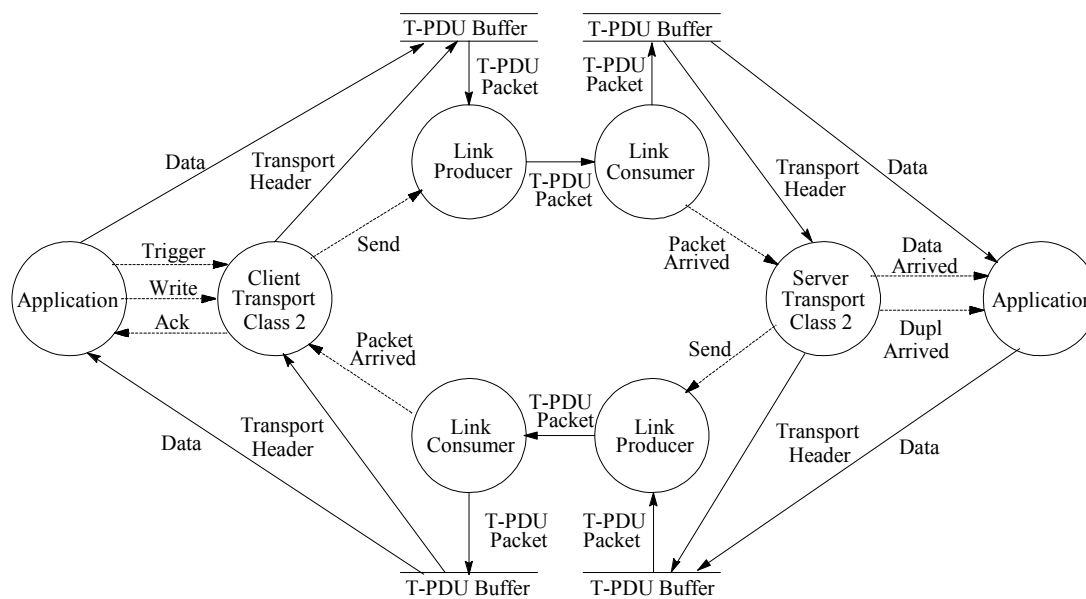
This transport class allows the client application to be notified that the server has received the transmitted packet. This acknowledgement tells the client application only that the data arrived. This does not mean that the server application has read the buffer or that a new sample can be sent without overwriting the buffer.

The main advantage of this class over class 3 (Verified) is that the acknowledgement is returned immediately after the packet arrives at the consumer, while the verify class requires the application to initiate the return verification. The delays associated with receiving the acknowledgement are small, approximately twice the time that is required for a one way transmission.

Transport class 2 uses two network connections: the originator-to-target connection, which can be either point-to-point or multipoint; and the return connection, which shall be point-to-point. A multipoint connection using transport class 2 shall have a point-to-point return connection corresponding to each client-to-server connection. *The acknowledgement notifies the client transport instance that the consuming node of the network connection has received the packet.*

If a server transport instance does not acknowledge receipt of the packet, the client transport instance triggers the client application to retransmit the most recently sent packet; if the client-to-server connection is one of the network connections of a multipoint connection, data is resent over all of the network connections of the multipoint connection. The server application can return data to the client application along with its acknowledgement.

Figure 75 shows the actions which take place during a data transfer using client transport class 2 and server transport class 2



NOTES:

- 1 The client and server transports each use two buffers: one for transmitting, and one for receiving.
- 2 The target application is not involved in sending the acknowledgement.

Figure 75 – Data flow diagram using client transport class 2 and server transport class 2

The server transport instance initiates and sends the acknowledgement to the client transport instance to notify the client application that it can write the next data packet to the client-side T-PDU transmit buffer. Each acknowledgement shall be received by the client transport instance before the next packet is sent over the same client-to-server network connection(s).

Figure 76 shows the sequence of actions which take place during data transfer using client transport class 2 and server transport class 2. In the example depicted, the acknowledgement is the only data transmitted in the server-to-client direction. The unshaded areas in Figure 76 indicate client-to-server data transmission, and the shaded areas indicate server-to-client transmission.

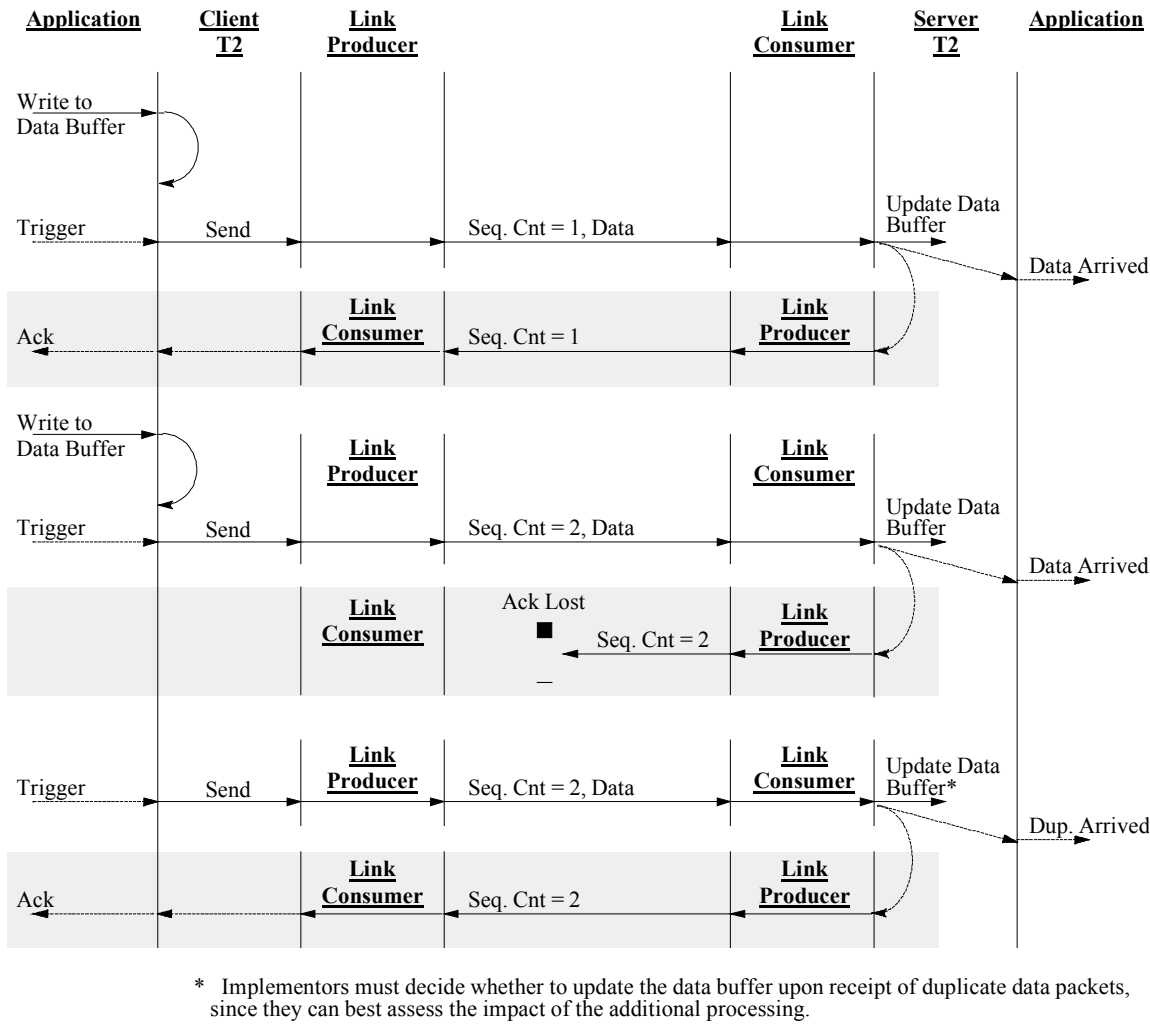


Figure 76 – Diagram of data transfer using client transport class 2 and server transport class 2 *without* returned data

At times, the server-to-client transmission comprises the acknowledgement *and* additional data.

Figure 77 shows the sequence in which actions take place during data transfer using client transport class 2 and server transport class 2 when the acknowledgement and data are returned to the client. Note that the returned data is written asynchronously to the receipt of data from the client. The unshaded areas in Figure 77 indicate client-to-server data transmission, and the shaded areas indicate server-to-client data transmission.

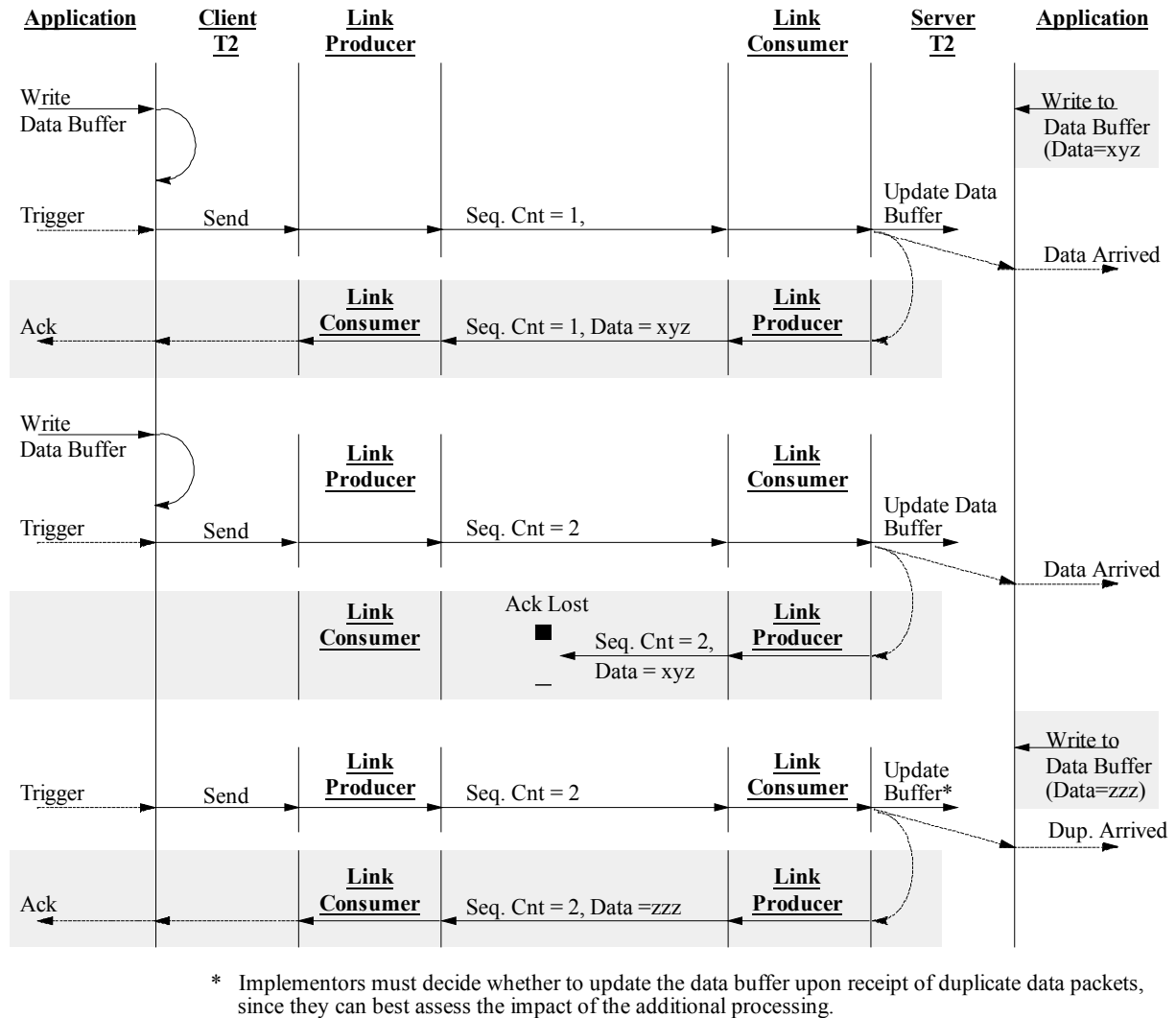


Figure 77 – Sequence diagram of data transfer using client transport class 2 and server transport class 2 with returned data

Possible uses of transport class 2 include master/slave applications and communication between controllers and operator interface devices.

5.6.3.7.2 Transport class 2 client

5.6.3.7.2.1 Functions

In the client-to-server direction, a class 2 client transport is responsible for:

- Accepting the client application's trigger that it has written a data packet to the client-side T-PDU transmit buffer;
- Writing a transport header to the client-side T-PDU transmit buffer for each packet that the client application writes to it

NOTE The transport header for each packet shall include a sequence count

- Initialising the sequence count in the transport header (for the first packet transmitted) or incrementing the sequence count (for subsequent packets of the same transmission);
- Triggering the producing node of the network connection to produce the T-PDU packet on the link and send it to the consuming node of the network connection.

In the server-to-client direction, a class 2 client transport is responsible for:

- Accepting notification from the consuming node of the return network connection that it has written data (the server transport instance's acknowledgement) to the client-side T-PDU receive buffer;
- Locating and reading the transport header of each data packet that the consuming node of the return network connection writes to the client-side T-PDU receive buffer;
- Notifying the client application that the consuming node of the return network connection has written data to the client-side T-PDU receive buffer;

Note that the application can update the data buffer or retrigger data before all of the acknowledgements have been received. If the client transport is retriggered while waiting for an acknowledgement, the previous acknowledgement received register is cleared. This means that the client transport class shall wait for all active consumers to return an acknowledgement before notifying the application.

5.6.3.7.2.2 States

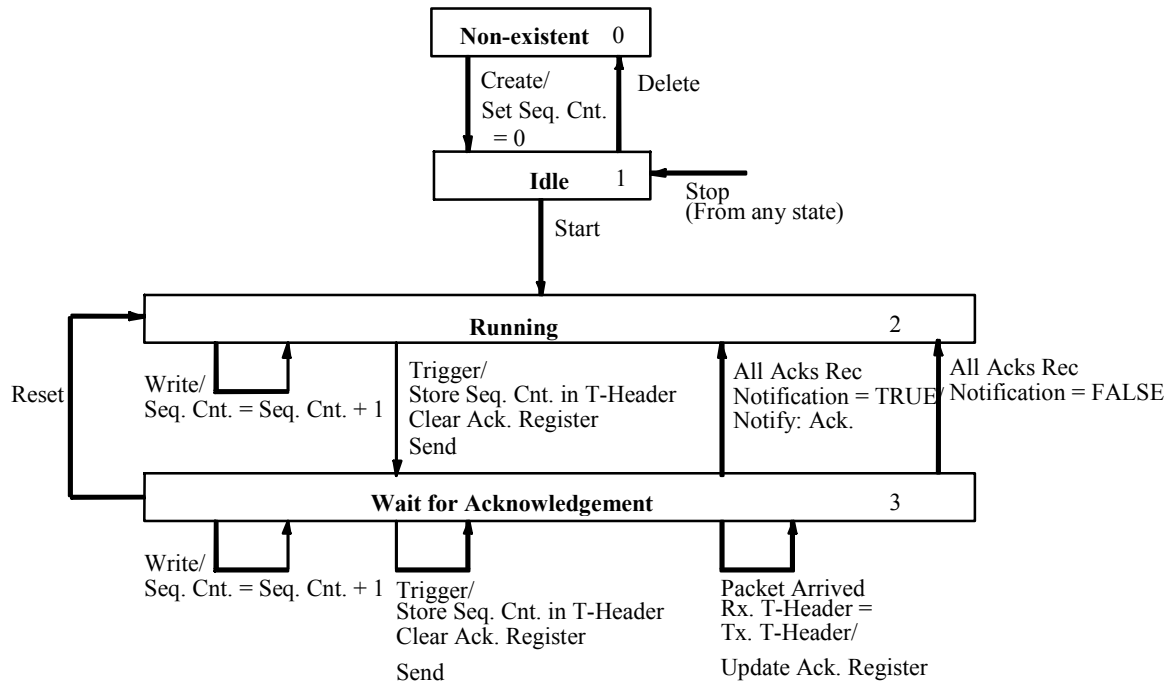
The defined states and their descriptions of the Class 2 Transport Client are listed in Table 362 below :

Table 362 – Class 2 transport client states

State	Description
Non-existent	The instance of Transport Class 2 does not exist.
Idle	The instance of Transport Class 2 has been created but not yet started.
Running	The instance of Transport Class 2 has been created and started.
Wait for acknowledgement	The instance of Transport Class 2 has been created and started, acknowledgement is pending.

5.6.3.7.2.3 State transitions

State transitions of the class 2 client transport are shown in Figure 78.



NOTE The sequence count is set to 0 by the create service. It is then incremented after every write event until it reaches a maximum value of $2^{16}-1$. After that it rolls over to 0.

Figure 78 – Class 2 client STD

5.6.3.7.2.4 State event matrix

State transitions of the class 2 client transport are shown in Table 363.

Table 363 – Class 2 client SEM

Event	State			
	Non-existent	Idle	Running	Wait for ack
Create	1) Seq. Cnt. = 0 2) Transition to Idle	Error	Error	Error
Delete	Error	Transition to Non-existent	Error	Error
Start	Error	Transition to Running	Error	Error
Stop	Error	No Action	Transition to Idle	Transition to Idle
Reset	Error	Error	No Action	Transition to Running
Write	Error	No Action	1) Seq. Cnt. = Seq. Cnt. + 1 2) Write TPDU	1) Seq.Cnt. = Seq.Cnt. +1 2) Write TPDU
Trigger	Error	No Action	1) Store Seq. Cnt. in TPDU 2) Clear Ack. register 3) Send 4) Transition to Wait for Ack.	1) Store Seq. Cnt. in TPDU 2) Clear Ack. Register 3) Send
Packet Arrived Rx. T-Header = Tx. T-Header	Error	No Action	No Action	1) Update Ack. Register
All Ack Received Ack. Notify = TRUE	Error	No Action	Error	1) Notify: Ack. 2) Transition to Running
All Ack Received Ack. Notify = FALSE	Error	No Action	Error	1) Transition to Running

NOTE The STD and SEM for client classes 2 and 3 are nearly identical. Verify has been substituted for acknowledgement.

5.6.3.7.3 Transport class 2 server

5.6.3.7.3.1 Functions

Server transport class 2 starts with the behaviour from transport class 1 and adds a return connection from the server to the client that is used to acknowledge a message. This return connection is used to identify the acknowledgement message.

After receiving a *packet arrived* event, the server transport shall read the received transport header, store it in the transmit transport header and invoke the *send* service. The application is not involved in the acknowledgement. The producer connection from the client node can be point to point or multipoint. The connection from the server to the client is point to point.

In the client-to-server direction, a class 2 server transport is responsible for

- Accepting the notification from the consuming node of the network connection that it has written a data packet to the server-side T-PDU receive buffer;
- Locating and reading the transport header of each packet that the consuming node of the network connection writes to the server-side T-PDU receive buffer;
- Comparing the sequence count of each packet in the server-side T-PDU receive buffer with that of the previous packet;
- Notifying the server application that the most recently arrived packet is a duplicate of the previous one when their sequence counts match;
- Notifying the server application that data has been written to the server-side T-PDU receive buffer. (ACK may contain server data)

In the server-to-client direction, a class 2 server transport is responsible for

- Writing the transport header (including sequence count) of the data in the server-side T-PDU receive buffer to the server-side T-PDU transmit buffer;
- Triggering the producing node of the return network connection to produce the T-PDU packet on the link and send it to the consuming node of the return network connection.

5.6.3.7.3.2 States

The defined states and their descriptions of the Class 2 Transport Server are listed in Table 364 below :

Table 364 – Class 2 transport server states

State	Description
Non-existent	The instance of Transport Class 2 does not exist.
Idle	The instance of Transport Class 2 has been created but not yet started.
Running	The instance of Transport Class 2 has been created and started.

5.6.3.7.3.3 State transitions

State transitions of the class 2 server transport are shown in Figure 79.

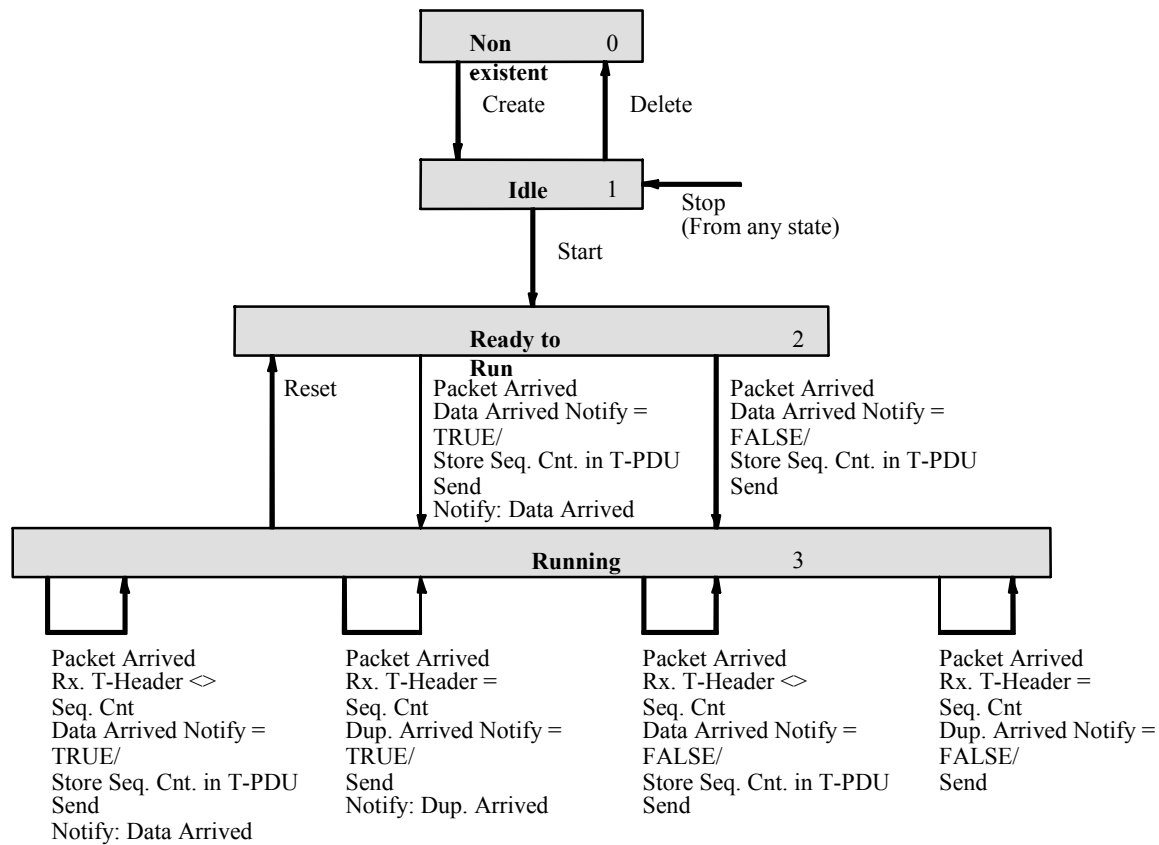


Figure 79 – Class 2 server STD

NOTE The send service is invoked for sequence counts that match and sequence counts that differ. This behaviour is required to ensure that a node that has sent an acknowledgement that was lost shall retransmit the acknowledgement on a retry.

5.6.3.7.3.4 State event matrix

State transitions of the class 2 server transport are shown in Table 365.

Table 365 – Class 2 server SEM

Event	State			
	Non-existent	Idle	Ready to run	Running
Create	Transition to Idle	Error	Error	Error
Delete	Error	Transition to Non-existent	Error	Error
Start	Error	Transition to Ready to Run	Error	Error
Stop	Error	No action	Transition to Idle	Transition to Idle
Reset	Error	Error	No action	Transition to Ready to Run
Packet Arrived Rx. T-Header <> Seq.Count Data Arrived Notify = TRUE Duplicate Arrived Notify = TRUE	No action	No action	1) Store Seq. Cnt. in Tx. TPDU 2) Send (Implies Ack) 3)Notify: Data Arrived 4) Transition to Running	1) Store Seq. Cnt. in Tx. TPDU 2) Send (Implies Ack) 3)Notify: Data Arrived
Packet Arrived Rx. T-Header = Seq. Count Data Arrived Notify = TRUE Duplicate Arrived Notify = TRUE	No action	No action	1) Store Seq. Cnt. in Tx. TPDU 2) Send (Implies Ack) 3)Notify: Data Arrived 4) Transition to Running	1) Send (Implies Ack) 2) Notify: Duplicate Arrived
Packet Arrived Rx. T-Header <> Seq.Count Data Arrived Notify = TRUE Duplicate Arrived Notify = FALSE	No action	No action	1) Store Seq. Cnt. in Tx. TPDU 2) Send (Implies Ack) 3)Notify: Data Arrived 4) Transition to Running	1) Store Seq. Cnt. in Tx. TPDU 2) Send (Implies Ack) 3)Notify: Data Arrived
Packet Arrived Rx. T-Header = Seq. Count Data Arrived Notify = TRUE Duplicate Arrived Notify = FALSE	No action	No action	1) Store Seq. Cnt. in Tx. TPDU 2) Send (Implies Ack) 3) Notify: Data Arrived 4) Transition to Running	1) Send (Implies Ack)
Packet Arrived Rx. T-Header <> Seq.Count Data Arrived Notify = FALSE Duplicate Arrived Notify = TRUE	No action	No action	1) Store Seq. Cnt. in Tx. TPDU 2) Send (Implies Ack) 3) Transition to Running	1) Store Seq. Cnt. in Tx. TPDU 2) Send (Implies Ack)
Packet Arrived Rx. T-Header = Seq. Count Data Arrived Notify = FALSE Duplicate Arrived Notify = TRUE	No action	No action	1) Store Seq. Cnt. in Tx. TPDU 2) Send (Implies Ack) 3) Transition to Running	1) Send (Implies Ack) 2) Notify: Duplicate Arrived
Packet Arrived Rx. T-Header <> Seq.Count Data Arrived Notify = FALSE Duplicate Arrived Notify = FALSE	No action	No action	1) Store Seq. Cnt. in Tx. TPDU 2) Send (Implies Ack) 3) Transition to Running	1) Store Seq. Cnt. in Tx. TPDU 2) Send (Implies Ack)
Packet Arrived Rx. T-Header = Seq. Count Data Arrived Notify = FALSE Duplicate Arrived Notify = FALSE	No action	No action	1) Store Seq. Cnt. in Tx. TPDU 2) Send (Implies Ack) 3) Transition to Running	1) Send (Implies Ack)

5.6.3.8 Transport state machines – class 3

5.6.3.8.1 Functions

Transport class 3 starts with the behaviour in class 1 and adds a return connection that verifies that the application has received the packet. The sequence count that shall be returned with the verify is the same as the sequence count sent with the data. This return connection is used to identify the verification message.

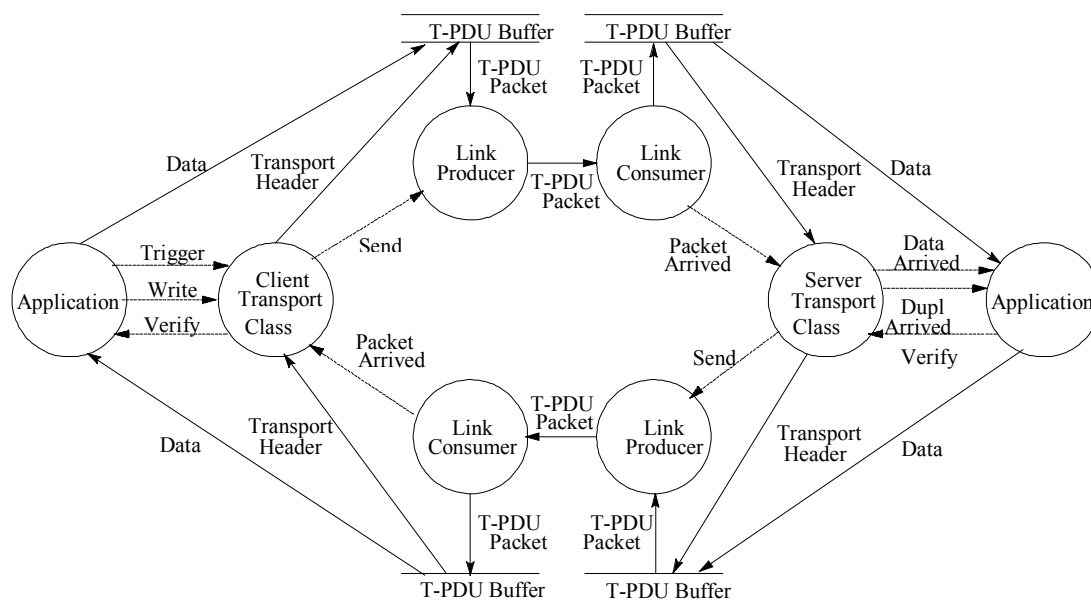
This transport class allows the application to be notified that the server application has responded to the transmitted packet. This verification tells the client application not only that the data arrived but that the server application has read the buffer and that a new sample can be sent without overwriting the buffer.

The main advantage of this class over class 2 (Acknowledgement) is that verification conveys an application response. The disadvantage is that the delay in receiving the verification can be very difficult to predict.

Transport class 3 uses two connections: the originator-to-target connection, which can be either point-to-point or multipoint; and the return connection, which shall be point-to-point. A multipoint connection using transport class 3 shall have a point-to-point return connection corresponding to each client-to-server connection. The verification notifies the client application that the server application has received and read the transmitted data. If a server application does not verify receipt of the packet, the client application retransmits the most recently sent packet; if the client-to-server connection is a multipoint connection, the data is retransmitted over all of the network connections of the multipoint connection.

The server application initiates and sends the verification *after* the consuming node of the network connection has written the packet to the T-PDU buffer, *after* the transport has read the packet's transport header, and *after* the application has read the packet. The server's verification signals the client application that the server-side T-PDU receive buffer is ready to accept the next data packet.

Figure 80 shows the actions which take place during a data transfer using client transport class 3 and server transport class 3.

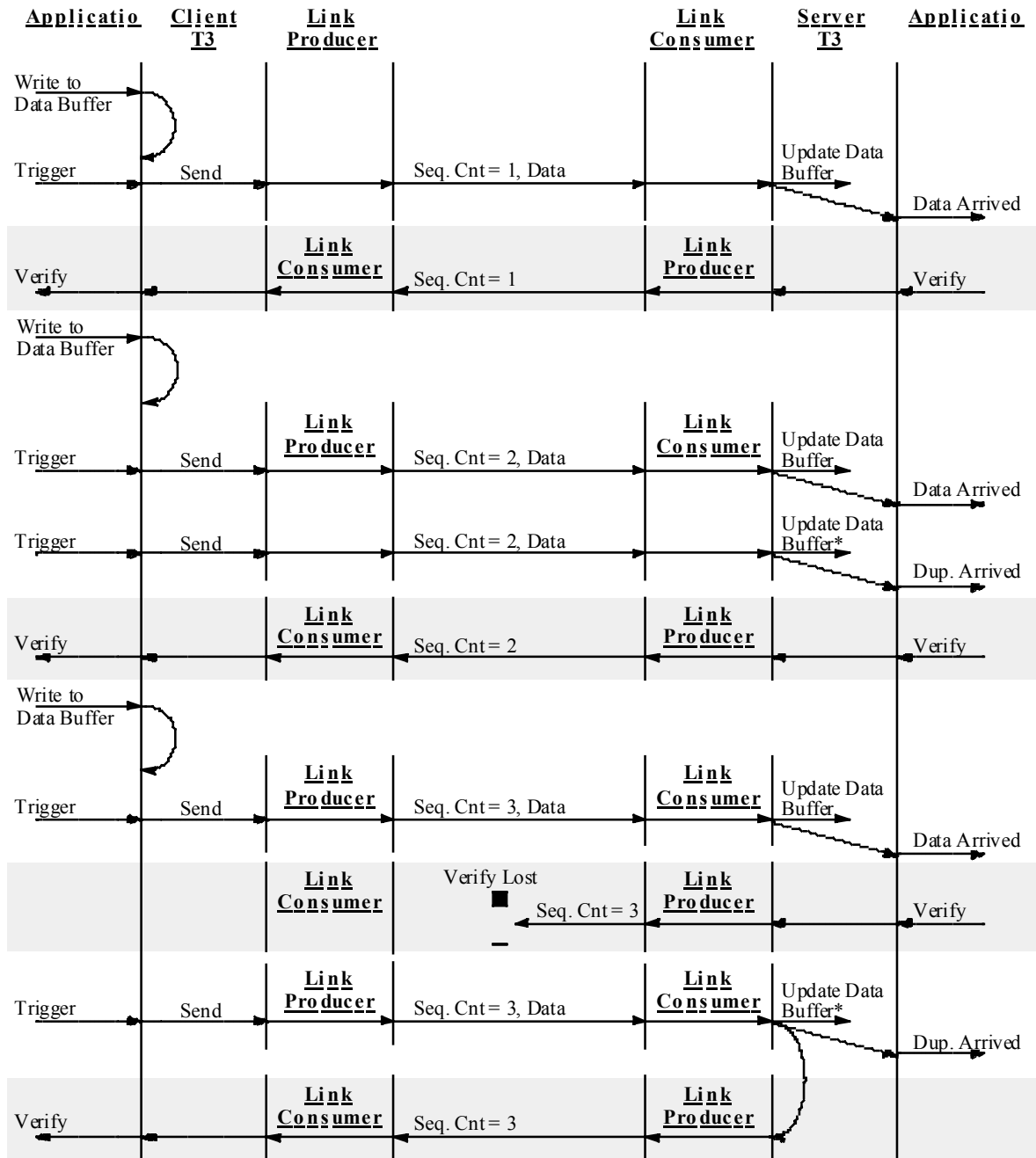


NOTES.

- 1 The client and server transports each use two buffers: one for transmitting, and one for receiving.
- 2 The server application can return data to the client application using the connection established for the verification.

Figure 80 – Data flow diagram using client transport class 3 and server transport class 3

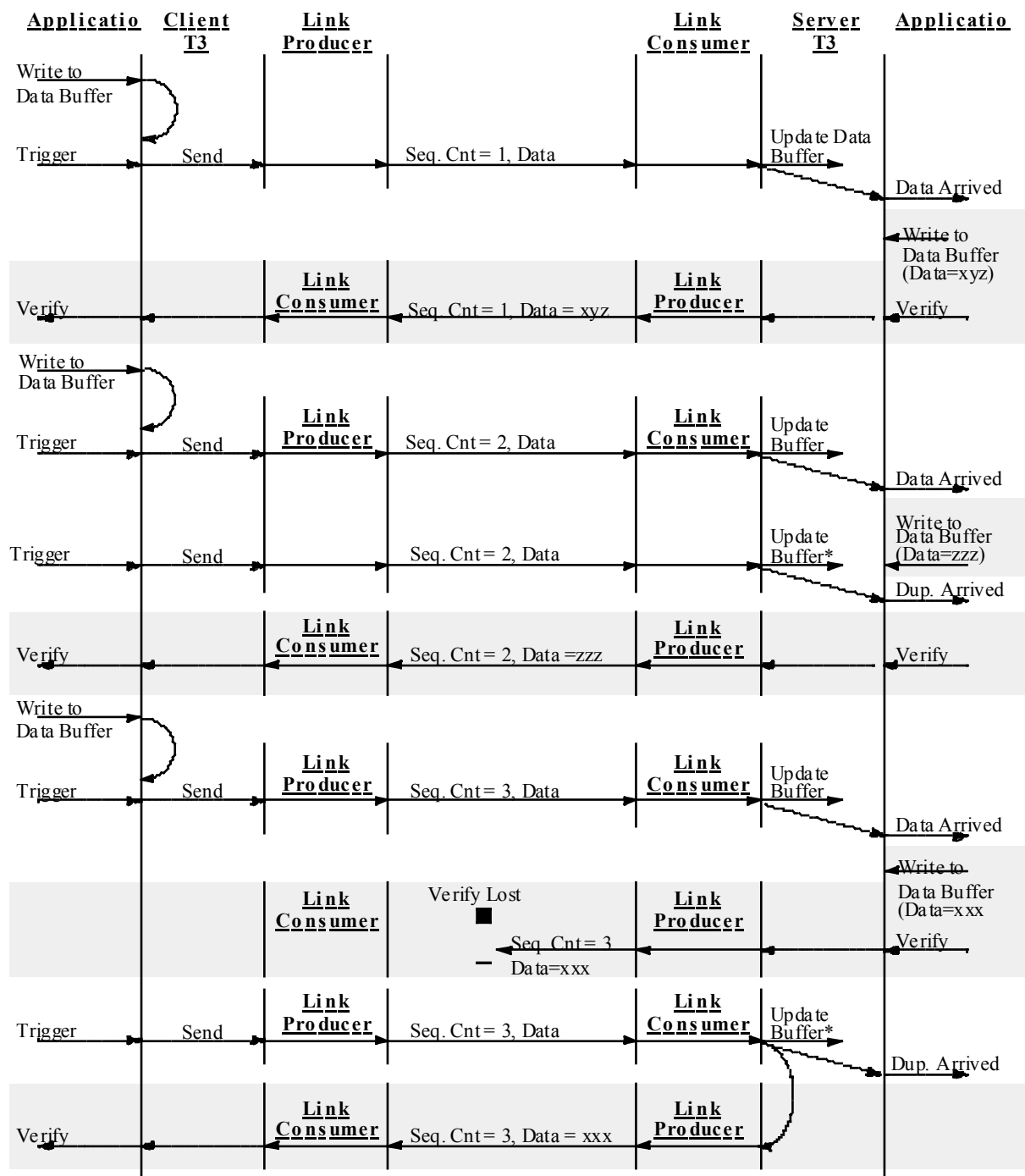
Figure 81 shows the sequence of actions which take place during data transfer using client transport class 3 and server transport class 3. In the example depicted, the verification is the only data transmitted in the server-to-client direction. The unshaded areas in Figure 81 indicate client-to-server data transmission, and the shaded areas indicate server-to-client transmission.



* Implementors must decide whether to update the data buffer upon receipt of duplicate data packets, since they can best assess the impact of the additional

Figure 81 – Sequence diagram of data transfer using client transport class 3 and server transport class 3 without returned data

At times, the server-to-client transmission comprises the verification *and* additional data. Figure 82 shows the sequence in which actions take place during data transfer using client transport class 3 and server transport class 3 when the verification and data are returned to the client. Note that the returned data is written asynchronously to the receipt of data from the client. The unshaded areas in Figure 82 indicate client-to-server data transmission, and the shaded areas indicate server-to-client transmission.



* Implementors must decide whether to update the data buffer upon receipt of duplicate data since they can best assess the impact of the additional processing.

Figure 82 – Sequence diagram of data transfer using client transport class 3 and server transport class 3 with returned data

Possible uses of transport class 3 include change-of-state inputs and outputs, asynchronous [independent] block transfers, event acknowledgements, communication between controllers and operator interface devices, uploads, downloads, and messaging.

5.6.3.8.2 Transport class 3 client

5.6.3.8.2.1 Functions

In the client-to-server direction, a class 3 client transport is responsible for:

- Accepting the client application's trigger that it has written a data packet to the client-side T-PDU transmit buffer;
- Writing a transport header to the client-side T-PDU transmit buffer for each packet that the client application writes to it;
- Initialising the sequence count in the transport header (for the first packet transmitted) or incrementing the sequence count (for subsequent packets of the same transmission);
- Triggering the producing node of the network connection to produce the T-PDU packet on the link and send it to the consuming node of the network connection.

In the server-to-client direction, a class 3 client transport is responsible for:

- Accepting notification from the consuming node of the return network connection that it has written data (the server transport instance's verification) to the client-side T-PDU receive buffer;
- Locating and reading the transport header of each data packet that the consuming node of the return network connection writes to the client-side T-PDU receive buffer;
- Notifying the client application that the consuming node of the return network connection has written data to the client-side T-PDU receive buffer.

Note that the application can update the data buffer or retrigger data before all of the verifications have been received. If the client transport is retriggered while waiting for verification, the previous verification received register is cleared. This means that the client transport class shall wait for all active consumers to return verification before notifying the application.

5.6.3.8.2.2 States

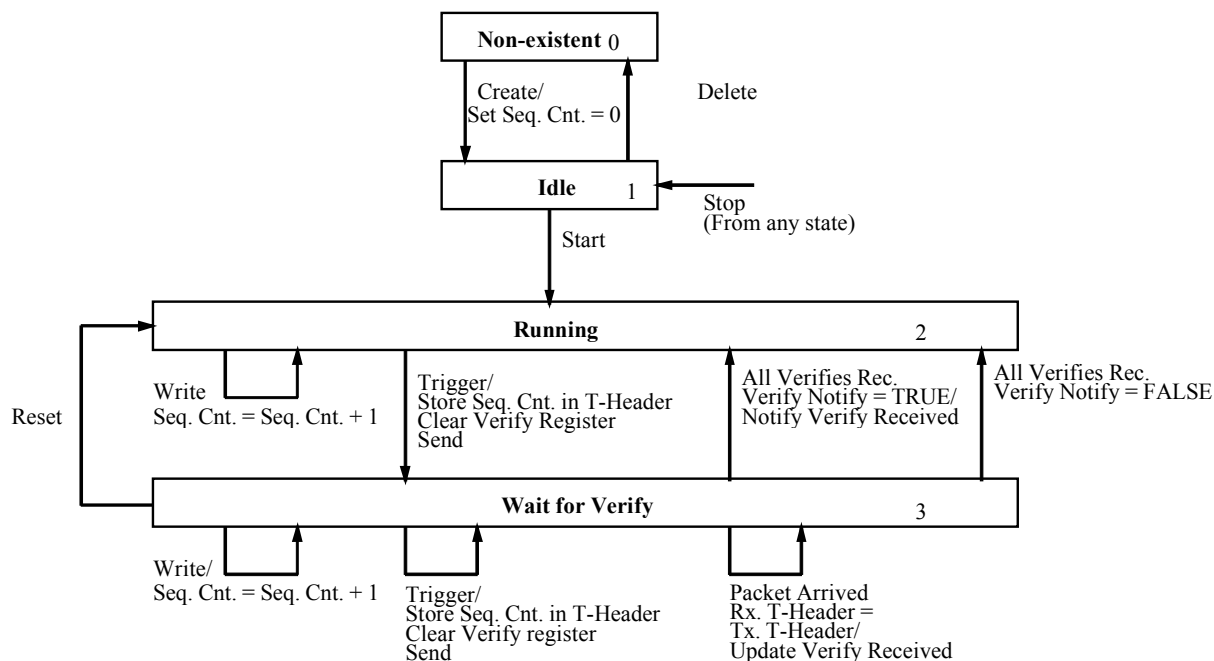
The defined states and their descriptions of the Class 3 Transport Client are listed in Table 366 below :

Table 366 – Class 3 transport client states

State	Description
Non-existent	The instance of Transport Class 3 does not exist.
Idle	The instance of Transport Class 3 has been created but not yet started.
Running	The instance of Transport Class 3 has been created and started.
Wait to verify	The instance of Transport Class 3 has been created and started, verification is pending.

5.6.3.8.2.3 State transitions

State transitions of the class 3 client transport are shown in Figure 83.



NOTE The sequence count is set to 0 by the create service. It is then incremented after every *write* event until it reaches a maximum value of $2^{16}-1$. After that it rolls over to 0.

Figure 83 – Class 3 client STD

5.6.3.8.2.4 State event matrix

State transitions of the class 3 client transport are shown in Table 367.

Table 367 – Class 3 client SEM

Event	State			
	Non-existent	Idle	Running	Wait for verify
Create	1) Seq. Cnt. = 0 2) Transition to Idle	Error	Error	Error
Delete	Error	Transition to Non-existent	Error	Error
Start	Error	Transition to Running	Error	Error
Stop	Error	No action	Transition to Idle	Transition to Idle
Reset	Error	Error	No Action	Transition to Running
Write	Error	No action	Seq. Cnt. = Seq. Cnt. + 1	Seq. Cnt. = Seq. Cnt. + 1
Trigger	Error	No action	1) Store Seq. Cnt. in TPDU 2) Clear Verify register 3) Send 4) Transition to Wait for Verify	1) Store Seq. Cnt. in TPDU 2) Clear Verify register 3) Send
Packet Arrived Rx. T- header = Tx. T-Header	No action	No action	No Action	1) Update Verify Register
All Verifies Received Notification = TRUE	No action	No action	Error	1) Notify: Verify 2) Transition to Running
All Verifies Received Notification = FALSE	No action	No action	Error	1) Transition to Running

NOTE The STD and SEM for client classes 2 and 3 are nearly identical. Acknowledgement has been substituted for verification.

5.6.3.8.3 Transport class 3 server

5.6.3.8.3.1 Functions

Transport class 3 starts with the behaviour in class 1 and adds a return connection that verifies that the application has received the packet. It is the responsibility of the application to verify receipt of data. The sequence count that shall be returned with the verification is the same as the sequence count sent with the data.

The application is required to read the TPDU buffer before invoking the verify service.

In the client-to-server direction, a class 3 server transport is responsible for:

- Accepting the notification from the consuming node of the network connection that it has written a data packet to the server-side T-PDU receive buffer;
- Locating and reading the transport header of each packet that the consuming node of the network connection writes to the server-side T-PDU receive buffer;
- Comparing the sequence count of each packet in the server-side T-PDU receive buffer with that of the previous packet;
- Notifying the server application that the most recently arrived packet is a duplicate of the previous one when their sequence counts match;
- Notifying the server application that data has been written to the server-side T-PDU receive buffer.

In the server-to-client direction, a class 3 server transport is responsible for:

- Writing the transport header of the data in the server-side T-PDU receive buffer to the server-side T-PDU transmit buffer;
- Correlating that transport header with any data that the server application wants to send to the client application and has written to the server-side T-PDU transmit buffer;
- Triggering the producing node of the return network connection to produce the T-PDU packet on the link and send it to the consuming node of the return network connection;

NOTE Data from the application can be return from the server to the client along the verification connection.

The send service is invoked when verify is received from an application or when a duplicate transmission is received while in the running state. Retransmitting when a previously verified packet has been received allows this class to recover from lost verifications.

5.6.3.8.3.2 States

The defined states and their descriptions of the Class 3 Transport Server are listed in Table 368 below :

Table 368 – Class 3 transport server states

State	Description
Non-existent	The instance of Transport Class 3 does not exist.
Idle	The instance of Transport Class 3 has been created but not yet started.
Read to run	The instance of Transport Class 3 has been created and started, waiting for the first packet to arrive.
Wait for verify	The instance of Transport Class 3 has been created and started, the first packet arrived, waiting for verification to be requested by the application.
Running	The instance of Transport Class 3 has been created and started, verification has been processed.

5.6.3.8.3.3 State transitions

State transitions of the class 3 server transport are shown in Figure 84.

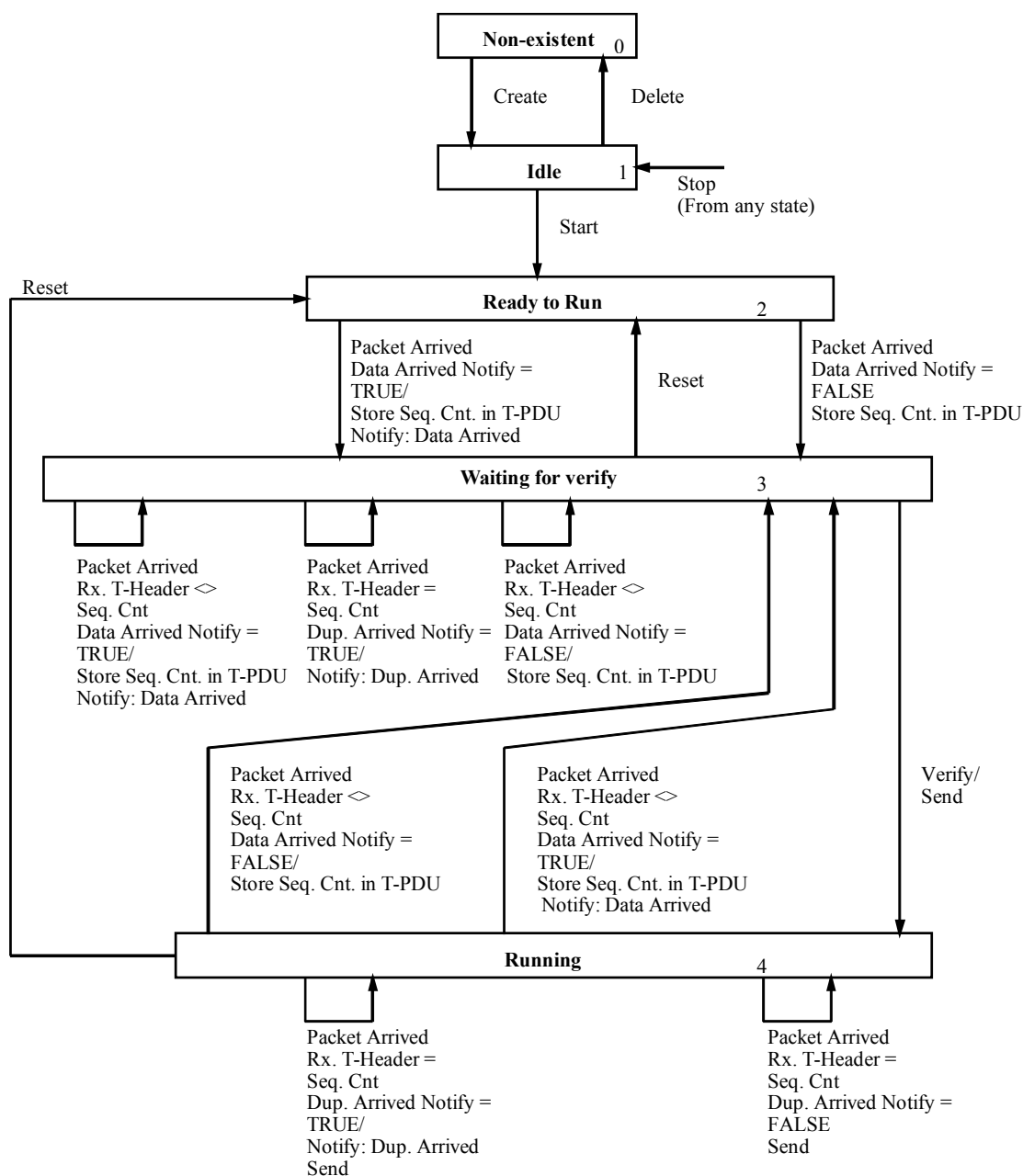


Figure 84 – Class 3 server STD

5.6.3.8.3.4 State event matrix

State transitions of the class 3 server transport are shown in Table 369.

Table 369 – Class 3 server SEM

Event	State				
	Non-existent	Idle	Ready to run	Waiting for verify	Running
Create	Transition to Idle	Error	Error	Error	Error
Delete	Error	Transition to Non-existent	Error	Error	Error
Start	Error	Transition to Ready to Run	Error	Error	Error
Stop	Error	No action	Transition to Idle	Transition to Idle	Transition to Idle
Reset	Error	Error	No action	Transition to Ready to Run	Transition to Ready to Run
Packet Arrived Rx. T-Header <> Seq.Count Data Arrived Notify = TRUE Dup Arrived Notify = TRUE	No action	No action	1) Store Seq. Cnt. in TPDU 2) Notify: Data Arrived 3) Transition to Waiting for Verify	1) Store Seq. Cnt. in TPDU 2) Notify: Data Arrived	1) Store Seq. Cnt. in TPDU 2) Notify: Data Arrived 3) Transition to Waiting for Verify
Packet Arrived Rx. T-Header = Seq.Count Data Arrived Notify = TRUE Dup Arrived Notify = TRUE	No action	No action	1) Store Seq. Cnt. in TPDU 2) Notify: Data Arrived 3) Transition to Waiting for Verify	1) Notify: Dup. Arrived	1) Notify: Dup. Arrived 2) Send (Implies Verify)
Packet Arrived Rx. T-Header <> Seq.Count Data Arrived Notify = TRUE Dup Arrived Notify = FALSE	No action	No action	1) Store Seq. Cnt. in TPDU 2) Notify: Data Arrived 3) Transition to Waiting for Verify	1) Store Seq. Cnt. in TPDU 2) Notify: Data Arrived	1) Store Seq. Cnt. in TPDU 2) Notify: Data Arrived 3) Transition to Waiting for Verify
Packet Arrived Rx. T-Header = Seq.Count Data Arrived Notify = TRUE Dup Arrived Notify = FALSE	No action	No action	1) Store Seq. Cnt. in TPDU 2) Notify: Data Arrived 3) Transition to Waiting for Verify	No action	1) Send (Implies Verify)
Packet Arrived Rx. T-Header <> Seq.Count Data Arrived Notify = FALSE Dup Arrived Notify = TRUE	No action	No action	1) Store Seq. Cnt. in TPDU 2) Transition to Waiting for Verify	1) Store Seq. Cnt. in TPDU	1) Store Seq. Cnt. in TPDU 2) Transition to Waiting for Verify
Packet Arrived Rx. T-Header = Seq.Count Data Arrived Notify = FALSE Dup Arrived Notify = TRUE	No action	No action	1) Store Seq. Cnt. in TPDU 2) Transition to Waiting for Verify	1) Notify: Dup. Arrived	1) Notify: Dup. Arrived 2) Send (Implies Verify)
Packet Arrived Rx. T-Header <> Seq.Count Data Arrived Notify = FALSE Dup Arrived Notify = FALSE	No action	No action	1) Store Seq. Cnt. in TPDU 2) Transition to Waiting for Verify	1) Store Seq. Cnt. in TPDU	1) Store Seq. Cnt. in TPDU 2) Transition to Waiting for Verify
Packet Arrived Rx. T-Header = Seq.Count Data Arrived Notify = FALSE Dup Arrived Notify = FALSE	No action	No action	1) Store Seq. Cnt. in TPDU 2) Transition to Waiting for Verify	No action	1) Send (Implies Verify)
Verify	Error	No action	No action	1) Send (Implies Verify) 2) Transition to Running	No action

5.6.3.9 Transport state machines – classes 4, 5, 6

5.6.3.9.1 General

This subclause explains the common elements of the transport classes 4 – 6, which provide higher-level features such as non-blocking, fragmentation/assembly, and transport-level acknowledgement. Transport classes 4 and 5 shall not be used with messaging. Messaging does not provide a way to match requests with responses. Use class 3 for messaging without fragmentation.

The fragmentation protocols assume that per-connection send order is preserved throughout the system; overwrite is permissible, but changing the order of the messages is not.

5.6.3.9.2 Class 4 and 5 data flow diagram

Figure 85 shows the relationship of the class 4 and 5 Transport to the Application and the Link Producer and Link Consumer, the data that move between these functions, and the events which are generated and used by them. (The class 6 data flow diagram is included with the class 6 definition, see 5.6.3.12.2.) Note some significant differences between these and the class 3 transport.

- The application data is entered into a queue, not placed directly into the T-PDU Buffer. This allows the application to give several requests to the transport without waiting for the acknowledgement, verification, or response to a previous request.
- The success or failure of an application request is placed into a queue for the application to access, rather than using a simple event. This is done because the execution of the application and the delivery of the application data are asynchronous.
- The events and data on the right hand side and the left hand side are identical. There is not a “Client” and “Server” transport for these classes. Both sides are capable of sending and receiving application data.
- The trigger timer is explicitly identified, rather than being a portion of the application. This is because the transport is using the trigger signal to perform timing functions.

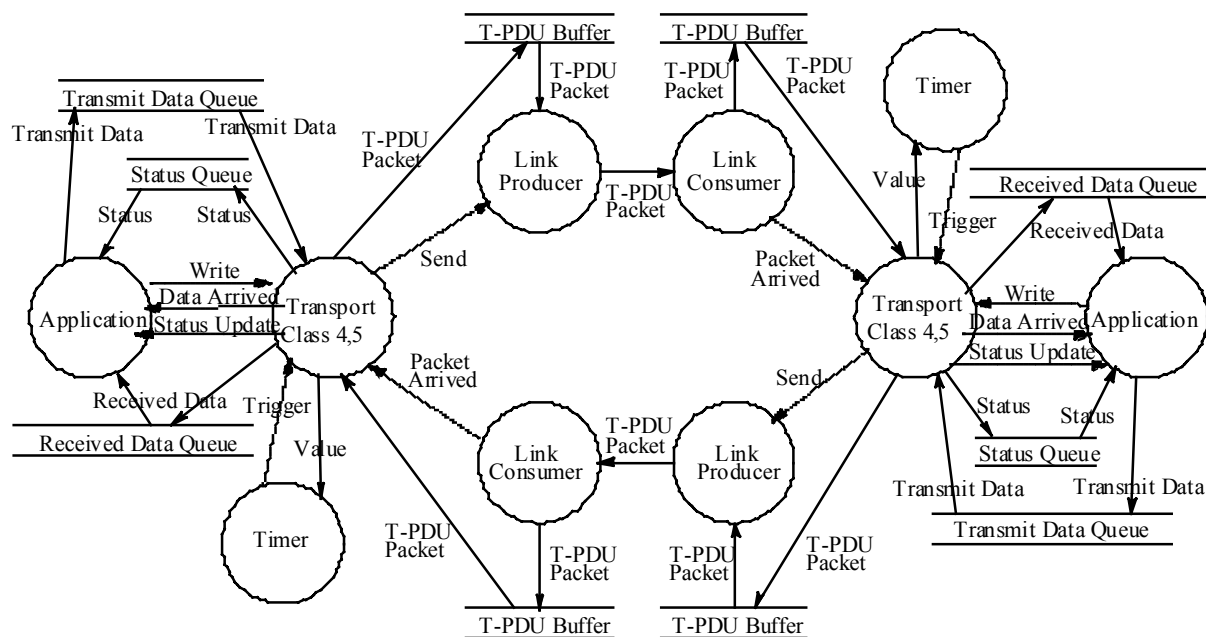


Figure 85 – Data flow diagram using transport classes 4 and 5

The Write event is used to notify the transport that new application data has been placed in the queue. The Trigger event is assumed to be provided by a resettable retry timer. These events are used in a different sense than they are used for classes 0 to 3. These are shown in Table 370.

Table 370 – Write and trigger events in class 4 and 5 transport

	Classes 0 to 3	Classes 4 to 6
Write event	“New data available”: Causes sequence count to increment.	“New data available”: Indicates queue is non-empty. If the transport is not otherwise busy, shall cause data to be sent.
Trigger event	Causes data to be sent.	Causes retry or keep alive to be sent.

5.6.3.9.3 Classes 4 and 5 sequence diagrams

5.6.3.9.3.1 Typical sequences

Some typical sequences are illustrated to aid understanding of the operation of these transport classes. Formal specifications of these transport classes are given in IEC 61158-5.

5.6.3.9.3.2 Typical message exchange sequence

Figure 86 shows the sequence in which actions take place during a messaging request/response exchange using transport classes 4 and 5. The unshaded areas in Figure 86 indicate data transmission in one direction, and the shaded areas indicate data transmission in the other direction. Each transmission shows the Command and Sequence Number used for each header (SND and RCV). SND gives the command in the direction of the data flow, and RCV is the acknowledgement of the command in the other direction.

When a transport has some data to send, it uses the *Only* command and when it has nothing to send, it uses the *Null* command. It is recommended that the Sender state machine send zero application data octets with the *Null* command. It is also recommended that the Receiver state machine accept *Null* commands with a non-zero number of application data octets, which it ignores.

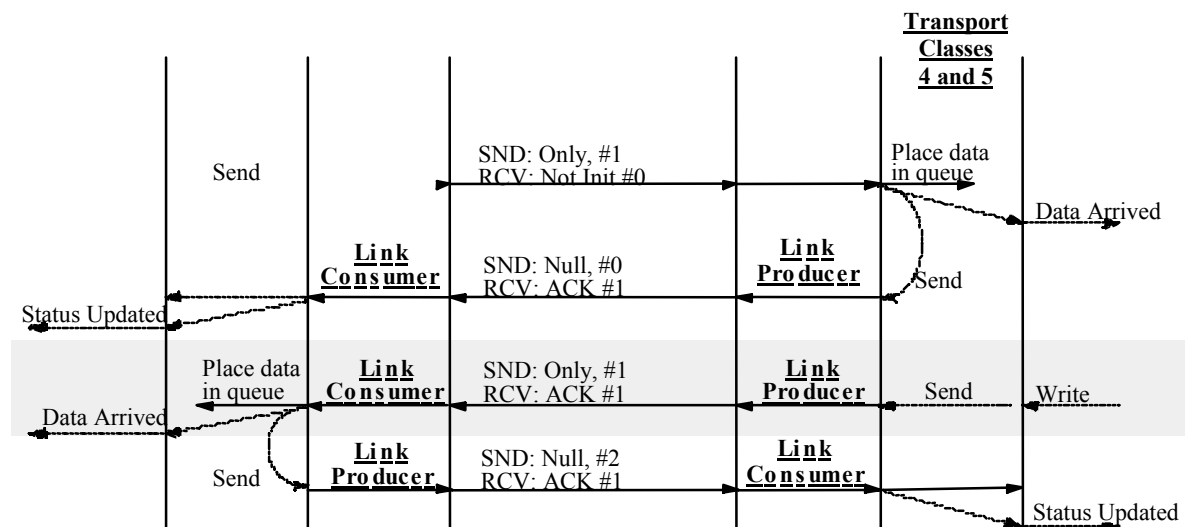


Figure 86 – Sequence diagram of message exchange using transport classes 4 and 5

5.6.3.9.3.3 Typical sequence with overwrite

It is permissible for a transport to send two data packets in a row. This implies that the second one might overwrite the first one in an intermediate module. This is desirable, since the second packet contains information that is newer than the first one and the data in the first one is no longer needed.

In the previous example (see 5.6.3.9.3.2), the transport on the right had nothing to send when it received the *Only* #1 from the transport on the left. It needed to send the *Null* #0, so that it could deliver the ACK #1. But when the application on the right did a WRITE, the transport immediately sent the *Only* #1 along with the ACK #1. The previous example shows both of them arriving separately. Figure 87 shows the left side's *Only* #1 overwriting the *Null* #0.

Figure 87 also shows a different situation where an overwrite can occur. The right side sends a *Null* #2 with the ACK #2 as the result of the keep-alive *Trigger* event. But, before it can be delivered, the right side receives the *Only* #3 from the left side. The left side immediately sends the *Null* #2, but this time with the newer ACK #3, which overwrites the ACK #2.

It is not necessary to wait for the ACK to a *Null* before sending an *Only*.

When sending an *Only*, it shall have a different sequence number than the *Null*, so sender knows which is being ACK'd by the other side.

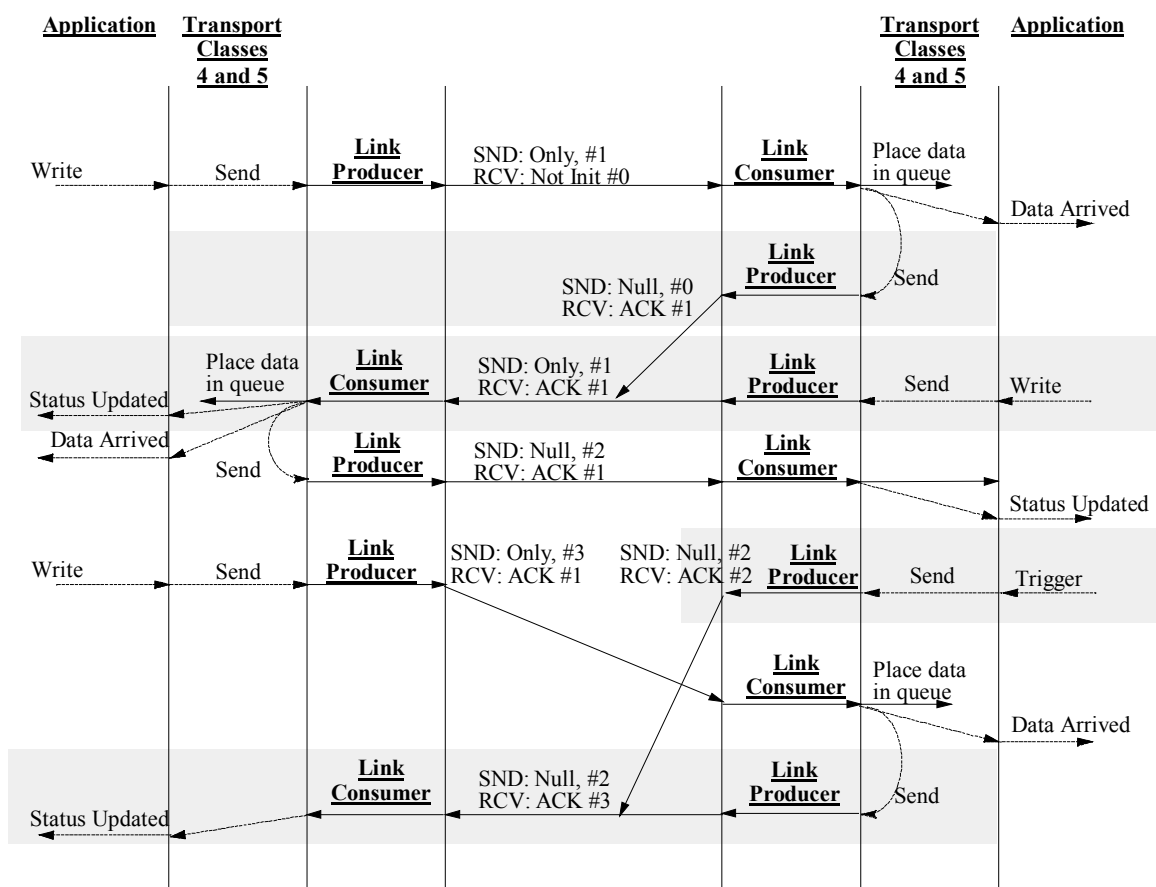


Figure 87 – Sequence diagram of messages overwriting each other

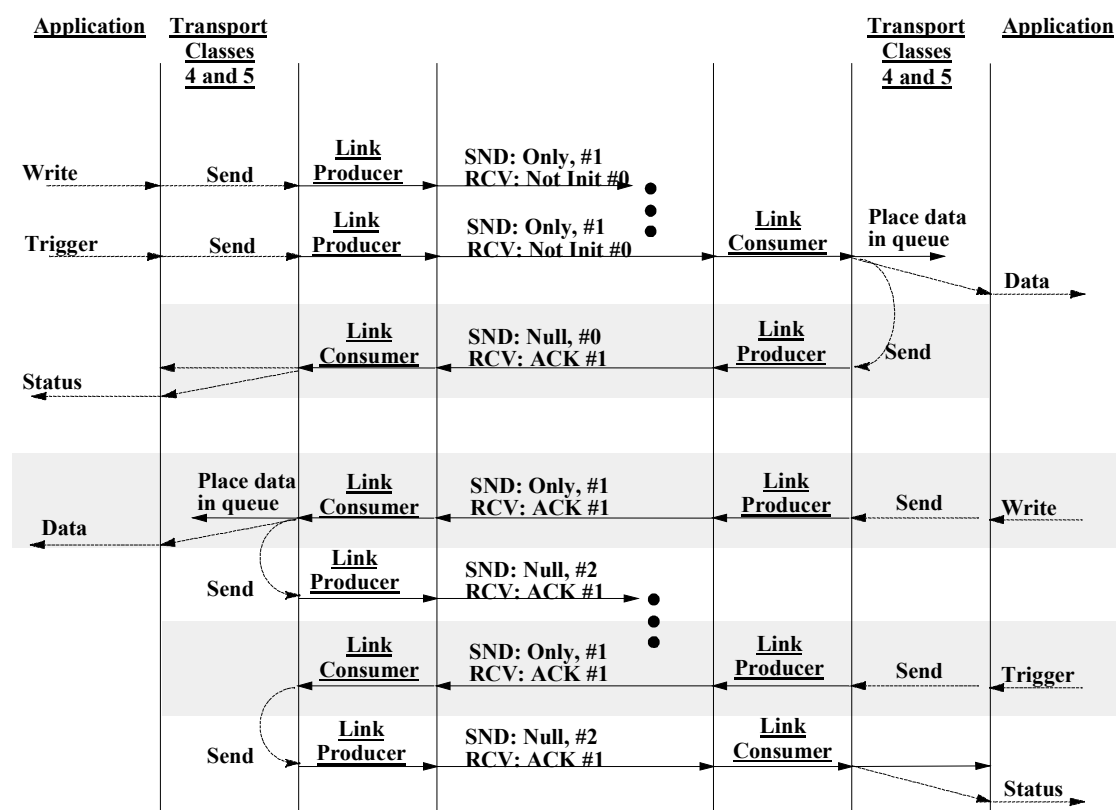


Figure 89 – Sequence diagram of retries using transport classes 4 and 5

5.6.3.9.3.6 Typical idle sequence

When a class 4 or 5 transport has nothing to send which shall impact the other state machine, it does not need to send anything. However, it is necessary to send some small amount of traffic to keep the connection alive. The Trigger event shall be used, with the timer set to a “keep alive” value. Figure 90 shows the *Null* command used to keep the connection alive, using the Trigger event.

NOTE the *Null* command does not need to be acknowledged. The other transport shall not retry a *Null* command. It shall continue to resend the *Null* command at the “keep alive” rate.

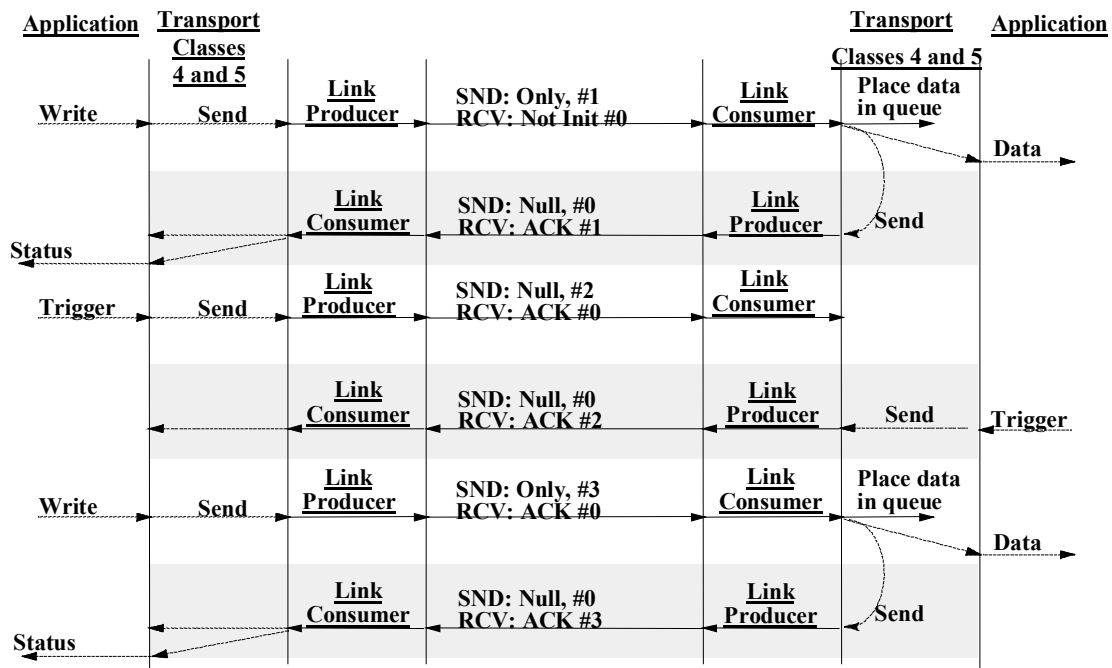


Figure 90 – Sequence diagram of idle traffic using transport classes 4 and 5

5.6.3.9.4 Classes 4 to 6 basic structure

To make the specification of the Non-Blocking classes (Classes 4 and 5) easier to read, the specification describes them as being made up of two state machines: one to handle the “Sender” function (Transport PDU’s from this device to the other) and the other to handle the “Receiver” function (Transport PDU’s from the other device to this one). (The class 6 transport has a client and a server state machine similar to the class 3 transport.)

The Sender and Receiver state machines in one node are coupled to each other via one event and three data items. The event and the data items come from the Receiver state machine to the Sender one. The *NewData* event indicates to the Sender state machine that one or more of the data items have changed. The *To_Send* and *From_Recv* data items are copies of the headers as shown in the diagram. The *Flag* data item is used by the Receiver state machine to tell the Sender state machine that it shall generate a Send event to the link producer, even if the Sender one has no reason to do so. This usually is accompanied by a change in the *From_Recv* data item, but not all changes in *From_Recv* need to be sent out immediately.

The Sender state machine has control of the Link Producer; the Receiver state machine accepts the data from the Link Consumer. The Sender state machine accepts data from the application; the Receiver state machine delivers data to the application. (This application data might be either Application Requests or Responses or data; the transport does not care nor know which it is.)

Figure 91 shows the relationship of these state machines. Both state machines are contained in one end node. The other end is identical.

Figure 92 shows the basic structure for class 6.

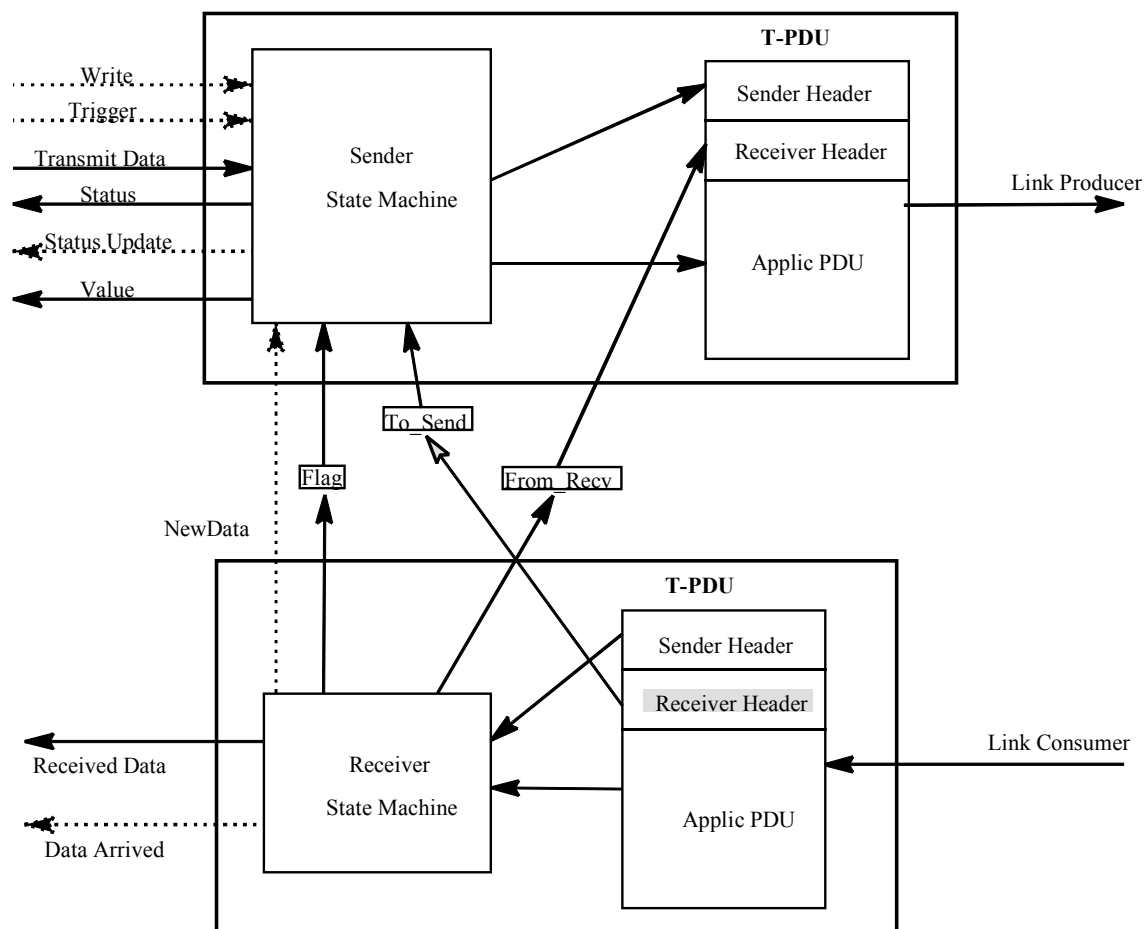


Figure 91 – Classes 4 and 5 basic structure

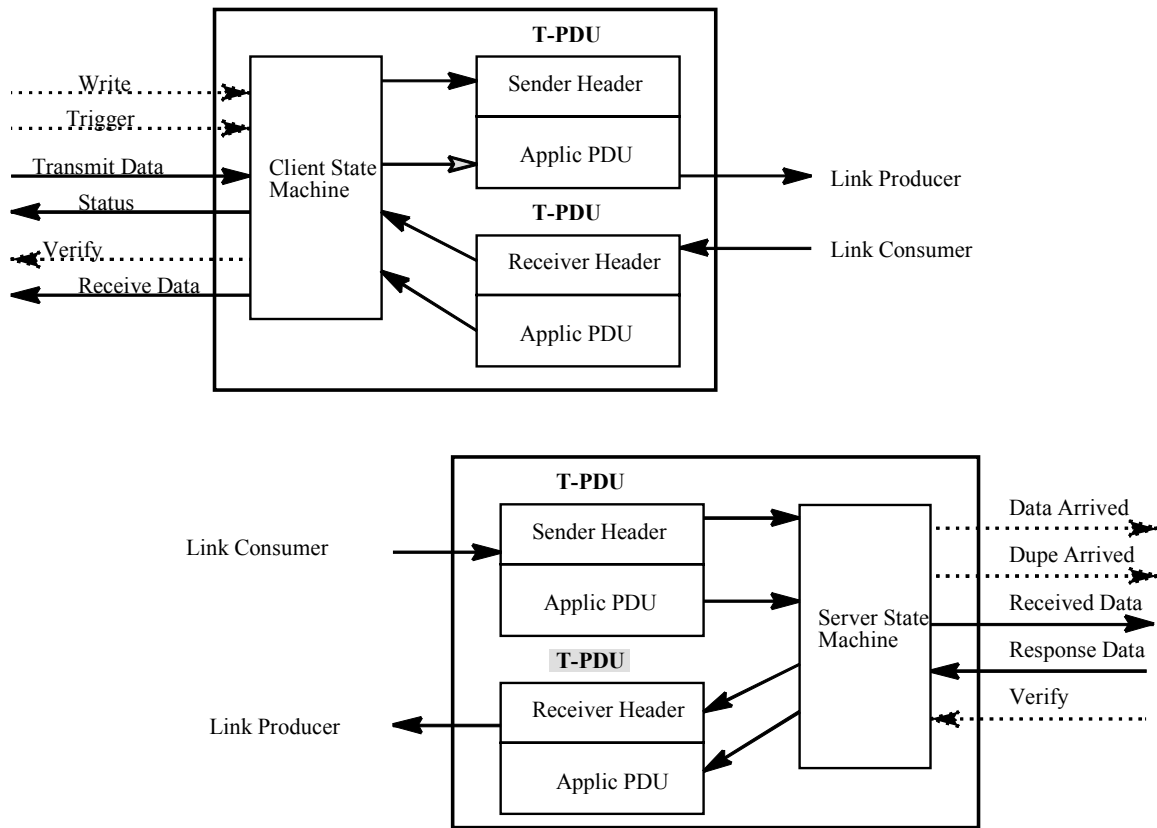


Figure 92 – Class 6 basic structure

5.6.3.9.5 Classes 4 to 6 general states

The defined states and their descriptions of the Class 4 Transport Sender are listed in Table 371 below :

Table 371 – Common states for transport classes 4 to 6

State	Description
Non-existent	The instance of Transport Class 4 to 6 does not exist.
Idle	The instance of Transport Class 4 to 6 has been created but not yet started.
Operational	The instance of Transport Class 4 to 6 has been created and started. NOTE this state has sub-states described in features particular to individual classes 4, 5 and 6.

5.6.3.9.6 Classes 4 to 6 general state transitions

Figure 93 shows the general behaviour of the transport classes. Each of the classes has some additional details added to the “Operational” state. This general behaviour shall **not** be repeated for each class in the interest of emphasizing the common behaviours and allowing those other descriptions to focus on the unique aspects of that class.

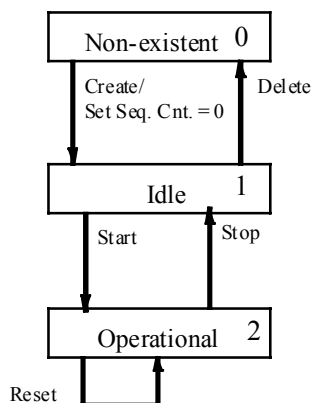


Figure 93 – Classes 4 to 6 general STD

5.6.3.9.7 Classes 4 to 6 general state event matrix

Table 372 defines the general state transitions of the transport classes.

Table 372 – Classes 4 to 6 general SEM

Event	State		
	Non-existent	Idle	Operational
Create	1) Seq. Cnt. = 0 2) Transition to Idle	Error	Error
Delete	Error	Transition to Non-existent	Error
Start	Error	Transition to Operational	Error
Stop	Error	No Action	Transition to Idle
Reset	Error	Error	Generally, stop waiting for outstanding events; details are transport class specific

5.6.3.9.8 Production triggers

IEC 61158-6 defines how these transport classes act as different trigger modes are specified when the Connection Manager creates an instance of the transport.

5.6.3.10 Transport state machines – class 4

5.6.3.10.1 Functions

The sequence count is set to 0 by the create service. It is incremented by the transport to distinguish a new Transport PDU from a retransmission of a previous one. It has a maximum value of 15. After that it rolls over to 0.

Transport class 4 is similar to having two class 2 transports, one in each direction, except that the class 4 uses two Network Connections instead of four. Transport class 4 can transfer application information in both directions. The delivery is acknowledged by the receiving transport. It uses a sequence number to identify which TPDU is being acknowledged. Retries are performed.

The main advantage of this class over using two Class 2 (Acknowledgement) transports is that class 4 uses fewer resources. The main advantage of class 4 over class 3 is that it does not block the application from sending a second request before the first response is received. This pipelining can improve the system performance in some circumstances. In addition, class 4 supports bi-directional, asynchronous data transmission, without imposing request-response type serialisation.

This transport class notifies the application that the server application has responded to the transmitted packet. This transport class shall automatically send the next item in the queue when it receives an acknowledge that the previous one has been acknowledged. The application shall be notified if the network connection fails.

Transport class 4 uses two connections: the originator-to-target connection, which shall be point-to-point; and the return connection, which also shall be point-to-point.

The receiving transport initiates and sends the acknowledgement *after* the consuming node of the network connection has written the packet to the T-PDU buffer, *after* the transport has read the packet's transport header, and *after* the transport has placed a copy of the data into a queue for the application. The receiver's acknowledgement signals the sending transport that the receiver-side T-PDU receive buffer is ready to accept the next data packet.

The limit to the number of application requests which can be sent before an application response is received is determined by the design of the target application. It shall typically be limited by the amount of memory which the receiving transport can allocate for holding the outstanding messages. An error shall be returned to the sending transport when a transmission attempts to exceed the limit.

5.6.3.10.2 Transport class 4 sender

5.6.3.10.2.1 Functions

The class 4 Sender transport is responsible for:

- Accepting the application's *write* that indicates the application has written a data packet to the transmit data queue;
 - Writing the application data and the transport headers to the Sender T-PDU transmit buffer for each packet that the application writes to it
- NOTE The transport header for each packet shall include a sequence count
- Initialising the sequence count in the transport header (for the first packet transmitted) or incrementing the sequence count (for subsequent packets of the same transmission);
 - Triggering the producing node of the network connection to produce the T-PDU packet on the link and send it to the consuming node of the network connection;
 - Performing retries if necessary when *triggered*;
 - Accepting notification from the associated Receiver state machine that it has written the Receiver header (the other transport instance's verification) to the shared data area;
 - Notifying the application that the application data has or has not been sent.

The following internal storage is assumed:

Current Sequence Number (SN)
Queue length
Try counter

5.6.3.10.2.2 States

The defined states and their descriptions of the Class 4 Transport Sender are listed in Table 373 below :

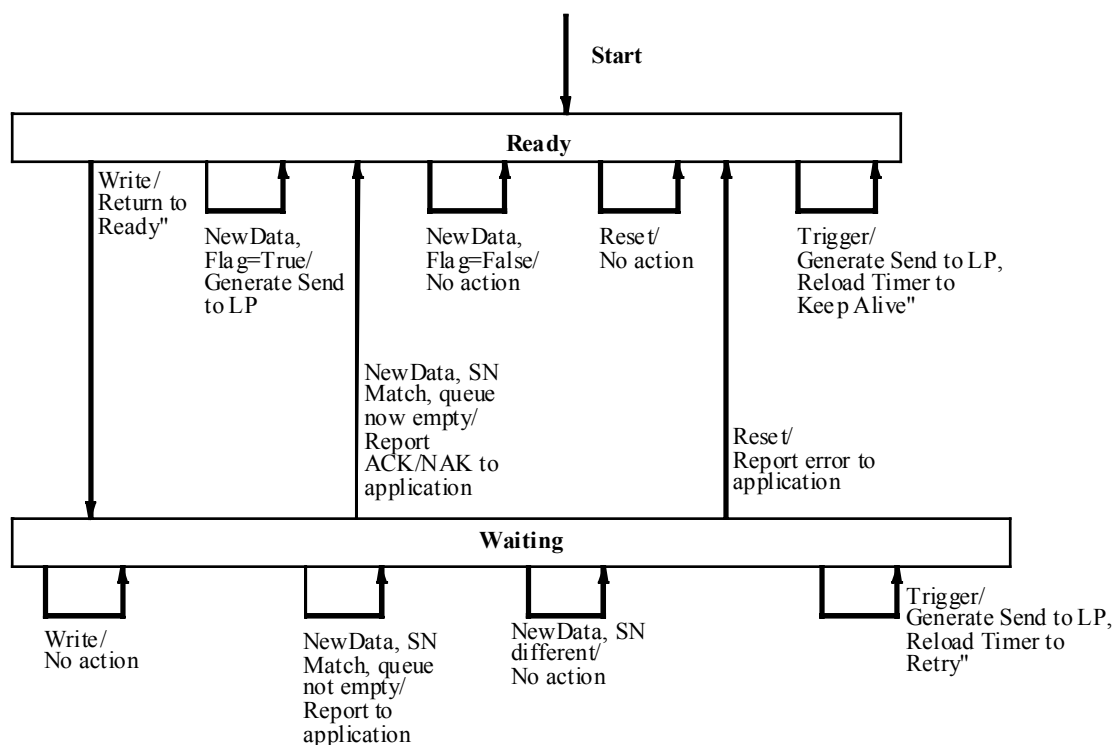
Table 373 – Class 4 transport sender states

State	Description
Ready	The instance of Transport Class 4 has been created and started. Queue is empty.
Waiting	The instance of Transport Class 4 has been created and started. Queue is not empty.

NOTE These two states are sub-states of the Running state described as the common feature for classes 4 to 6.

5.6.3.10.2.3 State transitions

State transitions of the class 4 sender transport are shown in Figure 94.

**Figure 94 – Class 4 sender STD**

NOTE This STD is incomplete. It shows the most common transitions. See Table 374 for the full specification of all events in all conditions.

The Command and Sequence Number columns in Table 374 refer to the values of those fields of the *To_Send* data item. In the action descriptions, the phrase "set CMD" refers to setting the value of the Command field of the Sender Header in the TPDU which shall be delivered to the Link Producer at the next *Send* event.

On all occurrences of the *New Data* event, the contents of the *From_Recv* data item are copied to the Receiver Header in the TPDU.

NOTE This action is not listed in Table 374.

5.6.3.10.2.4 State event matrix

State transitions of the class 4 sender transport are shown in Table 374.

Table 374 – Class 4 sender SEM

Event	Command	Sequence Number	State	
			Ready	Waiting
Reset	Don't care	Don't care	Do "return to Ready processing"	Take the application data off the queue, place error in Status queue, generate Status Updated event to application, do "return to Ready processing"
Write	Don't care	Don't care	Do "return to Ready processing"	Ignore (Shall be recognised as part of "return to Ready processing")
NewData	ACK	Different	(Not expecting anything). Do "Flag processing".	(Assume the other transport is ACKing something old) Do "Flag processing".
		Same		Take the application data off the queue, place Success in Status queue, generate Status Updated event to application, do "return to Ready processing"
	NAK	Different		(Assume the other transport is NAKing something old) Do "Flag processing".
		Same		Take the application data off the queue, place error in Status queue, generate Status Updated event to application, do "return to Ready processing"
	Other	Don't care		(Assume other end is confused.) Do "Flag processing".
Trigger (Retry or Keep alive timer)	Don't care	Don't care	Generate Send event to Link Producer (which re-sends the <i>Null</i> message to keep connection alive), reload timer with "keep alive" value.	Decrement Try counter, if zero: Take the application data off the queue, report error to application, generate Can't Send event to application, do "return to Ready processing" else: Generate Send event to Link Producer (which re-sends the <i>Only</i> message), reload timer with retry value.

The "Return to Ready Processing" includes the following:

- If the queue is now empty:
 - Increment Sequence number and place in Sender Header of TPDU
 - Set CMD to *Null*
 - Load timer with "keep alive" value
 - Do the "Flag Processing" (described below)
 - Go to Ready state
- If the queue is not empty:
 - Increment Sequence number and place in Sender Header of TPDU
 - Set CMD to *Only*
 - Set Try counter
 - Copy message to TPDU buffer
 - Generate Send event to Link Producer
 - Load timer with "retry" value
 - Go to Waiting state

The "Flag Processing" is defined as: Test the value of the Flag data item from the Receiver state machine. If the Flag has a value of True then generate the Send event to the Link Producer. If the Flag has a value of False, do nothing.

5.6.3.10.3 Transport class 4 receiver

5.6.3.10.3.1 Functions

In the link to application direction, a class 4 Receiver transport is responsible for:

- Accepting the notification from the consuming node of the network connection that it has written a data packet to the server-side T-PDU receive buffer;
- Locating and reading the transport headers of each packet that the consuming node of the network connection writes to the server-side T-PDU receive buffer;
- Performing the actions specified in the state event matrix for each packet which arrives;
- Notifying the server application when it places new data into the received data queue;
- Deliver the appropriate data (*To_Send*, *From_Recv*, and *Flag*) to the associated Sender transport state machine with the *NewData* event.

The Receiver Bi-directional Point-to-Point Non-Blocking transport has two states:

Initial

Ready

The basic approach is to start in the *Initial* state and transition to the *Ready* state when it receives something that indicates the receiver is synchronised with the sender. It remains in the *Ready* state until the *Reset* service is applied.

5.6.3.10.3.2 States

The defined states and their descriptions of the Class 4 Transport Receiver are listed in Table 375 below :

Table 375 – Class 4 transport receiver states

State	Description
Ready	The instance of Transport Class 4 has been created and started. Queue is empty.
Waiting	The instance of Transport Class 4 has been created and started. Queue is not empty.

NOTE These two states are sub-states of the Running state described as the common feature for classes 4 to 6.

5.6.3.10.3.3 State transitions

State transitions of the class 4 receiver transport are shown in Figure 95.

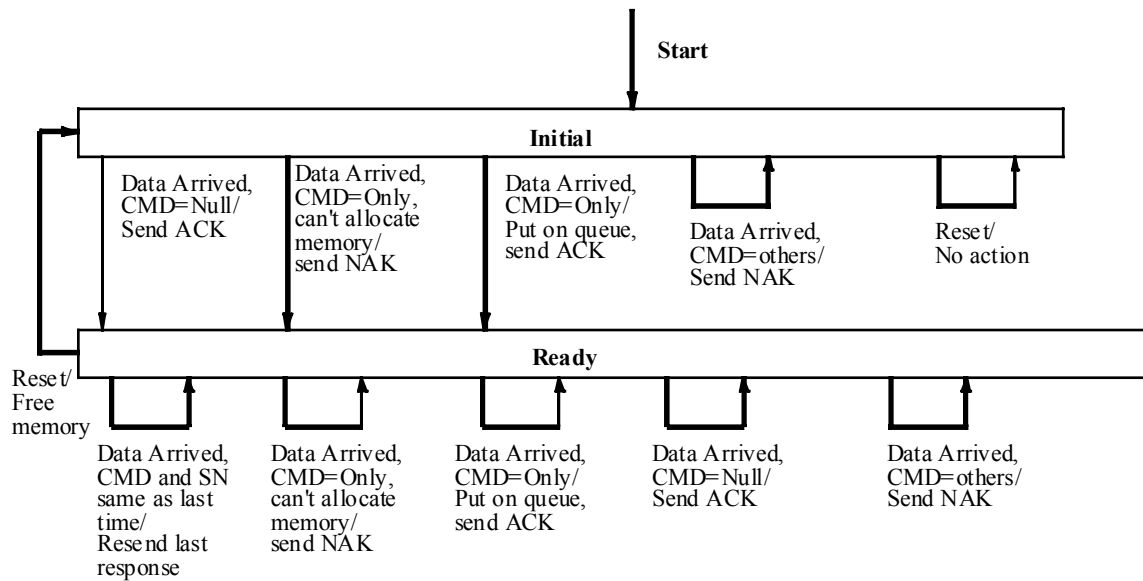


Figure 95 – Class 4 receiver STD

The following events can happen. Some of these are labelled on the Data Flow Diagram as the Packet Arrived event; I have taken the liberty to also factor in what arrived and if it has the same or different Sequence Number (SN):

- Data Arrived, CMD = *Null*;
- Data Arrived, CMD = *Only*;
- Data Arrived, CMD is *Illegal*;
- Reset;

NOTE 1 There is **no** timer event used by the Receiver state machine. (If there is a communications problem, the Sender state machine should time out. The application should have a timer to indicate problems with the other application).

On all occurrences of the *Data Arrived* event, the contents of the Receiver Header are copied to the *To_Send* data item.

NOTE 2 This action is not listed in Table 376.

The phrase “Set CMD” used in Table 376 means “set the Command field of the *From_Recv* data item”. When the value of CMD or SN is used to make a decision, it refers to the value of the Command or Sequence Number fields of the Sender Header in the TPDU received from the Link Consumer which caused the Data Arrived event.

The following internal storage is assumed:

Previous SN

Previous CMD received

5.6.3.10.3.4 State event matrix

State transitions of the class 4 receiver transport are shown in Table 376.

Table 376 – Class 4 receiver SEM

Event	State	
	Initial	Ready
Reset	No action	Set the Command field of the <i>From_Recv</i> data item to <i>Not_Init</i> and the Sequence Number of the <i>From_Recv</i> data item to zero. Go to Initial.
Data Arrived, CMD=Null, SN different	Copy SN to <i>From_Recv</i> , set CMD to ACK, set Flag to False, generate NewData event, go to Ready.	Copy SN to <i>From_Recv</i> , set CMD to ACK, set Flag to False, generate NewData event, stay in Ready.
Data Arrived, CMD=Null, SN same		If previous CMD was <i>Null</i> , then: (Assume other transport still has nothing for us to do, assume SN and CMD are OK.) Set Flag to False, generate NewData event, stay in Ready. (There is no need to rush out an ACK to a <i>Null</i> , let the sender's refresh send it out.) else: (Assume other transport made an error by sending a different CMD, but used the same SN). Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), set Flag to False, generate NewData event, stay in Ready.
Data Arrived, CMD=Only, SN different	Allocate buffer; if buffer allocation succeeds: Copy data to buffer, put on Received queue, generate Arrived event, copy SN to <i>From_Recv</i> , set CMD to ACK, set Flag to True, generate NewData event, go to Ready. else: Copy SN to <i>From_Recv</i> , set CMD to NAK (no memory), set Flag to True, generate NewData event, go to Ready.	Allocate buffer; if buffer allocation succeeds: Copy data to buffer, put on Received queue, generate Arrived event, copy SN to <i>From_Recv</i> , set CMD to ACK, set Flag to True, generate NewData event, stay in Ready else: Copy SN to <i>From_Recv</i> , set CMD to NAK (no memory), set Flag to True, generate NewData event, stay in Ready.
Data Arrived, CMD=Only, SN same		If previous CMD was <i>Only</i> , then: Set Flag to True, generate NewData event, stay in Ready. (This is a retry, assume SN and CMD are OK.) (Flag is TRUE to speed recovery from noise, at the possible expense of some additional link transmit time.) else: Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), set Flag to True, generate NewData event, stay in Ready.
Data Arrived, CMD=Illegal	Copy SN to <i>From_Recv</i> , set CMD to NAK (illegal command), set Flag to True, generate NewData event, stay in Initial.	Copy SN to <i>From_Recv</i> , set CMD to NAK (illegal command), set Flag to True, generate NewData event, stay in Ready.

5.6.3.11 Transport state machines – class 5

5.6.3.11.1 Functions

Possible uses of transport class 5 include exchanging messages and responses up to 65536 octets between two applications.

The sequence count is set to 0 by the create service. It is incremented by the transport to distinguish a new Transport PDU from a retransmission of a previous one. It has a maximum value of 15. After that it rolls over to 0 limiting the transport to fewer than 16 outstanding messages.

Transport class 5 is similar to class 4, except that class 5 adds Fragmentation and Reassembly at the transport layer. Transport class 5 can transfer application information in both directions. The delivery of each fragment is acknowledged by the receiving transport. A sequence number identifies which TPDU is being acknowledged. Retries are performed.

This transport class notifies the application that the server application has responded to the transmitted packet. This transport class shall automatically send the next item in the queue when it receives an acknowledgement that the previous one has been acknowledged. The application shall be notified if the network connection fails.

Transport Class 5 uses two connections: the originator-to-target connection, which shall be point-to-point, and the return connection, which also shall be point-to-point.

The receiving transport initiates and sends the acknowledgement *after* the consuming node of the network connection has written the packet to the T-PDU buffer, *after* the transport has read the packet's transport header, and *after* the transport has placed a copy of the data into a queue for the application. The receiver's acknowledgement signals the sending transport that the receiver-side T-PDU receive buffer is ready to accept the next data packet.

The limit to the number of application requests which can be sent before an application response is received is determined by the design of the target application. It shall typically be limited by the amount of memory. An error shall be returned when a transmission attempts to exceed the limit.

5.6.3.11.2 Class 5 sequence diagrams

5.6.3.11.2.1 Typical sequences

Some typical sequences are illustrated to aid understanding of the operation of this transport class. A formal specification of this transport class is given in IEC 61158-5.

NOTE The following sequences are identical for classes 4 and class 5. They are described under the class 4 specification and are not repeated here.

- Typical Message Exchange Sequence (see 5.6.3.9.3.2);
- Typical Sequence with Messages overwriting each other (see 5.6.3.9.3.3);
- Typical Queued Sequence (see 5.6.3.9.3.4);
- Typical Sequence with retries (see 5.6.3.9.3.5);
- Typical Idle Sequence (see 5.6.3.9.3.6).

5.6.3.11.2.2 Typical three fragment sequence

Figure 96 shows a typical sequence where the application data is too large to fit into one TPDU. In the example shown, the application data is split into three fragments. In general, there could be many *Middle* TPDUs. Each *Middle* TPDU increments the Sequence Number in the Sender Header. The *Last* TPDU shall also increment the Sequence Number.

At the receiver, the *First* allocates the memory (and supplies the data for the first fragment) and the *Last* indicates it is time to notify the receiving application that there is some data for it to process (as well as supplying the data for the last fragment).

The unshaded areas in the figures indicate data transmission in one direction, and the shaded areas indicate data transmission in the other direction.

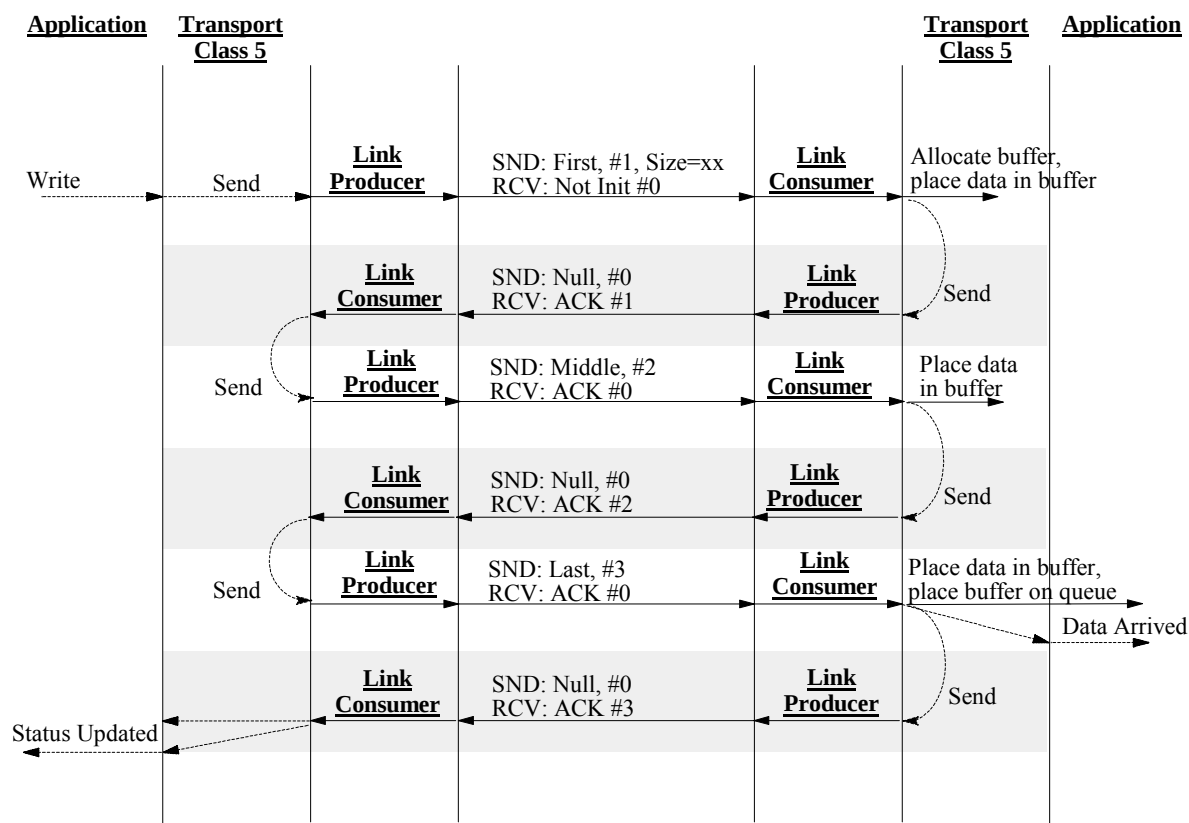


Figure 96 – Sequence diagram of three fragments using transport class 5

5.6.3.11.2.3 Typical fragmented sequence with retries

Figure 97 shows the processing when a transmission error causes something to be lost. In this example, two transmissions are lost, one in each direction.

NOTE The processing is very similar to class 4 (see 5.6.3.9.3.5).

The unshaded areas in the figures indicate data transmission in one direction, and the shaded areas indicate data transmission in the other direction.

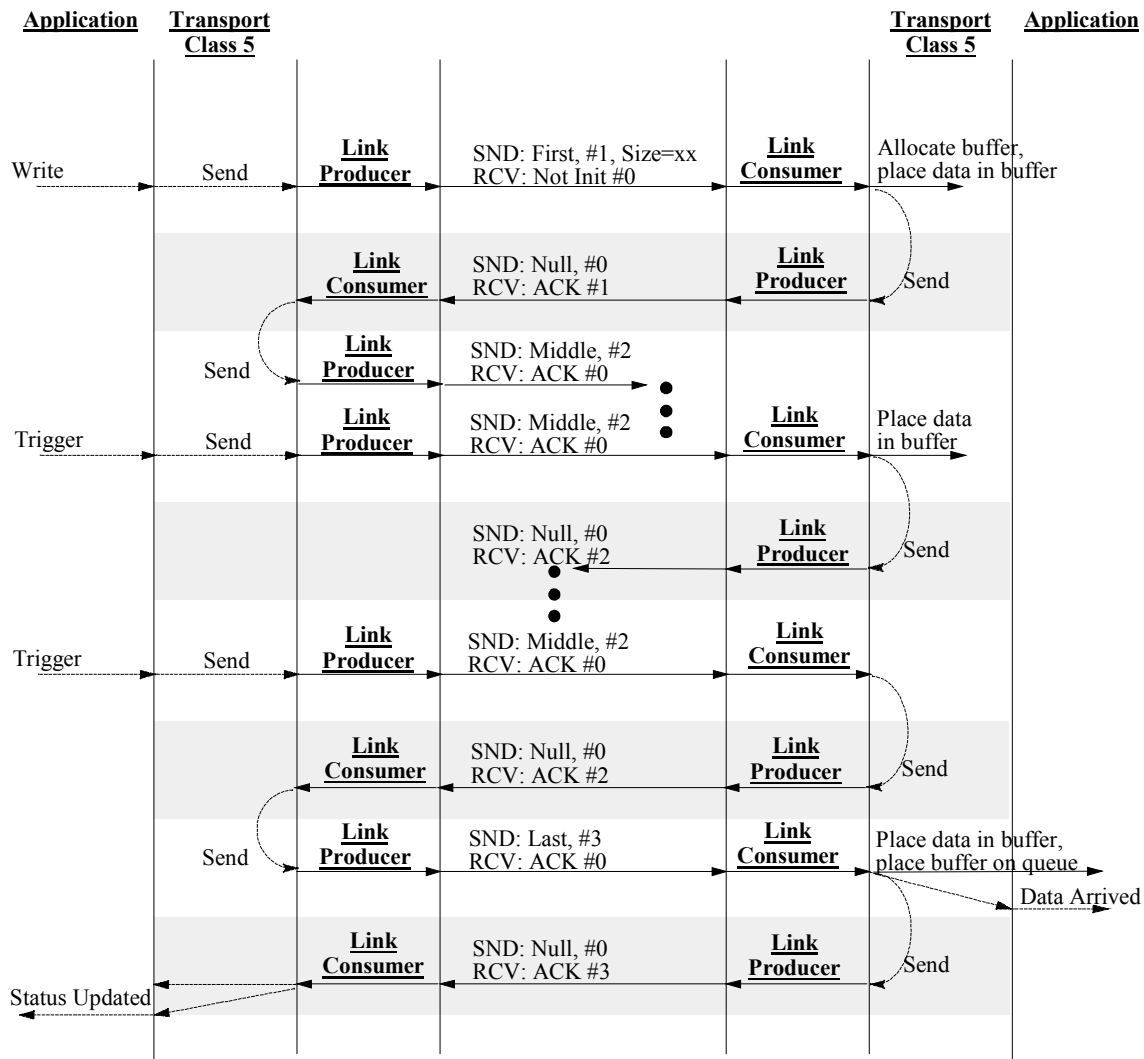


Figure 97 – Sequence diagram of fragmentation with retries using transport class 5

5.6.3.11.2.4 Typical two fragment sequence

Figure 98 shows a special case, where there are only two fragments. In this case, there is no *Middle* TPDU. Again, the *First* allocates the memory (and supplies the data for the first fragment) and the *Last* indicates it is time to notify the receiving application that there is some data for it to process (as well as supplying the data for the last fragment).

The unshaded areas in the figures indicate data transmission in one direction, and the shaded areas indicate data transmission in the other direction.

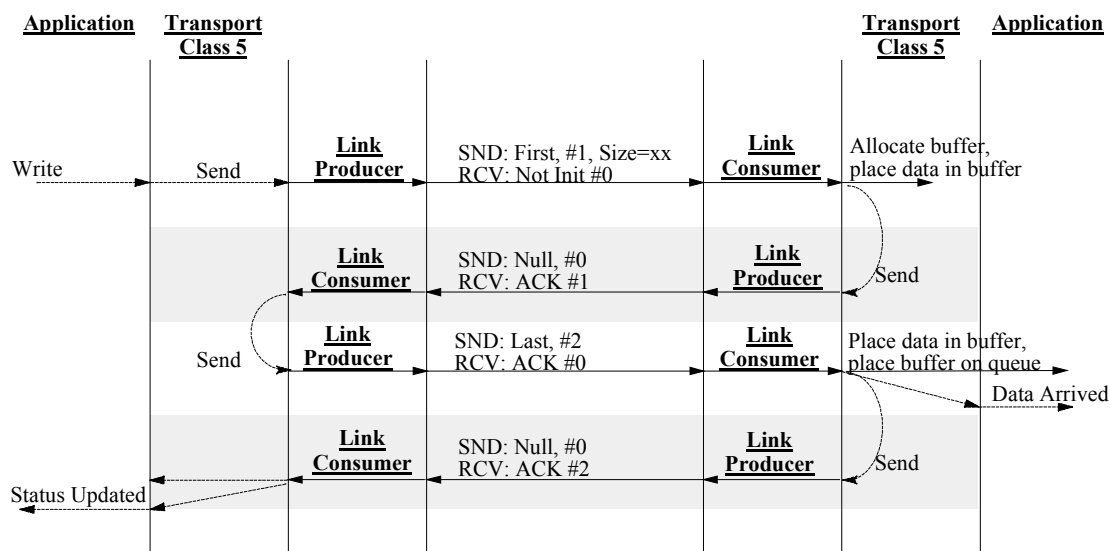


Figure 98 – Sequence diagram of two fragments using transport class 5

5.6.3.11.2.5 Typical aborted fragmented sequence

Figure 99 shows a receiver recovering when the sender has been reset in the middle of a fragmented sequence. When the receiver gets a *First* or *Only*, it assumes that any message which did not receive a *Last* has been aborted.

The unshaded areas in the figures indicate data transmission in one direction, and the shaded areas indicate data transmission in the other direction.

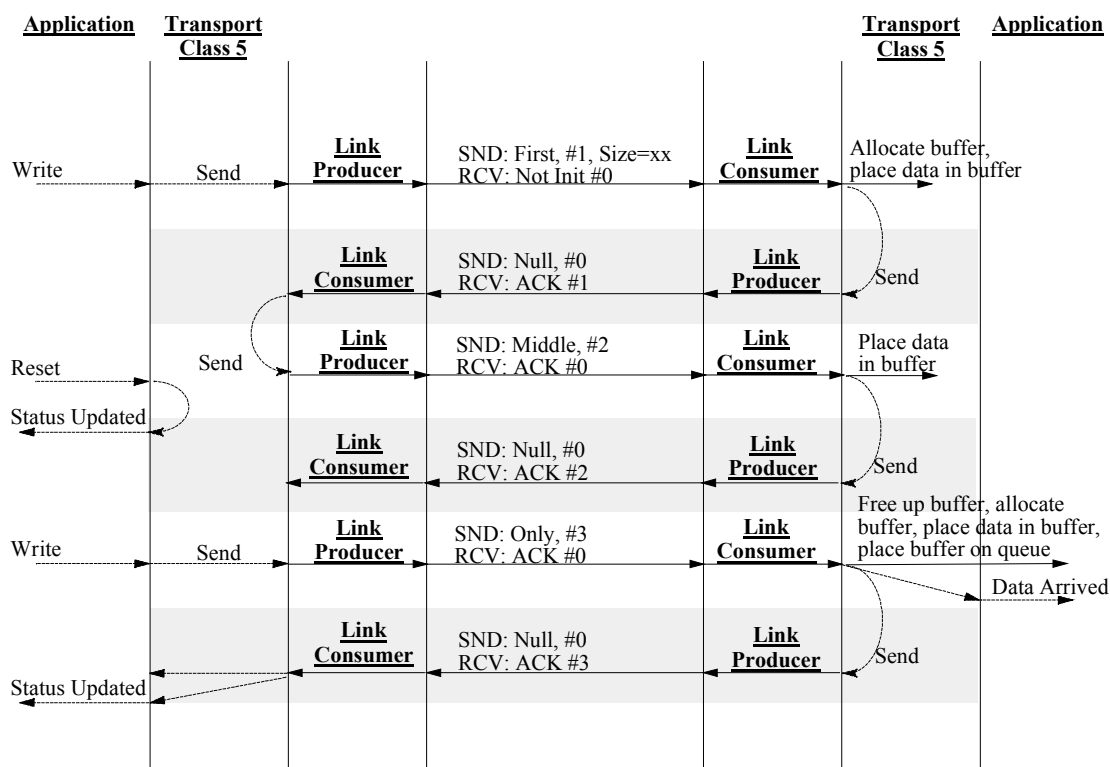


Figure 99 – Sequence diagram of aborted message using transport class 5

5.6.3.11.3 Transport class 5 sender

5.6.3.11.3.1 Functions

The Class 5 Sender transport is responsible for:

- Accepting the application's *write* that it has written a data packet to the transmit data queue;
- Writing the application data and the transport headers to the Sender T-PDU transmit buffer for each packet that the application writes to it;
- Initialising the sequence count in the transport header (for the first packet transmitted) or incrementing the sequence count (for subsequent packets of the same transmission);
- Triggering the producing node of the network connection to produce the T-PDU packet on the link and send it to the consuming node of the network connection;
- Performing retries if necessary when *triggered*;
- Accepting notification from the associated Receiver state machine that it has written the Receiver header (the other transport instance's verification) to the shared data area;
- Notifying the application that the application data has or has not been sent.

The following internal storage is assumed:

Current Sequence Number (SN);

Queue length;

Try counter.

5.6.3.11.3.2 States

The defined states and their descriptions of the Class 5 Transport Sender are listed in Table 377 below :

Table 377 – Class 5 transport sender states

State	Description
Ready	The instance of Transport Class 5 has been created and started. Queue is empty.
Waiting	The instance of Transport Class 5 has been created and started. Queue is not empty.

NOTE These two states are sub-states of the Running state described as the common feature for classes 4 to 6.

5.6.3.11.3.3 State transitions

State transitions of the class 5 sender transport are shown in Figure 100.

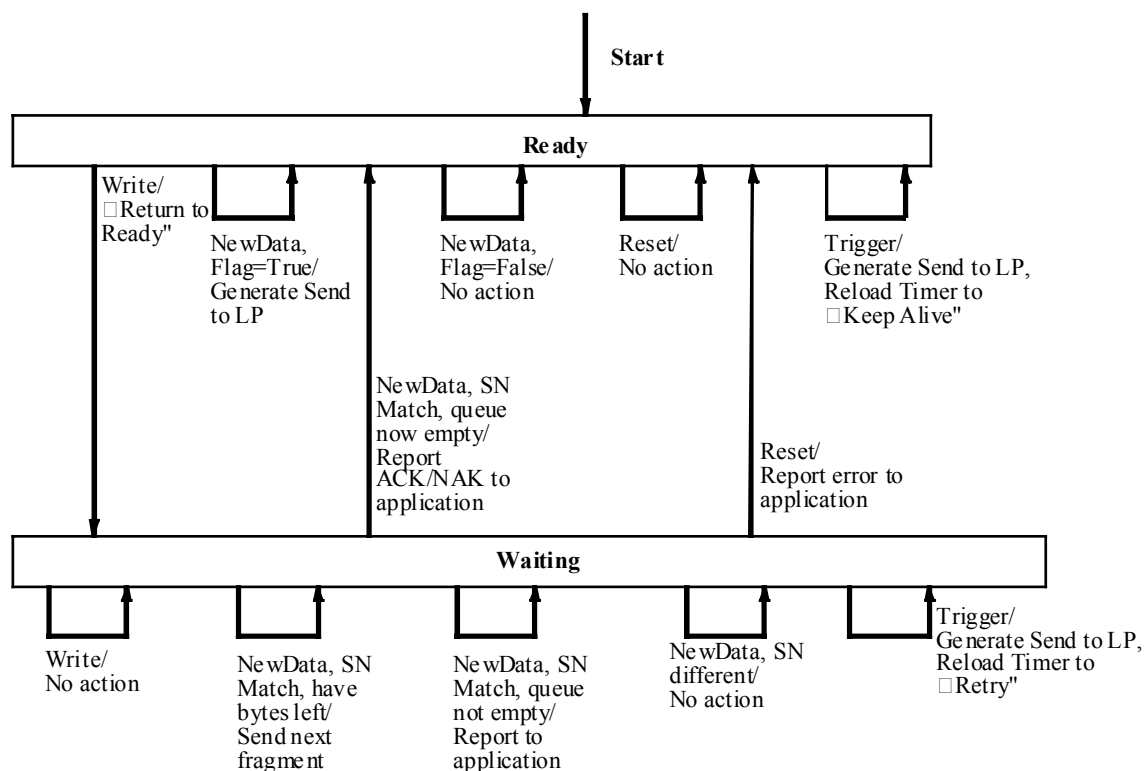


Figure 100 – Class 5 sender STD

NOTE This STD is incomplete. It shows some common transitions. See Table 378 for the full specification of all events in all conditions.

The Command and Seq. Number columns in Table 378 refer to the values of those fields of the *To_Send* data item. In the action descriptions, the phrase “set CMD” refers to setting the value of the Command field of the Sender Header in the TPDU which shall be delivered to the Link Producer at the next *Send* event. Also, the phrase “increment sequence number” refers to setting the value of the Sequence Number field of the Sender Header in the TPDU which shall be delivered to the Link Producer at the next *Send* event.

On all occurrences of the *New Data* event, the contents of the *From_Recv* data item are copied to the Receiver Header in the TPDU.

NOTE This action is not listed in Table 378.

5.6.3.11.3.4 State event matrix

State transitions of the class 5 sender transport are shown in Table 378.

Table 378 – Class 5 sender SEM

Event	Command	Sequence number	State	
			Ready	Waiting
Reset	n/a	n/a	Do "return to Ready processing"	Take the application data off the queue, place error in Status queue, generate Status Updated event to application, do "return to Ready processing"
Write	n/a	n/a	Do "return to Ready processing"	Ignore (Shall be recognised as part of "return to Ready processing")
NewData	ACK	Different	(Not expecting anything). Do "Flag processing".	(Assume the other transport is ACKing something old) Do "Flag processing".
		Same		If the whole application message has been sent, then take the application data off the queue, place Success in Status queue, generate Status Updated event to application, do "return to Ready processing" else, if the remainder fits in one fragment, then place the remainder in the Link Producer buffer, increment the sequence count, set the CMD to <i>Last</i> , and generate the Send event to the Link Producer else, place the next fragment in the Link Producer buffer, increment the sequence count, set the CMD to <i>Middle</i> , and generate the Send event to the Link Producer and stay in Waiting state.
	NAK	Different		(Assume the other transport is NAKing something old) Do "Flag processing".
		Same		Take the application data off the queue, place error in Status queue, generate Status Updated event to application, do "return to Ready processing"
	Other	n/a		(Assume other end is confused.) Do "Flag processing".
Trigger (Retry or Keep alive timer)	n/a	n/a	Generate Send event to Link Producer (which re-sends the <i>Null</i> message to keep connection alive), reload timer with "keep alive" value.	Decrement Try counter, if zero: Take the application data off the queue, report error to application, generate Can't Send event to application, do "return to Ready processing" else: Generate Send event to Link Producer (which re-sends the previous message), reload timer with retry value.

The "Return to Ready Processing" includes the following:

If the queue is now empty:

- Increment Sequence number
- Set CMD to *Null*
- Load timer with "Keep alive" value
- Do the "Flag Processing" (described below)
- Go to Ready state

If the queue is not empty:

- Increment Sequence number
- Test if the message shall fit in one fragment; if it does:
 - Set CMD to *Only*
 - Copy message to TPDU buffer

else

- Set CMD to First
- Calculate (total size / 256 octets) – 1; put into Sender Header
- Copy first fragment to TPDU buffer

Set Try counter

Load timer with “Retry” value

Generate Send event to Link Producer

Go to Waiting state

The “Flag Processing” is defined as: Test the value of the Flag data item from the Receiver state machine. If the Flag has a value of True then generate the Send event to the Link Producer. If the Flag has a value of False, do nothing.

5.6.3.11.4 Transport class 5 receiver

5.6.3.11.4.1 Functions

In the link to application direction, a Class 5 Receiver transport is responsible for:

- Accepting the notification from the consuming node of the network connection that it has written a data packet to the server-side T-PDU receive buffer;
- Locating and reading the transport headers of each packet that the consuming node of the network connection writes to the server-side T-PDU receive buffer;
- Performing the actions specified in the state event matrix for each packet which arrives;
- Notifying the server application when it places new data into the received data queue;
- Deliver the appropriate data and flags to the associated Sender transport.

The Receiver Bi-directional Point-to-Point Non-Blocking transport has three states:

Initial;

Assembling;

Ready.

The following internal storage is assumed:

Previous Sequence Number (SN);

Previous Command (CMD).

The basic approach is to start in the *Initial* state and transition to either the *Assembling* or the *Ready* state when it receives something that indicates the receiver is synchronised with the sender. It then stays in the *Ready* and *Assembling* states until the *Reset* service is applied. It goes into the *Assembling* state after receiving a *First* TPDU and remains there until getting the *Last* TPDU (unless it thinks the sender has aborted sending the current T-SDU).

5.6.3.11.4.2 States

The defined states and their descriptions of the Class 5 Transport Receiver are listed in Table 379 below :

Table 379 – Class 5 transport receiver states

State	Description
Initial	The instance of Transport Class 5 has been created and started.
Assembling	The instance of Transport Class 5 has been created and started. Segmented data is being assembled.
Ready	The instance of Transport Class 5 has been created and started. Segmented data has been assembled.

NOTE These three states are sub-states of the Running state described as the common feature for classes 4 to 6.

5.6.3.11.4.3 State transitions

State transitions of the class 5 receiver transport is shown in Figure 101.

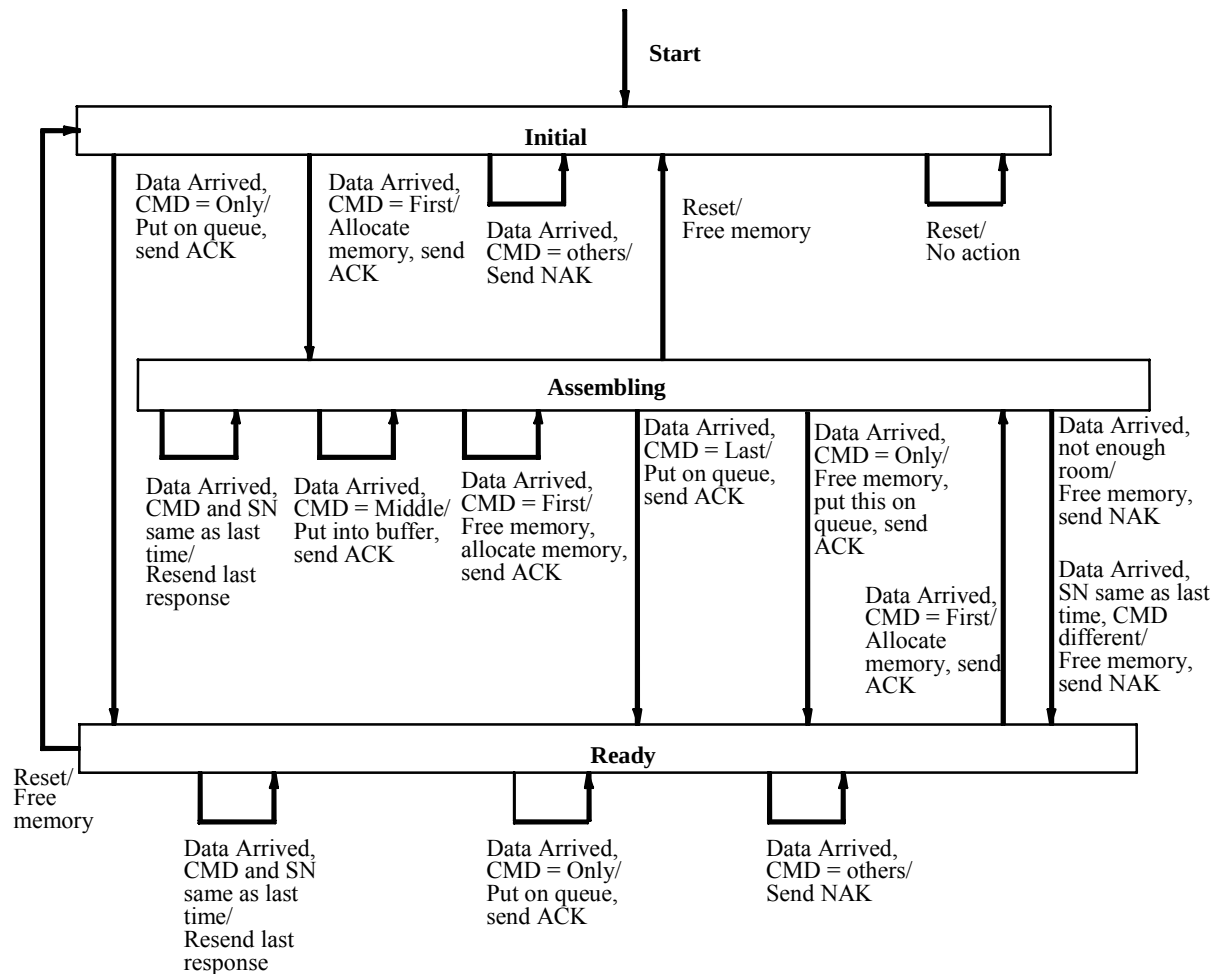


Figure 101 – Class 5 receiver STD

The following events can happen.

- *Reset* Event;
- Data Arrived Event, CMD= Null;
- Data Arrived Event, CMD= Only;
- Data Arrived Event, CMD= First;
- Data Arrived Event, CMD= Middle;
- Data Arrived Event, CMD= Last;
- *Data Arrived* Event, CMD= Illegal command.

NOTE There is **no** timer event used by the Receiver state machine. (If there is a communications problem, the Sender state machine should time out. The application should have a timer to indicate problems with the other application.)

The following internal storage is assumed:

Previous SN;
 Previous CMD received;
 Buffer Pointer;
 Room Left.

5.6.3.11.4.4 State event matrix

State transitions of the class 5 receiver transport is shown in Table 380.

Table 380 – Class 5 receiver SEM

Event	State		
	Initial	Ready	Assembling
Reset	No action.	Go to Initial.	Free memory previously allocated. Go to Initial.
Data Arrived, CMD=Null, SN different	Copy SN to From_Recv, set CMD to ACK, set Flag to False, generate NewData event, go to Ready.	Copy SN to From_Recv, set CMD to ACK, set Flag to False, generate NewData event, stay in Ready.	Free memory previously allocated. Copy SN to From_Recv, set CMD to ACK, set Flag to False, generate NewData event, stay in Assembling.
Data Arrived, CMD=Null, SN same		If previous CMD was <i>Null</i> , then: (Assume other transport still has nothing for us to do, assume SN and CMD are OK.) Set Flag to False, generate NewData event, stay in Ready. else: (Assume other transport made an error by sending a different CMD, but used the same SN). Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), set Flag to False, generate NewData event, stay in Ready.	If previous CMD was <i>Null</i> , then: (Assume other transport still has nothing for us to do, assume SN and CMD are OK.) Set Flag to False, generate NewData event, stay in Assembling. else: (Assume other transport made an error by sending a different CMD, but used the same SN). Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), set Flag to False, generate NewData event, go to Ready.
Data Arrived, CMD=Only, SN different	Allocate buffer; if buffer allocation succeeds: Copy data to buffer, put on Received queue, generate Arrived event, copy SN to <i>From_Recv</i> , set CMD to ACK, set Flag to True, generate NewData event, stay in Initial else: Copy SN to <i>From_Recv</i> , set CMD to NAK (no memory), set Flag to True, generate NewData event, go to Ready.	Allocate buffer; if buffer allocation succeeds: Copy data to buffer, put on Received queue, generate Arrived event, copy SN to <i>From_Recv</i> , set CMD to ACK, set Flag to True, generate NewData event, stay in Ready else: Copy SN to <i>From_Recv</i> , set CMD to NAK (no memory), set Flag to True, generate NewData event, stay in Ready.	Free memory previously allocated. Allocate buffer; if buffer allocation succeeds: Copy data to buffer, put on Received queue, generate Arrived event, copy SN to <i>From_Recv</i> , set CMD to ACK, set Flag to True, generate NewData event, go to Ready. else: Copy SN to <i>From_Recv</i> , set CMD to NAK (no memory), set Flag to True, generate NewData event, go to Ready.

Table 380 (continued)

Event	State		
	Initial	Ready	Assembling
Data Arrived, CMD=Only, SN same		If previous CMD was <i>Only</i> , then: Set Flag to True, generate NewData event, stay in Ready. (This is a retry, assume SN and CMD are OK.) (Flag is TRUE to speed recovery from noise, at the possible expense of some additional link transmit time.) else: Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), set Flag to True, generate NewData event, stay in Ready.	Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), set Flag to True, generate NewData event, stay in Assembling. (The sender changed the CMD but not the SN.)
Data Arrived, CMD=First, SN different	Allocate buffer; if buffer allocation succeeds: Copy data to buffer, copy SN to <i>From_Recv</i> , initialise Buffer Pointer and Room Left, set CMD to ACK, set Flag to True, generate NewData event, go to Assembling state. else: Copy SN to <i>From_Recv</i> , set CMD to NAK (no memory), set Flag to True, generate NewData event, go to Ready.	Allocate buffer; if buffer allocation succeeds: Copy data to buffer, copy SN to <i>From_Recv</i> , initialise Buffer Pointer and Room Left, set CMD to ACK, set Flag to True, generate NewData event, go to Assembling state. else: Copy SN to <i>From_Recv</i> , set CMD to NAK (no memory), set Flag to True, generate NewData event, stay in Ready.	Free memory previously allocated. Allocate buffer; if buffer allocation succeeds: Copy data to buffer, copy SN to <i>From_Recv</i> , initialise Buffer Pointer and Room Left, set CMD to ACK, set Flag to True, generate NewData event, stay in Assembling else: Copy SN to <i>From_Recv</i> , set CMD to NAK (no memory), set Flag to True, generate NewData event, go to Ready.
Data Arrived, CMD=First, SN same		If previous CMD was <i>First</i> , then: Set Flag to True, generate NewData event, stay in Ready. (This is a retry, previous attempt errored, assume SN and CMD are OK.) else: Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), set Flag to True, generate NewData event, stay in Ready.	If previous CMD was <i>First</i> , then: Set Flag to True, generate NewData event, stay in Assembling. (This is a retry, assume SN and CMD are OK.) else: Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), set Flag to True, generate NewData event, stay in Assembling.
Data Arrived, CMD=Middle, SN different	Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), set Flag to True, generate NewData event, stay in Initial.	Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), set Flag to True, generate NewData event, stay in Ready.	If size of data is less than Room Left, then: Copy data to buffer at Buffer Pointer, copy SN to <i>From_Recv</i> , update Buffer Pointer and Room Left, set CMD to ACK, set Flag to True, generate NewData event, stay in Assembling. else: Free memory previously allocated. Copy SN to <i>From_Recv</i> , set CMD to NAK (no memory), set Flag to True, generate NewData event, go to Ready.

Table 380 (continued)

Event	State		
	Initial	Ready	Assembling
Data Arrived, CMD=Middle, SN same			If previous CMD was <i>Middle</i> , then: Set Flag to False, generate NewData event, stay in Assembling. (This is a retry, assume SN and CMD are OK.) else: Free memory previously allocated. Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), set Flag to True, generate NewData event, go to Ready.
Data Arrived, CMD=Last, SN different	Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), set Flag to True, generate NewData event, stay in Initial.	Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), set Flag to True, generate NewData event, stay in Ready.	If size of data is less than Room Left, then: Copy data to buffer at Buffer Pointer, put buffer on Received queue, generate Arrived event, copy SN to <i>From_Recv</i> , set CMD to ACK, set Flag to True, generate NewData event, go to Ready. else: Free memory previously allocated. Copy SN to <i>From_Recv</i> , set CMD to NAK (no memory), set Flag to True, generate NewData event, go to Ready.
Data Arrived, CMD=Last, SN same		If previous CMD was <i>Last</i> , then: Set Flag to True, generate NewData event, stay in Ready. (This is a retry, assume SN and CMD are OK.) else: Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), set Flag to True, generate NewData event, stay in Ready.	If previous CMD was <i>Last</i> , then: Set Flag to False, generate NewData event, go to Ready. (This is a retry, assume SN and CMD are OK.) (NOTE This shall happen: If the previous CMD is <i>Last</i> , then we shall be in the Ready state.) else: Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), set Flag to True, generate NewData event, stay in Assembling.
Data Arrived, CMD=Illegal	Copy SN to <i>From_Recv</i> , set CMD to NAK (illegal command), set Flag to True, generate NewData event, stay in Initial.	Copy SN to <i>From_Recv</i> , set CMD to NAK (illegal command), set Flag to True, generate NewData event, stay in Ready.	Copy SN to <i>From_Recv</i> , set CMD to NAK (illegal command), set Flag to True, generate NewData event, stay in Assembling.

5.6.3.12 Transport state machines – class 6

5.6.3.12.1 Functions

Possible uses of transport class 6 include delivering event notification messages (up to 65536 octets) to multiple servers.

Transport class 6 is similar to class 3. Transport class 6 can transfer application information in both directions. Class 6 supports multicasting in the Client to Server direction. Class 6 adds Fragmentation and Reassembly at the transport layer, but only in the Client to Server (multicast) direction.

The delivery of each fragment is acknowledged by the receiving transport. A sequence number identifies the TPDU being acknowledged. Retries are performed.

The sequence count is set to 0 by the create service. It is incremented by the transport to distinguish a new Transport PDU from a retransmission of a previous one. It has a maximum value of 15. After that it rolls over to 0.

Class 6 is NOT a superset of class 5. Class 6 is blocking; that is, the next application request is not delivered until the current one is acknowledged. Functionally, class 6 can be thought of as a fragmenting version of class 3, rather than as a multicasting version of class 5. However, class 6 reuses the transport header format from class 5, instead of the class 3 transport header.

This transport class notifies the application that the server application has responded to the transmitted packet. The application shall be notified if the network connection fails.

Transport class 6 uses multiple network connections: the Client-to-Server connection, which may be a multipoint connection; and the return connections (one per Server), which shall be point-to-point.

One significant difference between class 3 and class 6, which results from class 6 doing fragmentation: the “write” event causes the first (or only) fragment to be transmitted; the “trigger” event is only a timer function which is used to signal retries or keep-alive transmissions.

5.6.3.12.2 Class 6 data flow diagram

Figure 102 shows the data flow diagram for class 6. Note that there is a separate T-SDU and TPDU in the Client to Server direction. The client places the fragments into the TPDU. The server assembles them from the TPDU into the T-SDU. Since there is no fragmentation in the Server to Client direction, the Application places the data directly into the Server’s outgoing TPDU (as is done in class 3).

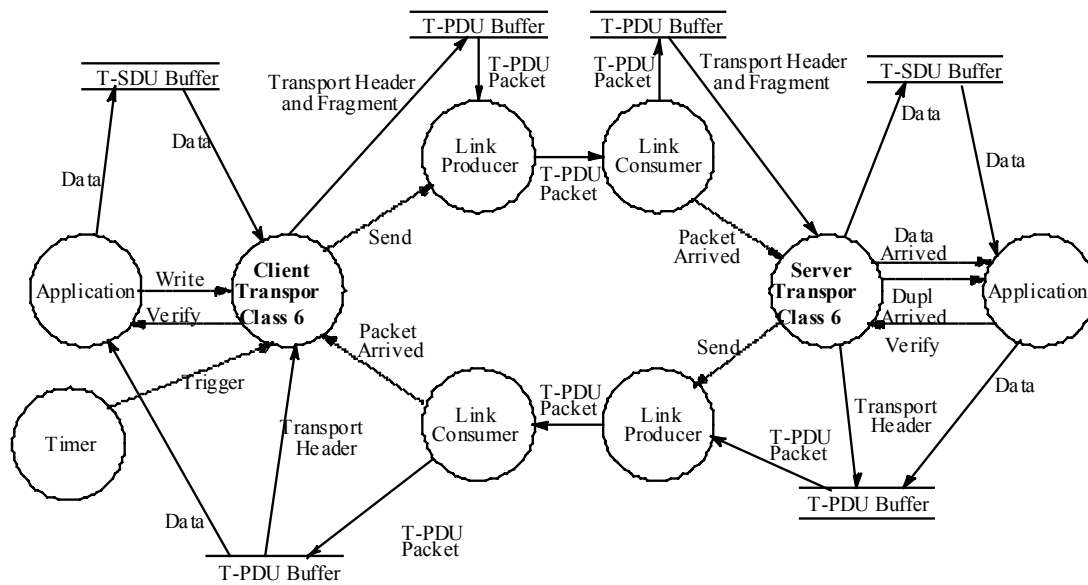


Figure 102 – Data flow diagram for transport class 6

5.6.3.12.3 Class 5 features missing in class 6

5.6.3.12.3.1 Function

It may be noted that the following two features of class 5 are not supported in class 6:

- The ability to have multiple outstanding requests queued between the receiving transport and its application;
- The support of fragmentation and reassembly in both directions.

This is a protocol design decision to exclude these features for the following reasoning. It is preferable that the class 6 transport header will not change size during the lifetime of a connection (due to the complexity of notifying all receivers of the change in header size). In the system, the multicasting receivers can enter and leave the relationship at will. Therefore, the number of multicasting receivers can change during the lifetime of the connection. Therefore, the size of the header shall be independent of the number of receivers. Since it is desirable to have no artificial limit to the number of receivers the decision was made to have **one** header field which all receivers use. This implies that all receivers shall be in the same state, which is maintained by having them all synchronised with the sender.

5.6.3.12.3.2 Blocking versus non-blocking

Assume there are several receivers of the multicast data. They are sending application responses to application requests to the multicasting sender. These application responses need to be acknowledged so that the transport can tell if it shall retry sending them or if it can now free up the resources used by that response. If the application responders are not synchronised with each other, they can be responding to different application requests. Therefore, each needs a field so that the application requester's transport can tell each of them about the successful delivery of the application response.

The design decision was not to have multiple fields, therefore all use the same field to decide if a retry is needed or if the resources can be released. Therefore, the transport delivery is synchronised with the application (the same as class 3).

5.6.3.12.3.3 Fragmentation in both directions

This was excluded by similar reasoning to the "Blocking vs. Non-Blocking" discussion. If the application responses from different responders require a different number of fragments, it would require the multicaster to send different transport headers to each of them.

5.6.3.12.4 Class 6 sequence diagrams

5.6.3.12.4.1 Typical sequences

Some typical sequences unique to class 6 are illustrated to aid understanding of the operation of this transport class. A formal specification of this transport class is given in IEC 61158-5.

The unshaded areas in the figures indicate data transmission in one direction, and the shaded areas indicate data transmission in the other direction.

5.6.3.12.4.2 Typical message exchange sequence

Figure 103 shows the sequence in which actions take place during a messaging request/response exchange using transport class 6. The unshaded areas in Figure 103 indicate data transmission in one direction, and the shaded areas indicate data transmission in the other direction. Each transmission shows the Command and Sequence Number used for each header.

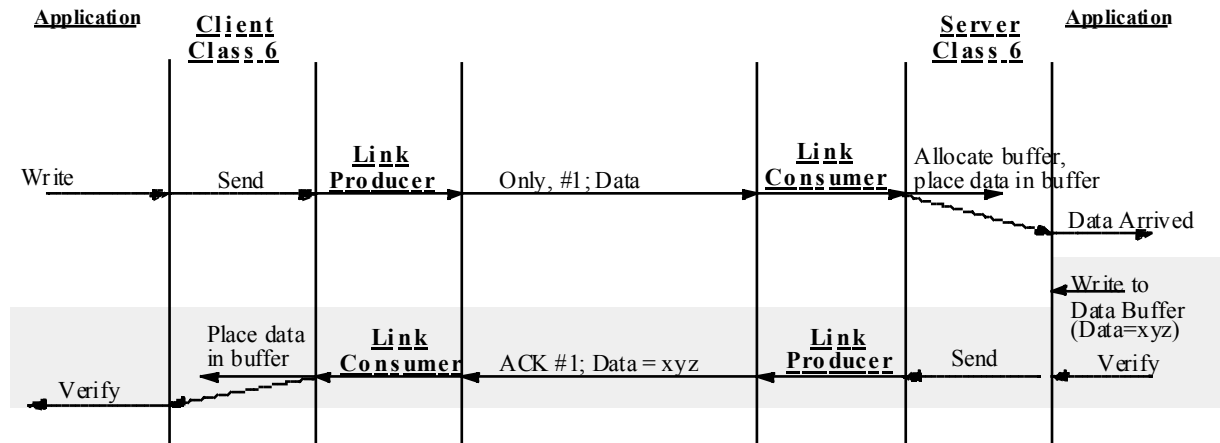


Figure 103 – Sequence diagram of message exchange using transport class 6

5.6.3.12.4.3 Typical message exchange sequence with retries

Retries are performed when a transport does not receive an Receiver header with the same sequence count as its current Sender header in the timeout period. The TPDU is not changed for a retry. Figure 104 shows two retry situations: where an *Only* command fails to be delivered and where an expected ACK fails to be delivered. This processing is very similar to class 3. The Trigger event is generated by the retry timer.

NOTE See 5.6.3.12.4.7 for a typical sequence with retries when the data is fragmented.

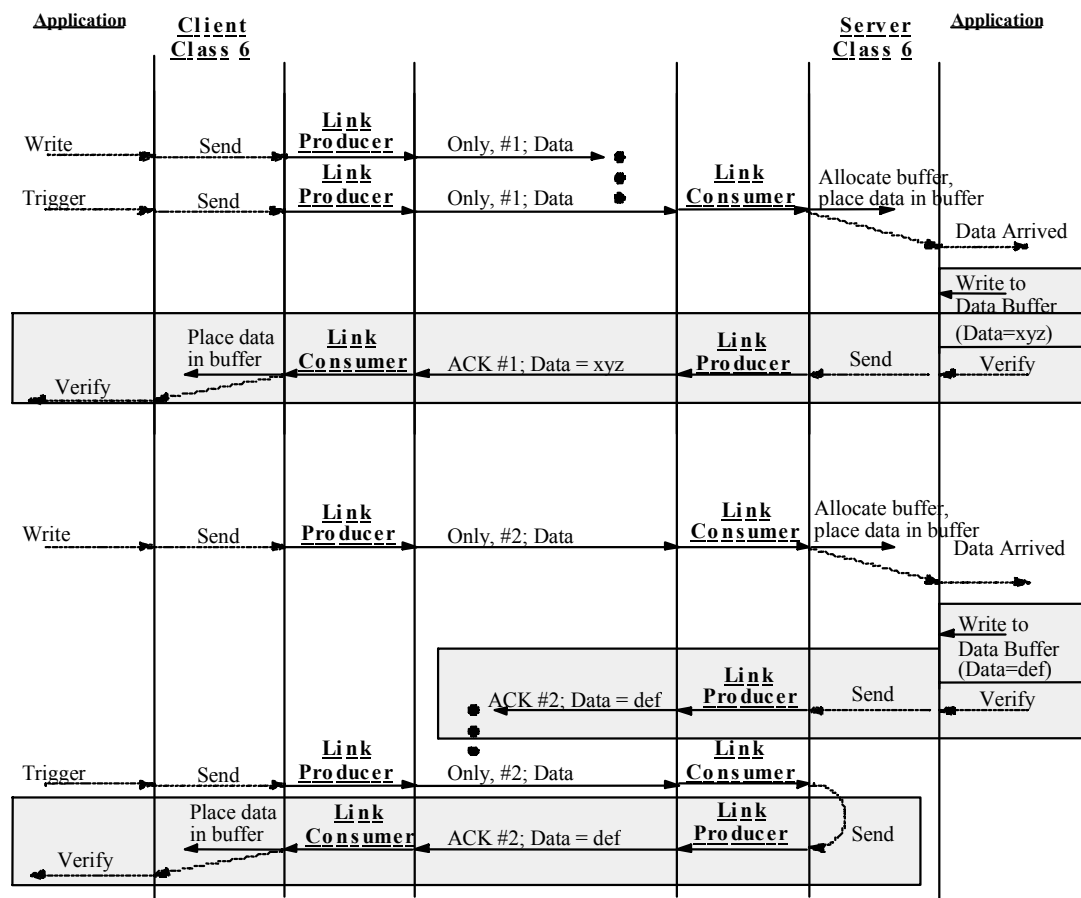


Figure 104 – Sequence diagram of retries using transport class 6

5.6.3.12.4.4 Typical idle sequence

When a class 6 Client transport has nothing to send which shall impact the other state machine, it does not need to send anything. However, it is necessary to send some small amount of traffic to keep the connection alive. The Trigger event shall be used, with the timer set to a “keep alive” value. Figure 105 shows the *Null* command used to keep the connection alive, using the Trigger event.

NOTE the Server does not have a timer, therefore the *Null* command shall be acknowledged by the server to keep the point to point network connection open. However, the client transport does not need to wait for the acknowledgement if the application gives it a Write event (see 5.6.3.12.4.5).

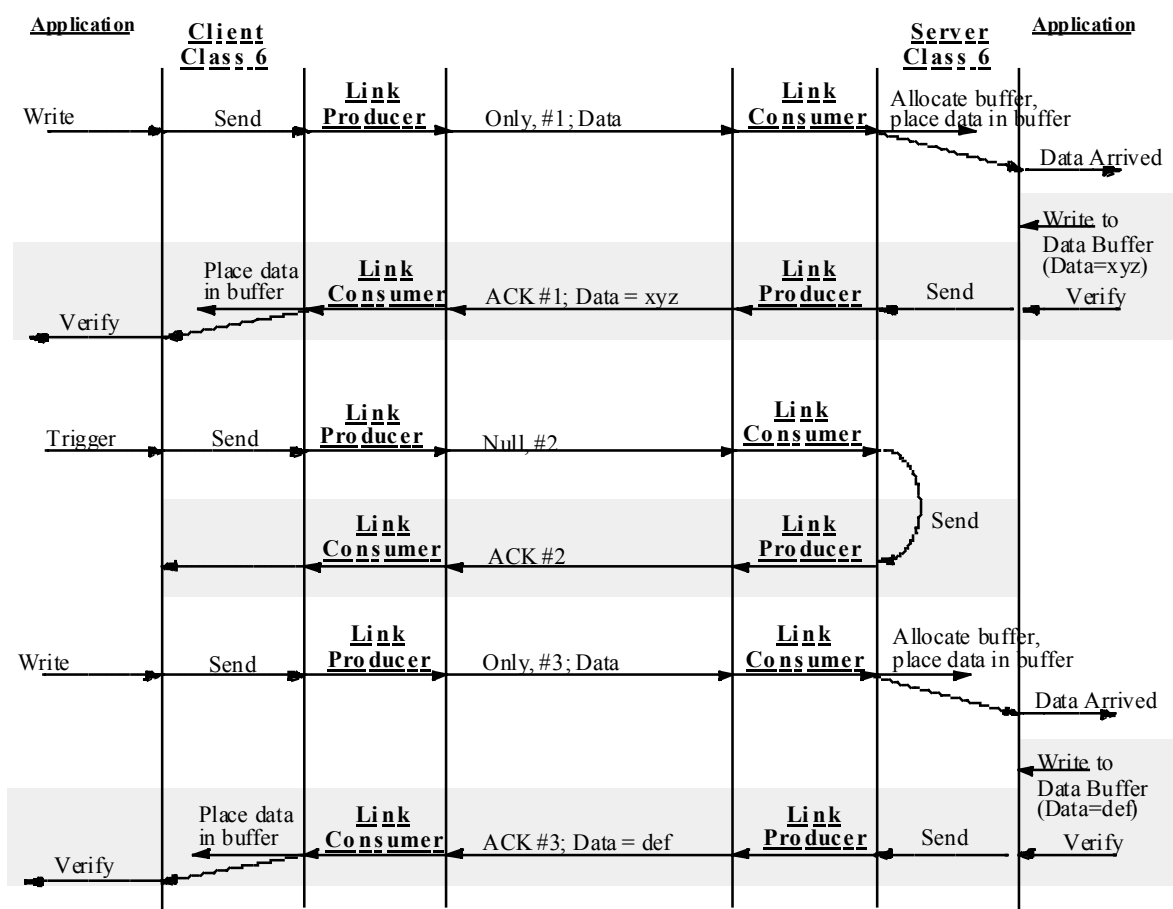


Figure 105 – Sequence diagram of idle traffic using transport class 6

5.6.3.12.4.5 Typical sequence with overwrite

It is permissible for a transport to send two data packets in a row. This implies that the second one might overwrite the first one in an intermediate module. This is desirable, since the second packet contains information that is newer than the first one and the data in the first one is no longer needed. There are two situations where this can occur when using Transport class 6.

In Figure 105 the transport on the left had nothing to send when it was triggered by the keep-alive timer. Therefore, it sent the *Null* #2. But when the application on the left did a WRITE, the transport immediately sent the *Only* #3. The previous example shows both of them arriving separately.

Figure 106 shows the left side's *Only* #3 overwriting the *Null* #2.

Figure 107 shows a different situation where an overwrite can occur. The left side sends a *Null* #2 as the result of the keep-alive *Trigger* event. But, before the ACK #2 can be delivered, the right side receives the *Only* #3 from the left side. The right side responded with the newer ACK #3, which overwrites the ACK #2.

The same situation shall occur if a *First* is used instead of the *Only*.

It is not necessary to wait for the ACK to a *Null* before sending an *Only* or a *First*.

When sending an *Only* or a *First*, it shall have a different sequence number than the *Null*, so sender knows which is being ACK'd by the other side.

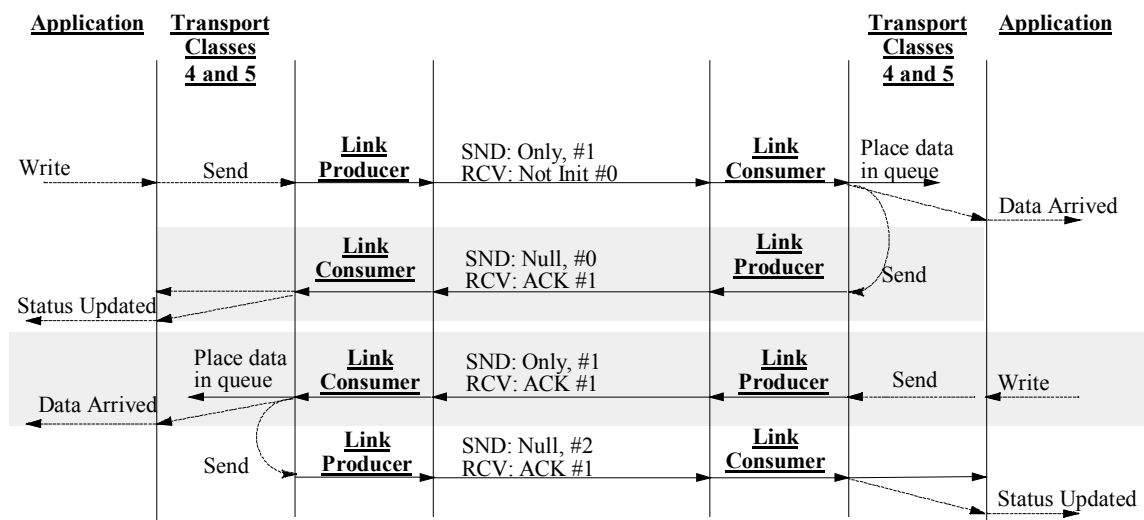


Figure 106 – Sequence diagram of request overwriting null

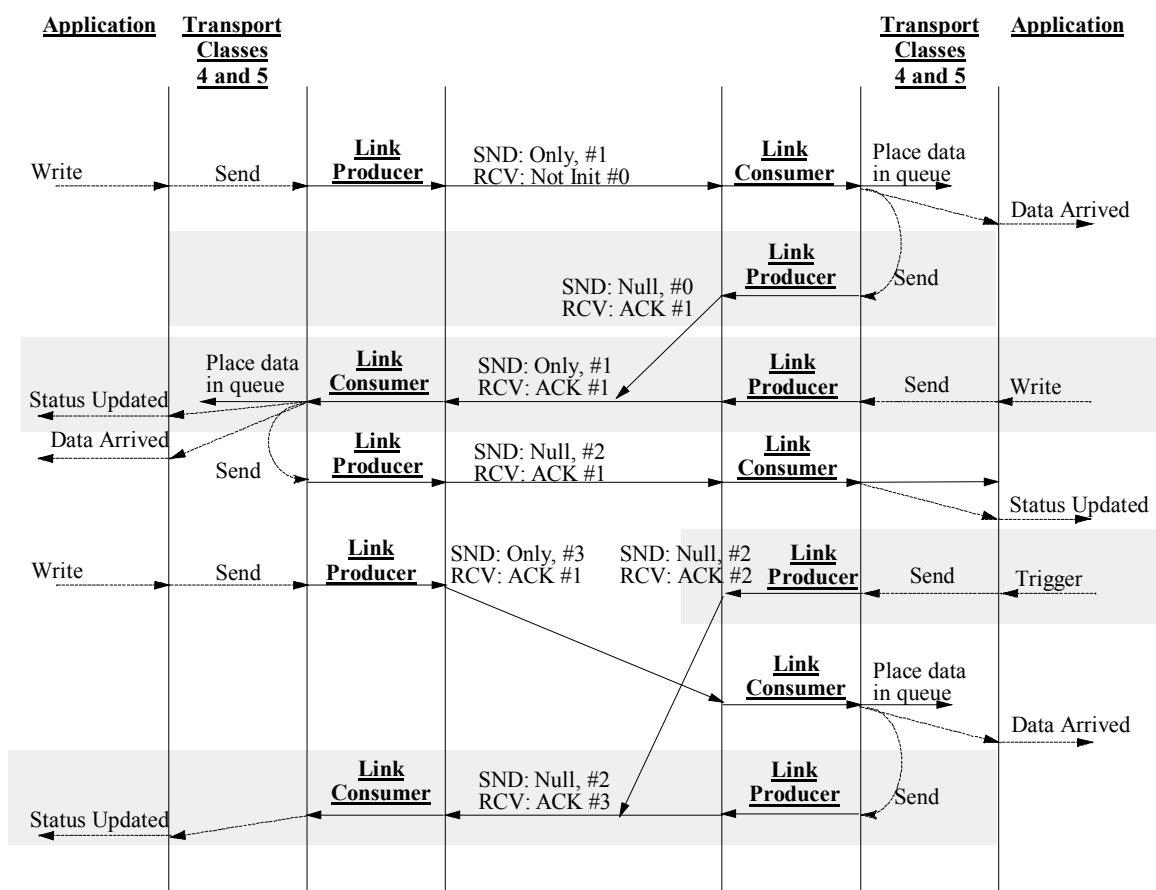


Figure 107 – Sequence diagram of response overwriting ACK of null

5.6.3.12.4.6 Typical three fragment sequence

Figure 108 shows a typical sequence where the application data is too large to fit into one TPDU. In the example shown, the application data is split into three fragments. In general, there could be many *Middle* TPDU. Each *Middle* TPDU increments the Sequence Number in the Sender Header. The *Last* TPDU shall also increment the Sequence Number.

At the receiver, the *First* allocates the memory (and supplies the data for the first fragment) and the *Last* indicates it is time to notify the receiving application that there is some data for it to process (as well as supplying the data for the last fragment).

The unshaded areas in the figures indicate data transmission in one direction, and the shaded areas indicate data transmission in the other direction.

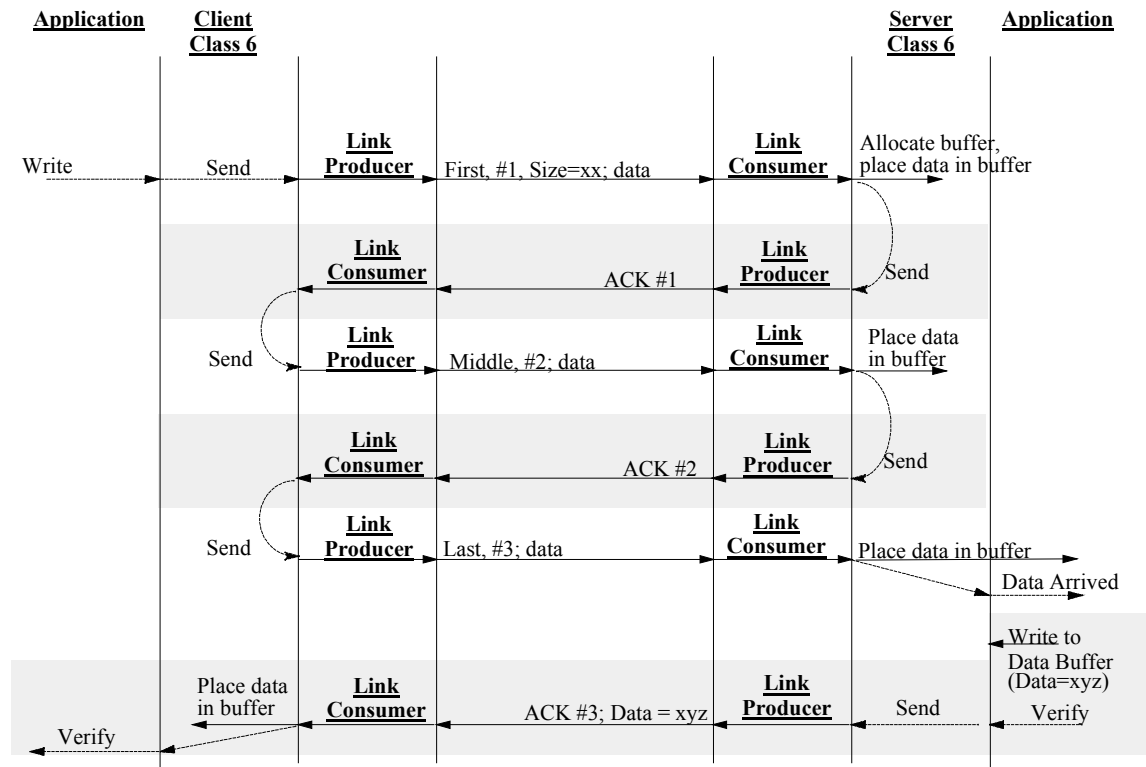


Figure 108 – Sequence diagram of three fragments using transport class 6

5.6.3.12.4.7 Typical fragmented sequence with retries

Figure 109 shows the processing when a transmission error causes something to be lost. In this example, two transmissions are lost, one in each direction. The processing is very similar to Class 5.

The unshaded areas in the figures indicate data transmission in one direction, and the shaded areas indicate data transmission in the other direction.

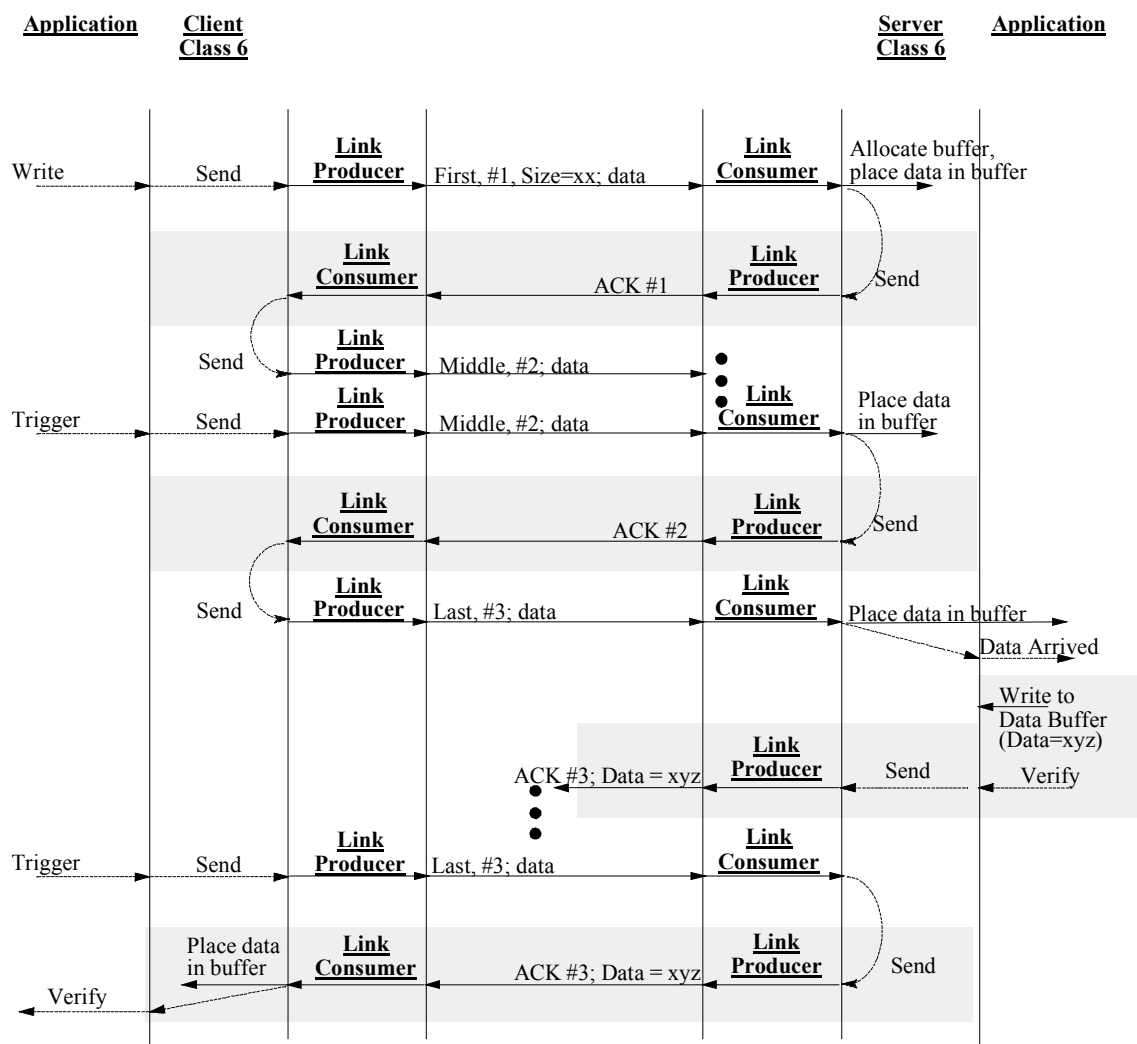


Figure 109 – Sequence diagram of fragmentation with retries using transport class 6

5.6.3.12.4.8 Typical two fragment sequence

Figure 110 shows a special case, where there are only two fragments. In this case, there is no *Middle* TPDU. Again, the *First* allocates the memory (and supplies the data for the first fragment) and the *Last* indicates it is time to notify the receiving application that there is some data for it to process (as well as supplying the data for the last fragment).

The unshaded areas in the figures indicate data transmission in one direction, and the shaded areas indicate data transmission in the other direction.

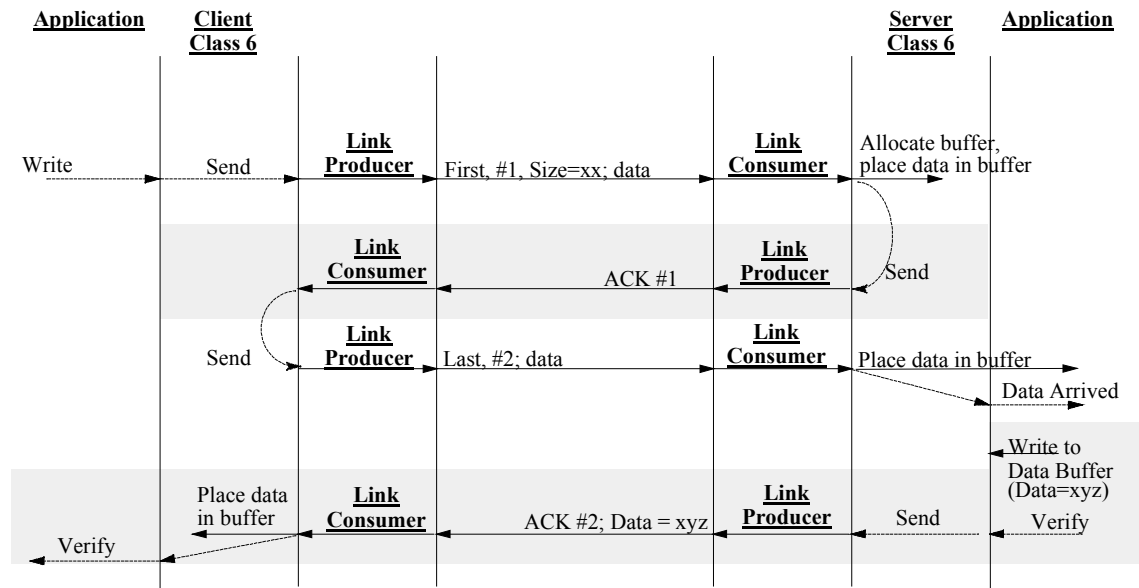


Figure 110 – Sequence diagram of two fragments using transport class 6

5.6.3.12.4.9 Typical aborted fragmented sequence

Figure 111 shows a receiver recovering when the Client has been reset in the middle of a fragmented sequence. When the Server gets a *First* or *Only*, it assumes that any message which did not receive a *Last* has been aborted.

The unshaded areas in the figures indicate data transmission in one direction, and the shaded areas indicate data transmission in the other direction.

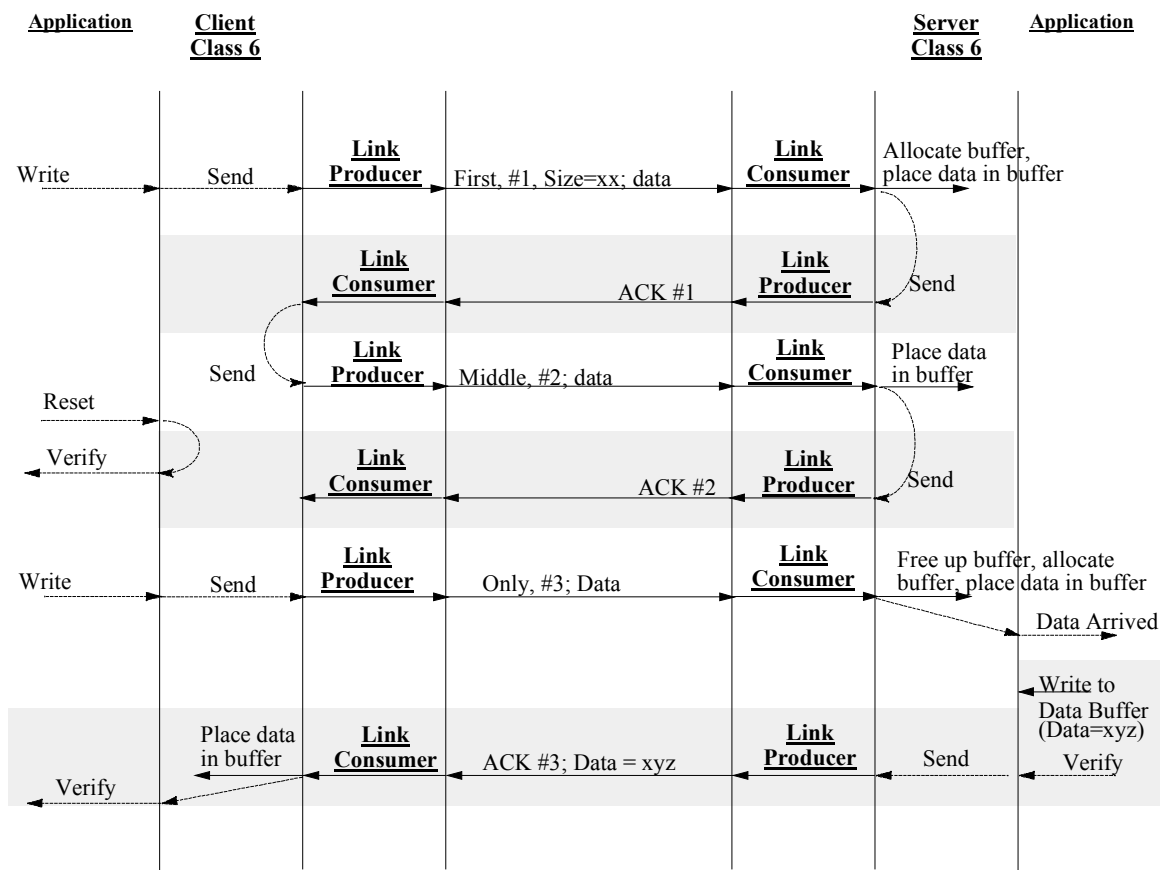


Figure 111 – Sequence diagram of aborted fragmented sequence using transport class 6

5.6.3.12.5 Transport class 6 client

5.6.3.12.5.1 Functions

The class 6 Client transport is responsible for:

- Accepting the application's *write* that it has written a data packet to the transmit data buffer;
- Writing the application data and the transport headers to the Client T-PDU transmit buffer for each packet that the application writes to it;
- Initialising the sequence count in the transport header (for the first packet transmitted) or incrementing the sequence count (for subsequent packets of the same transmission);
- Triggering the producing node of the network connection to produce the T-PDU packet on the link and send it to the consuming node of the network connection;
- Accumulating and tracking ACKs from the return connections;
- Performing retries if necessary when *triggered*;
- Notifying the application that the application data has or has not been sent.

The following internal storage is assumed:

Current sequence number (SN);
 Try counter;
 ACK accumulator (N server nodes).

5.6.3.12.5.2 States

The defined states and their descriptions of the Class 6 Transport Client are listed in Table 381 below :

Table 381 – Class 6 transport client states

State	Description
Ready	The instance of Transport Class 6 has been created and started.
Waiting	The instance of Transport Class 6 has been created and started. Application data has been written and the client is waiting for response.

NOTE These two states are sub-states of the Running state described as the common feature for classes 4 to 6.

5.6.3.12.5.3 State transitions

State transitions of the class 6 client transport is shown in Figure 112.

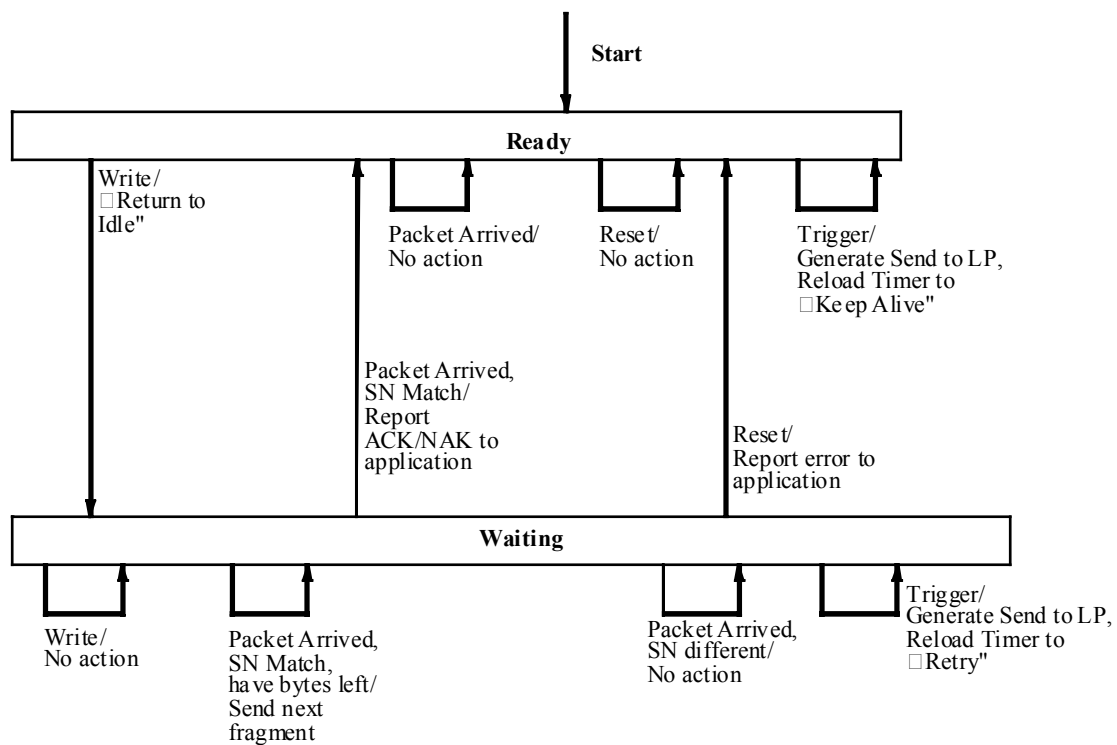


Figure 112 – Class 6 client STD

5.6.3.12.5.4 State event matrix

State transitions of the class 6 client transport is shown in Table 382.

Table 382 – Class 6 client state event matrix

Event	Command	Sequence number	State	
			Ready	Waiting
Reset	n/a	n/a	No action.	Do "Return to Ready processing" with "Reset" error status.
Write	n/a	n/a	Increment Sequence number. Test if the message fits in one fragment; if it does: Set CMD to <i>Only</i> Copy message to TPDU buffer else: Set CMD to <i>First</i> Calculate (total size / 256 octets) – 1; put into header Copy first fragment to TPDU buffer Set Try counter Load timer with time-out value Generate Send event to Link Producer Go to Waiting state	No action.
NewData	ACK	Different	(Not expecting anything). No action.	(Assume the other transport is ACKing something old) No action.
		Same		If the whole application message has been sent, then do "Return to Ready processing" with Success status. else, if the remainder fits in one fragment, then place the remainder in the Link Producer buffer, increment the sequence count, set the CMD to <i>Last</i> , and generate the Send event to the Link Producer else, place the next fragment in the Link Producer buffer, increment the sequence count, set the CMD to <i>Middle</i> , and generate the Send event to the Link Producer and stay in Waiting state.
	NAK	Different		(Assume the other transport is NAKing something old) No action.
		Same		Do "return to Ready processing" with NAK error status
	Other	Don't care		(Assume other end is confused.) No action.
Trigger (Retry or Keep alive timer)	Don't care	Don't care	Generate Send event to Link Producer (which re-sends the <i>Null</i> message to keep connection alive), reload timer with "keep alive" value.	Decrement Try counter, if zero: Do "return to Ready processing" with "Can't send" error status. else: Generate Send event to Link Producer (which re-sends the previous message), reload timer with retry value.

The "Return to Ready Processing" includes the following:

- Free the application data buffer;
- Generate *Verify* event to application with appropriate status;
- Increment Sequence number;
- Set CMD to *Null*;

- Generate the Send event to the Link Producer and reload the timer with the Keep Alive value;
- Go to Ready state.

5.6.3.12.6 Transport class 6 server

5.6.3.12.6.1 Functions

In the link to application direction, a Class 6 Server transport is responsible for:

- Accepting the notification from the consuming node of the network connection that it has written a data packet to the server-side T-PDU receive buffer;
- Locating and reading the transport headers of each packet that the consuming node of the network connection writes to the server-side T-PDU receive buffer;
- Performing the actions specified in the state event matrix for each packet which arrives;
- Notifying the server application when it places new data into the received data buffer.

The Server Bi-directional Point-to-Point Non-Blocking transport has four states:

Initial;
Ready;
Assembling;
Waiting (for application verification).

The following internal storage is assumed:

Previous Sequence Number (SN);
Previous Command (CMD).

5.6.3.12.6.2 States

The defined states and their descriptions of the Class 6 Transport Server are listed in Table 383 below :

Table 383 – Class 6 transport server states

State	Description
Initial	The instance of Transport Class 6 has been created and started.
Assembling	The instance of Transport Class 6 has been created and started. First packet of segmented application data has arrived.
Waiting	The instance of Transport Class 6 has been created and started. Subsequent packet(s) of segmented application data has(ve) arrived.
Ready	The instance of Transport Class 6 has been created and started. Last packet of segmented application data has arrived.

NOTE These four states are sub-states of the Running state described as the common feature for classes 4 to 6.

5.6.3.12.6.3 State transitions

State transitions of the class 6 server transport are shown in Figure 113.

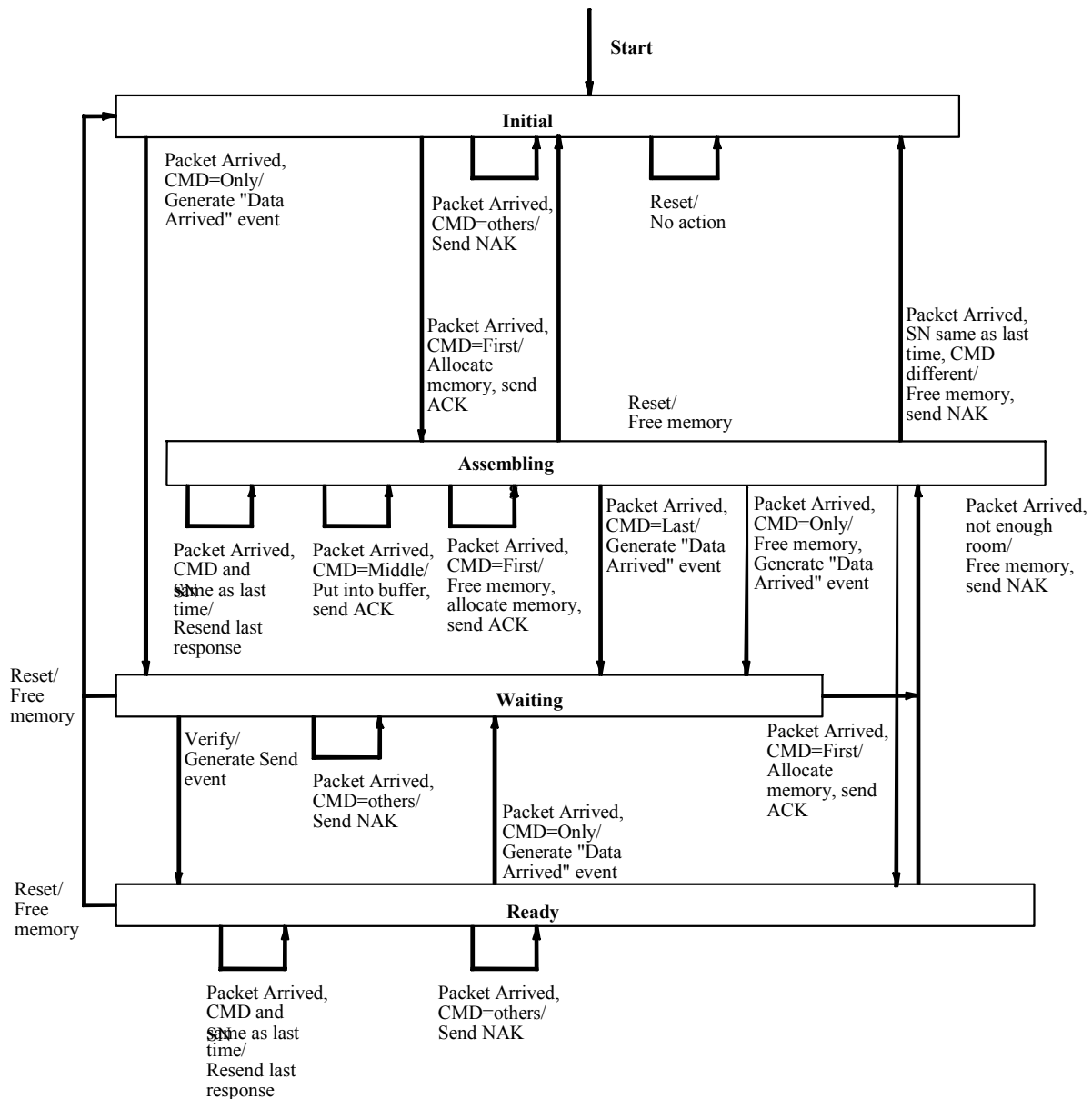


Figure 113 – Class 6 server STD

The class 6 server SEM is shown in Table 384.

The following events can happen.

- *Reset* Event;
- *Verify* Event;
- Packet Arrived Event, CMD= Null;
- Packet Arrived Event, CMD= Only
- Packet Arrived Event, CMD= First;
- Packet Arrived Event, CMD= Middle;
- Packet Arrived Event, CMD= Last;
- *Packet Arrived* Event, CMD= Illegal command.

NOTE There is **no** timer event used by the Server state machine. If there is a communications problem, the Client state machine should time out. The application should have a timer to indicate problems with the other application.

The following internal storage is assumed:

Previous SN;
Previous CMD received;
Buffer Pointer;
Room Left.

The following general guiding principles were used to develop the particular actions and transitions:

- Act like class 3 when possible;
- A *Null* shall not change state;
- When a *First* or *Only* is received when in the *Assembling* or *Waiting* states, free up the pending message and act like the server was in the *Ready* state;
- When a sequence error is detected, free up the resources and go to the *Ready* state (or stay in the *Initial* state);
- It is important to NOT change the CMD when responding to a retry. It might have been NAKed the last time; it shall be NAKed on a retry also.

5.6.3.12.6.4 State event matrix

State transitions of the class 6 server transport are shown in Table 384.

Table 384 – Class 6 server SEM

Event	State			
	Initial	Ready	Assembling	Waiting
Reset	No action.	Go to Initial.	Free memory previously allocated. Go to Initial.	Free memory previously allocated. Go to Initial.
Verify	Ignore.			Generate Send event, go to Ready.
Packet Arrived, CMD=Null, SN different	Copy SN to <i>From_Recv</i> , set CMD to ACK, generate Send event, go to Ready.	Copy SN to <i>From_Recv</i> , set CMD to ACK, generate Send event, stay in Ready.	Copy SN to <i>From_Recv</i> , set CMD to ACK, generate Send event, stay in Assembling.	Free memory previously allocated. Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), generate Send event, go to Ready. (If we ACK the <i>Null</i> , there is no way to ACK the original request. If we don't ACK the <i>Null</i> , the client shall assume we died.)
Packet Arrived, CMD=Null, SN same		If previous CMD was <i>Null</i> , then: (Assume other transport still has nothing for us to do, assume SN and CMD are OK.) Generate Send event, stay in Ready. else: (Assume other transport made an error by sending a different CMD, but used the same SN). Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), generate Send event, stay in Ready.	If previous CMD was <i>Null</i> , then: (Assume other transport still has nothing for us to do, assume SN and CMD are OK.) Generate Send event, stay in Assembling. else: (Assume other transport made an error by sending a different CMD, but used the same SN). Free memory previously allocated. Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), generate Send event, go to Ready.	
Packet Arrived, CMD=Only, SN different	Allocate buffer; if buffer allocation succeeds: Copy data to buffer, put on Received queue, generate Data Arrived event, copy SN to <i>From_Recv</i> , set CMD to ACK, go to Waiting. else: Copy SN to <i>From_Recv</i> , set CMD to NAK (no memory), generate Send event, go to Ready.		Free memory previously allocated. Allocate buffer; if buffer allocation succeeds: Copy data to buffer, put on Received queue, generate Data Arrived event, copy SN to <i>From_Recv</i> , set CMD to ACK, go to Waiting. else: Copy SN to <i>From_Recv</i> , set CMD to NAK (no memory), generate Send event, go to Ready.	

Table 384 (continued)

Event	State			
	Initial	Ready	Assembling	Waiting
Packet Arrived, CMD=Only, SN same	(Since this state does not know the previous Sequence Number, perform action as if the Sequence Number were different.)	If previous CMD was <i>Only</i> , then: Generate Send event, stay in Ready. (This is a retry, assume SN and CMD are OK. Since we are in Ready, and not Waiting, the application has verified it.) else: Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), generate Send event, stay in Ready.	Free memory previously allocated. Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), generate Send event, go to Ready. (NOTE Shall not be in this state if previous CMD was <i>Only</i> .)	If previous CMD was <i>Only</i> , then: Generate Duplicate Arrived event, stay in Waiting. (This is a retry, assume SN and CMD are OK.) else: Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), generate Send event, go to Ready.
Packet Arrived, CMD=First, SN different	Allocate buffer; if buffer allocation succeeds: Copy data to buffer, copy SN to <i>From_Recv</i> , initialise Buffer Pointer and Room Left, set CMD to ACK, generate Send event, go to Assembling state. else: Copy SN to <i>From_Recv</i> , set CMD to NAK (no memory), generate Send event, go to Ready.		Free memory previously allocated. Allocate buffer; if buffer allocation succeeds: Copy data to buffer, copy SN to <i>From_Recv</i> , initialise Buffer Pointer and Room Left, set CMD to ACK, generate Send event, go to Assembling. else: Copy SN to <i>From_Recv</i> , set CMD to NAK (no memory), generate Send event, go to Ready.	
Packet Arrived, CMD=First, SN same	(Since this state does not know the previous Sequence Number, perform action as if the Sequence Number were different.)	Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), generate Send event, stay in Ready. NOTE Shall not be in this state if previous CMD was <i>First</i> .	If previous CMD was <i>First</i> , then: Generate Send event, stay in Assembling. (This is a retry, assume SN and CMD are OK.) else: Free memory previously allocated. Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), generate Send event, go to Ready. (The Client changed the CMD but not the SN.)	Free memory previously allocated. Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), generate Send event, go to Ready. NOTE Shall not be in this state if previous CMD was <i>First</i> .

Table 384 (continued)

Event	State			
	Initial	Ready	Assembling	Waiting
Packet Arrived, CMD=Middle, SN different	Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), generate Send event, stay in Initial.	Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), generate Send event, stay in Ready.	If size of data is less than Room Left, then: Copy data to buffer at Buffer Pointer, copy SN to <i>From_Recv</i> , update Buffer Pointer and Room Left, set CMD to ACK, generate Send event, stay in Assembling. else: Free memory previously allocated. Copy SN to <i>From_Recv</i> , set CMD to NAK (no memory), generate Send event, go to Ready.	Free memory previously allocated. Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), generate Send event, go to Ready.
Packet Arrived, CMD=Middle, SN same			If previous CMD was <i>Middle</i> , then: Generate Send event, stay in Assembling. (This is a retry, assume SN and CMD are OK.) else: Free memory previously allocated. Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), generate Send event, go to Ready.	
Packet Arrived, CMD=Last, SN different	Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), generate Send event, stay in Initial.	Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), generate Send event, stay in Ready.	If size of data is less than Room Left, then: Copy data to buffer at Buffer Pointer, put buffer on Received queue, generate Data Arrived event, copy SN to <i>From_Recv</i> , set CMD to ACK, generate Send event, go to Ready. else: Free memory previously allocated. Copy SN to <i>From_Recv</i> , set CMD to NAK (no memory), generate Send event, go to Ready.	Free memory previously allocated. Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), generate Send event, go to Ready.

Table 384 (continued)

Event	State			
	Initial	Ready	Assembling	Waiting
Packet Arrived, CMD=Last, SN same		If previous CMD was <i>Last</i> , then: Generate Send event, stay in Ready. (This is a retry, assume SN and CMD are OK. Since we are in Ready, and not Waiting, the application has verified it.) else: Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), generate Send event, stay in Ready.	Free memory previously allocated. Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), generate Send event, go to Ready. (NOTE This shall not happen: If the previous CMD is <i>Last</i> , then we shall be in the Waiting or Ready state.)	If previous CMD was <i>Last</i> , then: Generate Duplicate Arrived event, stay in Waiting. (This is a retry, assume SN and CMD are OK.) else: Free memory previously allocated. Copy SN to <i>From_Recv</i> , set CMD to NAK (sequence error), generate Send event, go to Ready.
Packet Arrived, CMD=Illegal	Copy SN to <i>From_Recv</i> , set CMD to NAK (illegal command), generate Send event, stay in Initial.	Copy SN to <i>From_Recv</i> , set CMD to NAK (illegal command), generate Send event, stay in Ready.	Copy SN to <i>From_Recv</i> , set CMD to NAK (illegal command), generate Send event, stay in Assembling.	Copy SN to <i>From_Recv</i> , set CMD to NAK (illegal command), generate Send event, stay in Waiting.

5.7 DLL Mapping Protocol Machine (DMPM)

5.7.1 General

The DLL Mapping Protocol Machine is common to all the AREP types.

The network connections that interface to the Data Link layer include the link producer(s) and link consumer(s).

Data flow of the link producer and consumer on an instance of Transport connection is shown in Figure 114.

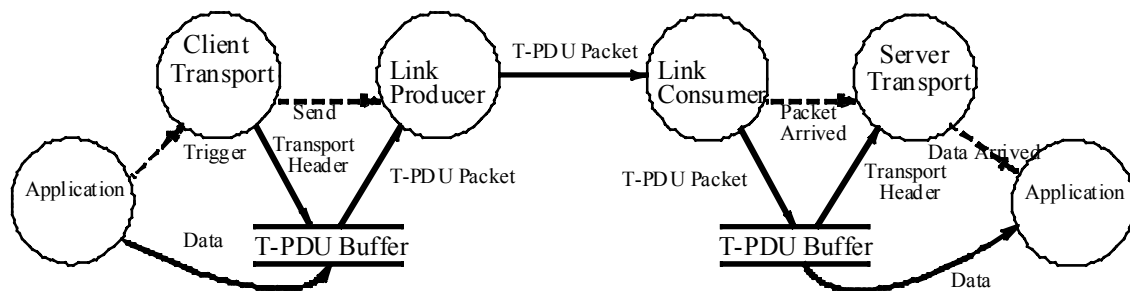


Figure 114 – Data flow diagram for a link producer and consumer

5.7.2 Link producer

The link producer shall be responsible for fetching data from the TPDU buffer and producing it on the data link after the *send* event has been received. TPDU buffer management schemes such as overwrite, double buffered, triple buffered are not excluded, but since they are implementation specific, they are not specified here.

The link producer shall read the TPDU packet from the TPDU buffer and transmit it unchanged. While the TPDU buffer is shown as a single buffer, it does not imply that the transport header and the data are stored in consecutive locations. An implementation that stored the transport header and data in non-consecutive locations is permitted as long as the link producer can find all parts of the TPDU packet.

The application, not the link producer, shall be responsible for ensuring that the TPDU buffer has been initialised before the first trigger has been issued.

5.7.3 Link consumer

The link consumer is responsible for receiving TPDU packets, storing them in the TPDU buffer and notifying the server transport class that a packet has arrived. Data buffer management schemes such as overwrite, double buffered, triple buffered are not excluded, but since they are implementation specific, they are not specified here.

The link consumer shall write the TPDU packet unchanged to the TPDU buffer. While the TPDU buffer is shown as a single buffer, it does not imply that the transport header and the data are stored in consecutive locations. There are no restrictions on actual implementations. An implementation that stored the transport header and data in non-consecutive locations is permitted as long as the consumer can find both parts of the TPDU packet.

5.7.4 Primitive definitions

5.7.4.1 Primitives exchanged between DMPM and ARPM

List of primitives exchanged between DMPM and ARPM is shown in Table 385 and Table 386.

Table 385 – Primitives issued by ARPM to DMPM

Primitive names	Source	Associated parameters	Functions
Send.req	ARPM	Service Dest-id TPDU	This primitive is used to request the DMPM to transfer the TPDU to the Data Link. It passes the TPDU to the DMPM as a DLSDU.

Table 386 – Primitives issued by DMPM to ARPM

Primitive names	Source	Associated parameters	Functions
Packet_Arrived.ind	DMPM	Service Source-id TPDU	This primitive is used to pass the TPDU received as a Data Link Layer service data unit to a designated ARPM.

5.7.4.2 Parameters of ARPM/DMPM primitives

The parameters used with the primitives exchanged between the ARPM and the DMPM are described in Table 387.

Table 387 – Parameters used with primitives exchanged between ARPM and DMPM

Parameter name	Description
TPDU	This parameter carries the transport PDU containing the user data and Transport header.
DL-SDU	This parameter carries the received Data Link SDU.

5.7.4.3 Primitives exchanged between Data Link Layer and DMPM

The request and confirmation services used to queue Lpackets with the Data Link Layer shall be of the form:

NOTE The primitives shown in Table 388 and their parameters are defined in detail in Type 2 clauses of IEC 61158-3.

Table 388 – Primitives exchanged between Data Link Layer and DMPM

Primitive names	Source	Associated parameters
DL_generic_Tag.req	ARPM	Request_DLS_user_id (handle) user_data [] QoS (priority) generic_TAG
DL_generic_Tag.cnf	Data Link layer	Request_DLS_user_id (handle) TX_status
DL_generic_Tag.ind	Data Link layer	Request_DLS_user_data [] generic_TAG
DL_fixed_Tag.req	ARPM	Request_DLS_user_id (handle) user_data [] QoS (priority) fixed_TAG destination-DLE-ID
DL_fixed_Tag.cnf	Data Link layer	Request_DLS_user_id (handle) TX_status
DL_fixed_Tag.ind	Data Link layer	user_data [] fixed_TAG source-DLE-ID
DL_tone.ind	Data Link layer	DLS_cycle

5.7.4.4 Parameters of DMPM/Data Link Layer primitives

The parameters used with the primitives exchanged between the DMPM and the Data Link Layer are identical to those defined in the Type 2 clauses of Data Link Layer Service Definition (IEC 61158-3).

The parameters are described in Table 389 .

Table 389 – Parameters used with primitives exchanged between DMPM and Data Link

Parameter name	Description
Request_DLS_user_id	Provides a local means of pairing the resulting confirm primitive with the causative request primitive.
User_data	Data to be transmitted between DLS-users without alteration by the DLS-provider.
QoS	Selects one of the following priorities : scheduled high low
Generic_TAG	Connection identification or service point address identifying the remote DLSAP(s) to which the DLS is to be provided.
Fixed_TAG	Specifies the destination service point in the station identified by the DLS- Destination-DLE-ID address.
Destination-DLE-ID	Address identifying the destination station for the message using its MAC ID address.
Source-DLE-ID	Address identifying the local station from which the fixed tag DLSDU has been sent. It is a station MAC ID address on the local link.
TX_status	Status of the requested transmission : a) "OK" message successfully sent; b) "TxAbort" sending process failed; c) "Flushed" message has been removed from the pending queue before sending.
DLS_cycle	Interval count for the Network Update Time (NUT) which has just been received within the overall cycle of scheduled access intervals.

5.7.5 DMPM state machine

5.7.5.1 DMPM states

5.7.5.1.1 Link producer

5.7.5.1.1.1 Link producer states

The defined states and their descriptions of the Link Producer are listed Table 390 below :

Table 390 – Link producer states

State	Description
Non-existent	The Link Producer does not exist.
Running	The Link Producer has been created and is running.

5.7.5.1.1.2 State transition diagram

Figure 115 shows the state transition diagram of a link producer.

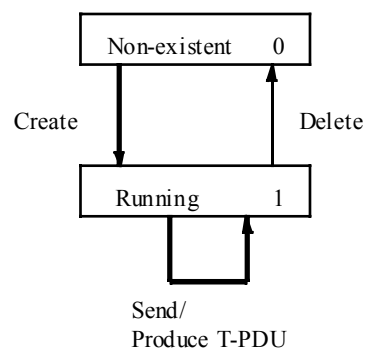


Figure 115 – State transition diagram for a link producer

5.7.5.1.1.3 State event matrix

State event matrix of the Link Producer is shown in Table 391.

Table 391 – State event matrix of link producer

Event	State	
	Non-existent	Running
Create	Transition to Running	Error
Delete	Error	Transition to Non-existent
Send	Error	1) Produce TPDU

5.7.5.1.2 Link consumer

5.7.5.1.2.1 Link consumer states

The defined states and their descriptions of the Link Consumer are listed in Table 392 below :

Table 392 – Link consumer states

State	Description
Non-existent	The Link Producer does not exist.
Running	The Link Producer has been created and is running.

5.7.5.1.2.2 State transition diagram

Figure 116 shows the state transition diagram of a link consumer.

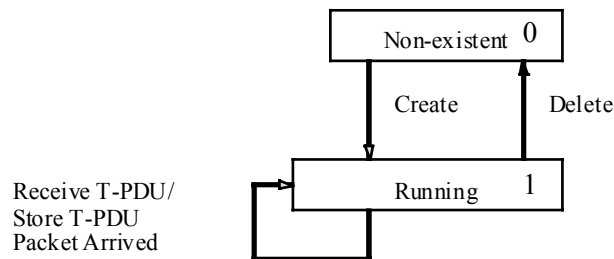


Figure 116 – State transition diagram for a link consumer

5.7.5.1.2.3 State event matrix

State event matrix of the Link Producer is shown in Table 393.

Table 393 – State event matrix of link consumer

Event	State	
	Non-existent	Running
Create	Transition to Running	Error
Delete	Error	Transition to Non-existent
Receive TPDU	Error	Store TPDU Packet Arrived

5.7.5.2 Functions used by DMPM

This fieldbus does not any specified functions.

5.7.6 Data Link Layer service selection

This fieldbus supports only the Data Link layer service whose primitives are noted in 5.7.4.3.

5.8 Alternate DLL mapping protocol machine

5.8.1 General

This subclause defines the requirements for mapping the transmission of TPDU's over an Ethernet-TCP/UDP/IP-based network, instead of the Type 2 Data Link Layer, using the TCP/UDP/IP protocol suite and the encapsulation protocol defined in 5.1.3.1.

5.8.2 Mapping of UCMM PDUs

5.8.2.1 General

UCMM PDUs shall be transmitted over a TCP/IP connection, using the encapsulation protocol defined in 5.8.5 and 5.1.3.1 (SendRRData command).

5.8.2.2 Common requirements for Connection Manager PDU's

This subclause contains the specific requirements for Connection Manager parameters in case of TCP/IP encapsulation (Connection Manager parameters are fully described in 5.1.1.5.2).

- The MULTIPOINT connection type shall be supported only for class 0 and class 1 transports.
- Class 0 and class 1 connections that use CHANGE_OF_STATE triggering shall use the production inhibit time segment (see 5.1.1.9.5).
- The connection size shall be no larger than 65511 octets.
- To reliably establish a connection that extends onto a TCP/IP link, the connection request time-out shall be large enough to allow the connection to be established, which could involve resolving a host name, or going through multiple gateways. Because of the large variation in connection request processing over TCP/IP, bridges in the connection path shall not subtract anything from the connection request timeout.
- The link address portion of a TCP/IP connection path segment shall be encoded within a port segment as specified in 5.1.1.9.3.
- For TCP/IP connections, the Network Connection ID shall be a 32-bit identifier meaningful to the device that chooses it. The Network Connection ID shall not be subdivided into any specific fields. The Network Connection ID shall not be reused until the connection has been closed or has timed out. When a device restarts, it shall not reuse Network Connection IDs from previously opened connections until those connections have been closed or have timed out.

5.8.2.3 Forward_open PDU for class 2 and class 3 connections

The Forward_open PDU for class 2 and class 3 connections shall be sent over a TCP connection using the SendRRData command defined in 5.1.3.1.

5.8.2.4 Forward_open for class 0 and class 1 connections

5.8.2.4.1 General

The Forward_open PDU for class 0 and class 1 connections shall be sent over a TCP connection using the SendRRData command defined in 5.1.3.1. The producer and consumer shall exchange the port numbers and IP multicast address (for multipoint connections) necessary to send the connected data, according to the following:

- For multipoint connections, the producer shall choose an IP multicast address to which to send the connected data. The IP multicast address shall be returned to the consumer in the Forward_open_reply via the Sockaddr Info item defined in 5.1.3.1. The Sockaddr Info item shall be placed after the Fwd_open_reply data in the encapsulation packet. The port number shall be the registered UDP port number (0x08ae) assigned by the IANA.
- For point-to-point connections, the consumer shall choose a UDP port number to which the connected data shall be sent. The port number shall be encoded in the Sockaddr Info item and shall be returned in the Forward_open or Forward_open_reply (depending on whether the consumer is the target or originator). The port number may be the registered port number (0x08ae) or may be a port number chosen by the consumer.
- If the Sockaddr Info items are not present, or are in error, a Forward_open_reply shall be returned with status code 0x01 and extended status 0x205.

NOTE After receiving the Forward_open_reply for a multipoint connection, the consuming Ethernet devices should join the desired IP Multicast Group in order to receive the IP Multicast datagrams. The exact method for doing this depends on the TCP/IP application programming interface in use on the device.

5.8.2.4.2 Mapping connections to IP multicast addresses

NOTE It is recommended, though not required, that producers use a unique IP multicast address for each active multipoint connection. Depending upon the implementation, this can reduce the amount of connection screening on the part of the consumer. It also allows the consumer to more evenly service incoming connected data from multiple connections.

Since a unique IP multicast addresses per multipoint connection is not required, consumers shall be able to handle the situation in which packets from multiple multipoint connections are being sent to the same IP multicast address. Consumers shall be able to screen the incoming packets based on the Connection ID (see 5.8.3.3 for screening requirements).

5.8.2.4.3 IP multicast scoping and address allocation

5.8.2.4.3.1 General

The following subclauses addresses two issues that shall be considered when implementing multipoint connections on Ethernet :

- IP multicast scoping refers to the practice of limiting how widely a given multicast datagram is propagated across the network
- IP multicast address allocation refers to the problem of how applications obtain IP multicast addresses that they may then use to send and receive multicast datagrams.

NOTE Once the IP multicast allocation architecture becomes standard and is deployed, requirements detailed in the two following subclauses may be updated as appropriate.

5.8.2.4.3.2 Scoping

Devices that send class 0 or class 1 multicast data shall use TTL scoping, and shall limit the connected datagrams to a single subnet. This is the safest approach as it prevents unwanted multicast traffic from affecting other subnets. With this approach, if a device wishes to connect to a device on a different subnet, then a point-to-point connection shall be used.

NOTE TTL scoping refers to the practice of using the TTL field in the IP header to confine multicast traffic. When sending IP multicast traffic, a host can set the TTL field in the IP header to an appropriate value based on how widely the packet should be propagated across the network. As the packet is routed across the network, the TTL field is decremented at each hop. Routers can be configured with TTL thresholds such that they will not forward a packet unless the remaining TTL is greater than the threshold. A multicast datagram with an initial TTL of one (1) is effectively limited to a single subnet.

5.8.2.4.3.3 Address allocation

IP multicast addresses used for multipoint connections shall be chosen from the multicast address space set aside by the Internet Assigned Numbers Authority (IANA), in the IPv4 Organization Local Scope: 239.192.0.0/14

NOTE The high-order /24 is reserved for relative assignments

Each device capable of producing multicast data shall choose a set of IP multicast addresses to use for multipoint connections. One of the following two mechanisms shall be used for choosing IP multicast addresses:

1. The user may explicitly configure the device with a set of IP multicast addresses to use. The actual configuration mechanism is product-specific
2. Devices may automatically choose a set of multicast addresses to use, based on their IP address, according to the following algorithm:

Each host is allowed to use a block of 32 multicast addresses, based on the host's IP address. If a subnet mask is in use, the subnet mask is applied to the IP address to obtain the relative host number: host 1 uses 239.192.1.0 through 239.192.1.31; host 2 uses 239.192.1.32 through 239.192.1.63, and so on. (Remember that the first 256 addresses in the scoped region are reserved for relative assignments, as noted above.)

To ensure that the IP multicast addresses are within the Organization Local Scope, if the subnet mask results in more than 10 bits of host address, only the low 10 bits are used. If no subnet mask is in use, then the HOST ID portion of the IP address is used. For Class A & B IP addresses only the low-order 10 bits of the HOST ID portion of the IP address are used, making it possible that different devices could produce data on the same multicast addresses. Receiving devices shall be tolerant of this (see below).

Since there is no mechanism to prevent multiple devices from using the same IP multicast addresses. Consequently, devices that receive connected multicast data shall screen the connected data based on the Connection ID and the IP address of the sending device, as described in 5.8.3.3.

5.8.3 Mapping of transport class 0 and class 1 PDUs

5.8.3.1 UDP datagrams

PDUs for transport class 0 and class 1 connections shall be encapsulated in UDP datagrams using the Common Packet Format defined in 5.1.3.3. PDUs for multipoint connections shall be transmitted using IP multicast.

The data portion of the UDP datagram for class 0 and class 1 PDUs shall be as shown in Table 394.

Table 394 – UDP data format for class 0 and class 1

Field name	Type	Value
Item Count	UINT	2
Type ID	UINT	0x8002 (Sequenced Address Type)
Length	UINT	8
Address Data	UDINT	Connection ID (from Forward_open reply)
	UDINT	Sequence Number
Type ID	UINT	0x00B1 (Connected Data Type)
Length	UINT	Number of octets in PDU to follow
Data		class 0 or class 1 PDU

5.8.3.2 Class 0 and class 1 packet ordering

NOTE 1 By definition class 0 and class 1 transports do not detect out-of-order packets. For class 0, every packet is considered to be new data. For class 1, only duplicate data is detected. If a new packet arrives and the sequence number in the transport PDU is different from the previous packet, then the new packet is considered to be new data even if the new packet has a sequence number that is less than the previous sequence number.

NOTE 2 When using UDP to transport Class 0 and class 1 connected data, there is no guarantee that packets arrive in the same order that they were sent. When both sender and receiver are on the same subnet, packets typically arrive in order. However, when going through routers, when there are multiple paths that a packet could take, it is possible for packets to arrive out of order.

For class 0 and class 1 connections over Ethernet, devices shall maintain a sequence number in the data portion of the UDP datagram defined in 5.8.3.1. The sequence number shall be maintained per connection. Each time an Ethernet device sends a Class 1 packet, it shall increment the sequence number for that connection. If the receiving Ethernet device receives a packet whose sequence number is less than the previously received packet, the packet with the smaller sequence number shall be discarded. Duplicate packets shall be accepted and given to the transport layer.

The sequence number shall be operated on with modular arithmetic to deal with sequence rollover.

NOTE 3 Dealing with 32-bit sequence numbers is described in RFC 0793 (the TCP definition).

5.8.3.3 Screening incoming connected data

Ethernet devices that receive class 0 and class 1 connected data shall screen incoming packets based on the Connection ID and IP address of the sending device. This is necessary for the following reasons:

- For multipoint connections, there is no guaranteed mechanism to prevent multiple devices from using the same IP multicast address. Consequently, a device could receive (bogus) multicast connected data from a device with which it has not established a connection.
- For multicast connections, a device is allowed to use the same IP multicast address for multiple class 0 and class 1 multipoint connections.
- To prevent Connection ID conflicts.

When a class 0 or class 1 connection is established, the target and originating Ethernet devices shall record the Connection ID on which they will receive connected data, coupled with the IP address of the device at the other end of the connection. When a device receives connected data, it shall confirm that the Connection ID is valid for the IP address of the sending device. If not, the packet shall be discarded.

5.8.4 Mapping of transport class 2 and class 3 PDU's

PDUs for transport class 2 and class 3 connections shall be transmitted over a TCP connection using the encapsulation protocol defined in 5.8.5 and 5.1.3.1 (SendUnitData command).

Multiple connections may be sent over a single TCP connection. An implementation need not support a specific number of connections per TCP connection. An implementation may impose an upper bound if it chooses.

Because of the full-duplex nature of TCP, the O2T and T2O link connections shall use the same TCP connection. However if a target subsequently originates a connection, then it shall be considered an originator, and a different TCP connection shall be used.

NOTE This standard defines no requirements for management of the TCP connection, such as inactivity timeouts, or closing the TCP connection when all native connections are closed. However, implementations are free to implement these.

5.8.5 Management of an encapsulation session

5.8.5.1 Phases of an encapsulation session

An encapsulation session shall have three phases:

- establishing a session;
- maintaining a session;
- closing a session.

5.8.5.2 Establishing a session

Session establishment shall proceed according to the following steps:

- The originator shall open a TCP/IP connection to the target, using the reserved port number (0xAF12), or if specified, the port number from the connection path;
- The originator shall send a RegisterSession command to the target;
- The target shall check the protocol version in the command message to verify it supports the same protocol version as the originator. If not, the target shall return a RegisterSession with an appropriate Status field along with the highest supported protocol version (see description of RegisterSession command).
- The target shall assign a new (unique) Session ID and shall send a RegisterSession reply to the originator

NOTE A TCP port number, 0xAF12, has been reserved with the Internet Assigned Numbers Authority (IANA) for use by the encapsulation protocol.

5.8.5.3 Terminating a session

Either the originator or the target may terminate the session. Sessions shall be terminated in either of two ways:

- The originator or target shall close the underlying TCP connection. The corresponding target or originator shall detect the loss of the TCP connection, and shall close its side of the connection;

- The originator or target shall send an UnRegisterSession command and shall wait to detect the closing of the TCP connection. The corresponding target or originator shall then close its side of the TCP connection. The sender of the UnRegisterSession shall detect the loss of the TCP connection, then it shall close its side of the connection.

NOTE The second method is preferred since it results in more timely clean up of the TCP connection.

5.8.5.4 Maintaining a session

Once a session is established, it shall remain established until one of the following occurs:

- the originator or target closes the TCP connection;
- the originator or target issues the UnRegisterSession command;
- the TCP connection is broken.

6 Type 3

6.1 FAL syntax description

6.1.1 APDU abstract syntax

Table 395 defines the abstract syntax of the DP Application Layer PDUs referred to as APDUs. The defined order of octets shall be used to convey the APDUs.

Table 395 – APDU syntax

APDU name	APDU structure
DataExchange-REQ-PDU	[Outp_Data*]
DataExchange-RES-PDU	[Inp_Data*]
RD_Input-RES-PDU	[Inp_Data*]
RD_Output-RES-PDU	[Outp_Data*]
Chk_Cfg-REQ-PDU	{[Identifier_Format*], [Special_Identifier_Format*]^ [Extended_Special_Identifier_Format*]}
Get_Cfg-RES-PDU	{[Identifier_Format*], [Special_Identifier_Format*]^ [Extended_Special_Identifier_Format*]}
Set_Prm-REQ-PDU	Station_status, WD_Fact_1, WD_Fact_2, min_TSDR, Ident_Number, Group_Ident, User_Prm_Data
Set_Ext_Prm-REQ-PDU	Structured_Prm_Data*
Diagnosis-RES-PDU	Station_status_1, Station_status_2, Station_status_3, Diag_Master_Add, Ident_Number, {[Device_Related_Diagnosis*]^([Alarm], [Status*],[Modul_Status],[DXB_Link_Status], { [Red_Status] ^[PrmCmdAck])}, [Identifier_Related_Diagnosis], [Channel_Related_Diagnosis*], [Revision_Number]} The field Device_Related_Diagnosis may be present if DPV1=FALSE, otherwise if DPV1=TRUE it shall not be present. In this case the fields Alarm, Status, Modul_Status may be present.
Set_Slave_Add-REQ-PDU	New_Slave_Add, Ident_Number, No_Add_Chg, [Rem_Slave_Data*]
Global_Control-REQ-PDU	Control_Command, Group_Select
Clock-Value-PDU	Clock_value_time_event, Clock_value_previous_TE, Clock_value_status1, Clock_value_status2
Initiate-REQ-PDU	Function_Num(0x57), reserved (3 Octets), Send_Timeout, Features_Supported, Profile_Features_Supported, Profile_Ident_Number, Add_Addr_Para
Initiate-RES-PDU	Function_Num(0x57), Max_Len_Data_Unit, Features_Supported, Profile_Features_Supported, Profile_Ident_Number, Add_Addr_Param
Initiate-NRS-PDU	Function_Num(0xD7), Error_Decode, Error_Code_1, Error_Code_2
Abort-REQ-PDU	Function_Num(0x58), Subnet, Instance_Reason_Code
Read-REQ-PDU	Function_Num(0x5E), Slot_Number, Index (0 .. 254), Length
Read-RES-PDU	Function_Num(0x5E), Slot_Number, Index (0 .. 254), Length, [Data*]
Read-NRS-PDU	Function_Num(0xDE), Error_Decode, Error_Code_1, Error_Code_2
Pull-NRS-PDU	
Write-REQ-PDU	Function_Num(0x5F), Slot_Number, Index (0 .. 254), Length, [Data*]
Write-RES-PDU	Function_Num(0x5F), Slot_Number, Index (0 .. 254), Length
Write-NRS-PDU Initiate_Load_NRS-PDU Push-NRS-PDU Terminate_Load-NRS-PDU Start-NRS-PDU Stop-NRS-PDU Resume-NRS-PDU Reset-NRS-PDU Call-NRS-PDU Get_FI_State-NRS-PDU	Function_Num(0xDF), Error_Decode, Error_Code_1, Error_Code_2
Alarm_Ack-REQ-PDU	Function_Num(0x5C), Slot_Number, Alarm_Type, Alarm_Specifier

APDU name	APDU structure
Alarm_Ack-RES-PDU	Function_Num(0x5C), Slot_Number, Alarm_Type, Alarm_Specifier
Alarm_Ack-NRS-PDU	Function_Num(0xDC), Error_Decode, Error_Code_1, Error_Code_2
Idle-REQ-PDU	Function_Num(0x48)
Idle-RES-PDU	Function_Num(0x48)
Data_Transport-REQ-PDU	Function_Num(0x51), Slot_Number, Index, Length, [Data*]
Data_Transport-RES-PDU	Function_Num(0x51), Slot_Number, Index, Length, [Data*]
Data_Transport-NRS-PDU	Function_Num(0xD1), Error_Decode, Error_Code_1, Error_Code_2
Initiate_Load-REQ-PDU	Function_Num(0x5F), Slot_Number, Index (255), Length, Extended_Function_Num (0x00), Options, LR_Index, LR_Length, Intersegment_Request_Timeout, [User_Specific]
Initiate_Load-RES-PDU	Function_Num(0x5E), Slot_Number, Index (255), Length, Extended_Function_Num (0x00), Max_Segment_Length, LR_Index, LR_Length, Max_Response_Delay
Push-REQ-PDU	Function_Num(0x5F), Slot_Number, Index (255), Length, Extended_Function_Num (0x01), Options, Sequence_Number, LR_Data
Pull-REQ-PDU	Function_Num(0x5E), Slot_Number, Index (255), Length
Pull-RES-PDU	Function_Num(0x5E), Slot_Number, Index (255), Length, Extended_Function_Num (0x02), Options, Sequence_Number, LR_Data
Terminate_Load-REQ-PDU	Function_Num(0x5F), Slot_Number, Index (255), Length, Extended_Function_Num (0x03), reserved (1 octet), reserved (2 Octets),
Start-REQ-PDU	Function_Num(0x5F), Slot_Number, Index (255), Length, Extended_Function_Num (0x04), reserved (1 octet), FI_Index, [Execution_Argument]
Stop-REQ-PDU	Function_Num(0x5F), Slot_Number, Index (255), Length, Extended_Function_Num (0x05), reserved (1 octet), FI_Index
Resume-REQ-PDU	Function_Num(0x5F), Slot_Number, Index (255), Length, Extended_Function_Num (0x06), reserved (1 octet), FI_Index, [Execution_Argument]
Reset-REQ-PDU	Function_Num(0x5F), Slot_Number, Index (255), Length, Extended_Function_Num (0x07), reserved (1 octet), FI_Index
Call-REQ-PDU	Function_Num(0x5F), Slot_Number, Index (255), Length, Extended_Function_Num (0x08), reserved (1 octet), FI_Index, [Execution_Argument]
Call-RES-PDU	Function_Num(0x5E), Slot_Number, Index (255), Length, Extended_Function_Num (0x08), reserved (1 octet), FI_Index, [Result_Argument]
Get_FI_State-REQ-PDU	Function_Num(0x5F), Slot_Number, Index (255), Length, Extended_Function_Num (0x09), reserved (1 octet), FI_Index
Get_FI_State-RES-PDU	Function_Num(0x5E), Slot_Number, Index (255), Length, Extended_Function_Num (0x09), reserved (1 octet), FI_Index, FI_State
Push-RES-PDU Terminate_Load-RES-PDU Start-RES-PDU Stop-RES-PDU Resume-RES-PDU Reset-RES-PDU	Function_Num(0x5F), Slot_Number, Index (255), Length
RM-REQ-PDU	Function_Num(0x56), Server_SAP, Send_Timeout
Get_Master_Diag-REQ-PDU	Function_Num(0x41), MDiag_Identifier
Get_Master_Diag-RES-PDU	Function_Num(0x41), MDiag_Identifier(=0..125), Diagnosis-RES-PDU Function_Num(0x41), MDiag_Identifier(=126), System_Diagnosis, Function_Num(0x41), MDiag_Identifier(=127), Master_Status Function_Num(0x41), MDiag_Identifier(=128), Data_Transfer_List
Start_Seq-REQ-PDU	Function_Num(0x42), Area_CodeUpDownload, Timeout
Start_Seq-RES-PDU	Function_Num(0x42), Area_CodeUpDownload, Timeout, Max_Len_Data_Unit
Download-REQ-PDU	Function_Num(0x43), Area_CodeUpDownload(=0..125), Add_Offset, DP_Slave_Parameter_Set Function_Num(0x43), Area_CodeUpDownload(=127), Add_Offset, Bus_Parameter_Set Function_Num(0x43), Area_CodeUpDownload(=129), Add_Offset, Statistic_Counters Function_Num(0x43), Area_CodeUpDownload(=136 to 139), Add_Offset, Master_Parameter_Set
Download-RES-PDU	Function_Num(0x43), Area_CodeUpDownload, Add_Offset
Upload-REQ-PDU	Function_Num(0x44), Area_CodeUpDownload, Add_Offset, Data_Len
Upload-RES-PDU	Function_Num(0x44), Area_CodeUpDownload(=0..125), Add_Offset, DP_Slave_Parameter_Set Function_Num(0x44), Area_CodeUpDownload(=127), Add_Offset, Bus_Parameter_Set Function_Num(0x44), Area_CodeUpDownload(=129), Add_Offset, Statistic_Counters Function_Num(0x43), Area_CodeUpDownload(=136 to 139), Add_Offset, Master_Parameter_Set
End_Seq-REQ-PDU	Function_Num(0x45)

APDU name	APDU structure
End_Seq-RES-PDU	Function_Num(0x45)
Act_Para_Brct-REQ-PDU	Function_Num(0x46), Area_CodeActBrct
Act_Param-REQ-PDU	Function_Num(0x47), Area_CodeAct, Activate
Act_Param-RES-PDU	Function_Num(0x47), Area_CodeAct, Activate
ZERO-PDU	Octet1(0)
NULL-PDU	
NOTE 1 If a APDU consists only of user data octets the distinction is made by use of different Data Link Layer service access points (see Protocol Machines).	
NOTE 2 A ZERO-PDU contains one octet with all bits set to zero. This APDU is used to indicate diagnosis from a device without inputs.	
NOTE 3 A NULL-PDU contains no octets. It is used to submit no DLSDU to data link layer.	

Table 396 defines structures for substitutions of elements of the APDU structures shown in Table 395.

Table 396 – Substitutions

Substitution name	Structure
Identifier_Format	Cfg_Identifier
Special_Identifier_Format	Special_Cfg_Identifier, [Length_Octet] , [Length_Octet], [Manufacturer_Specific_Data*] Length_Octet fields shall always start with fields that indicate output data if present.
Extended_Special_Identifier_Format	Special_Cfg_Identifier,[Extended_Length_Octet], [Extended_Length_Octet], [Data_Type*], [Manufacturer_Specific_Data*] Extended_Length_Octet fields shall always start with fields that indicate output data if present. The field Data_Type shall only be omitted in case of an empty slot. Otherwise it shall be present in relation to the length fields.
Device_Related_Diagnosis	Header_Octet, Diagnosis_User_Data*
Identifier_Related_Diagnosis	Header_Octet, Identifier_Diagnosis_Data_Array
Channel_Related_Diagnosis	Identifier_Number, (Channel_Number, Type_of_Diagnosis)*
Alarm	Header_Octet, Alarm_Type, Slot_Number, Alarm_Specifier, [Diagnosis_User_Data*]
Status	Header_Octet, Status_Type, Slot_Number, Status_Specifier, [Diagnosis_User_Data*]
Modul_Status	Header_Octet, Status_Type(=Modul_Status), Slot_Number(=0), Status_Specifier, Modul_Status_Array
DXB_Link_Status	Header_Octet, Status_Type(=DXB_Link_Status), Slot_Number(=0), Status_Specifier, (Publisher_Address, Publisher_Status)*
PrmCmdAck	Header_Octet, Status_Type(=PrmCmdAck), Slot_Number(=0), Status_Specifier, Function, Red_Status1, Red_Status2, Red_Status3
Red_Status	Header_Octet, Status_Type(=Red_Status), Slot_Number(=0), Status_Specifier, Function, Red_Status1, Red_Status2, Red_Status3
User_Prm_Data	[DPV1_Status_1, DPV1_Status_2, DPV1_Status_3], [User_Prm_Data_Element*]^ [Structured_Prm_Data*] The fields DPV1_Status_1, DPV1_Status_2 and DPV1_Status_3 shall be present if DPV1=TRUE. Structured_Prm_Data shall be present if the DPV1_Status_3.Prm_Structure is TRUE, otherwise it shall not be used.

Substitution name	Structure
Structured_Prm_Data	Structure_Length, Structure_Type, Slot_Number, reserved (1 octet), User_Prm_Data* Special Case DXB LinkTable: Structure_Length, Structure_Type(=3), Slot_Number(=0), reserved (1 octet), Version(=1), (Publisher_Addr, Publisher_Length, Sample_Offset, Sample_Length)* Special Case DXB-Subscribertable: Structure_Length, Structure_Type(=7), Slot_Number(=0), reserved (1 octet), Version(=1), (Publisher_Addr, Publisher_Length, Sample_Offset, Dest_Slot_Number, Offset_Data_Area, Sample_Length)* Special Case IsoM Parameter Structure_Length, Structure_Type(=4), Slot_Number(=0), reserved (1 octet), Version(=1), T _{BASE_DP} , T _{DP} , T _{MAPC} , T _{BASE_IO} , T _I , T _O , T _{DX} , T _{PLL_W} , T _{PLL_D} Special Case PrmCmd Structure_Length, Structure_Type(=2), Slot_Number(=0), Specifier, Function, Properties, Output_Hold_Time Special Case Time AR Parameter: Structure_Length, Structure_Type(=8), Slot_Number(=0), reserved (1 octet), Clock Sync Interval, [CS Delay Time]
Features_Supported	Features_Supported_1, Features_Supported_2
Profile_Features_Supported	Profile_Features_Supported_1, Profile_Features_Supported_1
Add_Addr_Para	S_Type, S_Len, D_Type, D_Len, S_API, S_SCL, [S_Network_Address], [S_MAC_Address], D_API, D_SCL, [D_Network_Address], [D_MAC_Address]
Master_Status	USIF_State, Ident_Number, Hardware_Release_DP, Firmware_Release_DP, Hardware_Release_User, Firmware_Release_User, reserved (9 Octets)
DP_Slave_Parameter_Set	Slave_Para_Len, SI_Flag, Slave_Type, Max_Diag_Data_Len, Max_Alarm_Len, Max_Channel_Data_Length, Diag_Upd_Delay, Alarm_Mode, Add_SI_Flag, C1_Timeout, reserved (4 Octets), Prm_Data_Len, Prm_Data, Cfg_Data_Len, Cfg_Data, Add_Tab_Len, Add_Tab, Slave_User_Data_Len, Slave_User_Data, Ext_Prm_Data_Len, [Ext_Prm_Data]
Bus_Parameter_Set	Bus_Para_Len, DL_Add, Data_rate, T _{SL} , min T _{SDR} , max T _{SDR} , T _{QUI} , T _{SET} , T _{TR} , G, HSA, max_retry_limit, Bp_Flag, Min_Slave_Interval, Poll_Timeout, Data_Control_Time, Alarm_Max, Max_User_Global_Control, reserved (4 Octets), Master_User_Data_Len, Master_Class2_Name, Master_User_Data*, T _{CT} , maxT _{SH}
Master_Parameter_Set	Bus_Parameter_Set, DP_Slave_Parameter_Set*
Statistic_Counters	Counter_Group*
Counter_Group	if Add_Offset 0..126: DLPDU_sent_count(Add_Offset), Error_count(Add_Offset) if Add_Offset 127: SD_sent_count, SD_error_count
Add_Tab	[Number_of_Entries], [Add_Tab_Entry_Header, I/O_Data_Length, IO_Config_Address, Host_Address]*
NOTE The DP_Slave_Parameter_Set and the Bus_Parameter_Set may exceed the maximum APDU size to a size up to 64 Koctets. The conveyance of the data is therefore segmented by means of a window addressed with the parameter Add_Offset.	

6.1.2 Data types

6.1.2.1 Notation for the Boolean type

Boolean ::= BOOLEAN

-- TRUE if the value is non-zero.

-- FALSE if the value is zero.

6.1.2.2 Notation for the Integer type

Integer8 ::= INTEGER (-128..+127)

-- range $-2^7 \leq I \leq 2^7-1$

Integer16 ::= INTEGER (-32768..+32767)

-- range $-2^{15} \leq I \leq 2^{15}-1$

Integer32 ::= INTEGER

-- range $-2^{31} \leq I \leq 2^{31}-1$

6.1.2.3 Notation for the Unsigned type

Unsigned8 ::= INTEGER (0..255)

-- range $0 \leq I \leq 2^8-1$

Unsigned16 ::= INTEGER (0..65535)

-- range $0 \leq I \leq 2^{16}-1$

Unsigned32 ::= INTEGER

-- range $0 \leq I \leq 2^{32}-1$

6.1.2.4 Notation for the Floating Point type

Floating32 ::= BIT STRING SIZE (4) -- IEC 60559 Single precision

6.1.2.5 Notation for the OctetString type

OctetString ::= OCTET STRING -- For generic use

6.1.2.6 Notation for VisibleString type

VisibleString2 ::= VISIBLE STRING -- For generic use

6.1.2.7 Notation for BinaryDate type

BinaryDate ::= OctetString7

6.1.2.8 Notation for TimeOfDay type

TimeOfDay with date indication ::= OctetString6

TimeOfDay without date indication ::= OctetString4

6.1.2.9 Notation for TimeDifference type

TimeDifference with date indication ::= OctetString6

TimeDifference without date indication ::= OctetString4

6.1.2.10 Notation for Network Time type

Network Time ::= OctetString8

6.1.2.11 Notation for Network Time Difference type

Network Time Difference ::= OctetString8

6.2 Transfer syntax

6.2.1 Coding of basic data types

6.2.1.1 Encoding of a Boolean value

- The encoding of a Boolean value shall be primitive. The ContentsOctets shall consist of a single octet.
- If the Boolean value is FALSE, the ContentsOctets shall be 0 (zero). If the Boolean value is TRUE, the ContentsOctets shall be 0xff.

6.2.1.2 Encoding of an Integer value

- The encoding of a fixed-length Integer value of Integer8, Integer16, and Integer32 types shall be primitive, and the ContentsOctets shall consist of exactly one, two, or four octets, respectively.
- The ContentsOctets shall be a two's complement binary number equal to the integer value, and consist of bits 7 to 0 of the first octet, followed by bits 7 to 0 of the second octet, followed by bits 7 to 0 of each octet in turn up to and including the last octet of the ContentsOctets.

NOTE The value of a two's complement binary number is derived by numbering the bits in the ContentsOctets, starting with bit 0 of the last octet as bit zero and ending the numbering with bit 7 of the first octet. Each bit is assigned a numerical value of 2^N , where N is its position in the above numbering sequence. The value of the two's complement binary number is obtained by adding the numerical values assigned to each bit for those bits which are set to one, excluding bit 7 of the first octet, and then reducing this value by the numerical value assigned to bit 7 of the first octet if that bit is set to one.

6.2.1.3 Encoding of an Unsigned value

- The encoding of a fixed-length Unsigned value of Unsigned8, Unsigned16, and Unsigned32 types shall be primitive, and the ContentsOctets shall consist of exactly one, two, or four octets, respectively.

- The ContentsOctets shall be a binary number equal to the Unsigned value, and consist of bits 7 to 0 of the first octet, followed by bits 7 to 0 of the second octet, followed by bits 7 to 0 of each octet in turn up to and including the last octet of the ContentsOctets.

NOTE The value of a binary number is derived by numbering the bits in the ContentsOctets, starting with bit 0 of the last octet as bit zero and ending the numbering with bit 7 of the first octet. Each bit is assigned a numerical value of 2^N , where N is its position in the above numbering sequence. The value of the binary number is obtained by adding the numerical values assigned to each bit for those bits which are set to one.

6.2.1.4 Encoding of a Floating-Point value

- The encoding of a Floating32 value shall be primitive, and the ContentsOctets shall consist of exactly four octets.
- The ContentsOctets shall contain floating-point values defined in conformance with IEC 60559. The sign is encoded in bit 7 of the first octet. It is followed by the exponent starting from bit 6 of the first octet, and then the mantissa starting from bit 6 of the second octet for Floating32.

6.2.1.5 Encoding of a Visible String value

- The encoding of a variable length VisibleString value shall be primitive.
- There is no Length field; the length is encoded implicitly.
- The ContentsOctets shall be a sequence of octets. The leftmost string element is encoded in the first octet, followed by the second octet, followed by each octet in turn up to and including the last octet as rightmost of the ContentsOctets.

6.2.1.6 Encoding of an Octet String value

- The encoding of a variable length OctetString value shall be primitive.
- There is no Length field; the length is encoded implicitly.
- The ContentsOctets shall be a sequence of octets. The leftmost string element is encoded in the first octet, followed by second octet, followed by each octet in turn up to and including the last octet as rightmost of the ContentsOctets.

6.2.1.7 Encoding of a BinaryDate value

- The encoding of a BinaryDate value shall be primitive.
- The Length field shall indicate as a binary number the number of octets in the BinaryDate value.
- The ContentsOctets shall be equal in value to the octets in the data value, as shown in Figure 117:

bits	7	6	5	4	3	2	1	0	
octets									
1	2 ¹⁵	2 ¹⁴	2 ¹³	2 ¹²	2 ¹¹	2 ¹⁰	2 ⁹	2 ⁸	0...59 999 ms
2	2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	
3	RSV	RSV	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	0...59 min
4	SU	RSV	RSV	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	0...23 hours
5	day of week			day of month					1...7 day of week
	2 ²	2 ¹	2 ⁰	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	1...31 day of month
6	RSV	RSV	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	1...12 months
7	RSV	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	0 ... 50 years 2000 to 2050 51 ... 99 years 1951 to 1999
msb									
NOTE To avoid the Y2K problem the values of the year are from 0 to 50 for the years 2000 to 2050 and from 51 to 99 for the years 1951 to 1999. This is according to the British Standards Institution (BSi) DISC PD2000-1:1998 "A Definition of Year 2000 Conformance Requirements" Rule 3 (b).									

Figure 117 – Coding of the data type BinaryDate

6.2.1.8 Encoding of a Time Of Day with and without date indication value

- The encoding of a Time Of Day with and without date indication value shall be primitive.
- The Length field shall indicate as a binary number the number of octets in the Time Of Day with and without date indication value.
- The ContentsOctets shall be equal in value to the octets in the data value, as shown in Figure 118:

bits	7	6	5	4	3	2	1	0	
octets									
1	0	0	0	0	2^{27}	2^{26}	2^{25}	2^{24}	number of milliseconds since midnight
2	2^{23}	2^{22}	2^{21}	2^{20}	2^{19}	2^{18}	2^{17}	2^{16}	
3	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	
4	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
5	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	number of days since 01.01.84 only with date indication
6	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
msb									

Figure 118 – Encoding of Time Of Day value

6.2.1.9 Encoding of a Time Difference with and without date indication value

- The encoding of a Time Difference with and without date indication value shall be primitive.
- The Length field shall indicate as a binary number the number of octets in the Time Difference with and without date indication value.
- The ContentsOctets shall be equal in value to the octets in the data value, as shown in Figure 119:

bits	7	6	5	4	3	2	1	0	
octets									milliseconds
1	2 ³¹	2 ³⁰	2 ²⁹	2 ²⁸	2 ²⁷	2 ²⁶	2 ²⁵	2 ²⁴	
2	2 ²³	2 ²²	2 ²¹	2 ²⁰	2 ¹⁹	2 ¹⁸	2 ¹⁷	2 ¹⁶	
3	2 ¹⁵	2 ¹⁴	2 ¹³	2 ¹²	2 ¹¹	2 ¹⁰	2 ⁹	2 ⁸	
4	2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	days only with date indication
5	2 ¹⁵	2 ¹⁴	2 ¹³	2 ¹²	2 ¹¹	2 ¹⁰	2 ⁹	2 ⁸	
6	2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	
msb									

Figure 119 – Encoding of Time Difference value

6.2.1.10 Encoding of a Network Time value

- The encoding of a Network Time value shall be primitive.
- The Length field shall indicate as a binary number the number of octets in the Time Difference value.
- The ContentsOctets shall be equal in value to the octets in the data value, as shown in Figure 120:

bits	7	6	5	4	3	2	1	0	
octets									
1	2^{31}	2^{30}	2^{29}	2^{28}	2^{27}	2^{26}	2^{25}	2^{24}	Sec. since 1.1.1900 0.00,00 or since 7.2. 2036 6.28,16 when time value less than 0x9dff4400.00000000
2	2^{23}	2^{22}	2^{21}	2^{20}	2^{19}	2^{18}	2^{17}	2^{16}	
3	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	
4	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
5	2^{31}	2^{30}	2^{29}	2^{28}	2^{27}	2^{26}	2^{25}	2^{24}	Fraction Part of Seconds one unit is $1/(2^{32})$ s
6	2^{23}	2^{22}	2^{21}	2^{20}	2^{19}	2^{18}	2^{17}	2^{16}	
7	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	
8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
msb									

Figure 120 – Encoding of Network Time value

6.2.1.11 Encoding of a Network Time Difference value

- The encoding of a Network Time Difference value shall be primitive.
- The Length field shall indicate as a binary number the number of octets in the Time Difference value.
- The ContentsOctets shall be equal in value to the octets in the data value, as shown in Figure 121.

bits	7	6	5	4	3	2	1	0	
octets									
1	2^{31}	2^{30}	2^{29}	2^{28}	2^{27}	2^{26}	2^{25}	2^{24}	Seconds as Integer32
2	2^{23}	2^{22}	2^{21}	2^{20}	2^{19}	2^{18}	2^{17}	2^{16}	
3	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	
4	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
5	2^{31}	2^{30}	2^{29}	2^{28}	2^{27}	2^{26}	2^{25}	2^{24}	Fraction Part of Seconds one unit is $1/(2^{32})$ s
6	2^{23}	2^{22}	2^{21}	2^{20}	2^{19}	2^{18}	2^{17}	2^{16}	
7	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	
8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
msb									

Figure 121 – Encoding of Network Time Difference value

6.2.1.12 Encoding of a Null value

- The encoding of a NULL value shall be primitive.
- The ContentsOctet shall be omitted.

6.2.2 Coding section related to data exchange PDUs

6.2.2.1 Coding of the field Outp_Data

One of the following data types shall be used for each Outp_Data field:

Boolean, Integer, Unsigned, Floating Point, Visible String, Octet String, Date, Time of Day, Time-Difference.

6.2.2.2 Coding of the field Inp_Data

One of the following data types shall be used for each Inp_Data field:

Boolean, Integer, Unsigned, Floating Point, Visible String, Octet String, Date, Time of Day, Time-Difference.

NOTE Data types with variable length (Visible String, Octet String, Time of Day, Time Difference) for Inp_Data or Outp_Data are only once present in the DataExchange-RES-PDU or DataExchange-REQ-PDU. The user data correspond to the contents of the Configuration-PDU.

6.2.3 Coding section related to slave diagnosis PDUs

6.2.3.1 Coding of the field Station_status_1

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0: Diag.Station_Non_Existent

Shall be set to zero in case of Diagnoses-RES-PDU

Value (0): means FALSE if Get_Master_Diag-RES-PDU

Value (1): means TRUE if Get_Master_Diag-RES-PDU

Bit 1: Diag.Station_Not_Ready

FALSE (0), TRUE (1)

Bit 2: Diag.Cfg_Fault (Configuration Fault)

FALSE (0), TRUE (1)

Bit 3: Diag.Ext_Diag (Extended Diagnosis)

FALSE (0), TRUE (1)

Bit 4: Diag.Not_Supported

FALSE (0), TRUE (1)

Bit 5: Diag.Invalid_Slave_Response

Shall be set to zero if Diagnoses-RES-PDU

Value (0): means FALSE if Get_Master_Diag-RES-PDU

Value (1): means TRUE if Get_Master_Diag-RES-PDU

Bit 6: Diag.Prm_Fault (Parameter Fault)

FALSE (0), TRUE (1)

Bit 7: Diag.Master_Lock

Shall be set to zero if Diagnoses-RES-PDU

Value (0): means FALSE if Get_Master_Diag-RES-PDU

Value (1): means TRUE if Get_Master_Diag-RES-PDU

6.2.3.2 Coding of the field Station_status_2

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0: Diag.Prm_Req (Parameterisation requested)

FALSE (0), TRUE (1)

Bit 1: Diag.Stat_Diag (Static Diagnosis)

FALSE (0), TRUE (1)

Bit 2: DP (DP Protocol)

Shall always set to ones

Bit 3: Diag.WD_On (Watchdog on)

FALSE (0), TRUE (1)

Bit 4: Diag.Freeze_Mode

FALSE (0), TRUE (1)

Bit 5: Diag.Sync_Mode (Synchronisation Mode)

FALSE (0), TRUE (1)

Bit 6: reserved**Bit 7: Diag.Deactivated**

Shall be set to zero if Diagnoses-RES-PDU

Value (0): means FALSE if Get_Master_Diag-RES-PDU

Value (1): means TRUE if Get_Master_Diag-RES-PDU

6.2.3.3 Coding of the field Station_status_3

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0 to 6: reserved**Bit 7: Diag.Ext_Diag_Overflow (Overflow of extended Diagnosis)**

FALSE (0), TRUE (1)

6.2.3.4 Coding of the field Diag_Master_Add

This field shall be coded as data type Unsigned8 with the following values:

Decimal (0-125)

means the address of the DP-master (Class 1) which has parameterised this DP-slave

Decimal (255)

means DP-slave has not been parameterised or has got invalid parameterisation.

Decimal (126-254)

not allowed

6.2.3.5 Coding of the field Ident_Number

This field shall be coded as data type Unsigned16.

6.2.3.6 Coding of the field Header_Octet

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0 to 5: Block_Length

It shall contain the length of the diagnosis block. The Header_Octet itself shall always be counted.

For the base diagnosis format of the Device Related Diagnosis the values shall be in the range of decimal 2 to 63. For the base diagnosis format of the Identifier Related Diagnosis the values shall be in the range of decimal 2 to 32. For the DPV1 diagnosis format for the Device Related Diagnosis the values shall be in the range of decimal 4 to 63.

NOTE The Identifier Related Diagnoses is limited to 32 because the identifier itself are limited to 244.

Bit 6 to 7: Selection

Decimal (0): means Device Related Diagnosis

Decimal (1): means Identifier Related Diagnosis

6.2.3.7 Coding of the field Alarm_Type

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0 to 6: Alarm_Type

The Alarm_Type shall contain the values according to Table 397:

Table 397 – Alarm_Type range

Value (decimal)	Meaning
0	reserved
1	Diagnostic_Alarm
2	Process_Alarm
3	Pull_Alarm
4	Plug_Alarm
5	Status_Alarm
6	Update_Alarm
7 – 31	reserved
32 – 126	manufacturer-specific
127	reserved

Bit 7: shall be zero (identifying Alarm)

6.2.3.8 Coding of the field Status_Type

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0 to 6: Status_Type

The Status_Type shall contain values according to Table 398.

Table 398 – Status_Type value range

Value (decimal)	Meaning
0	reserved
1	Status_Message
2	Modul_Status
3	DXB_Link_Status
4 to 29	reserved
30	PrmCmdAck
31	Red_Status
32 to 126	Manufacturer-specific
127	reserved

Bit 7: shall be one (identifying Status)

6.2.3.9 Coding of the field Slot_Number

This field shall be coded as data type Unsigned8. The value shall be taken from the range zero to 254. The value 255 is reserved.

6.2.3.10 Coding of the field Alarm_Specifier

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0 to 1: Alarm_Specifier

This bit group shall contain the alarm specifier with the values according to Table 399:

Table 399 – Alarm_Specifier

Value (decimal)	Meaning	Comment
0	no further differentiation	
1	Error appears and Slot disturbed	the slot generates an alarm due to an error
2	Error disappears and Slot is okay	the slot generates an alarm and indicates that the slot has no further errors
3	Error disappears and Slot is still disturbed	the slot generates an alarm and indicates that the slot has still further errors

Bit 2: Additional_Acknowledge

Value (0): means no additional acknowledge used

Value (1): means additional acknowledge is used

Bit 3 to 7: Sequence_Number

The value range shall be from zero to 31 (decimal).

6.2.3.11 Coding of the field Status_Specifier

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0 to 1: Status_Specifier

Decimal (0): means no further differentiation

Decimal (1): means status appears

Decimal (2): means status disappears

Decimal (3): reserved

Bit 2 to 7: reserved

6.2.3.12 Coding of the field Diagnosis_User_Data

One of the following data types shall be used for each Diagnosis_User_Data field:

Boolean, Integer, Unsigned, Floating Point, Visible String, Octet String, Date, Time of Day, Time-Difference.

6.2.3.13 Coding of the field Modul_Status_Array

The field Modul_Status_Array is an array of octets that shall contain at least one and at most 61 octets referred to as Modul_Status_Octet. A Modul_Status_Octet shall consist of at least one and at most 4 module stati referred to as Modul_Status_Entry_1 to Modul_Status_Entry_4 coded in two bits each. Therefore, the number of Modul_Status_Octet corresponds to the number of configured modules within a device and shall be calculated as follows

- a) $N = 1$ if $M_{\text{highest}} \leq 4$
- b) $N = M_{\text{highest}} \text{ DIV } 4 + 1$ if $M_{\text{highest}} \text{ MOD } 4 \neq 0$
- c) $N = M_{\text{highest}} \text{ DIV } 4$ if $M_{\text{highest}} \text{ MOD } 4 = 0$

where

N is the number of octets Modul_Status_Octet or the number of array elements,

M_{highest} is the highest number of configured modules within a device (maximum 244).

The last Modul_Status_Octet (octet N) may not contain all 4 module status entries. If $M_{\text{highest}} \text{ MOD } 4 \neq 0$ the octet N is not fully filled and the remaining bits shall be set to zero referred to as Padding Bits.

The stati of the device's modules shall be structured in ascending order without gaps. The status of module number one is always placed in Modul_Status_Entry_1 of the Modul_Status_Octet number one. The coding of a Modul_Status_Octet shall be according to 3.6.4 and the individual bits shall have the following meaning:

NOTE The term "configured modules" addresses physical or virtual modules of a device that have had a related format field in the Chk_Cfg-REQ-PDU. Modules that are not part of the configuration are purely local and therefore not related to the Modul_Status_Array field.

Bit 0 to 1: Modul_Status_Entry_1

It shall contain the module status of a configured module with the number that meets $m \text{ MOD } 4 = 1$. Padding bits shall not be used here.

Bit 2 to 3: Modul_Status_Entry_2

It shall contain the module status of a configured module with the number that meets $m \text{ MOD } 4 = 2$. Otherwise padding bits (00 binary) shall be used.

Bit 4 to 5: Modul_Status_Entry_3

It shall contain the module status of a configured module with the number that meets $m \text{ MOD } 4 = 3$. Otherwise padding bits (00 binary) shall be used.

Bit 6 to 7: Modul_Status_Entry_4

These 2 bits shall contain the module status of a configured module with the number that meets $m \text{ MOD } 4 = 0$. Otherwise padding bits (00 binary) shall be used.

where

m is the number of the current module with $1 \leq m \leq N$.

Figure 122 illustrates the arrangement of Modul_Status_Entry_(1 to 4) fields as an example for a device with 6 configured modules.

Bit Number							
7	6	5	4	3	2	1	0
Modul_Status_Octet 1		status module number 4		status module number 3		status module number 2	
Modul_Status_Octet 2		status module number 1		status module number 6		status module number 5	
Modul_Status_Entry_4		Modul_Status_Entry_3		Modul_Status_Entry_2		Modul_Status_Entry_1	

Figure 122 – Example Modul_Status_Array

Each entry Modul_Status_Entry_1 to Modul_Status_Entry_4 shall contain the module status according to Table 400:

Table 400 – Range of Modul_Status_Entry (1-4)

Value (decimal)	Meaning
0	data valid
1	data invalid: the data of the corresponding module are not valid due to an error (e.g. short circuit)
2	data invalid/wrong module: the data of the corresponding module are not valid, due to a wrong module in place
3	data invalid/no module: the data of the corresponding module are not valid, because there is no module in place

6.2.3.14 Coding of the field Identifier_Diagnosis_Data_Array

The field Identifier_Diagnosis_Data_Array is an array of octets that shall contain at least one and at most 31 octets referred to as Identifier_Diagnosis_Data_Octet. A Identifier_Diagnosis_Data_Octet shall consist of at least one and at most 8 diagnosis entries referred to as Identifier_Diagnosis_Entry_1 to Identifier_Diagnosis_Entry_8. Therefore, the number of Identifier_Diagnosis_Data_Octet corresponds to the number of configured modules within a device and shall be calculated as follows

- a) $N = 1$ if $M_{\text{highest}} \leq 8$
- b) $N = M_{\text{highest}} \text{ DIV } 8 + 1$ if $M_{\text{highest}} \text{ MOD } 8 \neq 0$
- c) $N = M_{\text{highest}} \text{ DIV } 8$ if $M_{\text{highest}} \text{ MOD } 8 = 0$

where

N is the number of octets Identifier_Diagnosis_Data_Octet or the number of array elements,

M_{highest} is the highest number of configured modules within a device (maximum 244).

The last Identifier_Diagnosis_Data_Octet (octet N) may not contain all 8 diagnosis entries. If $M_{\text{highest}} \text{ MOD } 8 \neq 0$ the octet N is not fully filled and the remaining bits shall be set to zero referred to as Padding Bits.

NOTE The term DIV stands for division without rest. The term MOD stands for the rest of the division.

The identifier diagnosis of the device's modules shall be structured in ascending order without gaps. The identifier diagnosis of module number one is always placed in Identifier_Diagnosis_Entry_1 of the Identifier_Diagnosis_Data_Octet number one. The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0: Identifier_Diagnosis_Entry_1

This bit shall be set to one if the module with the number that meets $m \text{ MOD } 8 = 1$ indicates diagnosis otherwise it shall be set to zero. A padding bit shall not be used here.

Bit 1: Identifier_Diagnosis_Entry_2

This bit shall be set to one if the module with the number that meets $m \text{ MOD } 8 = 2$ indicates diagnosis otherwise it shall be set to zero. A padding bit shall be used if there is no module configured.

Bit 2: Identifier_Diagnosis_Entry_3

This bit shall be set to one if the module with the number that meets $m \text{ MOD } 8 = 3$ indicates diagnosis otherwise it shall be set to zero. A padding bit shall be used if there is no module configured.

Bit 3: Identifier_Diagnosis_Entry_4

This bit shall be set to one if the module with the number that meets $m \text{ MOD } 8 = 4$ indicates diagnosis otherwise it shall be set to zero. A padding bit shall be used if there is no module configured.

Bit 4: Identifier_Diagnosis_Entry_5

This bit shall be set to one if the module with the number that meets $m \text{ MOD } 8 = 5$ indicates diagnosis otherwise it shall be set to zero. A padding bit shall be used if there is no module configured.

Bit 5: Identifier_Diagnosis_Entry_6

This bit shall be set to one if the module with the number that meets $m \text{ MOD } 8 = 6$ indicates diagnosis otherwise it shall be set to zero. A padding bit shall be used if there is no module configured.

Bit 6: Identifier_Diagnosis_Entry_7

This bit shall be set to one if the module with the number that meets $m \text{ MOD } 8 = 7$ indicates diagnosis otherwise it shall be set to zero. A padding bit shall be used if there is no module configured.

Bit 7: Identifier_Diagnosis_Entry_8

This bit shall be set to one if the module with the number that meets $m \text{ MOD } 8 = 0$ indicates diagnosis otherwise it shall be set to zero. A padding bit shall be used if there is no module configured.

where m is the number of the current module with $1 \leq m \leq N$.

6.2.3.15 Coding of the field Identifier_Number

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0 to 5: Identifier_Number

The values zero to 63 (decimal) are valid.

Bit 6 to 7: Selection

Decimal (2): means Channel Related Diagnosis

6.2.3.16 Coding of the field Channel_Number

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0 to 5: Channel_Number

The values zero to 63 (decimal) are valid.

Bit 6 to 7: Input_Output_Selection

Decimal (0): reserved

Decimal (1): means input

Decimal (2): means output

Decimal (3): means input and output

6.2.3.17 Coding of the field Type_of_Diagnosis

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0 to 4: Error_Type

The Error_Type shall be set as shown in Table 401.

Table 401 – Error type

Value (decimal)	Meaning
0	reserved
1	short circuit
2	Undervoltage
3	Overvoltage
4	Overload
5	Overtemperature
6	line break
7	upper limit value exceeded
8	lower limit value exceeded
9	Error
10	reserved
...	
15	reserved
16	manufacturer specific
...	
31	manufacturer specific

Bit 5 to 7: Channel_Type

Decimal (0): unspecific, may be used for any type

Decimal (1): means bit

Decimal (2): means 2 bit

Decimal (3): means 4 bit

Decimal (4): means octet

Decimal (5): means word

Decimal (6): means 2 words

Decimal (7): reserved

6.2.3.18 Coding of the field Revision_Number

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0 to 5: Revision_Number

The values one to 63 (decimal) are valid.

Bit 6: shall be set to one

Bit 7: shall be set to one

6.2.3.19 Coding of the field Publisher_Address

This field shall be coded as data type Unsigned8. The range of values shall be zero to 125. This parameter shall contain the address of the DP-slave acting as Publisher.

6.2.3.20 Coding of the field Publisher_Status

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 7: Link_Status

TRUE (1) means no error and active, FALSE (0) error or not active during the last monitoring period

Bit 6: Link_Error

TRUE (1) means length error, FALSE (0) means no error

Bit 0 to 5: reserved

6.2.3.21 Coding of the field RedSpecifier

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0 to 1: Status_Specifier

Decimal (0): means no further differentiation

Bit 3 to 7: Seq Number

Contains Sequence Number of the last PrmCmd processed

6.2.3.22 Coding of the field Function

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0: reserved

Bit 1: Primary

TRUE (1) means a Primary request was executed last, FALSE (0) means no Primary request was executed last

Bit 2: Start MS1

TRUE (1) means a Start MS1 request was executed last, FALSE (0) means no Start MS1 request was executed last

Bit 3: Stop MS1

TRUE (1) means a Stop MS1 request was executed last, FALSE (0) means no Stop MS1 request was executed last

Bit 4: Check_Properties

TRUE (1) means a Check_Properties request was executed last, FALSE (0) means no Check_Properties request was executed last

Bit 5: reserved

Bit 6: MasterStateClear

TRUE (1) means a MasterStateClear request was executed last, FALSE (0) means no MasterStateClear request was executed last

Bit 7: reserved

6.2.3.23 Coding of the field Red_Status1

This field represents the Status of the Component which sent the status. The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0: Backup

TRUE (1) means Component is in Backup State, FALSE (0) means Component is not in Backup State

Bit 1: Primary

TRUE (1) means Component is in Primary State, FALSE (0) means Component is not in Primary State

Bit 2: HW-Defect

TRUE (1) means Component has a Hardware Defect, FALSE (0) means has no Hardware Defect

Bit 3: DataExchange

TRUE (1) means Component is in DataExchange State, FALSE (0) means Component is not in DataExchange State

Bit 4: Fail Safe

TRUE (1) means Component is in Fail Safe State, FALSE (0) means Component is not in Fail Safe State

Bit 5: DatarateSet

TRUE (1) means Component has a Datarate set, FALSE (0) means has no Datarate set

Bit 6: TohStarted

TRUE (1) means Component has a started Toh, FALSE (0) means has not started Toh

Bit 7: reserved

6.2.3.24 Coding of the field Red_Status2

This field represents the Status of a Partner Component of the Component which sent the status. The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0: Backup

TRUE (1) means Component is in Backup State, FALSE (0) means Component is not in Backup State

Bit 1: Primary

TRUE (1) means Component is in Primary State, FALSE (0) means Component is not in Primary State

Bit 2: HW-Defect

TRUE (1) means Component has a Hardware Defect, FALSE (0) means has no Hardware Defect

Bit 3: DataExchange

TRUE (1) means Component is in DataExchange State, FALSE (0) means Component is not in DataExchange State

Bit 4: Fail Safe

TRUE (1) means Component is in Fail Safe State, FALSE (0) means Component is not in Fail Safe State

Bit 5: DatarateSet

TRUE (1) means Component has a Datarate set, FALSE (0) means has no Datarate set

Bit 6: TohStarted

TRUE (1) means Component has a started Toh, FALSE (0) means has not started Toh

Bit 7: reserved**6.2.3.25 Coding of the field Red_Status3**

This field shall be coded as data type Unsigned8. The values are user specific.

6.2.4 Coding section related to parameterisation PDU**6.2.4.1 Coding of the field Station_status**

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0: reserved**Bit 1: reserved****Bit 2: reserved****Bit 3: WD_On (Watchdog on)**

FALSE (0), TRUE (1)

Bit 4: Freeze_Req

Value(0): Freeze mode not requested

Value(1): Freeze mode requested

Bit 5: Sync_Req

Value(0): Sync mode not requested

Bit 6: Unlock_Req

Shall be used as described in Table 402.

Value(1): Sync mode requested

Bit 7: Lock_Req

Shall be used as described in Table 402.

Table 402 – Specification of the bits Lock_Req and Unlock_Req

Bit 7	Bit 6	Meaning
0	0	The parameter min T_{SDR} can be changed. All other parameters remain unchanged.
0	1	The DP-slave will be unlocked for other Masters.
1	0	The DP-slave is locked for other Masters, all parameters are accepted (exception: min $T_{SDR} = 0$)
1	1	The DP-slave is unlocked for other Masters.

6.2.4.2 Coding of the field WD_Fact_1

This field shall be coded as data type Unsigned8 with the following value range:

Decimal 1 to 255

6.2.4.3 Coding of the field **WD_Fact_2**

This field shall be coded as data type Unsigned8 with the following value range:

Decimal 1 to 255

6.2.4.4 Coding of the field **min_T_{SDR}**

This field shall be coded as data type Unsigned8 with the following value range:

Decimal(0 to max T_{SDR} (see Part 4)) if DP operation

Decimal(0): means current value remains unchanged

6.2.4.5 Coding of the field **Group_Ident**

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0: Group_1

Bit 1: Group_2

Bit 2: Group_3

Bit 3: Group_4

Bit 4: Group_5

Bit 5: Group_6

Bit 6: Group_7

Bit 7: Group_8

Each of the bits above shall set to one if the device belongs to the indicated group number. Otherwise it shall be set to zero.

NOTE A device may belong to no groups (Group_Ident=decimal(0)) or all groups (Group_Ident=decimal(255)).

6.2.4.6 Coding of the field **User_Prm_Data_Element**

One of the following data types shall be used for each User_Prm_Data_Element:

Boolean, Integer, Unsigned, Floating Point, Visible String, Octet String, Date, Time of Day, Time-Difference.

6.2.4.7 Coding of the field **DPV1_Status_1**

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0: reserved

Bit 1: reserved

Bit 2: WD_Base_1ms

Value(0): Watchdog time base equals 10 milliseconds

Value(1): Watchdog time base equals 1 millisecond

Bit 3 to 4: reserved

Bit 5: Publisher_Enable

FALSE(0), TRUE(1)

Bit 6: Fail_Safe

FALSE(0), TRUE(1)

Bit 7: DPV1_Enable

FALSE(0), TRUE(1)

6.2.4.8 Coding of the field DPV1_Status_2

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0: Check_Cfg_Mode

Value(0): normal check (restrictive)

Value(1): user specific check (more flexible)

Bit 1: reserved**Bit 2: Enable_Update_Alarm**

FALSE(0), TRUE(1)

Bit 3: Enable_Status_Alarm

FALSE(0), TRUE(1)

Bit 4: Enable_Manufacturer_Specific_Alarm

FALSE(0), TRUE(1)

Bit 5: Enable_Diagnostic_Alarm

FALSE(0), TRUE(1)

Bit 6: Enable_Process_Alarm

FALSE(0), TRUE(1)

Bit 7: Enable_Pull_Plug_Alarm

FALSE(0), TRUE(1)

6.2.4.9 Coding of the field DPV1_Status_3

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0 to 2: Alarm_Mode

Decimal(0): 1 alarm of each type

Decimal(1): 2 alarms in total

Decimal(2): 4 alarms in total

Decimal(3): 8 alarms in total

Decimal(4): 12 alarms in total

Decimal(5): 16 alarms in total

Decimal(6): 24 alarms in total

Decimal(7): 32 alarms in total

Bit 3: Prm_Structure

FALSE(0), TRUE(1)

Bit 4: IsoM_Req

FALSE(0), TRUE(1)

Bit 5 to 6: reserved**Bit 7: PrmCmd**

FALSE(0), TRUE(1)

6.2.4.10 Coding of the field Structure_Length

This field shall be coded as data type Unsigned8 with the following value range:

Decimal 5 to 244, 255

The field Structure_Length contains the number of octets including itself. A value of 255 means that all elements in the Prm_User_Data following this element belongs to this structure.

6.2.4.11 Coding of the field Structure_Type

This field shall be coded as data type Unsigned8 with the following value range:

Decimal 0 to 1: reserved

Decimal 2: PrmCmd

Decimal 3: DXB Linktable

Decimal 4: IsoM Parameter

Decimal 5 to 6: reserved

Decimal 7: DXB Subscribable

Decimal 8 to 31: reserved

Decimal 32 to 128: Manufacturer Specific

Decimal 129: User Prm Data

Decimal 130 to 255: reserved

6.2.4.12 Coding of the field Version

This field shall be coded as data type Unsigned8 with the following value range:

Decimal 0: reserved

Decimal 1: Version 1

Decimal 2 to 255: reserved

6.2.4.13 Coding of the field Publisher_Addr

This field shall be coded as data type Unsigned8 with the following value range:

Decimal 0 to 125, 128

Decimal 126, 127, 129 to 255: reserved

In case of the DXB-Subscribable the value 128 indicates an entry for the DP-master (Class 1).

6.2.4.14 Coding of the field Publisher_Length

This field shall be coded as data type Unsigned8 with the following value range:

Decimal 1 to 244

Decimal 0, 245 to 255: reserved

In case of a Master Data entry in the DXB-Subscribable the value in this field is not relevant and shall be set to 0.

6.2.4.15 Coding of the field Sample_Offset

This field shall be coded as data type Unsigned8 with the following value range:

Decimal 0 to 243

Decimal 244 to 255: reserved

6.2.4.16 Coding of the field Sample_Length

This field shall be coded as data type Unsigned8 with the following value range:

Decimal 1 to 244

Decimal 0, 245 to 255: reserved

6.2.4.17 Coding of the Dest_Slot_Number

This field shall be coded as data type Unsigned8 with the following value range:

Decimal 1 to 244

Decimal 0, 245 to 255: reserved

6.2.4.18 Coding of the Offset_Data_Area

This field shall be coded as data type Unsigned8 with the following value range:

Decimal 0 to 244

Decimal 245 to 255: reserved

6.2.4.19 Coding of the field T_{BASE_DP}

This field shall be coded as data type Unsigned32 with the allowed values of decimal 375, 750, 1500 (default), 3000, 6000, 12000. All other values are reserved.

6.2.4.20 Coding of the field T_{DP}

This field shall be coded as data type Unsigned16 with the value range from 154 to $2^{16}-1$.

6.2.4.21 Coding of the field T_{MAPC}

This field shall be coded as data type Unsigned8.

6.2.4.22 Coding of the field T_{BASE_IO}

This field shall be coded as data type Unsigned32 with the allowed values of decimal 375, 750, 1500 (default), 3000, 6000, 12000. All other values are reserved.

6.2.4.23 Coding of the field T_I

This field shall be coded as data type Unsigned16.

6.2.4.24 Coding of the field T_O

This field shall be coded as data type Unsigned16.

6.2.4.25 Coding of the field T_{DX}

This field shall be coded as data type Unsigned32.

6.2.4.26 Coding of the field T_{PLL_W}

This field shall be coded as data type Unsigned16 with the value range from 1 to $2^{16}-1$.

6.2.4.27 Coding of the field T_{PLL_D}

This field shall be coded as data type Unsigned16 with the value range from 0 to $2^{16}-1$.

6.2.4.28 Coding of the field Specifier

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0 to 1: Specifier

Decimal (0): means no further differentiation

Bit 3 to 7: Seq Number

Contains Sequence Number of the PrmCmd

6.2.4.29 Coding of the field Function

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0: reserved

Bit 1: Primary

TRUE (1) means a Primary request is requested, FALSE (0) means no Primary request is requested

Bit 2: Start MS1

TRUE (1) means a Start MS1 request is requested, FALSE (0) means no Start MS1 request is requested

Bit 3: Stop MS1

TRUE (1) means a Stop MS1 request is requested, FALSE (0) means no Stop MS1 request is requested

Bit 4: Check_Properties

TRUE (1) means a Check_Properties request is requested, FALSE (0) means no Check_Properties request is requested

Bit 5: reserved

Bit 6: MasterStateClear

TRUE (1) means the MasterState is Clear, FALSE (0) means the MasterState is Operate

Bit 7: reserved

6.2.4.30 Coding of the field Properties

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0: Primary

TRUE (1) means a Primary request is used, FALSE (0) means no Primary request is used

Bit 1: Start Stop MS1

TRUE (1) means a Start Stop MS1 request is used, FALSE (0) means no Start Stop MS1 request is used

Bit 2: AddressChange

TRUE (1) means Address Change is required with Primary/Backup role, FALSE (0) means no Address Change is required with Primary/Backup role

Bit 3: AddressOffset

TRUE (1) means Address Offset is 64, FALSE (0) means Address Offset is 0.

Bit 4 to 7: reserved

6.2.4.31 Coding of the field Output Hold Time

This field shall be coded as data type Unsigned16 with the value range from 0 to $2^{16}-1$.

6.2.4.32 Coding of the field Clock Sync Interval

This field shall be coded as data type Unsigned16 with the value range from 0 to $2^{16}-1$.

6.2.4.33 Coding of the field CS Delay Time

This field shall be coded as data type Network Time Difference.

6.2.5 Coding section related to configuration PDUs

6.2.5.1 Coding of the field Cfg_Identifier

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0 to 3: Data_Length

Decimal(0): means 1 octet or 1 word according to the Length_Format

Decimal(1): means 2 octets or 2 words according to the Length_Format

Decimal(2): means 3 octets or 3 words according to the Length_Format

Decimal(3): means 4 octets or 4 words according to the Length_Format

Decimal(4): means 5 octets or 5 words according to the Length_Format

...

Decimal(15): means 16 octets or 16 words according to the Length_Format

Bit 4 to 5: Input_Output_Selection

Decimal(0): shall not be used here, means specific identifier format

Decimal(1): means input

Decimal(2): means output

Decimal(3): means input and output

Bit 6: Length_Format

Value(0) : means octet structure used

Value(1) : means word structure used (high octet transferred first, big-endian)

Bit 7: Consistency

Value(0): means octet or word consistency

Value(1): means consistency over the whole length

6.2.5.2 Coding of the field Special_Cfg_Identifier

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0 to 3: Length_of_Manufacturer_Specific_Data

The length information of the manufacturer specific data shall be in dependency of the use in the Chk_Cfg-REQ-PDU or Get_Cfg-RES-PDU as shown in Table 403 and Table 404. Data type fields shall always be counted if present.

Bit 4 to 5: shall be set to zero (decimal)

Bit 6 to 7: Input_Output_Selection

Decimal(0): means empty slot, no input or output data for this module

Decimal(1): means one length octet for input data follows

Decimal(2): means one length octet for output data follows

Decimal(3): means one length octet for output data followed by one length octet for input data follows

Table 403 – Range of Length_of_Manufacturer_Specific_Data if used in Chk_Cfg-REQ-PDU

Value (decimal)	Meaning
0	No manufacturer specific data follow; no data in Real_Cfg_Data.
1 to 14	Manufacturer specific data of specified length follow; these shall be identical with the data in Real_Cfg_Data.
15	No manufacturer specific data follow; the verification can be omitted

Table 404 – Range of Length_of_Manufacturer_Specific_Data if used in Get_Cfg-RES-PDU

Value (decimal)	Meaning
0	No manufacturer specific data follow
1 to 14	Manufacturer specific data with specified length follow
15	Not allowed

6.2.5.3 Coding of the fields Length_Octet

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0 to 5: Data_Length

Decimal(0): means 1 octet or 1 word according to the Length_Format

Decimal(1): means 2 octets or 2 words according to the Length_Format

...

Decimal(63): means 64 octets or 64 words according to the Length_Format

Bit 6: Length_Format

Value(0): means octet structure used

Value(1): means word structure used (high octet transferred first, big-endian)

Bit 7: Consistency

Value(0): means octet or word consistency

Value(1): means consistency over the whole length indicated in Data_Length

6.2.5.4 Coding of the field Manufacturer_Specific_Data

One of the following data types shall be used for each Manufacturer_Specific_Data field:

Boolean, Integer, Unsigned, Floating Point, Visible String, Octet String, Date, Time of Day, Time-Difference.

6.2.5.5 Coding of the field Extended_Length_Octet

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0 to 5: Data_Length

Decimal(0): means 1 octet

Decimal(1): means 2 octets

Decimal(2): means 3 octets

Decimal(3): means 4 octets

Decimal(4): means 5 octets

...

Decimal(63): means 64 octets

Bit 6: Format

This bit shall be set to zero that means octet structure is used.

Bit 7: Consistency

This bit shall be set to one that means consistency over the whole length.

6.2.5.6 Coding of the field Data_Type

This field shall be coded as data type Unsigned8 with values according to the code column at Table 405.

Table 405 – Values (codes) for data types

Data type name	Code (decimal)	Number of octets for the data type
Boolean	1	1
Integer8	2	1
Integer16	3	2
Integer32	4	4
Unsigned8	5	1
Unsigned16	6	2
Unsigned32	7	4
Floating Point 32	8	4
Visible String	9	1, 2, 3, ...
Octet String	10	1, 2, 3, ...
	11	reserved
Time Of Day (with Date indication)	12	6
	13	reserved
	14 to 49, 51	reserved
Date	50	7
Time Of Day (without Date indication)	52	4
Time Difference (with Date indication)	53	6
Time Difference (without Date indication)	54	4
Network Time	57	8
Network Time Difference	58	8
	55, 56 , 59 to 70	reserved
	71 to 100	User specific
	101 to 127	reserved
	128 to 255	User specific (no data type coding)

Sequences of data types shall be coded as a list of simple data types. The sum of the lengths of the data types shall be corresponding the input- or output data length. Data types with variable length (visible string, octet string, time of day, time difference) shall not be handled in a sequence, but as single element.

The user specific coding (value $\geq$ 128) shall not be used in any combination with type specific coding (value $\leq$ 70) in one Chk_Cfg-REQ-PDU or Get_Cfg-REQ-PDU.

6.2.6 Coding section related to global control PDUs

6.2.6.1 Coding of the field Control_Command

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0: reserved

Bit 1: Clear_Data

Value(0): no command

Value(1): clear output (set output data in safe state)

Bit 2: UnFreeze

Value(0): no command

Value(1): cancels Freeze command

Bit 3: Freeze

Value(0): no command

Value(1): freeze input values

Bit 4: Unsync

Value(0): no command

Value(1): cancels Sync command

Bit 5: Sync

Value(0): no command

Value(1): synchronise output values

Bit 6: reserved

Bit 7: reserved

Table 406 illustrates the meaning and priority of the Bits 2 to 5:

Table 406 – Specification of the bits for Un-/Sync and Un-/Freeze

Bit 2 resp. 4	Bit 3 resp. 5	Meaning
0	0	no function
0	1	function is activated
1	0	function is deactivated
1	1	function is deactivated

6.2.6.2 Coding of the field Group_Select

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0: Group_1

Bit 1: Group_2

Bit 2: Group_3

Bit 3: Group_4

Bit 4: Group_5

Bit 5: Group_6

Bit 6: Group_7

Bit 7: Group_8

Each of the bits above shall set to one if the control command belongs to the indicated group number. Otherwise it shall be set to zero. Zero means that all assigned Slaves shall execute the Control Command.

NOTE A control command may belong to no groups (Group_Ident=decimal(0)) or all groups (Group_Ident=decimal(255)).

6.2.7 Coding section related to clock-value-PDUs

6.2.7.1 Coding of the field Clock_value_time_event

The coding of this field shall be as data type Network Time with the fixed length of 8.

6.2.7.2 Clock_value_previous_TE

The coding of this field shall be as data type Network Time with the fixed length of 8.

6.2.7.3 Coding of the field Clock_value_status1

The coding of this field shall be according to 3.6.4, with C and CV mapped to the parameter local time diff in the set time service. The individual bits shall have the following meaning:

Bit 0: reserved

Bit 1: reserved

Bit 2 to 6: CV (Correction Value)

Value(0): 0 min

Value(1): 30 min

Value(2): 60 min

Value(3): 90 min

...

Value(31): 930 min

Bit 7: C (Sign of CV)

Value(0): CV to be added from Time

Value(1): CV to be subtracted from Time

6.2.7.4 Coding of the field Clock_value_status1

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0: SYF (Synchronisation fault)

Value(0): Clock_value_time_event is synchronised

Value(1): Clock_value_time_event is not synchronised (with other clocks in the system)

Bit 1: reserved

Bit 2: reserved

Bit 3 to 4: CR (Accuracy)

Value(0): 1 ms

Value(1): 10 ms

Value(2): 100ms

Value(3): 1 s

Bit 5: reserved

Bit 6: SWT (Summer/Winter-Time)

Value(0): Winter Time

Value(1): Summer Time

Bit 7: ANH (Announcement Hour)

Value(0): No Change planned within the next hour

Value(1): A Change of SWT will occur within the next hour

6.2.8 Coding section related to function identification and errors

6.2.8.1 Coding of the field Function_Num

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meanings. The meaning of the complete field is defined in Table 407.

Bit 0 to 4: Function_Code / Error_Code

This bits contain the Function_Code or the Error_Code corresponding to the DLPDU_Selector. The Function_Code shall be set according to Table 407.

Bit 5 to 6: PDU_Identifier

These two bits shall be set to 2 (decimal) to indicate a DP-PDU.

Bit 7: DLPDU_Selector

Value(0): means request DLPDU or positive response DLPDU

Value(1): means error or negative response DLPDU

The field Function_Num is transferred in the request and response DLPDU. If the service is processed correctly, bit 7 of Function_Num in the response APDU shall be set to zero. In case of an error an error DLPDU is transferred. In this APDU Function_Num contains an error code and the bit 7 shall be set to one.

Table 407 – Coding of the Function_Code/ Function_Num

DLPDU_Selector (decimal)	PDU_Identifier (decimal)	Function_Code (decimal)	Meaning	Complete field: Function_Num (hexadecimal)
0	2	0	reserved	0x40
0	2	1	Get_Master_Diag	0x41
0	2	2	Start_Seq	0x42
0	2	3	Download	0x43
0	2	4	Upload	0x44
0	2	5	End_Seq	0x45
0	2	6	Act_Para_Brct	0x46
0	2	7	Act_Param	0x47
0	2	8	Idle	0x48
0	2	9 to 16	reserved	0x49 to 0x50
0	2	17	Data_Transport	0x51
0	2	18 to 21	reserved	0x52 to 0x55
0	2	22	RM	0x56
0	2	23	Initiate	0x57
0	2	24	Abort	0x58
0	2	25	reserved	0x59
0	2	26	reserved	0x5A
0	2	27	reserved	0x5B
0	2	28	Alarm_Ack	0x5C
0	2	29	reserved	0x5D
0	2	30	Read	0x5E
0	2	31	Write	0x5F

The Error_Code shall be set according to Table 408:

Table 408 – Coding of the Error_Code / Function_Num

DLPDU_Selector (decimal)	PDU_Identifier (decimal)	Error_Code (decimal)	Meaning	Complete field: Function_Num (hexadecimal)
1	2	0	reserved	0xC0
1	2	1	FE	0xC1
1	2	2	NI	0xC2
1	2	3	AD	0xC3
1	2	4	EA	0xC4
1	2	5	LE	0xC5
1	2	6	RE	0xC6
1	2	7	IP	0xC7
1	2	8	SC	0xC8
1	2	9	SE	0xC9
1	2	10	NE	0xCA
1	2	11	DI	0xCB
1	2	12	NC	0xCC
1	2	13	TO	0xCD
1	2	14	CA	0xCE
1	2	15 to 16	reserved	0xCF to 0xD0
1	2	17	Error Data_Transport	0xD1
1	2	18 to 22	reserved	0xD2 to 0xD6
1	2	23	Error Initiate	0xD7
1	2	24	reserved	0xD8
1	2	25	reserved	0xD9
1	2	26	reserved	0xDA
1	2	27	reserved	0xDB
1	2	28	Error Alarm_Ack	0xDC
1	2	29	reserved	0xDD
1	2	30	Error Read	0xDE
1	2	31	Error Write	0xDF

6.2.8.2 Coding of the field Error_Decode

The coding of this field shall be as data type Unsigned8 and the values shall be set according to Table 409:

Table 409 – Values of Error_Decode

Value(decimal)	Meaning
0 to 127	reserved
128 =	DPV1
129 to 253 =	reserved
254 to 255 =	PROFILE_SPECIFIC

The field Error_Decode defines the meaning of the following Octets Error_Code_x. The Error_Decode values PROFILE_SPECIFIC indicate that the parameters Error_Code_x shall be set as defined in the corresponding protocols:

PROFILE_SPECIFIC

Error_Code_1= taken from profile specification

Error_Code_2= taken from profile specification

6.2.8.3 Coding of the field Error_Code_1

If field Error_Decode = DPV1

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0 to 3: Error_Code

The values shall be set according to the Error_Code column in Table 410.

Bit 4 to 7: Error_Class

The values shall be set according to the Error_Class column in Table 410.

Table 410 – Coding of Error_Code_1 at DPV1

Error_Class (decimal)	Meaning	Error_Code (decimal)
0 to 9	not specified	not specified *)
10	application	0 = read error 1 = write error 2 = module failure 3 to 6 = not specified 7 = busy 8 = version conflict 9 = feature not supported 10 to 15 = User specific
11	access	0 = invalid index 1 = write length error 2 = invalid slot 3 = type conflict 4 = invalid area 5 = state conflict 6 = access denied 7 = invalid range 8 = invalid parameter 9 = invalid type 10 = backup 11 to 15 = User specific
12	resource	0 = read constrain conflict 1 = write constrain conflict 2 = resource busy 3 = resource unavailable 4 to 7 = not specified 8 to 15 = User specific
13 to 15	User specific	
NOTE Not specified values are used to serve legacy codes and are intended to be passed unchanged to the application.		

If field Error_Decode = PROFILE_SPECIFIC

Error_Code_1 = from profile specification, needs to be specified in the profile definition

6.2.8.4 Coding of the field Error_Code_2

If field Error_Decode = DPV1

The coding of this field shall be as data type Unsigned8. The values are user specific.

If field Error_Decode = PROFILE_SPECIFIC

Error_Code_2 = from profile specification, needs to be specified in the profile definition

6.2.9 Coding section related to master diagnosis PDU

6.2.9.1 Coding of the field MDiag_Identifier

The coding of this field shall be as data type Unsigned8 and the values shall be set according Table 411:

Table 411 – Values of MDiag_Identifier

Value (decimal)	Meaning
0-125	Diag_Data of the DP-slave
126	System_Diagnostis
127	Master_Status
128	Data_Transfer_List
129-255	reserved

6.2.9.2 Coding of the field System_Diagnosis

The coding of this field shall be as data type Octet String with the fixed length of 16. The 16 octets of System_Diagnosis shall be coded as follows

Octet 1

Bit 0 shall be set to one if station number 0 has reported diagnosis otherwise it shall be set to zero.

Bit 1 shall be set to one if station number 1 has reported diagnosis otherwise it shall be set to zero.

Bit 2 shall be set to one if station number 2 has reported diagnosis otherwise it shall be set to zero.

Bit 3 shall be set to one if station number 3 has reported diagnosis otherwise it shall be set to zero.

Bit 4 shall be set to one if station number 4 has reported diagnosis otherwise it shall be set to zero.

Bit 5 shall be set to one if station number 5 has reported diagnosis otherwise it shall be set to zero.

Bit 6 shall be set to one if station number 6 has reported diagnosis otherwise it shall be set to zero.

Bit 7 shall be set to one if station number 7 has reported diagnosis otherwise it shall be set to zero.

Octet 2

Bit 0 shall be set to one if station number 8 has reported diagnosis otherwise it shall be set to zero.

Bit 1 shall be set to one if station number 9 has reported diagnosis otherwise it shall be set to zero.

etc.

Octet 16

.

.

Bit 5 shall be set to one if station number 125 has reported diagnosis otherwise it shall be set to zero.

Bit 6 to 7 shall be set to zero (not used)

6.2.9.3 Coding of the field USIF_State

This field shall be coded as data type Unsigned8. The following values are defined:

Value(0x40): shall be set if operation mode of DP-master (Class 1) is STOP

Value(0x80): shall be set if operation mode of DP-master (Class 1) is CLEAR

Value(0xC0): shall be set if operation mode of DP-master (Class 1) is OPERATE

6.2.9.4 Coding of the field Hardware_Release_DP

This field shall be coded as data type Unsigned8.

6.2.9.5 Coding of the field Firmware_Release_DP

This field shall be coded as data type Unsigned8.

6.2.9.6 Coding of the field Hardware_Release_User

This field shall be coded as data type Unsigned8.

6.2.9.7 Coding of the field Firmware_Release_User

This field shall be coded as data type Unsigned8.

6.2.9.8 Coding of the field Data_Transfer_List

The coding of this field shall be as data type Octet String with the fixed length of 16. The 16 octets of Data_Transfer_List shall be coded as follows

Octet 1

Bit 0 shall be set to one if data exchange executed with station number 0 otherwise it shall be set to zero.

Bit 1 shall be set to one if data exchange executed with station number 1 otherwise it shall be set to zero.

Bit 2 shall be set to one if data exchange executed with station number 2 otherwise it shall be set to zero.

Bit 3 shall be set to one if data exchange executed with station number 3 otherwise it shall be set to zero.

Bit 4 shall be set to one if data exchange executed with station number 4 otherwise it shall be set to zero.

Bit 5 shall be set to one if data exchange executed with station number 5 otherwise it shall be set to zero.

Bit 6 shall be set to one if data exchange executed with station number 6 otherwise it shall be set to zero.

Bit 7 shall be set to one if data exchange executed with station number 7 otherwise it shall be set to zero.

Octet 2

Bit 0 shall be set to one if data exchange executed with station number 8 otherwise it shall be set to zero.

Bit 1 shall be set to one if data exchange executed with station number 9 otherwise it shall be set to zero.

etc.

Octet 16

.

Bit 5 shall be set to one if data exchange executed with station number 125 otherwise it shall be set to zero.

Bit 6 to 7 shall be set to zero (not used)

6.2.10 Coding section related to Upload/download/act para PDUs

6.2.10.1 Coding of the field Area_Code_UpDownload

The coding of this field shall be as data type Unsigned8 and the values shall be set according to Table 412.

Table 412 – Values for Area_Code_UpDownload

Value (decimal)	Meaning
0 to 125	DP-slave parameter set (see field DP_Slave_Parameter_Set)
126	reserved
127	Bus parameter set (see field Bus_Parameter_Set)
128	reserved
129	statistic counters (see field Statistic_Counter)
130 to 135	reserved
136 to 139	Master parameter set (see field Master_Parameter_Set)
140 to 254	reserved
255	No local access protection in Start_Seq-REQ-PDU or Start_Seq-RES-PDU

6.2.10.2 Coding of the field Timeout

The coding of this field shall be as data type Unsigned16 and the time base for the timer shall be 1ms.

6.2.10.3 Coding of the field Max_Len_Data_Unit

The coding of this field shall be as data type Unsigned8 and the value for the Max_Len_Data_Unit shall be in the range from one to 240.

NOTE The minimum value for the Max_Len_Data_Unit is 48 in case of Initiate-RES_PDU.

6.2.10.4 Coding of the field Add_Offset

The coding of this field shall be as data type Unsigned16.

6.2.10.5 Coding of the field Data

One of the following data types shall be used for each Data field:

Boolean, Integer, Unsigned, Floating Point, Visible String, Octet String, Date, Time of Day, Time-Difference.

6.2.10.6 Coding of the field Data_Len

The coding of this field shall be as data type Unsigned8 and the value for the Data_Len shall be in the range from one to 240.

6.2.10.7 Coding of the field Area_CodeActBrct

The coding of this field shall be as data type Unsigned8 and the values shall be set according to Table 413.

Table 413 – Values for Area_CodeActBrct

Value (decimal)	Meaning
0 to 126	reserved
127	Bus_parameter set
128 to 129	reserved
130 to 135	reserved
136 to 139	Master parameter set
140 to 254	reserved
255	reserved

6.2.10.8 Coding of the field Area_CodeAct

The coding of this field shall be as data type Unsigned8 and the values shall be set according Table 414.

Table 414 – Values for Area_CodeAct

Value (decimal)	Meaning
0 to 125	DP-slave parameter set The Active Flag in the DP-slave parameter set of the DP-master (Class 1) is influenced accordingly. The DP-slave concerned takes part in the cyclic data exchange mode or is removed from this mode and no longer addressed.
126	reserved
127	Bus parameter set
128	operation mode
129	reserved
130 to 135	reserved
136 to 139	Master parameter set
140 to 255	reserved

6.2.10.9 Coding of the field Activate

This field shall be coded as data type Unsigned8. The following values are defined:

Value(0x80) means activate the corresponding DP-slave parameter set if the field Area_CodeAct equals 0.. 125

Value(0x00) means deactivate if the field Area_CodeAct equals 0.. 125

Value(0xFF) means activate the bus parameter set if the field Area_CodeAct equals 127

Value(0x40) means set operation mode to STOP if the field Area_CodeAct equals 128

Value(0x80) means set operation mode to CLEAR if the field Area_CodeAct equals 128

Value(0xC0) means set operation mode to OPERATE if the field Area_CodeAct equals 128

6.2.11 Coding section related to the bus parameter set

6.2.11.1 Coding of the field Bus_Para_Len

This field shall contain the length of Bus_Para inclusive the field Bus_Para_Len itself. This field shall be coded as data type Unsigned16. The range of values shall be from 66 to $2^{16}-1$.

6.2.11.2 Coding of the field DL_Add

This field shall be coded as data type Unsigned8. The range of values shall be zero to 125. This parameter shall contain the own address of the DP-master.

6.2.11.3 Coding of the field Data_rate

This field shall be coded as data type Unsigned8. The values shall be set as shown at Table 415:

Table 415 – Values for Data_rate

Value (decimal)	Meaning
0	9,6kbit/s
1	19,2kbit/s
2	93,75kbit/s
3	187,5kbit/s
4	500 kbit/s
5	reserved
6	1500 kbit/s
7	3000 kbit/s
8	6000 kbit/s
9	12000 kbit/s
10	31,25 kbit/s
11	45,45 kbit/s
12 to 255	reserved

6.2.11.4 Coding of the fields T_{SL}, min T_{SDR}, max T_{SDR}

These parameters are described in part 3. The coding of these fields shall be as data type Unsigned16.

6.2.11.5 Coding of the fields T_{QUI}, T_{SET}, G, HSA, max_retry_limit

These parameters are described in part 3. The coding of these fields shall be as data type Unsigned8.

6.2.11.6 Coding of the field T_{TR} (Target Token Rotation time)

This field is described in part 3. The coding of this field shall be as data type Unsigned32.

6.2.11.7 Coding of the field Bp_Flag (Busparameter flag)

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0 to 2: reserved

Bit 3 to 4: Isochronous Mode

Decimal (0): Not Synchronized

Decimal (1): Buffered Synchronized

Decimal (2): Enhanced Synchronized

Decimal (3): reserved

Bit 5: IsoM Sync

This bit shall be set to one if the DP master (Class 1) shall sent a Global_Control-REQ-PDU with Control Command Sync in case of isochronous operation. Otherwise this bit shall be set to zero.

Bit 6: IsoM Freeze

This bit shall be set to one if the DP master (Class 1) shall sent a Global_Control-REQ-PDU with Control Command Freeze in case of isochronous operation. Otherwise this bit shall be set to zero.

Bit 7: Error_Action_Flag

This bit shall be set to one if the DP master (Class 1) shall change the operation mode in case of an error. Otherwise this bit shall be set to zero.

6.2.11.8 Coding of the field Min_Slave_Interval

The coding of this field shall be as data type Unsigned16 with the value range from 1 to $2^{16}-1$ and the time base for the timer shall be 100µs.

6.2.11.9 Coding of the field Poll_Timeout

The coding of this field shall be as data type Unsigned16 with the value range from 1 to $2^{16}-1$ and the time base for the timer shall be 1ms.

6.2.11.10 Coding of the field Data_Control_Time

The coding of this field shall be as data type Unsigned16 with the value range from 1 to $2^{16}-1$ and the time base for the timer shall be 10ms.

6.2.11.11 Coding of the field Alarm_Max

The coding of this field shall be as data type Unsigned8 with the value range from 7 to 32.

6.2.11.12 Coding of the field Max_User_Global_Control

The coding of this field shall be as data type Unsigned8 with the value range from one to 255. The value Zero is reserved.

NOTE This parameter defines the maximum number of Global_Control requests, which may be started by the User at the same time. The parameter describes the ability of the DP-master. A practicable value for Max_User_Global_Control is 16. For each of the 8 Slave groups one SYNC and one FREEZE command may be started at the same time.

6.2.11.13 Coding of the field Master_User_Data_Len

This field shall contain the length of Master_User_Data inclusive the field Master_User_Data_Len itself. This field shall be coded as data type Unsigned16. The range of values shall be from 2 to $(2^{16}-1)$.

6.2.11.14 Coding of the field Master_Class2_Name

The coding of this field shall be as data type Visible String with the fixed length of 32.

6.2.11.15 Coding of the field Master_User_Data

One of the following data types shall be used for each Master_User_Data field:

Boolean, Integer, Unsigned, Floating Point, Visible String, Octet String, Date, Time of Day, Time-Difference.

6.2.11.16 Coding of the field T_{CT}

The coding of this field shall be as data type Unsigned32 with the value range from 1 to $2^{24}-1$.

6.2.11.17 Coding of the field maxT_{SH}

The coding of this field shall be as data type Unsigned8 with the value range from 1 to 2^8-1 .

6.2.12 Coding section related to the slave parameter set

6.2.12.1 Coding of the field Slave_Para_Len

The coding of this field shall be as data type Unsigned16 and the value for the Slave_Para_Len shall be in the range from zero to $2^{16}-1$. This parameter shall contain the length of Slave_Para inclusive the length parameter itself. A DP-slave parameter set can be deleted by setting the Slave_Para_Len to zero.

6.2.12.2 Coding of the field SI_Flag (slave flag)

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0: reserved

Bit 1: Extra_Alarm_SAP

This bit shall be set if the DP-master (Class 1) shall acknowledge alarms via SAP 50. Otherwise, if the DP-master (Class 1) acknowledges alarms via SAP 51 this bit shall not be set.

Bit 2: DPV1_Data_Types

This bit shall be set if extended data types are used in configuration (Extended_Special_Identifier_Format). Otherwise, if only octet or word types with the Identifier_Format or Special_Identifier_Format this bit may not be set.

Bit 3: DPV1_Supported

This bit shall be set to enable the DPV1 extensions. Otherwise the extensions shall be disabled.

Bit 4: Publisher_Enable

This bit shall be set to enable the publisher function. Otherwise the publisher function shall be disabled.

Bit 5: Fail_Safe

This bit shall be set to force the DP-master (Class 1) to convey a NULL-PDU during operation mode Clear. Otherwise data with the value zero will be send during Clear.

Bit 6: New_Prm

This bit shall be set to force the DP-master (Class 1) to convey a new Set_Prm-REQ-PDU during the data transfer phase. Otherwise this bit shall be set to zero.

Bit 7: Active

This bit shall be set to force the DP-master (Class 1) to activate the corresponding DP-slave. Otherwise this bit shall be set to zero.

6.2.12.3 Coding of the field Slave_Type

This field shall be coded as data type Unsigned8 with values according to Table 416:

Table 416 – Values for Slave_Type

Value (decimal)	Meaning
0	DP-slave
1 to 15	reserved
16 to 255	manufacturer specific

6.2.12.4 Coding of the field Max_Diag_Data_Len

This field shall be coded as data type Unsigned8 and with values from the range 6 to 244.

6.2.12.5 Coding of the field Max_Alarm_Len

This field shall be coded as data type Unsigned8 and with values from the range 4 to Min(Max_Diag_Data_Len-6, 64).

6.2.12.6 Coding of the field Max_Channel_Data_Length

This field shall be coded as data type Unsigned8 and with values from the range 4 to 244.

NOTE This parameter defines the maximum APDU size for the corresponding DP-slaves on the MS1-AR.

6.2.12.7 Coding of the field Diag_Upd_Delay

This field shall be coded as data type Unsigned8 and with values from the range 0 to 15 (extendible up to 255).

NOTE The parameter is used to count the number of Slave_Diag.cnf in the state DIAG2 while Diag_Data.Prm_Req is still set (for Slaves with reduced performance).

6.2.12.8 Coding of the field Alarm_Mode

This parameter specifies the maximum number of possible active alarms.

This field shall be coded as data type Unsigned8 with values according to Table 417:

Table 417 – Values for Alarm_Mode

Value (decimal)	Meaning
0	1 alarm of each type
1	2 alarms in total
2	4 alarms in total
3	8 alarms in total
4	12 alarms in total
5	16 alarms in total
6	24 alarms in total
7	32 alarms in total

6.2.12.9 Coding of the field Add_SI_Flag

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0: NA_To_Abort

This bit shall be set if the DP-master (Class 1) shall proceed with data exchange in case the corresponding DP-slave is not responding. Otherwise this bit shall not be set.

Bit 1: Ignore_ACIr

This bit shall be set if the DP-master (Class 1) shall exclude the corresponding DP-slave from the auto-clear behaviour. Otherwise this bit shall not be set.

Bit 2 to 7: reserved

6.2.12.10 Coding of the field C1_Timeout

This field shall be coded as data type Unsigned16 with values from the range decimal 1 to $2^{16}-1$. The value 0 is reserved but may be used for back-up values.

6.2.12.11 Coding of the field Prm_Data_Len

This parameter contains the length of Prm_Data inclusive the length parameter.

This field shall be coded as data type Unsigned16 and with values from the range 9 to 246.

6.2.12.12 Coding of the field Prm_Data

This field shall be coded as a Set_Prm-REQ-PDU.

6.2.12.13 Coding of the field Cfg_Data_Len

This parameter contains the length of Cfg_Data inclusive the length parameter.

This field shall be coded as data type Unsigned16 and with values from the range 3 to 246.

6.2.12.14 Coding of the field Cfg_Data

This field shall be coded as a Chk_Cfg-REQ-PDU.

6.2.12.15 Coding of the field Add_Tab_Len

This parameter contains the length of Add_Tab inclusive the length parameter.

This field shall be coded as data type Unsigned16 and with values from the range 2 to $2^{16}-31$.

6.2.12.16 Coding of the field Number_of_Entries

This field contains the number of entries in the address table.

This field shall be coded as data type Unsigned16 and with values from the range 1 to 488.

NOTE This parameter contains the number of entries in the address assignment table in the host to the addressed DP-slave. At maximum 488 entries per DP-slave are allowed (maximal 244 configuration strings doubled for inputs and outputs are possible).

6.2.12.17 Coding of the field Add_Tab_Entry_Header

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0: IO_Selection

This bit shall be set if the following addresses belong to the output part. Otherwise this bit shall not be set and the following addresses belong to the input part.

Bit 1: Address_Format_Selection

This bit shall be set to one if the following addresses are coded as Unsigned32. This bit shall be set to zero if the following addresses are coded as Unsigned16.

Bit 2 to 7: reserved**6.2.12.18 Coding of the field I/O_Data_Length**

This field shall be coded as data type Unsigned8 with the values from one to 244.

6.2.12.19 Coding of the field I/O_Config_Address

This field represents the local address of the related configuration identifier.

This field shall be coded as data type Unsigned16 if the bit Address_Format_Selection in the field Add_Tab_Entry_Header is set to zero.

This field shall be coded as data type Unsigned32 if the bit Address_Format_Selection in the field Add_Tab_Entry_Header is set to one.

6.2.12.20 Coding of the field Host_Address

This field represents the local address of the related input or output data.

This field shall be coded as data type Unsigned16 if the bit Address_Format_Selection in the field Add_Tab_Entry_Header is set to zero.

This field shall be coded as data type Unsigned32 if the bit Address_Format_Selection in the field Add_Tab_Entry_Header is set to one.

6.2.12.21 Coding of the field Slave_User_Data_Len

This parameter contains the length of Slave_User_Data inclusive the length parameter.

This field shall be coded as data type Unsigned16 and with values from the range 2 to $2^{16}-31$.

6.2.12.22 Coding of the field Slave_User_Data

One of the following data types shall be used for each Slave_User_Data field:

Boolean, Integer, Unsigned, Floating Point, Visible String, Octet String, Date, Time of Day, Time-Difference.

6.2.12.23 Coding of the field Ext_Prm_Data_Len

This parameter contains the length of Ext_Prm_Data inclusive the length parameter (2 octets).

This field shall be coded as data type Unsigned16 and with values from the range 2 (no Ext_Prm_Data follow) and 7 to 246.

6.2.12.24 Coding of the field Ext_Prm_Data

This field shall be coded as a Set_Ext_Prm-REQ-PDU.

6.2.13 Coding section related to statistic counters

6.2.13.1 Coding of the field DLPDU_sent_count and SD_count

The coding of this field shall be as data type Unsigned32.

6.2.13.2 Coding of the field Error_count and SD_error_count

The coding of this field shall be as data type Unsigned16.

6.2.14 Coding section related to set slave address PDU

6.2.14.1 Coding of the field New_Slave_Add

This field shall be coded as data type Unsigned8 with values from the range zero to 125.

6.2.14.2 Coding of the field No_Add_Change

This field shall be coded as data type Boolean. The value TRUE shall be used to indicate that the change of the address is only once possible. The value FALSE shall be used to indicate that the change of the address is more than once possible.

6.2.14.3 Coding of the field Rem_Slave_Data

One of the following data types shall be used for each Rem_Slave _Data field:

Boolean, Integer, Unsigned, Floating Point, Visible String, Octet String, Date, Time of Day, Time-Difference.

6.2.15 Coding section related to initiate/abort PDUs

6.2.15.1 Coding of the field Features_Supported_1

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0: Read_Write

This bit shall be set to indicate that Read and Write services on MS2-AR are supported.

NOTE Read and Write are mandatory on a MS2-AR.

Bit 1 to 7: reserved

6.2.15.2 Coding of the field Features_Supported_2

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0 to 7: reserved

6.2.15.3 Coding of the field Profile_Features_Supported_1

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0 to 7: defined by profile

This field shall be set to zero (decimal) if no profile is used. Otherwise the value shall be taken from the appropriate profile specification.

6.2.15.4 Coding of the field Profile_Features_Supported_2

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0 to 7: defined by profile

This field shall be set to zero (decimal) if no profile is used. Otherwise the value shall be taken from the appropriate profile specification.

6.2.15.5 Coding of the field Profile_Ident_Number

The coding of this field shall be as data type Unsigned16. The value zero shall be set if no profile is used. Otherwise the value shall be taken from the appropriate profile specification.

6.2.15.6 Coding of the field S_Type (source type)

The coding of this field shall be as data type Unsigned8. The value zero shall be set to indicate the use of the short address format. The value one shall be set to indicate the use of the long address format. Other values are reserved.

NOTE The terms source and destination always belong to the direction of the used PDU.

6.2.15.7 Coding of the field D_Type (destination type)

The coding of this field shall be as data type Unsigned8. The value zero shall be set to indicate the use of the short address format. The value one shall be set to indicate the use of the long address format. Other values are reserved.

6.2.15.8 Coding of the field S_Len (source length)

This field shall be coded as data type Unsigned8.

6.2.15.9 Coding of the field D_Len (destination length)

This field shall be coded as data type Unsigned8.

6.2.15.10 Coding of the field S_API (source application identifier)

This field shall be coded as data type Unsigned8.

6.2.15.11 Coding of the field D_API (destination application identifier)

This field shall be coded as data type Unsigned8.

6.2.15.12 Coding of the field S_SCL (source security level)

This field shall be coded as data type Unsigned8. The value zero shall be set to indicate that no access level is used.

6.2.15.13 Coding of the field D_SCL (destination security level)

This field shall be coded as data type Unsigned8. The value zero shall be set to indicate that no access level is used.

6.2.15.14 Coding of the field S_Network_Address

The field shall only be present if the field S_Type is set to one.

The coding of this field shall be as data type Octet String with the fixed length of 6.

6.2.15.15 Coding of the field D_Network_Address

The field shall only be present if the field D_Type is set to one.

The coding of this field shall be as data type Octet String with the fixed length of 6.

6.2.15.16 Coding of the field S_MAC_Address

The field shall only be present if the field S_Type is set to one.

The coding of this field shall be as data type Octet String.

6.2.15.17 Coding of the field D_MAC_Address

The field shall only be present if the field D_Type is set to one.

The coding of this field shall be as data type Octet String.

6.2.15.18 Coding of the field Send_Timeout

This field shall be coded as data type Unsigned16. The time base for the timer shall be 10 ms. The values shall be set from the range 1 to $2^{16}-1$.

6.2.15.19 Coding of the field Server_SAP

This field shall be coded as data type Unsigned8. The values shall be set from the range zero to 48.

6.2.15.20 Coding of the field Subnet

The coding of this field shall be as data type Unsigned8 and the values shall be set according to Table 418.

Table 418 – Values for Subnet

Value (decimal)	Meaning
0	means no subnet
1	means subnet local
2	means subnet remote
3 to 255	reserved

6.2.15.21 Coding of the field Instance_Reason_Code

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0 to 3: Reason Code

The values shall be set according to Table 419 if Instance is set to DLL.

Table 419 – Values of reason code if instance is DLL

Value (decimal)	Meaning
1	UE (meaning see Part 3)
2	RR (meaning see Part 3)
3	RS (meaning see Part 3)
9	NR (meaning see Part 3)
10	DH (meaning see Part 3)
11	LR (meaning see Part 3)
12	RDL (meaning see Part 3)
13	RDH (meaning see Part 3)
14	DS (meaning see Part 3)
15	NA (meaning see Part 3)

The values shall be set according to Table 420 if Instance is set to MS2.

Table 420 – Values of reason code if instance is MS2

Value (decimal)	Meaning
1	ABT_SE means sequence error
2	ABT_FE means invalid request APDU received
3	ABT_TO means connection timed out
4	ABT_RE means invalid response APDU received
5	ABT_IV means invalid service from the User
6	ABT_STO means requested value of Send_Timeout was too short
7	ABT_IA means invalid additional address information
8	ABT_OC means S-Timer expired, response APDU has not been sent yet

Bit 4 to 5: Instance

The values shall be set as follows:

Decimal (0): DLL

Decimal (1): MS2

Decimal (2): USER

Decimal (3): reserved

Bit 6 to 7: reserved

6.2.16 Coding section related to read/write/data transport PDUs

6.2.16.1 Coding of the field Index

This field shall be coded as data type Unsigned8 with values from the range zero to 255.

6.2.16.2 Coding of the field Length

This field contains the number of octets of the Data field.

This field shall be coded as data type Unsigned8 with values from the range zero to 240.

6.2.17 Coding section related to load region and function invocation PDUs

6.2.17.1 Coding of the field Extended_Function_Num

In addition to the field Function_Num, as described in 6.2.8.1, this field specifies the function of the Load Region or Function Invocation service.

The coding of this field shall be as data type Unsigned8 and the values shall be set according to Table 421:

Table 421 – Values of Extended_Function_Num

Value (decimal)	Meaning
0	Initiate_Load
1	Push
2	Pull
3	Terminate
4	Start
5	Stop
6	Resume
7	Reset
8	Call
9	Get_FI_State
10 to 255	reserved

6.2.17.2 Coding of the field Options

The coding of this field shall be according to 3.6.4 and the individual bits shall have the following meaning:

Bit 0: More_follows

This bit is only valid for the Push APDU and Pull APDU. This bit shall be set if the Load Region sequence is not finished and additional Push APDUs or Pull APDUS follow subsequently. This bit shall be reset, if the sequence is finished and no additional Push APDUs or Pull APDUS follow.

For the Initiate_Load APDU this bit shall be reset.

Bit 1 to 6: reserved**Bit 7: Pull / Push**

This bit is only valid in the Initiate_Load APDU. This bit shall be set, if Pull LR sequence is initialised. This bit shall be reset, if a Push LR sequence is initialised.

For the Push APDU and Pull APDU this bit shall be reset.

6.2.17.3 Coding of the field Sequence_Number

This field shall be coded as data type Unsigned32.

6.2.17.4 Coding of the field LR_Data

This field shall be coded as data type Octet String.

6.2.17.5 Coding of the field Max_Segment_Length

This field shall be coded as data type Unsigned8.

6.2.17.6 Coding of the field LR_Index

This field shall be coded as data type Unsigned16.

6.2.17.7 Coding of the field LR_Length

This field shall be coded as data type Unsigned32.

6.2.17.8 Coding of the field Max_Response_Delay

This field shall be coded as data type Unsigned16.

6.2.17.9 Coding of the field Intersegment_Request_Timeout

This field shall be coded as data type Unsigned16.

6.2.17.10 Coding of the field User_Specific

This field contains userspecific information, e.g. the format of the LR_Data.

This field shall be coded as data type Octet String.

6.2.17.11 Coding of the field FI_Index

This field shall be coded as data type Unsigned16.

6.2.17.12 Coding of the field Execution_Argument

This field shall be coded as data type Octet String.

6.2.17.13 Coding of the field Result_Argument

This field shall be coded as data type Octet String.

6.2.17.14 Coding of the field FI_State

This field contains the state of the Function Invocation entity.

The coding of this field shall be as data type Unsigned8 and the values shall be set according to Table 422:

Table 422 – Values of FI_State

Value (decimal)	Meaning
0	IDLE
1	STARTING
2	RUNNING
3	STOPPING
4	STOPPED
5	RESUMING
6	UNRUNNABLE
7	RESETTING
8	WAIT-SET-IN-USE
9	WAIT-SET-IN-USE-2
10	WAIT-DEL-IN-USE
11	WAIT-DEL-IN-USE-2
12	EXEC-ERR
13	EXEC-TERM
14 to 255	reserved

6.2.18 Examples of diagnosis-RES-PDUs

Figure 123 illustrates an example for a diagnosis APDU with a device related status message, a device related process alarm and an identifier related diagnostic.

bits	7	6	5	4	3	2	1	0	
octets									
1	0	0	0	0	0	1	1	1	Headeroctet: device related diagnostic
2	1	0	0	0	0	0	0	1	Status_Type: Status_Message
3	0	0	0	0	0	0	1	0	Slot_Number: 2 (sensor A)
4	0	0	0	0	0	0	0	0	Specifier: no further differentiation
5	0	0	0	0	0	1	0	1	Diagnostic_User_Data: 5 (average temperature)
6									Diagnostic_User_Data: temperature value (Unsigned16)
7									
8	0	0	0	0	1	0	0	1	Headeroctet: device related diagnostic
9	0	0	0	0	0	0	1	0	Alarm_Type: Process_Alarm
10	0	0	0	0	0	0	1	1	Slot_Number: 3 (valve B)
11	0	0	0	0	0	0	0	1	Specifier: alarm appears
12	0	1	0	1	0	0	0	0	Diagnostic_User_Data: 0x50 (upper limit exceeded pressure)
13									Diagnostic_User_Data: time stamp (Time_of_day = 4 octets)
14									
15									
16									
17	0	1	0	0	0	0	1	0	Headeroctet: identifier related diagnostic
18	0	0	0	0	0	1	0	1	1 st identifier Number with diagnosis

Figure 123 – Example of Ext_Diag_Data in case of DPV1 diagnosis format with alarm and status PDU

Corresponding textual part

;text assignments for sensor A and valve B

Unit_Diag_Area = 24-27

Value(1) = “Minimum temperature”

Value(2) = “Maximum temperature”

Value(5) = “Average temperature”

Unit_Diag_Area_End

Unit_Diag_Area = 28-31

Value(1) = “lower limit exceeded pressure”

Value(5) = “upper limit exceeded pressure”

Unit_Diag_Area_End

Unit_Diag_Area = 8-15

Value(2) = “sensor A”

Value(3) = “valve B”

Unit_Diag_Area_End

Unit_Diag_Area = 16-17

Value(1) = “alarm/status appearing”

Value(2) = “alarm/status disappearing”

Unit_Diag_Area_End

Since these definitions are used for both alarms and status messages their values should be different. That means different values for alarms and status messages should be used at the same position within the diagnostic field.

Figure 124 illustrates an example for a diagnosis APDU with a device related userspecific diagnostic, an identifier related diagnostic and a channel related diagnostic.

bits	7	6	5	4	3	2	1	0	
octets									
1	0	0	0	0	0	1	0	0	device related diagnostic:
2	device specific								meaning of the bits is defined
3	diagnostic field								manufacturer specific
4	of length 3								
5	0	1	0	0	0	1	0	0	identifier related diagnostic:
6	0	0	0	0	0	0	0	1	identifier number 0 has diagnostic
7	0	0	0	1	0	0	0	0	identifier number 12 has diagnostic
8	0	0	0	0	0	1	0	0	identifier number 18 has diagnostic
9	1	0	0	0	0	0	0	0	channel related diagnostic: identifier number 0
10	0	0	0	0	0	0	1	0	channel 2
11	0	0	1	0	0	1	0	0	overload, channel bit organised
12	1	0	0	0	1	1	0	0	identifier number 12
13	0	0	0	0	0	1	1	0	channel 6
14	1	0	1	0	0	1	1	1	upper limit value exceeded, channel word organised
	msb								

Figure 124 – Example of Ext_Diag_Data in case of the basic diagnosis format

6.2.19 Example of Chk_Cfg-REQ-PDU

Figure 125 illustrates an example for a Chk_Cfg APDU with a special identifier format.

bits	7	6	5	4	3	2	1	0	
octets									
1	1	1	0	0	0	0	1	1	input/output, 3 octets manufacturer specific data
2	1	1	0	0	1	1	1	1	consistency, output, 16 words
3	1	1	0	0	0	1	1	1	consistency, input, 8 words
4	manufacturer								
5	specific								
6	data								
	msb								

Figure 125 – Example of a special identifier format

6.2.20 Examples of Chk_Cfg-REQ-PDUs with DPV1 data types

The data types are coded into the manufacturer-specific part.

Simple data types are coded in a octet which contains the code of the data type.

Sequences of data types are coded as a list of simple data types. The sum of the lengths of the data types has to correspond to the input- or output data length. Data types with variable length (visible string, octet string, time of day, time difference) cannot be handled in a sequence, but as single element. Comments may be added at the end of the list of the data types.

Figure 126 shows the special identifier format for an analogue input block containing a value (Floating Point), Status (Unsigned8) and Comment (3 Octets).

bits	7	6	5	4	3	2	1	0	
octets									
1	0	1	0	0	0	1	0	1	Manufacturer-specific format
2	1	0	0	0	0	1	0	0	Consistency, 5 Octets (Input)
3	0	0	0	0	1	0	0	0	Data type Floating Point
4	0	0	0	0	0	1	0	1	Data type Unsigned8
5	Manufacturer								Comment
6	Specific								Comment
7	Data								Comment
	msb								

Figure 126 – Example of a special identifier format with data types

Figure 127 shows the special identifier format for an analogue output block where the output value can be read back.

The representation of this analogue output block is structured as follows:

Output Value (Floating Point)
 Output Status (Unsigned8)
 read back Value (Floating Point)
 read back Status (Unsigned8)
 comment (1 octet)

bits	7	6	5	4	3	2	1	0	
octets									
1	1	1	0	0	0	1	0	1	Manufacturer-specific format
2	1	0	0	0	0	1	0	0	Consistency, 5 Octets (Output)
3	1	0	0	0	0	1	0	0	Consistency, 5 Octets (Input)
4	0	0	0	0	1	0	0	0	Data Type Floating Point
5	0	0	0	0	0	1	0	1	Data Type Unsigned8
6	0	0	0	0	1	0	0	0	Data Type Floating Point
7	0	0	0	0	0	1	0	1	Data Type Unsigned8
8	1	1	1	1	0	0	0	0	Comment
	msb								

Figure 127 – Example of a special identifier format with data types

Figure 128 shows the special identifier format for an empty slot with 3 octets comment.

The representation of this empty slot is structured as follows:

empty slot
 comment (3 octets)

bits	7	6	5	4	3	2	1	0	
octets									
1	0	0	0	0	0	0	1	1	Manufacturer-specific format
2	1	1	1	1	0	0	0	0	Comment
3	1	1	1	1	0	1	0	0	Comment
4	1	1	1	1	0	0	1	0	Comment
	msb								

Figure 128 – Example of a empty slot with data types

Figure 129 shows the use of the special identifier format for a multi-variable device which utilises the user specific coding instead of data types in the data type field.

The representation of this multi variable device is structured as follows:

Analog Input (AI)

Discrete Output (DO) with setpoint (SP_D) and readback (READBACK_D)

bits	7	6	5	4	3	2	1	0	Multi variable device, AI and DO
octets									Analog Input
1	0	1	0	0	0	0	1	0	Special Config Identifier for the example AI (0x42)
2	1	0	0	0	0	1	0	0	Extended Length Octet for input (0x84)
3	1	0	0	0	0	0	0	1	User specific instead of data type >128 (e.g block code for AI 0x81)
4	1	1	1	1	0	0	0	1	User specific (e.g. Identification of cyclic parameter for AI 0x81)
5	1	1	0	0	0	0	1	0	Discrete Output
6	1	0	0	0	0	0	0	1	Special Config Identifier for the example DO (0xC2)
7	1	0	0	0	0	0	0	1	Extended Length Octet for output (0x81)
8	1	0	0	0	0	1	0	0	Extended Length Octet for input (0x81)
9	1	0	0	0	0	0	1	1	User specific instead of data type >128 (e.g. block code for DO 0x84)
	msb								User specific instead of data type >128 (e.g. Identification of cyclic parameter for DO with SP_D and READBACK_D 0x83)

Figure 129 – Example for multi-variable device with AI and DO function blocks

6.2.21 Example structure of the Data_Unit for Data_Exchange

The structure of the Data_Unit for the service Data_Exchange is defined by the identifiers which are passed to the DP-slave with the service Chk_Cfg at configuration.

The following example illustrates how the data in the Data_Unit will be transferred, corresponding to the given identifiers. Figure 130 shows the data which shall be sent by Data_Exchange and their order as defined by the service Chk_Cfg.

1. ID	2. ID	3. ID	4. ID	5. ID	6. ID	7. ID	...
1 B_I	1 W_I	2 W_O	2 B_IO	1 B_O	2 W_I	3 W_A	...

Figure 130 – Identifiers (ID)

Figure 131 shows the type and length of the data corresponding to their Identifiers and the direction of their transfer, meaning whether they are Input or Output.

1 B_I: 1 Octet input;
 1 B_O: 1 Octet output;
 2 B_IO: 2 Octet out- and input
 1 W_I: 1 Word input
 1 W_O: 1 Word output
 etc.

Figure 131 – Identifier list

The sequence of data in the Data_Unit is shown in Figure 132.

Data_Exchange.req									Data_Exchange.res								
lsb									lsb								
Bit	7	6	5	4	3	2	1	0	Bit	7	6	5	4	3	2	1	0
octets									octets								
1	Word_high				(3. ID)				1	Data				(1. ID)			
2	Word_low				(3. ID)				2	Word_high				(2. ID)			
3	Data				(4. ID)				3	Word_low				(2. ID)			
4	Data				(4. ID)				4	Data				(4. ID)			
5	Data				(5. ID)				5	Data				(4. ID)			
6	1.Word_high				(7. ID)				6	1.Word_high				(6. ID)			
7	1.Word_low				(7. ID)				7	1.Word_low				(6. ID)			
8	2.Word_high				(7. ID)				8	2.Word_high				(6. ID)			
9	2.Word_low				(7. ID)				9	2.Word_low				(6. ID)			
10	3.Word_high				(7. ID)				10	etc.							
11	3.Word_low				(7. ID)				11								
12	etc.								12								
...									...								
n									n								

Figure 132 – Structure of the Data_Unit for the request- and response-DLPDU

For the configuration identifier “empty slot” no data will be transferred in the Data_Unit.

6.3 FAL protocol state machines

6.3.1 Overall structure

6.3.1.1 Fieldbus Service Protocol Machines (FSPM)

The FSPM State Machines co-ordinate the underlying state machines used for processing of the various services and application relations.

There exist one state machine for each device type (FSPMS for DP-slave, FSPMM1 for DP-master (Class 1) and FSPMM2 for DP-master (Class 2).

6.3.1.2 Master to Slave cyclic (MS0)

The MSCY1 State Machines are responsible for the cyclic data transfer between the DP-master (Class 1) and one DP-slave (MSCY1M/MSCY1S). This comprises the parameterisation, configuration, diagnostic and the data exchange as well as the Global Control commands. There exist one MSCY1S for a DP-slave and up to 125 MSCY1M for a DP-master (Class 1). To Subscribe Input Data objects a DP-slave use up to 125 SSCY1S.

6.3.1.3 Master (class 1) to Slave acyclic (MS1)

The MSAC1 State Machines (MSAC1M, MSAC1S) are responsible for the acyclic data transfer between the DP-master (Class 1) and one DP-slave. The MS1 communication relationship is coupled to the cyclic communication relationship and processes the services Read, Write and Alarm Ack as well as the Load Region and Function Invocation services.

Additionally the MSAL1M state machine is used for alarm handling at the DP-master.

There exist one MSAC1S for a DP-slave and up to 125 MSAC1M/MSAL1M for a DP-master (Class 1).

6.3.1.4 Master (class 2) to Slave acyclic (MS2)

The MS2 State Machines (MSAC2M, MSRM2S, MSAC2S) are responsible for the acyclic data transfer between the DP-master (Class 2) and one DP-slave. The MS2 communication relationship processes the services Read, Write and Data_Transport as well as the Load Region and Function Invocation services.

The Master Slave Resource Manager (MSRM2S) is responsible for the assignment of a DLSAP and the related resources to each established MSAC2 communication relationship in the DP-slave. The Master Slave Resource Manager additionally starts the appropriate MSAC2S State Machine.

At the DP-master (Class 2) an arbitrary number of MSAC2M state machines exist. The maximum number of MSAC2M state machines is given by the maximum number of DP-slaves (=125) where each DP-slave provides the maximum number of MS2 CRs (=49).

6.3.1.5 Master to Slave clock synchronisation (MS3)

By means of the MSCS1 State Machines the clock synchronisation can be accomplished between the DP-master (Class 1) and the DP-slaves.

There exist at most one state machine at DP-master (Class 1) and one at DP-slave.

6.3.1.6 Master Master acyclic (MM1/MM2)

By means of the MMAC State Machines the Master_Diag, the Master Parameter Set, Slave Parameter Set, Bus Parameter Set can be transferred between the DP-master (Class 1) and the DP-master (Class 2).

There exist at most one state machine at DP-master (Class 1) and one at DP-master (Class 2).

6.3.1.7 DLL Mapping Protocol Machines (DMPM)

The DL Mapping Protocol Machines (DMPM) connects the other State machines and Layer 2. DMPM provides the coordination of all state machines concerning the configuration and error handling of the Data Link Layer Usage. The functions are mapped by the DMPM to the DLL and DLM services of Layer 2. The DMPM generates the necessary Layer 2 parameters of the service, receives the confirmations and indications from Layer 2 and passes them to the appropriate DMPM-User.

There exist one state machine for each device type (FSPMS for DP-slave, FSPMM1 for DP-master (Class 1) and FSPMM2 for DP-master (Class 2)).

Interface between DMPM and Layer 2

The DMPM is directly put upon the DLL User – DLL interface and on the DLMS User – DLM interface as described in Data Link Layer Service Definitions (Refer to DL Service Definition Type 3 Fieldbus IEC 61158-3).

The DMPM uses for the mapping of the DMPM functions on the Layer 2 the following services:

- a) DLL services
 - Send and Request Data with Reply (SRD)
 - Send Data with No Acknowledge (SDN)

b) DLM services

Reset

Set Value

Get Value

Event

SAP Activate

SAP Activate Responder

SAP Deactivate

6.3.2 Assignment of state machines to devices

Table 423 shows the assignment of State Machines to devices. Some State Machines are always required and thus marked with 'M' (mandatory) others are marked with 'O' (optional). The Remarks show options in mandatory machines.

Table 423 – Assignment of state machines

Device type	Machine	Mandatory/ optional	Remarks
DP-slave	FSPMS	M	Alarm handling is optional
	MSCY1S	M	
	SSCY1S	O	
	MSAC1S	O	
	MSAC2S	O	
	MSRM2S	O	
	DMPMS	M	
DP-master (Class 1)	FSPMM1	M	
	MSCY1M	M	
	MSAL1M	O	
	MSAC1M	O	
	MMAC1	O	
	DMPMM1	M	
DP-master (Class 2)	FSPMM2	M	
	MSAC2M	O	
	MMAC2	O	
	DMPMM2	M	

6.3.3 Overview DP-slave

Figure 133 illustrates the general structure of the AL of a DP-slave by showing its state machines and the services used by them.

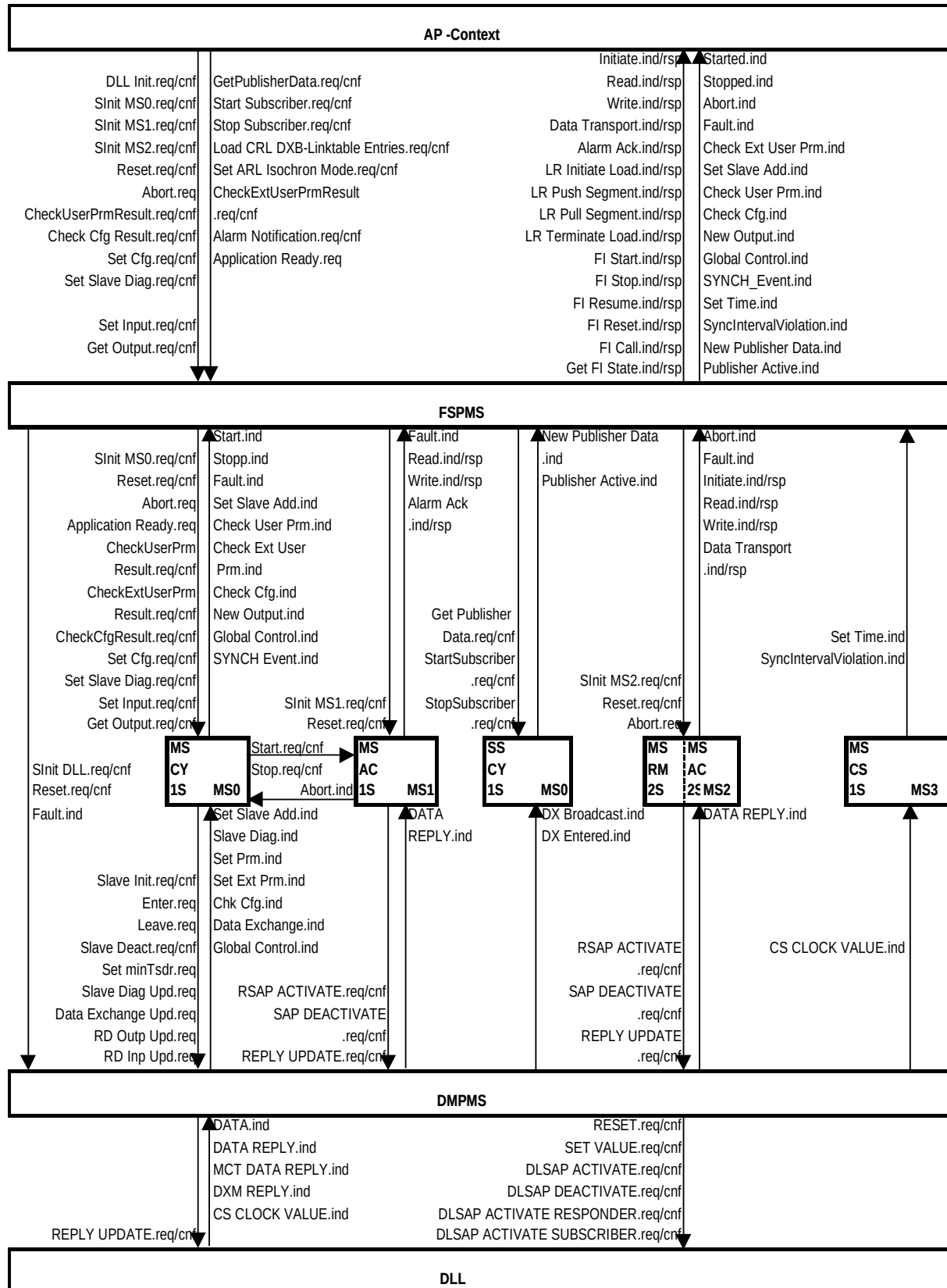


Figure 133 – Structuring of the protocol machines and adjacent layers in a DP-slave

6.3.4 Overview DP-master (class 1)

Figure 134 illustrates the general structure of the AL of a DP-master (Class 1) by showing its state machines and the services used by them.

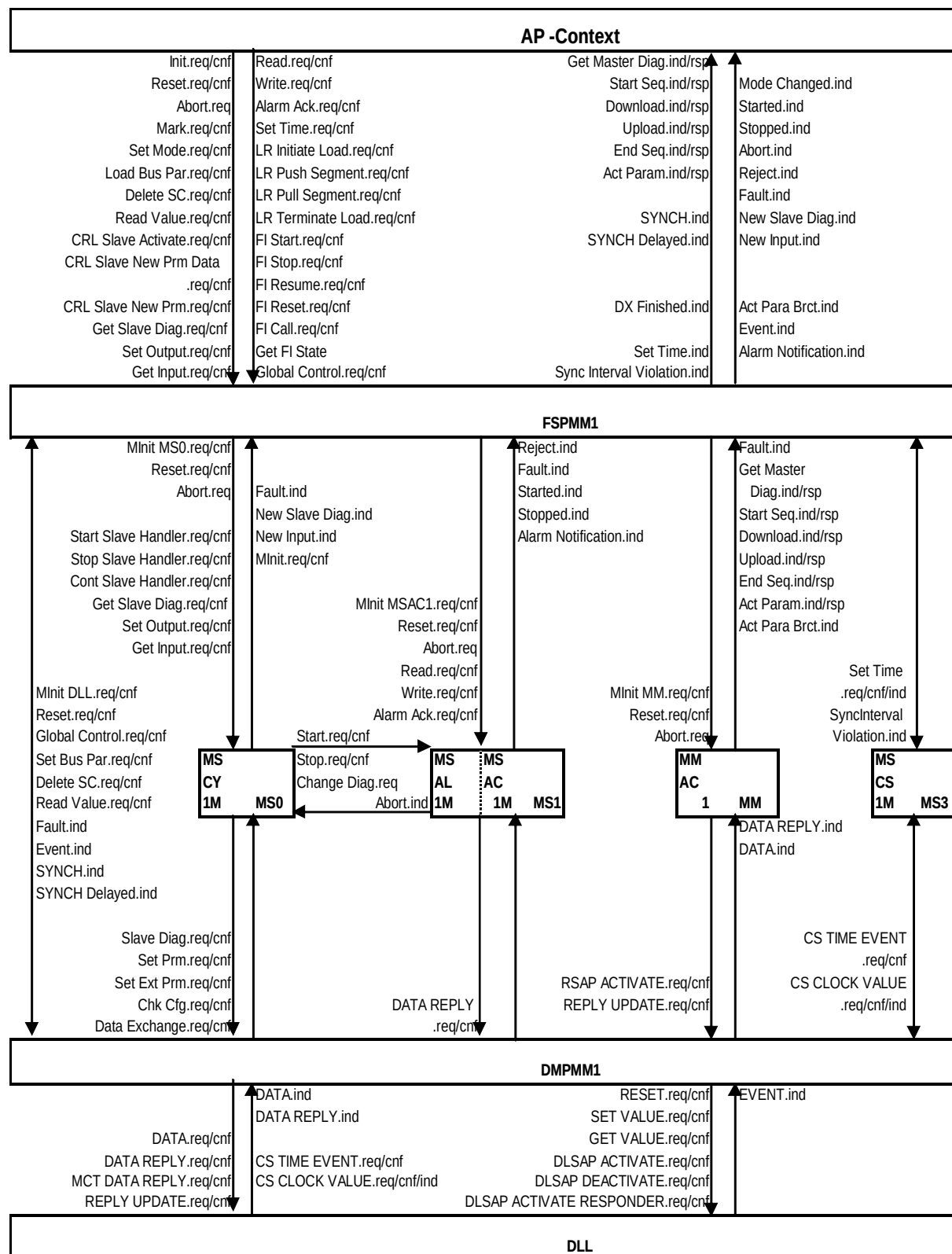


Figure 134 – Structuring of the protocol machines and adjacent layers in a DP-master (class 1)

6.3.5 Overview DP-master (class 2)

Figure 135 illustrates the general structure of the AL of a DP-master (Class 2) showing its state machines and the services used.

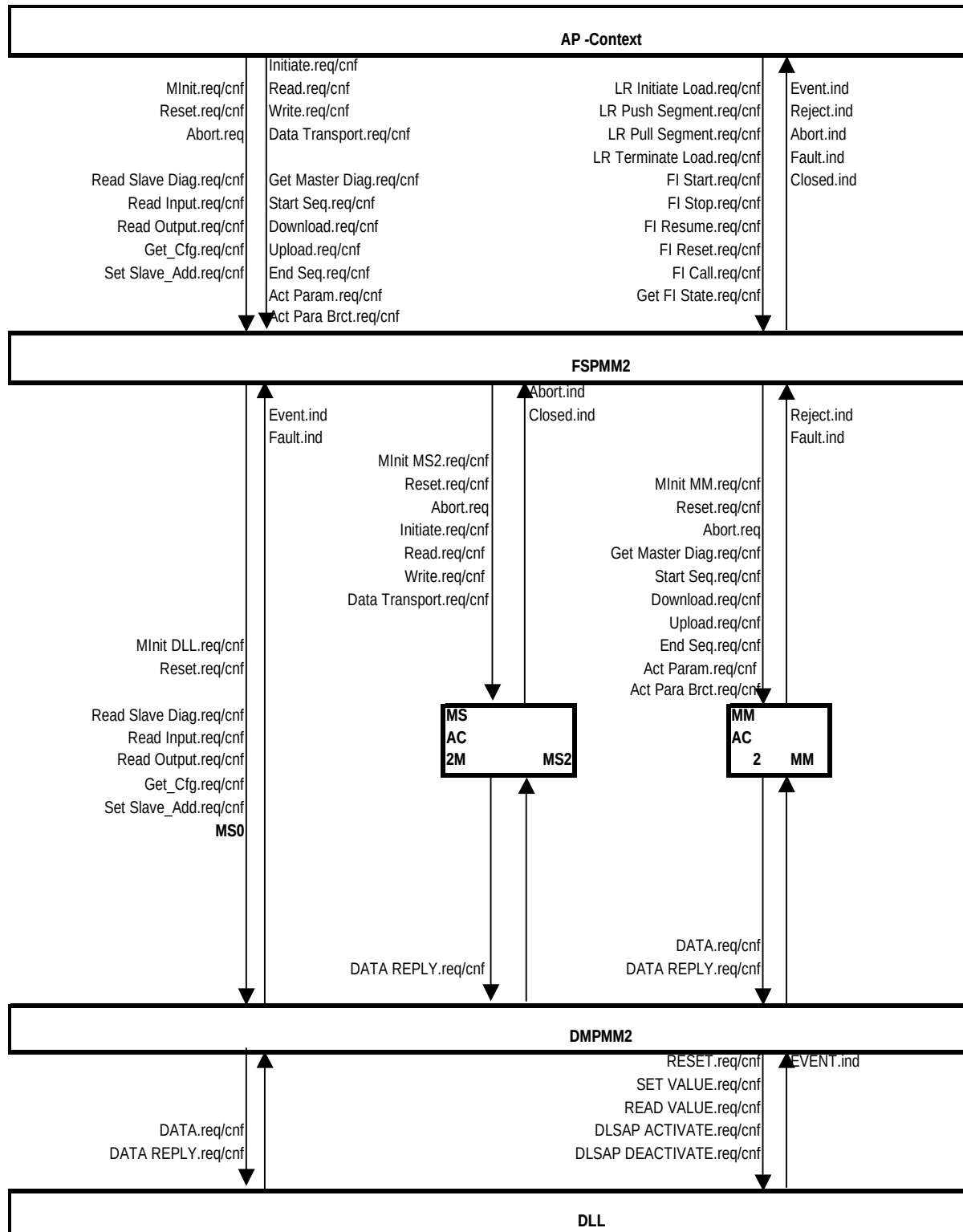


Figure 135 – Structuring of the protocol machines and adjacent layers in a DP-master (class 2)

6.3.6 Cyclic communication between DP-master (class 1) and DP-slave

The communication between DP-master and DP-slave is established with the sequence as shown in Figure 136:

If the DP-master wants to communicate with a DP-slave, the DP-master requests the Diag_Data of the DP-slave, in order to check the readiness for operation of the DP-slave.

This Diagnosis request will be repeated until the DP-slave responds with the requested data.

If the DP-slave responds with the requested diagnostic information, the DP-master checks whether another DP-master occupies this DP-slave. If this is not the case, the DP-slave will be parameterised and configured (Set_Prm / Chk_Cfg).

Subsequently the DP-master requests again the diagnostic data from the DP-slave. The following messages can occur:

- parameterisation or configuration error
- The DP-slave is occupied by another DP-master
- Static User diagnostics, no normal data transfer possible (for Example: start-up procedure for DP correctly executed, but some external parameterisation outstanding)
- DP-slave is not ready for operation

In the cases a) and b) the DP-master returns to the starting state and checks again the readiness for operation of the DP-slave.

In the cases c) and d) the DP-master requests the diagnostic information from the DP-slave until the mentioned messages disappear.

If no error messages occur the DP-master starts the data exchange to the DP-slave. The input and output data will be transferred to and from the DP-slave. The DP-master (Class 1) indicates his own operation mode to the DP-slaves for the first time.

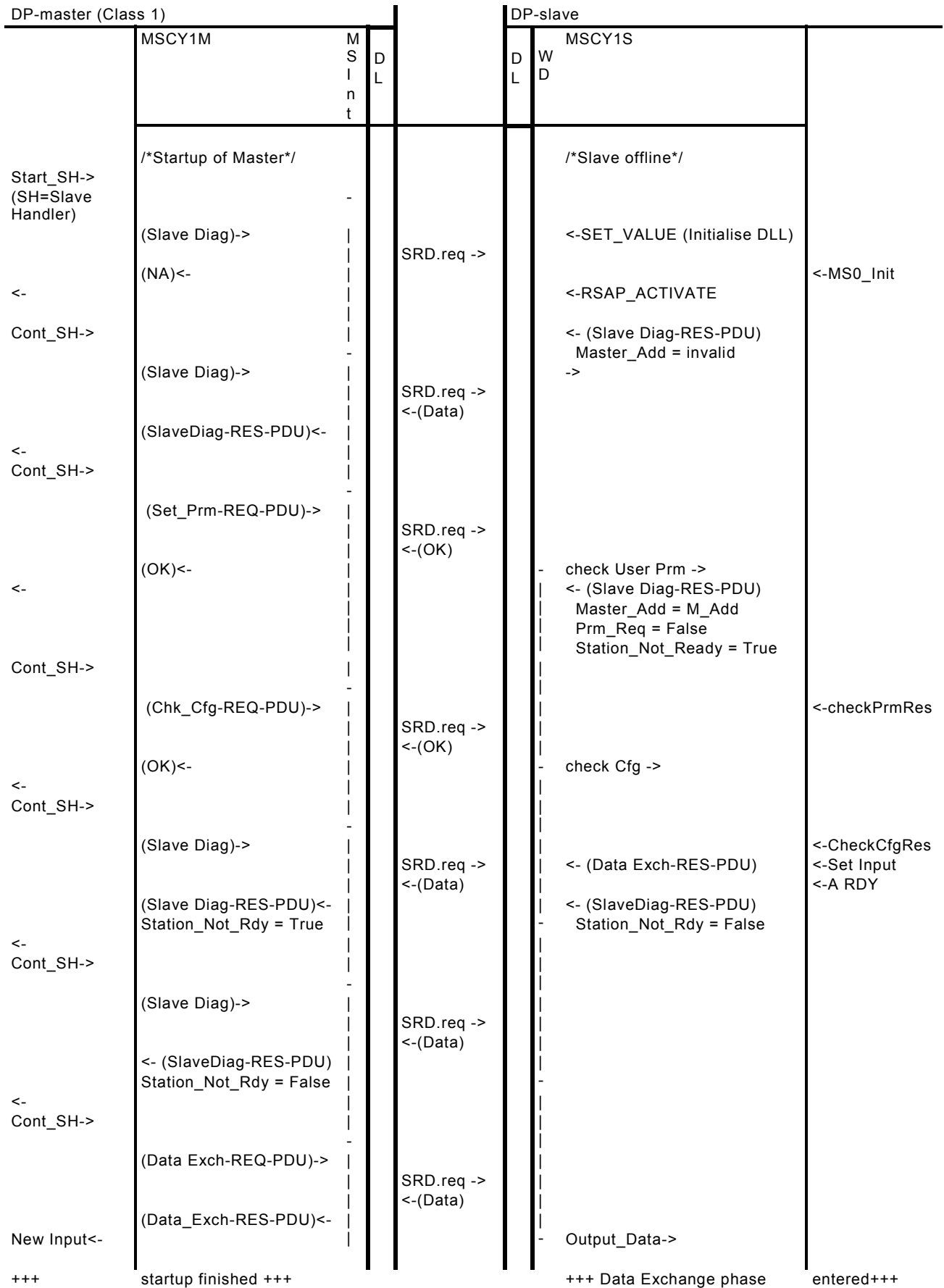


Figure 136 – Sequence of the communication between DP-master and DP-slave

6.3.7 Acyclic communication between DP-master (class 2) and DP-master (class 1)

The communication between DP-master (Class 2) and DP-master (Class 1) is handled in a fixed way (Request-Poll-Response), see Figure 137, with one exception.

This exception is the function Act_Para_Brct, which is transferred as a multicast from the DP-master (Class 2) to the corresponding DP-master (Class 1).

Communication is always initiated by the DP-master (Class 2). The DP-master (Class 2) sends a request to the DP-master (Class 1) and waits for the response. The service is processed by the MMAC2 of the DP-master (Class 2). After the receipt of the response the DP-master (Class 2) can make a new request.

At one point of time the DP-master (Class 1) can only communicate with one DP-master (Class 2).

The DP-master communication is initiated by a xxx.req from the DP-master (Class 2). The MMAC1 passes this request to Layer 2 and checks the status after receipt of the DMPM-response. If the response status is NR (no reply data) it will continue with a new SRD with no L_sdu to get response data. This procedure will be repeated by the DP-master (Class 2) until

an erroneous response occurs
or a SRD_RES_PDU with response data is received
or the Timer T1 expires.

The result will then be passed to the User of the DP-master (Class 2).

The timer T1 will be started after receipt of the xxx.req in the MMAC2 of the DP-master (Class 2) and will be stopped after release of the xxx.cnf. The value loaded in the timer T1 depends on the DLL parameter and the performance of the DP-master (Class 1).

The responder (DP-master (Class 1)) waits after a reset for a request. If the MMAC1 of the DP-master (Class 1) receives a valid request, this request is passed to the User by the corresponding xxx.ind. At this point of time an access protection was set to the local service access point (SAP) for the communication with the DP-master (Class 2). This is necessary to make sure that the response data will be sent to the right DP-master (Class 2).

After receipt of the response from the User (xxx.rsp) the response data will be passed with a REPLY_UPDATE.req to DMPM. Additionally, a timer will be started which controls the request for the response. If this timer expires and the response data are still not fetched by the DP-master (Class 2), the MMAC1 deletes the response data because the MMAC1 presumes that the DP-master (Class 2) is no longer available. In this case the access protection for the DP-master (Class 2) will be cancelled. This will also be done in the case of a single transaction if a DATA_REPLY.ind indicates that the response data are fetched by the DP-master (Class 2). The value loaded in the timer of the DP-master (Class 1) depends on the DL parameter and the performance of the DP-master (Class 2).

In case of Master-Master communication a sequence of requests can put into parentheses. Therefore the access protection will be set by the "Start_Seq" and released by the "End_Seq". To avoid errors the time between two requests will be supervised. The value for the time supervision is added to the Start_Seq request. With the Start_Seq.req the User can additionally send an areacode. By means of this areacode at the beginning of a sequence an area can be reserved at the User in the DP-master (Class 1).

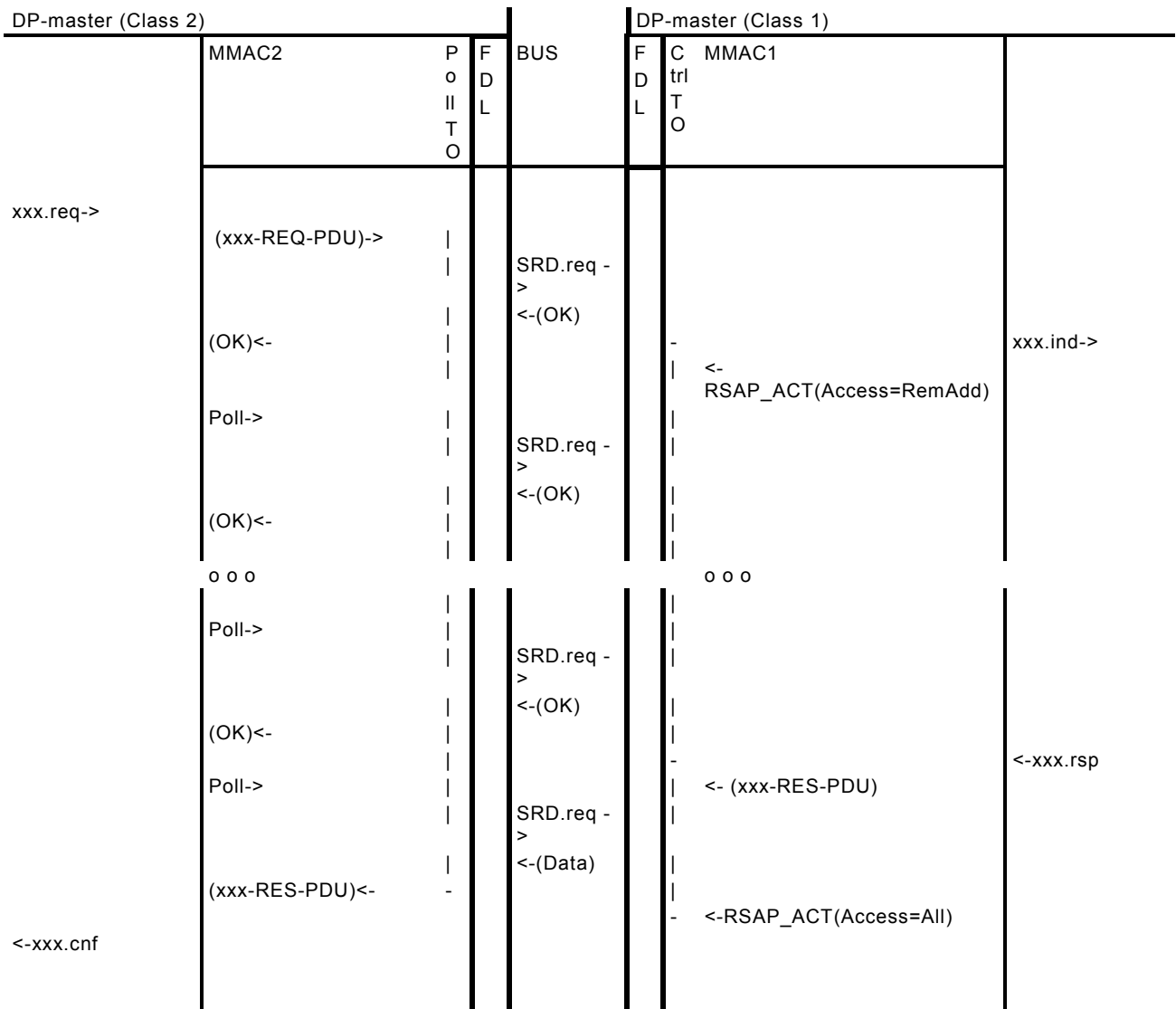


Figure 137 – Sequence of communication between DP-master (class 2) and DP-master (class 1)

6.3.8 Acyclic communication between DP-master (class 1) and DP-slave

The acyclic communication between DP-master (Class 1) and DP-slave is handled in a fixed way (Request-Poll-Response) on the MS1 application relationship, see Figure 138.

The communication is always initiated by the DP-master (Class 1). The connection is established if the cyclic communication MS0 AR has entered the data exchange mode. The DP-master (Class 1) issues a request and polls for the response.

The communication is initiated by a xxx.req from the DP-master (Class 1). The MSAC1M passes this request to Layer 2 and checks the status after receipt of the DMPMM1-response. If the response status is NR (no reply data) it will continue with a new SRD with a NULL-PDU to get response data. This procedure will be repeated by the DP-master (Class 1) until

- an erroneous response occurs
- or a SRD_RES_PDU with response data is received.

The result will then be passed to the User of the DP-master (Class 1) by means of a corresponding xxx.cnf.

The responder (DP-slave) waits for a request. If the MSAC1S of the DP-slave receives a valid request, this request is passed to the User by the corresponding xxx.ind.

After receipt of the response from the User (xxx.rsp) the response data will be passed with a REPLY_UPDATE.req to DMPMS.

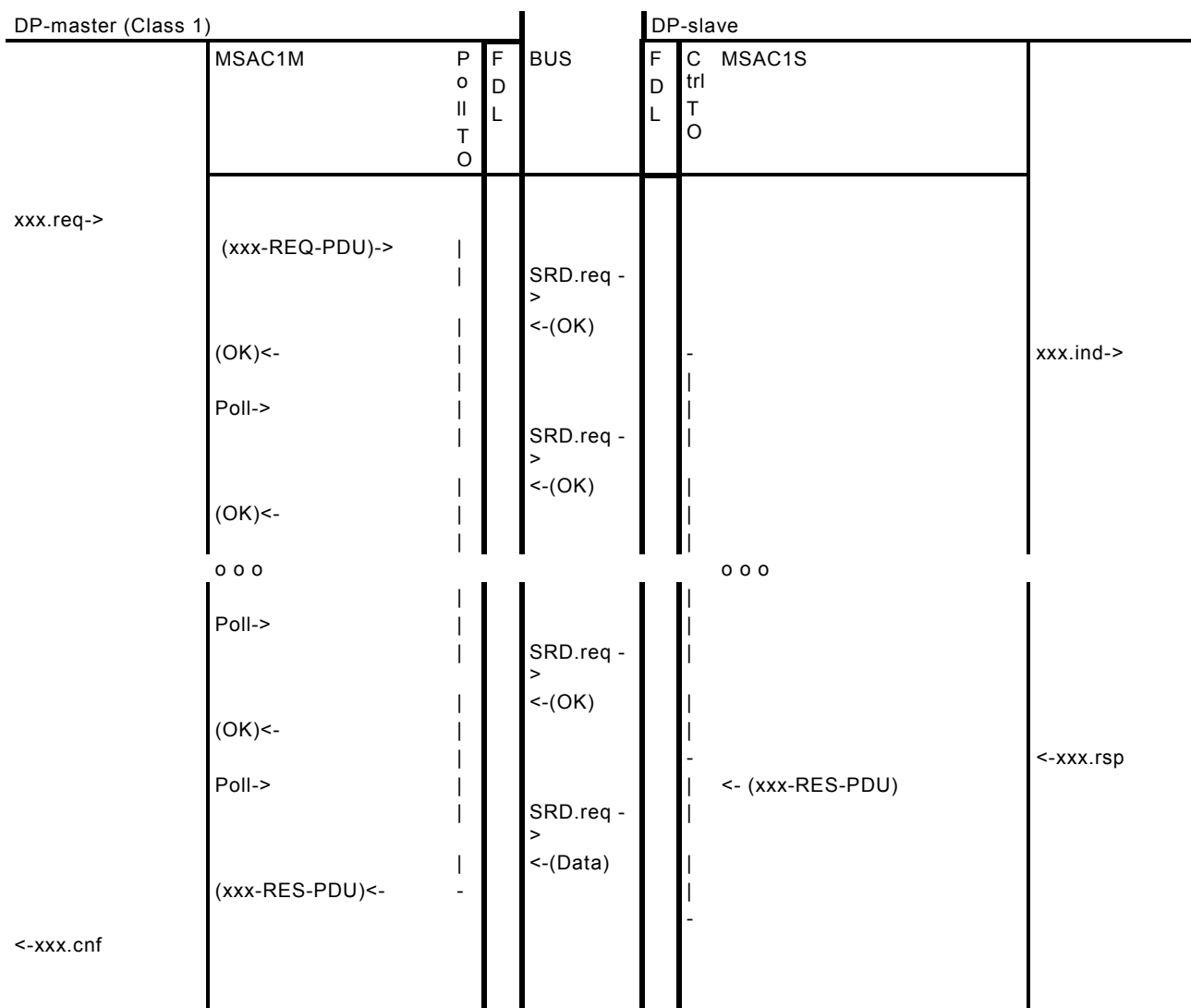


Figure 138 – Sequence of acyclic communication between DP-master (class 1) and DP-slave

6.3.9 Application relationship monitoring

6.3.9.1 Monitoring of the MS0 – AR

6.3.9.1.1 General

In industrial control systems there is a need for checking the correct functioning of each individual part. In a such a system a medium failure or a defect in a bus master shall not cause errors in the peripheral devices. In the above mentioned cases, a peripheral devices shall switch to a safe state. Additionally, the User of a bus master (Class 1) has to be informed about transmission breakdowns after a certain amount of time.

6.3.9.1.2 Control interval at the DP-slave

Watchdog control

This Timer, restarted by received Requests on the bus master side, will set the outputs of a bus slave to the safe state after the expiration of the timer.

6.3.9.1.3 Control intervals at the DP-master (class 1)

Min_Slave_Interval

This monitoring time specifies the smallest allowed period of time between two Slave poll cycles. This ensures that the sequence of function requests from the bus master can be handled by the bus slave. This period of time will be complied by the bus master (Class 1) for every master-slave function with exception of the function Global_Control. For the function Global_Control the User is responsible for compliance with Min_Slave_Interval.

Data_Control_Time

The Data_Control_Time defines the time in which a DP-master (Class 1) checks the data transfer to its associated DP-slaves. Furthermore, in half of this time the DP-master (Class 1) indicates his operation mode by means of a Global Control Service to the dedicated DP-slaves .

The Master parameter set contains the Data_Control_Time.

Rule

$$T_{WD} > T_{TR} \quad (1)$$

$$T_{WD} > \text{Min_Slave_Interval} \quad (2)$$

$$\text{Data_Control_Time} \geq 6 \cdot T_{WD} \quad (3)$$

6.3.9.2 Monitoring of the MS2 – AR

The objective of the connection monitoring is to detect the failure of the communication partner. Idle PDUs are exchanged to retrigger the monitoring timer of the communication partner.

The Idle service cycle initiated by a bus master is called Master-Idle. The Idle service cycle initiated by the Slave is called Slave-Idle.

All monitoring and idle sequences are controlled with a single timebase “Send Timeout”.

Send_Timeout

Each MSAC2 communication relationship is supervised by a Timer (Send_Timeout).

If this Timer expires the bus master (Class 2) aborts the MSAC2-communication relationship. Additionally the User of a bus master (Class 2) has to be informed.

This Timer has to be set up according to

the network load

the number of parallel connections per DP-master

Rule

$$T_{\text{Send_TO}}(TS) \geq (\sum \text{of all Activities of a Token cycle}) \cdot \text{Num_of_max acyclic connections}(TS) \quad (4)$$

NOTE The Send_Timeout parameter in the Initiate service is coded as an unsigned integer value, each unit represents 10ms. Due to Timer errors and the round up of integer, the next possible integer plus one has to be used as Send_Timeout.

It is assumed that one acyclic interaction is executed within a Token cycle at each DP-master. If more than one can be executed, the Send_TO may be divided by the number of acyclic interactions per token cycle.

Figure 139 shows an example for the establishment of an MS2 connection between a Master (Class 2) and a Slave. The connection monitoring starts immediately after this.

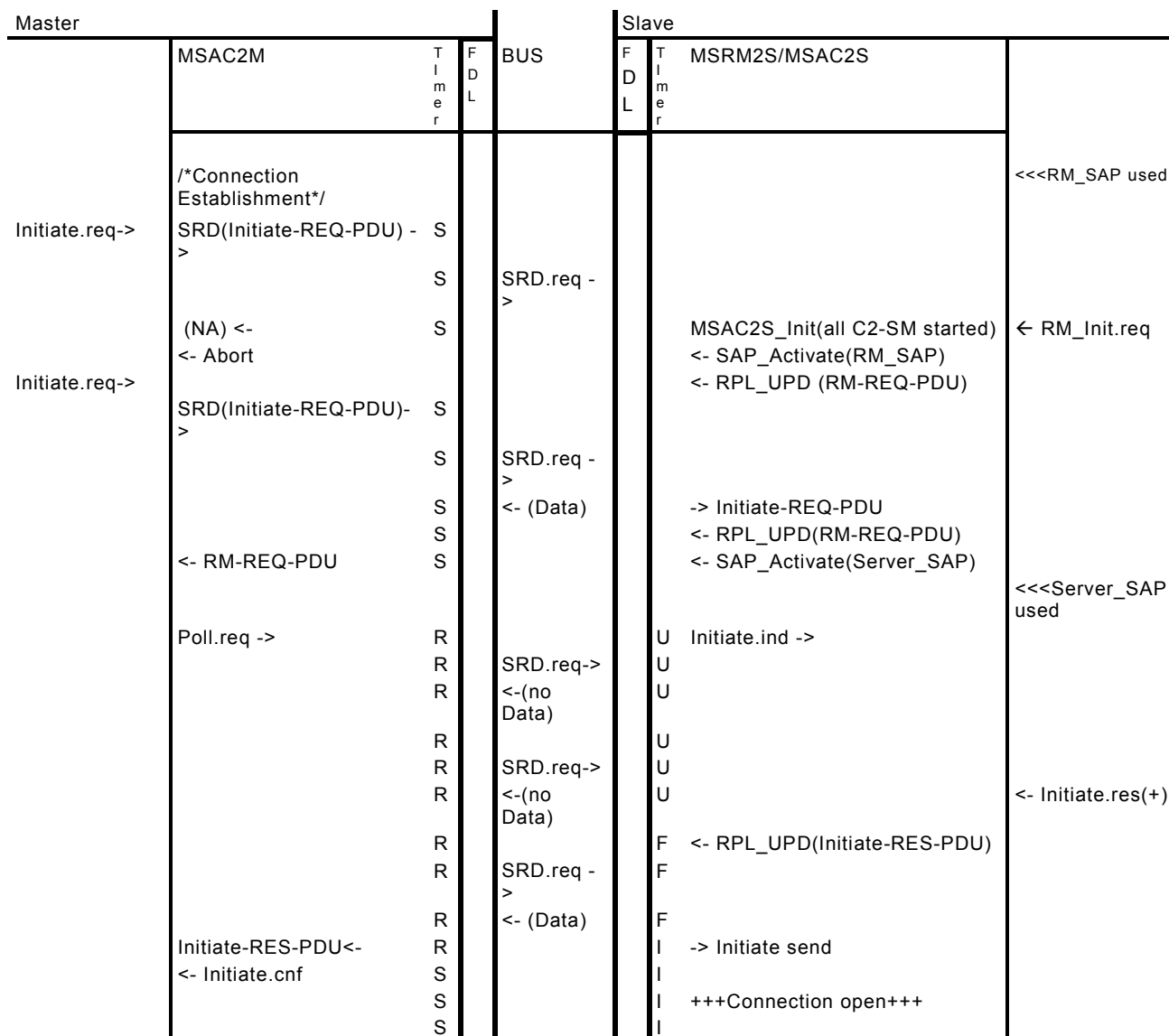


Figure 139 – Example for connection establishment on MS2

Phase a) Connection monitoring during pending MS2 service response or running Master-Idle:

The DP-master starts the R-Timer (receive timer) when the MS2 service request is transferred to the local DLL. The R-Timer monitors the remote MSAC2S and the local DLL.

The R-Timer is stopped if a response is received or restarted if a Slave-Idle request is received. The connection monitoring aborts the connection if a negative DL confirmation appears or the R-Timer expires.

The slave starts the U-Timer (user response timer) when a confirmed service indication is passed to the User. The U-Timer is stopped when the User provides the service response. If the U-Timer expires a Slave-Idle request is sent to retrigger the monitoring R-Timer of the Master.

The DP-slave starts the F-Timer (fetch response timer) when a service response or Slave-Idle request is passed to the local DLL. The F-Timer is stopped when the master has fetched the PDU. If the F-Timer expires the connection monitoring aborts the connection.

The connection monitoring of phase a) is shown in Figure 140.

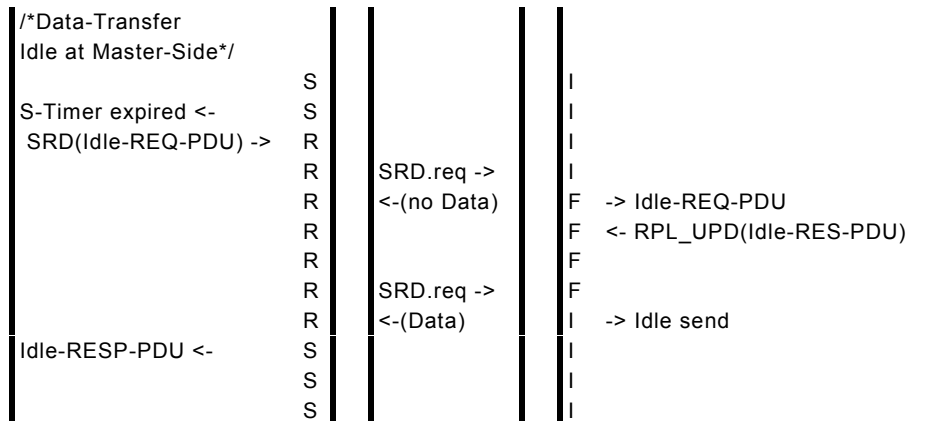


Figure 140 – Idle at master-side on MS2

Phase b) Connection monitoring without pending service response:

The bus master starts the S-Timer (send timer) if a response PDU or a Idle-PDU is received. The S-Timer is stopped if a service request is provided by the User. If the S-Timer expires a Master-Idle request is sent to stop the monitoring I-Timer of the Slave.

During pending Master-Idle responses the connection monitoring is handled as described in Phase a).

The Slave starts the I-Timer (indication timer) when the reply update buffer has been fetched by the Master. The I-Timer is stopped if a Master-Idle request or a service indication is received. If the I-Timer expires the connection monitoring aborts the connection.

The Master monitors the transmission of the Abort-REQ-PDU. The Master starts the S-Timer when the Abort.req is transferred to the local DLL. The S-Timer is stopped when the Abort.req is sent. If the S-Timer expires the connection is closed and the User is informed with a MSAC2M_Closed.ind.

The Slave monitors the fetching of the Abort-REQ-PDU by the Master. The Slave starts the F-Timer when the Abort.req is transferred to the local DL. The F-Timer is stopped if the Master has fetched the Abort-REQ-PDU. If the F-Timer expires the Server_SAP is deactivated and the connection is closed.

The connection monitoring of phase b) is shown in Figure 141.

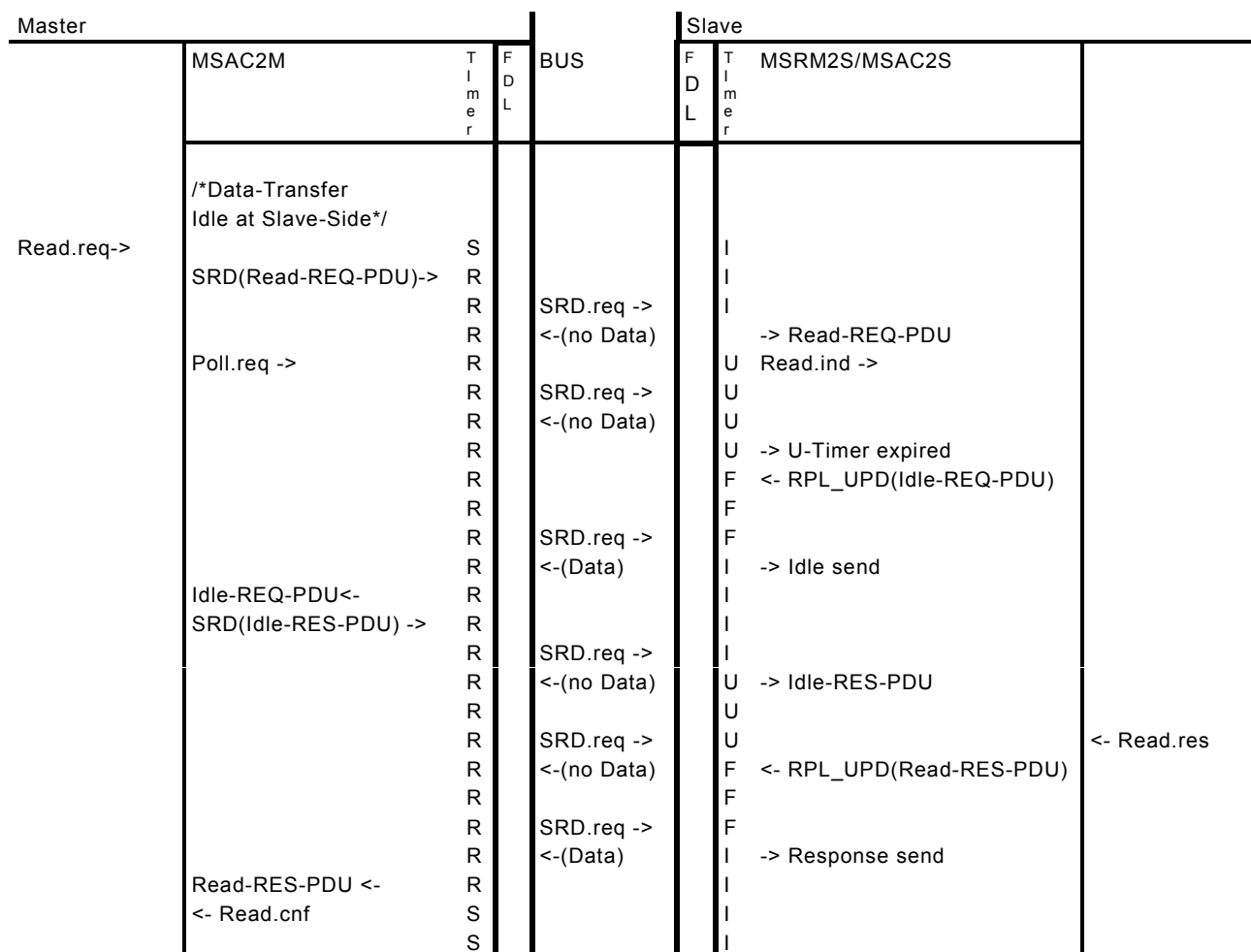


Figure 141 – Idle at slave-side on MS2

6.4 AP-Context state machine

There is no AP-Context State Machine defined for this Protocol.

6.5 FAL Service Protocol Machines (FSPMs)

6.5.1 FSPMS

6.5.1.1 Primitive definitions

6.5.1.1.1 Primitives exchanged between FSPMS and AP-Context

Table 424 shows the service primitives including their associated parameters issued by the AP-Context and received by the FSPMS.

Table 424 – Primitives issued by AP-Context to FSPMS

Primitive name	Source	Associated parameters	Functions
DLL Init DP slave.req	AP-Context	Bus Para	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
SInit MS0.req	AP-Context	AREP	
SInit MS1.req	AP-Context	AREP	
SInit MS2.req	AP-Context	AREP	
Reset DP slave.req	AP-Context	(none)	
Abort.req	AP-Context	AREP, Subnet, Instance, Reason Code	
DP slave Application Ready.req	AP-Context	AREP	
Check User Prm Result.req	AP-Context	AREP, Prm_OK	
Check Ext User Prm Result	AP-Context	AREP, Ext_Prm_OK	
Check Cfg Result.req	AP-Context	AREP, Cfg_OK, Input Data Len, Output Data Len	
Set Cfg.req	AP-Context	AREP, Cfg Data	
Set Slave Diag.req	AP-Context	AREP, Ext Diag Overflow, Ext Diag Flag, Ext Diag Data	
Set Input.req	AP-Context	AREP, Input Data	
Get Output.req	AP-Context	AREP	
Alarm Notification.req	AP-Context	AREP, Slot Number, Alarm Type, Seq Nr, Add Ack, Alarm Specifier, Alarm Data	
Initiate.rsp(+)	AP-Context	AREP, Max Len Data Unit, Features Supported, Profile Features Supported, Profile Ident Number, Add Addr Param	
Initiate.rsp(-)	AP-Context	AREP, Error Decode, Error Code 1 Error Code 2	
Read.rsp(+)	AP-Context	AREP, Length, Data	
Read.rsp(-)	AP-Context	AREP, Error Decode, Error Code 1 Error Code 2	
Write.rsp(+)	AP-Context	AREP, Length	
Write.rsp(-)	AP-Context	AREP, Error Decode, Error Code 1 Error Code 2	
Data Transport.rsp(+)	AP-Context	AREP, Length, Data	
Data Transport.rsp(-)	AP-Context	AREP, Error Decode, Error Code 1 Error Code 2	

Primitive name	Source	Associated parameters	Functions
Alarm Ack.rsp(+)	AP-Context	AREP, Slot Number, Alarm Type, Seq Nr	
Alarm Ack.rsp(-)	AP-Context	AREP, Slot Number, Alarm Type, Seq Nr	
Load ARL DP slave.req	AP-Context	Min Send Timeout, List of Resource Manager Entries, List of S_SAP_indices, List of ARL Entries	
Get ARL DP slave.req	AP-Context	(none)	
Load CRL DP slave.req	AP-Context	List of CRL Entries	
Get CRL DP slave.req	AP-Context	(none)	
Set ARL Isochronous Mode.req	AP-Context	Isochronous Mode	
Get Publisher Data.req	AP-Context	AREP, CREP	
Start Subscriber.req	AP-Context	AREP, CREP	
Stop Subscriber.req	AP-Context	AREP, CREP	
Load CRL DXB-Linktable Entries.req	AP-Context	AREP, DXB-Linktable	
Initiate Load.rsp(+)	AP-Context	AREP, Actual LR Size, Max Response Delay, Max Segment Length User Specific	
Initiate Load.rsp(-)	AP-Context	AREP, Error Code	
Pull Segment.rsp(+)	AP-Context	AREP, Segment Length, Segment Number, More Follows, Data	
Pull Segment.rsp(-)	AP-Context	AREP, Error Code	
Push Segment.rsp(+)	AP-Context	AREP	
Push Segment.rsp(-)	AP-Context	AREP, Error Code	
Terminate Load.rsp(+)	AP-Context	AREP	
Terminate Load.rsp(-)	AP-Context	AREP, Error Code	
Start.rsp(+)	AP-Context	AREP	
Start.rsp(-)	AP-Context	AREP Error Code	
Stop.rsp(+)	AP-Context	AREP	
Stop.rsp(-)	AP-Context	AREP Error Code	
Resume.rsp(+)	AP-Context	AREP	
Resume.rsp(-)	AP-Context	AREP Error Code	
Reset.rsp(+)	AP-Context	AREP	
Reset.rsp(-)	AP-Context	AREP Error Code	
Call.rsp(+)	AP-Context	AREP Result Argument	
Call.rsp(-)	AP-Context	AREP Error Code	
Get FI State.rsp(+)	AP-Context	AREP FI State	
Get FI State.rsp(-)	AP-Context	AREP Error Code	

Table 425 shows the service primitives including their associated parameters issued by the FSPMS and received by the AP-Context.

Table 425 – Primitives issued by FSPMS to AP-Context

Primitive name	Source	Associated parameters	Functions
Sinit DLL.cnf	FSPMS	(none)	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Sinit MS0.cnf	FSPMS	AREP	
Sinit MS1.cnf	FSPMS	AREP	
Sinit MS2.cnf	FSPMS	AREP	
Reset.cnf	FSPMS	(none)	
Check User Prm Result.cnf(+)	FSPMS	AREP	
Check User Prm Result.cnf(-)	FSPMS	AREP, Status	
Check Ext User Prm Result.cnf(+)	FSPMS	AREP	
Check Ext User Prm Result.cnf(-)	FSPMS	AREP, Status	
Check Cfg Result.cnf(+)	FSPMS	AREP	
Check Cfg Result.cnf(-)	FSPMS	AREP, Status	
Set Cfg.cnf(+)	FSPMS	AREP	
Set Cfg.cnf(-)	FSPMS	AREP	
Set Slave Diag.cnf	FSPMS	AREP	
Set Input.cnf(+)	FSPMS	AREP	
Set Input.cnf(-)	FSPMS	AREP	
Get Output.cnf(+)	FSPMS	AREP, Output Data, Clear Flag, New Flag	
Get Output.cnf(-)	FSPMS	AREP	
Started.ind	FSPMS	AREP, Actual Enabled Alarms, Alarm Sequence, Alarm Limit	
Stopped.ind	FSPMS	AREP	
Abort.ind	FSPMS	AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail	
Fault.ind	FSPMS	AREP	
Alarm Reject.ind	FSPMS	AREP, Slot Number, Alarm Type, Seq Nr	
Set Slave Add.ind	FSPMS	AREP, New Slave Add, Ident Number, No Add Chg, Rem Slave Data	
Check User Prm.ind	FSPMS	AREP, User Prm Data	
Check Ext User Prm.ind	FSPMS	AREP, Ext User Prm Data	
Check Cfg.ind	FSPMS	AREP, Cfg Data	
New Output.ind	FSPMS	AREP, Output Data, Clear Flag	
Global Control.ind	FSPMS	AREP, Clear Command, Sync Command, Freeze Command, Group Select	

Table 425 (continued)

Primitive name	Source	Associated parameters	Functions
Initiate.ind	FSPMS	AREP, Features Supported, Profile Features Supported, Profile Ident Number, Add Addr Param	
Read.ind	FSPMS	AREP, Slot Number, Index, Length	
Write.ind	FSPMS	AREP, Slot Number, Index, Length, Data	
Data Transport.ind	FSPMS	AREP, Slot Number, Index, Length, Data	
Alarm Ack.ind	FSPMS	AREP, Slot Number, Alarm Type, Seq Nr	
Load ARL DP slave.cnf(+)	FSPMS	(none)	
Load ARL DP slave.cnf(-)	FSPMS	Status	
Get ARL DP slave.cnf(+)	FSPMS	Min Send Timeout, List of Resource Manager Entries, List of S_SAP_indices, List of ARL Entries	
Get ARL DP slave.cnf(-)	FSPMS	Status	
Load CRL DP slave.cnf(+)	FSPMS	(none)	
Load CRL DP slave.cnf(-)	FSPMS	Status	
Get CRL DP slave.cnf(+)	FSPMS	List of CRL Entries	
Get CRL DP slave.cnf(-)	FSPMS	Status	
Set ARL Isochronous Mode.cnf(+)	FSPMS		
Set ARL Isochronous Mode.cnf(-)	FSPMS	Status	
SYNCH Event.ind	FSPMS	AREP, Status	
Publisher Active.ind	FSPMS	AREP, CREP, Status	
New Publisher Data.ind	FSPMS	AREP, CREP	
Get Publisher Data.cnf(+)	FSPMS	AREP, CREP, Data, New Flag	
Get Publisher Data.cnf(-)	FSPMS	AREP, CREP	
Start Subscriber.cnf(+)	FSPMS	AREP, CREP	
Start Subscriber.cnf(-)	FSPMS	AREP, CREP	
Stop Subscriber.cnf(+)	FSPMS	AREP, CREP	
Stop Subscriber.cnf(-)	FSPMS	AREP, CREP	
Load CRL DXB-Linktable Entries.cnf(+)	FSPMS	AREP	
Load CRL DXB-Linktable Entries.cnf(-)	FSPMS	AREP	

Table 425 (continued)

Primitive name	Source	Associated parameters	Functions
Set Time.ind	FSPMS	AREP, Time Value, Local Time Diff, Summertime, Accuracy, Synchronisation Active, Announcement Hour	
Sync Interval Violation.ind	FSPMS	AREP	
Initiate Load.ind	FSPMS	AREP, Slot Number, LR Index, Load Type, Load Image Size, Intersegment Request Timeout User Specific	
Pull Segment.ind	FSPMS	AREP, Slot Number, LR Index, Segment Length	
Push Segment.ind	FSPMS	AREP, Slot Number LR Index, Segment Length Segment Number, More Follows, Data	
Terminate Load.ind	FSPMS	AREP Slot Number LR Index	
Start.ind	FSPMS	AREP Slot Number FI Index Execution Argument	
Stop.ind	FSPMS	AREP Slot Number FI Index	
Resume.ind	FSPMS	AREP Slot Number FI Index Execution Argument	
Reset.ind	FSPMS	AREP Slot Number FI Index	
Call.ind	FSPMS	AREP Slot Number FI Index Execution Argument	
Get FI State	FSPMS	AREP Slot Number FI Index	

6.5.1.1.2 Parameters of FSPMS primitives

The parameters used with the primitives exchanged between the FSPMS and the AP-Context are described in FAL Service Definition Type 3 Fieldbus IEC 61158-5.

6.5.1.2 State machine description

This Machine is used to co-ordinate the Interactions between AP-Context and DMPMS, MSCY1S, MSAC1S, MSRM2S and MSAC2S. As Alarms requires the support of several State Machines the alarms are handled by this machine.

With the SInit_MS1.req, the local variables of the State Machines are set to the start-up values and MSAC1 State Machines is initialised.

When the MSCY1S enters the Data-Exchange-State (This is signalled by a start.ind) alarm messages can be generated by the User.

The parallel processing of alarm messages is limited by:

the number of Alarm_Classes (Alarm_Sequence = False)

the number of Alarms accepted by the Master (Alarm_Sequence = True)

the number of Alarms handled by this Slave (Alarm_Sequence=True).

Alarms may be processed only if the appropriate Alarm_Class is enabled by the DP-master. An Alarm issued by an Alarm_Notification will be transmitted via a diagnosis (MSCY1S) to a DP-master (Class 1). The Acknowledgement will be received by a Alarm Ack service processed by MSAC1S.

The State Machine needs local variables which are described below. Furthermore the machine uses functions and macros which are listed in 6.5.1.4.

Local variables

Alarm_Sequence

(Boolean)

This variable indicates whether

only one alarm of a specific Alarm_Class can be active at one time (Alarm_Sequence = False) or

several alarms (2 to 32) of any type can be active at one time (Alarm_Sequence=True).

Initial_Alarm_Sequence

(Boolean)

This variable stores the basic ability of the Slave.

Outstanding_Alarm_TABLE

(Array 0...7, 0...31 of Alarm_Status, Alarm_Type)

Alarm_Status: pending, not_pending

Alarm_Type: 1 to 6, 32 to 126

The Alarm_State_Table is a two dimensional array with 7 * 32 elements. Each element within the array stores information about the actual state of any alarm. The array is addressed with the Alarm_Class calculated from the Alarm_Type and the Seq_Nr of the alarm.

Alarm_Max

(Unsigned8)

This variable indicates the maximum number of parallel alarms of this Slave.

Range: 1..32

Alarm_Limit

(Unsigned8)

This variable contains the maximum number of Alarms allowed by the actual Master-Slave-connection.

Range: 1..32

Alarm_Decode

(Array 0...7 of Unsigned8)

Decoding the number of parallel alarms:

Alarm_Decode[0]= 1

Alarm_Decode[1]= 2

Alarm_Decode[2]= 4

Alarm_Decode[3]= 8

Alarm_Decode[4]=12

Alarm_Decode[5]=16

Alarm_Decode[6]=24

Alarm_Decode[7]=32

Alarm_Count

(Unsigned8)

This counter contains the number of alarms (0..32) which have actually been sent by the DP-slave. The counter is only used in the sequence mode.

Range: 0 to Alarm_Limit

Req_Alarm_FIFO

The Req_Alarm_FIFO is used to store the Alarms which are still to be send. Each element consists of an Alarm-PDU. The FIFO shall be able to hold up to 32 respectively Alarm_Max entries.

Actual_Enabled_Alarms

(Array of Bits)

This variable indicates the type of alarms which are actually supported by the Master.

Bit 0 = reserved

Bit 1 = reserved

Bit 2 = Update_Alarm

Bit 3 = Status_Alarm

Bit 4 = Manufacturer_Specific_Alarm

Bit 5 = Diagnostic_Alarm

Bit 6 = Process_Alarm

Bit 7 = Pull_Plug_Alarm

Alarms_Supported

(Array of Bits)

This variable indicates the type of alarms which are supported by the Slave.

Bit 0 = reserved

Bit 1 = reserved

Bit 2 = Update_Alarm

Bit 3 = Status_Alarm

Bit 4 = Manufacturer_Specific_Alarm

Bit 5 = Diagnostic_Alarm

Bit 6 = Process_Alarm

Bit 7 = Pull_Plug_Alarm

Local_Diag_Buffer

(Octet-String)

This local variable is used to store the Diag-Data except the first 6 Octets and the Alarm- and Status-PDUs.

L_Ext_Diag_Flag

(Bit)

This local variable is used to store the Ext_Diag_Flag.

Stored_Diag

(Boolean)

Flag which indicates, that new Diag_Data are stored in the Diag_Buffer while waiting for transmission of a previous Diagnosis.

Diag_Buffer

(Octet-String)

Buffer for storing a Slave-Diagnostic(Identification-Related-Diagnosis, Channel-Related-Diagnosis, Revision_Number)

Index

(Structure of Alarm_Class, Seq_Nr)

This index is used for addressing outstanding Alarm_Table.

LR_State

(Unsigned8)

This variable stores the state of the Load Region sequence.

CurrentBuffer

(Array of structure Empty, WriteLength and Data)

This buffer stores APDUs for Load Region for each AREP.

CurrentLoadType

(Unsigned8)

This variable stores whether a Pull or a Push sequence is active.

CurrentLRIndex

(Unsigned16)

This variable stores the LR type.

6.5.1.3 FSPMS state table

Table 426 contains the complete description of the FSPMS state machine.

Processing constrains for isochronous mode:

The time from Issuing the SYNCH.ind until the Service Requests from the MSCY1S to the DL shall be small enough to allow the MAC to send out all requests from MSCY1S within a cycle.

The following definition for a state set is used in Table 426.

t_state = W-START or W-DIA-UPD or W-FETCHED

Table 426 – FSPMS state table

#	Current State	Event /Condition =>Action	Next State
1	POWER-ON	Load ARL DP Slave.req (Min Send Timeout, List of Resource Manager Entries, List of S_SAP_indices, List of ARL Entries) /valid parameters => Load ARL DP Slave.cnf(+)	POWER-ON
2	POWER-ON	Load ARL DP Slave.req (Min Send Timeout, List of Resource Manager Entries, List of S_SAP_indices, List of ARL Entries) /invalid parameters => Status := NO Load ARL DP Slave.cnf(-) (Status)	POWER-ON
3	POWER-ON	Get ARL DP Slave.req /ARL loaded => Get ARL DP Slave.cnf(+) (Min Send Timeout, List of Resource Manager Entries, List of S_SAP_indices, List of ARL Entries)	POWER-ON
4	POWER-ON	Get ARL DP Slave.req /ARL not loaded => Status := NO Get ARL DP Slave.cnf(-) (Status)	POWER-ON
5	POWER-ON	Load CRL DP Slave.req (List of CRL Entries) /valid parameters => Load CRL DP Slave.cnf(+)	POWER-ON
6	POWER-ON	Load CRL DP Slave.req (List of CRL Entries) /invalid parameters => Status := NO Load CRL DP Slave.cnf(-) (Status)	POWER-ON
7	POWER-ON	Get CRL DP Slave.req /CRL loaded => Get CRL DP Slave.cnf(+) (List of CRL Entries)	POWER-ON
8	POWER-ON	Get CRL DP Slave.req /CRL not loaded => Status := NO Get CRL DP Slave.cnf(-) (Status)	POWER-ON
9	POWER-ON	FSPMS_SInit_MS1.req (AREP) /Alarm_Mode_Slave <> 0 => Alarm_Sequence := True Initial_Alarm_Sequence := Alarm_Sequence Alarm_Max := Alarm_Decompose[Alarm_Mode_Slave] Alarm_Max := max(Alarm_Max, 7) Local_Diag_Buffer := Default_Ext_Diag_Data L_Ext_Diag_Flag := Default_Ext_Diag FILL (Outstanding_Alarm_TABLE.Alarm_Status, not_pending) Stored_Diag:=False Reset_Req_Alarm_FIFO() Act_Ref := 0 MSAC1_SInit_MS1.req	W-START-C1

Table 426 (continued)

#	Current State	Event /Condition =>Action	Next State
11	W-START-C1	MSAC1S_SInit_MS1.cnf => FSPMS_SInit_MS1.cnf(AREP)	W-START
12	W-START	FSPMS_Abort.req(AREP) /AREP.AR_Type=MS0 => CurrentBuffer[AREP].Empty:=TRUE LR_State[AREP]:=W-LR-IND MSCY1S_Abort.req	W-START
13	W-START	FSPMS_Alarm_Notification.req(AREP, Slot_Number, Alarm_Type, Seq_Nr, Add_Ack, Alarm_Specifier, Alarm_Data) => FSPMS_Alarm_Notification.cnf(-)(AREP, Alarm_Type, Slot_Number, Seq_Nr, Status=No_Start)	W-START
14	W-START	FSPMS_Set_Slave_Diag.req(AREP,Ext_Diag_Flag, Ext_Diag) => L_Ext_Diag_Flag := Ext_Diag_Flag L_Ext_Diag_Overflow := Ext_Diag_Overflow Local_Diag_Buffer := Ext_Diag Act_Ref := Act_Ref + 1 MSCY1S_Set_Slave_Diag.req(Ext_Diag_Flag, Ext_Diag_Overflow, Ext_Diag, Reference := Act_Ref) FSPMS_Set_Slave_Diag.cnf(AREP)	W-START
15	W-START	MSAC1S_Alarm_Ack.ind(Slot_Number, Alarm_Type, Seq_Nr) => MSCY1S_Abort.req FSPMS_Abort.ind(AREP)	W-START
16	W-START	MSCY1S_Set_Slave_Diag.cnf(-) (Reference) => ignore	W-START
17	W-START	MSCY1S_Set_Slave_Diag.cnf(+) (Reference) => ignore	W-START
18	W-START	MSCY1S_Start.ind(Enabled_Alarms, Alarm_Mode_Master, Diag_Flag) /((Enabled Alarms & Alarms_Supported) = 0 => FSPMS DP Slave Started.ind(AREP,Actual_Enabled_Alarms, Alarm_Sequence, Alarm_Limit)	W-START
19	W-START	MSCY1S_Start.ind(Enabled_Alarms, Alarm_Mode_Master, Diag_Flag) /((Enabled Alarms & Alarms_Supported) = 0 => FSPMS DP Slave Started.ind(AREP,Actual_Enabled_Alarms, Alarm_Sequence, Alarm_Limit)	W-FETCHED
20	W-START	MSCY1S_Start.ind(Enabled_Alarms, Alarm_Mode_Master, Diag_Flag) / ((Enabled Alarms & Alarms_Supported) <> 0) && Alarm_Mode_Master <> 0 && Initial_Alarm_Sequence = TRUE && Diag_Flag=FALSE => Alarm_Sequence := True Actual_Enabled_Alarms := Enabled_Alarms Alarm_Limit := min(Alarm_Max, Alarm_Decode[Alarm_Mode_Master]) Alarm_Count := 0 FSPMS DP Slave Started.ind(AREP,Actual_Enabled_Alarms, Alarm_Sequence, Alarm_Limit)	W-DIA-UPD
21	W-START	MSCY1S_Start.ind(Enabled_Alarms, Alarm_Mode_Master, Diag_Flag) / ((Enabled Alarms & Alarms_Supported) <> 0) && Alarm_Mode_Master <> 0 && Initial_Alarm_Sequence =FALSE => MSCY1S_Abort.req	W-START
22	W-START	MSCY1S_Start.ind(Enabled_Alarms, Alarm_Mode_Master, Diag_Flag) / ((Enabled Alarms & Alarms_Supported) <> 0) && Alarm_Mode_Master = 0 && Diag_Flag=TRUE => Alarm_Sequence := False Actual_Enabled_Alarms := Enabled_Alarms FSPMS DP Slave Started.ind(AREP,Actual_Enabled_Alarms, Alarm_Sequence, Alarm_Limit)	W-FETCHED

Table 426 (continued)

#	Current State	Event /Condition =>Action	Next State
23	W-START	MSCY1S_Start.ind(Enabled_Alarms, Alarm_Mode_Master, Diag_Flag) / ((Enabled_Alarms & Alarms_Supported) <> 0) && Alarm_Mode_Master = 0 && Diag_Flag=FALSE => Alarm_Sequence := False Actual_Enabled_Alarms := Enabled_Alarms FSPMS_DP_Slave_Started.ind(AREP,Actual_Enabled_Alarms, Alarm_Sequence, Alarm_Limit)	W-DIA- UPD
24	W-START	MSCY1S_Stop.ind => FSPMS_Stopped.ind(AREP)	W-START
25	W-DIA- UPD	FSPMS_Abort.req(AREP) /AREP.AR_Type=MS0 => Index := Search_In_Outstanding_Alarm_TABLE(Alarm_Status:=pending) CurrentBuffer[AREP].Empty:=TRUE LR_State[AREP]:=W-LR-IND MSCY1S_Abort.req	RESET-P- LEAVE
26	W-DIA- UPD	FSPMS_Alarm_Notification.req(AREP, Slot_Number, Alarm_Type, Seq_Nr, Add_Ack, Alarm_Specifier, Alarm_Data) /ALARM_ENABLED=FALSE => FSPMS_Alarm_Notification.cnf(-)(AREP,Alarm_Type, Slot_Number, Seq_Nr, Status=Not_Enabled)	W-DIA- UPD
27	W-DIA- UPD	FSPMS_Alarm_Notification.req(AREP, Slot_Number, Alarm_Type, Seq_Nr, Add_Ack, Alarm_Specifier, Alarm_Data) /ALARM_ENABLED=TRUE && Alarm_Sequence=TRUE && Alarm_Count=Alarm_Limit => FSPMS_Alarm_Notification.cnf(-)(AREP, Alarm_Type, Slot_Number, Seq_Nr, Status=Limit_Expired)	W-DIA- UPD
28	W-DIA- UPD	FSPMS_Alarm_Notification.req(AREP, Slot_Number, Alarm_Type, Seq_Nr, Add_Ack, Alarm_Specifier, Alarm_Data) /ALARM_ENABLED=TRUE && Alarm_Sequence=TRUE && Alarm_Count<Alarm_Limit && Outstanding_Alarm_TABLE.Alarm_Status[Acls(Alarm_Type),Seq_Nr]=not_pending => Outstanding_Alarm_TABLE.Alarm_Status[Acls(Alarm_Type),Seq_Nr]:=pending Outstanding_Alarm_TABLE.Alarm_Type[Acls(Alarm_Type),Seq_Nr]:=Alarm_Type Alarm_Count:=Alarm_Count+1 Ext_Diag_Flag:=L_Ext_Diag_Flag Ext_Diag_Overflow:=L_Ext_Diag_Overflow Ext_Diag:=(Local_Diag_Buffer, Alarm(Slot_Number,Alarm_Type, Seq_Nr, Add_Ack, Alarm_Specifier, Alarm_Data)) Act_Ref := Act_Ref + 1 MSCY1S_Set_Slave_Diag.req(Ext_Diag_Flag, Ext_Diag_Overflow, Ext_Diag, Reference := Act_Ref) FSPMS_Alarm_Notification.cnf(+)(AREP,Alarm_Type, Slot_Number, Seq_Nr)	W- FETCHED
29	W-DIA- UPD	FSPMS_Alarm_Notification.req(AREP, Slot_Number, Alarm_Type, Seq_Nr, Add_Ack, Alarm_Specifier, Alarm_Data) /ALARM_ENABLED=TRUE && Alarm_Sequence=TRUE && Alarm_Count<Alarm_Limit && Outstanding_Alarm_TABLE.Alarm_Status[Acls(Alarm_Type),Seq_Nr]= pending => FSPMS_Alarm_Notification.cnf(-)(AREP,Alarm_Type, Slot_Number, Seq_Nr, Status=Sequence_Nr_Pending)	W-DIA- UPD
30	W-DIA- UPD	FSPMS_Alarm_Notification.req(AREP, Slot_Number, Alarm_Type, Seq_Nr, Add_Ack, Alarm_Specifier, Alarm_Data) /ALARM_ENABLED=TRUE && Alarm_Sequence=FALSE && Outstanding_Alarm_TABLE.Alarm_Status[Acls(Alarm_Type),Seq_Nr]= pending => FSPMS_Alarm_Notification.cnf(-)(AREP,Alarm_Type, Slot_Number, Seq_Nr, Status=Sequence_Nr_Pending)	W-DIA- UPD

Table 426 (continued)

#	Current State	Event /Condition =>Action	Next State
31	W-DIA-UPD	FSPMS_Alarm_Notification.req(AREP, Slot_Number, Alarm_Type, Seq_Nr, Add_Ack, Alarm_Specifier, Alarm_Data) /ALARM_ENABLED=TRUE && Alarm_Sequence=FALSE && Outstanding_Alarm_TABLE.Alarm_Status[Acls(Alarm_Type),Seq_Nr]=not_pending => Outstanding_Alarm_TABLE.Alarm_Status[Acls(Alarm_Type),Seq_Nr]:=pending Outstanding_Alarm_TABLE.Alarm_Type[Acls(Alarm_Type),Seq_Nr]:=Alarm_Type Ext_Diag_Flag:=L_Ext_Diag_Flag Ext_Diag_Overflow:=L_Ext_Diag_Overflow Ext_Diag:=(Local_Diag_Buffer, Alarm(Slot_Number,Alarm_Type, Seq_Nr, Add_Ack, Alarm_Specifier, Alarm_Data)) Act_Ref := Act_Ref + 1 MSCY1S_Set_Slave_Diag.req(Ext_Diag_Flag, Ext_Diag_Overflow, Ext_Diag, Reference := Act_Ref) FSPMS_Alarm_Notification.cnf(+)(AREP,Alarm_Type, Slot_Number, Seq_Nr)	W-FETCHED
32	W-DIA-UPD	FSPMS_Set_Slave_Diag.req(AREP,Ext_Diag_Flag, Ext_Diag) => if (PrmCmdAck in Ext_Diag or Stored_PrmCmdAck != NULL) Stored_PrmCmdAck := PrmCmdAck, Ext_Diag.Red_Status := PrmCmdAck endif L_Ext_Diag_Flag := Ext_Diag_Flag L_Ext_Diag_Overflow := Ext_Diag_Overflow Local_Diag_Buffer := Ext_Diag Act_Ref := Act_Ref + 1 MSCY1S_Set_Slave_Diag.req(Ext_Diag_Flag, Ext_Diag_Overflow, Ext_Diag, Reference := Act_Ref) FSPMS_Set_Slave_Diag.cnf(AREP)	W-DIA-UPD
33	W-DIA-UPD	MSAC1S_Alarm_Ack.ind(Slot_Number, Alarm_Type, Seq_Nr) /Alarm_Sequence=TRUE && Outstanding_Alarm_TABLE.Alarm_Status[Acls(Alarm_Type),Seq_Nr]=pending => Outstanding_Alarm_TABLE.Alarm_Status[Acls(Alarm_Type),Seq_Nr] := not_pending Alarm_Count:=Alarm_Count-1 FSPMS_Alarm_Ack.ind(Slot_Number, Alarm_Type, Seq_Nr) MSAC1S_Alarm_Ack.rsp(+) (Slot Number, Alarm Type, Seq Nr)	W-DIA-UPD
34	W-DIA-UPD	MSAC1S_Alarm_Ack.ind(Slot_Number, Alarm_Type, Seq_Nr) /Alarm_Sequence=FALSE && Outstanding_Alarm_TABLE.Alarm_Status[Acls(Alarm_Type),Seq_Nr] =pending => Outstanding_Alarm_TABLE.Alarm_Status[Acls(Alarm_Type),Seq_Nr] := not_pending FSPMS_Alarm_Ack.ind(Slot_Number, Alarm_Type, Seq_Nr) MSAC1S_Alarm_Ack.rsp(+) (Slot Number, Alarm Type, Seq Nr)	W-DIA-UPD
35	W-DIA-UPD	MSAC1S_Alarm_Ack.ind(Slot_Number, Alarm_Type, Seq_Nr) /Outstanding_Alarm_TABLE.Alarm_Status[Acls(Alarm_Type),Seq_Nr] =not_pending => Index := Search_In_Outstanding_Alarm_TABLE(Alarm_Status:=pending) MSCY1S_Abort.req FSPMS_Abort.ind(AREP)	RESET-P-LEAVE
36	W-DIA-UPD	MSCY1S_Set_Slave_Diag.cnf(-) (Reference) => R_Cnt:=0 FSPMS_DP_Slave_Fault.ind DMPMS_Reset.req MSCY1S_Reset.req MSAC1S_Reset.req MSRM2S_Reset.req	W-RESET
37	W-DIA-UPD	MSCY1S_Set_Slave_Diag.cnf(+) (Reference) /Reference < Act_Ref => ignore	W-DIA-UPD
38	W-DIA-UPD	MSCY1S_Set_Slave_Diag.cnf(+) (Reference) /Reference >= Act_Ref => Stored_PrmCmdAck := NULL	W-DIA-UPD

Table 426 (continued)

#	Current State	Event /Condition =>Action	Next State
39	W-DIA-UPD	MSCY1S_Start.ind(Enabled_Alarms, Alarm_Mode_Master, Diag_Flag) => MSCY1S_Abort.req	W-START
40	W-DIA-UPD	MSCY1S_Stop.ind => Alarm_Count:=0 Index := Search_In_Outstanding_Alarm_TABLE(Alarm_Status:=pending)	RESET-PENDING
41	RESET-PENDING	/Index.Seq_Nr <> 255 => Outstanding_Alarm_TABLE.Alarm_Status[Index] := not_pending Alarm_Type := Outstanding_Alarm_TABLE.Alarm_Type[Acls(Alarm_Type),Seq_Nr] Index := Search_In_Outstanding_Alarm_TABLE(Alarm_Status:=pending) FSPMS_Alarm_Notification.cnf(-)(AREP,Alarm_Type, Slot_Number=0, Index.Seq_Nr, Status=MSAL1S_Stopped)	RESET-PENDING
42	RESET-PENDING	/Index.Seq_Nr = 255 => Reset_Req_Alarm_FIFO() FSPMS DP Slave Stopped.ind(AREP)	W-START
43	RESET-P-LEAVE	/Index.Seq_Nr <> 255 => Outstanding_Alarm_TABLE.Alarm_Status[Index] := not_pending Alarm_Type:= Outstanding_Alarm_TABLE.Alarm_Type[Acls(Alarm_Type),Seq_Nr] Index := Search_In_Outstanding_Alarm_TABLE(Alarm_Status:=pending) FSPMS_Alarm_Notification.cnf(-)(AREP,Alarm_Type, Slot_Number=0, Index.Seq_Nr, Status=Stopped)	RESET-P-LEAVE
44	RESET-P-LEAVE	/Index.Seq_Nr = 255 => Reset_Req_Alarm_FIFO()	W-START
45	W-FETCHED	FSPMS_Abort.req(AREP) /AREP.AR_Type=MS0 && Alarm_Sequence=TRUE && Alarm_Count=0 => Stored_Diag:=False Ext_Diag_Flag:=L_Ext_Diag_Flag Ext_Diag_Overflow:=L_Ext_Diag_Overflow Ext_Diag:=(Local_Diag_Buffer)Act_Ref := Act_Ref + 1 CurrentBuffer[AREP].Empty:=TRUE LR_State[AREP]:=W-LR-IND MSCY1S_Set_Slave_Diag.req(Ext_Diag_Flag, Ext_Diag_Overflow, Ext_Diag, Reference := Act_Ref) MSCY1S_Abort.req	W-START
46	W-FETCHED	FSPMS_Abort.req(AREP) /AREP.AR_Type=MS0 && Alarm_Sequence=FALSE Alarm_Count<>0 => Stored_Diag:=False Ext_Diag_Flag:=L_Ext_Diag_Flag Ext_Diag_Overflow:=L_Ext_Diag_Overflow Ext_Diag:=(Local_Diag_Buffer) Index := Search_In_Outstanding_Alarm_TABLE(Alarm_Status:=pending) Act_Ref := Act_Ref + 1 CurrentBuffer[AREP].Empty:=TRUE LR_State[AREP]:=W-LR-IND MSCY1S_Set_Slave_Diag.req(Ext_Diag_Flag, Ext_Diag_Overflow, Ext_Diag, Reference := Act_Ref) MSCY1S_Abort.req	RESET-P-LEAVE
47	W-FETCHED	FSPMS_Alarm_Notification.req(AREP, Slot_Number, Alarm_Type, Seq_Nr, Add_Ack, Alarm_Specifier, Alarm_Data) /ALARM_ENABLED=FALSE => FSPMS_Alarm_Notification.cnf(-)(AREP,Alarm_Type, Slot_Number, Seq_Nr, Status=Not_Enabled)	W-FETCHED

Table 426 (continued)

#	Current State	Event /Condition =>Action	Next State
48	W-FETCHED	FSPMS_Alarm_Notification.req(AREP, Slot_Number, Alarm_Type, Seq_Nr, Add_Ack, Alarm_Specifier, Alarm_Data) /ALARM_ENABLED=TRUE && Alarm_Sequence=TRUE && Alarm_Count=Alarm_Limit => FSPMS_Alarm_Notification.cnf(-)(AREP,Alarm_Type, Slot_Number, Seq_Nr, Status=Limit_Expired)	W-FETCHED
49	W-FETCHED	FSPMS_Alarm_Notification.req(AREP, Slot_Number, Alarm_Type, Seq_Nr, Add_Ack, Alarm_Specifier, Alarm_Data) /ALARM_ENABLED=TRUE && Alarm_Sequence=TRUE && Alarm_Count<Alarm_Limit && Outstanding_Alarm_TABLE.Alarm_Status[AclsAlarm_Type),Seq_Nr]=not_pending => Outstanding_Alarm_TABLE.Alarm_Status[Acls(Alarm_Type),Seq_Nr]:=pending Alarm_Count:=Alarm_Count+1 Store_to_Req_Alarm_FIFO(Alarm(Slot_Number, Alarm_Type, Seq_Nr, Add_Ack, Alarm_Specifier, Alarm_Data)) FSPMS_Alarm_Notification.cnf(+)(AREP,Alarm_Type, Slot_Number, Seq_Nr)	W-FETCHED
50	W-FETCHED	FSPMS_Alarm_Notification.req(AREP, Slot_Number, Alarm_Type, Seq_Nr, Add_Ack, Alarm_Specifier, Alarm_Data) /ALARM_ENABLED=TRUE && Alarm_Sequence=TRUE && Alarm_Count<Alarm_Limit&& Outstanding_Alarm_TABLE.Alarm_Status[Acls(Alarm_Type),Seq_Nr]= pending => FSPMS_Alarm_Notification.cnf(-)(AREP,Alarm_Type, Slot_Number, Seq_Nr, Status=Sequence_Nr_Pending)	W-FETCHED
51	W-FETCHED	FSPMS_Alarm_Notification.req(AREP, Slot_Number, Alarm_Type, Seq_Nr, Add_Ack, Alarm_Specifier, Alarm_Data) /ALARM_ENABLED=TRUE && Alarm_Sequence=FALSE && Outstanding_Alarm_TABLE.Alarm_Status[Acls(Alarm_Type),Seq_Nr]= pending => FSPMS_Alarm_Notification.cnf(-)(AREP, Alarm_Type, Slot_Number, Seq_Nr, Status=Sequence_Nr_Pending)	W-FETCHED
52	W-FETCHED	FSPMS_Alarm_Notification.req(AREP, Slot_Number, Alarm_Type, Seq_Nr, Add_Ack, Alarm_Specifier, Alarm_Data) /ALARM_ENABLED=TRUE && Alarm_Sequence=FALSE && Outstanding_Alarm_TABLE.Alarm_Status[AclsL(Alarm_Type),Seq_Nr]=not_pending => Outstanding_Alarm_TABLE.Alarm_Status[AclsL(Alarm_Type),Seq_Nr] := pending Outstanding_Alarm_TABLE.Alarm_Type[Acls(Alarm_Type),Seq_Nr] := Alarm_Type Store_to_Req_Alarm_FIFO(Alarm(Slot_Number, Alarm_Type, Seq_Nr, Add_Ack, Alarm_Specifier, Alarm_Data)) FSPMS_Alarm_Notification.cnf(+)(AREP,Alarm_Type, Slot_Number, Seq_Nr)	W-FETCHED
53	W-FETCHED	FSPMS_Set_Slave_Diag.req(AREP,Ext_Diag_Flag, Ext_Diag) => if (PrmCmdAck in Ext_Diag or Stored_PrmCmdAck != NULL) Stored_PrmCmdAck := PrmCmdAck, Ext_Diag.Red_Status := PrmCmdAck endif Stored_Diag := True L_Ext_Diag_Flag := Ext_Diag_Flag L_Ext_Diag_Overflow := Ext_Diag_Overflow Local_Diag_Buffer := Ext_Diag FSPMS_Set_Slave_Diag.cnf(AREP)	W-FETCHED
54	W-FETCHED	MSAC1S_Alarm_Ack.ind(Slot_Number, Alarm_Type, Seq_Nr) /Alarm_Sequence=TRUE && Outstanding_Alarm_TABLE.Alarm_Status[Acls (Alarm_Type),Seq_Nr]=pending => Outstanding_Alarm_TABLE.Alarm_Status[Acls(Alarm_Type),Seq_Nr]:=not_pending Alarm_Count:=Alarm_Count-1 FSPMS_Alarm_Ack.ind(Slot_Number, Alarm_Type, Seq_Nr) MSAC1S_Alarm_Ack.rsp(+) (Slot Number, Alarm Type, Seq Nr)	W-FETCHED

Table 426 (continued)

#	Current State	Event /Condition =>Action	Next State
55	W-FETCHED	MSAC1S_Alarm_Ack.ind(Slot_Number, Alarm_Type, Seq_Nr) /Alarm_Sequence=FALSE && Outstanding_Alarm_TABLE.Alarm_Status[Acls(Alarm_Type),Seq_Nr]=pending => Outstanding_Alarm_TABLE.Alarm_Status[Acls(Alarm_Type),Seq_Nr]:=not_pending FSPMS_Alarm_Ack.ind(Slot_Number, Alarm_Type, Seq_Nr) MSAC1S_Alarm_Ack.rsp(+) (Slot Number, Alarm Type, Seq Nr)	W-FETCHED
56	W-FETCHED	MSAC1S_Alarm_Ack.ind(Slot_Number, Alarm_Type, Seq_Nr) /Outstanding_Alarm_TABLE.Alarm_Status[Acls(Alarm_Type),Seq_Nr]=not_pending => Index := Search_In_Outstanding_Alarm_TABLE(Alarm_Status:=pending) MSCY1S_Abort.req FSPMS_Abort.ind(AREP)	RESET-P-LEAVE
57	W-FETCHED	MSCY1S_Set_Slave_Diag.cnf(-) (Reference) => R_Cnt:=0 FSPMS_DP_Slave_Fault.ind DMPMS_Reset.req MSCY1S_Reset.req MSAC1S_Reset.req MSRM2S_Reset.req	W-RESET
58	W-FETCHED	MSCY1S_Set_Slave_Diag.cnf(+) (Reference) /Reference < Act_Ref => ignore	W-FETCHED
59	W-FETCHED	MSCY1S_Set_Slave_Diag.cnf(+) (Reference) /Reference >= Act_Ref => Stored_PrmCmdAck := NULL	CHK-TRANSMIT
60	W-FETCHED	MSCY1S_Start.ind(Enabled_Alarms, Alarm_Mode_Master, Diag_Flag) => MSCY1S_Abort.req	W-START
61	W-FETCHED	MSCY1S_Stop.ind /Alarm_Sequence=TRUE && Alarm_Count=0 => Stored_Diag:=False Ext_Diag_Flag:=L_Ext_Diag_Flag Ext_Diag_Overflow:=L_Ext_Diag_Overflow Ext_Diag:=(Local_Diag_Buffer) Act_Ref := Act_Ref + 1 CurrentBuffer[AREP].Empty:=TRUE LR_State[AREP]:=W-LR-IND MSCY1S_Set_Slave_Diag.req(Ext_Diag_Flag, Ext_Diag_Overflow, Ext_Diag, Reference := Act_Ref) FSPMS_Stopped.ind(AREP)	W-START
62	W-FETCHED	MSCY1S_Stop.ind /Alarm_Sequence=FALSE Alarm_Count<>0 => Stored_Diag:=False Alarm_Count:=0 Index := Search_In_Outstanding_Alarm_TABLE(Alarm_Status:=pending) Ext_Diag_Flag:=L_Ext_Diag_Flag Ext_Diag_Overflow:=L_Ext_Diag_Overflow Ext_Diag:=(Local_Diag_Buffer) Act_Ref := Act_Ref + 1 CurrentBuffer[AREP].Empty:=TRUE LR_State[AREP]:=W-LR-IND MSCY1S_Set_Slave_Diag.req(Ext_Diag_Flag, Ext_Diag_Overflow, Ext_Diag, Reference := Act_Ref)	RESET-PENDING
63	CHK-TRANSMIT	/Req_Alarm_FIFO_state=empty &&Stored_Diag = FALSE	W-DIA-UPD
64	CHK-TRANSMIT	/Req_Alarm_FIFO_state=empty && Stored_Diag = TRUE=>Stored_Diag:=FalseExt_Diag_Flag:=L_Ext_Diag_FlagExt_Diag_Overflow:=L_Ext_Diag_OverflowExt_Diag:=(Local_Diag_Buffer)Act_Ref := Act_Ref + 1MSCY1S_Set_Slave_Diag.req(Ext_Diag_Flag, Ext_Diag_Overflow, Ext_Diag, Reference := Act_Ref)	W-FETCHED

Table 426 (continued)

#	Current State	Event /Condition =>Action	Next State
65	CHK-TRANSMIT	/Req_Alarm_FIFO_state<>empty => Alarm:=Load_from_Req_Alarm_FIFO() Stored_Diag:=False Ext_Diag_Flag:=L_Ext_Diag_Flag Ext_Diag_Overflow:=L_Ext_Diag_Overflow Ext_Diag:=(Local_Diag_Buffer, Alarm) Act_Ref := Act_Ref + 1 MSCY1S_Set_Slave_Diag.req(Ext_Diag_Flag, Ext_Diag_Overflow, Ext_Diag, Reference := Act_Ref)	W-FETCHED
66	W-RESET	DMPMS_Reset.cnf /R_Cnt<3 => R_Cnt:=R_Cnt+1	W-RESET
67	W-RESET	MSCY1S_Reset.cnf /R_Cnt<3 => R_Cnt:=R_Cnt+1	W-RESET
68	W-RESET	MSAC1S_Reset.cnf /R_Cnt<3 => R_Cnt:=R_Cnt+1	W-RESET
69	W-RESET	MSRM2S_Reset.cnf /R_Cnt<3 => R_Cnt:=R_Cnt+1	W-RESET
70	W-RESET	DMPMS_Reset.cnf /R_Cnt>=3 => if (FSPMS_User_Reset=True) FSPMS_Reset DP Slave .cnf endif	POWER-ON
71	W-RESET	MSCY1S_Reset.cnf /R_Cnt>=3 => if (FSPMS_User_Reset=True) FSPMS_Reset DP Slave .cnf endif	POWER-ON
72	W-RESET	MSAC1S_Reset.cnf /R_Cnt>=3 => if (FSPMS_User_Reset=True) FSPMS_Reset DP Slave .cnf endif	POWER-ON
73	W-RESET	MSRM2S_Reset.cnf /R_Cnt>=3 => if (FSPMS_User_Reset=True) FSPMS_Reset DP Slave .cnfendif	POWER-ON
74	any state	DMPMS_SYCL_Clock_Value.ind (Clock Value Time Event, Clock Value previous TE, Clock Value Status, Receive Delay Time, Src add Clk msg) => FSPMS_SYCL_Clock_Value.ind (AREP, Clock Value Time Event, Clock Value previous TE, Clock Value Status, Receive Delay Time, Src add Clk msg)	SAME
75	any state	DMPMS_Fault.ind => R_Cnt:=0 FSPMS DP Slave Fault.ind DMPMS_Reset.req MSCY1S_Reset.req MSAC1S_Reset.req MSRM2S_Reset.req	W-RESET

Table 426 (continued)

#	Current State	Event /Condition =>Action	Next State
76	any state	MSAC1S_Fault.ind => R_Cnt:=0 FSPMS DP Slave Fault.ind DMPMS_Reset.req MSCY1S_Reset.req MSAC1S_Reset.req MSRM2S_Reset.req	W-RESET
77	any state	MSCY1S_Fault.ind => R_Cnt:=0 FSPMS DP Slave Fault.ind DMPMS_Reset.req MSCY1S_Reset.req MSAC1S_Reset.req MSRM2S_Reset.req	W-RESET
78	any state	MSRM2S_Fault.ind => R_Cnt:=0 FSPMS DP Slave Fault.ind DMPMS_Reset.req MSCY1S_Reset.req MSAC1S_Reset.req MSRM2S_Reset.req	W-RESET
79	any state	FSPMS_Reset DP Slave .req => R_Cnt:=0 FSPMS_User_Reset:=True DMPMS_Reset.req MSCY1S_Reset.req MSAC1S_Reset.req MSRM2S_Reset.req	W-RESET
80	W-FETCHED	Set_ARL_Isochron_Mode.req(Isochron_Mode) => Set_ARL_Isochron_Mode.cnf(+)	W-FETCHED
81	W-DIA-UPD	Set_ARL_Isochron_Mode.req(Isochron_Mode) => Set_ARL_Isochron_Mode.cnf(+)	W-DIA-UPD
82	W-START	Set_ARL_Isochron_Mode.req(Isochron_Mode) => Status:= NO Set_ARL_Isochron_Mode.cnf(-)(Status)	W-START
83	t-state	Load CRL DXB Linktable Entries.req(AREP, DXB-Linktable) /valid Parameters => Load CRL DXB Linktable Entries.cnf(+)	
84	t-state	Load CRL DXB Linktable Entries.req(AREP, DXB-Linktable) /invalid Parameters => Status:= NO Load CRL DXB Linktable Entries.cnf(-) (Status)	
85	t-state	FSPMS_DLL_Init_DP_Slave.req (Bus Para) => DMPMS_Sinit_DLL.req (Bus_Para)	SAME
86	t-state	FSPMS_Sinit MS0.req (AREP) => MSCY1S_Sinit MS0.req ()	SAME
87	t-state	FSPMS_Sinit MS2.req () => MSRM2S_Sinit MS2.req ()	SAME
88	t-state	FSPMS_Abort.req (AREP, Subnet, Instance, Reason Code) /AREP.AR_Type=MS2 => CurrentBuffer[AREP].Empty:=TRUE LR_State[AREP]:=W-LR-IND MSAC2S_Abort.req (Subnet, Instance, Reason Code)	SAME

Table 426 (continued)

#	Current State	Event /Condition =>Action	Next State
89	t-state	FSPMS_DP Slave Application Ready.req (AREP) /AREP.AR_Type=MS0 => MSCY1S_Application Ready.req ()	SAME
90	t-state	FSPMS_Check User Prm Result.req (AREP, Prm_OK) /AREP.AR_Type=MS0 => MSCY1S_Check User Prm Result.req (Prm_OK)	SAME
91	t-state	FSPMS_Check Cfg Result.req (AREP, Cfg_OK, Input Data Len, Output Data Len) /AREP.AR_Type=MS0 => MSCY1S_Check Cfg Result.req (Cfg_OK, Input Data Len, Output Data Len)	SAME
92	t-state	FSPMS_Set Cfg.req (AREP, Cfg Data) /AREP.AR_Type=MS0 => MSCY1S_Set Cfg.req (Cfg Data)	SAME
93	t-state	FSPMS_Set Input.req (AREP, Input Data) /AREP.AR_Type=MS0 => MSCY1S_Set Input.req (Input Data)	SAME
94	t-state	FSPMS_Get Output.req (AREP) /AREP.AR_Type=MS0 => MSCY1S_Get Output.req ()	SAME
95	t-state	FSPMS_Initiate.rsp(+) (AREP, Max Len Data Unit, Features Supported, Profile Features Supported, Profile Ident Number, Add Addr Param) /AREP.AR_Type=MS1 => MSAC2S_Initiate.rsp(+) (Res SAP, Max Len Data Unit, Features Supported, Profile Features Supported, Profile Ident Number, Add Addr Param)	SAME
96	t-state	FSPMS_Initiate.rsp(-) (AREP, Error Decode, Error Code 1 Error Code 2) /AREP.AR_Type=MS2 => MSAC2S_Initiate.rsp(-) (Res SAP, Error Decode, Error Code 1 Error Code 2)	SAME
97	t-state	FSPMS_Read.rsp(+) (AREP, Length, Data) /AREP.AR_Type=MS1 => MSAC1S_Read.rsp(+) (Length, Data)	SAME
98	t-state	FSPMS_Read.rsp(+) (AREP, Length, Data) /AREP.AR_Type=MS2 => MSAC2S_Read.rsp(+) (Length, Data)	SAME
99	t-state	FSPMS_Read.rsp(-) (AREP, Error Decode, Error Code 1 Error Code 2) /AREP.AR_Type=MS1 => MSAC1S_Read.rsp(-) (Error Decode, Error Code 1 Error Code 2)	SAME
100	t-state	FSPMS_Read.rsp(-) (AREP, Error Decode, Error Code 1 Error Code 2) /AREP.AR_Type=MS2 => MSAC2S_Read.rsp(-) (Error Decode, Error Code 1 Error Code 2)	SAME
101	t-state	FSPMS_Write.rsp(+) (AREP, Length) /AREP.AR_Type=MS1 => MSAC1S_Write.rsp(+) (Length)	SAME
102	t-state	FSPMS_Write.rsp(+) (AREP, Length) /AREP.AR_Type=MS2 => MSAC2S_Write.rsp(+) (Length)	SAME
103	t-state	FSPMS_Write.rsp(-) (AREP, Error Decode, Error Code 1 Error Code 2) /AREP.AR_Type=MS1 => MSAC1S_Write.rsp(-) (Error Decode, Error Code 1 Error Code 2)	SAME
104	t-state	FSPMS_Write.rsp(-) (AREP, Error Decode, Error Code 1 Error Code 2) /AREP.AR_Type=MS2 => MSAC2S_Write.rsp(-) (Error Decode, Error Code 1 Error Code 2)	SAME

Table 426 (continued)

#	Current State	Event /Condition =>Action	Next State
105	t-state	FSPMS_Data Transport.rsp(+) (AREP, Length, Data) /AREP.AR_Type=MS2 => MSAC2S_Data Transport.rsp(+) (Res SAP, Length, Data)	SAME
106	t-state	FSPMS_Data Transport.rsp(-) (AREP, Error Decode, Error Code 1 Error Code 2) /AREP.AR_Type=MS2 => MSAC2S_Data Transport.rsp(-) (Res SAP, Error Decode, Error Code 1 Error Code 2)	SAME
107	t-state	FSPMS_Alarm Ack.rsp(+) (AREP, Slot Number, Alarm Type, Seq Nr) /AREP.AR_Type=MS1 =>	SAME
108	t-state	FSPMS_Alarm Ack.rsp(-) (AREP, Slot Number, Alarm Type, Seq Nr) /AREP.AR_Type=MS1 =>	SAME
109	t-state	DMPMS_SInit DLL.cnf () => FSPMS_DLL Init DP Slave.cnf ()	SAME
110	t-state	MSCY1S_SInit MS0.cnf () => AREP=MS0-AR FSPMS_SInit MS0.cnf (AREP)	SAME
111	t-state	MSRM2S_SInit MS2.cnf () => FSPMS_SInit MS2.cnf ()	SAME
112	t-state	MSCY1S_Check User Prm Result.cnf(+) () => AREP=MS0-AR FSPMS_Check User Prm Result.cnf(+) (AREP)	SAME
113	t-state	MSCY1S_Check User Prm Result.cnf(-) (Status) => AREP=MS0-AR FSPMS_Check User Prm Result.cnf(-) (AREP, Status)	SAME
114	t-state	MSCY1S_Check Cfg Result.cnf(+) () => AREP=MS0-AR FSPMS_Check Cfg Result.cnf(+) (AREP)	SAME
115	t-state	MSCY1S_Check Cfg Result.cnf(-) (Status) => AREP=MS0-AR FSPMS_Check Cfg Result.cnf(-) (AREP, Status)	SAME
116	t-state	MSCY1S_Set Cfg.cnf(+) () => AREP=MS0-AR FSPMS_Set Cfg.cnf(+) (AREP)	SAME
117	t-state	MSCY1S_Set Cfg.cnf(-) () => AREP=MS0-AR FSPMS_Set Cfg.cnf(-) (AREP)	SAME
118	t-state	MSCY1S_Set Input.cnf(+) () => AREP=MS0-AR FSPMS_Set Input.cnf(+) (AREP)	SAME
119	t-state	MSCY1S_Set Input.cnf(-) () => AREP=MS0-AR FSPMS_Set Input.cnf(-) (AREP)	SAME
120	t-state	MSCY1S_Get Output.cnf(+) (Output Data, Clear Flag, New Flag) => AREP=MS0-AR FSPMS_Get Output.cnf(+) (AREP, Output Data, Clear Flag, New Flag)	SAME
121	t-state	MSCY1S_Get Output.cnf(-) () => AREP=MS0-AR FSPMS_Get Output.cnf(-) (AREP)	SAME

Table 426 (continued)

#	Current State	Event /Condition =>Action	Next State
122	t-state	MSCY1S_Set Slave Add.ind (New Slave Add, Ident Number, No Add Chg, Rem Slave Data) => AREP=MS0-AR FSPMS_Set Slave Add.ind (AREP, New Slave Add, Ident Number, No Add Chg, Rem Slave Data)	SAME
123	t-state	MSCY1S_Check User Prm.ind (User Prm Data) => AREP=MS0-AR FSPMS_Check User Prm.ind (AREP, User Prm Data)	SAME
124	t-state	MSCY1S_Check Cfg.ind (Check Cfg Mode, Cfg Data) => AREP=MS0-AR FSPMS_Check Cfg.ind (AREP, Check Cfg Mode, Cfg Data)	SAME
125	t-state	MSCY1S_New Output.ind (Clear Flag) => AREP=MS0-AR FSPMS_New Output.ind (AREP, Clear Flag)	SAME
126	t-state	MSCY1S_Global Control.ind (Clear Command, Sync Command, Freeze Command, Group Select) => AREP=MS0-AR FSPMS_Global Control.ind (AREP, Clear Command, Sync Command, Freeze Command, Group Select)	SAME
127	t-state	MSAC2S_Initiate.ind (Res SAP, Features Supported, Profile Features Supported, Profile Ident Number, Add Addr Param) /SetContext(Res_SAP, Service)=TRUE => FSPMS_Initiate.ind (AREP, Features Supported, Profile Features Supported, Profile Ident Number, Add Addr Param)	SAME
128	t-state	MSAC2S_Abort.ind (Res SAP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail) /SetContext(Res_SAP, Service)=TRUE => CurrentBuffer[AREP].Empty:=TRUE LR_State[AREP]:=W-LR-IND FSPMS_Abort.ind (AREP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail)	SAME
129	t-state	MSAC2S_Read.ind (Res SAP, Slot Number, Index, Length) /SetContext(Res_SAP, Service)=TRUE && Index<>255 => FSPMS_Read.ind (AREP, Slot Number, Index, Length)	SAME
130	t-state	MSAC1S_Read.ind (Res SAP, Slot Number, Index, Length) /SetContext(Res_SAP, Service)=TRUE && Index<>255 => FSPMS_Read.ind (AREP, Slot Number, Index, Length)	SAME
131	t-state	MSAC2S_Write.ind (Res SAP, Slot Number, Index, Length, Data) /SetContext(Res_SAP, Service)=TRUE && Index<>255 => FSPMS_Write.ind (AREP, Slot Number, Index, Length, Data)	SAME
132	t-state	MSAC1S_Write.ind (Res SAP, Slot Number, Index, Length, Data) /SetContext(Res_SAP, Service)=TRUE &&Index<>255 => FSPMS_Write.ind (AREP, Slot Number, Index, Length, Data)	SAME
133	t-state	MSAC2S_Data Transport.ind (Res SAP, Slot Number, Index, Length, Data) /SetContext(Res_SAP, Service)=TRUE => FSPMS_Data Transport.ind (AREP, Slot Number, Index, Length, Data)	SAME
134	t-state	FSPMS_Get_Publisher_Data.req (AREP, CREP) /AREP.AR_Type=MS0 => Rem_Add=CREP.Rem_Add SSCY1S_Get_Publisher_Data.req (Rem_Add)	SAME

Table 426 (continued)

#	Current State	Event /Condition =>Action	Next State
135	t-state	FSPMS_Start_Subscriber.req (AREP, CREP) /AREP.AR_Type=MS0 => Rem_Add=CREP.Rem_Add SSCY1S_Start_Subscriber.req (Rem_Add)	SAME
136	t-state	FSPMS_Stop_Subscriber.req (AREP, CREP) /AREP.AR_Type=MS0 => Rem_Add=CREP.Rem_Add SSCY1S_Stop_Subscriber.req (Rem_Add)	SAME
137	t-state	FSPMS_Check_Ext_User_Prm_Result.req (AREP, Ext_Prm_OK) /AREP.AR_Type=MS0 => MSCY1S_Check_Ext_User_Prm_Result.req (Ext_Prm_OK)	SAME
138	t-state	SSCY1S_Get_Publisher_Data.cnf(+) (Rem_Add, Data, New_Flag) /SetContext(Rem_Add, Service)=TRUE => FSPMS_Get_Publisher_Data.cnf(+) (AREP, CREP, Data, New_Flag)	SAME
139	t-state	SSCY1S_Get_Publisher_Data.cnf(-) (Rem_Add) /SetContext(Rem_Add, Service)=TRUE => FSPMS_Get_Publisher_Data.cnf(-) (AREP, CREP)	SAME
140	t-state	SSCY1S_Start_Subscriber.cnf(+) (Rem_Add) /SetContext(Rem_Add, Service)=TRUE => FSPMS_Start_Subscriber.cnf(+) (AREP, CREP)	SAME
141	t-state	SSCY1S_Start_Subscriber.cnf(-) (Rem_Add) /SetContext(Rem_Add, Service)=TRUE => FSPMS_Start_Subscriber.cnf(-) (AREP, CREP)	SAME
142	t-state	SSCY1S_Stop_Subscriber.cnf(+) (Rem_Add) /SetContext(Rem_Add, Service)=TRUE => FSPMS_Stop_Subscriber.cnf(+) (AREP, CREP)	SAME
143	t-state	SSCY1S_Stop_Subscriber.cnf(-) (Rem_Add) /SetContext(Rem_Add, Service)=TRUE => FSPMS_Stop_Subscriber.cnf(-) (AREP, CREP)	SAME
144	t-state	SSCY1S_New_Publisher_Data.ind (Rem_Add) /SetContext(Rem_Add, Service)=TRUE => FSPMS_New_Publisher_Data.ind(AREP, CREP)	SAME
145	t-state	SSCY1S_Publisher_Active.ind (Rem_Add, Status) /SetContext(Rem_Add, Service)=TRUE => FSPMS_Publisher_Active.ind(AREP, CREP, Status)	SAME
146	t-state	MSCY1S_Check_Ext_User_Prm.ind (Rem_Add, Ext_User_Prm_Data) => AREP=MS0-AR FSPMS_Check_Ext_User_Prm.ind (AREP, Ext_User_Prm_Data)	SAME
147	t-state	MSAC1S_Write.ind (Res SAP, Slot Number, Index, Length, Data) /LR_State[AREP]==W-LR-IND &SetContext(Res_SAP, Service)=TRUE && (Index=255 && Length >= 4) && (Data.w.IL.Extended_Function_Num=Initiate_Load) => CurrentBuffer[AREP].Empty:=TRUE CurrentBuffer[AREP].WriteLength:=Length Current Load Type[AREP]:=Data.w.IL.Load_Type LR Index:=Data.w.IL.LR_Index Load Type:=Data.w.IL.Load_Type Load Image Size:=Data.w.IL.Load_Image_Size User Specific:=Data.w.IL.User_Specific Intersegment Request Timeout:=Data.w.IL.Intersegment_Request_Timeout FSPMS_Initiate_Load.ind (AREP, Slot Number, LR Index, Load Type, Load Image Size, Intersegment Request Timeout, User Specific) LR_State[AREP]:=W-LR-RES	SAME

Table 426 (continued)

#	Current State	Event /Condition =>Action	Next State
148	t-state	MSAC2S_Write.ind (Res SAP, Slot Number, Index, Length, Data) /LR_State[AREP]==W-LR-IND &SetContext(Res_SAP, Service)=TRUE && (Index=255 && Length >= 4) && (Data.w.IL.Extended_Function_Num=Initiate_Load) => CurrentBuffer[AREP].Empty:=TRUE CurrentBuffer[AREP].WriteLength:=Length CurrentLoadType[AREP]:=Data.w.IL.Load_Type CurrentLRIndex[AREP]:=Data.w.IL.LR_Index LR_Index:=Data.w.IL.LR_Index Load_Type:=Data.w.IL.Load_Type Load Image Size:=Data.w.IL.Load_Image_Size User Specific:=Data.w.IL.User_Specific Intersegment Request Timeout:=Data.w.IL.Intersegment_Request_Timeout FSPMS_Initiate_Load.ind (AREP, Slot Number, LR Index, Load Type, Load Image Size, Intersegment Request Timeout, User Specific) LR_State[AREP]:=W-LR-RES	SAME
149	t-state	FSPMS_Initiate_Load.rsp(+) (AREP, Actual LR Size, Max Response Delay, Max Segment Length, User Specific) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS1 => CurrentBuffer[AREP].Empty:=FALSE CurrentBuffer[AREP].data.r.IL.Actual_LR_Size:=Actual LR Size CurrentBuffer[AREP].data.r.IL.Max_Response_Delay:=Max Response Delay CurrentBuffer[AREP].data.r.IL.Max_Segment_Length:=Max Segment Length CurrentBuffer[AREP].data.r.IL.User_Specific:=User Specific MSAC1S_Write.rsp(+) (Length) LR_State[AREP]:=W-LR-IND	SAME
150	t-state	FSPMS_Initiate_Load.rsp(+) (AREP, Actual LR Size, Max Response Delay, Max Segment Length, User Specific) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS2 => CurrentBuffer[AREP].Empty:=FALSE CurrentBuffer[AREP].data.r.IL.Actual_LR_Size:=Actual LR Size CurrentBuffer[AREP].data.r.IL.Max_Response_Delay:=Max Response Delay CurrentBuffer[AREP].data.r.IL.Max_Segment_Length:=Max Segment Length CurrentBuffer[AREP].data.r.IL.User_Specific:=User Specific MSAC2S_Write.rsp(+) (Length) LR_State[AREP]:=W-LR-IND	SAME
151	t-state	MSAC1S_Read.ind (Res SAP, Slot Number, Index, Length) /LR_State[AREP]==W-LR-IND &SetContext(Res_SAP, Service)=TRUE && (Index=255 && CurrentBuffer[AREP].Empty=FALSE) => CurrentBuffer[AREP].Empty:=TRUE Data:=CurrentBuffer[AREP].data Length:=sizeof(CurrentBuffer[AREP].data) MSAC1S_Read.rsp(+) (Length, Data) LR_State[AREP]:=W-LR-IND	SAME
152	t-state	MSAC2S_Read.ind (Res SAP, Slot Number, Index, Length) /LR_State[AREP]==W-LR-IND &SetContext(Res_SAP, Service)=TRUE && (Index=255 && CurrentBuffer[AREP].Empty=FALSE) => CurrentBuffer[AREP].Empty:=TRUE Data:=CurrentBuffer[AREP].data Length:=sizeof(CurrentBuffer[AREP].data) MSAC2S_Read.rsp(+) (Length, Data) LR_State[AREP]:=W-LR-IND	SAME

Table 426 (continued)

#	Current State	Event /Condition =>Action	Next State
153	t-state	FSPMS_Initiate_Load.rsp(-) (AREP, Error Code) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS1 => Error Decode:=DPV1 Error Code 1 :=Error Code Error Code 2:=0 MSAC1S_Write.rsp(-) (Error Decode, Error Code 1 Error Code 2) LR_State[AREP]:=W-LR-IND	SAME
154	t-state	FSPMS_Initiate_Load.rsp(-) (AREP, Error Code) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS2 => Error Decode:=DPV1 Error Code 1 :=Error Code Error Code 2:=0 MSAC2S_Write.rsp(-) (Error Decode, Error Code 1 Error Code 2) LR_State[AREP]:=W-LR-IND	SAME
155	t-state	MSAC1S_Read.ind (Res SAP, Slot Number, Index, Length) /LR_State[AREP]==W-LR-IND &SetContext(Res_SAP, Service)=TRUE && (Index=255 && CurrentBuffer[AREP].Empty=TRUE) => LR Index:=CurrentLRIndex[AREP] Segment Length:=Length FSPMS_Pull_Segment.ind (AREP, Slot Number, LR Index, Segment Length) LR_State[AREP]:=W-LR-RES	SAME
156	t-state	MSAC2S_Read.ind (Res SAP, Slot Number, Index, Length) /LR_State[AREP]==W-LR-IND &SetContext(Res_SAP, Service)=TRUE && (Index=255 && CurrentBuffer[AREP].Empty=TRUE) => LR Index:=CurrentLRIndex[AREP] Segment Length:=Length FSPMS_Pull_Segment.ind (AREP, Slot Number, LR Index, Segment Length) LR_State[AREP]:=W-LR-RES	SAME
157	t-state	FSPMS_Pull_Segment.rsp(+) (AREP, Segment Length, Segment Number, More Follows, Data) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS1 => Data.PULL.Extended_Function_Num=Pull Data.PULL.Sequence Number:=Segment Number Data.PULL.Options.More Follows:=More Follows Data.PULL.Region Data := Data Length:=6+Segment Length MSAC1S_Read.rsp(+) (Length, Data) LR_State[AREP]:=W-LR-IND	SAME
158	t-state	FSPMS_Pull_Segment.rsp(+) (AREP, Segment Length, Segment Number, More Follows, Data) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS2 => Data.PULL.Extended_Function_Num=Pull Data.PULL.Sequence Number:=Segment Number Data.PULL.Options.More Follows:=More Follows Data.PULL.Region Data := Data Length:=6+Segment Length MSAC2S_Read.rsp(+) (Length, Data) LR_State[AREP]:=W-LR-IND	SAME
159	t-state	FSPMS_Pull_Segment.rsp(-) (AREP, Error Code) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS1 => Error Decode:=DPV1 Error Code 1 :=Error Code Error Code 2:=0 MSAC1S_Read.rsp(-) (Error Decode, Error Code 1 Error Code 2) LR_State[AREP]:=W-LR-IND	SAME

Table 426 (continued)

#	Current State	Event /Condition =>Action	Next State
160	t-state	FSPMS_Pull_Segment.rsp(-) (AREP, Error Code) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS2 => Error Decode:=DPV1 Error Code 1 :=Error Code Error Code 2:=0 MSAC2S_Read.rsp(-) (Error Decode, Error Code 1 Error Code 2) LR_State[AREP]:=W-LR-IND	SAME
161	t-state	MSAC1S_Write.ind (Res SAP, Slot Number, Index, Length, Data) /LR_State[AREP]==W-LR-IND &SetContext(Res_SAP, Service)=TRUE && (Index=255 && Length >= 4) && (Data.w.PUSH.Extended_Function_Num=Push) => CurrentBuffer[AREP].Empty:=TRUE CurrentBuffer[AREP].WriteLength:=Length LR_Index:=Data.PUSH.LR_Index Segment Length:=Length Segment Number:=Data.PUSH.Segment_Number More Follows:=Data.PUSH.More_Follows Data:=Data.PUSH.data FSPMS_Push_Segment.ind (AREP, Slot Number, LR_Index, Segment Length, Segment Number, More Follows, Data) LR_State[AREP]:=W-LR-RES	SAME
162	t-state	MSAC2S_Write.ind (Res SAP, Slot Number, Index, Length, Data) /LR_State[AREP]==W-LR-IND &SetContext(Res_SAP, Service)=TRUE && (Index=255 && Length >= 4) && (Data.w.PUSH.Extended_Function_Num=Push) => CurrentBuffer[AREP].Empty:=TRUE CurrentBuffer[AREP].WriteLength:=Length LR_Index:=Data.PUSH.LR_Index Segment Length:=Length Segment Number:=Data.PUSH.Segment_Number More Follows:=Data.PUSH.More_Follows Data:=Data.PUSH.data FSPMS_Push_Segment.ind (AREP, Slot Number, LR_Index, Segment Length, Segment Number, More Follows, Data) LR_State[AREP]:=W-LR-RES	SAME
163	t-state	FSPMS_Push_Segment.rsp(+) (AREP) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS1 => Length:=CurrentBuffer[AREP].WriteLength MSAC1S_Write.rsp(+) (Length) LR_State[AREP]:=W-LR-IND	SAME
164	t-state	FSPMS_Push_Segment.rsp(+) (AREP) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS2 => Length:=CurrentBuffer[AREP].WriteLength MSAC2S_Write.rsp(+) (Length) LR_State[AREP]:=W-LR-IND	SAME
165	t-state	FSPMS_Push_Segment.rsp(-) (AREP, Error Code) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS1 => Error Decode:=DPV1 Error Code 1 :=Error Code Error Code 2:=0 MSAC1S_Write.rsp(-) (Error Decode, Error Code 1 Error Code 2) LR_State[AREP]:=W-LR-IND	SAME

Table 426 (continued)

#	Current State	Event /Condition =>Action	Next State
166	t-state	FSPMS_Push_Segment.rsp(-) (AREP, Error Code) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS2 => Error Decode:=DPV1 Error Code 1 :=Error Code Error Code 2:=0 MSAC2S_Write.rsp(-) (Error Decode, Error Code 1 Error Code 2) LR_State[AREP]:=W-LR-IND	SAME
167	t-state	MSAC1S_Write.ind (Res SAP, Slot Number, Index, Length, Data) /LR_State[AREP]==W-LR-IND &SetContext(Res_SAP, Service)=TRUE && (Index=255 && Length >= 4) && (Data.w.TL.Extended_Function_Num=Terminate_Load) => CurrentBuffer[AREP].Empty:=TRUE CurrentBuffer[AREP].WriteLength:=Length LR Index:=Data.w.TL.LR_Index FSPMS_Terminate_Load.ind (AREP, Slot Number, LR Index) LR_State[AREP]:=W-LR-RES	SAME
168	t-state	MSAC2S_Write.ind (Res SAP, Slot Number, Index, Length, Data) /LR_State[AREP]==W-LR-IND &SetContext(Res_SAP, Service)=TRUE && (Index=255 && Length >= 4) && (Data.w.TL.Extended_Function_Num=Terminate_Load) => CurrentBuffer[AREP].Empty:=TRUE CurrentBuffer[AREP].WriteLength:=Length LR Index:=Data.w.TL.LR_Index FSPMS_Terminate_Load.ind (AREP, Slot Number, LR Index) LR_State[AREP]:=W-LR-RES	SAME
169	t-state	FSPMS_Terminate_Load.rsp(+) (AREP) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS1 => Length:=CurrentBuffer[AREP].WriteLength MSAC1S_Write.rsp(+) (Length) LR_State[AREP]:=W-LR-IND	SAME
170	t-state	FSPMS_Terminate_Load.rsp(+) (AREP) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS2 => Length:=CurrentBuffer[AREP].WriteLength MSAC2S_Write.rsp(+) (Length) LR_State[AREP]:=W-LR-IND	SAME
171	t-state	FSPMS_Terminate_Load.rsp(-) (AREP, Error Code) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS1 => Error Decode:=DPV1 Error Code 1 :=Error Code Error Code 2:=0 MSAC1S_Write.rsp(-) (Error Decode, Error Code 1 Error Code 2) LR_State[AREP]:=W-LR-IND	SAME
172	t-state	FSPMS_Terminate_Load.rsp(-) (AREP, Error Code) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS2 => Error Decode:=DPV1 Error Code 1 :=Error Code Error Code 2:=0 MSAC2S_Write.rsp(-) (Error Decode, Error Code 1 Error Code 2) LR_State[AREP]:=W-LR-IND	SAME

Table 426 (continued)

#	Current State	Event /Condition =>Action	Next State
173	t-state	MSAC1S_Write.ind (Res SAP, Slot Number, Index, Length, Data) /LR_State[AREP]==W-LR-IND &SetContext(Res_SAP, Service)=TRUE && (Index=255 && Length >= 4) && (Data.w.CALL.Extended_Function_Num=CALL) => CurrentBuffer[AREP].Empty:=TRUE CurrentBuffer[AREP].WriteLength:=Length FI Index:=Data.w.CALL.FI_Index Execution Argument:=Data.w.CALL.Execution_Argument FSPMS_Call.ind (AREP, Slot Number, FI Index, Execution Argument) LR_State[AREP]:=W-LR-RES	SAME
174	t-state	MSAC2S_Write.ind (Res SAP, Slot Number, Index, Length, Data) /LR_State[AREP]==W-LR-IND &SetContext(Res_SAP, Service)=TRUE && (Index=255 && Length >= 4) && (Data.w.CALL.Extended_Function_Num=CALL) => CurrentBuffer[AREP].Empty:=TRUE CurrentBuffer[AREP].WriteLength:=Length FI Index:=Data.w.CALL.FI_Index Execution Argument:=Data.w.CALL.Execution_Argument FSPMS_Call.ind (AREP, Slot Number, FI Index, Execution Argument) LR_State[AREP]:=W-LR-RES	SAME
175	t-state	FSPMS_Call.rsp(+) (AREP, Result Argument) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS1 => CurrentBuffer[AREP].Empty:=FALSE CurrentBuffer[AREP].data.r.CALL.ResultArgument:=Result Argument Length:=CurrentBuffer[AREP].WriteLength MSAC1S_Write.rsp(+) (Length) LR_State[AREP]:=W-LR-IND	SAME
176	t-state	FSPMS_Call.rsp(+) (AREP, Result Argument) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS2 => CurrentBuffer[AREP].Empty:=FALSE CurrentBuffer[AREP].data.r.CALL.ResultArgument:=Result Argument Length:=CurrentBuffer[AREP].WriteLength MSAC2S_Write.rsp(+) (Length) LR_State[AREP]:=W-LR-IND	SAME
177	t-state	FSPMS_Call.rsp(-) (AREP, Error Code) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS1 => Error Decode:=DPV1 Error Code 1 :=Error Code Error Code 2:=0 MSAC1S_Write.rsp(-) (Error Decode, Error Code 1 Error Code 2) LR_State[AREP]:=W-LR-IND	SAME
178	t-state	FSPMS_Call.rsp(-) (AREP, Error Code) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS2 => Error Decode:=DPV1 Error Code 1 :=Error Code Error Code 2:=0 MSAC1S_Write.rsp(-) (Error Decode, Error Code 1 Error Code 2) LR_State[AREP]:=W-LR-IND	SAME

Table 426 (continued)

#	Current State	Event /Condition =>Action	Next State
179	t-state	MSAC1S_Write.ind (Res SAP, Slot Number, Index, Length, Data) /LR_State[AREP]==W-LR-IND &SetContext(Res_SAP, Service)=TRUE && (Index=255 && Length >= 4) && (Data.FIS.Extended_Function_Num=Start) => CurrentBuffer[AREP].Empty:=TRUE CurrentBuffer[AREP].WriteLength:=Length FI Index:=Data.FIS.FI_Index Execution Argument:=Data.FIS.Execution_Argument FSPMS_Start.ind (AREP, Slot Number, FI Index, Execution Argument) LR_State[AREP]:=W-LR-RES	SAME
180	t-state	MSAC2S_Write.ind (Res SAP, Slot Number, Index, Length, Data) /LR_State[AREP]==W-LR-IND &SetContext(Res_SAP, Service)=TRUE && (Index=255 && Length >= 4) && (Data.FIS.Extended_Function_Num=Start) => CurrentBuffer[AREP].Empty:=TRUE CurrentBuffer[AREP].WriteLength:=Length FI Index:=Data.FIS.FI_Index Execution Argument:=Data.FIS.Execution_Argument FSPMS_Start.ind (AREP, Slot Number, FI Index, Execution Argument) LR_State[AREP]:=W-LR-RES	SAME
181	t-state	FSPMS_Start.rsp(+) (AREP) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS1 => Length:=CurrentBuffer[AREP].WriteLength MSAC1S_Write.rsp(+) (Length) LR_State[AREP]:=W-LR-IND	SAME
182	t-state	FSPMS_Start.rsp(+) (AREP) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS2 => Length:=CurrentBuffer[AREP].WriteLength MSAC2S_Write.rsp(+) (Length) LR_State[AREP]:=W-LR-IND	SAME
183	t-state	FSPMS_Start.rsp(-) (AREP, Error Code) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS1 => Error Decode:=DPV1 Error Code 1 :=Error Code Error Code 2:=0 MSAC1S_Write.rsp(-) (Error Decode, Error Code 1 Error Code 2) LR_State[AREP]:=W-LR-IND	SAME
184	t-state	FSPMS_Start.rsp(-) (AREP, Error Code) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS2 => Error Decode:=DPV1 Error Code 1 :=Error Code Error Code 2:=0 MSAC2S_Write.rsp(-) (Error Decode, Error Code 1 Error Code 2) LR_State[AREP]:=W-LR-IND	SAME
185	t-state	MSAC1S_Write.ind (Res SAP, Slot Number, Index, Length, Data) /LR_State[AREP]==W-LR-IND &SetContext(Res_SAP, Service)=TRUE && (Index=255 && Length >= 4) && (Data.FIS.Extended_Function_Num=Stop) => CurrentBuffer[AREP].Empty:=TRUE CurrentBuffer[AREP].WriteLength:=Length FI Index:=Data.FIS.FI_Index FSPMS_Stop.ind (AREP, Slot Number, FI Index) LR_State[AREP]:=W-LR-RES	SAME

Table 426 (continued)

#	Current State	Event /Condition =>Action	Next State
186	t-state	MSAC2S_Write.ind (Res SAP, Slot Number, Index, Length, Data) /LR_State[AREP]==W-LR-IND &SetContext(Res_SAP, Service)=TRUE && (Index=255 && Length >= 4) && (Data.FIS.Extended_Function_Num=Stop) => CurrentBuffer[AREP].Empty:=TRUE CurrentBuffer[AREP].WriteLength:=Length FI Index:=Data.FIS.FI_Index FSPMS_Stop.ind (AREP, Slot Number, FI Index) LR_State[AREP]:=W-LR-RES	SAME
187	t-state	FSPMS_Stop.rsp(+) (AREP) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS1 => Length:=CurrentBuffer[AREP].WriteLength MSAC1S_Write.rsp(+) (Length) LR_State[AREP]:=W-LR-IND	SAME
188	t-state	FSPMS_Stop.rsp(+) (AREP) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS2 => Length:=CurrentBuffer[AREP].WriteLength MSAC2S_Write.rsp(+) (Length) LR_State[AREP]:=W-LR-IND	SAME
189	t-state	FSPMS_Stop.rsp(-) (AREP, Error Code) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS1 => Error Decode:=DPV1 Error Code 1 :=Error Code Error Code 2:=0 MSAC1S_Write.rsp(-) (Error Decode, Error Code 1 Error Code 2) LR_State[AREP]:=W-LR-IND	SAME
190	t-state	FSPMS_Stop.rsp(-) (AREP, Error Code) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS2 => Error Decode:=DPV1 Error Code 1 :=Error Code Error Code 2:=0 MSAC2S_Write.rsp(-) (Error Decode, Error Code 1 Error Code 2) LR_State[AREP]:=W-LR-IND	SAME
191	t-state	MSAC1S_Write.ind (Res SAP, Slot Number, Index, Length, Data) /LR_State[AREP]==W-LR-IND &SetContext(Res_SAP, Service)=TRUE && (Index=255 && Length >= 4) && (Data.FIS.Extended_Function_Num=Resume) => CurrentBuffer[AREP].Empty:=TRUE CurrentBuffer[AREP].WriteLength:=Length FI Index:=Data.FIS.FI_Index Execution Argument:=Data.FIS.Execution_Argument FSPMS_Resume.ind (AREP, Slot Number, FI Index, Execution Argument) LR_State[AREP]:=W-LR-RES	SAME
192	t-state	MSAC2S_Write.ind (Res SAP, Slot Number, Index, Length, Data) /LR_State[AREP]==W-LR-IND &SetContext(Res_SAP, Service)=TRUE && (Index=255 && Length >= 4) && (Data.FIS.Extended_Function_Num=Resume) => CurrentBuffer[AREP].Empty:=TRUE CurrentBuffer[AREP].WriteLength:=Length FI Index:=Data.FIS.FI_Index Execution Argument:=Data.FIS.Execution_Argument FSPMS_Resume.ind (AREP, Slot Number, FI Index, Execution Argument) LR_State[AREP]:=W-LR-RES	SAME

Table 426 (continued)

#	Current State	Event /Condition =>Action	Next State
193	t-state	FSPMS_Resume.rsp(+) (AREP) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS1 => Length:=CurrentBuffer[AREP].WriteLength MSAC1S_Write.rsp(+) (Length) LR_State[AREP]:=W-LR-IND	SAME
194	t-state	FSPMS_Resume.rsp(+) (AREP) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS2 => Length:=CurrentBuffer[AREP].WriteLength MSAC2S_Write.rsp(+) (Length) LR_State[AREP]:=W-LR-IND	SAME
195	t-state	FSPMS_Resume.rsp(-) (AREP, Error Code) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS1 => Error Decode:=DPV1 Error Code 1 :=Error Code Error Code 2:=0 MSAC1S_Write.rsp(-) (Error Decode, Error Code 1 Error Code 2) LR_State[AREP]:=W-LR-IND	SAME
196	t-state	FSPMS_Resume.rsp(-) (AREP, Error Code) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS2 => Error Decode:=DPV1 Error Code 1 :=Error Code Error Code 2:=0 MSAC2S_Write.rsp(-) (Error Decode, Error Code 1 Error Code 2) LR_State[AREP]:=W-LR-IND	SAME
197	t-state	MSAC1S_Write.ind (Res SAP, Slot Number, Index, Length, Data) /LR_State[AREP]==W-LR-IND &SetContext(Res_SAP, Service)=TRUE && (Index=255 && Length >= 4) && (Data.FIS.Extended_Function_Num=Reset) => CurrentBuffer[AREP].Empty:=TRUE CurrentBuffer[AREP].WriteLength:=Length FI Index:=Data.FIS.FI_Index FSPMS_Reset.ind (AREP, Slot Number, FI Index) LR_State[AREP]:=W-LR-RES	SAME
198	t-state	MSAC2S_Write.ind (Res SAP, Slot Number, Index, Length, Data) /LR_State[AREP]==W-LR-IND &SetContext(Res_SAP, Service)=TRUE && (Index=255 && Length >= 4) && (Data.FIS.Extended_Function_Num=Reset) => CurrentBuffer[AREP].Empty:=TRUE CurrentBuffer[AREP].WriteLength:=Length FI Index:=Data.FIS.FI_Index FSPMS_Reset.ind (AREP, Slot Number, FI Index) LR_State[AREP]:=W-LR-RES	SAME
199	t-state	FSPMS_Reset.rsp (+) (AREP) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS1 => Length:=CurrentBuffer[AREP].WriteLength MSAC1S_Write.rsp(+) (Length) LR_State[AREP]:=W-LR-IND	SAME
200	t-state	FSPMS_Reset.rsp (+) (AREP) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS2 => Length:=CurrentBuffer[AREP].WriteLength MSAC2S_Write.rsp(+) (Length) LR_State[AREP]:=W-LR-IND	SAME

Table 426 (continued)

#	Current State	Event /Condition =>Action	Next State
201	t-state	FSPMS_Reset.rsp (-) (AREP, Error Code) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS1 => Error Decode:=DPV1 Error Code 1 :=Error Code Error Code 2:=0 MSAC1S_Write.rsp(-) (Error Decode, Error Code 1 Error Code 2) LR_State[AREP]:=W-LR-IND	SAME
202	t-state	FSPMS_Reset.rsp (-) (AREP, Error Code) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS2 => Error Decode:=DPV1 Error Code 1 :=Error Code Error Code 2:=0 MSAC2S_Write.rsp(-) (Error Decode, Error Code 1 Error Code 2) LR_State[AREP]:=W-LR-IND	SAME
203	t-state	MSAC1S_Write.ind (Res SAP, Slot Number, Index, Length, Data) /LR_State[AREP]==W-LR-IND &SetContext(Res_SAP, Service)=TRUE && (Index=255 && Length >= 4) && (Data.w.STATE.Extended_Function_Num=Get_State) => CurrentBuffer[AREP].Empty:=TRUE CurrentBuffer[AREP].WriteLength:=Length FI Index:=Data.w.STATE.FI_Index FSPMS_Get_FI_State.ind (AREP, Slot Number, FI Index) LR_State[AREP]:=W-LR-RES	SAME
204	t-state	MSAC2S_Write.ind (Res SAP, Slot Number, Index, Length, Data) /LR_State[AREP]==W-LR-IND &SetContext(Res_SAP, Service)=TRUE && (Index=255 && Length >= 4) && (Data.w.STATE.Extended_Function_Num=Get_State) => CurrentBuffer[AREP].Empty:=TRUE CurrentBuffer[AREP].WriteLength:=Length FI Index:=Data.w.STATE.FI_Index FSPMS_Get_FI_State.ind (AREP, Slot Number, FI Index) LR_State[AREP]:=W-LR-RES	SAME
205	t-state	FSPMS_Get_FI_State.rsp (+) (AREP, FI State) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS1 => CurrentBuffer[AREP].Empty:=FALSE CurrentBuffer[AREP].data.r.STATE.FI_State:=FI Sate Length:=CurrentBuffer[AREP].WriteLength MSAC1S_Write.rsp(+) (Length) LR_State[AREP]:=W-LR-IND	SAME
206	t-state	FSPMS_Get_FI_State.rsp (+) (AREP, FI State) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS2 => CurrentBuffer[AREP].Empty:=FALSE CurrentBuffer[AREP].data.r.STATE.FI_State:=FI Sate Length:=CurrentBuffer[AREP].WriteLength MSAC2S_Write.rsp(+) (Length) LR_State[AREP]:=W-LR-IND	SAME
207	t-state	FSPMS_Get_FI_State.rsp(-) (AREP, Error Code) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS1 => Error Decode:=DPV1 Error Code 1 :=Error Code Error Code 2:=0 MSAC1S_Write.rsp(-) (Error Decode, Error Code 1 Error Code 2) LR_State[AREP]:=W-LR-IND	SAME

Table 426 (continued)

#	Current State	Event /Condition =>Action	Next State
208	t-state	FSPMS_Get_FI_State.rsp(-) (AREP, Error Code) /LR_State[AREP]==W-LR-RES &AREP.AR_Type=MS2 => Error Decode:=DPV1 Error Code 1 :=Error Code Error Code 2:=0 MSAC1S_Write.rsp(-) (Error Decode, Error Code 1 Error Code 2) LR_State[AREP]:=W-LR-IND	SAME

6.5.1.4 Functions

Table 427 contains the functions used by the FSPMS, their arguments and their descriptions.

Table 427 – Functions used by the FSPMS

Function name	Description
SetContext(Rem_Add, Service)	This function checks if there exist an entry for the inputs Rem_Add resp. Res_SAP and invoked Service in the ARL and related CRL of the DP-slave. A) If there exist an entry then it returns TRUE and sets the local context according to the current CREP and AREP. B) Otherwise it returns FALSE.
FILL(Outstanding_Alarm_TABLE.Alarm_Status, State)	This function sets each entry of the two dimensional Outstanding_Alarm_TABLE to the given initial value of State. Valid parameters for State are pending/not_pending. The function has no return value.
Search_In_Outstanding_Alarm_TABLE(Alarm_Status)	This function scans each entry of the two dimensional Outstanding_Alarm_TABLE for a specific Alarm_Status. The return value of this function is an index that refers to an Alarm_Table entry that fulfills the Alarm_Status criteria pending/not_pending.
SReset_Req_Alarm_FIFO()	This function resets the FIFO for alarms not yet sent. During this function all entries are cleared. The function has no return value.
Store_To_Req_Alarm_FIFO(Alarm)	By means of this function the Alarm is stored into the Req_Alarm_FIFO. The function has no return value.
Load_From_Req_Alarm_FIFO ()	By means of this function one Alarm is read from the Req_Alarm_FIFO. The return value is an Alarm.
Req_Alarm_FIFO_state ()	This function checks the FIFO for alarms not yet sent. The function has the return values empty/not_empty.
AcIs(Alarm_Type)	This function calculates the Alarm_Type related index as return value as follows: if (Alarm_Type = 1) return 0 if (Alarm_Type = 2) return 1 if (Alarm_Type = 3) return 2 if (Alarm_Type = 4) return 3 if (Alarm_Type = 5) return 4 if (Alarm_Type = 6) return 5 if (Alarm_Type >= 32) && (Alarm_Type <= 126) return 6
IsDynamicAttributeLoadable(Attribute)	This function checks the ARL and CRL if the given attribute is loadable in the current state. For the attribute DXB-Linktable the function returns TRUE: if all elements in the DXB-Linktable have a corresponding attribute in the CRL and non of the corresponding SSCY1S statemachines is in the state RUN.
SetDynamicAttribute(Attribute)	This function stores the given attribute in the ARL or CRL. For the attribute DXB-Linktable each entry is stored at the corresponding CRL attribute.

Macros**ALARM_ENABLED**

```
(
  ((Alarm_Type=3 OR Alarm_Type=4) &&
  Actual_Enabled_Alarms[7]=TRUE)
  OR ((Alarm_Type=2) && Actual_Enabled_Alarms[6]=TRUE)
  OR ((Alarm_Type=1) && Actual_Enabled_Alarms[5]=TRUE)
  OR ((Alarm_Type>=32 && Alarm_Type<=126)
  && Actual_Enabled_Alarms[4]=TRUE)
  OR ((Alarm_Type=5) && Actual_Enabled_Alarms[3]=TRUE)
  OR ((Alarm_Type=6) && Actual_Enabled_Alarms[2]=TRUE)
)
```

6.5.2 FSPMM1**6.5.2.1 Primitive definitions****6.5.2.1.1 Primitives exchanged between AP-Context and FSPMM1**

Table 428 shows the service primitives including their associated parameters issued by the AP-Context and received by the FSPMM1.

Table 428 – Primitives issued by AP-Context to FSPMM1

Primitive name	Source	Associated parameters	Functions
Init DP master CI1.req	AP-Context	Bus Para	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5 and Type 3 Fieldbus IEC 61158-3
Reset DP master CI1.req	AP-Context	(none)	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Load ARL DP master CI1.req	AP-Context	Alarm Max, List of ARL Entries	
Get ARL DP master CI1.req	AP-Context	(none)	
Load CRL DP master CI1.req	AP-Context	Update, List of CRL Entries	
Get CRL DP master CI1.req	AP-Context	(none)	
CRL Slave Activate.req	AP-Context	Activate	
CRL New Prm.req	AP-Context	New Prm	
CRL New Prm Data.req	AP-Context	Prm Data	
Abort DP master CI1.req	AP-Context	AREP	
Mark DP master CI1.req	AP-Context	AREP	
Set Mode DP master CI1.req	AP-Context	AREP, USIF State	
Load Bus Par DP master CI1.req	AP-Context	Bus Para	
Delete SC DP master CI1.req	AP-Context	Address	
Read Value DP master CI1.req	AP-Context	Variable	
Get Slave Diag.req	AP-Context	AREP, CREP	
Set Output.req	AP-Context	AREP, CREP, Slot Number, Output Data, Final	
Get Input.req	AP-Context	AREP, CREP, Slot Number	

Primitive name	Source	Associated parameters	Functions
Global Control.req	AP-Context	AREP, Sync Command, Freeze Command, Group Select	
Read.req	AP-Context	AREP, Slot Number, Index, Length	
Write.req	AP-Context	AREP, Slot Number, Index, Length, Data	
Alarm Ack.req(+)	AP-Context	AREP, Slot Number, Alarm Type, Seq Nr	
Get Master Diag.rsp(+)	AP-Context	AREP, Diagnosis Data	
Get Master Diag.rsp(-)	AP-Context	AREP, Status	
Start Seq.rsp(+)	AP-Context	AREP, Max Len Data Unit	
Start Seq.rsp(-)	AP-Context	AREP, Status	
Download.rsp(+)	AP-Context	AREP	
Download.rsp(-)	AP-Context	AREP, Status	
Upload.rsp(+)	AP-Context	AREP, Data	
Upload.rsp(-)	AP-Context	AREP, Status	
End Seq.rsp(+)	AP-Context	AREP	
End Seq.rsp(-)	AP-Context	AREP, Status	
Act Param.rsp(+)	AP-Context	AREP	
Act Param.rsp(-)	AP-Context	AREP, Status	
Set Time.req	AP-Context	AREP, Time Value Local Time Diff Summertime Accuracy Synchronisation Active Announcement Hour	
Initiate Load.req	AP-Context	AREP, Slot Number, LR Index, Load Type, Load Image Size, Intersegment Request, Timeout User_Specific	
Pull Segment.req	AP-Context	AREP, Slot Number, LR Index, Segment Length	
Push Segment.req	AP-Context	AREP, Slot Number LR Index, Segment Length Segment Number, More Follows, Data	
Terminate Load.req	AP-Context	AREP Slot Number LR Index	

Primitive name	Source	Associated parameters	Functions
Start.req	AP-Context	AREP Slot Number FI Index Execution Argument	
Stop.req	AP-Context	AREP Slot Number FI Index	
Resume.req	AP-Context	AREP Slot Number FI Index Execution Argument	
Reset.req	AP-Context	AREP Slot Number FI Index	
Call.req	FSPMS	AREP Slot Number FI Index Execution Argument	
Get_FI_State.req	FSPMS	AREP Slot Number FI Index	

Table 429 shows the service primitives including their associated parameters issued by the FSPMM1 and received by the AP-Context.

Table 429 – Primitives issued by FSPMM1 to AP-Context

Primitive name	Source	Associated parameters	Functions
Init DP master CI1.cnf	FSPMM1	(none)	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Reset DP master CI1.cnf	FSPMM1	(none)	
Load ARL DP master CI1.cnf(+)	FSPMM1	(none)	
Load ARL DP master CI1.cnf(-)	FSPMM1	Status	
Get ARL DP master CI1.cnf(+)	FSPMM1	Alarm Max, List of ARL Entries	
Get ARL DP master CI1.cnf(-)	FSPMM1	Status	
Load CRL DP master CI1.cnf(+)	FSPMM1	(none)	
Load CRL DP master CI1.cnf(-)	FSPMM1	Status	
Get CRL DP master CI1.cnf(+)	FSPMM1	List of CRL Entries	
Get CRL DP master CI1.cnf(-)	FSPMM1	Status	
CRL Slave Activate.cnf(+)	FSPMM1		
CRL Slave Activate.cnf(-)	FSPMM1	Status	
CRL New Prm.cnf(+)	FSPMM1		
CRL New Prm.cnf(-)	FSPMM1	Status	
CRL New Prm Data.cnf(+)	FSPMM1		
CRL New Prm Data.cnf(-)	FSPMM1	Status	
Mark DP master CI1.cnf(+)	FSPMM1	AREP, Dia	
Mark DP master CI1.cnf(-)	FSPMM1	AREP, Status	
Set Mode DP master CI1.cnf(+)	FSPMM1	AREP, Bus Accessible	
Set Mode DP master CI1.cnf(-)	FSPMM1	AREP, Bus Accessible	
Load Bus Par DP master CI1.cnf	FSPMM1	Status	
Delete SC DP master CI1.cnf	FSPMM1	Status	
Read Value DP master CI1.cnf	FSPMM1	Value, Status	
Get Slave Diag.cnf(+)	FSPMM1	AREP, CREP, Diag Data	
Get Slave Diag.cnf(-)	FSPMM1	AREP, CREP	
Set Output.cnf(+)	FSPMM1	AREP, CREP, Slot Number	
Set Output.cnf(-)	FSPMM1	AREP, CREP, Slot Number, Status	
Get Input.cnf(+)	FSPMM1	AREP, CREP, Slot Number, Input Data	
Get Input.cnf(-)	FSPMM1	AREP, CREP, Slot Number, Status	
Global Control.cnf(+)	FSPMM1	AREP	
Global Control.cnf(-)	FSPMM1	AREP, Status	
Read.cnf(+)	FSPMM1	AREP, Length, Data	
Read.cnf(-)	FSPMM1	AREP, Error Decode, Error Code 1 Error Code 2	

Table 429 (continued)

Primitive name	Source	Associated parameters	Functions
Write.cnf(+)	FSPMM1	AREP, Length	
Write.cnf(-)	FSPMM1	AREP, Error Decode, Error Code 1 Error Code 2	
Alarm Ack.cnf(+)	FSPMM1	AREP, Slot Number, Alarm Type, Seq Nr	
Alarm Ack.cnf(-)	FSPMM1	AREP Slot Number, Alarm Type, Seq Nr	
Get Master Diag.ind	FSPMM1	AREP, Mdiag Identifier	
Start Seq.ind	FSPMM1	AREP, Area Code, Timeout	
Download.ind	FSPMM1	AREP, Area Code, Add Offset, Data	
Upload.ind	FSPMM1	AREP, Area Code, Add Offset, Data Len	
End Seq.ind	FSPMM1	AREP	
Act Param.ind	FSPMM1	AREP, Area Code, Activate	
Act Para Brct.ind	FSPMM1	AREP, Area Code	
New Slave Diag.ind	FSPMM1	AREP, CREP	
New Input.ind	FSPMM1	AREP	
Alarm Notification.ind	FSPMM1	AREP, Slot Number, Alarm Type, Seq Nr, Add Ack, Alarm Specifier, Alarm Data	
DP master CI1 Mode Changed.ind	FSPMM1	AREP, USIF State	
DP master CI1 Event.ind		Event, Add Info	
DP master CI1 Started.ind	FSPMM1	AREP	
DP master CI1 Stopped.ind	FSPMM1	AREP	
Abort.ind	FSPMM1	AREP	
DP master CI1 Reject.ind	FSPMM1	AREP, Reason	
DP master CI1 Fault.ind	FSPMM1	(none)	
SYNCH.ind	FSPMM1	AREP	
SYNCH Delayed.ind	FSPMM1	AREP, T _{SH}	
DX Finished.ind	FSPMM1	AREP	
Set Time.ind	FSPMM1	AREP, Time Value, Local Time Diff, Summertime, Accuracy, Synchronisation Active, Announcement Hour	

Table 429 (continued)

Primitive name	Source	Associated parameters	Functions
Set Time.cnf	FSPMM1	AREP, Status	
Sync Interval Violation.ind	FSPMM1	AREP	
Initiate Load.cnf(+)	FSPMM1	AREP, Actual LR Size, Max Response Delay, Max Segment Length User Specific	
Initiate Load.cnf(-)	FSPMM1	AREP, Error Code	
Pull Segment.cnf(+)	FSPMM1	AREP, Segment Length, Segment Number, More Follows, Data	
Pull Segment.cnf(-)	FSPMM1	AREP, Error Code	
Push Segment.cnf(+)	FSPMM1	AREP	
Push Segment.cnf(-)	FSPMM1	AREP, Error Code	
Terminate Load.cnf(+)	FSPMM1	AREP	
Terminate Load.cnf(-)	FSPMM1	AREP, Error Code	
Start.cnf(+)	FSPMM1	AREP	
Start.cnf(-)	FSPMM1	AREP Error Code	
Stop.cnf(+)	FSPMM1	AREP	
Stop.cnf(-)	FSPMM1	AREP Error Code	
Resume.cnf(+)	FSPMM1	AREP	
Resume.cnf(-)	FSPMM1	AREP Error Code	
Reset.cnf(+)	FSPMM1	AREP	
Reset.cnf(-)	FSPMM1	AREP Error Code	
Call.cnf(+)	FSPMM1	AREP Result Argument	
Call.cnf(-)	FSPMM1	AREP Error Code	
Get FI State.cnf(+)	FSPMM1	AREP FI State	
Get FI State.cnf(-)	FSPMM1	AREP Error Code	

6.5.2.1.2 Parameters of FSPMM1 primitives

The parameters used with the primitives exchanged between the FSPMM1 and the AP-Context are described in FAL Service Definition Type 3 Fieldbus IEC 61158-5.

6.5.2.2 State machine description

This Machine is used to co-ordinate the Interactions between AP-Context and DMPMM1, MSCY1M, MSAC1M, MSAL1M and MMAC1. As the support of several State Machines for the communication with individual slave as well as timing constrains of the operation of slave require a consistent scheduling, this task is accomplished by FSPMM1 as well.

For each DP-slave an individual set of state machines (MSCY1M, MSAL1M, MSAC1M) will be established which will be controlled by the Scheduler in the DP-master (Class 1). The Scheduler will be controlled by the AP-Context with special services. The Slave-Handlers(MSCY1M) are triggered sequentially from the FSPMS-Scheduler to perform the necessary actions. The FSPMM1 notifies the operation modes CLEAR and OPERATE in fixed

time intervals (Dx_Control_Timer) to the DP-slaves. This occurs also at state transitions of FSPMM1.

The Scheduler provides four operation modes for the User:

OFFLINE

STOP

CLEAR

OPERATE

Local Variables

Go_AClr

(Boolean)

State variable which indicates by the value True that after finishing the current Slave cycle the FSPMM1 has to force all DP-slaves into a safe state (Scheduler-State = CLEAR).

Internal_Go_AClr

(Boolean)

Only after finishing the current Slave cycle the FSPMS has to go to state CLEAR if Go_AClr is True. During each cycle the state variable Internal_Go_AClr stores the actual value of Go_AClr. At the end of each cycle the value of Internal_Go_AClr is copied into Go_AClr.

Mark_Active

(Unsigned8)

Indicates the state of the local function Mark.

Range:

Possible values are:

0 => Mark is not active

1 => Mark is active, but not yet in execution

2 => Mark is active and in execution

Diag_Active

(Boolean)

Used to store the information if at least one of the DP-slaves is not in the data exchange mode (True). The information is issued to the User with Mark.cnf.

UGC_Count

(Unsigned8)

Global_Control requests of the User are possible at any time during the FSPMM1 operation modes CLEAR and OPERATE. Confirmations of these requests will be passed to the User. The FSPMM1 uses the service Global_Control to send cyclic Global_Control requests with its actual operation mode to all assigned DP-slaves. Confirmations of cyclic Global_Control requests have to be utilised by the FSPMM1. The FSPMM1 has to know which Global_Control.cnf belongs to which previously called Global_Control.req It is not possible to resolve this relationship only by using DL services. Therefore the FSPMM1 shall get this relationship information by using the sequential order of the received Global_Control confirmations. The counters UGC_Count and CGC_Count are used to store information about the order of the requests and also information about the order of the confirmations.

With the UGC_Count counter all User Global_Control requests are counted incrementally. The initial value is 0. It is decremented with each Global_Control.cnf. In addition the counter is related with the parameter Bus_Para.Max_User_Global_Control.

Range: 0 .. Bus_Para.Max_User_Global_Control

CGC_Count

(Unsigned8)

With this counter all cyclic Global_Control requests are counted which were started by the Scheduler. The initial value is 0. With each new cyclic Global_Control.req UGC_Count is incremented and its value is copied to CGC_Count. UGC_Count also is decremented with each new Global_Control.cnf. If this counter reaches the value 1, the actual Global_Control.cnf is a confirmation for a cyclic Global_Control.req

Range: 0 .. Bus_Para.Max_User_Global_Control

Bus_Accessible

(Boolean)

Indicates the actual state of the bus system. If services can be sent, its value is True.

Act_Rem_Add

(Unsigned8)

This variable stores the actually used station address of a DP-slave.

Range: 0 .. 125

Act_msac1m_Max_L_sdu_len

(Unsigned8)

This variable is used to store the actual value of the parameter msac1m_Max_L_sdu_len.

Range: 0 .. 240

User_Sched_Reset

(Boolean)

This variable indicates whether the actual reset was started by the User (AP-Context) or by any other State Machine.

Act_New_Bus_Para

(Bus_Para)

This structure is used to save an actual bus parameter set passed from the AP-Context to FSPMM1 temporarily.

DX_Lock

(Boolean)

This variable marks an active data exchange cycle to lock the data buffer and block user access. The value TRUE is set when the first Continue Slave Handler.req is issued until DX Finished.ind. The value remains FALSE until the next SYNCH.ind starts again the data exchange cycle.

DX_Synch

(Boolean)

This variable marks the interval between a SYNCH.ind and the next data exchange cycle. The value TRUE is set when the SYNCH.ind is received until the first Continue Slave Handler.req is issued. The value remains FALSE until the next SYNCH.ind.

Chg Buffer

(Boolean)

The value TRUE indicates that the buffers for input and output data below shall be changed with the next DX Finished.ind. It remains FALSE until the next Set Output.req (Final=TRUE) is received.

Bfr_Set_Output

(Octet String)

This variable is a local buffer for output data that belongs to the application. It allows asynchronous access from the application process to the output data in Buffered Synchronised Mode. The contents of the buffer is refreshed by the FSPMM1 when the application process has issued a Set Output.req (Final=TRUE) and the DX Finished.ind has been received. The buffer may be accessed again after the next SYNCH.ind.

Bfr_Get_Input

(Octet String)

This variable is a local buffer for input data that belongs to the application. It allows asynchronous access from the application process to the input data in Buffered Synchronised Mode. The refreshment is done together with the Bfr_Set_Output.

Set_Output

(Octet String)

This variable is a local buffer for output data that belongs to the network. It allows asynchronous access from the application process to the output data in Buffered Synchronised Mode. The contents of the buffer is refreshed by the FSPMM1 when the application process has issued a Set Output.req (Final=TRUE) and the DX Finished.ind has been received. The buffer may accessed again after the next SYNCH.ind.

Get_Input

(Octet String)

This variable is a local buffer for input data that belongs to the network. It allows asynchronous access from the application process to the input data in Buffered Synchronised Mode. The refreshment is done together with the Bfr_Set_Output

LR_State

(Array of Unsigned8)

This variable stores the state of the Load Region sequence of the corresponding CREP.

CurrentExtendedFunctionNum

(Array of Unsigned 8)

This variable stores the extended function number of the currently processed LR service of the corresponding CREP.

CurrentSlotNumber

(Array of Unsigned8)

This variable stores the slot number of the currently processed LR service of the corresponding CREP.

Timers**Min_SI_Interval_Timer**

This timer supervises the minimal access control time of the DP-slaves. It is evaluated only within the FSPMM1 states CCLEAR and COPERATE.

Dx_Control_Interval_Timer

If this timer expires the broadcast service Global_Control is executed in order to inform the DP-slave whether the DP-master (Class 1) is in the state CLEAR or OPERATE. If the timer expires and the confirmation of the last Global_Control has not yet been received a physical bus access problem has occurred.

Macros**INDICATE_BUS_ACCESSIBILITY**

If (Accessible = True) Bus_Accessible = True

6.5.2.3 FSPMM1 state table

Table 430 contains the complete description of the FSPMM1 state machine.

The following definition for a state set is used in Table 430.

t_state = STOP or CLEAR or OPERATE

Table 430 – FSPMM1 state table

#	Current State	Event /Condition =>Action	Next State
1	POWER-ON	Load ARL DP Master Cl1.req (Alarm Max, List of ARL Entries) /valid parameters => Load ARL DP Master Cl1.cnf(+)	POWER-ON
2	POWER-ON	Load ARL DP Master Cl1.req (Alarm Max, List of ARL Entries) /invalid parameters => Status := NO Load ARL DP Master Cl1.cnf(-) (Status)	POWER-ON
3	POWER-ON	Get ARL DP Master Cl1.req /ARL loaded => Get ARL DP Master Cl1.cnf(+) (Alarm Max, List of ARL Entries)	POWER-ON
4	POWER-ON	Get ARL DP Master Cl1.req /ARL not loaded => Status := NO Get ARL DP Master Cl1.cnf(-) (Status)	POWER-ON
5	POWER-ON	Load CRL DP Master Cl1.req (List of CRL Entries) /valid parameters => Load CRL DP Master Cl1.cnf(+)	POWER-ON
6	POWER-ON	Load CRL DP Master Cl1.req (List of CRL Entries) /invalid parameters => Status := NO Load CRL DP Master Cl1.cnf(-) (Status)	POWER-ON
7	POWER-ON	Get CRL DP Master Cl1.req /CRL loaded => Get CRL DP Master Cl1.cnf(+) (List of CRL Entries)	POWER-ON
8	POWER-ON	Get CRL DP Master Cl1.req /CRL not loaded => Status := NO Get CRL DP Master Cl1.cnf(-) (Status)	POWER-ON
9	POWER-ON	FSPMM1_Init DP Master Cl1.req(Bus_Para) => User_Init:=TRUE USER_ChangedCritical_Para := FALSE DX_Lock := FALSE Chg_Buffer := FALSE Current Extended Function Num[CREP]:=No_Service LR_State[CREP]:=W-LR-REQ DMPMM1_Minit_DLL.req(Bus_Para)	INIT-DMPM
10	INIT-DMPM	DMPMM1_Minit_DLL.cnf => Act_Rem_Add := 0 MSAC1M_Minit_MSAC1.req(Rem_Add:=Act_Rem_Add)	INIT-AC1
11			
12	INIT-AC1	MSAC1M_Minit_MSAC1.cnf(Rem_Add) /Act_Rem_Add<125 => Act_Rem_Add := Act_Rem_Add + 1 MSAC1M_Minit_MSAC1.req(Rem_Add:=Act_Rem_Add)	INIT-AC1
13	INIT-AC1	MSAC1M_Minit_MSAC1.cnf(Rem_Add) /Act_Rem_Add=125 => Act_Rem_Add := 0 MSAL1M_Minit.req(Rem_Add:=Act_Rem_Add)	INIT-AL1

Table 430 (continued)

#	Current State	Event /Condition =>Action	Next State
14	INIT-AL1	MSAL1M_MInit.cnf(Rem_Add) /Act_Rem_Add<125 => Act_Rem_Add := Act_Rem_Add + 1 MSAL1M_MInit.req(Rem_Add:=Act_Rem_Add)	INIT-AL1
15	INIT-AL1	MSAL1M_MInit.cnf(Rem_Add) /Act_Rem_Add = 125 => Act_Rem_Add := 0 MSCY1M_Minit_MS0.req(Rem_Add:=Act_Rem_Add)	INIT-CY1
16	INIT-CY1	MSCY1M_Minit_MS0.cnf(Rem_Add) /Act_Rem_Add<125 => Act_Rem_Add := Act_Rem_Add + 1 MSCY1M_Minit_MS0.req(Rem_Add:=Act_Rem_Add)	INIT-CY1
17	INIT-CY1	MSCY1M_Minit_MS0.cnf(Rem_Add) /Act_Rem_Add=125 && IsARExistent(MM)=TRUE => MMAC1_Minit_MM.req	INIT-MM1
18	INIT-CY1	MSCY1M_Minit_MS0.cnf(Rem_Add) /Act_Rem_Add=125 && IsARExistent(MM)=FALSE && USER_ChangedCritical_Para = FALSE => Bus_accessible := FALSE if(USER_SetMode_Offline=FALSE) FSPMM1_Init DP Master C11.cnf else FSPMM1_Set_Mode DP Master C11.cnf(+)(AREP, Bus_accessible) USER_SetMode_Offline:=FALSE endif	OFFLINE
19	INIT-CY1	MSCY1M_Minit_MS0.cnf(Rem_Add) /Act_Rem_Add=125 && IsARExistent(MM)=FALSE && USER_ChangedCritical_Para = TRUE => DMPMM1_Set_Bus_Par.req(Bus_Para:=Act_New_Bus_Para)	LDBP
20	INIT-MM1	MMAC1_Minit_MM.cnf /USER_ChangedCritical_Para = TRUE => DMPMM1_Set_Bus_Par.req(Bus_Para:=Act_New_Bus_Para)	LDBP
21	INIT-MM1	MMAC1_Minit_MM.cnf /USER_ChangedCritical_Para = FALSE => Bus_accessible := FALSE if(USER_SetMode_Offline=FALSE) FSPMM1_Init DP Master C11.cnf else FSPMM1_Set_Mode DP Master C11.cnf(+)(AREP, Bus_accessible) USER_SetMode_Offline:=FALSE endif	OFFLINE
22	OFFLINE	FSPMM1_Set_Mode.req(USIF_State) /USIF_State <> Stop && USIF_State <> Offline => FSPMM1_Set_Mode DP Master C11.cnf(-)(AREP, Bus_accessible)	OFFLINE
23	OFFLINE	FSPMM1_Set_Mode.req(USIF_State) /USIF_State = Offline => FSPMM1_Set_Mode DP Master C11.cnf(+)(AREP, Bus_accessible)	OFFLINE
24	OFFLINE	FSPMM1_Set_Mode.req(USIF_State) /USIF_State = Stop && Bus_Para invalid => FSPMM1_Set_Mode DP Master C11.cnf(-)(AREP, Bus_accessible)	OFFLINE
25	OFFLINE	FSPMM1_Set_Mode.req(USIF_State) /USIF_State = Stop && Bus_Para valid => Mark_Active := 0 Diag_Active := False DMPMM1_Set_Bus_Par.req(Bus_Para)	LDBP

Table 430 (continued)

#	Current State	Event /Condition =>Action	Next State
26	OFFLINE	FSPMM1_Mark.req(AREP) => FSPMM1_Mark.cnf(-)(AREP, Status:=NO)	OFFLINE
27	OFFLINE	FSPMM1_Global_Control.req(AREP, Sync_Command, Freeze_Command, Group_Select) => FSPMM1_Global_Control.cnf(-)(AREP, Status:=NO)	OFFLINE
28	OFFLINE	FSPMM1_Load_Bus_Par DP Master C11.req(Bus_Para) => FSPMM1_Load_Bus_Par.cnf(-)(Status:=NO)	OFFLINE
29	OFFLINE	FSPMM1_Delete_SC DP Master C11.req(Address) => DMPMM1_Delete_SC.req(Address)	DELSC-OFF
30	OFFLINE	FSPMM1_Read_Value DP Master C11.req(Variable) => DMPMM1_Read_Value.req(Variable)	RDVAL-OFF
31	LDBP	DMPMM1_Set_Bus_Par.cnf /USER_ChangeCritical_Para = FALSE => FSPMM1_Set_Mode DP Master C11.cnf(+)(AREP, Bus_accessible)	STOP
32	LDBP	DMPMM1_Set_Bus_Par.cnf /USER_ChangeCritical_Para = TRUE => FSPMM1_Load_Bus_Para.cnf(+)	STOP
33	DELSC-OFF	DMPMM1_Delete_SC.cnf => FSPMM1_Delete_SC DP Master C11.cnf	OFFLINE
34	RDVAL-OFF	DMPMM1_Read_Value.cnf(Status,Value) => FSPMM1_Read_Value DP Master C11.cnf(Status, Value)	OFFLINE
35	STOP	FSPMM1_Set_Mode.req(USIF_State) /USIF_State = Offline => USER_SetMode_Offline:=TRUE	RESET
36	STOP	FSPMM1_Set_Mode.req(USIF_State) /USIF_State = Stop => FSPMM1_Set_Mode DP Master C11.cnf(+)(AREP, Bus_accessible)	STOP
37	STOP	FSPMM1_Set_Mode.req(USIF_State) /USIF_State = Operate => FSPMM1_Set_Mode DP Master C11.cnf(-)(AREP, Bus_accessible)	STOP
38	STOP	FSPMM1_Set_Mode.req(USIF_State) /USIF_State = Clear => UGC_Count := 0 CGC_Count := 0 Bus_Accessible := True Go_AClr := Bus_Para.Error_Action_Flag Internal_Go_AClr := False Start Dx_Control_Interval_Timer(Bus_Para.Data_Control_Time/2) MSCY1M_Start_Slave_Handler.req(0 .. 125)	CLEAR
39	STOP	FSPMM1_Mark.req(AREP) => FSPMM1_Mark.cnf(-)(AREP, Status:=NO)	STOP
40	STOP	FSPMM1_Global_Control.req(AREP, Sync_Command, Freeze_Command, Group_Select) => FSPMM1_Global_Control.cnf(-)(AREP, Status:=NO)	STOP
41	STOP	FSPMM1_Load_Bus_Par DP Master C11.req(Bus_Para) /Bus_Para invalid => FSPMM1_Load_Bus_Par.cnf(-)(Status:=IV)	STOP

Table 430 (continued)

#	Current State	Event /Condition =>Action	Next State
42	STOP	FSPMM1_Load_Bus_Par DP Master CI1.req(Bus_Para) /(Bus_Para valid) && (critical parameters unchanged) => DMPMM1_Set_Bus_Par.req(Bus_Para)	LDBP-ST-CPU
43	STOP	FSPMM1_Load_Bus_Par DP Master CI1.req(Bus_Para) /(Bus_Para valid) && (critical parameters changed) => Act_New_Bus_Para := Bus_Para User_ChangedCritical_Para := TRUE	RESET
44	STOP	FSPMM1_Delete_SC DP Master CI1.req(Address) => DMPMM1_Delete_SC.req(Address)	DELSC-ST
45	STOP	FSPMM1_Read_Value DP Master CI1.req(Variable) => DMPMM1_Read_Value.req(Variable)	RDVAL-ST
46	LDBP-ST-CPU	DMPMM1_Set_Bus_Par.cnf => FSPMM1_Load_Bus_Para.cnf(+)	STOP
47	DELSC-ST	DMPMM1_Delete_SC.cnf => FSPMM1_Delete_SC DP Master CI1.cnf	STOP
48	RDVAL-ST	DMPMM1_Read_Value.cnf(Status,Value) => FSPMM1_Read_Value DP Master CI1.cnf(Status, Value)	STOP
49	CLEAR	FSPMM1_Set_Mode.req(USIF_State) /USIF_State = Offline => FSPMM1_Set_Mode DP Master CI1.cnf(-)(AREP, Bus_accessible)	CLEAR
50	CLEAR	FSPMM1_Mark DP Master CI1.req(AREP) /Mark_Active = 0 => Mark_Active := 1	CLEAR
51	CLEAR	FSPMM1_Mark DP Master CI1.req(AREP) /Mark_Active <> 0 => FSPMM1_Mark.cnf(-)(AREP, Status:=NO)	CLEAR
52	CLEAR	FSPMM1_Set_Mode.req(USIF_State) /USIF_State = Stop && Mark_Active = 0 => Stop Dx_Control_Interval_Timer MSCY1M_Stop_Slave_Handler.req(0..125)	SLHSTP
53	CLEAR	FSPMM1_Set_Mode.req(USIF_State) /USIF_State = Stop && Mark_Active <> 0 => Mark_Active := 0 Stop Dx_Control_Interval_Timer FSPMM1_Mark.cnf(-)(AREP; Status:=NO) MSCY1M_Stop_Slave_Handler.req(0..125)	SLHSTP
54	CLEAR	FSPMM1_Set_Mode.req(USIF_State) /USIF_State = Clear => FSPMM1_Set_Mode DP Master CI1.cnf(+)(AREP, Bus_accessible)	CLEAR
55	CLEAR	FSPMM1_Set_Mode.req(USIF_State) /USIF_State = Operate && Go_AClr = False && Internal_Go_AClr = False && CGC_Count > 0 => Start Dx_Control_Interval_Timer(Bus_Para.Data_Control_Time/2	SGC-CO-WGC

Table 430 (continued)

#	Current State	Event /Condition =>Action	Next State
56	CLEAR	FSPMM1_Set_Mode.req(USIF_State) /USIF_State = Operate && Go_AClr = False && Internal_Go_AClr = False && CGC_Count = 0 => UGC_Count := UGC_Count+1 CGC_Count := UGC_Count Start Dx_Control_Interval_Timer(Bus_Para.Data_Control_Time/2 DMPMM1_Global_Control.req(Rem_Add:=127, Control_Command:=0, Group_Select:=0)	SGC-CO
57	CLEAR	FSPMM1_Set_Mode.req(USIF_State) /USIF_State = Operate && (Go_AClr = True Internal_Go_AClr = True) => FSPMM1_Set_Mode DP Master Cl1.cnf(-)(AREP, Bus_accessible)	CLEAR
58	CLEAR	FSPMM1_Global_Control.req(AREP, Sync_Command, Freeze_Command, Group_Select) /NOT received all MSCY1M_Start_Slave_Handler.cnf => FSPMM1_Global_Control.cnf(-)(AREP, Status:=NO)	CLEAR
59	CLEAR	FSPMM1_Global_Control.req(AREP, Sync_Command, Freeze_Command, Group_Select) /received all MSCY1M_Start_Slave_Handler.cnf && CGC_Count = 0 && UGC_Count >= Bus_Para.Max_User_Global_Control => FSPMM1_Global_Control.cnf(-)(AREP, Status:=NO)	CLEAR
60	CLEAR	FSPMM1_Global_Control.req(AREP, Sync_Command, Freeze_Command, Group_Select) /received all MSCY1M_Start_Slave_Handler.cnf && CGC_Count = 0 && UGC_Count < Bus_Para.Max_User_Global_Control && (Isochronous Mode = Not Synchronized Group_Select < 0x80) => UGC_Count := UGC_Count+1 DMPMM1_Global_Control.req(Rem_Add:=CREP.Rem_Add, Control_Command.1:=1, Group_Select)	CLEAR
61	CLEAR	FSPMM1_Global_Control.req(AREP, Sync_Command, Freeze_Command, Group_Select) /received all MSCY1M_Start_Slave_Handler.cnf && CGC_Count > 0 && UGC_Count >= Bus_Para.Max_User_Global_Control+1 => FSPMM1_Global_Control.cnf(-)(AREP, Status:=NO)	CLEAR
62	CLEAR	FSPMM1_Global_Control.req(AREP, Sync_Command, Freeze_Command, Group_Select) /received all MSCY1M_Start_Slave_Handler.cnf && CGC_Count > 0 && UGC_Count < Bus_Para.Max_User_Global_Control+1 && (Isochronous Mode = Not Synchronized Group_Select < 0x80) => UGC_Count := UGC_Count+1 DMPMM1_Global_Control.req(Rem_Add:=CREP.Rem_Add, Control_Command.1:=1, Group_Select)	CLEAR
63	CLEAR	DMPMM1_Global_Control.cnf(Rem_Add, Status) /CGC_Count = 0 => INDICATE_BUS_ACCESSIBILITY UGC_Count := UGC_Count-1 FSPMM1_Global_Control.cnf(+)(AREP)	CLEAR

Table 430 (continued)

#	Current State	Event /Condition =>Action	Next State
64	CLEAR	DMPMM1_Global_Control.cnf(Rem_Add, Status) /CGC_Count = 1 => INDICATE_BUS_ACCESSIBILITY UGC_Count := UGC_Count-1 CGC_Count := 0	CLEAR
65	CLEAR	FSPMM1_Global_Control.req(AREP, Sync_Command, Freeze_Command, Group_Select) /received all MSCY1M_Start_Slave_Handler.cnf && CGC_Count = 0 && UGC_Count < Bus_Para.Max_User_Global_Control && Isochronous Mode = (Buffered Synchronized Enhanced Synchronized) && Group_Select >= 0x80 => FSPMM1_Global_Control.cnf(-)(AREP, Status:=GE)	CLEAR
66	CLEAR	FSPMM1_Global_Control.req(AREP, Sync_Command, Freeze_Command, Group_Select) /received all MSCY1M_Start_Slave_Handler.cnf && CGC_Count > 0 && UGC_Count < Bus_Para.Max_User_Global_Control+1 && Isochronous Mode = (Buffered Synchronized Enhanced Synchronized) && Group_Select >= 0x80 => FSPMM1_Global_Control.cnf(-)(AREP, Status:=GE)	CLEAR
67	CLEAR	DMPMM1_Global_Control.cnf(Rem_Add, Status) /CGC_Count > 1 => INDICATE_BUS_ACCESSIBILITY UGC_Count := UGC_Count-1 CGC_Count := CGC_Count-1 FSPMM1_Global_Control.cnf(+)(AREP)	CLEAR
68	CLEAR	MSCY1M_Start_Slave_Handler.cnf(Rem_Add) /NOT received all MSCY1M_Start_Slave_Handler.cnf => ignore	CLEAR
69	CLEAR	MSCY1M_Start_Slave_Handler.cnf(Rem_Add) /received all MSCY1M_Start_Slave_Handler.cnf && Isochronous Mode = Not Synchronized => Internal_Go_AClr := False Start_Min_SI_Interval_Timer(Bus_Para.Min_Slave_Interval) Set_Mode.cnf(Status:=OK, Bus_Accessible) MSCY1M_Cont_Slave_Handler.req(0..125, Output_Clear:=True)	CLEAR
70	CLEAR	MSCY1M_Start_Slave_Handler.cnf(Rem_Add) /received all MSCY1M_Start_Slave_Handler.cnf && Isochronous Mode <> Not Synchronized => Internal_Go_AClr := False Set_Mode.cnf(Status:=OK, Bus_Accessible) MSCY1M_Cont_Slave_Handler.req(0..125, Output_Clear:=True)	CLEAR
71	CLEAR	DMPMM1_SYNCH.ind => FSPMM1_DP Master C11 Fault.ind	RESET
72	CLEAR	MSCY1M_Cont_Slave_Handler.cnf(Rem_Add, Diag, No_AClr) /NOT received all MSCY1M_Cont_Slave_Handler.cnf && Mark_Active = 2 => Diag_Active := Diag_Active OR Diag Internal_Go_AClr := Internal_Go_AClr OR (Bus_Para.Error_Action_Flag AND NOT No_AClr)	CLEAR
73	CLEAR	MSCY1M_Cont_Slave_Handler.cnf(Rem_Add, Diag, No_AClr) /NOT received all MSCY1M_Cont_Slave_Handler.cnf && Mark_Active <> 2 => Internal_Go_AClr := Internal_Go_AClr OR (Bus_Para.Error_Action_Flag AND NOT No_AClr)	CLEAR

Table 430 (continued)

#	Current State	Event /Condition =>Action	Next State
74	CLEAR	MSCY1M_Cont_Slave_Handler.cnf(Rem_Add, Diag, No_AClr) /received all MSCY1M_Cont_Slave_Handler.cnf && Mark_Active = 0 => Internal_Go_AClr := Internal_Go_AClr OR (Bus_Para.Error_Action_Flag AND NOT No_AClr) DX_Lock := FALSE	CCLEAR
75	CLEAR	MSCY1M_Cont_Slave_Handler.cnf(Rem_Add, Diag, No_AClr) /received all MSCY1M_Cont_Slave_Handler.cnf && Mark_Active = 1 => Mark_Active:=2 Diag_Active := False Internal_Go_AClr := Internal_Go_AClr OR (Bus_Para.Error_Action_Flag AND NOT No_AClr) DX_Lock := FALSE	CCLEAR
76	CLEAR	MSCY1M_Cont_Slave_Handler.cnf(Rem_Add, Diag, No_AClr) /received all MSCY1M_Cont_Slave_Handler.cnf && Mark_Active = 2 => Diag_Active := Diag_Active OR Diag Internal_Go_AClr := Internal_Go_AClr OR (Bus_Para.Error_Action_Flag AND NOT No_AClr) Mark_Active:=0 DX_Lock := FALSE FSPMM1_Mark.cnf(+)(AREP, Dia:=Diag_Active)	CCLEAR
77	CLEAR	Dx_Control_Interval_Timer expired /CGC_Count = 0 => Start Dx_Control_Interval_Timer(Bus_Para.Data_Control_Time/2) UGC_Count := UGC_Count+1 CGC_Count := UGC_Count DMPMM1_Global_Control.req(Rem_Add:=127, Control_Command:=2, Group_Select:=0)	CLEAR
78	CLEAR	Dx_Control_Interval_Timer expired /CGC_Count > 0 => Start Dx_Control_Interval_Timer(Bus_Para.Data_Control_Time/2) Bus_Accessible := False	CLEAR
79	CLEAR	FSPMM1_Load_Bus_Par DP Master C11.req(Bus_Para) /Bus_Para invalid => FSPMM1_Load_Bus_Par.cnf(-)(Status:=IV)	CLEAR
80	CLEAR	FSPMM1_Load_Bus_Par DP Master C11.req(Bus_Para) /(Bus_Para valid) && (critical parameters changed) => FSPMM1_Load_Bus_Par.cnf(-)(Status:=NO)	CLEAR
81	CLEAR	FSPMM1_Load_Bus_Par DP Master C11.req(Bus_Para) /(Bus_Para valid) && (critical parameters unchanged) => DMPMM1_Set_Bus_Par.req(Bus_Para)	LDBP-CL-CPU
82	CLEAR	FSPMM1_Delete_SC DP Master C11.req(Address) => DMPMM1_Delete_SC.req(Address)	DELSC-CL
83	CLEAR	FSPMM1_Read_Value DP Master C11.req(Variable) => DMPMM1_Read_Value.req(Variable)	RDVAL-CL
84	SLHSTP	MSCY1M_Stop_Slave_Handler.cnf(Rem_Add) /NOT received all MSCY1M_Stop_Slave_Handler.cnf => ignore	SLHSTP
85	SLHSTP	MSCY1M_Stop_Slave_Handler.cnf(Rem_Add) /received all MSCY1M_Stop_Slave_Handler.cnf => Set_Mode.cnf(Status:=OK, Bus_Accessible)	STOP

Table 430 (continued)

#	Current State	Event /Condition =>Action	Next State
86	SGC-CO-WGC	Dx_Control_Interval_Timer expired => Start Dx_Control_Interval_Timer(Bus_Para.Data_Control_Time/2) Bus_Accessible := False FSPMM1_Set_Mode DP Master C11.cnf(-)(AREP, Bus_accessible)	CLEAR
87	SGC-CO-WGC	DMPMM1_Global_Control.cnf(Rem_Add, Status) /CGC_Count > 1 => INDICATE_BUS_ACCESSIBILITY UGC_Count := UGC_Count-1 CGC_Count := CGC_Count-1 FSPMM1_Global_Control.cnf(+)(AREP)	SGC-CO-WGC
88	SGC-CO-WGC	DMPMM1_Global_Control.cnf(Rem_Add, Status) /CGC_Count = 1 => INDICATE_BUS_ACCESSIBILITY CGC_Count := UGC_Count DMPMM1_Global_Control.req(Rem_Add:=127, Control_Command:=0, Group_Select:=0)	SGC-CO
89	SGC-CO	Dx_Control_Interval_Timer expired => Start Dx_Control_Interval_Timer(Bus_Para.Data_Control_Time/2) Bus_Accessible := False Set_Mode.cnf(Status:=OK, Bus_Accessible)	OPERATE
90	SGC-CO	DMPMM1_Global_Control.cnf(Rem_Add, Status) /CGC_Count > 1 => INDICATE_BUS_ACCESSIBILITY UGC_Count := UGC_Count-1 CGC_Count := CGC_Count-1 FSPMM1_Global_Control.cnf(+)(AREP)	SGC-CO
91	SGC-CO	DMPMM1_Global_Control.cnf(Rem_Add, Status) /CGC_Count = 1 => INDICATE_BUS_ACCESSIBILITY UGC_Count := UGC_Count-1 CGC_Count := 0 FSPMM1_Set_Mode DP Master C11.cnf(+)(AREP, Bus_accessible)	OPERATE
92	CCLEAR	Min_SI_Interval_Timer expired => Go_AClr := Internal_Go_AClr Internal_Go_AClr := False Start Min_SI_Interval_Timer(Bus_Para.Min_Slave_Interval) MSCY1M_Cont_Slave_Handler.req(0..125, Output_Clear:=True)	CLEAR
93	CCLEAR	DMPMM1_SYNCH.ind => DX_Lock := TRUE Output_Data = Set_Output MSCY1M_Cont_Slave_Handler.req(0..125, Output_Clear:=False) MSCY1M_Set_Output.req(Rem_Add:=0..125, Slot_Number:=ALL, Output_Data) MSCY1M_Get_Input.req(Rem_Add:=0..125, Slot_Number:=ALL) FSPMM1_SYNCH.ind (AREP)	CLEAR
94	LDBP-CL-CPU	DMPMM1_Set_Bus_Par.cnf => FSPMM1_Load_Bus_Par.cnf(+)	CLEAR
95	DELSC-CL	DMPMM1_Delete_SC.cnf => FSPMM1_Delete_SC DP Master C11.cnf	CLEAR
96	RDVAL-CL	DMPMM1_Read_Value.cnf(Status, Value) => FSPMM1_Read_Value DP Master C11.cnf(Status, Value)	CLEAR
97	OPERATE	FSPMM1_Set_Mode.req(USIF_State) /USIF_State <> Clear && USIF_State <> Operate => FSPMM1_Set_Mode DP Master C11.cnf(-)(AREP, Bus_accessible)	OPERATE

Table 430 (continued)

#	Current State	Event /Condition =>Action	Next State
98	OPERATE	FSPMM1_Set_Mode.req(USIF_State) /USIF_State = Operate => FSPMM1_Set_Mode DP Master CI1.cnf(+)(AREP, Bus_accessible)	OPERATE
99	OPERATE	FSPMM1_Set_Mode.req(USIF_State) /USIF_State = Clear && CGC_Count > 0 => Start Dx_Control_Interval_Timer(Bus_Para.Data_Control_Time/2)	SGC-OC-WGC
100	OPERATE	FSPMM1_Set_Mode.req(USIF_State) /USIF_State = Clear && CGC_Count = 0 => Start Dx_Control_Interval_Timer(Bus_Para.Data_Control_Time/2 UGC_Count := UGC_Count+1 CGC_Count := UGC_Count DMPMM1_Global_Control.req(Rem_Add:=127, Control_Command:=2, Group_Select:=0)	SGC-OC
101	OPERATE	FSPMM1_Mark DP Master CI1.req(AREP) /Mark_Active = 0 => Mark_Active := 1	OPERATE
102	OPERATE	FSPMM1_Mark DP Master CI1.req(AREP) /Mark_Active <> 0 => FSPMM1_Mark.cnf(-)(AREP, Status:=NO)	OPERATE
103	OPERATE	FSPMM1_Global_Control.req(AREP, Sync_Command, Freeze_Command, Group_Select) /Control_Command.1 = 1 => FSPMM1_Global_Control.cnf(-)(AREP, Status:=NO)	OPERATE
104	OPERATE	FSPMM1_Global_Control.req(AREP, Sync_Command, Freeze_Command, Group_Select) /Control_Command.1 = 0 && CGC_Count = 0 && UGC_Count >= Bus_Para.Max_User_Global_Control => FSPMM1_Global_Control.cnf(-)(AREP, Status:=NO)	OPERATE
105	OPERATE	FSPMM1_Global_Control.req(AREP, Sync_Command, Freeze_Command, Group_Select) /Control_Command.1 = 0 && CGC_Count = 0 && UGC_Count < Bus_Para.Max_User_Global_Control && (Isochronous Mode = Not Synchronized Group_Select < 0x80) => UGC_Count := UGC_Count+1 DMPMM1_Global_Control.req(Rem_Add:=CREP.Rem_Add, Control_Command.1:=0, Group_Select)	OPERATE
106	OPERATE	FSPMM1_Global_Control.req(AREP, Sync_Command, Freeze_Command, Group_Select) /Control_Command.1 = 0 && CGC_Count > 0 && UGC_Count >= Bus_Para.Max_User_Global_Control+1 => FSPMM1_Global_Control.cnf(-)(AREP, Status:=NO)	OPERATE
107	OPERATE	FSPMM1_Global_Control.req(AREP, Sync_Command, Freeze_Command, Group_Select) /Control_Command.1 = 0 && CGC_Count > 0 && UGC_Count < Bus_Para.Max_User_Global_Control+1 && (Isochronous Mode = Not Synchronized Group_Select < 0x80) => UGC_Count := UGC_Count+1 DMPMM1_Global_Control.req(Rem_Add:=CREP.Rem_Add, Control_Command.1:=0, Group_Select)	OPERATE

Table 430 (continued)

#	Current State	Event /Condition =>Action	Next State
108	OPERATE	DMPMM1_Global_Control.cnf(Rem_Add, Status) /CGC_Count = 0 => INDICATE_BUS_ACCESSIBILITY UGC_Count := UGC_Count-1 FSPMM1_Global_Control.cnf(+)(AREP)	OPERATE
109	OPERATE	DMPMM1_Global_Control.cnf(Rem_Add, Status) /CGC_Count = 1 => INDICATE_BUS_ACCESSIBILITY UGC_Count := UGC_Count-1 CGC_Count := 0	OPERATE
110	OPERATE	DMPMM1_Global_Control.cnf(Rem_Add, Status) /CGC_Count > 1 => INDICATE_BUS_ACCESSIBILITY UGC_Count := UGC_Count-1 CGC_Count := CGC_Count-1 FSPMM1_Global_Control.cnf(+)(AREP)	OPERATE
111	OPERATE	FSPMM1_Global_Control.req(AREP, Sync_Command, Freeze_Command, Group_Select) /Control_Command.1 = 0 && CGC_Count = 0 && UGC_Count < Bus_Para.Max_User_Global_Control && Isochronous Mode = (Buffered Synchronized Enhanced Synchronized) && Group_Select >= 0x80 => FSPMM1_Global_Control.cnf(-)(AREP, Status:=GE)	OPERATE
112	OPERATE	FSPMM1_Global_Control.req(AREP, Sync_Command, Freeze_Command, Group_Select) /Control_Command.1 = 0 && CGC_Count > 0 && UGC_Count < Bus_Para.Max_User_Global_Control+1 && Isochronous Mode = (Buffered Synchronized Enhanced Synchronized) && Group_Select >= 0x80 => FSPMM1_Global_Control.cnf(-)(AREP, Status:=GE)	OPERATE
113	OPERATE	DMPMM1_SYNCH.ind => FSPMM1_DP Master CI1 Fault.ind	RESET
114	OPERATE	MSCY1M_Cont_Slave_Handler.cnf(Rem_Add, Diag, No_AClr) /NOT received all MSCY1M_Cont_Slave_Handler.cnf && Mark_Active = 2 => Diag_Active:=Diag_Active OR Diag Go_AClr := Go_AClr OR (Bus_Para.Error_Action_Flag AND NOT No_AClr)	OPERATE
115	OPERATE	MSCY1M_Cont_Slave_Handler.cnf(Rem_Add, Diag, No_AClr) /NOT received all MSCY1M_Cont_Slave_Handler.cnf && Mark_Active <> 2 => Go_AClr := Go_AClr OR (Bus_Para.Error_Action_Flag AND NOT No_AClr)	OPERATE
116	OPERATE	MSCY1M_Cont_Slave_Handler.cnf(Rem_Add, Diag, No_AClr) /received all MSCY1M_Cont_Slave_Handler.cnf && Mark_Active = 0 && (Isochronous Mode = Not Synchronized Isochronous Mode = Enhanced Synchronized) => Go_AClr := Go_AClr OR (Bus_Para.Error_Action_Flag AND NOT No_AClr) DX_Lock := FALSE FSPMM1_DX_Finished.ind (AREP)	COPERATE

Table 430 (continued)

#	Current State	Event /Condition =>Action	Next State
117	OPERATE	MSCY1M_Cont_Slave_Handler.cnf(Rem_Add, Diag, No_AClr) /received all MSCY1M_Cont_Slave_Handler.cnf && Mark_Active = 1 && (Isochronous Mode = Not Synchronized Isochronous Mode = Enhanced Synchronized) => Mark_Active:=2 Diag_Active := False Go_AClr := Go_AClr OR (Bus_Para.Error_Action_Flag AND NOT No_AClr) DX_Lock := FALSE FSPMM1_DX_Finished.ind (AREP)	COPERATE
118	OPERATE	MSCY1M_Cont_Slave_Handler.cnf(Rem_Add, Diag, No_AClr) /received all MSCY1M_Cont_Slave_Handler.cnf && Mark_Active = 2 && (Isochronous Mode = Not Synchronized Isochronous Mode = Enhanced Synchronized) => Diag_Active:=Diag_Active OR Diag Go_AClr := Go_AClr OR (Bus_Para.Error_Action_Flag AND NOT No_AClr) Mark_Active:=0 DX_Lock := FALSE FSPMM1_Mark.cnf(+)(AREP, Dia:=Diag_Active) DX_Finished.ind (AREP)	COPERATE
119	OPERATE	MSCY1M_Cont_Slave_Handler.cnf(Rem_Add, Diag, No_AClr) /received all MSCY1M_Cont_Slave_Handler.cnf && Mark_Active = 0 && Isochronous Mode = Buffered Synchronized => Go_AClr := Go_AClr OR (Bus_Para.Error_Action_Flag AND NOT No_AClr) DX_Lock := FALSE Chg_Bfr (Bfr_Get_Input, Get_Input) FSPMM1_DX_Finished.ind (AREP)	COPERATE
120	OPERATE	MSCY1M_Cont_Slave_Handler.cnf(Rem_Add, Diag, No_AClr) /received all MSCY1M_Cont_Slave_Handler.cnf && Mark_Active = 1 && Isochronous Mode = Buffered Synchronized => Mark_Active:=2 Diag_Active := False Go_AClr := Go_AClr OR (Bus_Para.Error_Action_Flag AND NOT No_AClr) DX_Lock := FALSE Chg_Bfr (Bfr_Get_Input, Get_Input) FSPMM1_DX_Finished.ind (AREP)	COPERATE
121	OPERATE	MSCY1M_Cont_Slave_Handler.cnf(Rem_Add, Diag, No_AClr) /received all MSCY1M_Cont_Slave_Handler.cnf && Mark_Active = 2 && Isochronous Mode = Buffered Synchronized => Diag_Active:=Diag_Active OR Diag Go_AClr := Go_AClr OR (Bus_Para.Error_Action_Flag AND NOT No_AClr) Mark_Active:=0 DX_Lock := FALSE FSPMM1_Mark.cnf(+)(AREP, Dia:=Diag_Active) DX_Finished.ind (AREP)	COPERATE
122	OPERATE	Dx_Control_Interval_Timer expired /CGC_Count = 0 => Start Dx_Control_Interval_Timer(Bus_Para.Data_Control_Time/2) UGC_Count := UGC_Count+1 CGC_Count := UGC_Count DMPMM1_Global_Control.req(Rem_Add:=127, Control_Command:=0, Group_Select:=0)	OPERATE

Table 430 (continued)

#	Current State	Event /Condition =>Action	Next State
123	OPERATE	Dx_Control_Interval_Timer expired /CGC_Count > 0 => Start Dx_Control_Interval_Timer(Bus_Para.Data_Control_Time/2) Bus_accessible := False	OPERATE
124	OPERATE	FSPMM1_Load_Bus_Par DP Master C11.req(Bus_Para) /Bus_Para invalid => FSPMM1_Load_Bus_Par.cnf(-)(Status:=IV)	OPERATE
125	OPERATE	FSPMM1_Load_Bus_Par DP Master C11.req(Bus_Para) /(Bus_Para valid) && (critical parameters changed) => FSPMM1_Load_Bus_Par.cnf(-)(Status:=NO)	OPERATE
126	OPERATE	FSPMM1_Load_Bus_Par DP Master C11.req(Bus_Para) /(Bus_Para valid) && (critical parameters unchanged) => DMPMM1_Set_Bus_Par.req(Bus_Para)	LDBP-OP-CPU
127	OPERATE	FSPMM1_Delete_SC DP Master C11.req(Address) => DMPMM1_Delete_SC.req(Address)	DELSC-OP
128	OPERATE	FSPMM1_Read_Value DP Master C11.req(Variable) => DMPMM1_Read_Value.req(Variable)	RDVAL-OP
129	SGC-OC-WGC	Dx_Control_Interval_Timer expired /Go_AClr = False => Start Dx_Control_Interval_Timer(Bus_Para.Data_Control_Time/2) Bus_Accessible := False FSPMM1_Set_Mode DP Master C11.cnf(-)(AREP, Bus_accessible)	OPERATE
130	SGC-OC-WGC	Dx_Control_Interval_Timer expired /Go_AClr = True => Start Dx_Control_Interval_Timer(Bus_Para.Data_Control_Time/2) Bus_Accessible := False	SGC-OC-WGC
131	SGC-OC-WGC	DMPMM1_Global_Control.cnf(Rem_Add, Status) /CGC_Count > 1 => INDICATE_BUS_ACCESSIBILITY UGC_Count := UGC_Count-1 CGC_Count := CGC_Count-1 FSPMM1_Global_Control.cnf(+)(AREP)	SGC-OC-WGC
132	SGC-OC-WGC	DMPMM1_Global_Control.cnf(Rem_Add, Status) /CGC_Count = 1 => INDICATE_BUS_ACCESSIBILITY CGC_Count := UGC_Count DMPMM1_Global_Control.req(Rem_Add:=127, Control_Command:=2, Group_Select:=0)	SGC-OC
133	SGC-OC	Dx_Control_Interval_Timer expired /Go_AClr = False => Start Dx_Control_Interval_Timer(Bus_Para.Data_Control_Time/2) Bus_Accessible := False Set_Mode.cnf(Status:=OK, Bus_Accessible)	CLEAR
134	SGC-OC	Dx_Control_Interval_Timer expired /Go_AClr = True => Start Dx_Control_Interval_Timer(Bus_Para.Data_Control_Time/2) Bus_Accessible := False FSPMM1_DP Master C11 Mode_Changed.ind(AREP, USIF_State:=Clear) MSCY1M_Cont_Slave_Handler.req(0..125, Output_Clear:=True)	CLEAR

Table 430 (continued)

#	Current State	Event /Condition =>Action	Next State
135	SGC-OC	DMPMM1_Global_Control.cnf(Rem_Add, Status) /CGC_Count > 1 => INDICATE_BUS_ACCESSIBILITY UGC_Count := UGC_Count-1 CGC_Count := CGC_Count-1 FSPMM1_Global_Control.cnf(+)(AREP)	SGC-OC
136	SGC-OC	DMPMM1_Global_Control.cnf(Rem_Add, Status) /CGC_Count = 1 && Go_AClr = False => INDICATE_BUS_ACCESSIBILITY UGC_Count := UGC_Count-1 CGC_Count := 0 FSPMM1_Set_Mode DP Master Cl1.cnf(+)(AREP, Bus_accessible)	CLEAR
137	SGC-OC	DMPMM1_Global_Control.cnf(Rem_Add, Status) /CGC_Count = 1 && Go_AClr = True => INDICATE_BUS_ACCESSIBILITY UGC_Count := UGC_Count-1 CGC_Count := 0 FSPMM1_DP Master Cl1 Mode_Changed.ind(AREP, USIF_State:=Clear) MSCY1M_Cont_Slave_Handler.req(0..125, Output_Clear:=True)	CLEAR
138	COPERAT E	Min_SI_Interval_Timer expired /Go_AClr = False => Start Min_SI_Interval_Timer (Bus_Para.Min_Slave_Interval) MSCY1M_Cont_Slave_Handler.req(0..125, Output_Clear:=False)	OPERATE
139	COPERAT E	Min_SI_Interval_Timer expired /Go_AClr = True && CGC_Count > 0 => Start Dx_Control_Interval_Timer (Bus_Para.Data_Control_Time/2) Start Min_SI_Interval_Timer (Bus_Para.Min_Slave_Interval)	SGC-OC-WGC
140	COPERAT E	Min_SI_Interval_Timer expired /Go_AClr = True && CGC_Count = 0 => Start Dx_Control_Interval_Timer (Bus_Para.Data_Control_Time/2) UGC_Count := UGC_Count+1 CGC_Count := UGC_Count Start Min_SI_Interval_Timer (Bus_Para.Min_Slave_Interval) DMPMM1_Global_Control.req(Rem_Add:=127, Control_Command:=2, Group_Select:=0)	SGC-OC
141	COPERAT E	DMPMM1_SYNCH.ind /Go_AClr = False => DX_Lock := TRUE Output_Data = Set_Output MSCY1M_Cont_Slave_Handler.req(0..125, Output_Clear:=False) MSCY1M_Set_Output.req(Rem_Add:=0..125, Slot_Number:=ALL, Output_Data) MSCY1M_Get_Input.req(Rem_Add:=0..125, Slot_Number:=ALL) FSPMM1_SYNCH.ind (AREP)	COPERATE
142	COPERAT E	DMPMM1_SYNCH.ind /Go_AClr = True && CGC_Count > 0 => Start Dx_Control_Interval_Timer (Bus_Para.Data_Control_Time/2) DX_Lock := TRUE Output_Data = Set_Output MSCY1M_Set_Output.req(Rem_Add:=0..125, Slot_Number:=ALL, Output_Data) MSCY1M_Get_Input.req(Rem_Add:=0..125, Slot_Number:=ALL) FSPMM1_SYNCH.ind (AREP)	COPERATE

Table 430 (continued)

#	Current State	Event /Condition =>Action	Next State
143	COPERATE	DMPMM1_SYNCH.ind /Go_AClr = True && CGC_Count = 0 => Start Dx_Control_Interval_Timer (Bus_Para.Data_Control_Time/2) UGC_Count := UGC_Count+1 CGC_Count := UGC_Count DX_Lock := TRUE Output_Data = Set_Output DMPMM1_Global_Control.req(Rem_Add:=127, Control_Command:=2, Group_Select:=0) MSCY1M_Set_Output.req(Rem_Add:=0..125, Slot_Number:=ALL, Output_Data) MSCY1M_Get_Input.req(Rem_Add:=0..125, Slot_Number:=ALL) FSPMM1_SYNCH.ind (AREP)	COPERATE
144	LDBP-OP-CPU	DMPMM1_Set_Bus_Par.cnf => FSPMM1_Load_Bus_Para.cnf(+)	OPERATE
145	DELSC-OP	DMPMM1_Delete_SC.cnf => FSPMM1_Delete_SC DP Master CI1.cnf	OPERATE
146	RDVAL-OP	DMPMM1_Read_Value.cnf(Status,Value) => FSPMM1_Read_Value DP Master CI1.cnf(Status, Value)	OPERATE
147	RESET	/spontaneous => Act_Rem_Add := 0 MSCY1M_Reset.req(Act_Rem_Add)	RESET-MS0
148	RESET-MS0	MSCY1M_Reset.cnf(Act_Rem_Add) /Act_Rem_Add<125 => Act_Rem_Add := Act_Rem_Add+1 MSCY1M_Reset.req(Act_Rem_Add)	RESET-MS0
149	RESET-MS0	MSCY1M_Reset.cnf(Act_Rem_Add) /Act_Rem_Add=125 && IsARExistent(MS1)=TRUE => Act_Rem_Add := 0 MSAC1M_Reset.req(Act_Rem_Add)	RESET-MS1
150	RESET-MS0	MSCY1M_Reset.cnf(Act_Rem_Add) /Act_Rem_Add=125 && IsARExistent(MS1)=FALSE && && IsARExistent(MM1)=TRUE => MMAC1_Reset.req	RESET-MM1
151	RESET-MS0	MSCY1M_Reset.cnf(Act_Rem_Add) /Act_Rem_Add=125 && IsARExistent(MS1)=FALSE && && IsARExistent(MM1)=FALSE => DMPMM1_Reset.req	RESET-DMPM
152	RESET-MS1	MSAC1M_Reset.cnf(Act_Rem_Add) /Act_Rem_Add=125 => Act_Rem_Add := 0 MSAL1M_Reset.req(Act_Rem_Add)	RESET-MS1
153	RESET-MS1	MSAL1M_Reset.cnf(Act_Rem_Add) /Act_Rem_Add<125 => Act_Rem_Add := Act_Rem_Add + 1 MSAL1M_Reset.req(Act_Rem_Add)	RESET-MS1
154	RESET-MS1	MSAL1M_Reset.cnf(Act_Rem_Add) /Act_Rem_Add = 125 && IsARExistent(MM1)=TRUE => MMAC1_Reset.req	RESET-MM1
155	RESET-MS1	MSAL1M_Reset.cnf(Act_Rem_Add) /Act_Rem_Add = 125 && IsARExistent(MM1)=FALSE => DMPMM1_Reset.req	RESET-DMPM

Table 430 (continued)

#	Current State	Event /Condition =>Action	Next State
156	RESET-MM1	MMAC1_Reset.cnf => DMPMM1_Reset.req	RESET-DMPM
157	RESET-DMPM	DMPMM1_Reset.cnf /USER_SetMode_Offline = TRUE USER_ChangedCritical_Para = TRUE	INIT-DMPM
158	RESET-DMPM	DMPMM1_Reset.cnf /USER_SetMode_Offline = FALSE && USER_ChangedCritical_Para = FALSE => if(USER_sched_Reset = TRUE) FSPMM1_Reset DP Master Cl1.cnf endif	POWER-ON
159	t state	FSPMM1_Abort DP Master Cl1.req(AREP) /AREP.AR_Type=MS0 => Current Extended Function Num[CREP]:=No_Service LR_State[CREP]:=W-LR-REQ MSCY1M_Abort.req(Rem_Add:=CREP.Rem_Add)	SAME
160	t state	FSPMM1_Abort DP Master Cl1.req(AREP) /AREP.AR_Type=MS1 => Current Extended Function Num[CREP]:=No_Service LR_State[CREP]:=W-LR-REQ MSAC1M_Abort.req(Rem_Add:=CREP.Rem_Add)	SAME
161	t state	FSPMM1_Abort DP Master Cl1.req(AREP) /AREP.AR_Type=MM1 => MMAC1_Abort.req(Rem_Add:=CREP.Rem_Add)	SAME
162	t state	FSPMM1_Get_Slave_Diag.req(AREP, CREP) /AREP.AR_Type=MS0 => MSCY1M_Get_Slave_Diag.req(CREP.Rem_Add)	SAME
163	t state	MSCY1M_Get_Slave_Diag.cnf(+)(Rem_Add, Diag_Data) /SetContext(Rem_Add, Service)=TRUE => FSPMM1_Get_Slave_Diag.cnf(+)(AREP, CREP, Diag_Data)	SAME
164	t state	MSCY1M_Get_Slave_Diag.cnf(-)(Rem_Add) /SetContext(Rem_Add, Service)=TRUE => FSPMM1_Get_Slave_Diag.cnf(-)(AREP, CREP)	SAME
165	t state	MSCY1M_Set_Output.cnf(+)(Rem_Add, Slot_Number) /SetContext(Rem_Add, Service)=TRUE && Isochronous Mode = (Not Synchronized Enhanced Synchronized) => FSPMM1_Set_Output.cnf(+)(AREP, CREP, Slot_Number)	SAME
166	t state	MSCY1M_Set_Output.cnf(-)(Rem_Add, Slot_Number) /SetContext(Rem_Add, Service)=TRUE && Isochronous Mode = (Not Synchronized Enhanced Synchronized) => Status := SC FSPMM1_Set_Output.cnf(-)(AREP, CREP, Slot_Number, Status)	SAME
167	t state	MSCY1M_Get_Input.cnf(+)(Rem_Add, Slot_Number, Input_Data) /SetContext(Rem_Add, Service)=TRUE && Isochronous Mode = (Not Synchronized Enhanced Synchronized) => FSPMM1_Get_Input.cnf(+)(AREP, CREP, Slot_Number, Input_Data)	SAME
168	t state	MSCY1M_Get_Input.cnf(-)(Rem_Add, Slot_Number) /SetContext(Rem_Add, Service)=TRUE && Isochronous Mode = (Not Synchronized Enhanced Synchronized) => Status := SC FSPMM1_Get_Input.cnf(-)(AREP, CREP, Slot_Number, Status)	SAME
169	t state	FSPMM1_Read.req(AREP, Slot_Number, Index, Length) /AREP.AR_Type=MS1 => MSAC1M_Read.req(Rem_Add:=CREP.Rem_Add, Slot_Number, Index, Length)	SAME

Table 430 (continued)

#	Current State	Event /Condition =>Action	Next State
170	t state	MSAC1M_Read.cnf(+)(Rem_Add, Length, Data) /SetContext(Rem_Add, Service)=TRUE => FSPMM1_Read.cnf(+)(AREP, Length, Data)	SAME
171	t state	MSAC1M_Read.cnf(-)(Rem_Add, Error_Decode, Error_Code_1, Error_Code_2) /SetContext(Rem_Add, Service)=TRUE => FSPMM1_Read.cnf(-)(AREP, Rem_Add, Error_Decode, Error_Code_1, Error_Code_2)	SAME
172	t state	FSPMM1_Write.req(AREP, Slot_Number, Index, Length, Data) /AREP.AR_Type=MS1 => MSAC1M_Write.req(Rem_Add:=CREP.Rem_Add, Slot_Number, Index, Length, Data)	SAME
173	t state	MSAC1M_Write.cnf(+)(Rem_Add, Length) /SetContext(Rem_Add, Service)=TRUE => FSPMM1_Write.cnf(+)(AREP, Length)	SAME
174	t state	MSAC1M_Write.cnf(-)(Rem_Add, Error_Decode, Error_Code_1, Error_Code_2) /SetContext(Rem_Add, Service)=TRUE => FSPMM1_Write.cnf(-)(AREP, Rem_Add, Error_Decode, Error_Code_1, Error_Code_2)	SAME
175	t state	FSPMM1_Alarm_Ack.req(AREP, Slot_Number, Alarm_Type, Seq_Nr) /AREP.AR_Type=MS1 => MSAL1M_Alarm_Ack.req(Rem_Add:=CREP.Rem_Add, Slot_Number, Alarm_Type, Seq_Nr)	SAME
176	t state	MSAL1M_Alarm_Ack.cnf(+)(Rem_Add, Slot_Number, Alarm_Type, Seq_Nr) /SetContext(Rem_Add, Service)=TRUE => FSPMM1_Alarm_Ack.cnf(+)(AREP, Slot_Number, Alarm_Type, Seq_Nr)	SAME
177	t state	MSAL1M_Alarm_Ack.cnf(-)(Rem_Add) /SetContext(Rem_Add, Service)=TRUE => FSPMM1_Alarm_Ack.cnf(-)(AREP, Slot Number, Alarm Type, Seq Nr)	SAME
178	t state	FSPMM1_Get_Master_Diag.rsp(+)(AREP, Diagnosis_Data) /AREP.AR_Type=MM1 => MMAC1_Get_Master_Diag.rsp(Status:=OK, Diagnosis_Data)	SAME
179	t state	FSPMM1_Get_Master_Diag.rsp(-)(AREP, Status) /AREP.AR_Type=MM1 => MMAC1_Get_Master_Diag.rsp(Status)	SAME
180	t state	MMAC1_Get_Master_Diag.ind(Identifier) /SetContext(Rem_Add, Service)=TRUE => FSPMM1_Get_Master_Diag.ind(AREP, Mdiag_Identifier:=Identifier)	SAME
181	t state	FSPMM1_Start_Seq.rsp(+)(AREP, Max_Len_Data_Unit) /AREP.AR_Type=MM1 => MMAC1_Start_Seq.rsp(Status:=OK, Max_Length_Data_Unit)	SAME
182	t state	FSPMM1_Start_Seq.rsp(-)(AREP, Status) /AREP.AR_Type=MM1 => MMAC1_Start_Seq.rsp(Status)	SAME
183	t state	MMAC1_Start_Seq.ind(Req_Add, Area_Code, Timeout) /SetContext(Rem_Add, Service)=TRUE => FSPMM1_Start_Seq.ind(AREP, Area_Code, Timeout)	SAME
184	t state	FSPMM1_Download.rsp(+)(AREP) /AREP.AR_Type=MM1 => MMAC1_Download.rsp(Status:=OK)	SAME

Table 430 (continued)

#	Current State	Event /Condition =>Action	Next State
185	t state	FSPMM1_Download.rsp(-)(AREP, Status) /AREP.AR_Type=MM1 => MMAC1_Download.rsp(Status)	SAME
186	t state	MMAC1_Download.ind(Req_Add, Area_Code, Address_Offset, Data) /SetContext(Rem_Add, Service)=TRUE => FSPMM1_Download.ind(AREP, Area_Code, Add_Offset, Data)	SAME
187	t state	FSPMM1_Upload.rsp(+)(AREP, Data) /AREP.AR_Type=MM1 => MMAC1_Upload.rsp(Status:=OK, Data)	SAME
188	t state	FSPMM1_Upload.rsp(-)(AREP, Status) /AREP.AR_Type=MM1 => MMAC1_Upload.rsp(Status)	SAME
189	t state	MMAC1_Upload.ind(Req_Add, Area_Code, Address_Offset, Data_Length) /SetContext(Rem_Add, Service)=TRUE => FSPMM1_Upload.ind(AREP, Area_Code, Add_Offset, Data_Len:=Data_Length)	SAME
190	t state	FSPMM1_End_Seq.rsp(+)(AREP) /AREP.AR_Type=MM1 => MMAC1_End_Seq.rsp(Status:=OK)	SAME
191	t state	FSPMM1_End_Seq.rsp(-)(AREP, Status) /AREP.AR_Type=MM1 => MMAC1_End_Seq.rsp(Status)	SAME
192	t state	MMAC1_End_Seq.ind(Req_Add) /SetContext(Rem_Add, Service)=TRUE => FSPMM1_End_Seq.ind(AREP)	SAME
193	t state	FSPMM1_Act_Param.rsp(+)(AREP) /AREP.AR_Type=MM1 => MMAC1_Act_Param.rsp(Status:=OK)	SAME
194	t state	FSPMM1_Act_Param.rsp(-)(AREP, Status) /AREP.AR_Type=MM1 => MMAC1_Act_Param.rsp(Status)	SAME
195	t state	MMAC1_Act_Param.ind(Area_Code, Activate) /SetContext(Rem_Add, Service)=TRUE => FSPMM1_Act_Param.ind(AREP, Area_Code, Activate)	SAME
196	t state	MMAC1_Act_Para_Brct.ind(Area_Code) /SetContext(Rem_Add, Service)=TRUE => FSPMM1_Act_Para_Brct.ind(AREP, Area_Code)	SAME
197	t state	MSAC1M_Reject.ind(Rem_Add, Status) /SetContext(Rem_Add, Service)=TRUE => FSPMM1_DP Master CI1 Reject.ind(AREP, Reason:=Status)	SAME
198	t state	MSAL1M_Started.ind(Rem_Add, Alarm_Limit) /SetContext(Rem_Add, Service)=TRUE => FSPMM1_DP Master CI1 Started.ind(AREP, Alarm_Limit)	SAME
199	t state	MSAL1M_Stopped.ind(Rem_Add) /SetContext(Rem_Add, Service)=TRUE => Current Extended Function Num[CREP]:=No_Service LR_State[CREP]:=W-LR-REQ FSPMM1_DP Master CI1 Stopped.ind(AREP)	SAME

Table 430 (continued)

#	Current State	Event /Condition =>Action	Next State
200	t state	MSAL1M_Alarm_Notification.ind(Rem_Add, Alarm_Type, Slot_Number, Seq_Nr, Alarm_Specifier, Add_Ack, Alarm_Data) /SetContext(Rem_Add, Service)=TRUE => FSPMM1_Alarm_Notification.ind(AREP, Alarm_Type, Slot_Number, Seq_Nr, Alarm_Specifier, Add_Ack, Alarm_Data)	SAME
201	t state	MSCY1M_New_Slave_Diag.ind(Rem_Add) /SetContext(Rem_Add, Service)=TRUE => FSPMM1_New_Slave_Diag.ind(AREP, CREP)	SAME
202	t state	MSCY1M_New_Input.ind(Rem_Add) /SetContext(Rem_Add, Service)=TRUE => FSPMM1_New_Input.ind(AREP, CREP)	SAME
203	t state	FSPMM1_Set_Output.req(AREP, CREP, Slot_Number, Output_Data, Final) /AREP.AR_Type=MS0 && (DX_Lock Chg_Buffer) => Status := SE FSPMM1_Set_Output.cnf(-)(AREP, CREP, Slot_Number, Status)	SAME
204	t state	FSPMM1_Get_Input.req(AREP, CREP, Slot_Number) /AREP.AR_Type=MS0 && (DX_Lock Chg_Buffer) => Status := SE FSPMM1_Get_Input.cnf(-)(AREP, CREP, Slot_Number, Status)	SAME
205	t state	FSPMM1_Set_Output.req(AREP, CREP, Slot_Number, Output_Data, Final) /AREP.AR_Type=MS0 && !DX_Lock && !Chg_Buffer && Isochronous Mode <> Buffered Synchronized => MSCY1M_Set_Output.req(Rem_Add:=CREP.Rem_Add, Slot_Number, Output_Data)	SAME
206	t state	FSPMM1_Get_Input.req(AREP, CREP, Slot_Number) /AREP.AR_Type=MS0 && !DX_Lock && !Chg_Buffer && Isochronous Mode <> Buffered Synchronized => MSCY1M_Get_Input.req(Rem_Add:=CREP.Rem_Add, Slot_Number)	SAME
207	t state	FSPMM1_Set_Output.req(AREP, CREP, Slot_Number, Output_Data, Final) /AREP.AR_Type=MS0 && !DX_Lock && !Chg_Buffer && Isochronous Mode = Buffered Synchronized => Bfr_Set_Output := Output_Data Chg_Buffer := Final FSPMM1_Set_Output.cnf(+)(AREP, CREP, Slot_Number)	SAME
208	t state	FSPMM1_Get_Input.req(AREP, CREP, Slot_Number) /AREP.AR_Type=MS0 && !DX_Lock && !Chg_Buffer && Isochronous Mode = Buffered Synchronized => Input_Data := Bfr_Get_Input FSPMM1_Get_Input.cnf(+)(AREP, CREP, Slot_Number, Input_Data)	SAME
209	t state	MSCY1M_Set_Output.cnf(+)(Rem_Add, Slot_Number) /SetContext(Rem_Add, Service)=TRUE && Isochronous Mode = Buffered Synchronized =>	SAME
210	t state	MSCY1M_Set_Output.cnf(-)(Rem_Add, Slot_Number) /SetContext(Rem_Add, Service)=TRUE && Isochronous Mode = Buffered Synchronized => Set_Output = Nil	SAME
211	t state	MSCY1M_Get_Input.cnf(+)(Rem_Add, Slot_Number, Input_Data) /SetContext(Rem_Add, Service)=TRUE && Isochronous Mode = Buffered Synchronized && NOT received all MSCY1M_Get_Input.cnf => Get_Input = Input_Data	SAME

Table 430 (continued)

#	Current State	Event /Condition =>Action	Next State
212	t state	MSCY1M_Get_Input.cnf(+)(Rem_Add, Slot_Number, Input_Data) /SetContext(Rem_Add, Service)=TRUE && Isochronous Mode = Buffered Synchronized&& NOT received all MSCY1M_Get_Input.cnf => Chg_Buffer := FALSE Get_Input = Input_Data	SAME
213	t state	MSCY1M_Get_Input.cnf(-)(Rem_Add, Slot_Number) /SetContext(Rem_Add, Service)=TRUE && Isochronous Mode = Buffered Synchronized => Get_Input = Nil	SAME
214	ANY-STATE	DMPMM1_Event.ind(Event, Add_Info) => FSPMM1_DP Master CI1 Event.ind(Event, Add_Info)	SAME
215	ANY-STATE	DMPMM1_SYNCH_Delayed.ind (TSH) => FSPMM1_SYNCH_Delayed.ind (AREP, TSH)	SAME
216	ANY-STATE	DMPMM1_Fault.ind => USER_sched_Reset := False FSPMM1_DP Master CI1 Fault.ind	RESET
217	ANY-STATE	MSCY1M_Fault.ind => USER_sched_Reset := False FSPMM1_DP Master CI1 Fault.ind	RESET
218	ANY-STATE	MSAL1M_Fault.ind => USER_sched_Reset := False FSPMM1_DP Master CI1 Fault.ind	RESET
219	ANY-STATE	MSAC1M_Fault.ind => USER_sched_Reset := False FSPMM1_DP Master CI1 Fault.ind	RESET
220	ANY-STATE	MMAC1_Fault.ind => USER_sched_Reset := False FSPMM1_DP Master CI1 Fault.ind	RESET
221	ANY-STATE	FSPMM1_Reset DP Master CI1.req => USER_sched_Reset := True	RESET
222	ANY-STATE	DMPMM1_SYCL_Time_Event.cnf (Send Delay Time, Status) => FSPMM1_SYCL_Time_Event.cnf (AREP, Send Delay Time, Status)	SAME
223	ANY-STATE	FSPMM1_SYCL_Time_Event.req (AREP) => DMPMM1_SYCL_Time_Event.req	SAME
224	ANY-STATE	FSPMM1_SYCL_Clock_Value.req (AREP, Clock Value Time Event, Clock Value previous TE, Clock Value Status) => DMPMM1_SYCL_Clock_Value.req (Clock Value Time Event, Clock Value previous TE, Clock Value Status)	SAME
225	ANY-STATE	DMPMM1_SYCL_Clock_Value.cnf (Status) => FSPMM1_SYCL_Clock_Value.cnf (AREP, Status)	SAME
226	ANY-STATE	DMPMM1_SYCL_Clock_Value.ind (Clock Value Time Event, Clock Value previous TE, Clock Value Status, Receive Delay Time, Src add Clk msg) => FSPMM1_SYCL_Clock_Value.ind (AREP, Clock Value Time Event, Clock Value previous TE, Clock Value Status, Receive Delay Time, Src add Clk msg)	SAME

Table 430 (continued)

#	Current State	Event /Condition =>Action	Next State
227	t state	FSPMM1_Initiate_Load.req (AREP, Slot Number, LR Index, Load Type, Load Image Size, Intersegment Request Timeout, User Specific) /LR_State[CREP]==W-LR-REQ &AREP.AR_Type=MS1 => Current Extended Function Num[CREP]:=Initiate_Load Current Slot Number[CREP]:=Slot Number Index:=255 Length:=10+sizeof(User Specific) Data.w.IL.Extended_Function_Num:=Initiate_Load Data.w.IL.LR_Index:=LR Index Data.w.IL.Load_Type:=Load Type Data.w.IL.Load_Image_Size:=Load Image Size Data.w.IL.User_Specific:=User Specific Data.w.IL.Intersegment_Request_Timeout:=Intersegment Request Timeout MSAC1M_Write.req(Rem_Add:=CREP.Rem_Add, Slot_Number, Index, Length, Data) LR_State[CREP]:=W-LR-CNF	SAME
228	t state	MSAC1M_Write.cnf(+)(Rem_Add, Length) /LR_State[CREP]==W-LR-CNF &SetContext(Rem_Add, Service)=TRUE && Current Extended Function Num[CREP]:=Initiate_Load => Slot_Number:=Current Slot Number[CREP] Index:=255 Length:=10 MSAC1M_Read.req(Rem_Add:=CREP.Rem_Add, Slot_Number, Index, Length) LR_State[CREP]:=W-LR-CNF	SAME
229	t state	MSAC1M_Read.cnf(+)(Rem_Add, Length, Data) /LR_State[CREP]==W-LR-CNF &SetContext(Rem_Add, Service)=TRUE && Current Extended Function Num[CREP]:=Initiate_Load => Current Extended Function Num[CREP]:=No_Service Actual LR Size:=Data.r.IL.Actual_LR_Size Max Response Delay:=Data.r.IL.Max_Response_Delay Max Segment Length:=Data.r.IL.Max_Segment_Length User Specific:=Data.r.IL.User_Specific FSPMM1_Initiate_Load.cnf(+)(AREP, Actual LR Size, Max Response Delay, Max Segment Length, User Specific) LR_State[CREP]:=W-LR-REQ	SAME
230	t state	MSAC1M_Write.cnf(-)(Rem_Add, Error Decode, Error_Code_1, Error_Code_2) /LR_State[CREP]==W-LR-CNF &SetContext(Rem_Add, Service)=TRUE && Current Extended Function Num[CREP]:=Initiate_Load => Current Extended Function Num[CREP]:=No_Service Error Code:=Error_Code_1 FSPMM1_Initiate_Load.cnf(-)(AREP, Error Code) LR_State[CREP]:=W-LR-REQ	SAME
231	t state	FSPMM1_Pull_Segment.req (AREP, Slot Number, LR Index, Segment Length) /LR_State[CREP]==W-LR-REQ &AREP.AR_Type=MS1 => Current Extended Function Num[CREP]:=Pull Current LR Index[CREP]:=LR Index Index:=255 Length:=Segment Length MSAC1M_Read.req(Rem_Add:=CREP.Rem_Add, Slot_Number, Index, Length) LR_State[CREP]:=W-LR-CNF	SAME

Table 430 (continued)

#	Current State	Event /Condition =>Action	Next State
232	t state	MSAC1M_Read.cnf(+)(Rem_Add, Length, Data) /LR_State[CREP]==W-LR-CNF &SetContext(Rem_Add, Service)=TRUE && Current Extended Function Num[CREP]=Pull => Current Extended Function Num[CREP]=No_Service Segment Number:=Data.PULL.Sequence Number More Follows:=Data.PULL.Options.More Follows Data:=Data.PULL.Region Data Segment Length:=Length-6 FSPMM1_Pull_Segment.cnf(+)(AREP, Segment Length, Segment Number, More Follows, Data) LR_State[CREP]:=W-LR-REQ	SAME
233	t state	MSAC1M_Read.cnf(-)(Rem_Add, Error Decode, Error_Code_1, Error_Code_2) /LR_State[CREP]==W-LR-CNF &SetContext(Rem_Add, Service)=TRUE && Current Extended Function Num[CREP]=Pull => Current Extended Function Num[CREP]=No_Service Error Code:=Error_Code_1 FSPMM1_Pull_Segment.cnf(-)(AREP, Error Code) LR_State[CREP]:=W-LR-REQ	SAME
234	t state	FSPMM1_Push_Segment.req (AREP, Slot Number, LR Index, Segment Length, Segment Number, More Follows, Data) /LR_State[CREP]==W-LR-REQ &AREP.AR_Type=MS1 => Current Extended Function Num[CREP]:=Push Current Slot Number[CREP]:=Slot Number Index:=255 Length:=6+Segment Length Data.PUSH.Extended_Function_Num:=Push Data.PUSH.LR_Index:=LR Index Data.PUSH.Segment_Number:=Segment Number Data.PUSH.Option.More_Follows:=More Follows Data.PUSH.Data:=Data MSAC1M_Write.req(Rem_Add:=CREP.Rem_Add, Slot_Number, Index, Length, Data) LR_State[CREP]:=W-LR-CNF	SAME
235	t state	MSAC1M_Write.cnf(+)(Rem_Add, Length) /LR_State[CREP]==W-LR-CNF &SetContext(Rem_Add, Service)=TRUE && Current Extended Function Num[CREP]=Push => Current Extended Function Num[CREP]=No_Service Segment Number:=Data.PUSH.Sequence Number More Follows:=Data.PUSH.Options.More Follows Data:=Data.PUSH.Region Data Segment Length:=Length-6 FSPMM1_Push_Segment.cnf(+)(AREP) LR_State[CREP]:=W-LR-REQ	SAME
236	t state	MSAC1M_Write.cnf(-)(Rem_Add, Error Decode, Error_Code_1, Error_Code_2) /LR_State[CREP]==W-LR-CNF &SetContext(Rem_Add, Service)=TRUE && Current Extended Function Num[CREP]=Push => Current Extended Function Num[CREP]=No_Service Error Code:=Error_Code_1 FSPMM1_Push_Segment.cnf(-)(AREP, Error Code) LR_State[CREP]:=W-LR-REQ	SAME

Table 430 (continued)

#	Current State	Event /Condition =>Action	Next State
237	t state	FSPMM1_Terminate_Load.req (AREP, Slot Number, LR Index) /LR_State[CREP]==W-LR-REQ &AREP.AR_Type=MS1 => Current Extended Function Num[CREP]:=Terminate_Load Current Slot Number[CREP]:=Slot Number Index:=255 Length:=6 Data.TL.Extended_Function_Num:=Terminate_Load Data.TL.LR_Index:=LR Index MSAC1M_Write.req(Rem_Add:=CREP.Rem_Add, Slot_Number, Index, Length, Data) LR_State[CREP]:=W-LR-CNF	SAME
238	t state	MSAC1M_Write.cnf(+)(Rem_Add, Length) /LR_State[CREP]==W-LR-CNF &SetContext(Rem_Add, Service)=TRUE && Current Extended Function Num[CREP]:=Terminate_Load => Current Extended Function Num[CREP]:=No_Service FSPMM1_Terminate_Load.cnf(+)(AREP) LR_State[CREP]:=W-LR-REQ	SAME
239	t state	MSAC1M_Write.cnf(-)(Rem_Add, Error_Decompose, Error_Code_1, Error_Code_2) /LR_State[CREP]==W-LR-CNF &SetContext(Rem_Add, Service)=TRUE && Current Extended Function Num[CREP]:=Terminate_Load => Current Extended Function Num[CREP]:=No_Service Error Code:=Error_Code_1 FSPMM1_Terminate_Load.cnf(-)(AREP, Error Code) LR_State[CREP]:=W-LR-REQ	SAME
240	t state	FSPMM1_Call.req (AREP, Slot Number, FI Index, Execution Argument) /LR_State[CREP]==W-LR-REQ &AREP.AR_Type=MS1 => Current Extended Function Num[CREP]:=Call Current Slot Number[CREP]:=Slot Number Index:=255 Length:=4+sizeof(Execution Argument) Data.w.CALL.Extended_Function_Num:=Call Data.w.CALL.FI_Index:=FI Index Data.w.CALL.Execution_Argument:=Execution Argument MSAC1M_Write.req(Rem_Add:=CREP.Rem_Add, Slot_Number, Index, Length, Data) LR_State[CREP]:=W-LR-CNF	SAME
241	t state	MSAC1M_Write.cnf(+)(Rem_Add, Length) /LR_State[CREP]==W-LR-CNF &SetContext(Rem_Add, Service)=TRUE && Current Extended Function Num[CREP]:=Call => Slot_Number:=Current Slot Number[CREP] Index:=255 Length:=CREP.Max_PDU_Length MSAC1M_Read.req(Rem_Add:=CREP.Rem_Add, Slot_Number, Index, Length) LR_State[CREP]:=W-LR-CNF	SAME
242	t state	MSAC1M_Read.cnf(+)(Rem_Add, Length, Data) /LR_State[CREP]==W-LR-CNF &SetContext(Rem_Add, Service)=TRUE && Current Extended Function Num[CREP]:=Call => Current Extended Function Num[CREP]:=No_Service Result Argument:=Data.r.CALL.Result_Argument FSPMM1_Call.cnf(+)(AREP, Result Argument) LR_State[CREP]:=W-LR-REQ	SAME

Table 430 (continued)

#	Current State	Event /Condition =>Action	Next State
243	t state	MSAC1M_Write.cnf(-)(Rem_Add, Error_Decode, Error_Code_1, Error_Code_2) /LR_State[CREP]==W-LR-CNF &SetContext(Rem_Add, Service)=TRUE && Current Extended Function Num[CREP]=Call => Current Extended Function Num[CREP]=No_Service Error Code:=Error_Code_1 FSPMM1_Call.cnf(-) (AREP, Error Code) LR_State[CREP]:=W-LR-REQ	SAME
244	t state	FSPMM1_Start.req (AREP, Slot Number, FI Index, Execution Argument) /LR_State[CREP]==W-LR-REQ &AREP.AR_Type=MS1 => Current Extended Function Num[CREP]:=Start Current Slot Number[CREP]:=Slot Number Index:=255 Length:=4+sizeof(Execution Argument) Data.FIS.Extended_Function_Num:=Start Data.FIS.FI_Index:=FI Index Data.FIS.Execution_Argument:=Execution Argument MSAC1M_Write.req(Rem_Add:=CREP.Rem_Add, Slot_Number, Index, Length, Data) LR_State[CREP]:=W-LR-CNF	SAME
245	t state	MSAC1M_Write.cnf(+)(Rem_Add, Length) /LR_State[CREP]==W-LR-CNF &SetContext(Rem_Add, Service)=TRUE && Current Extended Function Num[CREP]=Start => Current Extended Function Num[CREP]=No_Service FSPMM1_Start.cnf(+)(AREP) LR_State[CREP]:=W-LR-REQ	SAME
246	t state	MSAC1M_Write.cnf(-)(Rem_Add, Error_Decode, Error_Code_1, Error_Code_2) /LR_State[CREP]==W-LR-CNF &SetContext(Rem_Add, Service)=TRUE && Current Extended Function Num[CREP]=Start => Current Extended Function Num[CREP]=No_Service Error Code:=Error_Code_1 FSPMM1_Start.cnf(-) (AREP, Error Code) LR_State[CREP]:=W-LR-REQ	SAME
247	t state	FSPMM1_Stop.req (AREP, Slot Number, FI Index) /LR_State[CREP]==W-LR-REQ &AREP.AR_Type=MS1 => Current Extended Function Num[CREP]:=Stop Current Slot Number[CREP]:=Slot Number Index:=255 Length:=4 Data.FIS.Extended_Function_Num:=Stop Data.FIS.FI_Index:=FI Index MSAC1M_Write.req(Rem_Add:=CREP.Rem_Add, Slot_Number, Index, Length, Data) LR_State[CREP]:=W-LR-CNF	SAME
248	t state	MSAC1M_Write.cnf(+)(Rem_Add, Length) /LR_State[CREP]==W-LR-CNF &SetContext(Rem_Add, Service)=TRUE && Current Extended Function Num[CREP]=Stop => Current Extended Function Num[CREP]=No_Service FSPMM1_Stop.cnf(+)(AREP) LR_State[CREP]:=W-LR-REQ	SAME

Table 430 (continued)

#	Current State	Event /Condition =>Action	Next State
249	t state	MSAC1M_Write.cnf(-)(Rem_Add, Error_Decode, Error_Code_1, Error_Code_2) /LR_State[CREP]==W-LR-CNF &SetContext(Rem_Add, Service)=TRUE && Current Extended Function Num[CREP]=Stop => Current Extended Function Num[CREP]=No_Service Error Code:=Error_Code_1 FSPMM1_Stop.cnf(-) (AREP, Error Code) LR_State[CREP]:=W-LR-REQ	SAME
250	t state	FSPMM1_Resume.req (AREP, Slot Number, FI Index, Execution Argument) /LR_State[CREP]==W-LR-REQ &AREP.AR_Type=MS1 => Current Extended Function Num[CREP]:=Resume Current Slot Number[CREP]:=Slot Number Index:=255 Length:=4+sizeof(Execution Argument) Data.FIS.Extended_Function_Num:=Resume Data.FIS.FI_Index:=FI Index Data.FIS.Execution_Argument:=Execution Argument MSAC1M_Write.req(Rem_Add:=CREP.Rem_Add, Slot_Number, Index, Length, Data) LR_State[CREP]:=W-LR-CNF	SAME
251	t state	MSAC1M_Write.cnf(+)(Rem_Add, Length) /LR_State[CREP]==W-LR-CNF &SetContext(Rem_Add, Service)=TRUE && Current Extended Function Num[CREP]=Resume => Current Extended Function Num[CREP]=No_Service FSPMM1_Resume.cnf(+)(AREP) LR_State[CREP]:=W-LR-REQ	SAME
252	t state	MSAC1M_Write.cnf(-)(Rem_Add, Error_Decode, Error_Code_1, Error_Code_2) /LR_State[CREP]==W-LR-CNF &SetContext(Rem_Add, Service)=TRUE && Current Extended Function Num[CREP]=Resume => Current Extended Function Num[CREP]=No_Service Error Code:=Error_Code_1 FSPMM1_Resume.cnf(-)(AREP, Error Code) LR_State[CREP]:=W-LR-REQ	SAME
253	t state	FSPMM1_Reset.req (AREP, Slot Number, FI Index) /LR_State[CREP]==W-LR-REQ &AREP.AR_Type=MS1 => Current Extended Function Num[CREP]:=Reset Current Slot Number[CREP]:=Slot Number Index:=255 Length:=4 Data.FIS.Extended_Function_Num:=Reset Data.FIS.FI_Index:=FI Index MSAC1M_Write.req(Rem_Add:=CREP.Rem_Add, Slot_Number, Index, Length, Data) LR_State[CREP]:=W-LR-CNF	SAME
254	t state	MSAC1M_Write.cnf(+)(Rem_Add, Length) /LR_State[CREP]==W-LR-CNF &SetContext(Rem_Add, Service)=TRUE && Current Extended Function Num[CREP]=Reset => Current Extended Function Num[CREP]=No_Service FSPMM1_Reset.cnf(+)(AREP) LR_State[CREP]:=W-LR-REQ	SAME

Table 430 (continued)

#	Current State	Event /Condition =>Action	Next State
255	t state	MSAC1M_Write.cnf(-)(Rem_Add, Error_Decode, Error_Code_1, Error_Code_2) /LR_State[CREP]==W-LR-CNF &SetContext(Rem_Add, Service)=TRUE && Current Extended Function Num[CREP]=Reset => Current Extended Function Num[CREP]=No_Service Error Code:=Error_Code_1 FSPMM1_Reset.cnf(-) (AREP, Error Code) LR_State[CREP]:=W-LR-REQ	SAME
256	t state	FSPMM1_Get_FI_State.req (AREP, Slot Number, FI Index) /LR_State[CREP]==W-LR-REQ &AREP.AR_Type=MS1 => Current Extended Function Num[CREP]:=Get_State Current Slot Number[CREP]:=Slot Number Index:=255 Length:=4 Data.w.STATE.Extended_Function_Num:=Get_State Data.w.STATE.FI_Index:=FI Index MSAC1M_Write.req(Rem_Add:=CREP.Rem_Add, Slot_Number, Index, Length, Data) LR_State[CREP]:=W-LR-CNF	SAME
257	t state	MSAC1M_Write.cnf(+)(Rem_Add, Length) /LR_State[CREP]==W-LR-CNF &SetContext(Rem_Add, Service)=TRUE && Current Extended Function Num[CREP]=Get_State => Slot_Number:=Current Slot Number[CREP] Index:=255 Length:=5 MSAC1M_Read.req(Rem_Add:=CREP.Rem_Add, Slot_Number, Index, Length) LR_State[CREP]:=W-LR-CNF	SAME
258	t state	MSAC1M_Read.cnf(+)(Rem_Add, Length, Data) /LR_State[CREP]==W-LR-CNF &SetContext(Rem_Add, Service)=TRUE && Current Extended Function Num[CREP]=Get_State => Current Extended Function Num[CREP]=No_Service FI State:=Data.r.STATE.State FSPMM1_Get_FI_State.cnf (+) (AREP, FI State) LR_State[CREP]:=W-LR-REQ	SAME
259	t state	MSAC1M_Write.cnf(-)(Rem_Add, Error_Decode, Error_Code_1, Error_Code_2) /LR_State[CREP]==W-LR-CNF &SetContext(Rem_Add, Service)=TRUE && Current Extended Function Num[CREP]=Get_State => Current Extended Function Num[CREP]=No_Service Error Code:=Error_Code_1 FSPMM1_Get_State.cnf(-) (AREP, Error Code) LR_State[CREP]:=W-LR-REQ	SAME

6.5.2.4 Functions

Table 431 contains the functions used by the FSPMM1, their arguments and their descriptions.

Table 431 – Functions used by the FSPMM1

Function name	Description
SetContext (Rem_Add, Service-Id)	This function checks if there exist an entry for the inputs Rem_Add resp. Res_SAP and Service in the ARL and related CRL of the DP-master (Class 1). Table 432 If there exist an entry then it returns TRUE and sets the local context according to the current CREP and AREP. B) Otherwise it returns FALSE.
IsARexistent (AR)	This function checks if there exist an entry with ARtype=AR.
Chg_Bfr(Buffer1, Buffer 2)	This function exchanges Buffer 1 and Buffer 2.

6.5.3 FSPMM2

6.5.3.1 Primitive definitions

6.5.3.1.1 Primitives exchanged between AP-Context and FSPMM2

Table 432 shows the service primitives including their associated parameters issued by the AP-Context and received by the FSPMM2.

Table 432 – Primitives issued by AP-Context to FSPMM2

Primitive name	Source	Associated parameters	Functions
Init DP master Cl2.req	AP-Context	Bus Para	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5 and Type 3 Fieldbus IEC 61158-3
Reset DP master Cl2.req	AP-Context	(none)	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Abort.req	AP-Context	AREP Subnet, Instance, Reason Code	
Read Slave Diag.req	AP-Context	AREP	
Read Output.req	AP-Context	AREP	
Read Input.req	AP-Context	AREP	
Get Cfg.req	AP-Context	AREP	
Set Slave Add.req	AP-Context	AREP, New Slave Add, Ident Number, No Add Chg, Rem Slave Data	
Initiate.req	AP-Context	AREP, Send Timeout, Features Supported, Profile Features Supported, Profile Ident Number, Add Addr Param	
Read.req	AP-Context	AREP, Slot Number, Index, Length	
Write.req	AP-Context	AREP, Slot Number, Index, Length, Data	
Data Transport.req	AP-Context	AREP, Slot Number, Index, Length, Data	
Get Master Diag.req	AP-Context	AREP, MDiag Identifier	

Primitive name	Source	Associated parameters	Functions
Start Seq.req	AP-Context	AREP, Area Code, Timeout	
Download.req	AP-Context	AREP, Area Code, Add Offset, Data	
Upload.req	AP-Context	AREP, Area Code, Add Offset, Data Len	
End Seq.req	AP-Context	AREP	
Act Param.req	AP-Context	AREP, Area Code, Activate	
Act Para Brct.req	AP-Context	AREP, Area Code	
Load ARL DP master CI2.req	AP-Context	List of ARL Entries	
Get ARL DP master CI2.req	AP-Context	(none)	
Load CRL DP master CI2.req	AP-Context	List of CRL Entries	
Get CRL DP master CI2.req	AP-Context	(none)	
Initiate Load.req	AP-Context	AREP, Slot Number, LR Index, Load Type, Load Image Size, Intersegment Request, Timeout	
Pull Segment.req	AP-Context	AREP, Slot Number, LR Index, Segment Length	
Push Segment.req	AP-Context	AREP, Slot Number LR Index, Segment Length Segment Number, More Follows, Data	
Terminate Load.req	AP-Context	AREP Slot Number LR Index	
Start.req	AP-Context	AREP Slot Number FI Index Execution Argument	
Stop.req	AP-Context	AREP Slot Number FI Index	
Resume.req	AP-Context	AREP Slot Number FI Index Execution Argument	
Reset.req	AP-Context	AREP Slot Number FI Index	
Call.req	FSPMS	AREP Slot Number FI Index Execution Argument	

Table 433 shows the service primitives including their associated parameters issued by the AP-Context and received by the FSPMS.

Table 433 – Primitives issued by FSPMM2 to AP-Context

Primitive name	Source	Associated parameters	Functions
Init DP master Cl2.cnf	FSPMM2	(none)	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Reset.cnf	FSPMM2	(none)	
Read Slave Diag.cnf(+)	FSPMM2	AREP, Diag Data	
Read Slave Diag.cnf(-)	FSPMM2	AREP, Status	
Get Cfg.cnf(+)	FSPMM2	AREP, Cfg Data	
Get Cfg.cnf(-)	FSPMM2	AREP, Status	
Read Output.cnf(+)	FSPMM2	AREP, Output Data	
Read Output.cnf(-)	FSPMM2	AREP, Status	
Read Input.cnf(+)	FSPMM2	AREP, Input Data	
Read Input.cnf(-)	FSPMM2	AREP, Status	
Set Slave Add.cnf(+)	FSPMM2	AREP	
Set Slave Add.cnf(-)	FSPMM2	AREP, Status	
Initiate.cnf(+)	FSPMM2	AREP, Max Len Data Unit, Features Supported, Profile Features Supported, Profile Ident Number, Add Addr Param	
Initiate.cnf(-)	FSPMM2	AREP, Error Decode, Error Code 1 Error Code 2	
Read.cnf(+)	FSPMM2	AREP, Length, Data	
Read.cnf(-)	FSPMM2	AREP, Error Decode, Error Code 1 Error Code 2	
Write.cnf(+)	FSPMM2	AREP, Length	
Write.cnf(-)	FSPMM2	AREP, Error Decode, Error Code 1 Error Code 2	
Data Transport.cnf(+)	FSPMM2	AREP, Length, Data	
Data Transport.cnf(-)	FSPMM2	AREP, Error Decode, Error Code 1 Error Code 2	
Get Master Diag.cnf(+)	FSPMM2	AREP, Diagnosis Data	
Get Master Diag.cnf(-)	FSPMM2	AREP, Status	
Start Seq.cnf(+)	FSPMM2	AREP, Max Len Data Unit	
Start Seq.cnf(-)	FSPMM2	AREP, Status	
Download.cnf(+)	FSPMM2	AREP	
Download.cnf(-)	FSPMM2	AREP, Status	
Upload.cnf(+)	FSPMM2	AREP, Data	

Table 433 (continued)

Primitive name	Source	Associated parameters	Functions
Upload.cnf(-)	FSPMM2	AREP, Status	
End Seq.cnf(+)	FSPMM2	AREP	
End Seq.cnf(-)	FSPMM2	AREP, Status	
Act Param.cnf(+)	FSPMM2	AREP	
Act Param.cnf(-)	FSPMM2	AREP, Status	
Act Para Brct.cnf(+)	FSPMM2	AREP	
Act Para Brct.cnf(-)	FSPMM2	AREP, Status	
Event.ind	FSPMM2	Event, Add Info	
DP master CI2 Reject.ind	FSPMM2	AREP, Reason	
Abort.ind	FSPMM2	AREP, Locally Generated, Subnet, Instance, Reason_Code, Additional_Detail	
DP master CI2 Fault.ind	FSPMM2	(none)	
Load ARL DP master CI2.cnf(+)	FSPMM2	(none)	
Load ARL DP master CI2.cnf(-)	FSPMM2	Status	
Get ARL DP master CI2.cnf(+)	FSPMM2	List of ARL Entries	
Get ARL DP master CI2.cnf(-)	FSPMM2	Status	
Load CRL DP master CI2.cnf(+)	FSPMM2	(none)	
Load CRL DP master CI2.cnf(-)	FSPMM2	Status	
Get CRL DP master CI2.cnf(+)	FSPMM2	List of CRL Entries	
Get CRL DP master CI2.cnf(-)	FSPMM2	Status	
Initiate Load.cnf(+)	FSPMM2	AREP, Actual LR Size, Max Response Delay, Max Segment Length	
Initiate Load.cnf(-)	FSPMM2	AREP, Error Code	
Pull Segment.cnf(+)	FSPMM2	AREP, Segment Length, Segment Number, More Follows, Data	
Pull Segment.cnf(-)	FSPMM2	AREP, Error Code	
Push Segment.cnf(+)	FSPMM2	AREP	
Push Segment.cnf(-)	FSPMM2	AREP, Error Code	
Terminate Load.cnf(+)	FSPMM2	AREP	
Terminate Load.cnf(-)	FSPMM2	AREP, Error Code	
Start.cnf(+)	FSPMM2	AREP	
Start.cnf(-)	FSPMM2	AREP Error Code Function Invocation State	
Stop.cnf(+)	FSPMM2	AREP	
Stop.cnf(-)	FSPMM2	AREP Error Code Function Invocation State	
Resume.cnf(+)	FSPMM2	AREP	

Table 433 (continued)

Primitive name	Source	Associated parameters	Functions
Resume.cnf(-)	FSPMM2	AREP Error Code Function Invocation State	
Reset.cnf(+)	FSPMM2	AREP	
Reset.cnf(-)	FSPMM2	AREP Error Code	
Call.cnf(+)	FSPMM2	AREP Result Argument	
Call.cnf(-)	FSPMM2	AREP Result Argument Error Code	

6.5.3.1.2 Parameters of FSPMM2 primitives

The parameters used with the primitives exchanged between the FSPMM2 and the AP-Context are described in FAL Service Definition Type 3 Fieldbus IEC 61158-5.

6.5.3.2 State machine description

This Machine is used to co-ordinate the Interactions between AP-Context and DMPMM1, MSCY1M, MSAC1M, MSAL1M and MMAC1. As the support of several State Machines for the communication with individual slave as well as timing constrains of the operation of slave require a consistent scheduling, this task is accomplished by FSPMM1 as well.

Local Variables

LR_State

(Array of Unsigned8)

This variable stores the state of the Load Region sequence of the corresponding AREP.

CurrentExtendedFunctionNum

(Array of Unsigned 8)

This variable stores the extended function number of the currently processed LR service of the corresponding AREP.

CurrentSlotNumber

(Array of Unsigned8)

This variable stores the slot number of the currently processed LR service of the corresponding AREP.

6.5.3.3 FSPMM2 state table

Table 434 contains the complete description of the FSPMM2 state machine.

Table 434 – FSPMM2 state table

#	Current state	Event / condition => action	Next state
1	POWER-ON	Load ARL DP master CI2.req (List of ARL Entries) /valid parameters => Load ARL DP master CI2.cnf(+)	POWER-ON
2	POWER-ON	Load ARL DP master CI2.req (List of ARL Entries) /invalid parameters => Status := NO Load ARL DP master CI2.cnf(-) (Status)	POWER-ON
3	POWER-ON	Get ARL DP master CI2.req /ARL loaded => Get ARL DP master CI2.cnf(+) (List of ARL Entries)	POWER-ON
4	POWER-ON	Get ARL DP master CI2.req /ARL not loaded => Status := NO Get ARL DP master CI2.cnf(-) (Status)	POWER-ON
5	POWER-ON	Load CRL DP master CI2.req (List of CRL Entries) /valid parameters => Load CRL DP master CI2.cnf(+)	POWER-ON
6	POWER-ON	Load CRL DP master CI2.req (List of CRL Entries) /invalid parameters => Status := NO Load CRL DP master CI2.cnf(-) (Status)	POWER-ON
7	POWER-ON	Get CRL DP master CI2.req /CRL loaded => Get CRL DP master CI2.cnf(+) (List of CRL Entries)	POWER-ON
8	POWER-ON	Get CRL DP master CI2.req /CRL not loaded => Status := NO Get CRL DP master CI2.cnf(-) (Status)	POWER-ON
9	PON	FSPMM2_Init DP master CI2 .req(Bus_Para) => StoreNumberIssuedMinit() Current Extended Function Num[AREP]=No_Service LR_State[AREP]:=W-LR-REQ DMPMM2_Minit_DLL.req(Bus_Para) MMAC_Minit_MM.req MSAC2M_Minit_MS2.req(First_MS2_AREP ..Last_MS2_AREP)	INIT-DLM
10	INIT-DLM	DMPMM2_Minit_DLL.cnf => ignore	INIT-DLM
11	INIT-DLM	MMAC2_Minit_MM.cnf => ignore	INIT-DLM
12	INIT-DLM	MSAC2M_Minit_MS2.cnf (CREP) /IsLastWaitingMinit()=FALSE => ignore	INIT-DLM
13	INIT-DLM	MSAC2M_Minit_MS2.cnf (CREP) /IsLastWaitingMinit()=TRUE => FSPMM2_Init DP master CI2 .cnf	RUN
14	RUN	Any FSPMM2.req /wrong AREP.AR_Type => ignore	RUN

Table 434 (continued)

#	Current state	Event / condition => action	Next state
15	RUN	Any MMAC1.cnf /SetContext(Rem_add, Service) = FALSE => ignore	RUN
16	RUN	Any MSAC2M.req /SetContext(CREP, Service) = FALSE => ignore	RUN
17	RUN	Any DMPMM2.req /SetContext(Rem_add, Service) = FALSE => ignore	RUN
18	RUN	FSPMM2_Abort.req(AREP, Subnet, Instance, Reason_Code) /AREP.AR_Type = MS0 => DMPMM2_Abort.req	RUN
19	RUN	FSPMM2_Abort.req(AREP, Subnet, Instance, Reason_Code) /AREP.AR_Type = MM1 => MMAC2_Abort.req	RUN
20	RUN	FSPMM2_Abort.req(AREP, Subnet, Instance, Reason_Code) /AREP.AR_Type = MS2 => Current Extended Function Num[AREP]=No_Service LR_State[AREP]:=W-LR-REQ MSAC2M_Abort.req(CREP, Subnet, Instance, Reason_Code)	RUN
21	RUN	MSAC2M_Abort.ind(CREP, Locally_Generated, Subnet, Instance, Reason_Code, Abort_Detail) /SetContext(CREP, Service) = TRUE => Current Extended Function Num[AREP]=No_Service LR_State[AREP]:=W-LR-REQ FSPMM2_Abort.ind(AREP, Locally_Generated, Subnet, Instance, Reason Code, Additional_Detail)	RUN
22	RUN	DMPMM2_Event.ind(Event, Add_info) => FSPMM2_Event.ind(Event=Event/Fault, Add_Info=Add_info)	RUN
23	RUN	FSPMM2_Read_Slave_Diag.req(AREP) /AREP.AR_Type = MS0 => DMPMM2_Read_Slave_Diag.req(CREP.Rem_Add)	RUN
24	RUN	DMPMM2_Read_Slave_Diag.cnf(+)(Rem_Add, Diag_Data) /SetContext(Rem_add, Service) = TRUE => FSPMM2_Read_Slave_Diag.cnf(+)(AREP, Diag_Data)	RUN
25	RUN	DMPMM2_Read_Slave_Diag.cnf(-)(Rem_Add, Status) /SetContext(Rem_add, Service) = TRUE => FSPMM2_Read_Slave_Diag.cnf(-)(AREP, Status=Status)	RUN
26	RUN	FSPMM2_Get_Cfg.req(AREP) /AREP.AR_Type = MS0 => DMPMM2_Get_Cfg.req(CREP.Rem_Add)	RUN
27	RUN	DMPMM2_Get_Cfg.cnf(+)(Rem_Add, Cfg_Data) /SetContext(Rem_add, Service) = TRUE => FSPMM2_Get_Cfg.cnf(+)(AREP, Cfg_Data)	RUN
28	RUN	DMPMM2_Get_Cfg.cnf(-)(Rem_Add, Status) /SetContext(Rem_add, Service) = TRUE => FSPMM2_Get_Cfg.cnf(-)(AREP, Status)	RUN
29	RUN	FSPMM2_Read_Input.req(AREP) /AREP.AR_Type = MS0 => DMPMM2_Read_Input.req(CREP.Rem_Add)	RUN

Table 434 (continued)

#	Current state	Event / condition => action	Next state
30	RUN	DMPMM2_Read_Input.cnf(+)(Rem_Add, Inp_Data) /SetContext(Rem_add, Service) = TRUE => FSPMM2_Read_Input.cnf(+)(AREP, Inp_Data)	RUN
31	RUN	DMPMM2_Read_Input.cnf(-)(Rem_Add, Status) /SetContext(Rem_add, Service) = TRUE => FSPMM2_Read_Input.cnf(-)(AREP, Status)	RUN
32	RUN	FSPMM2_Read_Output.req(AREP) /AREP.AR_Type = MS0 => DMPMM2_Read_Output.req(CREP.Rem_Add)	RUN
33	RUN	DMPMM2_Read_Output.cnf(+)(Rem_Add, Outp_Data) /SetContext(Rem_add, Service) = TRUE => FSPMM2_Read_Output.cnf(+)(AREP, Outp_Data)	RUN
34	RUN	DMPMM2_Read_Output.cnf(-)(Rem_Add, Status) /SetContext(Rem_add, Service) = TRUE => FSPMM2_Read_Output.cnf(-)(AREP, Status)	RUN
35	RUN	FSPMM2_Set_Slave_Add.req(AREP, New_Slave_Add, Ident_Number, No_Add_Chg, Rem_Slave_Data) /AREP.AR_Type = MS0 => DMPMM2_Set_Slave_Add.req(CREP.Rem_Add, New_Slave_Add, Ident_Number, No_Add_Chg, Rem_Slave_Data)	RUN
36	RUN	DMPMM2_Set_Slave_Add.cnf(+)(Rem_Add) /SetContext(Rem_add, Service) = TRUE => FSPMM2_Set_Slave_Add.cnf(+)(AREP)	RUN
37	RUN	DMPMM2_Set_Slave_Add.cnf(-)(Rem_Add, Status) /SetContext(Rem_add, Service) = TRUE => FSPMM2_Set_Slave_Add.cnf(-)(AREP, Status=Status)	RUN
38	RUN	Any DMPMM2.req /SetContext(Rem_add, Service) = FALSE => ignore	RUN
39	RUN	FSPMM2_Initiate.req(AREP, Send Timeout, Features Supported, Profile Features Supported, Profile Ident Number, Add Addr Param) /AREP.AR_Type = MS2 => MSAC2M_Initiate.req(CREP, Send Timeout, Features Supported, Profile Features Supported, Profile Ident Number, Add Addr Param)	RUN
40	RUN	MSAC2M_Initiate.cnf(+)(CREP, Max Len Data Unit, Features Supported, Profile Features Supported, Profile Ident Number, Add Addr Param) /SetContext(CREP, Service) = TRUE => FSPMM2_Initiate.cnf(+)(AREP, Max Len Data Unit, Features Supported, Profile Features Supported, Profile Ident Number, Add Addr Param)	RUN
41	RUN	MSAC2M_Initiate.cnf(-)(CREP, Error Decode, Error Code 1, Error Code 2) /SetContext(CREP, Service) = TRUE => FSPMM2_Initiate.cnf(-)(AREP, Error Decode, Error Code 1, Error Code 2)	RUN
42	RUN	FSPMM2_Read.req(AREP, Slot Number, Index, Length) /AREP.AR_Type = MS2 => MSAC2M_Read.req(CREP, Slot Number, Index, Length)	RUN
43	RUN	MSAC2M_Read.cnf(+)(CREP, Length, Data) /SetContext(CREP, Service) = TRUE => FSPMM2_Read.cnf(+)(AREP, Length, Data)	RUN

Table 434 (continued)

#	Current state	Event / condition => action	Next state
44	RUN	MSAC2M_Read.cnf(-)(CREP, Error Decode, Error Code 1, Error Code 2) /SetContext(CREP, Service) = TRUE => FSPMM2_Read.cnf(-)(AREP, Error Decode, Error Code 1, Error Code 2)	RUN
45	RUN	FSPMM2_Write.req(AREP, Slot Number, Index, Length, Data) /AREP.AR_Type = MS2 => MSAC2M_Write.req(CREP, Slot Number, Index, Length, Data)	RUN
46	RUN	MSAC2M_Write.cnf(+)(CREP, Length) /SetContext(CREP, Service) = TRUE => FSPMM2_Write.cnf(+)(AREP, Length)	RUN
47	RUN	MSAC2M_Write.cnf(-)(CREP, Error Decode, Error Code 1, Error Code 2) /SetContext(CREP, Service) = TRUE => FSPMM2_Write.cnf(-)(AREP, Error Decode, Error Code 1, Error Code 2)	RUN
48	RUN	FSPMM2_Data_Transport.req(AREP, Slot Number, Index, Length, Data) /AREP.AR_Type = MS2 => MSAC2M_Data_Transport.req(CREP, Slot Number, Index, Length, Data)	RUN
49	RUN	MSAC2M_Data_Transport.cnf(+)(CREP, Length, Data) /SetContext(CREP, Service) = TRUE => FSPMM2_Data_Transport.cnf(+)(AREP, Length, Data)	RUN
50	RUN	MSAC2M_Data_Transport.cnf(-)(CREP, Error Decode, Error Code 1, Error Code 2) /SetContext(CREP, Service) = TRUE => FSPMM2_Data_Transport.cnf(-)(AREP, Error Decode, Error Code 1, Error Code 2)	RUN
51	RUN	MSAC2M_Closed.ind(CREP) /SetContext(CREP, Service) = TRUE => FSPMM2_DP_master_CI2_Closed.ind(AREP)	RUN
52	RUN	MMAC2_Reject.ind(Rem_Add, Reason_Code) /SetContext(Rem_add, Service) = TRUE => FSPMM2_DP_master_CI2_Reject.ind(AREP, Reason)	RUN
53	RUN	FSPMM2_Get_Master_Diag.req(AREP, MDiag Identifier) /AREP.AR_Type = MM1 => MMAC2_Get_Master_Diag.req(AREP, MDiag Identifier)	RUN
54	RUN	MMAC2_Get_Master_Diag.cnf(+)(Rem_Add, Diagnosis Data) /SetContext(Rem_add, Service) = TRUE => FSPMM2_Get_Master_Diag.cnf(+)(AREP, Diagnosis Data)	RUN
55	RUN	MMAC2_Get_Master_Diag.cnf(-)(Rem_Add, Status) /SetContext(Rem_add, Service) = TRUE => FSPMM2_Get_Master_Diag.cnf(-)(AREP, Status)	RUN
56	RUN	FSPMM2_Start_Seq.req(AREP, Area Code, Timeout) /AREP.AR_Type = MM1 => MMAC2_Start_Seq.req(AREP, Area Code, Timeout)	RUN
57	RUN	MMAC2_Start_Seq.cnf(+)(Rem_Add, Max Len Data Unit) /SetContext(Rem_add, Service) = TRUE => FSPMM2_Start_Seq.cnf(+)(AREP, Max Len Data Unit)	RUN
58	RUN	MMAC2_Start_Seq.cnf(-)(Rem_Add, Status) /SetContext(Rem_add, Service) = TRUE => FSPMM2_Start_Seq.cnf(-)(AREP, Status)	RUN
59	RUN	FSPMM2_End_Seq.req(AREP) /AREP.AR_Type = MM1 => MMAC2_End_Seq.req(AREP)	RUN

Table 434 (continued)

#	Current state	Event / condition => action	Next state
60	RUN	MMAC2_End_Seq.cnf(+)(Rem_Add) /SetContext(Rem_add, Service) = TRUE => FSPMM2_End_Seq.cnf(+)(AREP)	RUN
61	RUN	MMAC2_End_Seq.cnf(-)(Rem_Add, Status) /SetContext(Rem_add, Service) = TRUE => FSPMM2_End_Seq.cnf(-)(AREP, Status)	RUN
62	RUN	FSPMM2_Download.req(AREP, Area Code, Add Offset, Data) /AREP.AR_Type = MM1 => MMAC2_Download.req(AREP, Area Code, Add Offset, Data)	RUN
63	RUN	MMAC2_Download.cnf(+)(Rem_Add) /SetContext(Rem_add, Service) = TRUE => FSPMM2_Download.cnf(+)(AREP)	RUN
64	RUN	MMAC2_Download.cnf(-)(Rem_Add, Status) /SetContext(Rem_add, Service) = TRUE => FSPMM2_Download.cnf(-)(AREP, Status)	RUN
65	RUN	FSPMM2_Upload.req(AREP, Area Code, Add Offset, Data Len) /AREP.AR_Type = MM1 => MMAC2_Upload.req(AREP, Area Code, Add Offset, Data Len)	RUN
66	RUN	MMAC2_Upload.cnf(+)(Rem_Add, Data) /SetContext(Rem_add, Service) = TRUE => FSPMM2_Upload.cnf(+)(AREP, Data)	RUN
67	RUN	MMAC2_Upload.cnf(-)(Rem_Add, Status) /SetContext(Rem_add, Service) = TRUE => FSPMM2_Upload.cnf(-)(AREP, Status)	RUN
68	RUN	FSPMM2_Act_Param.req(AREP, Area Code, Activate) /AREP.AR_Type = MM1 => MMAC2_Act_Param.req(AREP, Area Code, Activate)	RUN
69	RUN	MMAC2_Act_Param.cnf(+)(Rem_Add) /SetContext(Rem_add, Service) = TRUE => FSPMM2_Act_Param.cnf(+)(AREP)	RUN
70	RUN	MMAC2_Act_Param.cnf(-)(Rem_Add, Status) /SetContext(Rem_add, Service) = TRUE => FSPMM2_Act_Param.cnf(-)(AREP, Status)	RUN
71	RUN	FSPMM2_Act_Param_Brct.req(AREP, Area Code) /AREP.AR_Type = MM2 => MMAC2_Act_Param_Brct.req(AREP, Area Code, Activate)	RUN
72	RUN	MMAC2_Act_Param_Brct.cnf(+)(Rem_Add) /SetContext(Rem_add, Service) = TRUE => FSPMM2_Act_Param_Brct.cnf(+)(AREP)	RUN
73	RUN	MMAC2_Act_Param_Brct.cnf(-)(Rem_Add, Status) /SetContext(Rem_add, Service) = TRUE => FSPMM2_Act_Param_Brct.cnf(-)(AREP, Status)	RUN
74	any state	DMPMM2_SYCL_Time_Event.cnf (Send Delay Time, Status) => FSPMM2_SYCL_Time_Event.cnf (AREP, Send Delay Time, Status)	SAME
75	any state	FSPMM2_SYCL_Time_Event.req (AREP) => DMPMM2_SYCL_Time_Event.req	SAME

Table 434 (continued)

#	Current state	Event / condition => action	Next state
76	any state	FSPMM2_SYCL_Clock_Value.req (AREP, Clock Value Time Event, Clock Value previous TE, Clock Value Status) => DMPMM2_SYCL_Clock_Value.req (Clock Value Time Event, Clock Value previous TE, Clock Value Status)	SAME
77	any state	DMPMM2_SYCL_Clock_Value.cnf (Status) => FSPMM2_SYCL_Clock_Value.cnf (AREP, Status)	SAME
78	any state	DMPMM2_SYCL_Clock_Value.ind (Clock Value Time Event, Clock Value previous TE, Clock Value Status, Receive Delay Time, Src add Clk msg) => FSPMM2_SYCL_Clock_Value.ind (AREP, Clock Value Time Event, Clock Value previous TE, Clock Value Status, Receive Delay Time, Src add Clk msg)	SAME
79	any state	FSPMM2_Reset DP master CI2 .req => StoreNumberIssuedResets() FSPMM2_UserReset = TRUE DMPMM2_Reset.req MMAC2_Reset.req MSAC2M_Reset.req(First_MS2_AREP ..Last_MS2_AREP)	WAIT-RESET
80	any state	DMPMM2_Fault.ind => StoreNumberIssuedResets() FSPMM2_UserReset = FALSE DMPMM2_Reset.req MMAC2_Reset.req MSAC2M_Reset.req(First_MS2_AREP ..Last_MS2_AREP) FSPMM2 DP master CI2 Fault.ind	WAIT-RESET
81	any state	MMAC2_Fault.ind => StoreNumberIssuedResets() FSPMM2_UserReset = FALSE DMPMM2_Reset.req MMAC2_Reset.req MSAC2M_Reset.req(First_MS2_AREP ..Last_MS2_AREP) FSPMM2 DP master CI2 Fault.ind	WAIT-RESET
82	WAIT-RESET	DMPMM2_Reset.cnf => no action	WAIT-RESET
83	WAIT-RESET	MMAC2_Reset.cnf => no action	WAIT-RESET
84	WAIT-RESET	MSAC2M_Reset.cnf (CREP) /IsLastWaitingReset()=FALSE => no action	WAIT-RESET
85	WAIT-RESET	MSAC2M_Reset.cnf (CREP) /IsLastWaitingReset()=TRUE AND FSPMM2_UserReset = TRUE => FSPMM2_Reset DP master CI2 .cnf	PON
86	WAIT-RESET	MSAC2M_Reset.cnf (CREP) /IsLastWaitingReset()=TRUE AND FSPMM2_UserReset = FALSE => no action	PON

Table 434 (continued)

#	Current state	Event / condition => action	Next state
87	RUN	FSPMM2_Initiate_Load.req (AREP, Slot Number, LR Index, Load Type, Load Image Size, Intersegment Request Timeout, User Specific) /LR_State[AREP]==W-LR-REQ &AREP.AR_Type=MS2 => Current Extended Function Num[AREP]:=Initiate_Load Current Slot Number[AREP]:=Slot Number Index:=255 Length:=10+sizeof(User Specific) Data.w.IL.Extended_Function_Num:=Initiate_Load Data.w.IL.LR_Index:=LR Index Data.w.IL.Load_Type:=Load Type Data.w.IL.Load_Image_Size:=Load Image Size Data.w.IL.User_Specific:=User Specific Data.w.IL.Intersegment_Request_Timeout:=Intersegment Request Timeout MSAC2M_Write.req(CREP, Slot Number, Index, Length, Data) LR_State[AREP]:=W-LR-CNF	RUN
88	RUN	MSAC2M_Write.cnf(+)(CREP, Length) /LR_State[AREP]==W-LR-CNF &SetContext(CREP, Service)=TRUE && Current Extended Function Num[AREP]=Initiate_Load => Slot_Number:=Current Slot Number[AREP] Index:=255 Length:=10 MSAC2M_Read.req(CREP, Slot Number, Index, Length) LR_State[AREP]:=W-LR-CNF	RUN
89	RUN	MSAC2M_Read.cnf(+)(CREP, Length, Data) /LR_State[AREP]==W-LR-CNF &SetContext(CREP, Service)=TRUE && Current Extended Function Num[AREP]=Initiate_Load => Current Extended Function Num[AREP]=No_Service Actual LR Size:=Data.r.IL.Actual_LR_Size Max Response Delay:=Data.r.IL.Max_Response_Delay Max Segment Length:=Data.r.IL.Max_Segment_Length User Specific:=Data.r.IL.User_Specific FSPMM2_Initiate_Load.cnf(+)(AREP, Actual LR Size, Max Response Delay, Max Segment Length, User Specific) LR_State[AREP]:=W-LR-REQ	RUN
90	RUN	MSAC2M_Write.cnf(-)(CREP, Error Decode, Error Code 1, Error Code 2) /LR_State[AREP]==W-LR-CNF &SetContext(CREP, Service)=TRUE && Current Extended Function Num[AREP]=Initiate_Load => Current Extended Function Num[AREP]=No_Service Error Code:=Error_Code_1 FSPMM2_Initiate_Load.cnf(-)(AREP, Error Code) LR_State[AREP]:=W-LR-REQ	RUN
91	RUN	FSPMM2_Pull_Segment.req (AREP, Slot Number, LR Index, Segment Length) /LR_State[AREP]==W-LR-REQ &AREP.AR_Type=MS2 => Current Extended Function Num[AREP]:=Pull Current LR Index[AREP]:=LR Index Index:=255 Length:=Segment Length MSAC2M_Read.req(CREP, Slot Number, Index, Length) LR_State[AREP]:=W-LR-CNF	RUN

Table 434 (continued)

#	Current state	Event / condition => action	Next state
92	RUN	MSAC2M_Read.cnf(+)(CREP, Length, Data) /LR_State[AREP]==W-LR-CNF &SetContext(CREP, Service)=TRUE && Current Extended Function Num[AREP]=Pull => Current Extended Function Num[AREP]=No_Service Segment Number:=Data.PULL.Sequence Number More Follows:=Data.PULL.Options.More Follows Data:=Data.PULL.Region Data Segment Length:=Length-6 FSPMM2_Pull_Segment.cnf(+) (AREP, Segment Length, Segment Number, More Follows, Data) LR_State[AREP]:=W-LR-REQ	RUN
93	RUN	MSAC2M_Read.cnf(-)(CREP, Error Decode, Error Code 1, Error Code 2) /LR_State[AREP]==W-LR-CNF &SetContext(Service, Service)=TRUE && Current Extended Function Num[AREP]=Pull => Current Extended Function Num[AREP]=No_Service Error Code:=Error_Code_1 FSPMM2_Pull_Segment.cnf(-) (AREP, Error Code) LR_State[AREP]:=W-LR-REQ	RUN
94	RUN	FSPMM2_Push_Segment.req (AREP, Slot Number, LR Index, Segment Length, Segment Number, More Follows, Data) /LR_State[AREP]==W-LR-REQ &AREP.AR_Type=MS2 => Current Extended Function Num[AREP]:=Push Current Slot Number[AREP]:=Slot Number Index:=255 Length:=6+Segment Length Data.PUSH.Extended_Function_Num:=Push Data.PUSH.LR_Index:=LR Index Data.PUSH.Segment_Number:=Segment Number Data.PUSH.Option.More_Follows:=More Follows Data.PUSH.Data:=Data MSAC2M_Write.req(CREP, Slot Number, Index, Length, Data) LR_State[AREP]:=W-LR-CNF	RUN
95	RUN	MSAC2M_Write.cnf(+)(CREP, Length) /LR_State[AREP]==W-LR-CNF &SetContext(CREP, Service)=TRUE && Current Extended Function Num[AREP]=Push => Current Extended Function Num[AREP]=No_Service Segment Number:=Data.PUSH.Sequence Number More Follows:=Data.PUSH.Options.More Follows Data:=Data.PUSH.Region Data Segment Length:=Length-6 FSPMM2_Push_Segment.cnf(+) (AREP) LR_State[AREP]:=W-LR-REQ	RUN
96	RUN	MSAC2M_Write.cnf(-)(CREP, Error Decode, Error Code 1, Error Code 2) /LR_State[AREP]==W-LR-CNF &SetContext(CREP, Service)=TRUE && Current Extended Function Num[AREP]=Push => Current Extended Function Num[AREP]=No_Service Error Code:=Error_Code_1 FSPMM2_Push_Segment.cnf(-) (AREP, Error Code) LR_State[AREP]:=W-LR-REQ	RUN

Table 434 (continued)

#	Current state	Event / condition => action	Next state
97	RUN	FSPMM2_Terminate_Load.req (AREP, Slot Number, LR Index) /LR_State[AREP]==W-LR-REQ &AREP.AR_Type=MS2 => Current Extended Function Num[AREP]:=Terminate_Load Current Slot Number[AREP]:=Slot Number Index:=255 Length:=6 Data.TL.Extended_Function_Num:=Terminate_Load Data.TL.LR_Index:=LR Index MSAC2M_Write.req(CREP, Slot Number, Index, Length, Data) LR_State[AREP]:=W-LR-CNF	RUN
98	RUN	MSAC2M_Write.cnf(+)(CREP, Length) /LR_State[AREP]==W-LR-CNF &SetContext(CREP, Service)=TRUE && Current Extended Function Num[AREP]:=Terminate_Load => Current Extended Function Num[AREP]:=No_Service FSPMM2_Terminate_Load.cnf(+)(AREP) LR_State[AREP]:=W-LR-REQ	RUN
99	RUN	MSAC2M_Write.cnf(-)(CREP, Error Decode, Error Code 1, Error Code 2) /LR_State[AREP]==W-LR-CNF &SetContext(CREP, Service)=TRUE && Current Extended Function Num[AREP]:=Terminate_Load => Current Extended Function Num[AREP]:=No_Service Error Code:=Error_Code_1 FSPMM2_Terminate_Load.cnf(-)(AREP, Error Code) LR_State[AREP]:=W-LR-REQ	RUN
100	RUN	FSPMM2_Call.req (AREP, Slot Number, FI Index, Execution Argument) /LR_State[AREP]==W-LR-REQ &AREP.AR_Type=MS2 => Current Extended Function Num[AREP]:=Call Current Slot Number[AREP]:=Slot Number Index:=255 Length:=4+sizeof(Execution Argument) Data.w.CALL.Extended_Function_Num:=Call Data.w.CALL.FI_Index:=FI Index Data.w.CALL.Execution_Argument:=Execution Argument MSAC2M_Write.req(CREP, Slot Number, Index, Length, Data) LR_State[AREP]:=W-LR-CNF	RUN
101	RUN	MSAC2M_Write.cnf(+)(CREP, Length) /LR_State[AREP]==W-LR-CNF &SetContext(CREP, Service)=TRUE && Current Extended Function Num[AREP]:=Call => Slot_Number:=Current Slot Number[AREP] Index:=255 Length:=AREP.Max_PDU_Length MSAC2M_Read.req(CREP, Slot Number, Index, Length) LR_State[AREP]:=W-LR-CNF	RUN
102	RUN	MSAC2M_Read.cnf(+)(CREP, Length, Data) /LR_State[AREP]==W-LR-CNF &SetContext(CREP, Service)=TRUE && Current Extended Function Num[AREP]:=Call => Current Extended Function Num[AREP]:=No_Service Result Argument:=Data.r.CALL.Result_Argument FSPMM2_Call.cnf(+)(AREP, Result Argument) LR_State[AREP]:=W-LR-REQ	RUN

Table 434 (continued)

#	Current state	Event / condition => action	Next state
103	RUN	MSAC2M_Write.cnf(-)(CREP, Error Decode, Error Code 1, Error Code 2) /LR_State[AREP]==W-LR-CNF &SetContext(CREP, Service)=TRUE && Current Extended Function Num[AREP]=Call => Current Extended Function Num[AREP]=No_Service Error Code:=Error_Code_1 FSPMM2_Call.cnf(-) (AREP, Error Code) LR_State[AREP]:=W-LR-REQ	RUN
104	RUN	FSPMM2_Start.req (AREP, Slot Number, FI Index, Execution Argument) /LR_State[AREP]==W-LR-REQ &AREP.AR_Type=MS2 => Current Extended Function Num[AREP]:=Start Current Slot Number[AREP]:=Slot Number Index:=255 Length:=4+sizeof(Execution Argument) Data.FIS.Extended_Function_Num:=Start Data.FIS.FI_Index:=FI Index Data.FIS.Execution_Argument:=Execution Argument MSAC2M_Write.req(CREP, Slot Number, Index, Length, Data) LR_State[AREP]:=W-LR-CNF	RUN
105	RUN	MSAC2M_Write.cnf(+)(CREP, Length) /LR_State[AREP]==W-LR-CNF &SetContext(CREP, Service)=TRUE && Current Extended Function Num[AREP]=Start => Current Extended Function Num[AREP]=No_Service FSPMM2_Start.cnf(+) (AREP) LR_State[AREP]:=W-LR-REQ	RUN
106	RUN	MSAC2M_Write.cnf(-)(CREP, Error Decode, Error Code 1, Error Code 2) /LR_State[AREP]==W-LR-CNF &SetContext(CREP, Service)=TRUE && Current Extended Function Num[AREP]=Start => Current Extended Function Num[AREP]=No_Service Error Code:=Error_Code_1 FSPMM2_Start.cnf(-) (AREP, Error Code) LR_State[AREP]:=W-LR-REQ	RUN
107	RUN	FSPMM2_Stop.req (AREP, Slot Number, FI Index) /LR_State[AREP]==W-LR-REQ &AREP.AR_Type=MS2 => Current Extended Function Num[AREP]:=Stop Current Slot Number[AREP]:=Slot Number Index:=255 Length:=4 Data.FIS.Extended_Function_Num:=Stop Data.FIS.FI_Index:=FI Index MSAC2M_Write.req(CREP, Slot Number, Index, Length, Data) LR_State[AREP]:=W-LR-CNF	RUN
108	RUN	MSAC2M_Write.cnf(+)(CREP, Length) /LR_State[AREP]==W-LR-CNF &SetContext(CREP, Service)=TRUE && Current Extended Function Num[AREP]=Stop => Current Extended Function Num[AREP]=No_Service FSPMM2_Stop.cnf(+) (AREP) LR_State[AREP]:=W-LR-REQ	RUN
109	RUN	MSAC2M_Write.cnf(-)(CREP, Error Decode, Error Code 1, Error Code 2) /LR_State[AREP]==W-LR-CNF &SetContext(CREP, Service)=TRUE && Current Extended Function Num[AREP]=Stop => Current Extended Function Num[AREP]=No_Service Error Code:=Error_Code_1 FSPMM2_Stop.cnf(-) (AREP, Error Code) LR_State[AREP]:=W-LR-REQ	RUN

Table 434 (continued)

#	Current state	Event / condition => action	Next state
110	RUN	FSPMM2_Resume.req (AREP, Slot Number, FI Index, Execution Argument) /LR_State[AREP]==W-LR-REQ &AREP.AR_Type=MS2 => Current Extended Function Num[AREP]:=Resume Current Slot Number[AREP]:=Slot Number Index:=255 Length:=4+sizeof(Execution Argument) Data.FIS.Extended_Function_Num:=Resume Data.FIS.FI_Index:=FI Index Data.FIS.Execution_Argument:=Execution Argument MSAC2M_Write.req(CREP, Slot Number, Index, Length, Data) LR_State[AREP]:=W-LR-CNF	RUN
111	RUN	MSAC2M_Write.cnf(+)(CREP, Length) /LR_State[AREP]==W-LR-CNF &SetContext(CREP, Service)=TRUE && Current Extended Function Num[AREP]=Resume => Current Extended Function Num[AREP]=No_Service FSPMM2_Resume.cnf(+)(AREP) LR_State[AREP]:=W-LR-REQ	RUN
112	RUN	MSAC2M_Write.cnf(-)(CREP, Error Decode, Error Code 1, Error Code 2) /LR_State[AREP]==W-LR-CNF &SetContext(CREP, Service)=TRUE && Current Extended Function Num[AREP]=Resume => Current Extended Function Num[AREP]=No_Service Error Code:=Error_Code_1 FSPMM2_Resume.cnf(-)(AREP, Error Code) LR_State[AREP]:=W-LR-REQ	RUN
113	RUN	FSPMM2_Reset.req (AREP, Slot Number, FI Index) /LR_State[AREP]==W-LR-REQ &AREP.AR_Type=MS2 => Current Extended Function Num[AREP]:=Reset Current Slot Number[AREP]:=Slot Number Index:=255 Length:=4 Data.FIS.Extended_Function_Num:=Reset Data.FIS.FI_Index:=FI Index MSAC2M_Write.req(CREP, Slot Number, Index, Length, Data) LR_State[AREP]:=W-LR-CNF	RUN
114	RUN	MSAC2M_Write.cnf(+)(CREP, Length) /LR_State[AREP]==W-LR-CNF &SetContext(CREP, Service)=TRUE && Current Extended Function Num[AREP]=Reset => Current Extended Function Num[AREP]=No_Service FSPMM2_Reset.cnf(+)(AREP) LR_State[AREP]:=W-LR-REQ	RUN
115	RUN	MSAC2M_Write.cnf(-)(CREP, Error Decode, Error Code 1, Error Code 2) /LR_State[AREP]==W-LR-CNF &SetContext(CREP, Service)=TRUE && Current Extended Function Num[AREP]=Reset => Current Extended Function Num[AREP]=No_Service Error Code:=Error_Code_1 FSPMM2_Reset.cnf(-)(AREP, Error Code) LR_State[AREP]:=W-LR-REQ	RUN

Table 434 (continued)

#	Current state	Event / condition => action	Next state
116	RUN	FSPMM2_Get_FI_State.req (AREP, Slot Number, FI Index) /LR_State[AREP]==W-LR-REQ &AREP.AR_Type=MS2 => Current Extended Function Num[AREP]:=Get_State Current Slot Number[AREP]:=Slot Number Index:=255 Length:=4 Data.w.STATE.Extended_Function_Num:=Get_State Data.w.STATE.FI_Index:=FI Index MSAC2M_Write.req(CREP, Slot Number, Index, Length, Data) LR_State[AREP]:=W-LR-CNF	RUN
117	RUN	MSAC2M_Write.cnf(+)(CREP, Length) /LR_State[AREP]==W-LR-CNF &SetContext(CREP, Service)=TRUE && Current Extended Function Num[AREP]:=Get_State => Slot_Number:=Current Slot Number[AREP] Index:=255 Length:=5 MSAC2M_Read.req(CREP, Slot Number, Index, Length) LR_State[AREP]:=W-LR-CNF	RUN
118	RUN	MSAC2M_Read.cnf(+)(CREP, Length, Data) /LR_State[AREP]==W-LR-CNF &SetContext(CREP, Service)=TRUE && Current Extended Function Num[AREP]:=Get_State => Current Extended Function Num[AREP]=No_Service FI State:=Data.r.STATE.State FSPMM2_Get_FI_State.cnf (+) (AREP, FI State) LR_State[AREP]:=W-LR-REQ	RUN
119	RUN	MSAC2M_Write.cnf(-)(CREP, Error Decode, Error Code 1, Error Code 2) /LR_State[AREP]==W-LR-CNF &SetContext(CREP, Service)=TRUE && Current Extended Function Num[AREP]:=Get_State => Current Extended Function Num[AREP]=No_Service Error Code:=Error_Code_1 FSPMM2_Get_State.cnf(-) (AREP, Error Code) LR_State[AREP]:=W-LR-REQ	RUN

6.5.3.4 Functions

Table 435 contains the functions used by the FSPMM2, their arguments and their descriptions.

Table 435 – Functions used by the FSPMM2

Function name	Description
SetContext(Rem_Add/CREP, Service)	This function checks if there exist an entry for the inputs Rem_Add/CREP and Service in the ARL and related CRL of the DP-master (Class 2). A) If there exist an entry then it returns TRUE and sets the local context according to the current CREP and AREP. B) Otherwise it returns FALSE.
IsLastWaitingReset()	This function returns TRUE if all Reset service confirmation are received otherwise it returns FALSE.
StoreNumberIssuedResets()	This function checks stores the number of issued Reset service request in a local variable.
IsLastWaitingMInit()	This function returns TRUE if all MInit service confirmation are received otherwise it returns FALSE.
StoreNumberIssuedMInit()	This function checks stores the number of issued MInit service request in a local variable.

6.6 Application Relationship Protocol Machines (ARPMs)

6.6.1 MSCY1S

6.6.1.1 Primitive definitions

6.6.1.1.1 Primitives exchanged between MSCY1S and FSPMS

Table 436 shows the service primitives including their associated parameters issued by FSPMS and received by the MSCY1S.

Table 436 – Primitives issued by FSPMS to MSCY1S

Primitive name	Source	Associated parameters	Functions
SInit MS0.req	FSPMS	(none)	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Reset.req	FSPMS	(none)	
Abort.req	FSPMS	(none)	
Application Ready.req	FSPMS	(none)	
Check User Prm Result.req	FSPMS	Prm_OK	
Check Ext User Prm Result.req	FSPMS	Ext_Prm_OK	
Check Cfg Result.req	FSPMS	Cfg_OK, Input Data Len, Output Data Len	
Set Cfg.req	FSPMS	Cfg Data	
Set Slave Diag.req	FSPMS	Ext Diag Flag, Ext Diag Overflow, Ext Diag Data	
Set Input.req	FSPMS	Input Data	
Get Output.req	FSPMS	(none)	

Table 437 shows the service primitives including their associated parameters issued by MSCY1S and received by the FSPMS.

Table 437 – Primitives issued by MSCY1S to FSPMS

Primitive name	Source	Associated parameters	Functions
SInit MS0.cnf	MSCY1S	(none)	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Reset.cnf	MSCY1S	(none)	
Check User Prm Result.cnf(+)	MSCY1S	(none)	
Check User Prm Result.cnf(-)	MSCY1S	Status	
Check Ext User Prm Result.cnf(+)	MSCY1S	(none)	
Check Ext User Prm Result.cnf(-)	MSCY1S	Status	
Check Cfg Result.cnf(+)	MSCY1S	(none)	
Check Cfg Result.cnf(-)	MSCY1S	Status	
Set Cfg.cnf(+)	MSCY1S	(none)	
Set Cfg.cnf(-)	MSCY1S	(none)	
Set Slave Diag.cnf(+)	MSCY1S	(none)	
Set Slave Diag.cnf(-)	MSCY1S	(none)	
Set Input.cnf(+)	MSCY1S	(none)	
Set Input.cnf(-)	MSCY1S	(none)	
Get Output.cnf(+)	MSCY1S	Output Data, Clear Flag, New Flag	
Get Output.cnf(-)	MSCY1S	(none)	
Start.ind	MSCY1S	Actual Enabled Alarms, Alarm Sequence, Alarm Limit	
Stop.ind	MSCY1S	(none)	
Fault.ind	MSCY1S	(none)	
Set Slave Add.ind	MSCY1S	New Slave Add, Ident Number, No Add Chg, Rem Slave Data	
Check User Prm.ind	MSCY1S	User Prm Data	
Check Ext User Prm.ind	MSCY1S	Ext User Prm Data	
Check Cfg.ind	MSCY1S	Check Cfg Mode, Cfg Data	
New Output.ind	MSCY1S	Clear Flag	
SYNCH Event.ind	MSCY1S	Status	
Global Control.ind	MSCY1S	Clear Command, Sync Command, Freeze Command, Group Select	

6.6.1.1.2 Parameters of MSCY1S primitives

The parameters used with the primitives exchanged between the FSPMS and the MSCY1S are described in FAL Service Definition Type 3 Fieldbus IEC 61158-5.

6.6.1.2 State machine description

The MSCY1S State Machine handles the following services issued by the DP-master (Class 1):

diagnostic,
parameterisation,
configuration,
(cyclic) data exchange.

The MSCY1S State Machine starts (Start.req/cnf) and stops (Stop.req/cnf) the MSAC1S State Machine that handles acyclic services. The MSAC1S State Machine sends an Abort.ind to the MSCY1S State Machine that closes also the cyclic connection. The MSCY1S State Machine informs the FSPMS that the MSAC1S and the MSCY1S have been started (Start.ind) or stopped (Stop.ind).

If a Slave does not implement the parameterization structures DPV1_Status 1-3, the MSAL1S and MSAC1S State Machines, the following macros should be left empty. In this case the variable Operation_Mode remains in state V0:

PV0V0 shall return TRUE

OPERATION_MODE_OK shall return TRUE

NOPRMCMD shall return TRUE

ISO_MODE_OK shall return TRUE

STOP_ISOM

STOP_ASM

CHECK_ALARM_START

SET_OPERATION_MODE

SET_ALARM_CHKCFG

EXE_Prm_CMD

START_C1

STOP_C1

START_C1_CON

STOP_C1_CON

Rules for checking parameterization data

The rules according to Table 438 shall be used to check the octets DPV1_Status_1 to DPV1_Status_3 from the Set_Prm-REQ-PDU with local capabilities of the DP-slave.

Table 438 – Rules for DPV1_Status_1, DPV1_Status_2 and DPV1_Status_3 check

Bit(s) in Set_Prm-REQ-PDU	Rules for checking	
Reserved bits in field DPV1_Status_1 (Bit 0, Bit 1, Bit 3, Bit 4)	These bits shall not be checked (don't care). => okay for any combination	
Bit WD_Base_1ms in field DPV1_Status_1	= FALSE(0): => okay	= TRUE (1) & WD_Base_1ms_supp = 1: => okay = TRUE (1) & WD_Base_1ms_supp = 0: => okay, but the DP-slave shall load it's local WD-Timer with a corrected value (based on the next possible value with 10 ms base)
Bit Publisher_Enable in field DPV1_Status_1	= FALSE(0): => okay	= TRUE(1) & Publisher_supp = 1: => okay = TRUE(1) & Publisher_supp = 0: => Prm_Fault
Bit Fail_Safe in field DPV1_Status_1	= FALSE(0) & Fail_Safe_required = 1: => Prm_Fault = FALSE(0) & Fail_Safe_required = 0: => okay	= TRUE(1) & Fail_Safe_supp = 1: => okay = TRUE(1) & Fail_Safe_supp = 0: => Prm_Fault
Bit DPV1_Enable in field DPV1_Status_1	= FALSE(0) & C1_Read_Write_required=0: => okay = FALSE(0) & C1_Read_Write_required=1: => Prm_Fault	= TRUE(1) & C1_Read_Write_supp = 0: => Prm_Fault = TRUE(1) & C1_Read_Write_supp = 1: => okay, open MS1 application relationship

Bit(s) in Set_Prm-REQ-PDU	Rules for checking	
Bit Check_Cfg_Mode in field DPV1_Status_2	= 0 (strict check) : => okay The application process has to check the data of the following Chk_Cfg-REQ-PDU in a strict way (equality).	= 1 (more flexible check): => okay; The application process has to check the data of the following Chk_Cfg-REQ-PDU in a more flexible way (compatibility).
Reserved Bit 1 in field DPV1_Status_2	= 0: => okay	= 1: => Prm_Fault
Enable_xx_Alarm Bits in field DPV1_Status_2	= FALSE(0) & xx_Alarm_required = 1 & DPV1_Status_1.DPV1_Enable = FALSE (0): => Prm_Fault = FALSE(0) & xx_Alarm_required = 0 & DPV1_Status_1.DPV1_Enable = FALSE (0): => okay = FALSE(0) & xx_Alarm_required = 1 & DPV1_Status_1.DPV1_Enable = TRUE (1): => Prm_Fault = FALSE(0) & xx_Alarm_required = 0 & DPV1_Status_1.DPV1_Enable = TRUE (1): => okay	= TRUE(1) & DPV1_Status_1.DPV1_Enable = FALSE(0): => Prm_Fault = TRUE(1) & DPV1_Status_1.DPV1_Enable = TRUE (1): => okay
Bits Alarm_Mode in field DPV1_Status_3	0 => okay	<>0 & Alarm_Sequence_Mode_count<>0: => okay <>0 & Alarm_Sequence_Mode_count=0: => Prm_Fault
Bit Prm_Structure in field DPV1_Status_3	= FALSE(0) & Prm_Structure_required=FALSE: =>okay = FALSE(0) & Prm_Structure_required=TRUE: => Prm_Fault	= TRUE(1) & Prm_Structure_supp=TRUE: =>okay = TRUE(1) & Prm_Structure_supp=FALSE: => Prm_Fault
Bit IsoM_Req in field DPV1_Status_3	= FALSE(0) & Isochronous_Mode_required=FALSE: =>okay = FALSE(0) & Isochronous_Mode_required=TRUE: => Prm_Fault	= TRUE(1) & (Isochronous_Mode_supp = TRUE & DPV1_Status_1.Fail_Safe = TRUE & Group_Ident< 0x80 & Freeze_Req=0 & Sync_Req=0): => okay TRUE(1) & !(Isochronous_Mode_supp = TRUE & DPV1_Status_1.Fail_Safe = TRUE & Group_Ident< 0x80 & Freeze_Req=0 & Sync_Req=0): => Prm-Fault
Bit PrmCmd in field DPV1_Status_3	= FALSE(0) & PrmCmd_required=FALSE: =>okay = FALSE(0) & PrmCmd_required=TRUE: => Prm_Fault	= TRUE(1) & (PrmCmd_supp = TRUE): => okay TRUE(1) & (PrmCmd_supp = FALSE): => Prm-Fault
Reserved-Bits (Bit 5 and Bit 6) in field DPV1_Status_3	= 0: => okay	1: => Prm_Fault
NOTE The table above gives a comprehensive overview on all checks that belong to the parameter. However, some of them may also checked on other places within the state machine.		

Isochronous behaviour

Devices that support isochronous operation shall set the concerning AR attributes according to the configuration. The block for isochronous parameters belong to the application and are part of user data within the Set_Prm-REQ-PDU or Set_Ext_Prm-REQ-PDU. The DP-master sets the DP-slave into isochronous operation by setting the IsoM_Req-Bit TRUE. When the MSCY1S receives the Set_Prm-REQ-PDU it has to check the following condition:

If(Isochronous Mode = TRUE) then Failsafe shall be set and any Group shall not be set concerning the Set_Prm-REQ-PDU otherwise the DP-slaves shall generate a Prm_Fault and shall leave the Data Exchange state. DP-slaves that do not support isochronous operation

may be integrated in an isochronous working system by selecting “Group_8” for only this DP-slaves and using Sync and Freeze commands.

If the condition above is fulfilled the DP-slave operates isochronous. Additionally to normal cyclic data exchange the MSCY1S generates a SYNCH Event.ind with Status “IsoM Start” with receiving the first Global_Control-REQ-PDU, “IsoM SYNCH” with subsequent Global_Control-REQ-PDU's, and “IsoM Stop” when receiving Global_Control-REQ-PDU with Control Command “Clear” or leaving the data exchange cycle. This service triggers the PLL of the application process. The right access with Get Input and Set Output from the application point of view is monitored with the help of the PLL.

Redundancy behaviour

Devices that support Slave redundancy shall support the PrmCmd. This allows a DP-master (Class 1) to use a DP-slave as Primary or Backup. Additionally the DP-master (Class 1) can stop and start MS1 after a swichover of the DP-master. MSCY1S provides the basic mechanism to support a redundancy structure in the Application.

Local variables

Act_Ref

Counter for Reference Numbers for Diagnosis Updates.

Act_Cnt

Storage for Users Reference number

Ref_Cnt

Storage of Act_Ref at users Diagnosis.

B Sync

(Octet-String)

Intermediate storage for outputs if the DP-slave is operated in Sync Mode.

B Output

(Octet-String)

Output data of the DP-slave that can be fetched by the Get Output service. The requirements for the consistency as described in the Cfg_Data shall be taken into consideration.

B Freeze

(Octet-String)

Intermediate storage for input data.

B Input

(Octet-String)

Depending on the type (see description of Cfg_Data), the data shall be consistently loaded by the Set Input service, or may be captured freely.

B Real_Cfg

(Octet-String)

The configuration which was previously defined for the device, or the configuration which is determined in the start-up by the device, respectively. The structure of the individual octets corresponds to the structure of Cfg_Data (->Chk_Cfg).

B User Prm

(Octet-String)

Intermediate storage for User Prm data.

B Ext User Prm

(Octet-String)

Intermediate storage for Ext User Prm data.

B Cfg

(Octet-String)

Intermediate storage for Cfg data.

Diag

(Octet-String)

Intermediate storage for Diag data.

Ext Diag Data

(Octet-String)

Intermediate storage for Ext Diag Data, the diagnosis setup by the user.

Ext Diag Flag

(Boolean)

Intermediate storage for Ext Diag Flag, which indicates important user diagnosis.

Clear Command**Sync Command****Freeze Command**

Intermediate storage for global control commands.

Station_Address

(Unsigned8)

Own station address which may be set by the service "Set_Slave_Add" or which is locally stored.

Check_Prm_Add

(Unsigned8)

Masters station address which has invoked the last service "Set_Prm".

Check_Ext_Prm_Add

(Unsigned8)

Masters station address which has invoked the last service "Set_Ext_Prm".

Check_Cfg_Add

(Unsigned8)

Masters station address which has invoked the last service "Chk_Cfg".

Output Data Len

(Unsigned8)

Indicates the number of output octets that are actually used.

Input Data Len

(Unsigned8)

Indicates the number of input octets that are actually used.

Real_No_Add_Chg

(Boolean)

Indicates if an address change is possible (FALSE) or not. Can be statically defined or stored.

Active_Groups

(Unsigned8)

Contains the Group_Ident as set via Set_Prm. This is used at Global_Control for the decision if the station is addressed.

Diag_Flag

(Boolean)

This flag indicates that the Master shall be informed by a high prior Reply-Update that a change in the diagnostic information occurred.

Sync_Supp

(Boolean)

Indicates that the DP-slave does support the Sync-Mode.

Freeze_Supp

(Boolean)

Indicates that the DP-slave does support the Freeze-Mode.

Operation_Mode

(Boolean)

Local flag that stores the Operation_Mode set with the Set_Prm service. Operation_Mode=V0 means that the Slave does not support acyclic services on MSAC1 and alarms.

Prm_Pending

(Unsigned8)

Local flag that indicates that the User is still processing the last call of the function "Check UserPrm".

Ext_Prm_Pending

(Unsigned8)

Local flag that indicates that the User is still processing the last call of the function "Check Ext_Prm".

Cfg_Pending

(Unsigned8)

Local flag that indicates that the User is still processing the last call of the function "Check Cfg".

Input_Pending

(Unsigned8)

Local flag that indicates that the User has not yet set the Input data

New Output

(Unsigned8)

Local flag that indicates that there are Output data available not yet fetched with "Get Output".

DX_Entered

(Boolean)

Local flag which is set after the MSCY1S State Machine has entered the DATA-EXCH mode and at least one Data_Exchange service has been completed.

Start_State

(Unsigned8)

Local variable which is used to store the state of the MSAC1S-State Machine.

Possible values for Start_State:

Idle	MSAC1S is closed
Run	MSAC1S is open
B	Start_Operation is initiated
E	Stop_Operation is initiated
BE	Start_Operation is initiated, Stop_Operation is queued
EB	Stop_Operation is initiated, Start_Operation is queued
BEB	Start_Operation is initiated, Stop_Operation (first) and Start_Operation(second) are queued

Safe_State

(Boolean)

Indicates whether the outputs has to be switched to the safe state.

Chk Cfg Mode

(Boolean)

Indicates how to check the Cfg Data.

First Synch

(Boolean)

This local variable is FALSE if the Isochronous mode has been activated and at least one Global_Control-REQ-PDU with Group Select (Group_8) has been received.

6.6.1.3 MSCY1S state table

Table 439 contains the complete description of the MSCY1S state machine.

Table 439 – MSCY1S state table

#	Current State	Event /Condition =>Action	Next State
1	POWER-ON	MSCY1S Sinit MS0.req () => Diag.Ext_Diag_Data := NIL Diag.Ext Diag Flag := FALSE B Real Cfg := Real Cfg Data Chk Cfg Mode := FALSE Sync Supp := Sync Supported Freeze Supp := Freeze Supported Diag.Ident Number := Ident Number Real No Add Chg := No Add Chg Act_Ref := 0 Act_Cnt := 0 Ref_Cnt := 0 DMPMS Slave Init.req ()	WAIT-INI-CON
2	POWER-ON	MSCY1S Abort.req () => ignore	POWER-ON
3	POWER-ON	MSCY1S Set Slave Diag.req (Ext Diag Flag, Ext Diag Overflow, Ext Diag Data, Reference) => Act_Cnt := Reference MSCY1S Set Slave Diag.cnf(-) (Reference := Act_Cnt)	POWER-ON
4	POWER-ON	MSCY1S Set Cfg.req (Cfg Data) => MSCY1S Set Cfg.cnf(-) ()	POWER-ON
5	POWER-ON	MSCY1S Get Output.req () => MSCY1S Get Output.cnf(-) ()	POWER-ON
6	POWER-ON	MSCY1S Set Input.req (Input Data) => MSCY1S Set Input.cnf(-) ()	POWER-ON
7	POWER-ON	MSCY1S Check Cfg Result.req (Cfg_OK, Input Data Len, Output Data Len) => Status := Reset MSCY1S Check Cfg Result.cnf(-) (Status)	POWER-ON
8	POWER-ON	MSCY1S Application Ready.req() => ignore	POWER-ON
9	POWER-ON	MSCY1S Check User Prm Result.req (Prm_OK) => Status := Reset MSCY1S Check User Prm Result.cnf(-) (Status)	POWER-ON
10	POWER-ON	MSCY1S Check Ext User Prm Result.req (Ext_Prm_OK) => Status := Reset MSCY1S Check Ext User Prm Result.cnf(-) (Status)	POWER-ON

Table 439 (continued)

#	Current State	Event /Condition =>Action	Next State
11	WAIT-INIT-CON	DMPMS Slave Init.cnf () => Operation Mode := V0 Start_State := Idle Safe_State:=TRUE Diag.Prm_Req := TRUE Diag.Cfg_Fault := FALSE Diag.Prm_Fault := FALSE Diag.Not_Supported := FALSE Diag.Master_Add := Invalid Diag.WD_On := FALSE Diag.Station_Not_Ready := TRUE Diag.Sync_Mode := FALSE Diag.Freeze_Mode := FALSE Diag.Stat_Diag := FALSE Diag.Ext_Diag_Overflow := FALSE B User Prm := NIL B Cfg := NIL Diag_Flag := FALSE Diag Data := Diag Act_Ref := Act_Ref + 1 Prm Structure = FALSE DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref) DMPMS Get Cfg Upd.req (Cfg Data) MSCY1S SInit MS0.cnf ()	WAIT-PRM
12	WAIT-PRM	MSCY1S Abort.req () => ignore	WAIT-PRM
13	WAIT-PRM	MSCY1S Set Slave Diag.req (Ext Diag Flag, Ext Diag Overflow, Ext Diag Data, Reference) => Diag.Ext Diag Data := Ext Diag Data Diag.Ext Diag Flag := Ext Diag Flag Diag.Ext Diag Overflow := Ext Diag Overflow Diag Data := Diag Act_Ref := Act_Ref + 1 Act_Cnt := Reference Ref_Cnt := Act_Ref DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref) MSCY1S Set Slave Diag.cnf(+) (Reference := Act_Cnt)	WAIT-PRM
14	WAIT-PRM	MSCY1S Set Cfg.req (Cfg Data) => DMPMS Get Cfg Upd.req (Cfg Data) MSCY1S Set Cfg.cnf(+) ()	WAIT-PRM
15	WAIT-PRM	MSCY1S Get Output.req () => MSCY1S Get Output.cnf(-) ()	WAIT-PRM
16	WAIT-PRM	MSCY1S Set Input.req (Input Data) => MSCY1S Set Input.cnf(-) ()	WAIT-PRM
17	WAIT-PRM	MSCY1S Check User Prm Result.req (Prm_OK) /Prm_Pending = 0 => Status := Not pending MSCY1S Check User Prm Result.cnf(-) (Status)	WAIT-PRM
18	WAIT-PRM	MSCY1S Check User Prm Result.req (Prm_OK) /Prm_Pending = 2 => Status := New Prm Prm_Pending := 1 User Parameter Data := B User Prm MSCY1S Check User Prm Result.cnf(-) (Status) MSCY1S Check User Prm.ind (Prm Structure, User Parameter Data)	WAIT-PRM
19	WAIT-PRM	MSCY1S Check User Prm Result.req (Prm_OK) /Prm_Pending = 1 => Prm_Pending := 0 MSCY1S Check User Prm Result.cnf(+)()	WAIT-PRM

Table 439 (continued)

#	Current State	Event /Condition =>Action	Next State
20	WAIT-PRM	MSCY1S Check Cfg Result.req (Cfg_OK, Input Data Len, Output Data Len) /Cfg_Pending = 0 => Status := Not pending MSCY1S Check Cfg Result.cnf(-) (Status)	WAIT-PRM
21	WAIT-PRM	MSCY1S Check Cfg Result.req (Cfg_OK, Input Data Len, Output Data Len) /Cfg_Pending = 2 => Status := New Cfg Cfg_Pending := 1 Cfg Data := B Cfg MSCY1S Check Cfg Result.cnf(-) (Status) MSCY1S Check Cfg.ind (Cfg Data)	WAIT-PRM
22	WAIT-PRM	MSCY1S Check Cfg Result.req (Cfg_OK, Input Data Len, Output Data Len) /Cfg_Pending = 1 => Cfg_Pending := 0 MSCY1S Check Cfg Result.cnf(+) ()	WAIT-PRM
23	WAIT-PRM	MSCY1S Application Ready.req() => ignore	WAIT-PRM
24	WAIT-PRM	MSAC1S Abort.ind() => ignore	WAIT-PRM
25	WAIT-PRM	MSAC1S Start.cnf() => START_C1_CON	WAIT-PRM
26	WAIT-PRM	MSAC1S Stop.cnf() => Start_State := Idle	WAIT-PRM
27	WAIT-PRM	DMPMS Set Slave Add.ind(New Slave Add, Ident Number, No Add Chg, Rem Slave Data) /Real_No_Add_Chg = FALSE && Diag.Ident_Number = Ident_Number && New_Slave_Address <= 125 => MSCY1S Set Slave Add.ind (New Slave Add, Ident Number, No Add Chg, Rem Slave Data)	POWER-ON
28	WAIT-PRM	DMPMS Set Slave Add.ind(New Slave Add, Ident Number, No Add Chg, Rem Slave Data) /Real_No_Add_Chg = TRUE Diag.Ident_Number <> Ident_Number New_Slave_Add > 125 => ignore	WAIT-PRM
29	WAIT-PRM	DMPMS Slave Diag.ind(Req Add, Reference) => ignore	WAIT-PRM
30	WAIT-PRM	DMPMS Set Prm.ind(Req Add, Prm Data) /Prm_Data.len >= 7 && Lock_Req = FALSE && Unlock_Req = FALSE => DMPMS Set minTsdr.req (MinTsdr)	WAIT-PRM
31	WAIT-PRM	DMPMS Set Prm.ind(Req Add, Prm Data) /Prm_Data.len >= 7 && Unlock_Req = TRUE => ignore	WAIT-PRM
32	WAIT-PRM	DMPMS Set Prm.ind(Req Add, Prm Data) /Prm_Data.len >= 7 && Lock_Req = TRUE && Unlock_Req = FALSE && (Ident_Number <> Diag.Ident_Number !OPERATION_MODE_OK WD_On = TRUE && (WD_Fact_1=0 WD_Fact_2=0)) => Diag.Prm_Fault := TRUE Diag Data := Diag Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	WAIT-PRM

Table 439 (continued)

#	Current State	Event /Condition =>Action	Next State
33	WAIT-PRM	DMPMS Set Prm.ind(Req Add, Prm Data) /Prm_Data.len >= 7 && Lock_Req = TRUE && Unlock_Req = FALSE && Ident_Number = Diag.Ident_Number && OPERATION_MODE_OK && (WD_On = FALSE (WD_Fact_1 > 0 && WD_Fact_2 > 0)) && (Freeze_Req = TRUE && Freeze_Supp = FALSE Sync_Req = TRUE && Sync_Supp = FALSE Prm_Data[1].0, .1, .2 = TRUE) => Diag.Not_Supported := TRUE Diag_Data := Diag Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag_Data, Reference := Act_Ref)	WAIT-PRM
34	WAIT-PRM	DMPMS Set Prm.ind(Req Add, Prm Data) /Prm_Data.len < 7 => ignore	WAIT-PRM
35	WAIT-PRM	DMPMS Set Prm.ind(Req Add, Prm Data) /PRM_OK && Prm_Pending = 0 => Diag.Master_Add := Req_Add Check_Prm_Add := Req_Add FirstSynch := TRUE SET_OPERATION_MODE SET_WD Active_Groups := Group_Ident Diag.Prm_Fault := FALSE Diag.Prm_Req := FALSE Diag.Not_Supported := FALSE Prm_Pending := 1 User_Parameter_Data := Prm_Data.User_Prm SET_ALARM_CHKCFGM Input_Pending := FALSE Diag_Data := Diag Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag_Data, Reference := Act_Ref) DMPMS Set minTsdr.req (minTsdr) MSCY1S Check User Prm.ind (Prm_Structure, User_Parameter_Data)	WAIT-CFG
36	WAIT-PRM	DMPMS Set Prm.ind(Req Add, Prm Data) /PRM_OK && Prm_Pending > 0 => Diag.Master_Add := Req_Add Check_Prm_Add := Req_Add FirstSynch := TRUE SET_OPERATION_MODE SET_WD Active_Groups := Group_Ident Diag.Prm_Fault := FALSE Diag.Prm_Req := FALSE Diag.Not_Supported := FALSE Prm_Pending := 2 B User_Prm := Prm_Data.User_Prm SET_ALARM_CHKCFGM Input_Pending := FALSE Diag_Data := Diag Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag_Data, Reference := Act_Ref) DMPMS Set minTsdr.req (minTsdr)	WAIT-CFG
37	WAIT-PRM	DMPMS Chk_Cfg.ind(Req Add, Cfg Data) => ignore	WAIT-PRM
38	WAIT-PRM	DMPMS Set Ext_Prm.ind(Req_Add, Ext_Prm_Data) /Diag.Master_Add = Req_Add => ignore	WAIT-PRM
39	WAIT-PRM	MSCY1S Check Ext User Prm Result.req (Ext_Prm_OK) /Ext_Prm_Pending = 0 => Status := Not pending MSCY1S Check Ext User Prm Result.cnf(-) (Status)	WAIT-PRM

Table 439 (continued)

#	Current State	Event /Condition =>Action	Next State
40	WAIT-PRM	MSCY1S Check Ext User Prm Result.req (Ext_Prm_OK) /Ext_Prm_Pending = 2 => Status := New Prm Prm_Pending := 1 Ext User Parameter Data := B Ext User Prm MSCY1S Check Ext User Prm Result.cnf(-) (Status) MSCY1S Check Ext User Prm.ind (Ext User Parameter Data)	WAIT-PRM
41	WAIT-PRM	MSCY1S Check Ext User Prm Result.req (Ext_Prm_OK) /Ext_Prm_Pending = 1 => Ext_Prm_Pending := 0 MSCY1S Check Ext User Prm Result.cnf(+)()	WAIT-PRM
42	WAIT-CFG	MSCY1S Abort.req () => Operation_Mode := V0 Diag.Master_Add := invalid Diag.Prm_Req := TRUE Diag.Station_Not_Ready := TRUE StopTimer(WD) Diag.WD_On := FALSE STOP_C1 Diag Data := Diag Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	WAIT-PRM
43	WAIT-CFG	MSCY1S Set Slave Diag.req (Ext Diag Flag, Ext Diag Overflow, Ext Diag Data, Reference) => Diag.Ext Diag Data := Ext Diag Data Diag.Ext Diag Flag := Ext Diag Flag Diag.Ext Diag Overflow := Ext Diag Overflow Diag Data := Diag Act_Ref := Act_Ref + 1 Act_Cnt := Reference Ref_Cnt := Act_Ref DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref) MSCY1S Set Slave Diag.cnf(+)(Reference := Act_Cnt)	WAIT-CFG
44	WAIT-CFG	MSCY1S Set Cfg.req (Cfg Data) => DMPMS Get Cfg Upd.req (Cfg Data) MSCY1S Set Cfg.cnf(+)()	WAIT-CFG
45	WAIT-CFG	MSCY1S Get Output.req () => MSCY1S Get Output.cnf(-)()	WAIT-CFG
46	WAIT-CFG	MSCY1S Set Input.req (Input Data) /Input_Pending = FALSE => MSCY1S Set Input.cnf(-)()	WAIT-CFG
47	WAIT-CFG	MSCY1S Set Input.req (Input Data) /Input_Pending = TRUE && Operation_Mode = V0 => Inp Data := Input Data Diag.Cfg_Fault := FALSE Diag_Flag := TRUE Diag.Stat_Diag := TRUE Diag.Station_Not_Ready := FALSE DX_Entered := FALSE Diag Data := Diag Act_Ref := Act_Ref + 1 MSCY1S Set Input.cnf(+)() DMPMS Enter.req (Master Add, Input Data Len, Output Data Len) DMPMS RD Inp Upd.req (Inp Data) DMPMS Data Exchange Upd.req (Diag Flag, Inp Data) DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	DATA-EXCH

Table 439 (continued)

#	Current State	Event /Condition =>Action	Next State
48	WAIT-CFG	MSCY1S Set Input.req (Input Data) /Input_Pending = TRUE && Operation_Mode = V1 => B Input := Input Data START_C1 MSCY1S Set Input.cnf(+) ()	WAIT-CFG
49	WAIT-CFG	MSCY1S Check User Prm Result.req (Prm_OK) /Prm_Pending = 0 => Status := Not pending MSCY1S Check User Prm Result.cnf(-) (Status)	WAIT-CFG
50	WAIT-CFG	MSCY1S Check User Prm Result.req (Prm_OK) /Prm_Pending = 2 => Status := New Prm Prm_Pending := 1 User Parameter Data := B User Prm MSCY1S Check User Prm Result.cnf(-) (Status) MSCY1S Check User Prm.ind (Prm Structure, User Parameter Data)	WAIT-CFG
51	WAIT-CFG	MSCY1S Check User Prm Result.req (Prm_OK) /Prm_Pending = 1 && Diag.Master_Add <> Check_Prm_Add => Status := Inv Master Add Prm_Pending := 0 MSCY1S Check User Prm Result.cnf(-) (Status)	WAIT-CFG
52	WAIT-CFG	MSCY1S Check User Prm Result.req (Prm_OK) /Prm_Pending = 1 && Diag.Master_Add = Check_Prm_Add && Prm_OK = TRUE => Prm_Pending := 0 MSCY1S Check User Prm Result.cnf(+)()	WAIT-CFG
53	WAIT-CFG	MSCY1S Check User Prm Result.req (Prm_OK) /Prm_Pending = 1 && Diag.Master_Add = Check_Prm_Add && Prm_OK = FALSE => Prm_Pending := 0 Diag.Prm_Fault := TRUE Diag.Master_Add := Invalid Diag.Prm_Req := TRUE StopTimer(WD) Diag.WD_On := FALSE STOP_C1 Diag Data := Diag Act_Ref := Act_Ref + 1 MSCY1S Check User Prm Result.cnf(+)() DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	WAIT-PRM
54	WAIT-CFG	MSCY1S Check Cfg Result.req (Cfg_OK, Input Data Len, Output Data Len) /Cfg_Pending = 0 => Status := Not pending MSCY1S Check Cfg Result.cnf(-) (Status)	WAIT-CFG
55	WAIT-CFG	MSCY1S Check Cfg Result.req (Cfg_OK, Input Data Len, Output Data Len) /Cfg_Pending = 2 => Status := New Cfg Cfg_Pending := 1 Cfg Data := B Cfg MSCY1S Check Cfg Result.cnf(-) (Status) MSCY1S Check Cfg.ind (Cfg Data)	WAIT-CFG
56	WAIT-CFG	MSCY1S Check Cfg Result.req (Cfg_OK, Input Data Len, Output Data Len) /Cfg_Pending = 1 && Diag.Master_Add <> Check_Cfg_Add => Status := Inv Master Add Cfg_Pending := 0 MSCY1S Check Cfg Result.cnf(-) (Status)	WAIT-CFG

Table 439 (continued)

#	Current State	Event /Condition =>Action	Next State
57	WAIT-CFG	MSCY1S Check Cfg Result.req (Cfg_OK, Input Data Len, Output Data Len) /Cfg_Pending = 1 && Diag.Master_Add = Check_Cfg_Add && Cfg_OK = TRUE && Input Data Len > 0 => Inp Data Len := Input Data Len Outp Data Len := Output Data Len Cfg_Pending := 0 Input_Pending := TRUE MSCY1S Check Cfg Result.cnf(+) ()	WAIT-CFG
58	WAIT-CFG	MSCY1S Check Cfg Result.req (Cfg_OK, Input Data Len, Output Data Len) /Cfg_Pending = 1 && Diag.Master_Add = Check_Cfg_Add && Cfg_OK = TRUE && Operation_Mode = V1 && Input Data Len = 0 => Inp Data Len := 0 Outp Data Len := Output Data Len Inp Data := NIL Cfg_Pending := 0 START_C1 MSCY1S Check Cfg Result.cnf(+) ()	WAIT-CFG
59	WAIT-CFG	MSCY1S Check Cfg Result.req (Cfg_OK, Input Data Len, Output Data Len) /Cfg_Pending = 1 && Diag.Master_Add = Check_Cfg_Add && Cfg_OK = TRUE && Operation_Mode = V0 && Input Data Len = 0 => Inp Data Len := 0 Outp Data Len := Output Data Len Inp Data := NIL Cfg_Pending := 0 Diag.Cfg_Fault := FALSE Diag_Flag := TRUE Diag.Stat_Diag := TRUE Diag.Station_Not_Ready := FALSE DX_Entered := FALSE Diag Data := Diag Act_Ref := Act_Ref + 1 DMPMS Enter.req DMPMS Data Exchange Upd.req (Diag Flag, Inp Data) DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref) MSCY1S Check Cfg Result.cnf(+) ()	DATA-EXCH
60	WAIT-CFG	MSCY1S Check Cfg Result.req (Cfg_OK, Input Data Len, Output Data Len) /Cfg_Pending = 1 && Diag.Master_Add = Check_Cfg_Add && Cfg_OK = FALSE => Cfg_Pending := 0 Diag.Cfg_Fault := TRUE Diag.Prm_Req := TRUE Diag.Master_Add := Invalid StopTimer(WD) Diag.WD_On := FALSE STOP_C1 Diag Data := Diag Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref) MSCY1S Check Cfg Result.cnf(+) ()	WAIT-PRM
61	WAIT-CFG	MSCY1S Application Ready.req() => ignore	WAIT-CFG
62	WAIT-CFG	MSAC1S Abort.ind() => ignore	WAIT-CFG
63	WAIT-CFG	MSAC1S Start.cnf() /Start_State <> B => START_C1_CON	WAIT-CFG

Table 439 (continued)

#	Current State	Event /Condition =>Action	Next State
64	WAIT-CFG	MSAC1S Start.cnf() /Start_State = B => Diag.Cfg_Fault := FALSE Diag_Flag := TRUE Diag.Stat_Diag := TRUE Diag.Station_Not_Ready := FALSE DX_Entered := FALSE START_C1_CON Diag Data := Diag Act_Ref := Act_Ref + 1 DMPMS Enter.req DMPMS RD Inp Upd.req (Inp Data) DMPMS Data Exchange Upd.req (Diag Flag, Inp Data) DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	DATA-EXCH
65	WAIT-CFG	MSAC1S Stop.cnf() => STOP_C1_CON	WAIT-CFG
66	WAIT-CFG	MSAC1S Abort.ind() => Operation_Mode := V0 Diag.Master_Add := invalid Diag.Prm_Req := TRUE StopTimer(WD) Diag.WD_On := FALSE STOP_C1 Diag Data := Diag Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	WAIT-PRM
67	WAIT-CFG	DMPMS Set Slave Add.ind(New Slave Add, Ident Number, No Add Chg, Rem Slave Data) => ignore	WAIT-CFG
68	WAIT-CFG	DMPMS Slave Diag.ind(Req Add, Reference) /Diag.Master_Add = Req_Add => TRIG_WD	WAIT-CFG
69	WAIT-CFG	DMPMS Slave Diag.ind(Req Add, Reference) /Diag.Master_Add <> Req_Add => ignore	WAIT-CFG
70	WAIT-CFG	DMPMS Set Prm.ind(Req Add, Prm Data) /Prm_Data.len >= 7 && Unlock_Req = TRUE && Diag.Master_Add = Req_Add => Diag.Master_Add := invalid Diag.Prm_Req := TRUE StopTimer(WD) Diag.WD_On := FALSE STOP_C1 Diag Data := Diag Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	WAIT-PRM
71	WAIT-CFG	DMPMS Set Prm.ind(Req Add, Prm Data) /Prm_Data.len >= 7 && Unlock_Req = TRUE && Diag.Master_Add <> Req_Add => ignore	WAIT-CFG

Table 439 (continued)

#	Current State	Event /Condition =>Action	Next State
72	WAIT-CFG	DMPMS Set Prm.ind(Req Add, Prm Data) /Prm_Data.len >= 7 && Unlock_Req = FALSE && Lock_Req = TRUE && Diag.Master_Add = Req_Add && (Ident_Number <> Diag.Ident_Number OPERATION_MODE_OK = FALSE WD_On = TRUE && (WD_Fact_1=0 WD_Fact_2=0)) => Diag.Prm_Fault := TRUE Diag.Master_Add := Invalid Diag.Prm_Req := TRUE StopTimer(WD) Diag.WD_On := FALSE STOP_C1 Diag Data := Diag Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	WAIT-PRM
73	WAIT-CFG	DMPMS Set Prm.ind(Req Add, Prm Data) /Prm_Data.len >= 7 && Unlock_Req = FALSE && Lock_Req = TRUE && Diag.Master_Add = Req_Add && Ident_Number = Diag.Ident_Number && OPERATION_MODE_OK && (WD_On = FALSE WD_Fact_1>0 && WD_Fact_2>0) && (Freeze_Req = TRUE && Freeze Supp = FALSE Sync_Req = TRUE && Sync Supp = FALSE Prm_Data[1].0, .1, .2 = TRUE) => Diag.Not_Supported := TRUE Diag.Master_Add := Invalid Diag.Prm_Req := TRUE StopTimer(WD) Diag.WD_On := FALSE STOP_C1 Diag Data := Diag Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	WAIT-PRM
74	WAIT-CFG	DMPMS Set Prm.ind(Req Add, Prm Data) /Prm_Data.len >= 7 && Unlock_Req = FALSE && Lock_Req = TRUE && Diag.Master_Add <> Req_Add && (Ident_Number <> Diag.Ident_Number OPERATION_MODE_OK = FALSE WD_On = TRUE && (WD_Fact_1=0 WD_Fact_2=0) Freeze_Req = TRUE && Freeze Supp = FALSE Sync_Req = TRUE && Sync Supp = FALSE Prm_Data[1].0, .1, .2 = TRUE) => ignore	WAIT-CFG
75	WAIT-CFG	DMPMS Set Prm.ind(Req Add, Prm Data) /PRM_OK && Prm_Pending = 0 => Diag.Master_Add := Req_Add Check_Prm_Add := Req_Add SET_OPERATION_MODE SET_WD Active_Groups := Group_Ident Prm_Pending := 1 User Parameter Data := Prm Data.User Prm SET_ALARM_CHKCFGM Input_Pending := FALSE STOP_C1 Diag Data := Diag Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref) DMPMS Set minTsdreq (minTsdreq) MSCY1S Check User Prm.ind (Prm Structure, User Parameter Data)	WAIT-CFG

Table 439 (continued)

#	Current State	Event /Condition =>Action	Next State
76	WAIT-CFG	DMPMS Set Prm.ind(Req Add, Prm Data) /PRM_OK && Prm_Pending > 0 => Diag.Master_Add := Req_Add Check_Prm_Add := Req Add SET_OPERATION_MODE SET_WD Active_Groups := Group_Ident Prm_Pending := 2 B User Prm := Prm Data.User Prm SET_ALARM_CHKCFG Input_Pending := FALSE STOP_C1 Diag Data := Diag Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref) DMPMS Set minTsdr.req (minTsdr)	WAIT-CFG
77	WAIT-CFG	DMPMS Set Prm.ind(Req Add, Prm Data) /Prm_Data.len >= 7 && Unlock_Req = FALSE && Lock_Req = FALSE => DMPMS Set minTsdr.req (MinTsdr)	WAIT-CFG
78	WAIT-CFG	DMPMS Set Prm.ind(Req Add, Prm Data) /Prm_Data.len < 7 => ignore	WAIT-CFG
79	WAIT-CFG	DMPMS Chk Cfg.ind(Req Add, Cfg Data) /Diag.Master_Add <> Req_Add => ignore	WAIT-CFG
80	WAIT-CFG	DMPMS Chk Cfg.ind(Req Add, Cfg Data) /Diag.Master_Add = Req_Add && Cfg_Pending = 0 => Cfg_Pending := 1 Check_Cfg_Add := Req Add TRIG_WD MSCY1S Check Cfg.ind (Check Cfg Mode, Cfg Data)	WAIT-CFG
81	WAIT-CFG	DMPMS Chk Cfg.ind(Req Add, Cfg Data) /Diag.Master_Add = Req_Add && Cfg_Pending > 0 => Cfg_Pending := 2 Check_Cfg_Add := Req Add B Cfg := Cfg Data TRIG_WD	WAIT-CFG
82	WAIT-CFG	WDTimer expired => Diag.Master_Add := invalid Diag.Prm_Req := TRUE StopTimer(WD) Diag.WD_On := FALSE STOP_C1 Diag Data := Diag Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	WAIT-PRM
83	WAIT-CFG	DMPMS Set Ext Prm.ind(Req_Add, Ext Prm Data)/Diag.Master_Add = Req_Add &&Ext_Prm_Pending = 0=>TRIG_WDExt User Parameter Data := Ext User Prm DataExt_Prm_Pending := 1Check_Ext_Prm_Add := Req_AddMSCY1S Check Ext User Prm.ind (Ext User Parameter Data)	WAIT-CFG
84	WAIT-CFG	DMPMS Set Ext Prm.ind(Req_Add, Ext Prm Data) /Diag.Master_Add = Req_Add && Ext_Prm_Pending > 0 => TRIG_WD B Ext User Prm := Ext User Prm Data Ext_Prm_Pending := 2 Check_Ext_Prm_Add := Req_Add	WAIT-CFG

Table 439 (continued)

#	Current State	Event /Condition =>Action	Next State
85	WAIT-CFG	MSCY1S Check Ext User Prm Result.req (Ext_Prm_OK) /Ext_Prm_Pending = 0 => Status := Not pending MSCY1S Check Ext User Prm Result.cnf(-) (Status)	WAIT-CFG
86	WAIT-CFG	MSCY1S Check Ext User Prm Result.req (Ext_Prm_OK) /Ext_Prm_Pending = 2 => Status := New Prm Ext_Prm_Pending := 1 Ext User Parameter Data := B Ext User Prm MSCY1S Check Ext User Prm Result.cnf(-) (Status) MSCY1S Check Ext User Prm.ind (Ext User Parameter Data)	WAIT-CFG
87	WAIT-CFG	MSCY1S Check Ext User Prm Result.req (Ext_Prm_OK) /Ext_Prm_Pending = 1 && Diag.Master_Add <> Check_Ext_Prm_Add => Status := Inv Master Add Ext_Prm_Pending := 0 MSCY1S Check Ext User Prm Result.cnf(-) (Status)	WAIT-CFG
88	WAIT-CFG	MSCY1S Check Ext User Prm Result.req (Ext_Prm_OK) /Ext_Prm_Pending = 1 && Diag.Master_Add = Check_Ext_Prm_Add && Ext_Prm_OK = TRUE => Ext_Prm_Pending := 0 MSCY1S Check Ext User Prm Result.cnf(+)()	WAIT-CFG
89	WAIT-CFG	MSCY1S Check Ext User Prm Result.req (Ext_Prm_OK) /Ext_Prm_Pending = 1 && Diag.Master_Add = Check_Prm_Add && Ext_Prm_OK = FALSE => Ext_Prm_Pending := 0 Diag.Cfg_Fault := TRUE Diag.Master_Add := Invalid Diag.Prm_Req := TRUE StopTimer(WD) Diag.WD_On := FALSE STOP_C1 Diag Data := Diag Act_Ref := Act_Ref + 1 MSCY1S Check Ext User Prm Result.cnf(+)() DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	WAIT-PRM
90	DATA-EXCH	MSCY1S Abort.req () => LEAVE-MASTER Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	WAIT-PRM
91	DATA-EXCH	MSCY1S Set Slave Diag.req (Ext Diag Flag, Ext Diag Overflow, Ext Diag Data, Reference) => Diag.Ext Diag Data := Ext Diag Data Diag.Ext Diag Flag := Ext Diag Flag Diag.Ext Diag Overflow := Ext Diag Overflow Diag Data := Diag Diag Flag := TRUE Act_Ref := Act_Ref + 1 Act_Cnt := Reference Ref_Cnt := Act_Ref DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref) DMPMS Data Exchange Upd.req(Diag Flag, Inp Data)	DATA-EXCH
92	DATA-EXCH	MSCY1S Set Cfg.req (Cfg Data) => Diag.Cfg_Fault:=TRUE LEAVE-MASTER Act_Ref := Act_Ref + 1 DMPMS Get Cfg Upd.req (Cfg Data) MSCY1S Set Cfg.cnf(+)() DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	WAIT-PRM

Table 439 (continued)

#	Current State	Event /Condition =>Action	Next State
93	DATA-EXCH	MSCY1S Get Output.req () /DX_Entered = TRUE => Clear Flag := Safe_State New Flag := New Output New Output := FALSE Outp Data, Output Data := B Output DMPMS RD Outp Upd.req (Outp Data) MSCY1S Get Output.cnf(+) (Output Data, Clear Flag, New Flag)	DATA-EXCH
94	DATA-EXCH	MSCY1S Get Output.req () /DX_Entered = FALSE => MSCY1S Get Output.cnf(-) ()	DATA-EXCH
95	DATA-EXCH	MSCY1S Set Input.req (Input Data) /Diag.Freeze_Mode = FALSE => Inp Data, B Input := Input Data DMPMS Data Exchange Upd.req (Diag Flag, Inp Data) DMPMS RD Inp Upd.req (Inp Data) MSCY1S Set Input.cnf(+) ()	DATA-EXCH
96	DATA-EXCH	MSCY1S Set Input.req (Input Data) /Diag.Freeze_Mode = TRUE => B Freeze := Input Data MSCY1S Set Input.cnf(+) ()	DATA-EXCH
97	DATA-EXCH	MSCY1S Check User Prm Result.req (Prm_OK) /Prm_Pending = 0 => Status := Not pending MSCY1S Check User Prm Result.cnf(-) (Status)	DATA-EXCH
98	DATA-EXCH	MSCY1S Check User Prm Result.req (Prm_OK) /Prm_Pending = 2 => Status := New Prm Prm_Pending := 1 User Parameter Data := B User Prm MSCY1S Check User Prm Result.cnf(-) (Status) MSCY1S Check User Prm.ind (Prm Structure, User Parameter Data)	DATA-EXCH
99	DATA-EXCH	MSCY1S Check User Prm Result.req (Prm_OK) /Prm_Pending = 1 && Diag.Master_Add # Check_Prm_Add => Status := Inv Master Add Prm_Pending := 0 MSCY1S Check User Prm Result.cnf(-) (Status)	DATA-EXCH
100	DATA-EXCH	MSCY1S Check User Prm Result.req (Prm_OK) /Prm_Pending = 1 && Diag.Master_Add = Check_Prm_Add && Prm_OK = TRUE => Prm_Pending := 0 MSCY1S Check User Prm Result.cnf(+)()	DATA-EXCH
101	DATA-EXCH	MSCY1S Check User Prm Result.req (Prm_OK) /Prm_Pending = 1 && Diag.Master_Add = Check_Prm_Add && Prm_OK = FALSE => Diag.Prm_Fault := TRUE LEAVE-MASTER Prm_Pending := 0 Act_Ref := Act_Ref + 1 MSCY1S Check User Prm Result.cnf(+)() DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	WAIT-PRM
102	DATA-EXCH	MSCY1S Check Cfg Result.req (Cfg_OK, Input Data Len, Output Data Len) /Cfg_Pending = 0 => Status := Not pending MSCY1S Check Cfg Result.cnf(-) (Status)	DATA-EXCH

Table 439 (continued)

#	Current State	Event /Condition =>Action	Next State
103	DATA-EXCH	MSCY1S Check Cfg Result.req (Cfg_OK, Input Data Len, Output Data Len) /Cfg_Pending = 2 => Status := New Cfg Cfg_Pending := 1 Cfg Data := B Cfg MSCY1S Check Cfg Result.cnf(-) (Status) MSCY1S Check Cfg.ind (Cfg Data)	DATA-EXCH
104	DATA-EXCH	MSCY1S Check Cfg Result.req (Cfg_OK, Input Data Len, Output Data Len) /Cfg_Pending = 1 && Diag.Master_Add # Check_Cfg_Add => Status := Inv Master Add Cfg_Pending := 0 MSCY1S Check Cfg Result.cnf(-) (Status)	DATA-EXCH
105	DATA-EXCH	MSCY1S Check Cfg Result.req (Cfg_OK, Input Data Len, Output Data Len) /Cfg_Pending = 1 && Diag.Master_Add = Check_Cfg_Add && Cfg_OK = TRUE && Input Data Len = Inp Data Len && Output Data Len = Outp Data Len => Cfg_Pending := 0 MSCY1S Check Cfg Result.cnf(+) ()	DATA-EXCH
106	DATA-EXCH	MSCY1S Check Cfg Result.req (Cfg_OK, Input Data Len, Output Data Len) /Cfg_Pending = 1 && Diag.Master_Add = Check_Cfg_Add && Cfg_OK = TRUE && Input Data Len <> Inp Data Len && Output Data Len <> Outp Data Len => Diag.Cfg_Fault := TRUE LEAVE-MASTER Act_Ref := Act_Ref + 1 MSCY1S Check Cfg Result.cnf(+) () DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	WAIT-PRM
107	DATA-EXCH	MSCY1S Check Cfg Result.req (Cfg_OK, Input Data Len, Output Data Len) /Cfg_Pending = 1 && Diag.Master_Add = Check_Cfg_Add && Cfg_OK = FALSE => Diag.Cfg_Fault := TRUE LEAVE-MASTER Act_Ref := Act_Ref + 1 MSCY1S Check Cfg Result.cnf(+) () DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	WAIT-PRM
108	DATA-EXCH	MSCY1S Application Ready.req() /Diag.Stat_Diag = TRUE => Diag.Stat_Diag := FALSE Diag_Data := Diag Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref) DMPMS Data Exchange Upd.req(Diag Flag, Inp Data)	DATA-EXCH
109	DATA-EXCH	MSCY1S Application Ready.req() /Diag.Stat_Diag = FALSE => ignore	DATA-EXCH
110	DATA-EXCH	MSAC1S Abort.ind() => LEAVE-MASTER Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	WAIT-PRM
111	DATA-EXCH	MSAC1S Start.cnf() /Start_State <> B => START_C1_CON	DATA-EXCH
112	DATA-EXCH	MSAC1S Start.cnf() /Start_State = B => START_C1_CON MSCY1S_Start.ind (Actual_Enabled_Alarms, Alarm_Sequence, Alarm_Limit)	DATA-EXCH

Table 439 (continued)

#	Current State	Event /Condition =>Action	Next State
113	DATA-EXCH	MSAC1S Stop.cnf() /Start_State <> E => STOP_C1_CON	DATA-EXCH
114	DATA-EXCH	MSAC1S Stop.cnf() /Start_State = E => STOP_C1_CON MSCY1S_Stop.ind	DATA-EXCH
115	DATA-EXCH	DMPMS Set Slave Add.ind(New Slave Add, Ident Number, No Add Chg, Rem Slave Data) => ignore	DATA-EXCH
116	DATA-EXCH	DMPMS Slave Diag.ind(Req Add, Reference) /Diag.Master_Add = Req_Add && (Diag.Stat_Diag = TRUE Reference < Ref_Cnt) => TRIG_WD	DATA-EXCH
117	DATA-EXCH	DMPMS Slave Diag.ind(Req Add, Reference) /Diag.Master_Add = Req_Add && Diag.Stat_Diag = FALSE && Reference >= Ref_Cnt => Diag_Flag := FALSE TRIG_WD DMPMS Data Exchange Upd.req (Diag_Flag, Inp Data) DMPMS RD Inp Upd.req (Inp Data) MSCY1S Set Slave Diag.cnf(+) (Reference := Act_Cnt)	DATA-EXCH
118	DATA-EXCH	DMPMS Slave Diag.ind(Req Add, Reference) /Diag.Master_Add <> Req_Add => ignore	DATA-EXCH
119	DATA-EXCH	DMPMS Set Prm.ind(Req Add, Prm Data) /Prm_Data.len >= 7 && Lock_Req = FALSE && Unlock_Req = FALSE => DMPMS Set minTsdr.req (MinTsdr)	DATA-EXCH
120	DATA-EXCH	DMPMS Set Prm.ind(Req Add, Prm Data) /Prm_Data.len >= 7 && Unlock_Req = TRUE && Diag.Master_Add <> Req_Add => ignore	DATA-EXCH
121	DATA-EXCH	DMPMS Set Prm.ind(Req Add, Prm Data) /Prm_Data.len >= 7 && Unlock_Req = TRUE && Diag.Master_Add = Req_Add => LEAVE-MASTER Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	WAIT-PRM
122	DATA-EXCH	DMPMS Set Prm.ind(Req Add, Prm Data) /Prm_Data.len >= 7 && Lock_Req = TRUE && Unlock_Req = FALSE && Diag.Master_Add = Req_Add && (Ident_Number <> Diag.Ident_Number !OPERATION_MODE_OK WD_On && (WD_Fact_1=0 WD_Fact_2=0)) => Diag.Prm_Fault := TRUE LEAVE-MASTER Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	WAIT-PRM
123	DATA-EXCH	DMPMS Set Prm.ind(Req Add, Prm Data) /Prm_Data.len >= 7 && Lock_Req = TRUE && Unlock_Req = FALSE && Diag.Master_Add = Req_Add && Ident_Number = Diag.Ident_Number && OPERATION_MODE_OK && !WD_On (WD_Fact_1>0 && WD_Fact_2>0) && (Freeze_Req && Freeze_Supp = FALSE Sync_Req && Sync_Supp = FALSE Prm_Data[1].0, .1, .2 = TRUE) => Diag.Not_Supported := TRUE LEAVE-MASTER Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	WAIT-PRM

Table 439 (continued)

#	Current State	Event /Condition =>Action	Next State
124	DATA-EXCH	DMPMS Set Prm.ind(Req Add, Prm Data) /Prm_Data.len >= 7 && Lock_Req = TRUE && Unlock_Req = FALSE && Diag.Master_Add <> Req_Add && (Ident_Number <> Diag.Ident_Number !OPERATION_MODE_OK WD_On && (WD_Fact_1=0 WD_Fact_2=0) Freeze_Req && !Freeze Supp Sync_Req && !Sync Supp Prm_Data[1].0, .1, .2 = TRUE) => ignore	DATA-EXCH
125	DATA-EXCH	DMPMS Set Prm.ind(Req Add, Prm Data)/Diag.Master_Add = Req_Add && PRM_OK &&!PV0V0 && NOPRMCMD =>LEAVE-MASTERAct_Ref := Act_Ref + 1DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	WAIT-PRM
126	DATA-EXCH	DMPMS Set Prm.ind(Req Add, Prm Data) /Diag.Master_Add = Req_Add && PRM_OK && !NOPRMCMD && Prm_Pending = 0 => SET_WD Active_Groups := Group_Ident EXE_PRM_CMD Prm_Pending := 1 Check_Prm_Add := Req Add User Parameter Data := Prm Data.User Prm Diag_Data := Diag Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref) DMPMS Set minTsdr.req (minTsdr) MSCY1S Check User Prm.ind (Prm Structure, User Parameter Data)	DATA-EXCH
127	DATA-EXCH	DMPMS Set Prm.ind(Req Add, Prm Data) /Diag.Master_Add = Req_Add && PRM_OK && !NOPRMCMD && Prm_Pending > 0 => SET_WD Active_Groups := Group_Ident EXE_PRM_CMD Prm_Pending := 2 Check_Prm_Add := Req Add B User Prm := Prm Data.User Prm Diag_Data := Diag Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref) DMPMS Set minTsdr.req (minTsdr)	DATA-EXCH
128	DATA-EXCH	DMPMS Set Prm.ind(Req Add, Prm Data) /Diag.Master_Add = Req_Add && PRM_OK && NOPRMCMD && PV0V0 && Prm_Pending = 0 => SET_WD Active_Groups := Group_Ident Prm_Pending := 1 Check_Prm_Add := Req Add User Parameter Data := Prm Data.User Prm Diag_Data := Diag Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref) DMPMS Set minTsdr.req (minTsdr) MSCY1S Check User Prm.ind (Prm Structure, User Parameter Data)	DATA-EXCH
129	DATA-EXCH	DMPMS Set Prm.ind(Req Add, Prm Data) /Diag.Master_Add = Req_Add && PRM_OK && NOPRMCMD && PV0V0 && Prm_Pending > 0 =>SET_WD Active_Groups := Group_Iden tPrm_Pending := 2 Check_Prm_Add := Req Add B User Prm := Prm Data.User Prm Diag_Data := Diag Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref) DMPMS Set minTsdr.req (minTsdr)	DATA-EXCH

Table 439 (continued)

#	Current State	Event /Condition =>Action	Next State
130	DATA-EXCH	DMPMS Set Prm.ind(Req Add, Prm Data) /Diag.Master_Add <> Req_Add && PRM_OK && Prm_Pending = 0 => Diag.Sync_Mode := FALSE Diag.Freeze_Mode := FALSE Diag.Station_Not_Ready := TRUE Diag.Stat_Diag := FALSE SET_OPERATION_MODE SET_WD Active_Groups := Group_Ident Safe_State := TRUE Diag.Master_Add := Req_Add Check_Prm_Add := Req Add STOP_ASM STOP_C1 Prm_Pending := 1 User Parameter Data := Prm Data.User Prm SET_ALARM_CHKCFGM Diag Data := Diag Act_Ref := Act_Ref + 1 DMPMS Leave.req DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref) DMPMS Set minTsdr.req (minTsdr) MSCY1S Check User Prm.ind (Prm Structure, User Parameter Data)	WAIT-CFG
131	DATA-EXCH	DMPMS Set Prm.ind(Req Add, Prm Data) /Diag.Master_Add <> Req_Add && PRM_OK && Prm_Pending > 0 => Diag.Sync_Mode := FALSE Diag.Freeze_Mode := FALSE Diag.Station_Not_Ready := TRUE Diag.Stat_Diag := FALSE SET_OPERATION_MODE SET_WD Active_Groups := Group_Ident Safe_State := TRUE Diag.Master_Add := Req_Add Check_Prm_Add := Req Add STOP_ASM STOP_C1 Prm_Pending := 2 B User Prm := Prm Data.User Prm SET_ALARM_CHKCFGM Diag Data := Diag Act_Ref := Act_Ref + 1 DMPMS Leave.req DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref) DMPMS Set minTsdr.req (minTsdr)	WAIT-CFG
132	DATA-EXCH	DMPMS Set Prm.ind(Req Add, Prm Data) /Prm_Data.len < 7 => ignore	DATA-EXCH
133	DATA-EXCH	DMPMS Chk Cfg.ind(Req Add, Cfg Data) /Diag.Master_Add <> Req_Add => ignore	DATA-EXCH
134	DATA-EXCH	DMPMS Chk Cfg.ind(Req Add, Cfg Data) /Diag.Master_Add = Req_Add && Cfg_Pending = 0 => Cfg_Pending := 1 Check_Cfg_Add := Req Add TRIG_WD MSCY1S Check Cfg.ind (Check Cfg Mode, Cfg Data)	DATA-EXCH
135	DATA-EXCH	DMPMS Chk Cfg.ind(Req Add, Cfg Data) /Diag.Master_Add = Req_Add && Cfg_Pending > 0 => Cfg_Pending := 2 Check_Cfg_Add := Req Add B Cfg := Cfg Data TRIG_WD	DATA-EXCH

Table 439 (continued)

#	Current State	Event /Condition =>Action	Next State
136	DATA-EXCH	DMPMS Data Exchange.ind(Outp Data) /Outp_Data.len > 0 && Outp_Data.len = Outp Data Len && Diag.Sync_Mode = FALSE => B Output := Outp Data TRIG_WD Safe_State := FALSE Clear Flag := FALSE New Output := TRUE CHECK_ALARM_START MSCY1S New Output.ind (Clear Flag)	DATA-EXCH
137	DATA-EXCH	DMPMS Data Exchange.ind(Outp Data) /Outp_Data.len > 0 && Outp_Data.len = Outp Data Len && Diag.Sync_Mode = TRUE => B Sync := Outp Data TRIG_WD Safe_State := FALSE CHECK_ALARM_START	DATA-EXCH
138	DATA-EXCH	DMPMS Data Exchange.ind(Outp Data) /Outp_Data.len = 0 && Outp Data Len = 0 => TRIG_WD CHECK_ALARM_START	DATA-EXCH
139	DATA-EXCH	DMPMS Data Exchange.ind(Outp Data) /Outp_Data.len <> Outp Data Len && (Outp_Data.len = 0 && Fail_Safe_supp) => TRIG_WD Safe_State := TRUE Clear Flag := TRUE New Output := TRUE CHECK_ALARM_START MSCY1S New Output.ind (Clear Flag)	DATA-EXCH
140	DATA-EXCH	DMPMS Data Exchange.ind(Outp Data) /Outp_Data.len <> Outp Data Len && (Outp_Data.len <> 0 Fail_Safe_supp) => LEAVE-MASTERAct_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	WAIT-PRM
141	DATA-EXCH	WDTimer expired => LEAVE-MASTER Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	WAIT-PRM
142	DATA-EXCH	DMPMS Global Control.ind(Control Command, Group Select) / (Control_Command.0, .6, .7 = TRUE) => LEAVE-MASTER Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	WAIT-PRM
143	DATA-EXCH	DMPMS Global Control.ind(Control Command, Group Select) /Group_Select <> 0 && ((Active_Groups & Group_Select) = 0) && ((Group_Select < 0x80) Isochronous Mode = FALSE) && (Control_Command.0, .6, .7 = FALSE) => ignore	DATA-EXCH
144	DATA-EXCH	DMPMS Global Control.ind(Control Command, Group Select) /Control Command.Clear Command = FALSE && (Group_Select = 0 (Active_Groups & Group_Select) <> 0 ((Group_Select >= 0x80) && Isochronous Mode = TRUE)) && (Control_Command.0, .6, .7 = FALSE) => Clear Command := FALSE	CHECK-ISOM

Table 439 (continued)

#	Current State	Event /Condition =>Action	Next State
145	DATA-EXCH	DMPMS Global Control.ind(Control Command, Group Select) /Control Command.Clear Command = TRUE && (Group_Select = 0 (Active_Groups & Group_Select) <> 0 ((Group_Select >= 0x80) && Isochronous Mode = TRUE)) && (Control_Command.0, .6, .7 = FALSE) => Clear Command := TRUE Clear Flag := TRUE New Output := TRUE Safe_State := TRUE	CHECK-ISOM
146	DATA-EXCH	DMPMS Set Ext Prm.ind(Req_Add, Ext Prm Data) /Diag.Master_Add = Req_Add => Diag.Prm_Fault := TRUE LEAVE-MASTER Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	WAIT-PRM
147	DATA-EXCH	MSCY1S Check Ext User Prm Result.req (Ext_Prm_OK) /Ext_Prm_Pending = 0 => Status := Not pending MSCY1S Check Ext User Prm Result.cnf(-) (Status)	DATA-EXCH
148	DATA-EXCH	MSCY1S Check Ext User Prm Result.req (Ext_Prm_OK) /Ext_Prm_Pending = 2 => Status := New Prm Ext_Prm_Pending := 1 Ext_User Parameter Data := B Ext User Prm MSCY1S Check Ext User Prm Result.cnf(-) (Status) MSCY1S Check Ext User Prm.ind (Ext User Parameter Data)	DATA-EXCH
149	DATA-EXCH	MSCY1S Check Ext User Prm Result.req (Ext_Prm_OK) /Ext_Prm_Pending = 1 && Diag.Master_Add <> Check_Ext_Prm_Add => Status := Inv Master Add Ext_Prm_Pending := 0 MSCY1S Check Ext User Prm Result.cnf(-) (Status)	DATA-EXCH
150	DATA-EXCH	MSCY1S Check Ext User Prm Result.req (Ext_Prm_OK) /Ext_Prm_Pending = 1 && Diag.Master_Add = Check_Ext_Prm_Add && Ext_Prm_OK = TRUE => Ext_Prm_Pending := 0 MSCY1S Check Ext User Prm Result.cnf(+)()	DATA-EXCH
151	DATA-EXCH	MSCY1S Check Ext User Prm Result.req (Ext_Prm_OK) /Ext_Prm_Pending = 1 && Diag.Master_Add = Check_Ext_Prm_Add && Ext_Prm_OK = FALSE => Diag.Cfg_Fault := TRUE LEAVE-MASTER Ext_Prm_Pending := 0 Act_Ref := Act_Ref + 1 MSCY1S Check Ext User Prm Result.cnf(+)() DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	WAIT-PRM
152	CHECK-SYNC	/NOSYNC => Sync Command := 0	CHECK-FREEZE
153	CHECK-SYNC	/UNSYNC && Sync Supp = FALSE => Sync Command := 2	CHECK-FREEZE
154	CHECK-SYNC	/SYNC && Sync Supp = FALSE => LEAVE-MASTER Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	WAIT-PRM

Table 439 (continued)

#	Current State	Event /Condition =>Action	Next State
155	CHECK-SYNC	/UNSYNC && Sync Supp = TRUE && B Sync <> Nil => B Output := B Sync B Sync := Nil Sync Command := 2 Clear Flag := Safe_State New Output := TRUE Diag.Sync_Mode := FALSE Diag Data := Diag Act_Ref := Act_Ref + 1 MSCY1S New Output.ind (Clear Flag) DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	CHECK-FREEZE
156	CHECK-SYNC	/UNSYNC && Sync Supp = TRUE && B Sync = Nil => Sync Command := 2 Clear Flag := Safe_State Diag.Sync_Mode := FALSE Diag Data := Diag Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	CHECK-FREEZE
157	CHECK-SYNC	/SYNC && Sync Supp = TRUE && Diag.Sync_Mode = FALSE => B Sync := Nil Sync Command := 1 Diag.Sync_Mode := TRUE Diag Data := Diag Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	CHECK-FREEZE
158	CHECK-SYNC	/SYNC && Sync Supp = TRUE && Diag.Sync_Mode = TRUE && B Sync <> Nil => B Output := B Sync B Sync := Nil Clear Flag := Safe State New Output := TRUE Sync Command := 1 MSCY1S New Output.ind (Clear Flag)	CHECK-FREEZE
159	CHECK-SYNC	/SYNC && Sync Supp = TRUE && Diag.Sync_Mode = TRUE && B Sync = Nil => Sync Command := 1	CHECK-FREEZE
160	CHECK-ISOM	/(Group_Select >= 0x80) && Isochronous Mode = TRUE => if (FirstSynch = TRUE) Status:= IsoM Start FirstSynch:= FALSE else Status:=IsoM SYNCH endif MSCY1S SYNCH Event.ind(Status)	CHECK-SYNC
161	CHECK-ISOM	/(Group_Select < 0x80) Isochronous Mode = FALSE =>	CHECK-SYNC
162	CHECK-FREEZE	/NOFREEZE => Freeze Command := 0 MSCY1S Global Control.ind (Clear Command, Sync Command, Freeze Command)	DATA-EXCH
163	CHECK-FREEZE	/UNFREEZE && Freeze Supp = FALSE => Freeze Command := 2 MSCY1S Global Control.ind (Clear Command, Sync Command, Freeze Command)	DATA-EXCH
164	CHECK-FREEZE	/FREEZE && Freeze Supp = FALSE => LEAVE-MASTER Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref)	WAIT-PRM

Table 439 (continued)

#	Current State	Event /Condition =>Action	Next State
165	CHECK-FREEZE	/UNFREEZE && Freeze Supp = TRUE && B Freeze <> Nil => Inp Data := B Freeze B Freeze := Nil Freeze Command := 2 Diag.Freeze_Mode := FALSE Diag Data := Diag Act_Ref := Act_Ref + 1 DMPMS Data Exchange Upd.req (Diag Flag, Inp Data) DMPMS RD Inp Upd.req (Inp Data) DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref) MSCY1S Global Control.ind (Clear Command, Sync Command, Freeze Command)	DATA-EXCH
166	CHECK-FREEZE	/UNFREEZE && Freeze Supp = TRUE && B Freeze = Nil => Freeze Command := 2 Diag.Freeze_Mode := FALSE Diag Data := Diag Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref) MSCY1S Global Control.ind (Clear Command, Sync Command, Freeze Command)	DATA-EXCH
167	CHECK-FREEZE	/FREEZE && Freeze Supp = TRUE && Diag.Freeze_Mode = FALSE => B Freeze := Nil Freeze Command := 1 Diag.Freeze_Mode := FALSE Diag Data := Diag Act_Ref := Act_Ref + 1 DMPMS Slave Diag Upd.req (Diag Data, Reference := Act_Ref) MSCY1S Global Control.ind (Clear Command, Sync Command, Freeze Command)	DATA-EXCH
168	CHECK-FREEZE	/FREEZE && Freeze Supp = TRUE && Diag.Freeze_Mode = TRUE && B Freeze <> Nil => Inp Data := B Freeze B Freeze := Nil Freeze Command := 1 DMPMS Data Exchange Upd.req (Diag Flag, Inp Data) DMPMS RD Inp Upd.req (Inp Data) MSCY1S Global Control.ind (Clear Command, Sync Command, Freeze Command)	DATA-EXCH
169	CHECK-FREEZE	/FREEZE && Freeze Supp = TRUE && Diag.Freeze_Mode = TRUE && B Freeze = Nil => Freeze Command := 1 MSCY1S Global Control.ind (Clear Command, Sync Command, Freeze Command)	DATA-EXCH
170	any state	MSCY1S Reset.req () => DMPMS Slave Deact.req ()	SL-DEACT
171	any state	DMPMS Set Ext Prm.ind(Req_Add, Ext Prm Data) /Diag.Master_Add <> Req_Add => ignore	SAME
172	SL-DEACT	DMPMS Slave Deact.cnf () => MSCY1S Reset.cnf ()	

6.6.1.4 Functions

Table 440 contains the functions used by the MSCY1S and their descriptions.

Table 440 – Functions used by the MSCY1S

Name	Function
LEAVE-MASTER	Diag.Master_Add=invalid Diag.Sync_Mode=FALSE Diag.Freeze_Mode=FALSE Diag.Prm_Req=TRUE Diag.Station_Not_Ready=TRUE Stop Timer(WD), Diag.WD_On=FALSE Diag.Stat_Diag=FALSE, Operation_Mode=V0 Safe_State=TRUE STOP_ASM, STOP_C1, STOP_ISOM DMPMS_Leave.Req Diag_Data=Diag
PRM_OK	Prm_Data.len>=7 && Lock_Req=TRUE && Unlock_Req=FALSE && Ident_Number=Diag.Ident_Number && (WD_On=FALSE (WD_On=TRUE && WD_Fact_1>0 && WD_Fact_2>0)) && (Freeze_Req=FALSE Freeze_Supported) && (Sync_Req=FALSE Sync_Supported) && (Prm_Data[1].0, .1, .2=FALSE) && OPERATION_MODE_OK
SET_WD	if (WD_On=TRUE) Start Timer(WD), Diag.WD_On=TRUE) else Stop Timer(WD), Diag.WD_On=FALSE endif
TRIG_WD	if (Diag.WD_On=TRUE) Start Timer(WD) endif
NOSYNC	Control_Command.Sync=0 && Control_Command.UnSync=0
SYNC	Control_Command.Sync=1 && Control_Command.UnSync=0
UNSYNC	Control_Command.UnSync=1
NOFREEZE	Control_Command.Freeze=0 && Control_Command.UnFreeze=0
FREEZE	Control_Command.Freeze=1 && Control_Command.UnFreeze=0
UNFREEZE	Control_Command.UnFreeze=1
STOP_ISOM	if(Isochronous Mode Supp = TRUE && Isochronous Mode =TRUE && First Synch = FALSE) MSCY1S SYNCH Event.ind(Status:= IsoM Stop) First Synch = TRUE endif
STOP_ASM	if (DX_Entered=TRUE) MSCY1S_Stop.ind endif
CHECK_ALARM_START	if (DX_Entered=FALSE && Diag.Stat_Diag=FALSE) DX_Entered=TRUE, MSCY1S_Start.ind(Actual_Enabled_Alarms, Alarm_Sequence, Alarm_Limit) endif
SET_OPERATION_MODE	if (Prm_Data.len >= 10 && DPV1_Enable=TRUE) Operation_Mode=V1 else Operation_Mode=V0 endif if (Prm_Data.len >= 10 && IsoM_Req =TRUE) Isochron_Mode=True else Isochron_Mode=FALSE endif
PV0V0	Operation_Mode = V0 && (Prm_Data.len < 10 DPV1_Enable = FALSE)
OPERATION_MODE_OK	if (IsARexistent(MS1) && DPV1_Enable) further checks according the table "Rules for DPV1_Status_1, DPV1_Status_2, and DPV1_Status_3 check"

Table 440 (continued)

Name	Function
NOPRMCMD	!(DPV1_Status_3.PrmCmd = 1 && Prm_Data.len = 18 && PrmCmd_supp)
SET_ALARM_CHKCFG	<pre> if (Prm_Data.len >=10 && DPV1_Enable=TRUE) Enabled_Alarms=DPV1_Status_2.2 - DPV1_Status_2.7 else Enabled_Alarms=0 endif if (Prm_Data.len >=10) Alarm_Mode_Master=DPV1_Status_3.0 - DPV1_Status_3.2, Chk Cfg Mode = DPV1_Status_2.0, Prm Structure = DPV1_Status_3.4, if (DPV1_Status3.PrmCmd&& Prm_Data.PrmCmd exist) EXE_PRM_CMD else Alarm_Mode_Master=0, Chk Cfg Mode = FALSE, Prm Structure = FALSE endif endif </pre>
EXE_PRM_CMD	<pre> if (PrmCmd.Function.Primary) Primary=TRUE, if(Start_State=B Start_State=BEB Start_State=EB Start_State=Run) STOP_C1, START_C1 else START_C1 endif else Primary=FALSE endif if (PrmCmd.Function.Stop_MS1) if(Start_State=B Start_State=BEB Start_State=EB Start_State=Run) STOP_C1 endif endif if (PrmCmd.Function.Start_MS1) if(Start_State=E Start_State=BE Start_State=Idle) START_C1 endif endif endif </pre>
START_C1	<pre> if (Start_State=Idle) MSAC1S_Start.req(Diag.Master_Add), Start_State=B endif if (Start_State=E) Start_State=EB endif if (Start_State=BE) Start_State=BEB endif endif </pre>
STOP_C1	<pre> if (Start_State=B) Start_State=BE endif if (Start_State=Run) MSAC1S_Stop.req, Start_State=E endif if (Start_State=EB) Start_State=E endif if (Start_State=BEB) Start_State=BE endif endif </pre>
START_C1_CON	<pre> if (Start_State=B) Start_State=Run endif if (Start_State=BE) MSAC1S_Stop.req, Start_State=E endif if (Start_State=BEB) MSAC1S_Stop.req, Start_State=EB endif endif </pre>

Name	Function
STOP_C1_CON	<pre> if (Start_State=E) Start_State=Idle endif if (Start_State=EB) MSAC1S_Start.req(Diag.Master_Add), Start_State=B endif </pre>
LEAVE-MASTER	<pre> Diag.Master_Add=invalid Diag.Sync_Mode=FALSE Diag.Freeze_Mode=FALSE Diag.Prm_Req=TRUE Diag.Station_Not_Ready=TRUE Stop Timer(WD), Diag.WD_On=FALSE Diag.Stat_Diag=FALSE, Operation_Mode=V0 Safe_State=TRUE STOP_ASM, STOP_C1, STOP_ISOM DMPMS_Leave.Req Diag_Data=Diag </pre>

6.6.2 MSAC1S

6.6.2.1 Primitive definitions

6.6.2.1.1 Primitives exchanged between MSAC1S and FSPMS

Table 441 shows the service primitives including their associated parameters issued by the FSPMS and received by the MSAC1S.

Table 441 – Primitives issued by FSPMS to MSAC1S

Primitive name	Source	Associated parameters	Functions
SInit MS1.req	FSPMS	(none)	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Reset.req	FSPMS	(none)	
Read.rsp(+)	FSPMS	Length, Data	
Read.rsp(-)	FSPMS	Error Decode, Error Code 1 Error Code 2	
Write.rsp(+)	FSPMS	Length	
Write.rsp(-)	FSPMS	Error Decode, Error Code 1 Error Code 2	
Alarm Ack.rsp(+)	FSPMS	Slot Number, Alarm Type, Seq Nr	
Alarm Ack.rsp(-)	FSPMS	(none)	

Table 442 shows the service primitives including their associated parameters issued by MSAC1S and received by the FSPMS.

Table 442 – Primitives issued by MSAC1S to FSPMS

Primitive name	Source	Associated parameters	Functions
SInit MS1.cnf	MSAC1S	(none)	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Reset.cnf	MSAC1S	(none)	
Fault.ind	MSAC1S	(none)	
Read.ind	MSAC1S	Slot Number, Index, Length	
Write.ind	MSAC1S	Slot Number, Index, Length, Data	
Alarm Ack.ind	MSAC1S	Slot Number, Alarm Type, Seq Nr	

6.6.2.1.2 Primitives exchanged between MSAC1S and MSCY1S

Table 443 shows the service primitives including their associated parameters issued by MSCY1S and received by the MSAC1S.

Table 443 – Primitives issued by MSCY1S to MSAC1S

Primitive name	Source	Associated parameters	Functions
Start.req	MSCY1S	Master Add	Start Processing of acyclic services
Stop.req	MSCY1S	(none)	Stop Processing of acyclic services

Table 444 shows the service primitives including their associated parameters issued by the MSAC1S and received by the MSCY1S.

Table 444 – Primitives issued by MSAC1S to MSCY1S

Primitive name	Source	Associated parameters	Functions
Start.cnf	MSAC1S	(none)	MSAC1 is ready to execute acyclic services
Stop.cnf	MSAC1S	(none)	MSAC1 is disabled
Abort.ind	MSAC1S	(none)	Termination of MSAC1 due to a sequence error or wrong protocol elements

6.6.2.1.3 Parameter of MSAC1S primitives

The parameters used with the primitives exchanged between the FSPMS and the MSAC1S are described in FAL Service Definition Type 3 Fieldbus IEC 61158-5. Additional parameters are described in Table 445.

Table 445 – Parameter used with primitives exchanged between MSAC1S and MSCY1S

Parameter name	Description
MasterAdd	This parameter conveys the DL address of the assigned DP-master.

6.6.2.2 State machine description

After Power-On the MSAC1S waits for initiation by means of Slnit MS1.

In the state CLOSED the service Start called from the MSCY1S is expected. Then the access protection for the SAP 51 (50) will be activated by means of the function RSAP_ACTIVATE.

When receiving the confirmation the State Machine enters the OPEN state. In this state the services Read, Write and Alarm_Ack are allowed. Only one request from the DP-master is processed simultaneously on each SAP. Additionally to Read/Write at SAP 51 a Alarm_Ack may be executed at SAP 50.

When a valid service is invoked by the DP-master, the MSAC1S will indicate this primitive to the AP-Context (Read/Write or Alarm_Ack) and waits for response. After receiving the corresponding response MSAC1S will provide this PDU in the corresponding SAP update-buffer. The service is finished after DP-master has fetched the response.

The local service Stop called from the MSCY1S is closing the connection. The SAP 51 (50) is deactivated and a possibly stored response in the DL is deleted.

The abort of the connection is indicated to the MSCY1S with the local service Abort. If the User is still processing a response while the connection is aborted this response shall be discarded by the User.

Local variables of the MSAC1S

Server_SAP

(Unsigned8)

This SAP (51) is used for the MSAC1S_Read/_Write and MSAC1S_Alarm_Ack services.

Alarm_SAP

(Unsigned8)

This SAP (50) is reserved for MSAC1S_Alarm_Ack services.

Res_SAP

(Unsigned8)

Variable holding the SAP for the MSAC1S_Alarm_Ack -Response.

Service_Header

(Unsigned8)

The actual service header of the service processed. Used to check whether the User generates the correct response.

AA_State

MSAC1S_Alarm_Ack services may be executed simultaneously with the MSAC1S_Read/_Write-services. Thus, the actual state of the MSAC1S_Alarm_Ack processing is stored in the variable AA_State:

Values of AA_State:

Idle	No MSAC1S_Alarm_Ack in progress
WRes	MSAC1S_Alarm_Ack.ind is sent to the MSAL1S waiting for response
Upd	Response of the MSAL1S will be sent to the Update-Buffer
SRes	Response is stored – waiting for the next poll of the DP-master

VS_Upd

Indicates that a Response will be sent to the Update-Buffer.

6.6.2.3 MSAC1S state table

Table 446 contains the complete description of the MSAC1S state machine.

Table 446 – MSAC1S state table

#	Current state	Event / condition => action	Next state
1	POWER-ON	MSAC1S_SInit_MS1.req => Server_SAP:=51 Alarm_SAP:=50 MSAC1S_SInit_MS1.cnf	CLOSED
2	POWER-ON	MSAC1S_Reset.req => MSAC1S_Reset.cnf	POWER-ON
3	POWER-ON	MSAC1S_Alarm_Ack.rsp(+) (Alarm_Type, Slot_Number, Seq_Nr) => MSAC1S_Fault.ind	POWER-ON
4	POWER-ON	MSAC1S_Alarm_Ack.rsp(-) () => MSAC1S_Fault.ind	POWER-ON
5	CLOSED	MSAC1S_Start.req(Master_Add) => DMPMS_RSAP_ACTIVATE.req(SSAP=Server_SAP, Access=Master_Add, L_sdu_length_list=Nil, Indication_Mode=Unchanged)	SET-ACC51
6	CLOSED	MSAC1S_Stop.req => MSAC1S_Stop.cnf	CLOSED
7	SET-ACC51	DMPMS_RSAP_ACTIVATE.cnf(SSAP=Server_SAP, M_status) /M_status = OK => DMPMS_RSAP_ACTIVATE.req(SSAP=Alarm_SAP, Access=Master_Add, L_sdu_length_list=Nil, Indication_Mode=Unchanged)	SET-ACC50
8	SET-ACC51	DMPMS_RSAP_ACTIVATE.cnf(SSAP=Server_SAP, M_status) /M_status=NO/IV => MSAC1S_Fault.ind	POWER-ON
9	SET-ACC50	DMPMS_RSAP_ACTIVATE.cnf(SSAP=Alarm_SAP, M_status) /M_status = OK => AA_State:=Idle VS_Upd:=False MSAC1S_Start.cnf	OPEN
10	SET-ACC50	DMPMS_RSAP_ACTIVATE.cnf(SSAP=Alarm_SAP, M_status) /M_status=NO/IV => MSAC1S_Fault.ind	POWER-ON

Table 446 (continued)

#	Current state	Event / condition => action	Next state
11	OPEN	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Server_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /L_sdu.len=0 && Update_status=NO => MSAC1S_Fault.ind	POWER-ON
12	OPEN	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Alarm_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /L_sdu.len=0 && Update_status=NO => MSAC1S_Fault.ind	POWER-ON
13	OPEN	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Server_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /Read-REQ-PDU() => Service_Header:=Function_Num MSAC1S_Read.ind(Req_Add=Loc_add, Slot_Number,Index,Length)	VS-WRES
14	OPEN	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Server_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /Write-REQ-PDU() && L_sdu.len >= (Length+4) => Service_Header:=Function_Num MSAC1S_Write.ind(Req_Add=Loc_add, Slot_Number,Index,Length,Data)	VS-WRES
15	OPEN	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Server_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /Alarm_Ack-REQ-PDU() => AA_State:=WRes Res_SAP:=Server_SAP MSAC1S_Alarm_Ack.ind(Alarm_Type, Slot_Number,Seq_Nr)	AA-WRES
16	OPEN	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Alarm_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /Alarm_Ack-REQ-PDU() => AA_State:=WRes Res_SAP:=Alarm_SAP MSAC1S_Alarm_Ack.ind(Alarm_Type, Slot_Number,Seq_Nr)	AA-WRES
17	OPEN	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Server_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /NOT(Read-REQ-PDU() Write-REQ-PDU() && (L_sdu.len >= Length+4) Alarm_Ack-REQ-PDU()) => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
18	OPEN	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Alarm_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /NOT(Alarm_Ack-REQ-PDU()) => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
19	OPEN	DMPMS_REPLY_UPDATE.cnf(SSAP=Server_SAP,Serv_class,L_status) => MSAC1S_Fault.ind	POWER-ON
20	OPEN	MSAC1S_Read.rsp(+)(Length, Data) => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
21	OPEN	MSAC1S_Read.rsp(-)(Error Decode, Error_Code_1, Error_Code_2) => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
22	OPEN	MSAC1S_Write.rsp(+)(Length) => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51

Table 446 (continued)

#	Current state	Event / condition => action	Next state
23	OPEN	MSAC1S_Write.rsp(-)(Error_Decode, Error_Code_1, Error_Code_2) => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
24	OPEN	MSAC1S_Alarm_Ack.rsp(+) (Alarm_Type, Slot_Number, Seq_Nr) => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
25	OPEN	MSAC1S_Alarm_Ack.rsp(-) () => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
26	OPEN	MSAC1S_Stop.req => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP)	STOP-DEA51
27	OPEN	MSAC1S_Start.req(Master_Add) => MSAC1S_Fault.ind	POWER-ON
28	OPEN	MSAC1S_SInit_MS1.req => MSAC1S_Fault.ind	POWER-ON
29	OPEN	MSAC1S_Reset.req => MSAC1S_Reset.cnf	POWER-ON
30	VS-WRES	MSAC1S_Read.rsp(+)(Length, Data) /Service_Header=0x5E && Max_DLSDU_Length_Req_Low>=(Length+4) => VS_Upd:=True L_sdu:=Read-RES-PDU DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu, Serv_class=Low, Transmit=Single)	VS-SRES
31	VS-WRES	MSAC1S_Read.rsp(+)(Length, Data) /Service_Header=0x5E && Max_DLSDU_Length_Req_Low<(Length+4) => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
32	VS-WRES	MSAC1S_Write.rsp(+)(Length) /Service_Header=0x5F => VS_Upd:=True L_sdu:=Write-RES-PDU DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu, Serv_class=Low, Transmit=Single)	VS-SRES
33	VS-WRES	MSAC1S_Read.rsp(-)(Error_Decode, Error_Code_1, Error_Code_2) /Service_Header=0x5E => VS_Upd:=True L_sdu:=Read-NRS-PDU DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu, Serv_class=Low, Transmit=Single)	VS-SRES
34	VS-WRES	MSAC1S_Write.rsp(-)(Error_Decode, Error_Code_1, Error_Code_2) /Service_Header=0x5F => VS_Upd:=True L_sdu:=Write-NRS-PDU DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu, Serv_class=Low, Transmit=Single)	VS-SRES
35	VS-WRES	MSAC1S_Read.rsp(+)(Length, Data) /Service_Header<>0x5E => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
36	VS-WRES	MSAC1S_Read.rsp(-)(Error_Decode, Error_Code_1, Error_Code_2) /Service_Header<>0x5E => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51

Table 446 (continued)

#	Current state	Event / condition => action	Next state
37	VS-WRES	MSAC1S_Write.rsp(+)(Length) /Service_Header<>0x5F => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
38	VS-WRES	MSAC1S_Write.rsp(-)(Error Decode, Error_Code_1, Error_Code_2) /Service_Header<>0x5F => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
39	VS-WRES	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Server_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /L_sdu.len=0 && Update_status=NO => MSAC1S_Fault.ind	POWER-ON
40	VS-WRES	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Alarm_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /L_sdu.len=0 && Update_status=NO => MSAC1S_Fault.ind	POWER-ON
41	VS-WRES	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Server_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
42	VS-WRES	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Alarm_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /Alarm_Ack-REQ-PDU() && AA_State=Idle => AA_State:=WRes Res_SAP:=Alarm_SAP MSAC1S_Alarm_Ack.ind(Alarm_Type, Slot_Number, Seq_Nr)	VS-WRES
43	VS-WRES	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Alarm_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /Update_status=LO && L_sdu.len=0 && AA_State=SRes => AA_State:=Idle	VS-WRES
44	VS-WRES	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Alarm_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /NOT((Alarm_Ack-REQ-PDU() && AA_State=Idle) (Update_status=LO && L_sdu.len=0 && AA_State=SRes)) => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
45	VS-WRES	MSAC1S_Start.req(Master_Add) => MSAC1S_Fault.ind	POWER-ON
46	VS-WRES	MSAC1S_Stop.req => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP)	STOP-DEA51
47	VS-WRES	MSAC1S_Alarm_Ack.rsp(+)(Alarm_Type, Slot_Number, Seq_Nr) /AA_State<>WRes => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
48	VS-WRES	MSAC1S_Alarm_Ack.rsp(-)() /AA_State<>WRes => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
49	VS-WRES	MSAC1S_Alarm_Ack.rsp(+)(Alarm_Type, Slot_Number, Seq_Nr) /AA_State=WRes => AA_State:=Upd DMPMS_REPLY_UPDATE.req(SSAP=Res_SAP, L_sdu=Alarm_Ack-RES-PDU, Serv_class=Low, Transmit=Single)	VS-WRES

Table 446 (continued)

#	Current state	Event / condition => action	Next state
50	VS-WRES	MSAC1S_Alarm_Ack.rsp(-) () /AA_State=WRes => AA_State:=Upd DMPMS_REPLY_UPDATE.req(SSAP=Res_SAP, L_sdu=Alarm_Ack-NRS-PDU, Serv_class=Low, Transmit=Single)	VS-WRES
51	VS-WRES	DMPMS_REPLY_UPDATE.cnf(SSAP=Alarm_SAP, Serv_class, L_status) /L-status=OK && AA_State=Upd => AA_State:=SRes	VS-WRES
52	VS-WRES	DMPMS_REPLY_UPDATE.cnf(SSAP=Alarm_SAP, Serv_class, L_status) /L-status=OK && AA_State<>Upd => MSAC1S_Fault.ind	POWER-ON
53	VS-WRES	DMPMS_REPLY_UPDATE.cnf(SSAP=Server_SAP, Serv_class, L_status) /L_status=LS/IV/LR => MSAC1S_Fault.ind	POWER-ON
54	VS-WRES	MSAC1S_Sinit_MS1.req => MSAC1S_Fault.ind	POWER-ON
55	VS-WRES	MSAC1S_Reset.req => MSAC1S_Reset.cnf	POWER-ON
56	VS-SRES	DMPMS_REPLY_UPDATE.cnf(SSAP=Server_SAP, Serv_class, L_status) /L-status=OK && VS_Upd=True => VS_Upd:=False	VS-SRES
57	VS-SRES	DMPMS_REPLY_UPDATE.cnf(SSAP=Server_SAP, Serv_class, L_status) /L_status=LS/IV/LR => MSAC1S_Fault.ind	POWER-ON
58	VS-SRES	DMPMS_REPLY_UPDATE.cnf(SSAP=Server_SAP, Serv_class, L_status) /L-status=OK && VS_Upd=False => MSAC1S_Fault.ind	POWER-ON
59	VS-SRES	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add, L_sdu, Serv_class, Update_status) /L_sdu.len=0 && Update_status=NO => MSAC1S_Fault.ind	POWER-ON
60	VS-SRES	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Alarm_SAP, Loc_add, Rem_add, L_sdu, Serv_class, Update_status) /L_sdu.len=0 && Update_status=NO => MSAC1S_Fault.ind	POWER-ON
61	VS-SRES	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add, L_sdu, Serv_class, Update_status) /Update_status=LO && L_sdu.len=0 && AA_State=Idle && VS_Upd=False	OPEN
62	VS-SRES	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add, L_sdu, Serv_class, Update_status) /Update_status=LO && L_sdu.len=0 && AA_State=WRes && VS_Upd=False	AA-WRES
63	VS-SRES	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add, L_sdu, Serv_class, Update_status) /Update_status=LO && L_sdu.len=0 && (AA_State=SRes AA_State=Upd) && VS_Upd=False	AA-SRES
64	VS-SRES	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add, L_sdu, Serv_class, Update_status) /L_sdu.len=0 && VS_Upd=True => MSAC1S_Fault.ind	POWER-ON

Table 446 (continued)

#	Current state	Event / condition => action	Next state
65	VS-SRES	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Server_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /L_sdu.len<>0 => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
66	VS-SRES	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Alarm_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /Alarm_Ack-REQ-PDU() && AA_State=Idle => AA_State:=WRes Res_SAP:=Alarm_SAP MSAC1S_Alarm_Ack.ind(Alarm_Type, Slot_Number,Seq_Nr)	VS-SRES
67	VS-SRES	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Alarm_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /Update_status=LO && L_sdu.len=0 && AA_State=SRes => AA_State:=Idle	VS-SRES
68	VS-SRES	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Alarm_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /NOT((Alarm_Ack-REQ-PDU() && AA_State=Idle) (Update_status=LO && L_sdu.len=0 && AA_State=SRes)) => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
69	VS-SRES	MSAC1S_Start.req(Master_Add) => MSAC1S_Fault.ind	POWER-ON
70	VS-SRES	MSAC1S_Stop.req => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP)	STOP-DEA51
71	VS-SRES	MSAC1S_Read.rsp(+)(Length, Data) => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
72	VS-SRES	MSAC1S_Read.rsp(-)(Error Decode, Error_Code_1, Error_Code_2) => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
73	VS-SRES	MSAC1S_Write.rsp(+)(Length) => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
74	VS-SRES	MSAC1S_Write.rsp(-)(Error Decode, Error_Code_1, Error_Code_2) => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
75	VS-SRES	MSAC1S_Alarm_Ack.rsp(+) (Alarm_Type, Slot_Number, Seq_Nr) /AA_State<>WRes => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
76	VS-SRES	MSAC1S_Alarm_Ack.rsp(-) () /AA_State<>WRes => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
77	VS-SRES	MSAC1S_Alarm_Ack.rsp(+) (Alarm_Type, Slot_Number, Seq_Nr) /AA_State=WRes => AA_State:=Upd DMPMS_REPLY_UPDATE.req(SSAP=Res_SAP, L_sdu=Alarm_Ack-RES-PDU, Serv_class=Low,Transmit=Single)	VS-SRES

Table 446 (continued)

#	Current state	Event / condition => action	Next state
78	VS-SRES	MSAC1S_Alarm_Ack.rsp(-) () /AA_State=WRes => AA_State:=Upd DMPMS_REPLY_UPDATE.req(SSAP=Res_SAP, L_sdu=Alarm_Ack-NRS-PDU, Serv_class=Low,Transmit=Single)	VS-SRES
79	VS-SRES	DMPMS_REPLY_UPDATE.cnf(SSAP=Alarm_SAP,Serv_class,L_status) /L-status=OK && AA_State=Upd => AA_State:=SRes	VS-SRES
80	VS-SRES	DMPMS_REPLY_UPDATE.cnf(SSAP=Alarm_SAP,Serv_class,L_status) /L-status=OK && AA_State<>Upd => MSAC1S_Fault.ind	POWER-ON
81	VS-SRES	MSAC1S_SInit_MS1.req => MSAC1S_Fault.ind	POWER-ON
82	VS-SRES	MSAC1S_Reset.req => MSAC1S_Reset.cnf	POWER-ON
83	AA-WRES	MSAC1S_Alarm_Ack.rsp(+) (Alarm_Type, Slot_Number, Seq_Nr) => AA_State:=Upd L_sdu:=Alarm_Ack-RES-PDU DMPMS_REPLY_UPDATE.req(SSAP=Res_SAP, L_sdu, Serv_class=Low,Transmit=Single)	AA-SRES
84	AA-WRES	MSAC1S_Alarm_Ack.rsp(-) () => AA_State:=Upd L_sdu:=Alarm_Ack-NRS-PDU DMPMS_REPLY_UPDATE.req(SSAP=Res_SAP, L_sdu, Serv_class=Low,Transmit=Single)	AA-SRES
85	AA-WRES	DMPMS_REPLY_UPDATE.cnf(SSAP=Server_SAP,Serv_class,L_status) => MSAC1S_Fault.ind	POWER-ON
86	AA-WRES	MSAC1S_Read.rsp(+)(Length, Data) => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
87	AA-WRES	MSAC1S_Read.rsp(-)(Error Decode, Error_Code_1, Error_Code_2) => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
88	AA-WRES	MSAC1S_Write.rsp(+)(Length) => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
89	AA-WRES	MSAC1S_Write.rsp(-)(Error Decode, Error_Code_1, Error_Code_2) => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
90	AA-WRES	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Server_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /L_sdu.len=0 && Update_status=NO => MSAC1S_Fault.ind	POWER-ON
91	AA-WRES	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Alarm_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /L_sdu.len=0 && Update_status=NO => MSAC1S_Fault.ind	POWER-ON
92	AA-WRES	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Alarm_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51

Table 446 (continued)

#	Current state	Event / condition => action	Next state
93	AA-WRES	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Server_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /Read-REQ-PDU() && Res_SAP=Alarm_SAP => Service_Header:=Function_Num MSAC1S_Read.ind(Req_Add=Loc_add, Slot_Number,Index,Length)	VS-WRES
94	AA-WRES	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Server_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /Write-REQ-PDU() && Res_SAP=Alarm_SAP => Service_Header:=Function_Num MSAC1S_Write.ind(Req_Add=Loc_add, Slot_Number,Index,Length,Data)	VS-WRES
95	AA-WRES	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Server_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /NOT(Write-REQ-PDU() Read-REQ-PDU()) Res_SAP=Server_SAP => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
96	AA-WRES	MSAC1S_Start.req(Master_Add) => MSAC1S_Fault.ind	POWER-ON
97	AA-WRES	MSAC1S_Stop.req => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP)	STOP-DEA51
98	AA-WRES	MSAC1S_Reset.req => MSAC1S_Reset.cnf	POWER-ON
99	AA-WRES	MSAC1S_Sinit_MS1.req => MSAC1S_Fault.ind	POWER-ON
100	AA-SRES	DMPMS_REPLY_UPDATE.cnf(SSAP=Alarm_SAP,Serv_class,L_status) /L-status=OK && AA_State=Upd && Res_SAP=Alarm_SAP => AA_State:=SRes	AA-SRES
101	AA-SRES	DMPMS_REPLY_UPDATE.cnf(SSAP=Server_SAP,Serv_class,L_status) /L-status=LS/IV/LR => MSAC1S_Fault.ind	POWER-ON
102	AA-SRES	DMPMS_REPLY_UPDATE.cnf(SSAP=Alarm_SAP,Serv_class,L_status) /L-status=OK && (AA_State<>Upd Res_SAP<>Alarm_SAP) => MSAC1S_Fault.ind	POWER-ON
103	AA-SRES	DMPMS_REPLY_UPDATE.cnf(SSAP=Server_SAP,Serv_class,L_status) /L-status=OK && AA_State=Upd && Res_SAP=Server_SAP => AA_State:=SRes	AA-SRES
104	AA-SRES	DMPMS_REPLY_UPDATE.cnf(SSAP=Server_SAP,Serv_class,L_status) /L-status=OK && (AA_State<>Upd Res_SAP<>Server_SAP) => MSAC1S_Fault.ind	POWER-ON
105	AA-SRES	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Server_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /L_sdu.len=0 && Update_status=NO => MSAC1S_Fault.ind	POWER-ON
106	AA-SRES	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Alarm_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /L_sdu.len=0 && Update_status=NO => MSAC1S_Fault.ind	POWER-ON
107	AA-SRES	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Alarm_SAP,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /Update_status=LO && L_sdu.len=0 && AA_State=SRes => AA_State:=Idle	OPEN

Table 446 (continued)

#	Current state	Event / condition => action	Next state
108	AA-SRES	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Alarm_SAP,Loc_add,Rem_add,L_sdu,Service_class, Update_status) /(Update_status<>LO AA_State<>SRes) && Res_SAP=Alarm_SAP => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
109	AA-SRES	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Server_SAP,Loc_add,Rem_add,L_sdu,Service_class, Update_status) /Update_status=LO && L_sdu.len=0 && AA_State=SRes => AA_State:=Idle	OPEN
110	AA-SRES	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Server_SAP,Loc_add,Rem_add,L_sdu,Service_class, Update_status) /(Update_status<>LO L_sdu.len<>0 AA_State<>SRes) && Res_SAP=Server => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
111	AA-SRES	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Server_SAP,Loc_add,Rem_add,L_sdu,Service_class, Update_status) /Read-REQ-PDU() && RES_SAP=ALARM_SAP => Service_Header:=Function_Num MSAC1S_Read.ind(Req_Add=Loc_add, Slot_Number,Index,Length)	VS-WRES
112	AA-SRES	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Server_SAP,Loc_add,Rem_add,L_sdu,Service_class, Update_status) /Write-REQ-PDU() && RES_SAP=ALARM_SAP => Service_Header:=Function_Num MSAC1S_Write.ind(Req_Add=Loc_add, Slot_Number,Index,Length,Data)	VS-WRES
113	AA-SRES	DMPMS_DATA_REPLY.ind(SSAP,DSAP=Server_SAP,Loc_add,Rem_add,L_sdu,Service_class, Update_status) /NOT(Write-REQ-PDU() Read-REQ-PDU()) && RES_SAP=ALARM_SAP => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
114	AA-SRES	MSAC1S_Start.req(Master_Add) => MSAC1S_Fault.ind	POWER-ON
115	AA-SRES	MSAC1S_Stop.req => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP)	STOP-DEA51
116	AA-SRES	MSAC1S_Read.rsp(+)(Length, Data) => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
117	AA-SRES	MSAC1S_Read.rsp(-)(Error Decode, Error_Code_1, Error_Code_2) => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
118	AA-SRES	MSAC1S_Write.rsp(+)(Length) => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
119	AA-SRES	MSAC1S_Write.rsp(-)(Error Decode, Error_Code_1, Error_Code_2) => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
120	AA-SRES	MSAC1S_Alarm_Ack.rsp (Alarm_Type, Slot_Number, Seq_Nr) => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51
121	AA-SRES	MSAC1S_Alarm_Ack.rsp (Alarm_Type, Slot_Number, Seq_Nr) => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC1S_Abort.ind	ERR-DEA51

Table 446 (continued)

#	Current state	Event / condition => action	Next state
122	AA-SRES	MSAC1S_SInit_MS1.req => MSAC1S_Fault.ind	POWER-ON
123	AA-SRES	MSAC1S_Reset.req => MSAC1S_Reset.cnf	POWER-ON
124	STOP-DEA51	DMPMS_SAP_DEACTIVATE.cnf(SSAP=Server_SAP,M_status) /M_status = OK => DMPMS_SAP_DEACTIVATE.req(SSAP=Alarm_SAP)	STOP-DEA50
125	STOP-DEA51	DMPMS_SAP_DEACTIVATE.cnf(SSAP=Server_SAP,M_status) /M_status=NO/IV => MSAC1S_Fault.ind	POWER-ON
126	STOP-DEA50	DMPMS_SAP_DEACTIVATE.cnf(SSAP=Alarm_SAP,M_status) /M_status = OK => MSAC1S_Stop.cnf	CLOSED
127	STOP-DEA50	DMPMS_SAP_DEACTIVATE.cnf(SSAP=Alarm_SAP,M_status) /M_status=NO/IV => MSAC1S_Fault.ind	POWER-ON
128	ERR-DEA51	DMPMS_SAP_DEACTIVATE.cnf(SSAP=Server_SAP,M_status) /M_status = OK => DMPMS_SAP_DEACTIVATE.req(SSAP=Alarm_SAP)	ERR-DEA50
129	ERR-DEA51	DMPMS_SAP_DEACTIVATE.cnf(SSAP=Server_SAP,M_status) /M_status=NO/IV => MSAC1S_Fault.ind	POWER-ON
130	ERR-DEA50	DMPMS_SAP_DEACTIVATE.cnf(SSAP=Alarm_SAP,M_status) /M_status = OK	CLOSED
131	ERR-DEA50	DMPMS_SAP_DEACTIVATE.cnf(SSAP=Alarm_SAP,M_status) /M_status=NO/IV => MSAC1S_Fault.ind	POWER-ON

6.6.2.4 Functions

Table 447 contains the functions used by the MSAC1S, their arguments and their descriptions.

Table 447 – Functions used by the MSAC1S

Function name	Description
Read-REQ-PDU ()	This function will return TRUE if the corresponding L_sdu is a valid Read-REQ-PDU (L_sdu.len >= 4 && L_sdu[1] = 0x5E)
Write-REQ-PDU ()	This function will return TRUE if the corresponding L_sdu is a valid Write-REQ-PDU. (L_sdu.len >= 4 && L_sdu[1] = 0x5F)
Alarm_Ack-REQ-PDU ()	This function will return TRUE if the corresponding L_sdu is a valid Alarm_Ack-REQ-PDU. (L_sdu.len >= 4 && L_sdu[1] = 0x5C)

6.6.3 SSCY1S

6.6.3.1 Primitive definitions

6.6.3.1.1 Primitives exchanged between SSCY1S and FSPMS

Table 448 shows the service primitives including their associated parameters issued by the FSPMS and received by the SSCY1S.

Table 448 – Primitives issued by FSPMS to SSCY1S

Primitive name	Source	Associated parameters	Functions
Get Publisher Data.req	FSPMS	Rem_Add	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Start Subscriber.req	FSPMS	Rem_Add	
Stop Subscriber.req	FSPMS	Rem_Add	

Table 449 shows the service primitives including their associated parameters issued by the SSCY1S and received by the FSPMS.

Table 449 – Primitives issued by SSCY1S to FSPMS

Primitive name	Source	Associated parameters	Functions
Get Publisher Data.cnf(+)	SSCY1S	Rem_Add, Data, New Flag	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Get Publisher Data.cnf(-)	SSCY1S	Rem_Add	
Start Subscriber.cnf(+)	SSCY1S	Rem_Add	
Start Subscriber.cnf(-)	SSCY1S	Rem_Add	
Stop Subscriber.cnf(+)	SSCY1S	Rem_Add	
Stop Subscriber.cnf(-)	SSCY1S	Rem_Add	
New Publisher Data.ind	SSCY1S	Rem_Add	
Publisher Active.ind	SSCY1S	Rem_Add, Status	

6.6.3.1.2 Parameter of SSCY1S primitives

The parameters used with the primitives exchanged between the FSPMS and the SSCY1S are described in FAL Service Definition Type 3 Fieldbus IEC 61158-5.

6.6.3.2 State machine description

After Power-On the SSCY1S waits for initiation by means of DMPMS DX Entered.ind with the Status TRUE. It indicates that the MSCY1S is opened for data exchange by the associated DP-master.

In the state W-START the service Start Subscriber called from the FSPMS is expected. When receiving this event the transition initializes all local variables and enters the RUN state. Furthermore, the local monitoring timer with the WD value (MSCY1S) is started.

When receiving a DMPMS DX Broadcast.ind with publisher data in the OPEN state the sample data according to the filter table are stored. Furthermore, the local flags New Data and New Stat will be set to TRUE and a New Publisher Data.ind will be issued to the FSPMS. The application retrieves the sample data with the Get Publisher Data request service primitive.

When the monitoring timer expires the status is checked and the timer will be started again. If the current status (New Stat) differs from status (Status) of the last interval the change is indicated to the application by means of the Publisher Active indication service primitive. Therefore, every change of status from active to passiv is reported to the application.

The application may stop the subscribing procedure by sending a Stop Subscriber service request that sets the state machine back to W-START.

In any case, when a DMPMS DX Entered .ind with status FALSE (MSCY1S leaves the data exchange) is invoked the state machine is set back to POWER ON.

Local variables of the SSCY1S

Status

(Boolean)

This local variable is TRUE if the publisher was active during the last WD period. That means the data exchange mode was entered, the subscriber was started and at least one DXB had been received during the previous WD interval. Otherwise it is FALSE.

New_Stat

(Boolean)

This local variable is TRUE if the publisher is active. That means the Data Exchange is entered, the subscriber is started and at least one DXB has been received during the current WD interval. Otherwise it is FALSE

New Data

(Boolean)

This variable is TRUE if new data have arrived and are not retrieved by the Get Publisher Data service primitive. Otherwise it is FALSE.

Sample Data

(Octet-String)

The data buffer for publisher data.

6.6.3.3 SSCY1S state table

Table 450 contains the complete description of the SSCY1S state machine.

Table 450 – SSCY1S state table

#	Current state	Event / condition => action	Next state
1	POWER-ON	SSCY1S_Start_Subscriber.req (Rem_Add) => SSCY1S_Start_Subscriber.cnf (-) (Rem_Add)	POWER-ON
2	POWER-ON	SSCY1S_Stop_Subscriber.req (Rem_Add) => SSCY1S_Stop_Subscriber.cnf (-) (Rem_Add)	POWER-ON
3	POWER-ON	SSCY1S_Get_Publisher_Data.req (Rem_Add) => SSCY1S_Get_Publisher_Data.cnf (-) (Rem_Add)	POWER-ON
4	POWER-ON	DMPMS_DX_Broadcast.ind (Rem_Add, Data) => ignore	POWER-ON
5	POWER-ON	DMPMS_DX_Entered.ind (Rem_Add, Status) /Status =>	W-START
6	W-START	SSCY1S_Start_Subscriber.req (Rem_Add) => SET_WD, New_Stat := FALSE New_Data := FALSE, Sample_Data := 0 SSCY1S_Start_Subscriber.cnf (+) (Rem_Add)	RUN
7	W-START	SSCY1S_Stop_Subscriber.req (Rem_Add) => SSCY1S_Stop_Subscriber.cnf (-) (Rem_Add)	W-START

#	Current state	Event / condition => action	Next state
8	W-START	SSCY1S_Get_Publisher_Data.req (Rem_Add) => SSCY1S_Get_Publisher_Data.cnf (-) (Rem_Add)	W-START
9	W-START	DMPMS_DX_Broadcast.ind (Rem_Add, Data) => ignore	W-START
10	RUN	SSCY1S_Start_Subscriber.req (Rem_Add) => SSCY1S_Start_Subscriber.cnf (-) (Rem_Add)	RUN
11	RUN	SSCY1S_Stop_Subscriber.req (Rem_Add) => Stop Timer(WD) SSCY1S_Stop_Subscriber.cnf (+) (Rem_Add)	W-START
12	RUN	SSCY1S_Get_Publisher_Data.req (Rem_Add) => Data := Sample_Data, New_Flag := New_Data, New_Data := FALSE SSCY1S_Get_Publisher_Data.cnf (+) (Rem_Add, Data, New_Flag)	RUN
13	RUN	DMPMS_DX_Broadcast.ind (Rem_Add, Data) => New_Stat := TRUE, New_Data := TRUE, Sample_Data := Filter (Data) SSCY1S_New_Publisher_Data.ind (Rem_Add)	RUN
14	RUN	WDTimer expired /New_Stat => TRIG_WD, New_Stat := FALSE SSCY1S_Publisher_Active.ind (Rem_Add, Status)	RUN
15	RUN	WDTimer expired /!New_Stat => TRIG_WD, New_Stat := FALSE	RUN
16	any	DMPMS_DX_Entered.ind (Rem_Add, Status) /!Status => Stop Timer(WD), Publisher Address := 127, Status := FALSE, New_Stat := FALSE SSCY1S_Publisher_Active.ind (Rem_Add, Status)	POWER-ON

6.6.3.4 Functions

Table 451 contains the functions used by the SSCY1S, their arguments and their descriptions.

Table 451 – Functions used by the SSCY1S

Function name	Description
Filter(Data)	This function will return sample data (Octete-String) extracted from Data according to the offset and length from the DXB link attributes.
SET_WD	if (WD Enabled = TRUE) Start Timer (WD) else Stop Timer (WD)
TRIG_WD	if (WD Enabled = TRUE) Start Timer (WD)

6.6.4 MSRM2S

6.6.4.1 Primitive definitions

6.6.4.1.1 Primitives exchanged between MSRM2S and FSPMS

Table 452 shows the service primitives including their associated parameters issued by the FSPMS and received by the MSRM2S.

Table 452 – Primitives issued by FSPMS to MSRM2S

Primitive name	Source	Associated parameters	Functions
SInit MS2.req	FSPMS	(none)	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Reset.req	FSPMS	(none)	

Table 453 shows the service primitives including their associated parameters issued by the MSRM2S and received by the FSPMS.

Table 453 – Primitives issued by MSRM2S to FSPMS

Primitive name	Source	Associated parameters	Functions
SInit MS2.cnf	MSRM2S	(none)	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Reset.cnf	MSRM2S	(none)	
Fault.ind	MSRM2S	(none)	

6.6.4.1.2 Parameter of MSRM2S primitives

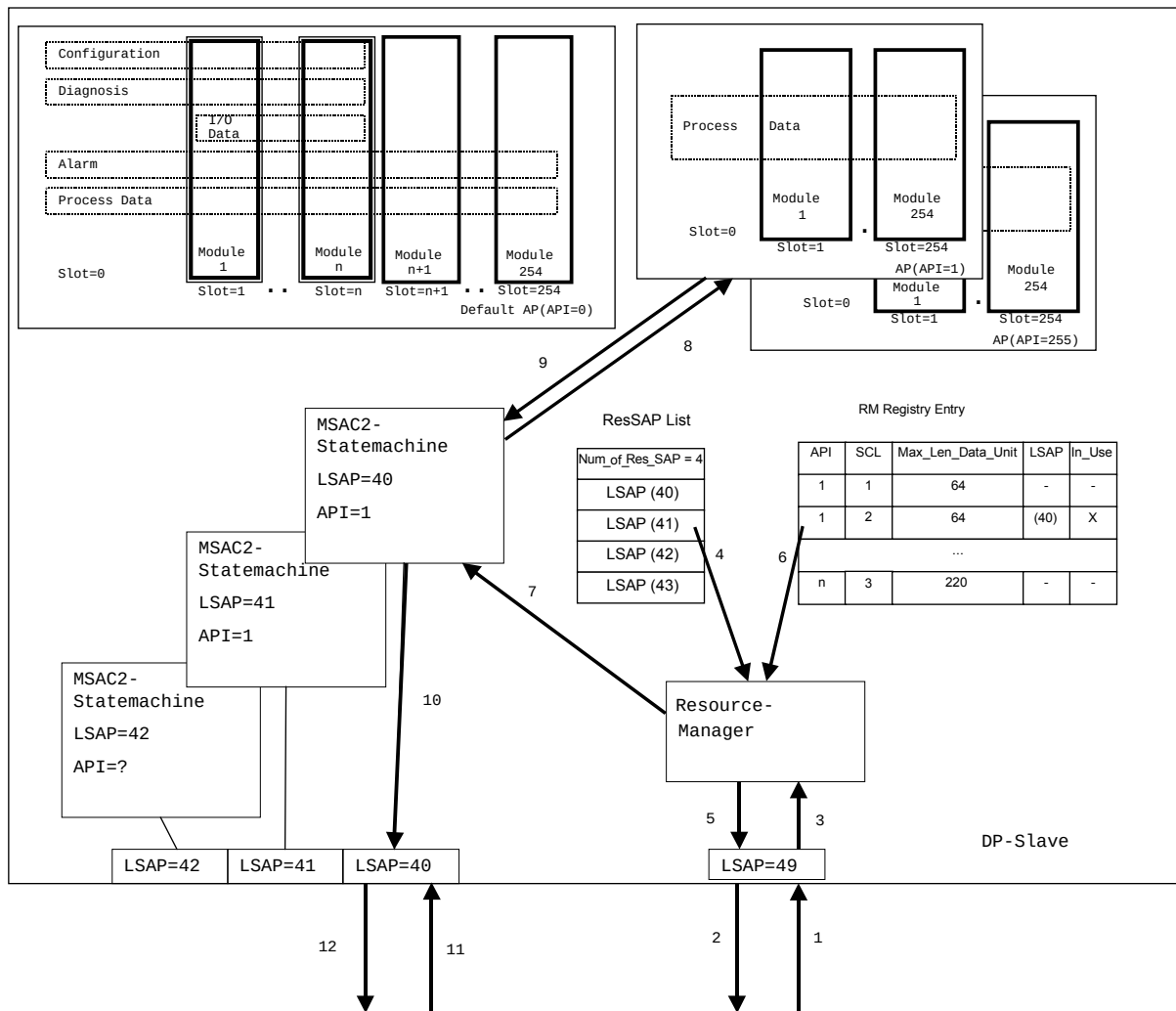
There are no parameters associated to the primitives.

6.6.4.2 State machine description

For acyclic communication the DSAPs are allocated dynamically by the so called Resource Manager (RM) of the DP-slave.

The Resource Manager (RM) co-ordinates the communication resources for MS2 AR of the DP-slave. The main features of the RM are described in the following sequence, see Figure 142:

- The DP-master requests a communication connection by means of the service Initiate (DSAP 49). If multiple application processes are used in the DP-slave it is possible to address these application processes through an Application Process Identifier (API).
- The RM of the DP-slave provides an unused SAP (SAP 0 - 48). The possible SAPs are defined locally in the Res_SAP_List.
- The RM responds the Initiate.req immediately with the RM_REQ-PDU containing the corresponding unused SAP.
- Further steps following the Initiate.req (Initiate.res, Read, Write, Abort) are handled by the corresponding MSAC2S State Machine.



- 1 Initiate-REQ-PDU (DSAP=49)
- 2 RM-REQ-PDU (Server_SAP=next SAP not In_Use)
- 3 Initiate-REQ-PDU
- 4 Search of the next SAP not In_Use
- 5 Reply_Update.req(L_sdu=RM-REQ-PDU with next SAP not In_Use)
- 6 Check RM_Registry_Entry for API, In_Use=FALSE
- 7 MSAC2S_Initiate.req (Server_SAP=40)
- 8 MSAC2S_Initiate.ind (Server_SAP=40)
- 9 MSAC2S_Initiate.res (Server_SAP=40)
- 10 Reply_Update.req(L_sdu=Initiate-RES-PDU)
- 11 Polling with (L_sdu.len=0)
- 12 Initiate-RES-PDU

Figure 142 – Example for connection establishment on MS2(server-side)

The Resource Manager is responsible for the allocation of SAPs to incoming requests for a MS2 AR communication.

If a Res_SAP is available when an Initiate-REQ-PDU is received on the RM_SAP, the Slave responds with a RM-REQ-PDU.

The only PDU which is accepted is an Initiate-REQ-PDU. The Resource Manager checks for an unused API/SCL in the RM_Registry and passes the Initiate-PDU with an Initiate to a MSAC2S State Machine.

If none of the entries in the RM_Registry fits to the requested API/SCL, the last SAP sent in the RM-REQ-PDU is temporarily used to send an Abort-REQ-PDU to the Master.

If no Res_SAP is available then the RM_SAP is deactivated.

If an MSAC2S State Machine indicates with anAbort.req that a connection has been closed, the corresponding entry in the RM_Registry is marked as unused and the corresponding DLSAP is released and can be used for further requests.

Local variables of the MSRM2S

List of RM entries

Figure 143 shows the structure of the RM entries as they are listed in the RM_Registry.

static part			dynamic part	
API	SCL	Max_Len_Data_Unit	SAP	In_Use

Figure 143 – Structure of RM entries in the RM_Registry

List of SSAPs

SSAP_Entry

(Unsigned8)

0..48

Stored_Req_Add,
Stored_Send_Timeout,
Stored_Features_Supported,
Stored_Profile_Features_Supported,
Stored_Profile_Ident_Number,
Stored_Add_Addr_Param

Storage of values of the last Initiate function (see part DP-Services).

RM_SAP

(constant)

Set to 49

Last_SAP

(Unsigned8)

0..48

SAP for previous established connection.

Server_SAP

(Unsigned8)

0..48

SAP which will be allocated to the next connection.

Macros

INITIATE-REQ-PDU-LEN

(

48 {64 for linking device}

)

RMREQ-PDU-LEN

(

4

)

6.6.4.3 MSRM2S state table

Table 454 contains the complete description of the MSRM2S state machine.

Table 454 – MSRM2S state table

#	Current state	Event / condition => action	Next state
1	POWER-ON	MSRM2S_SInit.req =>RM_SAP:=49 Res_SAP_entry_Index:=1 Server_SAP:=Get_from_List_of_SSAPs() DMPMS_RSAP_ACTIVATE.req(SSAP=RM_SAP, Access=All, L_sdu_length_list=(<INITIATE-REQ-PDU-LEN>,0,<RMREQ-PDU-LEN>,0) Indication_Mode=Data)	WAIT-ACT
2	WAIT-ACT	DMPMS_RSAP_ACTIVATE.cnf(SSAP,M_status) /M_status=OK =>RM_SAP:=49 Res_SAP_entry_Index:=1 Server_SAP:=Get_from_List_of_SSAPs() DMPMS_REPLY_UPDATE.req(SSAP=RM_SAP, L_sdu=RM-REQ-PDU, Serv_class=Low, Transmit=Single)	INIT-WUPD
3	WAIT-ACT	DMPMS_RSAP_ACTIVATE.cnf(SSAP,M_status) /M_status=NO/IV =>MSRM2S_Fault.ind	POWER-ON
4	INIT-WUPD	DMPMS_REPLY_UPDATE.cnf (SSAP=RM_SAP, Serv_class=Low, L_status) /L_status=OK =>MSRM2S_SInit.cnf	OPEN
5	INIT-WUPD	DMPMS_REPLY_UPDATE.cnf (SSAP=RM_SAP, Serv_class=Low, L_status) /L_status= LS/IV/LR =>MSRM2S_Fault.ind	POWER-ON
6	OPEN	DMPMS_DATA_REPLY.ind(SSAP, DSAP=RM_SAP, Loc_add, Rem_add,L_sdu, Serv_class=Low, Update_status) /Update_status=LO && L_sdu=Initiate-REQ-PDU && List_of_SSAPs<>empty =>Last_SAP := Server_SAP Stored_Req_Add:=Loc_add, Stored_Send_Timeout:= Send_Timeout Stored_Features_Supported:=Features_Supported Stored_Profile_Features_Supported:= Profile_Features_Supported Stored_Profile_Ident_Number:=Profile_Ident_Number Stored_Add_Addr_Param:= Add_Addr_Param Server_SAP:=Get_from_List_of_SSAPs() DMPMS_REPLY_UPDATE.req(SSAP=RM_SAP, L_sdu=RM-REQ-PDU, Serv_class=Low, Transmit=Single)	OW-UPDATE
7	OPEN	DMPMS_DATA_REPLY.ind(SSAP, DSAP=RM_SAP, Loc_add, Rem_add,L_sdu, Serv_class=Low, Update_status) /Update_status=LO && L_sdu=Initiate-REQ-PDU && List_of_SSAPs=empty =>Stored_Req_Add:=Loc_add, Stored_Send_Timeout:= Send_Timeout Stored_Features_Supported:=Features_Supported Stored_Profile_Features_Supported:= Profile_Features_Supported Stored_Profile_Ident_Number:=Profile_Ident_Number Stored_Add_Addr_Param:= Add_Addr_Param DMPMS_SAP_DEACTIVATE.req(SSAP=RM_SAP)	OW-DEACT
8	OPEN	DMPMS_DATA_REPLY.ind(SSAP, DSAP=RM_SAP, Loc_add, Rem_add,L_sdu, Serv_class=Low, Update_status) /Update_status=LO && (L_sdu<>Initiate-REQ-PDU) =>ignore Lsdu DMPMS_REPLY_UPDATE.req(SSAP=RM_SAP, L_sdu=RM-REQ-PDU, Serv_class=Low, Transmit=Single)	INIT-WUPD

Table 454 (continued)

#	Current state	Event / condition => action	Next state
9	OPEN	DMPMS_DATA_REPLY.ind(SSAP, DSAP=RM_SAP, Loc_add, Rem_add, L_sdu, Serv_class=Low, Update_status) /Update_status=NO =>ignore Lsdu	OPEN
10	OPEN	MSRM2S_Reset.req =>for (all CRL entries with AR-Type=MS2) MSAC2S_Reset.req(Res_SAP=SSAP)) endfor	RESET-AC2
11	OPEN	MSAC2S_Closed.ind(Res_SAP) =>Index := Search_in_List_of_RM_Entries(SAP:=Res_SAP, In_Use:=True)	AW-SEARCH
12	AW-SEARCH	/Index <> 0 =>List_of_RM_Entries.In_Use[Index]:=False Put_to_List_of_SSAPs(Stored_Res_SAP)	OPEN
13	AW-SEARCH	/Index = 0 =>MSRM2S_Fault.ind	POWER-ON
14	OW-UPDATE	DMPMS_REPLY_UPDATE.cnf (SSAP=RM_SAP, Serv_class=Low, L_status) /L_status=OK =>Index := Search_in_List_of_RM_Entries(API:=Add_Addr_Param.D_Addr.API, SCL:= Add_Addr_Param.D_Addr.SCL, In_Use:=False)	OW-SEARCH
15	OW-UPDATE	DMPMS_REPLY_UPDATE.cnf (SSAP=RM_SAP, Serv_class=Low, L_status) /L_status= LS/IV/LR =>MSRM2S_Fault.ind	POWER-ON
16	OW-SEARCH	/Index <> 0 =>List_of_RM_Entries.In_Use[Index]:=True List_of_RM_Entries.SAP[Index]:=Last_SAP Max_Len_Data_Unit:=List_of_RM_Entries.Max_Len_Data_Unit[Index] Send_Timeout:=Max(Stored_Send_Timeout, Stored_Min_Send_Timeout) MSAC2S_Initiate.req(Res_SAP=Last_SAP, Req_Add=Stored_Req_Add, Max_Len_Data_Unit, Send_Timeout, Features_Supported= Stored_Features_Supported, Profile_Features_Supported= Stored_Profile_Features_Supported, Profile_Ident_Number= Stored_Profile_Ident_Number, Add_Addr_Param= Stored_Add_Addr_Param)	OPEN
17	OW-SEARCH	/Index= 0 =>Send_Timeout:=Max(Stored_Send_Timeout, Stored_Min_Send_Timeout) MSAC2S_Abort.req(Res_SAP=Last_SAP, Req_Add, Subnet=NO, Instance=MSAC2, Reason_Code=ABT_IA, Send_Timeout)	OPEN
18	OW-DEACT	DMPMS_SAP_DEACTIVATE.cnf(SSAP,M_status) /M_status=OK =>Index := Search_in_List_of_RM_Entries(API:=Add_Addr_Param.D_Addr.API, SCL:= Add_Addr_Param.D_Addr.SCL, In_Use:=False)	CW-SEARCH
19	OW-DEACT	DMPMS_SAP_DEACTIVATE.cnf(SSAP,M_status) /M_status=NO/IV =>MSRM2S_Fault.ind	POWER-ON
20	CW-SEARCH	/Index <> 0 =>List_of_RM_Entries.In_Use[Index]:=True List_of_RM_Entries.SAP[Index]:=Last_SAP Max_Len_Data_Unit:=List_of_RM_Entries.Max_Len_Data_Unit[Index] Send_Timeout:=Max(Stored_Send_Timeout, Stored_Min_Send_Timeout) MSAC2S_Initiate.req(Res_SAP=Last_SAP, Req_Add=Stored_Req_Add, Max_Len_Data_Unit, Send_Timeout, Features_Supported= Stored_Features_Supported, Profile_Features_Supported= Stored_Profile_Features_Supported, Profile_Ident_Number= Stored_Profile_Ident_Number, Add_Addr_Param= Stored_Add_Addr_Param)	CLOSED
21	CW-SEARCH	/Index = 0 =>Send_Timeout:=Max(Stored_Send_Timeout, Stored_Min_Send_Timeout) MSAC2S_Abort.req(Res_SAP=Last_SAP, Req_Add, Subnet=NO, Instance=MSAC2, Reason_Code=ABT_IA, Send_Timeout)	CLOSED
22	CLOSED	MSRM2S_Reset.req =>for (all CRL entries with AR-Type=MS2) MSAC2S_Reset.req(Res_SAP=SSAP)) endfor	RESET-AC2

Table 454 (continued)

#	Current state	Event / condition => action	Next state
23	CLOSED	MSAC2S_Closed.ind(Res_SAP) =>Stored_Res_SAP:=Res_SAP Index := Search_in_List_of_RM_Entries(SAP:=Res_SAP, In_Use:=True)	REOW-SEARCH
24	REOW-SEARCH	/Index <> 0 =>List_of_RM_Entries.In_Use[Index]:=False Put_to_List_of_SSAPs(Stored_Res_SAP)	WAIT-ACT
25	REOW-SEARCH	/Index = 0 =>MSRM2S_Fault.ind	POWER-ON
26	RESET-AC2	for (all CRL entries with AR-Type=MS2) MSAC2S_Reset.cnf(Res_SAP=SSAP) endfor /M_status=OK =>MSRM2S_Reset.cnf	POWER-ON
27	ANY-STATE	MSAC2S_Fault.ind(Res_SAP) =>MSRM2S_Fault.ind	POWER-ON
28	ANY-STATE	unexpected DL reaction =>MSRM2S_Fault.ind	POWER-ON

6.6.5 MSAC2S

6.6.5.1 Primitive definitions

6.6.5.1.1 Primitives exchanged between MSAC2S and FSPMS

Table 455 shows the service primitives including their associated parameters issued by the FSPMS and received by the MSAC2S.

Table 455 – Primitives issued by FSPMS to MSAC2S

Primitive name	Source	Associated parameters	Functions
Initiate.rsp(+)	FSPMS	Res SAP, Max Len Data Unit, Features Supported, Profile Features Supported, Profile Ident Number, Add Addr Param	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Initiate.rsp(-)	FSPMS	Res SAP, Error Decode, Error Code 1 Error Code 2	
Read.rsp(+)	FSPMS	Res SAP, Length, Data	
Read.rsp(-)	FSPMS	Res SAP, Error Decode, Error Code 1 Error Code 2	
Write.rsp(+)	FSPMS	Res SAP, Length	
Write.rsp(-)	FSPMS	Res SAP, Error Decode, Error Code 1 Error Code 2	
Data Transport.rsp(+)	FSPMS	Res SAP, Length, Data	
Data Transport.rsp(-)	FSPMS	Res SAP, Error Decode, Error Code 1 Error Code 2	

Primitive name	Source	Associated parameters	Functions
Abort.req	FSPMS	Res SAP, Subnet, Instance, Reason Code	

Table 456 shows the service primitives including their associated parameters issued by the MSAC2S and received by the FSPMS.

Table 456 – Primitives issued by MSAC2S to FSPMS

Primitive name	Source	Associated parameters	Functions
Initiate.ind	MSAC2S	Res SAP, Features Supported, Profile Features Supported, Profile Ident Number, Add Addr Param	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Abort.ind	MSAC2S	Res SAP, Locally Generated, Subnet, Instance, Reason Code, Additional Detail	
Read.ind	MSAC2S	Res SAP, Slot Number, Index, Length	
Write.ind	MSAC2S	Res SAP, Slot Number, Index, Length, Data	
Data Transport.ind	MSAC2S	Res SAP, Slot Number, Index, Length, Data	

6.6.5.1.2 Primitives exchanged between MSAC2S and MSRM2S

Table 457 shows the service primitives including their associated parameters issued by MSAC2S and received by the MSRM2S.

Table 457 – Primitives issued by MSRM2S to MSAC2S

Primitive name	Source	Associated parameters	Functions
Reset.req	MSRM2S	Res SAP	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Initiate.req	MSRM2S	Res SAP, Features Supported, Profile Features Supported, Profile Ident Number, Add Addr Param	
RM2_Abort.req	MSRM2S	Res SAP, Subnet, Instance, Reason Code	

Table 458 shows the service primitives including their associated parameters issued by MSAC2S and received by the MSRM2S.

Table 458 – Primitives issued by MSAC2S to MSRM2S

Primitive name	Source	Associated parameters	Functions
Reset.cnf	MSAC2S	Res SAP	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Fault.ind	MSAC2S	Res SAP	
Closed.ind	MSAC2S	Res SAP	Signals the Termination of a MS2 Communication

6.6.5.1.3 Parameters of MSAC2S primitives

The parameters used with the primitives exchanged between the MSAC2S and FSPMS,MSRM2S are described in FAL Service Definition Type 3 Fieldbus IEC 61158-5. Additional parameters are described in Table 459.

Table 459 – Parameter used with primitives exchanged with MSAC2S

Parameter name	Description
Res SAP	This parameter conveys the SSAP for addressing of the various MS2 AR.

6.6.5.2 State machine description

In the state CLOSED the Initiate service is expected from the Resource Manager. Then the corresponding DLSAP is activated and the User gets the Initiate.ind. The User shall generate a MSAC2S_Initiate.rsp and the connection is established after the Master has fetched the response.

In the OPEN-state valid services (see macro <VALID_SERVICE>) are allowed. Only one request from the DP-master is processed simultaneously, otherwise the connection is aborted.

Both the DP-master and the DP-slave can close the connection using the Abort. The remote User gets an Abort.ind.

The connection monitoring is based on the U-Timer, F-Timer and I-Timer (see 6.3.9)

Local variables of the MSAC2S

Server_SAP

(Unsigned8)

The used SAP for the MS2 connection.

U-Timer

(Unsigned16)

The U-Timer controls the reaction of the User after delivering an indication. If the U-Timer expires an Idle-PDU is created and transferred to the local DLL. The U-Timer is stopped when the User provides the response.

F-Timer

(Unsigned16)

The F-Timer controls the Master fetching the previously provided PDU. If the F-Timer expires an Abort is generated, transferred to the local DLL and the F-Timer starts again. The F-Timer is stopped as soon as the Master fetches the PDU.

I-Timer

(Unsigned16)

The I-Timer controls the next activity of the client after the Master has fetched the previous PDU. When the I-Timer expires an Abort is generated, transferred to the local DLL and the F-Timer starts. The I-Timer is stopped when the Master has sent the next request.

NOTE Only one of the timers described above is running at the same time.

Service_Header

(Unsigned8)

The actual service header to check whether the User generates the correct response.

Return-State

INITIATE, VALID-SERVICE

This variable is used to store the state where to return after an idle cycle has been processed.

Go_Abort

(Boolean)

Local flag that indicates an abort reaction after finishing the present service response.

Service_Buffer

(Octet-String[244])

Local buffer that stores one complete Response-PDU (Initiate.rsp or <VALID_SERVICE>.rsp) during an outstanding Idle-RES-PDU.

Stored_Slot_Number**Stored_Index**

Stored values of the parameters of Read/Write-services

Stored_Instance**Stored_Reason**

Stored values of the parameters of MSAC2S_Abort-service

INITIATE-REQ-PDU-LEN

(local constant)

PDU-size (the 4 Octet Header is excluded) set to 48 (64 respectively for links)

ABORT_PDU_LEN

(local constant)

Length of an Abort-REQ-PDU = 4

Macros**Read-REQ-PDU**

This function will return True if the corresponding L_sdu is a valid Read-REQ-PDU

Write-REQ-PDU

This function will return True if the corresponding L_sdu is a valid Write-REQ-PDU

Data_Transport-REQ-PDU

This function will return True if the corresponding L_sdu is a valid Data_Transport-REQ-PDU

Abort-REQ-PDU

This function will return True if the corresponding L_sdu is a valid Abort-REQ-PDU

Idle-REQ-PDU

This function will return True if the corresponding L_sdu is a valid Idle-REQ-PDU

STORE_ABORT_PARAMETER

(

Stored-Instance=Instance

Stored-Reason_Code=Reason_Code

)

<VALID-SERVICE>

<

Service_Header=0x5E: Read

Service_Header=0x5F: Write

Service_Header=0x51: Data_Transport

>

VALID-SERVICE-REQ-PDU

```
(
(Read-REQ-PDU || Write-REQ-PDU || OR Data_Transport-REQ-PDU)
&& L_sdu.len >= 4
&& (L_sdu[1] = 0x5e || (L_sdu.len >= Length+4
&& Max_DLSDU_length_ind_low-4 >= Length)
)
```

6.6.5.3 MSAC2S state table

Table 460 contains the complete description of the MSAC2S state machine.

Table 460 – MSAC2S state table

#	Current state	Event / condition => action	Next state
1	POWER-ON	=> Server_SAP:=Res_SAP	CLOSED
2	CLOSED	MSAC2S_Initiate.req(Res_SAP, Req_Add, Max_Len_Data_Unit, Send_Timeout, Features_Supported, Profile_Features_Supported, Profile_Ident_Number, Add_Addr_Param) => Service_Header:=0x57 F-Timer:=Send_Timeout U-Timer:=Send_Timeout I-Timer:=2*Send_Timeout Stored_Max_Len_Data_Unit:= Max(Max_Len_Data_Unit, INITIATE-REQ-PDU-LEN) clear Service_Buffer Go_Abort:=FALSE MSAC2_Initiate.ind(Req_Add, Res_SAP, Features_Supported, Profile_Features_Supported, Profile_Ident_Number, Add_Addr_Param) DMPMS_RSAP_ACTIVATE.req(SSAP=Server_SAP, Access=Req_Add, L_sdu_length_list=(Stored_Max_Len_Data_Unit+4,0, Stored_Max_Len_Data_Unit+4,0) , Indication_Mode=Data)	SET-ACC-POS
3	CLOSED	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add,L_sdu, Serv_class=Low, Update_status) => ignore	CLOSED
4	CLOSED	MSAC2S_RM2_Abort.req(Res_SAP, Req_Add, Subnet, Instance, Reason_Code, Send_Timeout) => F-Timer:=Send_Timeout STORE_ABORT_PARAMETER DMPMS_RSAP_ACTIVATE.req(SSAP=Server_SAP, Access=Req_add, L_sdu_length_list=(ABORT_PDU_LEN,0, ABORT_PDU_LEN,0) Indication_Mode=Data)	SET-ACC-FE
5	CLOSED	MSAC2S_XXX.rsp(+/-)(Res_SAP) => Reason_Code:=ABT_SE MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	CLOSED
6	CLOSED	MSAC2S_Initiate.rsp(+/-)(Res_SAP) => Reason_Code:=ABT_SE MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	CLOSED
7	CLOSED	MSAC2S_Abort.req(Res_SAP, Subnet, Instance, Reason_Code) => ignore	CLOSED
8	SET-ACC-POS	DMPMS_RSAP_ACTIVATE.cnf(SSAP=Server_SAP,M_status) /M_status = OK Start U-Timer	INITI-WRES
9	SET-ACC-POS	DMPMS_RSAP_ACTIVATE.cnf(SSAP=Server_SAP,M_status) /M_status=NO/IV => MSAC2S_Fault.ind	POWER-ON

Table 460 (continued)

#	Current state	Event / condition => action	Next state
10	SET-ACCFE	DMPMS_RSAP_ACTIVATE.cnf(SSAP=Server_SAP,M_status) /M_status = OK => L_sdu:=Abort-REQ-PDU(with stored parameter) Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu, Serv_class=Low, Transmit=Single)	SABORT-WUPD
11	SET-ACCFE	DMPMS_RSAP_ACTIVATE.cnf(SSAP=Server_SAP,M_status) /M_status=NO/IV => MSAC2S_Fault.ind	POWER-ON
12	INITI-WRES	MSAC2S_Initiate.rsp(+)(Res_SAP) => L_sdu:=Initiate-RES-PDU(With Stored_MAx_Len_Data_Unit) Stop U-Timer Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu, Serv_class=Low, Transmit=Single)	INITI-WUPD
13	INITI-WRES	MSAC2S_Initiate.rsp(-)(Res_SAP) => L_sdu:=Initiate-NRS-PDU Stop U-Timer Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu, Serv_class=Low, Transmit=Single)	SABORT-WUPD
14	INITI-WRES	MSAC2S_XXX.rsp(+/-)(Res_SAP) => Reason_Code:=ABT_SE Stop U-Timer Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu=Abort-REQ-PDU(Subnet=NO, Instance=MSAC2, Reason_Code), Serv_class=Low, Transmit=Single) MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	SABORT-WUPD
15	INITI-WRES	MSAC2S_Abort.req(Res_SAP, Subnet, Instance, Reason_Code) => L_sdu:=Abort-REQ-PDU Stop U-Timer Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu, Serv_class=Low, Transmit=Single)	SABORT-WUPD
16	INITI-WRES	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add,L_sdu, Serv_class=Low, Update_status) /Abort-REQ-PDU => Stop U-Timer DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=FALSE, (Subnet, Instance, Reason_Code) from Abort-REQ-PDU)	WAIT-DEACT
17	INITI-WRES	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add,L_sdu, Serv_class=Low, Update_status) /NOT(Abort-REQ-PDU) => Reason_Code:=ABT_FE Stop U-Timer Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu=Abort-REQ-PDU(Subnet=NO, Instance=MSAC2, Reason_Code), Serv_class=Low, Transmit=Single) MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	SABORT-WUPD

Table 460 (continued)

#	Current state	Event / condition => action	Next state
18	INITI-WRES	U-Timer expired => L_sdu:=Idle-REQ-PDU Return-State:=INITIATE Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu, Serv_class=Low, Transmit=Single)	IDLE-WUPD
19	INITI-WUPD	DMPMS_REPLY_UPDATE.cnf(SSAP=Server_SAP, Serv_class=Low, L_status) /L_status=OK => ignore	INITI-SRES
20	INITI-WUPD	DMPMS_REPLY_UPDATE.cnf(SSAP=Server_SAP, Serv_class=Low, L_status) /L_status= LS/IV/LR => Stop F-Timer Stop I-Timer MSAC2S_Fault.ind	POWER-ON
21	INITI-SRES	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add, L_sdu, Serv_class=Low, Update_status) /Update_status=LO && L_sdu.len=0 && Go_Abort = FALSE => Stop F-Timer Start I-Timer	OPEN
22	INITI-SRES	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add, L_sdu, Serv_class=Low, Update_status) /Update_status=LO && L_sdu.len=0 && Go_Abort = TRUE => L_sdu:=Abort-REQ-PDU(With Stored Parameter) Stop F-Timer Start I-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu, Serv_class=Low, Transmit=Single)	SABORT-WUPD
23	INITI-SRES	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add, L_sdu, Serv_class=Low, Update_status) /Update_status=LO/NO && L_sdu.len<>0 && Abort-REQ-PDU => Stop F-Timer DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=FALSE, (Subnet, Instance, Reason_Code) from Abort-REQ-PDU)	WAIT-DEACT
24	INITI-SRES	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add, L_sdu, Serv_class=Low, Update_status) /Update_status=LO && L_sdu.len<>0 && NOT(Abort-REQ-PDU) => Reason_Code:=ABT_FE Stop F-Timer Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu=Abort-REQ-PDU(Subnet=NO, Instance=MSAC2, Reason_Code), Serv_class=Low, Transmit=Single) MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	SABORT-WUPD
25	INITI-SRES	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add, L_sdu, Serv_class=Low, Update_status) /Update_status=NO && L_sdu.len<>0 && NOT(Abort-REQ-PDU) => Go_Abort:=TRUE STORE_ABORT_PARAMETER Reason_Code:=ABT_FE MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	INITI-SRES

Table 460 (continued)

#	Current state	Event / condition => action	Next state
26	INITI-SRES	MSAC2S_XXX.rsp(+/-)(Res_SAP) => Go_Abort:=TRUE STORE_ABORT_PARAMETER Reason_Code:=ABT_SE MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	INITI-SRES
27	INITI-SRES	MSAC2S_Initiate.rsp(+/-)(Res_SAP) => Go_Abort:=TRUE STORE_ABORT_PARAMETER Reason_Code:=ABT_SE MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	INITI-SRES
28	INITI-SRES	MSAC2S_Abort.req(Res_SAP, Subnet, Instance, Reason_Code) => Go_Abort:=TRUE STORE_ABORT_PARAMETER	INITI-SRES
29	INITI-SRES	F-Timer expired => Reason_Code:=ABT_TO Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu=Abort-REQ-PDU(Subnet=NO, Instance=MSAC2, Reason_Code), Serv_class=Low, Transmit=Single) MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	SABORT-WUPD
30	IDLE-WUPD	DMPMS_REPLY_UPDATE.cnf(SSAP=Server_SAP, Serv_class=Low, L_status) /Status=OK	IDLE-SREQ
31	IDLE-WUPD	DMPMS_REPLY_UPDATE.cnf(SSAP=Server_SAP, Serv_class=Low, L_status) /L_status= LS/IV/LR => Stop F-Timer Stop I-Timer MSAC2S_Fault.ind	POWER-ON
32	IDLE-SREQ	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add,L_sdu, Serv_class=Low, Update_status) /Update_status=LO && L_sdu.len=0 && Go_Abort = FALSE => Stop F-Timer Start I-Timer	W-IDLE-CON
33	IDLE-SREQ	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add,L_sdu, Serv_class=Low, Update_status) /Update_status=LO && L_sdu.len=0 && Go_Abort = TRUE => L_sdu:=Abort-REQ-PDU(With Stored Parameter) Stop F-Timer Start I-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu, Serv_class=Low, Transmit=Single)	SABORT-WUPD
34	IDLE-SREQ	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add,L_sdu, Serv_class=Low, Update_status) /Update_status=LO/NO && L_sdu.len<>0 && Abort-REQ-PDU DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC2_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=FALSE, (Subnet, Instance, Reason_Code) from Abort-REQ-PDU) Stop F-Timer	WAIT-DEACT

Table 460 (continued)

#	Current state	Event / condition => action	Next state
35	IDLE-SREQ	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add, L_sdu, Serv_class=Low, Update_status) /Update_status=LO && L_sdu.len<>0 && NOT(Abort-REQ-PDU) => Reason_Code:=ABT_FE Stop F-Timer Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu=Abort-REQ-PDU(Subnet=NO, Instance=MSAC2, Reason_Code), Serv_class=Low, Transmit=Single) MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	SABORT-WUPD
36	IDLE-SREQ	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add, L_sdu, Serv_class=Low, Update_status) /Update_status=NO && L_sdu.len<>0 && NOT(Abort-REQ-PDU) => Go_Abort:=TRUE STORE_ABORT_PARAMETER Reason_Code:=ABT_FE MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	IDLE-SREQ
37	IDLE-SREQ	MSAC2S_XXX.rsp(+)(Res_SAP) /Service-Response fits stored Service_Header && Stored_Max_Len_Data_Unit >= Length && Service_Buffer empty => Service_Buffer:= <VALID-SERVICE>-RES-PDU(with Stored_Slot_Number, Stored_Index)	IDLE-SREQ
38	IDLE-SREQ	MSAC2S_XXX.rsp(-)(Res_SAP) /Service-Response fits stored Service_Header && Stored_Max_Len_Data_Unit >= Length && Service_Buffer empty => Service_Buffer:= <VALID-SERVICE>-NRS-PDU(with Stored_Slot_Number, Stored_Index)	IDLE-SREQ
39	IDLE-SREQ	MSAC2S_Initiate.rsp(+)(Res_SAP) /Service-Response fits stored Service_Header && Service_Buffer empty => Service_Buffer:= Initiate-RES-PDU	IDLE-SREQ
40	IDLE-SREQ	MSAC2S_Initiate.rsp(-)(Res_SAP) /Service-Response fits stored Service_Header && Service_Buffer empty => Service_Buffer:= Initiate-NRS-PDU	IDLE-SREQ
41	IDLE-SREQ	MSAC2S_XXX.rsp(+/-)(Res_SAP) /(NOT(Service-Response fits stored Service_Header && Stored_Max_Len_Data_Unit >= Length) NOT(Service_Buffer empty)) => Go_Abort:=TRUE STORE_ABORT_PARAMETER Reason_Code:=ABT_RE MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	IDLE-SREQ
42	IDLE-SREQ	MSAC2S_Initiate.rsp(+/-)(Res_SAP) /(NOT(Service-Response fits stored Service_Header) NOT(Service_Buffer empty)) => Go_Abort:=TRUE STORE_ABORT_PARAMETER Reason_Code:=ABT_RE MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	IDLE-SREQ
43	IDLE-SREQ	MSAC2S_Abort.req(Res_SAP, Subnet, Instance, Reason_Code) => Go_Abort:=TRUE STORE_ABORT_PARAMETER	IDLE-SREQ

Table 460 (continued)

#	Current state	Event / condition => action	Next state
44	IDLE-SREQ	F-Timer expired => Reason_Code:=ABT_TO Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu=Abort-REQ-PDU(Subnet=NO, Instance=MSAC2, Reason_Code), Serv_class=Low, Transmit=Single) MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	SABORT-WUPD
45	W-IDLE-CON	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add, L_sdu, Serv_class=Low, Update_status) /Idle-RES-PDU && Return-State=INITIATE && Go_Abort=FALSE && (Service_Buffer empty) => Stop I-Timer Start U-Timer	INITI-WRES
46	W-IDLE-CON	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add, L_sdu, Serv_class=Low, Update_status) /Idle-RES-PDU && Return-State=INITIATE && Go_Abort=FALSE && NOT(Service_Buffer empty) && Service_Buffer=Initiate-NRS-PDU => L_sdu:=Stored L_sdu from Service_Buffer clear Service_Buffer Stop I-Timer Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu, Serv_class=Low, Transmit=Single)	SABORT-WUPD
47	W-IDLE-CON	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add, L_sdu, Serv_class=Low, Update_status) /Idle-RES-PDU && Return-State=INITIATE && Go_Abort=FALSE && NOT(Service_Buffer empty) && Service_Buffer=Initiate-RES-PDU => L_sdu:=Stored L_sdu from Service_Buffer clear Service_Buffer Stop I-Timer Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu, Serv_class=Low, Transmit=Single)	INITI-WUPD
48	W-IDLE-CON	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add, L_sdu, Serv_class=Low, Update_status) /Idle-RES-PDU && Return-State=VALID-SERVICE && Go_Abort=FALSE && (Service_Buffer empty) => Stop I-Timer Start U-Timer	VS-WRES
49	W-IDLE-CON	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add, L_sdu, Serv_class=Low, Update_status) /Idle-RES-PDU && Return-State=VALID-SERVICE && Go_Abort=FALSE && NOT(Service_Buffer empty) => L_sdu:=Stored L_sdu from Service_Buffer clear Service_Buffer Stop I-Timer Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu, Serv_class=Low, Transmit=Single)	VS-WUPD
50	W-IDLE-CON	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add, L_sdu, Serv_class=Low, Update_status) /Idle-RES-PDU && Go_Abort=TRUE => L_sdu:=Abort-REQ-PDU(with stored Parameter) DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu, Serv_class=Low, Transmit=Single)	SABORT-WUPD

Table 460 (continued)

#	Current state	Event / condition => action	Next state
51	W-IDLE-CON	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add, L_sdu, Serv_class=Low, Update_status) /Abort-REQ-PDU => Stop I-Timer DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=FALSE, (Subnet, Instance, Reason_Code) from Abort-REQ-PDU)	WAIT-DEACT
52	W-IDLE-CON	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add, L_sdu, Serv_class=Low, Update_status) /NOT(Abort-REQ-PDU Idle-RES-PDU) => Reason_Code:=ABT_FE Stop I-Timer Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu=Abort-REQ-PDU(Subnet=NO, Instance=MSAC2, Reason_Code), Serv_class=Low, Transmit=Single) MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	SABORT-WUPD
53	W-IDLE-CON	MSAC2S_XXX.rsp(+)(Res_SAP) /Service-Response fits stored Service_Header && Stored_Max_Len_Data_Unit >= Length && Service_Buffer empty => Service_Buffer:= <VALID-SERVICE>-RES-PDU(with Stored_Slot_Number, Stored_Index)	W-IDLE-CON
54	W-IDLE-CON	MSAC2S_XXX.rsp(-)(Res_SAP) /Service-Response fits stored Service_Header && Stored_Max_Len_Data_Unit >= Length && Service_Buffer empty => Service_Buffer:= <VALID-SERVICE>-NRS-PDU (with Stored_Slot_Number, Stored_Index)	W-IDLE-CON
55	W-IDLE-CON	MSAC2S_XXX.rsp(+/-)(Res_SAP) /(NOT(Service-Response fits stored Service_Header && Stored_Max_Len_Data_Unit >= Length) NOT(Service_Buffer empty)) => Go_Abort:=TRUE STORE_ABORT_PARAMETER Reason_Code:=ABT_RE MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	W-IDLE-CON
56	W-IDLE-CON	MSAC2S_Initiate.rsp(+/-)(Res_SAP) /(NOT(Service-Response fits stored Service_Header) NOT(Service_Buffer empty)) => Go_Abort:=TRUE STORE_ABORT_PARAMETER Reason_Code:=ABT_RE MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	W-IDLE-CON
57	W-IDLE-CON	MSAC2S_Abort.req(Res_SAP, Subnet, Instance, Reason_Code) => Go_Abort:=TRUE STORE_ABORT_PARAMETER	W-IDLE-CON
58	W-IDLE-CON	I-Timer expired => Reason_Code:=ABT_TO Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu=Abort-REQ-PDU(Subnet=NO, Instance=MSAC2, Reason_Code), Serv_class=Low, Transmit=Single) MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	SABORT-WUPD

Table 460 (continued)

#	Current state	Event / condition => action	Next state
59	SABORT-WUPD	DMPMS_REPLY_UPDATE.cnf(SSAP=Server_SAP, Serv_class=Low, L_status) /Status=OK	SABORT-SRES
60	SABORT-WUPD	DMPMS_REPLY_UPDATE.cnf(SSAP=Server_SAP, Serv_class=Low, L_status) /L_status= LS/IV/LR => Stop F-Timer Stop I-Timer MSAC2S_Fault.ind	POWER-ON
61	SABORT-SRES	MSAC2S_Abort.req(Res_SAP, Subnet, Instance, Reason_Code) => ignore	SABORT-SRES
62	SABORT-SRES	MSAC2S_XXX.rsp(+/-)(Res_SAP) => Reason_Code:=ABT_SE MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	SABORT-SRES
63	SABORT-SRES	MSAC2S_Initiate.rsp(+/-)(Res_SAP) => Reason_Code:=ABT_SE MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	SABORT-SRES
64	SABORT-SRES	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add,L_sdu, Serv_class=Low, Update_status) /Update_status=LO && L_sdu.len=0 => Stop I-Timer Stop F-Timer DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP)	WAIT-DEACT
65	SABORT-SRES	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add,L_sdu, Serv_class=Low, Update_status) /Update_status=NO && L_sdu.len<>0 => ignore	SABORT-SRES
66	SABORT-SRES	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add,L_sdu, Serv_class=Low, Update_status) /Update_status=LO && L_sdu.len<>0 => ignore L_sdu Stop I-Timer Stop F-Timer DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP)	WAIT-DEACT
67	SABORT-SRES	I-Timer expired => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code=ABT_TO)	WAIT-DEACT
68	SABORT-SRES	F-Timer expired => DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code=ABT_TO)	WAIT-DEACT
69	WAIT-DEACT	DMPMS_SAP_DEACTIVATE.cnf(SSAP=Server_SAP,M_status) /M_status = OK => MSAC2S_Closed.ind(Res_SAP)	CLOSED
70	WAIT-DEACT	DMPMS_SAP_DEACTIVATE.cnf(SSAP=Server_SAP,M_status) /M_status=NO/IV => MSAC2S_Fault.ind	POWER-ON

Table 460 (continued)

#	Current state	Event / condition => action	Next state
71	OPEN	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add, L_sdu, Serv_class=Low, Update_status) /VALID-SERVICE-REQ-PDU => Service_Header:=Function_Num Stored_Slot_Number:=Slot_Number Stored_Index:=Index Stop I-Timer Start U-Timer MSAC2S_<VALID-SERVICE>.ind(Res_SAP=Server_SAP)	VS-WRES
72	OPEN	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add, L_sdu, Serv_class=Low, Update_status) /Idle-REQ-PDU {IDLE-PDU} => L_sdu:=Idle-RES-PDU Stop I-Timer Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu, Serv_class=Low, Transmit=Single)	VS-WUPD
73	OPEN	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add, L_sdu, Serv_class=Low, Update_status) /Abort-REQ-PDU => Stop I-Timer DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=FALSE, (Subnet, Instance, Reason_Code) from Abort-REQ-PDU)	WAIT-DEACT
74	OPEN	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add, L_sdu, Serv_class=Low, Update_status) /NOT(VALID-SERVICE-REQ-PDU (Abort-REQ-PDU) (Idle-REQ-PDU)) => Reason_Code:=ABT_FE Stop I-Timer Start I-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu=Abort-REQ-PDU(Subnet=NO, Instance=MSAC2, Reason_Code), Serv_class=Low, Transmit=Single) MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	SABORT-WUPD
75	OPEN	MSAC2S_Abort.req(Res_SAP, Subnet, Instance, Reason_Code) => L_sdu:=Abort-REQ-PDU Stop I-Timer Start I-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu, Serv_class=Low, Transmit=Single)	SABORT-WUPD
76	OPEN	MSAC2S_Initiate.rsp(+/-)(Res_SAP) => Reason_Code:=ABT_SE Stop I-Timer Start I-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu=Abort-REQ-PDU(Subnet=NO, Instance=MSAC2, Reason_Code), Serv_class=Low, Transmit=Single) MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	SABORT-WUPD
77	OPEN	MSAC2S_XXX.rsp(+/-)(Res_SAP) => L_sdu:=Abort-REQ-PDU(Subnet=NO, Instance=MSAC2, Reason_Code=ABT_TO) Stop I-Timer Start I-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu, Serv_class=Low, Transmit=Single)	SABORT-WUPD

Table 460 (continued)

#	Current state	Event / condition => action	Next state
78	OPEN	I-Timer expired => Reason_Code:=ABT_TO Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu=Abort-REQ-PDU(Subnet=NO, Instance=MSAC2, Reason_Code), Serv_class=Low, Transmit=Single) MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	SABORT-WUPD
79	VS-WRES	MSAC2S_XXX.rsp(+)(Res_SAP) /Service-Response fits stored Service_Header && Stored_Max_Len_Data_Unit >= Length => L_sdu:=<VALID-SERVICE>-RES-PDU(with Stored_Slot_Number,Stored_Index) Stop U-Timer Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu, Serv_class=Low, Transmit=Single)	VS-WUPD
80	VS-WRES	MSAC2S_XXX.rsp(-)(Res_SAP) /Service-Response fits stored Service_Header && Stored_Max_Len_Data_Unit >= Length => L_sdu:=<VALID-SERVICE>-NRS-PDU Stop U-Timer Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu, Serv_class=Low, Transmit=Single)	VS-WUPD
81	VS-WRES	MSAC2S_Initiate.rsp(+/-)(Res_SAP) => Reason_Code:=ABT_SE Stop U-Timer Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu=Abort-REQ-PDU(Subnet=NO, Instance=MSAC2, Reason_Code), Serv_class=Low, Transmit=Single) MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	SABORT-WUPD
82	VS-WRES	MSAC2S_XXX.rsp(+/-)(Res_SAP) /NOT(Service-Response fits stored Service_Header && Stored_Max_Len_Data_Unit >= Length) => Reason_Code:=ABT_RE Stop U-Timer Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu=Abort-REQ-PDU(Subnet=NO, Instance=MSAC2, Reason_Code), Serv_class=Low, Transmit=Single) MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	SABORT-WUPD
83	VS-WRES	MSAC2S_Abort.req(Res_SAP, Subnet, Instance, Reason_Code) => L_sdu:=Abort-REQ-PDU Stop U-Timer Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu, Serv_class=Low, Transmit=Single)	SABORT-WUPD
84	VS-WRES	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add,L_sdu, Serv_class=Low, Update_status) /Abort-REQ-PDU => Stop U-Timer DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=FALSE, (Subnet, Instance, Reason_Code) from Abort-REQ-PDU)	WAIT-DEACT

Table 460 (continued)

#	Current state	Event / condition => action	Next state
85	VS-WRES	DMPMS_DATA_REPLY.ind(SSAP=Server_SAP, Loc_add, Rem_add,L_sdu, Serv_class=Low, Update_status) /NOT(Abort-REQ-PDU) => Reason_Code:=ABT_FE Stop U-Timer Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu=Abort-REQ-PDU(Subnet=NO, Instance=MSAC2, Reason_Code), Serv_class=Low, Transmit=Single) MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	SABORT-WUPD
86	VS-WRES	U-Timer expired => L_sdu:=Idle-REQ-PDU Return-State:=VALID-SERVICE Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu, Serv_class=Low, Transmit=Single)	IDLE-WUPD
87	VS-WUPD	DMPMS_REPLY_UPDATE.cnf(SSAP=Server_SAP, Serv_class=Low, L_status) /Status=OK	VS-SRES
88	VS-WUPD	DMPMS_REPLY_UPDATE.cnf(SSAP=Server_SAP, Serv_class=Low, L_status) /L_status= LS/IV/LR => Stop F-Timer Stop I-Timer MSAC2S_Fault.ind	POWER-ON
89	VS-SRES	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add,L_sdu, Serv_class=Low, Update_status) /Update_status=LO && L_sdu.len=0 && Go_Abort = FALSE Stop F-Timer Start I-Timer	OPEN
90	VS-SRES	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add,L_sdu, Serv_class=Low, Update_status) /Update_status=LO && L_sdu.len=0 && Go_Abort = TRUE => L_sdu:=Abort-REQ-PDU with stored Parameter Stop F-Timer Start I-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu, Serv_class=Low, Transmit=Single)	SABORT-WUPD
91	VS-SRES	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add,L_sdu, Serv_class=Low, Update_status) /Update_status=LO/NO && L_sdu.len<>0 && Abort-REQ-PDU => Stop F-Timer DMPMS_SAP_DEACTIVATE.req(SSAP=Server_SAP) MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=FALSE, (Subnet, Instance, Reason_Code) from Abort-REQ-PDU)	WAIT-DEACT
92	VS-SRES	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add,L_sdu, Serv_class=Low, Update_status) /Update_status=LO && L_sdu.len<>0 && NOT (Abort-REQ-PDU) => Reason_Code:=ABT_FE Stop F-Timer Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu=Abort-REQ-PDU(Subnet=NO, Instance=MSAC2, Reason_Code), Serv_class=Low, Transmit=Single) MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	SABORT-WUPD

Table 460 (continued)

#	Current state	Event / condition => action	Next state
93	VS-SRES	DMPMS_DATA_REPLY.ind(SSAP, DSAP=Server_SAP, Loc_add, Rem_add, L_sdu, Serv_class=Low, Update_status) /Update_status=NO && L_sdu.len<>0 && NOT(Abort-REQ-PDU) => Go_Abort:=TRUE STORE_ABORT_PARAMETER Reason_Code:=ABT_FE MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	VS-SRES
94	VS-SRES	MSAC2S_XXX.rsp(+/-)(Res_SAP) => Go_Abort:=TRUE STORE_ABORT_PARAMETER Reason_Code:=ABT_SE MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	VS-SRES
95	VS-SRES	MSAC2S_Initiate.rsp(+/-)(Res_SAP) => Go_Abort:=TRUE STORE_ABORT_PARAMETER Reason_Code:=ABT_SE MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	VS-SRES
96	VS-SRES	MSAC2S_Abort.req(Res_SAP, Subnet, Instance, Reason_Code) => Go_Abort:=TRUE STORE_ABORT_PARAMETER	VS-SRES
97	VS-SRES	F-Timer expired => Reason_Code:=ABT_TO Start F-Timer DMPMS_REPLY_UPDATE.req(SSAP=Server_SAP, L_sdu=Abort-REQ-PDU(Subnet=NO, Instance=MSAC2, Reason_Code), Serv_class=Low, Transmit=Single) MSAC2S_Abort.ind(Res_SAP=Server_SAP, Locally_Generated=TRUE, Subnet=NO, Instance=MSAC2, Reason_Code)	SABORT-WUPD
98	ANY-STATE	unexpected DMPMS reaction => Stop U-Timer Stop F-Timer Stop I-Timer MSAC2S_Fault.ind	POWER-ON
99	DEACT-PON	DMPMS_SAP_DEACTIVATE.cnf(SSAP=Server_SAP, M_status) /M_status = OK	POWER-ON
100	DEACT-PON	DMPMS_SAP_DEACTIVATE.cnf(SSAP=Server_SAP, M_status) /M_status=NO/IV => MSAC2S_Fault.ind	POWER-ON
101	any state	MSAC2S_Reset.req(Res_SAP) => MSAC2S_Reset.cnf(Res_SAP)	POWER-ON

6.6.6 MSCS1S

6.6.6.1 Primitive definitions

6.6.6.1.1 Primitives exchanged between MSCS1S and FSPMS

Table 461 shows the service primitives including their associated parameters issued by MSCS1S and received by the FSPMS.

Table 461 – Primitives issued by MSCS1S to FSPMS

Primitive name	Source	Associated parameters	Functions
Set Time.ind	MSCS1S	AREP, Time Value, Local Time Diff, Summertime, Accuracy, Synchronisation Active, Announcement Hour	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Sync Interval Violation.ind	MSCS1S	AREP	

6.6.6.1.2 Parameter of MSCS1S primitives

The parameters used with the primitives exchanged between the FSPMS and the MSCS1S are described in FAL Service Definition Type 3 Fieldbus IEC 61158-5.

6.6.6.2 State machine description

This machine allways remains in IDLE state. It receives Clock Value indications from the DMPMS. A Set Time indication will be issued on detection of the end of a valid Clock Synchronisation sequence. Otherwise a Sync Interval Violation indication will be issued.

Local variables of the MSCS1S

Time Last Rcvd

(Network Time)

This local variable contains the Clock_Value_Time_Event of the last valid received Clock Value indication.

Error

(Unsigned8)

This local variable is a counter for invalid Clock Synchronisation sequences. It will be reset to 0, when a Sync Interval Violation is indicated.

Macros of the MSCS1S:

SET_TIME_ATTRIBUTES

```
(
  Local Time Diff :=CS_list.Clock_value_status.C*(-1)^CS_list.Clock_value_status.CV
  Summertime := CS_list.Clock_value_status.SWT
  Accuracy := CS_list.Clock_value_status.CR
  Synchronisation Active := CS_list.Clock_value_status.SYF
  Announcement Hour :=CS_list.Clock_value_status.ANH
)
```

6.6.6.3 MSCS1S state table

Table 462 contains the complete description of the MSCS1S state machine.

Table 462 – MSCS1S state table

#	Current state	Event / condition => action	Next state
1	Idle	CS_CLOCK_VALUE.ind (Time_Master_Addr, CS_list, CS_status, Receive_Delay_Time) /CS_status == SV && Error < 2 => Error := Error+1	Idle
2	Idle	CS_CLOCK_VALUE.ind (Time_Master_Addr, CS_list, CS_status, Receive_Delay_Time) /CS_status == OK && CS_list.Clock_Value_previous_TE != Time Last Rcvd && Error < 2 => Error := Error+1	Idle
3	Idle	CS_CLOCK_VALUE.ind (Time_Master_Addr, CS_list, CS_status, Receive_Delay_Time) /CS_status == SV && Error >= 2 => Error := 0 Sync Interval Violation.ind	Idle
4	Idle	CS_CLOCK_VALUE.ind (Time_Master_Addr, CS_list, CS_status, Receive_Delay_Time) /CS_status == OK && CS_list.Clock_Value_previous_TE != Time Last Rcvd && Error >= 2 => Error := 0 Sync Interval Violation.ind	Idle
5	Idle	CS_CLOCK_VALUE.ind (Time_Master_Addr, CS_list, CS_status, Receive_Delay_Time) /CS_status == OK && CS_list.Clock_Value_previous_TE == Time Last Rcvd => Error := 0 Time Last Rcvd := CS_list.Clock_Value_Time_Event Time Value := Time Last Rcvd + Receive Delay Time SET_TIME_ATTRIBUTES Set_Time.ind(Time Value, Local Time Diff, Summertime, Accuracy, Synchronisation Active, Announcement Hour)	Idle

6.6.7 MSCY1M

6.6.7.1 Primitive definitions

6.6.7.1.1 Primitives exchanged between FSPMM1 and MSCY1M

Table 463 shows the service primitives including their associated parameters issued by the FSPMM1 and received by the MSCY1M.

Table 463 – Primitives issued by FSPMM1 to MSCY1M

Primitive name	Source	Associated parameters	Functions
MInit MS0.req	FSPMM1	Rem Add	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Reset.req	FSPMM1	(none)	
Abort.req	FSPMM1	Rem Add, Subnet, Instance, Reason Code	
Start Slave Handler.req	FSPMM1	Rem Add	Start first Cycle of MSCY1M
Stop Slave Handler.req	FSPMM1	Rem Add	Stop processing of MSCY1M
Cont Slave Handler.req	FSPMM1	Rem Add, Output Clear	Start new Cycle of MSCY1M
Get Slave Diag.req	FSPMM1	Rem Add	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Set Output.req	FSPMM1	Rem Add, Slot_Number Output Data	
Get Input.req	FSPMM1	Rem Add, Slot Number	

Table 464 shows the service primitives including their associated parameters issued by the MSCY1M and received by the FSPMM1.

Table 464 – Primitives issued by MSCY1M to FSPMM1

Primitive name	Source	Associated parameters	Functions
MInit MS0.cnf	MSCY1M	Rem Add	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Reset.cnf	MSCY1M	(none)	
Start Slave Handler.cnf	MSCY1M	Rem Add	Start first Cycle of MSCY1M
Stop Slave Handler.cnf	MSCY1M	Rem Add	Stop processing of MSCY1M
Cont Slave Handler.cnf	MSCY1M	Rem Add, Diag, No ACIr	Start new Cycle of MSCY1M
Get Slave Diag.cnf(+)	MSCY1M	Rem Add, CREP, Diag Data	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Get Slave Diag.cnf(-)	MSCY1M	Rem Add	
Set Output.cnf(+)	MSCY1M	Rem Add, Slot Number	
Set Output.cnf(-)	MSCY1M	Rem Add, Slot Number	
Get Input.cnf(+)	MSCY1M	Rem Add, Slot_Number, Input Data	
Get Input.cnf(-)	MSCY1M	Rem Add	
New Slave Diag.ind	MSCY1M	Rem Add	
New Input.ind	MSCY1M	Rem Add	
Started.ind	MSCY1M	Rem Add, Alarm Limit	
Stopped.ind	MSCY1M	Rem Add	
Fault.ind	MSCY1M	Rem Add	

6.6.7.1.2 Parameters of MSCY1M primitives

The parameters used with the primitives exchanged between the FSPMM1 and MSCY1M are described in Table 465.

Table 465 – Parameters used with primitives exchanged between FSPMM1 and MSCY1M

Parameter name	Description
Rem Add	This parameter is used to identify the DP-slave which is assigned to that communication relation.
Output Clear	This parameter indicates to the MSCY1M state machine that the Outputs shall be cleared.
Diag	This parameter indicates to the FSPMM1 state machine that no Data Exchange was processed in the preceeding cycle or the Slave was deactivated.
No Aclr	This parameter indicates to the FSPMM1 state machine that the corresponding DP-slave is operated in the non Autoclear-Mode (this DP-slave has no influence to the Masters State (OPERATE or CLEAR).

Other parameters used with the primitives exchanged between the FSPMM1 and the MSCY1M are described in FAL Service Definition Type 3 Fieldbus IEC 61158-5.

6.6.7.2 State machine description

For each possible DP-slave a State Machine MSCY1M (Slave-Handler) is established and started by a superior State Machine, the FSPMM1. The remote address of the Slave (Rem_Add) is used as a local reference for MSCY1M.

The Slave-Handler manages the individual states for every DP-slave. The following main states will be distinguished:

- diagnostic
- parameterisation
- configuration
- data exchange

SI_Para_Exist

Access: -r

Local information indicating whether or not a Slave parameter set of the associated Rem_Add exists.

SPara

Access: -r/-w

Any Information from ARL/CRL related to this DP-slave.

BOutput

Access: -r

Local storage of Output data for the DP-slave.

BInput

Access: -w

Local storage of Input data for the DP-slave.

BDiag

Access: -r/-w

Local storage of Diagnostic information of the DP-slave.

MSCY1M changes the following flags of the diagnostic information:

Invalid_Slave_Response

Station_Non_Existent

Deactivated

DTrans

This variable is mapped on the Data Transfer List attribute of CR-List.

Access: -r/-w

The user data exchange mode between the DP-master (Class 1) and its assigned DP-slaves is supervised on the Master side. This means it is checked whether an user data transfer to the associated DP-slaves was executed within the last DP cycle or a diagnostic cycle, after which the directly following Slave state is again the data transfer state. Any other following state forces the associated bit of the Slave to be cleared.

SDiag

This variable is mapped on the System Diagnosis List attribute of CR-List.

Access: -r/-w

It is set while the MSCY1M state machine of an activated Slave is not able to establish the connection to the Slave without errors. E. g. it is set if a Slave does not respond, if he responds with a negative DL confirmation or if he does not respond with the correct number of input data during Data_Exchange. The bit is reset after the connection could be established without any error. It is also reset if the Slave is deactivated by the User or if the MSCY1M State Machine is stopped

Local variables

Delay_Count

(Unsigned8)

The Delay_Count is used to count the number of Slave_Diag.cnf in the state DIAG2 while Diag_Data.Prm_Req is still set (Slow Slave). The Delay_Count is initialised with the value of SPara.Diag_Upd_Delay.

Range: 0 to 15 (extendible up to 255)

MSAL1M_Started

(Boolean)

This variable is used to store the information whether the MSAL1M State Machine is started (True) or not (False).

Wait_for_Start_con

(Boolean)

This variable is used to store the information whether a Start.req has not yet been confirmed by the MSAL1M State Machine (True) or not (False).

Go-Abort

(Boolean)

This variable is used to store the information that an error occurred forcing the MSCY1M State Machine to abort the connection.

Macros

CHANGE_DIAG_IND

```
(
MSCY1M_New Slave Diag.ind(Rem_Add)
if (MSAL1M_Started = TRUE)
    MSAL1M_Change Diag.req(Rem_Add)
)
```

GO_ABORT

```
(
DTrans = 0
Go_Abort = TRUE
BInp = Nil
)
```

START_MSAL1M

```
(  
if ((SPara.DPV1_Supported= TRUE)  
  && (Wait_for_Start_con = FALSE)  
  && (MSAL1M_Started = FALSE))  
  MSAL1M_Start.req(Rem_Add),  
  MSAL1M_Started = TRUE  
)
```

6.6.7.3 MSCY1M state table

Table 466 contains the complete description of the MSCY1M state machine.

Table 466 – MSCY1M state table

#	Current state	Event / condition => action	Next state
1	POWER-ON	MSCY1M Minit MS0.req (Rem Add) => BDiag := NIL, SDiag := 0, DTrans := 0 Go_Abort := FALSE MSAL1M_Started := FALSE ExtPrmBsy:= FALSE MSCY1M Minit MS0.cnf(Rem Add)	STOP
2	STOP	MSCY1M Abort.req (Rem Add) => ignore	STOP
3	STOP	MSAL1 Abort.ind (Rem_Add) => ignore	STOP
4	STOP	MSCY1M Start Slave Handler.req (Rem Add) /SI_Para_Exist = FALSE SPara.Active = FALSE => MSCY1M Start Slave Handler.cnf(Rem Add)	DEACT
5	STOP	MSCY1M Start Slave Handler.req (Rem Add) /SI_Para_Exist = TRUE && SPara.Active = TRUE => SDiag := 1, BInput := Nil, BOutput := Nil BDiag.Deactivated := FALSE BDiag.Station_Non_Existent := TRUE MSCY1M Start Slave Handler.cnf(Rem Add)	DIAG1
6	STOP	MSCY1M Get Slave Diag.req (Rem Add) => MSCY1M Get Slave Diag.cnf(-) (Rem Add)	STOP
7	STOP	MSCY1M Set Output.req (Rem Add, Slot Number, Output Data) => MSCY1M Set Output.cnf(-) (Rem Add, Slot Number)	STOP
8	STOP	MSCY1M Get Input.req (Rem Add, Slot_Number) => MSCY1M Get Input.cnf(-) (Rem Add)	STOP
9	DEACT	MSCY1M Abort.req (Rem Add) => ignore	DEACT
10	DEACT	MSAL1 Abort.ind (Rem_Add) => ignore	DEACT
11	DEACT	MSCY1M Stop Slave Handler.req (Rem Add) => MSCY1M Stop Slave Handler.cnf(Rem Add)	STOP
12	DEACT	MSCY1M Cont Slave Handler.req (Rem Add, Output Clear) /SI_Para_Exist = FALSE SPara.Active = FALSE => Diag:=FALSE, No_AClr:=TRUE MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DEACT
13	DEACT	MSCY1M Cont Slave Handler.req (Rem Add, Output Clear) /SI_Para_Exist = TRUE && SPara.Active = TRUE => SDiag := 1, BInput := Nil, BOutput := Nil BDiag.Deactivated := FALSE BDiag.Station_Non_Existent := TRUE DMPMM1 Slave Diag.req (Rem Add)	WDIAG1

Table 466 (continued)

#	Current state	Event / condition => action	Next state
14	DEACT	MSCY1M Get Slave Diag.req (Rem Add) => MSCY1M Get Slave Diag.cnf(-) (Rem Add)	DEACT
15	DEACT	MSCY1M Set Output.req (Rem Add, Slot Number, Output Data) => MSCY1M Set Output.cnf(-) (Rem Add, Slot Number)	DEACT
16	DEACT	MSCY1M Get Input.req (Rem Add, Slot_Number) => MSCY1M Get Input.cnf(-) (Rem Add)	DEACT
17	DIAG1	MSCY1M Abort.req (Rem Add) => ignore	DIAG1
18	DIAG1	MSAL1 Abort.ind (Rem_Add) => ignore	DIAG1
19	DIAG1	MSCY1M Stop Slave Handler.req (Rem Add) => Bdiag.Deactivated := TRUE, Sdiag := 0 MSCY1M Stop Slave Handler.cnf(Rem Add)	STOP
20	DIAG1	MSCY1M Cont Slave Handler.req (Rem Add, Output Clear) => ExtPrmBsy:=FALSE DMPMM1 Slave Diag.req (Rem Add)	WDIAG1
21	DIAG1	MSCY1M Get Slave Diag.req (Rem Add) => Diag_Data := Bdiag MSCY1M Get Slave Diag.cnf(+) (Rem Add, Diag Data)	DIAG1
22	DIAG1	MSCY1M Set Output.req (Rem Add, Slot Number, Output Data) => BOutput(Slot_Number) := Output_Data MSCY1M Set Output.cnf(+) (Rem Add, Slot Number)	DIAG1
23	DIAG1	MSCY1M Get Input.req (Rem Add, Slot_Number) => Input_Data := BInput(Slot_Number) MSCY1M Get Input.cnf(+) (Rem Add, Input Data)	DIAG1
24	WDIAG1	MSCY1M Abort.req (Rem Add) => ignore	WDIAG1
25	WDIAG1	MSAL1 Abort.ind (Rem_Add) => ignore	WDIAG1
26	WDIAG1	MSCY1M Get Slave Diag.req (Rem Add) => Diag_Data := Bdiag MSCY1M Get Slave Diag.cnf(+) (Rem Add, Diag Data)	WDIAG1
27	WDIAG1	MSCY1M Set Output.req (Rem Add, Slot Number, Output Data) => BOutput(Slot_Number) := Output_Data MSCY1M Set Output.cnf(+) (Rem Add, Slot Number)	WDIAG1
28	WDIAG1	MSCY1M Get Input.req (Rem Add, Slot_Number) => Input_Data := BInput(Slot_Number) MSCY1M Get Input.cnf(+) (Rem Add, Input Data)	WDIAG1
29	WDIAG1	DMPMM1 Slave Diag.cnf(+) (Rem Add, Diag Data) /SPara.Active = FALSE => Diag:=FALSE, No_AClr:=TRUE Bdiag.Deactivated := TRUE, Sdiag := 0 MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DEACT
30	WDIAG1	DMPMM1 Slave Diag.cnf(-) (Rem Add, Status) /SPara.Active = FALSE => Diag:=FALSE, No_AClr:=TRUE Bdiag.Deactivated := TRUE, Sdiag := 0 MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DEACT

Table 466 (continued)

#	Current state	Event / condition => action	Next state
31	WDIAG1	DMPMM1 Slave Diag.cnf(-) (Rem Add, Status) /SPara.Active = TRUE && Status=NA => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr BDiag.Station_Non_Existent := TRUE MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DIAG1
32	WDIAG1	DMPMM1 Slave Diag.cnf(-) (Rem Add, Status) /SPara.Active = TRUE && Status=RE/RS/RR/UE/NR => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr BDiag.Invalid_Slave_Response := TRUE BDiag.Station_Non_Existent := FALSE MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DIAG1
33	WDIAG1	DMPMM1 Slave Diag.cnf(-) (Rem Add, Status) /SPara.Active = TRUE && Status=DS => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DIAG1
34	WDIAG1	DMPMM1 Slave Diag.cnf(+) (Rem Add, Diag Data) /SPara.Active = TRUE && Diag_Data.Master_Lock = TRUE => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr BDiag := Diag_Data MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DIAG1
35	WDIAG1	DMPMM1 Slave Diag.cnf(+) (Rem Add, Diag Data) /SPara.Active = TRUE && Diag_Data.Master_Lock = FALSE && Diag_Data.Master_Add <> invalid && SPara.Prm_Data.DPV1_Enable=TRUE && SPara.DPV1_Supported=TRUE => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr BDiag := Diag_Data MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	UNLCK
36	WDIAG1	DMPMM1 Slave Diag.cnf(+) (Rem Add, Diag Data) /SPara.Active = TRUE && Diag_Data.Master_Lock = FALSE && (Diag_Data.Master_Add = invalid SPara.Prm_Data.DPV1_Enable=FALSE SPara.DPV1_Supported=FALSE) => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr SPara.New_Prm := FALSE BDiag := Diag_Data, CHANGE_DIAG_IND MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	PRM
37	PRM	MSCY1M Abort.req (Rem Add) => ignore	PRM
38	PRM	MSAL1 Abort.ind (Rem_Add) => ignore	PRM
39	PRM	MSCY1M Stop Slave Handler.req (Rem Add) => BDiag.Deactivated := TRUE, SDiag := 0 MSCY1M Stop Slave Handler.cnf(Rem Add)	STOP
40	PRM	MSCY1M Cont Slave Handler.req (Rem Add, Output Clear) / ExtPrmBsy=FALSE => Prm_Data := SPara .Prm_Data Prm_Data.Lock_Req := TRUE Prm_Data.Unlock_Req := FALSE DMPMM1 Set Prm.req (Rem Add, Prm Data)	WPRM
41	PRM	MSCY1M Get Slave Diag.req (Rem Add) => Diag_Data := BDiag MSCY1M Get Slave Diag.cnf(+) (Rem Add, Diag Data)	PRM
42	PRM	MSCY1M Set Output.req (Rem Add, Slot Number, Output Data) => BOutput(Slot_Number) := Output_Data MSCY1M Set Output.cnf(+) (Rem Add, Slot Number)	PRM

Table 466 (continued)

#	Current state	Event / condition => action	Next state
43	PRM	MSCY1M Get Input.req (Rem Add, Slot_Number) => Input_Data := BInput(Slot_Number) MSCY1M Get Input.cnf(+) (Rem Add, Input Data)	PRM
44	WPRM	MSCY1M Abort.req (Rem Add) => ignore	WPRM
45	WPRM	MSAL1 Abort.ind (Rem_Add) => ignore	WPRM
46	WPRM	MSCY1M Get Slave Diag.req (Rem Add) => Diag_Data := BDiag MSCY1M Get Slave Diag.cnf(+) (Rem Add, Diag Data)	WPRM
47	WPRM	MSCY1M Set Output.req (Rem Add, Slot Number, Output Data) => BOutput(Slot_Number) := Output_Data MSCY1M Set Output.cnf(+) (Rem Add, Slot Number)	WPRM
48	WPRM	MSCY1M Get Input.req (Rem Add, Slot_Number) => Input_Data := BInput(Slot_Number) MSCY1M Get Input.cnf(+) (Rem Add, Input Data)	WPRM
49	WPRM	DMPMM1 Set Prm.cnf(+) (Rem Add) /SPara.Active = FALSE => Diag:=FALSE, No_AClr:=TRUE MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	UNLCK
50	WPRM	DMPMM1 Set Prm.cnf(-) (Rem Add, Status) /SPara.Active = FALSE => Diag:=FALSE, No_AClr:=TRUE MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	UNLCK
51	WPRM	DMPMM1 Set Prm.cnf(-) (Rem Add, Status) /SPara.Active = TRUE && Status = DS => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	PRM
52	WPRM	DMPMM1 Set Prm.cnf(-) (Rem Add, Status) /SPara.Active = TRUE && Status = NA => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr BDiag.Station_Non_Existent := TRUE MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DIAG1
53	WPRM	DMPMM1 Set Prm.cnf(-) (Rem Add, Status) /SPara.Active = TRUE && Status = RE/RS/RR/UE => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr BDiag.Invalid_Slave_Response := TRUE MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DIAG1
54	WPRM	DMPMM1 Set Prm.cnf(+) (Rem Add) /SPara.Active = TRUE && ExtPrmFlag = FALSE => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	CFG
55	CFG	MSCY1M Abort.req (Rem Add) => ignore	CFG
56	CFG	MSAL1 Abort.ind (Rem_Add) => ignore	CFG
57	CFG	MSCY1M Stop Slave Handler.req (Rem Add) => Prm_Data := SPara .Prm_Data Prm_Data.Unlock_Req := TRUE DMPMM1 Set Prm.req (Rem Add, Prm Data)	SUNLCK

Table 466 (continued)

#	Current state	Event / condition => action	Next state
58	CFG	MSCY1M Cont Slave Handler.req (Rem Add, Output Clear) => Cfg_Data:=SPara .Cfg_Data DMPMM1 Chk Cfg.req (Rem Add, Cfg Data)	WCFG
59	CFG	MSCY1M Get Slave Diag.req (Rem Add) => Diag_Data := BDiag MSCY1M Get Slave Diag.cnf(+) (Rem Add, Diag Data)	CFG
60	CFG	MSCY1M Set Output.req (Rem Add, Slot Number, Output Data) => BOutput(Slot_Number) := Output_Data MSCY1M Set Output.cnf(+) (Rem Add, Slot Number)	CFG
61	CFG	MSCY1M Get Input.req (Rem Add, Slot_Number) => Input_Data := BInput(Slot_Number) MSCY1M Get Input.cnf(+) (Rem Add, Input Data)	CFG
62	WCFG	MSCY1M Abort.req (Rem Add) => ignore	WCFG
63	WCFG	MSAL1 Abort.ind (Rem_Add) => ignore	WCFG
64	WCFG	MSCY1M Get Slave Diag.req (Rem Add) => Diag_Data := BDiag MSCY1M Get Slave Diag.cnf(+) (Rem Add, Diag Data)	WCFG
65	WCFG	MSCY1M Set Output.req (Rem Add, Slot Number, Output Data) => BOutput(Slot_Number) := Output_Data MSCY1M Set Output.cnf(+) (Rem Add, Slot Number)	WCFG
66	WCFG	MSCY1M Get Input.req (Rem Add, Slot_Number) => Input_Data := BInput(Slot_Number) MSCY1M Get Input.cnf(+) (Rem Add, Input Data)	WCFG
67	WCFG	DMPMM1 Chk Cfg.cnf(+) (Rem Add) /SPara.Active = FALSE => Diag:=FALSE, No_AClr:=TRUE MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	UNLCK
68	WCFG	DMPMM1 Chk Cfg.cnf(-) (Rem Add, Status) /SPara.Active = FALSE => Diag:=FALSE, No_AClr:=TRUE MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	UNLCK
69	WCFG	DMPMM1 Chk Cfg.cnf(-) (Rem Add, Status) /SPara.Active = TRUE && Status = DS => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	CFG
70	WCFG	DMPMM1 Chk Cfg.cnf(-) (Rem Add, Status) /SPara.Active = TRUE && Status = NA => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr BDiag.Station_Non_Existent := TRUE MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DIAG1
71	WCFG	DMPMM1 Chk Cfg.cnf(-) (Rem Add, Status) /SPara.Active = TRUE && Status = RE/RS/RR/UE => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr BDiag.Invalid_Slave_Response := TRUE MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DIAG1

Table 466 (continued)

#	Current state	Event / condition => action	Next state
72	WCFG	DMPMM1 Chk Cfg.cnf(+) (Rem Add) /SPara.Active = TRUE => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr MSAL1M_Started := FALSE Delay_Count := SPara .Diag_Upd_Delay Wait_for_Start_con := FALSE MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DIAG2
73	DIAG2	MSCY1M Abort.req (Rem Add) => GO_ABORT	DIAG2
74	DIAG2	MSAL1 Start.cnf (Rem Add) => Wait_for_Start_con:=FALSE MSCY1M_Started.ind(Rem_Add)	DIAG2
75	DIAG2	MSAL1 Abort.ind (Rem_Add) => GO_ABORT	DIAG2
76	DIAG2	MSCY1M Stop Slave Handler.req (Rem Add) /SPara.DPV1_Supported = FALSE => Go_Abort := FALSE, BInput := Nil DTrans := 0 Prm_Data := SPara .Prm_Data Prm_Data.Unlock_Req := TRUE DMPMM1 Set Prm.req (Rem Add, Prm Data)	SUNLCK
77	DIAG2	MSCY1M Stop Slave Handler.req (Rem Add) /SPara.DPV1_Supported = TRUE => Go_Abort := FALSE, BInput := Nil DTrans := 0 MSAL1 Stop.req (Rem Add)	WSTPCS
78	DIAG2	MSCY1M Cont Slave Handler.req (Rem Add, Output Clear) /Go_Abort = FALSE && Wait_for_Start_con = FALSE => DMPMM1 Slave Diag.req (Rem Add)	WDIAG2
79	DIAG2	MSCY1M Cont Slave Handler.req (Rem Add, Output Clear) /Go_Abort = FALSE && Wait_for_Start_con = TRUE => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DIAG2
80	DIAG2	MSCY1M Cont Slave Handler.req (Rem Add, Output Clear) /Go_Abort=TRUE && SPara.DPV1_Supported = FALSE => Go_Abort := FALSE No_AClr := SPara_.Ignore_AClr OR NOT SPara.Active Prm_Data := SPara .Prm_Data Prm_Data.Unlock_Req := TRUE DMPMM1 Set Prm.req (Rem Add, Prm Data)	WUNLCK
81	DIAG2	MSCY1M Cont Slave Handler.req (Rem Add, Output Clear) /Go_Abort=TRUE && SPara.DPV1_Supported = TRUE => Go_Abort := FALSE, Diag := TRUE No_AClr := SPara_.Ignore_AClr OR NOT SPara.Active MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr) MSAL1 Stop.req (Rem Add)	WSTPCC
82	DIAG2	MSCY1M Get Slave Diag.req (Rem Add) => Diag_Data := BDiag MSCY1M Get Slave Diag.cnf(+) (Rem Add, Diag Data)	DIAG2
83	DIAG2	MSCY1M Set Output.req (Rem Add, Slot Number, Output Data) => BOutput(Slot_Number) := Output_Data MSCY1M Set Output.cnf(+) (Rem Add, Slot Number)	DIAG2

Table 466 (continued)

#	Current state	Event / condition => action	Next state
84	DIAG2	MSCY1M Get Input.req (Rem Add, Slot_Number) => Input_Data := BInput(Slot_Number) MSCY1M Get Input.cnf(+) (Rem Add, Input Data)	DIAG2
85	WDIAG2	MSCY1M Abort.req (Rem Add) => GO_ABORT	WDIAG2
86	WDIAG2	MSAL1 Abort.ind (Rem_Add) => GO_ABORT	WDIAG2
87	WDIAG2	MSCY1M Get Slave Diag.req (Rem Add) => Diag_Data := BDiag MSCY1M Get Slave Diag.cnf(+) (Rem Add, Diag Data)	WDIAG2
88	WDIAG2	MSCY1M Set Output.req (Rem Add, Slot Number, Output Data) => BOutput(Slot_Number) := Output_Data MSCY1M Set Output.cnf(+) (Rem Add, Slot Number)	WDIAG2
89	WDIAG2	MSCY1M Get Input.req (Rem Add, Slot_Number) => Input_Data := BInput(Slot_Number) MSCY1M Get Input.cnf(+) (Rem Add, Input Data)	WDIAG2
90	WDIAG2	DMPMM1 Slave Diag.cnf(+) (Rem Add, Diag Data) /SPara.Active = FALSE => Diag:=FALSE, No_AClr:=FALSE BDiag := Diag_Data, GO_ABORT MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DIAG2
91	WDIAG2	DMPMM1 Slave Diag.cnf(-) (Rem Add, Status) /SPara.Active = FALSE => GO_ABORT, Diag:=FALSE, No_AClr:=FALSE MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DIAG2
92	WDIAG2	DMPMM1 Slave Diag.cnf(-) (Rem Add, Status) /SPara.Active = TRUE && Status = DS => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DIAG2
93	WDIAG2	DMPMM1 Slave Diag.cnf(-) (Rem Add, Status) /SPara.Active = TRUE && Status = NA && SPara_.NA_To_Abort = FALSE => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr BDiag.Station_Non_Existent := TRUE SDiag := 1, DTrans := 0 MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DIAG2
94	WDIAG2	DMPMM1 Slave Diag.cnf(-) (Rem Add, Status) /SPara.Active = TRUE && Status = NA && SPara_.NA_To_Abort = TRUE => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr, BDiag.Station_Non_Existent := TRUE, SDiag := 1, GO_ABORT MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DIAG2
95	WDIAG2	DMPMM1 Slave Diag.cnf(-) (Rem Add, Status) /SPara.Active = TRUE && Status = RE/RS/RR/UE/NR => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr, BDiag.Station_Non_Existent := TRUE, BDiag.Invalid_Slave_Response := TRUE, SDiag := 1, GO_ABORT MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DIAG2
96	WDIAG2	DMPMM1 Slave Diag.cnf(+) (Rem Add, Diag Data) /SPara.Active = TRUE && (Diag_Data.Prm_Req = TRUE Diag_Data.Master_Lock = TRUE) && Delay_Count > 0 => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr Delay_Count := Delay_Count-1, BDiag := Diag_Data BInput := 0, SDiag := 1, DTrans := 0 MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DIAG2

Table 466 (continued)

#	Current state	Event / condition => action	Next state
97	WDIAG2	DMPMM1 Slave Diag.cnf(+) (Rem Add, Diag Data) /SPara.Active = TRUE && (Diag_Data.Prm_Req = TRUE Diag_Data.Master_Lock = TRUE) && Delay_Count = 0 => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr BDiag := Diag_Data, SDiag := 1, GO_ABORT MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DIAG2
98	WDIAG2	DMPMM1 Slave Diag.cnf(+) (Rem Add, Diag Data) /SPara.Active = TRUE && Diag_Data.Prm_Req = FALSE && Diag_Data.Master_Lock = FALSE && Diag_Data.Station_Not_Ready = TRUE => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr SDiag := 1, DTrans := 0, BInput := Nil BDiag := Diag_Data MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DIAG2
99	WDIAG2	DMPMM1 Slave Diag.cnf(+) (Rem Add, Diag Data) /SPara.Active = TRUE && Diag_Data.Prm_Req = FALSE && Diag_Data.Master_Lock = FALSE && Diag_Data.Station_Not_Ready = FALSE && Diag_Data.Stat_Diag = TRUE => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr START_MSAL1M, SDiag := 1, DTrans := 0 BInput := Nil, BDiag := Diag_Data MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DIAG2
100	WDIAG2	DMPMM1 Slave Diag.cnf(+) (Rem Add, Diag Data) /SPara.Active = TRUE && Diag_Data.Prm_Req = FALSE && Diag_Data.Master_Lock = FALSE && Diag_Data.Station_Not_Ready = FALSE && Diag_Data.Stat_Diag=FALSE && DTrans = 0 => Diag:=TRUE, No_AClr:=TRUE SDiag := Diag_Data.Ext_Diag, Delay_Count := 0 BDiag := Diag_Data, START_MSAL1M, CHANGE_DIAG_IND MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DATA
101	WDIAG2	DMPMM1 Slave Diag.cnf(+) (Rem Add, Diag Data) /SPara.Active = TRUE && Diag_Data.Prm_Req = FALSE && Diag_Data.Master_Lock = FALSE && Diag_Data.Station_Not_Ready = FALSE && Diag_Data.Stat_Diag = FALSE && DTrans = 1 => Diag:=TRUE, No_AClr:=TRUE SDIAG := Diag_Data.Ext_Diag, CHANGE_DIAG_IND BDiag := Diag_Data MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DATA
102	DATA	MSCY1M Abort.req (Rem Add) => GO_ABORT	DATA
103	DATA	MSAL1 Start.cnf (Rem Add) => Wait_for_Start_con := FALSE, START_MSAL1M MSCY1M_Started.ind(Rem_Add)	DATA
104	DATA	MSAL1 Abort.ind (Rem_Add) => GO_ABORT	DATA
105	DATA	MSCY1M Stop Slave Handler.req (Rem Add) /SPara.DPV1_Supported = FALSE => Go_Abort := FALSE, BInput := Nil DTrans := 0 Prm_Data := SPara .Prm_Data Prm_Data.Unlock_Req := TRUE DMPMM1 Set Prm.req (Rem Add, Prm Data)	SUNLCK
106	DATA	MSCY1M Stop Slave Handler.req (Rem Add) /SPara.DPV1_Supported = TRUE => Go_Abort := FALSE, BInput :=Nil, DTrans := 0 MSAL1 Stop.req (Rem Add)	WSTPCS

Table 466 (continued)

#	Current state	Event / condition => action	Next state
107	DATA	MSCY1M Cont Slave Handler.req (Rem Add, Output Clear) /Go_Abort = FALSE && Wait_for_Start_con = FALSE && Output_Clear = TRUE && ((SPara.DPV1_Supported = FALSE && SPara.New_Prm = FALSE) SPara.DPV1_Supported = TRUE) && SPara.Fail_Safe = FALSE => Outp_Data:=0 DMPMM1 Data Exchange.req (Rem Add, Outp Data)	WDATA
108	DATA	MSCY1M Cont Slave Handler.req (Rem Add, Output Clear) /Go_Abort = FALSE && Wait_for_Start_con = FALSE && Output_Clear = TRUE && ((SPara.DPV1_Supported = FALSE && SPara.New_Prm = FALSE) SPara.DPV1_Supported = TRUE) && SPara.Fail_Safe = TRUE => Outp_Data.len:=0 DMPMM1 Data Exchange.req (Rem Add, Outp Data)	WDATA
109	DATA	MSCY1M Cont Slave Handler.req (Rem Add, Output Clear) /Go_Abort = FALSE && Wait_for_Start_con = FALSE && Output_Clear = FALSE && ((SPara.DPV1_Supported = FALSE && SPara.New_Prm = FALSE) SPara.DPV1_Supported = TRUE) => Outp_Data:=BOutput DMPMM1 Data Exchange.req (Rem Add, Outp Data)	WDATA
110	DATA	MSCY1M Cont Slave Handler.req (Rem Add, Output Clear) /Go_Abort = FALSE && SPara.DPV1_Supported = FALSE && SPara.New_Prm = TRUE => Prm_Data := SPara .Prm_Data Prm_Data.Lock_Req := TRUE Prm_Data.Unlock_Req := FALSE DMPMM1 Set Prm.req (Rem Add, Prm Data)	WDATA
111	DATA	MSCY1M Cont Slave Handler.req (Rem Add, Output Clear) /Go_Abort = FALSE && Wait_for_Start_con = TRUE => Diag:=TRUE, No_AClr:=TRUE MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DATA
112	DATA	MSCY1M Cont Slave Handler.req (Rem Add, Output Clear) /Go_Abort=TRUE && (SPara.DPV1_Supported = FALSE SPara_PrmCmd_Supported = TRUE) => Go_Abort := FALSE, Prm_Data := SPara .Prm_Data Prm_Data.Unlock_Req := TRUE DMPMM1 Set Prm.req (Rem Add, Prm Data)	WUNLCK
113	DATA	MSCY1M Cont Slave Handler.req (Rem Add, Output Clear) /Go_Abort=TRUE && SPara.DPV1_Supported = TRUE && SPara_PrmCmd_Supported = FALSE => Go_Abort := FALSE, Diag := TRUE No_AClr := SPara.Ignore_AClr OR NOT SPara.Active MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr) MSAL1 Stop.req (Rem Add)	WSTPCC
114	DATA	MSCY1M Get Slave Diag.req (Rem Add) => Diag_Data := BDiag MSCY1M Get Slave Diag.cnf(+) (Rem Add, Diag Data)	DATA
115	DATA	MSCY1M Set Output.req (Rem Add, Slot Number, Output Data) => BOutput(Slot_Number) := Output_Data MSCY1M Set Output.cnf(+) (Rem Add, Slot Number)	DATA
116	DATA	MSCY1M Get Input.req (Rem Add, Slot_Number) => Input_Data := BInput(Slot_Number) MSCY1M Get Input.cnf(+) (Rem Add, Input Data)	DATA
117	WDATA	MSCY1M Abort.req (Rem Add) => GO_ABORT	WDATA
118	WDATA	MSAL1 Abort.ind (Rem_Add) => GO_ABORT	WDATA

Table 466 (continued)

#	Current state	Event / condition => action	Next state
119	WDATA	MSCY1M Get Slave Diag.req (Rem Add) => Diag_Data := BDiag MSCY1M Get Slave Diag.cnf(+) (Rem Add, Diag Data)	WDATA
120	WDATA	MSCY1M Set Output.req (Rem Add, Slot Number, Output Data) => BOutput(Slot_Number) := Output_Data MSCY1M Set Output.cnf(+) (Rem Add, Slot Number)	WDATA
121	WDATA	MSCY1M Get Input.req (Rem Add, Slot_Number) => Input_Data := BInput(Slot_Number) MSCY1M Get Input.cnf(+) (Rem Add, Input Data)	WDATA
122	WDATA	DMPMM1 Data Exchange.cnf(+) (Rem Add, Diag Flag, Inp Data) /SPara.Active = FALSE => GO_ABORT, Diag:=FALSE, No_AClr:=TRUE MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DATA
123	WDATA	DMPMM1 Data Exchange.cnf(-) (Rem Add, Status) /SPara.Active = FALSE => GO_ABORT, Diag:=FALSE, No_AClr:=TRUE MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DATA
124	WDATA	DMPMM1 Data Exchange.cnf(-) (Rem Add, Status) /SPara.Active = TRUE && Status = DS => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr DTrans := 0, BInput := Nil MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DATA
125	WDATA	DMPMM1 Data Exchange.cnf(-) (Rem Add, Status) /SPara.Active = TRUE && Status = NA && SPara_.NA_To_Abort = FALSE => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr BDiag.Station_Non_Existent := TRUE, SDiag := 1 MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DATA
126	WDATA	DMPMM1 Data Exchange.cnf(-) (Rem Add, Status) /SPara.Active = TRUE && Status=NA && SPara_.NA_To_Abort = TRUE => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr BDiag.Station_Non_Existent := TRUE, SDiag := 1 GO_ABORT MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DATA
127	WDATA	DMPMM1 Data Exchange.cnf(-) (Rem Add, Status) /SPara.Active = TRUE && Status = RE/RS/RR/UE => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr BDiag.Invalid_Slave_Response := TRUE BDiag.Station_Non_Existent := FALSE SDiag := 1, GO_ABORT MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DATA
128	WDATA	DMPMM1 Data Exchange.cnf(+) (Rem Add, Diag Flag, Inp Data) /SPara.Active = TRUE && Diag_Flag = FALSE && Inp_Data.len = Exp_Inp_Len => Diag:=FALSE, No_AClr:=TRUE BDiag.Station_Non_Existent := FALSE BInput := Inp_Data, DTrans := 1 SDiag := BDiag.Ext_Diag MSCY1M_New_Input.ind(Rem_Add) MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DATA
129	WDATA	DMPMM1 Data Exchange.cnf(+) (Rem Add, Diag Flag, Inp Data) /SPara.Active = TRUE && Inp_Data.len <> Exp_Inp_Len && (Inp_Data.len <> 1 Exp_Inp_Len <> 0 Diag_Flag = FALSE) => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr BDiag.Invalid_Slave_Response := TRUE BDiag.Station_Non_Existent := FALSE SDiag := 1, GO_ABORT MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DATA

Table 466 (continued)

#	Current state	Event / condition => action	Next state
130	WDATA	DMPMM1 Data Exchange.cnf(+) (Rem Add, Diag Flag, Inp Data) /SPara.Active = TRUE && Diag_Flag = TRUE && Inp_Data.len = Exp_Inp_Len => Diag:=FALSE, No_AClr:=TRUE BDiag.Station_Non_Existent := FALSE BInput := Inp_Data, DTrans := 1 SDiag := BDiag.Ext_Diag MSCY1M_New_Input.ind(Rem_Add) MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DIAG2
131	WDATA	DMPMM1 Data Exchange.cnf(+) (Rem Add, Diag Flag, Inp Data) /SPara.Active = TRUE && Diag_Flag = TRUE && Inp_Data.len = 1 && Exp_Inp_Len = 0 => Diag:=FALSE, No_AClr:=TRUE BDiag.Station_Non_Existent := FALSE, DTrans := 1 SDiag := BDiag.Ext_Diag MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DIAG2
132	WDATA	DMPMM1 Set Prm.cnf(+) (Rem Add) /SPara.Active = FALSE => GO_ABORT, Diag:=FALSE, No_AClr:=TRUE MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DATA
133	WDATA	DMPMM1 Set Prm.cnf(-) (Rem Add, Status) /SPara.Active = FALSE => GO_ABORT, Diag:=FALSE, No_AClr:=TRUE MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DATA
134	WDATA	DMPMM1 Set Prm.cnf(-) (Rem Add, Status) /SPara.Active = TRUE && Status = DS => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DATA
135	WDATA	DMPMM1 Set Prm.cnf(-) (Rem Add, Status) /SPara.Active = TRUE && Status = NA && SPara_.NA_To_Abort = FALSE => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr BDiag.Station_Non_Existent := TRUE, SDiag := 1 MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DATA
136	WDATA	DMPMM1 Set Prm.cnf(-) (Rem Add, Status) /SPara.Active = TRUE && Status = NA && SPara_.NA_To_Abort = TRUE => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr BDiag.Station_Non_Existent := TRUE, SDiag := 1 GO_ABORT MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DATA
137	WDATA	DMPMM1 Set Prm.cnf(-) (Rem Add, Status) /SPara.Active = TRUE && Status = RE/RS/RR/UE => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr BDiag.Invalid_Slave_Response := TRUE BDiag.Station_Non_Existent := FALSE SDiag := 1, GO_ABORT MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DATA
138	WDATA	DMPMM1 Set Prm.cnf(+) (Rem Add) /SPara.Active = TRUE => Diag:=TRUE, No_AClr:=TRUE SPara.New_Prm := FALSE MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DATA
139	UNLCK	MSCY1M Abort.req (Rem Add) => ignore	UNLCK
140	UNLCK	MSAL1 Abort.ind (Rem_Add) => ignore	UNLCK

Table 466 (continued)

#	Current state	Event / condition => action	Next state
141	UNLCK	MSCY1M Stop Slave Handler.req (Rem Add) => Prm_Data := SPara .Prm_Data Prm_Data.Unlock_Req := TRUE DMPMM1 Set Prm.req (Rem Add, Prm Data)	SUNLCK
142	UNLCK	MSCY1M Cont Slave Handler.req (Rem Add, Output Clear) => Prm_Data := SPara .Prm_Data Prm_Data.Unlock_Req := TRUE DMPMM1 Set Prm.req (Rem Add, Prm Data)	WUNLCK
143	UNLCK	MSCY1M Get Slave Diag.req (Rem Add) => Diag_Data := BDiag MSCY1M Get Slave Diag.cnf(+) (Rem Add, Diag Data)	UNLCK
144	UNLCK	MSCY1M Set Output.req (Rem Add, Slot Number, Output Data) => BOutput(Slot_Number) := Output_Data MSCY1M Set Output.cnf(+) (Rem Add, Slot Number)	UNLCK
145	UNLCK	MSCY1M Get Input.req (Rem Add, Slot_Number) => Input_Data := BInput(Slot_Number) MSCY1M Get Input.cnf(+) (Rem Add, Input Data)	UNLCK
146	WUNLCK	MSCY1M Abort.req (Rem Add) => ignore	WUNLCK
147	WUNLCK	MSAL1 Abort.ind (Rem_Add) => ignore	WUNLCK
148	WUNLCK	MSCY1M Get Slave Diag.req (Rem Add) => Diag_Data := BDiag MSCY1M Get Slave Diag.cnf(+) (Rem Add, Diag Data)	WUNLCK
149	WUNLCK	MSCY1M Set Output.req (Rem Add, Slot Number, Output Data) => BOutput(Slot_Number) := Output_Data MSCY1M Set Output.cnf(+) (Rem Add, Slot Number)	WUNLCK
150	WUNLCK	MSCY1M Get Input.req (Rem Add, Slot_Number) => Input_Data := BInput(Slot_Number) MSCY1M Get Input.cnf(+) (Rem Add, Input Data)	WUNLCK
151	WUNLCK	DMPMM1 Set Prm.cnf(+) (Rem Add) /SPara.Active = TRUE => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DIAG1
152	WUNLCK	DMPMM1 Set Prm.cnf(+) (Rem Add) /SPara.Active = FALSE => Diag:=FALSE, No_AClr:=TRUE, SDiag := 0 BDiag.Deactivated := TRUE MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DEACT
153	WUNLCK	DMPMM1 Set Prm.cnf(-) (Rem Add, Status) /SPara.Active = TRUE => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DIAG1
154	WUNLCK	DMPMM1 Set Prm.cnf(-) (Rem Add, Status) /SPara.Active = FALSE => Diag:=FALSE, No_AClr:=TRUE, SDiag := 0 BDiag.Deactivated := TRUE MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	DEACT
155	SUNLCK	MSCY1M Abort.req (Rem Add) => ignore	SUNLCK

Table 466 (continued)

#	Current state	Event / condition => action	Next state
156	SUNLCK	MSAL1 Abort.ind (Rem_Add) => ignore	SUNLCK
157	SUNLCK	MSCY1M Get Slave Diag.req (Rem Add) => Diag_Data := BDiag MSCY1M Get Slave Diag.cnf(+) (Rem Add, Diag Data)	SUNLCK
158	SUNLCK	MSCY1M Set Output.req (Rem Add, Slot Number, Output Data) => BOutput(Slot_Number) := Output_Data MSCY1M Set Output.cnf(+) (Rem Add, Slot Number)	SUNLCK
159	SUNLCK	MSCY1M Get Input.req (Rem Add, Slot_Number) => Input_Data := BInput(Slot_Number) MSCY1M Get Input.cnf(+) (Rem Add, Input Data)	SUNLCK
160	SUNLCK	DMPMM1 Set Prm.cnf(+) (Rem Add) => SDiag := 0, BDiag.Deactivated := TRUE MSCY1M Stop Slave Handler.cnf(Rem Add)	STOP
161	SUNLCK	DMPMM1 Set Prm.cnf(-) (Rem Add, Status) => SDiag := 0, BDiag.Deactivated := TRUE MSCY1M Stop Slave Handler.cnf(Rem Add)	STOP
162	WSTPCC	MSAL1 Start.cnf (Rem Add) => MSAL1M_Started := FALSE	WSTPCC
163	WSTPCC	MSAL1 Stop.cnf (Rem Add) /MSAL1M_Started = TRUE => MSCY1M Stopped.ind (Rem Add)	UNLCK
164	WSTPCC	MSAL1 Stop.cnf (Rem Add) /MSAL1M_Started = FALSE => ignore	UNLCK
165	WSTPCC	MSCY1M Cont Slave Handler.req (Rem Add, Output Clear) => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr OR NOT SPara.Active MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No AClr)	WSTPCC
166	WSTPCC	MSCY1M Abort.req (Rem Add) => ignore	WSTPCC
167	WSTPCC	MSAL1 Abort.ind (Rem_Add) => ignore	WSTPCC
168	WSTPCC	MSCY1M Get Slave Diag.req (Rem Add) => Diag_Data := BDiag MSCY1M Get Slave Diag.cnf(+) (Rem Add, Diag Data)	WSTPCC
169	WSTPCC	MSCY1M Set Output.req (Rem Add, Slot Number, Output Data) => BOutput(Slot_Number) := Output_Data MSCY1M Set Output.cnf(+) (Rem Add, Slot Number)	WSTPCC
170	WSTPCC	MSCY1M Get Input.req (Rem Add, Slot_Number) => Input_Data := BInput(Slot_Number) MSCY1M Get Input.cnf(+) (Rem Add, Input Data)	WSTPCC
171	WSTPCS	MSAL1 Start.cnf (Rem Add) => MSAL1M_Started := FALSE	WSTPCS
172	WSTPCS	MSAL1 Stop.cnf (Rem Add) /MSAL1M_Started = TRUE => Prm_Data := SPara .Prm_Data Prm_Data.Unlock_Req := TRUE MSCY1M Stopped.ind (Rem Add) DMPMM1 Set Prm.req (Rem Add, Prm Data)	SUNLCK

Table 466 (continued)

#	Current state	Event / condition => action	Next state
173	WSTPCS	MSAL1 Stop.cnf (Rem Add) /MSAL1M_Started = FALSE => Prm_Data := SPara .Prm_Data Prm_Data.Unlock_Req := TRUE DMPMM1 Set Prm.req (Rem Add, Prm Data)	SUNLCK
174	WSTPCS	MSCY1M Abort.req (Rem Add) => ignore	WSTPCS
175	WSTPCS	MSAL1 Abort.ind (Rem_Add) => ignore	WSTPCS
176	WSTPCS	MSCY1M Get Slave Diag.req (Rem Add) => Diag_Data := BDiag MSCY1M Get Slave Diag.cnf(+) (Rem Add, Diag Data)	WSTPCS
177	WSTPCS	MSCY1M Set Output.req (Rem Add, Slot Number, Output Data) => BOutput(Slot_Number) := Output_Data MSCY1M Set Output.cnf(+) (Rem Add, Slot Number)	WSTPCS
178	WSTPCS	MSCY1M Get Input.req (Rem Add, Slot_Number) => Input_Data := BInput(Slot_Number) MSCY1M Get Input.cnf(+) (Rem Add, Input Data)	WSTPCS
179	ANY-STATE	MSCY1M Reset.req (Rem Add) => MSCY1M Reset.cnf (Rem Add)	POWER-ON
180	WPRM	DMPMM1 Set Prm.cnf(+) (Rem Add) /SPara.Active = TRUE && ExtPrmFlag = TRUE => ExtPrmBsy:=TRUE MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No ACIr)	PRM
181	PRM	MSCY1M Cont Slave Handler.req (Rem Add, Output Clear) / ExtPrmBsy=TRUE => Prm_Data := SPara .ExtPrm_Data DMPMM1 Set ExtPrm.req (Rem Add, Prm Data)	WPRM
182	WPRM	DMPMM1 Set ExtPrm.cnf(+) (Rem Add) /SPara.Active = TRUE => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr, ExtPrmBsy:=FALSE MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No ACIr)	CFG
183	WPRM	DMPMM1 Set ExtPrm.cnf(+) (Rem Add) /SPara.Active = FALSE => Diag:=FALSE, No_AClr:=TRUE, ExtPrmBsy:=FALSE MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No ACIr)	UNLCK
184	WPRM	DMPMM1 Set ExtPrm.cnf(-) (Rem Add, Status) /SPara.Active = TRUE && Status = DS => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No ACIr)	PRM
185	WPRM	DMPMM1 Set ExtPrm.cnf(-) (Rem Add, Status) /SPara.Active = TRUE && Status = NA => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr BDiag.Station_Non_Existent := TRUE, ExtPrmBsy:=FALSE MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No ACIr)	DIAG1
186	WPRM	DMPMM1 Set ExtPrm.cnf(-) (Rem Add, Status) /SPara.Active = TRUE && Status = RE/RS/RR/UE => Diag:=TRUE, No_AClr:=SPara_.Ignore_AClr BDiag.Invalid_Slave_Response := TRUE, ExtPrmBsy:=FALSE MSCY1M Cont Slave Handler.cnf(Rem Add, Diag, No ACIr)	DIAG1

6.6.8 MSAL1M

6.6.8.1 Primitive definitions

6.6.8.1.1 Primitives exchanged between FSPMM1 and MSAL1M

Table 467 shows the service primitives including their associated parameters issued by the FSPMM1 and received by the MSAL1M.

Table 467 – Primitives issued by FSPMM1 to MSAL1M

Primitive name	Source	Associated parameters	Functions
MInit.req	FSPMM1	(none)	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Reset.req	FSPMM1	(none)	
Abort.req	FSPMM1	(none)	

Table 468 shows the service primitives including their associated parameters issued by the MSAL1M and received by the FSPMM1.

Table 468 – Primitives issued by MSAL1M to FSPMM1

Primitive name	Source	Associated parameters	Functions
MInit.cnf	MSAL1M	(none)	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Reset.cnf	MSAL1M	(none)	
Fault.ind	MSAL1M	(none)	
Started.ind	MSAL1M	Alarm Limit	
Stopped.ind	MSAL1M	(none)	
Alarm Notification.ind	MSAL1M	Slot Number, Alarm Type, Seq Nr, Add Ack, Alarm Specifier, Alarm Data	

6.6.8.1.2 Primitives exchanged between MSCY1M and MSAL1M

Table 469 shows the service primitives including their associated parameters issued by the MSCY1M and received by the MSAL1M.

Table 469 – Primitives issued by MSCY1M to MSAL1M

Primitive name	Source	Associated parameters	Functions
Start.req	MSCY1M	Rem Add	Start MS1-processing
Stop.req	MSCY1M	Rem Add	Stop MS1-processing
Change Diag.req	MSCY1M	Rem Add, Diag Data	Indicates change of diagnosis

Table 470 shows the service primitives including their associated parameters issued by the MSAL1M and received by the MSCY1M.

Table 470 – Primitives issued by MSAL1M to MSCY1M

Primitive name	Source	Associated parameters	Functions
Abort.ind	MSAL1M	Rem Add	MSAL1 detected an error
Start.cnf	MSAL1M	Rem Add	Start completed
Stop.cnf	MSAL1M	Rem Add	Stop completed

6.6.8.1.3 Parameters of MSAL1M primitives

The parameters used with the primitives exchanged between the MSAL1M and MSCY1M are described in Table 471.

Table 471 – Parameter used with primitives exchanged between MSAL1M and MSCY1M

Parameter name	Description
Rem Add	This parameter conveys the DL address of the assigned DP-slave.
Diag Data	This parameter conveys the new diagnosis information of a DP-slave

6.6.8.2 State machine description

The Alarm State Machine of a DP-master (Class 1) handles alarm messages, which have been indicated by a DP-slave.

When started the State Machine waits for being started by MSCY1M. The Start.req follows, when the assigned DP-slave has reached the state DATA-EXCH. MSAL1M will be stopped by MSCY1M if the corresponding DP-slave leaves the state DATA-EXCH.

The service Slave_Diag.cnf provides new Diag_Data to MSCY1M. MSCY1M copies the diagnosis information into the User data interface. Additionally MSCY1M hands over an Alarm in Ext_Diag_Data to MSAL1M.

Alarm-Requests are indicated to the User. The User has to confirm the Alarm-Indications with the service Alarm_Ack.

The MSAC1 State Machine indicates the termination of the alarm sequence with the service primitive Alarm_Ack.cnf.

Two types of alarm handling are provided by a DP-master (Class 1) the type mode and the sequence mode. Depending on the chosen mode the number of outstanding alarms per DP-slave differs. In type mode one pending alarm per type and per DP-slave is allowed. 6 different alarm types are defined. In sequence mode the alarm type is irrelevant. Only the number of simultaneously processed alarms for each DP-slave is limited to a value between 2 and 32.

Local variables

Alarm_Sequence

(Boolean)

This variable indicates whether:

only one alarm of a specific alarm type can be active at one time (type mode: Alarm_Sequence=False) or

several alarms (2 to 32) of any type can be active at one time (sequence mode: Alarm_Sequence=True).

Alarm_State_Table

(Unsigned8)

The Alarm_State_Table is a two dimensional array with 7 * 32 elements. Each element within the array stores information about the actual state of any alarm. The array is indexed with the alarm class calculated from the Alarm_Type and the Seq_Nr of the alarm.

Range

Table 472 shows the possible values for the actual states of alarms in the Alarm_State_Table.

Table 472 – Possible values in the Alarm_State_Table

Value	Meaning
idle	No service is being processed according to the actual index.
w_res	Wait for MSAL1_Alarm.res according to the actual index.
w_req	For this index (Alarm_Type,Seq_Nr) an Alarm_Acknowledge is stored in the Alarm_Ack_FIFO and MSAL1M waits for another MSAC1M_Alarm_Ack.cnf.
w_con	Wait for MSAC1M_Alarm_Ack.cnf according to the actual index.

Alarm_Limit

(Unsigned8)

This variable indicates the maximum number of parallel alarms of each Slave.

Range: 7..Bus_Para.Alarm_Max

Alarm_Decompose

(Array 0..7 of Unsigned8)

Table for Decoding the number of parallel alarms.

Range:

Alarm_Decompose[0]= 1
 Alarm_Decompose[1]= 2
 Alarm_Decompose[2]= 4
 Alarm_Decompose[3]= 8
 Alarm_Decompose[4]=12
 Alarm_Decompose[5]=16
 Alarm_Decompose[6]=24
 Alarm_Decompose[7]=32

Actual_Max_Alarm_Len

(Unsigned8)

Actual_Max_Alarm_Len contains the actual maximum length of an Alarm.

Range: 4..64

Alarm_Count

(Unsigned8)

This counter contains the number of alarms which have actually been sent by the Slave. The counter is only used in the sequence mode.

Range: 0..32

Waiting_For_Alarm_Ack

(Boolean)

This variable shows, whether the Alarm State Machine is actually waiting for the confirmation of any alarm acknowledge (True) or not (False).

Alarm_Ack_FIFO

The Alarm_Ack_FIFO is used to store the Alarm_Ack which are still to be sent. Each element of this alarm acknowledge FIFO has 3 Unsigned8 entries: Alarm_Type, Slot_Number and Seq_Nr. The FIFO shall be able to hold up to 32 respectively Bus_Para.Alarm_Max entries.

Alarm_Ack_FIFO_Filled

(Boolean)

This variable indicates whether the Alarm_Ack_FIFO contains at least one entry (True) or it is empty (False). The value of the variable is adapted with each access to the FIFO.

Act_Alarm_PDU

(Octet-String)

The structure Act_Alarm_PDU is temporarily used to store one Alarm while being evaluated.

Stored_Alarm_PDU

(Octet-String)

The structure Stored_Alarm_PDU is used to store exactly one Alarm-PDU. Up to one Alarm-PDU shall be saved locally during waiting for MSAC1M_Alarm_Ack.cnf of the according Seq_Nr, because only one channel for alarm acknowledges exists.

Actual_Enabled_Alarms

(Bitarea[0..7])

This variable indicates the type of alarms which are actually supported by the Master.

- Bit 0 = reserved
- Bit 1 = reserved
- Bit 2 = Update_Alarm
- Bit 3 = Status_Alarm
- Bit 4 = Manufacturer_Specific_Alarm
- Bit 5 = Diagnostic_Alarm
- Bit 6 = Process_Alarm
- Bit 7 = Pull_Plug_Alarm

Functions**Fill_Alarm_State_Table(State)**

This function sets each entry of the two dimensional Alarm_State_Table to the given initial value of State. Valid parameters for State are idle, w_res, w_req and w_con. The function has no return value.

Reset_Alarm_Ack_FIFO()

This function resets the FIFO for alarm acknowledges. During this function all entries are cleared and the variable Alarm_Ack_FIFO_Filled is thereby set to False. The function has no return value.

Store_To_Alarm_Ack_FIFO(Alarm_Type, Slot_Number, Seq_Nr)

By means of this function one set of the 3 alarm recognition parameters Alarm_Type, Slot_Number and Seq_Nr is stored from the given parameters into the Alarm_Ack_FIFO. The function has no return value.

Load_From_Alarm_Ack_FIFO(Alarm_Type, Slot_Number, Seq_Nr)

By means of this function one set of the 3 alarm recognition parameters Alarm_Type, Slot_Number and Seq_Nr is read from the Alarm_Ack_FIFO and stored into the given parameters. The function has no return value.

AcIs(Alarm_Type)

This function calculates the Alarm_Type related index as return value as follows:

- if (Alarm_Type = 1) return 0
- if (Alarm_Type = 2) return 1
- if (Alarm_Type = 3) return 2
- if (Alarm_Type = 4) return 3
- if (Alarm_Type = 5) return 4
- if (Alarm_Type = 6) return 5
- if (Alarm_Type >= 32) && (Alarm_Type <= 126) return 6

Macros**ALARM_ENABLED**

```
(
  ((Alarm_Type=3 OR Alarm_Type=4)
  && Actual_Enabled_Alarms[7]=TRUE)
  || ((Alarm_Type=2) && Actual_Enabled_Alarms[6]=TRUE)
  || ((Alarm_Type=1) && Actual_Enabled_Alarms[5]=TRUE)
  || ((Alarm_Type>=32 && Alarm_Type<=126)
```

```

&& Actual_Enabled_Alarms[4]=TRUE)
|| ((Alarm_Type=5) && Actual_Enabled_Alarms[3]=TRUE)
|| ((Alarm_Type=6) && Actual_Enabled_Alarms[2]=TRUE)
)

```

6.6.8.3 MSAL1M state table

Table 473 contains the complete description of the MSAL1M state machine.

Table 473 – MSAL1M state table

#	Current state	Event / condition => action	Next state
1	POWER-ON	MSAL1M_Minit_MS1.req(Rem_Add) =>MSAL1M_Minit_MS1.cnf(Rem_Add)	WSTART
2	POWER-ON	MSAL1M_Abort.req(Rem_Add) =>ignore	POWER-ON
3	POWER-ON	MSAL1M_Alarm_Ack.req(Rem_Add, Alarm_Type, Slot_Number, Seq_Nr) =>MSAL1M_Alarm_Ack.cnf(-)(Rem_Add, Status:=Not_Initialized)	POWER-ON
4	WSTART	MSCY1M_Start.req(Rem_Add) /Alarm_Mode = 0 =>Alarm_Sequence := False Alarm_Limit := 7 Actual_Enabled_Alarms := Enabled_Alarms Actual_Max_Alarm_Len := Max_Alarm_Len Fill_Alarm_State_Table(idle) Reset_Alarm_Ack_FIFO() Alarm_Count := 0 Clear_Stored_Alarm_PDU Waiting_For_Alarm_Ack := False MSAL1M_Started.ind(Rem_Add, Alarm_Limit) MSCY1M_Start.cnf(Rem_Add)	W-DIA-EVENT
5	WSTART	MSCY1M_Start.req(Rem_Add) /Alarm_Mode <> 0 =>Alarm_Sequence := True Alarm_Limit := min(Bus_Para.Alarm_Max, Alarm_Decompose[Alarm_Mode]) Actual_Enabled_Alarms := Enabled_Alarms Actual_Max_Alarm_Len := Max_Alarm_Len Fill_Alarm_State_Table(idle) Reset_Alarm_Ack_FIFO() Alarm_Count := 0 Clear_Stored_Alarm_PDU Waiting_For_Alarm_Ack := False MSAL1M_Started.ind(Rem_Add, Alarm_Limit) MSCY1M_Start.cnf(Rem_Add)	W-DIA-EVENT
6	WSTART	MSCY1M_Stop.req(Rem_Add) =>MSAL1M_Fault.ind	POWER-ON
7	WSTART	MSCY1M_Change_Diag.req(Rem_Add, Diag_Block) =>MSAL1M_Fault.ind	WSTART
8	WSTART	MSAL1M_Abort.req(Rem_Add) =>ignore	WSTART
9	WSTART	MSAL1M_Alarm_Ack.req(Rem_Add, Alarm_Type, Slot_Number, Seq_Nr) =>MSAL1M_Alarm_Ack.cnf(-)(Rem_Add, Status:=Not_Started)	WSTART
10	WSTART	MSAC1M_Alarm_Ack.cnf(+)(Rem_Add, Slot_Number, Alarm_Type, Seq_Nr) =>MSAL1M_Fault.ind	POWER-ON
11	W-DIA-EVENT	MSAL1M_Minit_MS1.req(Rem_Add) =>MSAL1M_Fault.ind	POWER-ON
12	W-DIA-EVENT	MSAL1M_Abort.req(Rem_Add) =>MSAL1M_Abort.ind(Rem_Add)	WSTART

Table 473 (continued)

#	Current state	Event / condition => action	Next state
13	W-DIA-EVENT	MSCY1M_Start.req(Rem_Add) =>MSAL1M_Fault.ind	POWER-ON
14	W-DIA-EVENT	MSCY1M_Stop.req(Rem_Add) =>MSAL1M_Stopped.ind(Rem_Add) MSCY1M_Stop.cnf(Rem_Add)	WSTART
15	W-DIA-EVENT	MSCY1M_Change_Diag.req(Rem_Add, Diag_Block) =>Act_Alarm_PDU := Alarm-PDU	CHK-DIA-ALARM
16	W-DIA-EVENT	MSAL1M_Alarm_Ack.req(Rem_Add, Alarm_Type, Slot_Number, Seq_Nr) /ALARM_ENABLED(Alarm_Type) = False =>MSAL1M_Alarm_Ack.cnf(-)(Rem_Add, Status:=Not_Enabled)	W-DIA-EVENT
17	W-DIA-EVENT	MSAL1M_Alarm_Ack.req(Rem_Add, Alarm_Type, Slot_Number, Seq_Nr) /ALARM_ENABLED(Alarm_Type) = True && Alarm_State_Table[Acls(Alarm_Type), Seq_Nr] <> w_res =>MSAL1M_Alarm_Ack.cnf(-)(Rem_Add, Status:=Alarm_Not_Pending)	W-DIA-EVENT
18	W-DIA-EVENT	MSAL1M_Alarm_Ack.req(Rem_Add, Alarm_Type, Slot_Number, Seq_Nr) /ALARM_ENABLED(Alarm_Type) = True && Alarm_Sequence = False && Alarm_State_Table[Acls(Alarm_Type), Seq_Nr] = w_res && Waiting_For_Alarm_Ack = False =>Alarm_State_Table[Acls(Alarm_Type), Seq_Nr] := w_con Waiting_For_Alarm_Ack := True MSAC1M_Alarm_Ack.req(Rem_Add, Slot_Number, Alarm_Type, Seq_Nr)	W-DIA-EVENT
19	W-DIA-EVENT	MSAL1M_Alarm_Ack.req(Rem_Add, Alarm_Type, Slot_Number, Seq_Nr) /ALARM_ENABLED(Alarm-PDU.Alarm_Type) = True && Alarm_Sequence = False && Alarm_State_Table[Acls(Alarm_Type), Seq_Nr] = w_res && Waiting_For_Alarm_Ack = True =>Alarm_State_Table[Acls(Alarm_Type), Seq_Nr] := w_req Store_To_Alarm_Ack_FIFO(Alarm_Type, Slot_Number, Seq_Nr)	W-DIA-EVENT
20	W-DIA-EVENT	MSAL1M_Alarm_Ack.req(Rem_Add, Alarm_Type, Slot_Number, Seq_Nr) /ALARM_ENABLED(Alarm_Type) = True && Alarm_Sequence = True && Alarm_State_Table[Acls(Alarm_Type), Seq_Nr] = w_res && Waiting_For_Alarm_Ack = False =>Alarm_State_Table[Acls(Alarm_Type), Seq_Nr] := w_con Alarm_Count := Alarm_Count-1 Waiting_For_Alarm_Ack := True MSAC1M_Alarm_Ack.req(Rem_Add, Slot_Number, Alarm_Type, Seq_Nr)	W-DIA-EVENT
21	W-DIA-EVENT	MSAL1M_Alarm_Ack.req(Rem_Add, Alarm_Type, Slot_Number, Seq_Nr) /ALARM_ENABLED(Alarm_Type) = True && Alarm_Sequence = True && Alarm_State_Table[Acls(Alarm_Type), Seq_Nr] = w_res && Waiting_For_Alarm_Ack = True =>Alarm_State_Table[Acls(Alarm_Type), Seq_Nr] := w_req Store_To_Alarm_Ack_FIFO(Alarm_Type, Slot_Number, Seq_Nr)	W-DIA-EVENT
22	W-DIA-EVENT	MSAC1M_Alarm_Ack.cnf(+)(Rem_Add, Slot_Number, Alarm_Type, Seq_Nr) /Waiting_For_Alarm_Ack = False =>MSAL1M_Fault.ind	POWER-ON
23	W-DIA-EVENT	MSAC1M_Alarm_Ack.cnf(+)(Rem_Add, Slot_Number, Alarm_Type, Seq_Nr) /Waiting_For_Alarm_Ack = True && Alarm_State_Table[Acls(Alarm_Type), Seq_Nr] <> w_con =>MSAL1M_Abort.ind(Rem_Add) MSAL1M_Stopped.ind(Rem_Add)	WSTART

Table 473 (continued)

#	Current state	Event / condition => action	Next state
24	W-DIA-EVENT	MSAC1M_Alarm_Ack.cnf(+)(Rem_Add, Slot_Number, Alarm_Type, Seq_Nr) /Waiting_For_Alarm_Ack = True && Alarm_State_Table[Acls(Alarm_Type), Seq_Nr] = w_con =>Alarm_State_Table[Acls(Alarm_Type), Seq_Nr] := idle Waiting_For_Alarm_Ack := False MSAL1M_Alarm_Ack.cnf(+)(Rem_Add, Slot_Number, Alarm_Type, Seq_Nr)	SEND-NEXT-ALARM
25	CHK-DIA-ALARM	/Act_Alarm_PDU not present =>ignore	W-DIA-EVENT
26	CHK-DIA-ALARM	/Act_Alarm_PDU present && ALARM_ENABLED(Act_Alarm_PDU.Alarm_Type) = False =>MSAL1M_Abort.ind(Rem_Add) MSAL1M_Stopped.ind(Rem_Add)	WSTART
27	CHK-DIA-ALARM	/Act_Alarm_PDU present && ALARM_ENABLED(Act_Alarm_PDU.Alarm_Type) = True && Alarm_Sequence = False && Alarm_State_Table[Acls(Act_Alarm_PDU.Alarm_Type), Act_Alarm_PDU.Seq_Nr] = w_res/w_req =>MSAL1M_Abort.ind(Rem_Add) MSAL1M_Stopped.ind(Rem_Add)	WSTART
28	CHK-DIA-ALARM	/Act_Alarm_PDU present && ALARM_ENABLED(Act_Alarm_PDU.Alarm_Type) = True && Alarm_Sequence = False && Alarm_State_Table[Acls(Act_Alarm_PDU.Alarm_Type), Act_Alarm_PDU.Seq_Nr] = w_con && Stored_Alarm_PDU present =>MSAL1M_Abort.ind(Rem_Add) MSAL1M_Stopped.ind(Rem_Add)	WSTART
29	CHK-DIA-ALARM	/Act_Alarm_PDU present && ALARM_ENABLED(Act_Alarm_PDU.Alarm_Type) = True && Alarm_Sequence = False && Alarm_State_Table[Acls(Act_Alarm_PDU.Alarm_Type), Act_Alarm_PDU.Seq_Nr] = idle =>Alarm_State_Table[Acls(Act_Alarm_PDU.Alarm_Type), Act_Alarm_PDU.Seq_Nr] := w_res MSAL1M_Alarm_Notification.ind(Rem_Add, Alarm_Type:=Act_Alarm_PDU.Alarm_Type, Slot_Number:=Act_Alarm_PDU.Slot_Number, Seq_Nr:=Act_Alarm_PDU.Seq_Nr, Alarm_Specifier:=Act_Alarm_PDU.Alarm_Specifier, Add_Ack:=Act_Alarm_PDU.Add_Ack, Alarm_Data:=Act_Alarm_PDU.Diagnostic_User_Data)	W-DIA-EVENT
30	CHK-DIA-ALARM	/Act_Alarm_PDU present && ALARM_ENABLED(Act_Alarm_PDU.Alarm_Type) = True && Alarm_Sequence = False && Alarm_State_Table[Acls(Act_Alarm_PDU.Alarm_Type), Act_Alarm_PDU.Seq_Nr] = w_con && Stored_Alarm_PDU not present =>Stored_Alarm_PDU := Act_Alarm_PDU	W-DIA-EVENT
31	CHK-DIA-ALARM	/Act_Alarm_PDU present && ALARM_ENABLED(Act_Alarm_PDU.Alarm_Type) = True && Alarm_Sequence = True && Alarm_Count >= Alarm_Limit =>MSAL1M_Abort.ind(Rem_Add) MSAL1M_Stopped.ind(Rem_Add)	WSTART

Table 473 (continued)

#	Current state	Event / condition => action	Next state
32	CHK-DIA-ALARM	/Act_Alarm_PDU present && ALARM_ENABLED(Act_Alarm_PDU.Alarm_Type) = True && Alarm_Sequence = True && Alarm_Count < Alarm_Limit && Alarm_State_Table[Acls(Act_Alarm_PDU.Alarm_Type), Act_Alarm_PDU.Seq_Nr] = w_res/w_req =>MSAL1M_Abort.ind(Rem_Add) MSAL1M_Stopped.ind(Rem_Add)	WSTART
33	CHK-DIA-ALARM	/Act_Alarm_PDU present && ALARM_ENABLED(Act_Alarm_PDU.Alarm_Type) = True && Alarm_Sequence = True && Alarm_Count < Alarm_Limit && Alarm_State_Table[Acls(Act_Alarm_PDU.Alarm_Type), Act_Alarm_PDU.Seq_Nr] = idle =>Alarm_State_Table[Acls(Act_Alarm_PDU.Alarm_Type), Act_Alarm_PDU.Seq_Nr] := w_res Alarm_Count := Alarm_Count+1 MSAL1M_Alarm_Ack.ind(Rem_Add, Alarm_Type:=Act_Alarm_PDU.Alarm_Type, Slot_Number:=Act_Alarm_PDU.Slot_Number, Seq_Nr:=Act_Alarm_PDU.Seq_Nr, Alarm_Specifier:=Act_Alarm_PDU.Alarm_Specifier, Add_Ack:=Act_Alarm_PDU.Add_Ack, Alarm_Data:=Act_Alarm_PDU.Diagnostic_User_Data)	W-DIA-EVENT
34	CHK-DIA-ALARM	/Act_Alarm_PDU present && ALARM_ENABLED(Act_Alarm_PDU.Alarm_Type) = True && Alarm_Sequence = True && Alarm_Count < Alarm_Limit && Alarm_State_Table[Acls(Act_Alarm_PDU.Alarm_Type), Act_Alarm_PDU.Seq_Nr] = w_con && Stored_Act_Alarm_PDU present =>MSAL1M_Abort.ind(Rem_Add) MSAL1M_Stopped.ind(Rem_Add)	WSTART
35	CHK-DIA-ALARM	/Act_Alarm_PDU present && ALARM_ENABLED(Act_Alarm_PDU.Alarm_Type) = True && Alarm_Sequence = True && Alarm_Count < Alarm_Limit && Alarm_State_Table[Acls(Act_Alarm_PDU.Alarm_Type), Act_Alarm_PDU.Seq_Nr] = w_con && Stored_Alarm_PDU not present =>Stored_Alarm_PDU := Act_Alarm_PDU Alarm_Count := Alarm_Count+1	W-DIA-EVENT
36	SEND-NEXT-ALARM	/Stored_Alarm_PDU not present =>ignore	READ-NEXT-ACK
37	SEND-NEXT-ALARM	/Stored_Alarm_PDU present =>Alarm_State_Table[Acls(Stored_Alarm_PDU.Alarm_Type), Stored_Alarm_PDU.Seq_Nr] := w_res Clear Stored_Alarm_PDU MSAL1M_Alarm_Notification.ind(Rem_Add, Alarm_Type:=Stored_Alarm_PDU.Alarm_Type, Slot_Number:=Stored_Alarm_PDU.Slot_Number, Seq_Nr:=Stored_Alarm_PDU.Seq_Nr, Alarm_Specifier:=Stored_Alarm_PDU.Alarm_Specifier, Add_Ack:=Stored_Alarm_PDU.Add_Ack, Alarm_Data:=Stored_Alarm_PDU.Diagnostic_User_Data)	READ-NEXT-ACK
38	READ-NEXT-ACK	/Alarm_Ack_FIFO_Filled = False =>ignore	W-DIA-EVENT

Table 473 (continued)

#	Current state	Event / condition => action	Next state
39	READ-NEXT-ACK	/Alarm_Ack_FIFO_Filled = True =>Load_From_Alarm_Ack_FIFO(New_Alarm_Type, New_Slot_Number, New_Seq_Nr)	SEND-NEXT-ACK
40	SEND-NEXT-ACK	/Alarm_State_Table[Acls(New_Alarm_Type), New_Seq_Nr] <> w_req =>MSAL1M_Abort.ind(Rem_Add) MSAL1M_Stopped.ind(Rem_Add)	WSTART
41	SEND-NEXT-ACK	/Alarm_State_Table[Acls(New_Alarm_Type), New_Seq_Nr] = w_req =>Alarm_State_Table[Acls(New_Alarm_Type), New_Seq_Nr] := w_con Alarm_Count := Alarm_Count-1 Waiting_For_Alarm_Ack := True MSAC1M_Alarm_Ack.req(Rem_Add, Slot_Number:=New_Slot_Number, Alarm_Type:=New_Alarm_Type, Seq_Nr:=New_Seq_Nr)	W-DIA-EVENT
42	ANY-STATE	MSAL1M_Reset.req(Rem_Add) =>MSAL1M_Reset.cnf(Rem_Add)	POWER-ON

6.6.9 MSAC1M

6.6.9.1 Primitive definitions

6.6.9.1.1 Primitives exchanged between FSPMM1 and MSAC1M

Table 474 shows the service primitives including their associated parameters issued by the FSPMM1 and received by the MSAC1M.

Table 474 – Primitives issued by FSPMM1 to MSAC1M

Primitive name	Source	Associated parameters	Functions
Read.req	FSPMM1	Rem Add, Slot Number, Index, Length	
Write.req	FSPMM1	Rem Add, Slot Number, Index, Length, Data	
Alarm Ack.req	FSPMM1	Rem Add, Slot Number, Alarm Type, Seq Nr	

Table 475 shows the service primitives including their associated parameters issued by the MSAC1M and received by the FSPMM1.

Table 475 – Primitives issued by MSAC1M to FSPMM1

Primitive name	Source	Associated parameters	Functions
Reject.ind	MSAC1M	Rem Add, Status	
Read.cnf(+)	MSAC1M	Rem Add, Length, Data	
Read.cnf(-)	MSAC1M	Rem Add, Error Decode, Error Code 1 Error Code 2	
Write.cnf(+)	MSAC1M	Rem Add, Length	
Write.cnf(-)	MSAC1M	Rem Add, Error Decode, Error Code 1 Error Code 2	
Alarm Ack.cnf(+)	MSAC1M	Rem Add, Slot Number, Alarm Type, Seq Nr	
Alarm Ack.cnf(-)	MSAC1M	Rem Add, Slot Number, Alarm Type, Seq Nr	

6.6.9.1.2 Primitives exchanged between MSAC1M and MSAL1M

Table 476 shows the service primitives including their associated parameters issued by MSAL1M and received by the MSAC1M.

Table 476 – Primitives issued by MSAL1M to MSAC1M

Primitive name	Source	Associated parameters	Functions
SInit MSAC1.req	MSAL1M	Rem Add	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Reset.req	MSAL1M	Rem Add	
Start.req	MSAL1M	Rem Add	Start MS1-processing
Stop.req	MSAL1M	Rem Add	Stop MS1-processing

Table 477 shows the service primitives including their associated parameters issued by MSAC1M and received by the MSAL1M.

Table 477 – Primitives issued by MSAC1M to MSAL1M

Primitive name	Source	Associated parameters	Functions
SInit MSAC1.cnf	MSAC1M	Rem Add	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Reset.cnf	MSAC1M	Rem Add	
Start.cnf	MSAC1M	Rem Add	Start completed
Stop.cnf	MSAC1M	Rem Add	Stop completed
Fault.ind	MSAC1M	(none)	
Abort.ind	MSAC1M	Rem Add	MSAC1 detected an error

6.6.9.1.3 Parameters of MSAL1M primitives

The parameters used with the primitives exchanged between the MSAL1M and MSCY1M are described in Table 478.

Table 478 – Parameter used with primitives exchanged between MSAL1M and MSCY1M

Parameter name	Description
Rem Add	This parameter conveys the DL address of the assigned DP-slave.

6.6.9.2 State machine description

The MSAC1M State Machine has the three states POWER-ON, CLOSED and OPEN. With the Init.req from the FSPMM1, the State Machine is initialised and changes to CLOSED. The state transition from CLOSED to OPEN is caused by the Start.req from the Slave-Handler (MSCY1M). The Slave-Handler is calling this service as soon as the DP-slave has entered the DATA-EXCH-state. When the DP-slave leaves the DATA-EXCH-state, the Stop.req service is called and the MSAC1M State Machine changes from OPEN to CLOSED. A Read/_Write request with an illegal parameter or a parallel service request while the last Read/Write is still being processed, is rejected to the User with a Reject.ind. An Alarm_Ack.req containing illegal service parameters or a parallel Alarm_Ack.req while the last Alarm_Ack.req is still being processed, is causing a state transition to POWER-ON and a Fault.ind to the FSPMM1.

The MSAC1M State Machine takes into account that the Alarm_Ack can be handled either on the SAP 51 as the Read/Write-service or on SAP 50.

The responses for Read and Write services are monitored by means of C1 Timer.

MSAC1M_Alarm_Ack is handled on the same SAP as MSAC1M_Read/Write

If a MSAC1M_Alarm_Ack is requested by the MSAL1M State Machine or a Read/_Write is requested by the User while no service is being processed, the service request is passed directly to the DL. If a Alarm_Ack.req is requested and a Read/Write is being processed, the Alarm_Ack.req is stored and sent as soon as the confirmation of the Read/Write is received. When a Read/Write.req is requested and a Alarm_Ack is being processed, the Read/Write is stored and sent as soon as the confirmation of the Alarm_Ack is received.

When recognising an error while a service request is being processed, the MSAC1M State Machine

aborts the MS0 connection by indicating a Abort.ind to the MSCY1S,
deletes an eventually stored service,
informs the User with a Reject.ind in case of an outstanding Read/_Write service
and changes from state OPEN to CLOSE.

When a Stop.req is requested by the Slave-Handler, the MSAC1M first waits for an eventually outstanding DATA-REPLY.cnf. After this the Stop.req will be processed. A Read/_Write-confirmation is passed to the User and a possibly stored Alarm_Ack.req is deleted. In case of an Alarm_Ack confirmation, this confirmation is ignored, and a possibly outstanding Read/_Write is deleted and the User is informed with a Reject.ind. After that, the Stop.cnf is passed to the Slave-Handler and the MSAC1M State Machine changes from state OPEN to CLOSE.

MSAC1M_Alarm_Ack is handled on SAP 50

When a Alarm_Ack is requested by the MSAL1M State Machine or a Read/_Write service is requested by the User, the service request is passed directly to the DL.

When recognising an error while a service request is being processed on the other SAP, the MSAC1M State Machine

– waits for the confirmation of the service on the other SAP

- then aborts the MSCY1M connection by indicating a Abort.ind to the Slave-Handler MSCY1S,
- informs the User with a Reject.ind in case of an outstanding _Read/_Write service
- and changes from state OPEN to CLOSE.

When a Stop.req is requested by the Slave-Handler, the MSAC1M first waits for eventually outstanding DL-DATA-REPLY.cnf. After this the Stop.req will be processed. An outstanding MSAC1M_Read/_Write confirmation is passed to the User and an outstanding Alarm_Ack confirmation is ignored. Then the Stop.cnf is passed to the Slave-Handler and the MSAC1M State Machine changes from state OPEN to CLOSE.

Local Variables

Go_Abort

(Boolean)

This variable is set TRUE if the Stop.req is in use.

Internal_Abort

(Boolean)

This variable is set TRUE if an error in the Slave response occurs.

Max_Data_Length

(Unsigned8)

This variable defines the maximum size of MS1-PDU for all DP-slaves.

Alarm_DSAP

(Unsigned8)

This variable contains the DSAP the Alarm_Ack is sent to.

Service_Header

(Unsigned8)

This variable contains the Function_Num of the MS1-Service actually sent to the DSAP 51.

Service_Buffer

(Octet-String)

This variable contains a Read or Write Request-PDU if a Alarm_Ack is in use (DSAP 51).

Alarm_Buffer

(Octet-String)

This variable contains a Alarm_Ack if a Read or Write Request-PDU is in use (DSAP 51).

Src

(Unsigned8)

This variable contains the number of the actual processed Read or Write Services (only 0 or 1 is possible).

Arc

(Unsigned8)

This variable contains the number of the actual Alarm_Ack Services (only 0 or 1 is possible)

Stop_Pending

(Boolean)

This variable is set TRUE if an Stop.cnf is pending.

Start_C1_Monitoring

(Boolean)

This variable is set TRUE if the user issues a Read or Write request service primitive. It is used to distinguish the first polling from later once and to start the C1 Timer for monitoring the user response.

C1_Timer_Expired

(Boolean)

This variable is set TRUE if the C1 Timer expires. With the next confirmation it is used to abort the connection if no response is available. It is only relevant if at least one polling for the response has been issued.

C1_Timer

Timer with 10 ms base

This timer is used to monitor the response of the DP-slave according the values specified in the GSD of the DP-slave (stored in the CRL). The function Start C1 Timer starts or restarts this timer with the value C1_Response_Timeout taken from the appropriate CRL entry. The function Stop C1 Timer stops the operation no matter of it was running before.

Macros and Replacements

<REQPARAM>

```
<
  XXX=Read : Slot_Number, Index, Length
  XXX=Write : Slot_Number, Index, Length, Data
>
```

<VALID_SERVICE_FUNCTION_NUM>

```
<
  XXX=Read : 0x5E
  XXX=Write : 0x5F
>
```

<LOAD_SERVICE_BUFFER>

```
<
  XXX=Read : Service_Buffer[1]=0x5E
    Service_Buffer[2]=Slot_Number
    Service_Buffer[3]=Index
    Service_Buffer[4]=Length
  XXX=Write : Service_Buffer[1]=0x5F
    Service_Buffer[2]=Slot_Number
    Service_Buffer[3]=Index
    Service_Buffer[4]=Length
    Service_Buffer[5..Length+4]=Data
>
```

<LOAD_ALARM_BUFFER>

```
<
  Alarm_Buffer[1]=0x5C
  Alarm_Buffer[2]=Slot_Number
  Alarm_Buffer[3]=Alarm_Type
  Alarm_Buffer[4]=Seq_Nr*8
>
```

IS_VALID_SERVICE_REQ

```
(
  Length ≤ Max_Data_Length
)
```

IS_VALID_SERVICE_RES_PDU

```
(
  L_sdu[1] = Service_Buffer[1]
  && L_sdu.len ≥ 4
  && (L_sdu[1] <> 0x5E || L_sdu.len ≥ L_sdu[4]+4)
  && Max_Data_Length ≥ L_sdu[4]
)
```

IS_VALID_SERVICE_NRS_PDU

```
(
  L_sdu[1] = (Service_Buffer[1]+0x80)
  && L_sdu.len = 4
)
```

IS_ALARM_ACK_PDU

```
(
  L_sdu[2..4] = Alarm_Buffer[2..4]
  && (L_sdu[1] AND 0x7F = (Service_Buffer[1]))
  && L_sdu.len = 4
)
```

<VALID_SERVICE>

```
<
  Service_Header=0x5E : MSAC1M_Read
  Service_Header=0x5F : MSAC1M_Write
>
```

<VALID_SERVICE_RES_PARAM>

```
<
  Service_Header=0x5E : Length=L_sdu[4],
    Data=L_sdu[5..Length+4]
  Service_Header=0x5F : Length=L_sdu[4]
>
```

<VALID_SERVICE_NRS_PARAM>

```
<
  Error_Decode=L_sdu[2],
  Error_Code_1=L_sdu[3],
  Error_Code_2=L_sdu[4],
>
```

REJECT_VALID_SERVICE

```
(
  if (Stop_Pending=True)
    MSAC1M_Reject.ind(Rem_Add, Status=REJ_SE)
  else
    if.(Go_Abort=True)
      MSAC1M_Reject.ind(Rem_Add, Status=REJ_ABORT)
    else
      if (Src <> 0)
        MSAC1M_Reject.ind(Rem_Add, Status=REJ_PS)
      else
        if (Length<=Max_Data_Length)
          MSAC1M_Reject.ind(Rem_Add, Status=REJ_LE)
        else
          MSAC1M_Reject.ind(Rem_Add, Status=REJ_IV)
      )
    )
```

CONTINUE_VALID_SERVICE

```
(
  if (Src <> 0 && Alarm_Dsap=51)
    Service_Header = Service_Buffer[1]
    Start_C1_Monitoring = TRUE
    DMPMM1_DATA_REPLY.req(SSAP=51, DSAP=51,
      Rem_Add,L_sdu=Service_Buffer, Serv_class=Low)
  )
```

CONTINUE_ALARM_ACK

```
(
  if (Arc <> 0 && Alarm_Dsap=51)
    Service_Header = Alarm_Buffer[1]
    DMPMM1_DATA_REPLY.req(SSAP=51, DSAP=51, Rem_Add,
      L_sdu=Alarm_Buffer, Serv_class=Low)
  )
```

IS_INVALID_SERVICE_RES

```
(
  (L_status = DL &&
    (L_sdu[1]<> Service_Header || L_sdu.len<4) &&
    (L_sdu[1]<> (Service_Header+0x80) || L_sdu.len<> 4)
  ) ||
  L_status = DS/UE/RS/RDL/NA/RR/LR/DH/RDH
)
```

IS_INVALID_ALARM_ACK_RES

```
(
  (L_status = DL &&
  (L_sdu[1..4]<> Alarm_Buffer[1..4] || L_sdu.len<> 4)
  ) ||
  L_status = DS/UE/RS/RDL/NA/RR/LR/DH/RDH
)
```

CHECK_STOP_PENDING

```
(
  if (Stop_Pending = TRUE)
    MSAC1M_Stop.cnf(Rem_Add)
  else
    MSAC1M_Abort.ind(Rem_Add)
)
```

6.6.9.3 MSAC1M state table

Table 479 contains the complete description of the MSAC1M state machine.

Table 479 – MSAC1M state table

#	Current state	Event / condition => action	Next state
1	POWER-ON	MSAC1M_Minit_MSAC1.req(Rem_Add) => MSAC1M_Minit_MSAC1.cnf(Rem_Add)	CLOSED
2	CLOSED	MSAC1M_Start.req(Rem_Add) => Alarm_Buffer := empty Service_Buffer := empty Stop_Pending := False Go_Abort := False Start_C1_Monitoring := FALSE C1_Timer_Expired := FALSE Src := 0 Arc := 0 Service_Header := 0 Max_Data_Length := Max_Channel_Data_Length if (Extra_Alarm_Sap=FALSE) (Alarm_Dsap:=51) ELSE (Alarm_Dsap:=50) MSAC1M_Start.cnf(Rem_Add)	OPEN
3	CLOSED	MSAC1M_Stop.req(Rem_Add) => MSAC1M_Stop.cnf(Rem_Add)	CLOSED
4	CLOSED	MSAC1M_XXX.req(Rem_Add, <REQPARAM>) => MSAC1M_Reject.ind(Rem_Add, Status:=REJ_SE)	CLOSED
5	CLOSED	MSAC1M_Alarm_Ack.req(Rem_Add, Slot_Number, Alarm_Type, Seq_Nr) => ignore	CLOSED
6	CLOSED	DMPMM1_DATA_REPLY.cnf(SSAP, DSAP, Rem_Add, L_sdu, Serv_class, L_status) => MSAC1M_Fault.ind(Rem_Add)	POWER-ON
7	OPEN	MSAC1M_Start.req(Rem_Add) => MSAC1M_Fault.ind(Rem_Add)	POWER-ON

Table 479 (continued)

#	Current state	Event / condition => action	Next state
8	OPEN	MSAC1M_Stop.req(Rem_Add) /Arc = 0 && Src = 0 => MSAC1M_Stop.cnf(Rem_Add)	CLOSED
9	OPEN	MSAC1M_Stop.req(Rem_Add) /Arc <> 0 Src <> 0 => Stop_Pending:=True	OPEN
10	OPEN	MSAC1M_XXX.req(Rem_Add, <REQPARAM>) /Src = 0 && Stop_Pending = False && Go_Abort = False && IS_VALID_SERVICE_REQ && (Arc = 0 Alarm_Dsap <> 51) => Service_Header := <VALID_SERVICE_FUNCTION_NUM> Src := Src+1 Start_C1_Monitoring:=TRUE <LOAD_SERVICE_BUFFER> DMPMM1_DATA_REPLY.req(SSAP:=51, DSAP:=51, Rem_Add, L_sdu:=Service_Buffer, Serv_class:=Low)	OPEN
11	OPEN	MSAC1M_XXX.req(Rem_Add, <REQPARAM>) /Src = 0 && Stop_Pending = False && Go_Abort = False && IS_VALID_SERVICE_REQ && Arc <> 0 && Alarm_Dsap = 51 => Src := Src+1 <LOAD_SERVICE_BUFFER>	OPEN
12	OPEN	MSAC1M_XXX.req(Rem_Add, <REQPARAM>) /Src <> 0 Stop_Pending = True Go_Abort = True NOT (IS_VALID_SERVICE_REQ) => REJECT_VALID_SERVICE	OPEN
13	OPEN	MSAC1M_Alarm_Ack.req(Rem_Add, Slot_Number, Alarm_Type, Seq_Nr) /Arc = 0 && Stop_Pending = False && Go_Abort = False && Alarm_Dsap <> 51 => Arc := Arc+1 <LOAD_ALARM_BUFFER> DMPMM1_DATA_REPLY.req(SSAP:=51, DSAP:=Alarm_Dsap, Rem_Add, L_sdu:=Alarm_Buffer, Serv_class:=Low))	OPEN
14	OPEN	MSAC1M_Alarm_Ack.req(Rem_Add, Slot_Number, Alarm_Type, Seq_Nr) /Arc = 0 && Stop_Pending = False && Go_Abort = False && Src = 0 && Alarm_Dsap = 51 => Arc := Arc+1 DMPMM1_DATA_REPLY.req(SSAP:=51, DSAP:=Alarm_Dsap, Rem_Add, L_sdu:=Alarm_Buffer, Serv_class:=Low))	OPEN
15	OPEN	MSAC1M_Alarm_Ack.req(Rem_Add, Slot_Number, Alarm_Type, Seq_Nr) /Arc = 0 && Stop_Pending = False && Go_Abort = False && Src <> 0 && Alarm_Dsap = 51 => Arc := Arc+1 <LOAD_ALARM_BUFFER>	OPEN
16	OPEN	MSAC1M_Alarm_Ack.req(Rem_Add, Slot_Number, Alarm_Type, Seq_Nr) /Arc=0 && (Stop_Pending = True Go_Abort = True) => ignore	OPEN
17	OPEN	MSAC1M_Alarm_Ack.req(Rem_Add, Slot_Number, Alarm_Type, Seq_Nr) /Arc <> 0 => DMPMM1_Fault.ind(Rem_Add, Status:=MSAC1_Fault)	POWER-ON

Table 479 (continued)

#	Current state	Event / condition => action	Next state
18	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=51, Rem_Add, L_sdu, Serv_class=LOW, L_status) /L_status = DL && IS_VALID_SERVICE_RES_PDU && Src <> 0 && L_sdu[1] = Service_Header && Stop_Pending = False && Go_Abort = False => Stop C1 Timer() C1_Timer_Expired := FALSE Src := 0 CONTINUE_ALARM_ACK MSAC1_<VALID_SERVICE>.cnf(+)(Rem_Add, <VALID_SERVICE_RES_PARAM>)	OPEN
19	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=51, Rem_Add, L_sdu, Serv_class=LOW, L_status) /L_status = DL && IS_VALID_SERVICE_RES_PDU && Src <> 0 && L_sdu[1] = Service_Header && Stop_Pending = True && (Arc=0 Alarm_Dsap=51) => Stop C1 Timer() C1_Timer_Expired := FALSE Src := 0 Arc := 0 MSAC1_<VALID_SERVICE>.cnf(+)(Rem_Add, <VALID_SERVICE_RES_PARAM>) MSAC1M_Stop.cnf(Rem_Add)	CLOSED
20	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=51, Rem_Add, L_sdu, Serv_class=LOW, L_status) /L_status = DL && IS_VALID_SERVICE_RES_PDU && Src <> 0 && L_sdu[1] = Service_Header && Stop_Pending = True && Arc <> 0 && Alarm_Dsap <> 51 => Stop C1 Timer() C1_Timer_Expired := FALSE Src := 0 MSAC1_<VALID_SERVICE>.cnf(+)(Rem_Add, <VALID_SERVICE_RES_PARAM>)	OPEN
21	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=51, Rem_Add, L_sdu, Serv_class=LOW, L_status) /L_status = DL && IS_VALID_SERVICE_RES_PDU && Src <> 0 && L_sdu[1] = Service_Header && Stop_Pending = False && Go_Abort = True => Stop C1 Timer() C1_Timer_Expired := FALSE Src := 0 MSAC1_<VALID_SERVICE>.cnf(+)(Rem_Add, <VALID_SERVICE_RES_PARAM>) MSAC1M_Abort.ind(Rem_Add)	CLOSED
22	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=51, Rem_Add, L_sdu, Serv_class=LOW, L_status) /L_status = DL && IS_VALID_SERVICE_NRS_PDU && Src <> 0 && L_sdu[1] = (Service_Header+0x80) && Stop_Pending = False && Go_Abort = False => Stop C1 Timer() C1_Timer_Expired := FALSE Src := 0 CONTINUE_ALARM_ACK MSAC1_<VALID_SERVICE>.cnf(-)(Rem_Add, <VALID_SERVICE_NRS_PARAM>)	OPEN
23	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=51, Rem_Add, L_sdu, Serv_class=LOW, L_status) /L_status = DL && IS_VALID_SERVICE_NRS_PDU && Src <> 0 && L_sdu[1] = (Service_Header+0x80) && Stop_Pending = True && (Arc=0 Alarm_Dsap=51) => Stop C1 Timer() C1_Timer_Expired := FALSE Src := 0 Arc := 0 MSAC1_<VALID_SERVICE>.cnf(-)(Rem_Add, <VALID_SERVICE_NRS_PARAM>) MSAC1M_Stop.cnf(Rem_Add)	CLOSED

Table 479 (continued)

#	Current state	Event / condition => action	Next state
24	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=51, Rem_Add, L_sdu, Serv_class=LOW, L_status) /L_status = DL && IS_VALID_SERVICE_NRS_PDU && Src <> 0 && L_sdu[1] = (Service_Header+0x80) && Stop_Pending = True && Arc <> 0 && Alarm_Dsap <> 51 => Stop C1 Timer() C1_Timer_Expired := FALSE Src := 0 MSAC1_<VALID_SERVICE>.cnf(-)(Rem_Add, <VALID_SERVICE_NRS_PARAM>)	OPEN
25	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=51, Rem_Add, L_sdu, Serv_class=LOW, L_status) /L_status = DL && IS_VALID_SERVICE_NRS_PDU && Src <> 0 && L_sdu[1] = (Service_Header+0x80) && Stop_Pending = False && Go_Abort = True => Stop C1 Timer() C1_Timer_Expired := FALSE Src := 0 MSAC1_<VALID_SERVICE>.cnf(-)(Rem_Add, <VALID_SERVICE_NRS_PARAM>) MSAC1M_Abort.ind(Rem_Add)	CLOSED
26	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=Alarm_Dsap, Rem_Add, L_sdu, Serv_class=LOW, L_status) /L_status = DL && IS_VALID_ALARM_ACK_PDU && Arc <> 0 && L_sdu[1]=Service_Buffer[1] && (Alarm_Dsap<>51 L_sdu[1]=Service_Header) && Stop_Pending = False && Go_Abort = False => Arc := 0 CONTINUE_VALID_SERVICE MSAC1S_Alarm_Ack.cnf(+)(Rem_Add, Slot_Number, Alarm_Type, Seq_Nr)	OPEN
27	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=Alarm_Dsap, Rem_Add, L_sdu, Serv_class=LOW, L_status) /L_status = DL && IS_VALID_ALARM_ACK_PDU && Arc <> 0 && L_sdu[1]=(Service_Buffer[1] OR 0x80) && (Alarm_Dsap<>51 L_sdu[1]=Service_Header) && Stop_Pending = False && Go_Abort = False => Arc := 0 CONTINUE_VALID_SERVICE MSAC1S_Alarm_Ack.cnf(-)(Rem_Add, Slot_Number, Alarm_Type, Seq_Nr)	OPEN
28	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=Alarm_Dsap, Rem_Add, L_sdu, Serv_class=LOW, L_status) /L_status = DL && IS_VALID_ALARM_ACK_RES_PDU && Arc <> 0 && (Alarm_Dsap<>51 L_sdu[1]=Service_Header) && Stop_Pending = True && Src = 0 => Arc := 0 MSAC1M_Stop.cnf(Rem_Add)	CLOSED
29	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=Alarm_Dsap, Rem_Add, L_sdu, Serv_class=LOW, L_status) /L_status = DL && IS_VALID_ALARM_ACK_PDU && Arc <> 0 && Alarm_Dsap = 51 && L_sdu[1] = Service_Header && Stop_Pending = True && Src <> 0 => Arc := 0 Src := 0 MSAC1M_Reject.ind(Rem_Add, Status=REJ_ABORT) MSAC1M_Stop.cnf(Rem_Add)	CLOSED
30	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=Alarm_Dsap, Rem_Add, L_sdu, Serv_class=LOW, L_status) /L_status = DL && IS_VALID_ALARM_ACK_PDU && Arc <> 0 && Alarm_Dsap <> 51 && Stop_Pending = True && Src <> 0 => Arc := 0	OPEN

Table 479 (continued)

#	Current state	Event / condition => action	Next state
31	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=Alarm_Dsap, Rem_Add, L_sdu, Serv_class=LOW, L_status) /L_status = DL && IS_VALID_ALARM_ACK_PDU && Arc <> 0 && (Alarm_Dsap <> 51 L_sdu[1]=Service_Header) && Stop_Pending = False && Go_Abort = True => Arc := 0 MSAC1M_Abort.ind(Rem_Add)	CLOSED
32	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=51, Rem_Add, L_sdu, Serv_class=LOW, L_status) /L_status = NR && Src <> 0 && Service_Header <> Alarm_Ack && Stop_Pending = False && Go_Abort = False && Start_C1_Monitoring = TRUE => Start Timer C1() Start_C1_Monitoring = FALSE DMPMM1_DATA_REPLY.req(SSAP=51, DSAP, Rem_Add, L_sdu.len=0, Serv_class=Low)	OPEN
33	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=51, Rem_Add, L_sdu, Serv_class=LOW, L_status) /L_status = NR && Src <> 0 && Service_Header <> Alarm_Ack && Stop_Pending = False && Go_Abort = False && Start_C1_Monitoring = FALSE && C1_Timer_Expired = FALSE => DMPMM1_DATA_REPLY.req(SSAP=51, DSAP, Rem_Add, L_sdu.len=0, Serv_class=Low)	OPEN
34	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=51, Rem_Add, L_sdu, Serv_class=LOW, L_status) /L_status = NR && Src <> 0 && Service_Header <> Alarm_Ack && Stop_Pending = False && Go_Abort = False && Start_C1_Monitoring = FALSE && C1_Timer_Expired = TRUE => C1_Timer_Expired = FALSE Src:=0 MSAC1M_Reject.ind(Rem_Add, Status=REJ_ABORT) MSAC1M_Abort.ind(Rem_Add)	CLOSED
35	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=51, Rem_Add, L_sdu, Serv_class=LOW, L_status) /L_status = NR && Src <> 0 && Service_Header <> Alarm_Ack && Stop_Pending = True && (Arc=0 Alarm_Dsap=51) => Src := 0 Arc := 0 MSAC1M_Reject.ind(Rem_Add, Status=REJ_ABORT) MSAC1M_Stop.cnf(Rem_Add)	CLOSED
36	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=51, Rem_Add, L_sdu, Serv_class=LOW, L_status) /L_status = NR && Src <> 0 && Service_Header <> Alarm_Ack && Stop_Pending = True && Arc <> 0 && Alarm_Dsap <> 51 => Src := 0 MSAC1M_Reject.ind(Rem_Add, Status=REJ_ABORT)	OPEN
37	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=51, Rem_Add, L_sdu, Serv_class=LOW, L_status) /L_status = NR && Src <> 0 && Service_Header <> Alarm_Ack && Stop_Pending = False && Go_Abort = True => Src := 0 MSAC1M_Reject.ind(Rem_Add, Status=REJ_ABORT) MSAC1M_Abort.ind(Rem_Add)	CLOSED
38	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=Alarm_Dsap, Rem_Add, L_sdu, Serv_class=LOW, L_status) /L_status = NR && Arc <> 0 && (Service_Header=Alarm_Ack Alarm_Dsap <> 51) && Stop_Pending = False && Go_Abort = False => DMPMM1_DATA_REPLY.req(SSAP=51, DSAP, Rem_Add, L_sdu.len=0, Serv_class=Low)	OPEN

Table 479 (continued)

#	Current state	Event / condition => action	Next state
39	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=Alarm_Dsap, Rem_Add, L_sdu, Serv_class=LOW, L_status) /L_status = NR && Arc <> 0 && (Service_Header=Alarm_Ack Alarm_Dsap<>51) && Stop_Pending = True && Src = 0 => Arc := 0 MSAC1M_Stop.cnf(Rem_Add)	CLOSED
40	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=Alarm_Dsap, Rem_Add, L_sdu, Serv_class=LOW, L_status) /L_status = NR && Arc <> 0 && Service_Header = Alarm_Ack && Alarm_Dsap = 51 && Stop_Pending = True && Src <> 0 => Arc := 0 Src := 0 MSAC1M_Reject.ind(Rem_Add, Status=REJ_ABORT) MSAC1M_Stop.cnf(Rem_Add)	CLOSED
41	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=Alarm_Dsap, Rem_Add, L_sdu, Serv_class=LOW, L_status) /L_status = NR && Arc <> 0 && Alarm_Dsap <> 51 && Stop_Pending = True && Src <> 0 => Arc := 0	OPEN
42	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=Alarm_Dsap, Rem_Add, L_sdu, Serv_class=LOW, L_status) /L_status = NR && Arc <> 0 && (Service_Header=Alarm_Ack Alarm_Dsap<>51) && Stop_Pending = False && Go_Abort = True => Arc := 0 MSAC1M_Abort.ind(Rem_Add)	CLOSED
43	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=51, Rem_Add, L_sdu, Serv_class=LOW, L_status) /IS_INVALID_SERVICE_RES && Src <> 0 && Service_Header <> Alarm_Ack && (Arc=0 Alarm_Dsap=51) => Src := 0 Arc := 0 CHECK_STOP_PENDING MSAC1M_Reject.ind(Rem_Add, Status=REJ_ABORT)	CLOSED
44	OPEN	DMPMM1_DATA_REPLY.cnf(SSAP=51, DSAP=51, Rem_Add, L_sdu, Serv_class=LOW, L_status) /IS_INVALID_SERVICE_RES && Src <> 0 && Service_Header <> Alarm_Ack && Arc <> 0 && Alarm_Dsap <> 51 => Src := 0 if Stop_Pending := False Go_Abort := True MSAC1M_Reject.ind(Rem_Add, Status=REJ_ABORT)	OPEN
45	OPEN	C1 Timer expired => C1_Timer_Expired := TRUE	OPEN
46	CLOSED	C1 Timer expired => no action	CLOSED

6.6.10 MMAC1

6.6.10.1 Primitive definitions

6.6.10.1.1 Primitives exchanged between FSPMM1 and MMAC1

Table 480 shows the service primitives including their associated parameters issued by the FSPMM1 and received by the MMAC1.

Table 480 – Primitives issued by FSPMM1 to MMAC1

Primitive name	Source	Associated parameters	Functions
SInit MM.req	FSPMM1	(none)	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Reset.req	FSPMM1	(none)	
Abort.req	FSPMM1	(none)	
Get Master Diag.rsp(+)	FSPMM1	Diagnosis Data	
Get Master Diag.rsp(-)	FSPMM1	Status	
Start Seq.rsp(+)	FSPMM1	Max Len Data Unit	
Start Seq.rsp(-)	FSPMM1	Status	
Download.rsp(+)	FSPMM1	(none)	
Download.rsp(-)	FSPMM1	Status	
Upload.rsp(+)	FSPMM1	Data	
Upload.rsp(-)	FSPMM1	Status	
End Seq.rsp(+)	FSPMM1	(none)	
End Seq.rsp(-)	FSPMM1	Status	
Act Param.rsp(+)	FSPMM1	(none)	
Act Param.rsp(-)	FSPMM1	Status	

Table 481 shows the service primitives including their associated parameters issued by the MMAC1 and received by the FSPMM1.

Table 481 – Primitives issued by MMAC1 to FSPMM1

Primitive name	Source	Associated parameters	Functions
SInit MM.cnf	MMAC1	(none)	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Reset.cnf	MMAC1	(none)	
Abort.ind	MMAC1	(none)	
Fault.ind	MMAC1	(none)	
Get Master Diag.ind	MMAC1	Mdiag Identifier	
Start Seq.ind	MMAC1	Area Code, Timeout	
Download.ind	MMAC1	Area Code, Add Offset, Data	
Upload.ind	MMAC1	Area Code, Add Offset, Data Len	
End Seq.ind	MMAC1	(none)	
Act Param.ind	MMAC1	Area Code, Activate	
Act Para Brct.ind	MMAC1	Area Code	

6.6.10.1.2 Parameters of MMAC1 primitives

The parameters used with the primitives exchanged between the FSPMM1 and the MMAC1 are described in FAL Service Definition Type 3 Fieldbus IEC 61158-5.

6.6.10.2 State machine description

The following MMAC1 state machine for Master-Master communication describes the responder side. Normally, this means a DP-interface of an automation device such as a Programmable Logic Controller.

After Reset, the MMAC1 changes to the "POWER-ON" state and waits for initialisation.

The state machine remains in the state "WAIT-REQ" as long as it receives a valid request from Layer 2. After issuing this request to the User, the state machine remains in the "WAIT-RES" state until the User offers the response. After storing the response, data collection is monitored. If a service-sequence is executed, MMAC1 is waiting for the next service request in the state "WAIT-SEQ-REQ".

At any time, maximally one request shall be in execution.

Local Variables

T1

(Unsigned16)

Time in which a Master (Class 2) shall receive at latest a response from the Master (Class 1).

T2

(Unsigned16)

Time that can elapse during the execution of a service-sequence between response and the following request.

Client_Add

(Unsigned8)

Address of the DP-master (Class 2) just communicating with.

Service-Header

(Octet-String of the length 4)

Intermediate memory for the first four octets of the last request.

Sequence_Mode

(Boolean)

Indicates that a service-sequence is just in execution. During a service-sequence the DP-master (Class 1) is reserved for the respective DP-master (Class 2).

6.6.10.3 MMAC1 state table

Table 482 contains the complete description of the MMAC1 state machine.

Table 482 – MMAC1 state table

#	Current state	Event / condition => action	Next state
1	POWER-ON	MMAC1_Minit_MM.req =>MMAC1_Minit_MM.cnf	WAIT-REQ
2	WAIT-REQ	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class,Update_status) /L_sdu.len=0 =>ignore	WAIT-REQ
3	WAIT-REQ	MMAC1_XXX.rsp =>MMAC1_Reject.ind(Reason:=REJ_PS)	WAIT-REQ
4	WAIT-REQ	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class,Update_status) /L_sdu.len=2 && L_sdu[1]=0x41 =>Service_Header:=L_sdu[1-2] Client_Add:=Loc_add MMAC1_Get_Master_Diag.ind(Identifier:=L_sdu[2]) DMPMM1_RSAP_ACTIVATE.req(SSAP:=54, Access:=Loc_add, Indication_Mode:=Unchanged)	WAIT-PROT
5	WAIT-REQ	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class,Update_status) /L_sdu.len=4 && L_sdu[1]=0x42 =>Sequence_Mode:=True Service_Header:=L_sdu[1-4] T2:=L_sdu[3-4] Client_Add:=Loc_add MMAC1_Start_Seq.ind(Req_Add:=Loc_add, Area_Code:=L_sdu[2], Timeout:=L_sdu[3-4]) DMPMM1_RSAP_ACTIVATE.req(SSAP:=54, Access:=Loc_add, Indication_Mode:=Unchanged)	WAIT-PROT
6	WAIT-REQ	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class,Update_status) /L_sdu.len>=5 && L_sdu[1]=0x43 =>Service_Header:=L_sdu[1-4] Client_Add:=Loc_add MMAC1_Download.ind(Req_Add:=Loc_add, Area_Code:=L_sdu[2], Address_Offset:=L_sdu[3-4], Data:=L_sdu[5-L_sdu.len]) DMPMM1_RSAP_ACTIVATE.req(SSAP:=54, Access:=Loc_add, Indication_Mode:=Unchanged)	WAIT-PROT
7	WAIT-REQ	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class,Update_status) /L_sdu.len=5 && L_sdu[1]=0x44 =>Service_Header:=L_sdu[1-4] Client_Add:=Loc_add MMAC1_Upload.ind(Req_Add:=Loc_add, Area_Code:=L_sdu[2], Address_Offset:=L_sdu[3-4], Data_Length:=L_sdu[5]) DMPMM1_RSAP_ACTIVATE.req(SSAP:=54, Access:=Loc_add, Indication_Mode:=Unchanged)	WAIT-PROT
8	WAIT-REQ	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class,Update_status) /L_sdu.len=1 && L_sdu[1]=0x45 =>Client_Add:=Loc_add DMPMM1_RSAP_ACTIVATE.req(SSAP:=54, Access:=Loc_add, Indication_Mode:=Unchanged)	END-SEQ-PROT
9	WAIT-REQ	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class,Update_status) /L_sdu.len=3 && L_sdu[1]=0x47 =>Service_Header:=L_sdu[1-3] Client_Add:=Loc_add MMAC1_Act_Param.ind(Area_Code:=L_sdu[2], Activate:=L_sdu[3]) DMPMM1_RSAP_ACTIVATE.req(SSAP:=54, Access:=Loc_add, Indication_Mode:=Unchanged)	WAIT-PROT
10	WAIT-REQ	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class,Update_status) /invalid MMAC-REQ-PDU =>Client_Add:=Loc_add DMPMM1_RSAP_ACTIVATE.req(SSAP:=54, Access:=Loc_add, Indication_Mode:=Unchanged)	ERR-PROT

Table 482 (continued)

#	Current state	Event / condition => action	Next state
11	WAIT-REQ	DMPMM1_DATA.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /L_sdu.len=2 && L_sdu[1]=0x46 =>MMAC1_Act_Para_Brct.ind(Area_Code:=L_sdu[2])	WAIT-REQ
12	WAIT-REQ	DMPMM1_DATA.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /L_sdu.len<>2 L_sdu[1]<>0x46 =>ignore	WAIT-REQ
13	WAIT-SEQ-REQ	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /L_sdu.len=0 =>ignore	WAIT-REQ
14	WAIT-SEQ-REQ	MMAC1_xxx.rsp =>Sequence_Mode:=False Stop T2 DMPMM1_RSAP_ACTIVATE.req(SSAP:=54, Access:=All, Indication_Mode:=Unchanged)	WAIT-UNPROT
15	WAIT-SEQ-REQ	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /L_sdu.len=2 && L_sdu[1]=0x41 =>Service_Header:=L_sdu[1-2] Stop T2 MMAC1_Get_Master_Diag.ind(Identifier:=L_sdu[2])	WAIT-RES
16	WAIT-SEQ-REQ	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /L_sdu.len=4 && L_sdu[1]=0x42 =>Sequence_Mode:=False Stop T2 DMPMM1_REPLY_UPDATE.req(SSAP:=54, L_sdu[1]:=0xC9, Serv_class:=Low, Transmit:=Single) {SE error code}	WAIT-UPD
17	WAIT-SEQ-REQ	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /L_sdu.len>=5 && L_sdu[1]=0x43 =>Service_Header:=L_sdu[1-4] Stop T2 MMAC1_Download.ind(Req_Add:=Loc_add, Area_Code:=L_sdu[2], Address_Offset:=L_sdu[3-4], Data:=L_sdu[5-L_sdu.len])	WAIT-RES
18	WAIT-SEQ-REQ	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /L_sdu.len=5 && L_sdu[1]=0x44 =>Service_Header:=L_sdu[1-4] Stop T2 MMAC1_Upload.ind(Req_Add:=Loc_add, Area_Code:=L_sdu[2], Address_Offset:=L_sdu[3-4], Data_Length:=L_sdu[5])	WAIT-RES
19	WAIT-SEQ-REQ	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /L_sdu.len=1 && L_sdu[1]=0x45 =>Sequence_Mode:=False Service_Header:=L_sdu[1] Stop T2 MMAC1_End_Seq.ind(Req_Add:=Loc_add)	WAIT-RES
20	WAIT-SEQ-REQ	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /L_sdu.len=3 && L_sdu[1]=0x47 =>Service_Header:=L_sdu[1-3] Stop T2 MMAC1_Act_Param.ind(Area_Code:=L_sdu[2], Activate:=L_sdu[3])	WAIT-RES
21	WAIT-SEQ-REQ	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /invalid MMAC-REQ-PDU && Sequence_Mode=True =>Sequence_Mode:=False Stop T2 DMPMM1_REPLY_UPDATE.req(SSAP:=54, L_sdu[1]:=0xC1, Serv_class:=Low, Transmit:=Single) {FE error code}	WAIT-UPD
22	WAIT-SEQ-REQ	T2 expired =>Sequence_Mode:=False DMPMM1_RSAP_ACTIVATE.req(SSAP:=54, Access:=All, Indication_Mode:=Unchanged)	WAIT-UNPROT

Table 482 (continued)

#	Current state	Event / condition => action	Next state
23	WAIT-SEQ-REQ	DMPMM1_DATA.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class) /L_sdu.len=2 && Loc_add=Client_Add && L_sdu[1]=0x46 =>MMAC1_Act_Para_Brct.ind(Area_Code:=L_sdu[2])	WAIT-SEQ-REQ
24	WAIT-SEQ-REQ	DMPMM1_DATA.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class) /L_sdu.len<>2 Loc_add<>Client_Add L_sdu[1]<>0x46 =>ignore	WAIT-SEQ-REQ
25	END-SEQ-PROT	DMPMM1_RSAP_ACTIVATE.cnf(SSAP=54, M_status) /M_status=OK =>DMPMM1_REPLY_UPDATE.req(SSAP:=54, L_sdu[1]:=0xC9, Serv_class:=Low, Transmit:=Single) {SE error code}	WAIT-UPD
26	ERR-PROT	DMPMM1_RSAP_ACTIVATE.cnf(SSAP=54, M_status) /M_status=OK =>DMPMM1_REPLY_UPDATE.req(SSAP:=54, L_sdu[1]:=0xC1, Serv_class:=Low, Transmit:=Single) {FE error code}	WAIT-UPD
27	WAIT-PROT	DMPMM1_RSAP_ACTIVATE.cnf(SSAP=54, M_status) /M_status=OK	WAIT-RES
28	WAIT-RES	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /Sequence_Mode=False Client_Add<>Loc_add L_sdu.len=0 =>ignore	WAIT-RES
29	WAIT-RES	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /Sequence_Mode=True && Client_Add=Loc_add && L_sdu.len>0 =>Sequence_Mode:=False ignore L_sdu	WAIT-RES
30	WAIT-RES	MMAC1_Get_Master_Diag.rsp(Status, Diagnostic_Data) /Service_Header[1]=0x41 && Status=OK =>DMPMM1_REPLY_UPDATE.req(SSAP:=54, L_sdu:=(Service_Header, Diagnostic_Data), Serv_class:=Low, Transmit:=Single)	WAIT-UPD
31	WAIT-RES	MMAC1_Start_Seq.rsp(Status, Max_Length_Data_Unit) /Service_Header[1]=0x42 && Status=OK =>DMPMM1_REPLY_UPDATE.req(SSAP:=54, L_sdu:=(Service_Header, Max_Length_Data_Unit), Serv_class:=Low,Transmit:=Single)	WAIT-UPD
32	WAIT-RES	MMAC1_Download.rsp(Status) /Service_Header[1]=0x43 && Status=OK =>DMPMM1_REPLY_UPDATE.req(SSAP:=54, L_sdu:=Service_Header, Serv_class:=Low,Transmit:=Single)	WAIT-UPD
33	WAIT-RES	MMAC1_Upload.rsp(Status, Data) /Service_Header[1]=0x44 && Status=OK =>DMPMM1_REPLY_UPDATE.req(SSAP:=54, L_sdu:=(Service_Header, Data), Serv_class:=Low,Transmit:=Single)	WAIT-UPD
34	WAIT-RES	MMAC1_End_Seq.rsp(Status) /Service_Header[1]=0x45 && Status=OK =>DMPMM1_REPLY_UPDATE.req(SSAP:=54, L_sdu:=Service_Header, Serv_class:=Low,Transmit:=Single)	WAIT-UPD
35	WAIT-RES	MMAC1_Act_Param.rsp(Status) /Service_Header[1]=0x47 && Status=OK =>DMPMM1_REPLY_UPDATE.req(SSAP:=54, L_sdu:=Service_Header, Serv_class:=Low,Transmit:=Single)	WAIT-UPD
36	WAIT-RES	MMAC1_xxx.rsp(Status) /Service_Header[1]=Service_Code && Status <> OK =>Start T1 DMPMM1_REPLY_UPDATE.req(SSAP:=54, L_sdu[1]:=Status, Serv_class:=Low, Transmit:=Single)	WAIT-UPD
37	WAIT-RES	MMAC1_XXX.rsp(Status) /Service_Header[1]<>Service_Code =>Sequence_Mode:=False DMPMM1_REPLY_UPDATE.req(SSAP:=54, L_sdu[1]:=0xC6, Serv_class:=Low, Transmit:=Single) {RE}	WAIT-UPD

Table 482 (continued)

#	Current state	Event / condition => action	Next state
38	WAIT-UPD	DMPMM1_REPLY_UPDATE.cnf(SSAP=54, Serv_class, L_status) /Status=OK =>Start T1	WAIT-IND
39	WAIT-UPD	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /Loc_add<>Client_Add && Update_status=NO =>ignore	WAIT-UPD
40	WAIT-UPD	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /Loc_add=Client_Add && L_sdu.len>0 && Update_status=NO =>Sequence_Mode := False ignore L_sdu	WAIT-UPD
41	WAIT-UPD	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /Update_status=LO/HI =>MMAC1_Fault.ind	POWER-ON
42	WAIT-UPD	DMPMM1_REPLY_UPDATE.cnf(SSAP=54) /Status=NO/IV =>MMAC1_Fault.ind	POWER-ON
43	WAIT-UPD	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /(Update_status=LO/HI) (Update_status=NO && L_sdu.len=0) =>MMAC1_Fault.ind	POWER-ON
44	WAIT-UPD	DMPMM1_DATA.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class) /L_sdu.len=2 && L_sdu[1]=0x46 && (Sequence_Mode=False Loc_add=Client_Add) =>MMAC1_Act_Para_Brct.ind(Area_Code:=L_sdu[2])	WAIT-UPD
45	WAIT-UPD	DMPMM1_DATA.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class) /L_sdu.len<>2 L_sdu[1]<>0x46 Sequence_Mode=True Loc_add<>Client_Add =>ignore	WAIT-UPD
46	WAIT-IND	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /Loc_add<>Client_Add && Update_status=NO =>ignore	WAIT-IND
47	WAIT-IND	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /Update_status=LO/HI && Loc_add<>Client_Add =>stop T1 MMAC1_Fault.ind	POWER-ON
48	WAIT-IND	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /(Update_status=LO && Loc_add<>Client_Add) (Update_status=NO && L_sdu.len=0) (Update_status=HI) =>MMAC1_Fault.ind	POWER-ON
49	WAIT-IND	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /Loc_add=Client_Add && L_sdu.len>0 && Update_status=NO =>Sequence_Mode = False ignore L_sdu	WAIT-IND
50	WAIT-IND	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class, Update_status) /Update_status=LO && Sequence_Mode=False && Loc_add = Client_Add =>ignore L_sdu, if existent Stop T1 DMPMM1_RSAP_ACTIVATE.req(SSAP:=54, Access:=All, Indication_Mode:=Unchanged)	WAIT-UNPROT

Table 482 (continued)

#	Current state	Event / condition => action	Next state
51	WAIT-IND	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class,Update_status) /Update_status=LO && Sequence_Mode=True && L_sdu.len=0 && Loc_add = Client_Add =>Stop T1 Start T2	WAIT-SEQ-REQ
52	WAIT-IND	DMPMM1_DATA_REPLY.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class,Update_status) /Update_status=LO && Sequence_Mode=True && L_sdu.len>0 && Loc_add = Client_Add =>Sequence_Mode:=False ignore L_sdu, if existent STOP T1 DMPMM1_RSAP_ACTIVATE.req(SSAP:=54, Access:=All, Indication_Mode:=Unchanged)	WAIT-UNPROT
53	WAIT-IND	DMPMM1_DATA.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class) /L_sdu.len=2 && L_sdu[1]=0x46 && (Sequence_Mode=False Loc_add=Client_Add) =>MMAC1_Act_Para_Brct.ind(Area_Code:=L_sdu[2])	WAIT-IND
54	WAIT-IND	DMPMM1_DATA.ind(SSAP,DSAP=54,Loc_add,Rem_add,L_sdu,Serv_class) /L_sdu.len<>2 L_sdu[1]<>0x46 (Sequence_Mode=True && Loc_add<>Client_Add) =>ignore	WAIT-IND
55	WAIT-IND	T1 expired =>Sequence_Mode:=False DMPMM1_REPLY_UPDATE.req(SSAP:=54, L_sdu.len:=0, Serv_class:=Low, Transmit:=Single)	WAIT-DEL
56	WAIT-DEL	DMPMM1_REPLY_UPDATE.cnf(SSAP=54) /L_status=OK =>DMPMM1_RSAP_ACTIVATE.req(SSAP:=54, Access:=All, Indication_Mode:=Unchanged)	WAIT-UNPROT
57	ANY-STATE	MMAC1_Reset.req =>Stop T1 MMAC1_Reset.cnf	POWER-ON

6.6.11 MSCS1M

6.6.11.1 Primitive definitions

6.6.11.1.1 Primitives exchanged between MSCS1M and FSPMM1

Table 483 shows the service primitives including their associated parameters issued by the FSPMM1 and received by the MSCS1M.

Table 483 – Primitives issued by FSPMM1 to MSCS1M

Primitive name	Source	Associated parameters	Functions
Set Time.req	FSPMM1	AREP, Time Value, Local Time Diff, Summertime, Accuracy, Synchronisation Active, Announcement Hour	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5

Table 484 shows the service primitives including their associated parameters issued by the MSCS1M and received by the FSPMM1.

Table 484 – Primitives issued by MSCS1M to FSPMM1

Primitive name	Source	Associated parameters	Functions
Set Time.ind	MSCS1M	AREP, Time Value, Local Time Diff, Summertime, Accuracy, Synchronisation Active, Announcement Hour	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Set Time.cnf	MSCS1M	AREP, Status	
Sync Interval Violation.ind	MSCS1M	AREP	

6.6.11.1.2 Parameter of MSCS1M primitives

The parameters used with the primitives exchanged between the FSPMM1 and the MSCS1M are described in FAL Service Definition Type 3 Fieldbus IEC 61158-5.

6.6.11.2 State machine description

In IDLE state this machine waits either for a Clock Value indication from a valid Time master conveyed by DMPMM1 to enter StopTM state and deactivate the own Time master capability or for a Set Time request from the local user conveyed by the FSPMM1 to enter WTEc state and become an active Time master.

In the states WTEc and WCVc the machine issues a clock synchronisation sequence. On completion of this sequence a Set Time confirmation is issued and the machine returns to IDLE. In case of an interruption of the sequence by a valid Clock Value indication the machine branches to StopTM state and the Time master capability becomes inactive.

The StopTM state will be left, if no valid clock synchronisation sequence is detected.

Local variables of the MSCS1M

Time Last Rcvd (Network Time)

This local variable contains the Clock_Value_Time_Event of the last valid received Clock Value indication.

Error (Unsigned8)

This local variable is a counter for invalid Clock Synchronisation sequences. It will be reset to 0, when a Sync Interval Violation is indicated.

Clock (Network Time)

This Variable is used to hold the time value between issuing a Time Event service request and receiving the corresponding Time Event service confirmation.

T_State (Compound)

This local variable is used for internal storage of service parameters for clock synchronisation.

T_Last (Network Time)

This local variable is used for internal storage of the Time Value of the last valid clock synchronisation sequence. It is used to check the validity of subsequent clock synchronisations.

Macros of the MSCS1M:**SET_TIME_ATTRIBUTES**

```
(
  Local Time Diff := CS_list.Clock_value_status.C * (-1)^CS_list.Clock_value_status.CV
  Summertime := CS_list.Clock_value_status.SWT
  Accruacy := CS_list.Clock_value_status.CR
  Synchronisation Active := CS_list.Clock_value_status.SYF
  Announcement Hour := CS_list.Clock_value_status.ANH
)
```

GET_TIME_ATTRIBUTES

```
(
  CS_list.Clock_value_status.C * (-1)^CS_list.Clock_value_status.CV := Local Time Diff
  CS_list.Clock_value_status.SWT := Summertime
  CS_list.Clock_value_status.CR := Accruacy
  CS_list.Clock_value_status.SYF := Synchronisation Active
  CS_list.Clock_value_status.ANH := Announcement Hour
)
```

6.6.11.3 MSCS1M state table

Table 485 contains the complete description of the MSCS1M state machine.

Table 485 – MSCS1M state table

#	Current state	Event / condition => action	Next state
1	Idle	CS_CLOCK_VALUE.ind (Time_Master_Addr, CS_list, CS_status, Receive_Delay_Time) /CS_status == SV && Error < 2 => Error := Error+1	Idle
2	Idle	CS_CLOCK_VALUE.ind (Time_Master_Addr, CS_list, CS_status, Receive_Delay_Time) /CS_status == OK && CS_list.Clock_Value_previous_TE != Time Last Rcvd && Error < 2 => Time Last Rcvd:= CS_list.Clock_Value_Time_Event Error := Error+1	Idle
3	Idle	CS_CLOCK_VALUE.ind (Time_Master_Addr, CS_list, CS_status, Receive_Delay_Time) /CS_status == SV && Error >= 2 => Error := 0 Sync Interval Violation.ind	Idle
4	Idle	CS_CLOCK_VALUE.ind (Time_Master_Addr, CS_list, CS_status, Receive_Delay_Time) /CS_status == OK && CS_list.Clock_Value_previous_TE != Time Last Rcvd && Error >= 2 => Time Last Rcvd:= CS_list.Clock_Value_Time_Event Error := 0 Sync Interval Violation.ind	Idle

Table 485 (continued)

#	Current state	Event / condition => action	Next state
5	Idle	CS_CLOCK_VALUE.ind (Time_Master_Addr, CS_list, CS_status, Receive_Delay_Time) /CS_status == OK && CS_list.Clock_Value_previous_TE == Time Last Rcvd && !(Time_Master_Addr < TS) => ignore	
6	Idle	CS_CLOCK_VALUE.ind (Time_Master_Addr, CS_list, CS_status, Receive_Delay_Time) /CS_status == OK && CS_list.Clock_Value_previous_TE == Time Last Rcvd && Time_Master_Addr < TS => Error := 0 Time Last Rcvd:= CS_list.Clock_Value_Time_Event Time Value := Time Last Rcvd + Receive Delay Time Local Time Diff, Summertime, Accuracy, Synchronisation Active, Announcment Hour from CS-list.Clock_Value_Status Set_Time.ind(Time Value, Local Time Diff, Summertime, Accuracy, Synchronisation Active, Announcment Hour)	StopTM
7	Idle	Set_Time.req(Time Value, Local Time Diff, Summertime, Accuracy, Synchronisation Active, Announcment Hour) => T_State from Local Time Diff, Summertime, Accuracy, Synchronisation Active, Announcment Hour Clock:=Time Value CS-TIME_EVENT.req()	WTEc
8	WTEc	CS-TIME-EVENT.cnf (Send_Delay_Time, DL_status) /DL_status = OK => CS_list.Clock_Value_Previous_TE:= T_Last T_Last:= Clock + Send_Delay_Time CS_list.Clock_Value_Time_Event:= T_Last DL-CS-CLOCK-VALUE.req (CS_list)	WCVc
9	WTEc	CS-TIME-EVENT.cnf (Send_Delay_Time, DL_status) /DL_status != OK => Status:= DL_Status Set_Time.cnf(Status)	Idle
10	WTEc	Set_Time.req(Time Value, Local Time Diff, Summertime, Accuracy, Synchronisation Active, Announcment Hour) => Status:= SV Set_Time.cnf(Status)	WTEc
11	WCVc	CS-CLOCK-VALUE.cnf (DL_status) => Status:= DL_Status Set_Time.cnf(Status)	Idle
12	WCVc	Set_Time.req(Time Value, Local Time Diff, Summertime, Accuracy, Synchronisation Active, Announcment Hour) => Status:= SV Set_Time.cnf(Status)	WCVc
13	StopTM	CS_CLOCK_VALUE.ind (Time_Master_Addr, CS_list, CS_status, Receive_Delay_Time) /CS_status == SV && Error < 2 => Error := Error+1	StopTM
14	StopTM	CS_CLOCK_VALUE.ind (Time_Master_Addr, CS_list, CS_status, Receive_Delay_Time) /CS_status == OK && CS_list.Clock_Value_previous_TE != Time Last Rcvd && Error < 2 => Time Last Rcvd:= CS_list.Clock_Value_Time_Event Error := Error+1	StopTM

Table 485 (continued)

#	Current state	Event / condition => action	Next state
15	StopTM	CS_CLOCK_VALUE.ind (Time_Master_Addr, CS_list, CS_status, Receive_Delay_Time) /CS_status == SV && Error >= 2 => Error := 0 Sync Interval Violation.ind	Idle
16	StopTM	CS_CLOCK_VALUE.ind (Time_Master_Addr, CS_list, CS_status, Receive_Delay_Time) /CS_status == OK && CS_list.Clock_Value_previous_TE != Time Last Rcvd && Error >= 2 => Time Last Rcvd:= CS_list.Clock_Value_Time_Event Error := 0 Sync Interval Violation.ind	Idle
17	StopTM	CS_CLOCK_VALUE.ind (Time_Master_Addr, CS_list, CS_status, Receive_Delay_Time) /CS_status == OK && CS_list.Clock_Value_previous_TE == Time Last Rcvd && Time_Master_Addr < TS => Error := 0 Time Last Rcvd:= CS_list.Clock_Value_Time_Event Time Value := Time Last Rcvd + Receive Delay Time Local Time Diff, Summertime, Accuracy, Synchronisation Active, Announcement Hour from CS-list.Clock_Value_Status Set_Time.ind(Time Value, Local Time Diff, Summertime, Accuracy, Synchronisation Active, Announcement Hour)	StopTM
18	StopTM	CS_CLOCK_VALUE.ind (Time_Master_Addr, CS_list, CS_status, Receive_Delay_Time) /CS_status == OK && CS_list.Clock_Value_previous_TE == Time Last Rcvd && !(Time_Master_Addr < TS) => ignore	StopTM
19	StopTM	Set_Time.req(Time Value, Local Time Diff, Summertime, Accuracy, Synchronisation Active, Announcement Hour) => Status:= SV Set_Time.cnf(Status)	StopTM

6.6.12 MSAC2M

6.6.12.1 Primitive definitions

6.6.12.1.1 Primitives exchanged between FSPMM2 and MSAC2M

Table 486 shows the service primitives including their associated parameters issued by FSPMM2 and received by the MSAC2M.

Table 486 – Primitives issued by FSPMM2 to MSAC2M

Primitive name	Source	Associated parameters	Functions
MInit MS2.req	FSPMM2	C_Ref	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Reset.req	FSPMM2	(none)	
Abort.req	FSPMM2	C_Ref	
Initiate.req	FSPMM2	C_Ref, Send Timeout, Features Supported, Profile Features Supported, Profile Ident Number, Add Addr Param	
Read.req	FSPMM2	C_Ref, Slot Number, Index, Length	
Write.req	FSPMM2	C_Ref, Slot Number, Index, Length, Data	
Data Transport.req	FSPMM2	C_Ref, Slot Number, Index, Length, Data	

Table 487 shows the service primitives including their associated parameters issued by the MSAC2M and received by the FSPMM2.

Table 487 – Primitives issued by MSAC2M to FSPMM2

Primitive name	Source	Associated parameters	Functions
MInit MS2.cnf	MSAC2M	C_Ref	
Reset.cnf	MSAC2M	(none)	
Initiate.cnf(+)	MSAC2M	C_Ref, Max Len Data Unit, Features Supported, Profile Features Supported, Profile Ident Number, Add Addr Param	
Initiate.cnf(-)	MSAC2M	C_Ref, Error Decode, Error Code 1 Error Code 2	
Read.cnf(+)	MSAC2M	C_Ref, Length, Data	
Read.cnf(-)	MSAC2M	C_Ref, Error Decode, Error Code 1 Error Code 2	
Write.cnf(+)	MSAC2M	C_Ref, Length	
Write.cnf(-)	MSAC2M	C_Ref, Error Decode, Error Code 1 Error Code 2	
Data Transport.cnf(+)	MSAC2M	C_Ref, Length, Data	
Data Transport.cnf(-)	MSAC2M	C_Ref, Error Decode, Error Code 1 Error Code 2	
Reject.ind	MSAC2M	C_Ref, Reason Code	
Abort.ind	MSAC2M	C_Ref, Locally_Generated, Subnet, Instance, Reason_Code, Abort_Detail	
Fault.ind	MSAC2M	C_Ref	

6.6.12.1.2 Parameters of MSAC2M primitives

The parameters used with the primitives exchanged between the FSPMM2 and the MSAC2M are described in FAL Service Definition Type 3 Fieldbus IEC 61158-5. Additional parameters are described in Table 488.

Table 488 – Parameters used with primitives exchanged with MSAC2M

Parameter name	Description
C_Ref	Local identifier of a MSAC2 state machine

6.6.12.2 State machine description

On POWER-ON each MSAC2M State Machine waits for initialisation.

In the state CLOSED the Initiate service called from the AP-Context is expected and sent to the DP-slave. In the state START-POLL-RES the Master is waiting for the positive Initiate.cnf and makes the transition into the state OPEN.

In the OPEN-state the services Read, Write and Data_Transport are allowed. The State Machine controls that only one request from the User is processed simultaneously, otherwise the connection is aborted.

The connection can be aborted by the User of the Master or by the User of the Slave. It is aborted automatically if a protocol error is detected.

Local Variables

Client_SAP

(Unsigned8)

The SAP used for the connection.

Server_SAP

(Unsigned8)

The SAP used for the connection at the server side.

Server_Add

(Unsigned8)

The DL address (0-126) of the remote server.

S-Timer

(Unsigned16)

This timer monitors the sending of service request PDUs on the connection if no service is outstanding. The S-Timer expires if no service request PDU has been sent during the S-Timer interval. In this case an Idle-REQ-PDU will be sent.

R-Timer

(Unsigned16)

This Timer monitors the liveliness of the connection if there is an outstanding service response. The R-Timer expires if no service response or IDLE-PDU during the R-Timer interval has been received. In this case the connection will be aborted. The R-Timer interval is 3 times the S-Timer interval.

Service-Header

Local buffer for the Function_Num of the last transmitted request.

Go_Abort

(Boolean)

Local flag that indicates an abort reaction after receiving the remote response.

Service_Buffer

(Octet-String[255])

Local buffer that stores one complete service during an outstanding Idle-RES-PDU.

Macros

;XXX handles Data_Transport, Read and Write Services.

<REQPARAM>

<

XXX=Read : Slot_Number, Index, Length

XXX=Write : Slot_Number, Index, Length, Data

XXX=Data_Transport : Slot_Number, Index, Length, Data

>

<VALID_SERVICE_FUNCTION_NUM>

```

<
  XXX=Read : 0x5E
  XXX=Write : 0x5F
  XXX=Data_Transport : 0x51
>

```

<LOAD_SERVICE_BUFFER>

```

<
  XXX=Read : Service_Buffer[1]=0x5E
    Service_Buffer[2]=Slot_Number
    Service_Buffer[3]=Index
    Service_Buffer[4]=Length
  XXX=Write : Service_Buffer[1]=0x5F
    Service_Buffer[2]=Slot_Number
    Service_Buffer[3]=Index
    Service_Buffer[4]=Length
    Service_Buffer[5..Length+4]=Data
  XXX=Data_Transport : Service_Buffer[1]=0x51
    Service_Buffer[2]=Slot_Number
    Service_Buffer[3]=Index
    Service_Buffer[4]=Length
    Service_Buffer[5..Length+4]=Data
>

```

VALID_SERVICE

```

(
  Length ≤ Max_Data_Length
)

```

VALID_SERVICE_RES_PDU

```

(
  L_sdu[1] = Service_Buffer[1] &&
  L_sdu.len ≥ 4 &&
  (L_sdu[1] <> 0x5E || L_sdu.len ≥ L_sdu[4]+4) &&
  Max_Data_Length ≥ L_sdu[4]
)

```

VALID_SERVICE_NRS_PDU

```

(
  L_sdu[1] = (Service_Buffer[1]+0x80) &&
  L_sdu.len = 4
)

```

<VALID_SERVICE>

```
<
  Service_Header=0x5E : Read
  Service_Header=0x5F : Write
  Service_Header=0x51 : Data_Transport
>
```

<VALID_SERVICE_RES_PARAM>

```
<
  Service_Header=0x5E : Length=L_sdu[4],
    Data=L_sdu[5..Length+4]
  Service_Header=0x5F : Length=L_sdu[4]
  Service_Header=0x51 : Length=L_sdu[4],
    Data=L_sdu[5..Length+4]
>
```

<VALID_SERVICE_NRS_PARAM>

```
<
  Error_Decode=L_sdu[2],
  Error_Code_1=L_sdu[3],
  Error_Code_2=L_sdu[4],
>
```

VALID_SERVICE_REQ_PDU

```
(
  (Read-REQ-PDU) ||
  (Write-REQ-PDU) ||
  (Data_Transport-REQ-PDU)
)
```

VALID_SERVICE_RES_PDU

```
(
  (Read-RES-PDU) ||
  (Write-RES-PDU) ||
  (Data_Transport-RES-PDU)
)
```

VALID_SERVICE_NRS_PDU

```
(
  (Read-NRS-PDU) ||
  (Write-NRS-PDU) ||
  (Data_Transport-NRS-PDU)
)
```

STORE_ABORT_PARAMETER

```
(  
  Stored-Subnet=Subnet  
  Stored-Instance=Instance  
  Stored-Reason_Code=Reason_Code  
)
```

6.6.12.3 MSAC2M state table

Table 489 contains the complete description of the MSAC2M state machine.

Table 489 – MSAC2M state table

#	Current state	Event / condition => action	Next state
1	POWER-ON	MSAC2M_Minit_MS2.req(C_Ref) => Stored_RM_SAP := 49 Stored_C_Ref := C_Ref Stored_Max_Local_Data_Length := Max_Local_Data_Length Client_SAP := 50 MSAC2M_Minit_MS2.cnf(C_Ref)	CLOSED
2	CLOSED	MSAC2M_Initiate.req(C_Ref, Send_Timeout, Features_Supported, Profile_Features_Supported, Profile_Ident_Number, Add_Addr_Param) => Server_Add := Rem_Add S-Timer := Send_Timeout R-Timer := 3*Send_Timeout Go_Abort := FALSE Service_Buffer := empty store Initiate-REQ-PDU Start S-Timer DMPMM2_DATA_REPLY.req(SSAP:=Client_SAP, DSAP:=STORED_RM_SAP, Rem_Add:=Server_Add, L_sdu:=Initiate-REQ-PDU, Serv_class:=Low)	START-POLL-RMREQ
3	CLOSED	MSAC2M_Abort.req(C_Ref=Stored_C_Ref, Subnet, Instance, Reason_Code) => MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_SE) MSAC2M_Closed.ind(C_Ref:=Stored_C_Ref)	CLOSED
4	CLOSED	MSAC2M_XXX.req(C_Ref=Stored_C_Ref, <REQPARAM>) => MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_SE) MSAC2M_Closed.ind(C_Ref:=Stored_C_Ref)	CLOSED
5	CLOSED	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_RM_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = IV/LS => ignore	CLOSED
6	START-POLL-RMREQ	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_RM_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DL && L_sdu = RM-REQ-PDU && S-Timer >= Send_Timeout && Go_Abort = FALSE => Stored_Server_SAP := Server_SAP Stop S-Timer Start R-Timer DMPMM2_DATA_REPLY.req(SSAP:=Client_SAP, DSAP:=Stored_Server_SAP, Rem_Add:=Server_Add, L_sdu.len:=0, Serv_class:=Low)	START-POLL-RES
7	START-POLL-RMREQ	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_RM_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DL && L_sdu = RM-REQ-PDU && Go_Abort = TRUE => Stored_Server_SAP := Server_SAP Stop S-Timer Start R-Timer DMPMM2_DATA_REPLY.req(SSAP:=Client_SAP, DSAP:=Stored_Server_SAP, Rem_Add:=Server_Add, L_sdu.len:=0, Serv_class:=Low)	START-POLL-RES

Table 489 (continued)

#	Current state	Event / condition => action	Next state
8	START-POLL-RMREQ	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_RM_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DL && L_sdu = RM-REQ-PDU && S-Timer < Send_Timeout => C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_STO, Additional_Detail:=Send_Timeout, STORE_ABORT_PARAMETER Stored_Server_SAP := Server_SAP Go_Abort := TRUE Stop S-Timer Start R-Timer MSAC2M_Abort.ind(C_Ref, Locally_Generated, Subnet, Instance, Reason_Code, Additional_Detail) DMPMM2_DATA_REPLY.req(SSAP:=Client_SAP, DSAP:=Stored_Server_SAP, Rem_Add:=Server_Add, L_sdu.len:=0, Serv_class:=Low)	START-POLL-RES
9	START-POLL-RMREQ	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_RM_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DL && L_sdu <> RM-REQ-PDU => Stop S-Timer MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_FE) MSAC2M_Closed.ind(C_Ref:=Stored_C_Ref)	CLOSED
10	START-POLL-RMREQ	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_RM_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = RR && Go_Abort = FALSE => DMPMM2_DATA_REPLY.req(SSAP:=Client_SAP, DSAP:=STORED_RM_SAP, Rem_Add:=Server_Add, L_sdu:=stored Initiate-REQ-PDU, Serv_class:=Low)	START-POLL-RMREQ
11	START-POLL-RMREQ	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_RM_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = RR && Go_Abort = TRUE => Stop S-Timer MSAC2M_Closed.ind(C_Ref:=Stored_C_Ref)	CLOSED
12	START-POLL-RMREQ	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_RM_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = RS/NR/RDL/RDH/LR/NA/DH/DS/UE => C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=FALSE, Instance:=DLL, Reason_Code:=L_status ignore L_sdu if present Stop S-Timer MSAC2M_Abort.ind(C_Ref, Locally_Generated, Subnet, Instance, Reason_Code) MSAC2M_Closed.ind(C_Ref:=Stored_C_Ref)	CLOSED
13	START-POLL-RMREQ	MSAC2M_Abort.req(C_Ref=Stored_C_Ref, Subnet, Instance, Reason_Code) => Go_Abort := TRUE STORE_ABORT_PARAMETER	START-POLL-RMREQ
14	START-POLL-RMREQ	MSAC2M_Initiate.req(C_Ref, Send_Timeout, Features_Supported, Profile_Features_Supported, Profile_Ident_Number, Add_Addr_Param) => C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_SE, Go_Abort := TRUE STORE_ABORT_PARAMETER MSAC2M_Abort.ind(C_Ref, Locally_Generated, Subnet, Instance, Reason_Code)	START-POLL-RMREQ

Table 489 (continued)

#	Current state	Event / condition => action	Next state
15	START-POLL-RMREQ	MSAC2M_XXX.req(C_Ref=Stored_C_Ref, <REQPARAM>) => C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_SE, Go_Abort := TRUE STORE_ABORT_PARAMETER MSAC2M_Abort.ind(C_Ref, Locally_Generated, Subnet, Instance, Reason_Code)	START-POLL-RMREQ
16	START-POLL-RMREQ	S-Timer expired => MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_TO)	FINAL-WAIT-CON
17	START-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = RS/NR => DMPMM2_DATA_REPLY.req(SSAP:=Client_SAP, DSAP:=Stored_Server_SAP, Rem_Add:=Server_Add, L_sdu.len:=0, Serv_class:=Low)	START-POLL-RES
18	START-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DL && L_sdu = Initiate-RES-PDU && Go_Abort = FALSE => Stored_Max_Data_Lenth := Max_Len_Data_Unit Stop R-Timer Start S-Timer MSAC2M_Initiate.cnf(+, C_Ref:=Stored_C_Ref, Features_Supported, Profile_Features_Supported, Profile_Ident_Number, Add_Addr_Param)	OPEN
19	START-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DL && L_sdu = Initiate-RES-PDU && Go_Abort = TRUE => Stop R-Timer Start S-Timer DMPMM2_DATA_REPLY.req(SSAP:=Client_SAP, DSAP:=Stored_Server_SAP, Rem_Add:=Server_Add, L_sdu:=Abort-REQ-PDU with stored Parameter, Serv_class:=Low)	END-POLL-RES
20	START-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DS/UE => Stop R-Timer MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=DLL, Reason_Code:=L_status) MSAC2M_Closed.ind(C_Ref:=Stored_C_Ref)	CLOSED
21	START-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = RDL/NA/LR/DH/RDH/RR => ignore L_sdu if present Stop R-Timer Start S-Timer MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=DLL, Reason_Code:=L_status) DMPMM2_DATA_REPLY.req(SSAP:=Client_SAP, DSAP:=Stored_Server_SAP, Rem_Add:=Server_Add, L_sdu:=Abort-REQ-PDU, Serv_class:=Low)	END-POLL-RES
22	START-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DL && L_sdu = Abort-REQ-PDU => Stop R-Timer MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=FALSE, Subnet, Instance, Reason_Code) MSAC2M_Closed.ind(C_Ref:=Stored_C_Ref)	CLOSED

Table 489 (continued)

#	Current state	Event / condition => action	Next state
23	START-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DL && L_sdu = (Initiate-NRS-PDU) && Go_Abort = FALSE => Stop R-Timer MSAC2M_Initiate.cnf(-,C_Ref:=Stored_C_Ref, Error Decode,Error_Code_1,Error_Code_2) MSAC2M_Closed.ind(C_Ref:=Stored_C_Ref)	CLOSED
24	START-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DL && L_sdu = (Initiate-NRS-PDU) && Go_Abort = TRUE => Stop R-Timer MSAC2M_Closed.ind(C_Ref:=Stored_C_Ref)	CLOSED
25	START-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DL && NOT (L_sdu = Initiate-RES-PDU L_sdu = (Initiate-NRS-PDU) L_sdu = Abort-REQ-PDU L_sdu = Idle-REQ-PDU) => ignore L_sdu Stop R-Timer Start S-Timer MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_RE) DMPMM2_DATA_REPLY.req(SSAP:=Client_SAP, DSAP:=Stored_Server_SAP, Rem_Add:=Server_Add, L_sdu:=Abort-REQ-PDU, Serv_class:=Low)	END-POLL-RES
26	START-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DL && L_sdu = Idle-REQ-PDU && Go_Abort = FALSE => Stop R-Timer Start R-Timer DMPMM2_DATA_REPLY.req(SSAP:=Client_SAP, DSAP:=Stored_Server_SAP, Rem_Add:=Server_Add, L_sdu:=Idle-RES-PDU, Serv_class:=Low)	START-POLL-RES
27	START-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DL && L_sdu = Idle-REQ-PDU && Go_Abort = TRUE => Stop R-Timer Start S-Timer DMPMM2_DATA_REPLY.req(SSAP:=Client_SAP, DSAP:=Stored_Server_SAP, Rem_Add:=Server_Add, L_sdu:=Abort-REQ-PDU with stored Parameter, Serv_class:=Low)	END-POLL-RES
28	START-POLL-RES	MSAC2M_Initiate.req(C_Ref, Send_Timeout, Features_Supported, Profile_Features_Supported, Profile_Ident_Number, Add_Addr_Param) => C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_SE, Go_Abort := TRUE STORE_ABORT_PARAMETER MSAC2M_Abort.ind(C_Ref, Locally_Generated, Subnet, Instance, Reason_Code)	START-POLL-RES
29	START-POLL-RES	MSAC2M_Abort.req(C_Ref=Stored_C_Ref, Subnet, Instance, Reason_Code) => Go_Abort := TRUE STORE_ABORT_PARAMETER	START-POLL-RES
30	START-POLL-RES	MSAC2M_XXX.req(C_Ref=Stored_C_Ref, <REQPARAM>) => C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_SE, Go_Abort := TRUE STORE_ABORT_PARAMETER MSAC2M_Abort.ind(C_Ref, Locally_Generated, Subnet, Instance, Reason_Code)	START-POLL-RES

Table 489 (continued)

#	Current state	Event / condition => action	Next state
31	START-POLL-RES	R-Timer expired => C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_TO, STORE_ABORT_PARAMETER Start S-Timer MSAC2M_Abort.ind(C_Ref, Locally_Generated, Subnet, Instance, Reason_Code)	END-WAIT-CON
32	OPEN	MSAC2M_XXX.req(C_Ref=Stored_C_Ref, <REQPARAM>) /Length <= (Stored_Max_Data_Length) && Length <= (Stored_Max_Local_Data_Length) => Service_Header := Function_Num Stop S-Timer Start R-Timer DMPMM2_DATA_REPLY.req(SSAP:=Client_SAP, DSAP:=Stored_Server_SAP, Rem_Add:=Server_Add, L_sdu:=VALID_SERVICE_REQ_PDU, Serv_class:=Low)	VALID-SERVICE-POLL-RES
33	OPEN	MSAC2M_XXX.req(C_Ref=Stored_C_Ref, <REQPARAM>) /Length > (Stored_Max_Data_Length) Length > (Stored_Max_Local_Data_Length) => MSAC2M_Reject.ind(C_Ref:=Stored_C_Ref, Reason_Code:=REJ_LE)	OPEN
34	OPEN	S-Timer expired => Start R-Timer DMPMM2_DATA_REPLY.req(SSAP:=Client_SAP, DSAP:=Stored_Server_SAP, Rem_Add:=Server_Add, L_sdu:=Idle-REQ-PDU, Serv_class:=Low)	IDLE-POLL-RES
35	OPEN	MSAC2M_Abort.req(C_Ref=Stored_C_Ref, Subnet, Instance, Reason_Code) => Stop S-Timer Start S-Timer DMPMM2_DATA_REPLY.req(SSAP:=Client_SAP, DSAP:=Stored_Server_SAP, Rem_Add:=Server_Add, L_sdu:=Abort-REQ-PDU, Serv_class:=Low)	END-POLL-RES
36	OPEN	MSAC2M_Initiate.req(C_Ref, Send_Timeout, Features_Supported, Profile_Features_Supported, Profile_Ident_Number, Add_Addr_Param) => Stop S-Timer Start S-Timer MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_SE) DMPMM2_DATA_REPLY.req(SSAP:=Client_SAP, DSAP:=Stored_Server_SAP, Rem_Add:=Server_Add, L_sdu:=Abort-REQ-PDU, Serv_class:=Low)	END-POLL-RES
37	VALID-SERVICE-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = NR => DMPMM2_DATA_REPLY.req(SSAP:=Client_SAP, DSAP:=Stored_Server_SAP, Rem_Add:=Server_Add, L_sdu.len:=0, Serv_class:=Low)	VALID-SERVICE-POLL-RES
38	VALID-SERVICE-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DL && L_sdu = Idle-REQ-PDU => Stop R-Timer Start R-Timer DMPMM2_DATA_REPLY.req(SSAP:=Client_SAP, DSAP:=Stored_Server_SAP, Rem_Add:=Server_Add, L_sdu:=Idle-RES-PDU, Serv_class:=Low)	VALID-SERVICE-POLL-RES
39	VALID-SERVICE-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DL && L_sdu = VALID_SERVICE_RES_PDU && Function_Num = Service_Header && Go_Abort = FALSE => Stop R-Timer Start S-Timer MSAC2M_<VALID_SERVICE>.cnf(+, C_Ref:=Stored_C_Ref, <VALID_SERVICE_RES_PARAM>)	OPEN

Table 489 (continued)

#	Current state	Event / condition => action	Next state
40	VALID-SERVICE-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DL && L_sdu = VALID_SERVICE_NRS_PDU && Function_Num = (Service_Header+0x80) && Go_Abort = FALSE => Stop R-Timer Start S-Timer MSAC2M_<VALID_SERVICE>.cnf(-, C_Ref:=Stored_C_Ref, Rem_Add, <VALID_SERVICE_NRS_PARAM>)	OPEN
41	VALID-SERVICE-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DL && NOT(Abort-REQ-PDU) && Go_Abort = TRUE => ignore L_sdu Stop R-Timer Start S-Timer DMPMM2_DATA_REPLY.req(SSAP:=Client_SAP, DSAP:=Stored_Server_SAP, Rem_Add:=Server_Add, L_sdu:=Abort-REQ-PDU with stored Parameter, Serv_class:=Low)	END-POLL-RES
42	VALID-SERVICE-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DS/UE/RS => Stop R-Timer MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=DLL, Reason_Code:=L_status) MSAC2M_Closed.ind(C_Ref:=Stored_C_Ref)	CLOSED
43	VALID-SERVICE-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DL && Go_Abort = FALSE && NOT(L_sdu = VALID_SERVICE_RES_PDU && (Function_Num = Service_Header)) && NOT(L_sdu = VALID_SERVICE_NRS_PDU && (Function_Num = (Service_Header+0x80))) && NOT(L_sdu = Abort-REQ-PDU) && NOT(L_sdu = Idle-REQ-PDU) => Stop R-Timer Start S-Timer MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_RE) DMPMM2_DATA_REPLY.req(SSAP:=Client_SAP, DSAP:=Stored_Server_SAP, Rem_Add:=Server_Add, L_sdu:=Abort-REQ-PDU, Serv_class:=Low)	END-POLL-RES
44	VALID-SERVICE-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = RDL/NA/RR/LR/DH/RDH => ignore L_sdu if present Stop R-Timer Start S-Timer MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=DLL, Reason_Code:=L_status) DMPMM2_DATA_REPLY.req(SSAP:=Client_SAP, DSAP:=Stored_Server_SAP, Rem_Add:=Server_Add, L_sdu:=Abort-REQ-PDU, Serv_class:=Low)	END-POLL-RES
45	VALID-SERVICE-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DL && L_sdu = Abort-REQ-PDU => Stop R-Timer MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=FALSE, Subnet, Instance, Reason_Code) MSAC2M_Closed.ind(C_Ref:=Stored_C_Ref)	CLOSED
46	VALID-SERVICE-POLL-RES	MSAC2M_XXX.req(C_Ref:=Stored_C_Ref, <REQPARAM>) => MSAC2M_Reject.ind(C_Ref:=Stored_C_Ref, Reason_Code:=REJ_PS)	VALID-SERVICE-POLL-RES

Table 489 (continued)

#	Current state	Event / condition => action	Next state
47	VALID-SERVICE-POLL-RES	MSAC2M_Initiate.req(C_Ref, Send_Timeout, Features_Supported, Profile_Features_Supported, Profile_Ident_Number, Add_Addr_Param) => C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_SE, Go_Abort := TRUE STORE_ABORT_PARAMETER MSAC2M_Abort.ind(C_Ref, Locally_Generated, Subnet, Instance, Reason_Code)	VALID-SERVICE-POLL-RES
48	VALID-SERVICE-POLL-RES	MSAC2M_Abort.req(C_Ref=Stored_C_Ref, Subnet, Instance, Reason_Code) => Go_Abort := TRUE STORE_ABORT_PARAMETER	VALID-SERVICE-POLL-RES
49	VALID-SERVICE-POLL-RES	R-Timer expired => C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_TO, STORE_ABORT_PARAMETER Start S-Timer MSAC2M_Abort.ind(C_Ref, Locally_Generated, Subnet, Instance, Reason_Code)	END-WAIT-CON
50	END-WAIT-CON	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /(L_status = DL && L_sdu <> Abort-REQ-PDU) (L_status = NR) => Stop S-Timer Start S-Timer DMPMM2_DATA_REPLY.req(SSAP:=Client_SAP, DSAP:=Stored_Server_SAP, Rem_Add:=Server_Add, L_sdu:=Abort-REQ-PDU with stored Parameter, Serv_class:=Low)	END-POLL-RES
51	END-WAIT-CON	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DS/UE/RS/RDL/NA/RR/LR/RDH/DH => C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=DLL, Reason_Code:=L_status Stop S-Timer MSAC2M_Abort.ind(C_Ref, Locally_Generated, Subnet, Instance, Reason_Code)	CLOSED
52	END-WAIT-CON	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DL && L_sdu = Abort-REQ-PDU => Stop S-Timer MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=FALSE, Subnet, Instance, Reason_Code) MSAC2M_Closed.ind(C_Ref:=Stored_C_Ref)	CLOSED
53	END-WAIT-CON	MSAC2M_XXX.req(C_Ref=Stored_C_Ref, <REQPARAM>) => MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_SE)	END-WAIT-CON
54	END-WAIT-CON	MSAC2M_Abort.req(C_Ref=Stored_C_Ref, Subnet, Instance, Reason_Code) => MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_SE)	END-WAIT-CON
55	END-WAIT-CON	MSAC2M_Initiate.req(C_Ref, Send_Timeout, Features_Supported, Profile_Features_Supported, Profile_Ident_Number, Add_Addr_Param) => MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_SE)	END-WAIT-CON
56	END-WAIT-CON	S-Timer expired => MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_OC)	FINAL-WAIT-CON

Table 489 (continued)

#	Current state	Event / condition => action	Next state
57	IDLE-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = NR => DMPMM2_DATA_REPLY.req(SSAP:=Client_SAP, DSAP:=Stored_Server_SAP, Rem_Add:=Server_Add, L_sdu.len:=0, Serv_class:=Low)	IDLE-POLL-RES
58	IDLE-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DL && L_sdu = Idle-RES-PDU && Service_Buffer = empty && Go_Abort = FALSE => Stop R-Timer Start S-Timer	OPEN
59	IDLE-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DL && L_sdu = Idle-RES-PDU && Service_Buffer <> empty && Go_Abort = FALSE => Service_Header:= Service_Buffer[1] Service_Buffer:=empty Stop R-Timer Start R-Timer DMPMM2_DATA_REPLY.req(SSAP:=Client_SAP, DSAP:=Stored_Server_SAP, Rem_Add:=Server_Add, L_sdu:=Service_Buffer, Serv_class:=Low)	VALID-SERVICE-POLL-RES
60	IDLE-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DL && L_sdu = Idle-RES-PDU && Go_Abort = TRUE => Stop R-Timer Start S-Timer DMPMM2_DATA_REPLY.req(SSAP:=Client_SAP, DSAP:=Stored_Server_SAP, Rem_Add:=Server_Add, L_sdu:=Abort-REQ-PDU with stored Parameter, Serv_class:=Low)	END-POLL-RES
61	IDLE-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DL && L_sdu = Abort-REQ-PDU => Stop R-Timer MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=FALSE, Subnet, Instance, Reason_Code) MSAC2M_Closed.ind(C_Ref:=Stored_C_Ref)	CLOSED
62	IDLE-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DL && L_sdu <> Idle-RES-PDU && L_sdu <> Abort-REQ-PDU => Stop R-Timer Start S-Timer MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_RE) DMPMM2_DATA_REPLY.req(SSAP:=Client_SAP, DSAP:=Stored_Server_SAP, Rem_Add:=Server_Add, L_sdu:=Abort-REQ-PDU, Serv_class:=Low)	END-POLL-RES
63	IDLE-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DS/UE/RS => Stop R-Timer MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=DLL, Reason_Code:=L_status) MSAC2M_Closed.ind(C_Ref:=Stored_C_Ref)	CLOSED

Table 489 (continued)

#	Current state	Event / condition => action	Next state
64	IDLE-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = RDL/NA/RR/LR/DH/RDH => ignore L_sdu if present Stop R-Timer Start S-Timer MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=DLL, Reason_Code:=L_status) DMPMM2_DATA_REPLY.req(SSAP:=Client_SAP, DSAP:=Stored_Server_SAP, Rem_Add:=Server_Add, L_sdu:=Abort-REQ-PDU, Serv_class:=Low)	END-POLL-RES
65	IDLE-POLL-RES	MSAC2M_XXX.req(C_Ref=Stored_C_Ref, <REQPARAM>) /Length <= (Stored_Max_Data_Length) && Length <= (Stored_Max_Local_Data_Length) && Service_Buffer = empty => <LOAD_SERVICE_BUFFER>	IDLE-POLL-RES
66	IDLE-POLL-RES	MSAC2M_XXX.req(C_Ref=Stored_C_Ref, <REQPARAM>) /(Length > (Stored_Max_Data_Length) Length > (Stored_Max_Local_Data_Length)) && Service_Buffer = empty => MSAC2M_Reject.ind(C_Ref:=Stored_C_Ref, Reason_Code:=REJ_LE)	IDLE-POLL-RES
67	IDLE-POLL-RES	MSAC2M_XXX.req(C_Ref=Stored_C_Ref, <REQPARAM>) //Service_Buffer <> empty => MSAC2M_Reject.ind(C_Ref:=Stored_C_Ref, Reason_Code:=REJ_PS)	IDLE-POLL-RES
68	IDLE-POLL-RES	MSAC2M_Initiate.req(C_Ref, Send_Timeout, Features_Supported, Profile_Features_Supported, Profile_Ident_Number, Add_Addr_Param) => C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_SE, Go_Abort := TRUE STORE_ABORT_PARAMETER MSAC2M_Abort.ind(C_Ref, Locally_Generated, Subnet, Instance, Reason_Code)	IDLE-POLL-RES
69	IDLE-POLL-RES	MSAC2M_Abort.req(C_Ref=Stored_C_Ref, Subnet, Instance, Reason_Code) => Go_Abort:=TRUE STORE_ABORT_PARAMETER	IDLE-POLL-RES
70	IDLE-POLL-RES	R-Timer expired => C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_TO, STORE_ABORT_PARAMETER Start S-Timer MSAC2M_Abort.ind(C_Ref, Locally_Generated, Subnet, Instance, Reason_Code)	END-WAIT-CON
71	END-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = NR => Stop S-Timer MSAC2M_Closed.ind(C_Ref:=Stored_C_Ref)	CLOSED
72	END-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DL => Stop S-Timer ignore L_sdu MSAC2M_Closed.ind(C_Ref:=Stored_C_Ref)	CLOSED

Table 489 (continued)

#	Current state	Event / condition => action	Next state
73	END-POLL-RES	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status = DS/UE/RS/RDL/NA/RR/LR/RDH/DH => ignore L_sdu if present Stop S-Timer MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=DLL, Reason_Code:=L_status) MSAC2M_Closed.ind(C_Ref:=Stored_C_Ref)	CLOSED
74	END-POLL-RES	MSAC2M_XXX.req(C_Ref=Stored_C_Ref, <REQPARAM>) => MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_SE)	END-POLL-RES
75	END-POLL-RES	MSAC2M_Initiate.req(C_Ref, Send_Timeout, Features_Supported, Profile_Features_Supported, Profile_Ident_Number, Add_Addr_Param) => MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_SE)	END-POLL-RES
76	END-POLL-RES	MSAC2M_Abort.req(C_Ref=Stored_C_Ref, Subnet, Instance, Reason_Code) => ignore	END-POLL-RES
77	END-POLL-RES	S-Timer expired => MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_OC)	FINAL-WAIT-CON
78	FINAL-WAIT-CON	DMPMM2_DATA_REPLY.cnf(SSAP=Client_SAP, DSAP=Stored_Server_SAP, Rem_add=Server_Add, L_sdu, Serv_class=Low, L_Status) /L_status <> IV/LS => ignore L_sdu if present MSAC2M_Closed.ind(C_Ref:=Stored_C_Ref)	CLOSED
79	FINAL-WAIT-CON	MSAC2M_XXX.req(C_Ref=Stored_C_Ref, <REQPARAM>) => MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_OC)	FINAL-WAIT-CON
80	FINAL-WAIT-CON	MSAC2M_Initiate.req(C_Ref, Send_Timeout, Features_Supported, Profile_Features_Supported, Profile_Ident_Number, Add_Addr_Param) => MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_OC)	FINAL-WAIT-CON
81	FINAL-WAIT-CON	MSAC2M_Abort.req(C_Ref=Stored_C_Ref, Subnet, Instance, Reason_Code) => MSAC2M_Abort.ind(C_Ref:=Stored_C_Ref, Locally_Generated:=TRUE, Subnet:=NO, Instance:=MSAC2M, Reason_Code:=ABT_OC)	FINAL-WAIT-CON
82	ANY-STATE	MSAC2M_Reset.req => Stop R-Timer Stop S-Timer MSAC2M_Reset.cnf	POWER-ON

6.6.13 MMAC2

6.6.13.1 Primitive definitions

6.6.13.1.1 Primitives exchanged between MMAC2 and FSPMM2

Table 490 shows the service primitives including their associated parameters issued by the FSPMM2 and received by the MMAC2.

Table 490 – Primitives issued by FSPMM2 to MMAC2

Primitive name	Source	Associated parameters	Functions
MInit MM.req	FSPMM2	Max Local Data Length	
Reset.req	FSPMM2	(none)	
Abort.req	FSPMM2	(none)	
Get Master Diag.req	FSPMM2	Rem Add, MDiag Identifier	
Start Seq.req	FSPMM2	Rem Add, Area Code, Timeout	
Download.req	FSPMM2	Rem Add, Area Code, Add Offset, Data	
Upload.req	FSPMM2	Rem Add, Area Code, Add Offset, Data Len	
End Seq.req	FSPMM2	Rem Add	
Act Param.req	FSPMM2	Rem Add, Area Code, Activate	
Act Para Brct.req	FSPMM2	Rem Add, Area Code	

Table 491 shows the service primitives including their associated parameters issued by the MMAC2 and received by the FSPMM2.

Table 491 – Primitives issued by MMAC2 to FSPMM2

Primitive name	Source	Associated parameters	Functions
MInit MM.cnf	MMAC2		
Reset.cnf	MMAC2	(none)	
Get Master Diag.cnf(+)	MMAC2	Rem Add, Diagnosis Data	
Get Master Diag.cnf(-)	MMAC2	Rem Add, Status	
Start Seq.cnf(+)	MMAC2	Rem Add, Max Len Data Unit	
Start Seq.cnf(-)	MMAC2	Rem Add, Status	
Download.cnf(+)	MMAC2	Rem Add	
Download.cnf(-)	MMAC2	Rem Add, Status	
Upload.cnf(+)	MMAC2	Rem Add, Data	
Upload.cnf(-)	MMAC2	Rem Add, Status	
End Seq.cnf(+)	MMAC2	Rem Add	
End Seq.cnf(-)	MMAC2	Rem Add, Status	
Act Param.cnf(+)	MMAC2	Rem Add	
Act Param.cnf(-)	MMAC2	Rem Add, Status	
Act Para Brct.cnf(+)	MMAC2	Rem Add	
Act Para Brct.cnf(-)	MMAC2	Rem Add, Status	
Abort.ind	MMAC2	Rem Add	
Fault.ind	MMAC2	(none)	

6.6.13.1.2 Parameters of MMAC2 primitives

The parameters used with the primitives exchanged between MMAC2 and FSPMM2 are described in FAL Service Definition Type 3 Fieldbus IEC 61158-5. Additional parameters are described in Table 492.

Table 492 – Parameters used with primitives exchanged with MMAC2

Parameter name	Description
Rem_Add	DL Address of the remote station

6.6.13.2 State machine description

The state machine for Master-Master communication specifies the requester side. Typically, these are programming or diagnostic devices.

After reset, the MMAC2 changes to the "POWER-ON" state and waits for initialisation by the User.

The state machine resumes as long in the "WAIT-REQ" state as it receives a valid request from the User. After this request was sent, the state machine remains in the "WAIT-RES" state, until the service is terminated successfully or incorrectly.

At any time, maximally one service request is in execution.

Local Variables

T1

(Unsigned16)

Time which can elapse at most between a request and the receipt of the associated response.

Server_Add

(Unsigned8)

Address of the DP-master (Class 1) just communicating with.

Service-Header

(Octet-String of length 4)

Intermediate memory for the first four octets of the last request.

Srv_lst

This list is used to hold all information which are necessary for the service SAP_ACTIVATE.

Upload_Data_Len

(Unsigned8)

This parameter specifies the length of the requested data.

6.6.13.3 MMAC2 state table

Table 493 contains the complete description of the MMAC2 state machine.

Table 493 – MMAC2 state table

#	Current state	Event / condition => action	Next state
1	POWER-ON	MMAC2_Minit_MM.req =>MMAC2_Minit_MM.cnf	WAIT-REQ
2	WAIT-REQ	MMAC2_Get_Master_Diag.req(Rem_Add,Identifier) =>Service_Header[1]:=0x41 Service_Header[2]:=Identifier Server_Add:=Rem_Add Start T1 DMPMM2_DATA_REPLY.req(SSAP:=54, DSAP:=54, Rem_add:=Rem_Add, L_sdu:=Service_Header, Serv_class:=Low)	WAIT-RES
3	WAIT-REQ	MMAC2_Start_Seq.req(Rem_Add,Area_Code,Timeout) =>Service_Header[1]:=0x42 Service_Header[2]:=Area_Code Service_Header[3-4]:=Timeout Server_Add:=Rem_Add Start T1 DMPMM2_DATA_REPLY.req(SSAP:=54, DSAP:=54, Rem_add:=Rem_Add, L_sdu:=Service_Header, Serv_class:=Low)	WAIT-RES
4	WAIT-REQ	MMAC2_Download.req(Rem_Add,Area_Code,Add_Offset,Data) =>Service_Header[1]:=0x42 Service_Header[2]:=Area_Code Service_Header[3-4]:=Add_Offset Server_Add:=Rem_Add Start T1 DMPMM2_DATA_REPLY.req(SSAP:=54, DSAP:=54, Rem_add:=Rem_Add, L_sdu:=(Service_Header,Data), Serv_class:=Low)	WAIT-RES
5	WAIT-REQ	MMAC2_Upload.req(Rem_Add,Area_Code,Add_Offset, Data_Len) =>Service_Header[1]:=0x44 Service_Header[2]:=Area_Code Service_Header[3-4]:=Add_Offset Upload_Data_Len := Data_Len Server_Add:=Rem_Add Start T1 DMPMM2_DATA_REPLY.req(SSAP:=54, DSAP:=54, Rem_add:=Rem_Add, L_sdu:=(Service_Header,Data_Len), Serv_class:=Low)	WAIT-RES
6	WAIT-REQ	MMAC2_End_Seq.req(Rem_Add) =>Service_Header[1]:=0x45 Server_Add:=Rem_Add Start T1 DMPMM2_DATA_REPLY.req(SSAP:=54, DSAP:=54, Rem_add:=Rem_Add, L_sdu:=Service_Header[1], Serv_class:=Low)	WAIT-RES
7	WAIT-REQ	MMAC2_Act_Param.req(Rem_Add, Area_Code, Activate) =>Service_Header[1]:=0x47 Service_Header[2]:=Area_Code Service_Header[3]:=Activate Server_Add:=Rem_Add Start T1 DMPMM2_DATA_REPLY.req(SSAP:=54, DSAP:=54, Rem_add:=Rem_Add, L_sdu:=Service_Header[1-3], Serv_class:=Low)	WAIT-RES
8	WAIT-REQ	MMAC2_Act_Para_Brct.req(Rem_Add, Area_Code) =>DMPMM2_DATA.req(SSAP:=54, DSAP:=54, Rem_add:=Rem_Add, L_sdu:=(0x46,Area_Code), Serv_class:=Low)	WAIT-CON
9	WAIT-CON	MMAC2_XXX.req =>MMAC2_Reject.ind(Rem_Add,Reason_Code:=REJ_CO)	WAIT-CON
10	WAIT-CON	DMPMM2_DATA.cnf(SSAP=54, DSAP=54, Rem_Add, Serv_Class, L_status) /L_status=OK/DS =>MMAC2_Act_Para_Brct.cnf(Rem_Add,Status:=L_status)	WAIT-REQ
11	WAIT-RES	MMAC2_XXX.req =>MMAC2_Reject.ind(Rem_Add,Reason_Code:=REJ_PS)	WAIT-RES
12	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=NR =>DMPMM2_DATA_REPLY.req(SSAP:=54, DSAP:=54, Rem_add:=Server_Add, L_sdu.len:=0, Serv_class:=Low)	WAIT-RES
13	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=IV/LS =>MMAC2_Fault.ind	POWER-ON

Table 493 (continued)

#	Current state	Event / condition => action	Next state
14	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=DS/NA/UE/RR/RS && Service_Header[1]=0x41 =>Stop T1 MMAC2_Get_Master_Diag.cnf(Rem_Add, Status:=L_status, Diagnostic_Data:=L_sdu)	WAIT-REQ
15	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=DS/NA/UE/RR/RS && Service_Header[1]=0x42 =>Stop T1 MMAC2_Start_Seq.cnf(Rem_Add, Status:=L_status)	WAIT-REQ
16	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=DS/NA/UE/RR/RS && Service_Header[1]=0x43 =>Stop T1 MMAC2_Download.cnf(Rem_Add, Status:=L_status)	WAIT-REQ
17	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=DS/NA/UE/RR/RS && Service_Header[1]=0x44 =>Stop T1 MMAC2_Upload.cnf(Rem_Add, Status:=L_status)	WAIT-REQ
18	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=DS/NA/UE/RR/RS && Service_Header[1]=0x45 =>Stop T1 MMAC2_End_Seq.cnf(Rem_Add, Status:=L_status)	WAIT-REQ
19	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=DS/NA/UE/RR/RS && Service_Header[1]=0x47 =>Stop T1 MMAC2_Act_Param.cnf(Rem_Add, Status:=L_status)	WAIT-REQ
20	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=RDL/RDH && Service_Header[1]=0x41 =>Stop T1 MMAC2_Get_Master_Diag.cnf(Rem_Add, Status:=RR)	WAIT-REQ
21	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=RDL/RDH && Service_Header[1]=0x42 =>Stop T1 MMAC2_Start_Seq.cnf(Rem_Add, Status:=RR)	WAIT-REQ
22	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=RDL/RDH && Service_Header[1]=0x43 =>Stop T1 MMAC2_Download.cnf(Rem_Add, Status:=RR)	WAIT-REQ
23	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=RDL/RDH && Service_Header[1]=0x44 =>Stop T1 MMAC2_Upload.cnf(Rem_Add, Status:=RR)	WAIT-REQ
24	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=RDL/RDH && Service_Header[1]=0x45 =>Stop T1 MMAC2_End_Seq.cnf(Rem_Add, Status:=RR)	WAIT-REQ
25	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=RDL/RDH && Service_Header[1]=0x47 =>Stop T1 MMAC2_Act_Param.cnf(Rem_Add, Status:=RR)	WAIT-REQ
26	WAIT-RES	T1 expired /Service_Header[1]=0x41 =>MMAC2_Get_Master_Diag.cnf(Rem_Add, Status:=TO)	TIMO
27	WAIT-RES	T1 expired /Service_Header[1]=0x42 =>MMAC2_Start_Seq.cnf(Rem_Add, Status:=TO)	TIMO

Table 493 (continued)

#	Current state	Event / condition => action	Next state
28	WAIT-RES	T1 expired /Service_Header[1]=0x43 =>MMAC2_Download.cnf(Rem_Add, Status:=TO)	TIMO
29	WAIT-RES	T1 expired /Service_Header[1]=0x44 =>MMAC2_Upload.cnf(Rem_Add, Status:=TO)	TIMO
30	WAIT-RES	T1 expired /Service_Header[1]=0x45 =>MMAC2_End_Seq.cnf(Rem_Add, Status:=TO)	TIMO
31	WAIT-RES	T1 expired /Service_Header[1]=0x47 =>MMAC2_Act_Param.cnf(Rem_Add, Status:=TO)	TIMO
32	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=DL && L_sdu[1]=FE/NE/AD/EA/LE/RE/IP && Service_Header[1]=0x41 =>Stop T1 MMAC2_Get_Master_Diag.cnf(Rem_Add, Status:=L_sdu[1])	WAIT-REQ
33	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=DL && L_sdu[1]=FE/NE/AD/EA/LE/RE/IP/NI/SE/SC && Service_Header[1]=0x42 =>Stop T1 MMAC2_Start_Seq.cnf(Rem_Add, Status:=L_sdu[1])	WAIT-REQ
34	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=DL && L_sdu[1]=FE/NE/AD/EA/LE/RE/NC/SC/NI && Service_Header[1]=0x43 =>Stop T1 MMAC2_Download.cnf(Rem_Add, Status:=L_sdu[1])	WAIT-REQ
35	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=DL && L_sdu[1]=FE/NE/AD/EA/LE/RE/NI/SC && Service_Header[1]=0x44 =>Stop T1 MMAC2_Upload.cnf(Rem_Add, Status:=L_sdu[1])	WAIT-REQ
36	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=DL && L_sdu[1]=FE/NE/AD/EA/LE/RE/NI/SE && Service_Header[1]=0x45 =>Stop T1 MMAC2_End_Seq.cnf(Rem_Add, Status:=L_sdu[1])	WAIT-REQ
37	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=DL && L_sdu[1]=FE/NE/AD/EA/LE/RE/IP/SC/NI/DI && Service_Header[1]=0x47 =>Stop T1 MMAC2_Act_Param.cnf(Rem_Add, Status:=L_sdu[1])	WAIT-REQ
38	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=DH/LR L_status=DL && L_sdu[1]<>FE/NE/AD/EA/LE/RE/IP && Service_Header[1]=0x41 && (L_sdu.len<3 L_sdu[1-2]<>Service_Header[1-2]) =>Stop T1 MMAC2_Get_Master_Diag.cnf(Rem_Add, Status:=RE)	WAIT-REQ
39	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=DH/LR L_status=DL && L_sdu[1]<>FE/NE/AD/EA/LE/RE/IP/NI/SE/SC && Service_Header[1]=0x42 && (L_sdu.len<>5 L_sdu[1-4]<>Service_Header[1-4]) =>Stop T1 MMAC2_Start_Seq.cnf(Rem_Add, Status:=RE)	WAIT-REQ

Table 493 (continued)

#	Current state	Event / condition => action	Next state
40	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=DH/LR L_status=DL && L_sdu[1]<>FE/NE/AD/EA/LE/RE/NC/SC/NI && Service_Header[1]=0x43 && (L_sdu[1-4]<>Service_Header[1-4] L_sdu.len<>4) =>Stop T1 MMAC2_Download.cnf(Rem_Add, Status:=RE)	WAIT-REQ
41	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=DH/LR L_status=DL && L_sdu[1]<>FE/NE/AD/EA/LE/RE/NI/SC && Service_Header[1]=0x44 && (L_sdu.len<5 L_sdu[1-4]<>Service_Header[1-4] L_sdu.len> Upload_Data_Len +4) =>Stop T1 MMAC2_Upload.cnf(Rem_Add, Status:=RE)	WAIT-REQ
42	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=DH/LR L_status=DL && L_sdu[1]<>FE/NE/AD/EA/LE/RE/NI/SE && Service_Header[1]=0x45 && (L_sdu[1]<>Service_Header[1] L_sdu.len<>1) =>Stop T1 MMAC2_End_Seq.cnf(Rem_Add, Status:=RE)	WAIT-REQ
43	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=DH/LR L_status=DL && L_sdu[1]<>FE/NE/AD/EA/LE/RE/IP/SC/NI/DI && Service_Header[1]=0x47 && (L_sdu[1-3]<>Service_Header[1-3] L_sdu.len<>3) =>Stop T1 MMAC2_Act_Param.cnf(Rem_Add, Status:=RE)	WAIT-REQ
44	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=DL && L_sdu[1]<>FE/NE/AD/EA/LE/RE/IP && Service_Header[1]=0x41 && (L_sdu.len>=3 && L_sdu[1-2]=Service_Header[1-2]) =>Stop T1 MMAC2_Get_Master_Diag.cnf(Rem_Add, Status:=OK, Diagnostic_Data:=L_sdu[3-L_sdu.len])	WAIT-REQ
45	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=DL && L_sdu[1]<>FE/NE/AD/EA/LE/RE/IP/NI/SE/SC && Service_Header[1]=0x42 && (L_sdu.len=5 && L_sdu[1-4]=Service_Header[1-4]) =>Stop T1 MMAC2_Start_Seq.cnf(Rem_Add, Status :=OK, Max_Length_Data_Unit:=L_sdu[5])	WAIT-REQ
46	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=DL && L_sdu[1]<>FE/NE/AD/EA/LE/RE/NC/SC/NI && Service_Header[1]=0x43 && (L_sdu[1-4]=Service_Header[1-4] && L_sdu.len=4) =>Stop T1 MMAC2_Download.cnf(Rem_Add, Status:=OK)	WAIT-REQ
47	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=DL && L_sdu[1]<>FE/NE/AD/EA/LE/RE/NI/SC && Service_Header[1]=0x44 && (L_sdu.len>=5 && L_sdu[1-4]=Service_Header[1-4] && L_sdu.len<= Upload_Data_Len +4) =>Stop T1 MMAC2_Upload.cnf(Rem_Add, Status:=OK,Data:=L_sdu[5-L_sdu.len])	WAIT-REQ

Table 493 (continued)

#	Current state	Event / condition => action	Next state
48	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=DL && L_sdu[1]<>FE/NE/AD/EA/LE/RE/NI/SE && Service_Header[1]=0x45 && (L_sdu[1]=Service_Header[1] && L_sdu.len=1) =>Stop T1 MMAC2_End_Seq.cnf(Rem_Add, Status:=OK)	WAIT-REQ
49	WAIT-RES	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status) /L_status=DL && L_sdu[1]<>FE/NE/AD/EA/LE/RE/IP/SC/NI/DI && Service_Header[1]=0x47 && (L_sdu=Service_Header && L_sdu.len=3) =>Stop T1 MMAC2_Act_Param.cnf(Rem_Add, Status:=OK)	WAIT-REQ
50	TIMO	DMPMM2_DATA_REPLY.cnf(SSAP=54, DSAP=54, Rem_Add, L_sdu, Serv_Class, L_status)	WAIT-REQ
51	TIMO	MMAC2_XXX.req =>MMAC2_Reject.ind(Rem_Add, Reason_Code:=REJ_CO)	TIMO
52	any state	unexpected DL reaction =>Stop T1 MMAC22_Fault.ind	POWER-ON
53	any state	MMAC2_Reset.req =>Stop T1	WAIT-REQ

6.7 DLL Mapping Protocol Machines (DMPMs)

6.7.1 DMPMS

6.7.1.1 Primitive definitions

6.7.1.1.1 Primitives exchanged between DMPMS and FSPMS

Table 494 shows the service primitives including their associated parameters issued by the FSPMS and received by the DMPMS.

Table 494 – Primitives issued by FSPMS to DMPMS

Primitive name	Source	Associated parameters	Functions
SInit DLL.req	FSPMS	(none)	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Reset.req	FSPMS	(none)	

Table 495 shows the service primitives including their associated parameters issued by the DMPMS and received by the FSPMS.

Table 495 – Primitives issued by DMPMS to FSPMS

Primitive name	Source	Associated parameters	Functions
SInit DL.cnf	DMPMS	Loc_Station_Address	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Reset.cnf	DMPMS	(none)	
Fault.ind	DMPMS	(none)	

6.7.1.1.2 Primitives exchanged between DMPMS and MSCY1S

Table 496 shows the service primitives including their associated parameters issued by the MSCY1S and received by the DMPMS.

Table 496 – Primitives issued by MSCY1S to DMPMS

Primitive name	Source	Associated parameters	Functions
Slave Init req	MSCY1S	(none)	Initialising of the resorces used by DP
Slave Deact.req	MSCY1S	(none)	Deactivation of LSAPs for MS0, MS1
Enter.req	MSCY1S	(none)	Preparing the DLL resources for Data Exchange
Leave.req	MSCY1S	(none)	Release the DLL resources for Data Exchange
Set minTsdr.req	MSCY1S	MinTsdr	Change of the minTsdr Parameter in DLL
Slave Diag Upd.req	MSCY1S	Diag Data, Reference	Setup of the DL update buffer for Diagnosis
Data Exchange Upd.req	MSCY1S	Diag Flag, Inp Data	Transfer of the Inp Data buffer to the DL update buffer
RD Outp Upd.req	MSCY1S	Outp Data	Setup a copy of the Outp Data buffer
RD Inp Upd.req	MSCY1S	Inp Data	Setup a copy of the Inp Data buffer
Get Cfg Upd.req	MSCY1S	Cfg Data	Setting of configuration data

Table 497 shows the service primitives including their associated parameters issued by the DMPMS and received by the MSCY1S.

Table 497 – Primitives issued by DMPMS to MSCY1S

Primitive name	Source	Associated parameters	Functions
Slave Init.cnf	DMPMS	(none)	Success of Initialisation
Reset.cnf	DMPMS	(none)	Termination of Reset service
Slave Deact.cnf	DMPMS	(none)	Success of deactivation of LSAPs for MS0, MS1
Set Slave Add.ind	DMPMS	New Slave Add, Ident Number, No Add Chg, Rem Slave Data	Change of the DL address of a DP-slave. Setting of remanent Slave Data.
Slave Diag.ind	DMPMS	Req Add, Reference	A DP-master fetches diagnosis information.
Set Prm.ind	DMPMS	Req Add, Prm Data	Parameter data are delivered to the DP-slave. The parameterization is first done in the start-up phase and is also while the DP-slave is in the Data Exchang mode.
Chk Cfg.ind	DMPMS	Req Add, Cfg Data	Received configuration data to the DP-slave for checking. They contain the format of input and output areas as well as the information about the data consistency.
Data Exchange.ind	DMPMS	Outp Data	Transmits output data to a DP-slave and requests input data.
Global Control.ind	DMPMS	Control Command, Group Select	Control Command accepted only from DP master (Class1) to which this Slave is assigned. Master reports his Operation Mode with this command.

6.7.1.1.3 Primitives exchanged between DMPMS and SSCY1S

Table 498 shows the service primitives including their associated parameters issued by the DMPMS and received by the SSCY1S.

Table 498 – Primitives issued by DMPMS to SSCY1S

Primitive name	Source	Associated parameters	Functions
DX_Entered.ind	DMPMS	Rem_add, Status	entered the DATA-EXCH mode
DX_Broadcast.ind	DMPMS	Rem_add, Data	Broadcast received

6.7.1.1.4 Primitives exchanged between DMPMS and MSAC1S, MSRM2S, MSAC2S

Table 499 shows the service primitives including their associated parameters issued by the MSAC1S, MSRM2S and MSAC2S and received by the DMPMS.

Table 499 – Primitives issued by MSAC1S, MSRM2S, MSAC2S to DMPMS

Primitive name	Source	Associated parameters	Functions
RSAP_ACTIVATE.req	MSAC1S, MSRM2S, MSAC2S	SSAP, Access, L_sdu_length_list, Indication_Mode	Activates local SAP
SAP_DEACTIVATE.req	MSAC1S, MSRM2S, MSAC2S	SSAP	Deactivates local SAP
REPLY_UPDATE.req	MSAC1S, MSRM2S, MSAC2S	SSAP, L_sdu, Serv_class, Transmit	Write Data in the Update Buffer for Transmission

Table 500 shows the service primitives including their associated parameters issued by the DMPMS and received by the MSAC1S, MSRM2S and MSAC2S.

Table 500 – Primitives issued by DMPMS to MSAC1S, MSRM2S, MSAC2S

Primitive name	Source	Associated parameters	Functions
RSAP_ACTIVATE.cnf	DMPMS	SSAP, M_Status	Activation finished
SAP_DEACTIVATE.cnf	DMPMS	SSAP, M_Status	Deactivation finished
REPLY_UPDATE.cnf	DMPMS	SSAP, Serv_class, L_status	Update finished
DATA_REPLY.ind	DMPMS	SSAP, DSAP, Rem_Add, L_sdu, Serv_class, Loc_Add, Update_status	Data/Request received

6.7.1.1.5 Primitives exchanged between DMPMS and MSCS1S

Table 501 shows the service primitives including their associated parameters issued by DMPMS and received by the MSCS1S.

Table 501 – Primitives issued by DMPMS to MSCS1S

Primitive name	Source	Associated parameters	Functions
CS_CLOCK_VALUE.ind	DMPMS	Time_master_addr CS_list, CS_status, Receive_delay_time	Indicates receiving of a clock message

6.7.1.1.6 Primitives exchanged between DMPMS and DL

Table 502 shows the service primitives including their associated parameters issued by the DMPMS and received by the DL.

Table 502 – Primitives issued by DMPMS to DL

Primitive name	Source	Associated parameters	Functions
DL-REPLY-UPDATE.req	DMPMS	Service_class, S_SAP_index, DLSDU, Transmit_strategy, Reference	
DLM-RESET.req	DMPMS	(none)	
DLM-SET-VALUE.req	DMPMS	Variable_name(1 to n), Desired_value (1 to n)	
DLM-DLSAP-ACTIVATE.req	DMPMS	S_SAP_index, Access, Service_list	
DLM-DLSAP-ACTIVATE-RESPONDER.req	DMPMS	S_SAP_index, Access, DLSDU_length_list, Indication_mode, Publisher_enabled	
DLM-DLSAP-ACTIVATE-SUBSCRIBER.req	DMPMS	S_SAP_index, DLSDU_length_list	
DLM-DLSAP-DEACTIVATE.req	DMPMS	S_SAP_index	

Table 503 shows the service primitives including their associated parameters issued by the DL and received by the DMPMS.

Table 503 – Primitives issued by DL to DMPMS

Primitive name	Source	Associated parameters	Functions
DL-DATA.ind	DL	Service_class, D_addr, D_SAP_index, S_addr, S_SAP_index, DLSDU	Refer to DL Service Definition Type 3 Fieldbus IEC 61158-3
DL-DATA-REPLY.ind	DL	Service_class, D_addr, D_SAP_index, S_addr, S_SAP_index, DLSDU, Update_Status Reference	
DL-MCT-DATA-REPLY.ind	DL	Service_class, D_addr, D_SAP_index, S_addr, S_SAP_index, DLSDU, Update_Status Reference	
DL-REPLY-UPDATE.cnf	DL	Service_class, S_SAP_index, DL_status	
DL-DXM-REPLY.ind	DL	D_addr, D_SAP_index, S_addr, S_SAP_index, DLSDU	

Primitive name	Source	Associated parameters	Functions
DL-CS-CLOCK-VALUE.ind	DL	D_addr, D_SAP_index, S_addr, S_SAP_index, DLSDU, Receive_delay_time, CS_Status	
DLM-RESET.cnf	DL	DLM_Status	
DLM-SET-VALUE.cnf	DL	Current_value (1 to n), DLM_status	
DLM-DLSAP-ACTIVATE.cnf	DL	S_SAP_index, DLM_status	
DLM-DLSAP-ACTIVATE-RESPONDER.cnf	DL	S_SAP_index, DLM_status	
DLM-DLSAP-ACTIVATE-SUBSCRIBER.cnf	DL	S_SAP_index, DLM_status	
DLM-DLSAP-DEACTIVATE.cnf	DL	S_SAP_index, DLM_status	

6.7.1.1.7 Parameters of DMPMS primitives

The parameters used with the primitives exchanged between DMPMS and other state machines of the Slave AL are described in Table 504.

Table 504 – Parameters used with primitives exchanged with DMPMS

Parameter name	Description
MasterAdd	This parameter conveys the DL address of the assigned DP-master.
Input Data Len	This parameter is used to setup the length of Input Data for this Slave.
Output Data Len	This parameter is used to setup the length of Output Data for this Slave.
Min Tsdr	This parameter conveys the value of the DLL variable minTsdr.
Diag Data	This parameter conveys the Diagnosis-RES-PDU
Diag Flag	This parameter indicates that a DP-slave has new Diag Data
Inp Data	This parameter conveys the DataExchange-RES-PDU
Outp Data	This parameter conveys the DataExchange-REQ-PDU
Cfg Data	This parameter conveys the Chk_Cfg-REQ-PDU
New Slave Add	This parameter conveys the new DL address for this DP-slave.
Ident Number	This parameter conveys the device type identifier for this DP-slave.
No Add Chg	This parameter indicates that there is no further address change allowed.
Rem Slave Data	This parameter conveys remanent parameter for this DP-slave.
Req Add	This parameter conveys the DL address of the initiator of this service.
Prm Data	This parameter conveys the Set_Prm-REQ-PDU.
Control Command	This parameter conveys the sync, freeze actions and the state of the DP-master.
Group Select	This parameter determines which groups shall be addressed by this command.
SSAP	Local Identifier for Service Access Point
Access	Defines the use of a SAP for one or several Remote Address (Stations)
L_sdu_length_list	Defines the maximum length of incoming or outgoing L_sdu
Indication_Mode	Defines the Mode for Indication of poll-sequences without user data
Serv_class	Indicates priority of the service
M_status	Status of the service execution
L_status	Status of the service execution
Transmit	Defines the operation mode of the update buffer (single = buffer is transmitted once)
Rem_Add	DL Address of the remote station
Loc_Add	DL Address of this station
Update_status	Indicates the transmission of an update buffer
Reference	Handle to identify diagnosis data
Data	This parameter conveys the data
Time_master_addr	DL address of the active Time Master

Parameter name	Description
CS_list	List of Time Value at the transmission of this service (Clock_value_time_event), Time Value at the previous transmission (Clock_value_previous_TE) and status information related to the clock value (Clock_value_status)
CS_status	Indicates the success or failure of the Clock Synchronisation sequence
Receive_delay_time	Time expired between receipt of the Time_Event and the invocation of that service primitive

6.7.1.2 State machine description

The DMPMS (DL Mapping Protocol Machine) makes the connection between the MS0-, MS1-, MS2-related Protocol Machines and Layer 2. All services are mapped by the DMPMS to the DL and its management services of Layer 2. The DMPMS delivers the necessary Layer 2 parameters of the function call (SSAP, DSAP, Serv_class, ...) for Layer 2, receives the confirmations and indications from Layer 2 and passes them to the MS0-, MS1-, MS2-related Protocol Machines.

Local Variables

Loc_Station_Address

(Unsigned8)

Local variable for the intermediate storage of the own station address which was delivered with the last Set_Value.

Prev_Diag_Flag

(Boolean)

Indicates the state of the Diag_Flag at the last Data_Exchange_Update.req By means of this flag it is detected whether a high prior update is present.

6.7.1.3 DMPMS state table

Table 505 contains the complete description of the DMPMS state machine.

Table 505 – DMPMS state table

#	Current state	Event / condition => action	Next state
1	PON	DMPMS_SInit_DLL.req(Bus Para) => Act_cnt := 0 DLM_SET_VALUE.req(Variable_name1:= "TS ", Variable_name2:= "Data_rate ", Variable_name3:= "TSL ", Variable_name4:= "min TSDR ", Desired_Value1:= Bus_Para.TS Desired_Value2:= Bus_Para.Data_rate, Desired_Value3:= Bus_Para.TSL, Desired_Value4:= 11)	DLM-SET-VALUE
2	DLM-SET-VALUE	DLM_SET_VALUE.cnf(DLM_status(1-5) /DLM_status(1-4)=OK => no action DMPMS_SInit_DLL.cnf	RUN
3	RUN	DMPMS_Enter.req => DMPMS_ActivateControl := DMPMS_Enter Status:=TRUE DMPMS_DX_Entered.ind (0..125, Status)	DLM-CHECK-RSAP-ACT

Table 505 (continued)

#	Current state	Event / condition => action	Next state
4	RUN	DMPMS_Slave_Init.req /ActivateRSAPFromCRLList(DMPMS_ActivateControl)=TRUE => DMPMS_ActivateControl := DMPMS_Init DLSAP_ACTIVATE_RESPONDER.req(S_SAP_index:=CREP.DSAP, Access:=CREP.Rem Add, DLSDU_length_list:=BuildRSAPServiceListFromCR(), Indication_mode:=CREP.Indication Mode, Publisher_enabled:=FALSE)	DLM-ACT-RSAP
5	DLM-CHECK-RSAP-ACT	/ActivateRSAPFromCRLList(DMPMS_ActivateControl)=TRUE => DLSAP_ACTIVATE_RESPONDER.req(S_SAP_index:=CREP.DSAP, Access:=CREP.Rem Add, DLSDU_length_list:=BuildRSAPServiceListFromCR(), Indication_mode:=CREP.Indication Mode, Publisher_enabled:= CREP.Publisher_enabled)	DLM-ACT-RSAP
6	DLM-CHECK-RSAP-ACT	/ActivateRSAPFromCRLList(DMPMS_ActivateControl)=FALSE && DMPMS_ActivateControl=DMPMS_Init => DMPMS_Slave_Init.cnf	RUN
7	DLM-CHECK-RSAP-ACT	/ActivateRSAPFromCRLList(DMPMS_ActivateControl)=FALSE && DMPMS_ActivateControl=DMPMS_Enter => DLSAP_ACTIVATE.req(S_SAP_index:=58, Access:=CREP.Rem_Add, (Service_activate:=SDN, Role_in_service:=Responder, DLSDU_length_list:=(0,2,0,0)))	DLM-ACT-SAP
8	DLM-ACT-RSAP	DLSAP_ACTIVATE_RESPONDER.cnf(SSAP, DLM_status) /DLM_status=OK	DLM-CHECK-RSAP-ACT
9	DLM-ACT-SAP	DLSAP_ACTIVATE.cnf(S_SAP_index, DLM_status) /DLM_status=OK && EnableSubscriber() = FALSE => Prev_Diag_Flag := False	RUN
10	RUN	DMPMS_Leave.req => DMPMS_DeactControl:=DMPMS_Leave Status:=FALSE DMPMS_DX_Entered.ind (0..125, Status)	CHECK-SAP-DEACT
11	RUN	DMPMS_Slave_Deact.req => DMPMS_DeactControl:=DMPMS_Deact	CHECK-SAP-DEACT
12	CHECK-SAP-DEACT	/DeactivateSAPFromCRLList(DMPMS_DeactControl)=TRUE => DLSAP_DEACTIVATE.req(S_SAP_index:=CREP.DSAP)	SAP-DEACT
13	CHECK-SAP-DEACT	/DeactivateSAPFromCRLList(DMPMS_DeactControl)=FALSE => DMPMS_Slave_Deact.cnf	RUN
14	SAP-DEACT	DLSAP_DEACTIVATE.cnf(S_SAP_index, DLM_status) /DLM_status=OK	CHECK-SAP-DEACT
15	RUN	DMPMS_Data_Exchange_Upd.req(Diag_Flag, Inp_Data) /Diag_Flag = False && Prev_Diag_Flag = False && Inp_Data.len > 0 => DL_REPLY_UPDATE.req(S:SAP_index:=NIL, DLSDU:=Inp_Data, Service_Class:=Low, Transmit_startegy:=Multiple, Reference:=Act_cnt)	RUN

Table 505 (continued)

#	Current state	Event / condition => action	Next state
16	RUN	DMPMS_Data_Exchange_Upd.req(Diag_Flag, Inp_Data) /Diag_Flag = False && Prev_Diag_Flag = False && Inp_Data.len = 0 => no action	RUN
17	RUN	DMPMS_Data_Exchange_Upd.req(Diag_Flag, Inp_Data) /Diag_Flag = False && Prev_Diag_Flag = True && Inp_Data.len > 0 => Prev_Diag_Flag := False DL_REPLY_UPDATE.req(S_SAP_index:=NIL, DLSDU:=Inp_Data, Service_Class:=Low, Transmit_strategy:=Multiple, Reference:=Act_cnt) DL_REPLY_UPDATE.req(S_SAP_index:=NIL, DLSDU:=NULL-PDU, Service_Class:=High, Transmit_strategy:=Single, Reference:=Act_cnt)	RUN
18	RUN	DMPMS_Data_Exchange_Upd.req(Diag_Flag, Inp_Data) /Diag_Flag = False && Prev_Diag_Flag = True && Inp_Data.len = 0 => Prev_Diag_Flag := False DL_REPLY_UPDATE.req(S_SAP_index:=NIL, DLSDU:=NULL-PDU, Service_Class:=High, Transmit_strategy:=Single, Reference:=Act_cnt)	RUN
19	RUN	DMPMS_Data_Exchange_Upd.req(Diag_Flag, Inp_Data) /Diag_Flag = True && Inp_Data.len > 0 => Prev_Diag_Flag := True DL_REPLY_UPDATE.req(S_SAP_index:=NIL, DLSDU:=Inp_Data, Service_Class:=High, Transmit_strategy:=Multiple, Reference:=Act_cnt)	RUN
20	RUN	DMPMS_Data_Exchange_Upd.req(Diag_Flag, Inp_Data) /Diag_Flag = True && Inp_Data.len = 0 => Prev_Diag_Flag := True DL_REPLY_UPDATE.req(S_SAP_index:=NIL, DLSDU:=ZERO-PDU, Service_Class:=High, Transmit_strategy:=Multiple, Reference:=Act_cnt)	RUN
21	RUN	DL_MCT_DATA_REPLY.ind(S_SAP_index, D_SAP_index, S_addr, D_addr, DLSDU, Update_status, Reference) /D_SAP_index=NIL => DMPMS_Data_Exchange.ind(Outp_Data:=DLSDU)	RUN
22	RUN	DL_DATA_REPLY.ind(S_SAP_index, D_SAP_index, S_addr, D_addr, DLSDU, Service_class, Update_status, Reference) /D_SAP_index=NIL => DMPMS_Data_Exchange.ind(Outp_Data:=DLSDU)	RUN
23	RUN	DMPMS_Slave_Diag_Upd.req(Diag_Data, Reference) => Act_cnt := Reference DL_REPLY_UPDATE.req(S_SAP_index:=60, DLSDU:=Diag_Data, Service_Class:=Low, Transmit_strategy:=Multiple, Reference:=Act_cnt)	RUN
24	RUN	DL_DATA_REPLY.ind(S_SAP_index, D_SAP_index, S_addr, D_addr, DLSDU, Service_class, Update_status, Reference) /D_SAP_index=60 => Act_cnt := Reference DMPMS_Slave_Diag.ind(Req_Add:=S_addr, Reference := Act_cnt)	RUN
25	RUN	DMPMS_Get_Cfg_Upd.req(Cfg_Data) => DL_REPLY_UPDATE.req(S_SAP_index:=59, DLSDU:=Cfg_Data, Service_class:=Low, Transmit_strategy:=Multiple, Reference:=Act_cnt)	RUN
26	RUN	DL_DATA_REPLY.ind(S_SAP_index, D_SAP_index, S_addr, D_addr, DLSDU, Service_class, Update_status, Reference) /D_SAP_index=59 => no action	RUN

Table 505 (continued)

#	Current state	Event / condition => action	Next state
27	RUN	DL_DATA.ind(S_SAP_index, D_SAP_index, S_addr, D_addr, DLSDU, Service_class) / D_SAP_index=58 && DLSDU = Global_Control-REQ-PDU => DMPMS_Global_Control.ind(Control_Command:=Global_Control-REQ-PDU.Control_Command, Group_Select:=Global_Control-REQ-PDU.Group_Select)	RUN
28	RUN	DL_DATA.ind(S_SAP_index, D_SAP_index, S_addr, D_addr, DLSDU, Service_class) / D_SAP_index=58 && DLSDU <> Global_Control-REQ-PDU => no action	RUN
29	RUN	DMPMS_RD_Inp_Upd.req(Inp_Data) /Inp_Data.len > 0 => DL_REPLY_UPDATE.req(S_SAP_index:=56, DLSDU:=Inp_Data, Service_Class:=Low, Transmit_strategy:=Multiple, Reference:=Act_cnt)	RUN
30	RUN	DMPMS_RD_Inp_Upd.req(Inp_Data) /Inp_Data.len = 0 => no action	RUN
31	RUN	DL_DATA_REPLY.ind(S_SAP_index, D_SAP_index, S_addr, D_addr, DLSDU, Service_class, Update_status, Reference) /D_SAP_index=56 => no action	RUN
32	RUN	DMPMS_RD_Outp_Upd.req(Outp_Data) /Outp_Data.len > 0 => DL_REPLY_UPDATE.req(S_SAP_index:=57, DLSDU:=Outp_Data, Service_Class:=Low, Transmit_strategy:=Multiple, Reference:=Act_cnt)	RUN
33	RUN	DMPMS_RD_Outp_Upd.req(Outp_Data) /Outp_Data.len = 0 => no action	RUN
34	RUN	DL_DATA_REPLY.ind(S_SAP_index, D_SAP_index, S_addr, D_addr, DLSDU, Service_class, Update_status, Reference) /D_SAP_index=57 => no action	RUN
35	RUN	DMPMS_RSAP_ACTIVATE.req(SSAP, Access, L_sdu_length_list, Indication_Mode) => DLSAP_ACTIVATE_RESPONDER.req(S_SAP_index:=SSAP, Access:=Access, DLSDU_length_list:=L_sdu_length_list, Indication_mode:=Indication_Mode, Publisher_enabled:=FALSE)	DLM-PASS-ACT
36	DLM-PASS-ACT	DLSAP_ACTIVATE_RESPONDER.cnf(SSAP, DLM_status) => DMPMS_RSAP_ACTIVATE.cnf(SSAP:=S_SAP_index, M_Status:=DLM_status)	RUN
37	RUN	DL_DATA_REPLY.ind(S_SAP_index, D_SAP_index, S_addr, D_addr, DLSDU, Service_class, Update_status, Reference) /D_SAP_index<52 => DMPMS_DATA_REPLY.ind(SSAP:=S_SAP_index, DSAP:=D_SAP_index, Rem_Add:=D_addr, L_sdu:=DLSDU, Serv_class:=Service_class, Loc_Add:=S_addr, Update_status:=Update_status)	RUN
38	RUN	DL_DATA_REPLY.ind(S_SAP_index, D_SAP_index, S_addr, D_addr, DLSDU, Service_class, Update_status, Reference) /D_SAP_index=55 && DLSDU=Set_Slave_Add-REQ-PDU => DMPMS_Set_Slave_Add.ind(New_Slave_Add:=Set_Slave_Add-REQ-PDU.New_Slave_Add, Ident_Number:=Set_Slave_Add-REQ-PDU.Ident_Number, No_Add_Chg:=Set_Slave_Add-REQ-PDU.No_Add_Chg, Rem_Slave_Data:=Set_Slave_Add-REQ-PDU.Rem_Slave_Data)	RUN

Table 505 (continued)

#	Current state	Event / condition => action	Next state
39	RUN	DL_DATA_REPLY.ind(S_SAP_index, D_SAP_index, S_addr, D_addr, DLSDU, Service_class, Update_status, Reference) /D_SAP_index=55 && DLSDU<>Set_Slave_Add-REQ-PDU => no action	RUN
40	RUN	DL_DATA_REPLY.ind(S_SAP_index, D_SAP_index, S_addr, D_addr, DLSDU, Service_class, Update_status, Reference) /D_SAP_index=62 => DMPMS_Chk_Cfg.ind(Req_Add:=S_addr, Cfg_Data:=DLSDU)	RUN
41	RUN	DMPMS_Set_minTsdr.req(minTsdr) /MinTsdr = 0 => no action	RUN
42	RUN	DMPMS_Set_minTsdr.req(minTsdr) /minTsdr > 0 => DLM_SET_VALUE.req(Variable_name1:="minTSDR", Desired_value1:=minTsdr)	SMINTSD R
43	SMINTSD R	DLM_SET_VALUE.cnf(DLM_status) /DLM_Status=OK => no action	RUN
44	RUN	DL_DATA_REPLY.ind(S_SAP_index, D_SAP_index, S_addr, D_addr, DLSDU, Service_class, Update_status, Reference) /D_SAP_index=61 => DMPMS_Set_Prm.ind(Req_Add:=S_addr, Prm_Data:=DLSDU)	RUN
45	RUN	DMPMS_SAP_DEACTIVATE.req(SSAP) => DLSAP_DEACTIVATE.req(S_SAP_index:=SSAP)	DLM-PASS-DEACT
46	DLM-PASS-DEACT	DLSAP_DEACTIVATE.cnf(S_SAP_index, DLM_status) => DMPMS_SAP_DEACTIVATE.cnf(SSAP:=S_SAP_index, M_Status:=DLM_status)	RUN
47	RUN	DMPMS_REPLY_UPDATE.req(SSAP, L_sdu, Serv_class, Transmit) /SSAP<52 => DL_REPLY_UPDATE.req(S_SAP_index:=SSAP, DLSDU:=L_sdu, Service_class:=Serv_class, Transmit_strategy:=Transmit, Reference:=Act_cnt)	RUN
48	RUN	DL_REPLY_UPDATE.cnf(S_SAP_index, Service_class, DL_status) /S_SAP_index<52 => DMPMS_REPLY_UPDATE.cnf(SSAP:=S_SAP_index, Serv_class:=Service_class, L_status:=DL_status)	RUN
49	any state	DMPMS_Reset.req => DLM_RESET.req	WAIT-RESET
50	WAIT-RESET	DLM_RESET.cnf(DLM_status) /DLM_status=OK => DMPMS_Reset.cnf	PON
51	RUN	DL_REPLY_UPDATE.cnf(S_SAP_index, Service_class, DL_status) /(S_SAP_index=NIL S_SAP_index=62 S_SAP_index=61 S_SAP_index=60 S_SAP_index=59 S_SAP_index=57 S_SAP_index=56) && DL_status=OK => no action	RUN
52	any state	DL_CS_CLOCK_VALUE.ind(D_addr, D_SAP_index, S_addr, S_SAP_index, CS_list, CS_Status, Receive_delay_time) => CS_CLOCK_VALUE.ind(Time_master_addr := S_addr, CS_list, CS_Status, Receive Delay Time := Receive_delay_time)	SAME
53	any state	DLM_SET_VALUE.cnf(DLM_status) /DLM_status=NO/IV => DMPMS_Fault.ind	PON

Table 505 (continued)

#	Current state	Event / condition => action	Next state
54	any state	DLSAP_ACTIVATE_RESPONDER.cnf(SSAP, DLM_status) /DLM_status=NO/IV => DMPMS_Fault.ind	PON
55	any state	DLSAP_ACTIVATE.cnf(S_SAP_index, DLM_status) /DLM_status=NO/IV => DMPMS_Fault.ind	PON
56	any state	DLSAP_DEACTIVATE.cnf(S_SAP_index, DLM_status) /DLM_status=NO/IV => DMPMS_Fault.ind	PON
57	any state	DL_REPLY_UPDATE.cnf(S_SAP_index, Service_class, DL_status) /DL_status=LS/LR/IV => DMPMS_Fault.ind	PON
58	any state	unexpected DL-primitive /D_SAP_index>54 => DMPMS_Fault.ind	PON
59	any state	unknown DL-primitive /D_SAP_index>54 => DMPMS_Fault.ind	PON
60	any state	unexpected DLM-primitive => DMPMS_Fault.ind	PON
61	any state	unknown DLM-primitive => DMPMS_Fault.ind	PON
62	RUN	DL_DXM_REPLY.ind(S_SAP_index, D_SAP_index, S_addr, D_addr, DLSDU) /IsFilterEntryExistent(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class) =TRUE => Rem_Add:=Publisher Address Data:=DLSDU DMPMS_DX_Broadcast.ind (Rem_Add, Data)	RUN
63	RUN	DL_DXM_REPLY.ind(S_SAP_index, D_SAP_index, S_addr, D_addr, DLSDU) /IsFilterEntryExistent(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class) =FALSE => no action	RUN
64	RUN	DL_DATA_REPLY.ind(S_SAP_index, D_SAP_index, S_addr, D_addr, DLSDU, Service_class, Update_status, Reference) /D_SAP_index=53 => DMPMS_Set_Ext_Prm.ind(Req_Add:=S_add, Prm_Data:=DLSDU)	RUN
65	DLM-ACT-SAP	DLSAP_ACTIVATE.cnf(S_SAP_index, DLM_status) /DLM_status=OK && EnableSubscriber() = TRUE => DLSAP_ACTIVATE_SUBSCRIBER.req(S_SAP_index:=NIL, DLSDU_length_list:=(244,244))	DLM-ACT-SUB
66	DLM-ACT-SAP	DLSAP_ACTIVATE_SUBSCRIBER.cnf(S_SAP_index, DLM_status) /DLM_status=OK => Prev_Diag_Flag := False	RUN

6.7.1.4 Functions

Table 506 contains the functions used by the DMPMS, their arguments and their descriptions.

Table 506 – Functions used by the DMPMS

Function name	Description
DeactivateSAPFromCRLList (DMPMS_DeactControl)	This function checks the CRL DP-slave if there are CRL entries that belongs to MS0-AR and which SSAP has not been deactivated with the DLM-SAP-DEACTIVATE.req service primitive yet. A) If DMPMS_DeactControl=DMPMS_Deact and if there exist one of the following CRL entries that has not been deactivated yet: Entries with DSAP=NIL or DSAP=58 or DSAP=57 or DSAP=56 or DSAP=62 or DSAP=61 or DSAP=60 or DSAP=59 or DSAP=55 then it returns TRUE and sets the local context according to the current CREP. B) If DMPMS_DeactControl=DMPMS_Leave and if there exist one of the following CRL entries that has not been deactivated yet: Entries with DSAP=NIL or DSAP=58 or DSAP=57 or DSAP=56 then it returns TRUE and sets the local context according to one not activated CREP. C) Otherwise it returns FALSE.
ActivateRSAPFromCRLList (DMPMS_ActivateControl)	This function checks the CRL DP-slave if there are CRL entries that belongs to MS0-AR and which SSAP has not been activated with the DLM-RSAP-ACTIVATE.req service primitive yet. A) If DMPMS_ActivateControl=DMPMS_Init and if there exist one of the following CRL entries that has not been activated yet: Entries with DSAP=62 or DSAP=61 or DSAP=60 or DSAP=59 or DSAP=55 then it returns TRUE and sets the local context according to the current CREP. B) If DMPMS_ActivateControl=DMPMS_Enter and if there exist one of the following CRL entries that has not been activated yet: Entries with DSAP=NIL or DSAP=58 or DSAP=57 or DSAP=56 then it returns TRUE and sets the local context according to one not activated the current CREP. C) Otherwise it returns FALSE.
BuildRSAPServiceListFromCR()	This function builds the Service_list parameter for DL-RSAP-ACTIVATE.req primitive from the current CRL context. It returns DLSDU_length_list.Max_DLSDU_length_req_low = CREP.Max L_sdu Length Req Low DLSDU_length_list.Max_DLSDU_length_req_high = CREP.Max L_sdu Length Req High DLSDU_length_list.Max_DLSDU_length_ind/cnf_low = CREP.Max L_sdu Length Ind Low DLSDU_length_list.Max_DLSDU_length_ind/cnf_high = CREP.Max L_sdu Length Ind High
IsFilterEntryExistent(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class)	This function checks the CRL DP-slave if there are CRL entries that belongs to MS0-AR which contain DXB-Link entries . A) If (S_SAP_Index && D_SAP_Index)=NIL and (Service_class=R-Brct) and if there exist one entry with (DLSDU = Input Data Length Publisher && S_addr =Publisher Address): then it returns TRUE and sets the local context according to the current CREP. B) Otherwise it returns FALSE.
BuildServiceListFromCR()	This function builds the Service_list parameter for DLSAP-ACTIVATE.req primitive from the current CRL context. A) If Enable Publisher = TRUE && EnableSubscriber() =FALSE it returns Service_list.Service_activate[1] = SRD(R-Brct) Service_list.Role_in_Service[1] = Initiator Service_list.DLSDU_length_list[1].Max_DLSDU_length_req_low=0 Service_list.DLSDU_length_list[1].Max_DLSDU_length_req_high=0 Service_list.DLSDU_length_list[1].Max_DLSDU_length_ind/cnf_low=0 Service_list.DLSDU_length_list[1].Max_DLSDU_length_ind/cnf_high=0 B) If Enable Publisher = FALSE && EnableSubscriber() =TRUE it returns Service_list.Service_activate[1] = SRD(R-Brct) Service_list.Role_in_Service[1] = Responder Service_list.DLSDU_length_list[1].Max_DLSDU_length_req_low=0 Service_list.DLSDU_length_list[1].Max_DLSDU_length_req_high=0 Service_list.DLSDU_length_list[1].Max_DLSDU_length_ind/cnf_low=244 Service_list.DLSDU_length_list[1].Max_DLSDU_length_ind/cnf_high=244 C) If Enable Publisher = TRUE && EnableSubscriber() =TRUE it returns Service_list.Service_activate[1] = SRD(R-Brct) Service_list.Role_in_Service[1] = Both Service_list.DLSDU_length_list[1].Max_DLSDU_length_req_low=0 Service_list.DLSDU_length_list[1].Max_DLSDU_length_req_high=0 Service_list.DLSDU_length_list[1].Max_DLSDU_length_ind/cnf_low=244 Service_list.DLSDU_length_list[1].Max_DLSDU_length_ind/cnf_high=244
EnableSubscriber()	This function checks the CRL DP-slave if there are CRL entries that belongs to MS0-AR which contain DXB-Link entries . A) If there exist at least one entry: then it returns TRUE and sets the local context according to the current CREP. B) Otherwise it returns FALSE.

6.7.2 DMPMM1

6.7.2.1 Primitive definitions

6.7.2.1.1 Primitives exchanged between FSPMM1 and DMPMM1

Table 507 shows the service primitives including their associated parameters issued by the FSPMM1 and received by the DMPMM1.

Table 507 – Primitives issued by FSPMM1 to DMPMM1

Primitive name	Source	Associated parameters	Functions
MInit DLL.req	FSPMM1	Bus Para	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Reset.req	FSPMM1	(none)	
Global Control.req	FSPMM1	Control Command, Group Select	
Set Bus Par.req	FSPMM1	Bus Para	
Delete SC.req	FSPMM1	Address	
Read Value.req	FSPMM1	Variable	

Table 508 shows the service primitives including their associated parameters issued by the DMPMM1 and received by the FSPMM1.

Table 508 – Primitives issued by DMPMM1 to FSPMM1

Primitive name	Source	Associated parameters	Functions
MInit DLL.cnf	DMPMM1	(none)	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Reset.cnf	DMPMM1	(none)	
Global Control.cnf(+)	DMPMM1	(none)	
Global Control.cnf(-)	DMPMM1	Status	
Set Bus Par.cnf	DMPMM1	Status	
Delete SC.cnf	DMPMM1	Status	
Read Value.cnf	DMPMM1	Value, Status	
Fault.ind	DMPMM1	(none)	
Event.ind	DMPMM1	Event, Add Info	

6.7.2.1.2 Primitives exchanged between MSCY1M and DMPMM1

Table 509 shows the service primitives including their associated parameters issued by the MSCY1M and received by the DMPMM1.

Table 509 – Primitives issued by MSCY1M to DMPMM1

Primitive name	Source	Associated parameters	Functions
Slave Diag.req	MSCY1M	Rem Add	Fetch Diagnosis of a DP-slave
Set Prm.req	MSCY1M	Rem Add, Prm Data	Parameter data are delivered to the DP-slave. The parameterization is first done in the start-up phase and also while the DP-slave is in the Data Exchange mode.
Set ExtPrm.req	MSCY1M	Rem Add, Prm Data	Parameter data are delivered to the DP-slave. The parameterization is first done in the start-up phase and also while the DP-slave is in the Data Exchange mode.
Chk Cfg.req	MSCY1M	Rem Add, Cfg Data	Send configuration data to the DP-slave for checking. They contain the format of input and output areas as well as the information about the data consistency.
Data Exchange.req	MSCY1M	Rem Add, Outp Data	Transmits output data to a DP-slave and requests input data.

Table 510 shows the service primitives including their associated parameters issued by the DMPMM1 and received by the MSCY1M.

Table 510 – Primitives issued by DMPMM1 to MSCY1M

Primitive name	Source	Associated parameters	Functions
Slave Diag.cnf(+)	DMPMS	Rem Add, Diag Data	Get Diagnosis from a DP-slave.
Slave Diag.cnf(-)	DMPMS	Rem Add, Status	This primitive is used to convey a Slave Diag negative confirmation to DMPMM..
Set Prm.cnf(+)	DMPMS	Rem Add	Parametrisation successfully transmitted, the parameters are not checked.
Set Prm.cnf(-)	DMPMS	Rem Add, Status	This primitive is used to convey a Set Prm (Parameter Setting) negative confirmation to DMPMM.
Set ExtPrm.cnf(+)	DMPMS	Rem Add	Parametrisation successfully transmitted, the parameters are not checked.
Set ExtPrm.cnf(-)	DMPMS	Rem Add, Status	This primitive is used to convey a Set ExtPrm (Parameter Setting) negative confirmation to DMPMM.
Chk Cfg.cnf(+)	DMPMS	Rem Add	Configuration data successfully transmitted, the parameters are not checked.
Chk Cfg.cnf(-)	DMPMS	Rem Add, Status	This primitive is used to convey a Chk Cfg (configuration Check) negative confirmation to DMPMM.
Data Exchange.cnf(+)	DMPMS	Rem Add, Diag Flag, Inp Data	Get Input Data and Diag lag from a DP-slave
Data Exchange.cnf(-)	DMPMS	Rem Add, Status	This primitive is used to convey a Data Exchange negative confirmation to DMPMS.

6.7.2.1.3 Primitives exchanged between DMPMM1 and MSAL1M, MSAC1M

Table 511 shows the service primitives including their associated parameters issued by MSAL1M and MSAC1M and received by the DMPMM1.

Table 511 – Primitives issued by MSAL1M, MSAC1M to DMPMM1

Primitive name	Source	Associated parameters	Functions
DATA_REPLY.req	MSAL1M, MSAC1M	SSAP, DSAP, Rem_Add, L_sdu, Serv_class	Send Service

Table 512 shows the service primitives including their associated parameters issued by the DMPMM1 and received by the MSAL1M and MSAC1M.

Table 512 – Primitives issued by DMPMM1 to MSAL1M, MSAC1M

Primitive name	Source	Associated parameters	Functions
DATA_REPLY.cnf	DMPMM1	SSAP, DSAP, Rem_Add, L_sdu, Serv_class, L_status	Send Service executed

6.7.2.1.4 Primitives exchanged between DMPMM1 and MMAC1

Table 513 shows the service primitives including their associated parameters issued by MMAC1 and received by the DMPMM1.

Table 513 – Primitives issued by MMAC1 to DMPMM1

Primitive name	Source	Associated parameters	Functions
RSAP_ACTIVATE.req	MMAC1	SSAP, Access, L_sdu_length_list, Indication_Mode	Activates local SAP
REPLY_UPDATE.req	MMAC1	SSAP, L_sdu, Serv_class, Transmit	Write Data in the Update Buffer for Transmission

Table 514 shows the service primitives including their associated parameters issued by the DMPMM1 and received by the MMAC1.

Table 514 – Primitives issued by DMPMM1 to MMAC1

Primitive name	Source	Associated parameters	Functions
RSAP_ACTIVATE.cnf	DMPMM1	SSAP, M_Status	Activation finished
REPLY_UPDATE.cnf	DMPMM1	SSAP, Serv_class, L_status	Update finished
DATA.ind	DMPMM1	SSAP, DSAP, Rem_Add, L_sdu, Serv_class,	Data/Request received
DATA_REPLY.ind	DMPMM1	SSAP, DSAP, Rem_Add, L_sdu, Serv_class, Update_status	Data/Request received

6.7.2.1.5 Primitives exchanged between DMPMM1 and MSCS1M

Table 515 shows the service primitives including their associated parameters issued by MSCS1M and received by the DMPMM1.

Table 515 – Primitives issued by MSCS1M to DMPMM1

Primitive name	Source	Associated parameters	Functions
CS_TIME_EVENT.req	FSPMM1	(none)	Refer to DL Service Definition Type 3 Fieldbus IEC 61158-3
CS_CLOCK_VALUE.req	FSPMM1	CS_list	

Table 516 shows the service primitives including their associated parameters issued by the DMPMM1 and received by the MSCS1M.

Table 516 – Primitives issued by DMPMM1 to MSCS1M

Primitive name	Source	Associated parameters	Functions
CS_CLOCK_VALUE.ind	DMPMM1	Time_master_addr CS_list Receive Delay Time CS_status	Refer to DL Service Definition Type 3 Fieldbus IEC 61158-3
CS_TIME_EVENT.cnf	DMPMM1	Send_delay_time, Status	
CS_CLOCK_VALUE.cnf	DMPMM1	Status	

6.7.2.1.6 Primitives exchanged between DMPMM1 and DL

Table 517 shows the service primitives including their associated parameters issued by the DMPMM1 and received by the DL.

Table 517 – Primitives issued by DMPMM1 to DL

Primitive name	Source	Associated parameters	Functions
DL-DATA.req	DMPMM1	Service_class, D_addr, D_SAP_index, S_SAP_index, DLSDU	Refer to DL Service Definition Type 3 Fieldbus IEC 61158-3
DL-DATA-REPLY.req	DMPMM1	Service_class, D_addr, D_SAP_index, S_SAP_index, DLSDU	
DL-REPLY-UPDATE.req	DMPMM1	Service_class, S_SAP_index, DLSDU, Transmit_strategy	
DLM-RESET.req	DMPMM1	(none)	
DLM-SET-VALUE.req	DMPMM1	Variable_name(1 to n), Desired_value (1 to n)	
DLM-GET-VALUE.req	DMPMM1	Variable_name(1 to n)	
DLSAP-ACTIVATE.req	DMPMM1	S_SAP_index, Access, Service_list	
DLSAP-ACTIVATE-RESPONDER.req	DMPMM1	S_SAP_index, Access, DLSDU_length_list, Indication_mode, Publisher_enabled	
DLSAP-DEACTIVATE.req	DMPMM1	S_SAP_index	
DL-CS_TIME_EVENT.req	DMPMM1	D_addr, D_SAP_index, S_SAP_index	
DL-CS_CLOCK_VALUE.req	DMPMM1	D_addr, D_SAP_index, S_SAP_index, DLSDU	

Table 518 shows the service primitives including their associated parameters issued by the DL and received by the DMPMM1.

Table 518 – Primitives issued by DL to DMPMM1

Primitive name	Source	Associated parameters	Functions
DL-DATA.cnf	DL	Service_class, D_addr, D_SAP_index, S_SAP_index, DL_status	Refer to DL Service Definition Type 3 Fieldbus IEC 61158-3
DL-DATA-REPLY.cnf	DL	Service_class, D_addr, D_SAP_index, S_SAP_index, DLSDU, DL_status	
DL-DATA.ind	DL	Service_class, D_addr, D_SAP_index, S_addr, S_SAP_index, DLSDU	
DL-DATA-REPLY.ind	DL	Service_class, D_addr, D_SAP_index, S_addr, S_SAP_index, DLSDU, Update_Status	
DL-REPLY-UPDATE.cnf	DL	Service_class, S_SAP_index, DL_status	
DLM-RESET.cnf	DL	DLM_Status	
DLM-SET-VALUE.cnf	DL	Current_value (1 to n), DLM_status	
DLM-GET-VALUE.cnf	DL	Current_value (1 to n), DLM_status (1 to n)	
DLM-EVENT.ind	DL	Event/Fault	
DLSAP-ACTIVATE.cnf	DL	S_SAP_index, DLM_status	
DLSAP-ACTIVATE-RESPONDER.cnf	DL	S_SAP_index, DLM_status	
DLSAP-DEACTIVATE.cnf	DL	S_SAP_index, DLM_status	
DL-CS_CLOCK_VALUE.ind	DL	D_addr, D_SAP_index, S_addr, S_SAP_index, DLSDU, Receive_delay_time, CS_Status	
DL-CS_TIME_EVENT.cnf	DL	Send_delay_time, Status	
DL-CS_CLOCK_VALUE.cnf	DL	Status	

6.7.2.1.7 Parameters of DMPMM1 primitives

The parameters used with the primitives exchanged between DMPMM1 and the other state machines of the Master (Class 1) AL are described in Table 519.

Table 519 – Parameters used with primitives exchanged with DMPMM1

Parameter name	Description
Input Data Len	This parameter is used to setup the length of Input Data for this Slave.
Output Data Len	This parameter is used to setup the length of Output Data for this Slave.
Min Tsdr	This parameter conveys the value of the DLL variable minTsdr.
Rem Add	This parameter conveys the DL address of the receiver of this service.
Diag Data	This parameter conveys the Diagnosis-RES-PDU
Diag Flag	This parameter indicates that a DP-slave has new Diag Data
Prm Data	This parameter conveys the Set_Prm-REQ-PDU.
Cfg Data	This parameter conveys the Chk_Cfg-REQ-PDU
Inp Data	This parameter conveys the DataExchange-RES-PDU
Outp Data	This parameter conveys the DataExchange-REQ-PDU
Status	This parameter contains the cause of a negative confirmation-
SSAP	Local Identifier for Service Access Point
DSAP	Identifier for the remote Service Access Point
Access	Defines the use of a SAP for one or several Remote Address (Stations)
L_sdu_length_list	Defines the maximum length of incoming or outgoing L_sdu
Indication_Mode	Defines the Mode for Indication of poll-sequences without user data
Serv_class	Indicates priority of the service
M_status	Status of the service execution
L_status	Status of the service execution
L_sdu	Defines the DP PDU to be transmitted
Transmit	Defines the operation mode of the update buffer (single = buffer is transmitted once)
Rem_Add	DL Address of the remote station
Update_status	Indicates the transmission of an update buffer
Time_master_addr	DL address of the active Time Master
CS_list	List of Time Value at the transmission of this service (Clock_value_time_event), Time Value at the previous transmission (Clock_value_previous_TE) and status information related to the clock value (Clock_value_status)
CS_status	Indicates the success or failure of the Clock Synchronisation sequence
Receive_delay_time	Time expired between receipt of the Time_Event and the invocation of that service primitive

Table 520 shows the possible values of the parameter Status and the corresponding source entities.

Table 520 – Possible values of status

Value	Meaning	Source
DS	Local-DLL/PHY Entity is not in logical token-ring or disconnected from line	Local DLL
NA	Negative ack, no reaction from remote station	Local DLL
NR	No Response data	Remote DLL
OK	Acknowledgement positive	Local DLL Remote DLL
RE	Format-Error in a Response-DLPDU	Local DMPM
RR	Resources of the remote-DLL Entity not sufficient or not available	Remote DLL
RS	Service or remote-address at remote-LSAP or remote-LSAP not activated; remote-station is no DP-Station remote-station is not yet ready for these functions remote-station is associated with an other Requester optional service not available	Remote DLL
UE	Remote-MMAC/DLL interface error	Remote DLL

6.7.2.2 State machine description

The DMPMM1 makes the connection between the MS0,MS1,MM1,MM2 specific parts and Layer 2. Services are mapped by the DMPMM1 to the DL and management of Layer 2. The DMPMM1 delivers the necessary Layer 2 parameters of the function call (SSAP, DSAP, Serv_class, ...) for Layer 2, receives the confirmations and indications from Layer 2 and passes them to the MS0,MS1,MM1/2-Handler.

Local Variables

Act_Max_Diag_Len

(Unsigned8)

Contains the maximal length of the diagnostic information which can be stored locally.

Range: 6 .. 244

Loc_Station_Address

(Unsigned8)

Local variable for the intermediate storage of the own station address which has been transferred with the last Set_Bus_Par.

Range: 0 .. 125

Srv_lst

This list is used to store all information which are necessary for the service SAP_ACTIVATE.

Act_msac1m_Max_L_sdu_len

(Unsigned8)

This variable is used to store the actual value of the parameter Max_L_sdu_len locally.

Range: 0 .. 240

6.7.2.3 DMPMM1 state table

Table 521 contains the complete description of the DMPMM1 state machine.

Table 521 – DMPMM1 state table

#	Current State	Event /Condition =>Action	Next State
1	PON	DMPMM1_Minit_DLL.req(Bus_Para) => if (Isochronous Mode = Not Synchronized) IsoM On:=FALSE DLSDU:= NIL-SDU else IsoM On:=TRUE DL IsoM On:=FALSE DLSDU:=Global_Control-REQ-PDU(Control_Command:= 2, Group_Select:=Group_8) endif DLM_SET_VALUE.req(MSAP_0 , Variable_name1:= "TS ", Variable_name2:= "Baud_rate ", Variable_name3:= "TSL ", Variable_name4:= "min TSDR ", Variable_name5:= "max TSDR ", Variable_name6:= "TQUI ", Variable_name7:= "TSET ", Variable_name8:= "TTR ", Variable_name9:= "G ", Variable_name10:= "HSA ", Variable_name11:= "max_retry_limit ", Variable_name12:= "in_ring_desired", Variable_name13:= "SYNCHT", Variable_name14:= "maxTsh", Variable_name15:= "Tct", Desired_Value1:= Bus_Para.TS, Desired_Value2:= Bus_Para.Baud_rate, Desired_Value3:= Bus_Para.TSL, Desired_Value4:= Bus_Para.min TSDR, Desired_Value5:= Bus_Para.max TSDR, Desired_Value6:= Bus_Para.TQUI, Desired_Value7:= Bus_Para.TSET, Desired_Value8:= Bus_Para.TTR, Desired_Value9:= Bus_Para.G, Desired_Value10:= Bus_Para.HSA, Desired_Value11:= Bus_Para.max_retry_limit, Desired_Value12:= TRUE, Desired_Value13:= DLSDU, Desired_Value14:= maxTsh, Desired_Value15:= Tct)	SET-VALUE
2	PON	DLM_RESET.cnf(DLM_status) /DLM_status=OK => DMPMM1_Reset.cnf	PON
3	SET-VALUE	DLM_SET_VALUE.cnf(DLM_status(1-15)) /DLM_status(1-15)=OK => DLSAP_ACTIVATE.req(S_SAP_index:=SSAP, Access:=All, Service_list:=BuildServiceListFromCR())	SAP-ACT
4	SAP-ACT	DLSAP_ACTIVATE.cnf(S_SAP_index, DLM_status) /DLM_status=OK && ActivateSAPFromCRList()==TRUE => DLSAP_ACTIVATE.req(S_SAP_index:=SSAP, Access:=All, Service_list:=BuildServiceListFromCR())	SAP-ACT
5	SAP-ACT	DLSAP_ACTIVATE.cnf(S_SAP_index, DLM_status) /DLM_status=OK && ActivateSAPFromCRList()==FALSE => DLSAP_ACTIVATE_RESPONDER.req(S_SAP_index:=54, Access:=All, DLSDU_length_list:=BuildLengthListFromCR())	RSAP-ACT
6	RSAP-ACT	DLSAP_ACTIVATE.cnf(S_SAP_index, DLM_status) /DLM_status=OK => DMPMM2_Minit_DLL.cnf	RUN

Table 521 (continued)

#	Current State	Event /Condition =>Action	Next State
7	RUN	DMPMM1_Set_Bus_Par.req(Bus_Para) => if (Isochron Mode = Not Synchronized) IsoM On:=FALSE DLSDU:= NIL-SDU else IsoM On:=TRUE DL IsoM On:=FALSE DLSDU:=Global_Control-REQ-PDU(Control_Command:= 2, Group_Select:=Group_8) endif DLM_SET_VALUE.req(MSAP_0 , Variable_name1:= "TS ", Variable_name2:= "Baud_rate ", Variable_name3:= "TSL ", Variable_name4:= "min TSDR ", Variable_name5:= "max TSDR ", Variable_name6:= "TQUI ", Variable_name7:= "TSET ", Variable_name8:= "TTR ", Variable_name9:= "G ", Variable_name10:= "HSA ", Variable_name11:= "max_retry_limit ", Variable_name12:= "in_ring_desired", Variable_name13:= "SYNCHT", Variable_name14:= "maxTsh", Variable_name15:= "Tct", Desired_Value1:= Bus_Para.TS, Desired_Value2:= Bus_Para.Baud_rate, Desired_Value3:= Bus_Para.TSL, Desired_Value4:= Bus_Para.min TSDR, Desired_Value5:= Bus_Para.max TSDR, Desired_Value6:= Bus_Para.TQUI, Desired_Value7:= Bus_Para.TSET, Desired_Value8:= Bus_Para.TTR, Desired_Value9:= Bus_Para.G, Desired_Value10:= Bus_Para.HSA, Desired_Value11:= Bus_Para.max_retry_limit, Desired_Value12:= TRUE, Desired_Value13:= DLSDU, Desired_Value14:= maxTsh, Desired_Value15:= Tct)	SV1
8	SV1	DLM_SET_VALUE.cnf(DLM_status(1-15)) /DLM_status(1-15)=OK => DMPMM1_Set_Bus_Par.cnf(+)	RUN
9	RUN	DMPMM1_Delete_SC.req(Address) => DLM_SET_VALUE.req(Variable_name1:="DLPDU_Sent_Count", Index1:=Address, Variable_name2:="Error_Count", Index2:=Address, Desired_Value1:=0, Desired_Value2:=0)	SV2
10	SV2	DLM_SET_VALUE.cnf(DLM_status(1-2)) /DLM_status(1-2)=OK => DMPMM1_Delete_SC.cnf(+)	RUN
11	RUN	DMPMM1_Read_Value.req(Variable) => DLM_GET_VALUE.req(Variable_name1:=Variable)	RUN
12	RUN	DLM_GET_VALUE.cnf(Current_value(1), DLM_status(1)) /DLM_status(1)=OK/NO => DMPMM1_Read_Value.cnf(Status:=DLM_status(1),Value:=Current_value(1))	RUN
13	RUN	DLM_EVENT.ind(Event/Fault, Tsh)/Event/Fault<>Synch_Delay &&Event/Fault<>Synch=>DMPMM1_Event.ind(Event:=Event/Fault)	RUN

Table 521 (continued)

#	Current State	Event /Condition =>Action	Next State
14	RUN	DLM_EVENT.ind(Event/Fault, Tsh) /Event/Fault=Synch_Delay => DMPMM1_SYNCH_Delayed.ind(TSH:=Tsh)	RUN
15	RUN	DLM_EVENT.ind(Event/Fault, Tsh) /Event/Fault=Synch => DMPMM1_SYNCH.ind	RUN
16	RUN	DMPMM1_RSAP_ACTIVATE.req(SSAP, Access, L_sdu_length_list, Indication_Mode) => DLSAP_ACTIVATE_RESPONDER.req(S_SAP_index:=SSAP, Access:=Access, DLSDU_length_list:= L_sdu_length_list, Indication_mode:= Indication_Mode)	RUN
17	RUN	DLSAP_ACTIVATE_RESPONDER.cnf(SSAP, DLM_status) /SSAP = 54 && DLM_status=OK => DMPMM1_RSAP_ACTIVATE.cnf(SSAP:=SSAP,DLM_status)	RUN
18	RUN	DMPMM1_REPLY_UPDATE.req(SSAP, L_sdu, Serv_class, Transmit) => DL_REPLY_UPDATE.req(S_SAP_index:=SSAP, DLSDU:=L_sdu, Service_class:=Serv_class, Transmit_strategy:=Transmit)	RUN
19	RUN	DL_REPLY_UPDATE.cnf(S_SAP_index, Service_class, DL_status) => DMPMM1_REPLY_UPDATE.cnf (SSAP:=S_SAP_index, Serv_class:=Service_class, L_status:=DL_status)	RUN
20	RUN	DMPMM1_DATA_REPLY.req (SSAP, DSAP, Rem_Add, L_sdu, Serv_class) => DL_DATA_REPLY.req (S_SAP_index:=SSAP, D_SAP_index:=DSAP, S_addr:=Rem_Add, DLSDU:=L_sdu, Service_class:=Serv_class)	RUN
21	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index<>62 && S_SAP_index<>NIL && DL_status<>LS/IV => DMPMM1_DATA_REPLY.cnf (SSAP:=S_SAP_index, DSAP:=D_SAP_index, Rem_Add:=S_addr, L_sdu:=DLSDU, Serv_class:=Service_class, L_status:=DL_status)	RUN
22	RUN	DL_DATA.ind(S_SAP_index, D_SAP_index, D_addr, S_Addr, DLSDU, Service_class) => DMPMM1_DATA.ind (SSAP:=S_SAP_index, DSAP:=D_SAP_index, Rem_Add:=S_addr, L_sdu:=DLSDU, Serv_class:=Service_class)	RUN
23	RUN	DL_DATA_REPLY.ind(S_SAP_index, D_SAP_index, S_addr, D_addr, DLSDU, Service_class, Update_status) => DMPMM1_DATA_REPLY.ind(SSAP:=S_SAP_index, DSAP:=D_SAP_index, Rem_Add:=S_addr, L_sdu:=DLSDU, Serv_class:=Service_class, Update_status:=Update_status)	RUN
24	RUN	DMPMM1_Data_Exchange.req(Rem_Add, Outp_Data) /GetCRAAttribute(PublisherFlag) = TRUE => DL_MCT_DATA_REPLY.req(S_SAP_index:=NIL, D_SAP_index:=NIL, D_addr:=Rem_Add, DLSDU:=Outp_Data, Service_class:=High)	RUN
25	RUN	DMPMM1_Data_Exchange.req(Rem_Add, Outp_Data) /GetCRAAttribute(PublisherFlag)=FALSE => DL_DATA_REPLY.req(S_SAP_index:=NIL, D_SAP_index:=NIL, D_addr:=Rem_Add,DLSDU:=Outp_Data, Service_class:=High)	RUN
26	RUN	DL_MCT_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index=NIL && D_SAP_index=NIL && DL_status=NR => DMPMM1_Data_Exchange.cnf(+)(Rem_Add:=S_addr, Diag_Flag:=False, Inp_Data.len:=0)	RUN

Table 521 (continued)

#	Current State	Event /Condition =>Action	Next State
27	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index=NIL && D_SAP_index=NIL && DL_status=NR => DMPMM1_Data_Exchange.cnf(+)(Rem_Add:=S_addr, Diag_Flag:=False, Inp_Data.len:=0)	RUN
28	RUN	DL_MCT_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index=NIL && D_SAP_index=NIL && DL_status=DL => DMPMM1_Data_Exchange.cnf(+)(Rem_Add:=S_addr, Diag_Flag:=False, Inp_Data:=DLSDU)	RUN
29	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index=NIL && D_SAP_index=NIL && DL_status=DL => DMPMM1_Data_Exchange.cnf(+)(Rem_Add:=S_addr, Diag_Flag:=False, Inp_Data:=DLSDU)	RUN
30	RUN	DL_MCT_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index=NIL && D_SAP_index=NIL && DL_status=DH => DMPMM1_Data_Exchange.cnf(+)(Rem_Add:=S_addr, Diag_Flag:=True, Inp_Data:=DLSDU)	RUN
31	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index=NIL && D_SAP_index=NIL && DL_status=DH => DMPMM1_Data_Exchange.cnf(+)(Rem_Add:=S_addr, Diag_Flag:=True, Inp_Data:=DLSDU)	RUN
32	RUN	DL_MCT_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index=NIL && D_SAP_index=NIL && DL_status=DS/UE/RS/NA => DMPMM1_Data_Exchange.cnf(-)(Rem_Add:=S_addr, Status:=DL_status)	RUN
33	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index=NIL && D_SAP_index=NIL && DL_status=DS/UE/RS/NA => DMPMM1_Data_Exchange.cnf(-)(Rem_Add:=S_addr, Status:=DL_status)	RUN
34	RUN	DL_MCT_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index=NIL && D_SAP_index=NIL && DL_status=RR/RDL/RDH => DMPMM1_Data_Exchange.cnf(-)(Rem_Add:=S_addr, Status:=RR)	RUN
35	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index=NIL && D_SAP_index=NIL && DL_status=RR/RDL/RDH => DMPMM1_Data_Exchange.cnf(-)(Rem_Add:=S_addr, Status:=RR)	RUN
36	RUN	DMPMM1_Slave_Diag.req(Rem_Add) => DL_DATA_REPLY.req(S_SAP_index:=62, D_SAP_index:=60, S_addr:=Rem_Add, DLSDU:=ZERO-PDU, Service_class:=High)	RUN
37	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index=62 && D_SAP_index=60 && DL_status=DL && DLSDU.len >= 6 && DLSDU.len <= Act_Max_Diag_Len && (Diagnosis-RES-PDU.Diag_Master_Add=255 Diagnosis-RES-PDU.Diag_Master_Add=Loc_Station_Address) => DMPMM1_Slave_Diag.cnf(+)(Rem_Add:=S_addr, Diag_Data:=DLSDU)	RUN

Table 521 (continued)

#	Current State	Event /Condition =>Action	Next State
38	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index=62 && D_SAP_index=60 && DL_status=DL && DLSDU.len > Act_Max_Diag_Len && (Diagnosis-RES-PDU.Diag_Master_Add<>255 && Diagnosis-RES-PDU.Diag_Master_Add<>Loc_Station_Address) => cut Diagnosis-RES-PDU until Act_Max_Diag_Len Diagnosis-RES-PDU.Station_status_3.Ext_Diag_Overflow:=1 Diagnosis-RES-PDU.Station_status_1.Master_Lock := 1 DMPMM1_Slave_Diag.cnf(+)(Rem_Add:=S_addr, Diag_Data:=Diagnosis-RES-PDU)	RUN
39	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index=62 && D_SAP_index=60 && DL_status=DL && DLSDU.len > Act_Max_Diag_Len && (Diagnosis-RES-PDU.Diag_Master_Add=255 Diagnosis-RES-PDU.Diag_Master_Add=Loc_Station_Address) => cut Diagnosis-RES-PDU until Act_Max_Diag_Len Diagnosis-RES-PDU.Station_status_3.Ext_Diag_Overflow:=1 DMPMM1_Slave_Diag.cnf(+)(Rem_Add:=S_addr, Diag_Data:=Diagnosis-RES-PDU)	RUN
40	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index=62 && D_SAP_index=60 && L_status=DL && DLSDU.len >= 6 && DLSDU.len <= Act_Max_Diag_Len && (Diagnosis-RES-PDU.Diag_Master_Add<>255 && Diagnosis-RES-PDU.Diag_Master_Add<>Loc_Station_Address) => Diagnosis-RES-PDU.Station_status_1.Master_Lock := 1 DMPMM1_Slave_Diag.cnf(+)(Rem_Add:=S_addr, Diag_Data:=Diagnosis-RES-PDU)	RUN
41	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index=62 && D_SAP_index=60 && DL_status=DL && DLSDU.len < 6 => DMPMM1_Slave_Diag.cnf(-)(Rem_Add:=S_addr, Status:=RE)	RUN
42	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index=62 && D_SAP_index=60 && DL_status=DH/RDL/RDH => DMPMM1_Slave_Diag.cnf(-)(Rem_Add:=S_addr, Status:=RE)	RUN
43	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index=62 && D_SAP_index=60 && DL_status=DS/UE/RS/NA/NR/RR => DMPMM1_Slave_Diag.cnf(-)(Rem_Add:=S_addr, Status:=DL_status)	RUN
44	RUN	DMPMM1_Global_Control.req(Rem_Add, Control_Command, Group_Select) /Isochron Mode Supp = FALSE Isochron Mode = Not Synchronized Group_Select <> 0 Control_Command&2=2 && DL IsoM On=FALSE Control_Command&2=0 && DL IsoM On=TRUE => DL_DATA.req(S_SAP_index:=62, D_SAP_index:=58, D_addr:=Rem_Add,DLSDU:=Global_Control-REQ-PDU,Service_class:=High)	RUN

Table 521 (continued)

#	Current State	Event /Condition =>Action	Next State
45	RUN	DMPMM1_Global_Control.req(Rem_Add, Control_Command, Group_Select) /Isochron Mode Supp = TRUE && Isochron Mode <>Not Synchronized && Group_Select=0 && Control_Command&2=0 && DL IsoM On=FALSE => DL IsoM On:=TRUE Control_Command:=0 if(IsoM Sync) Control_Command.Sync:=1 endif if(IsoM Freeze) Control_command.Freeze:=1 endif DLSDU:=Global_Control-REQ-PDU(Control_Command, Group_Select:=Group_8) DL_DATA.req(S_SAP_index:=62, D_SAP_index:=58, D_addr:=Rem_Add, DLSDU:=Global_Control-REQ-PDU, Service_class:=High) DLM_SET_VALUE.req(Variable_name1:"SYNCHT", Variable_name2:"maxTsh", Variable_name3:"Tct", Desired_Value1:=DLSDU, Desired_Value2:=maxTsh, Desired_Value3:=Tct)	RUN
46	RUN	DMPMM1_Global_Control.req(Rem_Add, Control_Command, Group_Select) /Isochron Mode Supp = TRUE && Isochron Mode <>Not Synchronized && Group_Select=0 && Control_Command&2=2 && DL IsoM On=TRUE => DL IsoM On:=FALSE DLSDU:=Global_Control-REQ-PDU(Control_Command:= 2, Group_Select:=Group_8) DL_DATA.req(S_SAP_index:=62, D_SAP_index:=58, D_addr:=Rem_Add, DLSDU:=Global_Control-REQ-PDU, Service_class:=High) DLM_SET_VALUE.req(Variable_name1:"SYNCHT", Variable_name2:"maxTsh", Variable_name3:"Tct", Desired_Value1:=DLSDU, Desired_Value2:=maxTsh, Desired_Value3:=Tct)	RUN
47	RUN	DL_DATA.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_status) /S_SAP_index=62 && D_SAP_index=58 && DL_status=OK/DS => DMPMM1_Global_Control.cnf(Rem_Add:=S_addr, Status:=DL_status)	RUN
48	RUN	DMPMM1_Chk_Cfg.req(Rem_Add, Cfg_Data) => DL_DATA_REPLY.req(S_SAP_index:=62, D_SAP_index:=62, S_addr:=Rem_Add, DLSDU:=Cfg_Data, Service_class:=High)	RUN
49	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index=62 && D_SAP_index=62 && DL_status=NR => DMPMM1_Chk_Cfg.cnf(+)(Rem_Add:=S_addr)	RUN
50	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index=62 && D_SAP_index=62 && DL_status=DS/NA/RS/RR/UE => DMPMM1_Chk_Cfg.cnf(-)(Rem_Add:=S_addr, Status:=DL_status)	RUN
51	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index=62 && D_SAP_index=62 && DL_status=RDL/RDH/DL/DH => DMPMM1_Chk_Cfg.cnf(-)(Rem_Add:=S_addr, Status:=RE)	RUN
52	RUN	DMPMM1_Set_Prm.req(Rem_Add, Prm_Data) => DL_DATA_REPLY.req(S_SAP_index:=62, D_SAP_index:=61, S_addr:=Rem_Add, DLSDU:=Prm_Data, Service_class:=High)	RUN

Table 521 (continued)

#	Current State	Event /Condition =>Action	Next State
53	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index=62 && D_SAP_index=61 && DL_status=NR => DMPMM1_Set_Prm.cnf(+)(Rem_Add:=S_addr)	RUN
54	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index=62 && D_SAP_index=61 && DL_status=DS/NA/RS/RR/UE => DMPMM1_Set_Prm.cnf(-)(Rem_Add:=S_addr, Status:=DL_status)	RUN
55	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index=62 && D_SAP_index=61 && DL_status=RDL/RDH/DL/DH => DMPMM1_Set_Prm.cnf(-)(Rem_Add:=S_addr, Status:=RE)	RUN
56	RUN	DMPMM1_CS_TIME_EVENT.req => D_addr=127, D_SAP_index=CS, S_SAP_index=CS DL_CS_TIME_EVENT.req(D_addr, D_SAP_index, S_SAP_index)	RUN
57	RUN	DL_CS_TIME_EVENT.cnf(D_addr, D_SAP_index, S_SAP_index, Send_delay_time, DL_status) => Send_Delay_Time=Send_delay_time, DL_Status=DL_status DMPMM1_CS_TIME_EVENT.cnf(Send_Delay_Time, DL_Status)	RUN
58	RUN	DMPMM1_CS_CLOCK_VALUE.req (Clock Value Time Event, Clock Value previous TE, Clock Value Status) => D_addr=127, D_SAP_index=CS, S_SAP_index=CS, DLSDU=Clock Value Time Event, Clock Value previous TE, Clock Value Status DL_CS_CLOCK_VALUE.req (D_addr, D_SAP_index, S_SAP_index, DLSDU)	RUN
59	RUN	DL_CS_CLOCK_VALUE.cnf (D_addr, D_SAP_index, S_SAP_index, DL_status) => DL_Status=DL_status DMPMM1_CS_CLOCK_VALUE.cnf (DL_Status)	RUN
60	RUN	DL_CS_CLOCK_VALUE.ind (D_addr, D_SAP_index, S_addr, S_SAP_index, DLSDU, Receive_delay_time, CS_Status) => Time_Master_Addr=S_addr, CS_list=DLSDU, CS_status=CS_Status, Receive_Delay_Time=Receive_delay_time DMPMM1_CS_CLOCK_VALUE.ind (Time_Master_Addr, CS_list, CS_status, Receive_Delay_Time)	RUN
61	RUN	DMPMM1_Set_ExtPrm.req(Rem_Add, Prm_Data) => DL_DATA_REPLY.req(S_SAP_index:=62, D_SAP_index:=53, S_addr:=Rem_Add, DLSDU:=Prm_Data, Service_class:=High)	RUN
62	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index=62 && D_SAP_index=53 && DL_status=NR => DMPMM1_Set_ExtPrm.cnf(+)(Rem_Add:=S_addr)	RUN
63	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index=62 && D_SAP_index=53 && DL_status=DS/NA/RS/RR/UE => DMPMM1_Set_ExtPrm.cnf(-)(Rem_Add:=S_addr, Status:=DL_status)	RUN
64	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_Status) /S_SAP_index=62 && D_SAP_index=53 && DL_status=RDL/RDH/DL/DH => DMPMM1_Set_ExtPrm.cnf(-)(Rem_Add:=S_addr, Status:=RE)	RUN
65	RUN	DLM_SET_VALUE.cnf(DLM_status(1-3)) /DLM_status(1-3)=OK => no action	RUN

Table 521 (continued)

#	Current State	Event /Condition =>Action	Next State
66	RUN	DLM_SET_VALUE.cnf(DLM_status(1)) /DLM_status(1)=OK => no action	RUN
67	any state	DMPMM1_Reset.req => DLM_RESET.req(DLMSAP_0)	PON
68	any state	DLM_RESET.cnf(DLM_status) /DLM_status=OK => DMPMM1_Reset.cnf	PON
69	any state	inadmissible or unknown DL primitive => DMPMM1_Fault.ind	PON

6.7.2.4 Functions

Table 522 contains the functions used by the DMPMM1, their arguments and their descriptions.

Table 522 – Functions used by the DMPMM1

Function name	Description
ActivateSAPFromCRList()	This function checks the CRL DP-master (Class 1) if there are CRL entries which SSAP has not been activated with the DLAP-ACTIVATE.req service primitive yet. A) If there exist one of the following CRL entries that has not been activated yet: Entries with SSAP=NIL or SSAP=62 or SSAP=51 or SSAP=54 than it returns TRUE and sets the local context according to the current CREP. B) Otherwise it returns FALSE.
BuildServiceListFromCR()	This function builds the Service_list parameter for DLSAP-ACTIVATE.req primitive from the current CRL context. A) If SSAP = NIL(MS0) it returns Service_list.Service_activate[1] = SRD Service_list.Role_in_Service[1] = Initiator Service_list.DLSDU_length_list[1].Max_DLSDU_length_req_low=0 Service_list.DLSDU_length_list[1].Max_DLSDU_length_req_high=244 Service_list.DLSDU_length_list[1].Max_DLSDU_length_ind/cnf_low=244 Service_list.DLSDU_length_list[1].Max_DLSDU_length_ind/cnf_high=244 B) If SSAP = 62(MS0) it returns Service_list.Service_activate[1] = SRD Service_list.Role_in_Service[1] = Initiator Service_list.DLSDU_length_list[1].Max_DLSDU_length_req_low=0 Service_list.DLSDU_length_list[1].Max_DLSDU_length_req_high=244 Service_list.DLSDU_length_list[1].Max_DLSDU_length_ind/cnf_low=244 Service_list.DLSDU_length_list[1].Max_DLSDU_length_ind/cnf_high=0 Service_list.Service_activate[2] = SDN Service_list.Role_in_Service[2] = Initiator Service_list.DLSDU_length_list[2].Max_DLSDU_length_req_low=0 Service_list.DLSDU_length_list[2].Max_DLSDU_length_req_high=2 Service_list.DLSDU_length_list[2].Max_DLSDU_length_ind/cnf_low=0 Service_list.DLSDU_length_list[2].Max_DLSDU_length_ind/cnf_high=0 C) If SSAP = 51(MS1) it returns Service_list.Service_activate[1] = SRD Service_list.Role_in_Service[1] = Initiator Service_list.DLSDU_length_list[1].Max_DLSDU_length_req_low=244 Service_list.DLSDU_length_list[1].Max_DLSDU_length_req_high=0 Service_list.DLSDU_length_list[1].Max_DLSDU_length_ind/cnf_low=244 Service_list.DLSDU_length_list[1].Max_DLSDU_length_ind/cnf_high=0 D) If SSAP = 54(MM) it returns Service_list.Service_activate[1] = SDN Service_list.Role_in_Service[1] = Receiver Service_list.DLSDU_length_list[1].Max_DLSDU_length_req_low=0 Service_list.DLSDU_length_list[1].Max_DLSDU_length_req_high=0 Service_list.DLSDU_length_list[1].Max_DLSDU_length_ind/cnf_low=4 Service_list.DLSDU_length_list[1].Max_DLSDU_length_ind/cnf_high=0
BuildLengthFromCR()	This function builds the Service_list parameter for DLSAP-ACTIVATE-RESPONDER.req primitive from the current CRL context. DLSDU_length_list.Max_DLSDU_length_req_low = 244 DLSDU_length_list.Max_DLSDU_length_req_high = 0 DLSDU_length_list.Max_DLSDU_length_ind_low = 244 DLSDU_length_list.Max_DLSDU_length_ind_high = 0

6.7.3 DMPMM2

6.7.3.1 Primitive definitions

6.7.3.1.1 Primitives exchanged between FSPMM2 and DMPMM2

Table 523 shows the service primitives including their associated parameters issued by the FSPMM2 and received by the DMPMM2.

Table 523 – Primitives issued by FSPMM2 to DMPMM2

Primitive name	Source	Associated parameters	Functions
MInit DLL.req	FSPMM2	Bus Para	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5 and Type 3 Fieldbus IEC 61158-3
Reset.req	FSPMM2	(none)	
Read Slave Diag.req	FSPMM2	Rem Add	
Read Output.req	FSPMM2	Rem Add	
Read Input.req	FSPMM2	Rem Add	
Get Cfg.req	FSPMM2	Rem Add	
Set Slave Add.req	FSPMM2	Rem Add, New Slave Add, Ident Number, No Add Chg, Rem Slave Data	

Table 524 shows the service primitives including their associated parameters issued by the DMPMM2 and received by the FSPMM2.

Table 524 – Primitives issued by DMPMM2 to FSPMM2

Primitive name	Source	Associated parameters	Functions
MInit DLL.cnf	DMPMM2	(non)	Refer to FAL Service Definition Type 3 Fieldbus IEC 61158-5
Reset.cnf	DMPMM2	(none)	
Read Slave Diag.cnf(+)	DMPMM2	Rem Add, Diag Data	
Read Slave Diag.cnf(-)	DMPMM2	Rem Add, Status	
Get Cfg.cnf(+)	DMPMM2	Rem Add, Cfg Data	
Get Cfg.cnf(-)	DMPMM2	Rem Add, Status	
Read Output.cnf(+)	DMPMM2	Rem Add, Output Data	
Read Output.cnf(-)	DMPMM2	Rem Add, Status	
Read Input.cnf(+)	DMPMM2	Rem Add, Input Data	
Read Input.cnf(-)	DMPMM2	Rem Add, Status	
Set Slave Add.cnf(+)	DMPMM2	Rem Add	
Set Slave Add.cnf(-)	DMPMM2	Rem Add, Status	
Event.ind	DMPMM2	Event, Add Info	
Fault.ind	DMPMM2	Rem Add	

6.7.3.1.2 Primitives exchanged between DMPMM2 and MSAC2M

Table 525 shows the service primitives including their associated parameters issued by the MSAC2M and received by the DMPMM2.

Table 525 – Primitives issued by MSAC2M to DMPMM2

Primitive name	Source	Associated parameters	Functions
DATA_REPLY.req	MSAC2M	SSAP, DSAP, Rem_Add, L_sdu, Serv_class	Send Service

Table 526 shows the service primitives including their associated parameters issued by the DMPMM2 and received by the MSAC2M.

Table 526 – Primitives issued by DMPMM2 to MSAC2M

Primitive name	Source	Associated parameters	Functions
DATA_REPLY.cnf	DMPMM2	SSAP, DSAP, Rem_Add, L_sdu, Serv_class, L_status	Send Service executed

6.7.3.1.3 Primitives exchanged between DMPMM2 and MMAC2

Table 527 shows the service primitives including their associated parameters issued by the MMAC2 and received by the DMPMM2.

Table 527 – Primitives issued by MMAC2 to DMPMM2

Primitive name	Source	Associated parameters	Functions
DATA_REPLY.req	MMAC2	SSAP, DSAP, Rem_Add, L_sdu, Serv_class	Send Service
DATA.req	MMAC2	SSAP, DSAP, Rem_Add, L_sdu, Serv_class	Send Service

Table 528 shows the service primitives including their associated parameters issued by the DMPMM2 and received by the MMAC2.

Table 528 – Primitives issued by DMPMM2 to MMAC2

Primitive name	Source	Associated parameters	Functions
DATA_REPLY.cnf	DMPMM2	SSAP, DSAP, Rem_Add, L_sdu, Serv_class, L_status	Send Service executed
DATA.cnf	DMPMM2	SSAP, DSAP, Rem_Add, Serv_class, L_status	Send Service executed

6.7.3.1.4 Primitives exchanged between DMPMM2 and DL

Table 529 shows the service primitives including their associated parameters issued by the DMPMM2 and received by the DL.

Table 529 – Primitives issued by DMPMM2 to DL

Primitive name	Source	Associated parameters	Functions
DL-DATA.req	DMPMM2	Service_class, D_addr, D_SAP_index, S_SAP_index, DLSDU	Refer to DL Service Definition Type 3 Fieldbus IEC 61158-3
DL-DATA-REPLY.req	DMPMM2	Service_class, D_addr, D_SAP_index, S_SAP_index, DLSDU	
DLM-RESET.req	DMPMM2	(none)	
DLM-SET-VALUE.req	DMPMM2	Variable_name(1 to n), Desired_value (1 to n)	
DLSAP-ACTIVATE.req	DMPMM2	S_SAP_index, Access, Service_list	
DLSAP-DEACTIVATE.req	DMPMM2	S_SAP_index	

Table 530 shows the service primitives including their associated parameters issued by the DL and received by the DMPMM2.

Table 530 – Primitives issued by DL to DMPMM2

Primitive name	Source	Associated parameters	Functions
DL-DATA.cnf	DL	Service_class, D_addr, D_SAP_index, S_SAP_index, DL_status	Refer to DL Service Definition Type 3 Fieldbus IEC 61158-3
DL-DATA-REPLY.cnf	DL	Service_class, D_addr, D_SAP_index, S_SAP_index, DLSDU, DL_status	
DLM-RESET.cnf	DL	DLM_Status	
DLM-SET-VALUE.cnf	DL	Current_value (1 to n), DLM_status	
DLM-EVENT.ind	DL	Event/Fault	
DLSAP-ACTIVATE.cnf	DL	S_SAP_index, DLM_status	
DLSAP-DEACTIVATE.cnf	DL	S_SAP_index, DLM_status	

6.7.3.1.5 Parameters of FSPMM2 primitives

The parameters used with the primitives exchanged between DMPMM2 and FSPMM2 are described in FAL Service Definition Type 3 Fieldbus IEC 61158-5. Additional parameters are described in Table 531.

Table 531 – Parameters used with primitives exchanged with DMPMM2

Parameter name	Description
SSAP	Local Identifier for Service Access Point
DSAP	Identifier for the remote Service Access Point
Serv_class	Indicates priority of the service
L_status	Status of the service execution
L_sdu	Defines the DP PDU to be transmitted
Rem_Add	DL Address of the remote station

6.7.3.2 State machine description

The DMPMM2 makes the connection between the MS0,MS2,MM1/2 specific parts and Layer 2. Services are mapped by the DMPMM2 to the DL and management of Layer 2. The DMPMM2 delivers the necessary Layer 2 parameters of the function call (SSAP, DSAP, Serv_class, ...) for Layer 2, receives the confirmations and indications from Layer 2 and passes them to the MS0,MS2-Handler.

6.7.3.3 DMPMM2 state table

Table 532 contains the complete description of the DMPMM2 state machine.

Table 532 – DMPMM2 state table

#	Current state	Event / condition => action	Next state
1	PON	DMPMM2_Minit_DLL.req(Bus_Para) => DLM_SET_VALUE.req(Variable_name1:= "TS ", Variable_name2:= "Data_rate ", Variable_name3:= "TSL ", Variable_name4:= "min TSDR ", Variable_name5:= "max TSDR ", Variable_name6:= "TQUI ", Variable_name7:= "TSET ", Variable_name8:= "TTR ", Variable_name9:= "G ", Variable_name10:= "HSA ", Variable_name11:= "max_retry_limit ", Variable_name12:= "in_ring_desired", Desired_Value1:= Bus_Para.TS Desired_Value2:= Bus_Para.Data_rate, Desired_Value3:= Bus_Para.TSL, Desired_Value4:= Bus_Para.min TSDR, Desired_Value5:= Bus_Para.max TSDR, Desired_Value6:= Bus_Para.TQUI, Desired_Value7:= Bus_Para.TSET, Desired_Value8:= Bus_Para.TTR, Desired_Value9:= Bus_Para.G, Desired_Value10:= Bus_Para.HSA, Desired_Value11:= Bus_Para.max_retry_limit, Desired_Value12:= TRUE)	SET-VALUE
2	PON	DLM_RESET.cnf(DLM_status) /DLM_status=OK => DMPMM2_Reset.cnf	PON
3	SET-VALUE	DLM_SET_VALUE.cnf(DLM_status(1-11)) /DLM_status(1-11)=OK => no action	SAP-CHECK
4	SAP-CHECK	spontaneous /ActivateSAPFromCRList()=TRUE => DLSAP_ACTIVATE.req(S_SAP_index:=SSAP, Access:=All, Service_list:=BuildServiceListFromCR())	SAP-ACT

Table 532 (continued)

#	Current state	Event / condition => action	Next state
5	SAP-CHECK	spontaneous /ActivateSAPFromCRList()=FALSE => DMPMM2_Minit_DLL.cnf	RUN
6	SAP-ACT	DLSAP_ACTIVATE.cnf(S_SAP_index, DLM_status) /DLM_status=OK => no action	SAP-CHECK
7	RUN	DLM_EVENT.ind(Event/Fault, Add_info) => DMPMM2_Event.ind(Event:=Event/Fault, Add_Info:=Add_info)	RUN
8	RUN	DMPMM2_Read_Slave_Diag.req(Rem_Add) => DL_DATA_REPLY.req(S_SAP_index:=62, D_SAP_index:=60, S_addr:=Rem_Add, DLSDU.len:=0, Service_class:=High)	RUN
9	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_status) /S_SAP_index=62 && D_SAP_index=60 && DL_status=DL && DLSDU.len >= 6 => DMPMM2_Read_Slave_Diag.cnf(+)(Rem_Add:=S_addr, Diag_Data:=DLSDU)	RUN
10	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_status) /S_SAP_index=62 && D_SAP_index=60 && DL_status=DL && DLSDU < 6 => DMPMM2_Read_Slave_Diag.cnf(-)(Rem_Add:=S_addr, Status:=RE)	RUN
11	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_status) /S_SAP_index=62 && D_SAP_index=60 && DL_status=DH/RDL/RDH => DMPMM2_Read_Slave_Diag.cnf(-)(Rem_Add:=S_addr, Status:=RE)	RUN
12	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_status) /S_SAP_index=62 && D_SAP_index=60 && DL_status=DS/UE/RS/NA/NR/RR => DMPMM2_Read_Slave_Diag.cnf(-)(Rem_Add:=S_addr, Status:=DL_status)	RUN
13	RUN	DMPMM2_Get_Cfg.req(Rem_Add) => DL_DATA_REPLY.req(S_SAP_index:=62, D_SAP_index:=59, S_addr:=Rem_Add, DLSDU.len:=0, Service_class:=High)	RUN
14	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_status) /S_SAP_index=62 && D_SAP_index=59 && DL_status=DL => DMPMM2_Get_Cfg.cnf(+)(Rem_Add:=S_addr, Cfg_Data:=DLSDU)	RUN
15	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_status) /S_SAP_index=62 && D_SAP_index=59 && DL_status=DS/UE/RS/NA/NR/RR => DMPMM2_Get_Cfg.cnf(-)(Rem_Add:=S_addr, Status:=DL_status)	RUN
16	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_status) /S_SAP_index=62 && D_SAP_index=59 && DL_status=DH/RDL/RDH => DMPMM2_Get_Cfg.cnf(-)(Rem_Add:=S_addr, Status:=RE)	RUN
17	RUN	DMPMM2_Read_Input.req(Rem_Add) => DL_DATA_REPLY.req(S_SAP_index:=62, D_SAP_index:=56, S_addr:=Rem_Add, DLSDU.len:=0, Service_class:=High)	RUN
18	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_status) /S_SAP_index=62 && D_SAP_index=56 && DL_status=DL => DMPMM2_Read_Input.cnf(+)(Rem_Add:=S_addr, Inp_Data:=DLSDU)	RUN

Table 532 (continued)

#	Current state	Event / condition => action	Next state
19	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_status) /S_SAP_index=62 && D_SAP_index=56 && DL_status=DS/UE/RS/NA/NR/RR => DMPMM2_Read_Input.cnf(-)(Rem_Add:=S_addr, Status:=DL_status)	RUN
20	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_status) /S_SAP_index=62 && D_SAP_index=56 && DL_status=DH/RDL/RDH => DMPMM2_Read_Input.cnf(-)(Rem_Add:=S_addr, Status:=RE)	RUN
21	RUN	DMPMM2_Read_Output.req(Rem_Add) => DL_DATA_REPLY.req(S_SAP_index:=62, D_SAP_index:=57, S_addr:=Rem_Add, DLSDU.len:=0, Service_class:=High)	RUN
22	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_status) /S_SAP_index=62 && D_SAP_index=57 && DL_status=DL => DMPMM2_Read_Output.cnf(+)(Rem_Add:=S_addr, Outp_Data:=DLSDU)	RUN
23	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_status) /S_SAP_index=62 && D_SAP_index=57 && DL_status=DS/UE/RS/NA/NR/RR => DMPMM2_Read_Output.cnf(-)(Rem_Add:=S_addr, Status:=DL_status)	RUN
24	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_status) /S_SAP_index=62 && D_SAP_index=57 && DL_status=DH/RDL/RDH => DMPMM2_Read_Output.cnf(-)(Rem_Add:=S_addr, Status:=RE)	RUN
25	RUN	DMPMM2_Set_Slave_Add.req(Rem_Add, New_Slave_Add, Ident_Number, No_Add_Chg, Rem_Slave_Data) => DL_DATA_REPLY.req(S_SAP_index:=62, D_SAP_index:=55, S_addr:=Rem_Add, DLSDU:=Set_Slave_Add-REQ-PDU, Service_class:=High)	RUN
26	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_status) /S_SAP_index=62 && D_SAP_index=55 && DL_status=NR => DMPMM2_Set_Slave_Add.cnf(+)(Rem_Add:=S_addr)	RUN
27	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_status) /S_SAP_index=62 && D_SAP_index=55 && DL_status=DS/UE/RR/RS/NA => DMPMM2_Set_Slave_Add.cnf(-)(Rem_Add:=S_addr, Status:=DL_status)	RUN
28	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_status) /S_SAP_index=62 && D_SAP_index=55 && DL_status=RDL/RDH/DL/DH => DMPMM2_Set_Slave_Add.cnf(-)(Rem_Add:=S_addr, Status:=RE)	RUN
29	RUN	DMPMM2_DATA_REPLY.req (SSAP, DSAP, Rem_Add, L_sdu, Serv_class) => DL_DATA_REPLY.req (S_SAP_index:=SSAP, D_SAP_index:=DSAP, S_addr:=Rem_Add, DLSDU:=L_sdu, Service_class:=Serv_class)	RUN
30	RUN	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_status) /S_SAP_index<>62 && DL_status<>LS/IV/LR => DMPMM2_DATA_REPLY.cnf (SSAP:=S_SAP_index, DSAP:=D_SAP_index, Rem_Add:=S_addr, L_sdu:=DLSDU, Serv_class:=Service_class, L_status:=DL_status)	RUN
31	RUN	DMPMM2_DATA.req (SSAP, DSAP, Rem_Add, L_sdu, Serv_class) => DL_DATA.req (S_SAP_index:=SSAP, D_SAP_index:=DSAP, S_addr:=Rem_Add, DLSDU:=L_sdu, Service_class:=Serv_class)	RUN

Table 532 (continued)

#	Current state	Event / condition => action	Next state
32	RUN	DL_DATA.cnf(S_SAP_index, D_SAP_index, S_addr, Service_class, DL_status) /DL_status<>LS/IV/LR => DMPMM2_DATA.cnf (SSAP:=S_SAP_index, DSAP:=D_SAP_index, Rem_Add:=S_addr, Serv_class:=Service_class, L_status:=DL_status)	RUN
33	any state	DLSAP_ACTIVATE.cnf(S_SAP_index, DLM_status) /DLM_status<>OK => DMPMM2_Fault.ind	PON
34	any state	DLM_SET_VALUE.cnf(DLM_status(1-11)) /NOT (DLM_status(1-11)=OK) => DMPMM2_Fault.ind	PON
35	any state	DL_DATA_REPLY.cnf(S_SAP_index, D_SAP_index, S_addr, DLSDU, Service_class, DL_status) /DL_status=LS/IV/LR => DMPMM2_Fault.ind	RUN
36	any state	inadmissible or unknown DL primitive => DMPMM2_Fault.ind	RUN
37	any state	DL_DATA.cnf(S_SAP_index, D_SAP_index, S_addr, Service_class, DL_status) /DL_status=LS/IV/LR => DMPMM2_Fault.ind	PON
38	any state	DMPMM2_Reset.req => DLM_RESET.req	PON

6.7.3.4 Functions

Table 533 contains the functions used by the DMPMM2, their arguments and their descriptions.

Table 533 – Functions used by DMPMM2

Function name	Description
ActivateSAPFromCRList()	This function checks the CRL DP-master (Class 2) if there are CRL entries which SSAP has not been activated with the DLM-SAP-ACTIVATE.req service primitive yet. A) If there exist one of the following CRL entries that has not been activated yet: Entries with SSAP=62 or SSAP=50 or SSAP=54 than it returns TRUE and sets the local context according to the current CREP. B) Otherwise it returns FALSE.
BuildServiceListFromCR()	This function builds the Service_list parameter for DLSAP-ACTIVATE.req primitive from the current CRL context. A) If SSAP = 62(MS0) it returns Service_list.Service_activate[1] = SRD Service_list.Role_in_Service[1] = Initiator Service_list.DLSDU_length_list[1].Max_DLSDU_length_req_low=0 Service_list.DLSDU_length_list[1].Max_DLSDU_length_req_high=244 Service_list.DLSDU_length_list[1].Max_DLSDU_length_ind/cnf_low=244 Service_list.DLSDU_length_list[1].Max_DLSDU_length_ind/cnf_high=0 B) If SSAP = 50(MS2) it returns Service_list.Service_activate[1] = SRD Service_list.Role_in_Service[1] = Initiator Service_list.DLSDU_length_list[1].Max_DLSDU_length_req_low=244 Service_list.DLSDU_length_list[1].Max_DLSDU_length_req_high=0 Service_list.DLSDU_length_list[1].Max_DLSDU_length_ind/cnf_low=244 Service_list.DLSDU_length_list[1].Max_DLSDU_length_ind/cnf_high=0 C) If SSAP = 54(MM) it returns Service_list.Service_activate[1] = SRD Service_list.Role_in_Service[1] = Initiator Service_list.DLSDU_length_list[1].Max_DLSDU_length_req_low=244 Service_list.DLSDU_length_list[1].Max_DLSDU_length_req_high=0 Service_list.DLSDU_length_list[1].Max_DLSDU_length_ind/cnf_low=244 Service_list.DLSDU_length_list[1].Max_DLSDU_length_ind/cnf_high=0 Service_list.Service_activate[2] = SDN Service_list.Role_in_Service[2] = Initiator Service_list.DLSDU_length_list[2].Max_DLSDU_length_req_low=2 Service_list.DLSDU_length_list[2].Max_DLSDU_length_req_high=0 Service_list.DLSDU_length_list[2].Max_DLSDU_length_ind/cnf_low=0 Service_list.DLSDU_length_list[2].Max_DLSDU_length_ind/cnf_high=0

6.8 Parameters for a DP-slave

Table 534 contains limitations for datarate-depending AL timing parameters of Slaves.

Table 534 – Bus parameter/reaction times for a DP-slave

Datarate (kbit/s)	≤187,5	500	1500	3000	6000	12000
Min_Slave_Interval (ms)	≤ 20	≤ 6	≤ 2	≤ 1,5	≤ 1	≤ 0,6
Send_Timeout (sec) (1)	4	2	1	1	1	1
C1_Timeout (sec) (2)	40	20	10	10	10	10
NOTE 1 The parameter Send_Timeout represents a performance feature of a DP-slave implementation. Each DP-slave implementation has to ensure that the parameter Send_Timeout reaches the smallest value that is possible.						
NOTE 2 The parameter C1_Timeout represents a performance feature of a DP-slave. Each DP-slave implementation has to ensure that the parameter C1_Timeout reaches the smallest value that is possible.						

Another performance feature for a DP-slave implementation is the time Min_Slave_Interval. Each DP-slave implementation has to ensure that the Min_Slave_Interval reaches the smallest possible value. This means, that in a DP-System with more than ten stations the Min_Slave_Interval shall not be the dominant factor for the cycle time.

7 Type 4

7.1 FAL syntax description

7.1.1 FAL-AR PDU abstract syntax

7.1.1.1 General

The information stored in an APDU depends on whether the APDU holds a request or a response. The role of the state machine that encodes the APDU (the FSPM) determines how the APDU is encoded.

APDUs always consist of an APDU Header and an APDU Body. In response APDUs the APDU body may be empty.

7.1.1.2 Abstract Syntax of APDU Header

Table 535 defines the contents of the APDU Header.

Table 535 – APDU Header

Field name	Subfield name	Possible values	Constraint (present if)	Comment
ControlStatus	Instruction	Errorcode Write Read And Or Test-And-Set Segmented Read Segmented Write		
ControlStatus	Errorcode	Described in Figure 145 to Figure 147	ControlStatus.Instruction = Errorcode	
ControlStatus	Addressing method	Variable Object Flat	ControlStatus.Instruction <> Errorcode	
ControlStatus	ActualDataError	NoActualError ActualError	ControlStatus.Instruction <> Errorcode	Used by the responding user application to indicate, that an actual error may affect the accessed Variable Object
ControlStatus	HistoricalDataError	NoHistoricalError HistoricalError	ControlStatus.Instruction <> Errorcode	Used by the responding user application to indicate, that an error may have affected the accessed Variable Object
DataFieldFormat	Offset/Attribute	No Offset/Attribute Offset/Attribute		Indicates, whether the APDU Body holds an Offset/Attribute field
DataFieldFormat	Variable Identifier Format	Simple Complex	APDU is a request APDU	Indicates the format of the Variable Identifier in a request APDU
DataFieldFormat	Offset/Attribute size	Integer16 Integer32	APDU is a response APDU AND DataFieldFormat.Offset/Attribute = Offset/Attribute	Indicates the size of the Offset/Attribute field of the APDU Body
DataLength		min. 2		Indicates the total length of the APDU Body. MaxDataSize indicates the max length of the data part of the APDU Body.

7.1.1.3 Abstract Syntax of APDU Body

The APDU Header indicates the interpretation of the contents of the APDU Body.

Table 536 defines the contents of the APDU Body.

Table 536 – APDU Body

Field name	Subfield name	Possible values	Constraint (present if)	Comment
VariableIdentifier	Code.Bitaddressing	No BitAddressing BitAddressing	APDU is a request APDU AND APDU Header indicates Complex VariableIdentifier	If this field indicates BitAddressing, the VariableIdentifier also holds a Bit-no
VariableIdentifier	Code.Bit-no	0-7	APDU is a request APDU AND APDU Header indicates Complex VariableIdentifier AND VariableIdentifier indicates BitAddressing	Bit-no selects a bit within one octet. Bit-no = 0 selects bit 1 etc. The octet is selected by Offset/Attribute.
VariableIdentifier	Code.Offset/Attribute size	Integer16 Integer32	APDU is a request APDU AND DataFieldFormat.Variable Identifier Format = Complex AND DataFieldFormat.Offset/ Attribute = Offset/Attribute	
VariableIdentifier	ID	-32768 - +32767	APDU is a request APDU AND DataFieldFormat.Variable Identifier Format = Simple	
VariableIdentifier	ID	-8388608 - +8388607	APDU is a request APDU AND DataFieldFormat.Variable Identifier Format = Complex	
Offset/Attribute		-32768 - +32767	APDU is a request APDU AND DataFieldFormat.Offset/ Attribute = Offset/Attribute AND VariableIdentifier.Code. Offset/Attribute size = Integer16	Negative values select attribute, positive values select part of constructed variable
Offset/Attribute		-2147483648 - +2147483647	APDU is a request APDU AND DataFieldFormat.Offset/ Attribute = Offset/Attribute AND VariableIdentifier.Code. Offset/Attribute size = Integer32	Negative values select attribute, positive values select part of constructed variable
Offset/Attribute		-32768 - +32767	APDU is a response APDU AND DataFieldFormat.Offset/ Attribute = Offset/Attribute AND DataFieldFormat.Offset/ Attribute size= Integer16	Negative values select attribute, positive values select part of constructed variable
Offset/Attribute		-2147483648 - +2147483647	APDU is a response APDU AND DataFieldFormat.Offset/ Attribute = Offset/Attribute AND DataFieldFormat.Offset/ Attribute size= Integer32	Negative values select attribute, positive values select part of constructed variable

Field name	Subfield name	Possible values	Constraint (present if)	Comment
Data		Any		
RequestedLength		0-65535	APDU is a request APDU AND ControlStatus.Instruction indicates Read OR Segmented Read	Indicates the length of data to Read, as the number of octets
Sequence		0-2	APDU is a request APDU AND ControlStatus.Instruction indicates Segmented Read OR Segmented Write	Indicates whether this request is the first, one in the middle, or the last of a segmented transfer.

7.1.2 Data types

The notation for data types is the same as Type 1 for the following types:

- Integer, Integer8, Integer16, Integer32
- Unsigned, Unsigned8, Unsigned16
- Floating32, Floating64

7.2 Transfer syntaxes

7.2.1 APDU encoding

7.2.1.1 APDU Header encoding

7.2.1.1.1 APDU Header structure

he abstract syntax of the APDU Header is defined in 7.1.1.2. This subclause describes the encoding of the header. The APDU Header consists of three fields, as shown in Figure 144.

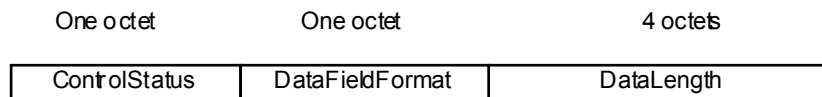


Figure 144 – APDU Header structure

7.2.1.1.2 ControlStatus

ControlStatus is coded into one octet. The interpretation of this octet depends on the instruction subfield. The coding of the instruction subfield is shown in Figure 145.

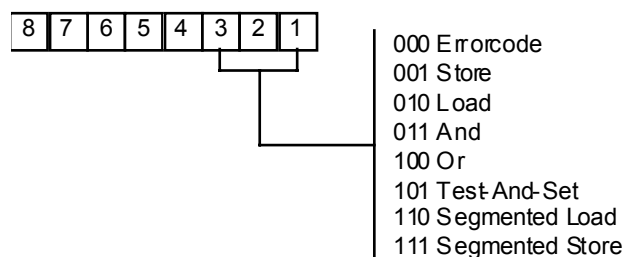


Figure 145 – Instruction subfield of ControlStatus

If the instruction is = 000 (= Errorcode), the remaining five bits of ControlStatus holds the error code. The possible values are shown in Figure 146.

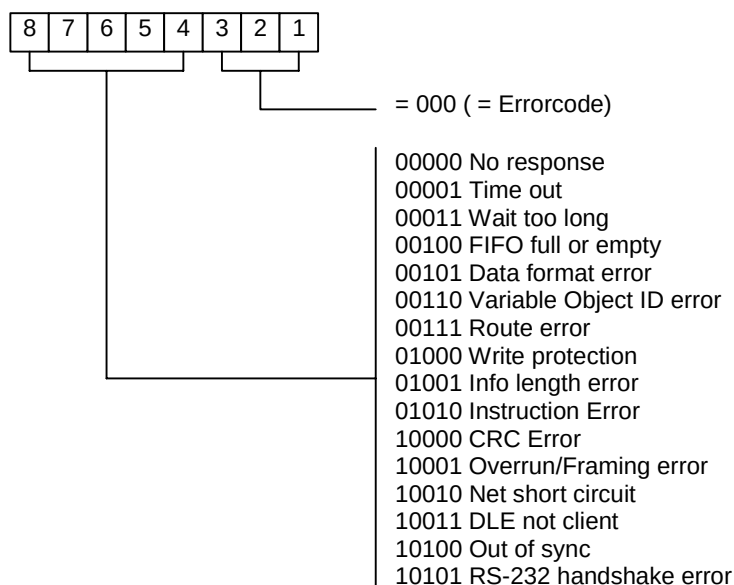


Figure 146 – Errorcode subfield of ControlStatus

If the instruction is $\neq$ 000 ($\neq$ Errorcode), the remaining five bits of ControlStatus holds the subfields Addressing method, ActualDataError and HistoricalDataError. The coding of these fields is shown in Figure 147.

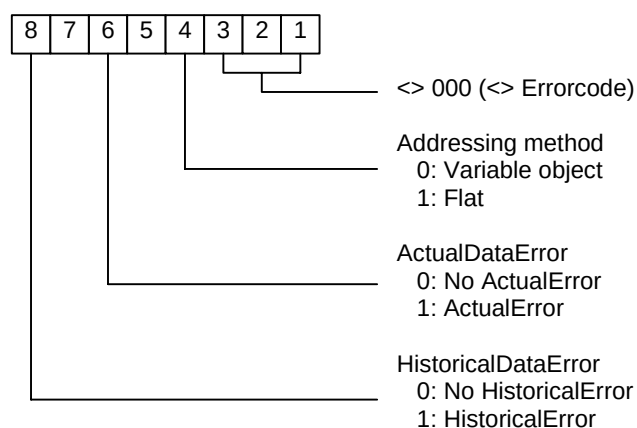


Figure 147 – Remaining subfields of ControlStatus

7.2.1.1.3 DataFieldFormat

DataFieldFormat is coded into one octet. The coding of this octet is shown in Figure 148.

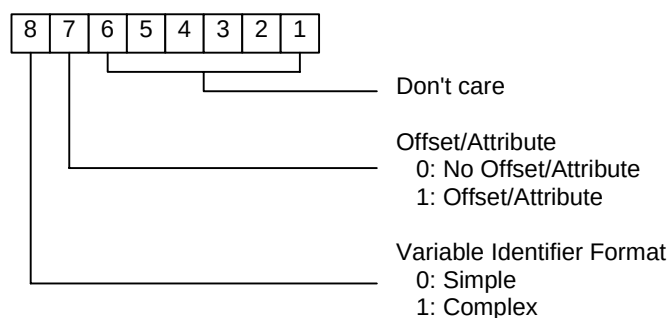


Figure 148 – DataFieldFormat encoding

7.2.1.1.4 DataLength

DataLength is an Integer32, indicating the total length of the APDU Body.

7.2.1.2 APDU Body encoding

7.2.1.2.1 APDU Body structure

The abstract syntax for the APDU Body is described in 7.1.1.3. This subclause describes the encoding. The interpretation of the APDU Body is indicated by the APDU Header.

A request APDU Body may consist of up to four of five possible fields, as shown in Figure 149.

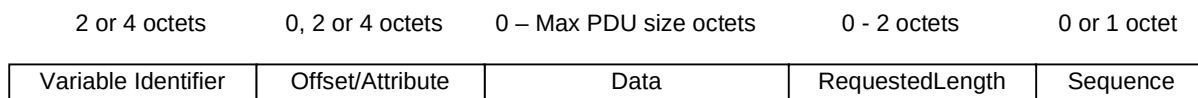


Figure 149 – Structure of request APDU Body

A response APDU Body may consist of up to two fields, as shown in Figure 150.

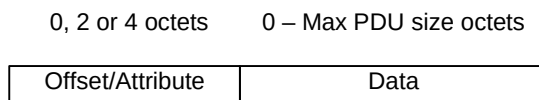


Figure 150 – Structure of response APDU Body

7.2.1.2.2 Variable Identifier

The Variable Identifier can be either Simple or Complex. If it is Simple, it consists of only one subfield, ID, which is of type Integer16. If it is complex, it consists of two subfields, Code (1 octet) and ID (3 octets) as shown in Figure 151.



Figure 151 – Variable Identifier

The coding of the Code subfield of the Variable Identifier is shown in Figure 152.

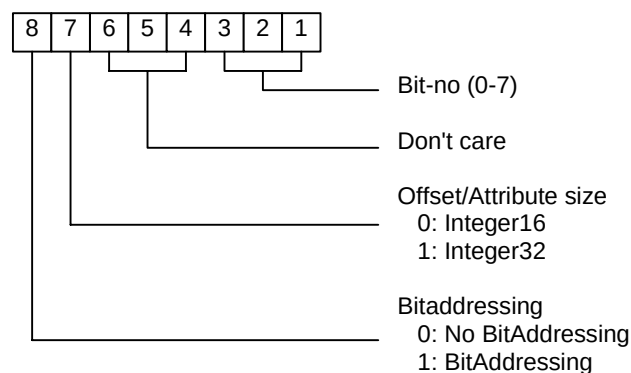


Figure 152 – Code subfield of Variable Identifier

7.2.1.2.3 Offset/Attribute

The Offset/Attribute subfield of the APDU Body may or may not be present. If present, Offset/Attribute is an Integer16 or an Integer32. A negative value selects an attribute of the Variable Object. A positive value selects a part of the data value of the Variable Object, by indicating the offset in octets to the starting octet of the data block to be transferred, relative to the first octet of the variable.

7.2.1.2.4 RequestedLength

The RequestedLength subfield may or may not be present.

If the APDU is a request APDU, and the ControlStatus.Instruction subfield of the APDU Header indicates Read or Segmented Read, the RequestedLength subfield is present. It indicates the octet length of the requested data or attribute.

The RequestedLength subfield is an Unsigned8 or an Unsigned16. As there is no Data subfield in the APDU if there is a RequestedLength subfield, the size of RequestedLength is implicitly given by the DataLength parameter. If the value of RequestedLength is less than 256, RequestedLength shall be of type Unsigned8.

7.2.1.2.5 Data

The Data subfield of the APDU Body holds either:

- Data, coded and packed as described in 7.2.2.
- An attribute of a Variable Object, coded and packed as described in 7.2.2.

As there is no RequestedLength subfield in the APDU if there is a Data subfield, the size of the Data subfield is implicitly given by the DataLength parameter.

7.2.1.2.6 Sequence

The Sequence subfield is one octet, with the following legal values:

- 0: Indicating, that this is the first request of a segmented Read or Write.
- 1: Indicating, that this is one of the following requests of a segmented Read or Write.
- 2: Indicating, that this is the last request of a segmented Read or Write.

7.2.2 Variable Object encoding and packing

7.2.2.1 Encoding of Simple Variables

7.2.2.1.1 Encoding of a Boolean value

- a) The encoding of a Boolean value shall be primitive. The ContentsOctets shall consist of a single octet.
- b) If the Boolean value is FALSE, bit 1 of the ContentsOctets shall be 0 (zero). If the Boolean value is TRUE, bit 1 of the ContentsOctets shall be 1 (one).

7.2.2.1.2 Encoding of an Integer value

As defined in Type 1, Transfer syntax 1, Encoding of an Integer Value.

7.2.2.1.3 Encoding of an Unsigned value

As defined in Type 1, Transfer syntax 1, Encoding of an Unsigned Value, types Unsigned8 and Unsigned 16.

7.2.2.1.4 Encoding of a Floating Point value

As defined in Type 1, Transfer syntax 1, Encoding of a Floating-Point Value.

7.2.2.2 Encoding of Constructed Variables

7.2.2.2.1 Encoding of a String value

- a) The encoding of a variable length String value shall be primitive.
- b) The Length field shall indicate as a binary number the number of elements in the String value.
- c) The Length field shall be in the first octet.

7.2.2.2.2 Encoding of a BitString value

- a) The encoding of a BitString value shall be primitive.
- b) There is no Length field in the BitString.
- c) The value of the BitString, commencing with the first bit and proceeding to the trailing bit, shall be placed in bits 1 to 8 of the first octet, followed by bits 1 to 8 of the second octet, followed by bits 1 to 8 of each octet up to and including the last octet of the ContentsOctets.
- d) Unused bits, if any, shall be placed in bits 2-8 of the last octet.

7.2.2.3 Alignment

7.2.2.3.1 General

This section describes, how fields and elements of constructed variables are aligned and transferred.

The general alignment is two.

- Fields and elements of basic type, and with a size of 1 octet, shall be transferred immediately after the previous field or element.
- Fields and elements with a size of more than 1 octet, shall be transferred with an even offset relative to the first octet of the constructed variable.
- Fields and elements of constructed type shall be transferred with an even offset relative to the first octet of the constructed variable.

7.2.2.3.2 Array Elements transfer syntax

Table 537 shows an example of transfer syntax for a variable "ArrayVar" of type
ARRAY[1..2] of ARRAY[1..3] of Integer8.

Table 537 – Transfer syntax for Array

1. octet	ArrayVar[1,1]
2. octet	ArrayVar[1,2]
3. octet	ArrayVar[1,3]
4. octet	ArrayVar[2,1]
5. octet	ArrayVar[2,2]
6. octet	ArrayVar[2,3]

7.2.2.3.3 Structure Fields transfer syntax

Table 538 shows an example of transfer syntax for a variable "StructVar" of type

```

STRUCTURE
  Field1: Integer8;
  Field2: STRUCTURE
    Sub1: Integer16;
    Sub2: BitString[8];
  END;
  Field3: Integer8;
  Field4: BitString[8];
  Field5: Integer8;
END;
```

Table 538 – Transfer Syntax for Structure

1. octet	StructVar.Field1
2. octet	Dummy
3. octet	StructVar.Field2.Sub1, Most significant octet
4. octet	StructVar. Field2.Sub1, Least significant octet
5. octet	StructVar.Field2.Sub2
6. octet	StructVar.Dummy
7. octet	StructVar.Field3
8. octet	StructVar.Dummy
9. octet	StructVar.Field4
10. octet	StructVar.Field5

7.2.2.4 Variable Object attributes

All Variable Objects may have the optional attributes defined in Table 539.

Table 539 – Common Variable Object Attributes

Attribute name	Index	Format
Variable Type Identifier	-20	Unsigned8
Octet Length	-24	Integer32
Read enable	-28	Boolean
Write enable	-29	Boolean
Write protected	-30	Boolean

The Key attribute, Variable Identifier, is used to identify the Variable Object, and is not an accessible attribute.

The attributes of a Variable Object may be accessed from the network. If a Variable Object does not support access of an attribute, it shall respond to an access attempt with the Errorcode "Variable Object ID error". Only one attribute can be accessed as a result of one service invocation.

There is one set of attributes for each Variable Object. This means, the subfields or elements of constructed variables do not have their own attributes.

Table 540 shows the variable identifier values for the Variable types.

Table 540 – Variable Type Identifiers

Variable type	Identifier
Boolean	43
Integer8	49
Integer16	34
Integer32	35
Unsigned8	48
Unsigned16	50
Float32	36
Float64	37
UNICODE Char	51
Complex	61
String	40
BitString	62
FIFO	63

Variable Objects of type FIFO shall have the mandatory additional attributes defined in Table 541.

Table 541 – FIFO Variable Object Attributes

Attribute name	Index	Format
Next Element In	-2	Integer16
Next Element Out	-4	Integer16
Free elements	-6	Integer16
Used elements	-8	Integer16
Reread	-9	Boolean
Rewrite	-10	Boolean

7.2.3 Error Codes

This subclause specifies the hexadecimal values for error codes in the error status parameter of the Variable ASE service RESPONSE. The possible values are shown in Table 542.

Table 542 – Error Codes

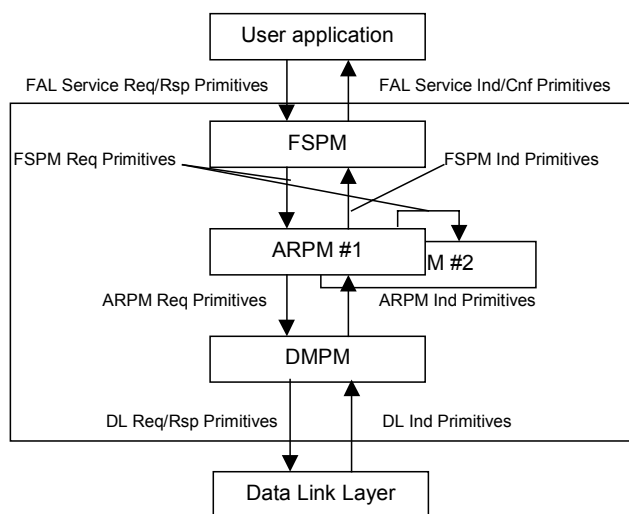
Error description	Error code (binary)
Instruction Error	01010000
Info length error	01001000
FIFO full or empty	00100000
Write protection	01000000
Data format error	00101000
Variable Object ID error	00110000
Time Out	00001000
Actual data error	x010xaaa
Historical data error	1xxxaaa
Route error	00111000
No response	00000000
Wait too long	00011000
Out of sync	10100000
CRC error	10000000
DLE not Client	10011000
Net shortcircuit	10010000
Overrun/Framing error	10001000
RS-232 handshake error	10101000
"x" means the bit is don't care.	
"aaa" means at least one of the three bits is true (1).	

7.3 FAL Protocol State Machines

Interface to FAL services and protocol machines are specified in the following sections.

The behaviour of the FAL is described by three integrated protocol machines. The three protocol machines are: the FAL Service Protocol Machine (FSPM), the Application Relationship Protocol

Machine (ARPM), and the Data Link Layer Mapping Protocol Machine (DMPM). The relationship among these protocol machines as well as primitives exchanged among them are depicted in Figure 153.

**Figure 153 – Summary of FAL Architecture**

The FSPM describes the service interface between the FAL user and a REP. The FSPM is responsible for the following activities:

- To accept service primitives from the FAL service user and convert them into FAL internal primitives.
- To select the AREP identified by the Destination Route parameter supplied by the user application and send FAL internal primitives to the selected AREP.
- To accept FAL internal primitives from the AREP, perform the actions specified in these primitives, and return the result to the AREP from which they are received. This applies for indications holding Request APDUs.
- To accept FAL internal primitives from the AREP, convert them into service, and deliver these service primitives to the FAL user, as a result of the FAL user invocation of the RESPONSE service. This applies for indications holding Response APDUs.

The ARPM describes exchange of FAL PDUs with remote ARPMs. It does not have any state changes. The ARPM is responsible for the following activities:

- To accept FAL internal primitives from the FSPM and create and send other FAL internal primitives to the DMPM.
- To accept FAL internal primitives from the DMPM and send them to the either the FSPM as indications, or another ARPM as requests.

The DMPM describes the mapping between the FAL and the DLL. It does not have any state changes. The DMPM is responsible for the following activities:

- To accept FAL internal primitives from the ARPM, prepare DLL service primitives, and
- To receive DLL indication primitives from the DLL and send them to the ARPM in a form of FAL internal primitives.

7.4 AP-Context State Machine

The AP-Context State Machine is not present.

7.5 FAL Service Protocol Machine (FSPM)

7.5.1 Primitives exchanged between FAL User and FSPM

The primitives exchanged between FAL user and FSPM are shown in Table 543.

Table 543 – Primitives exchanged between FAL-User and FSPM

Primitive name	Source	Associated parameters	Comments
REQUEST.req	FAL user	REP, Variable Object ID, Variable Service, Data length, Offset/Attribute, Bit-no, Data	This primitive is used to convey a FAL user request to a REP
RESPONSE.cnf	FSPM	REP, Variable Service, Data length, Error status, Data	This primitive is used to convey a response from a REP to the FAL user

7.5.2 FSPM states

7.5.2.1 General

The following states are defined for an REP: IDLE RESERVED, WAITING FOR RESPONSE, RESPONSE RECEIVED, NOT IN USE.

An REP can be in the role of Proxy object (Client) or Real object (Server). As the behaviour is very different for Proxy object REPs and Real object REPs, they are described in separate subclauses.

7.5.2.2 FSPM proxy object states

7.5.2.2.1 Overview

The following states apply to a REP in the Proxy object role:

NOT IN USE

The REP is currently not in use. The only service primitive allowed is ReserveREP.req. All other primitives shall be rejected.

IDLE RESERVED

The REP is reserved by the FAL user, but is currently not active. Allowed service primitives in this state are REQUEST.req, Get REP Attribute.req, Set REP Attribute.req and Free REP.req. All other service primitives shall be rejected.

WAITING FOR RESPONSE

The REP has initiated a transmission, and is monitoring this by a timer. The only service primitive allowed is AR Send.ind from the ARPM, holding the response. All other service primitives shall be rejected.

RESPONSE RECEIVED

The REP has received a response (by an AR Send.ind from the ARPM), or has timed out, and is ready to deliver the response to the FAL user by a RESPONSE.cnf service primitive. Allowed service primitives in this state are REQUEST.req, Get REP Attribute.req, Set REP Attribute.req and Free REP.req from the FAL user. All other service primitives shall be rejected.

Figure 154 depicts the FSP Proxy object state machine.

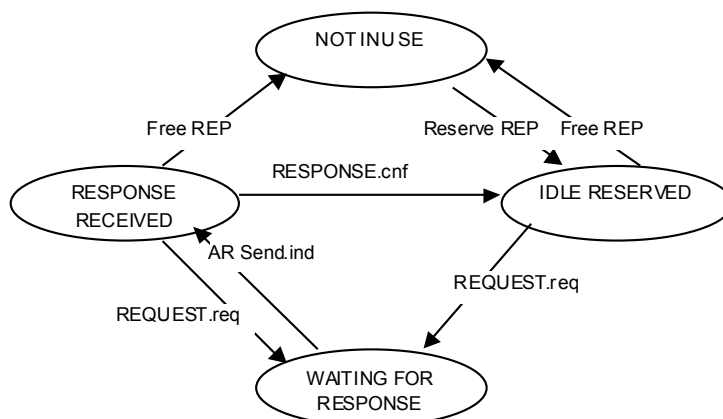


Figure 154 – FSPM proxy object state machine

7.5.2.2.2 Sender state transitions

7.5.2.2.2.1 REQUEST.req

As a result of this service invocation, the FSPM shall build APDU Header, APDU Body and Route Information, and send them to the ARPM in the form of an AR Send primitive. If anything fails, the request is rejected by returning REQUEST result FAILURE. Constraints are listed in Table 544.

Table 544 – REQUEST.req FSPM constraints

Condition ID	Condition description
1	REP Attribute State shall be IDLE RESERVED or RESPONSE RECEIVED
2	REP Attribute Role shall be Proxy object
3	If REP attribute Confirmation indicates Confirmed, no elements of REP attribute Destination Route may hold the Broadcast address (126)
4	First element of REP attribute Destination Route shall hold the address of an AREP in state OPEN
5	If parameter Data length exceeds addressed AREP attribute MaxDataSize, Variable Service may only be Read or Write
6	If parameter Variable Service is Test-And-Set, Data length shall be 1
7	If parameter Bit-no indicates bit addressing, Data length shall be 1, and parameter Variable Service shall be Read, Write or Test-And-Set

If all of the above is fulfilled, the AR Send service primitives are built as described in Table 545.

Table 545 – REQUEST.req FSPM actions

Action ID	Action description
1	The REP attribute Destination Route is copied to Route info.Destination Route
2	The REP attribute Source Route is copied to Route info.Source Route
3	The REP attribute Endpoint Address is appended to Route info.Source Route
4	If the REP attribute Confirmation indicates Unconfirmed, and no elements of Destination route holds the Broadcast node address (126), the No response node address (0) is appended to Route info.Source Route
5	The REP attribute Priority is copied to Route info.Priority
6	The REP attribute Remote LAN Server ID (if present) is copied to Route info. Remote LAN Server ID
7	Set internal variable Local Bit-no to indicate not bit-addressing, and copy parameter Data length to local variable Data length
8	Handle bit-addressing as described in the following, if parameter Bit-no indicates bit-addressing, and REP attribute Capabilities indicates bit addressing is illegal : Save parameter Bit-no in internal variable Local Bit-no, change parameter Bit-no to indicate not bit-addressing, and increment Local Bit-no by one If Variable Service parameter is Write and parameter Data.Bit1 is TRUE then Set APDU Header.ControlStatus.Instruction to Or, and clear all bits in parameter Data and set bit indicated by Local Bit-no If Variable Service parameter is Write and parameter Data.Bit1 is FALSE then Set APDU Header.ControlStatus.Instruction to And, and Set all bits in parameter Data and clear bit indicated by Local Bit-no If Variable Service parameter is And, Or or Test-And-Set then Set APDU Header.ControlStatus.Instruction to And, Or, Test-And-Set respectively, and rotate parameter Data.Bit1 to position indicated by Local Bit-no If Variable Service parameter is Read then Set APDU Header.ControlStatus.Instruction to Read
9	If parameter Data length exceeds addressed AREP attribute MaxDataSize, and parameter Variable Service is Read, set APDU Header.ControlStatus.Instruction to Segmented Read
10	If parameter Data length exceeds addressed AREP attribute MaxDataSize, and parameter Variable Service is Write, set APDU Header.ControlStatus.Instruction to Segmented Write
11	If actions 8, 9 and 10 do not apply, set then set APDU Header ControlStatus.Instruction according to the following: Parameter Variable Service Write -> Write Parameter Variable Service Read -> Read Parameter Variable Service And -> And Parameter Variable Service Or -> Or Parameter Variable Service Test-And-Set -> Test-And-Set
12	If REP attribute Flat addressing indicates Flat addressing, set APDU Header ControlStatus.Addressing method to Flat. If not, set to Variable Object addressing

Action ID	Action description
13	If parameter Bit-no indicates bit-addressing, or the value of parameter Variable Object ID is lower than –32768 or higher than +32767, or the value of parameter Offset/Attribute is lower than –32768 or higher than +32767, then set APDU Header DataFieldFormat.Variable Identifier Format to Complex, and set APDU Header Data length to 4. If this does not apply, then set APDU Header DataFieldFormat.Variable Identifier Format to Simple, and set APDU Header Data length to 2.
14	If the value of parameter Offset/Attribute is zero, set APDU Header DataFieldFormat.Offset/Attribute to indicate no Offset/Attribute. If not, set to indicate Offset/Attribute
15	If the value of parameter Offset/Attribute is zero, set APDU Header DataFieldFormat.Offset/Attribute to indicate no Offset/Attribute. If not, set to indicate Offset/Attribute
16	If parameter Bit-no indicates bit-addressing, set APDU Body Variable Identifier.Code to indicate Bit-addressing, and insert Bit-no
17	If the value of parameter Offset/Attribute is lower than –32768 or higher than +32767, set APDU Body Variable Identifier.Code to indicate Integer32 Offset/Attribute size
18	If the value of parameter Offset/Attribute is not zero, and within the range of Integer16, copy lower 2 octets to APDU Body Offset/Attribute, and increment APDU Header Data length by 2. If this does not apply, copy parameter Offset/Attribute to APDU Body Offset/Attribute, and increment APDU Header Data length by 4
19	If APDU Header.ControlStatus.Instruction indicates Segmented Read, set RequestedLength in APDU Body Data first octet to "addressed AREP attribute MaxDataSize", if that value is less than or equal to 256, and set APDU Body Data first 2 octets to "addressed AREP attribute MaxDataSize", if that value is higher than 256. Set next octet of APDU Body Data to 0, indicating that this is the first request of a segmented transaction. Increment APDU Header Data length by 2 or 3, depending on size of RequestedLength. Decrement local variable Data length by "addressed AREP attribute MaxDataSize"
20	If APDU Header.ControlStatus.Instruction indicates Segmented Write, copy the first "addressed AREP attribute MaxDataSize – 2" octets from parameter Data to APDU Body Data. Set next octet of APDU Body Data to 0, indicating that this is the first request of a segmented transaction. Increment APDU Header Data length by "addressed AREP attribute MaxDataSize – 1". Decrement local variable Data length by "addressed AREP attribute MaxDataSize – 2"
21	If APDU Header.ControlStatus.Instruction indicates Read, set RequestedLength in APDU Body Data first octet to the value of parameter Data length, and increment APDU Header Data length by 1
22	If APDU Header.ControlStatus.Instruction indicates Write, And, Or or Test-And-Set, copy "parameter Data length" octets from parameter Data to APDU Body Data set, and increment APDU Header Data length by the value of "parameter Data length"
23	Send an AR Send request to the addressed AREP
24	If the request fails, return REQUEST.cnf result FAILURE. If the request succeeds, return REQUEST.cnf result OK
25	If the request succeeds, and should be confirmed, set REP Attribute State to WAITING FOR RESPONSE.
26	If the request succeeds, and should not be confirmed, and is a sequenced transaction, build the next APDU in the sequence. This continues, until the complete operation is performed, or an error occurs

7.5.2.2.3 Receiver state transitions

7.5.2.2.3.1 RESPONSE.cnf

As a result of this service invocation, the FSPM shall check, if a response has been received from the ARPM (FSPM state RESPONSE RECEIVED). If this is the case, deliver the received Variable Service, Data length, Error status and Data to the FAL user. If this is not the case, deliver the Error status "No response". Constraints are listed in Table 546.

Table 546 – RESPONSE.cnf FSPM constraints

Condition ID	Condition description
1	REP Attribute State shall be RESPONSE RECEIVED

If the above is fulfilled, the RESPONSE.cnf primitives are built as described in Table 547.

Table 547 – RESPONSE.cnf FSPM actions

Action ID	Action description
1	Copy lower 3 bits of local variable ControlStatus to parameter Variable Service
2	Copy local variable Data length to parameter Data length
3	Copy local variable ControlStatus to parameter Error status
4	Copy local variable Data to parameter Data
5	Set REP Attribute State to IDLE RESERVED

7.5.2.2.3.2 AR Send.ind

As a result of this service invocation, the FSPM shall check, if a response is expected from the ARPM (FSPM state WAITING FOR RESPONSE). If this is the case, either parse the received APDU into local variables and change REP state to RESPONSE RECEIVED, or initiate the next AR Send.req if sequenced transaction. If anything fails, do nothing. Constraints are listed in Table 548.

Table 548 – AR Send.ind proxy FSPM constraints

Condition ID	Condition description
1	REP Attribute State shall be WAITING FOR RESPONSE

If the above is fulfilled, the AR Send indication is handled as described in Table 549.

Table 549 – AR Send.ind proxy FSPM actions

Action ID	Action description
1	If APDU Header.ControlStatus.Instruction indicates Sequenced Read or Sequenced Write, and this is not the final, build next APDU, and Send a new AR Send request to the addressed AREP. If sequenced Read copy received APDU Body Data to local variable Data.
2	If not sequenced transaction, or sequenced transaction finished, copy received APDU Body Data to local variable Data. If Read or Test-And-Set and bit-addressing, copy received APDU Body Data bit "local variable Bit-no" to bit 1 of local Variable Data. Copy received APDU Header ControlStatus to local variable ControlStatus. Copy accumulated received APDU Header Data length to local variable Data length. Set REP Attribute State to IDLE RESERVED

7.5.2.3 FSPM real object state machine description**7.5.2.3.1 Overview**

The following states apply to a REP in the Real object role:

NOT IN USE

The REP is currently not in use. The only service primitive allowed is ReserveREP.req. All other service primitives shall be rejected. Typically, an REP in the Real object role will never be in this state, but will automatically go into the IDLE RESERVED state.

IDLE RESERVED

Typically the initial state of an REP in the Real object role. The REP is reserved by the FAL user, or entered this state automatically, but is not currently active. Allowed service primitives in this state for Real object role REPs are Get REP Attribute.req, Set REP Attribute.req and Free REP.req from the FAL user, and AR Send.ind from the ARPM. All other service primitives shall be rejected.

WAITING FOR RESPONSE

An REP in the Real Object role will never go into this state.

RESPONSE RECEIVED

An REP in the Real Object role will never go into this state.

Figure 155 depicts the states of the FSPM Real object state machine.

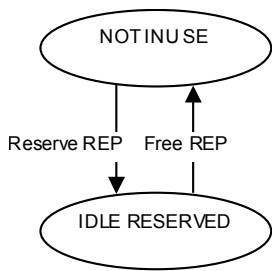


Figure 155 – FSPM real object state machine

7.5.2.3.2 State transitions

7.5.2.3.2.1 AR Send.ind

As a result of this service invocation, the FSPM shall check, if it is ready to receive an indication from the ARPM (FSPM state IDLE RESERVED). If this is the case, check the received APDU, and if OK interact directly with the addressed Variable Object, which returns the result. Finally, the FSPM shall issue an AR Send.req primitive, to deliver the result. Constraints are listed in Table 550.

Table 550 – AR Send.ind real FSPM constraints

Condition ID	Condition description
1	REP Attribute State shall be IDLE RESERVED
2	APDU Body field Variable Object Identifier shall indicate a legal Variable Object

If the above is fulfilled, the AR Send indication is handled as described in Table 551.

Table 551 – AR Send.ind real FSPM Actions

Action ID	Action description
1	If the addressed Variable Object cannot respond within the time specified by AREP Attribute MaxIndicationDelay, issue an AR Acknowledge service primitive. The Destination Route parameter shall be a copy of the Source Route parameter of the AR Send indication. The Source Route parameter shall be a copy of the Destination Route parameter of the AR Send indication. The Priority parameter shall be 0. The Confirmation parameter shall be Unconfirmed. The Remote LAN Server ID parameter shall be a copy of the Remote LAN Server ID parameter of the AR Send indication.
2	Interact directly with the addressed Variable Object, which shall build the appropriate response APDU.
3	Issue an AR Send service primitive. The Destination Route parameter shall be a copy of the Source Route parameter of the AR Send indication. The Source Route parameter shall be a copy of the Destination Route parameter of the AR Send indication. The Priority parameter shall be 0. The Confirmation parameter shall be Unconfirmed. The Remote LAN Server ID parameter shall be a copy of the Remote LAN Server ID parameter of the AR Send indication. APDU Header and APDU Body parameters shall be as build by the Variable Object.

7.6 Application Relationship Protocol Machine (ARPM)

7.6.1 Primitives exchanged between ARPM and FSPM

The primitives exchanged between ARPM and FSPM, and between one ARPM and another are shown in Table 552 through Table 554.

Table 552 – Primitives issued by FSPM to ARPM

Primitive name	Source	Associated parameters	Comments
AR Send.req	FSPM	Route info, APDU Header, APDU Body	This primitive is used to convey an APDU holding a request or a response from the FSPM to the ARPM
AR Acknowledge.req	FSPM	Route info	This primitive is used by the FSPM to indicate, that the Variable Object is not able to handle the preceding indication immediately

Table 553 – Primitives issued by ARPM to FSPM

Primitive name	Source	Associated parameters	Comments
AR Send.ind	ARPM	Route info, APDU Header, APDU Body	This primitive is used to convey an APDU holding a request or a response from the ARPM to the FSPM

Table 554 – Primitives issued by ARPM to ARPM

Primitive name	Source	Associated parameters	Comments
AR Send.ind	ARPM	Route info, APDU Header, APDU Body	This primitive is used to convey an APDU holding a request or a response from the ARPM to another ARPM

7.6.2 ARPM States

7.6.2.1 General

The following states are defined for an AREP: OPEN, as depicted in Figure 156.

7.6.2.2 Overview



Figure 156 – ARPM State Machine

OPEN

The AREP is initialised, and ready to accept service primitives.

7.6.2.3 Sender state transitions

7.6.2.3.1 AR Send.req

As a result of this service invocation, the ARPM shall deliver the received parameters to the addressed DLPM by a new AR Send service invocation. If this is not possible, the request is rejected by returning AR Send result Route error.

Constraints are listed in Table 555.

Table 555 – AR Send.req ARPM Constraints

Condition ID	Condition description
1	AREP Attribute State shall be OPEN
2	First address of parameter Route info, Destination route shall specify a DLPM in state OPEN

If the above is fulfilled, the AR Send parameters are delivered to the DLPM as described in Table 556.

Table 556 – AR Send.req ARPM Actions

Action ID	Action description
1	Issue an AR Send service primitive to the DLPM, with the same parameters as this indication

7.6.2.3.2 AR Acknowledge.req

As a result of this service invocation, the ARPM shall deliver the received parameters to the addressed DLPM by a new AR Acknowledge service invocation. If this is not possible, the request is rejected.

Constraints are listed in Table 557.

Table 557 – AR Acknowledge.req ARPM Constraints

Condition ID	Condition description
1	AREP Attribute State shall be OPEN
2	First address of parameter Route info, Destination route shall specify a DLPM in state OPEN

If the above is fulfilled, the AR Acknowledge parameters are delivered to the DLPM as described in Table 558.

Table 558 – AR Acknowledge.req ARPM Actions

Action ID	Action description
1	Issue an AR Acknowledge service primitive to the DLPM, with the same parameters as this indication

7.6.2.4 Receiver state transitions

7.6.2.4.1 AR Send.ind

As a result of this service invocation, the ARPM shall deliver the received parameters to the addressed destination by a new AR Send service invocation. The addressed destination may be a REP or another AREP. If it is not possible to deliver the received parameters, the request is rejected by returning AR Send result Route error.

Constraints are listed in Table 559.

Table 559 – AR Send.ind ARPM Constraints

Condition ID	Condition description
1	AREP Attribute State shall be OPEN
2	First address of parameter Route info, Destination route shall either indicate an REP in state IDLE RESERVED or WAITING FOR RESPONSE, or an AREP in state OPEN

If the above is fulfilled, the AR Send parameters are delivered to the FSPM as described in Table 560.

Table 560 – AR Send.req ARPM Actions

Action ID	Action description
1	If the first address of parameter Route info, Destination route indicates an REP, issue an AR Send service primitive to the addressed REP, with the same parameters as this indication
2	If the first address of parameter Route info, Destination route indicates another AREP, issue an AR Send service primitive to the addressed AREP with the same parameters as this indication, and issue an AR Acknowledge primitive to the DLPM from which this indication was received, with the Route info, Destination route parameter equal to the Route info, Source route parameter of this indication, and the Route info, Source route parameter equal to the Route info, Destination route parameter of this indication

7.7 DLL Mapping Protocol Machine (DMPM)

7.7.1 Data Link Layer service selection

7.7.1.1 General

This subclause briefly describes the Data Link Layer services utilized by the FAL. These Data Link Layer services are fully defined in the Data Link Layer service specification (IEC 61158-3, Type 4).

7.7.1.2 DL-UNITDATA request

This service is used to transmit an APDU from an AREP, and deliver it as a DLSDU to a DLE.

7.7.1.3 DL-UNITDATA indication

This service is used to receive a DLSPDU from a DLE and deliver it as an APDU to an AREP.

7.7.1.4 DL-UNITDATA response

This service is used to inform the DLE, that a response will not be prepared in time.

7.7.1.5 DLM-Set primitive and parameters

This service is used to update the values of attributes of the DLE locally.

7.7.1.6 DLM-Get primitive and parameters

This service is used to read the values of attributes of the DLE locally.

7.7.2 Primitives exchanged between ARPM and DLPM

The primitives exchanged between DLPM and ARPM are shown in Table 561 through Table 562.

Table 561 – Primitives issued by ARPM to DLPM

Primitive name	Source	Associated parameters	Comments
AR Send.req	ARPM	Route info, APDU Header, APDU Body	This primitive is used to convey an APDU holding a request or a response from the ARPM to the DLPM
AR Acknowledge.req	ARPM	Route info	This primitive is used by the ARPM to indicate, that the Variable Object is not able to handle the preceding indication immediately

Table 562 – Primitives issued by DLPM to ARPM

Primitive name	Source	Associated parameters	Comments
AR Send.ind	ARPM	Route info, APDU Header, APDU Body	This primitive is used to convey an APDU holding a request or a response from the DLPM to the ARPM

7.7.3 Primitives exchanged between DLPM and Data Link Layer

The primitives exchanged between DLPM and ARPM are shown in Table 563 through Table 564.

Table 563 – Primitives issued by DLPM to Data Link Layer

Primitive name	Source	Associated parameters	Comments
DL-UNITDATA request	DLPM	Destination-DL-route, Source-DL-route, Priority, Maximum retry time, Control-status, Data-field-format, DLSDU	This primitive is used to convey a DLSDU holding a request or a response to the DLE
DL-UNITDATA response	DLPM	Destination-DL-route, Source-DL-route	This primitive is used to indicate to the DLE, that the preceding indication cannot be handled immediately

Table 564 – Primitives issued by Data Link Layer to DLPM

Primitive name	Source	Associated parameters	Comments
DL-UNITDATA indication	DLL	Destination-DL-route, Source-DL-route, Confirmation expected, Control-status, Data-field-format, DLSDU	This primitive is used to convey a DLSDU holding a request or a response from the DLE to the DLPM

7.7.4 DLPM states

7.7.4.1 States

The following states are defined for a DLPM: OPEN, as depicted in Figure 157.

7.7.4.2 Overview

**Figure 157 – DLPM State Machine**

OPEN

The DLPM is initialised, and ready to accept service primitives.

7.7.4.3 Sender state transitions

7.7.4.3.1 AR Send.req

As a result of this service invocation, the DLPM shall deliver the received parameters to the DLE by a DL-UNITDATA request primitive. If this is not possible, the request is rejected.

Constraints are listed in Table 565.

Table 565 – AR Send.req DLPM Constraints

Condition ID	Condition description
1	DLPM state shall be OPEN

If the above is fulfilled, the AR Send parameters are converted and delivered to the DLE as described in Table 566.

Table 566 – AR Send.req DLPM Actions

Action ID	Action description
1	First address of parameter Route info, Destination Route, is removed. The result is delivered as the Destination-DL-route parameter of the DL-UNITDATA request primitive
2	Parameter Route info, Source Route is delivered as the Source-DL-route parameter of the DL-UNITDATA request primitive
3	Parameter Route info, Priority is delivered as the Priority parameter of the DL-UNITDATA request primitive
4	AREP attribute MaxRetryTime is delivered as the Maximum retry time parameter of the DL-UNITDATA request primitive
5	Parameter APDU Header, ControlStatus is delivered as the Control-status parameter of the DL-UNITDATA request primitive
6	Parameter APDU Header, DataFieldFormat is delivered in bits 7 and 8, and APDU Header, DataLength is delivered in bits 1 to 6 of the Data-field-format parameter of the DL-UNITDATA request primitive
7	Parameter APDU Body is delivered as the DLSDU parameter of the DL-UNITDATA request primitive

7.7.4.3.2 AR Acknowledge.req

As a result of this service invocation, the DLPM shall deliver the received parameters to the DLE by a DL-UNITDATA response primitive. If this is not possible, no action is taken.

Constraints are listed in Table 567.

Table 567 – AR Acknowledge.req DLPM Constraints

Condition ID	Condition description
1	DLPM state shall be OPEN

If the above is fulfilled, the AR Acknowledge parameters are converted and delivered to the DLE as described in Table 568.

Table 568 – AR Acknowledge.req DLPM Actions

Action ID	Action description
1	First address of parameter Route info, Destination Route, is removed. The result is delivered as the Destination-DL-route parameter of the DL-UNITDATA response primitive
2	Parameter Route info, Source Route is delivered as the Source-DL-route parameter of the DL-UNITDATA response primitive

7.7.4.4 Receiver state transitions

7.7.4.4.1 DL-UNITDATA indication

As a result of a DL-UNITDATA indication service invocation, the DLPM shall deliver the parameters received from the DLE to the AREP by an AR Send service invocation. If this is not possible, the request is rejected.

Constraints are listed in Table 569.

Table 569 – DL-UNITDATA.ind DLPM Constraints

Condition ID	Condition description
1	AREP Attribute State shall be OPEN

If the above is fulfilled, the AR Send parameters are delivered to the ARPM as described in Table 570.

Table 570 – DL-UNITDATA.ind DLPM Actions

Action ID	Action description
1	AREP address is inserted in front of all elements in the parameter Source-DL-route. The result is delivered as the Route info, Source route parameter of the AR Send primitive
2	Parameter Destination-DL-route is delivered as the Route info, Source route parameter of the AR Send primitive
3	Parameter Route info, Priority is set to 0
5	Parameter Confirmation Expected is delivered as the Confirmation parameter of the AR Send primitive
6	Parameter Control-status is delivered as the APDU Header, ControlStatus parameter of the AR Send primitive
7	Bits 1 to 6 of the Data-field-format parameter are delivered as the APDU Header, DataLength parameter of the AR Send primitive
8	Bits 7 and 8 of the Data-field-format parameter are delivered as the APDU Header, DataFieldFormat parameter of the AR Send primitive
9	Parameter DLSDU is delivered as the APDU Body parameter of the AR Send primitive

7.8 Protocol options

There are no options to select for Type 4.

8 Type 5

8.1 Overview

The Type 5 services and protocols are provided by the Type 5 AE, referred to as the *Field Device Access (FDA) Agent* throughout this specification.

The FDA Agent conveys Type 9 APDUs using Type 9 Application Relationships, referred to as *FDA Sessions*, or more simply as *sessions* throughout this Type 5 specification.

The FDA Agent may provide gateway functions that map between Type 5 services and corresponding services that operate over a Type “X” DLL. Networks that employ the Type “X” DLL are referred to as *Type “X” networks*.

8.2 FAL Syntax Description

8.2.1 PDU abstract syntax

The abstract syntax of APDUs is combined with their transfer syntax and is specified in 8.3.

8.2.2 Abstract syntax of data types

The abstract syntax of data types is combined with their transfer syntax and is specified in 8.3.

8.3 Transfer syntax

8.3.1 General

The transfer syntax combines the specification of the abstract syntax and their encodings as a set of fixed format APDUs. Each APDU contains a header and an APDU body. An optional trailer is also defined. The presence of the trailer is indicated in the APDU header.

The format of the APDU header and trailer is specified in 8.3.3 and 8.3.4. The APDU header contains the fields common to all messages that are mandatory. The APDU trailer contains the fields common to all APDUs that are optional.

The formats of the APDU bodies are specified in 8.3.5. If a service request or response APDU has no parameters, it is stated in the subclause that describes the APDU body. If the service has no response or error APDU body associated with it, then the corresponding subclause is not present. Common error parameters are used in many service-specific APDUs. Their definitions are specified in 8.3.6.1.

User data, when present in an APDU, is defined as a service-specific APDU body parameter. The content and length of the user data are dependent upon the service and the object being accessed. Therefore, neither is specified as part of the APDU body. However, the user data length can be calculated from the APDU length contained in the APDU header.

The APDU header, trailer, and bodies are composed of a subset of data types defined in IEC 61158-5. The encodings for these data types are defined next, followed by the definitions of the header, trailer, and APDU bodies.

8.3.2 Data type encodings

The following data types are used in APDUs as listed in Table 571. Other data types may be conveyed as user data.

Table 571 – Data Types

Data Type	Octet Length	Comment
Boolean	1	0 = False, Non-Zero = TRUE
Integer8	1	
Integer16	2	
Unsigned8	1	
Unsigned16	2	
Unsigned32	4	The length of a VisibleString is defined by the field definition. The length of an OctetString is defined by the field definition.
VisibleString	-	
OctetString	-	
Time Value	8	

For all data types, the bit and octet order of transmission is as defined in Type 9. The unsigned integer value for each bit position is shown in Table 572.

Table 572 – Data Types

Bit Position	Hex Value	Decimal Value
8	0x80	128
7	0x40	64
6	0x20	32
5	0x10	16
4	0x08	8
3	0x04	4
2	0x02	2
1	0x01	1

8.3.3 Type 5 Header

8.3.3.1 APDU header format

The header is the same for each APDU and has a fixed length.

All header fields are mandatory.

The format of the header is shown in Table 573.

Table 573 – Type 5 APDU Header Format

Parameter name	Octet offset	Data type	Octet length	Description
Type 5 APDU Version	0	Unsigned8	1	Specifies the Version number of the Type 5 APDU format. The version number of the Type 5 APDUs specified in this document is 1. 1 = Version 1
Options	1	Unsigned8	1	Bit 8: 1 = APDU Number present in the Trailer Bit 7: 1 = Invoke Id present in the Trailer (this bit shall be set for all client / server sessions) Bit 6: 1 = Time Stamp present in the Trailer Bit 5: Reserved, set to 0 Bit 4: Extended Control Field present in the Trailer Bit 3-1: Reserved, set to 0
ASE Id And Confirmed Msg Type	2	Unsigned8	1	Bits 8-3: ASE Id – Used to specify the ASE whose service is being conveyed by the APDU. They are interpreted as a 6-bit unsigned integer with the following values: 0 = Unused 1 = AR ASE 2 = SMK ASE 3 = Type 9 ASEs/Type 5 VFD ASE 4 = LAN Redundancy ASE 5-63 : Reserved Bits 1, 2: Confirmed Msg Type – Used to differentiate between request, response, and error APDUs. Not used with unconfirmed services. These bits are interpreted as a 2-bit unsigned integer with the following values: 0 = Request APDU 1 = Response APDU 2 = Error APDU 3 = Reserved
Service	3	Unsigned8	1	Specifies the service within the specified protocol. Bit 8: Confirmed Flag (Confirmed=1) and (Unconfirmed=0). When this flag is set for a request APDU, the corresponding response (either a response APDU or an error APDU) is always returned after the request has been processed to indicate the result of the processing. Bits 7-1 Service Id of the service (7-bit unsigned integer). The value used for each Service Id is specified in its APDU Body specification below. It is the number in parenthesis that follows the service name in the header for each service. All other values are reserved.
FDA Address	4	Unsigned32	4	The format and use of the FDA Address is dependent on the ASE Id and the type of S-VFD Context. The next subclause describes this field.
APDU Length	8	Unsigned32	4	Specifies the number of octets contained in the entire APDU, including the header and the trailer.

8.3.3.2 FDA address use**8.3.3.2.1 FDA address use summary**

This subclause specifies the values for the FDA Address field of the header. See Table 574 for details. In the descriptions that follow, a Type 9 device is a device with the Type 9 AL and a DLL whose addressing can be represented as a Type 1 DLL address. A Type 9 SMK is the SMK in a Type 9 device.

Table 574 – FDA Address Use

ASE	VCR	Bits	Use
AR	N/A	32-1	Service-specific.
LAN Redundancy	N/A	32-1	Not Used, set to 0.
SMK	N/A	32-17 16-1	Link Id Set to 0 if the destination SMK is located in the Type 5 device receiving the APDU. If the device is a Linking Device, and the destination SMK is in a Type 9 device attached to the Linking Device, then the FDA Address field contains the link id used to access the Type 9 device. Node.Selector The node value is the high order 8 bits and the selector value is the low order 8 bits, when taken as a 16-bit unsigned integer value. When used to identify a Type 5 SMK (when the Link Id is zero), the node value is set to 0 and the selector value is set to 2. When used to identify an Type 9 SMK (when the Link Id is non-zero), this is the individual or group Type 9 node.selector address. See the description of the Find Tag Query APDU for a detailed description of its use of the FDA Address.
Type 9	Client/Server	32-17 16-1	Link Id If the server S-VFD is located in the Type 5 device, then this is 0. If the device is a Linking Device, and the destination VFD is in a Type 9 device attached to the Linking Device, then this is the Link Id of the link to which the Type 9 device is attached. Type 5 Selector Initiate Request: The use of this field is dependent on the Connect Option selected for the request. See the description of the Initiate APDU for the values used. Initiate Response: The Type 5 FAL AE returns the VCR Id of the initiated VCR. All Others: The Selector returned in the Initiate Response.
Type 9	Publisher / Subscriber	32-17 16-1	Link Id If the publisher VFD is located in the Type 5 device sending the APDU, then this is combined with the Selector bits to form a 32-bit flat Dlcep address. If the device is a Linking Device, and the publisher is in a Type 9 device attached to the Linking Device, then this is the Link Id of the link to which the publishing device is attached. Dlcep Selector If the publisher is located in the Type 5 device sending the APDU, then this is part of the 32-bit flat Dlcep address assigned to the published data. If the device is a Linking Device, and the publisher is in a Type 9 device attached to the Linking Device, then this is the Dlcep Selector of the publisher buffer.
Type 9	Report Distribution	32-17 16-1	Link Id If the report source VFD is located in the Type 5 device sending the APDU, then this is combined with the Selector bits to form a 32-bit flat DLSAP address. If the device is a Linking Device, and the report source is in a Type 9 device attached to the Linking Device, then this is the Link Id of the link to which the reporting device is attached. S-VFD Context Id or Dlsap Selector If the report source VFD is located in the Type 5 device sending the APDU, then this is part of the 32-bit flat Dlsap address assigned to the Report Source VFD. If the device is a Linking Device, and the report source is in a Type 9 device attached to the Linking Device, then this is the Dlsap Selector of the report source.
NOTE The Link Id, Node, Dlcep Selector, and Dlsap Selectors are all Type 1 DLL address components.			

8.3.3.2.2 APDU sent by a client VCR endpoint

Table 575 illustrates the use of the FDA Address header field in APDUs sent by a client VCR endpoint.

Table 575 – FDA Address header field APDUs sent by a client VCR endpoint

Message type	Destination AP	FDA address	
		Bits	Value
Type 5 Initiate Request APDU Connect Option 1	Non-MIB VFD of Type 5 device	Bits 32-17 Bits 16-1	0 (Local Type 5 Device) Generic Selector, Returned by Find Tag Query, or obtained by some other means
	Non-MIB VFD of Type 9 device connected to Linking Device	Bits 32-17 Bits 16-1	Link Id of Type 9 Link Client Type 9 VCR Index
Type 5 Initiate Request APDU Connect Option 2	MIB VFD of Type 5 device	Bits 32-17 Bits 16-1	0 (Local Type 5 Device) 0.0
	MIB VFD of Linking Device Type 9 Link Interface	Bits 32-17 Bits 16-1	Link Id of Type 9 Link 0.0
	MIB VFD of Type 9 device connected to Linking Device	Bits 32-17 Bits 16-1	Link Id of Type 9 Link Node.0
All Other Request Messages	MIB VFD of Type 5 device	Bits 32-1	FDA Address returned in Initiate Response message
	Non-MIB VFD	Bits 32-1	FDA Address returned in Initiate Response message
	MIB VFD of Linking Device Type 9 Link Interface	Bits 32-1	FDA Address returned in Initiate Response message
	MIB VFD of Type 9 device connected to Linking Device	Bits 32-1	FDA Address returned in Initiate Response message
	Non-MIB AP of Type 9 device connected to Linking Device	Bits 32-1	FDA Address returned in Initiate Response message

8.3.3.2.3 APDUs sent by a server AR VCR endpoint

Table 576 illustrates the use of the FDA Address header field in APDUs sent by a server VCR endpoint.

Table 576 – FDA Address header field APDUs sent by a server VCR endpoint

Message type	Destination AP	FDA address	
		Bits	Value
Type 5 Initiate Response APDU Connect Option 1	Non-MIB VFD of Type 5 device	Bits 32-17 Bits 16-1	0 (Local Type 5 Device) Type 5 VCR Id ¹
	Non-MIB VFD of Type 9 device connected to Linking Device	Bits 32-17 Bits 16-1	Link Id of Type 9 Link Type 5 VCR Id ¹
Type 5 Initiate Response APDU Connect Option 2	MIB VFD of Type 5 device	Bits 32-17 Bits 16-1	0 (Local Type 5 Device) Type 5 VCR Id ¹
	MIB VFD of Linking Device Type 9 Link Interface	Bits 32-17 Bits 16-1	Link Id of Type 9 Link Interface Type 5 VCR Id ¹
	MIB VFD of Type 9 device connected to Linking Device	Bits 32-17 Bits 16-1	Link Id of Type 9 Link Type 5 VCR Id ¹
	Non-MIB VFD of Type 9 device connected to Linking Device	Bits 32-1	Same as Request Message Same as Request Message
All Other Response Messages	MIB VFD of Type 5 device	Bits 32-1	Same as Request Message Same as Request Message
	Non MIB VFD of Type 5 device	Bits 32-1	Same as Request Message Same as Request Message
	MIB VFD of Linking Device Type 9 Link	Bits 32-1	Same as Request Message Same as Request Message
	MIB VFD of Type 9 device connected to Linking Device	Bits 32-1	Same as Request Message Same as Request Message
	Non-MIB VFD of Type 9 device connected to Linking Device	Bits 32-1	Same as Request Message Same as Request Message

NOTE The selector value (bits 17-32) returned is the Type 5 VCR Index of the dynamically created VCR.

8.3.3.2.4 APDUs sent by a Publisher VCR Endpoint

Table 577 illustrates the use of the FDA Address in messages sent by a publisher VCR endpoint.

Table 577 – FDA Address header field APDUs sent by a publisher VCR endpoint

Message type	Publisher AP	FDA address	
		Bits	Value
Type 9 Request Message	AP in the Type 5 device	Bits 32-1	Flat 32-bit Type 9 address assigned to the published data buffer
	VFD of Type 9 device connected through a Type 9 link of a Linking Device	Bits 32-17	Link Id of Type 9 Link
		Bits 16-1	Node.Selector Dlcep of the published data buffer

8.3.3.2.5 Messages sent by a Report Source VCR Endpoint

Table 578 illustrates the use of the FDA Address in APDUs sent by a report source VCR endpoint.

Table 578 – FDA Address header field APDUs sent by a report source VCR endpoint

Message type	Report source AP	FDA address	
		Bits	Value
Type 9 Request Message	AP in the Type 5 device	Bits 32-1	Flat 32-bit Type 9 address assigned to the report source VFD.
	VFD of Type 9 device connected through a Type 9 link of a Linking Device	Bits 32-17	Link Id of Type 9 Link
		Bits 16-1	Node.Selector Dlsep of the source of the report

8.3.4 Trailer

FDA Sessions define which trailer fields are defined for the Type 9 APDUs that they convey. The use of trailer fields by SMK ASE and LAN Redundancy APDUs is service specific. See the SMK ASE and LAN Redundancy APDU definitions for their use of trailer fields.

The presence of trailer fields is negotiated for Client / Server sessions and configured for Publisher / Subscriber and Report Distribution sessions. Their presence is indicated in the Message Header Options field of the each message. The fields that are present are positioned after the user data in the order shown in Table 579.

Table 579 – APDU Trailer Fields

Field name	Order	Data type	Octet length	Description
APDU Number	1	Unsigned32	4	Monotonically increasing by 1, 0 based, number of the APDU within a specific session.
Invoke Id	2	Unsigned32	4	Request/response identifier used to match responses with requests. It is required for all request / response exchanges. Invoke Ids are unique within the context of an Application Relationship.
Time Stamp	3	OctetString	8	System Time at which sender created APDU (in Time Value format). It can be used by receivers to calculate the one-way transfer time from the sender.
Extended Control Field	4	Unsigned32	4	Reserved For Future Use

8.3.5 APDU Body

8.3.5.1 General

Each service has a set of APDU bodies that are conveyed in a fixed format in each APDU body. The length of the APDU body formats may vary only if the last field of the APDU body repeats or its length is variable. Variable length fields are supported only as the last field of a APDU body (before the APDU Trailer) so that their offset from the beginning of the APDU body is constant.

The number in parenthesis that follows the name of the service in each subclause header below is the value that is used for the Service Id field in the Type 5 APDU Header.

8.3.5.2 FDA Session ASE Services

8.3.5.2.1 Open Session Service (Confirmed Service Id = 1)

8.3.5.2.1.1 Service Overview

This service is used to open a Client/Server AR between a client AR endpoint and a server AR endpoint. The source and destination address pair identifies each AR endpoint.

The Version field in the Type 5 APDU Header in the request message indicates the version of the FDA Agent that is being requested for the AR. The responder may negotiate the version down.

The Options field in the Type 5 APDU Header in the request indicates the options that are being requested for the AR. The Invoke Id bit is always set. The responder can refuse all other options. Returning 0 in the bit position representing the option indicates refusal.

If the message number option is requested, then the trailer field contains the initial value to be used by both endpoints of the session. This value is returned in the response message. If the requester does not receive a response to the request, and elects to resend the request, it increments the message number in the trailer, while keeping the invoke id the same as the original. This causes the request to be delivered to the AR endpoint created if the first request was received and a positive response was returned. If the original request was not received, or it caused a negative response to be returned (that was not received), then the new request is treated like a new request because there is no existing AR endpoint to process the message.

The FDA Address in the APDU Header is not used and is set to 0.

Successful exchange of the request and response APDUs results in the establishment of an AR that may be used to send Type 9 requests and responses.

The server receives requests at a generic endpoint address and creates a new endpoint at a previously unused address to process each request and to return the response. The newly created endpoint is used for all subsequent APDUs sent and received on the AR.

The request APDU conveys AR attributes for validation and use by the responder. If the responder is able to support certain of these, it is permitted to negotiate them, as defined in Table 580. If the requester is not prepared to operate with the negotiated attributes returned by the responder, it is free to not open the AR.

If a request is received with invalid parameter or unsupported values, a negative response is returned with an error class of "service" and a error code of "parameter-inconsistent" for the offending parameter. This negative response also uses the additional code value of 1, and the additional description to propose an acceptable values for each of the parameters indicated below that can be used to open the session. To convey these values the 16 octets of the

additional description is interpreted as 4 Unsigned32 integers instead of as a VisibleString. The location of each parameter value in the 16 octets is specified in the descriptions below.

If the responder receives an Open Session Request APDU for an AR on which it has already sent a response message, it closes the AR.

If the responder does not have an available Session Index for the new session, then a negative response is returned with error class = "resource" and error code = "max sessions exceeded".

If the responder does not have the resources to support the new session, then a negative response is returned with error class = "resource" and error code = "object creation failure" or some other appropriate error code within the "resource" error class.

ARs opened for MIB VFD Configuration Use are referred to as configuration ARs. Only one configuration AR is allowed at a time. If a request is received to open a configuration AR and one is already open, or if there is an Type 9 VCR to any MIB VFD (Type 5 or Type 9) with update services supported already open, then the configuration AR is refused. The AR ASE returns a negative response with error class = "service" and error code = "config-access-already-open".

Opening Publisher / Subscriber and Report Distribution ARs is local, and does not result in the exchange of Open Session Service APDUs. As a result, the Open Session Request APDU parameters for these AR types are preconfigured instead of negotiated.

8.3.5.2.1.2 Request APDU Parameters

Table 580 – Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
AR Index	0	Unsigned32	4	This parameter contains the numeric identifier of the client AR endpoint, if one. If not, 0 is used. For the response, this parameter contains the numeric identifier of the newly opened AR.
Max Buffer Size	4	Unsigned32	4	This parameter defines the maximum buffer length in octets (the buffer may contain concatenated Type 9 APDUs) that the sender of this APDU may send or receive on this AR. This parameter can be negotiated down (but not up) by the responder using the Max Buffer Size parameter. If the negotiation fails for any parameter, an acceptable value for this parameter is returned in the additional description field as an Unsigned32 at offset 0. If the requested value is supported, but another parameter failed the negotiation, then the requested value is returned at offset 0.
Max Message Length	8	Unsigned32	4	This parameter defines the maximum length in octets of APDUs to be sent on this AR by the sender of this APDU. It is used by the receiving AR endpoint as its Max Message Length attribute. This parameter cannot be negotiated. The server may reject the value supplied by the client if it cannot support it using error class = "access" and error code = "unsupported max APDU length". If the negotiation fails for any parameter, an acceptable value for this parameter is returned in the additional description field as an Unsigned32 at offset 4. If the requested value is supported, but another parameter failed the negotiation, then the requested value is returned at offset 4.

Parameter name	Octet offset	Data type	Octet length	Description
Reserved	12	Unsigned8	1	Unused, set to 0.
Configuration Use	13	Unsigned8	1	0 = Configuration Not Permitted 1 = Configuration Permitted
Inactivity Close Time	14	Unsigned16	2	This parameter identifies how long, in seconds, that the AR stays open without receiving an APDU. After experiencing inactivity on the AR for this period of time, the AREP is closed, along with all VCR activity associated with the AREP. The responder can negotiate this value down. The value 0 is not permitted. If the negotiation fails for any parameter, an acceptable value for this parameter is returned in the additional description field as an Unsigned32 at offset 8. If the requested value is supported, but another parameter failed the negotiation, then the requested value is returned at offset 8.
Transmit Delay Time	16	Unsigned32	4	This parameter is used to set the Transmit Delay Time attribute in the receiving AREP. Its value may not be negotiated. Its value is expressed in milliseconds. If the negotiation fails for any parameter, an acceptable value for this parameter is returned in the additional description field as an Unsigned32 at offset 12. If the requested value is supported, but another parameter failed the negotiation, then the requested value is returned at offset 12.
SMK PD Tag	20	VisibleString	32	SMK PD Tag of the server. If the SMK PD Tag in the request message does not match the PD Tag of the server, then the server rejects the request with error class "access" and error code "object-access-denied".

8.3.5.2.1.3 Response APDU Parameters

Same as Request APDU

8.3.5.2.1.4 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.2.2 Idle (Confirmed Service Id = 3)

8.3.5.2.2.1 Service Overview

This APDU is used to inform the receiver that the sender is present. .

8.3.5.2.2.2 Request APDU Parameters

None.

8.3.5.2.2.3 Response APDU Parameters

None.

8.3.5.3 SMK ASE Services

8.3.5.3.1 SM Find Tag Query (Unconfirmed Service Id = 1)

8.3.5.3.1.1 Service Overview

The network address used to send the FIND_TAG_QUERY message and the FDA Address in the Type APDU Header together indicate which SMKs are to receive and process the message.

The Find Tag Query message may be sent to one or more SMKs in Type 5 and/or Type 9 devices. When sent to a linking device, the linking device Type 5 SMK responds if the linking device contains the queried object. The Type 9 interface SMKs in the linking device are used only to forward the query to Type 9 links. They never respond to the queries.

Although this service is unconfirmed, it uses the Invoke Id in the Message Trailer to allow it to be matched to the returned SM Find Tag Reply message.

Table 581 describes the distribution that may be requested for this service. The FDA Addresses in the table are shown in hexadecimal.

Table 581 – SMK FDA Address Values

FDA address meaning	FDA address		Destination address	Distribution
	LLLL	NN.SS		
Type 5 Device SMK	00 00	00.02	SMK Multicast	All Type 5 SMKs
			Individual	Individual Type 5 SMK
Type 5 and Type 9 SMK Local Link Group Address	00 00	01.09 ¹	SMK Multicast	All Type 5 SMKs and all Type 9 SMKs
			Individual	Individual LD SMK and all Type 9 SMKs accessible through the LD. This does not include the SMKs of the linking device's Type 9 interfaces.
Type 9 SMK Local Link Group Address	LL LL	01.09 ¹	SMK Multicast	All Type 9 SMKs on Type 9 link LL LL
			Individual	All Type 9 SMKs on Type 9 link LL LL accessible through the LD. This does not include the SMK of the linking device's Type 9 LL LL interface.
Individual Type 9 Address	LL LL	NN.02	Individual	Individual Type 9 SMK accessible through the LD
NOTE Specified in IEC 61158-6 as the Type 1 link specific address for "the SMAEs of all DLEs on the link"				

8.3.5.3.1.2 Request APDU Parameters

Table 582 lists the SMK FDA address values.

Table 582 – SMK FDA Address Values

Parameter name	Octet offset	Data type	Octet length	Description
Query Type	0	Unsigned8	1	Indicates the type of query: 0 = PD Tag query for primary or non-redundant device 1 = VFD tag query 2 = Function-Block tag query 3 = Element Id query 4 = PD Tag/VFD Reference query 5 = Device Index query 6 = PD Tag query for secondary or member of redundant set All other values are reserved.
Reserved	1	Octetstring	3	Reserved, set to 0
Element Id Or VFD Reference	4	Unsigned32	4	Service Parameter When not used, set to binary 0.
PD Tag Or Object Tag	8	VisibleString	32	Service Parameter When not used, set to binary 0.
VFD Tag	40	VisibleString	32	Service Parameter When not used, set to binary 0.

8.3.5.3.2 SM Find Tag Reply (Unconfirmed Service Id = 2)

8.3.5.3.2.1 Service Overview

This message is used to reply to the SM Find Tag request message.

Although this service is unconfirmed, it uses the Invoke Id in the Type 5 APDU Trailer to allow it to be matched to the corresponding SM Find Tag Query message.

8.3.5.3.2.2 Request APDU Parameters

Table 583 lists the request APDU parameters.

Table 583 – Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Query Type	0	Unsigned8	1	Indicates the type of query: 0 = PD Tag query for primary or non-redundant device 1 = VFD tag query 2 = Function-Block tag query 3 = Element Id query 4 = PD Tag/VFD Reference query 5 = Device Index query 6 = PD Tag query for secondary or member of redundant set All other values are reserved.
Type 9 Node Address	1	Unsigned8	1	Service Parameter Set to 0 if the Reply is for a Type 5 device
Type 9 Link Id	2	Unsigned16	2	Service Parameter Set to 0 if the Reply is for a Type 5 device.
VFD Reference	4	Unsigned32	4	Service Parameter
Queried Object Numeric Id	8	Unsigned32	4	Service Parameter
Queried Object Network Address	12	Octetstring	16	Service Parameter
Queried Object OD Version Number	28	Unsigned32	4	Service Parameter
Queried Object Device ID	32	VisibleString	32	Service Parameter Unused octets in this field are set to ASCII space characters.
Queried Object PD Tag	64	VisibleString	32	Service Parameter Unused octets in this field are set to ASCII space characters.
Duplicate Detection State	96	Unsigned8	1	Service Parameter Encoded as follows. Bits 8-3: Reserved, set to 0. Bit 2: 1 = Duplicate PD Tag Detected 0 = Duplicate PD Tag Not Detected Bit 1: 1 = Duplicate Device Index Detected 0 = Duplicate Device Index Not Detected
Reserved	97	Unsigned8	1	Reserved, set to 0.
Number in List of FDA Addresses	98	Unsigned16	2	This parameter indicates how many FDA Address Selectors are contained in the List of FDA Address Selectors parameter. This value of this attribute is 0 if the Query Type is "physical-device query".
List of FDA Addresses	100	Unsigned16	2x number in list	Service Parameter Each Selector is an Unsigned32 that immediately follows its predecessor in the series (no spaces or padding between them).

8.3.5.3.3 SM Identify (Confirmed Service Id = 3)**8.3.5.3.3.1 Service Overview**

This service is used to request the identity of a single Type 5 or Type 9 device or of a group of Type 5 devices.

Table 584 specifies the network addresses and FDA Addresses used with the SM Identify request message.

Table 584 – SMK FDA Address Values For SM Identify

Identification of	FDA address LLLL.NN.SS	Network address	Request message delivered to
Type 5 SMK device, including Live List Version Number List*	00 00.00.02	SM Multicast	All Type 5 SMKs
		Individual	Individual Type 5 SMK
Linking Device Node Version Number List for Specific Type 9 Interface	LL LL.00.02	SM Multicast	All Linking Device Type 5 SMKs – the one with the requested Link Id responds
	LL LL.00.02	Individual	Linking Device Type 5 SMK
Type 9 device SMK	LL LL.NN.02	Individual	Linking Device Type 5 SMK -> Type 9 SMK
* The Live List Version Number List is returned by Linking Device SMKs			

8.3.5.3.3.2 Request APDU Parameters

None.

8.3.5.3.3.3 Response APDU Parameters

Same as SM Device Annunciation Request APDU Parameters.

8.3.5.3.3.4 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.3.4 SMK Clear Address (Confirmed Service Id = 12)**8.3.5.3.4.1 Service Overview**

This service is always confirmed and, therefore, always uses the Invoke Id in the Message Trailer.

Table 585 specifies the Network Address and FDA Address used with the SM Clear Address request and response APDUs.

Table 585 – SMK FDA Address Values For SMK Set Assignment Info Request APDUs

Use	FDA address LLLL.NN.SS	Network address	Request path
Type 5 Clear Address	00 00.00.02	Individual	Type 5 Requester -> Type 5 SMK
Type 9 Clear Address	LL LL.NN.02	Individual	Type 5 Requester -> LD FDA Agent -> Type 9 SMK

8.3.5.3.4.2 Request APDU Parameters

Table 586 specifies the request APDU parameters.

Table 586 – SMK Clear Address Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Device Id	0	VisibleString	32	Service Parameter Unused octets in this field are set to ASCII space characters.
Physical Device Tag	32	VisibleString	32	Service Parameter Unused octets in this field are set to ASCII space characters.
InterfaceToClear	64	Unsigned8	1	Service Parameter Encoded as follows: 0 = Interface A 1 = Interface B 255 = All interfaces
Reserved	65	Octetstring	3	Reserved, set to 0.

8.3.5.3.4.3 Response APDU Parameters

None.

8.3.5.3.4.4 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.3.5 SMK Set Assignment Info (Confirmed Service Id = 14)**8.3.5.3.5.1 Service Overview**

This service is always confirmed and, therefore, always uses the Invoke Id in the Message Trailer.

Table 587 specifies the Network Address and FDA Address used with the SM Set Assignment Info request and response APDUs.

Table 587 – SMK FDA Address Values For SMK Set Assignment Info Request APDUs

Use	FDA address LLLL.NN.SS	Network address	Request path
Type 5 Set Assignment Info	00 00.00.02	Individual	Type 5 Requester -> Type 5 SMK
Type 9 Set Assignment Info	LL LL.NN.02	Individual	Type 5 Requester -> LD FDA Agent -> Type 9 SMK

8.3.5.3.5.2 Request APDU Parameters

Table 588 specifies the request APDU parameters.

Table 588 – SMK Set Assignment Info Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Device Id	0	VisibleString	32	Service Parameter Unused octets in this field are set to ASCII space characters.
Physical Device Tag	32	VisibleString	32	Service Parameter Unused octets in this field are set to ASCII space characters.
Type 9 New Address	64	Unsigned8	1	Service Parameter The field contains the new Type 9 address for the Type 9 device identified in the FDA Address of the Type 5 APDU Header. The value of this field is 0 if the FDA Address does not identify a Type 9 device or this APDU is not being used to change the Type 9 device's address.
Device Redundancy State	65	Unsigned8	2	Service Parameter Unused and set to 0 if the FDA Address identifies a Type 9 device. The value 0 indicates that the device is not participating in device redundancy. If it is to participate in device redundancy, the value of bit pair 2 and 1 is non-zero. The value of bit pair 4 and 3 is non-zero whenever bits 2 & 1 indicate "External Switchover Control Device". Otherwise they are unused: - Bit 8 = Reserved, set to 0 - Bit 7 = Reserved, set to 0 - Bit 6 = Reserved, set to 0 - Bit 5 = Reserved, set to 0 - Bits 4 & 3 –External Switchover Control Device Redundancy Role 0 = Not used 1 = Primary 2 = Secondary - Bits 2 & 1 – Assigned Device Redundancy Type 0 = Non-redundant 1 = External Switchover Control 2 = Internal Switchover Control
LAN Redundancy Socket Address	66	Unsigned16	2	Service Parameter Unused and set to 0 if the FDA Address identifies a Type 9 device.
Annunciation Repeat Time	68	Unsigned32	4	Service Parameter Unused and set to 0 if the FDA Address identifies a Type 9 device.
Device Index	72	Unsigned16	2	Service Parameter Encoded as follows. 0 = Index not assigned 1 to Max Device Index = Device Index
Max Device Index	74	Unsigned16	2	Service Parameter Unused and set to 0 if the FDA Address identifies a Type 9 device.
Operational Network Address	76	Octetstring	16	Service Parameter Unused and set to 0 if the FDA Address identifies an Type "X" device.
Reserved	92	Octetstring	3	Reserved, set to 0.

Parameter name	Octet offset	Data type	Octet length	Description
Clear Duplicate Detection State	95	Unsigned8	2	Service Parameter Unused and set to 0 if the FDA Address identifies a Type 9 device. Encoded as follows: Bits 8-3: Reserved, set to 0. Bit 2: 1 = Do not clear Duplicate PD Tag Detected 0 = Clear Duplicate PD Tag Detected Bit 1: 1 = Do not clear Duplicate Device Index Detected 0 = Clear Duplicate Device Index Detected

8.3.5.3.5.3 Response APDU Parameters

Table 589 specifies the response APDU parameters

Table 589 – SMK Set Assignment Info Response APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Reserved	0	Unsigned16	2	Reserved, set to 0.
Max Device Index	2	Unsigned16	2	Service Parameter Unused and set to 0 if the FDA Address identifies a Type 9 device.
Annunciation Repeat Time	4	Unsigned32	4	Service Parameter Unused and set to 0 if the FDA Address identifies a Type 9 device.

8.3.5.3.5.4 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.3.6 SMK Clear Assignment Info (Confirmed Service Id = 15)

8.3.5.3.6.1 Service Overview

This service is always confirmed and, therefore, always uses the Invoke Id in the Message Trailer.

Table 590 specifies the Network Address and FDA Address used with the SM Clear Assignment Info request and response APDUs.

Table 590 – SMK FDA Address Values For SMK Device Clear Assignment Info APDUs

Use	FDA address LLLL.NN.SS	Network address	Request path
Type 5 Clear Assignment Info	00 00.00.02	Individual	Type 5 Requester -> Type 5 SMK
Type 9 Clear Assignment Info	LL LL.NN.02	Individual	Type 5 Requester -> LD FDA Agent -> Type 9 SMK

8.3.5.3.6.2 Request APDU Parameters

Table 591 defines the request APDU parameters.

Table 591 – SMK Clear Assignment Info Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Device Id	0	VisibleString	32	Service Parameter Unused octets in this field are set to ASCII space characters. Service Parameter Unused octets in this field are set to ASCII space characters.
Physical Device Tag	32	VisibleString	32	

8.3.5.3.6.3 Response APDU Parameters

None.

8.3.5.3.6.4 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.3.7 SMK Device Annunciation (Unconfirmed Service Id = 16)**8.3.5.3.7.1 Service Overview**

The request APDU body format defined below is used for the SM Device Annunciation service and also for the SM Identify service.

This service is always unconfirmed and no APDU Trailer fields are used.

Table 592 specifies the Network Address and FDA Address used with the SMK Device Annunciation request APDUs.

Table 592 – SMK FDA Address Values For SMK Device Annunciation Request APDUs

Use	FDA address LLLL.NN.SS	Network address	Request path
Device Annunciation	00 00.00.02	SM Multicast	Type 5 SMK -> Configuration Applications

8.3.5.3.7.2 Request APDU Parameters

Table 593 defines the request APDU parameters.

Table 593 – SMK Device Annunciation Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
SMK State	0	Unsigned8	1	Service Parameter Encoded as follows. 0 = Reserved 1 = NO_TAG 2 = OPERATIONAL 3 – 255 = Reserved
Device Type	1	Unsigned8	1	Service Parameter Encoded as follows. Each bit identifies a different device type. It is the destination device type for request messages and the source device for response messages. A combination of bits 8-6 and 2,1 may be set for devices. Bit 5 is used only for Type “X” devices, and only in SM Identify responses. Non Devices participating in device assignment set the value of bits 8 through 3 of this parameter to 0. Bit 8 = Linking Device Bit 7 = I/O Gateway Bit 6 = Field Device Bit 5 = Type “X” Device Bit 4 = Reserved Bit 3 = Reserved Bits 2 & 1 – Redundant Device Type Capability 0 = Non-redundant 1 = External Switchover Control 2 = Internal Switchover Control
Device Redundancy State	2	Unsigned8	2	Service Parameter Encoded as follows. Encoded as follows. Bit 8 = Reserved, set to 0 Bit 7 = Reserved, set to 0 Bit 6 = Reserved, set to 0 Bit 5 = Reserved, set to 0 Bits 4 & 3 – Device Redundancy Role 0 = Non-redundant 1 = Primary 2 = Secondary Bits 2 & 1 – Assigned Redundant Device Type 0 = Non-redundant 1 = External Switchover Control 2 = Internal Switchover Control
Duplicate Detection State	3	Unsigned8	2	Service Parameter Encoded as follows. Bits 8-3: Reserved, set to 0. Bit 2: 1 = Duplicate PD Tag Detected 0 = Duplicate PD Tag Not Detected Bit 1: 1 = Duplicate Device Index Detected 0 = Duplicate Device Index Not Detected
Device Index	4	Unsigned16	2	Service Parameter Encoded as follows. 0 = Index not assigned 1 to Max Device Index = Device Index
Max Device Index	6	Unsigned16	2	Service Parameter
Operational Network Address	8	Octetstring	16	Service Parameter
Device Id	24	VisibleString	32	Service Parameter Unused octets in this field are set to ASCII space characters.
Physical Device Tag	56	VisibleString	32	Service Parameter Unused octets in this field are set to ASCII space characters.
Annunciation Repeat Time	92	Unsigned32	4	Service Parameter

Parameter name	Octet offset	Data type	Octet length	Description
LAN Redundancy Socket Address	96	Unsigned16	2	Service Parameter
Reserved	98	Unsigned16	2	Service Parameter
APDU Version Number	100	Unsigned32	4	Service Parameter
Device Version Number	100	Unsigned32	4	Service Parameter
Number of Entries in Version Number List	104	Unsigned32	4	This parameter indicates how many version numbers are contained in the Version Number List field below. Its value is 0 if the device is not a linking device or a gateway.
Version Number List	108	Unsigned32	4 x Number of Entries	Service Parameter This field is present only for linking devices and I/O gateways. This field contains a list of numbers. Each is an Unsigned32 number. The number of entries in the list is specified by the field above "Number of Entries in Version Number List". This field supports two types of lists, as determined by the Link Id portion of the FDA Address, as follows: Link Id = 0 For linking devices, this is a list of Type 9 live list version numbers, one for each Type 9 interface attached to the linking device. Each element in the list is constructed as follows: Bits 32-17: Type 9 Link Id Bits 16-9: Reserved, set to 0 Bits 8-1: Version Number The list is ordered by Type 9 bridge interface number with the Live List Version for the first interface occurring first in the list. The value 0 is used for the Link Id bits if the interface has not been assigned a link Id. Link Id > 0 This is a List of Type "X" Node Address Version Numbers, one for each node id currently in the live list for the specified Type "X" link, including the LD Type "X" interface. The List of Type "X" Node Address Version Numbers is used only when this message format is used for SM Identify Responses that describe an LD Type "X" interface. The list begins with the version number of the specified Type "X" Interface. The remainder of the list is sorted in ascending order by Type "X" node address, with two version numbers packed into each Unsigned32 value, as follows: Bits 33-25: Type "X" Node Address Bits 24-17: Version Number Bits 16-9: Type "X" Node Address Bits 8-1: Version Number

8.3.5.4 Type 9 ASE Services

8.3.5.4.1 General Note

NOTE Unless specified otherwise, all parameters of the Type 9 Service APDUs and their values are defined in IEC 61158-5.

8.3.5.4.2 Reject

The Type 9 Reject Service is used only to indicate to the requester that the response is too long for Type 9 to convey it. Instead of defining a separate APDU here to convey this information, the Common Error APDU conveys it, using the Error Class "Reject" and the Reject Code as defined in the Type 9 abstract syntax for the Error Code.

8.3.5.4.3 Initiate (Confirmed Service Id = 96)**8.3.5.4.3.1 Request APDU Parameters**

Table 594 defines the request APDU parameters

Table 594 – Initiate Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Connect Option	0	Unsigned8	1	This field indicates which option is to be used to open the S-VFD Context. Its values are: 1 = VCR Selector 2 = MIB Access 3 = FBAP Access
Access Protection Supported Calling	8	Boolean	1	Service Parameter
Password and Access Groups Calling	2	Unsigned16	2	Service Parameter
Version OD Calling	4	Integer16	2	Service Parameter
Profile Number Calling	6	Unsigned16	2	Service Parameter
PD Tag	8	OctetString	32	The SMK Physical Device Tag of the device containing the destination VFD.

8.3.5.4.3.2 Response APDU Parameters

Table 595 lists the response APDU parameters.

Table 595 – Initiate Response APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Version OD Called	0	Integer16	2	Service Parameter
Profile Number Called	2	Unsigned16	2	Service Parameter

8.3.5.4.3.3 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1. See Type 9 Initiate specific error codes.

8.3.5.4.4 Abort (Unconfirmed Service Id = 112)**8.3.5.4.4.1 Request APDU Parameters**

Table 596 specifies the request APDU parameters.

Table 596 – Abort Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Abort Detail	0	OctetString	16	Supplied by the sender of the APDU.
Abort Identifier	16	Unsigned8	1	Service parameter. Values defined for Type 9 PDUs.
Reason Code	17	Unsigned8	1	Service Parameter
Reserved	18	Unsigned16	2	Reserved, set to 0

8.3.5.4.5 Get Status (Confirmed Service Id = 0)**8.3.5.4.5.1 Request APDU Parameters**

None

8.3.5.4.5.2 Response APDU Parameters

Table 597 defines the response APDU parameters.

Table 597 – Get Response APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Logical Status	0	Unsigned8	1	Service Parameter
Physical Status	1	Unsigned8	1	Service Parameter
Reserved	2	Unsigned16	2	Reserved, set to 0
Local Detail	4	OctetString	4	Service Parameter. The high order octet is reserved and set to 0. The low order three octets contain the 3-octet Local Detail value

8.3.5.4.6 Status Notification (Unconfirmed Service Id = 1)

8.3.5.4.6.1 Request APDU Parameters

Same as Get Status Response APDU

8.3.5.4.7 Identify (Confirmed Service Id = 1)

8.3.5.4.7.1 Request APDU Parameters

None

8.3.5.4.7.2 Response APDU Parameters

Table 598 specifies the response APDU parameters.

Table 598 – Identify Response APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Vendor Name	0	VisibleString	32	Service Parameter
Model Identifier	32	VisibleString	32	Service Parameter
Vendor Revision	64	VisibleString	32	Service Parameter

8.3.5.4.8 Get OD (Confirmed Service Id = 4)

8.3.5.4.8.1 Request APDU Parameters

Table 599 specifies the request APDU parameters.

Table 599 – Get OD Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
All Attributes	0	Boolean	1	Service Parameter
Start Index Flag	1	Boolean	1	Service Parameter
Reserved	2	Unsigned16	2	Reserved, set to 0
Index	4	Unsigned32	4	Service Parameter

8.3.5.4.8.2 Response APDU Parameters

Table 600 specifies the response APDU parameters.

Table 600 – Get OD Response APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
More Follows	0	Boolean	1	Service Parameter
Number of Object Descriptions	1	Unsigned8	1	Service Parameter
Reserved	2	Unsigned16	2	Reserved, set to 0
List of Object Descriptions	4	OctetString	N	Defined in 8.3.5.6.2

8.3.5.4.8.3 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.4.9 Initiate Put OD (Confirmed Service Id = 28)**8.3.5.4.9.1 Request APDU Parameters**

Table 601 specifies the request APDU parameters.

Table 601 – Initiate Put OD Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Reserved	0	Unsigned16	2	Reserved, set to 0
Consequence	2	Unsigned16	2	Service Parameter. Values defined for Type 9 PDUs.

8.3.5.4.9.2 Response APDU Parameters

None

8.3.5.4.9.3 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.4.10 Put OD (Confirmed Service Id = 29)**8.3.5.4.10.1 Request APDU Parameters**

Table 602 specifies the request APDU parameters.

Table 602 – Put OD Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Number of Object Descriptions	0	Unsigned8	1	Number of entries in List of Object Descriptions
Reserved	1	OctetString	3	Reserved, set to 0
List of Object Descriptions	4	OctetString	N	Defined in 8.3.5.6.2.

8.3.5.4.10.2 Response APDU Parameters

None

8.3.5.4.10.3 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.4.11 Terminate Put OD (Confirmed Service Id = 30)

8.3.5.4.11.1 Request APDU Parameters

None

8.3.5.4.11.2 Response APDU Parameters

None

8.3.5.4.11.3 Error APDU Parameters

Defined by the OD Error APDU Parameters in 8.3.6.

8.3.5.4.12 Generic Initiate Download Sequence (Confirmed Service Id = 31)

8.3.5.4.12.1 Service Overview

This service corresponds to the Type 9 Initiate Load Service with the following parameter settings:

- Load Region Index Called: Load region of the responder (Server)
- Load Type: DOWNLOAD
- Load Service to Use: Push Segment (Generic Download Segment)
- Load Service Initiator: TRUE (Initiator (Client) initiates the load service (The Push Segment service))

8.3.5.4.12.2 Request APDU Parameters

Table 603 specifies the request APDU parameters.

Table 603 – Generic Initiate Download Sequence Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter

8.3.5.4.12.3 Response APDU Parameters

None

8.3.5.4.12.4 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.4.13 Generic Download Segment (Confirmed Service Id = 32)

8.3.5.4.13.1 Service Overview

This service corresponds to the Type 9 Generic Download Segment Service.

8.3.5.4.13.2 Request APDU Parameters

See Table 604.

Table 604 – Generic Download Segment Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter
More Follows	4	Boolean	1	Service Parameter
Reserved	5	OctetString	3	Set to 0.
Load Data	8	OctetString	N	Service Parameter

8.3.5.4.13.3 Response APDU Parameters

None

8.3.5.4.13.4 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.4.14 Generic Terminate Download Sequence (Confirmed Service Id = 33)**8.3.5.4.14.1 Service Overview**

This service corresponds to the Type 9 Generic Terminate Download Sequence Service.

8.3.5.4.14.2 Request APDU Parameters

See Table 605.

Table 605 – Generic Terminate Download Sequence Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter

8.3.5.4.14.3 Response APDU Parameters

See Table 606.

Table 606 – Response APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Terminate Reason	0	Boolean	1	Service Parameter
Reserved	1	OctetString	3	Reserved, set to 0

8.3.5.4.14.4 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.4.15 Initiate Download Sequence (Confirmed Service Id = 9)**8.3.5.4.15.1 Service Overview**

This service corresponds to the Type 9 Initiate Download Sequence Service with the following parameter settings:

- Load Region Index Called: Load region of the responder (Server)
- Load Type: DOWNLOAD
- Load Service to Use: Pull Segment (Download Segment)
- Load Service Initiator: FALSE (Remote AP (Server) initiates the load service (The Pull Segment service))

8.3.5.4.15.2 Request APDU Parameters

See Table 607.

Table 607 – Initiate Download Sequence Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter

8.3.5.4.15.3 Response APDU Parameters

None

8.3.5.4.15.4 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.4.16 Download Segment (Confirmed Service Id = 10)

8.3.5.4.16.1 Service Overview

This service corresponds to the Type 9 Download Segment Service.

8.3.5.4.16.2 Request APDU Parameters

See Table 608.

Table 608 – Download Segment Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter

8.3.5.4.16.3 Response APDU Parameters

See Table 609.

Table 609 – Download Segment Response APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
More Follows	0	Boolean	1	Service Parameter
Reserved	2	OctetString	3	Reserved, each octet is set to binary 0
Load Data	4	OctetString	N	Service Parameter

8.3.5.4.16.4 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.4.17 Terminate Download Sequence (Confirmed Service Id = 11)

8.3.5.4.17.1 Service Overview

This service corresponds to the Type 9 Terminate Download Sequence Service.

8.3.5.4.17.2 Request APDU Parameters

See Table 610.

Table 610 – Terminate Download Sequence Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter
Reserved	4	Octetstring	3	Reserved, each octet is set to binary 0
Terminate Reason	7	Boolean	1	Service Parameter

8.3.5.4.17.3 Response APDU Parameters

None

8.3.5.4.17.4 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.4.18 Initiate Upload Sequence (Confirmed Service Id = 12)**8.3.5.4.18.1 Service Overview**

This service corresponds to the Type 9 Initiate Upload Sequence Service with the following parameter settings:

- Load Region Index Called: Load region of the responder (Server)
- Load Type: UPLOAD
- Load Service to Use: Pull Segment (Upload Segment)
- Load Service Initiator: TRUE (Initiator (Client) initiates the load service (The Pull Segment service))

8.3.5.4.18.2 Request APDU Parameters

See Table 611.

Table 611 – Initiate Upload Sequence Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter

8.3.5.4.18.3 Response APDU Parameters

None

8.3.5.4.18.4 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.4.19 Upload Segment (Confirmed Service Id = 13)**8.3.5.4.19.1 Service Overview**

This service corresponds to the Type 9 Upload Segment Service.

8.3.5.4.19.2 Request APDU Parameters

See Table 612.

Table 612 – Upload Segment Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter

8.3.5.4.19.3 Response APDU Parameters

See Table 613.

Table 613 – Upload Segment Response APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
More Follows	0	Boolean	1	Service Parameter
Reserved	2	OctetString	3	Reserved, each octet is set to binary 0
Load Data	4	OctetString	N	Service Parameter

8.3.5.4.19.4 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.4.20 Terminate Upload Sequence (Confirmed Service Id = 14)**8.3.5.4.20.1 Service Overview**

This service corresponds to the Type 9 Terminate Upload Sequence Service.

8.3.5.4.20.2 Request APDU Parameters

See Table 614.

Table 614 – Terminate Upload Sequence Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter

8.3.5.4.20.3 Response APDU Parameters

None

8.3.5.4.20.4 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.4.21 Request Domain Download (Confirmed Service Id = 15)**8.3.5.4.21.1 Service Overview**

This service corresponds to the Type 9 Request Domain Download Service.

8.3.5.4.21.2 Request APDU Parameters

See Table 615 for definitions.

Table 615 – Request Domain Download Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter
Additional Information	4	VisibleString	N	Service Parameter

8.3.5.4.21.3 Response APDU Parameters

None

8.3.5.4.21.4 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.4.22 Request Domain Upload (Confirmed Service Id = 16)**8.3.5.4.22.1 Service Overview**

This service corresponds to the Type 9 Create Program Invocation Service.

8.3.5.4.22.2 Request APDU Parameters

See Table 616 for definitions.

Table 616 – Request Domain Upload Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter
Additional Information	4	VisibleString	N	Service Parameter

8.3.5.4.22.3 Response APDU Parameters

None

8.3.5.4.22.4 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.4.23 Create Program Invocation (Confirmed Service Id = 17)**8.3.5.4.23.1 Service Overview**

The Create Program Invocation corresponds to to the Type 9 Create Program Invocation service.

8.3.5.4.23.2 Request APDU Parameters

See Table 617 for definitions.

Table 617 – Create Program Invocation Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Reusable	0	Boolean	1	Service Parameter
Reserved	1	Unsigned8	1	Reserved, set to 0
Number of Domain Indexes	2	Unsigned16	2	The number of domain indexes in this service
List Of Domain Indexes	4	Unsigned32	4 X Number of Domain Indexes	Service Parameter

8.3.5.4.23.3 Response APDU Parameters

See Table 618 for definitions.

Table 618 – Create Program Invocation Response APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter

8.3.5.4.23.4 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.4.24 Delete Program Invocation (Confirmed Service Id = 18)

8.3.5.4.24.1 Service Overview

The Delete Program Invocation corresponds to to the Type 9 Delete Program Invocation service.

8.3.5.4.24.2 Request APDU Parameters

See Table 619 for definitions.

Table 619 – Delete Program Invocation Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter

8.3.5.4.24.3 Response APDU Parameters

None

8.3.5.4.24.4 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.4.25 Start (Confirmed Service Id = 19)

8.3.5.4.25.1 Request APDU Parameters

See Table 620 for definitions.

Table 620 – Start Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter
Execution Argument	4	OctetString	N	Service Parameter

8.3.5.4.25.2 Response APDU Parameters

None

8.3.5.4.25.3 PI Error APDU Parameters

Defined by the Common PI Error APDU Parameters in 8.3.6.2.

8.3.5.4.26 Stop (Confirmed Service Id = 20)**8.3.5.4.26.1 Request APDU Parameters**

See Table 621 for definitions.

Table 621 – Stop Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter

8.3.5.4.26.2 Response APDU Parameters

None

8.3.5.4.26.3 Error APDU Parameters

Defined by the Common PI Error APDU Parameters in 8.3.6.2.

8.3.5.4.27 Resume (Confirmed Service Id = 21)**8.3.5.4.27.1 Request APDU Parameters**

See Table 622 for definitions.

Table 622 – Resume Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter
Execution Argument	4	OctetString	N	Service Parameter

8.3.5.4.27.2 Response APDU Parameters

None

8.3.5.4.27.3 Error APDU Parameters

Defined by the Common PI Error APDU Parameters in 8.3.6.2.

8.3.5.4.28 Reset (Confirmed Service Id = 22)**8.3.5.4.28.1 Request APDU Parameters**

See Table 623 for definitions.

Table 623 – Reset Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter

8.3.5.4.28.2 Response APDU Parameters

None

8.3.5.4.28.3 Error APDU Parameters

Defined by the Common PI Error APDU Parameters in 8.3.6.2.

8.3.5.4.29 Kill (Confirmed Service Id = 23)**8.3.5.4.29.1 Request APDU Parameters**

See Table 624 for definitions.

Table 624 – Kill Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter

8.3.5.4.29.2 Response APDU Parameters

None

8.3.5.4.29.3 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.4.30 Read (Confirmed Service Id = 2)**8.3.5.4.30.1 Request APDU Parameters**

See Table 625 for definitions.

Table 625 – Read Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter

8.3.5.4.30.2 Response APDU Parameters

See Table 626 for definitions.

Table 626 – Read Response APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Value	0	OctetString	N	Service Parameter

8.3.5.4.30.3 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.4.31 Read with Subindex (Confirmed Service Id = 82)**8.3.5.4.31.1 Service Overview**

This service corresponds to the Type 9 Read service.

8.3.5.4.31.2 Request APDU Parameters

See Table 627 for definitions.

Table 627 – Read with Subindex Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter
Component ID	4	Unsigned32	4	Service Parameter

8.3.5.4.31.3 Response APDU Parameters

See Table 628 for definitions.

Table 628 – Read with Subindex Response APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Value	0	OctetString	N	Service Parameter

8.3.5.4.31.4 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.4.32 Write (Confirmed Service Id = 3)**8.3.5.4.32.1 Request APDU Parameters**

See Table 629 for definitions.

Table 629 – Write Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter
Value	4	OctetString	N	Service Parameter

8.3.5.4.32.2 Response APDU Parameters

None

8.3.5.4.32.3 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.4.33 Write with Subindex (Confirmed Service Id = 83)**8.3.5.4.33.1 Service Overview**

This service corresponds to the Type 9 Write service.

8.3.5.4.33.2 Request APDU Parameters

See Table 630 for definitions.

Table 630 – Write with Subindex Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter
Component ID	4	Unsigned32	4	Service Parameter
Value	8	OctetString	N	Service Parameter

8.3.5.4.33.3 Response APDU Parameters

None

8.3.5.4.33.4 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.4.34 Define Variable List (Confirmed Service Id = 7)**8.3.5.4.34.1 Service Overview**

This service corresponds to the Type 9 Define Variable List service.

8.3.5.4.34.2 Request APDU Parameters

See Table 631 for definitions.

Table 631 – Define Variable List Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Number of Indexes	0	Unsigned32	4	Number of Variable Indexes in the list that follows.
List Of Indexes	4	Unsigned32	4 x Number of Indexes	Service Parameter

8.3.5.4.34.3 Response APDU Parameters

See Table 632 for definitions.

Table 632 – Define Variable List Response APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter

8.3.5.4.34.4 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.4.35 Delete Variable List (Confirmed Service Id = 8)**8.3.5.4.35.1 Service Overview**

This service corresponds to the Type 9 Delete Variable List service.

8.3.5.4.35.2 Request APDU Parameters

See Table 633 for definitions.

Table 633 – Delete Variable List Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter

8.3.5.4.35.3 Response APDU Parameters

None

8.3.5.4.35.4 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.4.36 Information Report (Unconfirmed Service Id = 0)**8.3.5.4.36.1 Request APDU Parameters**

See Table 634 for definitions.

Table 634 – Information Report Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter
Value	4	OctetString	N	Service Parameter

8.3.5.4.37 Information Report with Subindex (Unconfirmed Service Id = 16)**8.3.5.4.37.1 Service Overview**

This service corresponds on the Type 9 Information Report service.

8.3.5.4.37.2 Request APDU Parameters

See Table 635 for definitions.

Table 635 – Information Report with Subindex Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter
Component ID	4	Unsigned32	4	Service Parameter
Value	8	OctetString	N	Service Parameter

8.3.5.4.38 Information Report On Change (Unconfirmed Service Id = 17)**8.3.5.4.38.1 Service Overview**

This service corresponds to the Type 9 Information Report service.

This APDU is used to send data that is published only when its value has changed. The VCR that controls the publishing indicates if this service is to be used.

8.3.5.4.38.2 Request APDU Parameters

See Table 636 for definitions.

Table 636 – Information Report On Change Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter
Value	4	OctetString	N	Service Parameter

8.3.5.4.39 Information Report On Change with Subindex (Unconfirmed Service Id = 18)**8.3.5.4.39.1 Service Overview**

This service corresponds to the Type 9 Information Report service.

This APDU is used to send data that is published only when its value has changed. The VCR that controls the publishing indicates if this service is to be used.

8.3.5.4.39.2 Request APDU Parameters

See Table 637 for definitions.

Table 637 – Information Report On Change with Subindex Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter
Component ID	4	Unsigned32	4	Service Parameter
Value	8	OctetString	N	Service Parameter

8.3.5.4.40 Event Notification (Unconfirmed Service Id = 2)**8.3.5.4.40.1 Request APDU Parameters**

See Table 638 for definitions.

Table 638 – Event Notification Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter
Event Number	4	Unsigned32	4	Service Parameter
Data	8	OctetString	N	Service Parameter

8.3.5.4.41 Alter Event Condition Monitoring (Confirmed Service Id = 24)**8.3.5.4.41.1 Request APDU Parameters**

See Table 639 for definitions.

Table 639 – Alter Event Condition Monitoring Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter
Reserved	4	Octetstring	3	Reserved, each octet is set to binary 0
Enabled	7	Boolean	1	Service Parameter

8.3.5.4.41.2 Response APDU Parameters

None

8.3.5.4.41.3 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.4.42 Acknowledge Event Notification (Confirmed Service Id = 25)**8.3.5.4.42.1 Request APDU Parameters**

See Table 640 for definitions.

Table 640 – Acknowledge Event Notification Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Service Parameter
Event Number	4	Unsigned32	4	Service Parameter

8.3.5.4.42.2 Response APDU Parameters

None

8.3.5.4.42.3 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.5 LAN Redundancy Services**8.3.5.5.1 LAN Redundancy Diagnostic Message (Unconfirmed Service Id = 1)****8.3.5.5.1.1 Request APDU Parameters**

See Table 641 for definitions.

Table 641 – LAN Redundancy Diagnostic Message Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Device Index	0	Unsigned16	2	Service parameter 0 = Index not assigned 1 – 65535 = Device Index
Number of Interfaces	2	Unsigned8	1	Service parameter.
Transmission Interface	3	Unsigned8	1	Identifies the interface from which this APDU was transmitted 1 = Interface A 2 = Interface B
Diagnostic Message Interval	4	Unsigned 32	4	Service parameter.
SMK Physical Device Tag	8	VisibleString	32	Service parameter. Unused octets in this field are set to ASCII space characters.
Reserved	40	Unsigned8	1	Reserved. Set to zero
Duplicate Detection State	41	Unsigned8	1	Service parameter. Encoded as follows: Bits 8-3: Reserved, set to 0. Bit 2: 1 = Duplicate PD Tag Detected 0 = Duplicate PD Tag Not Detected Bit 1: 1 = Duplicate Device Index Detected 0 = Duplicate Device Index Not Detected
Number of Interface Statuses	42	Unsigned16	2	The number of Unsigned32 entries in the four Array of Interface Statuses that follow. The value of this field is 1/32 nd of the Max Device Index value configured using the SM Set Assignment service.
Array of Interface AtoA Statuses	44	Array of Unsigned32	4* Number of Interface Statuses	Each 32-bit value represents the status of 32 devices. (see note) 1 bit represents each device. Each bit indicates the status of the transmission path from interface A of the reporting device to interface A of this device. The following values are used for each bit: 0 = Diagnostic messages successfully 1 = Diagnostic messages not received
Array of Interface BtoA Statuses	44 + 4* Number of Interface Statuses	Array of Unsigned32	4* Number of Interface Statuses	Each 32-bit value represents the status of 32 devices. (see note) 1 bit represents each device. Each bit indicates the status of the transmission path from interface B of the reporting device to interface A of this device. The following values are used for each bit: 0 = Diagnostic messages successfully 1 = Diagnostic messages not received
Array of Interface AtoB Statuses	44 + 8* Number of Interface Statuses	Array of Unsigned32	4* Number of Interface Statuses	Each 32-bit value represents the status of 32 devices. (see note) 1 bit represents each device. Each bit indicates the status of the transmission path from interface A of the reporting device to interface B of this device. The following values are used for each bit: 0 = Diagnostic messages successfully 1 = Diagnostic messages not received
Array of Interface BtoB Statuses	44 + 12* Number of Interface Statuses	Array of Unsigned32	4* Number of Interface Statuses	Each 32-bit value represents the status of 32 devices. (see note) 1 bit represents each device. Each bit indicates the status of the transmission path from interface B of the reporting device to interface B of this device. The following values are used for each bit: 0 = Diagnostic messages successfully 1 = Diagnostic messages not received

NOTE The bit that identifies a device is located by its bit position in the array. Dividing the device's Device Index by 32 yields the index of the zero-based Unsigned32 array element. The remainder from the division identifies the specific bit in the Unsigned32 array element, counting from the most significant bit (= remainder 1). Therefore, a device with Device Index 33 would be represented by the most significant bit (bit 32) of the second Unsigned32 element of the array, and a device with Device Index 34 would be represented by the next most significant bit (bit 31) of the second Unsigned32 element of the array.

8.3.5.5.2 LAN Redundancy Get Information (Confirmed Service Id = 1)

8.3.5.5.2.1 Service Overview

This service is always confirmed and, therefore, always uses the Invoke Id in the Message Trailer.

8.3.5.5.2.2 Request APDU Parameters

None

8.3.5.5.2.3 Response APDU Parameters

See Table 642 for definitions.

Table 642 – LAN Redundancy Get Information Response APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
LAN Redundancy Attributes Version	0	Unsigned32	4	Service Parameter
Max Sequence Number Difference	4	Unsigned8	1	Service Parameter
LAN Redundancy Flags	5	Unsigned8	1	Service Parameter. Encoded as follows; 0 = False, 1 = True Bit 1 Single Mcast APDU Tx I/F Enabled Bit 2 Crossed Cable Detection Enabled Bit 3 Single Mcast APDU Rcpt I/F Enabled Bit 4 Diagnosis using own APDUs Enabled Bit 5 Load Balancing Enabled Bits 6-8 Reserved, set to 0
Reserved	6	Unsigned16	2	Reserved. Set to zero.
Diagnostic Message Interval	8	Unsigned32	4	Service Parameter
AgingTime	12	Unsigned32	4	Service Parameter
Diagnostic Message Interface A Send Address	16	OctetString	16	This parameter is part of the Diagnostic Message Send Addresses service parameter. It defines the destination address for diagnostic messages sent from interface A.
Diagnostic Message Interface A Receive Address	32	OctetString	16	This parameter is part of the Diagnostic Message Receive Addresses service parameter. It defines the destination address for diagnostic messages received on interface A.
Diagnostic Message Interface B Send Address	48	OctetString	16	This parameter is part of the Diagnostic Message Send Addresses service parameter. It defines the destination address for diagnostic messages sent from interface B.
Diagnostic Message Interface B Receive Address	64	OctetString	16	This parameter is part of the Diagnostic Message Receive Addresses service parameter. It defines the destination address for diagnostic messages received on interface B.

8.3.5.5.2.4 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.5.3 LAN Redundancy Put Information (Confirmed Service Id = 2)

8.3.5.5.3.1 Service Overview

This service is always confirmed and, therefore, always uses the Invoke Id in the Message Trailer.

8.3.5.5.3.2 Request APDU Parameters

Same as LAN Redundancy Get Information response parameters. Only those fields following the APDU Version Number are used by the receiving LAN Redundancy Entity.

8.3.5.5.3.3 Response APDU Parameters

Same as Request Message Parameters. It includes the updated LAN Redundancy Attributes Version.

8.3.5.5.3.4 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.5.4 LAN Redundancy Get Statistics (Confirmed Service Id = 3)

8.3.5.5.4.1 Service Overview

This service is always confirmed and, therefore, always uses the Invoke Id in the Message Trailer.

8.3.5.5.4.2 Request APDU Parameters

None

8.3.5.5.4.3 Response APDU Parameters

See Table 643 for definitions.

Table 643 – LAN Redundancy Get Statistics Request APDU Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Diagnostic Messages Received On Interface A	0	Unsigned32	4	Service Parameter
Diagnostic Messages Missed On Interface A	4	Unsigned32	4	Service Parameter
Remote Device Diagnostic Message Receive Faults Detected On Interface A	8	Unsigned32	4	Service Parameter
Diagnostic Messages Received On Interface B	12	Unsigned32	4	Service Parameter
Diagnostic Messages Missed On Interface B	16	Unsigned32	4	Service Parameter
Remote Device Diagnostic Message Receive Faults Detected On Interface B	20	Unsigned32	4	Service Parameter
Number of Crossed Cable Statuses	24	Unsigned32	4	This field is number of Unsigned32 values in the List of Crossed Cable Status. 0 means that the list is not present. If the list is present, then this field contains 1/32nd of the value of the Max Device Index assigned to the device in the SM Set Assignment Info message.
Array of Crossed Cable Status	28	Array of Unsigned32	4 * Number of Crossed Cable Statuses	Service Parameter. Each bit in this array (see note) represents a device that sends diagnostic messages to the device that is sending this message (the reporting device). Each remote device's bit represents whether the interface it is using to send diagnostic messages, as indicated by the Interface Used for Transmission of this Diagnostic Message field, is the same interface used by the reporting device to receive the messages. The following values are used for each bit: 0 = Cables not crossed or Crossed Cable Detection Enabled is false. 1 = Cables are crossed (Diagnostic message sent from Interface A was received on Interface B, or diagnostic message sent from Interface B was received on Interface A).

NOTE The bit that identifies a device is located by its bit position in the array. Dividing the device's Device Index by 32 yields the index of the zero-based Unsigned32 array element. The remainder from the division identifies the specific bit in the Unsigned32 array element, counting from the most significant bit (= remainder 1). Therefore, a device with Device Index 33 would be represented by the most significant bit (bit 32) of the second Unsigned32 element of the array, and a device with Device Index 34 would be represented by the next most significant bit (bit 31) of the second Unsigned32 element of the array.

8.3.5.5.4.4 Error APDU Parameters

Defined by the Common Error APDU Parameters in 8.3.6.1.

8.3.5.6 Structure of Object Descriptions

8.3.5.6.1 General

This subclause defines the object descriptions conveyed using Type 5 Get OD and Put OD services. VFDs using 32-bit OD indexes are required to use the HSE object description formats defined below. VFDs using 16-bit OD indexes may use either H1 or HSE object description formats.

The FDA Agent may convert Type 9 formatted object descriptions it receives from local VFDs or from Type 9 devices to the corresponding Type 5 formats, but it is not required to do so. In addition, the FDA Agent may, but is not required to, convert HSE formatted object descriptions that it receives from Type 5 in PutOD request messages to Type 9 format for delivery to or forwarding to VFDs that use Type 9 formats.

When an FDA Agent that does not support conversion from Type 5 to Type 9 format receives such a request message requiring conversion, it may simply forward the message and rely on the receiving VFD to return the appropriate error message, or it may return a negative response with error class "od" and error code ="type 5 to type 9 format conversion not supported".

The fields contained in each OD construct below are defined in the Type 9 Specification.

Their definitions are the same here, however, their *index*, *data type*, and *length* fields have changed to 32-bits, and their order have changed to align on 32-bit boundaries.

For fields defined to have Boolean values, the value 0 indicates "FALSE" and any non-zero value indicates "TRUE".

To maintain compatibility with Type 9 object descriptions, the first field in an object description conveyed by the FDA Agent is the OD Tag. This OD Tag contains either the tag of a Type 9 object description or a tag reserved for Type 5 object descriptions (hex value 0xFFFF). Following the OD Tag is the object description indicated by the tag. Each Type 5 object description uses one of the formats specified in this subclause.

Each Type 9 object description contains a Type 5 tag plus a Type 5 object description, as specified by Type 9. The Type 5 tag is the encoded Type 9 ASN.1 tag specified for the GetOD Response and the PutOD Request messages. The Type 9 Object Description is specified as packed data.

The Type 9 tag consists of one or two octets. The first four most significant bits of the first octet (most significant byte) has the value 0. The next four bits is the length of the object description. If this 4-bit length field contains a value less than the hex value 0xF, then the H1 tag is one octet and the 4-bit length field specifies the length of the object description that follows.

Following the Type 5 Type Flag field, all Type 5 Object Descriptions have five additional fields. The format of this common Type 5 Object Description header is shown in Table 644.

Table 644 – Object Description Header

Field name	Octet offset	Data type	Octet length	Description
OD Tag	0	Unsigned16	2	Set to FFFF ₁₆ to indicate an Type 5 Object Description
Object Code	2	Integer8	1	Object Code of the Object
All Attributes Flag	3	Unsigned8	1	Indicates long form (=1) or short form (=0) Object Description
Object Description Length	4	Unsigned16	2	Total length in octets of the Object Description
Reserved (set to zero)	6	Unsigned16	2	Set to zero
Index32	8	Unsigned32	4	32-bit Object Index

A null object in Type 5 is used to indicate an unused object index, as it is in Type 9. It is represent in Type 5 as shown in Table 645.

Table 645 – Null Object

Field name	Value
OD Tag	FF FF ₁₆
Object Code	0
All Attributes Flag	0
Object Description Length	12
Reserved (set to zero)	0
Index32	Index of Null Object

8.3.5.6.2 Structure of the List of Object Descriptions

See Table 646 for definitions.

Table 646 – Structure of the List of Object Descriptions

Field name	Octet offset	Data type	Octet length
Short Form Fields			
OD Tag = FF FF ₁₆	0	Unsigned16	2
Object Code = 16	2	Integer8	1
All Attributes Flag	3	Unsigned8	1
Object Description Length	4	Unsigned16	2
Reserved (set to zero)	6	Unsigned16	2
Index32	8	Unsigned32	4
ROM/RAM Flag	12	Unsigned8	1
Access Protection Supported	13	Unsigned8	1
Reserved (set to zero)	14	Unsigned16	2
Name Length	16	Unsigned32	4
Version OD	20	Unsigned32	4
ST-OD Length	24	Unsigned32	4
First Index S-OD	28	Unsigned32	4
S-OD Length	32	Unsigned32	4
First Index DV-OD	36	Unsigned32	4
DV-OD Length	40	Unsigned32	4
First Index DP-OD	44	Unsigned32	4
DP-OD Length	48	Unsigned32	4
Long Form Additional Fields (see note)			
	52		
Local Address OD-ODES	0 + Long Form Offset	Unsigned64	8
Local Address ST-OD	8 + Long Form Offset	Unsigned64	8
Local Address S-OD	16 + Long Form Offset	Unsigned64	8
Local Address DV-OD	24 + Long Form Offset	Unsigned64	8
Local Address DP-OD	32 + Long Form Offset	Unsigned64	8
NOTE If the Short Form is requested, these fields are not present			

8.3.5.6.3 Structure of a Load Region in the S-OD

See Table 647 for definitions.

Table 647 – Structure of a Load Region in the S-OD

Field name	Octet offset	Data type	Octet length
Short Form Fields			
OD Tag = FF FF ₁₆	0	Unsigned16	2
Object Code = 18	2	Integer8	1
All Attributes Flag	3	Unsigned8	1
Object Description Length	4	Unsigned16	2
Reserved (set to zero)	6	Unsigned16	2
Index32	8	Unsigned32	4
Domain State	12	Unsigned8	1
Upload State	13	Unsigned8	1
Reserved (set to zero)	14	Unsigned16	2
Max Octets	16	Unsigned32	4
Counter	20	Unsigned32	4
Long Form Additional Fields (see note 1)			
	24		
Local Address	0 + Long Form Offset	Unsigned64	8
Password	8 + Long Form Offset	Unsigned8	1
Access Groups	9 + Long Form Offset	Unsigned8	1
Access Rights	10 + Long Form Offset	Unsigned16	2
Extension Length	12 + Long Form Offset	Unsigned32	4
Extension Data	16 + Long Form Offset	OctetString	Extension Length
Domain Name	16 + Long Form Offset + Extension Length	VisibleString	Name Length (see note 2)
NOTE 1 If the Short Form is requested, these fields are not present.			
NOTE 2 If Name Length in the OD Object Description is 0, then this field is not present			

8.3.5.6.4 Structure of a Function Invocation in the DP-OD

See Table 648 for definitions.

Table 648 – Structure of a Function Invocation in the DP-OD

Field name	Octet offset	Data type	Octet length
Short Form Fields			
OD Tag = FF FF ₁₆	0	Unsigned16	2
Object Code = 19	2	Integer8	1
All Attributes Flag	3	Unsigned8	1
Object Description Length	4	Unsigned16	2
Reserved (set to zero)	6	Unsigned16	2
Index32	8	Unsigned32	4
Deletable	12	Unsigned8	1
Reusable	13	Unsigned8	1
PI-State	14	Unsigned8	1
Reserved	15	Unsigned8	3
Number Of Domains	16	Unsigned32	4
List of Domain Indexes	20	Unsigned32	Number Of Domains x 4 (see note 1)
Long Form Additional Fields (see note 2)	20 + (Number Of Domains x 4)		
Password	0 + Long Form Offset	Unsigned8	1
Access Groups	1 + Long Form Offset	Unsigned8	1
Access Rights	2 + Long Form Offset	Unsigned16	2
Extension Length	4 + Long Form Offset	Unsigned32	4
Extension Data	8 + Long Form Offset	OctetString	Extension Length
PI Name	8 + Long Form Offset + Extension Length	VisibleString	Name Length (see note 3)
NOTE 1 If Number Of Domains is 0, then this field is not present			
NOTE 2 If the Short Form is requested, these fields are not present.			
NOTE 3 If Name Length in the OD Object Description is 0, then this field is not present			

8.3.5.6.5 Structure of an Event in the S-OD

See Table 649 for definitions.

Table 649 – Structure of an Event in the S-OD

Field name	Octet offset	Data type	Octet length
Short Form Fields			
OD Tag = FF FF ₁₆	0	Unsigned16	2
Object Code = 20	2	Integer8	1
All Attributes Flag	3	Unsigned8	1
Object Description Length	4	Unsigned16	2
Reserved (set to zero)	6	Unsigned16	2
Index32	8	Unsigned32	4
Reserved	12	OctetString	3
Enabled	15	Unsigned8	1
Index Event Data	16	Unsigned32	4
Length	20	Unsigned32	4
Long Form Additional Fields (see note 1)	24		
Password	0 + Long Form Offset	Unsigned8	1
Access Groups	1 + Long Form Offset	Unsigned8	1
Access Rights	2 + Long Form Offset	Unsigned16	2
Extension Length	4 + Long Form Offset	Unsigned32	4
Extension Data	8 + Long Form Offset	OctetString	Extension Length
Event Name	8 + Long Form Offset + Extension Length	VisibleString	Name Length (see note 2)
NOTE 1 If the Short Form is requested, these fields are not present.			
NOTE 2 If Name Length in the OD Object Description is 0, then this field is not present			

8.3.5.6.6 Structure of a Data Type in the ST-OD

See Table 650 for definitions.

Table 650 – Structure of a Data Type in the ST-OD

Field name	Octet offset	Data type	Octet length
Short Form Fields			
OD Tag = FF FF ₁₆	0	Unsigned16	2
Object Code = 21	2	Integer8	1
All Attributes Flag	3	Unsigned8	1
Object Description Length	4	Unsigned16	2
Reserved (set to zero)	6	Unsigned16	2
Index32	8	Unsigned32	4
Reserved	10	Unsigned16	2
Length of Data Type	12	Unsigned32	4
Symbolic Name Length	16	Unsigned32	4
*Symbolic Name	20	VisibleString	Symbolic Name Length
NOTE If the Symbolic Name Length is 0, then this field is not present			

8.3.5.6.7 Structure of a Data Type Structure Description in the ST-OD

See Table 651 for definitions.

Table 651 – Structure of a Data Type Structure Description in the ST-OD

Field name	Octet offset	Data type	Octet length
Short Form Fields			
OD Tag = FF FF ₁₆	0	Unsigned16	2
Object Code = 22	2	Integer8	1
All Attributes Flag	3	Unsigned8	1
Object Description Length	4	Unsigned16	2
Reserved (set to zero)	6	Unsigned16	2
Index32	8	Unsigned32	4
Reserved (set to zero)	12	Unsigned16	2
Number of Elements	14	Unsigned16	2
List of Elements	16	Structure	8 x Number of Elements
Data Type Index		Unsigned32	4
Length		Unsigned32	4

8.3.5.6.8 Structure of a Simple Variable in the S-OD

See Table 652 for definitions.

Table 652 – Structure of a Simple Variable in the S-OD

Field name	Octet offset	Data type	Octet length
Short Form Fields			
OD Tag = FF FF ₁₆	0	Unsigned16	2
Object Code = 23	2	Integer8	1
All Attributes Flag	3	Unsigned8	1
Object Description Length	4	Unsigned16	2
Reserved (set to zero)	6	Unsigned16	2
Index32	8	Unsigned32	4
Data Type Index	12	Unsigned32	4
Length	16	Unsigned32	4
Long Form Additional Fields (see note 1)			
	20		
Local Address	0 + Long Form Offset	Unsigned64	8
Password	8 + Long Form Offset	Unsigned8	1
Access Groups	9 + Long Form Offset	Unsigned8	1
Access Rights	10 + Long Form Offset	Unsigned16	2
Extension Length	12 + Long Form Offset	Unsigned32	4
Extension Data	16 + Long Form Offset	OctetString	Extension Length
Variable Name	16 + Long Form Offset + Extension Length	VisibleString	Name Length (see note 2)
NOTE 1 If the Short Form is requested, these fields are not present.			
NOTE 2 If Name Length in the OD Object Description is 0, then this field is not present			

8.3.5.6.9 Structure of an Array in the S-OD

See Table 653 for definitions.

Table 653 – Structure of an Array in the S-OD

Field name	Octet offset	Data type	Octet length
Short Form Fields			
OD Tag = FF FF ₁₆	0	Unsigned16	2
Object Code = 24	2	Integer8	1
All Attributes Flag	3	Unsigned8	1
Object Description Length	4	Unsigned16	2
Reserved (set to zero)	6	Unsigned16	2
Index32	8	Unsigned32	4
Number of Elements	12	Unsigned32	2
Data Type Index	16	Unsigned32	4
Length	20	Unsigned32	4
Long Form Additional Fields (see note 1)			
	24		
Local Address	0 + Long Form Offset	Unsigned64	8
Password	8 + Long Form Offset	Unsigned8	1
Access Groups	9 + Long Form Offset	Unsigned8	1
Access Rights	10 + Long Form Offset	Unsigned16	2
Extension Length	12 + Long Form Offset	Unsigned32	4
Extension Data	16 + Long Form Offset	OctetString	Extension Length
Variable Name	16 + Long Form Offset + Extension Length	VisibleString	Name Length (see note 2)
NOTE 1 If the Short Form is requested, these fields are not present.			
NOTE 2 If Name Length in the OD Object Description is 0, then this field is not present			

8.3.5.6.10 Structure of a Record in the S-OD

See Table 654 for definitions.

Table 654 – Structure of a Record in the S-OD

Field name	Octet offset	Data type	Octet length
Short Form Fields			
OD Tag = FF FF ₁₆	0	Unsigned16	2
Object Code = 25	2	Integer8	1
All Attributes Flag	3	Unsigned8	1
Object Description Length	4	Unsigned16	2
Reserved (set to zero)	6	Unsigned16	2
Index32	8	Unsigned32	4
Data Type Index	12	Unsigned32	4
Reserved	16	Unsigned32	4
Long Form Additional Fields (see note 1)	20		
List of Local Addresses	0 + Long Form Offset	Unsigned64	K x 8 (see note 2)
Password	(K x 8) + Long Form Offset	Unsigned8	1
Access Groups	1 + (K x 8) + Long Form Offset	Unsigned8	1
Access Rights	2 + (K x 8) + Long Form Offset	Unsigned16	2
Extension Length	4 + (K x 8) + Long Form Offset	Unsigned32	4
Extension Data	8 + (K x 8) + Long Form Offset	OctetString	Extension Length
Record Name	8 + (K x 8) + Long Form Offset + Extension Length	VisibleString	Name Length (see note 3)
NOTE 1 If the Short Form is requested, these fields are not present.			
NOTE 2 K represents the number of fields in the record.			
NOTE 3 If Name Length in the OD Object Description is 0, then this field is not present			

8.3.5.6.11 Structure of a Variable List in the DV-OD

See Table 655 for definitions.

Table 655 – Structure of a Variable List in the DV-OD

Field name	Octet offset	Data type	Octet length
Short Form Fields			
OD Tag = FF FF ₁₆	0	Unsigned16	2
Object Code = 26	2	Integer8	1
All Attributes Flag	3	Unsigned8	1
Object Description Length	4	Unsigned16	2
Reserved (set to zero)	6	Unsigned16	2
Index32	8	Unsigned32	4
Deletable	12	Unsigned8	1
Reserved	13	Unsigned8	1
Number of Elements	14	Unsigned16	2
List of Element Indexes	16	Unsigned32	4 x Number of Elements
Long Form Additional Fields (see note 1)	20 + (K x 4) (see note 2)		
Password	0 + Long Form Offset	Unsigned8	1
Access Groups	1 + Long Form Offset	Unsigned8	1
Access Rights	2 + Long Form Offset	Unsigned16	2
Extension Length	4 + Long Form Offset	Unsigned32	4
Extension Data	8 + Long Form Offset	OctetString	Extension Length
Variable List Name	8 + Long Form Offset + Extension Length	VisibleString	Name Length (see note 3)
NOTE 1 If the Short Form is requested, these fields are not present.			
NOTE 2 K represents the Number of Elements in the record.			
NOTE 3 If Name Length in the OD Object Description is 0, then this field is not present			

8.3.6 Error APDU Bodies

8.3.6.1 Common Error Parameters

The following error parameter format as shown in Table 656 is used for many services.

Table 656 – Common Error Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Error Class	0	Unsigned8	1	This parameter identifies the category of the error. Its values are defined in the Error Class production of the Type 9 Abstract Syntax. The values are the tagged values. The value 9 has been added for the Type 5 Initiate Error APDU use.
Error Code	1	Unsigned8	1	This parameter identifies the specific type of error within the class. Its values are defined in the Error Class production of the Type 9 Abstract Syntax. The values are the enumerated values with parenthesized values, such as other = 0. Values for error class 11 have been added for Initiate Error message use.
Reserved	2	Unsigned8	1	Reserved set to 0
Additional Code	3	Unsigned8	1	This parameter provides an additional code that is associated with the error.
Additional Description	4	VisibleString	16	This parameter provides 16 octets of additional information that is associated with the error.

8.3.6.2 PI Error Parameters

See Table 657 for definitions.

Table 657 – PI Error Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Pi State	0	Unsigned8	1	Defined in Service Parameter.
Reserved	1	OctetString	3	Reserved, set to 0
Error Class	4	Unsigned8	1	Defined in Service Parameter.
Error Code	5	Unsigned8	1	Defined in Service Parameter.
Reserved	6	Unsigned8	1	Reserved, set to 0
Additional Code	7	Unsigned8	1	Defined in Service Parameter.
Additional Description	8	VisibleString	16	Defined in Service Parameter.

8.3.6.3 OD Error Parameters

See Table 658 for definitions.

Table 658 – OD Error Parameters

Parameter name	Octet offset	Data type	Octet length	Description
Index	0	Unsigned32	4	Defined in the Service Parameter.
Error Class	4	Unsigned8	1	Defined in the Service Parameter.
Error Code	5	Unsigned8	1	Defined in the Service Parameter.
Reserved	6	Unsigned8	1	Reserved, set to 0
Additional Code	7	Unsigned8	1	Defined in the Service Parameter.
Additional Description	8	VisibleString	16	Defined in the Service Parameter.

8.3.6.4 Error Class and Error Code Values

Table 659 specifies the FDA Agent error codes for each error class.

Table 659 – Error Class and Error Code Values

Class	Description: Code	Values:	
		Class	Code
vfd state	other	1	0
application reference	other	2	0
	application unreachable	2	1
definition	other	3	0
	object undefined	3	1
	object attributes inconsistent	3	2
	name already exists	3	3
resource	other	4	0
	memory unavailable	4	1
	max outstanding requests per session exceeded	4	2
	max sessions exceeded	4	3
	object creation failure	4	4
service	other	5	0
	object state conflict	5	1
	pdu size	5	2
	object constraint conflict	5	3
	parameter inconsistent	5	4
	illegal parameter	5	5
	unsupported service	5	6
	unsupported version	5	7
	invalid options	5	8
	unsupported protocol	5	9
	reserved	5	10
	key parameter mismatch	5	11
	assignments already made	5	12
	unsupported device redundancy state	5	13
	response time-out	5	14
	duplicate PD Tag detected	5	15
access	other	6	0
	object invalidated	6	1
	hardware fault	6	2
	object access denied	6	3
	invalid address	6	4
	object attribute inconsistent	6	5
	object access unsupported	6	6
	object non existent	6	7
	type conflict	6	8
	named access unsupported	6	9
	access to element unsupported	6	10
	config access already open	6	11
	reserved	6	12
	unrecognized FDA Address	6	13
od	other	7	0
	name length overflow	7	1
	od overflow	7	2
	od write protected	7	3
	extension length overflow	7	4
	od description length overflow	7	5
	operational problem	7	6
	type 5 to type 9 format conversion not supported	7	7

Class	Description: Code	Values:	
		Class	Code
other	other	8	0
reject	pdu size	9	5
sm reason code	other	10	0
	DLL Error - insufficient resources	10	1
	DLL Error - sending queue full	10	2
	DLL Error - time-out before transmission	10	3
	DLL Error - reason unspecified	10	4
	Device failed to respond to SET_PD_TAG	10	5
	Device failed to respond to WHO_HAS_PD_TAG	10	6
	Device failed to respond to SET_ADDR	10	7
	Device failed to respond to IDENTIFY	10	8
	Device failed to respond to ENABLE_SM_OP	10	9
	Device failed to respond to CLEAR_ADDRESS	10	10
	Multiple Response from WHO_HAS_PD_TAG	10	11
	Non-Matching PD_TAG from WHO_HAS_PD_TAG	10	12
	Non-Matching PD_TAG from IDENTIFY	10	13
	Non-Matching DEV_ID from IDENTIFY	10	14
	Remote Error Invalid State	10	15
	Remote Error PD-Tag doesn't match	10	16
	Remote Error Dev-ID doesn't match	10	17
	Remote Error SMIB object write failed	10	18
	Remote Error Starting SM Operational	10	19
initiate	other	11	0
	max-fms-pdu-size-insufficient	11	1
	feature-not-supported	11	2
	version-od-incompatible	11	3
	user-initiate-denied	11	4
	password-error	11	5
	profile-number-incompatible	11	6

8.4 FAL Protocol State Machine Structure

The FAL Protocol state machine structure defined for Type 9 applies.

8.5 SMK State Machine

8.5.1 General

The SMK protocol machine (SMKPM) defines the behavior of the SMK for those states that affect its communications.

In the protocol machine that follows:

1. The Invoke Id parameter is not included explicitly. Instead, it is considered to be an integral part of all confirmed service and Find Tag primitives. It is received from the sending SMK user and conveyed implicitly to the remote SMK user.
2. The response address is not included explicitly. It is always taken from the source network address of the request.
3. To send and receive SMK APDUs, ARs are not used. Instead, the SMKPM performs the necessary AR functions.
4. The double forward slash, i.e. "//" marks the beginning of a comment in a line.

8.5.2 Messages Received by the SMKPM

The SMKPM receives APDUs from the FDA Agent. These APDUs are modeled as Events of SMKPM transitions. The RcvMsg() function is used to represent the arrival of an APDU.

8.5.3 SMKPM Service Primitives and Local APs, Type 5 SMK Service Processors, and Type "X" Interface Functions

Table 660 shows the service primitives (in alphabetic order) used in the SMKPM. They are used as follows:

- Request service primitives are used as Events of SMKPM transitions. They are received from Local APs.
- Request service primitives are used also in Actions of SMKPM transitions. When used in this fashion, they cause the corresponding event to be delivered to the SMKPM and are processed immediately, ahead of all other queued events.
- Indication service primitives are used in Actions of SMKPM transitions. They may be delivered to Local APs or to Type "X" Interface Functions. There is one interface function for each Type "X" interface in a linking device. Each is identified by the link id of the FDA Address contained in the received APDU. Link Id 00 00 when used with the Type "X" NN.SS SMK group address means all Type "X" Interface Functions. Service names preceded by "LD_" identify services provided by Type "X" Interface Functions. These services take the sm_svc parameter of the received APDU as a parameter. They invoke one or more Type "X" SM services to satisfy requests received from the Type 5 SMK. Issuing one of these service requests causes the Type 5 SMK to save the Invoke Id so that it can match the response with the request. The Type "X" services used and their sequence is implementation dependent. If the Type "X" Interface Function receives an LD_FindTagQuery request and locates the queried object, it responds by issuing an SM_FindTagReply request.
- Response service primitives are used as Events of SMKPM transitions. They may be received from a Local AP or from an Interface Functions.
- Response service primitives are used also in Actions of SMKPM transitions. When used in this fashion, they cause the corresponding event to be delivered to the SMKPM and are processed immediately, ahead of all other queued events.
- Confirmation service primitives are used in Actions of SMKPM transitions. They are delivered to the requesting Local APs.

Table 660 – SMKPM Service Primitives

Service	Primitives			
	.req	.ind	.rsp*	.cnf
Type_X_ClearAddress		X	X	
Type_X_ClearAssignmentInfo		X	X	
Type_X_FindTagQuery		X		
Type_X_FindTagReply		X		
Type_X_SetAssignmentInfo		X	X	
SM_ClearAddress	X	X	X	X
SM_ClearAssignmentInfo	X	X	X	X
SM_DeviceAnnunciation	X			
SM_FindTagQuery	X	X		
SM_FindTagReply	X	X		
SM_Identify	X	X	X	X
SM_SetAssignmentInfo	X	X	X	X
*.rsp is the positive response primitive as well as the negative response primitive				

8.5.4 State Table Transition Processing

The state tables that follow are composed of a series of numbered transitions. The order of processing the transitions is based on the following rules:

- When receiving an event, locate the first transition in the table containing an instance of the event. A communications event is identified by its indication, e.g. SM_Clear Address.ind.
- Beginning with that transition, proceed down the state table, transition by transition, until a transition is located in which its Current state matches the current state of the state machine and its Event matches the received event.
- Test the conditions. If the result is false, proceed down to the next transition that matches the Current state and Event.
- If no transition matches, then ignore the event.
- When a transition is located, perform the specified action and change the current state of the state machine to the Next state specified by the transition. Then wait for the next event.

8.5.5 States

Table 661 and Figure 158 define the states and their transitions.

Table 661 – SMKPM states

No Address	The device does not have a network address on any of its Type 5 interfaces. If it has a network address on any of its Type 5 interfaces, it is not in this state. In this state, the SMKPM waits for an event that receives a network address through a local interface from the underlying layers.
No Tag	The device does not have valid assignment info. In this state, the SMK sends Annunciate requests and listens for a valid Set Assignment Info APDU. It also responds to Clear Address and SM Identify Request APDUs.
Operational	The device has a network address and valid assignment info. This is the normal operating state of the SMK.

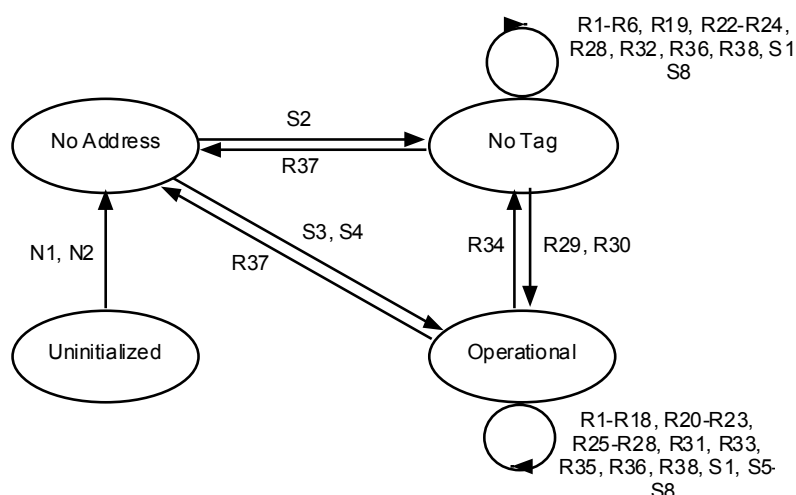


Figure 158 – State Transition Diagram for SMK

8.5.6 State Tables

Table 662 through Table 664 define the state machine.

Table 662 – SMKPM state table – Initialization

#	Current state	Event or condition => action	Next state
N1	Uninitialized	Powerup && OperationalRestore () = "false" => RestoreDefaults ()	No Address
N2	Uninitialized	Powerup && OperationalRestore () = "true" => // No action	No Address

Table 663 – SMKPM state table – Receive Transitions

#	Current state	Event or condition => action	Next state
R1	Operational No Tag	RcvMsg() = "Any Unconfirmed SM Req Message" && FdaAddressType (sm_svc) = "UNRECOGNIZED" => // Do nothing - no response for unconfirmed messages	Same
R2	Operational No Tag	RcvMsg() = "Any Confirmed SM Req Message" && FdaAddressType (sm_svc) = "UNRECOGNIZED" => SM_ConfirmedService.err { sm_service_type = SvcType (sm_svc), error_class = "access" error_code = "unrecognized FDA Address" addl_code = 2 }	Same
R3	Operational No Tag	RcvMsg() = "Any Confirmed SM Req Message" && FdaAddressType (sm_svc) = "GROUP SMK" => SM_ConfirmedService.err { sm_service_type = SvcType (sm_svc), error_class = "access" error_code = "unrecognized FDA Address" addl_code = 3 }	Same
R4	Operational No Tag	RcvMsg() = "Any Confirmed SM Req Message" && IsValid (sm_svc) = "false" => SM_ConfirmedService.err { sm_service_type = SvcType (sm_svc), error_class = "service" error_code = "parameter inconsistent" addl_code = GetAddlCode () }	Same
R5	Operational No Tag	RcvMsg() = "Any Confirmed SM Rsp Message" RcvMsg() = "Any Confirmed SM Error Message" => SM_ConfirmedService.cnf {} *	Same
R6	Operational No Tag	RcvMsg() = "SM_FindTagQuery" && IsValid (sm_svc) = "false" => // Do nothing - no response for unconfirmed messages	Same

Table 663 (continued)

#	Current state	Event or condition => action	Next state
R7	Operational	RcvMsg() = "SM_FindTagQuery" && (FdaAddressType (sm_svc) = "HSE SMK" FdaAddressType (sm_svc) = "GROUP SMK") && DeviceRedundancyState () = "secondary" && (QueryType (sm_svc) = "Secondary PD Tag" QueryType (sm_svc) = "Device Index") && QueryMatch (sm_svc) = "true" => SM_FindTagReply.req {}	Same
R8	Operational	RcvMsg() = "SM_FindTagQuery" && DeviceRedundancyState () = "secondary" => // Discard message and do nothing. Secondaries do not respond to // any other queries	Same
R9	Operational	RcvMsg() = "SM_FindTagQuery" && (FdaAddressType (sm_svc) = "HSE SMK" FdaAddressType (sm_svc) = "GROUP SMK") && (QueryType (sm_svc) = "Device Index" QueryType (sm_svc) = "VFD Reference") && QueryMatch (sm_svc) = "true" => SM_FindTagReply.req {}	Same
R10	Operational	RcvMsg() = "SM_FindTagQuery" && FdaAddressType (sm_svc) = "HSE SMK" && (QueryType (sm_svc) = "Primary PD Tag" QueryType (sm_svc) = "VFD Tag") && QueryMatch (sm_svc) = "true" => SM_FindTagReply.req {}	Same
R11	Operational	RcvMsg() = "SM_FindTagQuery" && FdaAddressType (sm_svc) = "GROUP SMK" && (QueryType (sm_svc) = "Primary PD Tag" QueryType (sm_svc) = "VFD Tag") && QueryMatch (sm_svc) = "true" => SM_FindTagReply.req {} Type_X_FindTagQuery.ind {}	Same
R12	Operational	RcvMsg() = "SM_FindTagQuery" && FdaAddressType (sm_svc) = "GROUP SMK" && (QueryType (sm_svc) = "Primary PD Tag" QueryType (sm_svc) = "VFD Tag") && QueryMatch (sm_svc) = "false" => Type_X_FindTagQuery.ind {}	Same
R13	Operational	RcvMsg() = "SM_FindTagQuery" && FdaAddressType (sm_svc) = "HSE SMK" && (QueryType (sm_svc) = "Function Block Tag" QueryType (sm_svc) = "Element ID") => SM_FindTagQuery.ind {}	Same
R14	Operational	RcvMsg() = "SM_FindTagQuery" && FdaAddressType (sm_svc) = "GROUP SMK" && (QueryType (sm_svc) = "Function Block Tag" QueryType (sm_svc) = "Element ID") => SM_FindTagQuery.ind {} Type_X_FindTagQuery.ind {}	Same
R15	Operational	RcvMsg() = "SM_FindTagQuery" && (FdaAddressType (sm_svc) = "Type_X SMK" FdaAddressType (sm_svc) = "LD Type_X INTERFACE SMK") && (QueryType (sm_svc) = "Primary PD Tag" QueryType (sm_svc) = "Function Block Tag" QueryType (sm_svc) = "Element ID" QueryType (sm_svc) = "VFD Tag") => Type_X_FindTagQuery.ind {}	Same

Table 663 (continued)

#	Current state	Event or condition => action	Next state
R16	Operational	RcvMsg() = "SM_FindTagReply" && FdaAddressType (sm_svc) = "HSE SMK" && DuplicateQueryIdMatch (sm_svc) = "true" && DevId_Match (sm_svc) = "false" => Set_DuplicatePdTagFlag () SM_DeviceAnnunciation.req {} Restart_HseRepeatTimer ()	Same
R17	Operational	RcvMsg() = "SM_FindTagReply" && FdaAddressType (sm_svc) = "HSE SMK" && DuplicateQueryIdMatch (sm_svc) = "true" && DevId_Match (sm_svc) = "true" => // Do nothing - the response is from this device	Same
R18	Operational	RcvMsg() = "SM_FindTagReply" && FdaAddressType (sm_svc) = "HSE SMK" => SM_FindTagReply.ind {} * Deliver indication to the AP that issued the Find Tag Query. How the SMKPM locates the requesting AP is not specified. There may be more than one way to do this. Error checking is performed on the response by the requesting AP.	Same
R19	No Tag	RcvMsg() = "SM_IdentifyReq" && (FdaAddressType (sm_svc) = "Type_X SMK" FdaAddressType (sm_svc) = "LD Type_X INTERFACE SMK") => SM_ConfirmedService.err { sm_service_type = SvcType (sm_svc), error_class = "service" error_code = "object state conflict " addl_code = 19 }	Same
R20	Operational	RcvMsg() = "SM_IdentifyReq" && FdaAddressType (sm_svc) = "Type_X SMK" && SmCacheEntry(sm_svc) = "false" => SM_ConfirmedService.err { sm_service_type = SvcType (sm_svc), error_class = "access" error_code = "object invalidated" addl_code = 20 }	Same
R21	Operational No Tag	RcvMsg() = "SM_IdentifyReq" => SM_Identify.rsp {}	Same
R22	Operational No Tag	(RcvMsg() = "SM_SetAssignmentInfoReq" RcvMsg() = "SM_ClearAssignmentInfoReq" RcvMsg() = "SM_ClearAddressReq") && FdaAddressType (sm_svc) = "LD Type_X INTERFACE SMK" => SM_ConfirmedService.err { sm_service_type = SvcType (sm_svc), error_class = "access" error_code = "object access denied" addl_code = 22 }	Same
R23	Operational No Tag	(RcvMsg() = "SM_SetAssignmentInfoReq" RcvMsg() = "SM_ClearAssignmentInfoReq" RcvMsg() = "SM_ClearAddressReq") && DevId_Match (sm_svc) = "false" => SM_ConfirmedService.err { sm_service_type = SvcType (sm_svc), error_class = "service" error_code = "key parameter mismatch" addl_code = GetAddlCode () }	Same

Table 663 (continued)

#	Current state	Event or condition => action	Next state
R24	No Tag	<pre> (RcvMsg() = "SM_SetAssignmentInfoReq" RcvMsg() = "SM_ClearAssignmentInfoReq" RcvMsg() = "SM_ClearAddressReq") && FdaAddressType (sm_svc) = "Type_X SMK" => SM_ConfirmedService.err { sm_service_type = SvcType (sm_svc), error_class = "service" error_code = "object state conflict" addl_code = 24 } </pre>	Same
R25	Operational	<pre> (RcvMsg() = "SM_SetAssignmentInfoReq" RcvMsg() = "SM_ClearAssignmentInfoReq" RcvMsg() = "SM_ClearAddressReq") && FdaAddressType (sm_svc) = "Type_X SMK" && DeviceRedundancyState () = "secondary" => SM_ConfirmedService.err { sm_service_type = SvcType (sm_svc), error_class = "access" error_code = "object access denied" addl_code = 25 } </pre>	Same
R26	Operational	<pre> (RcvMsg() = "SM_SetAssignmentInfoReq" RcvMsg() = "SM_ClearAssignmentInfoReq" RcvMsg() = "SM_ClearAddressReq") && PdTag_Match() = "false" => SM_ConfirmedService.err { sm_service_type = SvcType (sm_svc), error_class = "service" error_code = "key parameter mismatch" addl_code = GetAddlCode () } </pre>	Same
R27	Operational	<pre> RcvMsg() = "SM_SetAssignmentInfoReq" && FdaAddressType (sm_svc) = "Type_X SMK" => Type_X_SetAssignmentInfo.ind{} </pre>	Same
R28	Operational No Tag	<pre> RcvMsg() = "SM_SetAssignmentInfoReq" && PdTagDeviceIndex_Check(sm_svc) = "false" => SM_ConfirmedService.err { sm_service_type = SvcType (sm_svc), error_class = "service" error_code = "parameter inconsistent" addl_code = GetAddlCode () } </pre>	Same
R29	No Tag	<pre> RcvMsg() = "SM_SetAssignmentInfoReq" && FdaAddressType (sm_svc) = "HSE SMK" && DeviceRedundancyState () = "Primary" => Set_Assignment_Data (sm_svc) Clear_DuplicatePdTagFlag () SM_SetAssignmentInfo.rsp {} SM_DeviceAnnunciation.req {} Restart_HseRepeatTimer () SM_FindTagQuery.req {} </pre>	Operational
R30	No Tag	<pre> RcvMsg() = "SM_SetAssignmentInfoReq" && FdaAddressType (sm_svc) = "HSE SMK" && DeviceRedundancyState () = "Secondary" => Set_Assignment_Data (sm_svc) Clear_DuplicatePdTagFlag () SM_SetAssignmentInfo.rsp {} SM_DeviceAnnunciation.req {} Restart_HseRepeatTimer () </pre>	Operational

Table 663 (continued)

#	Current state	Event or condition => action	Next state
R31	Operational	RcvMsg() = "SM_SetAssignmentInfoReq" && FdaAddressType (sm_svc) = "HSE SMK" => Set_Assignment_Data (sm_svc) SM_SetAssignmentInfo.rsp {} SM_DeviceAnnunciation.req {} Restart_HseRepeatTimer ()	Same
R32	No Tag	RcvMsg() = "SM_ClearAssignmentInfoReq" && FdaAddressType (sm_svc) = "HSE SMK" => // This will force the device back to defaults RestoreDefaults () SM_ClearAssignmentInfo.rsp {} SM_DeviceAnnunciation.req {} Restart_HseRepeatTimer ()	Same
R33	Operational	RcvMsg() = "SM_ClearAssignmentInfoReq" && FdaAddressType (sm_svc) = "Type_X SMK" => Type_X_ClearAssignmentInfo.ind{}	Same
R34	Operational	RcvMsg() = "SM_ClearAssignmentInfoReq" && FdaAddressType (sm_svc) = "HSE SMK" => RestoreDefaults () SM_ClearAssignmentInfo.rsp {} SM_DeviceAnnunciation.req {} Restart_HseRepeatTimer ()	No Tag
R35	Operational	RcvMsg() = "SM_ClearAddressReq" && FdaAddressType (sm_svc) = "Type_X SMK" => Type_X_ClearAddress.ind{}	Same
R36	Operational No Tag	RcvMsg() = "SM_ClearAddressReq" && FdaAddressType (sm_svc) = "HSE SMK" && (AddressToClear(sm_svc) = "None" ConfigurationSessionActive = "True") => SM_ConfirmedService.err { sm_service_type = SvcType (sm_svc), error_class = "service" error_code = "object constraint conflict" addl_code = GetAddlCode () }	Same
R37	Operational No Tag	RcvMsg() = "SM_ClearAddressReq" && FdaAddressType (sm_svc) = "HSE SMK" && (NumberOfAssignedAddresses() = 1 AddressToClear(sm_svc) = "All") => SM_ClearAddress.rsp {} Clear_Address (interface_to_clear)	No Address
R38	Operational No Tag	RcvMsg() = "SM_ClearAddressReq" && FdaAddressType (sm_svc) = "HSE SMK" && NumberOfAssignedAddresses() > 1 => Clear_Address (interface_to_clear) SM_ClearAddress.rsp {} SM_DeviceAnnunciation.req {} Restart_HseRepeatTimer ()	Same

Table 664 – SMKPM state table – Internal Events

#	Current state	Event or condition => action	Next state
S1	Operational No Tag	HseRepeatTimerExpires() => SM_DeviceAnnunciation.req {} Restart_HseRepeatTimer ()	Same
S2	No Address	RcvNewNetworkAddress (interface, address) && AssignmentInfo_Set () = "false" => RestoreDefaults () NewAddress (interface, address) SM_DeviceAnnunciation.req {} Restart_HseRepeatTimer ()	No Tag
S3	No Address	RcvNewNetworkAddress (interface, address) && AssignmentInfo_Set () = "true" && DeviceRedundancyState () = "Primary" => Clear_DuplicatePdTagFlag () NewAddress (interface, address) SM_DeviceAnnunciation.req {} Restart_HseRepeatTimer () SM_FindTagQuery.req {}	Operational
S4	No Address	RcvNewNetworkAddress (interface, address) && AssignmentInfo_Set () = "true" && DeviceRedundancyState () = "Secondary" => Clear_DuplicatePdTagFlag () NewAddress (interface, address) SM_DeviceAnnunciation.req {} Restart_HseRepeatTimer ()	Operational
S5	Operational	// for new address for second HSE interface or for changing the address of either // HSE interface RcvNewNetworkAddress (interface, address) && NetworkAddressChange (interface, address) = "false" => NewAddress (interface, address) SM_DeviceAnnunciation.req {} Restart_HseRepeatTimer ()	Same
S6	Operational	SM_UnconfirmedService.req {} SM_ConfirmedService.req {} SM_ConfirmedService.rsp {} Type_X_ConfirmedService.rsp {} => Send_SM_ReqRspMessage (sm_svc)	Same
S7	Operational	SM_ConfirmedService.err {} Type_X_ConfirmedService.err {} => Send_SM_CommonErrorRsp (sm_service_type = SvcType (sm_svc), error_class = "result error class" error_code = "result error code" addl_code = "result error addl_code" addl_description = "result error addl_description")	Same
S8	Operational No Tag	SntpSyncLost() => SM_DeviceAnnunciation.req {} Restart_HseRepeatTimer ()	Same

8.5.7 Function Descriptions

8.5.7.1 General

The following descriptions define the functions referenced by the preceding state tables.

8.5.7.2 Event Functions

8.5.7.2.1 Summary

These functions are *callback* functions used to receive an event. They do not cause any modification of the data received. The other events are Service Primitives defined previously. See Table 665 through Table 668 for definitions of the functions.

8.5.7.2.2 HseRepeatTimerExpires ()

Table 665 – HseRepeatTimerExpires ()

Function	
This function is invoked when the Hse Repeat Timer expires.	
Input Parameters	Output Parameters
none	none

8.5.7.2.3 RcvNewNetworkAddress (interface, address)

Table 666 – RcvNewNetworkAddress (interface, address)

Function	
This function is invoked when an network address is received from the underlying layers. The input parameters identify the Type 5 interface and the received network address.	
Input Parameters	Output Parameters
interface, address	none

8.5.7.2.4 RcvMsg ()

Table 667 – RcvMsg ()

Function	
This function is invoked when an APDU is received. It conveys the sm_svc parameter to the state machine. This parameter contains all the fields of the APDU, including the header and trailer fields. The syntax RcvMsg() = "SM_XXX" is used to indicate that an APDU for SM service XXX has been received.	
Input Parameters	Output Parameters
sm_svc	none

8.5.7.2.5 SntpSyncLost ()

Table 668 – SntpSyncLost ()

Function	
This function is invoked when the state of synchronization of time between the device and the selected remote time server changes from true to false.	
Input Parameters	Output Parameters
none	none

8.5.7.3 Condition Functions

8.5.7.3.1 Summary

These functions are used to test an event. They do not cause any modification of the data tested. See Table 669 through Table 685 for definitions of the functions.

8.5.7.3.2 AddressToClear (sm_svc)**Table 669 – AddressToClear (sm_svc)**

Function	
Returns which Type 5 interfaces are being requested to be cleared. The values are OperationalNetworkAddress, Non-OperationalNetworkAddress, All, and None. None indicates that the specified Type 5 interface does not have an address or is not capable of dynamically requesting a new address..	
Input Parameters	Output Parameters
none	HseInterface

8.5.7.3.3 AssignmentInfo_Set ()**Table 670 – AssignmentInfo_Set ()**

Function	
Returns true if the SMK assignment information has been previously set by the Set Assignment service and is currently still valid as defined by that service.	
Input Parameters	Output Parameters
none	true or false

8.5.7.3.4 ConfigurationSessionActive ()**Table 671 – ConfigurationSessionActive ()**

Function	
Returns true if there is a configuration AR established.	
Input Parameters	Output Parameters
none	true or false

8.5.7.3.5 DeviceRedundancyState ()**Table 672 – DeviceRedundancyState ()**

Function	
Returns the redundancy state of the device.	
Input Parameters	Output Parameters
none	primary or secondary

8.5.7.3.6 DevId_Match (sm_svc)**Table 673 – DevId_Match (sm_svc)**

Function	
Returns true if the Device ID in the APDU exactly matches the Device ID of this device.	
Input Parameters	Output Parameters
sm_svc	true or false

8.5.7.3.7 DuplicateQueryIdMatch (sm_svc)**Table 674 – DuplicateQueryIdMatch (sm_svc)**

Function	
Returns true if the Invoke Id matches the Invoke Id of the Find Tag Request that was sent to determine if another device has the same PD Tag as this device.	
Input Parameters	Output Parameters
sm_svc	true or false

8.5.7.3.8 DuplicatePdTagDetected ()

Table 675 – DuplicatePdTagDetected ()

Function	
Returns true if the device has detected another device with the same Physical Device Tag as this device.	
Input Parameters	Output Parameters
sm_svc	true or false

8.5.7.3.9 FdaAddressType (sm_svc)

Table 676 – FdaAddressType (sm_svc)

Function	
Evaluates the FDA Address component of the sm_svc parameter and returns which SMK it identifies. For the return values defined below, LL LL identifies an Type "X" link connected to the linking device, and NN is the node address for that link interface. Returns: Type 5 SMK if FDA Address = 00 00.00.SM, where SM is the SMK selector GROUP SMK if FDA Address = 00 00.GG GG or LL LL.GG GG, where GG GG is the Type "X" SMK group address LD Type "X" Interface SMK if FDA Address = LL LL.NN.SM and NN is the node address of an Type "X" interface of the LD, Type "X" SMK if FDA Address = LL LL.NN.SM UNRECOGNIZED if FDA Address is not one of the above	
Input Parameters	Output Parameters
sm_svc	SMK Identifier

8.5.7.3.10 IsValid (sm_svc)

Table 677 – IsValid (sm_svc)

Function	
Returns true if the parameters in the APDU are valid for the service type. Data is considered valid if it has the correct data type and it conforms to the values defined for it. Parameter values are defined in the SMK service specifications in IEC 61158-5 and in the APDU specifications in this specification. The values are checked for validity in this function. Note the contrast with other condition functions defined for the SMKPM, such as PdTag_Match(sm_svc) that check for the use of the value.	
Input Parameters	Output Parameters
sm_svc	true or false

8.5.7.3.11 NetworkAddressChange (interface, address)

Table 678 – NetworkAddressChange (interface, address)

Function	
Returns true if the specified interface already has a valid network address.	
Input Parameters	Output Parameters
interface, address	true or false

8.5.7.3.12 NumberOfAssignedAddresses ()

Table 679 – NumberOfAssignedAddresses ()

Function	
Returns the number of Type 5 Interfaces that currently have an network address.	
Input Parameters	Output Parameters
none	NumberOfInterfaces

8.5.7.3.13 OperationalRestore ()**Table 680 – OperationalRestore ()**

Function	
Returns true if Operational Powerup is true and the last state of the SMKPM was Operational.	
Input Parameters	Output Parameters
none	true or false

8.5.7.3.14 PdTag_Match (sm_svc)**Table 681 – PdTag_Match (sm_svc)**

Function	
Returns true if the PD Tag in the APDU exactly matches the PD Tag of the device identified by the FDA Address of the APDU. Group FDA Addresses cause false to be returned.	
Input Parameters	Output Parameters
sm_svc	true or false

8.5.7.3.15 PdTagDeviceIndex_Check (sm_svc)**Table 682 – PdTagDeviceIndex_Check (sm_svc)**

Function	
Returns false if: 1. the service would cause PD Tag or the Device Index to be cleared or remain clear (both shall be set), or. 2. the PD Tag in the device is set before the service begins and the service is attempting to change it (the device index can be changed, but the PD Tag cannot). Otherwise, returns true.	
Input Parameters	Output Parameters
sm_svc	true or false

8.5.7.3.16 Query_Match (sm_svc)**Table 683 – Query_Match (sm_svc)**

Function	
Returns true if the device contains the queried object.	
Input Parameters	Output Parameters
sm_svc	true or false

8.5.7.3.17 QueryType (sm_svc)**Table 684 – QueryType (sm_svc)**

Function	
Returns the Query Type for a Find Tag Query or Find Tag Reply.	
Input Parameters	Output Parameters
sm_svc	query type

8.5.7.3.18 SmCacheEntry (sm_svc)**Table 685 – SmCacheEntry (sm_svc)**

Function	
Returns true if the SMK has cached the identification information for the Type "X" device identified by the fda_address component of the sm_svc parameter.	
Input Parameters	Output Parameters
sm_svc	true or false

8.5.7.4 Action Functions

8.5.7.4.1 Summary

These functions are used in the Action portion of the transitions. See Table 686 through Table 696 for definitions of the functions.

8.5.7.4.2 Clear_Address (interface_to_clear)

Table 686 – Clear_Address (interface_to_clear)

Function	
Close all FDA sessions and Type 9 VCRs for the address to be cleared. Stop the Type 5 Repeat Timer if it is running. Clear the address for the specified Type 5 interface in the local network address array . Request the underlying layers to clear the address. If the Operational network address is no longer valid, set it to the address of the other Type 5 interface if that address is valid.	
Input Parameters	Output Parameters
interface_to_clear	none

8.5.7.4.3 Clear_DuplicatePdTagFlag ()

Table 687 – Clear_DuplicatePdTagFlag ()

Function	
Performs the equivalent of DuplicateDetectedState := DuplicateDetectedState & NOT "DuplicatePdTag"	
Input Parameters	Output Parameters
none	none

8.5.7.4.4 Get_AddlCode ()

Table 688 – Get_AddlCode ()

Function	
Determines the value of the additional code variable. These values are defined later in this subclause	
Input Parameters	Output Parameters
none	addl_code

8.5.7.4.5 New_Address (interface, address)

Table 689 – New_Address (interface, address)

Function	
If the existing network address for the specified Type 5 interface is valid, close all client/server FDA sessions and Type 9 VCRs for that address. Set or change the address for the specified Type 5 interface in the local network address array. If the Operational network address is not valid or no longer valid, set it to this address. If the Operational Network Address was valid and was changed, then immediately move all existing publisher and report source VCRs and FDA Sessions to the new Operational Network Address.	
Input Parameters	Output Parameters
interface, address	none

8.5.7.4.6 Restart_HseRepeatTimer ()**Table 690 – Restart_HseRepeatTimer ()**

Function	
This function stops the HseRepeatTimer if it is running, and restarts it with the period of time defined by the Type 5 Repeat Time attribute. When this timer expires, it generates the HseRepeatTimerExpires event for the SM state machine.	
Input Parameters	Output Parameters
None	None

8.5.7.4.7 Restore_Defaults ()**Table 691 – Restore_Defaults ()**

Function	
Restore this device to its 'as manufactured' condition, except that network addresses and the Operational network address are retained. This closes all sessions and VCRs and removes all configured and operational data.	
Input Parameters	Output Parameters
none	none

8.5.7.4.8 Send_SM_CommonErrorRsp (sm_service_type, svc_spec_params)**Table 692 – Send_SM_CommonErrorRsp (sm_service_type, svc_spec_params)**

Function	
Builds and sends the error response APDU for the service identified in the sm_service_type parameter.	
Input Parameters	Output Parameters
sm_service_type, svc_spec_params	none

8.5.7.4.9 Send_SM_ReqRspMessage (sm_svc)**Table 693 – Send_SM_ReqRspMessage (sm_svc)**

Function	
Builds and sends the request or response APDU for the service identified in the sm_svc parameter.	
Input Parameters	Output Parameters
sm_svc	none

8.5.7.4.10 Set_Assignment_Data (sm_svc)**Table 694 – Set_Assignment_Data (sm_svc)**

Function	
Copies assignment data for an Type 5 device from the APDU to matching SMK attributes. If the Operational IP Address changes, then immediately move all existing publisher and report source VCRs and FDA Sessions to the Operational IP Address.	
Input Parameters	Output Parameters
sm_svc	none

8.5.7.4.11 Set_DuplicatePdTagFlag ()**Table 695 – Set_DuplicatePdTagFlag ()**

Function	
Sets the DuplicateDetectedState attribute to reflect that this device has detected that this device and another device have the same PD Tag.	
Input Parameters	Output Parameters
none	none

8.5.7.4.12 SvcType (sm_svc)

Table 696 – SvcType (sm_svc)

Function	
This function decodes the data that is conveyed in the sm_svc parameter and returns the service type.	
Input Parameters	Output Parameters
sm_svc	service type

8.5.8 Error Classes and Codes

The Error Classes and Codes defined in 8.3.6.4 apply.

8.5.9 Additional Code Definition

Table 697 and Table 698 define the values that may be used in the Additional Code octet of the FDA Common Error classes. This definition is local to SM services.

Table 697 – Additional Code Used by Error Class and Code

FDA error class	FDA error code	Additional code value for SMKPM	Provided By
service	key parameter mismatch	see Table 698	GetAddlCode ()
	parameter inconsistent	see Table 698	GetAddlCode ()
	object constraint conflict	see Table 698	GetAddlCode ()

Table 698 – Additional Code Parameter Ids

Parameter	Additional code
Clear Duplicate Detection State	5
Device ID	10
Device Index	20
Device Redundancy State	25
Element ID	30
FDA Address	35
Function Block Tag	40
Type "X" New Address	45
Interface to Clear	50
Invoke ID	55
PD Tag	60
Type 5 Repeat Time	65
LAN Redundancy Addr	70
Max Device Index	75
Operational IP Address	80
Query Type	85
VFD Reference	90
VFD Tag	95
Interface does not have an address	101
Interface has a fixed address	102
Interface has a configuration session	103

8.6 VCR State Machine

The Type 5 VCR state machine is defined by the Type 9 VCR State Machine, with the following modifications:

- The Type 5 Initiate service replaces the Type 9 Initiate service. Type 9 Initiate service parameters that are not defined for the Type 5 Initiate service are ignored in the state machine.

- References to the AR Abort service by the Type 9 Connection Establishment state transitions are modified as follows. The Abort request primitive is replaced by the sending of an Type 5 VCR Abort request APDU, and the receipt of a Type 9 Abort indication primitive is replaced by the receipt of an Type 5 VCR Abort request APDU. The only exception to this rule is when an Type 9 Abort.request primitive is issued in response to the receipt of an Type 9 Initiate request PDU when the VCR is in the Connection Established state. In this case, the FDA Agent returns an Type 5 Initiate Error APDU with the same error class and error code supplied in the Type 9 Abort.request primitive.
- The Type 5 VCR Client and Server endpoints use Type 5 Idle APDUs as "keep alive" APDUs as follows.
- Client Type 5 VCR endpoints send Idle request APDUs only when the Idle timer expires.
- Server Type 5 VCR endpoints immediately send Idle Response APDUs when they receive Idle request APDUs.
- Client endpoints restart their Idle Timers whenever they send a Type 5 APDU.
- Server endpoints do not maintain an Idle timer.
- Upon expiration of their Inactivity timer, client and server Type 5 VCR endpoints terminate after sending an Abort APDU.
- Requests to establish an Type 5 VCR of type "Type 9 CLIENT" using the VCR Selector option are rejected if the remote selector address of the Type 9 Client VCR indicates that the remote VFD is the MIB VFD. The error class used is "access" and the error code is "object access denied".
- Server endpoints established for MIB access respond negatively to received Type 9 update service requests using error class "access" and error code "object access denied" if the underlying AR is not a configuration AR. The Type 9 update service requests are:

Generic Initiate Download Sequence
 Generic Download Segment
 Generic Terminate Download Sequence
 Initiate Download Sequence
 Download Segment
 Terminate Download
 Request Domain Download
 Write
 Write with Subindex

8.7 FAL Service Protocol Machine (FSPM)

8.7.1 Type 5 FSPM

The formal interface between an Type 5 VCR and its underlying AR maintains compatibility with the Type 9 VCR state transitions used for the Type 5 VCR State Machine.

The Type 5 FSPM is defined by the Type 9 FSPM. It is used in its entirety with the following modifications:

- The Confirmed Send and Unconfirmed Send service primitives for sending and receiving have been extended to contain the Type 5_VCR_ID parameter to identify the VCR that uses the session endpoint.
- All references to remote DLCEP addresses are replaced by references to remote addresses. When used with ARs that operate over connections, the remote address parameter identifies the connection.
- The Abort service is not conveyed between the FSPM and the Type 5 VCR state machines. Instead, the Type 5 VCR Abort APDU is conveyed between the two state machines using the Unconfirmed Send service.

- Some service parameter differences exist in the service primitives exchanged between the Type 5 FSPM and the Type 5 ARPM. The service primitives and their parameters are defined in 8.8.1.1.1.

8.8 Application Relationship Protocol Machines (ARPMs)

8.8.1 Client / Server Session ARPM

8.8.1.1 Primitive Definitions

8.8.1.1.1 Primitives Exchanged between ARPM and FSPM

Table 699 and Table 700 define the primitives exchanged between the ARPM and the FSPM.

Table 699 – Primitives issued by FSPM to ARPM

Primitive name	Source	Associated parameters	Functions
EST_req	FSPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish request primitive from the FSPM to the ARPM.
EST_rsp(+)	FSPM	arep_id, user_data,	This is an FAL internal primitive used to convey an Establish response(+) primitive from the FSPM to the ARPM.
EST_rsp(-)	FSPM	arep_id, user_data, service_type	This is an FAL internal primitive used to convey an Establish response(-) primitive from the FSPM to the ARPM.
Abort_req	FSPM	vcr_id, abort_detail, abort_identifier, reason_code	This is an FAL internal primitive used to convey an Abort request primitive from the FSPM to the ARPM.
CS_req	FSPM	vcr_id, service_type, user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) request primitive from the FSPM to the ARPM.
CS_rsp	FSPM	vcr_id, service_type, user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) response primitive from the FSPM to the ARPM.
UCS_req	FSPM	vcr_id, service_type, user_data	This is an FAL internal primitive used to convey an Unconfirmed Send (UCS) request primitive from the FSPM to the ARPM.

Table 700 – Primitives issued by ARPM to FSPM

Primitive name	Source	Associated parameters	Functions
EST_ind	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish indication primitive from the ARPM to the FSPM.
EST_cnf(+)	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish confirmation(+) primitive from the ARPM to the FSPM.
EST_cnf(-)	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish confirmation(-) primitive from the ARPM to the FSPM.
Abort_ind	ARPM	vcr_id, abort_detail, abort_identifier, reason_code	This is an FAL internal primitive used to convey an Abort primitive from the ARPM to the FSPM.
CS_ind	ARPM	vcr_id, service_type, user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) indication primitive from the ARPM to the FSPM.
CS_cnf(+)	ARPM	vcr_id, service_type, user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) confirmation(+) primitive from the ARPM to the FSPM.
CS_cnf(-)	ARPM	vcr_id, service_type, user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) confirmation(-) primitive from the ARPM to the FSPM.
UCS_ind	ARPM	vcr_id, service_type, user_data	This is an FAL internal primitive used to convey an Unconfirmed Send (UCS) indication primitive from the ARPM to the FSPM.

8.8.1.1.2 Parameters of FSPM/ARPM Primitives

The parameters used with the primitives exchanged between the FSPM and the ARPM are described in Table 701.

Table 701 – Parameters used with primitives exchanged between FSPM and ARPM

Parameter name	Description
arep_id	This parameter is used to unambiguously identify an instance of the AREP that has issued a primitive. A means for such identification is not specified by this standard.
user_data	This parameter conveys FAL-User data.
abort_identifier	This parameter conveys the value that is used for the Abort_Identifier parameter.
reason_code	This parameter conveys the value that is used for the Reason_Code parameter.
abort_detail	This parameter conveys the value that is used for the Abort_Detail parameter.
vcr_id	This parameter is used to unambiguously identify an instance of the VCR that issued a primitive or that is the primitive destination. A means for such identification is not specified by this standard.
service_type	This parameter conveys the protocol Id and the service within the protocol.

8.8.1.2 DLL Mapping of Client / Server AREP Class

8.8.1.2.1 Formal Definition

This subclause describes the mapping of the Client / Server AREP Class to the Socket model. The DLL Mapping attributes and their permitted values used with the Client / Server AREP class are defined in this subclause.

CLASS: ClientServer

PARENT CLASS: AR Endpoint

ATTRIBUTES:

- | | | |
|-----|-------------------|---------------------------|
| 1 | (m) KeyAttribute: | LocalAndRemoteAddresses |
| 1.1 | (m) KeyAttribute: | LocalAddress |
| 1.2 | (m) KeyAttribute: | RemoteAddress |
| 2 | (m) Attribute: | TransferType (CONN, CNLS) |
| 3 | (m) Attribute: | BufferSize |
| 4 | (m) Attribute: | TransmitDelayTime |
| 5 | (m) Attribute: | InactivityCloseTime |
| 6 | (m) Attribute: | HdrOptions |
| 7 | (m) Attribute: | GuardBand |
| 8 | (m) Attribute: | MibConfigurationUse |
| 9 | (m) Attribute: | MaxAduLength |
| 10 | (m) Attribute: | MaxOutstanding |
| 11 | (m) Attribute: | ServerPhysicalDeviceTag |

8.8.1.2.2 Attributes

LocalAndRemoteAddresses

This key attribute identifies the AREP. It is composed of the local and remote addresses of the underlying layer for the AREP.

TransferType

This attribute specifies whether the connection-oriented (CONN) or connectionless (CNLS) services of the underlying layer are used by the AREP.

BufferSize

This attribute specifies the size in octets of the session endpoint's send buffer. The buffer holds an integral number of APDUs. If an APDU is received for transfer that would overflow the buffer, the buffer is sent first, and the APDU is then added as the first APDU in the buffer.

TransmitDelayTime

This attribute specifies the amount of time that a sending session endpoint accumulates APDUs to send in its buffer. The time period starts when the first APDU is placed into the transmit buffer after either the buffer has been initialized or after it has been emptied by the previous transmission.

If the endpoint has no APDUs to send (the buffer is empty) and it receives an APDU to send, it loads the APDU into the buffer and waits for for additional APDUs to concatenate into the buffer. If the buffer fills before the Transmit Delay Time has been reached, the Type 5 FAL AE stops the timer and sends the buffer. When this time interval has expired and the buffer has not yet filled, it sends the contents of the buffer.

InactivityCloseTime

This attribute is used by Client and Server endpoints to determine when to close their endpoints because of inactivity.

HdrOptions

This attribute specifies which APDU Trailer fields are present in the APDUs sent and received on the session.

GuardBand

This attribute specifies the lower and upper bound for the field being protected from rollover.

MibConfigurationUse

This Boolean attribute specifies, when TRUE, that the AR is used for MIB configuration. This is not a configurable attribute. Client applications are expected to know that they need configuration ARs and request them with the Establish request.

MaxAduLength

This attribute specifies the maximum permitted length in octets of APDUs sent by the CLIENT AREPs and received by SERVER AREPs.

MaxOutstanding

This attribute specifies the maximum number of requests that the receiving endpoint may have outstanding at any point in time. Outstanding means that the endpoint has received a request message, but has not returned the corresponding response message. Its initial value is set by the device manufacturer. If a request message is received that causes this attribute to be exceeded, a negative response is returned with error class = "resource" and error code = "max outstanding requests per session exceeded".

ServerPhysicalDeviceTag

This attribute specifies the SMK PhysicalDeviceTag of the server endpoint.

8.8.1.3 Client / Server AREP State Machine

8.8.1.3.1 Client / Server ARPM States

The defined states and their descriptions of the Client / Server ARPM are shown in Table 702 and Figure 159. If the transfer type = CONN, then the AR operates over an underlying connection, and the state machine does not take effect until the underlying connection has been established. If the first message received on the underlying connection is not an Establish request, then the underlying connection is terminated.

Table 702 – Client / Server ARPM states

States	Description
CLOSED	Client and Server AREPs shall be created prior to use. They are created in the CLOSED state. When the FDA Agent receives an OpenSession Request FDA-PDU it dynamically creates a new server AREP and delivers the received FDA-PDU to it. AREPs in the CLOSED state are not capable of sending or receiving FDA-PDUs except for OpenSession FDA-PDUs. Transitioning an AREP to the CLOSED state causes its timers to be stopped, its underlying socket connection to be closed if necessary, and it is then deleted.
OPEN	The AREP is defined and capable of sending or receiving FAL-PDUs.
REQUESTING	The AREP has sent an Establish Request FAL-PDU and is waiting for a response from the remote AREP.
RESPONDING	The AREP has received an Establish Request FAL-PDU, delivered an Establish.ind primitive and is waiting for a response from its user.
SAME	Indicates that the next state is the same as the current state.

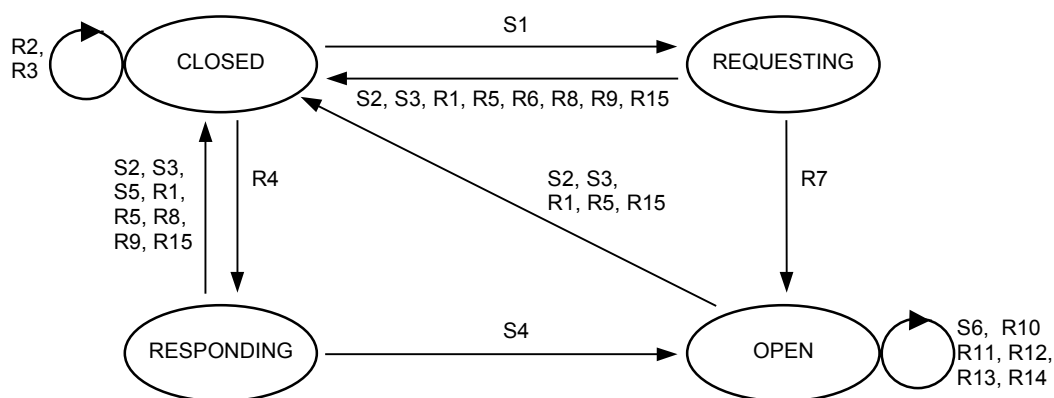
**Figure 159 – State Transition Diagram of Client / Server ARPM****8.8.1.3.2 Client / Server ARPM state table**

Table 703 and Table 704 define the Client/Sever ARPM state machine.

Table 703 – Client / Server ARPM state table – Sender transitions

#	Current state	Event or condition => action	Next state
S1	CLOSED	EST_req && EndpointType = "CLIENT" => FAL-PDU_req { smpm_service_name = "SMPM_Send", arep_id = GetArepld (), remote_address = RemoteAddress, fda_pdu = BuildFAL-ReqRspPDU (service_type = "AR Establish Req", fal_data = user_data) }, StartInactivityCloseTimer()	REQUESTING
S2	RESPONDING REQUESTING OPEN	Abort_req =>	CLOSED
S3	REQUESTING RESPONDING OPEN	InactivityCloseTimerExpires =>	CLOSED
S4	RESPONDING	EST_rsp(+) => FAL-PDU_req { smpm_service_name = "SMPM_Send", arep_id = GetArepld (), remote_address = RemoteAddress, fda_pdu = BuildFAL-ReqRspPDU (service_type = "AR Establish Rsp", fal_data = user_data) } StartInactivityCloseTimer()	OPEN
S5	RESPONDING	EST_rsp(-) => FAL-PDU_req { smpm_service_name = "SMPM_Send", arep_id = GetArepld (), remote_address = RemoteAddress, fda_pdu = BuildFAL-ReqRspPDU (service_type = "AR Establish Err", fal_data = user_data) }	CLOSED
S6	OPEN	UCS_req CS_req CS_rsp => FAL-PDU_req { smpm_service_name = "SMPM_Send", arep_id = GetArepld (), remote_address = RemoteAddress, fda_pdu = BuildFAL-ReqRspPDU (service_type = service_type, fal_data = user_data) }	OPEN

Table 704 – Client / Server ARPM state table – Receiver transitions

#	Current state	Event or condition => action	Next state
R1	REQUESTING RESPONDING OPEN	FAL-PDU_ind && FDA_PDU_Valid(fal_pdu) = False => <pre> FAL-PDU_req* { smpm_service_name = "SMPM_Send", arep_id = GetArepld (), remote_address = FAL_Pdu_RemoteAddress(fal_pdu), fal_pdu = BuildFAL-ErrPDU(request_pdu = fal_pdu, error_class = "service", error_code = appropriate error code for detected error) } </pre> * Used only when the received FAL PDU is a DTC Request	CLOSED
R2	CLOSED	FAL-PDU_ind && EndpointType = "SERVER" && FAL_Pdu_SvcType(fda_pdu) = "AR Establish Req" && FAL_PDU_Valid(fda_pdu) = False => <pre> FAL-PDU_req { smpm_service_name = "SMPM_Send", arep_id = GetArepld (), remote_address = FAL_Pdu_RemoteAddress (fal_pdu), fda_pdu = BuildFAL-ErrPDU(request_pdu = fal_pdu, error_class = "service", error_code = appropriate error code for detected error) } </pre>	CLOSED
R3	CLOSED	FAL-PDU_ind && EndpointType = "SERVER" && FAL_Pdu_SvcType (fal_pdu) = "AR Establish Req" && ConfigurationArCheckOK (fal_pdu) = False => <pre> FAL-PDU_req { smpm_service_name = "SMPM_Send", arep_id = GetArepld (), remote_address = FAL_Pdu_RemoteAddress (fal_pdu), fal_pdu = BuildFAL-ErrPDU(request_pdu = fal_pdu, error_class = "access", error_code = "config access already open") } </pre>	CLOSED
R4	CLOSED	FAL-PDU_ind && EndpointType = "SERVER" && FAL_Pdu_SvcType (fal_pdu) = "AR Establish Req" && ConfigurationArCheckOK(fal_pdu) = True => <pre> RemoteAddress = FAL_Pdu_RemoteAddress () EST_ind* { arep_id = GetArepld (), user_data = GetUserData (fal_pdu) } StartInactivityCloseTimer() </pre> * The server entity that receives this indication is responsible for performing parameter negotiations.	RESPONDING
R5	OPEN RESPONDING REQUESTING	FAL-PDU_ind && FAL_Pdu_SvcType (fal_pdu) = "AR Establish Req" => <pre> FAL-PDU_req { smpm_service_name = "SMPM_Send", arep_id = GetArepld (), remote_address = FAL_Pdu_RemoteAddress (), fal_pdu = BuildFAL-ErrPDU(request_pdu = fal_pdu, error_class = "service", error_code = "object state conflict") } </pre>	CLOSED

Table 704 (continued)

#	Current state	Event or condition => action	Next state
R6	REQUESTING	FAL-PDU_ind && FAL_Pdu_SvcType (fal_pdu) = "AR Establish Err" => EST_cnf(-) { arep_id = GetArepld (), user_data = FAL_Pdu_ServiceSpecificParameters(fal_pdu) }	CLOSED
R7	REQUESTING	FAL-PDU_ind && FAL_Pdu_SvcType (fal_pdu) = "AR Establish Rsp" => BufferSize = FAL_Pdu_BufferSize(fal_pdu) InactivityCloseTime = GetInactivityCloseTime(fal_pdu) EST_cnf(+) { arep_id = GetArepld (), user_data = FAL_Pdu_ServiceSpecificParameters(fal_pdu) } StartInactivityCloseTimer()	OPEN
R8	RESPONDING REQUESTING	FDA-PDU_ind && FDA_Pdu_SvcType (fda_pdu) = "CS_ReqPDU" => FDA-PDU_req { smpm_service_name = "SMPM_Send", arep_id = GetArepld (), remote_address = FDA_Pdu_RemoteAddress (fda_pdu), fda_pdu = BuildFDA-ErrPDU(request_pdu = fda_pdu, error_class = "service", error_code = "object state conflict") } StartInactivityCloseTimer()	CLOSED
R9	RESPONDING	FAL-PDU_ind && FAL_Pdu_SvcType (fal_pdu) = "ANY FAL PDU" =>	CLOSED
R10	OPEN	FAL-PDU_ind && FAL_Pdu_SvcType (fal_pdu) = "UCS_ReqPDU" && FAL_Pdu_GetVcrId() <> NULL => UCS_ind { arep_id = GetArepld (), vcr_id = FAL_Pdu_GetVcrId(), service_type = FAL_Pdu_SvcType(fal_pdu), user_data = FAL_Pdu_ServiceSpecificParameters(fal_pdu) } StartInactivityCloseTimer()	OPEN
R11	OPEN	FAL-PDU_ind && FAL_Pdu_SvcType (fal_pdu) = "CS_ReqPDU" && FAL_Pdu_GetVcrId() = NULL => FAL-PDU_req { smpm_service_name = "SMPM_Send", arep_id = GetArepld (), remote_address = FAL_Pdu_RemoteAddress (fal_pdu), fal_pdu = BuildFAL-ErrPDU(request_pdu = fal_pdu, error_class = "access", error_code = "unrecognized FDA Address") } StartInactivityCloseTimer()	OPEN
R12	OPEN	FAL-PDU_ind && FAL_Pdu_SvcType (fal_pdu) = "CS_ReqPDU" => CS_ind { arep_id = GetArepld (), vcr_id = FAL_Pdu_GetVcrId(), service_type = FAL_Pdu_SvcType(fal_pdu), user_data = FAL_Pdu_ServiceSpecificParameters(fal_pdu) } StartInactivityCloseTimer()	OPEN

Table 704 (continued)

#	Current state	Event or condition => action	Next state
R13	OPEN	FAL-PDU_ind && FAL_Pdu_SvcType (fda_pdu) = "DTC_ReqPDU" && MaxOutstandingReached() = True => FAL-PDU_req { smpm_service_name = "SMPM_Send", arep_id = GetArepld (), remote_address = FAL_Pdu_RemoteAddress (fal_pdu), fda_pdu = BuildFAL-ErrPDU(request_pdu = fal_pdu, error_class = "resource", error_code = "max outstanding requests per session exceeded") } StartInactivityCloseTimer()	OPEN
R14	OPEN	FAL-PDU_ind && (FAL_Pdu_SvcType (fal_pdu) = "CS_RspPDU" FAL_Pdu_SvcType (fal_pdu) = "CS_ErrPDU") => CS_cnf { arep_id = GetArepld (), vcr_id = FAL_Pdu_GetVcrlid(), service_type = FAL_Pdu_SvcType(fal_pdu), user_data = FAL_Pdu_ServiceSpecificParameters(fal_pdu) } StartInactivityCloseTimer()	OPEN
R15	OPEN REQUESTING RESPONDING	ErrorToARPM && ErrorType = "socket disconnect" =>	CLOSED

8.8.2 ARPM

8.8.2.1 General

This ARPM is used by Publisher/Subscriber and Report Distribution ARs.

8.8.2.2 Primitive definitions

8.8.2.2.1 Primitives exchanged between ARPM and FSPM

Table 705 and Table 706 list the primitives exchanged between the ARPM and the FSPM.

Table 705 – Primitives issued by FSPM to ARPM

Primitive name	Source	Associated parameters	Functions
EST_req	FSPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish request primitive from the FSPM to the ARPM.
Abort_req	FSPM	vcr_id, abort_detail, abort_identifier, reason_code	This is an FAL internal primitive used to convey an Abort request primitive from the FSPM to the ARPM.
UCS_req	FSPM	vcr_id, service_type, user_data	This is an FAL internal primitive used to convey an Unconfirmed Send (UCS) request primitive from the FSPM to the ARPM.

Table 706 – Primitives issued by ARPM to FSPM

Primitive name	Source	Associated parameters	Functions
EST_cnf(+)	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish response(+) primitive from the ARPM to the FSPM.
Abort_ind	ARPM	vcr_id, abort_detail, abort_identifier, reason_code	This is an FAL internal primitive used to convey an Abort primitive from the ARPM to the FSPM.
UCS_ind	ARPM	vcr_id, service_type, user_data	This is an FAL internal primitive used to convey an Unconfirmed Send (UCS) indication primitive from the ARPM to the FSPM.

8.8.2.2.2 Parameters of FSPM/ARPM primitives

The parameters used with the primitives exchanged between the FSPM and the ARPM are described in Table 707.

Table 707 – Parameters used with primitives exchanged between FSPM and ARPM

Parameter name	Description
arep_id	This parameter is used to unambiguously identify an instance of the AREP that has issued a primitive. A means for such identification is not specified by this standard.
vcr_id	This parameter is used to unambiguously identify an instance of the VCR that issued a primitive or that is the primitive destination. A means for such identification is not specified by this standard.
user_data	This parameter conveys AR-User data (Including the service specific parameters).
abort_detail	This parameter conveys the value that is used for the Abort_Detail parameter.
abort Identifier	This parameter conveys the value that is used for the Identifier parameter.
reason_code	This parameter conveys the value that is used for the Reason_Code parameter.
remote_address	This parameter conveys the remote address of the underlying layer.
service_type	This parameter conveys the protocol Id and the service within the protocol.

8.8.2.3 DLL Mapping of MulticastAREP Class

8.8.2.3.1 Formal Definition

This subclause describes the mapping of the Publisher / Subscriber and Report Distribution AREP Classes to the Socket model. The Socket Mapping attributes and their permitted values used with the Publisher / Subscriber and Report Distribution AREP classes are defined in this subclause.

CLASS: Multicast

PARENT CLASS: AR Endpoint

ATTRIBUTES:

- 1 (m) KeyAttribute: LocalAndRemoteAddresses
- 1.1 (m) KeyAttribute: LocalAddress
- 1.2 (m) KeyAttribute: RemoteAddress
- 2 (m) Attribute: TransferType (CONN, CNLS)
- 4 (c) Constraint: Role = SUBSCRIBER | REPORT SINK
- 4.1 (m) Attribute: GuardBand
- 5 (c) Constraint: Role = PUBLISHER | REPORT SOURCE
- 5.1 (m) Attribute: TransmitDelayTime
- 6 (m) Attribute: BufferSize
- 7 (m) Attribute: HdrOptions
- 8 (c) Constraint: Role = SUBSCRIBER | REPORT SINK
- 8.1 (m) Attribute: MaxApduLength

8.8.2.3.2 Attributes

LocalAndRemoteAddresses

This key attribute identifies the AREP. It is composed of the local and remote addresses of the underlying layer for the AREP.

TransferType

This attribute specifies whether the connection-oriented (CONN) or connectionless (CNLS) services of the underlying layer are used by the AREP.

GuardBand

This conditional attribute specifies the lower and upper bound for the field being protected from rollover. It is present if the ROLE is SUBSCRIBER or REPORT SINK.

TransmitDelayTime

This conditional attribute specifies the amount of time that a sending session endpoint accumulates messages to send in its buffer. It is present if the ROLE is PUBLISHER or REPORT SOURCE. The time period starts when the first message is placed into the transmit buffer after either the buffer has been initialized or after it has been emptied by the previous transmission.

If the endpoint has no messages to send (the buffer is empty) and it receives a message to send, it loads the message into the buffer and waits for additional messages to concatenate into the buffer. If the buffer fills before the Transmit Delay Time has been reached, the Type 5 FAL AE stops the timer and sends the buffer. When this time interval has expired and the buffer has not yet filled, it sends the contents of the buffer.

BufferSize

This attribute specifies the size in octets of the session endpoint's send buffer. The buffer holds an integral number of messages. If a message is received for transfer that would overflow the buffer, the buffer is sent first, and the message is then added as the first message in the buffer.

HdrOptions

This attribute specifies which Message Trailer fields are present in the messages sent and received on the session.

MaxApduLength

This conditional attribute specifies the maximum permitted length in octets of APDUs sent and received on the AR. It is present if the ROLE is SUBSCRIBER or REPORT SINK.

8.8.2.4 MulticastARPM State Machine

8.8.2.4.1 MulticastARPM States

The defined states and their descriptions of the Publisher / Subscriber ARPM are shown in Table 708 and Figure 160. For CONN, the closed state is never left until the connection has been established. How this connection is established is implementation dependent.

Table 708 – Publisher / Subscriber ARPM States

CLOSED	The AREP is defined, but not capable of sending or receiving arbitrary FAL-PDUs. It may send or receive Establish service FAL-PDUs while in this state.
OPEN	The AREP is defined and capable of sending or receiving FAL-PDUs.

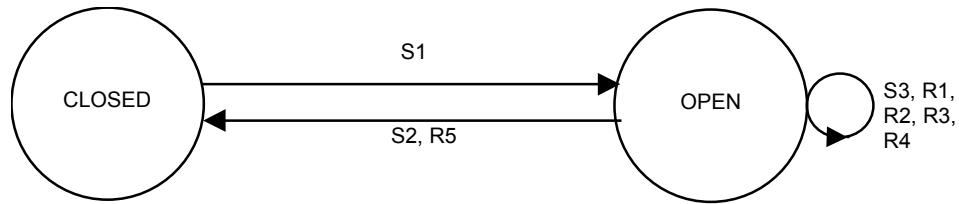


Figure 160 – State Transition diagram of the publisher / subscriber ARPM

8.8.2.4.2 Multicast ARPM state table

Table 709 and Table 710 define the Multicast ARPM state machine.

Table 709 – MulticastARPM state table – sender transitions

#	Current state	Event or condition => action	Next state
S1	CLOSED	EST_req => EST_cnf(+){ arep_id := GetArepId (), user_data := "null" }	OPEN
S2	OPEN	Abort_req =>	CLOSED
S3	OPEN	UCS_req && Role = "PUBLISHER" "REPORT SOURCE" => FAL-PDU_req { dmpm_service_name := "SMPM_Send", arep_id := GetArepId (), remote_address := RemoteAddress*, fal_pdu := BuildFAL-ReqRspPDU (fal_service_type = service_type, fal_data := user_data) }	OPEN

Table 710 – MulticastARPM state table – receiver transitions

#	Current state	Event or condition => action	Next state
R1	OPEN	FAL-PDU_ind && FAL_Pdu_Valid(fal_pdu) = False => Report to local management	OPEN
R2	OPEN	FAL-PDU_ind && Endpoint Type = "SUBSCRIBER" "REPORT SINK" && FAL_Pdu_Confirmed(fal_pdu) = True => Report to local management	OPEN
R3	OPEN	FAL-PDU_ind && Endpoint Type = "SUBSCRIBER" && FAL_Pdu_GetVcrId() <> NULL => UCS_ind* { arep_id = GetArepld (), vcr_id = GetVcrId (), service_type = GetServiceType(fal_pdu), user_data = GetUserData(fal_pdu) } * Delivered to each VCR that subscribes to the FDA Address in the APDU Header	OPEN
R4	OPEN	FAL-PDU_ind && Endpoint Type = "REPORT SINK" => UCS_ind* { arep_id = GetArepld (), vcr_id = GetVcrId (), service_type = GetServiceType(fal_pdu), user_data = GetUserData(fal_pdu) } * Delivered to all VCRs associated with this AREP	OPEN
R5	OPEN	ErrorToARPM && ErrorType = "socket disconnect" => Abort_ind* { arep_id = GetArepld (), locally_generated = "True", identifier = "FAL", reason_code = "Socket disconnected" } * Delivered to all VCR associated with this AREP	CLOSED

8.8.3 Functions used by ARPMs

Table 711 through Table 727 define the functions used by the ARPMs.

8.8.3.1 BuildFAL-ErrPDU()

Table 711 – BuildFAL-ErrPDU()

Function	
Returns a fal_pdu that is an error response with the specified errors. The request_pdu input parameter provides the necessary information to construct the appropriate error response pdu.	
Input Parameters	Output Parameters
request_pdu error_class error_code	fal_pdu

8.8.3.2 BuildFAL-ReqRspPDU()

Table 712 – BuildFAL-ReqRspPDU()

Function	
Builds an FAL PDU out of the parameters given as input variables. This includes: Building the FAL pdu header Adding the fal_data (AR user data) Generating the FAL-PDU trailer	
Input Parameters	Output Parameters
service_type fal_data	fal_pdu

8.8.3.3 GetArepId()

Table 713 – GetArepId()

Function	
Returns a value that can unambiguously identify the current AREP.	
Input Parameters	Output Parameters
None	AREP Identifier

8.8.3.4 ConfigurationArCheckOK()

Table 714 – ConfigurationArCheckOK()

Function	
Returns True if: The request fal_pdu is not a configuration request The request fal_pdu is a configuration request and there are no other configuration ARs established	
Input Parameters	Output Parameters
fal_pdu	True False

8.8.3.5 FAL_Pdu_BufferSize()

Table 715 – FAL_Pdu_BufferSize()

Function	
This function returns the buffer size from the AR Establish fal_pdu.	
Input Parameters	Output Parameters
fal_pdu	BufferSize

8.8.3.6 FAL_Pdu_Confirmed()

Table 716 – FAL_Pdu_Confirmed()

Function	
This function returns true if the fal_pdu is confirmed	
Input Parameters	Output Parameters
fal_pdu	True False

8.8.3.7 FAL_Pdu_DuplicateMsg ()

Table 717 – FAL_Pdu_DuplicateMsg ()

Function	
This function returns True if the received fal_pdu is determined to be a duplicate based on its message number and the vcr_id parameter. Otherwise, it returns False.	
Input Parameters	Output Parameters
vcr_id, fal_pdu	True False

8.8.3.8 FAL_Pdu_GetVcrlId()**Table 718 – FAL_Pdu_GetVcrlId()**

Function	
Using the FDA Address in the fal_pdu header, returns a value that identifies the destination VCR endpoint associated with the AR. May return a list if the AREP is a subscriber. Returns NULL if the FDA Address does not identify a VCR endpoint.	
Input Parameters	Output Parameters
fal_pdu	VCR Identifier

8.8.3.9 FAL_Pdu_InactivityCloseTime()**Table 719 – FAL_Pdu_InactivityCloseTime()**

Function	
This function returns the InactivityCloseTime from the AR Establish fal_pdu.	
Input Parameters	Output Parameters
AR Establish fal_pdu	InactivityCloseTime

8.8.3.10 FAL_Pdu_TransmitDelayTime()**Table 720 – FAL_Pdu_TransmitDelayTime()**

Function	
This function returns the TransmitDelayTime from the AR Establish fal_pdu.	
Input Parameters	Output Parameters
AR Establish fal_pdu	TransmitDelayTime

8.8.3.11 FAL_Pdu_SvcType()**Table 721 – FAL_Pdu_SvcType()**

Function	
This function decodes the FAL-PDU that is conveyed in the fal_pdu parameter and retrieves the FAL-PDU service type.	
Input Parameters	Output Parameters
fal_pdu	service type

8.8.3.12 FAL_Pdu_RemoteAddress()**Table 722 – FAL_Pdu_RemoteAddress()**

Function	
Returns the address of the sender of the received fal_pdu.	
Input Parameters	Output Parameters
fal_pdu	address of sender

8.8.3.13 FAL_Pdu_TrailerFields()**Table 723 – FAL_Pdu_TrailerFields()**

Function	
This function returns the trailer fields of the fal_pdu.	
Input Parameters	Output Parameters
fal_pdu	Trailer fields present in the FAL-PDU.

8.8.3.14 FAL_Pdu_ServiceSpecificParameters()

Table 724 – FAL_Pdu_ServiceSpecificParameters()

Function	
This function returns the service specific parameters, including user data, from an fal_pdu.	
Input Parameters	Output Parameters
fal_pdu	Service Specific Parameters

8.8.3.15 FAL_Pdu_Valid()

Table 725 – FAL_Pdu_Valid()

Function	
The function returns False if the ARPM can determine that: 1. the fal_pdu header or trailer contains invalid values or inconsistent values, or 2. the fal_pdu header or trailer contains options that do not match those negotiated for the AR, 3. the fal_pdu cannot be properly decoded or identified. The Service Specific Parameters are not examined by this function.	
Input Parameters	Output Parameters
fal_pdu	True False

8.8.3.16 MaxOutstandingReached()

Table 726 – MaxOutstandingReached()

Function	
The function tests the local counter for the current number of outstanding requests to determine if it has reached the limit defined by the MaxOutstanding session attribute. It returns True if the limit has been reached. If not, it increments the local counter for the current number of outstanding requests and returns False.	
Input Parameters	Output Parameters
None	True False

8.8.3.17 StartInactivityCloseTimer()

Table 727 – StartInactivityCloseTimer()

Function	
The function (re)starts a timer for the period of time defined by the Inactivity Close Time.	
Input Parameters	Output Parameters
None	None

8.9 DLL Mapping Protocol Machine (DMPM)

8.9.1 General

The DLL mapping is represented by the Socket model.

The Socket model defines an abstract service interface that permits the underlying layer to exist at the data link layer, network layer, or transport layer. Two types of services are supported by the Socket model, connection oriented (CONN) and connectionless (CNLS).

8.9.2 Primitive definitions

8.9.2.1 Primitives Exchanged between DMPM and ARPM

Table 728 and Table 729 define the primitives exchanged between the DMPM and the ARPM.

Table 728 – Primitives issued by ARPM to DMPM

Primitive name	Source	Associated parameters	Functions
FAL-PDU_req	ARPM	smpm_service_name, remote_address, fal_pdu	This primitive is used request the DMPM to transfer an FAL-PDU. The list of smpm_service_names is: SMPM_Send

Table 729 – Primitives issued by DMPM to ARPM

Primitive name	Source	Associated parameters	Functions
FAL-PDU_ind	DMPM	smpm_service_name, remote_address, fal_pdu	This primitive is used to pass an FAL-PDU received as a Socket model service data unit to a designated ARPM. It also carries some of the Socket model parameters that are referenced in the ARPM. The list of smpm_service_names is: SMPM_Receive
ErrorToARPM	DMPM	originator, reason	This primitive is used to convey selected communication errors reported by the Socket model to a designated ARPM.

8.9.2.2 Parameters of ARPM/DMPM Primitives

The parameters used with the primitives exchanged between the ARPM and the DMPM are described in Table 730.

Table 730 – Parameters used with primitives exchanged between ARPM and DMPM

Parameter name	Description
fal_pdu	This parameter conveys the value of the socket_user_data parameter.
smpm_service_name	This parameter conveys a Socket model service name.
remote_address	This parameter conveys the value of the remote address of the underlying layer.
originator	This parameter identifies the entity that detected the error that is reported using the ErrorToARPM primitive.
reason	This parameter identifies the cause of the error that is reported using the ErrorToARPM primitive.

8.9.2.3 Primitives Exchanged between the Socket model and DMPM

See Table 731 for the definitions.

NOTE 1 The following primitives and their parameters are exchanged between the Socket model and the DMPM.

NOTE 2 The DMPM instance uses either CNLS or CONN services.

NOTE 3 It is assumed that each ARPM is bound to the local address and therefore this is never an argument with any primitive.

Table 731 – Primitives exchanged between the Socket model and DMPM

Primitive name	Source	Associated parameters
socket_send.ind	Socket model	socket_user_data
socket_sendto.ind	Socket model	socket_remote_address, socket_user_data
socket_sendto.req	DMPM	socket_remote_address, socket_user_data
socket_send.req	DMPM	socket_user_data

8.9.2.4 Parameters of DMPM/Socket model primitives

See Table 732 for the definitions.

Table 732 – Parameters of DMPM/Socket model primitives

Parameter name	Description
socket_remote_address	This parameter conveys the value of the remote Socket model address.
socket_user_data	The complete message that the socket model shall send to / received from specified destination

8.9.3 DMPM state machine

8.9.3.1 DMPM states

The defined state of the DMPM together with its description is listed in Table 733 and Figure 161.

Table 733 – DMPM state descriptions

State Name	Description
ACTIVE	The DMPM in the ACTIVE state is ready to transmit or receive primitives to or from the Socket model and the ARPM.

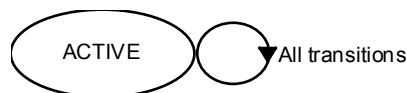


Figure 161 – State transition diagram of DMPM

8.9.3.2 DMPM state table

Table 734 and Table 735 define the DMPM state machine.

NOTE 1 Although each primitive contains all the available parameters, only those applicable to particular ARPM are relevant.

NOTE 2 Parameters starting with a capital letter, "TransmitDelayTime " for instance, refer to those defined in the attribute list of each ARPM. Therefore, they are not conveyed by the service primitives defined here.

Table 734 – DMPM state table – sender transitions

#	Current state	Event or condition => action	Next state
S1	ACTIVE	FDA-PDU_req && RemainingBufferSizeCheck (arep_id, fal_pdu) = "FIRST MESSAGE" => LoadBuffer(arep_id, fal_pdu) StartTransmitDelayTimer(arep_id)	ACTIVE
S2	ACTIVE	FDA-PDU_req && RemainingBufferSizeCheck (arep_id, fal_pdu) = "LESS THAN" => LoadBuffer(arep_id, fal_pdu)	ACTIVE
S3	ACTIVE	FDA-PDU_req && ConnectionOriented() = True && RemainingBufferSizeCheck (arep_id, fal_pdu) = "EQUAL" => socket_send.req { user_data = GetBufferedData(arep_id) }	ACTIVE
S4	ACTIVE	FDA-PDU_req && ConnectionOriented() = FALSE && RemainingBufferSizeCheck (arep_id, fal_pdu) = "EQUAL" => socket_sendto.req { remote_address = remote_address, user_data = GetBufferedData(arep_id) }	ACTIVE
S5	ACTIVE	FDA-PDU_req && ConnectionOriented() = True && RemainingBufferSizeCheck (arep_id, fal_pdu) = "GREATER THAN" => socket_send.req { user_data = fal_pdu } LoadBuffer(arep_id, fal_pdu) StartTransmitDelayTimer(arep_id)	ACTIVE
S6	ACTIVE	FDA-PDU_req && ConnectionOriented() = FALSE && RemainingBufferSizeCheck (arep_id, fal_pdu) = "GREATER THAN" => socket_sendto.req { remote_address = remote_address, user_data = GetBufferedData(arep_id) } LoadBuffer(arep_id, fal_pdu) StartTransmitDelayTimer(arep_id)	ACTIVE
S7	ACTIVE	Transmit delay timer expires && ConnectionOriented = False => socket_sendto.req { remote_address = GetRemoteAddress(arep_id), user_data = GetBufferedData(arep_id) }	ACTIVE
S8	ACTIVE	Transmit delay timer expires && ConnectionOriented() = True => socket_send.req { user_data = GetBufferedData(arep_id) }	ACTIVE

Table 735 – DMPM state table – receiver transitions

#	Current state	Event or condition => action	Next state
R1	ACTIVE	socket_sendto.ind => FAL-PDU_ind { smpm_service_name := "SMPM_Receive", remote_address := socket_remote_address, fal_pdu := socket_user_data }	ACTIVE
R2	ACTIVE	socket_send.ind => FAL-PDU_ind { smpm_service_name := "SMPM_Receive", remote_address := GetConnectionId(arep_id) , fal_pdu := socket_user_data }	ACTIVE
R3	ACTIVE	"Connection breaks" => ErrorToArpm { originator= local_socket, reason = "Socket disconnected" }	ACTIVE

8.9.3.3 Functions used by DMPM

Table 736 through Table 741 define the functions used by the DMPM.

8.9.3.3.1 Function ConnectionOriented

Table 736 – ConnectionOriented

Name	ConnectionOriented	Used in	ARPM
Input		Output	
		True False	
Function	This function returns True if the SMPM instance is connection oriented (CONN) and False if the SMPM instance in connection less (CNLS).		

8.9.3.3.2 Function GetBufferedData

Table 737 – GetBufferedData

Function	
This function returns the contents of the AREP send buffer, and marks the buffer as empty.	
Input Parameters	Output Parameters
Arepld	buffered data

8.9.3.3.3 Function GetConnectionId

Table 738 – GetConnectionId

Function	
This function returns the connection id of a connection-oriented socket.	
Input Parameters	Output Parameters
Arepld	remote_address

8.9.3.3.4 Function LoadBuffer

Table 739 – LoadBuffer

Function	
This function concatenates the user_data into the AREP buffer	
Input Parameters	Output Parameters
Arepld, user_data	None

8.9.3.3.5 Function RemainingBufferSizeCheck**Table 740 – RemainingBufferSizeCheck**

Function	
This function compares the size of the user_data with the size of the space remaining in the AREP send buffer. It returns: "FIRST MESSAGE" the buffer is currently empty and the size of the user_data less than the size of the buffer. "EQUAL" the size of the user_data is the same size as the space remaining in the buffer. "LESS THAN" the size of the user_data is less than the size of the space remaining in the buffer. "GREATER THAN" the size of the user_data is greater than the size of the space remaining in the buffer.	
Input Parameters	Output Parameters
Arepld, user_data	buffer_status

8.9.3.3.6 StartTransmitDelayTimer**Table 741 – StartTransmitDelayTimer**

Function	
This function starts or restarts the Transmit Delay Timer of the AREP identified by the Arepld parameter	
Input Parameters	Output Parameters
Arepld	None

9 Type 6

9.1 Reference

Type 6 uses the Type 1 AL protocol except for changes listed here.

Type 6 does not use any of the AREP state machines of Type 1. Type 6 does not use the DMPM of Type 1.

9.2 Usage of Parameters

The additional detail parameter is not used.

9.3 AP Context State Tables

Transition S1 of Table 19 shall be replaced by transition S1 of Table 742.

Table 742 – Type 6 specific transition

#	Current state	Event or condition => action	Next state
S1	CLOSED	Initiate.req && ApContextTest = "True" && ApExplicitConnection = "True" => EST.req { FalApduBody := "Establish-CommandPDU" }	REQ

All of Table 20 shall apply with the exception of transitions R6, R8, R9, R10 and R11.

9.4 Application Relationship protocol machines (ARPMs)

9.4.1 General

None of Type 1 ARPMs are used. Type 6 defines two Application Relations (ARs) and their associated ARPMs, which are:

- Queued Usertriggered Bidirectional-Permanent Connection (QUB-PC) ARPM
- Buffered Networkscheduled Unidirectional-Permanent Connection (BNU-PC) ARPM

9.4.2 Queued Usertriggered Bidirectional-Permanent Connection (QUB-PC) ARPM

9.4.2.1 Primitive definitions

9.4.2.1.1 Primitives exchanged between ARPM and FSPM

Table 743 and Table 744 list the service primitives between the ARPM and the FSPM in support of QUB-PC.

Table 743 – Primitives issued by FSPM to ARPM

Primitive name	Source	Associated parameters	Functions
EST_req	FSPM	user_data, local_dlc_identifier	This is an FAL internal primitive used to convey an Establish request primitive from the FSPM to the ARPM.
EST_rsp(+)	FSPM	user_data	This is an FAL internal primitive used to convey an Establish response(+) primitive from the FSPM to the ARPM.
EST_rsp(-)	FSPM	user_data	This is an FAL internal primitive used to convey an Establish response(-) primitive from the FSPM to the ARPM.
Abort_req	FSPM	identifier, reason_code	This an FAL internal primitive used to convey an Abort request primitive from the FSPM to the ARPM.
CS_req	FSPM	user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) request primitive from the FSPM to the ARPM.
CS_rsp	FSPM	user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) response primitive from the FSPM to the ARPM.
UCS_req	FSPM	user_data	This is an FAL internal primitive used to convey a Unconfirmed Send (UCS) request primitive from the FSPM to the ARPM.

Table 744 – Primitives issued by ARPM to FSPM

Primitive name	Source	Associated parameters	Functions
EST_ind	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish indication primitive from the ARPM to the FSPM.
EST_cnf(+)	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish response(+) primitive from the ARPM to the FSPM.
EST_cnf(-)	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Establish response(-) primitive from the ARPM to the FSPM.
Abort_ind	ARPM	arep_id, locally_generated, identifier, reason_code	This is an FAL internal primitive used to convey an Abort primitive from the ARPM to the FSPM.
CS_ind	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) indication primitive from the ARPM to the FSPM.
CS_cnf	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey a Confirmed Send (CS) confirmation primitive from the ARPM to the FSPM.
UCS_ind	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey a Unconfirmed Send (UCS) indication primitive from the ARPM to the FSPM.

9.4.2.1.2 Parameters of FSPM/ARPM primitives

The parameters used with the primitives exchanged between the FSPM and the ARPM are described in Table 745.

Table 745 – Parameters used with primitives exchanged between FSPM and ARPM

Parameter name	Description
arep_id	This parameter is used to unambiguously identify an instance of the AREP that has issued a primitive. A means for such identification is not specified by this standard.
user_data	This parameter conveys FAL-User data.
locally_generated	This parameter conveys value that is used for the Locally_Generated parameter.
identifier	This parameter conveys value that is used for the Identifier parameter.
reason_code	This parameter conveys value that is used for the Reason_Code parameter.
reported_status	This parameter conveys a Data Link Layer event status.
Local_dlc_identifier	This parameter conveys value of the local DLC-identifier.
NOTE This DLL uses permanently open connections. The correspondence of local DLC-identifiers to local-DLCEP-addresses and remote-DLCEP-addresses is established during Bus-configuration, as described in 61158-4, Type 6.	

9.4.2.2 DLL Mapping of QUB-PC AREP class

9.4.2.2.1 General

This subclause describes the mapping of the QUB-PC AREP Class to the Type 6 Fieldbus Data Link Layer. It does not redefine the DLCEP attributes or Bus-configuration attributes that are or will be defined in the Data link layer standard; rather, it defines how they are used by this AR class.

The DLL Mapping attributes and their permitted values and the DLL services used with the QUB AREP class are defined in this clause.

CLASS: Qub-PC

PARENT CLASS: QueuedUser-TriggeredBidirectionalPermanentConnectionAREP

ATTRIBUTES:

- 1 (m) KeyAttribute: LocalDlcIdentifier
- 2 (c) Constraint: Role = Client
- 2.1 (m) Attribute: Initiator (True)
- 3 (m) Attribute: DlcType (GeneralPurposeConfirmed)
- 5 (m) Attribute: MaxConfirmDelay
- 6 (m) Attribute: MaxDisduSizes
- 6.1 (m) Attribute: MaxDisduSizeFromRequestor
- 6.2 (m) Attribute: MaxDisduSizeFromResponder
- 6.3 (m) Attribute: MaxDisduSizeFromRequestorNegotiated
- 6.4 (m) Attribute: MaxDisduSizeFromResponderNegotiated
- 6.5 (m) Attribute: MaxDisduLength

DLL SERVICES:

- 1 (m) OpsService: DL-Data

9.4.2.2.2 Attributes

9.4.2.2.2.1 LocalDlcIdentifier

This attribute specifies dynamically the local DLC-identifier of a DL-Data-request primitive and thus it identifies the DLCEP.

NOTE This DLL uses permanently open connections. The correspondence of local DLC-identifiers to local-DLCEP-addresses and remote-DLCEP-addresses is established during Bus-configuration, as described in 61158-4, Type 6.

9.4.2.2.2.2 Role

This attribute specifies possible roles of this AREP. The value of "Client" means that this AREP issues FAL confirmed service request PDUs to a remote AREP whose Role is "Server", and may also receive FAL unconfirmed service requests from the "Server". An AREP whose Role is "Server" is a mirror of one whose Role is "Client."

NOTE This value is established during Bus-configuration, as described in 61158-4, Type 6.

9.4.2.2.2.3 Initiator

This attribute specifies whether this AREP is capable of initiating traffic on this AR or not. The value of "True" means it is capable of doing it. If the value of this attribute is "False," it can only respond to traffic from the remote AREP.

NOTE This value is established during Bus-configuration, as described in 61158-4, Type 6.

9.4.2.2.2.4 DlcType

This attribute specifies type of DLC used by the AR and thus the behavior of the DLCEP which is attached to the AREP. Its value is always General-Purpose-Confirmed (GPC).

NOTE This value is established during Bus-configuration, as described in 61158-4, Type 6.

9.4.2.2.2.5 MaxConfirmDelay

This attribute specifies the maximum confirmation delay of the DL-Data service, when a response has been requested. The Status parameter of the DL-Data-confirm primitive indicates if no response is received within MaxConfirmDelay.

NOTE This value is established during Bus-configuration, as described in 61158-4, Type 6.

9.4.2.2.2.6 MaxDisduSizeFromRequestor

This attribute specifies the configured value of the maximum length of an FAL-PDU that can be sent from the AREP whose Initiator attribute has a value of “True” to the remote AREP.

This attribute is compared to the value of max-DLSDU-length, defined in 61158-4, Type 6.

9.4.2.2.2.7 MaxDisduSizeFromResponder

This attribute specifies the configured value of the maximum length of an FAL-PDU that can be sent from the AREP whose Initiator attribute has a value of “False” to the remote AREP.

This attribute is compared to the value of max-DLSDU-length, defined in 61158-4, Type 6.

9.4.2.2.2.8 MaxDisduSizeFromRequestorNegotiated

This attribute specifies the negotiated value of the maximum length of an FAL-PDU that can be sent from the AREP whose Initiator attribute has a value of “True” to the remote AREP.

9.4.2.2.2.9 MaxDisduSizeFromResponderNegotiated

This attribute specifies the negotiated value of the maximum length of an FAL-PDU that can be sent from the AREP whose Initiator attribute has a value of “False” to the remote AREP.

9.4.2.2.2.10 MaxDisduLength

This attribute is the value of max-DLSDU-length, defined in 61158-4, Type 6.

NOTE This value is established during Bus-configuration, as described in 61158-4, Type 6.

9.4.2.2.3 DLL services

Refer to 9.5.10, Data Link Layer Service Selection, for DLL service descriptions.

9.4.2.2.4 QUB-PC ARPM states

The defined states and their descriptions of the QUB ARPM are listed in Table 746 and Figure 162:

Table 746 – QUB ARPM states

State	Description
CLOSED	The AREP is defined, but not capable of sending or receiving FAL-PDUs. It may send or receive Establish service FAL-PDUs while in this state.
OPEN	The AREP is defined and capable of sending or receiving FAL-PDUs.
REQUESTING (REQ)	The AREP has sent an Establish Request FAL-PDU and is waiting for a response from the remote AREP.
RESPONDING (RSP)	The AREP has received an Establish Request FAL-PDU, delivered an Establish.ind primitive and is waiting for a response from its user.
REPLIED (REPL)	The Server AREP has issued an EST_rsp(+) primitive and is awaiting a connection established indication from the DLL.
SAME	Indicates that the next state is the same as the current state.

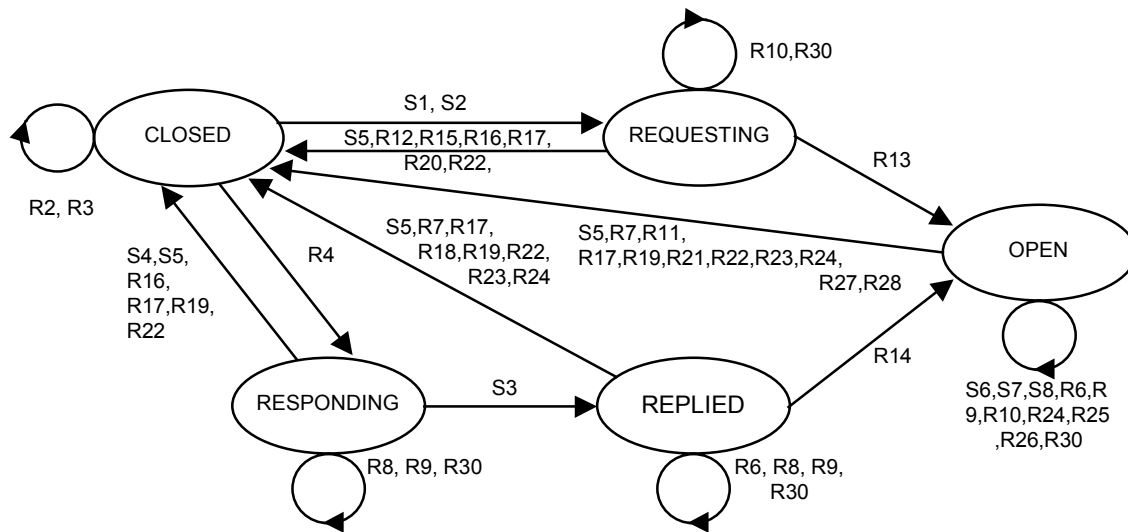


Figure 162 – State transition diagram of QUB-PC ARPM

9.4.2.2.5 QUB-PC ARPM state table

The QUB-PC state machines are defined in Table 747 and Table 748.

Table 747 – QUB-PC ARPM state table – sender transactions

#	Current state	Event or condition => action	Next state
S1	CLOSED	EST_req && Initiator = "True" && MaxFalPduLengthTest = "False" => Est_cnf(-) { arep_id := GetArepld (), user_data = "null" }	CLOSED
S2	CLOSED	EST_req && Initiator = "True" && MaxFalPduLengthTest => FAL-PDU_req { dmpm_service_name := "DMPM_Connect_req", arep_id := GetArepld (), local_dlcep_address := LocalDlclIdentifier, response_request := "True", dlsdu := BuildFAL-PDU (fal_pdu_name := "EST_ReqPDU", fal_data := user_data) }	REQ
S3	RSP	EST_rsp(+) => FAL-PDU_req { dmpm_service_name := "DMPM_Connect_rsp", arep_id := GetArepld (), local_dlcep_address := LocalDlclIdentifier, dlsdu := BuildFAL-PDU (fal_pdu_name := "EST_RspPDU", fal_data := user_data) }	REPL

#	Current state	Event or condition => action	Next state
S4	RSP	EST_rsp(-) => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "connection rejection-transient condition", dlsdu := BuildFAL-PDU (fal_pdu_name := "EST_ErrPDU", fal_data := user_data) } }	CLOSED
S5	NOT CLOSED	Abort_req => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "disconnection-normal condition", dlsdu := BuildFAL-PDU (fal_pdu_name := "Abort_PDU", fal_id := identifier, fal_reason_code := reason_code } }	CLOSED
S6	OPEN	CS_req && Role = "Client" => FAL-PDU_req { dmpm_service_name := "DMPM_Data_req", arep_id := GetArepld (), response_request := "True", dlsdu := BuildFAL-PDU (fal_pdu_name := "CS_ReqPDU", fal_data := user_data) } } confirmed_service = "True"	OPEN
S7	OPEN	CS_rsp && Role = "Server" => FAL-PDU_req { dmpm_service_name := "DMPM_Data_req", arep_id := GetArepld (), dlsdu := BuildFAL-PDU (fal_pdu_name := "CS_RspPDU", fal_data := user_data) } }	OPEN
S8	OPEN	UCS_req && Role = "Client" => FAL-PDU_req { dmpm_service_name := "DMPM_Data_req", arep_id := GetArepld (), response_request := "False", dlsdu := BuildFAL-PDU (fal_pdu_name := "UCS_ReqPDU", fal_data := user_data) } } confirmed_service = "False"	OPEN

Table 748 – QUB-PC ARPM state table – receiver transactions

#	Current state	Event or condition => action	Next state
R2	CLOSED	Connect_ind && Initiator = "False" && FAL_Pdu_Type (dls_user_data) <> "EST_ReqPDU" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }	CLOSED
R3	CLOSED	Connect_ind && Initiator = "False" && FAL_Pdu_Type (dls_user_data) = "EST_ReqPDU" && MaxFalPduLengthTest := "False" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Data-too-long", dlsdu := "null" }	CLOSED
R4	CLOSED	Connect_ind && Initiator = "False" && FAL_Pdu_Type (dls_user_data) = "EST_ReqPDU" && MaxFalPduLengthTest => LocalDlcIdentifier := calling_dlc_identifier, MaxDlsduSizeFromRequestorNegotiated := dlsdu_size_from_requestor, MaxDlsduSizeFromResponderNegotiated := dlsdu_size_from_responder, EST_ind { arep_id := GetArepld (), user_data := dls_user_data }	RSP
R9	RSP REPL OPEN	Connect_ind && Initiator = "False" && FAL_Pdu_Type (dls_user_data) <> "EST_ReqPDU" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }	SAME
R12	REQ	Connect_cnf && FAL_Pdu_Type (dls_user_data) <> "EST_RspPDU" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU" }	CLOSED
R13	REQ	Connect_cnf && FAL_Pdu_Type (dls_user_data) = "EST_RspPDU" => MaxDlsduSizeFromRequestorNegotiated := dlsdu_size_from_requestor, MaxDlsduSizeFromResponderNegotiated := dlsdu_size_from_responder, EST_cnf(+) { arep_id := GetArepld (), user_data := dls_user_data }	OPEN

Table 748 – (continued)

#	Current state	Event or condition => action	Next state
R14	REPL	FAL-PDU_ind => (no actions taken)	OPEN
R15	REQ	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && FAL_Pdu_Type (fal_pdu) = "EST_ErrPDU" => EST_cnf(-) { arep_id := GetArepld (), user_data := fal_pdu }	CLOSED
R16	REQ RSP	FAL-PDU_ind && dmpm_service_name <> "DMPM_Disconnect_ind" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid DL-Event", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid DL-Event" }	CLOSED
R17	NOT CLOSED	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu <> "null" && FAL_Pdu_Type (fal_pdu) = "Abort_PDU" => Abort_ind { arep_id := GetArepld (), locally_generated := "False", identifier := AbortIdentifier (fal_pdu), reason_code := AbortReason (fal_pdu) }	CLOSED
R18	REPL	FAL-PDU_ind && ((dmpm_service_name <> "DMPM_Disconnect_ind") && (dmpm_service_name <> "DM_Connection_Established_ind")) => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid DL-Event", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid DL-Event" }	CLOSED
R19	REPL RSP OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu <> "null" && FAL_Pdu_Type (fal_pdu) <> "Abort_PDU" => Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU" }	CLOSED

Table 748 – (continued)

#	Current state	Event or condition => action	Next state
R20	REQ	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu <> "null" && ((FAL_Pdu_Type (fal_pdu) <> "Abort_PDU") && (FAL_Pdu_Type (fal_pdu) <> "EST_ErrPDU")) => Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU" } }	CLOSED
R21	OPEN	FAL-PDU_ind && ((dmpm_service_name <> "DMPM_Disconnect_ind") && (dmpm_service_name <> "DMPM_Data_ind")) => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid DL-Event", dlsdu := "null" }, Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid DL-Event" } }	CLOSED
R22	NOT CLOSED	FAL-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fal_pdu = "null" => Abort_ind{ arep_id := GetArepld (), locally_generated := "False", identifier := "Data Link Layer", reason_code := "Invalid DL-Event" } }	CLOSED
R23			
R24	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Server" && FAL_Pdu_Type (fal_pdu) = "UCS_ReqPDU" => UCS_ind { arep_id := GetArepld (), user_data := fal_pdu } }	OPEN
R25	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Server" && FAL_Pdu_Type (fal_pdu) = "CS_ReqPDU" => CS_ind { arep_id := GetArepld (), user_data := fal_pdu } }	OPEN
R26	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Client" && FAL_Pdu_Type (fal_pdu) = "CS_RspPDU" => CS_cnf { arep_id := GetArepld (), user_data := fal_pdu } }	OPEN

Table 748 – (continued)

#	Current state	Event or condition => action	Next state
R27	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Server" && FAL_Pdu_Type (fal_pdu) <> "CS_ReqPDU" && FAL_Pdu_Type (fal_pdu) <> "UCS_ReqPDU" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU" } 	CLOSED
R28	OPEN	FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Client" && FAL_Pdu_Type (fal_pdu) <> "CS_RspPDU" => FAL-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), reason := "Invalid FAL-PDU", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAL", reason_code := "Invalid FAL-PDU" } 	CLOSED
R30	NOT CLOSED	ErrorToARPM (No actions taken. See note.)	SAME
NOTE It is a local matter to report this error status to network management entities. The ARPM does not abort the existing connections. The FAL user may issue an Abort request to disconnect the current connection, depending on the type of status information conveyed by the ErrorToARPM primitive.			

9.4.2.2.6 Functions used by QUB-PC ARPM

Table 749 through Table 759 list the functions used by the QUB-PC ARPM state machines.

Table 749 – Function GetArepld ()

Name	GetArepld ()	Used in	ARPM
Input		Output	
(none)		AREP Identifier	
Function	Returns a value that can unambiguously identify the current AREP.		

Table 750 – Function BuildFAL-PDU

Name	BuildFAL-PDU	Used in	ARPM
Input	fal_pdu_name, fal_data, fal_id,	Output	
	fal_reason_code	dlsdu	
Function	Builds an FAL-PDU out of the parameters given as input variables.		

Table 751 – Function FAL_Pdu_Type

Name	FAL_Pdu_Type	Used in	ARPM
Input		Output	
dls_user_data fal_pdu		One of the FAL-PDU types defined in clause 4.	
Function	This function decodes the FAL-PDU that is conveyed in the dls_user_data or fal_pdu parameter and retrieves one of the FAL-PDU types.		

Table 752 – Function AbortIdentifier

Name	AbortIdentifier	Used in	ARPM
Input		Output	
fal_pdu		The Identifier parameter of the Abort service.	
Function			
This function decodes the Abort_PDU that is conveyed in the fal_pdu parameter and extracts the Identifier parameter.			

Table 753 – Function AbortReason

Name	AbortReason	Used in	ARPM
Input		Output	
fal_pdu		The Reason Code parameter of the Abort service.	
Function			
This function decodes the Abort_PDU that is conveyed in the fal_pdu parameter and extracts the Reason Code parameter.			

Table 754 – Function AbortDetail

Name	AbortDetail	Used in	ARPM
Input		Output	
fal_pdu		The Additional Detail parameter of the Abort service.	
Function			
This function decodes the Abort_PDU that is conveyed in the fal_pdu parameter and extracts the Additional Detail parameter.			

Table 755 – Function ApExplicitConnection

Name	ApExplicitConnection		
Input	Arep_Id	Output	True
Function	Refer to AREP to determine if this AP_ASE supports explicit connection between APs.		

Table 756 – Function ImmediateAcknowledge

Name	ImmediateAcknowledge		
Input	Arep_Id	Output	True
Function	Refer to AREP to determine if this AP_ASE requires immediate acknowledge in the underlying layer.		

Table 757 – Function ArFspmService

Name	ArFspmService		
Input	Arep_Id	Output	FAL Service or AR FSPM Service
Function	Generate an AR ASE service primitive. Relationship between FAL Services and AR FSPM Services shall be as follows: AR-Confirmed Send CS AR-Establish EST AR-Abort Abort AR-Unconfirmed Send UCS AR-Unconfirmed Send Acknowledge UCA		

Table 758 – Function ArAcceeSupported

Name	ArAcceeSupported		
Input	Arep_Id	Output	True
Function	Determines if the AP accepts AR ASE Send (ConfirmedSend or UnconfirmedSend) services.		

Table 759 – Function MaxFalPduLengthTest

Name	MaxFalPduLengthTest		
Input	Arep_Id	Output	True or False
Function	Upon receipt of an Initiate-RequestPDU, the called AP_ASE shall check the requested maximum PDU length against its defined context. The following tests are performed: 1. If (ConfiguredMaxPDUSizeSending > MaxPDUSizeReceiving of the Initiate PDU), this test fails; 2. If (ConfiguredMaxPDUSizeReceiving < MaxPDUSizeSending of the Initiate PDU), this test fails. 3. If (ConfiguredMaxPDUSizeSending > MaxDisduLength), this test fails; 4. If (ConfiguredMaxPDUSizeReceiving > MaxDisduLength), this test fails.		

9.4.3 Bufferscheduled Unidirectional-Permanent Connection (BNU-PC) ARPM

9.4.3.1 Primitive definitions

9.4.3.1.1 Primitives exchanged between ARPM and FSPM

The primitives exchanged between the ARPM and the FSPM are listed in Table 760 and Table 761.

Table 760 – Primitives issued by FSPM to ARPM

Primitive name	Source	Associated parameters	Functions
EST_req	FSPM	user_data	This is an FAL internal primitive used to convey an Establish request primitive from the FSPM to the ARPM.
UCS_req	FSPM	user_data	This is an FAL internal primitive used to convey an Unconfirmed Send (UCS) request primitive from the FSPM to the ARPM.
GBM_req	FSPM	(none)	This is an FAL internal primitive used to convey a Get-Buffered-Message (GBM) request primitive from the FSPM to the ARPM.

Table 761 – Primitives issued by ARPM to FSPM

Primitive name	Source	Associated parameters	Functions
UCS_ind	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey an Unconfirmed Send (UCS) indication primitive from the ARPM to the FSPM.
GBM_cnf(+)	ARPM	arep_id, user_data	This is an FAL internal primitive used to convey a Get-Buffered-Message (GBM) positive confirmation primitive from the ARPM to the FSPM.
GBM_cnf(-)	ARPM	arep_id	This is an FAL internal primitive used to convey a Get-Buffered-Message (GBM) negative confirmation primitive from the ARPM to the FSPM.
FSTS_ind	ARPM	arep_id, reported_status	This is an FAL internal primitive used to convey a FAL-Status (FSTS) indication primitive from the ARPM to the FSPM.

9.4.3.1.2 Parameters of FSPM/ARPM primitives

The parameters used with the primitives exchanged between the FSPM and the ARPM are described in Table 762.

Table 762 – Parameters used with primitives exchanged between FSPM and ARPM

Parameter name	Description
arep_id	This parameter is used to unambiguously identify an instance of the AREP that has issued a primitive. A means for such identification is not specified by this standard.
user_data	This parameter conveys FAL-User data.
identifier	This parameter conveys value that is used for the Identifier parameter.
reason	This parameter conveys value that is used for the Reason_Code parameter.
status	This parameter conveys value that is used for the Status parameter.
reported_status	This parameter conveys a Data Link Layer event status.

9.4.3.2 DLL Mapping of BNU-PC AREP class

This clause describes the mapping of the BNU-PC AREP class to the Fieldbus Data Link Layer. It does not redefine the DLCEP attributes that are or will be defined in the Data Link Layer standard; rather, it defines how they are used by this AR class.

The DLL Mapping attributes and their permitted values and the DLL services used with the BNU AREP class are defined in this clause.

CLASS: Bnu-PC

PARENT CLASS: BufferedNetworkScheduledUnidirectionalPermanentConnectionAREP

ATTRIBUTES:

- 1 (m) KeyAttribute: LocalDlcIdentifier
- 2 (m) Attribute: MaxPduLength
- 3 (m) Attribute: Role (Publisher, Subscriber)
 - 3.1 (m) Constraint: DlcClass = Publisher
 - 3.1.1 (m) Attribute: MaxDlsduSizeFromRequestor
 - 3.1.1.1 (m) Constraint: MaxDlsduSizeFromRequestor > MaxPduLength
 - 3.1.1.1.1 (m) Attribute: DlcType (ExSCAN)
 - 3.1.1.2 (m) Constraint: MaxDlsduSizeFromRequestor ≤ MaxPduLength
 - 3.1.1.2.1 (m) Attribute: DlcType (SCAN)
 - 3.2 (m) Constraint: DlcClass = Subscriber
 - 3.2.1 (m) Attribute: MaxDlsduSizeFromResponder

DLL SERVICES:

- 1 (m) OpsService: DL-Put
- 2 (m) OpsService: DL-Get
- 3 (m) OpsService: DL-Buffer-Received
- 4 (m) OpsService: DL-Buffer-Sent

9.4.3.2.1 Attributes

9.4.3.2.1.1 LocalDlcIdentifier

This attribute specifies dynamically the local DLC-identifier of a DL-Put or DL-Get primitive and thus it identifies the DLCEP.

NOTE The Type 6 DLL uses permanently open connections. The correspondence of local DLC-identifiers to Publisher-DLCEP-address and Subscriber-DLCEP-addresses is established during Bus-configuration, as described in 61158-4, Type 6.

9.4.3.2.1.2 DlcType

This attribute specifies type of DLC used by the AR and thus the behavior of the DLCEP which is attached to the AREP. Its value is always SCAN.

NOTE This value is established during Bus-configuration, as described in 61158-4, Type 6.

9.4.3.2.1.3 Role

This attribute specifies the role of this AREP. The value of “Publisher” indicates that this AREP is used as a publisher. The value of “Subscriber” means that it is used as a subscriber.

NOTE This value is established during Bus-configuration, as described in 61158-4, Type 6.

9.4.3.2.1.4 MaxDisduSizeFromRequestor

This attribute is used if the Role attribute has a value of “Publisher” and specifies the maximum length of an FAL-PDU that can be sent from this AREP.

This attribute is compared to the value of max-DLPDU-length, defined in 61158-4, Type 6.

9.4.3.2.1.5 MaxDisduSizeFromResponder

This attribute is used if the Role attribute has a value of “Subscriber” and specifies the maximum length of an FAL-PDU that can be received by this AREP.

This attribute is compared to the value of max-DLPDU-length, defined in 61158-4, Type 6.

9.4.3.2.1.6 MaxPduLength

This attribute is the value of max-DLSDU-length, defined in 61158-4, Type 6.

NOTE This value is established during Bus-configuration, as described in 61158-4, Type 6.

9.4.3.2.2 DLL services

Refer to 9.5.10, Data Link Layer Service Selection, for DLL service descriptions.

9.4.3.2.3 BNU-PC AREP state machine

9.4.3.2.4 BNU-PC ARPM states

The defined states together with their descriptions of the BNU ARPM are listed in Table 763 and Figure 163.

Table 763 – BNU ARPM states

State	Description
CLOSED	The AREP is defined but not capable of sending or receiving FAL-PDUs.
OPEN	The AREP is defined and capable of sending or receiving FAL-PDUs.

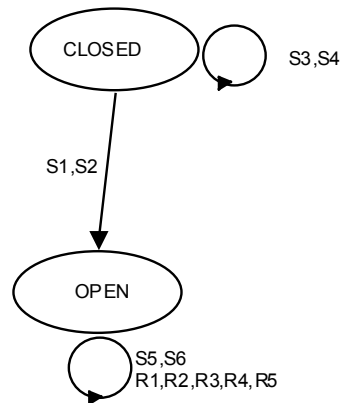


Figure 163 – State transition diagram of the BNU-PC ARPM

9.4.3.2.5 BNU-PC ARPM state table

Table 764 and Table 765 define the BNU-PC state machines.

Table 764 – BNU-PC ARPM state table - sender transactions

#	Current state	Event or condition => action	Next state
S1	CLOSED	EST_req && Role = "Publisher" && MaxFalPduLengthTest => Est_cnf(+) { arep_id := GetArepld (), user_data = "null" }	OPEN
S2	CLOSED	EST_req && Role = "Subscriber" && MaxFalPduLengthTest => Est_cnf(+) { arep_id := GetArepld (), user_data = "null" }	OPEN
S3	CLOSED	EST_req && Role = "Publisher" && MaxFalPduLengthTest = "False" => Est_cnf(-) { arep_id := GetArepld (), user_data = "null" }	CLOSED
S4	CLOSED	EST_req && Role = "Subscriber" && MaxFalPduLengthTest = "False" => Est_cnf(-) { arep_id := GetArepld (), user_data = "null" }	CLOSED
S5	OPEN	UCS_req && Role = "Publisher" => FAL-PDU_req { dmpm_service_name := "DMPM_Put_req", arep_id := GetArepld (), dlsdu := BuildFAL-PDU (fal_pdu_name := "UCS_PDU", fal_data := user_data) }	OPEN

#	Current state	Event or condition => action	Next state
S6	OPEN	GBM_req && Role = "Subscriber" => FAL-PDU_req { dmpm_service_name := "DMPM_Get_req", arep_id := GetArepId () }	OPEN

Table 765 – BNU-PC ARPM state table - receiver transactions

#	Current state	Event or condition => action	Next state
R1	OPEN	FAL-PDU_ind && Role = "Subscriber" && dmpm_service_name = "DMPM_Buffer_Received_ind" && FAL_Pdu_Type (fal_pdu) = "UCS_PDU" => UCS_ind { arep_id := GetArepId (), user_data := fal_pdu, }	OPEN
R2	OPEN	FAL-PDU_ind && Role = "Subscriber" && dmpm_service_name = "DMPM_Get_cnf" && reason = "success" => GBM_cnf(+) { arep_id := GetArepId (), user_data := fal_pdu, }	OPEN
R3	OPEN	FAL-PDU_ind && Role = "Subscriber" && dmpm_service_name = "DMPM_Get_cnf" && reason <> "success" => GBM_cnf(-) { arep_id := GetArepId () }	OPEN
R4	OPEN	FAL-PDU_ind && Role = "Publisher" && dmpm_service_name = "DMPM_Buffer_Sent_ind" => FSTS_ind { arep_id := GetArepId (). reported_status := "Buffer-Sent" }	OPEN
R5	OPEN	ErrorToARPM => (no actions taken; see note)	OPEN

NOTE It is a local matter to report this error status to network management entities. The ARPM does not abort the existing connections. The FAL user may issue an Abort request to disconnect the current connection, depending on the type of status information conveyed by the ErrorToARPM primitive.

9.4.3.2.6 Functions used by BNU ARPM

Table 766 through Table 769 define the functions used by the BNU-PC state machines.

Table 766 – Function GetArepId ()

Name	GetArepId ()	Used in	ARPM
Input		Output	
(none)		AREP Identifier	
Function	Returns a value that can unambiguously identify the current AREP.		

Table 767 – Function BuildFAL-PDU

Name	BuildFAL-PDU	Used in	ARPM
Input		Output	
	fal_pdu_name, fal_data, fal_id,	disdu	
Function	Builds an FAL-PDU out of the parameters given as input variables.		

Table 768 – Function FAL_Pdu_Type

Name	FAL_Pdu_Type	Used in	ARPM
Input		Output	
fal_pdu		One of the FAL-PDU types defined in the clause 4.	
Function	This function decodes the FAL-PDU that is conveyed in the fal_pdu parameter and retrieves one of the FAL-PDU types.		

Table 769 – Function MaxFalPduLengthTest

Name	MaxFalPduLengthTest	Used in	
Input	Arep_Id	Output	True or False
Function	Upon receipt of an Est_req, the AP_ASE shall check the requested maximum PDU length against its defined context. The following tests are performed:		
	1. If (ConfiguredMaxPDUSizeFromPublisher > MaxPduLength), this test fails; 2. If (ConfiguredMaxPDUSizeTo Subscriber > MaxPduLength), this test fails.		

9.5 DLL Mapping protocol machine (DMPM)

9.5.1 General

The DLL Mapping Protocol Machine is common to all the AREP types. Only applicable primitives are different among different AREP types.

9.5.2 A remark about Dlc-identifiers

The Type 6 Data Link Layer standard defines local Dlc-identifiers to distinguish the permanent connections established by the Bus-configuration service. Since these Dlc-identifiers are the KeyAttributes of the FAL ARPMs, they are included as parameters of the DMPM primitives.

9.5.3 A remark about DLSDU-length

The Type 6 Data Link Layer standard defines DLSDU-length parameters to distinguish the end of each DLSDU. In a real implementation, the use of these parameters is implementation dependent. Therefore, this DMPM does not include DLSDU-length parameters.

9.5.4 A remark about DLS-user identification

It is assumed that the connection between one ARPM instance and one DMPM instance is established by other than protocol means. Therefor DLS-user_identification parameters are not used in DMPM primitives.

9.5.5 A remark about buffer or queue identifiers

The Type 6 Data Link Layer standard uses parameters to identify a queue or a buffer that are shared between the Data Link Layer and the DLS-user. Although they are useful to clarify the operations of the Data Link Layer, none of them affects protocol behavior of the FAL and DLL. In a real implementation, these parameters are implementation dependent. Therefore, this

standard does not include parameters that directly correspond to these buffer or queue identifiers. A means for identifying the buffers and queues between the FAL and the DLL is a local matter.

9.5.6 A remark about configuration and initialization of the Data Link Layer

The Type 6 Data Link Layer standard defines Bus_configuration services to setup resources and permanent connections within the layer. Although they are useful to clarify the operations of the Data Link Layer, none of them affects protocol behavior of the FAL and DLL. Therefore, the FAL assumes that such initialization procedures have been executed prior to the operations of the FAL state machines. For the same reason, these DL-services are not listed in Data Link Layer service selection.

9.5.7 A remark about DMPM pseudo-services

The Type 6 Data Link Layer standard does not define a DL-Connect service since all connections are permanently established. The DMPM uses the initial set of DL-Data primitives exchanged on a permanent connection, prior to the AREP reaching the OPEN state, to carry connection establishment information. This set of primitives constitutes a DMPM pseudo-service. Subsequent sets of DL-Data primitives on that connection carry CS, UCS, UCA or Abort information.

9.5.8 Primitive definitions

9.5.8.1 Primitives exchanged between DMPM and ARPM

Table 770 and Table 771 list the service primitives between the ARPM and the DMPM

Table 770 – Primitives issued by ARPM to DMPM

Primitive names	Source	Associated parameters	Functions
FAL-PDU_req	ARPM	dmpm_service_name, arep_id, local_dlc_identifier, reason, response_request dlsdu	This primitive is used to request the DMPM to transfer an FAL-PDU, or to request an abort without transferring an FAL-PDU. It passes the FAL-PDU to the DMPM as a DLSDU. It also carries some of the Data Link Layer parameters that are referenced there.

Table 771 – Primitives issued by DMPM to ARPM

Primitive names	Source	Associated parameters	Functions
FAL-PDU_ind	DMPM	dmpm_service_name, reason, response_requesst fal_pdu	This primitive is used to pass an FAL-PDU received as a Data Link Layer service data unit to a designated ARPM. It also carries some of the Data Link Layer parameters that are referenced in the ARPM.
Connect_ind	DMPM	local_dlc_identifier, max_confirm_delay, qos_set, response_request max_dl_sdu_size dl_sdu_length dl_s_user_data	This primitive is used to convey a DL_Connect.ind primitive to the ARPM to process connection establishment. All the parameters that are associated with the DL_Data.ind primitive are carried with this primitive.
Connect_cnf	DMPM	local_dlc_identifier, status, dl_s_user_data	This primitive is used to convey a DL_Connect.cnf primitive to the ARPM. All the parameters that are associated with the DL_Data.cnf primitive are carried with this primitive.
ErrorToARPM	DMPM	reason	This primitive is used to convey selected communication errors reported by the Data Link Layer to a designated ARPM.

9.5.8.2 Parameters of ARPM/DMPM primitives

The parameters used with the primitives exchanged between the ARPM and the DMPM are described in Table 772.

Table 772 – Parameters used with primitives exchanged between ARPM and DMPM

Parameter name	Description
arep_id	This parameter carries a local identifier to specify the associated AR instance.
dmpm_service_name	This parameter conveys a DMPM pseudo-service name or a Data Link Layer service name. Possible values are represented as DMPM_XXXX_yyy.
dl_s_user_data	This parameter conveys the value of the dl_dl_s_user_data parameter.
dl_sdu	This parameter conveys the value of the dl_dl_s_user_data parameter.
fal_pdu	This parameter conveys the value of the dl_dl_s_user_data parameter.
local_dlc_identifier	This parameter conveys the value of the RequestingAREP parameter supplied with the EST_ReqPDU and the value of the RespondingAREP parameter supplied with the EST_ReqPDU..
max_confirm_delay	This parameter conveys the value of the dl_max_confirm_delay parameter.
max_dl_sdu_size	This parameter conveys the value of the dl_max_dl_sdu_size parameter.
qos_set	This parameter conveys the value of the dl_qos_set parameter.
reason	This parameter conveys the value of the dl_reason parameter.
response_request	This parameter conveys the value of the dl_response_request parameter.
status	This parameter conveys the value of the dl_status parameter.

9.5.8.3 Primitives exchanged between Data Link Layer and DMPM

NOTE The following primitives and their parameters are defined in IEC 61158-3.

Table 773 summarizes the primitives exchanged between the DLL and the DMPM.

Table 773 – Primitives exchanged between Data Link Layer and DMPM

Primitive names	Source	Associated parameters
DL_Data.ind	Data Link Layer	dl_local_dlc_identifier, dl_qos_set, dl_response_request dl_dls_user_data
DL_Data.cnf	Data Link Layer	dl_local_dlc_identifier, dl_status
DL_Disconnect.ind	Data Link Layer	(none)
DL_Buffer_Received.ind	Data Link Layer	dl_local_dlc_identifier
DL_Buffer_Sent.ind	Data Link Layer	dl_local_dlc_identifier
DL_Get.req(out)	Data Link Layer	dl_local_dlc_identifier, dl_dls_user_data, dl_status
DL_Disconnect.req	DMPM	dl_reason dl_dls_user_data
DL_Data.req	DMPM	dl_local_dlc_identifier, dl_qos_set, dl_response_request dl_dls_user_data
DL_Get.req(in)	DMPM	dl_local_dlc_identifier
DL_Put.req(in)	DMPM	dl_local_dlc_identifier, dl_dls_user_data,

9.5.8.4 Parameters of DMPM/Data Link Layer primitives

The parameters used with the primitives exchanged between the DMPM and the Data Link Layer are identical to those defined in the Data Link Layer Service Definition (IEC 61158-3). They are prefixed by “dl_” to indicate that they are used by the FAL.

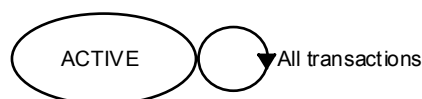
9.5.9 DMPM state machine

9.5.9.1 DMPM states

The defined state of the DMPM together with its description are listed in Table 774 and Figure 164.

Table 774 – DMPM state descriptions

State name	Description
ACTIVE	The DMPM in the ACTIVE state is ready to transmit or receive primitives to or from the Data Link Layer and the ARPM.

**Figure 164 – State transition diagram of DMPM**

9.5.9.2 DMDP state table

The DMPM state machines are defined in Table 775 and Table 776.

NOTE 1 Although each primitive contains all the available parameters, only those applicable to particular ARPM are relevant.

NOTE 2 Parameters starting with a capital letter, “Role” for instance, refer to those defined in the attribute list of each ARPM. Therefore, they are not conveyed by the service primitives defined here.

Table 775 – DMPM state table – sender transactions

#	Current state	Event or condition => action	Next state
S1	ACTIVE	<pre> FAL-PDU_req && dmpm_service_name = "DMPM_Connect_req" && DL_role:= Master => PickArep (arep_id), DL_Data.req { dl_local_dlc_identifier := LocalDlcIdentifier, dl_dlc_type := GeneralPurposeConfirmed, dl_qos_set := qos_set, dl_response_request := "True", dl_max_confirm_delay := MaxConfirmDelay, dl_max_dlsdu_length := MaxDlsduLength, dl_dlsdu_size_from_requestor := MaxDlsduSizeFromRequestor, dl_dlsdu_size_from_responder := MaxDlsduSizeFromResponder, dl_dls_user_data := dlsdu } </pre>	ACTIVE
S2	ACTIVE	<pre> FAL-PDU_req && dmpm_service_name = "DMPM_Connect_req" && DL_role:= Slave => PickArep (arep_id), ErrorToARPM reason:= InvalidDlcepRole } </pre>	ACTIVE
S3	ACTIVE	<pre> FAL-PDU_req && dmpm_service_name = "DMPM_Connect_rsp" && DL_role:= Slave => PickArep (arep_id), DL_Data.req { dl_local_dlc_identifier := LocalDlcIdentifier, dl_dlc_type := GeneralPurposeConfirmed, dl_qos_set := qos_set, dl_response_request := "True", dl_max_confirm_delay := MaxConfirmDelay, dl_max_dlsdu_length := MaxDlsduLength, dl_dlsdu_size_from_requestor := MaxDlsduSizeFromRequestorNegotiated, dl_dlsdu_size_from_responder := MaxDlsduSizeFromResponderNegotiated, dl_dls_user_data := dlsdu } </pre>	ACTIVE
S4	ACTIVE	<pre> FAL-PDU_req && dmpm_service_name = "DMPM_Connect_req" && DL_role:= Master => PickArep (arep_id), ErrorToARPM reason:= InvalidDlcepRole } </pre>	ACTIVE
S5	ACTIVE	<pre> FAL-PDU_req && dmpm_service_name = "DMPM_Disconnect_req" => PickArep (arep_id), DL_Disconnect.req { dl_reason := reason, } </pre>	ACTIVE

#	Current state	Event or condition => action	Next state
S6	ACTIVE	FAL-PDU_req && dmpm_service_name = "DMPM_Data_req" => PickArep (arep_id), DL_Data.req { dl_local_dlc_identifier := LocalDlcIdentifier, dl_dlc_type := GeneralPurposeConfirmed, dl_qos_set := qos_set, dl_response_request := response_request, dl_max_confirm_delay := MaxConfirmDelay, dl_max_dlsdu_length := MaxDlsduLength, dl_dls_user_data := dlsdu } 	ACTIVE
S7	ACTIVE	FAL-PDU_req && dmpm_service_name = "DMPM_Put_req" => PickArep (arep_id), DL_Put.req { dl_local_dlc_identifier := LocalDlcIdentifier, dl_dls_user_data := dlsdu, } 	ACTIVE
S8	ACTIVE	FAL-PDU_req && dmpm_service_name = "DMPM_Get_req" => PickArep (arep_id), DL_Get.req(in) { } DL_Get.req(out) FAL-PDU_ind { dmpm_service_name := "DMPM_Get_cnf", dl_local_dlc_identifier := LocalDlcIdentifier, fal_pdu := dlsdu, } 	ACTIVE

Table 776 – DMPM state table – receiver transactions

#	Current state	Event or condition => action	Next state
R1	ACTIVE	DL_Data.ind && AREP-state = NOT OPEN && LocateArep (dl_dls_user_data) = "NOT FOUND" => DL_Disconnect.req { dl_reason := "AREP Not Found", dl_dls_user_data := "null" }	ACTIVE
R2	ACTIVE	DL_Data.ind && AREP-state = NOT OPEN && LocateArep (dl_dls_user_data) = "NOT Relevant" => DL_Disconnect.req { dl_reason := "Invalid AREP Type", dl_dls_user_data := "null" }	ACTIVE
R3	ACTIVE	DL_Data.ind && AREP-state = NOT OPEN && DL_role:= Slave && LocateArep (dl_dls_user_data) = "True" => Connect_ind { local_dlc_identifier := dl_local_dlc_identifier , dl_dlc_type := GeneralPurposeConfirmed, qos_set:= qos_set, response_request:= "True", max_confirm_delay:= dl_max_confirm_delay, dlsdu_size_from_requestor := dl_dlsdu_size_from_requestor, dlsdu_size_from_responder := dl_dlsdu_size_from_responder, max_dlsdu_length:= dl_max_dlsdu_length, dls_user_data := dl_dls_user_data }	ACTIVE
R4	ACTIVE	DL_Data.cnf && AREP-state = NOT OPEN && FindAREP () = "False" => DL_Disconnect.req { reason := "DLCEP Not Found", dl_dls_user_data := "null" }	ACTIVE
R5	ACTIVE	DL_Data.ind && AREP-state = NOT OPEN && DL_role=Master && FindAREP () = "True" => Connect_cnf { local_dlc_identifier := dl_local_dlc_identifier , dl_dlc_type := GeneralPurposeConfirmed, qos_set:= qos_set, response_request:= "True", max_confirm_delay:= dl_max_confirm_delay, dlsdu_size_from_requestor := dl_dlsdu_size_from_requestor, dlsdu_size_from_responder := dl_dlsdu_size_from_responder, max_dlsdu_length:= dl_max_dlsdu_length, dls_user_data := dl_dls_user_data } FAL-PDU_ind { dmpm_service_name := "DMPM_Connection_Established_ind", fal_pdu := "null" }	ACTIVE
R6	ACTIVE	DL_Disconnect.ind && FindAREP () = "False" => (no actions taken)	ACTIVE
R7	ACTIVE	DL_Disconnect.ind && FindAREP () = "True" => FAL-PDU_ind { dmpm_service_name := "DMPM_Disconnect_ind", reason := dl_reason, }	ACTIVE

#	Current state	Event or condition => action	Next state
R8	ACTIVE	DL_Data.ind && AREP-state = OPEN && FindAREP () = "False" => (no actions taken)	ACTIVE
R9	ACTIVE	DL_Data.ind && AREP-state = OPEN && FindAREP () = "True" => FAL-PDU_ind { dmpm_service_name := "DMPM_Data_ind", local_dlc_identifier := dl_local_dlc_identifier , dl_dlc_type := GeneralPurposeConfirmed, qos_set:= qos_set, response_request:= "True", max_confirm_delay:= dl_max_confirm_delay, max_dlsdu_length:= dl_max_dlsdu_length, fal_pdu := dl_dls_user_data }	ACTIVE
R10	ACTIVE	DL_Data.cnf && AREP-state = OPEN && dl_status = "data-transferred" => (no actions taken)	ACTIVE
R11	ACTIVE	DL_Data.cnf && AREP-state = OPEN && dl_status <> "data-transferred" => FAL-PDU_ind { dmpm_service_name := "DMPM_Data_cnf", reason := dl_status } ErrorToARPM { reason := dl_status }	ACTIVE
R12	ACTIVE	DL_Buffer_Received.ind && FindAREP () = "False" => (no actions taken)	ACTIVE
R13	ACTIVE	DL_Buffer_Received.ind && FindAREP () = "True" => DL_Get.req(in) { } DL_Get.req(out) && dl_status = "data-transferred" FAL-PDU_ind { dmpm_service_name := "DMPM_Buffer_Received_ind", fal_pdu := dl_dls_user_data, } DL_Get.req(out) && dl_status <> "data-transferred" FAL-PDU_ind { dmpm_service_name := "DMPM_Get_cnf", reason := dl_status }	ACTIVE
R14	ACTIVE	DL_Buffer_Sent.ind && FindAREP () = "False" => (no actions taken)	ACTIVE
R15	ACTIVE	DL_Buffer_Sent.ind && FindAREP () = "True" => FAL-PDU_ind { dmpm_service_name := "DMPM_Buffer_Sent_ind" }	ACTIVE

9.5.9.3 Functions used by DMPM

Table 777 through Table 779 define the functions used by the DMPM.

Table 777 – Function PickArep

Name	PickArep	Used in	DMPM
Input		Output	
arep_id		(all the attributes of the specified AREP)	
Function	Selects the attributes for the AREP specified by the arep_id parameter. After this function is executed, the attributes of the selected AREP are available to the state machine.		

Table 778 – Function FindAREP

Name	FindAREP	Used in	DMPM
Input		Output	
local_dlc_identifier		True False	
Function	This function identifies the AREP that shall be bound with an active DMPM. True means the AREP exists. If it does, this function also returns a means to send a DMPM primitive to that AREP.		

Table 779 – Function LocateArep

Name	LocateArep	Used in	DMPM
Input		Output	True Not Found Not Relevant
data (of DL_Connect.ind)		Access to attributes of the located AREP.	
Function	This function returns the value of True and a means to get access to the attributes of the located AREP if all of the following conditions are met. Otherwise, it returns the value of either "Not Found" or "Not Relevant."		
a)	Decodes the dlsdu that is conveyed by the dl_dls_user_data argument and checks if the FAL PDU type is "EST_ReqPDU."		
b)	Decodes the FAL-PDU and extracts the RequestingAREP and RespondingAREP parameters.		
c)	Looks for the AREPs that that are bound to the connection.		

9.5.10 Data Link Layer service selection

NOTE This section briefly describes the Type 6 Data Link Layer services utilized by the FAL. These Data Link Layer services are fully defined in the Data Link Layer standard (IEC 61158-3).

9.5.10.1 Introduction

NOTE The FAL assumes that connections and resources, such as buffers or queues, are set up prior to any operations of FAL protocol machines by the Bus-configuration service. Therefore, this service is not listed in this section.

9.5.10.1.1 DL-DISCONNECT

This service is used to release an existing connection or leave from it and refers to the Disconnect service.

9.5.10.1.2 DL-DATA

This service is used for connection-oriented data transfer mode and refers to the Data service. This service is used to transfer an FAL-PDU from one AREP. When the AREP associated with this primitive is NOT OPEN the primitive carries connection establishment information. When the AREP associated with this primitive is OPEN the primitive carries CS, UCS, UCA or Abort information.

9.5.10.1.3 DL-PUT

This service is used to copy an FAL-PDU to a buffer. It refers to the Put service.

9.5.10.1.4 DL-GET

This service is used to read an FAL-PDU from a buffer. It refers to the Get service.

9.5.10.1.5 DL-BUFFER-RECEIVED

The DL-BUFFER-RECEIVED service is used to inform the FAL of a new update on the specified receive buffer.

9.5.10.1.6 DL-BUFFER-SENT

The DL-BUFFER-SENT service is used to inform the FAL that the specified buffer content has just been sent.

10 Type 7

10.1 Abstract syntax of data type

10.1.1 General

This present standard uses the following terms defined in the abstract syntax number 1 (ASN1). Different or added definitions are the following :

10.1.2 Boolean

A Boolean value is encoded on an octet all of whose bits are null for FALSE; any other combination means TRUE.

Example of notation of the type:

Response::= BOOLEAN

Example of value notation and encoding:

Response value::= TRUE

value is encoded, e.g., FFh or 6Dh.

10.1.3 Integer

The INTEGER value is encoded as a sequence of M octets describing a binary number:

- either as a complement of 2 if the values can be positive and negative: the most and least significant bits are respectively bit 8 of the first octet and bit 1 of the last octet. Bit 8 of the first octet represents the integer sign. If the size is the number of bits of the integer (i.e., $\text{size} = M \times 8$), the value of the integer is between $-2^{**}(\text{size}-1)$ and $+2^{**}(\text{size}-1)-1$;

Example of notation of the type:

Signed8::= INTEGER (-128..127)

Example of value notation and encoding:

integer Signed8::= -128

integer is encoded: FFh.

- or as an absolute value if the values are all positive or all negative: the most and least significant bits are respectively bit 8 of the first octet and bit 1 of the last octet. If the size is the number of bits of the integer (i.e., $\text{size} = M \times 8$), the value of the integer is between $-2^{**}(\text{size})$ and $+2^{**}(\text{size})-1$.

Example of type notation:

Unsigned8::= INTEGER (0..255)

Example of value notation and encoding:

integer Unsigned8::= 255

integer is encoded: FFh.

10.1.4 Bit string

The value of BIT STRING is encoded on a sequence of M octets; the first bit of the binary string is aligned with bit 8 of the least significant octet of the sequence of octets and its number is zero. The number of unused bits shall be between bit 1 and 7 of the most significant octet of the sequence of octets. The true value of a bit of the bit string takes the binary value 1; unused bits can take the value 0 or 1.

Example of notation of the type:

Bitstring ::= BIT STRING SIZE(32)

Example of value notation and encoding:

binaryvalue Bitstring ::= '5F291CD0'H

binary value is encoded: 5Fh 29h 1Ch D0h

10.1.5 Octet string

The value of OCTET STRING is encoded on a sequence of M octets; the first octet of the OCTET STRING is aligned with the least significant octet of the sequence of octets.

Example of type notation:

OctetString ::= OCTET STRING SIZE(4)

Example value notation and encoding:

octetvalue OctetString ::= '5F291CD0'H

octetvalue is encoded: 5Fh 29h 1Ch D0h.

10.1.6 Sequence (sequence or sequence of)

SEQUENCE or SEQUENCE OF encoding is the juxtaposition of the element encoding which comprise it.

Example of type notation:

INFO1 ::= SEQUENCE {name String, ok BOOLEAN}

String ::= OCTET STRING SIZE(5) --string of 5 octets

Example of value notation and encoding:

value INFO1 ::= {name "SMITH", ok TRUE}

value is encoded as shown in Table 780.

Table 780 – Example of encoding of a SEQUENCE

0006h	53h	4Dh	49h	54h	48h	FFh
SEQUENCE length	"S"	"M"	"I"	"T"	"H"	TRUE

The top line represents the encoding proper and the bottom line describes what is encoded:

Example of type:

INFO2::= SEQUENCE OF date

date::= OCTET STRING SIZE (8) --YYYYMMDD

Example of value notation and encoding:

value INFO2::= { { date "19571111"}, {date "19590717"}}

value is encoded as shown in Table 781.

Table 781 – Example of encoding of a SEQUENCE OF

00h	10h				
number	of octets				
31h	39h	35h	37h	31h	31h
"1"	"9"	"5"	"7"	"1"	"1"
31h	31h	31h	39h	35h	39h
"1"	"1"	"1"	"9"	"5"	"9"
30h	37h	31h	37h		
"0"	"7"	"1"	"7"		

In each row pair, the top line represents the encoding proper and the bottom line describes what is encoded. Each table is read from left to right.

10.1.7 Choice

The encoding of CHOICE is the encoding of the possibility selected in this CHOICE.

Example of type notation:

INFO::= CHOICE {name [0] String, age [1] Unsigned8}

String::= OCTET STRING SIZE(5) --string of 5 octets

Unsigned8::= INTEGER (0..127) -- unsigned 8 bit integer

Example value notation and encoding:

value INFO::= {name "SMITH"}

value is encoded as shown in Table 782.

Table 782 – Example of encoding of a CHOICE

80h	53h	4Dh	49h	54h	48h
identification	"S"	"M"	"I"	"T"	"H"

The top line represents the encoding proper and the bottom line describes what is encoded:

10.1.8 Null

No content encoding is associated with the null element.

Identification encoding is possible if NULL is a possibility of a choice (Cf. section 10.1.2.5.2).

Example of notation of the type:

```
Type-Room ::= SEQUENCE {
    number Unsigned8,
    person CHOICE {
        name [0] String,
        default [1] NULL --room not in use
    }
}
```

Unsigned8 ::= INTEGER (0..127)

String ::= OCTET STRING SIZE(12)

Example of value notation and encoding:

value-Room Type-Room ::= {number 30h, person { default NULL}}

value-Room is encoded 0002h 30h 81h

Length encoding is compulsory if NULL is optional (in this case, the length is necessarily equal to zero).

10.1.9 Object identifier

Encoding is comprised of an ordered list of encoding of concatenated sub-identifiers.

Every sub-identifier is represented by a sequence of one or more octets. Every octet is encoded as follows:

- Bit 8 of each octet indicates that it is the last in the sequence:
- Bit 8 of the last octet is on one;
- Bit 8 of each previous octet is on zero.
- Bits 7 to 1 of the octets in the sequence collectively encode the sub-identifier, knowing that:

Bits 5, 6 and 7 are zero ;

- Bits 1 to 4 are used to encode each digit of each sub-identifier. Encoding occurs in the form of an unsigned binary number.

Example: {iso standard 9506 part(2) mms-general-module-version1(2)} is encoded as shown in Table 783.

Table 783 – Example of encoding of an object identifier

81h	80h	09h	05h	00h
"iso"	"standard"	9	5	0

86h	82h	82h
6	part(2)	mms...(2)

In each table, the top line represents the encoding proper and the bottom line describes what is encoded. Each table is read from left to right.

10.1.10 Default

The keyword DEFAULT is ignored.

10.1.11 Example of encoding

This section illustrates the FER encoding rules specified in the present standard, by showing the representation in octets of an element defined in FIELDBUS ASN.1.

```

PDU ::= CHOICE {
    req [0] IMPLICIT PDUreq,
    rep [1] IMPLICIT PDUrep, ....
}

PDUrep ::= SEQUENCE {
    invokeID Unsigned32,
    Response }

Response ::= CHOICE {
    status                               [0] IMPLICIT StatusResponse, ...
    getprog                             [45] IMPLICIT GetProgramInvocationAttributesResponse, ...}

GetProgramInvocationAttributesResponse ::= SEQUENCE {
    pi_state                            [0] IMPLICIT PiState,
    listOfDomainId                     [1] IMPLICIT ListOfObjectID,
    mmsdeletable                       [2] IMPLICIT BOOLEAN,
    reusable                           [3] IMPLICIT BOOLEAN,
    monitor                            [4] IMPLICIT BOOLEAN,
    executionargument                  [5] IMPLICIT OCTET STRING OPTIONAL
}

PiState ::= Unsigned8

Unsigned8 ::= INTEGER (0..127) --8 bit unsigned integer

ListOfObjectID ::= SEQUENCE OF Identifier

Identifier ::= Unsigned16

Unsigned16 ::= INTEGER (0..32767) --16 bit unsigned integer

Unsigned32 ::= INTEGER(0..2147483647) --32 bit unsigned integer

The registered value is:
{ rep { invokeID 1,
    getprog { pi-state 8,
        listOfDomainId { {1024}, {1025}, {1026}, {1027} },
        mmsdeletable TRUE,
        reusable FALSE,
        monitor TRUE,
        executionargument "ARGUMENT" }
    }
}
    
```

Its representation is as follows: octet encoding is described using the two-line tables below in which the top line represents the encoding proper and the bottom line describes what is encoded. The tables are read from left to right. See Table 784 for details.

Table 784 – Example of encoding of a PDU

81h	00h	00h	00h	01h	ADh		
rep	invokeID	(32 b ...		...)	getprog		

08h	00h	08h	04h	00h	04h	01h	
pi-state	number	of octets	1024	(16 b)	1025	(16b)	

04h	02h	04h	03h	FFh	00h	FFh	00h
1026	(16b)	1027	(16b)	TRUE	FALSE	TRUE	number

08h	41h	52h	47h	55h	4Dh	45h	4Eh
of octets	"A"	"R"	"G"	"U"	"M"	"E"	"N"

54h	
"T"	

10.2 Transfer Syntaxes

10.2.1 General

The encoding rules applied to the PDU type values represented by using the ASN1 (ISO 8824) abstract notation syntax, yield the transfer syntax.

In the MPS environment, the values associated with the PDU types, are globally encoded according to the encoding rules of ASN1 (ISO/IEC 8825), with some restrictions.

The encoding rules which yield the MPS transfer syntax associate a set of octets with each of the different PDU types.

This octet sequence comprises three components:

- identification
- size
- contents

In the general case, the encoding rules permit contents which are recursively structured themselves in Identification, Size, Contents.

NOTE In the case of a CompactValue type PDU, the encoding has only one level.

The organization of an encoding level corresponds to that of the ISO/IEC 8825.

Encoding structure is shown in Figure 165.

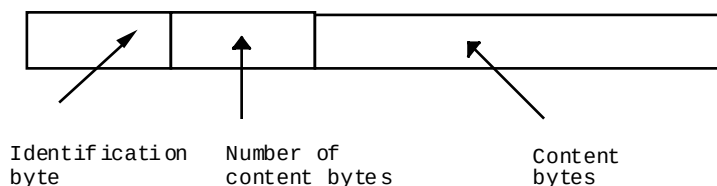


Figure 165 – Encoding of a CompactValue

The following restrictions on the encoding relative to the MPS standard apply:

The Identification is encoded on 1 octet.

The Contents is encoded on a maximum of 126 octets.

10.2.2 MPS ASE FAL PDU data types

10.2.2.1 Summary

The specification of the protocol provides a set of communication rules expressed in terms of data interchanges and procedures that shall be supported by the application entities.

AL TYPE 7 conformance is based on the compliance to these communication rules which are guidelines for conformance testing in order to achieve interoperability.

This protocol specification consists of:

- the description of the various protocol data units (PDU) interchanged between the communicating entities. This description is affected by use of an abstract syntax.
- encoding rules which transform the abstract syntax into transfer syntax.
- the description of the protocol procedures in terms of interactions between the primitives of each of the application services and the associated data link service primitives.

NOTE In the MPS environment, there is no direct interaction between a producer application entity and a consumer entity during the interchanges associated with a service primitive.

The specification of A_PDUs in the MPS environment is performed with a notation defined independently from the encoding. This notation constitutes the abstract syntax.

The abstract syntax is subsequently transformed by a set of encoding rules which are applied to the various types of A_PDU. These encoding rules yield the transfer syntax.

The conjunction of the abstract syntax and the encoding rules specifies the content of the octets interchanged between application entities by using the data link services.

NOTE The actual representation at the interface between the user and an application entity is out of the scope of this standard and is implementation dependent.

10.2.2.2 Syntax notation

The specification of the different A_PDUs associated with the MPS services is performed with the abstract notation syntax ASN1 defined in the ISO/IEC 8824.

All the ASN1 definitions of this standard are part of the module: " AL TYPE 7 - -MPS-1 ". All the grammar elements which constitute the Part5-3-MPS-1 module correspond to the first version of MPS.

Declaration of the AL TYPE 7 - -MPS-1 module:
AL TYPE 7 - -MPS-1 DEFINITIONS:= BEGIN

NOTE In order to meet the requirements of the ASN1 notation, all the keywords of the language are written in capital letters and the final symbol names begin with a capital letter. Furthermore, all the grammar items existing in this standard which take part in the definition of the AL TYPE 7 - -MPP-1 module are outlined with a double line before and after them.

General quote: The present standard specifies the value of each octet of an encoding by the sentences 'the most significant bit' and 'the least significant bit'. The bits of an octet are numbered from 8 to 1: bit 8 is 'the most significant bit' and bit 1 'the least significant bit'. This encoding is summarized in Figure 166.

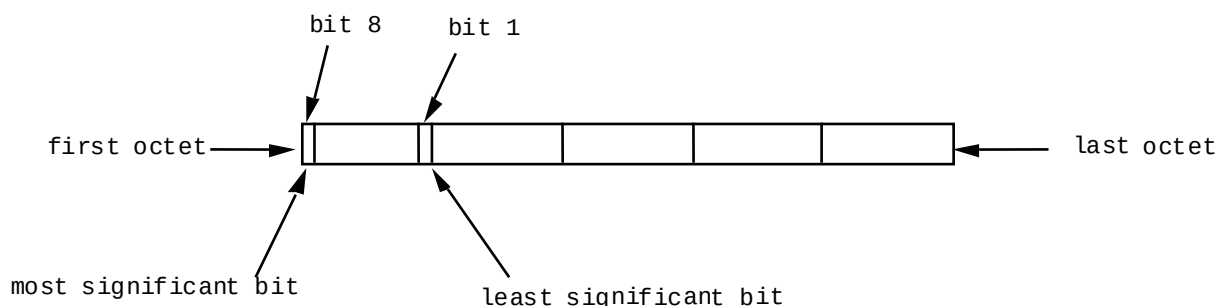


Figure 166 – Organisation of the bits and octets within a PDU

10.2.2.3 Simple predefined types

The PDU specification uses simple predefined types.

Some simple types are predefined by means of a statement that defines limits for these type values.

```
Unsigned4      ::= INTEGER          -- values limited between 0 and 15
Unsigned8      ::= INTEGER          -- values limited between 0 and 255
Unsigned16 ::= INTEGER              -- values limited between 0 and 65535
Symbol ::= Visible String          -- number of characters limited between 0 and 16
```

In the MPS environment the variable names, the access names, the type names and the list names are described as a combination of 16 character maximum length.

The characters belong to the set defined for the 'visible string' type of the ASN1 (ISO 8824) syntax notation.

In this set, the following characters are permitted:

```
Capital letters      (A..Z)
Letters              (a..z)
Digits               (0..9)
Underscore            ( _ )
Currency symbol ($)  ( $ )
```

The 'space' character is not permitted.

Strings may not begin with a number.

The 'Unsigned16', 'Unsigned8' and 'Unsigned4' types are used in this standard to describe the limits of the maximum values of the MPS types.

10.2.2.4 PDU Types

10.2.2.4.1 Top Level Definition

The PDUs specified in the MPS environment convey the information associated with *Variable* objects. The PDU type used depends directly on the value of the *Construction* and *A_Name* attributes of the instance of the *Type constructor* associated with the variable and is summarized in Table 785.

Table 785 – MPS PDU types

Construction		PDU type
SIMPLE ARRAY STRUCTURE		CompactValuePDU
		CompactValuePDU
		CompactValuePDU
PREDEFINED	A_Name = K_DVAR	VariableDescriptionPDU
	A_Name = K_DACCESS	AccessDescriptionPDU
	A_Name = K_DTYPE	TypeDescriptionPDU
	A_Name = K_DLIST	ListDescriptionPDU
EXPLICIT		ExplicitPDU

The encoding of values of these different kinds of PDU uses the transfer syntax associated with ASN1 part 2 (ISO 8825). Nevertheless, array compacting and compressing rules can be implemented to optimize the flow on the bus.

NOTE These compacting rules allow the representation of structured type variables by using an octet string.

Specification of PDU type used in MPS:

MPS-pdu:: = CHOICE

```
{
  CompactedVariableValue      [APPLICATION 0]      IMPLICIT CompactValuePDU,
  ExplicitVariableValue       [APPLICATION 1]      IMPLICIT ExplicitPDU,
  VariableDescription         [APPLICATION 2]      IMPLICIT VariableDescriptionPDU,
  AccessDescription          [APPLICATION 3]      IMPLICIT AccessDescriptionPDU,
  TypeDescription            [APPLICATION 4]      IMPLICIT TypeDescriptionPDU,
  ListDescription             [APPLICATION 5]      IMPLICIT ListDescriptionPDU,
  extensions                  [APPLICATION 6]      ANY
}
```

NOTE The [APPLICATION 7] to [APPLICATION 15] type PDUs are reserved by this standard for a future definition of other PDU types. The [APPLICATION 16] to [APPLICATION 30] type PDUs are reserved for the use of the Network Management standard. Moreover, the ANY type PDU is used to allow the future definition of additional PDU types in companion standards.

10.2.2.4.2 VariableCompactValue PDU

10.2.2.4.2.1 General

This PDU represents the variable value of SIMPLE, STRUCTURE or ARRAY construction type, and optionally the associated production status. These types are used for variables of NORMAL or SYNCHRONIZATION class.

The PDU of CompactValuePDU type is represented by using the OCTET STRING type:

Compact-value-pdu:: = OCTET STRING

-- The total length of the PDU shall not exceed 128 octets.

NOTE The fields of this PDU permits encoding of the following attributes in the MPS application layer objects. See Table 786.

Table 786 – Fields of a CompactValuePDU

Field	Class	Attribute
Compact_value_PDU	Variable	Public value Transmitted status value

The ASN1 OCTET STRING type will be then encoded under the ASN1 (ISO 8825 standard) rules as specified in the chapter concerning the transfer syntax.

NOTE The representation of a MPS variable under an ASN1 OCTET STRING type is only possible considering the following restrictions:

- Producers and consumers of the variable have a prior knowledge of the MPS type of the variable, and do not need to transmit the type description with the value of the variable. The knowledge is available by access to the description variable and description type variables.
- The type of the variable has a fixed length and has no conditional or optional fields.

The representation of the public value associated with a variable by using compacting rules is performed from the first content octet of the CompactValuePDU.

The production status are represented on the last octet when this option is selected.

The PDU's length octet value represents the octet count used to encode the variable value and the production status octet when it exists.

The representation of the production status on this octet is performed as follows:

- The refreshment status is encoded on bit 1. This bit is set to '1' when the refreshment status is 'TRUE', and '0' when it is 'FALSE'.
- The punctual refreshment status is encoded on bit 2. This bit is set to '1' when the punctual refreshment status is 'TRUE', and '0' when it is 'FALSE'.
- The bits 3 to 8 are reserved.

The encoding of a given variable is represented with a constant number of octets derived from the type specification of this variable.

10.2.2.4.2.2 BOOLEAN value encoding rule

The boolean encoding is performed on a single octet.

If the value is:

FALSE: the octet is 00_{16}

TRUE: the octet may take any value different from zero chosen by the producer entity.

10.2.2.4.2.3 INTEGER value encoding rule

The encoding of an integer value is performed on one or several octets. The maximum number of octets is fixed by the maximum size authorized for the INTEGER type, that is, 32 octets.

The number of octets used for encoding an INTEGER type corresponds to $E[(e-1)/8] + 1$ where E is the integer part, and e the value of the *size* attribute associated with this type.

The content octets shall be a two-complement binary number equal to the integer value and be composed of bits 8 to 1 of the first octet, followed by bits 8 to 1 of the second octet, followed by bits 8 to 1 of each following octet, up to and including the last octet of the contents.

NOTE The value of a two-complement binary number is derived by numbering the bits of the content octets, starting from bit 1 of the last octet and ending at bit 8 of the first octet. Each bit is assigned a numerical value of 2^{n-1} , where n is the bit position within the numbering sequence. The value of the two-complement binary number is obtained by adding the numerical values assigned to each of the bits set to '1' (except for bit 8 of the first octet) from which the signed value from bit 8 of the first octet is derived. The sign is negative when bit 8 of the first octet is set to '1'.

10.2.2.4.2.4 BITSTRING value encoding rule

The encoding of a bitstring value is performed on one or several octets as shown in Figure 167.

The maximum number of octets is fixed by the maximum size authorized for a BITSTRING type, that is, 32 octets.

The number of octets used for encoding a BITSTRING type corresponds to $E[(e-1)/8] + 1$ where E is the integer part and e the size of this type.

The bits of the bit string, starting from the first bit and ending at the last bit, shall be placed in sequence and in this order:

- in bits 8 to 1 of the first octet for the first 8 bits of the string,
- in bits 8 to 1 of the second octet for the 8 following bits,
- then in bits 8 to 1 of each following octet,
- followed by as many bits of the final octet as necessary always starting with bit 1.

The unused bits of the last octet have a value chosen by the producer entity.

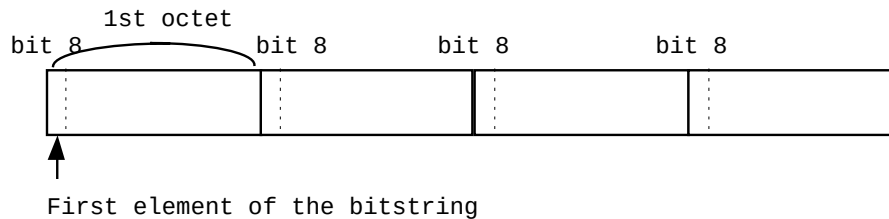


Figure 167 – Encoding of a Bitstring

10.2.2.4.2.5 UNSIGNED value encoding rule

The encoding of an unsigned integer value is performed on one or several octets. The maximum number of octets is fixed by the maximum size permitted for an UNSIGNED type, that is, 32 octets. The number of octets used for encoding a UNSIGNED type corresponds to $E[(e-1)/8] + 1$ where E is the integer part and e the value of the *size* attribute associated with this type.

The content octets shall be a binary number equivalent to the integer value, composed of bits 8 to 1 of the first octet, followed by bits 8 to 1 of the second octet, followed by bits 8 to 1 of each following octet, up to and including the last octet of the contents.

NOTE The value of a binary number is obtained by numbering the bits of the content octets, starting from bit 1 of the last octet and ending at bit 8 of the first octet. Each bit is assigned a numerical value of 2^{n-1} , where n is the bit position within the numbering sequence.

10.2.2.4.2.6 OCTETSTRING value encoding rule

The encoding of an OCTETSTRING value contains zero, one or several content octets whose value is equal to that of the data value octets, in the same sequence as they are placed in the data value, the most significant bit of a data value being aligned with the most significant bit of an octet of the contents.

The number of octets used for encoding an OCTETSTRING type corresponds directly to the number of terms of this string within the limit of 126 octets.

10.2.2.4.2.7 VISIBLE STRING value encoding rule

The value of a visible string is composed of a string of characters from the set specified in ISO 8824. Each of the characters of this string is encoded in one octet.

The number of octets used for encoding a VISIBLE STRING type corresponds directly to the number of elements of the string within the limit of 126 octets.

10.2.2.4.2.8 GENERALIZED TIME value encoding rule

The encoding of the value is performed in the same manner as a 14 character VISIBLE STRING value.

The generalized time (specified in ISO 8824) is characterized by a specific number of characters encoded on octets.

The generalized time is a juxtaposition of characters in the following sequence:

- 4 digits for the year
- 2 digits for the month

- 2 digits for the day
- 2 digits for the hour
- 2 digits for the minutes
- 2 digits for the seconds

The first octets are those including the encoding of the year.

The generalized time expresses directly a local time.

10.2.2.4.2.9 FLOATING POINT value encoding rule

The encoding of a floating point value is performed on 4 or 8 octets according to the size attribute corresponding to single or double precision.

The presentation formats defined by the ANSI/IEEE/754 standard are as shown in Figure 168.

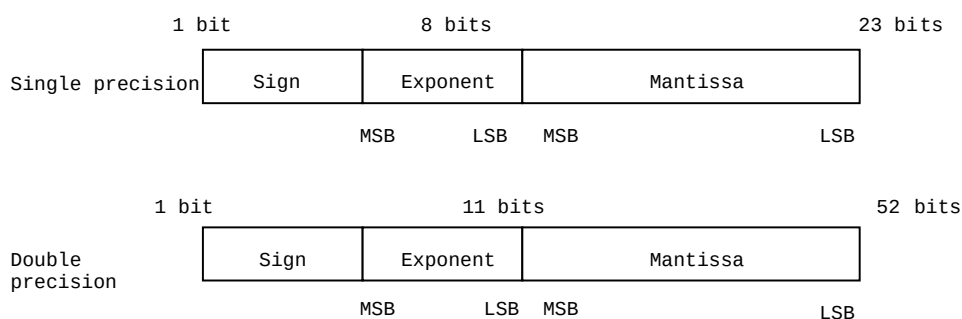


Figure 168 – Encoding of a Floating point

The content octets contain the sign, the exponent and the mantissa in the previous order starting from the first to the last octet.

In all cases the sign is encoded by using bit 8 of the first octet. It is followed by the the exponent starting from bit 7 of the first octet, and then the mantissa starts from bit 7 of the second octet for the simple precision and from bit 4 of the second octet for the double precision.

The encoding of the mantissa value and the exponent value is described in the ANSI/IEEE/754 standard.

10.2.2.4.2.10 BINARY TIME value encoding rule

The encoding of a binary time value is performed on a number of octets defined by the size, and describes a number of elementary time units. The value of the elementary time unit depends on the selected size.

The content octets contain a binary value equivalent to the binary time value, and composed of bits 8 to 1 of the first octet, followed by bits 8 to 1 of each following octet, up to and including the last octets of the contents.

NOTE The binary value is obtained by numbering the bits of the content octets, starting from bit 1 of the last octet and ending at bit 8 of the first octet. Each bit set to '1' is assigned a numerical value of 2^{n-1} , where n is the bit position within the numbering.

10.2.2.4.2.11 BCD value encoding rule

The BCD encoding is performed on one octet. The binary value associated with the BCD value is encoded on bits 1 to 4 of the octet and obtained by starting from bit 1 up to bit 4. Each bit set to '1' is assigned a numerical value of 2^{n-1} , where n is the position of the bit within the octet.

10.2.2.4.2.12 Structure encoding rule

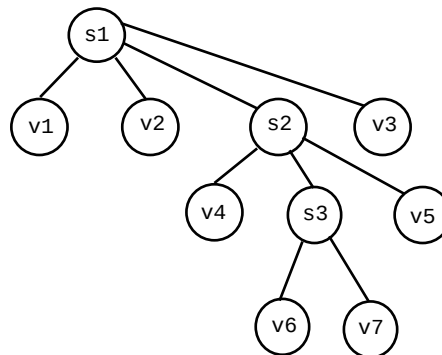
The encoding of a structure type involves the encoding rules of each of the simple types and a concatenation rule of the types composing this structure.

The encoding of the various types composing a structure are juxtaposed in order of appearance within the structure.

The value associated with the first structure field is encoded in the first octets of the content. The value associated with the second field is encoded in the following octets, and so on, up to the last field of the structure.

A structure can comprise several levels of abstraction; in this case, the encoding juxtaposition of the values associated with the various fields is performed by a first in depth traverse over the type tree associated with the structure as shown in Figure 169.

Consider the following type tree:



Representation within the octets of the contents:

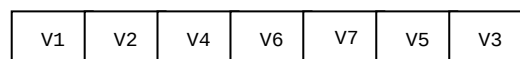


Figure 169 – Encoding of a structure

10.2.2.4.2.13 Array encoding

The array encoding involves the encoding rule of a simple type and a concatenation rule of the elements which compose the array.

The encoded values of the array elements are juxtaposed in the statement sequence of the array: these elements may be of simple, structure or array type.

The first array element is contained in the first octet of the content octets.

The array types may be optionally specified with optimisation rules of the associated encoding.

These optimisation rules do not preclude a partial access to an array element and preserve the integrity of the handled data.

10.2.2.4.2.14 Compacted encoding rule for a BOOLEAN array

The normal encoding of a BOOLEAN array type value uses one octet for each element of the array.

The compacted encoding of a BOOLEAN array type value is performed using one octet for a set of eight booleans.

Thus, the content octets used for encoding a compacted BOOLEAN array are organized as shown in Figure 170.

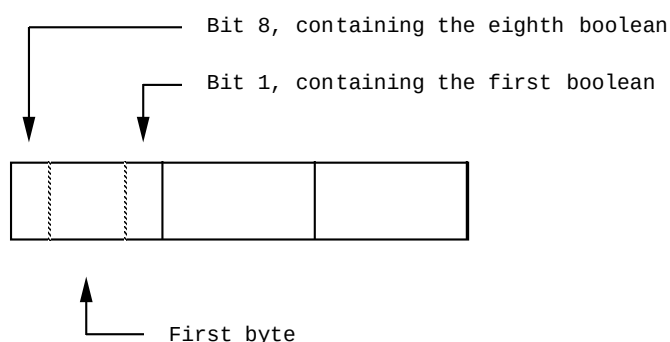


Figure 170 – Encoding of a boolean array

In the compacted encoding of a boolean array, a boolean value is 'TRUE' when the corresponding bit is set to '1'.

Every compacted boolean array is encoded using a number of octets equal to $E[(e-1)/8] + 1$ where E is the integer part and e is the number of elements of this boolean array.

10.2.2.4.2.15 Compacted encoding rule for a BCD array

The normal encoding of a BCD array type value uses one octet for each element of the array.

The compact encoding of a BCD array type value is performed by using one octet for a pair of BCDs.

Thus, within the content octets used for encoding a compacted BCD array, bits 8 to 5 of the first octet contain the first BCD of the array, and bits 4 to 1 of the first octet contain the second BCD of the array, and so on up to the last BCD of the array.

The binary value associated with the value of the first BCD of a octet is obtained by starting from bit 5 up to bit 8: each bit is assigned a numerical value of 2^{n-5} , where n is the position of the bit within the octet.

The binary value associated with the value of the second BCD of a octet is obtained by starting from bit 1 up to bit 4: each bit is assigned a numerical value of 2^{n-1} , where n is the position of the bit within the octet.

10.2.2.4.3 ExplicitVariableValue PDU

The Explicit PDU type PDU allows the representation of the values of the variables for which the type construction is EXPLICIT.

This PDU supports variable values which are not of constant length.

```
ExplicitPDU ::= CHOICE
{
    array                [0] IMPLICIT SEQUENCE OF ExplicitPDU,
    structure            [1] IMPLICIT SEQUENCE OF ExplicitPDU,
    boolean              [2] IMPLICIT BOOLEAN,
    bitString            [3] IMPLICIT BITSTRING,
    integer              [4] IMPLICIT INTEGER,
    unsigned             [5] IMPLICIT INTEGER,
    floating             [6] IMPLICIT OCTET STRING,
    octetString          [7] IMPLICIT OCTET STRING,
    visibleString        [8] IMPLICIT OCTET STRING,
    generalizedTime      [9] IMPLICIT OCTET STRING,
    binaryTime           [10] IMPLICIT OCTET STRING,
    bcd                  [1] IMPLICIT INTEGER
}
```

The total length of the PDU shall not exceed 128 octets.

10.2.2.4.4 VariableDescription PDU

This PDU represents the value of the variables whose type is K_DVAR. This type is used for the description variables.

```
VariableDescriptionPDU ::= SEQUENCE
{
    FixedDescription      [0] IMPLICIT Description,
                        -- predefined type specified hereafter
    variableName          [1] IMPLICIT Symbol,
    resynchronisationReference [2] IMPLICIT ObjectReference OPTIONAL,
    productionPeriod      [3] IMPLICIT Unsigned16 OPTIONAL,
    synchronisationReference [4] IMPLICIT ObjectReference OPTIONAL,
    ProductionSlot        [5] IMPLICIT Unsigned16 OPTIONAL,
    punctualSynchroReference [6] IMPLICIT ObjectReference OPTIONAL,
    listReference          [7] IMPLICIT ObjectReference OPTIONAL,
    otherDetails           [8] IMPLICIT ANY OPTIONAL,
    vendorInformation      [9] IMPLICIT OCTET STRING OPTIONAL
}
```

The total length of the PDU shall not exceed 128 octets. See Table 787.

NOTE The fields of this PDU permits encoding of the following attributes in the MPS application layer objects.

Table 787 –Fields of a VariableDescriptionPDU

Field	Class	Attribute
FixedDescription	Variable	Identifier Network Period Class Consistency Variable Significant Status Reference Type Constructor
Variable Name	Variable	A_Name
ResynchronizationReference	Resynchronization in Production	Reference (Synchronization) Variable
ProductionPeriod	Refreshment	Production Variable
SynchronizationReference	Refreshment	Reference (Synchronization) Variable
ProductionSlot	Punctual Refreshment	Production Time Slot
PunctualSynchroReference	Punctual Refreshment	Reference (Synchronization) Variable
ListReference	Variable	Reference Variable List

The type description of this PDU uses the Description type which contains the non optional part of constant size of this PDU.

Description:: = OCTET STRING -- Size fixed to 7 octets

Octet semantics:

octets 1 + 2: Described variable identifier

(Most significant bit of the identifier in bit 8 of the first octet and least significant bit in bit 1 of the second octet)

octets 3 + 4: Network period

(Most significant bit of the network period in bit 8 of the first octet, in multiples of 1 millisecond; a 0 value indicates an aperiodic variable)

octet 5: *Bit 8, and 7:* Variable class

00: NORMAL

01: SYNCHRONIZATION

10: DESCRIPTION

All the other combinations are reserved

Bit 6: Sub-class of a synchronization variable

1: Consistency variable

0: Synchronization variable but not consistency variable

Bits 5 and 4: Significant status

00: no significant status

01: significant refreshment

10: significant punctual refreshment

11: significant refreshment and punctual refreshment

Bits 3 to 1 reserved.

octets 6 + 7: Type reference

(encoded on 16 bits by using the description variable identifier of this type. The most significant bit is bit 8 of the octet 6 and the least significant one bit 1 of the octet 7)

The type description of this PDU uses the non terminal grammar element ObjectReference referring to variables, lists and types by means of the identifier of the associated description variable. These identifiers are encoded on two octets: the most significant bit of the identifier is encoded in bit 8 of the first octet and the least significant one in bit 1 of the second octet.

ObjectReference::= OCTET STRING -- Size fixed to 2 octets

The ANY types included in the definition of the PDU, as well as the reserved bits within this PDU, used to support, the description complements specified elsewhere.

Moreover, optional octet strings permit, insertion of vendor specific information into this PDU.

10.2.2.4.5 AccessDescription PDU

This PDU supports the variable value whose type is K_DACCESS. This type is used for the access description variables. See Table 788.

AccessDescriptionPDU ::= SEQUENCE

```
{
  accessName                [0] IMPLICIT Symbol,
  rootReference              [1] IMPLICIT ObjectReference,
  accessPath                 [2] IMPLICIT SEQUENCE OF CHOICE
  {
    structureFieldName       [0] IMPLICIT Symbol,
    arrayElementIndex        [1] IMPLICIT Unsigned8
  } OPTIONAL
  otherDetails               [3] IMPLICIT ANY OPTIONAL,
  vendorInformation          [4] IMPLICIT OCTET STRING OPTIONAL
}
```

NOTE The fields of this PDU permits encoding of the following attributes in the MPS application layer objects.

Table 788 –Fields of AccessDescriptionPDU

Field	Class	Attribute
Access Name	Access	A Name
Root Reference	Access	Reference Variable
Access Path	Type Constructor	Field Name

The ANY types include description complements specified elsewhere in the definition of the PDU.

Moreover, optional octet strings permit insertion of vendor specific information into this PDU.

10.2.2.4.6 TypeDescription PDU

This PDU supports the variable value whose type is K_DTYPE. This type is used for the type description variables.

TypeDescriptionPDU ::= SEQUENCE

```
{
  typeName                  [0] IMPLICIT Symbol,
  typeDescription           [1] CHOICE
  {
    array                   [0] IMPLICIT SEQUENCE
    {
      compressed            [0] IMPLICIT BOOLEAN DEFAULT FALSE,
      dimension              [1] IMPLICIT Unsigned8,
      elementReference       [2] IMPLICIT ObjectReference
    },
    structure                [1] IMPLICIT SEQUENCE OF SEQUENCE
    {
      fieldName              [0] IMPLICIT Symbol OPTIONAL,
      fieldTypeReference     [1] IMPLICIT ObjectReference
    },
    -- simple                Size                Type Class
    boolean                  [2] IMPLICIT NULL,        -- BOOLEAN
    binaryString             [3] IMPLICIT Unsigned8, -- BITSTRING
    integer                  [4] IMPLICIT Unsigned8, -- INTEGER
    unsigned                 [5] IMPLICIT Unsigned8, -- UNSIGNED
    floatingPoint            [6] IMPLICIT BOOLEAN, -- FLOATING POINT
    octetString              [7] IMPLICIT Unsigned8, -- OCTET STRING
    visibleString            [8] IMPLICIT Unsigned8, -- VISIBLE STRING
    universalTime            [9] IMPLICIT NULL,        -- UNIVERSAL TIME
    binaryTime               [10] IMPLICIT Unsigned4, -- BINARY TIME
    bcd                      [11] IMPLICIT NULL,       -- BCD
    predefined               [12] IMPLICIT NULL,
    explicit                 [13] IMPLICIT NULL,
    otherDetails             [2] IMPLICIT ANY OPTIONAL,
    vendorInformation         [3] IMPLICIT OCTET STRING OPTIONAL
  }
}
```

-- The total length of the PDU shall not exceed 128 octets. See Table 789.

NOTE The fields of this PDU permits encoding of the following attributes in the MPS application layer objects.

Table 789 – Fields of TypeDescriptionPDU

Field	Class	Attribute
TypeName	TypeConstructor	A_Name
Compressed	TypeConstructor	Compressed
Dimension	TypeConstructor	Dimension
ElementReference	TypeConstructor	TypeConstructor
FieldName	TypeConstructor	FieldName
FieldTypeReference	TypeConstructor	TypeConstructor
BinaryString	TypeConstructor	Size
Integer	TypeConstructor	Size
Unsigned	TypeConstructor	Size
FloatingPoint	TypeConstructor	Size
OctetString	TypeConstructor	Size
VisibleString	TypeConstructor	Size
BinaryTime	TypeConstructor	Size

The ANY types include description complements specified elsewhere in the definition of the PDU.

Moreover, optional octet strings permit insertion of information specific to the vendors into this PDU.

ListDescription PDU

This PDU supports the variable value whose type is K_DLIST. This type is used for the list description variables.

```

ListDescriptionPDU ::= SEQUENCE
{
  listName                [0] IMPLICIT Symbol,
  variableReferenceList    [1] IMPLICIT ListComponents,
  inconsistencyDetection    [2] IMPLICIT Unsigned8 OPTIONAL,
                           -- 0UNDEFINED
                           -- 1PERMANENT
                           -- 2PERMANENT
                           -- The other values are reserved
  synchronizationReference [3] IMPLICIT ObjectReference OPTIONAL,
  consistencyVariableList  [4] IMPLICIT ListConsistencyVariables OPTIONAL,
  recoveryPeriod           [5] IMPLICIT Unsigned16 OPTIONAL,
  otherDetails             [6] IMPLICIT ANY OPTIONAL,
  vendorInformation        [7] IMPLICIT OCTET STRING OPTIONAL
}

```

-- The total length of the PDU shall not exceed 128 octets. See Table 790.

NOTE The fields of this PDU permits encoding of the following attributes in the MPS application layer objects.

Table 790 – Fields of a ListDescriptionPDU

Field	Class	Attribute
ListName	VariableList	A_Name
VariableReferenceList	VariableList	List of Reference ConsumedVariable
InconsistencyDetection	VariableList	InconsistencyDetection
SynchronizationReference	VariableList	Reference (Synchronization)Variable
ConsistencyVariableList	VariableList	List of Reference (Consistency) Variable
RecoveryPeriod	VariableList	RecoveryPeriod

The type description of this PDU uses the non terminal grammar element 'listComponents' which defines the representation format of the references to the variables which compose the described list.

ListComponents ::= OCTET STRING

-- The size of this string depends on that of the list.

Each variable name which comprise this list is encoded on two octets of this octet string, starting from the first two octets up to the last two octets within the size limits provided by this PDU. The list variable names are represented on two octets in a unique way by means of the identifier of the description variable associated with each variable of this list. The most significant bit of the identifier is encoded in bit 8 of the first octet, and the least significant one in bit 1 of the second octet.

The type description of this PDU uses the non terminal grammar element 'listConsistencyVariables' which defines the representation format of the references to the consistency variables associated with the variable list in the case of the operation of the spatial consistency mechanism.

ListConsistencyVariables::= OCTET STRING -- size relative to the number of application entities involved in the spatial consistency.

Each consistency variable name, which takes part in the elaboration of the spatial consistency of this list, is encoded on two octets of this octet string, starting from the first two octets up to the last two octets within the size limits provided by this PDU. The consistency variable names are represented on two octets in an unique way by means of the identifier of the description variable associated with each consistency variable.

The most significant bit of the identifier is encoded in bit 8 of the first octet, and the least significant one in bit 1 of the second octet.

10.2.2.4.7 PDU Extensions

The ANY types include description complements specified elsewhere in the PDU definition.

For, all the grammar elements participating in the definition of the module being defined, it is necessary to declare the end of this module.

END -- End of the module AL TYPE 7 - -MPS-1

10.2.3 MCS AR ASE AL PDU types

10.2.3.1 Coding rules

All MCS PDUs contain a whole number of octets. The octets of a PDU are numbered from 1 to a maximum of 256 for the longest PDUs. These octets are transmitted via the bus in order from the first octet to the last. Each of the octets of a PDU has bits numbered 1 to 8, where bit 1 is the least significant and bit 8 the first transmitted on the line. Finally, when a number of consecutive octets represents a binary number, the least significant octet contains the most significant value.

In the following figures, the least significant number octets are always on the left when represented horizontally, and at the top when represented vertically. Also, the bits in a octet are always represented with bit 8 on the left and bit 1 on the right.

10.2.3.2 Structure of the PDUs

10.2.3.2.1 Top Level Definition

As a general rule, all MCS PDUs consist, in order, of:

- the header containing:
- the header size,
- the fixed part containing what is mandatory,
- the variable part containing what is optional.
- the data, which are optionally present.

Representation is shown in Figure 171

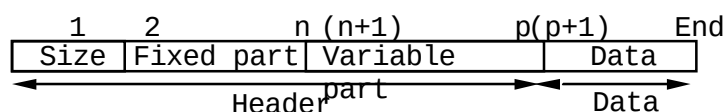


Figure 171 – Representation of a MCS PDU

a) Size:

The size is equal to sum of the numbers of octets of the fixed part and the variable part of the PDU header.

The value of the size is coded in the first octet of the header as a binary number.

The maximum size value is 255, which is coded as 1111 1111.

b) Fixed part

Whatever the PDU type, the fixed part contains its code followed by coding of a list of parameters which are mandatory for the code value.

The values of the PDU codes associated with the different types of PDUs are as shown in Table 791.

Table 791 – Coding of the different MCS PDU types

PDU values	Meaning
1000 0000	association establishment request
1100 0000	association establishment response
1001 0000	association termination request
1101 0000	association termination response
1011 0000	association revocation request
00X0 0XXX	associated transfer
0110 0000	associated transfer acknowledgement
00X1 0XXX	non-associated transfer
0111 0XXX	non-associated transfer acknowledgement

The bits of certain codes marked X have the values indicated in the descriptions of the associated PDU types.

NOTE All other codes are pending in this standard.

c) Variable part

This field is intended to receive optional parameters, which may be present or absent depending on the classes of conformity of the devices and the requirements of the user.

Each service parameter encoded in the variable part is structured as shown in Table 792.

Table 792 – Coding of the variable part of the PDU

Mapping	Meaning
octet 1	Parameter code
octet 2	Parameter length (length = m)
octet 2 + 1 (to)	Parameter value start (continued)
octet 2 + m	Parameter value end

Each of the parameters has a parameter code which is universal in the MCS environment, and which is therefore covered by this standard (these are defined for each PDU type implementing them).

The length of the parameter is coded in one octet. The value of the parameter is coded in as many octets as is specified in the length and the semantics of the coding is specific to each parameter (the semantics are defined at the level of each type of PDU implementing them).

d) Data

This field is transparent for MCS as it contains data specific to one of the specific application service elements projected onto MCS.

10.2.3.2.2 Association establishment request (AARQ)

AARQ structure is as shown in Table 793.

Table 793 – Structure of Association Establishment Request

Mapping	Meaning
octet 1	Size
octet 2	PDU code
octet 3	Calling AEI access point
octet 4	ditto
octet 5	ditto
octet 6	Called AEI access point
octet 7	ditto
octet 8	ditto
octet 9	Class of conformity
octet 10	Variable part
to	ditto
octet p	ditto
octet p+1	User data
to	ditto
end octet	ditto

a) Size

The minimum size value is 8 (coded: 0000 1000) when the parameters relating to this PDU are not present, and the maximum size value is reached when all the parameters are present.

(For instance, the maximum size value is 51 octets)

b) Fixed part

The following fields are present in the fixed part.

- The PDU code takes the value “1000 0000”.
- The calling AEI access point, which corresponds to a link address (of an ADAEI type) is represented by bit I/G as the high order bit in the lowest number octet.

Representation:

```

octet_3:   I / G x x x   x x x x
octet_4:   x x x x x x x x
octet_5:   S / R x x x   x x x x
    
```

- The called AEI access point which corresponds to a liaison address (of ADAEI type) is represented with the high order bit in the lowest number octet.

Representation:

```

octet_6 :   I / G x x x   x x x x
octet_7 :   x x x x x x x x
octet_8 :   S / R x x x   x x x x
    
```

NOTE The default value used for this access point when the association establishment initiator does not wish to fill it in is the following value:

```

octet_6 :   1111   1111
octet_7 :   1111   1111
octet_8 :   1111   1110
    
```

It is a link address value which is found in the generalised group address space reserved under the standard.

- The class of conformity

Representation:

octet_9: b8 b7 b6 b5 b4 b3 b2 b1

Where:

b8 = 1, if PV_RE = 2
 b7 = 1, if PH_AK = 2
 b6 = 1, if PH_RA = 2
 b5 = 1, if PH_SR = 2
 b4 = 1, if PH_CF = 2

The possible combinations of these bits shall allow for the constraints placed on the classes of conformity

c) Variable part

The following parameters shall be present in the variable part.

- Transfer rate:

Parameter code: 0000 0000

Parameter length: 4 octets (0000 0100)

Parameter value:

xxxx xxxx	:	Calling to Called
xxxx xxxx		
yyyy yyyy	:	Called to Calling
yyyy yyyy		

The value of these two rates is represented by a binary number equal to the time interval expressed in ms between successive messages.

The value 0000 0000 0000 0001 corresponds to 1msg/ms and the value 1110 1010 0110 0000 corresponds to 1msg/min.

The value 0000 0000 0000 0000 conventionally represents a null guaranteed minimum flow in one direction of transfer.

This parameter can only be present if b4 (PH_CF) of the Class of conformity parameter is set to 1.

- Anticipation factor:

Parameter code: 0000 0001

Parameter length: 2 octets (0000 0010)

Parameter value:

xxxx xxxx	:	Calling to Called
yyyy yyyy	:	Called to Calling

The anticipation factor is represented by a binary number making it possible to discriminate between 255 levels. The anticipation level of “i” means that it is possible to have a maximum of “i” ATAK_SM_RQ current in the association.

The use of value 0 is illegal.

This parameter can only be present if the b7(PH_AK) of the Class of conformity parameter is 1 .

- Application context:

Parameter code: 0000 0010
 Parameter length: 3 octets (0000 0011)
 Parameter value:
 xxxx xxxx : Specific ASE type
 yyyy yyyy : Abstract syntax type
 zzzz zzzz : Transfer syntax type

The value of these different types is specific to each of the application ASEs. The high order bit of each of these octets indicates whether it is a type defined in a standard (b8=1) or specific to an application (b8=0).

- Calling entity identification:

Parameter code: 0000 0011
 Parameter length: 6 octets (0000 0110)
 Parameter value:
 xxxx xxxx : AEI identifier
 yyyy yyyy : API identifier
 zzzz zzzz : AE qualifier
 uuuu uuuu : AP title
 uuuu uuuu
 uuuu uuuu

The first octet of the value designates the AEI identifier by means of a binary number making it possible to discriminate between 255 AEIs for an API in the context of a given AE. Value 0000 0000 is the default value used to not fill in this parameter.

The second octet designates the API identifier by means of a binary number making it possible to discriminate between 255 APIs for a given AP. The value 0000 0000 is the default value used to not fill in this parameter.

The third octet designates the AE qualifier by means of a binary number making it possible to discriminate between 255 AEs for a given AP. The value 0000 0000 is the default value used to not fill in this parameter.

The last free octets designate the AP title by means of a binary number. These three octets are set to 0000 0000 if this parameter is not adopted.

- Called entity identification:

Parameter code: 0000 0100
 Parameter length: 6 octets (0000 0110)
 Parameter value:
 xxxx xxxx : AEI identifier
 yyyy yyyy : API identifier
 zzzz zzzz : AE qualifier
 uuuu uuuu : AP title
 uuuu uuuu
 uuuu uuuu

The first octet of the value designates the AEI identifier by means of a binary number making it possible to discriminate between 255 AEIs for one API in the context of a given AE. The value 0000 0000 is the default value used to not fill in this parameter.

The second octet designates the API identifier by means of a binary number making it possible to discriminate between 255 APIs for a given AP. The value 0000 0000 is the default value used to not fill in this parameter.

The third octet designates the AE qualifier by means of a binary number making it possible to discriminate between 255 AEs for a given AP. The value 0000 0000 is the default value used to not fill in this parameter.

The last three octets designate the AP title by means of a binary number. These three octets are set to 0000 0000 if this parameter is not adopted.

- SDU size:

Parameter code: 0000 0101

Parameter length: 2 octets (0000 0010)

Parameter value:

xxxx xxxx : Calling to Called

yyyy yyyy : Called to Calling

SDU size is represented by a binary number providing for a maximum SDU size of 256 DLPDUs, i.e. 64 octets for a maximum DLPDU size of 256 octets.

Value 0 of this parameter is illegal.

This parameter may only be present if b5(PH_SR) of the Class of conformity parameter is set to 1.

- PDU size:

Parameter code: 0000 0110

Length: 1 octet (0000 0001)

Parameter value:

xxxx xxxx : SDU size

PDU size is represented by a binary number allowing a maximum PDU size of 256 octets.

This parameter can be present whatever the value of the class of conformity.

- Number of retries:

Parameter code: 0000 0111

Length: 1 octet (0000 0001)

Parameter value: 0000 xxxx

The number of retries is represented by a binary number allowing a maximum of 16 retries coded in bits xxxx.

Value 0 of this parameter is illegal.

This parameter can only be present if b6(PH_RA) of the Class of conformity parameter is set to 1.

- “Cyclic flow” option:

Parameter code: 0000 1000

Length: 0 octet

d) Data

The data carried in this PDU is coded in a manner interpretable in the context of the application.

10.2.3.2.3 Association establishment response (AARP)

AARP structure is shown in Table 794.

Table 794 – Structure of an associated establishment response

Mapping	Meaning
octet 1	Size
octet 2	PDU code
octet 3	Calling AEI access point
octet 4	ditto
octet 5	ditto
octet 6	Called AEI access point
octet 7	ditto
octet 8	ditto
octet 9	Class of conformity
octet 10	Result
octet 11	Variable part
to	ditto
octet p	ditto
octet p+1	User data
to	ditto
end octet	ditto

a) Size

The minimum size value is 9 octets (coded 0000 1001), when the parameters relating to this PDU are not present and the maximum size value is reached when all the parameters are present.

b) Fixed part

The following fields are present in this part:

- The PDU code takes the value “1100 0000”.
- The calling AEI access point, which corresponds to a link address (of ADAEI type) is represented with bit I/G as the high order bit in the lowest number octet.

Representation:

```

octet_3:    I / G x x x    x x x x
octet_4:    x  x x x      x x x x
octet_5:    S / R x x x    x x x x
    
```

- The called AEI access point which corresponds to a link address (of ADAEI type), is represented with the high order bit in the lowest number octet.

Representation:

```

octet_6:    I / G x x x    x x x x
octet_7:    x  x x x      x x x x
octet_8:    S / R x x x    x x x x
    
```

NOTE The default value used for this access point when the association establishment initiator does not wish to fill it in is the following value:

```

octet_6: 1111    1111
octet_7: 1111    1111
octet_8: 1111    1110
    
```

It is a link address value which is found in the generalised group address space reserved under the standard.

- The class of conformity

Representation:

octet_9: b8 b7 b6 b5 b4 b3 b2 b1

Where:

b8 = 1, if PV_RE = 2
 b7 = 1, if PH_AK = 1
 b6 = 1, if PH_RA = 2
 b5 = 1, if PH_SR = 2
 b4 = 1, if PH_CF = 2

The possible combinations of these bits shall allow for the constraints on the classes of conformity.

- The result subsequent to an association establishment request is coded in one octet and indicated if the response is positive or negative, in the latter case providing information on the nature and location of the fault.

The values of the level 10 octet contain the following values in the following cases:

0000 0000: (+) Positive
 0000 0001: (-) MCS provider (general)
 0000 0010: (-) Responder user
 0000 0011: (-) linked to "Cyclic rate"
 0000 0100: (-) linked to "Transfer rate"
 0000 0101: (-) linked to "Context name"
 0000 0110: (-) linked to "Called entity identification"
 0000 0111: (-) linked to "Calling entity identification"
 0000 1000: (-) linked to "Service size"
 xxxx xxxx: Spare

c) Variable part

The following parameters can be present in the variable part.

- Transfer rate:

The coding and semantics of this parameter are equivalent to those of the parameter of the same name in the association establishment request.

- Anticipation factor:

The coding and semantics of this parameter are equivalent to those of the parameter of the same name in the association establishment request PDU

- Application context:

The coding and semantics of this parameter are equivalent to those of the parameter of the same name in the association establishment request PDU (see 10.2.3.2.2.c).

- Called entity identification:

The coding and semantics of this parameter are equivalent to those of the parameter of the same name in the association establishment request PDU

- SDU size:

The coding and semantics of this parameter are equivalent to those of the parameter of the same name in the association establishment request PDU (see 10.2.3.2.2.c).

- PDU size:

The coding and semantics of this parameter are equivalent to those of the parameter of the same name in the association establishment request PDU (see 10.2.3.2.2.c).

- Number of retries:

The coding and semantics of this parameter are equivalent to those of the parameter of the same name in the association establishment request PDU (see 10.2.3.2.2.c).

d) Data

The data carried in this PDU are coded in a manner interpretable in the context of the application.

10.2.3.2.4 Association termination request (RERQ)

The RERQ structure is shown as in Table 795.

Table 795 – Structure of an association termination request

Mapping	Meaning
octet 1	Size
octet 2	PDU code
octet 3	User data
to	ditto
end octet	ditto

a) Length

The size value is 1 octet (coded 0000 0001)

b) Fixed part

The single field containing the fixed part is the PDU code which takes the value “1001 0000”.

c) Variable part

There is no variable part in this PDU.

d) Data

Data carried in this PDU has coding which is interpretable in the context of the application.

10.2.3.2.5 Association termination response (RERP)

The RERP structure is as in Table 796.

Table 796 – Structure of an association termination response

Mapping	Meaning
octet 1	Size
octet 2	PDU code
octet 3	Result
octet 4	User data
to	ditto
end octet	ditto

a) Size

The size value is 2 octets (coded: 0000 0010).

b) Fixed part

The fixed part of the header contains the following two fields:

- PDU code: “1101 0000”
- The result which is coded in one octet, indicating whether the result is positive or negative, and in the second phase providing information on the fault, to be defined later.

0000 0000: (+) Positive
 0000 0001: (-) Negative
 xxxx xxxx: Spare (Defaults)

c) Variable part

This PDU has no variable part.

d) Data

The data carried in this PDU has coding which is interpretable in the context of the application.

10.2.3.2.6 Association revocation request (ABRQ)

The ABRQ structure is as in Table 797.

Table 797 – Structure of an association revocation request

Mapping	Meaning
octet 1	Size
octet 2	PDU code
octet 3	Invocation origin
octet 4	User data
to	ditto
end octet	ditto

a) Size

The size value is 2 octets (coded: 0000 0010).

b) Fixed part

The following two fields make up the fixed part:

- PDU code taking value “1011 0000”.
- Invocation origin, coded in one octet and taking two values:

0000 0000: MCS user
 1111 1111: MCS provider
 xxxx xxxx: Other codes reserved

c) Variable part

This PDU has no variable part.

d) Data

The data carried in this PDU has coding which is interpretable in the context of the application.

10.2.3.2.7 Associated transfer request (ATRQ)

The ATRQ structure is as in Table 798.

Table 798 – Structure of an associated transfer request

Mapping	Meaning
octet 1	Size
octet 2	PDU code
octet 3	Sequence number
octet 4	ditto
octet 5	User data
to	ditto
end octet	ditto

a) Size

The size value is 3 octets (coded: 0000 0011).

b) Fixed part

The following two fields constitute the fixed part of the header:

- PDU code, taking values:

0000 0000: for unacknowledged transfers

0010 0XXX: for acknowledged transfers

where XXX is used to code the packet nature in the case of segmentation.

- The packet nature is coded as follows:

000: NIL (no segmentation)

100: STT (first packet)

101: MID (middle packet)

110: END (last packet)

111: ALL (first and last packet).

- The sequence number is represented in two octets by means of a binary number. This makes it possible to discriminate between 65536 data transfers. This number is not significant in the case of unacknowledged transfer.

c) Variable part

This PDU has no variable part.

d) Data

The data carried in this PDU have coding which is interpretable in the context of the application specific to the association.

10.2.3.2.8 Associated transfer acknowledgement (AKAT)

The AKAT structure is as in Table 799.

Table 799 – Structure of an associated transfer acknowledgement

Mapping	Meaning
octet 1	Size
octet 2	PDU code
octet 3	Sequence number
octet 4	ditto

a) Size

The size value is 3 octets (coded: 0000 0011).

b) Fixed part

The following field is contained in the fixed part:

- PDU number, taking the value “0110 0000”.
- Sequence number, attributed by the numbering entity.

c) Variable part

This PDU has no variable part.

d) Data

This PDU has no user data.

10.2.3.2.9 Non-associated transfer request (NTRQ)

The NTRQ structure is as in Table 800.

Table 800 – Structure of a non-associated transfer request

Mapping	Meaning
octet 1	Size
octet 2	PDU code
octet 3	Invocation identification
octet 4	ditto
octet 5	Variable part
to	ditto
octet p	ditto
octet p+1	User data
to	ditto
end octet	ditto

a) Size

The minimum size value is 3 octets (coded: 0000 0011) when the parameters relating to the PDU parameter are not present, and the maximum value is reached when all the parameters are present.

b) Fixed part

The following fields are present in the fixed part:

- PDU code, taking values:
 - 0001 0xxx: for unacknowledged transfers
 - 0011 0xxx: for acknowledged transfers
 where the value of xxx is used to code service priority.
- Service priority, coded in 3 bits making it possible to establish 8 priority levels.
- Invocation identification, represented in 2 octets by a binary number. This makes it possible to discriminate 65536 current simultaneous data transfers.

c) Variable part

The following parameters may be present in the variable part:

- Application context

The coding and semantics of this parameter are equivalent to those of the parameter with the same name in the association establishment request PDU (see 10.2.3.2.2).

- Calling entity identification

The coding and semantics of this parameter are equivalent to those of the parameter with the same name in the association establishment request PDU (see 10.2.3.2.2).

- Called entity identification

The coding and semantics of this parameter are equivalent to those of the parameter with the same name in the association establishment request PDU (see 10.2.3.2.2).

d) Data

The data carried in this PDU have coding which is interpretable in the context of the application.

10.2.3.2.10 Non-associated transfer acknowledgement (AKNT)

The AKNT structure is as in Table 801.

Table 801 – Structure of a non-associated transfer acknowledgement

Mapping	Meaning
octet 1	Size
octet 2	PDU code
octet 3	Invocation identification
octet 4	ditto

a) Size

The size value is 3 octets (coded: 0000 0011).

b) Fixed part

The following field is contained in the fixed part:

- PDU number, taking the value “0111 0XXX”.

The 3 bits XXX are reserved for coding a fault which can arise at the acceptor.

Fault coding is as follows:

000: AKNT+, no fault

≠000: AKNT-, fault

where:

001: non-specialised fault line

010 to 111: faults, the specialisation of which is reserved under this standard.

NOTE The acknowledgement does not carry the priority of the transfer request which gave rise to the acknowledgement as it is implicit that any requester will reserve the resources to receive the acknowledgement it awaits.

- Invocation identification is represented by 2 octets with a binary number. This makes it possible to discriminate 65536 current simultaneous data transfers.

c) Variable part

This PDU has no variable part.

d) Data

This PDU has no user data.

10.2.4 SUB MMS ASE FAL PDU type

10.2.4.1 SUB-MMS-PDU

10.2.4.1.1 Top Level Definition

This section describes the PDUs used in the SUB-MMS protocol.

SUB-MMS-General-module-1 {iso standard 9605
 part-protocol (2)
 sub-mms-general-module-version1 (5) }

DEFINITION:: = BEGIN

EXPORT SUB-MMSpdu;

-- By this affirmation we mean that any production subjected to SUB-MMSpdu is exportable.--

```
SUB-MMSpdu :: =
  confirmed-reqPDU      [0] IMPLICIT ConfirmedReqPDU,
  confirmed-respPDU     [1] IMPLICIT ConfirmedRespPDU,
  confirmed-errorPDU    [2] IMPLICIT ConfirmedErrorPDU,
  unconfirmedPDU        [3] IMPLICIT UnconfirmedPDU,
  rejectPDU             [4] IMPLICIT RejectPDU,
  initiate-reqPDU       [5] IMPLICIT InitiateReqPDU,
  initiate-respPDU      [6] IMPLICIT InitiateRespPDU,
  initiate-errorPDU     [7] IMPLICIT InitiateErrorPDU,
  conclude-reqPDU       [8] IMPLICIT ConcludeReqPDU,
  conclude-respPDU      [9] IMPLICIT ConcludeRespPDU,
  conclude-errorPDU     [10] IMPLICIT ConcludeErrorPDU
}
```

The SUB-MMSpdu protocol defines 11 types of PDU

The companion Standards can increase this list.

10.2.4.1.2 Definition of confirmed-req-PDU

```
ConfirmedReqPDU:: =
  invoke-ID             UNSIGNED-32,
  confirmed-service-req  ConfirmedServiceReq}.
```

The confirmed-req-pdu includes two parameters:

- invoke-ID has the same significance as that given in the ISO/IEC 9506-2 standard.
- ConfirmedServiceReq is defined below.

10.2.4.1.3 Definition of Unconfirmed-PDU

```
UnconfirmedPDU:: =
  invoke-ID             UNSIGNED-32,
  unconfirmed-service   UnconfirmedService}.
```

The unconfirmed-pdu includes two parameters:

- invoke-ID has the same significance as that given by the ISO/IEC 9506-2 standard.

UnconfirmedService is defined below.

10.2.4.1.4 Definition of confirmed-resp-pdu

```
ConfirmedRespPDU:: =
  invoke-ID             UNSIGNED-32,
  confirmed-service-resp ConfirmedServiceResp}.
```

The confirmed RespPDU includes two parameters:

The invoke-ID has the same significance as that given by the ISO 9506-2 standard.

The ConfirmedServiceResp is defined below.

10.2.4.1.5 Definition of confirmed-error-pdu

```
ConfirmedErrorPDU:: =
  invoke-ID             UNSIGNED-32,
  service-error         ErrorType}.
```

The ConfirmedErrorPDU includes two parameters:

- a) invoke-ID has the same significance as that given by the ISO 9506-2 standard.

Error is defined below.

10.2.4.1.6 Definition of "object" and "service"

10.2.4.1.6.1 Definition of a class of objects

A class of objects is a qualifying expression attributed to a type of set of objects to express the fact that the same action applied to these objects always gives the same result. We attribute a name to each class of objects. Consequently a class of objects shall be identified by its name. The CSs can increase this class of objects. See Table 802.

Table 802 – Definitions of object classes

Name of class	Type of object	Creation
vmd	visible part of the server	I/T
dom	domain	I/T/E
pi	executable program	I/T/E
var	variable	I/T
var-list	variables-list	I/T/E
event	event	I/T
association	connection	I/T/E
OD-header	OD-header	I/T
data type	type descriptor	I/T
LEGEND		
I	Implicit creation	
T	Creation following a remote loading	
E	Explicit Creation by Service	

10.2.4.1.6.2 Definition of a service

A type of action applied to an object belonging to a given class is called service. We attach a name to each of the services. Consequently, a service will be identified by its name. The CSs can increase this list of services. See Table 803.

Table 803 – Definition of Sub-MMS Services

Name of service		Object class	Note
1- confirmed services:	Status	vmd	note 1
	Identify	vmd	note 1
	Get Name List	vmd	note 1
	Delete-domain	dom	note 1
	Initiate-download-sequence	dom	note 1
	Download-segment	dom	note 1
	Terminate-download-sequence	dom	note 1
	Initiate-upload-sequence	dom	note 1
	Upload-segment	dom	note 1
	Terminate-upload-sequence	dom	note 1
	Get Domain Attributes	dom	note 1
	Create-program-invocation	pi	note 1
	Delete-program-invocation	pi	note 1
	Start	pi	note 1
	Stop	pi	note 1
	Resume	pi	note 1
	Reset	pi	note 1
	Kill	pi	note 1
	Get Program Invocation Attributes	pi	note 1
	Read	var/var-list	note 1
	Write	var/var-list	note 1
	Get Variable Access Attributes	var	note 1
	Define-variable-list	var-list	note 1
	Delete-variable-list	var-list	note 1
	Get Variable List Attributes	var-list	note 1
	Get-alarm-summary	event	note 1
	Acknowledge-event-notification	event	note 1
	Alter-event-condition-monitoring	event	note 1
	Get Event Condition Attributes	event	note 1
	Initiate	association	note 2
	Conclude	association	note 1
	Generic-Init-Download	OD	note 1
	Generic-Download	OD	note 1
	Generic-terminate-Download	OD	note 1
	Get OD Header/Data Type Attributes	OD	note 1
2-Unconfirmed services:	Unsolicited-Status	Vmd	note 1
	Information-Report	Var	note 1
	Event-Notification	Event	note 1
	Abort	association	note 2
	Reject	PDU	note 3
NOTE 1 service negotiable via the Initiate service.			
NOTE 2 service compulsory if the equipment supports the negotiable connection.			
NOTE 3 service compulsory for all the equipment supporting the indications of confirmed services (rejection of confirmed services requests).			

10.2.4.1.7 Definition of confirmed-service-req

The structure of a service request includes:

- 1) the service label ([X] = service code),
- 2) the service parameters (called "req-serv"),
- 3) the parameter extensions ensured by the CSs (called "req-detail").

For reasons of convenience parameters 2 and 3 are described apart.

NOTE The requests and responses of confirmed services are identified by the same service code. The requests of unconfirmed services are identified by service codes which are distinct from those of the confirmed services.

```
ConfirmedServiceReq:: =
--DOMAIN services--
  initiate-download-sequence
    req-serv
    req-detail
  download-segment
    req-serv
    req-detail
  terminate-download-sequence
    req-serv
    req-detail
  initiate-upload-sequence
    req-serv
    req-detail
  upload-segment
    req-serv
    req-detail
  terminate-upload-sequence
    req-serv
    req-detail
  delete-domain
    req-serv
    req-detail
--VMD services
  status
    req-serv
    req-detail
  identify
    req-serv
    req-detail
--PROGRAM INVOCATION services--
  create-program-invocation
    req-serv
    req-detail
  delete-program-invocation
    req-serv
    req-detail
  start
    req-serv
    req-detail
  stop
    req-serv
    req-detail

  resume
    req-serv
    req-detail
  reset
    req-serv
    req-detail
  kill
    req-serv
    req-detail
--VARIABLE /VARIABLE-LIST services--
  read
    req-serv
    req-detail
  write
    req-serv
    req-detail
  define-variable-list
    req-serv
    req-detail
  delete-variable-list
    req-serv
    req-detail
--EVENT services--
  get-alarm-summary
    req-serv
    req-detail
  acknowledge event notification
    req-serv
    req-detail
  alter-event-cond-monitor
    req-serv
    req-detail
```

```
CHOICE{
[6] IMPLICIT SEQUENCE{
  [0] IMPLICIT InitiateDownloadSequenceRequest,
  [1] IMPLICIT CSInitiateDownloadSequenceRequest},
[7] IMPLICIT SEQUENCE{
  [0] DownloadSegmentRequest,
  [1] IMPLICIT CSDownloadSegmentRequest},
[8] IMPLICIT SEQUENCE{
  [0] IMPLICIT TerminateDownloadSequenceRequest,
  [1] IMPLICIT CSTerminateDownloadSequenceRequest},
[9] IMPLICIT SEQUENCE{
  [0] InitiateUploadSequenceRequest,
  [1] IMPLICIT CSInitiateUploadSequenceRequest},
[10] IMPLICITSEQUENCE{
  [0] UploadSegmentRequest,
  [1] IMPLICIT CSUploadSegmentRequest},
[11] IMPLICITSEQUENCE{
  [0] TerminateUploadSequenceRequest,
  [1] IMPLICIT CSTerminateUploadSequenceRequest},
[12] IMPLICITSEQUENCE{
  [0] DeleteDomainRequest,
  [1] IMPLICIT CSDeleteDomainRequest},

[15] IMPLICITSEQUENCE{
  [0] IMPLICIT StatusRequest,
  [1] IMPLICIT CSStatusRequest},
[16] IMPLICIT SEQUENCE{
  [0] IMPLICIT IdentifyRequest,
  [1] IMPLICIT CSIdentifyRequest},

[20] IMPLICITSEQUENCE{
  [0] IMPLICIT CreateProgramInvocationRequest,
  [1] IMPLICIT CSCreateProgramInvocationRequest},
[21] IMPLICITSEQUENCE{
  [0] DeleteProgramInvocationRequest,
  [1] IMPLICIT CSDeleteProgramInvocationRequest},
[22] IMPLICITSEQUENCE{
  [0] IMPLICIT StartRequest,
  [1] IMPLICIT CSStartRequest},
[23] IMPLICITSEQUENCE{
  [0] StopRequest,
  [1] IMPLICIT CSStopRequest},

[24] IMPLICITSEQUENCE{
  [0] IMPLICIT ResumeRequest,
  [1] IMPLICIT CSResumeRequest},
[25] IMPLICITSEQUENCE{
  [0] ResetRequest,
  [1] IMPLICIT CSResetRequest},
[26] IMPLICITSEQUENCE{
  [0] KillRequest,
  [1] IMPLICIT CSKillRequest},

[30] IMPLICITSEQUENCE{
  [0] ReadRequest,
  [1] IMPLICIT CSReadRequest},
[31] IMPLICITSEQUENCE{
  [0] IMPLICIT WriteRequest,
  [1] IMPLICIT CSWriteRequest},
[35] IMPLICITSEQUENCE{
  [0] IMPLICIT DefineVariableListRequest,
  [1] IMPLICIT CSDefineVariableListRequest},
[36] IMPLICITSEQUENCE{
  [0] DeleteVariableListRequest,
  [1] IMPLICIT CSDeleteVariableListRequest},

[39] IMPLICITSEQUENCE{
  [0] GetAlarmSummaryRequest,
  [1] IMPLICIT CSGetAlarmSummaryRequest},
[40] IMPLICITSEQUENCE{
  [0] IMPLICIT AcknowledgeEventNotificationRequest,
  [1] IMPLICIT CSAcknowledgeEventNotificationRequest},
[41] IMPLICITSEQUENCE{
  [0] IMPLICIT AlterEventConditionMonitoringRequest,
  [1] IMPLICIT CSAAlterEventConditionMonitoringRequest},
```

```

--Get Attributes services--
  get-name-list           [45] IMPLICITSEQUENCE{
    req-serv              [0] IMPLICIT GetNameListRequest,
    req-detail            [1] IMPLICIT CSGetNameListRequest},
  get-domain-attributes   [46] IMPLICITSEQUENCE{
    req-serv              [0] GetDomainAttributesRequest,
    req-detail            [1] IMPLICIT CSGetDomainAttributesRequest},
  get-program-invocation-attributes [47] IMPLICITSEQUENCE{
    req-serv              [0] GetProgramInvocationAttributesRequest,
    req-detail            [1] IMPLICIT CSGetProgramInvocationAttributesRequest},
  get-variable-access-attributes [48] IMPLICITSEQUENCE{
    req-serv              [0] GetVariableAccessAttributesRequest,
    req-detail            [1] IMPLICIT CSGetVariablesAccessAttributesRequest},
  get-variable-list-attributes [49] IMPLICITSEQUENCE{
    req-serv              [0] GetVariableListAttributesRequest,
    req-detail            [1] IMPLICIT CSGetVariableListAttributesRequest},
  get-event-condition-attributes [50] IMPLICITSEQUENCE{
    req-serv              [0] GetEventConditionAttributesRequest,
    req-detail            [1] IMPLICIT CSGetEventConditionAttributesRequest},
--OD services--
  generic-init-download    [55] IMPLICITSEQUENCE{
    req-serv              [0] IMPLICIT GenericInitDownloadRequest,
    req-detail            [1] IMPLICIT CSGenericInitDownloadRequest},
  generic-download         [56] IMPLICITSEQUENCE{
    req-serv              [0] IMPLICIT GenericDownloadRequest,
    req-detail            [1] IMPLICIT CSGenericDownloadRequest},
  generic-terminate-download [57] IMPLICITSEQUENCE{
    req-serv              [0] IMPLICIT GenericTerminateDownloadRequest,
    req-detail            [1] IMPLICIT CSGenericTerminateDownloadRequest},
  get-OD-header/datatype-attributes [58] IMPLICITSEQUENCE{
    req-serv              [0] IMPLICIT GetODHeaderDataTypeRequest,
    req-detail            [1] IMPLICIT CSGetODHeaderDataTypeRequest},
}

```

The ConfirmedServiceReq includes:

- 1) [X] identifies the service,
- 2) req-serv references the production corresponding to the selected service,
- 3) req-detail references the production of the Companion Standards (CS) corresponding to the selected service.

10.2.4.1.8 Definition of unconfirmed-service

The unconfirmed services are transmitted spontaneously. The reason for their transmission is not defined by this document (local issue).

The choice of the channel(s) via which a unconfirmed service is transmitted is also a local issue. Consequently, a notification can take place in point to point and/or in multipoint and/or in broadcasting.

The structure of the service includes:

- 1) the service label ([X] = service code),
- 2) the parameter services (called "req-serv"),
- 3) the parameter extensions by the CS (called "req-detail").

For reasons of convenience, parameters 3 and 4 are described separately.

NOTE The requests of unconfirmed services are identified by service codes distinct from those of the confirmed services.

```

UnconfirmedService ::= CHOICE{
--VMD service--
    unsolicited-status          [3] IMPLICIT SEQUENCE{
        req-serv                [0] IMPLICIT UnsolicitedStatus,
        req-detail              [1] IMPLICIT CSUnsolicitedStatus},
--EVENT service:
    event-notification          [4] IMPLICIT SEQUENCE{
        req-serv                [0] IMPLICIT EventNotification,
        req-detail              [1] IMPLICIT CSEventNotification},
--VARIABLE service--
    information-report          [5] IMPLICIT SEQUENCE{
        req-serv                [0] IMPLICIT InformationReport,
        req-detail              [1] IMPLICIT CSInformationReport},
}

```

The unconfirmedService includes:

- 1) [X] identifies the service,
- 2) req-serv reference of the production corresponding to the selected service,
- 3) req-detail references the production of the Companion Standards (CS) corresponding to the selected service.

10.2.4.1.9 Definition of confirmed-service-resp

The structure of the service includes:

- 1) the service label([X] = service code),
- 2) the service parameters (called "rsp-serv"),
- 3) the parameter extensions made by the CS (called "rsp-detail").

For reasons of convenience, parameters 2 and 3 are described elsewhere.

NOTE The requests and responses of confirmed services are identified by the same service code. The requests of unconfirmed services are identified by service codes which are distinct from those of the confirmed services.

```

ConfirmedServiceResp ::= CHOICE{
--DOMAIN services--
    initiate-download-sequence  [6] IMPLICIT SEQUENCE{
        rsp-serv                [0] IMPLICIT InitiateDownloadSequenceResponse,
        rsp-detail              [1] IMPLICIT CSInitiateDownloadSequenceResponse},
    download-segment            [7] IMPLICIT SEQUENCE{
        rsp-serv                [0] IMPLICIT DownloadSegmentResponse,
        rsp-detail              [1] IMPLICIT CSDownloadSegmentResponse},
    terminate-download-sequence [8] IMPLICIT SEQUENCE{
        rsp-serv                [0] IMPLICIT TerminateDownloadSequenceResponse,
        req-detail              [1] IMPLICIT CSTerminateDownloadSequenceResponse},
    initiate-upload-sequence    [9] IMPLICIT SEQUENCE{
        rsp-serv                [0] IMPLICIT InitiateUploadSequenceResponse,
        rsp-detail              [1] IMPLICIT CSInitiateUploadSequenceResponse},
    upload-segment              [10] IMPLICIT SEQUENCE{
        rsp-serv                [0] IMPLICIT UploadSegmentResponse,
        rsp-detail              [1] IMPLICIT CSUploadSegmentResponse},
    terminate-upload-sequence    [11] IMPLICIT SEQUENCE{
        rsp-serv                [0] IMPLICIT TerminateUploadSequenceResponse,
        rsp-detail              [1] IMPLICIT CSTerminateUploadSequenceResponse},
    delete-domain               [12] IMPLICIT SEQUENCE{
        rsp-serv                [0] IMPLICIT DeleteDomainResponse,
        rsp-detail              [1] IMPLICIT CSDeleteDomainResponse},
--VMD services
    status                      [15] IMPLICIT SEQUENCE{
        rsp-serv                [0] IMPLICIT StatusResponse,
        rsp-detail              [1] IMPLICIT CSStatusResponse},
    identify                    [16] IMPLICIT SEQUENCE{
        rsp-serv                [0] IMPLICIT IdentifyResponse,
        rsp-detail              [1] IMPLICIT CSIIdentifyResponse},
}

```

```

--PROGRAM INVOCATION services--
  create-program-invocation
    rsp-serv
    rsp-detail
  delete-program-invocation
    rsp-serv
    rsp-detail
  start
    rsp-serv
    rsp-detail
  stop
    rsp-serv
    rsp-detail
  resume
    rsp-serv
    rsp-detail
  reset
    rsp-serv
    rsp-detail
  kill
    rsp-serv
    rsp-detail
--VARIABLE/VARIABLE-LIST services--
  read
    rsp-serv
    rsp-detail
  write
    rsp-serv
    rsp-detail
  define-variable-list
    rsp-serv
    rsp-detail
  delete-variable-list
    rsp-serv
    rsp-detail
--EVENT services--
  get-alarm-summary
    rsp-serv
    rsp-detail
  acknowledge-event-notification
    rsp-serv
    rsp-detail
  alter-event-cond-monitor
    rsp-serv
    rsp-detail
--Get Attributes services--
  get-name-list
    req-serv
    req-detail
  get-domain-attributes
    req-serv
    req-detail
  get-program-invocation-attributes
    req-serv
    req-detail
  get-variable-access-attributes
    req-serv
    req-detail
  get-variable-list-attributes
    req-serv
    req-detail
  get-event-condition-attributes
    req-serv
    req-detail
--OD services--
  generic-init-download
    req-serv
    req-detail
  generic-download
    req-serv
    req-detail
  generic-terminate-download
    req-serv
    req-detail
  get-OD-header/datatype-attributes
    req-serv
    req-detail

```

[20] IMPLICITSEQUENCE{
 [0] IMPLICIT CreateProgramInvocationResponse,
 [1] IMPLICIT CSCreateProgramInvocationResponse},

[21] IMPLICITSEQUENCE{
 [0] IMPLICIT DeleteProgramInvocationResponse,
 [1] IMPLICIT CSDeleteProgramInvocationResponse},

[22] IMPLICITSEQUENCE{
 [0] IMPLICIT StartResponse,
 [1] IMPLICIT CSStartResponse},

[23] IMPLICITSEQUENCE{
 [0] IMPLICIT StopResponse,
 [1] IMPLICIT CSStopResponse},

[24] IMPLICITSEQUENCE{
 [0] IMPLICIT ResumeResponse,
 [1] IMPLICIT CSResumeResponse},

[25] IMPLICITSEQUENCE{
 [0] IMPLICIT ResetResponse,
 [1] IMPLICIT CSResetResponse},

[26] IMPLICITSEQUENCE{
 [0] IMPLICIT KillResponse,
 [1] IMPLICIT CSKillResponse},

[30] IMPLICITSEQUENCE{
 [0] IMPLICIT ReadResponse,
 [1] IMPLICIT CSReadResponse},

[31] IMPLICITSEQUENCE{
 [0] IMPLICIT WriteResponse,
 [1] IMPLICIT CSWriteResponse},

[35] IMPLICITSEQUENCE{
 [0] IMPLICIT DefineVariableListResponse,
 [1] IMPLICIT CSDefineVariableListResponse},

[36] IMPLICITSEQUENCE{
 [0] IMPLICIT DeleteVariableListResponse,
 [1] IMPLICIT CSDeleteVariableListResponse},

[39] IMPLICITSEQUENCE{
 [0] IMPLICIT GetAlarmSummaryResponse,
 [1] IMPLICIT CSGetAlarmSummaryResponse}

[40] IMPLICITSEQUENCE{
 [0] IMPLICIT AcknowledgeEventNotificationResponse,
 [1] IMPLICIT CSAcknowledgeEventNotificationResponse},

[41] IMPLICITSEQUENCE{
 [0] IMPLICIT AlterEventConditionMonitoringResponse,
 [1] IMPLICIT CSAAlterEventConditionMonitoringResponse},

[45] IMPLICITSEQUENCE{
 [0] GetNameListResponse,
 [1] IMPLICIT CSGetNameListResponse},

[46] IMPLICITSEQUENCE{
 [0] IMPLICIT GetDomainAttributesResponse,
 [1] IMPLICIT CSGetDomainAttributesResponse},

[47] IMPLICITSEQUENCE{
 [0] IMPLICIT GetProgramInvocationAttributesResponse,
 [1] IMPLICIT CSGetProgramInvocationAttributesResponse},

[48] IMPLICITSEQUENCE{
 [0] IMPLICIT GetVariableAccessAttributesResponse,
 [1] IMPLICIT CSGetVariablesAccessAttributesResponse},

[49] IMPLICITSEQUENCE{
 [0] IMPLICIT GetVariableListAttributesResponse,
 [1] IMPLICIT CSGetVariableListAttributesResponse},

[50] IMPLICITSEQUENCE{
 [0] IMPLICIT GetEventConditionAttributesResponse,
 [1] IMPLICIT CSGetEventConditionAttributesResponse}

[55] IMPLICITSEQUENCE{
 [0] IMPLICIT GenericInitDownloadRequest,
 [1] IMPLICIT CSGenericInitDownloadRequest},

[56] IMPLICITSEQUENCE{
 [0] IMPLICIT GenericDownloadRequest,
 [1] IMPLICIT CSGenericDownloadRequest},

[57] IMPLICITSEQUENCE{
 [0] IMPLICIT GenericTerminateDownloadRequest,
 [1] IMPLICIT CSGenericTerminateDownloadRequest},

[58] IMPLICITSEQUENCE{
 [0] IMPLICIT GetODHeaderDataTypeRequest,
 [1] IMPLICIT CSGetODHeaderDataTypeRequest},

}

ConfirmedServiceResp includes three parameters:

- 1) [X] identifies the service,
- 2) rsp-serv references the production corresponding to the selected service,
- 3) rsp-detail references the production of the CS corresponding to the selected service.

10.2.4.1.10 Definition of error-type

```
ErrorType ::= CHOICE{
--Choice of a sub-set of MMS standard errors. The CSs can lengthen the MMS standard error list.--
--MMS standard errors:
    vmd [0] IMPLICIT UNSIGNED-16{
        other (0)},
    application-reference [1] IMPLICIT UNSIGNED-16{
        other (0),
        application-unreachable (1),
        -- value (2) reserved for connection lost,
        application-reference-invalid (3),
        context-unsupported (4)},
    definition [2] IMPLICIT UNSIGNED-16 {
        other (0),
        object-undefined (1),
        --value (2) reserved for invalid-address --
        type-unsupported (3),
        type-inconsistent (4),
        object exist (5),
        object-attribute-inconsistent (6)},
    resource [3] IMPLICIT UNSIGNED-16 {
        other (0),
        memory-unavailable (1),
        --value (2) reserved for processor resource unavailable,--
        --value (3) reserved for mass storage unavailable,--
        capability-unavailable (4),
        capability-unknown (5)},
    service [4] IMPLICIT UNSIGNED-16 {
        other (0),
        primitive-out-of-sequence (1),
        object-state-conflict (2)+256*valeur "pi-state",
        --value (3) reserved for further definition,--
        --value (4) reserved for continuation invalid,--
        object-constraint-conflict (5)+256*valeur "pi-state",
        service-preempt [5] IMPLICIT UNSIGNED-16 {
            other (0),
            timeout (1),
            --value (2) reserved for deadlock,--
            --value (3) reserved for cancel--},
        --choice-value [6] reserved for time resolution,--
        access [7] IMPLICIT UNSIGNED-16 {
            other (0),
            object-access-unsupported (1),
            object-non-existent (2),
            object-access-denied (3),
            object-invalidated (4)},
        initiate [8] IMPLICIT UNSIGNED-16 {
            other (0),
            --values (1) and (2) are reserved for further definition--
            max serv outstanding calling insufficient (3),
            max serv outstanding called insufficient (4),
            service CBB insufficient (5),
            parameter CBB insufficient (6),
            nesting level insufficient (7) },
        conclude [9] IMPLICIT UNSIGNED-16 {
            other (0),
            further-communication-required (1)},
        --choice value [10] reserved for cancel,--
        --choice value [11] reserved for file,--
        other [12] IMPLICIT UNSIGNED-16 {
            --values reserved for local matter--
        }
}
```

In case of failure of the Start, Stop, Resume and Reset services, the Error-Type also contains information on the "class error" and "code error" the value of "pi-state". In case of failure of the other services, the Error-Type should contain information on "class error" and "code error" and the value of the "pi-state" equals (0).

NOTE 1 [x] represents the error class; this is identical to the equivalent MMSs,

NOTE 2 (y) represents the error code; this is identical to the equivalent MMS,

NOTE 3 The authorized values of the "pi-state" are those defined

10.2.4.1.11 Definitions of the data types

Specifications of the types of data exchanged (data = content of a variable or "ErrorType") allows description of the exact composition of data, indicating the type and length.

```
TypeDataSpecification ::= CHOICE{
--The CSs and the Sub-MMS users can increase this list.--
--Each type extension shall be identified by [X] IMPLICIT "Y". The values of "X" varying from 15 to 25 are reserved
for types defined by the Companion Standards. The values of "X" varying from 26 to 127 are reserved for types
defined by the users. "Y" shall be defined using the EN 50170 - Volume 3 ASN.1 notation (see appendix A)--
--DATA-TYPE--
--constructed data type --
    array-type                                [0] IMPLICIT SEQUENCE{
        number-of-elements                    UNSIGNED-8,
        element-type                          TypeDataSpecification
    --recommended: nest=1--},
    structure-type                             [1] IMPLICIT SEQUENCE OF{
        TypeDataSpecification
    --recommended: nest=1--},
--Simple data type--
    Boolean-type                               [2] IMPLICIT UNSIGNED-8,
    --UNSIGNED-8 represents the length; the length is the number of bits used to encode the Boolean value;
    authorized length values = 8.
    integer-type                               [3] IMPLICIT UNSIGNED-8,
    --UNSIGNED-8 represents the length; length is the number of bits used to encode the integer value; authorized
    length values = multiple of 8.--
    unsigned-type                              [4] IMPLICIT UNSIGNED-8,
    --UNSIGNED-8 represents the length; length is the number of bits used to encode the unsigned value; authorized
    length values = multiples of 8.--
    bitstring-type                             [5] IMPLICIT UNSIGNED-8,
    --UNSIGNED-8 represents the length; length is the number of bits used to encode the bitstring value --
    visiblestring-type                         [6] IMPLICIT UNSIGNED-8,
    --UNSIGNED-8 represents the length; length is the number of octets used to encode the visiblestring value.--
    octetstring-type                          [7] IMPLICIT UNSIGNED-8,
    --UNSIGNED-8 represents the length; length is the number of octets used to encode the octetstring value.--
    time-difference                           [8] IMPLICIT UNSIGNED-8,
    --UNSIGNED-8 represents the length; length is the number of octets used to encode the time-difference value;
    authorized length value = 4.--
    time-of-day-type                          [9] IMPLICIT UNSIGNED-8,
    --UNSIGNED-8 represents the length; length is the number of octets used to encode the time of day value;
    authorized length value = 6.--
    single-floating-type                       [10] IMPLICIT UNSIGNED-8,
    --UNSIGNED-8 represents the length; length is the number of octets used to encode the single floating value;
    authorized length value= 4.--
    double-floating-type                      [11] IMPLICIT UNSIGNED-8,
    --UNSIGNED-8 represents the length; length is the number of octets used to encode the double floating value;
    authorized length value = 8 --
    bcd-type                                  [12] IMPLICIT UNSIGNED-8,
    --UNSIGNED-8 represents the length; length is the number of digits--
    Boolean-array-type                        [13] IMPLICIT UNSIGNED-8,
    --UNSIGNED-8 represents the length; length is the number of bits used to encode the value of the Boolean array.--
--ERROR-TYPE--
    error-type                                [14] IMPLICIT UNSIGNED-8
    --UNSIGNED-8 represents the length; length is the number of octets used to encode the error value; authorized
    length value = 3.--
}
```

10.2.4.1.12 Definitions of the data values (without length or type)

- The CSs and the Sub-MMS users can increase this list of Data Values by using the ASN.1 notation.
- The following values, resulting from encoding of the types of ASN.1 data and completed by other types of data judged useful, correspond to data whose type and length are either implicitly known or specified by TypeDataSpecification, or explicitly specified in a service. The type and length not included in the encoding, however the value subsists. In the framework of variable management and events, the number of octets used for encoding the value shall correspond with that specified by TypeDataSpecification. These data are used for example in the Read, Write, Information Report services, etc.--

DATA VALUE::= CHOICE{
 --Structured data values--
 Array::= OCTETSTRING SIZE (n)
 --OCTETSTRING-SIZE (n) is composed of the juxtaposition of the simple data values composing it--
 Structure::= OCTETSTRING SIZE (n)
 --OCTETSTRING-SIZE (n) is composed of the juxtaposition of the simple data values composing it--
 --Simple data values--
 BOOLEAN::= BOOLEAN
 --the encoding is ensured on 1 octet; binary value 00H signifies "false" and binary value ≠ 00H signifies "true"
 UNSIGNED-n::= Integer(0.. 2**n - 1)
 --"n" represents the number of bits used; "n" is a multiple of 8--
 --the encoding is done on a series of M octets with M=n/8 the first octet is called "LSB octet", the last octet is called "MSM octet". The binary value is between: 0 and 2**n-1. The MSB of the binary number is aligned with bit 8 of the LSB of the octet string and so on--
 INTEGER-n::= Integer(-2**(n - 1)... +2**(n-1)-1)
 --"n" represents the number of bits used; "n" is a multiple of 8--
 --the value of an integer is encoded on a string of M octets with M = n/8 the first octet is called "LSB octet", the last octet is called "MSM octet". The binary value complement of 2 is between: 0: -2**(n-1) and + 2**(n-1)-1. The sign of the integer is aligned with bit 8 of the LSB octet of the octet string. The MSB is aligned with bit 7 of the first octet and so on--
 BITSTRING-n::= BITSTRING SIZE(n)
 --"n" represents the number of bits used; "n" is a whole number equal to 0, 1,--
 --the encoding is done on a series of M octets with M=n/8; the first octet is called "LSB octet," the last bit is called "MSB octet". Bit No. 0 is aligned with bit 8 of the LSB octet of the octet string and so on--
 TimeOfDay::= OCTET STRING SIZE(6)
 --the encoding is done on a series of 6 octets. The value contains two items of juxtaposed information, the first indicating the number of days since 1984 encoded in UNSIGNED-16, the second indicating the number of milliseconds since midnight, encoded in UNSIGNED-32; the high order octet of the UNSIGNED-16 is aligned with the low order octet of the string of octets. The high order octet of the UNSIGNED-32 is aligned with the LSB+2 octet of the string of octets--
 TimeDifference::= OCTET STRING SIZE(4)
 --the encoding is done on a string of 4 octets. The value indicates the number of milliseconds since midnight encoded in UNSIGNED-32; the MSB octet of the UNSIGNED-32 is aligned with the LSB of the string of octets--
 BCD-n::= OCTET STRING SIZE(M)
 --"n" is the number of bits used; "n" is a whole number equal to 0, 1, 2--
 --the encoding is done on a series of M octets with M=n/4. The value represents "n" digits. Each digit is encoded in binary number on a quartet; the first digit(most significant) is aligned with the higher order octet of the higher order octet. The unused quartet, if any, shall be the lower order quartet of the higher order octet. The values permitted for a quartet are 0 to 9. The MSB of the quartet is aligned as required with bit 8 or 4.
 BooleanArray-n::= BITSTRING SIZE(n)
 --"n" is the number of bits used; "n" is a whole number equal to 0, 1, 2--
 --The encoding of the Boolean Array is identical to that of the BITSTRING; it differs only by the naming of the bits. The bits are named according to the following rule: bit No. 0 is aligned with bit 2**0 of the 1st octet, bit No. 7 is aligned with bit 2**7 of the 1st octet, bit No. 8 is aligned with bit 2**0 of the 2nd octet, bit No. 15 is aligned with bit 2**7 of the 2nd octet, etc.--
 SingleFloating::= OCTET STRING SIZE(4)
 --the encoding is done on a series of 4 octets according to the single precision format of IEEE 754 --
 DoubleFloating::= OCTET STRING SIZE(8)
 --"n" is the number of bits used; "n" is a whole number equal to 0, 1, 2, 3--
 --the encoding is done on a series of 8 octets according to the double precision format of IEEE 754--
 OCTETSTRING-n::= OCTET STRING SIZE(n)
 --"n" is the number of bits used; "n" is a whole number equal to 0, 1, 2--
 --the encoding is done on a string of "n" octets--
 VisibleString-n::= OCTET STRING SIZE(n)
 --the encoding is identical to that of the OCTETSTRING-n with however the following restriction: each octet represent a visible character to be chosen from the standard MMS characters: "A" to "Z", "a" to "z", "\$", "_", "0" to "9"--

10.2.4.1.13 Complementary definitions

```

Identification ::= CHOICE {
    index                UNSIGNED-16 (0 .. 32K-1).
    name                 CHOICE {
        undefined-length-name    [0] IMPLICIT OCTETSTRING SIZE(0 .. 32)
        cs-defined-length-name   [5] IMPLICIT OCTETSTRING SIZE(X)—see note
    }
}
NOTE CS defined-length-name can only be used if the Companion Standards have defined the value X ≤ 32. In
this case the length of OCTET STRING SIZE (X) is no longer encoded .

```

Discard::= OCTET STRING SIZE(0..3)
 -- in case of loading of a domain with success, its content is empty, otherwise its content is ErrorType. Discard
 is used only in "Terminate Download Sequence"--

```
ListOfAccessResult ::= BIT STRING
--each bit fills the result of the access at one variable. A bit at 1 signifies that the access is a success.--
```

ListOfObjectId:= SEQUENCE OF Identification

ListOfObjectType ::= SEQUENCE OF TypeDataSpecification

ListOfObjectValue ::= OCTET STRING

--it is the juxtaposition of the values of the variables and/or error types encoded in compliance with "Data Value" or Error type

ListOfErrorType ::= OCTET STRING
 --contains the juxtaposition of several ErrorType values encoded in compliance with Error Type (See 10.2.4.1.10).

```

{
ObjectClass ::= UNSIGNED-8
    Null          (0),
    Domain        (1),
    PI            (2),
    Variable      (3),
    Variable-List (4),
    Event         (5),
    OD Header     (6),
    Data Type     (7).
}

```

10.2.5 VMD ASE Protocol

10.2.5.1 Environment management protocol

10.2.5.1.1 General

The association management protocol defines the PDUs necessary for execution of the following services:

- 1) Initiate,
- 2) Conclude,
- 3) Abort,
- 4) Reject.

10.2.5.1.2 Definition of the confirmed initiate service protocol

NOTE Init Detail Request and Init Detail Response are defined in the SUB-MMS-CORE, module,

```
InitiateReqPDU ::= SEQUENCE {
    quality-of-service-calling          QualityOfServiceCalling
    extension-calling                   ExtensionCalling,
    proposed-max-serv-outstanding-calling INTEGER-16,
    proposed-max-serv-outstanding-called INTEGER-16,
    proposed-data-structure-nesting     INTEGER-8,
    --recommended nest=1; shall be ≥ 1--
    init-detail-request                 [0] IMPLICIT InitDetailRequest
}
```

```

InitiateRespPDU ::= SEQUENCE {
    quality-of-service-called          QualityOfServiceCalled,
    extension-called                  ExtensionCalled,
    negotiated-max-serv-outstanding-calling    INTEGER-16,
    negotiated-max-serv-outstanding-called    INTEGER-16,
    negotiated-data-structure-nesting    INTEGER-8,
    --recommended nest=1; shall be ≥ 1--
    init-detail-response                [0] IMPLICIT InitDetailResponse
    --see 10.2.8--
}

QualityOfServiceCalling ::= BITSTRING-8 {
    quality-of-service                (0),
    -- quality of service 1 (value of the bit = 1)--
    --quality of service 2 (value of the bit = 0)--
}

ExtensionCalling ::= OCTETSTRING
    -- For quality 1 the content shall be empty--
    -- For quality 2 the content shall be AccessControl--
    -- For service quality extensions, the format of this parameter shall be specified.

QualityOfServiceCalled ::= BITSTRING-8 {
    quality-of-service                (0)
    -- quality of service 1 (value of the bit = 1): no information concerning this quality shall be included in
    ExtensionCalling.--
    --quality of service 2 (value of the bit = 0): the information concerning this quality shall be of the AccessControl
    type; this information shall be present in ExtensionCalling and be included in the first row (first two octets of
    OCTETSTRING content)--
}

ExtensionCalled : = OCTETSTRING
    }

AccessControl : = BITSTRING-16 {
    password bit 8                    (0),
    password bit 7                    (1),
    password bit 6                    (2),
    password bit 5                    (3),
    password bit 4                    (4),
    password bit 3                    (5),
    password bit 2                    (6),
    password bit 1                    (7),

    access group 8                    (8),
    access group 7                    (9),
    access group 6                    (10),
    access group 5                    (11),
    access group 4                    (12),
    access group 3                    (13),
    access group 2                    (14),
    access group 1                    (15) }

InitiateErrorPDU ::= ErrorType

ParameterCBBSupportOption ::= BITSTRING{
    --true value=parameter supported. The CSs can extend this list.--
    --types of data
    arr (array)                        (0),--bit No. 0
    str (structure)                    (1),--bit No. 1
    vlis (variable-list)                (2),--bit No. 2
    valt (partial access)                (3),--bit No. 3
}

ServiceSupportOption ::= BITSTRING {
    --A service is supported when its bit takes the true value (1). The CSs can lengthen this list by adding new
    services.
    a) the sub-set of bits No. 0 to No. 39 indicates the support of the indications of services; this sub-set shall be
    present in the BITSTRING,
    b)the sub-set of bits No. 40 to No. 79 indicates the support of the requests for services and the bits extending
    from No. 40 to No. 81 indicate respectively the support of the name for the indications of services and the
    requests for services; this sub-set is optional for the request and conditional for the response .
    The length of the bitstring can thus take the value 39, or the value 81--

```

--Bits No. 0 to 32 are mandatory--vmd management services supported

status.ind (0), --bit No. 0
 unsolicited status.ind (unconfirmed) (1), --bit No. 1
 identify.ind (2), --bit No. 2

-- domain management services supported

delete domain.ind (3), --bit No. 3
 initiate download sequence.ind (4), --bit No. 4
 download segment.ind (5), --bit No. 5
 terminate download sequence.ind (6), --bit No. 6
 initiate upload sequence.ind (7), --bit No. 7
 upload segment.ind (8), --bit No. 8
 terminate upload sequence.ind (9), --bit No. 9

-- program invocations management services supported

create program invocation.ind (10), --bit No. 10
 delete program invocation.ind (11), --bit No. 11
 start.ind (12), --bit No. 12
 stop.ind (13), --bit No. 13
 resume.ind (14), --bit No. 14
 reset.ind (15), --bit No. 15
 kill.ind (16), --bit No. 16

-- variable management services supported

read.ind (17), --bit No. 17
 write.ind (18), --bit No. 18
 information report.ind (unconfirmed) (19), --bit No. 19
 define variable-list.ind (20), --bit No. 20
 delete variable-list.ind (21), --bit No. 21

-- event management services supported

event notification.ind (unconfirmed) (22), --bit No. 22
 alter event condition monitoring.ind (23), --bit No. 23
 acknowledge event notification.ind (24), --bit No. 24
 get alarm summary.ind (25), --bit No. 25

-- context management services supported

conclude.ind (26), --bit No. 26

-- attributes management services supported

get name list.ind (27), --bit No. 27
 get domain attributes.ind (28), --bit No. 28
 get program invocation attributes.ind (29), --bit No. 29
 get variable access attributes.ind (30), --bit No. 30
 get variable-list attributes.ind (31), --bit No. 31
 get event condition attributes.ind (32), --bit No. 32

-- OD services

generic-init-download.ind (33), --bit No. 33
 generic-download.ind (34), --bit No. 34
 generic-terminate-download.ind (35), --bit No. 35
 get OD Header/Data Type.ind (36), --bit No. 36

--Bits No. 37 to 39 are reserved for extensions of services

--The set of bits from 40 to 80 is optional for the request and conditional for the response

--vmd management services supported

status.req (40), --bit No. 40
 unsolicited status.req (unconfirmed) (41), --bit No. 41
 identify.req (42), --bit No. 42

-- domain management services supported

delete domain.req (43), --bit No. 43
 initiate download sequence.req (44), --bit No. 44
 download segment.req (45), --bit No. 45
 terminate download sequence.req (46), --bit No. 46
 initiate upload sequence.req (47), --bit No. 47
 upload segment.req (48), --bit No. 48
 terminate upload sequence.req (49), --bit No. 49

-- program invocation management services supported

create program invocation.req (50), --bit No. 50
 delete program invocation.req (51), --bit No. 51
 start.req (52), --bit No. 52
 stop.req (53), --bit No. 53
 resume.req (54), --bit No. 54
 reset.req (55), --bit No. 55
 kill.req (56), --bit No. 56

-- variable management services supported

read.req (57), --bit No. 57
 write.req (58), --bit No. 58
 information report.req (unconfirmed) (59), --bit No. 59
 define variable-list.req (60), --bit No. 60
 delete variable-list.req (61), --bit No. 61

```

-- event management services supported
  event notification.req (unconfirmed) (62), --bit No. 62
  alter event condition monitoring.req (63), --bit No. 63
  acknowledge event notification.req (64), --bit No. 64
  get alarm summary.req (65), --bit No. 65
-- context management service supported
  conclude.req (66), --bit No. 66
-- attributes management services supported
  get name list.req (67), --bit No. 67
  get domain attributes.req (68), --bit No. 68
  get program invocation attributes.req (69), --bit No. 69
  get variable access attributes.req (70), --bit No. 70
  get variable-list attributes.req (71), --bit No. 71
  get event condition attributes.req (72), --bit No. 72
-- OD services
  generic-init-download.ind (73), --bit No. 73
  generic-download.ind (74), --bit No. 74
  generic-terminate-download.ind (75), --bit No. 75
  get OD Header/Data Type.ind (76), --bit No. 76
-- Bits No. 77 to 79 are reserved for extensions of services
-- Type of identification supported
  name support for service.ind (80), --bit No. 80
  name support for service.req (81), --bit No. 81
}

```

10.2.5.1.3 Definition of confirmed conclude service protocol

ConcludeReqPDU ::= NULL.

ConcludeRespPDU ::= NULL

ConcludeErrorPDU ::= ErrorType

10.2.5.1.4 Definition of the unconfirmed abort service protocol

The Abort service is mapped directly on the A-Abort service of the lower layer ensuring the connections .

10.2.5.1.5 Definition of the unconfirmed reject service protocol

The reject service is described by Reject PDU

```

RejectPDU ::= SEQUENCE{
  original-invocation-Id          UNSIGNED-32,
  --not significant if the reject concerns Conclude--
  reject-reason                   RejectReason}

RejectReason ::= CHOICE {
  --The CSs can lengthen the following list --
  confirmed-request-PDU           [0] IMPLICIT UNSIGNED-8{
    other                          (0),
    unrecognized-service          (1),
    invalid invocation Id         (3),
    invalid-argument              (4),
    max-serv-outstanding-exceeded (6),
    max-recursion-exceeded       (8),
    value-out-of-range           (9)},
  pdu-error                       [4] IMPLICIT UNSIGNED-8{
    unknown-type-pdu             (0),
    invalid-pdu                  (1),
    illegal-mapping              (2)},
  conclude-request-PDU           [8] IMPLICIT UNSIGNED-8{
    other                         (0),
    invalid-argument             (1)},
}

```

10.2.5.2 Protocol concerning the vmd management

10.2.5.2.1 General

This section defines the protocols of the following services:

- 1) Status,
- 2) UnsolicitedStatus,
- 3) Identify
- 4) Get Name List

10.2.5.2.2 Definition of the confirmed status service protocol

– request parameters.

StatusRequest ::= BOOLEAN --true for extended derivation

– response parameters.

```
StatusResponse ::= SEQUENCE{
    vmd-logical-status          UNSIGNED-8{
        state-change-allowed    (0),
        no-state-change-allowed (1),
        others                   (2 to 5),
        --reserved for complementary states. These states shall be explained in the PICS.
        These complementary states allow ensuring the compatibility with MMS, FMS. Values 2 to 3 are reserved to
        ensure compatibility of Sub-MMS with MMS. Value 2 is reserved for the limited service permitted state. Value 3
        is reserved for the support service allowed state. Values 4 to 5 are reserved to ensure the compatibility of Sub-
        MMS with FMS. Value 4 is reserved for the state "loading of objects without consequences for the users". Value
        5 is reserved for the state "loading of objects with consequences for the users".
        (6 to n)
    },
    --reserved for the C.S
},
    vmd-physical-status          UNSIGNED-8{
        operational              (0),
        partially-operational    (1),
        inoperable               (2),
        needs-commissioning      (3) },
    local-detail                 BITSTRING SIZE( 0..128)
    --the content of BIT STRING is empty if the local detail is not used --
}
```

- 1) "State change allowed" (value 0 for UNSIGNED-8):
all the services of Sub-MMS are authorized.
- 2) "No state change allowed" (value 1 for UNSIGNED-8):
the following services are authorized.

Initiate,
Conclude,
Abort,
Status,
Identify.
Read
Get Alarm Summary
Get Name List
Get Domain Attributes
Get Program Invocation Attributes
Get Variable Access Attributes
Get Variable List Attributes
Get Event Condition Attributes

10.2.5.2.3 Definition of the unconfirmed unsolicited status service protocol

- service parameters.

UnsolicitedStatus ::= StatusResponse

10.2.5.2.4 Definition of the confirmed identify service protocol

- request parameters.

IdentifyRequest ::= NULL

- response parameters.

```
IdentifyResponse ::= SEQUENCE {
    vendor's name    VISIBLESTRING,
    model name       VISIBLESTRING,
    revision         VISIBLESTRING
}
```

10.2.5.2.5 Definition of the confirmed get name list service protocol

- request parameters.

```
GetNameListRequest ::= SEQUENCE {
    object class      OBJECTCLASS {
        --values authorized
        null           (0),
        domain         (1),
        pi             (2),
        variable        (3),
        variable-list   (4),
        event           (5),
        ODHeader        (6),
        Datatype        (7),
    }
    continue after    identification
}
```

- response parameters.

```
GetNameListResponse ::= CHOICE {
    list-of-ID        SEQUENCE {
        --selected if the object class value is different from 0
        list-of-identification ListOfObjectID,
        -- if an object supports the double identification (name and index) only the index will be transmitted--
        more follows          BOOLEAN },
    list-of-description SEQUENCE {
        --selected if the value of the object class is equal to 0
        object description     CHOICE {
            gives the description of the object
            domain-description  GetDomainAttributesResponse
            pi-description       GetProgramInvocationAttributesResponse
            variable-description GetVariableAccessAttributesResponse
            variable-list-description GetVariableListAttributesResponse
            event-description    GetEventConditionAttributesResponse
            od-header-description GetODHeader/DataTypeAttributesResponse
            data-type-description GetODHeader/DataTypeAttributesResponse
        }
        more follows          BOOLEAN
    }
}
```

10.2.6 Domain ASE protocol

10.2.6.1 General

This section defines the protocols of the following services:

- 1) DeleteDomain,
- 2) InitiateDownloadSequence,
- 3) DownloadSegment,
- 4) TerminateDownloadSequence,
- 5) InitiateUploadSequence,
- 6) UploadSegment,
- 7) TerminateUploadSequence
- 8) Get Domain Attributes

NOTE The status of a Domain can implicitly take the values: "predefined/non-existent", "ready" and "in-use".

– a domain can preexist or can be created by a Sub-MMS service.

– a preexistent domain can either come up with its total data or partial data; the rest of its data can be loaded by the Sub-MMS services.

– the Delete Domain service can be executed locally.

10.2.6.2 Definition of the confirmed delete domain service protocol

– Request parameters.

DeleteDomainRequest: = Identification.

– Response parameters.

DeleteDomainResponse::= NULL

10.2.6.3 Definition of the confirmed initiate download sequence service protocol

– Request parameters.

```
InitDownloadSequenceRequest::= SEQUENCE{
    domain-identification      Identification,
    list-of-capabilities        SEQUENCE OF OCTETSTRING
-- the content can be blank (caution: the SEQUENCE OF encoding continues)
    sharable                   BOOLEAN--true for sharable
    domain-access-protection    DomainAccessProtection
}
```

NOTE The "domain-access-protection" parameter is assigned to the Domain object created by the service. It characterizes the level of protection of the object with respect to the CLIENTS.

DomainAccessProtection::= BITSTRING{

--For a service 1 quality, the content of BITSTRING shall be empty.

For a service 2 quality of the content the BITSTRING shall correspond to the following definition--

--password--

password-bit 8	(0),
password-bit 7	(1),
password-bit 6	(2),
password-bit 5	(3),
password-bit 4	(4),
password-bit 3	(5),
password-bit 2	(6),
password-bit 1	(7),

```
--access group--
access-group 8          (8),
access-group 7          (9),
access-group 6          (10),
access-group 5          (11),
access-group 4          (12),
access-group 3          (13),
access-group 2          (14),
access-group 1          (15),

--access object protection--
--value (16) is reserved--
    Ug                  (17),
--If the value of the bit = 1 and the client belongs to one of the groups specified in the "access group" attribute,
then the domain can be attached to a PI by the Create PI service--
    Wg                  (18),
--If the value of the bit = 1 and the client belongs to one of the groups specified in the "access group" attribute,
the domain can be downloaded/deleted by the Initiate Download Sequence, Download Segment, Terminate
Download Sequence and Delete Domain services--
    Rg                  (19),
--If the value of the bit = 1 and the client belongs to one of the groups specified in the "access group" attribute,
then the domain can be uploaded by the Initiate Upload Sequence, Upload Segment and Terminate Upload
Sequence services--
--value (20) is reserved--
    U                  (21),
--If the value of the bit = 1 and the client has the required password identical to the attribute "password" then
the domain can be linked to a PI by the Create PI service--
    W                  (22),
--If the value of the bit = 1 and the client has given the required password identical to the "password" attribute,
the domain can be downloaded by the Initiate Download Sequence, Download Segment, Terminate Download
Sequence and Delete Domain services--
    R                  (23),
--If the value of the bit = 1 and the client has given the required password identical to the attribute "password",
the domain can be uploaded by the Initiate Upload Sequence, Upload Segment and Terminate Upload
Sequence services--
--The values (24), (25), (26), (27), (28) are reserved --
    Ua                  (29),
--If the value of the bit = 1, all the clients can link the domain to a PI by the Create PI service--
    Wa                  (30),
--If the value of the bit = 1, all the clients can download/delete the domain by the Initiate Download Sequence,
Download Segment, Terminate Download Sequence and Delete Domain services--
    Ra                  (31),
--If the value of the bit = 1 all the clients can then upload the domain by the Initiate Upload Sequence, Upload
Segment and Terminate Upload Sequence services-
}

```

– Response parameters.

InitDownloadSequenceResponse ::= NULL
Definition of the confirmed download segment service protocol

– Request parameters

DownloadSegmentRequest ::= Identification.

– Response parameters

DownloadSegmentResponse ::= SEQUENCE{
 load-data OCTETSTRING,
 more-follow BOOLEAN
 --If the value is true, this means that there are other segments to be downloaded--
}

Definition of the confirmed terminate download sequence service protocol

– Request parameters.

```
TermDownloadSequenceRequest ::= SEQUENCE {
    domain-Id          Identification,
    discard             Discard
    --the content of Discard is empty if the loading of the domain is successful, otherwise it contains ErrorType
    (caution: the length encoding continues as long as the content of the Discard is empty) see 10.2.4.1.11.
}
```

– Response parameters.

```
TermDownloadSequenceResponse ::= NULL
```

Definition of the confirmed initiate upload sequence service protocol

– Request parameters

```
InitUploadSequenceRequest ::= Identification.
```

– Response parameters

```
InitUploadSequenceResponse ::= SEQUENCE {
    ulsm Index          Index,
    --Index is of the UNSIGNED-16 type. if the index is significant (the value of the UNSIGNED-16 is ≤32K-1) the
    saving should be continued with the ulsm index; if, on the contrary, the index is non-significant (the value of the
    UNSIGNED-16 is 32K) the saving shall continue with the domain identifier(the Name or Index of the domain). The
    resolution of the non ambiguity between domain Index and ULSM Index is at the server's charge.—
    list-of-capabilities SEQUENCE OF OCTETSTRING }
    --the content of the list of capabilities can be empty (caution: the SEQUENCE OF encoding continues)—
```

10.2.6.4 Definition of the confirmed upload segment service protocol

– Request parameters

```
UploadSegmentRequest ::= Identification
```

--See the above constraints of the Initiate Upload Sequence service on the significance of the "Identification".

"Identification" corresponds either to the name or to the domain Index or to the ULSM Index depending on the context.--

– Response parameters

```
UploadSegmentResponse ::= SEQUENCE {
    load-data          OctetString,
    more-follow        BOOLEAN
    -- true if there are other segments to be saved
}
```

10.2.6.5 Definition of the confirmed terminate upload sequence service protocol

– Request parameters

```
TerminateUploadSequenceRequest ::= Identification
```

--See the constraints of the Initiate Upload Sequence service on the significance of "Identification".

"Identification" corresponds either to the domain name or Index or to the ULSM Index, depending on the context.--

– Response parameters

TerminateUploadSequenceResponse ::= NULL

10.2.6.6 Definition of the confirmed get domain attributes service protocol

– Request parameters

GetDomainAttributesRequest ::= Identification

– Response parameters

```
GetDomainAttributesResponse ::= SEQUENCE {
    list-of-identification      SEQUENCE OF IDENTIFICATION
    object-class                ObjectClass
                                --Permitted value "Domain"
    list-of-capabilities        SEQUENCE OF OCTETSTRING,
    --the content can be empty (caution the encoding of SEQUENCE OF continues)--
    state                       DomainState,
    predefined                  BOOLEAN, --true if predefined--
    sub-mms-sharable           BOOLEAN, --true if sharable--
    list-of-pi                  ListOfObjectId
    upload-in-progress          UNSIGNED-8,
    --number of upload in progress--
    domain-access-protection    DomainAccessProtection
    --if the object is not protected its content shall be empty--
    extension                   OCTETSTRING}

DomainState ::= UNSIGNED-8
{
    non-existent/predefined    (0),
    loading                    (1),
    ready                      (2),
    in-use                     (3),
    complete                   (4),
    incomplete                  (5)
}
```

10.2.7 Program Invocation ASE protocol

10.2.7.1 General

This section defines the protocol concerning the following services:

- 1) CreateProgramInvocation,
- 2) DeleteProgramInvocation,
- 3) Start,
- 4) Stop,
- 5) Resume,
- 6) Reset,
- 7) Kill,
- 8) Get program invocation attributes

NOTE The first 7 services can be executed locally.

10.2.7.2 Definition of the confirmed create program invocation service protocol

– Request_parameters

```
CreateProgramInvocationRequest ::= SEQUENCE{
    pi-identification                Identification,
    list-of-domains-identification   ListOfObjectID,
    reusable                         BOOLEAN,
    --true for reusable--
    pi-access-protection             PiAccessProtection
    extension                        OCTETSTRING}
```

NOTE The "pi-access-protection" parameter is assigned to the PI object created by the service. It characterizes the level of protection of the object with respect to CLIENTS.

```
PiAccessProtection ::= BITSTRING{
    --For a quality of service 1 the content of BITSTRING shall be empty
    For a quality of service 2 the content of BITSTRING shall correspond to the following definition--
    --password--
    password-bit 8                (0),
    password-bit 7                (1),
    password-bit 6                (2),
    password-bit 5                (3),
    password-bit 4                (4),
    password-bit 3                (5),
    password-bit 2                (6),
    password-bit 1                (7),
    --access_group--
    access-group 8                (8),
    access-group 7                (9),
    access-group 6                (10),
    access-group 5                (11),
    access-group 4                (12),
    access-group 3                (13),
    access-group 2                (14),
    access-group 1                (15),
    --access object protection--
    --value (16) is reserved--
    Dg                            (17),
    --If the bit value = 1 and the client belongs to one of the groups specified in the "access groups" attribute then
    he can delete the PI by the Kill, Delete-PI services--
    Hg                            (18),
    --If the value of the bit = 1 and the client belongs to one of the groups specified in the "access groups"
    attribute, then he can stop the execution of the PI by the Stop service --
    Sg                            (19),
    --If the value of the bit = 1 and the client belongs to one of the groups specified in the "access groups"
    attribute, then he can start execution of the PI by the Start, Resume and Reset services --
    --value (20) is reserved--
    D                             (21),
    --If the value of the bit = 1 and the client has provided a password identical to that of the "password" attribute
    he can delete the PI by the Kill, Delete-PI services--
    H                             (22),
    --If the value of the bit = 1 and the client has provided a password identical to that of the "password" attribute,
    then he can stop execution of the PI by the Stop service--
    S                             (23),
    --If the value of the bit = 1 and the client has provided a password identical to that of the "access group"
    attribute, he can start execution of the PI by the Start, Resume and Reset services--
    --The values (24), (25), (26), (27), (28) are reserved--
    Da                            (29),
    --If the value of the bit = 1 all the clients can delete the PI by the Kill, Delete-PI services--
    Ha                            (30),
    --If the value of the bit = 1 all the clients can stop execution of the PI by the Stop service--
    Sa                            (31),
    --If the value of the bit = 1 all the clients can start execution of the PI by the Start, Resume and Reset
    services--
}
```

– Response parameters

```
CreateProgramInvocationResponse ::= Index
```

The server can inform the client that he accepts to assign the proposed identifier to the created object by sending back in its response an UNSIGNED_16 type index with a non-significant value (Value = 32K).

The server can signify to the client that it is assigning to the created object an identifier of the index type by sending back in its response an index with a significant value (Value between 0 and 32K-1); this identifier replaces the proposed identifier if it is of the index type, otherwise the created object supports the name and the index as identifier.--

10.2.7.3 Definition of the confirmed delete program invocation service protocol

- Request parameters.

DeleteProgInvocRequest::= Identification.

- Response parameters.

DeleteProgInvocResponse::= NULL

10.2.7.4 Definition of the confirmed start service protocol

- Request parameters

StartRequest::= SEQUENCE{
 pi-identification Identification,
 execution-argument OCTETSTRING }
 --the content of the execution argument can be empty--

- Response parameters

StartResponse::= NULL

10.2.7.5 Definition of the confirmed stop service protocol

- Request parameters

StopRequest::= Identification

- Response parameters

StopResponse::= NULL

10.2.7.6 Definition of the confirmed resume service protocol

- Request parameters

ResumeRequest::= SEQUENCE{
 pi-identification Identification,
 execution-argument OCTETSTRING}
 --the content of the execution argument can be empty--

- Response parameters

Resume Response::= NULL

10.2.7.7 Definition of the confirmed reset service protocol

- Request parameters

ResetRequest::= Identification

- Response parameters

ResetResponse::= NULL

10.2.7.8 Definition of the confirmed kill service protocol

- Request parameters

KillRequest::= Identification

- Response parameters

KillResponse::= NULL

10.2.7.9 definition of the confirmed service protocol get program invocation attributes

- Request parameters

GetProgramInvocationAttributesRequest::= Identification

- Response parameters

```
GetProgramInvocationAttributesResponse::= SEQUENCE {
  list-of-identification      SEQUENCE OF Identification
  object-class                ObjectClass --Permitted value "PI"--
  pi-state                    PISState
  list-of-domain-id          ListOfObjetid
  sub-mms-deletable          BOOLEAN, --true if deletable
  reusable                    BOOLEAN, --true if reusable
  execution-argument          OCTETSTRING,
  --the content of OCTETSTRING can be empty, otherwise it should contain the last value supplied by the Start
  or Resume service executed with success--
  pi-access-protection        PIAccessProtection
  --the content should be empty if the object is not protected
  extension                   OCTETSTRING }

PISTATE::= UNSIGNED-8 {
  non-existent                (0)
  unrunnable                  (1)
  idle                        (2)
  running                     (3)
  stopped                     (4)
  starting                    (5),
  stopping                    (6)
  resuming                    (7)
  resetting                   (8)
}
```

10.2.8 Variable ASE protocol

10.2.8.1 Summary

This section defines the following service parameters:

- 1) Read (a variable-list)
- 2) Write (a variable-list)
- 3) InformationReport (a variable-list)
- 4) DefineVariableList (a variable-list),
- 5) DeleteVariableList (a variable-list),
- 6) Get Variable Access Attributes (a variable),
- 7) Get Variable List Attributes (a variable-list).

10.2.8.2 Common definitions

The following production is used by the Read (without result), Write and Information Report services--

```
VariableDescription::= CHOICE {
    variable_list [0]          IMPLICIT Identification,
    list of variables          [1] IMPLICIT SEQUENCE OF VariableAccessDescription
}
```

--The following production is used only by the Read service (with result)--

```
VariableDescriptionWithType::= CHOICE {
    variable_list [2]          IMPLICIT Identification,
    list of variables          [3] IMPLICIT SEQUENCE OF VariableAccessDescription
}
```

--The following production is used by the Read, Write and Information Report services. It allows global or partial access to variables identified by their index or their name--

```
VariableAccessDescription::= CHOICE {
--global-access-choice
    object-index          UNSIGNED-16 (0..32K-1),
    object-name           CHOICE {
        undefined-length-name [0] IMPLICIT OCTETSTRING SIZE (0..32),
        CS-fixed-length-name  [5] IMPLICIT OCTETSTRING SIZE (X)-- see note 1
    }
--partial access choice
--list-of-comp/elem-selection [(n≥1) th level nesting]
    object-index/list-of-selection [1] IMPLICIT SEQUENCE {
        object-index          UNSIGNED-16 (0..32K-1),
        list-of-selection     [0] IMPLICIT Selections
    },
    object-name/list-of-selection CHOICE {
        undefined-length-name/list-of-selection [2] IMPLICIT SEQUENCE {
            undefined-length-name OCTETSTRING SIZE (0..32),
            list-of-selection     [0] IMPLICIT Selections
        },
        cs-fixed-length-name/list-of-selection [6] IMPLICIT SEQUENCE {
            cs-fixed-length-name OCTETSTRING SIZE (X)—see note 1
            list-of-selection     [0] IMPLICIT Selection
        },
        --one-comp/elem-selection [1 st level nesting]
        object-index/one-selection [3] IMPLICIT SEQUENCE {
            object-index          UNSIGNED-16 (0..32K-1),
            one-selection         UNSIGNED-16
        },
        object-name-one-selection CHOICE {
            undefined-length-name/one-selection [4] IMPLICIT SEQUENCE {
                undefined-length-name OCTETSTRING SIZE (0..32),
                one-selection         UNSIGNED-16
            },
            cs-fixed-length-name/one-selection [7] IMPLICIT SEQUENCE {
                cs-fixed-length-name OCTETSTRING SIZE (X)—see note 1
                one-selection         UNSIGNED-16
            }
        }
    }
}
```

NOTE 1 CS-fixed-length-name can only be used if the Companion Standards have defined a value at X ($X \leq 32$). In this case, the length of OCTETSTRING SIZE (X) is no longer encoded.

```
Selections::= SEQUENCE {
    alternate access          SEQUENCE OF {--the index number shall be ≥ 0.--
        successive deeper comp/elem index UNSIGNED-16
    }
    access                   CHOICE {
        list of comp/elem index [0] IMPLICIT SEQUENCE OF UNSIGNED-16
        range of elements       [1] IMPLICIT SEQUENCE {
            start element index UNSIGNED-16
            number of elements  UNSIGNED-16
        }
    }
}
```

NOTE 2 A table is composed of several elements. Each element can be of the simple, table or structure type. Each element is identified by an index of the UNSIGNED-16 type. The first element is identified by an index with the value 0, the second element having the value 1, etc. The composition of a table is assumed to be known by all the clients.

NOTE 3 A structure is composed of several fields. Each field can be of the simple, table or structure type. Each field is identified by an index of the UNSIGNED-16 type. No rule is imposed on the values to be assigned to the field identifiers. The composition of a structure and the values of the field identifiers are assumed to be known by all the clients.

10.2.8.3 Definition of the confirmed read service protocol

– Request parameters

```
ReadRequest ::= CHOICE {
  without result variable access specification  VariableDescription ,
    --This choice indicates a variable-list to be read. It also indicates that the response shall not contain
    descriptors of the types of values read--
  with result variable access specification      VariableDescriptionWithType
    --This choice indicates a variable-list to be read. It also indicates that the response shall contain the
    descriptors of the type of variables--
}
```

– Response parameters.

-- The read procedure for a variable executed with success results in returning the variable value. An unsuccessful read procedure for a variable results in returning the ErrorType value--

```
ReadResponse ::= SEQUENCE {
  list-of-access-result      ListOfAccessResult,
    --ListOfAccessResult is of the BITSTRING type. Bit (0) is assigned to the first variable read, bit (1) to
    the second variable read and so on. For each variable read, the bit assigned to it indicates if the
    read operation has been executed successfully or has been unsuccessful. A bit set to 0 indicates
    that the read attempt has been unsuccessful--
  list-of-Type-Data-Specif   ListOfObjectType ,
    --ListOfObjectType does not contain any descriptor if the request does not require their restitution.
    In other cases, it is composed of descriptors of the read variables. If the reading of a variable is
    executed with success, the descriptor should indicate the type of the value read. On the contrary if it
    ends by a failure, the descriptor should indicate the type of the ErrorType value.
    If all the readings of variables are successfully executed, and the values read are of the same type,
    ListOfObjectType can then contain a single descriptor describing the type in question. The
    authorized types are those specified by "TypeDataSpecification" (see section 10.2.4.1.11)--
  list-of-Type-Data-value    ListOfObjectValue
    --The content of ListOfObjectValue is composed of the data of the variables read. These data are
    juxtaposed. If the reading of a variable is executed with success, the data should be of the value
    read. (see "Data Value" section 10.2.4.1.12 for the different possible values). If the reading of a
    variable ends in failure, the data shall be of the ErrorType value. (see section 10.2.4.1.10)--
}
```

10.2.8.4 Definition of the confirmed write service protocol

– Request parameters.

```
WriteRequest ::= SEQUENCE{
  variables access specification  VariableDescription,
    --VariableDescription indicates a variable class object identifier list or a variable-list class object
    identifier to be written--
  list-of-Type-Data-Specif. ListOfObjectType,
    --The content of ListOfObjectType could be without any type descriptor.
    If not, it should contain the descriptor of each of the values of the variables to be written.
    If it contains a single descriptor, this signifies that the values of the variables to be written are all of
    the type indicated by the descriptor. The authorized types are those specified by
    "TypeDataSpecification" (see section 10.2.4.1.11)--
  list-of-Type-Data-Value      ListOfObjectValue}
    --The content of ListOfObjectValue shall correspond to the values of the variables to be written. These
    values are juxtaposed. The permitted values are those specified by "Data Value" (see
    section 10.2.4.1.12)--
```

– Response parameters.

```
WriteResponse ::= SEQUENCE {
    list-of-access-result      ListOfAccessResult,
    --ListOfAccessResult is of the BITSTRING type. The bit (0) is assigned to the first written variable, the
    bit (1) is assigned to the second written variable and so on. For each written variable, the bit which is
    assigned to it indicates if the write operation has been successfully executed or that it was unsuccessful.
    A bit set to value 0 indicates that the write operation was unsuccessful--
    list-of-ErrorType          ListOfErrorType
    --ListOfErrorType contains no value if all the writing operations of the variables have been successfully
    executed. If not, it contains the juxtaposition of ErrorType values of all the writings of variables which
    ended unsuccessfully. The first value should correspond to the first failure, the second to the second
    failure and so on (for the ErrorType value See section 10.2.4.1.10)--
}
```

10.2.8.5 Definition of the unconfirmed information report service protocol

– Request parameters

```
InformationReportRequest ::= SEQUENCE{
    local detail              OCTETSTRING,
    --the content of the OCTETSTRING can be empty--
    variable access specification VariablesDescription,
    --VariableDescription indicates a list of variable class object identifiers or a variable list class object
    identifier whose values are provided in the ListOfObjectValue parameter--
    list-of-Type-Data-Specif  ListOfObjectType,
    --the content of ListOfObjectType could be empty.
    Otherwise, it is composed of the descriptors of values present in the ListOfObjectValue parameter. If it
    contains only one descriptor this signifies that all the values of the "ListOfObjectValue" parameter are of
    the same type. The authorized types are those specified by "TypeDataSpecification" with the exception
    of error type (see section 10.2.4.1.10)--
    list-of-Data-Value        ListOfObjectValue }
    --The content of ListOfObjectValue is composed of the data of variables identified by the
    "VariablesDescription" parameter. These data are juxtaposed. The authorized values are those specified
    by "Data Value" (see section 10.2.4.1.11)--
```

10.2.8.6 Definition of the confirmed define variable list service protocol

– Request parameters

```
DefineVariableListRequest ::= SEQUENCE{
    proposed-Var-list-Identification Identification,
    --the proposed identifier can be either a name or an Index.--
    list-of-var-Identification SEQUENCE OF Identification
    --Identifiers of the variables (no variable-list) to be linked to the created variable-list--
    variable-list access protection VariableListAccessProtection,
    extension OCTETSTRING
    --allows characterizing the type of user, extension, etc.--
}
```

Review:

- 1) The "variable-list access protection" parameter is assigned to the created Variable-list object. It characterizes the level of protection of the object with respect to CLIENTS.


```

VarListAccessProtection ::= BITSTRING{
    --for a service quality 1, the content of BITSTRING should be empty.
    For a service quality 2, the content of BITSTRING should correspond to the following definition--
    --password--
    password-bit 8,
    password-bit 7 (1),
    password-bit 6 (2),
    password-bit 5 (3),
    password-bit 4 (4),
    password-bit 3 (5),
    password-bit 2 (6),
    password-bit 1 (7),

    --access group--
    access-group 8 (8),
    access-group 7 (9),
    access-group 6 (10),
    access-group 5 (11),
    access-group 4 (12),
    access-group 3 (13),
    access-group 2 (14),
    access-group 1 (15),

    --access object protection--
    --value (16) is reserved--
    Dg (17),
    --If the bit value = 1 and the client has the "access group" rights required, he can delete the Variable-list by the
    Delete Variable-list service--
    Wg (18),
    --If the bit value = 1 and the client has the "access group" rights required, he can write in the variables linked to
    the variable -list by the Write service (variable-list) --
    Rg (19),
    --If the bit value = 1 and the client has the required "access group" rights, he can read the variables linked to
    the variable-list by the Read service (Variable-list)--
    --value (20) is reserved--
    D (21),
    --If the bit value = 1 and the client has the required "password" right, he can delete the variable-list by the
    Delete variable-list service--
    W (22),
    --If the bit value = 1 and the client has the required "password" rights, he can write in the variables linked to the
    variable-list by the Write service (Variable-list)--
    R (23),
    --If the bit value = 1 and the client has the required "access group" right he can read the variables linked to the
    variable-list by the Read (Variable-list) service--
    --values (24), (25), (26), (27), (28) are reserved--
    Da (29),
    --If the bit value = 1 all the clients can delete the variable list object by the Delete variable-list service--
    Wa (30),
    --If the bit value = 1, all the clients can write the variables linked to the variable-list by the Write (Variable-list)
    service--
    Ra (31)
    --If the bit value = 1 all the clients can read the variables linked to the variable-list by the Read(Variable-list)
    service--
}

```

– Response parameters

DefineVariableListResponse ::= Index

The server can indicate to the client that it accepts to assign the proposed identifier to the created object by returning in its response an index with a non-significant value (Value = 32K).

The server can indicate to the client that it is assigning an identifier of the index type to the created object by returning in its response an index with a significant value (Value between 0 and 32K-1); this identifier replaces the proposed identifier if it is of the index type, otherwise the created object supports both the name and the index as identifier--

10.2.8.7 Definition of the confirmed delete variable list service protocol

- Request parameters.

DeleteVariableListRequest ::= Identification.

"Identification" should be the identifier of the created object variable-list. The type of identification borne can be either a Name, or an Index, or a Name and an Index. See the explanations given by the DEFINE VARIABLE-LIST service.--

- Response parameters.

DeleteVariableListResponse ::= NULL

10.2.8.8 Definition of the confirmed get variable access attributes service protocol

- Request parameters.

GetVariableAccessAttributesRequest ::= Identification.

- Response parameters.

```
GetVariableAccessAttributesResponse ::= SEQUENCE {
    list-of-identification      SEQUENCE OF IDENTIFICATION
    object-class                Object-Class
                                --Permitted value "variable"--
    type data description      TypeDataSpecification,
    variableaccessprotection   VariableAccessProtection
    extension OCTETSTRING
}
```

VariableAccessProtection ::= BITSTRING{
 --the content shall be empty if the object is not protected, otherwise, it should contain the information defined below--

```
--password--
password-bit 8          (0),
password-bit 7          (1),
password-bit 6          (2),
password-bit 5          (3),
password-bit 4          (4),
password-bit 3          (5),
password-bit 2          (6),
password-bit 1          (7),
--access group--
access-group 8          (8),
access-group 7          (9),
access-group 6          (10),
access-group 5          (11),
access-group 4          (12),
access-group 3          (13),
access-group 2          (14),
access-group 1          (15),
--access object protection--
--values (16), (17) reserved--
Wg                      (18),
--If the bit value = 1 and the client belongs to one of the groups specified in the "group access" attribute, he
can write the variable by the Write service (variable) --
Rg                      (19),
--If the bit value = 1 and the client belongs to one of the groups specified in the "access group" attribute, he
can read the variable by the Read service (Variable)--
-- values (20), (21) are reserved--
W                        (22),
--If the bit value = 1 and the client has the "password" rights required, he can write the variable by the Write
service (variable) --
R                        (23),
--If the bit value = 1 and the client has the "group access" rights required, he can read the variable by the Read
service (Variable) --
--values (24), (25), (26), (27), (28), (29) are reserved --
Wa                      (30),
```

--If the bit value = 1 all the clients can write the variable by the Write service (variable) -
 Ra (31)
 --If the bit value = 1 all the clients can read the variable by the Read service (Variable)--
 extension OCTETSTRING}

10.2.8.9 Definition of the confirmed get variable list attributes service protocol

– Request parameters.

GetVariableListAttributesRequest ::= Identification.

– Response parameters.

GetVariableListAttributesResponse ::= SEQUENCE {
 list-of-identification SEQUENCE OF IDENTIFICATION
 object-class ObjectClass
 sub-mms-deletable BOOLEAN, --true if deletable
 list-of-variable ListOfObjectID,
 variable-list-access-protection VariableListAccessProtection
 --the content should be empty if the object is not protected.--
 extension OCTETSTRING }

VariableListAccessProtection ::= BITSTRING{
 --the content shall be empty if the object is not protected, otherwise, it should contain the information defined below--

--password--

password-bit 8 (0),
 password-bit 7 (1),
 password-bit 6 (2),
 password-bit 5 (3),
 password-bit 4 (4),
 password-bit 3 (5),
 password-bit 2 (6),
 password-bit 1 (7),

--access group--

access-group 8 (8),
 access-group 7 (9),
 access-group 6 (10),
 access-group 5 (11),
 access-group 4 (12),
 access-group 3 (13),
 access-group 2 (14),
 access-group 1 (15),

--access object protection--

--value (16) is reserved--

Dg (17)

--If the bit value = 1 and the client belongs to one of the groups specified in the "Access Groups" attribute, then he can delete the Variable-List object by the Delete Variable-List service--

Wg (18),

--If the bit value = 1 and the client belongs to one of the groups specified in the "Access Groups" attribute, then he can write the variable by the Write (Variable-List) service--

Rg (19),

--If the bit value = 1 and the client belongs to one of the groups specified in the "Access Groups" attribute, then he can read the variable by the Read (Variable-List) service--

-- value (20) is reserved--

D (21)

--If the bit value = 1 and the client has given a password identical to that specified in the "Password" attribute, he can delete the Variable-List object by the Delete Variable-List service--

W (22),

--If the bit value = 1 and the client has given a password identical to that specified in the "Password" attribute, he can write the variable by the Write (Variable-List) service.

R (23),

--If the bit value = 1 and the client has given a password identical to that specified in the "Password" attribute, he can read the variable by the Read (Variable-List) service--

--values (24), (25), (26), (27), (28) are reserved --

Da (29)

--If the value of the bit = 1 all the clients can delete the Variable-List object by the Delete Variable-List service--

Wa (30),

--If the bit value = 1 all the clients can write the variable by the Write service (Variable list) -

Ra (31)

--If the bit value = 1 all the clients can read the variable by the Read service (Variable list)--

}

10.2.9 Event ASE protocol

10.2.9.1 General

This section defines the protocol for the following services:

- 1) Event Notification,
- 2) Acknowledge Event Notification
- 3) Alter Event Condition Monitoring,
- 4) Get Alarm Summary,
- 5) Get Event Condition Attributes

10.2.9.2 Definition of the unconfirmed event notification service protocol

– Request parameters.

```
EventNotification ::= SEQUENCE {
    common-information          CommonInformation
    list-of-event-identification ListOfObjectId,
    list-of-event-message-description SEQUENCE OF TypeDataSpecification
```

--NOTE

--The "list-of-event-message-description" parameter gives the list of descriptors of the "EventMessage" attributes of the objects listed in the ListOfObjectId parameter. This list could be reduced to ZERO descriptor. Each TypeDataSpecification describes the EventMessage type.

Each "Event Message" is a structure having one or more fields according to the type of the event object.

According to the type of the object, the description of an "EventMessage" shall be that of a structure having:

a) either only one field:

- this field describes Event Data. The authorized types are those defined by TypeDataSpecification with an exception of error type.

b) or two fields:

- the first field describes Event Occurrence ID. The authorized type is OCTETSTRING SIZE(2).

- the second field describes Event Data. The authorized types are those defined by TypeDataSpecification excepting error type.

c) or three fields:

- the first field describes Event State. The authorized type is BITSTRING-16.

- the second field describes Event Occurrence ID. The authorized type is OCTETSTRING SIZE(2).

- the third field describes Event Data. The authorized types are those defined by TypeDataSpecification excepting error type.

If all the "EventMessage" attributes are of the same type, the list could be reduced to a single "Event Message" descriptor--

```
list-of-event-message-value          OCTETSTRING
```

--This parameter contains the juxtaposition of EventMessage values.

NOTE

Depending on the type of object, the value of an "EventMessage" comprises:

a) a single field:

this field gives the Event Data value. The permitted values are those defined by Data Value (see section 10.2.4.1.12).

b) or two fields:

- the first gives the value of Event Occurrence ID. The value authorized is OCTETSTRING-2,

- The second gives the value of Event Data. The values authorized are those defined by Data Value (see section 10.2.4.1.12),

c) or three fields:

- the first gives the value of Event State. The value permitted is BITSTRING-16,

- the second gives the value of Event Occurrence ID. The value authorized is OCTETSTRING-2,

- the third gives the value of Event Data. The values permitted are those defined by Data Value (see section 10.2.4.1.12).--

CommonInformation ::= OCTETSTRING

--The contents of OCTETSTRING is composed of the ordered juxtaposition of the following information:

1- the value of the first information is a BITSTRING-16 and represents "Event Type". This information mandatory, indicates the type of event objects of which one event is included in the notification.

2- the value of the second information is an OCTETSTRING-2 and represents "EventTransactionOccurrenceID".

3- the value of the third information, optional, is known to the user; it occupies the rest of the octets of the OCTETSTRING content.--

```

EventType ::= BITSTRING-16 {
  common event detection support (0), --supported if the value is 1.
  acknowledge action support (1), --supported if the value is 1
  enable action support (2), --supported if the value is 1
  C.S/User action support (3 to n) --to be defined by C.S/User
}

```

EventTransactionOccurrenceID ::= OCTETSTRING-2

--the semantic is a local issue; it shall be described in the PICS.--

10.2.9.3 Definition of the confirmed acknowledge event notification service protocol

– Request parameters

This service enables acknowledgement of several object events. The acknowledgement of an object is done by forcing the object attribute Ack State to the value: True..The "Ack State" attribute is a component of the "Event State" attribute.--

```

AcknowledgeEventNotificationRequest ::= SEQUENCE {
  list of event identification ListOfObjectId
  list of event occurrence ID type description SEQUENCE OF TypeDataSpecification
  --The number of TypeDataSpecification can be null when the types are implicitly known. The
  description of each TypeDataSpecification shall be that of OCTETSTRING-2. Since all the type
  descriptors are identical, it is possible to reduce the list to a single descriptor--
  list of event occurrence Id value OCTETSTRING
  --The OCTET STRING content is composed of the ordered juxtaposition of values "EventOccurrenceID".
  Each of these values is an OCTETSTRING-2--
}

```

– Response parameters

```

AcknowledgeEventNotificationResponse ::= SEQUENCE {
  list-of-access-result ListOfAccessResult,
  --ListOfAccessResult is of the BITSTRING type. Bit (0) is assigned to the first acknowledged object, bit
  (1) is assigned to the second acknowledged object and so on. For each object acknowledged, the bit
  which is assigned to it indicates if the acknowledgement operation has been successfully carried out or
  has ended in a failure. The bit set to 0 signifies that the acknowledgement is a failure.
  list-of-ErrorType ListOfErrorType
  --the content of the ListOfErrorType is empty if all the acknowledgements have been carried out
  successfully, otherwise it contains the juxtaposition of the ErrorType values of all the acknowledgements
  which have ended in a failure. The first value shall correspond to the first failure, the second value shall
  correspond to the second failure and so on. The authorized Error Type values are those defined by the
  section 10.2.4.1.10--
}

```

10.2.9.4 Definition of the confirmed alter event condition monitoring service protocol

– Request parameters

This service enables modifying the attributes of Enable State or Extension States of several event objects. These attributes are the components of the "Event State" attribute. The C.S or the user can define other actions.

The modification of an "Event State" attribute can entail a new notification of the events of the same object. The present document does not indicate how or when this new notification took place--

```
AlterEventConditionMonitoringRequest::= SEQUENCE {
    list of event state action ID      SEQUENCE OF EventStatePathAccess
    --This parameter indicates the list of Event State attributes to be modified. For each modification it
    indicates the identification of the object followed by the row of the action. See below the description of
    the EventState with the possible actions--
    list of event state description    SEQUENCE OF TypeDataSpecification
    --The number of TypeDataSpecification can be null when the types are implicitly known. The
    description of each "Event State", made by TypeDataSpecification, shall be that of BITSTRING-16. When
    all the type descriptors are identical it is possible to reduce the list to a single descriptor--
    list of event state value          OCTETSTRING
    --The content of OCTET STRING is composed of the ordered juxtaposition of the "EventState" values.
    Each Event State value is represented by a BITSTRING-16 value with the following constraints: the bits
    which do not belong to the expected action shall be forced to 0. See below the definition of the "Event
    State" actions--
}

EventStatePathAccess::= CHOICE {
    event-index/action-path-description [3] IMPLICIT SEQUENCE {
        event-index      UNSIGNED-16 (0..32K-1),
        action-rank      ActionRank
    },
    event-name/action-path-description CHOICE {
        undefined-length-name [4] IMPLICIT SEQUENCE {
            event-name      OCTETSTRING SIZE (0..32),
            action-rank      ActionRank
        },
        cs-defined-length-name [7] IMPLICIT SEQUENCE {
            event-name      OCTETSTRING SIZE (X),
            action-rank      ActionRank
        }
    }
}
```

NOTE CS-defined-length-name can only be used if the Companion Standards have defined the value $X \leq 32$. In this case the length of OCTETSTRING SIZE (X), is no longer encoded. (See encoding in appendix 10.A).

```
ActionRank::= UNSIGNED-16 {
    rank 0      (0), --reserved value
    rank 1      (1), --validation/invalidation
    rank 2 to n (2) to (n), --user action/CS
}

EventState::= BITSTRING-16 {
    acknowledgement (0), --true if the object is acknowledged
    validation       (1), --true if the object is validated
    extension-state  (2) to (p), --the user or the CS can lengthen the list of actions.
```

To this end, we recommend on one hand to break down the bits from 2 to p into n ordered sets, each including 1 or several bits and, on the other hand, to assign to each set an action of rank 2 to n--

– Response parameters

```
AlterEventConditionMonitoringResponse::= SEQUENCE {
    list-of-access-result      ListOfAccessResult,
    --ListOfAccessResult is of the BITSTRING type. Bit (0) is assigned to the first altered object, Bit (1) is
    assigned to the second altered object and so on. For each altered object, the bit which is assigned to it
    indicates if the alteration operation has been successfully carried out or if it ended in a failure. A bit
    positioned to 0 value signifies that the modification ended in a failure--
    list-of-error-type         ListOfErrorType
    --the content of ListOfErrorType is empty if all the modifications have been successfully executed,
    otherwise it contains the juxtaposition of the ErrorType values of all the modifications which have ended
    in a failure. The first value shall correspond to the first failure, the second value shall correspond to the
    second failure and so on. The authorized values for Error Type are those defined by section 10.2.4.1.10-
}
```

10.2.9.5 Definition of the confirmed get alarm summary service protocol

– Request parameters.

```
GetAlarmSummaryRequest ::= CHOICE {
  without result events access [1] IMPLICIT ListOfObjectID ,
    --This choice indicates a list of event objects whose Event_Message attribute is to be read. It also
    indicates that the response shall not contain descriptors of the read value type--
  with result events access [3] IMPLICIT ListOfObjectID
    --This choice indicates a list of object events whose Event_Message attribute is to be read. It also
    indicates that the response shall contain the descriptors of the type of values read--
}
```

– Response parameters.

-- The reading procedure for a successfully executed Event_Message attribute causes returning its value. The reading procedure of an unsuccessful Event_Message attribute causes returning of the ErrorType value--

```
GetAlarmSummaryResponse ::= SEQUENCE {
  list-of-access-result ListOfAccessResult,
    --ListOfAccessResult is of the BITSTRING type. Bit (0) is assigned to the first attribute read, bit (1) is
    assigned to the second attribute read and so on. For each attribute read, the bit which is assigned to it
    indicates if the reading operation has been carried out successfully or has ended in a failure. A bit
    positioned at 0 value signifies that the reading is a failure--
  list-of-event-message-description SEQUENCE OF TypeDataSpecification --see note 1--
  list-of-event-message-value OCTETSTRING --see note 2--
}
```

NOTE 1 This parameter contains a list of descriptors. This list could be reduced to ZERO descriptor if the request does not demand the descriptor types. Each TypeDataSpecification describes either an EventMessage type if reading operation was successful, or an Error Type if reading operations were a failure. If all the reading operations were carried out successfully and the descriptors were of the same type, the list could be reduced to only one descriptor of Event Message.

NOTE 2 Depending on the type of the object the "EventMessage" descriptor shall be that of a structure having:

- a) either only one field:
 - this field describes the Event Data type. The authorized types are those defined by TypeDataSpecification except error type.
- b) or two fields:
 - the first describes the Event Occurrence ID. The authorized type is OCTETSTRING-2,
 - the second describes the Event Data Type. The authorized types are those defined by TypeDataSpecification except error type.
- c) or three fields:
 - the first describes the Event State type. The authorized type is BITSTRING-16,
 - the second describes the type of Event Occurrence ID. The authorized type is OCTETSTRING-2,
 - the third describes the Event Data type. The authorized types are those described by TypeDataSpecification except error type.--

This parameter contains the juxtaposition of Event Message or Error Type values.

Each reading operation carried out successfully restores the Event Message value. Each reading operation ended in a failure restores Error Type value.

NOTE 3 According to the type of the object, the value of an "EventMessage" comprises:

- a) either only one field:
 - this field gives the Event Data value. The authorized values are those defined by Data Value (see 8.1.10.2).
- b) or two fields:
 - the first field gives the Event Occurrence ID value. The authorized value is OCTETSTRING-2,
 - the second gives the Event Data value. The authorized values are those defined by Data Value (see 8.1.10.2),

c) or three fields:

- the first field gives the Event State value. The authorized value is BITSTRING-16.
- the second field gives the Event Occurrence ID value. The authorized value is OCTETSTRING-2,
- the third field gives the Event Data value. The authorized values are those defined by Data Value (see 8.1.10.2).--

10.2.9.6 Definition of the confirmed get event condition attribute service protocol

– Request parameters.

GetEventConditionAttributesRequest ::= Identification

– Response parameters.

```
GetEventConditionAttributesResponse ::= SEQUENCE {
    list-of-identification      SEQUENCE OF IDENTIFICATION
    object-class                ObjectClass          --permitted value "Event"--
    event type                  EventType
    --This parameter indicates the type of object. For the definition of Event Type, see the Event Notification
    --service (section 8.7.2)--
    event-message-type-description TypeDataSpecification
    --This parameter gives the descriptor of "Event Message" of an object. The response of the "Get Alarm
    --Summary" service (section 8.7.5) explains in detail how to build a description of Event Message--

    Event-access-protection ::= EventAccessProtection
    --The content of this parameter is empty if the object is not protected--
}
```

```
EventAccessProtection ::= BITSTRING {
    --The content is empty if the object is not protected, otherwise it shall correspond to the following description.
    --password--
    password bit 8          (0),
    password bit 7          (1),
    password bit 6          (2),
    password bit 5          (3),
    password bit 4          (4),
    password bit 3          (5),
    password bit 2          (6),
    password bit 1          (7),
    --access groups--
    access group 8          (8),
    access group 7          (9),
    access group 6          (10),
    access group 5          (11),
    access group 4          (12),
    access group 3          (13),
    access group 2          (14),
    access group 1          (15),
    --bit (16) reserved
    Dg                      (17),
    --if the value is 1 and if the client has the group access rights required he can validate/invalidate and read the
    --object by means of the Alter Event Condition Monitoring and Get Alarm Summary services--
    Wg                      (18),
    --if the value is 1 and if the client has the group access rights required he can acknowledge and read the object
    --by means of Acknowledge Event Notification and Get Alarm Summary services.--
    --bits (19), (20) reserved
    D                       (21),
    --if the value is 1 and if the client provides a password identical to that specified in the "password" attribute, he
    --can validate/invalidate and read the object by means of the Alter Event Condition Monitoring and Get Alarm
    --Summary--
    W                       (22),
    --if the value is 1 and the client has the password rights required he can acknowledge and read the object by
    --means of Acknowledge Event Notification and Get Alarm Summary services.--
    --bits (23), (24), (25), (26), (27), (28) reserved
    Da                      (29),
    --if the value is 1 all the clients can validate/invalidate and read the object by means of Alter Event Condition
    --Monitoring and Get Alarm Summary services.--
    Wa                      (30),
    --if the value is 1 all the clients can acknowledge and read the object by means of Acknowledge Event
    --Notification and Get Alarm Summary services.--
    --bit (31) reserved
    extension               OCTETSTRING}
```


10.3 Structure of Protocol Machines

Interface to FAL services and protocol machines are specified in this section.

NOTE The state machines and ARPMs defined in this section only define the valid events for each. It is a local matter to handle these invalid events.

The behavior of the FAL is described by three integrated protocol machines. Specific sets of these protocol machines are defined for different AREP types. The three protocol machines are: FAL Service Protocol Machine (FSPM), the Application Relationship Protocol Machine (ARPM), and the Data Link Layer Mapping Protocol Machine (DMPM). The relationship among these protocol machines as well as primitives exchanged among them are depicted in Figure 172.

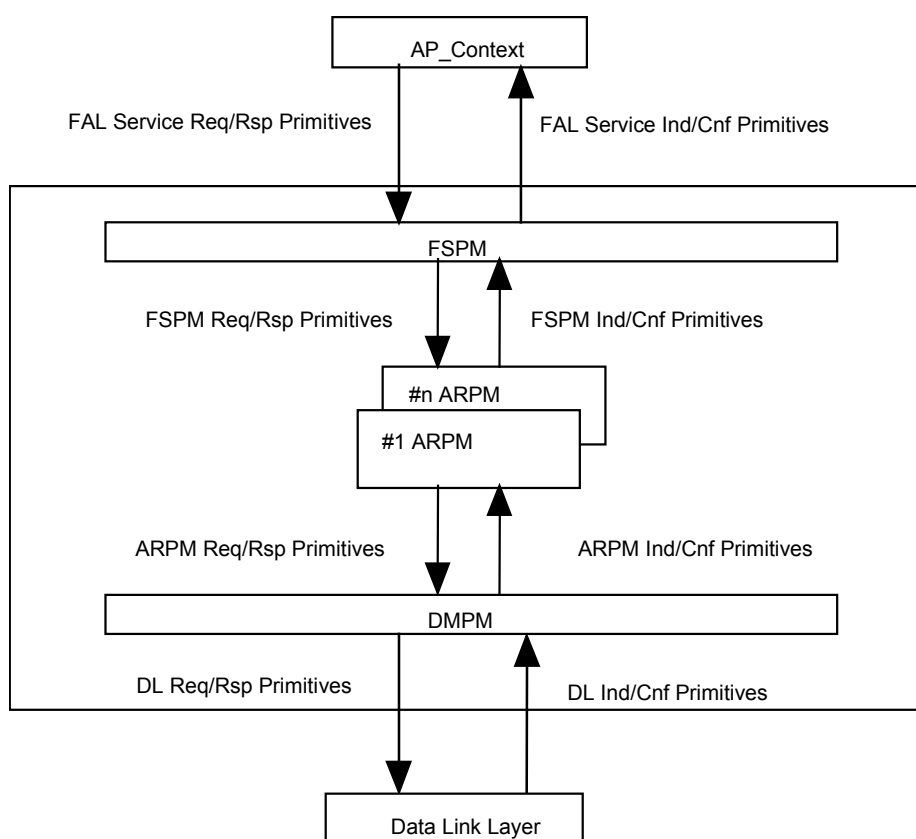


Figure 172 – Relationships among Protocol Machines and Adjacent Layers

The FSPM describes the service interface between the AP-Context and a particular AREP. The FSPM is common to all the AREP classes and does not have any state changes. The FSPM is responsible for the following activities:

- to accept service primitives from the FAL service user and convert them into FAL internal primitives;
- to select an appropriate ARPM state machine based on the AREP Identifier parameter supplied by the AP-Context and send FAL internal primitives to the selected ARPM;
- to accept FAL internal primitives from the ARPM and convert them into service primitives for the AP-Context;
- to deliver the FAL service primitives to the AP-Context based on the AREP Identifier parameter associated with the primitives.

The ARPM describes the establishment and release of an AR and exchange of FAL-PDUs with a remote ARPM(s). The ARPM is responsible for the following activities:

- a) to accept FAL internal primitives from the FSPM and create and send other FAL internal primitives to either the FSPM or the DMPM, based on the AREP and primitive types;
- b) to accept FAL internal primitives from the DMPM and send them to the FSPM as a form of FAL internal primitives;
- c) if the primitives are for the Establish or Abort service, it shall try to establish or release the specified AR.

The DMPM describes the mapping between the FAL and the DLL. It is common to all the AREP types and does not have any state changes. The DMPM is responsible for the following activities:

- a) to accept FAL internal primitives from the ARPM, prepare DLL service primitives, and send them to the DLL;
- b) to receive DLL indication or confirmation primitives from the DLL and send them to the ARPM in a form of FAL internal primitives.

For the Type-7 Application Layer protocol: the following restrictions apply:

- AP-Context State machine is not used
- FAL Service protocol state machines are not used for MPS ASE but are used globally for Sub-MMS (VMD ASE, Domain ASE; Program Invocation ASE, Variable ASE, Event ASE, Directory ASE).
- Application Relationship Protocol Machine (ARPM) and Data Link Layer Mapping Protocol Machine (DMPM) are combined together for both MCS AR ASE and MPS ASE:
- One ARPM-DMPM protocol machine for MPS ASE
- One ARPM-DMPM protocol machine for MCS AR ASE

10.4 AP-Context state machine

Feature not used

10.5 Sub-MMS FAL Service Protocol Machine (FSPM)

10.5.1 General

For the needs of the fieldbus messaging, this section imposes the MCS lower layer defined in the standard.

10.5.2 Projection of the SUB-MMS PDUs on the MCS services

SUB-MMSpdu	MCS lower layer service
Confirmed Request PDU	(A-Data or A-Unidata) with/without ack.
Confirmed Response PDU	(A-Data or A-Unidata) with/without ack.
Confirmed Error PDU	(A-Data or A-Unidata) with/without ack.
Unconfirmed Request PDU	(A-Data or A-Unidata) with/without ack.
Reject PDU	(A-Data or A-Unidata) with/without ack.
Initiate Request PDU	A-Associate.Request,
Initiate Response PDU	A-Associate.Response(result +),
Initiate Error PDU	A-Associate.Response(result -),
Conclude Request PDU	A-Release.Request,
Conclude Response PDU	A-Release.Response(result +),
Conclude Error PDU	A-Release.Response(result -).

10.5.3 Projection of the SUB-MMS abort service on the MCS services

The SUB-MMS Abort service without PDU, shall therefore be directly projected on the MCS A-Abort service; consequently the following actions will be ensured by the supplier of the SUB-MMS service:

- a) on receiving an Abort request service (generated by the Sub-MMS user) it shall generate a A-Abort request towards the lower layer,
- b) on receiving an A-Abort service indication (generated by the lower layer) it shall generate an Abort indication towards the USER.

10.5.4 Construction of a SUB-MMS-PDU from a service primitive

On receiving a service primitive (request/response) other than "Abort" the SUB-MMS service supplier shall:

- a) construct a PDU in conformity with the SUB-MMS protocol,
- b) provide the built PDU to the lower layer via an MCS service in conformity with the Sub-MMS PDU projection requirements of the Sub-MMS PDU for the MCS services.

10.5.5 Extraction of a valid service primitive from a SUB-MMS-PDU

On receiving an MCS service primitive other than A_Abort the following actions are to be carried out in the order indicated by the SUB-MMS service supplier:

- a) Check that the "user-data" parameter of the primitive contains a valid PDU (in conformity with the SUB-MMS protocol),
- b) if the received PDU is valid, extract a SUB-MMS service primitive and deliver it to the Sub-MMS user,
- c) if the received PDU is invalid and the transfer has been performed by means of an association being established (reception with success of "InitiateReqPDU" and no successful transmission of "InitiateRespPDU") then the two following actions can be carried out:
 - 1) generate an Abort indication and deliver it to the Sub-MMS,
 - 2) generate an "A-Abort" request to the MCS so that the latter frees the reserved resources. This request will not be transferred to the remote correspondent because the MCS is incapable of it (See MCS ASE).
- d) if the PDU is invalid and if the transfer of the PDU has been performed by means of a negotiated association, a predefined association or without association then IEC 61158-5:2003¹, Type 7, shall be complied with to execute the two following actions:
 - 1) construct a RejectPDU and deliver to the layer in compliance with the projection requirements of the Sub-MMS PDU for the MCS services.
 - 2) generate an indication of the Reject service and deliver it to the Sub-MMS user.

10.5.6 Negotiation of an abstract syntax and a transfer syntax commonly called presentation-context

This paragraph defines the negotiation in the following framework: the lower layer is MCS. It does not show the reductions made by the various intervenants during the negotiation. These reductions are explained in detail in a Sub-MMS Initiate service.

¹ Expected date of concurrent publication.

Review: the MCS standard authorises only one abstract syntax and only one transfer syntax per association.

Sequencing of an association opening

1- Calling user:

The Sub-MMS user wishing to open an association with a remote partner generates a primitive Initiate.Request to the Sub-MMS service supplier.

2- Sub-MMS service supplier of the calling user:

On receiving the primitive Initiate.Request initiated by the local Sub-MMS user, the Sub-MMS service supplier reserves the necessary resources, constructs an Initiate.Request PDU and generates towards the MCS layer the A_Associate.Request service with the User Data parameter = Initiate.Request PDU.

3- Sub-MMS service supplier of the called user:

On receiving an A_Associate.Indication service initiated by the MCS layer, the Sub-MMS layer carries out the following actions as required:

- a) if it notes that the User Data parameter of A_Associate service does not contain a valid Initiate.RequestPDU or if it is incapable of accepting the opening of an association for various reasons, it generates a primitive A_Associate.Response to the MCS layer with the User Data = Empty and Result = negative parameters because of the user (the user is the Sub-MMS service supplier).
- b) if it notes that the User Data parameter of the A_Associate service contains a valid Initiate.Request PDU and accepts the opening of an association, then it reserves the necessary resources and generates the primitive Initiate.Indication to the Sub-MMS user.

4- Called user:

The Sub-MMS user can either accept or refuse the association.

1- Acceptance of the association:

Following the reception of the primitive Initiate.Indication the Sub-MMS user generates towards the Sub-MMS service supplier a primitive Initiate.Response(+) and arranges for the future exchange of messages via this communication channel.

2- Refusal of the association:

Following the reception of the primitive Initiate.Indication the Sub-MMS user generates towards the Sub-MMS service supplier a primitive Initiate.Response(-) and it considers the proposed association inexistent.

5- Sub-MMS service supplier of the called user:

The behaviour of the layer varies according to the Sub-MMS user's response.

- 1- On receiving a primitive Initiate.Response(+), the Sub-MMS service supplier builds an Initiate.Response PDU and generates towards the MCS layer the A_Associate.Response service with the User Data = Initiate.ResponsePDU and Results = positive parameters and arranges for the future exchanges of messages via this communication channel.
- 2- On receiving a primitive Initiate.Response(-), the Sub-MMS service supplier frees the reserved resources, builds an Initiate.Error PDU and generates towards the MCS layer the A_Associate.Response service with the User Data = Initiate.ErrorPDU and Result = negative parameters because of the user and it considers the proposed association inexistent.

6- Sub-MMS service supplier of the calling user:

The behaviour of the Sub-MMS service supplier varies according to the response received.

- 1- On receiving an A_Associate.Confirm(+) primitive, the Sub-MMS service supplier carries out the following actions as required:
 - a) if it notes that the User Data parameter of the A_Associate service does not contain a valid Initiate.ResponsePDU, it ignores the service.
 - b) if it notes that the A_Associate service user data parameter contains a valid Initiate.Response PDU, it generates the Initiate.Confirm(+)primitive towards the Sub-MMS user and makes arrangements for the future message exchanges via this communication channel.
- 2- On receiving an A_Associate.Confirm(-) primitive, the Sub-MMS service supplier carries out the following actions as required:
 - a) if it notes that the A_Associate service user data parameter does not contain a valid Initiate.ErrorPDU, it ignores the service.
 - b) if it notes that the A_Associate service user data parameter contains a valid Initiate.ErrorPDU it generates the Initiate.Confirm(-)primitive towards the Sub-MMS user, frees the reserved resources and considers the proposed association inexistent.

7- Called user:

The behaviour of the Sub-MMS user varies according to the response received.

- 1- Positive response:
Following the reception of the Initiate.Confirm(+) primitive the Sub-MMS user takes his dispositions for the future exchanges of messages via this communication channel.
- 2- Negative response:
Following reception of the Initiate.Confirm(-)primitive the Sub-MMS user considers the proposed association inexistent.

NOTE 1 If is necessary to check the state of a negotiated or predefined association in the absence of traffic the Sub-MMS service supplier can periodically generate an MCS A_Data request service with the parameter User data = empty. The implementation of this mechanism of the client is a local issue.

NOTE 2 If is necessary to check the state of a negotiated or predefined association in the absence of traffic the Sub-MMS service supplier can demand the periodic reception of a Sub-MMS A_Data service request with the parameter User data = empty.

10.5.7 Identification of the SUB-MMS core abstract syntax

- 1- The abstract syntax of the CORE-SUB-MMS module is identified by an object-identifier and an object-descriptor. This identification characterises the abstract syntax of the Sub-MMS core.

The present document assigns the following values to them:

FIELD BUS ASN.1 object-identifier value:
{iso standard 9506 part-protocol (2)
SUB-MMS-abstract-syntax-version1 (4) }

FIELD BUS ASN.1 object-descriptor value:
"sub-mms-abstract-syntax-major-version1".

- 2- The SUB-MMS-General-module-1 is identified by an object identifier and an object-descriptor. This identification characterizes the type of ASE Sub-MMS because it enables generating different abstract syntaxes.

This document assigns to them the following values.

FIELD BUS ASN.1 object-identifier value:
 {iso standard 9506 part_protocol (2)
 sub-mms-general-module-version1 (5) }
 FIELD BUS ASN.1 object-descriptor value:
 "sub-mms-general-module-major-version1"

3-The transfer syntax relative to the abstract syntax of the CORE-SUB-MMS and the CS is identified by an object-identifier and an object-descriptor:

FIELD BUS ASN.1 object-identifier value:
 {iso standard 8825 fieldbus-encoding (5) }.
 FIELD BUS ASN.1 object-descriptor value:
 "fieldbus-encoding-rules",
 The major version of this document is 1.
 The minor version of this document is 1.

This document permits other transfer syntaxes.

10.5.8 Identification of the application context name

For the specific use of the application layer, this document proposes the application layer containing "CORE-SUB-MMS" as ASE identified by an object-identifier and an object-descriptor.

This document assigns them the following values.

FIELD BUS ASN.1 object-identifier value:
 {iso standard 9506 part-protocol (2)
 sub-mms-application-context-version1 (6)}
 FIELD BUS ASN.1 object-descriptor value:
 "iso sub-mms" .

10.5.9 Identification of the ASE of the core abstract syntax and the transfer syntax

This paragraph defines the following equivalences on behalf of MCS:

1- For the following object-identifier:

FIELD BUS ASN.1 object-identifier value (Sub-MMS general model):
 {iso standard 9506 part_protocol (2)
 sub-mms-general-module-version1 (5) }

we assign the binary value {0} for the "TYPE of ASE" parameter of the A_Associate service of MCS.

2-For the following object-identifier (Sub-MMS core):

FIELD BUS ASN.1 object-identifier value:
 {iso standard 9506 part-protocol (2)
 SUB-MMS-abstract-syntax-version1 (4) }

we assign the binary value {0} for the "ABSTRACT SYNTAX" parameter of the A_Associate service of MCS.

3-For the following object-identifier (Sub-MMS encoding rule):

FIELD BUS ASN.1 object-identifier value:
 {iso standard 8825 fieldbus-encoding (5) }.

we assign the binary value {0} for the "TRANSFERT SYNTAX" parameter of the A_Associate service.

10.6 DLL Mapping Protocol Machine (DMPM) and Association Relationship Protocol Machine (ARPM)

10.6.1 MPS ARPM and DMPM

10.6.1.1 General

The protocol procedures specify:

- the interactions between the MPS application services and link layer services.
- the interaction between the MPS application service elements and the link layer services. The service elements are characterized by application layer internal mechanisms that cannot be locally accessed by the primitives offered to the user.

The mapping between the parameters specifying link faults and the *Type of Error* parameters of the application layer service primitives is not specified. The error types described are simply recommendations.

10.6.1.2 Service procedures

10.6.1.2.1 General

The application services protocol procedures describe the interactions with the link services by use of evaluation nets.

Nevertheless the behaviour described by the evaluation nets does not presume the internal implementation of the MPS finite state machines services.

Furthermore, the service procedures describe the mapping on the link services in terms of primitives and parameters.

10.6.1.2.2 Local read service

10.6.1.2.2.1 Service Description

The local read service only interacts with the link services when reading a non resynchronized variable. When a variable is resynchronized the service performs a read on the *Private Value* and *Private Status* attributes of the *Consumed Variable Local Image* application object, situated at the application layer level.

10.6.1.2.2.2 Evaluation net

The A_Readloc service evaluation net is shown in Figure 173.

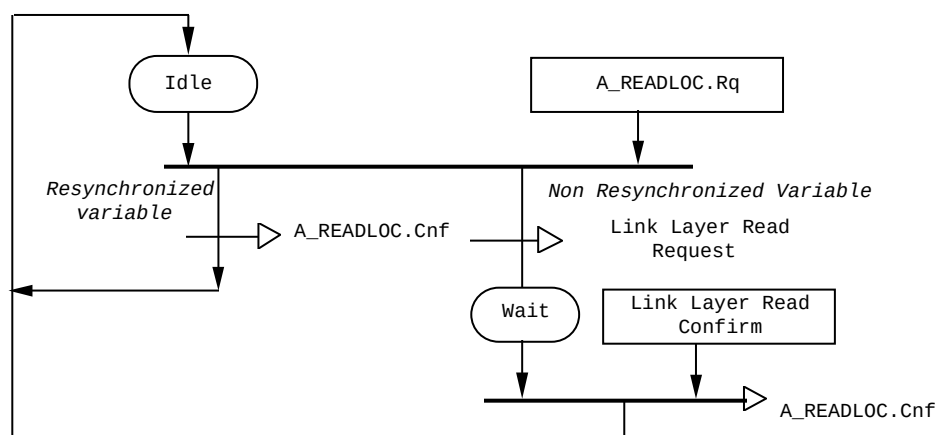


Figure 173 – A_Readloc service evaluation net

The confirmation associated with a A_READLOC.rq read request shall be negative whenever the data link confirmation was itself negative.

10.6.1.2.2.3 Mapping on Link services

The mapping on the link layer services is as follows:

Link Layer Read Request: L_GET.request(IDENTIFIER)

Link Layer Read Confirmation: L_GET.confirm(IDENTIFIER,STATUS, L_DATA)

10.6.1.2.2.4 Mapping on Link parameters

Mapping of MPS objects attributes on link services parameters is performed as follows:

IDENTIFIER: This parameter corresponds to the attribute *Identifier* of the *Consumed Variable Local Image* object.

L_DATA: This parameter corresponds to the transfer syntax PDU resulting from the encoding rules. The components of the PDU contents are available in the *Public value* and in the *Transmitted status value* attributes of the *Consumed Variable Local Image* object.

STATUS: When the confirmation is negative, this parameter specifies the error type.

10.6.1.2.3 Local write service

10.6.1.2.3.1 Service Description

The local write service only interacts with the link services when writing a non resynchronized variable. When a variable is resynchronized the service writes the value of the *Private Value* attributes of the *Produced Variable Local Image* application object, situated at the application layer level.

10.6.1.2.3.2 Evaluation net

The A_Writeloc service evaluation net is shown in Figure 174.

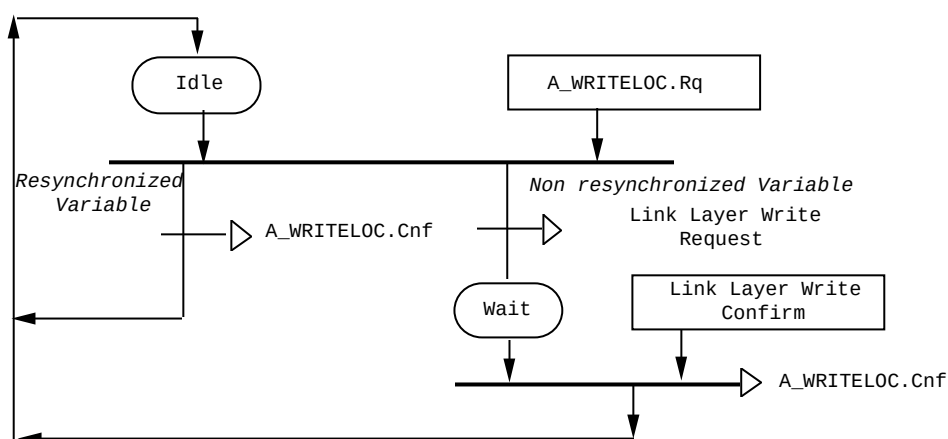


Figure 174 – A_Writeloc service evaluation net

The confirmation associated with a A_WRITELOC.rq write request shall be negative whenever the data link confirmation was itself negative.

10.6.1.2.3.3 Mapping on the Link services

The mapping on the link layer services is as follows:

Link Layer Write Request: L_PUT.request(IDENTIFIER, L_DATA)

Link Layer Write Confirmation: L_PUT.confirm(IDENTIFIER, STATUS)

10.6.1.2.3.4 Mapping on Link parameters

Mapping of MPS objects attributes on link services parameters is performed as follows:

IDENTIFIER: This parameter corresponds to the attribute *Identifier* of the *Produced Variable Local Image* object.

L_DATA: This parameter corresponds to the transfer syntax PDU resulting from the encoding rules. The components of the PDU contents are available in the Public value and the Transmitted status value attributes of the Produced Variable Local Image object.

STATUS: When the confirmation is negative, this parameter specifies the error type.

10.6.1.2.4 Update service

10.6.1.2.4.1 Evaluation net

The A_Update service evaluation net is shown in Figure 175.

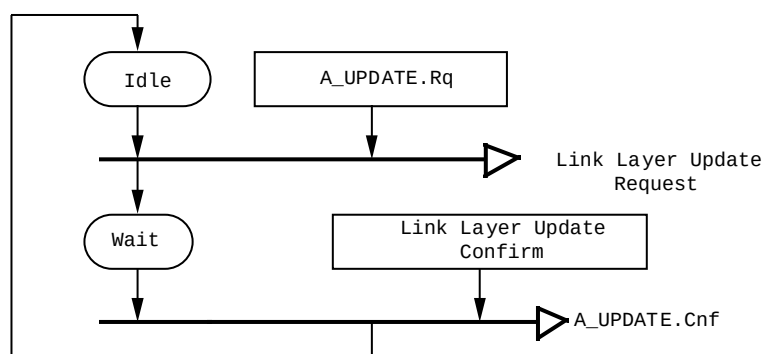


Figure 175 – A_Update service evaluation net

The confirmation associated with a A_UPDATE.rq update request shall be negative whenever the data link confirmation was itself negative.

10.6.1.2.4.2 Mapping on the Link services

The mapping on the data link layer services is performed as follows:

Link Layer Update Request: L_FREE_UPDATE.request(SEQUENCE OF REQUESTED IDENTIFIERS, PRIORITY)

Link Layer Update Confirmation: L_FREE_UPDATE.confirm(STATUS)

10.6.1.2.4.3 Mapping on Link parameters

Mapping of MPS object attributes on link services parameters is performed as follows:

SEQUENCE OF REQUESTED IDENTIFIERS: This parameter corresponds to the attribute *Identifier* of one of the following objects: *Consumed Variable Local Image*, *Produced Variable Local Image* or *Third Part Variable Local Image* object. This attribute corresponds to the variable specified in the *Variable Specification* parameter of the A_UPDATE service primitive.

PRIORITY: This parameter corresponds to the *Priority* parameter of the A_UPDATE service primitive.

STATUS: When the confirmation is negative, this parameter specifies the error type.

10.6.1.2.5 Remote read service

10.6.1.2.5.1 Service Description

When a variable is resynchronized, the remote read service performs only a link update request. The read value being the *Private value* attribute of the *Consumed Variable Local Image* object

When a variable is not resynchronized the remote read service first performs a link update request and, after confirmation of this request, the service performs a local read on the *Public value* attribute of the object *Consumed Variable Local Image*.

10.6.1.2.5.2 Evaluation net

The A_Readfar service evaluation net is shown in Figure 176.

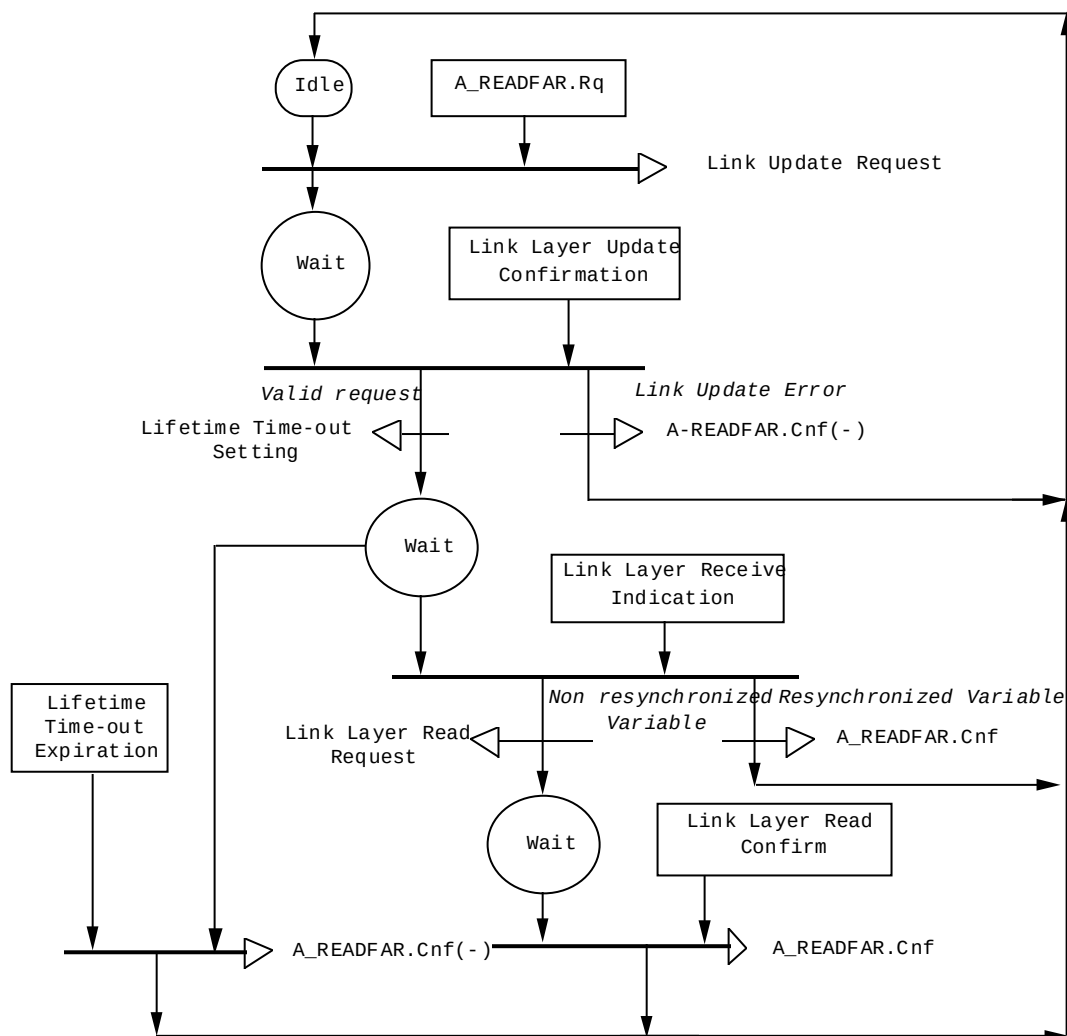


Figure 176 – A_Readfar service evaluation net

10.6.1.2.5.3 Mapping on the Link services

The mapping on the data Link Layer is made as follows:

Link Layer Read Request:	L_GET.(IDENTIFIER)
Link Layer Read Confirmation:	L_GET.confirm(IDENTIFIER,STATUS, L_DATA)
Link Layer Update Request:	L_FREE_UPDATE.request(SEQUENCE OF REQUESTED IDENTIFIERS, PRIORITY)
Link Layer Update Confirmation:	L_FREE_UPDATE.confirm(STATUS)
Link Layer Receive Indication:	L_RECEIVED.indication(IDENTIFIER)

10.6.1.2.5.4 Mapping on Link parameters

Mapping of MPS object attributes on link service parameters is performed as follows:

IDENTIFIER: This parameter corresponds to the attribute *Identifier* of the *Consumed Variable Local Image* object.

L_DATA: This parameter corresponds to the transfer syntax PDU resulting from the encoding rules. The components of the PDU contents are available in the *Public value* and in the *Transmitted status value* attributes of the *Consumed Variable Local Image* object.

SEQUENCE OF REQUESTED IDENTIFIERS: This parameter corresponds to the attribute *Identifier* of the *Consumed Variable Local Image* object.

PRIORITY: This parameter corresponds to the *Priority* parameter of the A_READFAR service primitive.

STATUS: When the confirm primitive is negative, this parameter specifies the type of the error.

10.6.1.2.6 Remote write service

10.6.1.2.6.1 Service Description

When a variable is resynchronized, the remote write service performs only a link update request, the value of the variable having been already write on the *Private value* attribute of the *Produced Variable Local Image* object

When a variable is not resynchronized the remote write service first performs a local write on the *Public value* attribute of the object *Produced Variable Local Image*, then after confirmation of the local write, the service performs a link update request.

10.6.1.2.6.2 Evaluation net

The A_writefar service evaluation net is shown in Figure 177.

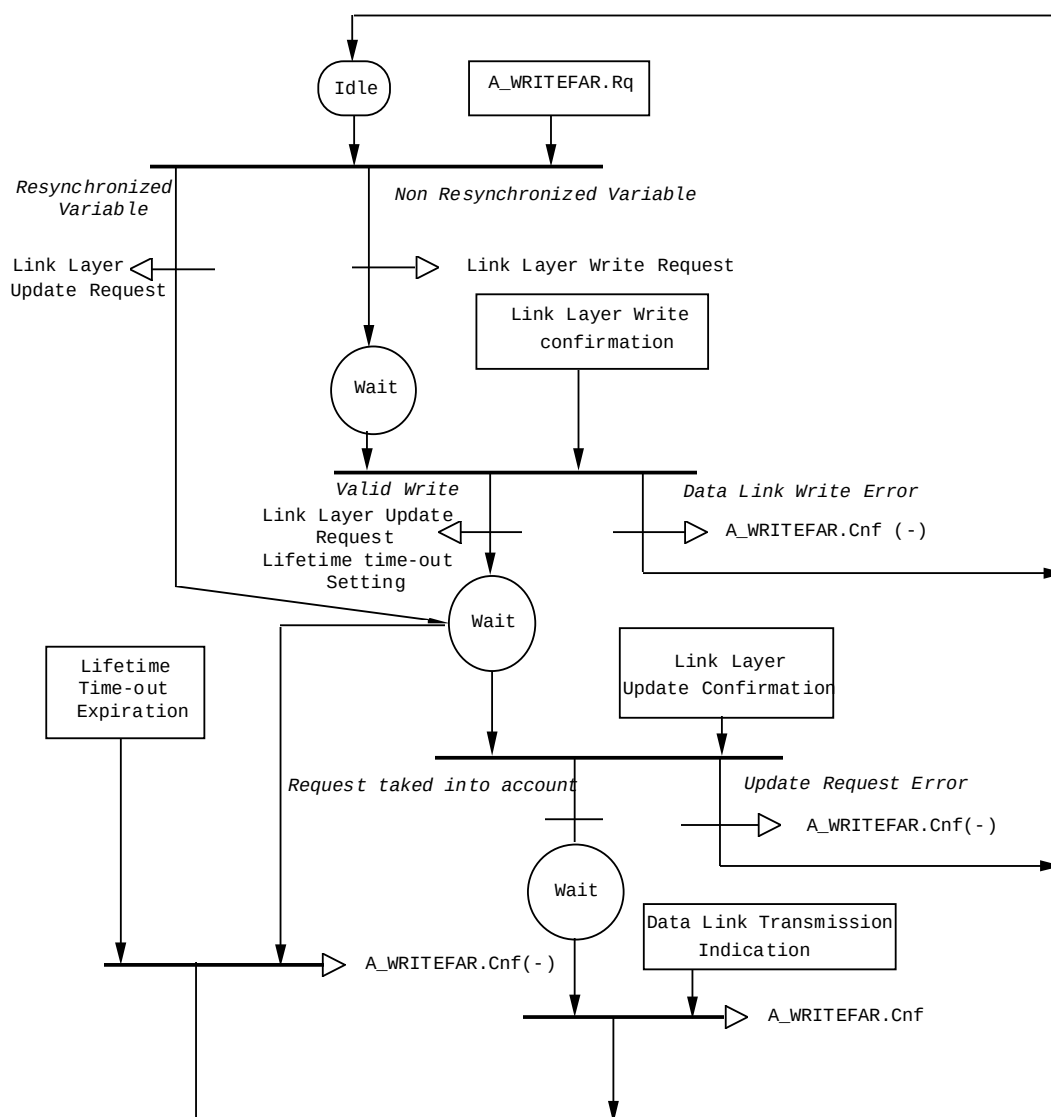


Figure 177 – A_writefar service evaluation net

10.6.1.2.6.3 Mapping on the Link services

The mapping on the data Link Layer is performed as follows:

Link Layer Write Request:	L_PUT.request(IDENTIFIER, L_DATA)
Link Layer Write Confirmation:	L_PUT.confirm(IDENTIFIER, STATUS)
Link Layer Update Request:	REQUESTED IDENTIFIERS, PRIORITY)
Link Layer Update Confirmation:	L_FREE_UPDATE.confirm(STATUS)
Link Layer Transmission Indication:	L_SENT.indication(IDENTIFIER)

10.6.1.2.6.4 Mapping on the Link parameters

The mapping of MPS object attributes on link services parameters is performed as follows:

IDENTIFIER: This parameter corresponds to the attribute *Identifier* of the *Produced Variable Local Image* object

L_DATA: This parameter corresponds to the transfer syntax PDU resulting from the encoding rules. The components of the PDU contents are available in the *Public value* and in the *Transmitted status value* attributes of the *Produced Variable Local Image* object.

SEQUENCE OF REQUESTED IDENTIFIERS: This parameter corresponds to the attribute *Identifier* of the *Produced Variable Local Image* object.

PRIORITY: This parameter corresponds to the *Priority* parameter of the A_WRITEFAR service primitive.

STATUS: When the confirm primitive is negative, this parameter specifies the type of error.

10.6.1.2.7 Transmission Indication Service

10.6.1.2.7.1 Evaluation net

The A_Sent service evaluation net is shown in Figure 178.

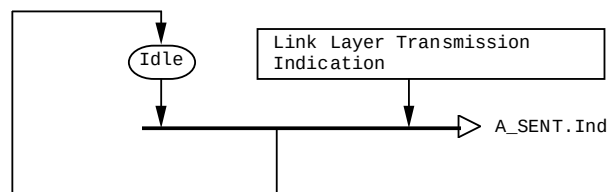


Figure 178 – A_Sent service evaluation net

10.6.1.2.7.2 Mapping on Link services

The mapping on the Link Layer services is performed as follows:

Link Layer Transmission Indication: L_SENT.indication(IDENTIFIER)

10.6.1.2.7.3 Mapping on the Link parameters

The mapping of MPS object attributes on link services parameters is performed as follows:

IDENTIFIER: This parameter corresponds to the attribute *Identifier* of the *Produced Variable Local Image* object of the variable which implements the transmission indication service.

10.6.1.2.8 Reception indication service.

10.6.1.2.8.1 Evaluation net

The A_Received service evaluation net is shown in Figure 179.

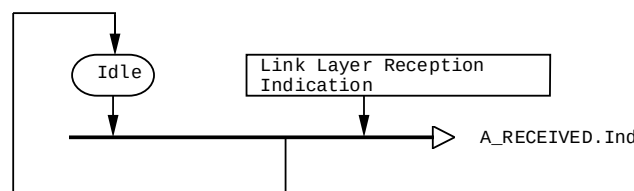


Figure 179 – A_Received service evaluation net

10.6.1.2.8.2 Mapping on the Link services.

The mapping on the Link Layer services is performed as follows:

Link Layer Reception Indication: L_RECEIVED.indication(IDENTIFIER)

10.6.1.2.8.3 Mapping on the Link parameters

Mapping of MPS objects attributes on link services parameters is performed as follows:

IDENTIFIER: This parameter corresponds to the attribute *Identifier* of the *Consumed Variable Local Image Object* of the variable which implements the reception indication service.

10.6.1.3 Service elements procedures

10.6.1.3.1 General

The application services elements are defined by Application Layer internal mechanisms that cannot be accessed by dedicated application primitives.

The protocol procedures associated with service elements describe the interaction with the link services by using evaluation nets; besides, the service procedures describe the mapping of those mechanisms on the link services of the AL TYPE 7 environment in terms of primitives and parameters.

10.6.1.3.2 Mapping of request and actions of the production validity status elaboration mechanisms:

An object *Produced Variable Local Image* has a service element procedure associated with the production validity status either the variable is resynchronized on production, or not resynchronized.

Status updating:

This action is mapped on the service primitive of data link level

L_PUT.request(IDENTIFIER,L_DATA)

The parameter IDENTIFIER, corresponds to the value of the attribute *Identifier* of the *Produced Variable Local Image* object.

The parameter L_DATA corresponds to the values of the attributes *Public Value* and *Transmitted Status Value* of the *Produced Variable Local Image* object.

Synchronization Order:

This request is mapped on a service primitive of data link level

L_RECEIVED.indication(IDENTIFIER) or

L_SENT.indication(IDENTIFIER)

The parameter IDENTIFIER, corresponds to the value of the attribute *Identifier* of the object *Consumed Variable Local Image* (of synchronization) or *Produced Variable Local Image* object (of synchronization) associated with the synchronization variable involved in the production validity status elaboration.

When the two objects *Consumed Variable Local Image* and *Produced Variable Local Image* associated with the synchronization variable are resident in the same application entity, the synchronization order is mapped on the consumed variable reception.

10.6.1.3.3 Mapping of request, and actions of the transmission validity status elaboration mechanisms:

A *Consumed Variable Local Image* object has a service element procedure associated with the transmission validity status whether the variable is resynchronized or not.

Variable reception:

This request is mapped on the data link level service primitive

L_RECEIVED.indication(IDENTIFIER)

The parameter IDENTIFIER corresponds to the value of the attribute *Identifier* of the *Consumed Variable Local Image* object.

Synchronization order:

This request is mapped on the link layer level services primitives

L_RECEIVED.indication (IDENTIFIER) or

L_SENT.indication(IDENTIFIER)

The parameter IDENTIFIER, corresponds to the value of the attribute *Identifier* of the object *Consumed Variable Local Image* (of synchronization) or *Produced Variable Local Image* object (of synchronization) associated with the synchronization variable involved in the transmission validity status elaboration mechanism.

10.6.1.3.4 Mapping of request, and actions of the recovery list elaboration mechanisms

The contents of the recovery list is managed by one service element which takes into account firstly, the synchronization order associated with the list, and secondly, the reception of the various variables of the list.

Recovery List Content Management

List Content Initialization:

These actions are mapped on the Data Link level service primitive

L_SPEC_UPDATE.request(SPECIFIED IDENTIFIER, SEQUENCE OF REQUESTED IDENTIFIERS)

The SPECIFIED IDENTIFIER parameter corresponds to the attribute *Recovery List Identifier* of the *List of Variables Local Image* object.

The SEQUENCE OF REQUESTED IDENTIFIERS corresponds to the attribute *Recovery List Content* of the *List of Variables Local Image* object.

List Synchronization Order:

This action is mapped on the data link level service primitive

L_RECEIVED.indication (IDENTIFIER)

The parameter IDENTIFIER corresponds to the attribute *Reference (synchronization) Variable* of the *Variables List Local Image* object.

When the two objects *Consumed Variable Local Image* and *Produced Variable Local Image* are resident in the same application entity, the synchronization order is mapped on the consumed variable reception.

Reception of the Variable List:

This action is mapped on the data link layer service primitive L_RECEIVED.indication (IDENTIFIER)

The parameter IDENTIFIER corresponds to the attribute *Reference Consumed Variable* of the *List of Variables Local Image* object.

NOTE When the attribute *Recovery Nature* of the *List of Variables Local Image* object has the value FRAGMENTED, the RQ_INHIBED link layer indicator shall have the value FALSE. When the attribute has the value INDIVISIBLE the indicator shall have the value TRUE.

10.6.1.3.5 Mapping of request, and actions of the resynchronization mechanisms.

When a variable is resynchronized, the involved mechanism uses:

- resynchronization order.
- The transfer of the public value into the private value for a consumed resynchronized variable.
- The transfer of the private value into the public value for a produced resynchronized variable.

Resynchronization Order:

This request is mapped on the data link layer level service primitives

L_RECEIVED.indication(IDENTIFIER) or

L_SENT.indication(IDENTIFIER)

The parameter IDENTIFIER, corresponds to the value of the attribute *Identifier* of the object *Consumed Variable Local Image* (of synchronization) or *Produced Variable Local Image* object (of synchronization) associated with the synchronization variable involved in the resynchronization mechanism.

When the two objects *Consumed Variable Local Image* and *Produced Variable Local Image* are resident in the same application entity, the synchronization order is the consumed variable reception.

Private Value into Public Value Transfer:

This action is mapped on the data link level service primitive

L_PUT.request(IDENTIFIER,L_DATA)

immediately after the update of the public context by means of the private context of the resynchronized variable.

The IDENTIFIER parameter corresponds to the attribute *Identifier* of the *Produced Variable Local Image* object.

The L_DATA parameter corresponds to the value of the attributes *Public Value* and *Transmitted Status Value* of the *Produced Variable Local Image* object.

Public Value into Private Value Transfer:

This action is mapped on the data link level service primitive

L_GET.request(IDENTIFIER,L_DATA)

immediately after the update of the private context by means of the public context of the resynchronized variable.

The IDENTIFIER parameter corresponds to the attribute *Identifier* of the *Consumed Variable Local Image* object.

The L_DATA parameter corresponds to the value of the attributes *Public Value* and *Transmitted Status Value* of the *Consumed Variable Local Image* object

10.6.1.3.6 Mapping of request, and actions of the consistency variable elaboration mechanisms

Each consistency variable value is elaborated in several application entities by using application service elements.

The value elaboration is done from the two following events: reception of the components of the variable list and reception of the synchronization variable of the list.

Consistency Variable Management

Consistency Variable Value Initialization:

These actions are mapped on the data link level service primitives L_SPEC_UPDATE.request (SPECIFIED IDENTIFIER, SEQUENCE OF REQUESTED IDENTIFIERS)

The parameter SPECIFIED IDENTIFIER corresponds to the attribute *Reference Produced Variable (of consistency)* of the object *List of Variable Local Image*.

The parameter SEQUENCE OF REQUESTED IDENTIFIERS corresponds to the attribute *Public Value* of the object *Produced Variable Local Image* which is referenced in the attribute *Reference Produced Variable (of consistency)* of the object *List of variables Local Image*.

Synchronisation List Order:

This action is mapped on the data link service primitive L_RECEIVED.indication(IDENTIFIER) or

L_SENT.indication(IDENTIFIER).

The parameter IDENTIFIER corresponds to the attribute *Reference (synchronization) Variable* of the object *List of variables Local Image*.

When the two objects *Consumed Variable Local Image* and *Produced Variable Local Image* are resident in the same application entity, the synchronization order is mapped on the consumed variable reception.

Reception of the Variable List:

This action is mapped on the data link level service primitive

L_RECEIVED.indication(IDENTIFIER)

The parameter IDENTIFIER corresponds to the attribute *Reference Consumed Variable* of the *List of Variables Local Image* object.

10.6.2 MCS ARPM and DMPM

10.6.2.1 General

The MCS protocols define the procedures which define the interactions between remote application entities and between data link layer entities, for the various services.

List of procedures:

- Association establishment
- Association termination
- Association revocation
- Associated mode data transfer
- Numbering
- Retry/Antiduplication
- Multiple anticipation
- Segmentation/Reassembly
- Non-associated mode data transfer
- Invocation identification

This definition, also features the different types of application data unit types implemented for carrying out the services.

- AARQ: Association establishment request
- AARP: Association establishment response
- RERQ: Association termination request
- RERP: Association termination response
- ABRQ: Association revocation request
- ATRQ: Associated transfer request
- AKAT: Associated transfer acknowledgement
- NTRQ: Non-associated transfer request
- AKNT: Non-associated transfer acknowledgement

NOTE Details of the coding of these application data units is covered in 3.8.6.

It is also important to know that all the protocol procedures consider the L_MSG_XX.rq/cnf/ind generic link primitives, and that it is only in 3.8.7 that an exact projection onto the link layer is to be found.

10.6.2.2 Association establishment

10.6.2.2.1 General

The association establishment procedure consists of two parts: the association establishment requesting element and responding element. These two elements of the procedure call upon functions which process the values of parameters and manage the resources of the association.

10.6.2.2.2 Requester element

a) For the association establishment requester element, the events and conditions accounted for are the following:

- invocation of the A_ASSOCIATE.rq() primitive by the user,
- reception of the AARP-APDU in the data of the L_MSG_XX.ind() primitive. This PDU contains the “Result” parameter which indicates whether establishment is accepted,
- a time-out which monitors the “Establishment duration” designed to limit the requester wait,
- invocation by the data link layer of the L_MSG_XX.cnf(+/-) primitive, indicating whether sending has taken place correctly or not,
- an establishment reinitialisation occurrence which corresponds to switching of the association status from OPEN to NON-EXISTENT.

NOTE This transition can be obtained by invoking the A_ABORT service by the user or the provider element, or any local initiative under network management responsibility.

- the association status value NON-EXISTENT governs acceptance of the A_ASSOCIATE.rq() primitive by the requester element.

b) The actions performed by this requester element are the following:

- invocation of the A_ASSOCIATE.cnf(+/-) primitive,
- transmission of an AARQ-APDU in the L_MSG_XX.rq() primitive data,
- resetting of the timer which monitors the “Establishment duration”, designed to limit the requester’s wait,
- assigns the value OPENING, ESTABLISHED or NON-EXISTENT to the association status.

c) Requester element mechanism:

An instantiation of this mechanism at the level of an application entity which supports the association establishment procedure causes the creation of an association. One or more associations can be in the process of being established simultaneously depending on the capability of the application entity.

The calling entity has invoked this mechanism for a given association.

In this mechanism, all interactions with the user are in the context of an “AE address” pair which identifies the two application entities concerned by the association opening, and all interactions with the data link layer are in the context of the link address (of ADAE type) which is associated with the AE address of the caller to localise the requester mechanism. See Figure 180.

NOTE It is important to understand that not all A_ASSOCIATE.rq() service invocation requests are necessarily followed by an A_ASSOCIATE.cnf(+/-) confirmation. Indeed, in the case of establishment reinitialisation, it is the A_ABORT.ind() primitive which follows the establishment request and thus takes the place of confirmation.

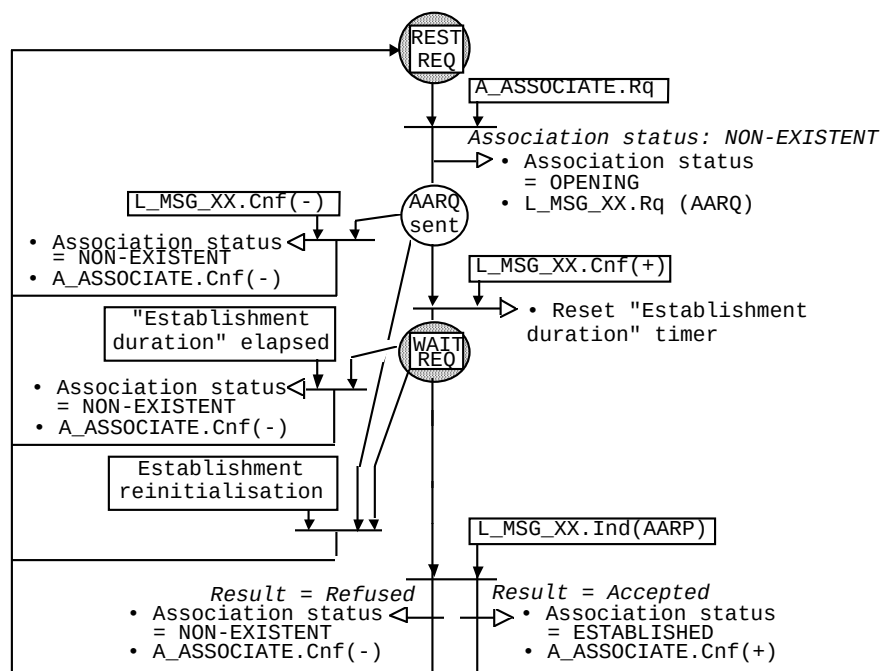


Figure 180 – Association establishment: Requester element state machine

d) Description of this mechanism

As soon as a A_ASSOCIATE.rq establishment request is made at a calling AEI access point for which there is no association (which corresponds to NON-EXISTENT association status), the latter switches to OPENING status and an establishment opening PDU is sent.

Sending consists of invoking the L_MSG_XX.rq() link service which is locally confirmed by L_MSG_XX.cnf() when the message has effectively disappeared from the link queues, subsequent to a successful transmission, or subsequent to a failure.

In the event of a link transmission failure, the requester element is reset to REST REQ status and the association to NON-EXISTENT status.

After sending an establishment request, the mechanism resets the “Establishment duration” timer and awaits an establishment response.

On arrival of the establishment response, the association changes to ESTABLISHED status or NON-EXISTENT status, depending on whether the establishment request has been accepted or refused by the responder to the request.

In addition, awaiting the response will be interrupted and the association set to NON-EXISTENT status if an establishment time-out is reached.

Finally, from the moment when the establishment procedure is started, i.e. when the requester element has “AARQ send” or “WAIT REQ” status, the latter is reinitialised in “REST REQ” status, when the status of the association changes from OPENING to NON-EXISTENT.

NOTE This change in the association status results from invoking the ABORT service by the local or remote user, or the local or remote provider element.

10.6.2.2.3 Responder element

a) For the establishment responder element, the events and conditions accounted for are the following:

- invoking the A_ASSOCIATE.rp() primitive by the user,
- receiving an AARQ-APDU in the L_MSG_XX.ind() primitive data,
- invoking the L_MSG_XX.cnf(+/-) primitive by the data link layer, indicating whether or not sending has taken place correctly,

- the occurrence of reinitialisation, which corresponds to changing the status of the association from OPENING to NON-EXISTENT,
- the association status value NON-EXISTENT governs acceptance of the receipt of an AARQ by the responder element.

b) The actions carried out by this responder element are the following:

- transmission of an AARP-APDU in the L_MSG_XX.rq() primitive data,
- assigning values OPENING, ESTABLISHED or NON-EXISTENT to the association status.

c) Responder element mechanism:

An instantiation of this mechanism at the level of an application entity which supports the association establishment procedure causes the creation of an association. One or more associations can be in the process of being established simultaneously depending on the capacity of the application entity.

NOTE Consequently, all establishment requests arriving after the maximum number of simultaneous establishment requests is reached are lost and there is no corresponding response. When such a loss occurs, the requesters wait until establishment time-out before repeating their requests.

The called entity is that which invokes the mechanism for a given association.

In this mechanism, all interactions with the user take place in the context of a pair of “AE addresses” which identify the two application identities concerned by the association opening, and all interactions with the data link layer are in the context of a link address (of ADAE type) which is associated with the AE address of the called entity for localising the responder mechanism. See Figure 181.

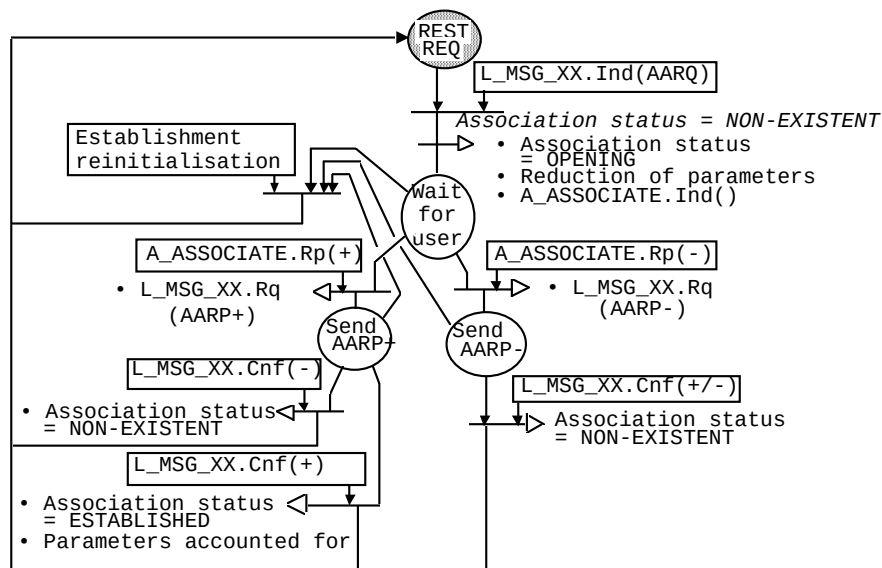


Figure 181 – Association establishment: Responder element state machine

d) Description of mechanism

As soon as the L_MSG_XX.ind() service is invoked with AARQ-APDU for data, when the association was previously NON-EXISTENT, the association is placed in OPENING status, the A_ASSOCIATE.ind() primitive is invoked, and an quality of service parameter reduction may occur.

The mechanism then enters an A_ASSOCIATE.rp() primitive wait state. As soon as the latter occurs, an AARP, which is positive or negative depending on the establishment response, is sent by invoking the L_MSG_XX.rq primitive.

This link service returns a local confirmation when the message has effectively disappeared from the link queues, after a successful transmission or after a failure.

In the event of failure in transmission of an AARP or success in transmission of a negative AARP signifying establishment refusal, the association status returns to NON-EXISTENT and the resources reserved on receiving the request are freed. The association status changes to ESTABLISHED in the event of successfully transmitting and AARP signifying establishment acceptance.

Finally, as soon as the establishment procedure is begun, i.e. the responder element status is other than “REST REQ”, the latter is reinitialised when the association status changes from OPENING to NON-EXISTENT.

10.6.2.3 Association termination

10.6.2.3.1 General

This association termination procedure consists of two parts, one relating to the termination requester and the other to the termination responder.

10.6.2.3.2 Requester element

a) For the termination requester element, the events and conditions accounted for are the following:

- invocation of the A_RELEASE.rq() primitive by the user,
- reception of an RERP-APDU in the data of the L_MSG_XX.ind() primitive. This PDU contains the “Result” parameter which indicates whether the termination is accepted,
- a “Termination duration” time-out, designed to limit the requester’s wait,
- the invoking of the L_MSG_XX.cnf(+/-) primitive by the data link layer, indicating whether or not the sending has occurred correctly,
- the occurrence of reinitialisation of termination which corresponds to the changing of the association status from CLOSING to ESTABLISHED,
- the association status established value governs the acceptance by the requesting element of the A_RELEASE.rq primitive,
- condition CN = LSNS.

b) The actions carried out by this requester element are the following:

- invoking of the A_RELEASE.cnf(+/-) primitive,
- sending of an RERQ-APDU in the data of the L_MSG_XX.rq() primitive,
- setting the time-out which monitors the “Termination duration”, designed to limit the requester’s wait,
- assigning of association status values CLOSING, ESTABLISHED and NON-EXISTENT.

c) Requester element mechanism:

An association in which it is possible to implement the termination procedure supports one single instance of this mechanism at called and calling entity levels. Both ends of the association can request termination.

This mechanism requires not only that all the interactions with the user take place in the context of the association to which the mechanism belongs but also that all interaction with the data link layer take place in the context of the pair of addresses (of ADAEI type) specific to the association. See Figure 182.

NOTE It is important to understand that the A_RELEASE.rq() service invocation request is not necessarily followed by an A_RELEASE.cnf(+/-) confirmation. Indeed, in the case of establishment reinitialisation, it is the A_ABORT.ind() primitive which follows a termination request and thus replaces the confirmation.

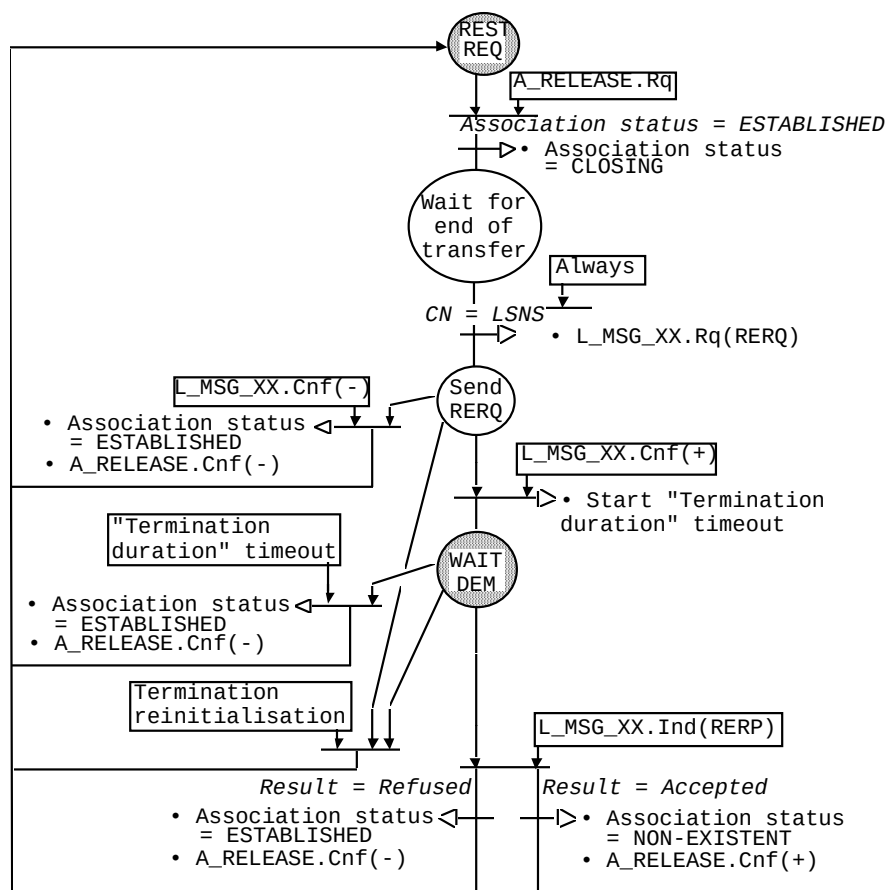


Figure 182 – Association termination: Requester element state machine

d) Description of mechanism:

As soon as the A_RELEASE.rq termination request is carried out in an ESTABLISHED association, its status changes to CLOSING and a PDU is sent requesting termination as soon as all the data transfers in progress in the association are finished (CN=LSNS).

Sending is by invoking the L_MSG_XX.rq() link service which is locally confirmed by L_MSG_XX.cnf() when the message has effectively disappeared from the link queues, subsequent to a successful transmission or subsequent to a failure.

In the case of a link transmission failure, the requester element is reset to quiescent status and the association is set to ESTABLISHED status.

NOTE Termination is a procedure in which priority is given to opening in the event of execution faults.

After transmitting a termination request, the mechanism awaits a termination response, after having started a "Termination duration" time-out.

On the arrival of the termination response, the association status changes to ESTABLISHED or to NON-EXISTENT, depending on whether the termination request was accepted or refused by the responder to this request.

In addition, the wait for a response is interrupted, and the association status becomes ESTABLISHED, if the termination time-out is reached. Finally, as soon as the termination procedure is begun, i.e. the requester element status is "Send RERQ" or "WAIT REQ", the latter is reinitialised with "REST REQ" status when the association changes from CLOSING to NON-EXISTENT.

NOTE This association state transition results from invocation of the ABORT service by the local or remote user or by the local or remote provider element for simultaneous termination purposes.

10.6.2.3.3 Responder element

a) For the termination responder element, the events and conditions accounted for are the following:

- invoking the A_RELEASE.rp() primitive by the user,
- receiving a RERQ-APDU in the data of the L_MSG_XX.ind() primitive,
- invoking the L_MSG_XX.cnf(+/-) primitive by the data link layer, indicating whether the transmission has taken place properly or not,
- an occurrence of termination reinitialisation, which corresponds to changing the association from the CLOSING to the ESTABLISHED state,
- the association status value CLOSING governs the acceptance of reception of an RERQ by the responder element.

b) The actions carried out by this responder element are the following:

- sending an RERP-APDU in the data of the L_MSG_XX.rq() primitive,
- assigning association status value CLOSING, ESTABLISHED and NON-EXISTENT,
- a provider ABORT request.

c) Responder element mechanism:

An association in which it is possible to implement the termination procedure supports a single instance of the mechanism at calling and called entity levels. Both ends of the association can respond to a termination.

In addition, if both ends request simultaneously, the termination collision is detected by the responder mechanism, which then proceeds to request a provider ABORT for the association.

The mechanism also requires that all interactions with the user be made in the context of the association to which the mechanism belongs and that all interactions with the data link level are in the context of the pair of addresses (of ADAEI type) specific to the association. See Figure 183.

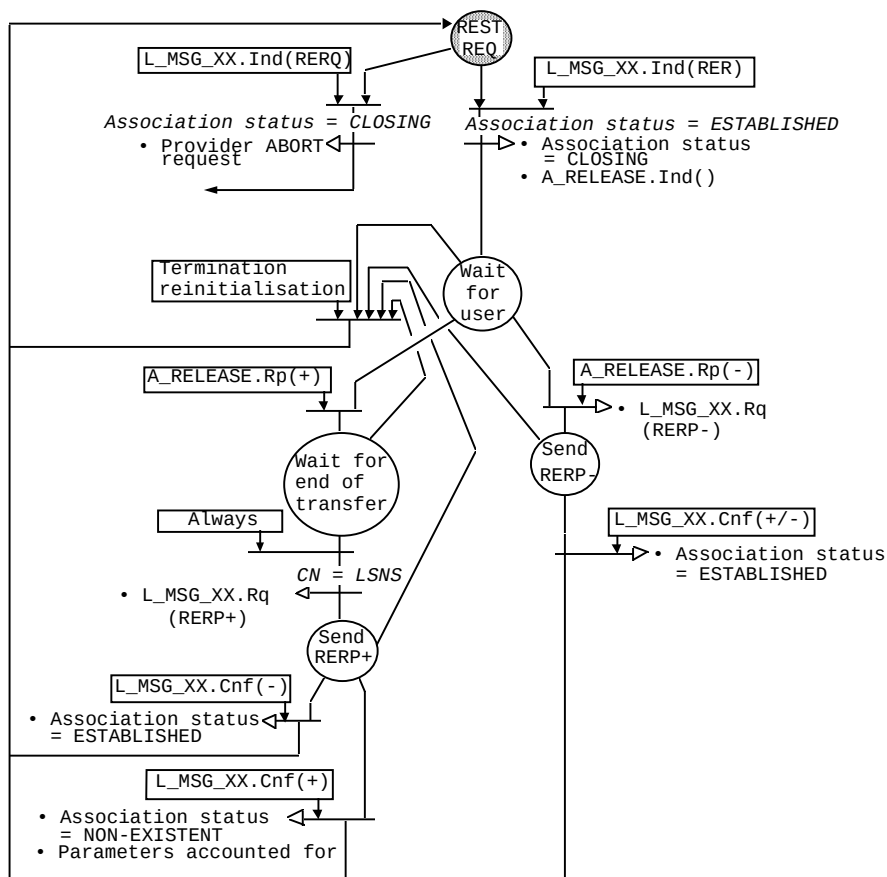


Figure 183 – Association termination: Responder element state machine

d) Description of mechanism:

As soon as the L_MSG_XX.ind() service is invoked with an RERQ-APDU for data, when the association is ESTABLISHED, the association state changes to CLOSING and the A_RELEASE.ind() is invoked.

The mechanism then awaits the A_RELEASE.rp() primitive. As soon as this arises, depending on whether it is negative or positive, an RERP termination response is sent by invoking the L_MSG_XX.rq() primitive immediately or as soon as CN = LSNS.

This link service returns a local confirmation when the message has effectively disappeared from the link queues, subsequent to a successful termination or subsequent to a failure.

In the event of a failure, or of success in transmission of an RERP signifying refusal of termination, the association returns to ESTABLISHED status.

In all other cases, the status of the association changes to NON-EXISTENT.

In the case of receiving a termination request (i.e. invoking the L_MSG_XX.ind(RERQ) service when the association status is already CLOSING, a provider ABORT request is made. Indeed, receiving a termination request in a state corresponding to a termination collision resulting from simultaneous termination requests by the two correspondents of the association.

Finally, as soon as the termination procedure is begun, i.e. the status of the responder element is other than "REST REQ", the latter is reinitialised when the association status changes from CLOSING to NON-EXISTENT.

10.6.2.4 Association revocation

10.6.2.4.1 General

This association revocation procedure consists of two parts: requester and acceptor.

NOTE The difference between the responder and the acceptor is that for the first association revocation is a confirmed service and for the second it is not.

10.6.2.4.2 Requester element

The requester element is that which causes the sending of an association revocation request to an association.

a) For an association revocation requester element, the events and conditions accounted for are the following:

- invoking the A_ABORT.rq() primitive by the user,
- invoking the L_MSG_XX.cnf(+/-) primitive by the data link layer,
- indicating whether or not sending has taken place correctly,
- the association status values OPENING, ESTABLISHED AND CLOSING govern acceptance of the A_ABORT.rq() primitive by the requester element.

b) The actions which are carried out by this requester element are the following:

- invoking the A_ABORT.ind() primitive,
- sending an ABRQ-APDU in the data of the L_MSG_XX.rq() primitive,
- setting association status value to NON-EXISTENT.

c) Mechanism of requester element:

An association in which it is possible to implement the revocation procedure supports a single instance of this mechanism at called and calling entity level.

This mechanism requires all interaction with the user to take place in the context of the association to which the mechanism belongs and all interaction with the data link level being in the context of the pair of addresses (of ADAEI type) specific to the association. See Figure 184.

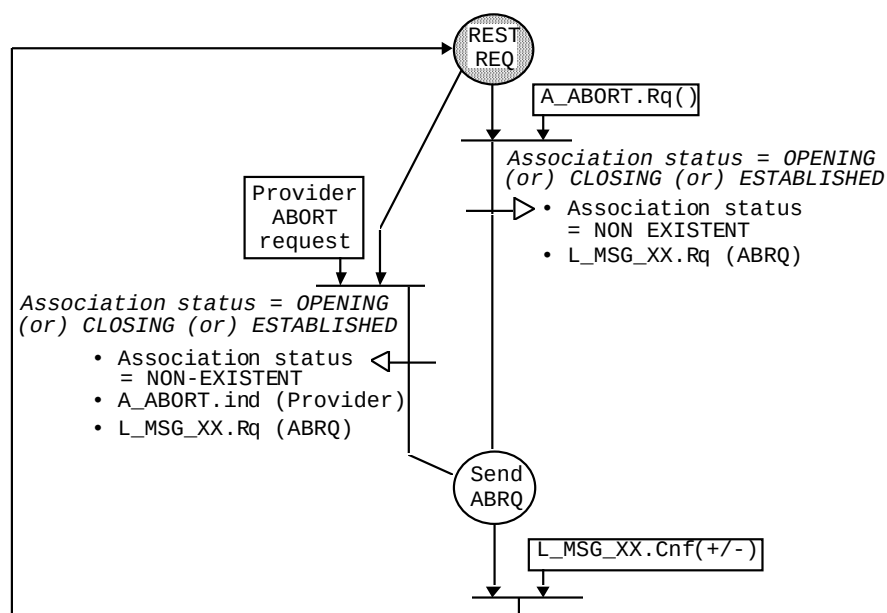


Figure 184 – Association revocation: Requester element state machine

d) Description of mechanism:

As soon as a revocation request from user A_ABORT.rq or the provider is made on an association which is ESTABLISHED, OPENING or CLOSING, the status of the latter changes to NON-EXISTENT and a revocation request PDU is sent. In addition, in the event of a provider attempt, the A_ABORT.ind primitive is invoked to warn the local user.

Whatever the results of sending, all the resources locally reserved for the association are freed.

10.6.2.4.3 Acceptor element

The acceptor element is that which receives an association revocation request and which processes it regardless of its origin.

a) For the revocation acceptor element, the events and conditions accounted for are the following:

- receiving an ABRQ-APDU in the data of the L_MSG_XX.ind() primitive,
- the OPENING, CLOSING or ESTABLISHED status of the association governs the acceptance by the acceptor element of the receipt of an ABRQ.

b) The actions carried out by this acceptor element are the following:

- invoking the A_ABORT.ind() primitive,
- setting the status of the association to NON-EXISTENT.

c) Acceptor element mechanism:

An association in which it is possible to implement the revocation process, supports a single instance of the mechanism at called and calling entity level.

This mechanism requires all interaction with the user to take place in the context of the association to which the mechanism belongs and for all interaction with the data link layer to be in the context of the pair of addresses (of ADAEI type) specific to the association. See Figure 185.

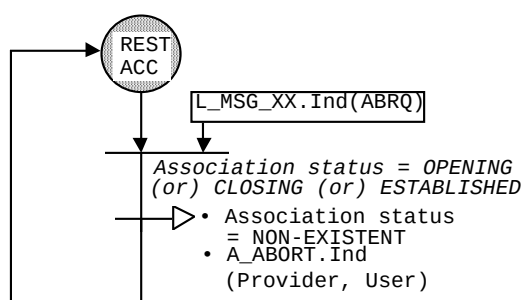


Figure 185 – Association revocation: Acceptor element state machine

NOTE Invocation of the A_ABORT.ind() service of which the origin is the "User" are necessarily from the correspondent, whereas one of "Provider" origin may be local or remote.

d) Description of mechanism:

As soon as the L_MSG_XX() service is invoked with an ABRQ-APDU for data, with the association ESTABLISHED, OPENING or CLOSING, the association status becomes NON-EXISTENT and the A_ABORT.ind() primitive is invoked.

10.6.2.5 Associated mode data transfer

10.6.2.5.1 General

In the associated mode, the complexity of the data transfer protocol procedure varies with the supported mechanisms adopted for the association.

The overall procedure is thus composed of a number of state machines respectively associated with the various mechanisms which can be implemented in association, interacting via calls and responses or via global variables.

The various state machines defined for the mechanisms which can be implemented in an association at data transfer requester or acceptor level are the following:

- TS_SM_RQ, TS_SM_ACC:
Transfer service machine for requester and acceptor.
- AT_SM_RQ, AT_SM_ACC:
Non-acknowledged transfer machine for the requester and the acceptor.
- ATA_K_SM_RQ, ATA_K_SM_ACC:
Acknowledged transfer machine for the requester and the acceptor.
- NB_SM:
Numbering machine at requester level.
- RC_SM:
Retry machine at requester level.
- DU_SM:
Antiduplication machine at acceptor level.
- MA_SM:
Anticipation machine at requester level.
- SG_SM:
Segmentation machine at requester level.
- RS_SM:
Reassembly machine at acceptor level.

NOTE The term “acceptor” is used for the receiver of a data transfer, as it is an unconfirmed service. For a confirmed service, the term “responder” would be used.

All state machines interact both:

- between themselves in the context of an association. These interactions take place by MCS local identification of calls/responses between machines,
- with other application service elements participating at the same association. These take place by local identification between ASEs,
- with the data link layer in the case of a pair of link addresses of the ADAEI type, which localises the ends of the association.

Figure 186 shows interaction between the various state machines.

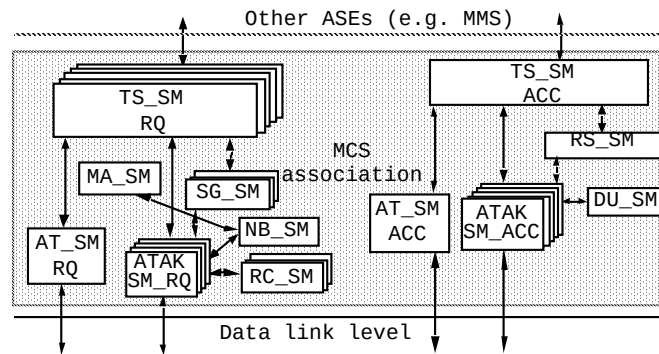


Figure 186 – Interactions between state machine in an associated mode data transfer

This breakdown of the associated mode data transfer procedure uses modularity associated with the classes of conformity. It is thus that the only necessary machine is that of the transfer service which, in conjunction with the acknowledged or unacknowledged transfer machine, constitutes the simplest classes of conformity for transfer of data in the associated mode.

10.6.2.5.2 Definitions

10.6.2.5.2.1 Variables

The various state machines implemented in the context of an association for the transfer of data in the associated mode are co-ordinated by variables. There are two types of variables:

- local variables, which are specific to one status machine and which have a value which exists and may vary in the context of each of the invocations of the machine,
- global variables, which makes it possible to manage interaction between the different mechanisms and which have a value which exists and possibly varies in the context of each association.

The local variables are described at the level of each of the mechanisms to which they belong, and the global variables are described below.

SA (Service Access): Access to the service used to specify whether a new transfer service can be invoked in the association. (TS_SM_RQ), in order to avoid interlacing of the data transfers associated with a number of service invocations. This variable is shared by TS_SM_RQ and SEG_SM machines.

The value varies dynamically between:

- FREE if another invocation is possible,
- BUSY if it is temporarily impossible to invoke another transfer service.

Typically, the status of the variable changes from FREE to BUSY when the transfer service machine, on the requester side, activates a segmentation machine.

This it remains in the BUSY state for the time interval during which the segmentation machine divides the messages into packets and instantiates as many ATAK_SM_RQ acknowledged transfer protocol machines as there are packets. The status of the SA variable changes from BUSY to FREE in two cases:

- when all the packets subsequent to the message have been invoked by an ATAK_SM_RQ machine or
- if the service is interrupted after negative confirmation of a packet (other than the last one) originating from the ATAK_SM_RQ machine invoked for the packet. Indeed, in this case the message is considered to have been corrupted and the current transfer is interrupted, whereafter another transfer can be envisaged.

AC (Anticipation Credit): The anticipation credit is utilised to specify that another acknowledged transfer (ATAK_SM_RQ) can be invoked in the association.

Another transfer can only be invoked within the quantitative limits set by the anticipation factor provided that the difference between the sequence number of the earliest invocation and the current number is less than the value of the anticipation factor.

The value varies dynamically between:

- FREE if a new invocation is possible,
- BUSY if it is temporarily impossible to invoke another acknowledged transfer.

On opening an association, the variable is initialised at FREE.

The mechanism controlling variation of the value of this variable is MA_SM.

CN (Current Number): The sequence number to be allocated to the next acknowledged data transfer.

This variable is shared by the machines NB_SM, MA_SM, SEG_SM and ATAK_SM_RQ.

The value varies under the control of the numbering mechanism (NB_SM).

This variable is initialised at “0” on opening the association.

LSNS (Lowest Sequence Number Send): The lowest sequence number attributed to an acknowledge transfer being exchanges and known at transfer initiator level.

The sequence numbers (SN) identify the invocations of transfer in the associated mode. Each of the PDUs sent contains a sequence number, which is necessary for numerous mechanisms.

This variable is initialised at “0” on opening the association.

S (Size): Size of service expressed in terms of the number of packets of maximum size.

a (Anticipation factor): This is the anticipation factor adopted for the association. The minimum value of a is 1.

RC: Specifies whether retry (RC_SM) is adopted in the association. The value of this variable is static throughout the life of an association and is governed by the fact that the two devices participating in it belong to a class of conformity which supports it.

The values are the following:

- USED: mechanism adopted, necessarily being supported by the devices,
- UNUSED: mechanism not adopted as not supported, or not supported by any one of the devices.

DU: Specifies whether antiduplication (DU_SM) is adopted in the association. The value of this variable is static throughout the life of an association and is governed by the fact that the two devices participating in it belong to a class of conformity which supports it.

The values are the following:

- USED: mechanism adopted, necessarily being supported by the devices,
- UNUSED: mechanism not adopted as not supported, or not supported by any one of the devices.

SG: Specifies whether segmentation (SG_SM) is adopted in the association. The value of this variable is static throughout the life of an association and is governed by the fact that the two devices participating in it belong to a class of conformity which supports it.

The values are the following:

- USED: mechanism adopted, necessarily being supported by the devices,
- UNUSED: mechanism not adopted as not supported, or not supported by any one of the devices.

RS: Specifies whether reassembly (RS_SM) is adopted in the association. The value of this variable is static throughout the life of an association and is governed by the fact that the two devices participating in it belong to a class of conformity which supports it.

The values are the following:

- USED: mechanism adopted, necessarily being supported by the devices,
- UNUSED: mechanism not adopted as not supported, or not supported by any one of the devices.

10.6.2.5.2.2 Operators

The different state machines have conditional transitions which are expressed with arithmetic in $2^{\text{exp}16}$ modulo numbers. Indeed, all MCS-APDU identification is carried out with 16-bit binary-coded numbers which vary between 0 and $2^{\text{exp}16}-1 = \text{Max}$.

Consequently, to facilitate expression of conditions, operators have been defined:

- **“successor”**
When $y = \text{success}(x)$; $y = x + 1$, if $x < \text{Max}$; and $y = 0$, if $x = \text{Max}$
- **“distance”**
When $y = \text{dist}(a, b)$; $y = a - b$, if $a \geq b$; and $y = a + \text{max} + 1 - b$, if $a < b$
- **“between”**
When $(x) \text{between}(a, b) = y$;
 $y = \text{TRUE}$, if $b > a$ and $x \geq a$ and $x < b$,
 if $a > b$ and $x \geq a$,
 if $a > b$ and $x < b$
 $y = \text{FALSE}$, in all other cases.

NOTE In the special case where a is equal to b , y takes the value FALSE.

10.6.2.5.3 Transfer service (TS_SM)

10.6.2.5.3.1 General

The transfer service machine consists of two parts, one relating to the requester and the other to the acceptor.

10.6.2.5.3.2 Requester element (TS_SM_RQ)

The requester element is that which causes the transmission of a data transfer request on an association. See Figure 187.

- a) For the transfer service request element, the following events and conditions are accounted for:

A_DATA.rq(Request_ack), where Request_ack = TRUE (or) FALSE

dt_noack_cnf(+/-) (see 10.6.2.5.4.2)

dt_ack_cnf(+/-) (see 10.6.2.5.5)

- segment_cnf(+/-) (see 10.6.2.5.9)
- Association status = ESTABLISHED
- AC = FREE
- SA = FREE
- SG = USED or UNUSED.

- b) The actions carried out by the requester element are the following:

A_DATA.cnf(+/-)

- dt_noack_rq() ,
- dt_ack_rq(Packet nature), where Packet nature = NIL (see 10.6.2.5.2.1),
- segment_rq() (see 10.6.2.5.9).

- c) Mechanism of requester element

An association necessarily has a transfer service mechanism and can support one or more instances at called and calling entity level. The multiplicity of instances is limited by the maximum anticipation value specified in an association. Indeed, the number of instances possible is equal to the maximum anticipation value if there is no segmentation, and less than or equal to it otherwise.

NOTE If the segmentation mechanism is adopted, the number of instances of the mechanism possible in an association is equal, on average, to maximum anticipation “a” divided by the average number of packets necessary for the exchange of a service.

Machine: TS_SM_RQ

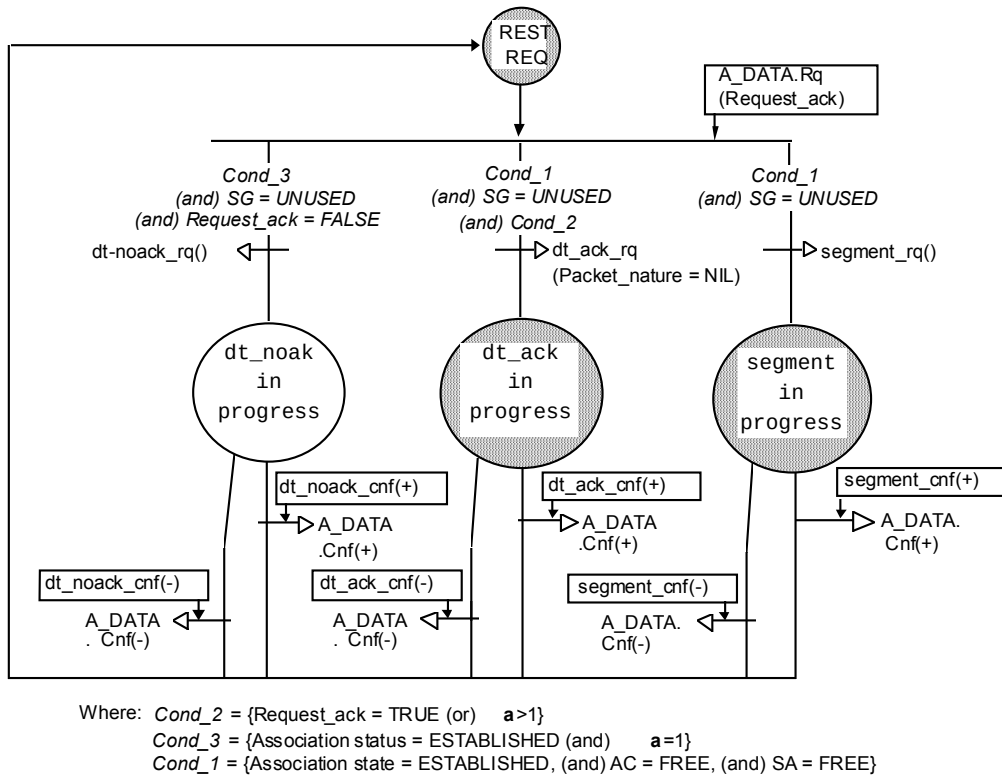


Figure 187 – Transfer service – Requester element state machine

d) Description of mechanism:

As soon as a A_DATA.rq(Request_ack) data transfer request is made by the user, a call to another mechanism is made depending on the value of the global variables of the association and the “Request_ack” parameter transferred in the request.

Thus, an acknowledged transfer, or an unacknowledged transfer or a segmentation are begun.

In the case of an association which implements segmentation, the “Request_ack” parameter can only have the value TRUE.

10.6.2.5.3.3 Acceptor element (TS_SM_ACK)

The acceptor element is that which receives a data transfer in an association.

a) For the transfer service acceptor element, the following events and conditions are accounted for:

- dt_noack_ind() (see 10.6.2.5.4.3)
- dt_ack_ind() (see 10.6.2.5.5.3)
- reassemb_ind() (see 10.6.2.5.10)
- Association status = ESTABLISHED (or) CLOSING
- RS = USED (or) UNUSED

b) The actions carried out by this acceptor element are the following:

- A_DATA.ind (Request_ack), where Request_ack = TRUE, (or) FALSE

NOTE This parameter necessarily has the value TRUE if reassembly takes place.

c) Acceptor element mechanism (See Figure 188):

An association necessarily has a transfer service mechanism and may support a number of instances of the machine for local anticipation purposes at device level.

Machine: TS_SM_ACC

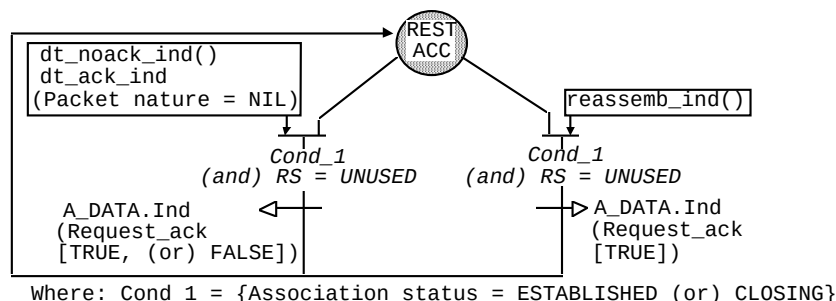


Figure 188 – Transfer service: Acceptor element state machine

d) Description of mechanism:

As soon as another RS_SM or AT_SM_ACC, or ATAK_SM_ACC machine signals receiving the data transfer, this indication is accounted for depending on the value of the global variables of the association.

Its accounting for involves returning an A_DATA.ind(Request_ack) data transfer indication to the user. In the case where an association implements reassembly, the parameter "Request_ack" can only take the value TRUE.

10.6.2.5.4 Unacknowledged transfer (AT_SM)

10.6.2.5.4.1 General

In unacknowledged associated mode data transfer, the machine consists of two parts, one relating to the requester and the other to the acceptor.

10.6.2.5.4.2 Requester element (AT_SM_RQ)

The requester element is that which causes sending of a data transfer request in an association.

a) For the unacknowledged transfer requester element, the following events and conditions are accounted for:

- dt_noack_rq(),
- L_MSG_XX.cnf(+/-) (see IEC 61158-3 Type 7).

b) The actions carried out by this requester element are the following:

- L_MSG_XX.rq(ATRQ[noack]) (see IEC 61158-3 Type 7),
- dt_noack_cnf(+/-).

c) Requester element mechanism (See Figure 189):

An association in which it is possible to implement the non-acknowledged data transfer machine may support a number of instances of this machine for local anticipation purposes at device level.

Machine: AT_SM_RQ

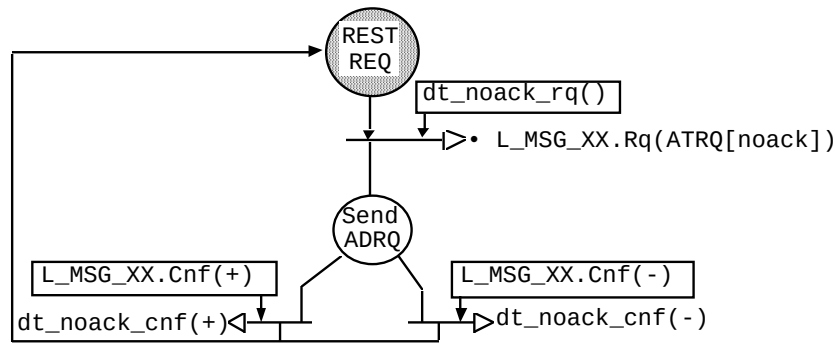


Figure 189 – Unacknowledged transfer: Requester element state machine

d) Description of mechanism:

As soon as a call is made to this machine with `dt_noack_rq`, an ATRQ data transfer which does not contain an acknowledgement request is sent.

Whatever the results of sending, the mechanism returns to its initial status.

NOTE Sending errors may be accounted for by equipment fault management.

10.6.2.5.4.3 Acceptor element (AT_SM_ACC)

The acceptor element is that which receives a data transfer with no reception acknowledgement requests and which transfers it to the user.

a) For this element, the following events and conditions are accounted for:

`L_MSG_XX.ind(ATRQ[noack])` (see IEC 61158-3 Type 7).

b) The following actions are carried out by this acceptor element:

- `dt_noack_ind()`

c) Acceptor element mechanism (See Figure 190):

An association in which it is possible to implement the non-acknowledged data transfer procedure may support a number of instances of this machine for local anticipation purposes at device level.

Machine: AT_SM_ACC

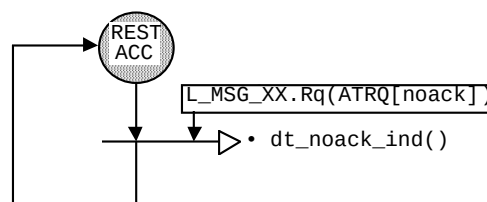


Figure 190 – Unacknowledged transfer: Acceptor element state machine

d) Description of mechanism:

As soon as an `L_MSG_XX.ind()` link service is invoked, the data of which correspond to an ATRQ without an acknowledgement request, a `dt_noack_ind` signal is given.

10.6.2.5.5 Acknowledged transfer (ATAK_SM)

10.6.2.5.5.1 General

The acknowledged associated mode data transfer machine consists of two parts, one relating to the requester and the other to the acceptor.

10.6.2.5.5.2 Requester element (ATAK_SM_RQ)

The requester element is that which causes transmission of a data transfer request and receives the corresponding acknowledgement in an association. See Figure 191.

a) For this requester element, the events and conditions accounted for are the following:

- dt_ack_rq(Packet nature), where Packet nature = STT, (or) MID, (or) END, (or) ALL, (or) NIL.

Packet nature NIL is used when this requester element is invoked outside segmentation.

- L_MSG_XX.cnf(+/-) (see IEC 61158-3 Type 7)
- L_MSG_XX.ind(ATAK[Packet_num])
- acknowledgement time-out
- retry_cnf(+/-) (see 10.6.2.5.6)
- revocation AT_SM_RQ
- RC = USED (or) UNUSED
- SN = Packet_num
- (SN) between (FSN,LSN) = TRUE

b) The following actions are carried out by this requester element:

- SN = Take number (see 10.6.2.5.5.4)
- L_MSG_XX.rq(ATRQ[ack]) (see IEC 61158-3 Type 7)
- resetting of acknowledgement time-out
- retry_rq() (see 10.6.2.5.6)
- dt_ack_cnf(+/-)
- free(SN)
- retry revocation

c) Request element mechanism:

For an association in which it is possible to implement the data transfer mechanism with acknowledgement, the number of instances of the machine which can be implemented simultaneously both at calling entity and called entity level is linked to the value of the anticipation factor and the value of the anticipation credit. The anticipation can be implemented simultaneously both at calling entity and called entity level is linked to the value of the anticipation factor and the value of the anticipation credit. The anticipation factor determines the greatest possible number and the credit establishes the greatest instantaneous number on the basis of constraints associated with the retry and antiduplication mechanism designed to increase transmission reliability.

Machine: ATAK_SM_RQ

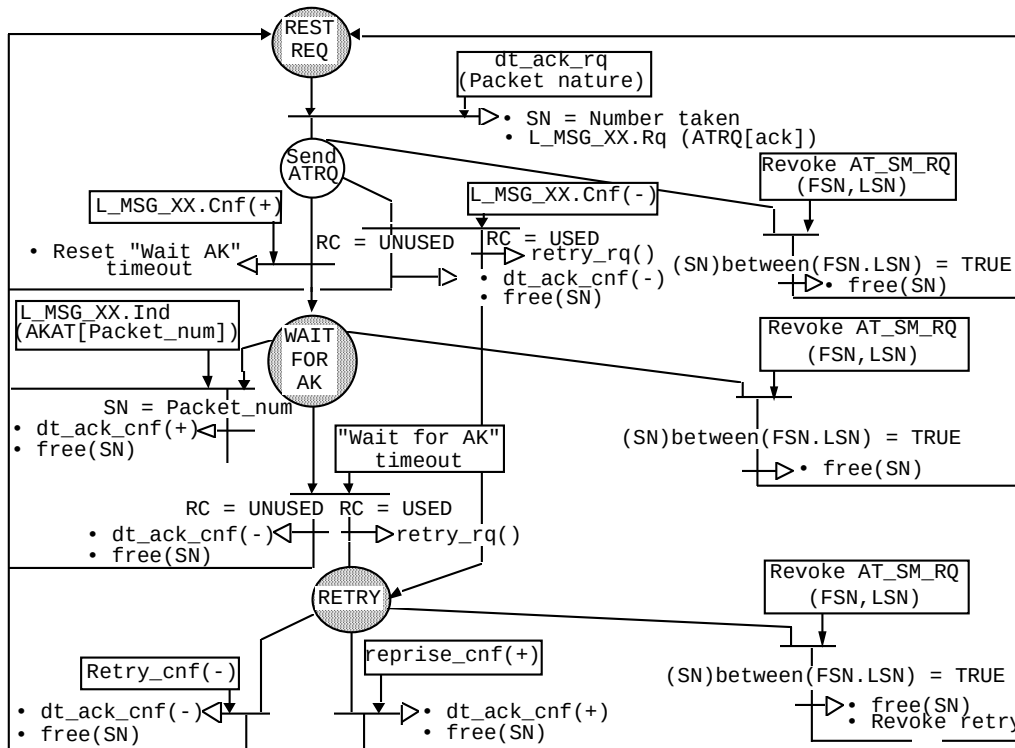


Figure 191 – Acknowledged transfer: Requester element state machine

d) Description of mechanism:

As soon as the unacknowledged data transfer request is made, an ATRQ containing an acknowledgement request is sent.

The "Packet nature" parameter contains values STT, MID, END and ALL if filled in by the segmentation machine and NIL otherwise. This parameter is coded in the ATRQ.

If the result of sending return by the data link layer is positive, an acknowledgement time-out is begun. The state is one of waiting for acknowledgement, which either arrives, in which case the mechanism returns to its initial state, or the acknowledgement time-out expires and a retry may take place before returning to the same initial status.

Furthermore, if after a transfer request, the data link level returns a negative result, a retry is immediately requested if the latter is utilised.

Any invoking of this machine involves taking a sequence number from the global numbering machine of the association. This is stored in the "SN" variable throughout its lifetime. Finally, any event leading to the revocation of an invocation, whether completed normally or abnormally, results in freeing of the sequence number to manage the value of the lowest sequence number (LSNS) still current in the association.

Finally, as soon as an instantiation of this mechanism is in another state than "REST-REQ", it can be reinitialised at a "Revocation AT_SM_RQ" request from the segmentation mechanism.

10.6.2.5.5.3 Acceptor element (ATAK_SM_ACC)

The acceptor element is that which receives the data transfer and the acknowledgement in an association. See Figure 192.

a) For this acceptor element, the following events and conditions are taken into account:

- L_MSG_XX.ind(ATRQ[ack]) (see IEC 61158-3 Type 7)

antiduplication_cnf(Message), where Message = REPETITION, (or) NEW

- L_MSG_XX.cnf(+/-) (see IEC 61158-3 Type 7)
- DU = USED, (or) UNUSED

b) The actions carried out by this acceptor element are the following:

- antiduplication_rq(Sequence number), Sequence number has the value derived from ATRQ[ack]
- L_MSG_XX.rq(AKAT) (see IEC 61158-3 Type 7)
- dt_ack_ind(Sequence number; Packet nature) where Packet nature has the value derived from the ATRQ received and Packet nature = STT, (or) MID, (or) END, (or) ALL, (or) NIL (see 10.6.2.5.5.2).

c) Acceptor element mechanism:

An association in which it is possible to implement the data transfer procedure may support a number of instances of this machine for local anticipation purposes at device level.

Machine: ATA_K_SM_ACC

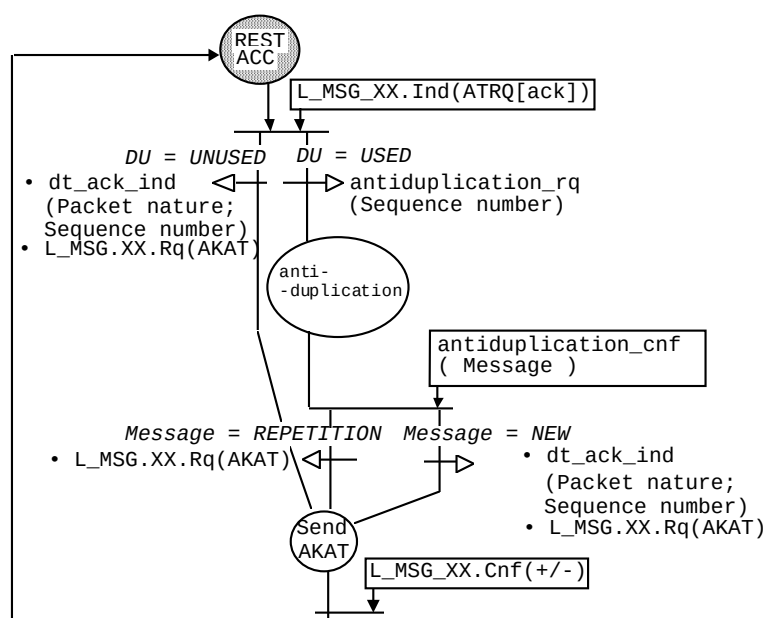


Figure 192 – Acknowledged transfer: Acceptor element state machine

d) Description of mechanism:

As soon as the L_MSG_XX.ind primitive is invoked, with data which corresponds to an ATRQ with acknowledgement request, a condition is evaluated before proceeding with sending an acknowledgement, and return of the message. This condition concerns the fact that antiduplication is implemented in the association.

If this is adopted, depending on whether it considers the message as NEW or as a REPETITION, it is or is not returned.

In all cases, an acknowledgement is returned to the data transfer sender, containing a sequence number which is equal to that of the data transfer acknowledged.

Finally, whether or not sending of the transmission fails at link level, the mechanism returns to its initial state.

10.6.2.5.5.4 Numbering (NB_SM)

On using the data transfer service with acknowledgement in an association, the numbering element is implemented to manage allocation of sequence numbers (SN). The values of this number are transited together with the data in the transfers using the ATRQ type PDU, and which are also found in the respectively associated acknowledgements, of which the PDU is of the AKAT type. See Figure 193.

In addition, the state machines implemented at requester and acceptor levels, for acknowledged data transfers, retain this number during their lifetime to make them identifiable by the protocol in the context of the association.

Furthermore, this machine manages freeing of the sequence numbers. Freeing takes place each time a data transfer with acknowledgement terminates normally or abnormally. This management system makes it possible to update the value of the LSNS variable.

a) This mechanism accounts for the following events and conditions:

- “Take number”
- A_ASSOCIATE.rp(+), (or) A_ASSOCIATE.cnf(+)
- A_RELEASE.rp(+), or A_RELEASE.cnf(+), or A_ABORT.ind()
- free(SN).

b) The actions triggered by this mechanism are the following:

- CN = success(CN)
- LSNS management
- ma_rq()
- LSNS = CN = 0
- LACT[i] = Not received, when $0 \leq i \leq \text{Max}$.

c) Mechanism:

There is a single instance of this mechanism per association at calling entity level and at called entity level. This mechanism shall necessarily exist at the level of one association if the latter supports data transfer with acknowledgement.

Machine: NB_SM

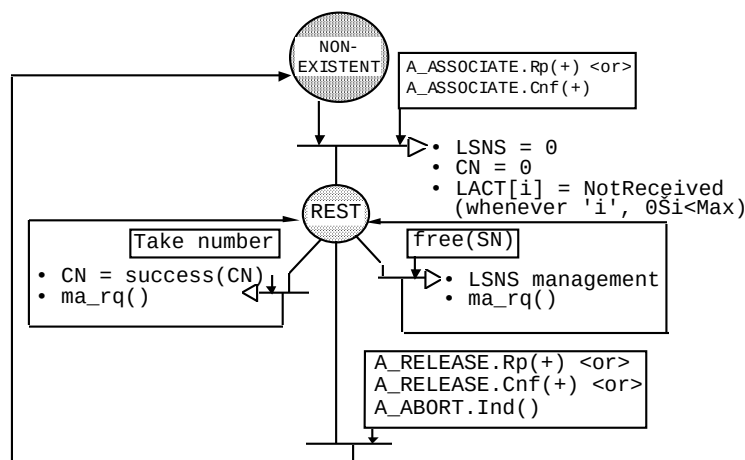


Figure 193 – Numbering mechanism state machine

d) Description of mechanism:

A numbering entity (NB_SM) is instantiated and utilised on opening an association. Whenever the “Take number” function is invoked by another mechanism causing reading of the value followed by incrementation. The value taken is used to identify an acknowledged data transfer. In addition, this value circulates in the acknowledged data transfer PDUs but also in the associated acknowledgement.

Whenever the free function (SN) is invoked by another mechanism management of the LSNS variable takes place. Management of the LSNS is based on a list LACT[i] containing “Max” elements (see 10.6.2.5.2.2) and proceeds to the following operations:

- if $SN \neq LSNS$, then account for the reception by setting $LACT[SN] = \text{Received}$,
- if $SN = LSNS$, then repeat $LSNS = \text{success}(LSNS)$ and $LACT[SN] = \text{Not received}$ until $LACT[LSNS] \neq \text{Not received}$.

10.6.2.5.6 Retry (RC_SM)

On using the data transfer service with acknowledgement in an association, the retry element can be implemented if adopted in the association. See Figure 194.

This element can only be implemented conjointly with that of antiduplication (10.6.2.5.7).

This element makes it possible to reiterate, a number of times, a data transfer with acknowledgement in the event of absence of receiving an acknowledgement after a specified time limit has elapsed.

a) The following events and conditions are accounted for by this mechanism:

- `retry_rq()`
- Acknowledgement time-out
- Counter: retry counter
- Rpm_{max}: maximum value of retry counter adopted for the association
- `L_MSG_XX.cnf(+/-)` (see IEC 61158-3 Type 7)
- `L_MSG_XX.ind(AKAT)` (see IEC 61158-3 Type 7)
- Retry revocation.

b) The following actions are triggered by this mechanism:

- `retry.cnf(+/-)`
- `L_MSG_XX.rq(ATRQ)` (see IEC 61158-3 Type 7)
- Incrementation of counter
- Starting of “Wait for AK” time-out.

c) Mechanism:

There is one instance of this mechanism per request for data transfer with acknowledgement being retried. Each of these mechanisms has its own current counter value (Counter). As for the maximum value of the retry counter (Rpm_{max}), it is that which is specific to the association in which it is implemented.

Machine: RC_SM

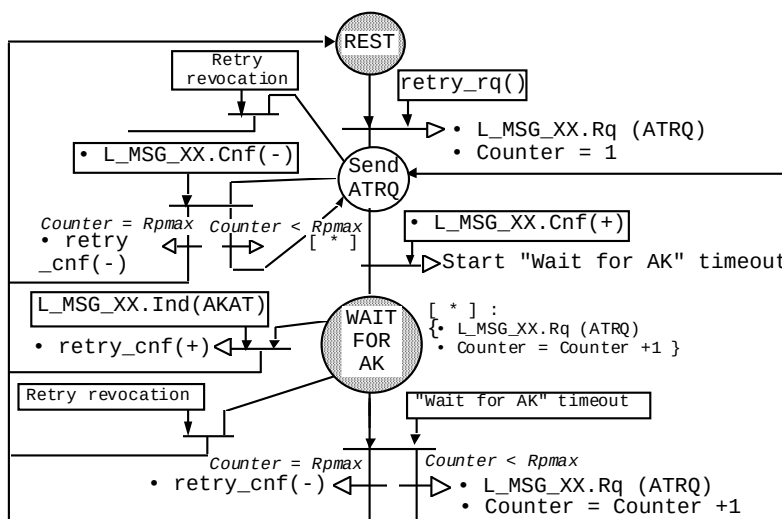


Figure 194 – Retry mechanism state machine

d) Description of mechanism:

This mechanism simply proceeds with resending a data transfer within the limit of the maximum number set per association, knowing that a first transmission has already taken place.

Resending takes place in the event of negative confirmation by the data link layer or in the event of positive confirmation if the acknowledgement arrives after the time interval during which it is expected.

10.6.2.5.7 Antiduplication (DU_SM)

During data transfers with acknowledgement in an association, the antiduplication element is implemented if adopted in the association. It is used conjointly with the retry element (see 10.6.2.5.6).

This element makes it possible to detect, at acceptor level, an ATRQ which has already been the subject of a data transfer with acknowledgement, as a number of repetitions of the same ATRQ can occur after a transmission fault.

a) The following events are accounted for by this mechanism:

- antiduplication_rq(Sequence number)

NOTE The acknowledged data transfer acceptor mechanism which invokes this mechanism transmits the sequence number available to the ATRQ for which it wants to make an antiduplication check.

b) The following actions are triggered by this mechanism:

- antiduplication_cnf(Message), where Message = REPETITION or NEW.

c) Mechanism:

There is a single instance of this mechanism per association at calling entity and called entity level.

This antiduplication mechanism is purely transformational and there is therefore no state machine but simply a description of the data and processing associated with it.

- Data description:

The processing is based on an “antiduplication list” containing information concerning the latest data transfers with acknowledgement which have taken place.

This antiduplication list is an LAD[i] structure table in which the number of elements is equal to the anticipation factor adopted for the association, and of which each structure features a sequence number and reception status associated with the transfer LAD[i] = (sn[i], Etr[i]).

At any given moment, the first element in the table has a sequence number that was the greatest (HSNA) of those received in the ATRQs for which the antiduplication mechanism was invoked. Consequently, this first element necessarily has reception status value YES.

The other elements contain successively smaller sequence numbers and a reception status with a value of YES or NO depending on whether the antiduplication mechanism has been invoked or not.

NOTE It is important to understand that the value of the sequence number allocated by the numbering element increases as modulo $2^{\exp(n)}$ where $n=16$, in view of 16-bit representation of the sequence number in the PDUs. See Table 804.

Table 804 – Structure of the antiduplication list

Sequence number	Reception status
sn(1)	YES
sn(2) where, $\text{dist}(\text{sn}(1), \text{sn}(2))=1$	YES/NO
sn(3) where, $\text{dist}(\text{sn}(1), \text{sn}(3))=2$	YES/NO
sn(a) where, $\text{dist}(\text{sn}(1), \text{sn}(a))=a-1$	YES/NO

- Processing description:

When the mechanism is invoked with $\text{antiduplication_rq}(\text{Sequence number})$, the value of the parameter SN sequence number governs the processing:

- When sn belongs to the antiduplication list, the nature of the processing in accordance with the value of the reception status associated with it in the list is as follows:
- if reception status is NO, it is set to YES and the message is considered as NEW,
- if reception status is YES, it stays at YES and the message is considered as a REPETITION.
- When it does not belong to the antiduplication list, it is considered to be new and the processing consists in offsetting the elements in the antiduplication list so that the value of sn(1) corresponds to the received SN sequence number.

The processing consists of three stages, after assessment of the distance $N = \text{dist}(\text{SN}, \text{sn}(1))$.

- if $N < a$, then for the elements in the LAD[i] list, where i decreases from a to N+1, make $\text{LAD}[i] = \text{LAD}[i-N]$. This constitutes shifting the elements of the structure by N cells.
- when i increases from 2 to $\text{Min}(N, a)$, make $\text{sn}[i] = M$ so that $\text{dist}(\text{SN}, M) = i-1$ and $\text{Etr}[i] = \text{NO}$. It is a question of setting to not received the reception status associated with the sequence numbers contained exclusively within the values of sn(1) and SN.
- make $\text{sn}(1) = \text{SN}$, and $\text{Etr}(1) = \text{YES}$. It is a question of setting the first element of the list to received.

10.6.2.5.8 Anticipation (MA_SM)

10.6.2.5.8.1 General

During transfer of data with acknowledgement in an association, the anticipation element is implemented if adopted in the association.

This element monitors the traffic in the association.

10.6.2.5.8.2 Requirements

Anticipation relates to transfers of data with acknowledgement in an association. It makes it possible to envisage commitment of a new acknowledged transfer when the acknowledgement of other current transfers has not yet arrived. This makes it possible to optimise the pass-band by multiplexing the data transfers and acknowledgements on the bus.

The anticipation possibilities offered by an association are linked both to the anticipation factor which characterises the association and to the anticipation credit which may temporarily reduce them.

10.6.2.5.8.3 Anticipation factor

Anticipation factor “a” corresponds to the maximum number of transfers which can be awaiting acknowledgement.

The anticipation factor “a” specific to each direction of transmission in an association establishes both:

- the maximum number of acceptor and requester elements for data transfer with acknowledgement which can be invoked simultaneously,
- and the size of the antiduplication list necessary at the acceptor in the event of implementation of retry and antiduplication.

10.6.2.5.8.4 Anticipation credit

The anticipation credit which is specific to each end of the association, establishes its capacity whether or not to allow invocation of new requester elements of data transfer with acknowledgement within the limits of the anticipation factor. See Figure 195.

The anticipation credit (AC) is characterised by two values:

- FREE, if it is possible to invoke an acknowledged data transfer requester element,
- BUSY, if it is temporarily impossible to invoke a requester element.

Anticipation credit changes between states depending on the following events:

- ma_rq()

Anticipation credit changes make allowance for two other variables:

- LSNS: the lowest sequence number attributed to an ATRQ sent and awaiting acknowledgement (Lowest Sequence Number Send),
- CN: the current value of the numbering entity for transfer of data with acknowledgement.

Machine: MA_SM

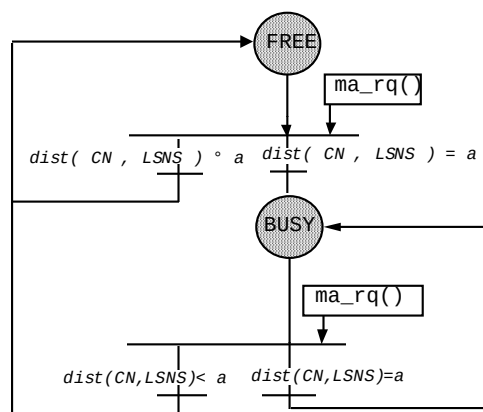


Figure 195 – Anticipation mechanism state machine

10.6.2.5.8.5 Optimum performance

For the anticipation factor adopted for a given association to constitute the optimum as regards the traffic adopted for it, it shall allow for the following factors:

- ensuring best use of the traffic adopted for the association.

NOTE 1 If it is too large, the maximum number of instances will never be reached, and if it is too low the pass-band will be wasted.

- ensure that the time necessary for chaining the maximum number of retries will be close to that necessary for passing the anticipation traffic.

NOTE 2 This ensures that the data transfer requester user will not be unable to access the association due to using up the anticipation credit by the current repetitions.

As for the user of the acknowledged data transfer service, it will choose an anticipation factor and a traffic which corresponds to these requirements in terms of:

- the capacity to relay a number of data transfer requests (A_DATA) in a given time.

NOTE 3 In the MMS environment, this capacity integrates both the “Outstanding” services and the time taken for execution of the services by the VMD.

- data transfer size, which in the case of segmentation implements a number of MCS transfers for different packets.

10.6.2.5.9 Segmentation (SG_SM)

A segmentation element can be implemented on an association. This element makes it possible to divide the data unit (SDU) associated with a data transfer service into one or more packets depending on its size, and to manage the acknowledged exchange of the different packets. The implementation of this element at data transfer initiator level is only possible conjointly with that of reassembly at receptor level. See Figure 196.

Global variables utilised by this mechanism:

- AC (Anticipation Credit), global for all association mechanisms,
- SA (Service Access) global for all association mechanisms
- LSNS (Last Smallest Sequence Number), the smaller sequence number attributed to a transfer currently exchanged.

Local variables of the mechanism:

- PN (Packet Number), the number of packets of an SDU remaining to be sent,
- LSN (Last Sequence Number), the sequence number of the next lowest value, in the modulo sense, to that attributed to the latest packet.

- FSN (First Sequence Number), the sequence number attributed to the first packet arising from segmentation,
- Count_ack, the received acknowledgement counter, relating to transfer of packets of a service.

a) This mechanism accounts for the following events and conditions:

- segment_rq(),
- “End of division (n)”, resulting from the division operation, where parameter (n) is the number of packets obtained,
- Always, permanent event to indicate that the transition is receptive at all times,
- dt_ack_cnf(+/-),
- AC = FREE, to account for the end of sending of packets belonging to the same SDU, or otherwise to indicate whether the packet is the last or not,
- PN > 1; PN = 1; PN = 0; to test for the end of sending of packages belonging to the same SDU, or otherwise to indicate whether the packet is the last or not,
- Count_ack = n, to verify that all the acknowledgements associated with the packets of a service have been received.

b) The following actions are triggered by this mechanism:

- Setting SA to BUSY or FREE,
- “SDU division request”,
- PN = n; initialisation of PN at value n of the number of packets resulting from division of the SDU,
- Packet nature = STT; for the first packet of a segmentation, or ALL if the number of packets arising from subdivision is 1,
- FSN=CN and LSN=x so that dist(x,CN)=n; initialisation of the variables FSN and LSN,
- Count_ack = 0, initialisation at zero of the acknowledgement counter,
- Count_ack = Count_ack+1, incrementation of the acknowledgement counter,
- dt_ack_rq(Packet nature), where Packet nature = MID, END,
- PN=PN-1; decrementation of the number of packets remained to be sent,
- Packet number = MID or END, depending on whether it is the last packet or not,
- segment_cnf(+/-),
- AT_SM_RQ revocation; revocation of all ATAK_SM_RQ machines of which the associated sequence number is between LSN (value excluded) and FSN (value included). It being impossible to transmit one of these packets, the operation becomes globally impossible and results in interruption of transmission of the packets not yet sent. Thus, the lowest sequence number during transmission (LSNS), will necessarily change to a value equal or greater than LSN.

NOTE 1 Revocation of the ATAK_SM_RQ is atomic with regard to the lifetime of all the machines of the association.

c) Mechanism:

For an association in which it is possible to implement segmentation, the number of instances is bounded by the maximum anticipation value specified in the association. Indeed, the number of possible instances is equal to the maximum anticipation if the SDUs exchanged only call a single packet, and otherwise less.

NOTE 2 The average number of instances of this mechanism in an association is equal to the maximum anticipation (a) divided by the average number of packets necessary for exchange of a service.

Machine: SG_SM

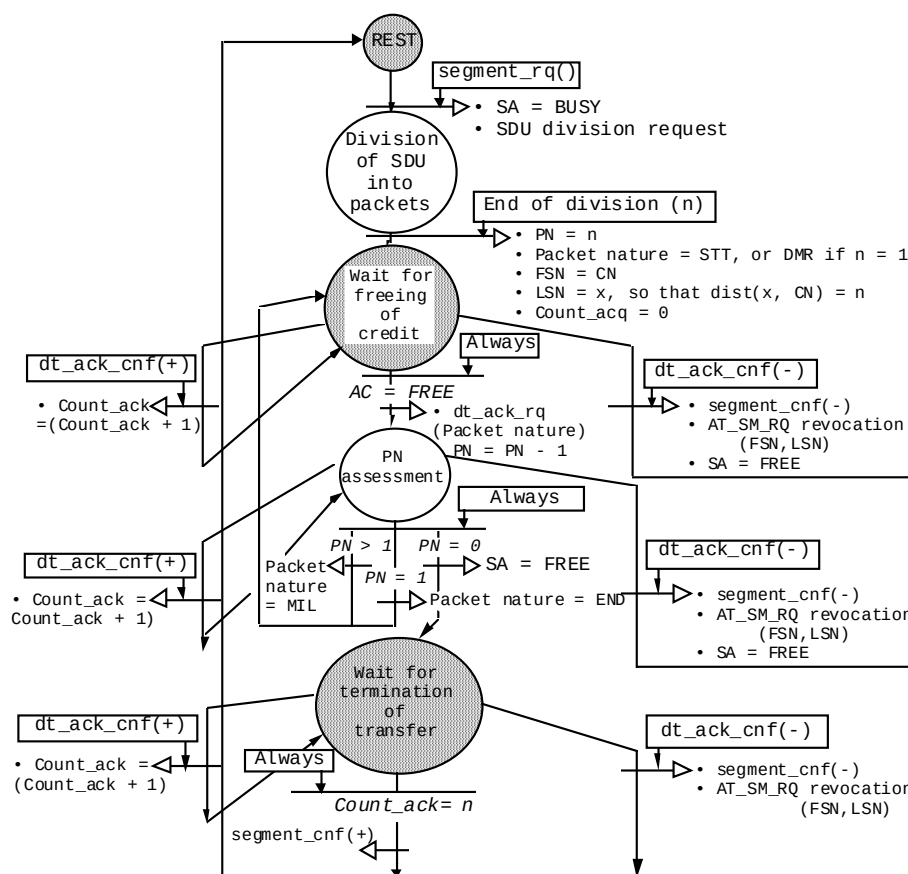


Figure 196 – Segmentation mechanism state machine

NOTE 3 The various mechanisms which can be implemented in an association can, in some cases, operate in parallel. It is therefore necessary to specify that after crossing of a transition, a mechanism retains the new target status until all the actions induced by the transition are terminated. Thus the SG_SM machine retains "PN assessment" status as long as dt_ack_rq(), then number taking, then ma_rq() have not been accounted for by machines ATAK_SM_RQ, NB_SM and MA_SM respectively.

d) Description of mechanism:

As soon as a data transfer service segmentation request is made, service access (SA) is inhibited until all the packets are assigned to an acknowledged data transfer machine. Service access is then freed, allowing new segmentation requests to be accounted for, even if transfer of the packets of the preceding request is not terminated.

Segmentation request processing always begins with division of the MCS-SDU into n packets, starting at the beginning. The different packets are the subject of individual acknowledged transfer. The first packet resulting from division is attributed packet nature STT (or ALL if the MCS-SDU is contained in a single packet) and an FSN sequence number. The following packets have a packet nature of MID, except for the last which has a packet nature of END.

During transmission of the different packets associated with the segmentation of the service, the segmentation machine passes the packets one by one as the acknowledged transfer machine invocations are freed. The availability of acknowledged data transfer machines can be monitored using anticipation credit value FREE.

In addition, if a transmission failure is detected by an acknowledged transfer machine, sending of the remaining packets to be sent is suspended and revocation of the acknowledged transfer machine is effected for each packet in the process of being transferred. This suspension makes it possible to immediately free transfer machines which become available for other SDU exchanges. The acknowledged transfer positive confirmation primitives are counted to detect determination of the transfer of all the packets, and in order to obtain positive confirmation of the segmentation request.

10.6.2.5.10 Reassembly (RS_SM)

A reassembly element can be implemented in an association. This element makes it possible to reconstitute the data associated with a data transfer service which has been received in a number of packets. Implementation of this element at data transfer receiver level can only be carried out conjointly with that of segmentation at initiator level.

This reassembly element is both reactive and transformational, it is consequently necessary to provide a state machine and describe the associated processing. See Figure 197.

a) Description of variables specific to the mechanism:

The data specific to this mechanism can be represented by a reassembly list as shown in Table 805. This reassembly list is an LR[i] structure table, in which the number of elements is equal to (a+s), where “a” is the association anticipation factor and “s” is the maximum number of MCS-PDUs for an MCS-SDU, in the context of an association. All structures of the table include a sequence number, the packet reassembly status associated with the sequence number as well as the packet nature $LR[i] = (sn[i], Etrs[i]$ and $NatP[i])$.

Reassembly status can be either:

- UNAVAILABLE, if the packet has not been received and therefore not restored to the user,
- AVAILABLE, if the packet has been received.

The packet nature is either:

- STT, first packet of SDU,
- MID, for intermediate packets of the SDU,
- END, for the last packet of the SDU,

At any given time, the sequence number of the first element of the table is the largest of all those received and has a reassembly status value which is necessarily other than available.

Each of the other elements in the table has a sequence number which decreases by one modulo increment relative to that preceding it, and one of the two possible reassembly status values.

- ALL, for a packet containing the entire SDU.

NOTE The packet nature cannot have the value NIL in the reassembly machine case.

Table 805 – Structure of the reassembly list

Sequence number	Reassembly status	Packet nature
sn(1)	AVAILABLE	STT/MID/END/ALL
sn(2) such that $\text{dist}(sn(1), sn(2)) = 1$	AVAIL/UNAVAIL	STT/MID/END/ALL
.....		
sn(a) such that $\text{dist}(sn(1), sn(a)) = a-1$	AVAIL/UNAVAIL	STT/MID/END/ALL
sn(a+1) such that $\text{dist}(sn(1), sn(a+1)) = a$	AVAIL/UNAVAIL	STT/MID/END/ALL
.....		
sn(a+s) such that $\text{dist}(sn(1), sn(a+s)) = a+s-1$	AVAIL/UNAVAIL	STT/MID/END/ALL

b) The following events and conditions are accounted for by the mechanism:

- dt_ack_ind(Packet nature, NS).
- “End of update”, “End of shift”, “End of operation”, corresponding to termination of the operations.

- Packet nature ≠ NIL (and) SN do not belong to “Reassembly list”, so as to be able to ascertain that the packet does in fact originate from a segmentation machine and that the sequence number belongs to the antiduplication window.
 - Packet nature ≠ NIL (and) SN do not belong to “Reassembly list”, so as to consider a sequence number greater than those received to date.
 - Packet nature = END (or) ALL, possibility of detecting an entire message.
 - Packet nature = STT (or) MID, impossible to detect an entire message.
 - SDU = COMPLETE or INCOMPLETE, reception of all or not all of the packets of a transfer.
- c) The following actions are triggered by the mechanism:
- Updating of “reassembly list”, “reassembly list” shift, “reassembly list” scanning, initiation of associated processing,
 - reassemb_ind.

d) Mechanism:

For an association in which it is possible to implement reassembly, a single instance of this machine is necessary.

Machine: RS_SM

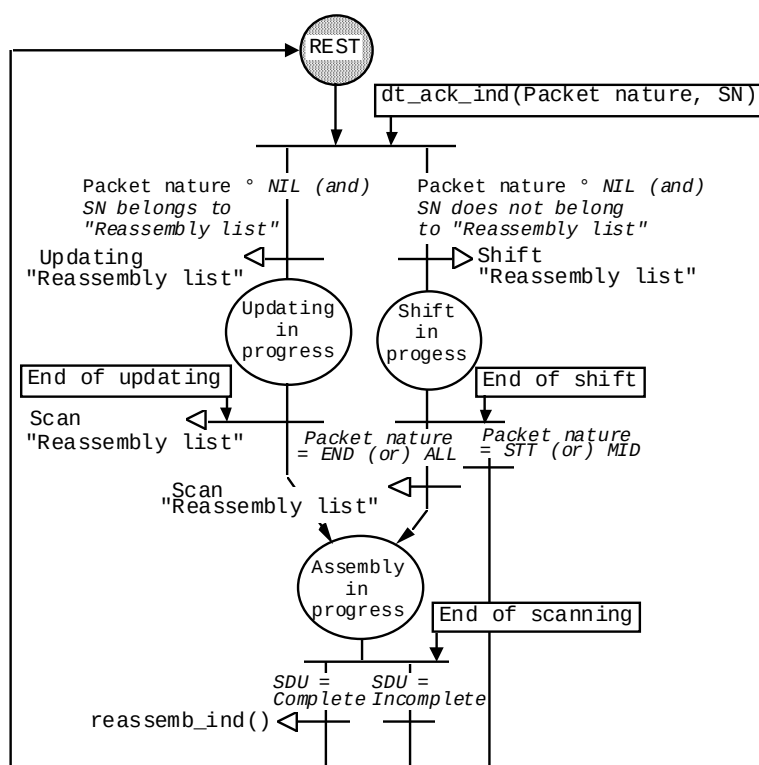


Figure 197 – Reassembly mechanism state machine

e) Description of mechanism:

When a packet is received, depending on whether or not its sequence number belongs to the reassembly list, “Update” or “Shift” processing is carried out on the reassembly list.

In the case where the assembly list shall be shifted whereas the packet nature is STT or MID, no further action is required. In all other cases, the list is scanned for complete SDUs, any found being restored to the user by a reassembly indication.

Description of the processing used by the machine:

- “Update”: Updating of the reassembly list consists of updating the “reassembly status” and “packet nature” fields of that element with the same sequence number as that received.

This updating consists of setting the packet nature to STT, MID, END or ALL depending on the value received, and reassembly status to AVAILABLE.

NOTE 1 The reassembly machine cannot receive the same packet twice as prior filtering is carried out by the antiduplication machine.

- “Shift”: This processing consists of shifting the elements in the reassembly list so that the value of $sn(1)$ corresponds to the SN sequence number received.

This processing involves the following operations, after evaluation of distance $N = dist(SN, sn(1))$.

- if $N < a+s$, then for all the elements of all the $LR[i]$ list, where i decreases from $a+s$ to $N+1$, set $LR[i] = LR[i-N]$. This shifts the elements of the structure by N cells.
- where i increases from 2 to $Min(N, a+s)$ set $sn[i] = M$ so that $dist(SN, M) = i-1$ and $Etrs[i] = UNAVAILABLE$. This is a question of setting to UNAVAILABLE the reception status associated with the sequence numbers contained exclusively within the values of $sn(1)$ and SN.
- set $sn(1) = SN$, and $Etrs(1) = AVAILABLE$, and $NatP(1) = Packet\ nature$. It is a question of setting the first element of the list to AVAILABLE while retaining its packet nature.
- “Scan”: Scanning consists of determining whether the packet received contains an ALL packet nature or scanning the reassembly list on either side of the packet received to find any series of consecutive numbers, marked available, bounded by two packets of packet natures END and STT respectively.
- This processing involves the following operations, after having identified m , such that $sn(m) = SN$:
- if $NatP(m) = ALL$, reconstitute the SDU.
- if $NatP(m) \neq ALL$, set $j = m$ and $Finexp = FALSE$, then perform the following operations until $Finexp \neq FALSE$ or $j = 0$:
- if $Etrs(j) = UNAVAILABLE$, set $Finexp = TRUE$,
- if $Etrs(j) = AVAILABLE$ and $NatP(j) = END$, then set $Finexp = CONTINUE$,
- $j = j + 1$.
- if $Finexp = CONTINUE$, set $k = m$, then perform the following operations until $Finexp \neq CONTINUE$ or $k = a+s$:
- $k = k+1$,
- if $Etrs(k) = UNAVAILABLE$, set $Finexp = TRUE$,
- if $Etrs(k) = AVAILABLE$ and $NatP(k) = STT$, then set $Finexp = TRUE$, reconstitute the SDU.

NOTE 2 The numbers of the first and last packet of the SDU are k and $j+1$ respectively.

10.6.2.6 Non-associated mode data transfer

10.6.2.6.1 General

Two protocol procedures can be implemented in the context of non-associated mode data transfer, depending on whether or not acknowledgement is requested. See Figure 198.

NOTE Non-associated mode data transfer mechanisms do not feature retry and antiduplication or segmentation and reassembly.

The following state machines are defined:

- NT_SM_RQ, NT_SM_ACC:
Non-associated mode unacknowledged transfer machine for requester and acceptor.
- NTAK_SM_RQ, NTAK_SM_ACC:
Non-associated mode acknowledged transfer machine for requester and acceptor.

In the non-associated mode, the state machines concerned with these transfers interact with both:

- the other application service elements available to the application entity. These are carried out by local identification.
- and the data link layer in the case where an ADAE type link address pair localises the pair of application entities communicating.

The following diagram shows all the state machines implemented by the non-associated transfers at the level of an application entity, and situates them relative to associated transfers of the same entity.

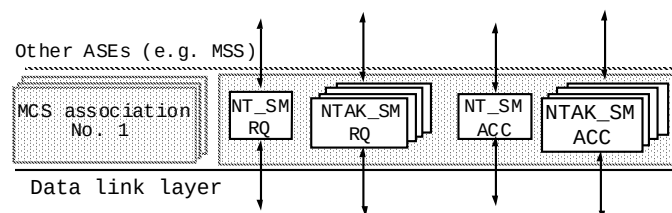


Figure 198 – Interaction of state machine in a non associated data transfer

This shows that an application entity can have a number of associations possessing their own machines, as well as a set of machines delegated to non-associated transfers which are shared by all the corresponding entities.

10.6.2.6.2 Unacknowledged transfer (NT_SM)

10.6.2.6.2.1 General

The unacknowledged non-associated mode data transfer procedure consists of two parts, relating to the requester and the acceptor respectively.

10.6.2.6.2.2 Requester element (NT_SM_RQ)

The requester element is that which causes the sending of a data transfer request from an application entity to a destination in another application entity. See Figure 199.

a) The following events and conditions are accounted for:

- A_UNIDATA.rq (Request_ack), Request_ack = FALSE.
- L_MSG_XX.cnf(+/-) (See IEC 61158-3 Type-7)

b) The following actions are performed:

- L_MSG_XX.rq(NTRQ[noack]) (See IEC 61158-3 Type-7)
- A_UNIDATA.cnf(+/-).

c) Mechanism:

An application entity in which it is possible to implement the unacknowledged data transfer procedure supports one or possibly a number of instances of the mechanism for local anticipation purposes at device level.

Machine: NT_SM_RQ

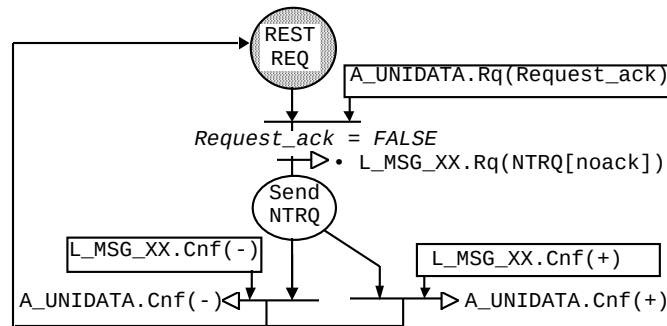


Figure 199 – Unacknowledged transfer: Requester element state machine

d) Description of mechanism:

As soon as an A_UNIDATA.rq(Request_ack) unacknowledged data transfer request is made where (Request_ack = FALSE) with a target application entity, a data transfer (NTRQ) is sent.

Whatever the results of the send, the mechanism returns to its initial state.

10.6.2.6.2.3 Acceptor element (NT_SM_ACC)

The acceptor element is that which receives a data transfer for which acknowledgement is not requested and transfers it to the user. See Figure 200.

a) The following events and conditions are accounted for:

- L_MSG_XX.ind(NTRQ[noack]) (See IEC 61158-3 Type-7)

b) The following actions are carried out by the acceptor element:

- A_UNIDATA.ind (Request_ack = FALSE)

c) Acceptor element mechanism:

An application entity in which it is possible to implement the unacknowledged data transfer procedure supports one or possibly a number of instances of the mechanism for local anticipation purposes.

Machine: NT_SM_ACC

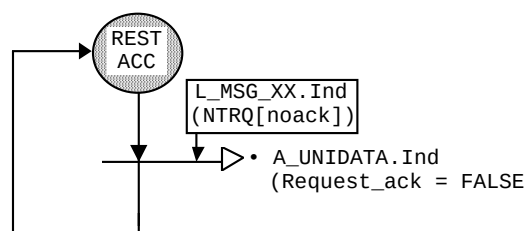


Figure 200 – Unacknowledged transfer: Acceptor element state machine

d) Description of mechanism:

On receiving an L_MSG_XX.ind() primitive whose data correspond to an NTRQ for which acknowledgement is not requested, when the application entity has the resources necessary for instantiation of the mechanism, the primitive A_UNIDATA.ind(Request_ack = FALSE) is invoked.

NOTE The resources associated with a mechanism are, indeed, shared between all the correspondents of the application entity.

10.6.2.6.3 Acknowledged transfer (NTAK_SM)**10.6.2.6.3.1 General**

The acknowledged non-associated mode data transfer procedure consists of two parts, relating respectively to the requester and the acceptor.

10.6.2.6.3.2 Requester element (NTAK_SM_RQ)

The requester element is that which causes the sending of a data transfer request from one application entity to a destination in another application entity and which receives the corresponding acknowledgement. See Figure 201.

a) The following events and conditions are accounted for:

- A_UNIDATA.rq (Request_ack = TRUE),
- L_MSG_XX.cnf(+/-) (See IEC 61158-3 Type-7),
- reception of an AKNT in the data of the L_MSG_XX.ind primitive containing an acknowledgement request,
- acknowledgement time-out.

b) The following actions are carried out by the requester element:

- calling of the “Identification” function to assign an invocation identification to each transfer,
- L_MSG_XX.rq(NTRQ[ack]) (See IEC 61158-3 Type-7),
- starting of acknowledgement time-out,
- A_UNIDATA.cnf(+/-).

c) Requester element mechanism:

For an application entity in which it is possible to implement the acknowledged data transfer procedure, a number of instances of the mechanism can exist simultaneously with one or more per destination application entity.

Machine: NTAK_SM_RQ

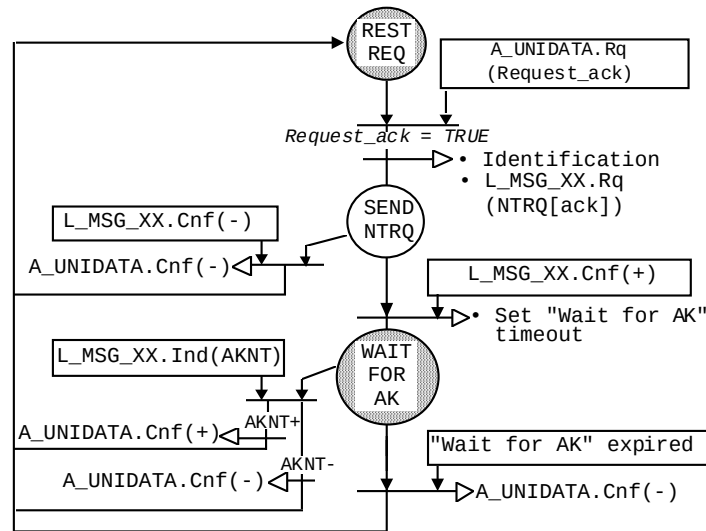


Figure 201 – Acknowledged transfer: Requester element state machine

d) Description of mechanism:

As soon as an acknowledged data transfer is requested, an NTRQ is sent.

When the result of the send returned by the data link layer is positive, an acknowledgement time-out is started. Either the expected acknowledgement arises and the mechanism returns to its initial state after returning a confirmation to the user which can be positive or negative depending on whether or not the acknowledgement received is positive, or the acknowledgement time-out expires and a negative confirmation is sent to the user.

10.6.2.6.3.3 Acceptor element (NTAK_SM_ACC)

The acceptor element is that which receives the data transfer and which sends the corresponding acknowledgement, and relays the data to the user. See Figure 202.

a) The following events and conditions are accounted for:

- L_MSG_XX.ind(NTRQ[ack]) (See IEC 61158-3 Type-7)
- L_MSG_XX.cnf(+/-) (See IEC 61158-3 Type-7)

b) The following actions are carried out by the requester element:

- L_MSG_XX.rq(AKNT[+/-]) (See IEC 61158-3 Type-7)
- A_UNIDATA.ind(Request_ack = TRUE)

c) Acceptor element mechanism:

For an entity in which it is possible to implement the acknowledged data transfer procedure, a number of instances of the machine can exist simultaneously with one or more per source entity.

Machine: NTAK_SM_ACC

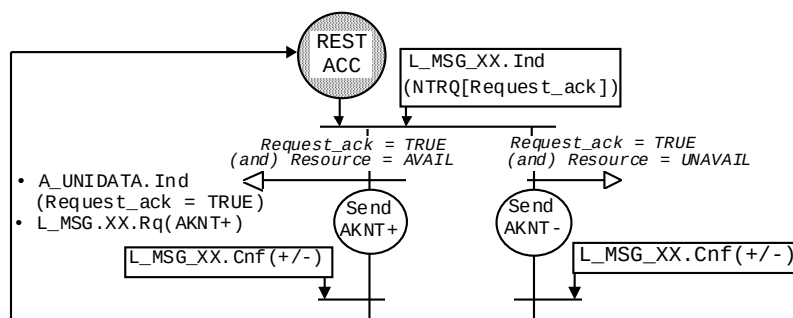


Figure 202 – Acknowledged transfer: Acceptor element state machine

d) Description of mechanism:

As soon as the L_MSG_XX.ind primitive, the data of which corresponds to an NTRQ with an acknowledgement request, the data transfer sender is returned an acknowledgement containing a sequence number equal to that of the acknowledged data transfer.

Depending on whether or not the message can be supplied to the user, the acknowledgement return is positive or negative. The various reasons preventing an application entity from returning data to the user are modelled by the resources not being available.

Finally, whether or not there is a failure during sending of the acknowledgement at link level, the mechanism returns to its initial state.

10.6.2.6.4 Invocation identification

When using the acknowledged data transfer service with a pair of application entities, the identification function supplies an invocation identification which transits with the data in the transfers using the NTRQ type PDU, and is also found in the corresponding acknowledgements of which the PDU is of the AKNT type.

There is a single instance of this mechanism per application entity. The value of the invocation identification is random, provided it makes it possible to discriminate between the different transfers pending acknowledgement in the same pair of application entities.

NOTE There is no special initial value or rules for changing the value of the invocation identification.

10.6.2.6.5 Anticipation

Multiple anticipation is authorised in the case of acknowledged data transfers, provided the resources are available at the requester and the acceptor.

10.7 Protocol options

10.7.1 Conformance classes

10.7.1.1 General

Conformance classes in AL define various levels of quality of service. These levels are qualified in terms of the services provided by a device and in terms of the supported mechanisms. Three series of parameters are used to define the conformance class. These characterize the horizontal, vertical and type conformance.

10.7.1.2 Vertical conformance

10.7.1.2.1 General

Vertical conformance describes the application level services supported by a device.

To define the device vertical conformance there are three independent parameters on which some constraints apply. A set of values of those parameters defines a vertical conformance block. Each parameter is representative of a functionality or group of functionalities provided by the application level services, and it has a set of values featuring the ability of the device to support these functionalities.

The values of those parameters are set in upward compatibility. So, a device supporting the services characterized by the parameter value **n** will also support all the functionalities provided by services characterized by values inferior to **n** of this same parameter.

For the AL TYPE 7 application layer, the parameters are:

10.7.1.2.2 Read / write parameter: PV_R/W

This parameter defines the various read / write possibilities supported by the device depending on of the provided services. See Table 806. It can take the following values.

PV_R/W[1] Local write Service (Primitive A_WRITELOC)

PV_R/W[2] is equal to PV_R/W[1] plus local read Service (Primitive A_READLOC)

PV_R/W[3] is equal to PV_R/W[2] plus update Service (Primitive A_UPDATE)

PV_R/W[4] is equal to PV_R/W[3] plus remote read and write Services (Primitives A_READFAR plus A_WRITEFAR)

PV_R/W[5] is equal to PV_R/W[4] plus universal read and write Services (Primitives A_READ and A_WRITE).

Table 806 – PV_R/W parameter values

Parameter PV_R/W	1	2	3	4	5
A_WRITELOC	X	X	X	X	X
A_READLOC		X	X	X	X
A_UPDATE			X	X	X
A_WRITEFAR				X	X
A_READFAR				X	X
A_WRITE					X
A_READ					X

10.7.1.2.3 Indication parameter: PV_IND

This parameter defines the various possibilities of the indication services supported by the application layer. See Table 807. It can take the following values:

PV_IND[0] No indication service supported

PV_IND[1] Reception and transmission indication services (Primitives A_SENT and A_RECEIVED)

Table 807 – PV_IND parameter values

Parameter PV_IND	0	1
A_SENT		X
A_RECEIVED		X

10.7.1.2.4 List service parameter: PV_LIS

This parameter defines the access services to variable lists supported by the application layer. See Table 808.

PV_LIS[0] No list access services.

PV_LIS[1] Read variable list service (Primitive A_READLIST)

Table 808 – PV_LIS parameter values

Parameter PV_LIS	0	1
A_READLIST		X

10.7.1.2.5 Constraints on parameters**10.7.1.2.5.1 General**

For each parameter, a table specifies the constraints by giving the minimum value of the other parameters to which this parameter is linked.

10.7.1.2.5.2 Parameter PV_R/W

No constraint.

10.7.1.2.5.3 Parameter PV_IND

No constraint.

10.7.1.2.5.4 Parameter PV_LIS

See Table 809 for the definitions.

Table 809 – Constraints on PV_LIS parameter

Constraints on PV_LIS	0	1
PV_R/W		PV_R/W[2]

10.7.1.2.5.5 Vertical Conformance parameters for Sub-MMS AEs

The vertical parameters make it possible to specify the subset of primitives of services offered.

a) See Table 810 for the definitions of the associated mode data transfer parameter.

Table 810 – PV_AT parameter values

PV_AT	1	2	3	
A_ASSOCIATE			X	
A_ABORT			X	
A_DATA		X	X	

b) See Table 811 for the definitions of the association termination parameter.

Table 811 – PV_RE parameter values

PV_RE	1	2		
A_RELEASE		X		

c) See Table 812 for the definitions of the non-association mode data transfer parameter.

Table 812 – PV_UT parameter values

PV_UT	1	2		
A_UNIDATA		X		

See Table 813 for the expression of the constraints which remain between the various vertical parameters.

Table 813 – Constraints on PV_RE parameter

Constraints on PV_RE	1	2		
PV_AT		[3]		

10.7.1.3 Horizontal conformance

10.7.1.3.1 Asynchronous refreshment parameter: PH_R_A

This parameter, as shown in Table 814, describes the asynchronous refreshment status elaboration mechanisms that are supported by an information producer. It can take the following values:

PH_R_A[0] No asynchronous refreshment status is elaborated.

PH_R_A[1] A mechanism to elaborate the asynchronous refreshment status is implemented.

Table 814 – PH_R_A parameter values

Parameter PH_R_A	0	1
Asynchronous refreshment		X

10.7.1.3.2 Synchronous refreshment parameter: PH_R_S

This parameter, as shown in Table 815, describes the synchronous refreshment status elaboration mechanisms that are supported by an information producer. It can take the following values:

PH_R_S[0] No synchronous refreshment status is elaborated.

PH_R_S[1] A mechanism to elaborate the synchronous refreshment status is implemented.

Table 815 – PH_R_S parameter values

Parameter PH_R_S	0	1
Synchronous refreshment		X

10.7.1.3.3 Punctual refreshment parameter: PH_R_P

This parameter, as shown in Table 816, describes the punctual refreshment status elaboration mechanisms, supported by the application layer of an information producer. It can take the following values:

PH_R_P[0] No punctual refreshment status is elaborated.

PH_R_P[1] A mechanism to elaborate the punctual refreshment status is implemented.

Table 816 – PH_R_P parameter values

Parameter PH_R_P	0	1
Punctual refreshment		X

10.7.1.3.4 Asynchronous promptness parameter: PH_P_A

This parameter, as shown in Table 817, describes the asynchronous promptness status elaboration mechanisms that are supported by an information producer. It can take the following values:

PH_P_A[0] No asynchronous promptness status is elaborated.

PH_P_A[1] A mechanism to elaborate the asynchronous promptness status is implemented.

Table 817 – PH_P_A parameter values

Parameter PH_P_A	0	1
Asynchronous promptness		X

10.7.1.3.5 Synchronous promptness parameter: PH_P_S

This parameter, as shown in Table 818, describes the synchronous promptness status elaboration mechanisms that are supported by an information producer. It can take the following values:

PH_P_S[0] No synchronous promptness status is elaborated.

PH_P_S[1] A mechanism to elaborate the synchronous promptness status is implemented.

Table 818 – PH_P_S parameter values

Parameter PH_P_S	0	1
Synchronous promptness		X

10.7.1.3.6 Punctual promptness parameter: PH_P_P

This parameter, as shown in Table 819, describes the variable punctual promptness status elaboration mechanisms supported by the application layer of an information consumer. It can take the following values:

PH_P_P[0] No punctual promptness status is elaborated.

PH_P_P[1] A mechanism to elaborate the punctual promptness status is implemented.

Table 819 – PH_P_P parameter values

Parameter PH_P_P	0	1
Punctual promptness		X

10.7.1.3.7 Variables list consistency parameter: PH_COH

This parameter, as shown in Table 820, describes the various production or transmission status elaboration mechanisms associated with a variable list at the level of a consumer entity. It can take the following values:

PH_COH[0] No consistency mechanism is supported

PH_COH[1] Consistency status elaboration mechanisms implemented

Table 820 – PH_COH parameter values

Parameter PH_COH	0	1
Consistency		X

10.7.1.3.8 List reliability parameter: PH_FIA

This parameter, as shown in Table 821, describes the various mechanisms supported to make the variable transmission reliable at the consuming application entity level.

PH_FIA[0] No reliability mechanism is supported

PH_FIA[1] List recovery mechanism implemented

Table 821 – PH_FIA parameter values

Parameter PH_FIA	0	1
Recovery		X

10.7.1.3.9 Transitory spatial consistency status: PH_SPF

This parameter, as shown in Table 822, describes the elaboration mechanism of the contents of the produced consistency variable. This variable is used in the elaboration of the transitory spatial consistency status of the lists of variables at the consumer application entity level.

PH_SPF[0] No transitory spatial consistency mechanism is supported

PH_SPF[1] A mechanism of transitory spatial consistency on lists is implemented

Table 822 – PH_SPF parameter values

Parameter PH_SPF	0	1
Transitory spatial consist.		X

10.7.1.3.10 Permanent spatial consistency parameter: PH_SPM

This parameter, as shown in Table 823, describes the contents elaboration of the produced consistency variable mechanism. This variable is used in the elaboration of the permanent spatial consistency status of the lists of variables at the consumer application entity level.

PH_SPM[0] No permanent spatial consistency mechanism is supported

PH_SPM[1] A mechanism of permanent spatial consistency on lists is implemented

Table 823 – PH_SPM parameter values

Parameter PH_SPM	0	1
Permanent spatial consist.		X

10.7.1.3.11 Access parameter: PH_ACC

This parameter, as shown in Table 824, describes the various variable access modes. It can take the following values:

PH_ACC[0] Application variable global access

PH_ACC[1] is equal to PH_ACC[0] plus variable partial access

PH_ACC[2] is equal to PH_ACC[1] plus variable renamed access

Table 824 – PH_ACC parameter values

Parameter PH_ACC	0	1	2
Partial access		X	X
Renamed access			X

10.7.1.3.12 Resynchronization parameter: PH_RES

This parameter, as shown in Table 825, defines whether or not the resynchronization mechanism is supported by the application layer. It can take the following values:

PH_RES[0] The resynchronization mechanism is not supported

PH_RES[1] The resynchronization mechanism is supported

Table 825 – PH_RES parameter values

Parameter PH_RES	0	1
Resynchronization		X

10.7.1.3.13 Horizontal conformance parameters for Sub-MMS ASEs

The horizontal parameters make it possible to specify the various procedures supported.

a) Acknowledgement parameter: See Table 826.

Table 826 – PH_AK parameter values

PH_AK	1	2		
AT_SM		C		
NT_SM		C		
NB_SM		C		
MA_SM		C		

As soon as PV_AT and PV_UT have value 2, if PH_AK has the value 2, they shall offer the acknowledged transfer.

b) Retry/antiduplication parameter: See Table 827.

Table 827 – PH_RA parameter values

PH_RA	1	2		
RC_SM		X		
DU_SM		X		

c) Segmentation/reassembly parameter: See Table 828.

Table 828 – PH_SR parameter values

PH_SR	1	2		
SG_SM		X		
RS_SM		X		

d) Flow control parameter: See Table 829.

Table 829 – PH_CF parameter values

PH_CF	1	2		
Flow control		X		

Expression of the constraints which remain between the various horizontal parameters. See Table 830 and Table 831.

Table 830 – Constraints on PH_RA parameter

Constraints on PH_RA	1	2		
PH_AK		[2]		

Table 831 – Constraints on PH_SR parameter

Constraints on PH_SR	1	2		
PH_AK		[2]		

10.7.1.4 Type conformance

10.7.1.4.1 General

The set of types available for the AL TYPE 7 application variables is issued from all the possible instances of the Type Constructor object. Within this set are defined a certain number of subsets related to the various categories of variables handled by a device. The conformance type blocks define the supported types subsets on a device.

10.7.1.4.2 Primitive type parameters

10.7.1.4.2.1 Octet string parameter PT_OCT

This parameter specifies whether or not the device supports the octet string type. See Table 832.

PT_OCT[0] Octet string type not supported

PT_OCT[1] Octet string type supported

Table 832 – PT_OCT parameter values

Parameter PT_OCT	0	1
Octet string		X

10.7.1.4.2.2 Binary string parameter PT_BIN

This parameter specifies whether or not the device supports the binary string type. See Table 833.

PT_BIN[0] Binary string type not supported

PT_BIN[1] Binary string type supported

Table 833 – PT_BIN parameter values

Parameter PT_BIN	0	1
Bit string		X

10.7.1.4.2.3 Visible string parameter PT_VIS

This parameter specifies whether or not the device supports the visible string type. See Table 834.

PT_VIS[0] Visible string type not supported

PT_VIS[1] Visible string type supported

Table 834 – PT_VIS parameter values

Parameter PT_VIS	0	1
Visible string		X

10.7.1.4.2.4 Boolean parameter PT_BOO

This parameter specifies whether or not the device supports the boolean type. See Table 835.

PT_BOO[0] Boolean type not supported

PT_BOO[1] Boolean type supported

Table 835 – PT_BOO parameter values

Parameter PT_BOO	0	1
Boolean		X

10.7.1.4.2.5 BCD parameter PT_BCD

This parameter specifies whether or not the device supports the BCD type. See Table 836.

PT_BCD[0] BCD type not supported

PT_BCD[1] BCD type supported

Table 836 – PT_BCD parameter values

Parameter PT_BCD	0	1
BCD		X

10.7.1.4.2.6 Binary time parameter PT_BTM

This parameter specifies whether or not the device supports the binary time type. See Table 837.

PT_BTM[0] Binary time type not supported

PT_BTM[1] Binary time type supported

Table 837 – PT_BTM parameter values

Parameter PT_BTM	0	1
Binary time		X

10.7.1.4.2.7 Integer parameter PT_INT

This parameter specifies whether or not the device supports the integer type. See Table 838.

PT_INT[0] Integer type not supported

PT_INT[1] Integer type supported

Table 838 – PT_INT parameter values

Parameter PT_INT	0	1
Integer		X

10.7.1.4.2.8 Unsigned parameter PT_UNUS

This parameter specifies whether or not the device supports the unsigned type. See Table 839.

PT_UNUS[0] Unsigned type not supported

PT_UNUS[1] Unsigned type supported

Table 839 – PT_UNUS parameter values

Parameter PT_UNUS	0	1
Unsigned		X

10.7.1.4.2.9 Floating point parameter PT_FPT

This parameter specifies whether or not the device supports the floating point type. See Table 840.

PT_FPT[0] Floating point type not supported

PT_FPT[1] Floating point type supported

Table 840 – PT_FPT parameter values

Parameter PT_FPT	0	1
Floating point		X

10.7.1.4.2.10 Generalized time PT_GTM

This parameter specifies whether or not the device supports the generalized time type. See Table 841.

PT_GTM[0] Generalized time type not supported

PT_GTM[1] Generalized time type supported

Table 841 – PT_GTM parameter values

Parameter PT_GTM	0	1
Generalised time		X

10.7.1.4.3 Array type parameter PT_TAB

This parameter describes the various possibilities for a device to implement the array type. See Table 842. It can take the following values:

PT_TAB[0] Array type not supported

PT_TAB[1] Array composed of primitive types

PT_TAB[2] is equal to PT_TAB[1] plus complex structured type arrays (array, structures)

Table 842 – PT_TAB parameter values

Parameter PT_TAB	0	1	2
Primitive type array		X	X
Structured type array			X

10.7.1.4.4 Structure type parameter PT_STR

This parameter describes the several possibilities for a device to implement the structure type. See Table 843. It can take the following values:

PT_STR[0] Structure type not supported

PT_STR[1] Structure composed of primitive types

PT_STR[2] is equal to PT_STR[1] plus complex structured type structures (array, structures)

Table 843 – PT_STR parameter values

Parameter PT_STR	0	1	2
Primitive type structure		X	X
Complex type structure			X

10.7.1.4.5 Constraints on parameters

10.7.1.4.5.1 General

For each parameter, a table specifies the constraints by giving the minimum value of the other parameters to which it is linked.

10.7.1.4.5.2 Parameter PT_TAB

See Table 844.

Table 844 – Constraints on PT_TAB parameter

Constraints on PT_TAB	0	1	2
PT_TAB			PT_TAB[1]

10.7.1.4.5.3 Parameter PT_STR

See Table 845.

Table 845 – Constraints on PT_STR parameter

Constraints on PT_STR	0	1	2
PT_STR			PT_STR[1]

There are no particular constraints on the other parameters specifying type conformity.

10.7.1.5 Protocol Implementation Conformance statement

10.7.1.5.1 General

This part of the document describes the "Protocol Implementation Conformance Statement Proforma (PICS). Each implementation should fill all the PICS.

- 1- Part 1 of the PICS shall indicate the information relative to the implementation and to the system.
- 2- Part 2 of the PICS shall indicate the CBB services implemented
- 3- Part 3 of the PICS shall indicate the CBB services and the values implemented.
- 4- Part 4 of the PICS shall indicate the "local implementations values".

10.7.1.5.2 PICS Part 1 :

PICS Serial Number :

Date:

Vendor's Name,

Model Name,

Revision Identifier,

Machine Name(s) and Version Number(s),

Operating System(s),

Sub-mms Abstract syntax,

Version Number of the Sub-MMS Abstract Syntax Supported (minor number),
CS Sub-mms abstract syntax,

Version Number of the CS Sub-MMS Abstract Syntax Supported (minor number),

Calling (yes or no),

Called (yes or no),

List of standardized names.

10.7.1.5.3 PICS part 2 :

PICS Serial Number :

Date issued :

Initiate,

(server,client or both)

Conclude,

Unsolicited Status,

Identify

Status,

Get name list,

Delete domain,

Initiate download sequence,

Download segment,

Terminate download sequence,

Initiate upload sequence,

Upload segment,

Terminate upload sequence,

Get domain attributes,

Create program invocation,

Delete program invocation,

Start,

Stop,

Resume,

Reset,

Kill,

Get program invocation attributes,

Read,

Write,

Get variable access attributes,

Information report,

Define variable list,

Delete variable list,

Get variable list attributes,

Event notification,

Get alarm summary,

Alter event condition monitoring,

Acknowledge event notification,

Get event condition attributes.

Generic Init Download
 Generic Download
 Generic Terminate Download
 Get OD Header/Data Type Request

10.7.1.5.4 PICS part 3 :

PICS Serial number : Date :
 Object identification by name
 Object identification by index
 Recursivity of variable data
 variable-list
 Partial access

10.7.1.5.5 PICS part 4 :

PICS Serial number : Date :
 List of Data Types supported with limit values
 Max time-out value in seconds,
 Time with system-clock,
 Granularity of time in milliseconds,
 Uninterruptible-access to variables,
 Local detail & execution argument for all services,
 Max outstanding calling,
 Max outstanding called,
 Load data format for Download segment,
 Max number of simultaneous upload.
 Max number of variables read in one service invocation (1 to n)
 Max number of variables written in one service invocation (1 to n)
 Max number of variables notified in one service invocation (1 to n)
 Max number of events notified in one service invocation (0 to n)
 Max number of events read in one service invocation (1 to n)
 Maximum number of acknowledged event objects (1 to n)
 Maximum number of modified event objects (1 to n)
 Object Access Protection (supported or not)

10.7.1.5.6 Minimum conformance classes

The following conformance classes concern only the SERVER devices.

The minimum conformance classes are listed by management types.

The types of management are:

- a) association management,
- b) VMD management,
- c) domain management,
- d) program invocation management,

- e) variable management,
- f) event management.

There are 5 conformance classes for each type of management. Each of these conformance classes defines the services, the data types, the complex variable objects and the obligatorily supported mechanisms.

The conformance class of a device is composed of 6 values. Each value corresponds to the conformance class of one type of management. These 6 values can be different from one another.

It is not forbidden to have a device belonging to one conformance class services, data types, complex variable objects and mechanisms not obligatory for its class.

1- Conformance classes for environment management: See Table 846.

Table 846 – Conformance classes for environment management

Services	Class 1	Class 2	Class 3	Class 4	Class 5
initiate				M	M
conclude					M
abort				M	M
status					
unsolicited status					
identify					
get name list					
initiate download sequence					
download segment					
terminate download sequence					
initiate upload sequence					
upload segment					
terminate upload sequence					
delete domain					
get domain attributes					
create program invocation					
delete program invocation					
start					
stop					
resume					
reset					
kill					
get program invocation attributes					
read					
write					
information report					
define variable list					
delete variable list					
get variable access attributes					
get variable list attributes					
event notification					
acknowledge event notification					
alter event condition monitoring					
get alarm summary					
get event condition attributes					
Variable Data Type					
simple data					
structured data (arr/str)					
Complex Variable Objects					
Variable-list					
Mechanisms					
partial access to the variables					
controlled access by protections					
association(predef/negotiated)		M	M	M	M
without association	M		M	M	M

2- Conformance classes for VMD management: See Table 847.

Table 847 – Conformance classes for VMD management

Services	Class 1	Class 2	Class 3	Class 4	Class 5
initiate					
conclude					
abort					
status		M	M	M	M
unsolicited status				M	M
identify			M	M	M
get name list					M
initate download sequence					
download segment					
terminate download sequence					
initiate upload sequence					
upload segment					
terminate upload sequence					
delete domain					
get domain attributes					
create program invocation					
delete program invocation					
start					
stop					
resume					
reset					
kill					
get program invocation attributes					
read					
write					
information report					
define variable list					
delete variable list					
get variable access attributes					
get variable list attributes					
event notification					
acknowledge event notification					
alter event condition monitoring					
get alarm summary					
get event condition attributes					
Variable Data Type					
simple data					
structured data (arr/str)					
Complex Variable Objects					
Variable-list					
Mechanisms					
partial access to the variables					
controlled access by protections					
association(predef/negotiated)					
without association					

3- Conformance classes for domain management: See Table 848.

Table 848 – Conformance classes for PI management

Services	Class 1	Class 2	Class 3	Class 4	Class 5
initiate					
conclude					
abort					
status					
unsolicited status					
identify					
get name list					
initiate download sequence		M	M	M	M
download segment		M	M	M	M
terminate download sequence		M	M	M	M
initiate upload sequence			M	M	M
upload segment			M	M	M
terminate upload sequence			M	M	M
delete domain				M	M
get domain attributes					M
create program invocation					
delete program invocation					
start					
stop					
resume					
reset					
kill					
get program invocation attributes					
read					
write					
information report					
define variable list					
delete variable list					
get variable access attributes					
get variable list attributes					
event notification					
acknowledge event notification					
alter event condition monitoring					
get alarm summary					
get event condition attributes					
Variable Data Type					
simple data					
structured data (arr/str)					
Complex Variable Objects					
Variable-list					
Mechanisms					
partial access to the variables					
controlled access by protections					M
association(predef/negotiated)					
without association					

4- Conformance classes for PI management: See Table 849.

Table 849 – Conformance classes for domain management

Services	Class 1	Class 2	Class 3	Class 4	Class 5
initiate					
conclude					
abort					
status					
unsolicited status					
identify					
get name list					
initate download sequence					
download segment					
terminate download sequence					
initiate upload sequence					
upload segment					
terminate upload sequence					
delete domain					
get domain attributes					
create program invocation				M	M
delete program invocation				M	M
start		M	M	M	M
stop		M	M	M	M
resume		M	M	M	M
reset			M	M	M
kill				M	M
get program invocation attributes					M
read					
write					
information report					
define variable list					
delete variable list					
get variable access attributes					
get variable list attributes					
event notification					
acknowledge event notification					
alter event condition monitoring					
get alarm summary					
get event condition attributes					
Variable Data Type					
simple data					
structured data (arr/str)					
Complex Variable Objects					
Variable-list					
Type of access					
partial access to the variables					
controlled access by protections					M
association(predef/negotiated)					
without association					

5- Conformance classes for the variable/variable-list management: See Table 850.

Table 850 – Conformance classes for variable/variable list management

Services	Class 1	Class 2	Class 3	Class 4	Class 5
initiate					
conclude					
abort					
status					
unsolicited status					
identify					
get name list					
initiate download sequence					
download segment					
terminate download sequence					
initiate upload sequence					
upload segment					
terminate upload sequence					
delete domain					
get domain attributes					
create program invocation					
delete program invocation					
start					
stop					
resume					
reset					
kill					
get program invocation attributes					
read	M	M	M	M	M
write	M	M	M	M	M
information report	M	M	M	M	M
define variable list				M	M
delete variable list				M	M
get variable access attributes					M
get variable list attributes					M
event notification					
acknowledge event notification					
alter event condition monitoring					
get alarm summary					
get event condition attributes					
Variable Data Type					
simple data	M	M	M	M	M
structured data (arr/str)		M	M	M	M
Complex Variable Objects					
Variable-list				M	M
Type of access					
partial access to the variables			M	M	M
controlled access by protections					M
association(predef/negotiated)					
without association					

6- Conformance classes for event management: See Table 851.

Table 851 – Conformance classes for event management

Services	Class 1	Class 2	Class 3	Class 4	Class 5
initiate					
conclude					
abort					
status					
unsolicited status					
identify					
get name list					
initate download sequence					
download segment					
terminate download sequence					
initiate upload sequence					
upload segment					
terminate upload sequence					
delete domain					
get domain attributes					
create program invocation					
delete program invocation					
start					
stop					
resume					
reset					
kill					
get program invocation attributes					
read					
write					
information report					
define variable list					
delete variable list					
get variable access attributes					
get variable list attributes					
event notification		M	M	M	M
acknowledge event notification			M	M	M
alter event condition monitoring				M	M
get alarm summary					M
get event condition attributes					M
Variable Data Type					
simple data					
structured data (arr/str)					
Complex Variable Objects					
Variable-list					
Mechanisms					
partial access to the variables					
controlled access by protections					M
association(predef/negotiated)					
without association					

11 Type 8

11.1 FAL Syntax Description

11.1.1 FAL-AR PDU Abstract Syntax

11.1.1.1 Top Level Definition

The productions defined here shall be used with the Peripherals Encoding Rules (see 11.2.2) for APDU encoding.

```
APDU ::= CHOICE {
    [PRIVATE 0] ConfirmedSend-RequestPDU,
    [PRIVATE 1] ConfirmedSend-ResponsePDU,
    [PRIVATE 2] UnconfirmedSend-PDU,
    [PRIVATE 3] UnconfirmedAcknowledgedSend-CommandPDU,
    [PRIVATE 4] Establish-RequestPDU,
    [PRIVATE 5] Establish-ResponsePDU,
    [PRIVATE 6] Establish-ErrorPDU,
    [PRIVATE 7] Abort-PDU,
    [PRIVATE 8] DataSendAcknowledge-PDU
}
```

11.1.1.2 Confirmed Send Service

```
ConfirmedSend-RequestPDU ::= SEQUENCE {
    [APPLICATION 0] AddressAREP,
    InvokeID,
    ConfirmedServiceRequest
}

ConfirmedSend-ResponsePDU ::= SEQUENCE {
    [APPLICATION 1] AddressAREP,
    InvokeID,
    pduBody CHOICE {
        ConfirmedServiceResponse,
        ConfirmedServiceError
    }
}
```

11.1.1.3 Unconfirmed Send Service

```
UnconfirmedSend-PDU ::= SEQUENCE {
    [APPLICATION 2] AddressAREP,
    InvokeID,
    pduBody CHOICE {
        UnconfirmedServiceRequest,
        UnconfirmedSendPD-PDU
    }
}
```

11.1.1.4 Unconfirmed Acknowledge Send Service

```
UnconfirmedAcknowledgeSend-CommandPDU ::= SEQUENCE {
    [APPLICATION 3] AddressAREP,
    InvokeID,
    UnconfirmedServiceRequest
}
```

11.1.1.5 Establish Service

```
Establish-RequestPDU ::= SEQUENCE {
    [APPLICATION 4] AddressAREP,
    ConType,
    MaxOSCC,
    MaxOSCS,
    MAXUCSC,
    MAXUCSS,
    CIU,
    InvokeID,
    initiateRequest
}

[PRIVATE 0] IMPLICIT Initiate-RequestPDU
```

```
Establish-ResponsePDU ::= SEQUENCE {
    [APPLICATION 5] AddressAREP,
    InvokeID,
    initiateResponse
    [PRIVATE 0] IMPLICIT Initiate-ResponsePDU
}
```

```
Establish-ErrorPDU ::= SEQUENCE {
    [APPLICATION 6] AddressAREP,
    InvokeID,
    initiateError
    [PRIVATE 0] IMPLICIT Initiate-ErrorPDU
}
```

The following types are specified in 4.1.3.

- MaxOSCC
- MaxOSCS
- MaxUCSC
- MaxUCSS
- CIU

The following parameter is specified in 4.1.4.2.

- InvokeID

11.1.1.6 ConType

```
ConType ::= ENUMERATED {
    mmaz (0)
}
```

11.1.1.7 Data Send Acknowledge Service

```
DataSendAcknowledge-PDU ::= SEQUENCE {
    Protocol-Code, [APPLICATION 8], Address2ARP, --- Protocol-Code in the higher nibble of the
    octet!!!
    Block-Number,
    Block-Length,
    Protocol-Data,
}
```

11.1.1.8 Protocol-Code

```
Protocol-Code ::= ENUMERATED {
    TCP/IP (1) --- TCP/IP protocol
}
```

11.1.1.9 Block-Number

```
Block-Number ::= Unsigned8
    --- 0 no blocking
    ---- 1...254 block number
    ---- 255 last block
```

11.1.1.10 Block-Length

```
Block-Length ::= Unsigned
```

11.1.1.11 Protocol-Data

```
Protocol-Data ::= Any
    --- transparent protocol transfer
```

11.1.1.12 Address2 ARP

```

Address2ARP ::= SEQUENCE {
    Destination-Address,
    Source-Address,
    Destination-Node,
    Destination-Subnode,
    Source-Node,
    Source-Subnode
}

```

11.1.1.13 Destination-Address

```

Destination-Address ::= OctetString --- 3 Octets

```

11.1.1.14 Source-Address

```

Source-Address ::= OctetString --- 3 Octets

```

11.1.1.15 Destination-Node

```

Destination-Node ::= Unsigned8

```

11.1.1.16 Source-Node

```

Source-Node ::= Unsigned8

```

11.1.1.17 Destination-Subnode

```

Destination-Subnode ::= Unsigned8

```

11.1.1.18 Source-Subnode

```

Source-Subnode ::= Unsigned8

```

11.1.2 Abstract Syntax of PDUBody**11.1.2.1 Abort Service**

```

Abort-PDU ::= SEQUENCE {
    [APPLICATION 7] AddressAREP,
    Identifier,
    ReasonCode,
    AdditionalDetail
}

```

11.1.2.2 ConfirmedServiceRequest

```

ConfirmedServiceRequest ::= CHOICE {
    read-Request          [0] IMPLICIT Read-RequestPDU,
    write-Request         [3] IMPLICIT Write-RequestPDU,
    start-Request         [6] IMPLICIT Start-RequestPDU,
    stop-Request          [9] IMPLICIT Stop-RequestPDU,
    status-Request        [15] IMPLICIT Status-RequestPDU,
    identify-Request      [18] IMPLICIT Identify-RequestPDU,
    getAttributes1-Request [21] IMPLICIT GetAttributes-RequestPDU, -- short form
    getAttributes2-Request [35] IMPLICIT GetAttributes-RequestPDU, -- long form
    reset-Request         [36] IMPLICIT Reset-RequestPDU,
    resume-Request        [39] IMPLICIT Resume-RequestPDU
}

```

11.1.2.3 ConfirmedServiceResponse

```
ConfirmedServiceResponse ::= CHOICE {
    read-Response           [1] IMPLICIT Read-ResponsePDU,
    write-Response          [4] IMPLICIT Write-ResponsePDU,
    start-Response          [7] IMPLICIT Start-ResponsePDU,
    stop-Response           [10] IMPLICIT Stop-ResponsePDU,
    status-Response         [16] IMPLICIT Status-ResponsePDU,
    identify-Response       [19] IMPLICIT Identify-ResponsePDU,
    getAttributes-Response  [22] IMPLICIT GetAttributes-ResponsePDU,
    reset-Response          [37] IMPLICIT Reset-ResponsePDU,
    resume-Response         [40] IMPLICIT Resume-ResponsePDU
}
```

11.1.2.4 ConfirmedServiceError

```
ConfirmedServiceError ::= CHOICE {
    read-Error              [2] IMPLICIT ErrorType,
    write-Error             [5] IMPLICIT ErrorType,
    start-Error             [8] IMPLICIT ErrorFiType,
    stop-Error              [11] IMPLICIT ErrorFiType,
    status-Error            [17] IMPLICIT ErrorType,
    identify-Error          [20] IMPLICIT ErrorType,
    getAttributes-Error     [23] IMPLICIT ErrorType,
    reset-Error             [38] IMPLICIT ErrorFiType,
    resume-Error            [41] IMPLICIT ErrorFiType
}
```

11.1.2.5 Error Type

Specified in 4.1.4.6.

11.1.2.6 Error Fi Type

```
ErrorFiType ::= SEQUENCE {
    errorClass              [0] IMPLICIT ErrorClass,
    additionalCode          [1] IMPLICIT Integer16 OPTIONAL,
    fiState                 [3] IMPLICIT Integer8
}
```

11.1.2.7 Error Class

Specified in 4.1.4.7.

11.1.2.8 Unconfirmed PDUs

```
UnconfirmedServiceRequest ::= CHOICE {
    informationReport-Request [12] IMPLICIT InformationReport-RequestPDU,
    reject-Request           [34] IMPLICIT Reject-RequestPDU
}
```

UnconfirmedSendPD-PDU ::= BIT STRING

11.1.2.9 Management ASE

11.1.2.9.1 Get Attributes Service

```
GetAttributes-Request-PDU ::= SEQUENCE {
    listOfAttributes          [PRIVATE 0] IMPLICIT Mn_SelectedAttributes,
    accessSpecification CHOICE {
        index                 [1] IMPLICIT Gn_NumericID,
        variableName          [2] IMPLICIT Gn_Name,
        fiName                [5] IMPLICIT Gn_Name,
        startIndex            [7] IMPLICIT Gn_NumericID
    }
}
```

```
GetAttributes-ResponsePDU ::= SEQUENCE {
    more                     [PRIVATE 0] IMPLICIT Gn_MoreFollows,
    listOfObjectDefinition  [PRIVATE 1] IMPLICIT SEQUENCE OF Gn_ObjectDefinition
}
```

11.1.2.10 Application Process ASE

11.1.2.10.1 Get Status Service

Status-RequestPDU ::= NULL

Status-ResponsePDU ::= SEQUENCE {
 logicalStatus [PRIVATE 0] IMPLICIT ENUMERATED {
 ready-for-communication (0),
 limited-services-permitted (2),
 },
 physicalStatus [PRIVATE 1] IMPLICIT ENUMERATED {
 operational (0),
 partially-operational (1),
 inoperable (2),
 needs-commissioning (3),
 },
 localDetail [PRIVATE 2] IMPLICIT BitString OPTIONAL
 }

11.1.2.10.2 Identify Service

Identify-RequestPDU ::= NULL

Identify-ResponsePDU ::= SEQUENCE {
 vendorName [PRIVATE 0] IMPLICIT VisibleString,
 modelIdentifier [PRIVATE 1] IMPLICIT VisibleString,
 vendorRevision [PRIVATE 2] IMPLICIT VisibleString
 }

11.1.2.10.3 Initiate Service

Initiate-RequestPDU ::= SEQUENCE {
 versionObjectDefinitionsCalling [PRIVATE 0] IMPLICIT Integer16,
 apDescriptorCalling [PRIVATE 1] IMPLICIT OctetString,
 accessProtectionSupportedCalling [PRIVATE 2] IMPLICIT Ap_AccessProtectionSupported,
 passwordAndAccessGroupsCalling [PRIVATE 3] IMPLICIT Ap_AccessControl,
 configuredMaxPduSizeSending [PRIVATE 4] IMPLICIT Unsigned8,
 configuredMaxPduSizeReceiving [PRIVATE 5] IMPLICIT Unsigned8,
 listOfSupportedServicesCalling [PRIVATE 6] IMPLICIT Mn_PduSupportedMap
 }

Initiate-ResponsePDU ::= SEQUENCE {
 versionObjectDefinitionsCalled [PRIVATE 0] IMPLICIT Integer16,
 apDescriptorCalled [PRIVATE 1] IMPLICIT OctetString,
 accessPrivilegeSupportedCalled [PRIVATE 2] IMPLICIT Ap_AccessProtectionSupported,
 passwordAndAccessGroupsCalled [PRIVATE 3] IMPLICIT Ap_AccessControl
 }

Initiate-ErrorPDU ::= SEQUENCE {
 errorCode [PRIVATE 0] IMPLICIT ENUMERATED {
 other (0),
 max-fal-pdu-size-insufficient (1),
 service-not-supported (2),
 version-obj-def-incompatible (3),
 user-initiate-denied (4),
 password-error (5),
 profile-number-incompatible (6),
 },
 maxPduLengthSendingCalled [PRIVATE 1] IMPLICIT Unsigned8,
 maxPduLengthReceivingCalled [PRIVATE 2] IMPLICIT Unsigned8,
 listOfSupportedServicesCalled [PRIVATE 3] IMPLICIT Mn_PduSupportedMap
 }

11.1.2.10.4 Reject Service

Reject-RequestPDU ::= SEQUENCE {
 original-invokeID [PRIVATE 0] IMPLICIT Integer8,
 reject-code [PRIVATE 1] IMPLICIT ENUMERATED {
 (5)
 }
 }

11.1.2.11.1 Reset Service

11.1.2.11.1 Reset Service

Reset-ResponsePDU ::= NULL

Resume-ResponsePDU ::= NULL

Start-ResponsePDU ::= NULL

Stop-ResponsePDU ::= NULL

11.1.2.12.1 Information Report Service

11.1.2.12.1 Information Report Service

11.1.2.12.2 Read Service

11.1.2.12.2 Read Service

11.1.2.12.3 Write service

11.1.2.12.3 Write service

Write-ResponsePDU ::= NULL

11.1.3.1 AP ASE Types

11.1.3.1 AP ASE Types

The AP ASE Types

- Ap_AccessProtectionSupported
- Ap_AccessControl

are specified in 4.1.4.15.1.

11.1.3.2 AR ASE Types

11.1.3.2.1 Reason Code and Additional Detail

The AR ASE Types

- Reason Code
- Additional Detail

are specified in 4.1.4.15.2.

11.1.3.2.2 AREP

AddressAREP ::= Unsigned8

-- Least significant octet of DLCEP address

11.1.3.2.3 Abort Types

11.1.3.2.3.1 Identifier

```
Identifier ::= ENUMERATED {
    fal-user                (0),
    fal-APO-ASE             (1),
    fal-AR-ASE              (2),
    dll                     (3)
}
```

11.1.3.3 Function Invocation ASE Types

The Function Invocation ASE Types

- Fi_AccessPrivilege
- Fi_State

are specified in 4.1.4.15.6.

11.1.3.4 Management ASE Types

11.1.3.4.1 Mn_PduSupportedMap

```
Mn_PduSupportedMap ::= BIT STRING {
    getAttributes-RequestPDU      (1),      -- Requester
    start-RequestPDU              (8),
    stop-RequestPDU               (9),
    resume-RequestPDU             (9),
    reset-RequestPDU              (9),
    read-RequestPDU               (11),
    write-RequestPDU              (12),
    informationReport-RequestPDU  (17),
    getAttributes-ResponsePDU     (25),     -- Responder
    start-ResponsePDU             (33),
    stop-ResponsePDU              (33),
    resume-ResponsePDU            (33),
    reset-ResponsePDU             (33),
    read-ResponsePDU              (35),
    write-ResponsePDU             (36),
    informationReport-ResponsePDU (41)
}
```

11.1.3.5 Variable ASE Types

The Variable ASE type

- Vr_AccessPrivilege

is specified in 4.1.4.15.10.

11.1.3.6 General Types

11.1.3.6.1 Referenced General Types

The general types

- Gn_Deletable
- Gn_Reusable
- Gn_KeyAttribute
- Gn_Length
- Gn_MoreFollows
- Gn_Name
- Gn_NumericID
- Gn_ObjectDefinition
- Gn_SubIndex

are specified in 4.1.4.15.7.

11.1.3.6.2 Gn_ObjectClass

```
Gn_ObjectClass ::= ENUMERATED {
    functionInvocation      (3),
    fixedLengthStringDataType (5),
    structuredDataType       (6),
    fixedLengthStringVariable (7),
    arrayVariable            (8),
    dataStructureVariable    (9),
}
```

11.1.3.6.3 Gn_TypeDescription

```
Gn_TypeDescription ::= CHOICE {
    boolean           [1] Gn_Length,
    integer8          [2] Gn_Length,
    integer16         [3] Gn_Length,
    integer32         [4] Gn_Length,
    unsigned8         [5] Gn_Length,
    unsigned16        [6] Gn_Length,
    unsigned32        [7] Gn_Length,
    float             [8] Gn_Length,
    visiblestring     [9] Gn_Length,
    octetstring       [10] Gn_Length,
    binaryDate        [11] Gn_Length,
    timeOfDay          [12] Gn_Length,
    timeDifference     [13] Gn_Length,
    bitstring         [14] Gn_Length,
}
```

11.1.3.7 Object Definitions

11.1.3.7.1 Top Level Definition

```
Object-Definition ::= CHOICE {
    [PRIVATE 0] ListHeader,
    [PRIVATE 1] DataTypeList,
    [PRIVATE 2] StaticList,
    [PRIVATE 3] FunctionInvocationDefinition
}
```

11.1.3.7.2 ListHeader

```

ListHeader ::= SEQUENCE {
    numericId                      [PRIVATE 0] IMPLICIT Unsigned16,
    romRamFlag                    [PRIVATE 1] IMPLICIT Boolean,
    maxNameLength                 [PRIVATE 2] IMPLICIT Unsigned8,
    accessProtectionSupported     [PRIVATE 3] IMPLICIT Boolean,
    versionOfObjectDefinition     [PRIVATE 4] IMPLICIT Integer16,
    localReferenceOfListHeader    [PRIVATE 5] IMPLICIT Unsigned32 OPTIONAL,
    numberOfEntriesInDataTypeList [PRIVATE 6] IMPLICIT Unsigned16,
    localReferenceOfDataTypeList  [PRIVATE 7] IMPLICIT Unsigned32 OPTIONAL,
    firstNumericIdOfStaticList    [PRIVATE 8] IMPLICIT Unsigned16,
    numberOfEntriesInStaticList   [PRIVATE 9] IMPLICIT Unsigned16,
    localReferenceOfStaticList    [PRIVATE 10] IMPLICIT Unsigned32 OPTIONAL,
    firstNumericIdOfVariableListDefinition [PRIVATE 11] IMPLICIT Unsigned16,
    numberOfEntriesInVariableListDefinition [PRIVATE 12] IMPLICIT Unsigned16,
    localReferenceOfVariableListDefinition [PRIVATE 13] IMPLICIT Unsigned32 OPTIONAL,
    firstNumericIdOfFunctionInvocationDefinition [PRIVATE 14] IMPLICIT Unsigned16,
    numberOfEntriesInFunctionInvocationDefinition [PRIVATE 15] IMPLICIT Unsigned16,
    localReferenceOfFunctionInvocationDefinition [PRIVATE 16] IMPLICIT Unsigned32 OPTIONAL
}

```

11.1.3.7.3 DataTypeList

```

DataTypeList ::= CHOICE {
    [PRIVATE 0] DataTypeDefinition,
    [PRIVATE 1] StructuredDataTypeDefinition
}

DataTypeDefinition ::= SEQUENCE {
    numericId                      Gn_NumericID,
    objectClass                   Gn_ObjectClass,
    dataTypeNameLength            Gn_Length,
    dataTypeName                  [PRIVATE 0] IMPLICIT VisibleString OPTIONAL
}

StructuredDataTypeDefinition ::= SEQUENCE {
    numericId                      Gn_NumericID,
    objectClass                   Gn_ObjectClass,
    numberOfElements              [PRIVATE 0] IMPLICIT Integer8,
    recordList                    SEQUENCE OF SEQUENCE {
        numericIdOfDataTypeDefinition Gn_NumericID,
        dataLength                    Gn_Length
    }
}

```

11.1.3.7.4 StaticList

```

StaticList ::= CHOICE {
    [PRIVATE 0] VariableDefinition,
    [PRIVATE 1] ArrayDefinition,
    [PRIVATE 2] StructureDefinition
}

VariableDefinition ::= SEQUENCE {
    numericId                      Gn_NumericID,
    objectClass                   Gn_ObjectClass,
    numericIdOfDataTypeDefinition Gn_NumericID,
    dataLength                    Gn_Length,
    accessPrivilege               Vr_AccessPrivilege OPTIONAL,
    localReferenceOfVariable      [PRIVATE 0] IMPLICIT Unsigned32 OPTIONAL,
    variableName                  [PRIVATE 1] IMPLICIT VisibleString OPTIONAL,
    extension                     [PRIVATE 2] IMPLICIT OctetString OPTIONAL
}

ArrayDefinition ::= SEQUENCE {
    numericId                      Gn_NumericID,
    objectClass                   Gn_ObjectClass,
    numericIdOfDataTypeDefinition Gn_NumericID,
    dataLength                    Gn_Length,
    numberOfElements              [PRIVATE 0] IMPLICIT Unsigned8,
    accessPrivilege               Vr_AccessPrivilege OPTIONAL,
    localReferenceOfArray         [PRIVATE 1] IMPLICIT Unsigned32 OPTIONAL,
    arrayName                     [PRIVATE 2] IMPLICIT VisibleString OPTIONAL,
    extension                     [PRIVATE 3] IMPLICIT OctetString OPTIONAL
}

```

```

StructureDefinition ::= SEQUENCE {
    numericId                Gn_NumericID,
    objectClass              Gn_ObjectClass,
    numericIdOfDataTypeDefinition Gn_NumericID,
    accessPrivilege          Vr_AccessPrivilege OPTIONAL,
    structureName            [PRIVATE 0] IMPLICIT VisibleString OPTIONAL,
    extension                [PRIVATE 1] IMPLICIT OctetString OPTIONAL,
    localReferenceOfElement  [PRIVATE 2] IMPLICIT SEQUENCE OF Unsigned32 OPTIONAL
}

```

11.1.3.7.5 FunctionInvocationDefinition

```

FunctionInvocationDefinition ::= SEQUENCE {
    numericId                Gn_NumericID,
    objectClass              Gn_ObjectClass,
    numberOfRelatedObjects   [PRIVATE 0] IMPLICIT Unsigned8,
    accessPrivilege          Fi_AccessPrivilege OPTIONAL,
    deletable                Gn_Deletable,
    reusable                 Gn_Reusable,
    functionInvocationState   FI_State,
    numericIdOfLoadRegion    SEQUENCE OF Gn_NumericID,
    functionInvocationName    [PRIVATE 1] IMPLICIT VisibleString OPTIONAL,
    extension                [PRIVATE 2] IMPLICIT OctetString OPTIONAL
}

```

11.1.4 Abstract Syntax of Data Types

11.1.4.1 Referenced Data Types

The notations of the following Data Types are specified in 4.1.5.

- Boolean
- Integer8
- Integer16
- Unsigned8
- Unsigned16
- Unsigned32
- Floating32
- BitString
- OctetString
- VisibleString
- BinaryDate
- TimeOfDay
- TimeDifference

11.1.4.2 Notation for BinaryDate2000 Type

BinaryDate2000 ::= OctetString8

11.2 Transfer Syntax

11.2.1 Peripherals Encoding Rules (PER)

The Peripherals Encoding Rule is a preferable encoding rule for the encoding of the PDU types defined in 11.1.1.

11.2.2 Encoding of APDU types

11.2.2.1 Overview of APDU encoding

The APDUs encoded with the PER shall have a uniform format. The APDUs shall consist of two major parts, the “APDU Header” part and the “APDU Body” part as shown in Figure 203:

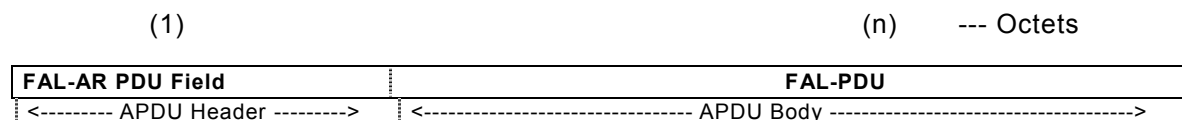


Figure 203 – APDU overview

11.2.2.2 APDU Header encoding

In 11.1.1, an APDU header is defined as a tagged type of the tag class 'APPLICATION'.

The APDU Header (also called FAL-AR PDU Field) shall be encoded using one or more octets as described in this subclause.

Within the APDU header, the PDU Type tag (e.g. Establish-RequestPDU, ConfirmedSend-RequestPDU...) and the address information (AddressAREP) is encoded .

If the value of the PDU type is < 3, the APDU header shall be encoded using one octet as defined in Figure 204.

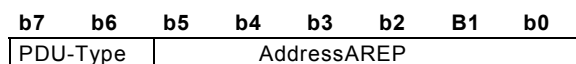


Figure 204 – APDU Header

If the value of the PDU type is ≥ 3 an extension octet is used for the encoding of the tag. The use of a tag extension octet shall be indicated by setting both bits in the PDU type field of the first octet to 1. If the address information (AddressAREP) is greater than 62, an address extension octet shall be used for the encoding of the address value. In this case, the use of an address extension shall be indicated by setting all bits of the address field in the first octet to 1 (see Figure 205 and Figure 206).



Figure 205 – PDU with Type Extension

The S field is reserved for system management extensions. For FAL-PDUs this flag is set to 0.



Figure 206 – PDU with Address Extension

Figure 207 shows the encoding of an APDU header using the tag extension and the address extension octet:

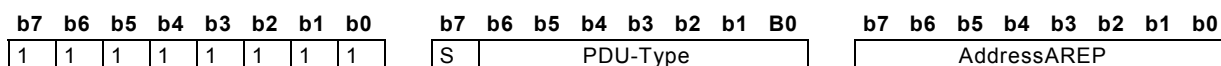


Figure 207 – PDU with Type and Length Extension

Special encoding of the Establish-Request PDU type:

The encoding of the Establish-Request PDU differs from the rules described above, since in this PDU additional parameters, which contain the AR context, are passed. The parameters are the connection type (master-master connection for acyclic data transfer), the parameter CIU (Control Interval User-triggered) and the outstanding service counters. The connection type parameter is encoded in the most-significant three bits. The least-significant 5 bits contain the number of the additional establish parameters.

If the tag for the address requires an extension (the Establish PDU type has an extension), a minimum PDU length of 9 octets is used in an establish request (see Figure 208).

Example for Establish-RequestPDU:

Establish-RequestPDU tag : 4
 AddressAREP : 64
 ConnectionType : 0 for MMAZ
 CIU : FFFFFFFF
 S : 0
 MaxOSCC : 1
 MaxOSCS : 1
 MaxUCSC : 1
 MaxUCSS : 1

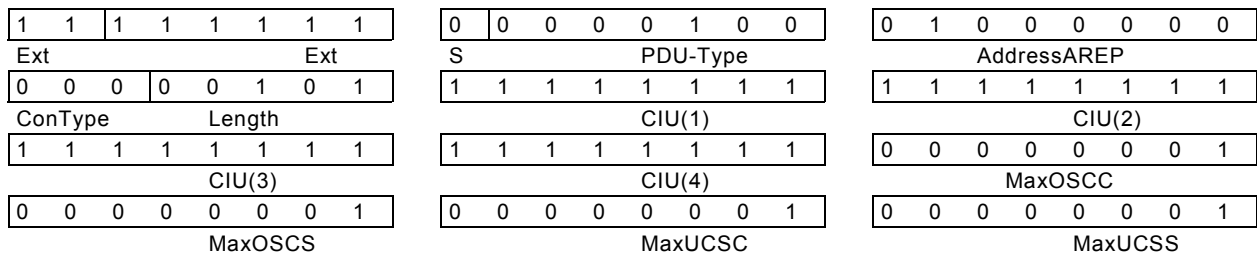


Figure 208 – Example of an Establish-Request PDU

11.2.2.3 APDU Body encoding

The format of the APDU body is described in 11.1.1 using the ASN.1 syntax. In the following clauses the encoding of the used language elements is described.

11.2.3 Encoding of tagged type values

11.2.3.1 Encoding of tagged type values of tag class PRIVATE

Tagged type values with tags of class PRIVATE shall be encoded in the same way as their base types. Tags of class PRIVATE are not explicit encoded (see Figure 209). They are used only to reach syntactical correctness of the ASN.1 definitions.

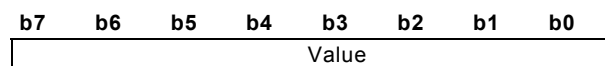


Figure 209 – Encoding of a PRIVATE tagged value

The following restrictions exist for the use of PRIVATE tagged types:

- The length and definition of the encoding of a PRIVATE tagged type shall not change.
- In a SEQUENCE construct no more than one PRIVATE tagged type may be OPTIONAL.
- A private tagged type shall not be contained in a CHOICE construct.

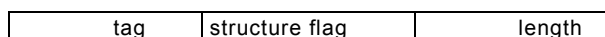
11.2.3.2 Encoding of context specific tagged type values

11.2.3.2.1 Overview

Context specific tagged type values are coded as a sequence of one or more Identification information (ID-info) octets, followed by one or more Value octets (ContentsOctets) as defined in the following clauses. The resulting format is shown in Figure 210.

**Figure 210 – Encoding of a context specific tagged value****11.2.3.2.2 Encoding of the Identification Information (ID-info)**

The ID-info is composed of three fields: tag, structure flag and length field (see Figure 211).

**Figure 211 – Identification information fields**

The length field contains either the number of the subsequent data octets or the number of the following values, as defined in this subclause.

If the structure flag is set to 1, it indicates that a structure follows. Within the structure encoding each value is identified by its own ID-info. In this case, the number of the values within the structure is encoded in the length field of the ID-info. In the length field of the last value of the structure, the length of the values encoding in octets is contained.

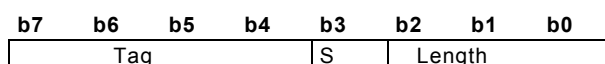
If the structure flag is 0, and the value is not part of a structure as described before, the following values are not tagged. In this case the types and lengths of the contained values are known implicitly; the number of the contained values(parameters) is encoded in the length field.

If the structure flag is 0, and the value is part of another structure, the length field shall contain:

the number of the following values, if this value is not the last value within the structure,

the number of the value encoding octets, if this value is the last value within the structure.

The tag field contains the tag value defined within the ASN.1 syntax definition. If the tag value is < 15 and the length value is < 7, the ID-info shall be encoded using 1 octet (see Figure 212).

**Figure 212 – ID-info for Tag 0 .. 14 , Length entry 0 .. 6**

NOTE Figure 213 is not used.

If the tag value is ≥ 15 and the length value is < 7, the ID-info shall be encoded using 2 octets (see Figure 214).

**Figure 214 – ID-info for Tag 15 .. 255 , Length entry 0 .. 6**

If the tag value is < 15 and the length value is ≥ 7 , the ID-info shall be encoded using 2 octets (see Figure 215).

**Figure 215 – ID-info for Tag 0 .. 14 , Length entry 7 .. 255**

If the tag value is ≥ 15 and the length value is ≥ 7 , the ID-info shall be encoded using 3 octets (see Figure 216).

b7	b6	b5	b4	b3	b2	b1	b0	b7	b6	b5	b4	b3	b2	b1	b0	b7	b6	b5	b4	b3	b2	b1	b0
1	1	1	1	S	1	1	1	Tag								Length entry							

Figure 216 – ID-info for Tag 15 .. 255 , Length entry 7 .. 255

11.2.3.2.3 Example

Data of the data type Boolean, for example, is coded as shown in Figure 217 and Figure 218:

b7	b6	b5	b4	b3	b2	b1	b0	b7	b6	b5	b4	b3	b2	b1	b0
Tag								1	1	1	1	1	1	1	1

Figure 217 – Encoding of Boolean value TRUE

b7	b6	b5	b4	b3	b2	b1	b0	b7	b6	b5	b4	b3	b2	b1	b0
Tag								0	0	0	0	0	0	0	0

Figure 218 – Encoding of Boolean value FALSE

11.2.4 Encoding of simple values

11.2.4.1 Encoding of a Boolean value

- 1) The encoding of a Boolean value shall be primitive, and the ContentsOctets shall consist of exactly one octet.
- 2) The value TRUE is coded by setting all bits of the octet to 1, the value FALSE is coded by setting all bits of the octet to 0.

11.2.4.2 Encoding of String values (visible string, octet string, bit string)

In the encoding of strings, length entries, which give the number of following octets, are placed in the first octet of the individual string elements (see Figure 219). User data without a tag are provided with no ID-info. The data is identified by its position within the PDU and the length is implicitly known and fixed.

Octet 1	Octet 2	Octet 3		Octet n + 1
Length = n	Element 1	Element 2		Element n

Figure 219 – Encoding of Strings

11.2.4.3 Encoding of an Integer value

- a) The encoding of a fixed-length Integer value of Integer8, Integer16, and Integer32 types shall be primitive, and the ContentsOctets shall consist of exactly one, two, or four octets, respectively.
- b) The ContentsOctets shall be a two's complement binary number equal to the integer value, and consist of bits 8 to 1 of the first octet, followed by bits 8 to 1 of the second octet, followed by bits 8 to 1 of each octet in turn up to and including the last octet of the ContentsOctets.

11.2.4.4 Encoding of an Unsigned value

- a) The encoding of a fixed-length Unsigned value of Unsigned8, Unsigned16, and Unsigned32 types shall be primitive, and the ContentsOctets shall consist of exactly one, two, or four octets, respectively.
- b) The ContentsOctets shall be a binary number equal to the Unsigned value, and consist of bits 8 to 1 of the first octet, followed by bits 8 to 1 of the second octet, followed by bits 8 to 1 of each octet in turn up to and including the last octet of the ContentsOctets.

11.2.4.5 Encoding of a floating-point value

- a) The encoding of a Floating32 value shall be primitive, and the ContentsOctets shall consist of exactly four or eight octets, respectively.

- b) The ContentsOctets shall contain floating-point values defined in conformance with ANSI/IEEE Standard 754. The sign is encoded in bit 8 of the first octet. It is followed by the exponent starting from bit 7 of the first octet, and then the mantissa starting from bit 7 of the second octet for Floating32.

11.2.4.6 Encoding of a BinaryDate value

- a) The encoding of a BinaryDate value shall be primitive.
- b) The Length field shall indicate as a binary number the number of octets in the BinaryDate value.
- c) The ContentsOctets shall be equal in value to the octets in the data value, as shown in Figure 220:

bits	8	7	6	5	4	3	2	1	
octets									
1	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	
2	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	0...59 999 ms
3	0	0	2^5	2^4	2^3	2^2	2^1	2^0	0...59 min
4	SU	0	0	2^4	2^3	2^2	2^1	2^0	0...23 hours
	day of week			day of month					1...7 d. of w.
5	2^2	2^1	2^0	2^4	2^3	2^2	2^1	2^0	1...31 d. of m.
6	0	0	2^5	2^4	2^3	2^2	2^1	2^0	1...12 months
7	0	2^6	2^5	2^4	2^3	2^2	2^1	2^0	0 ... 99 years
msb									

Figure 220 – Encoding of BinaryDate Value

11.2.4.7 Encoding of a BinaryDate2000 value

- a) The encoding of a BinaryDate2000 value shall be primitive.
- b) The Length field shall indicate as a binary number the number of octets in the BinaryDate2000 value.
- c) The ContentsOctets shall be equal in value to the octets in the data value, as shown in Figure 221:

bits	8	7	6	5	4	3	2	1	
octets									
1	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	
2	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	0...59 999 ms
3	0	0	2^5	2^4	2^3	2^2	2^1	2^0	0...59 min
4	SU	0	0	2^4	2^3	2^2	2^1	2^0	0...23 hours
	day of week			day of month					1...7 d. of w.
5	2^2	2^1	2^0	2^4	2^3	2^2	2^1	2^0	1...31 d. of m.
6	0	0	2^5	2^4	2^3	2^2	2^1	2^0	1...12 months
7	0	0	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	0 ... 9999 years
8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	with century
msb									

Figure 221 – Encoding of BinaryDate2000 Value

11.2.4.8 Encoding of a Time of Day value

- The encoding of a Time Of Day value shall be primitive.
- The Length field shall indicate as a binary number the number of octets in the Time Of Day value.
- The ContentsOctets shall be equal in value to the octets in the data value, as shown in Figure 222:

bits	8	7	6	5	4	3	2	1	
octets									
1	0	0	0	0	2^{27}	2^{26}	2^{25}	2^{24}	number of milliseconds since midnight
2	2^{23}	2^{22}	2^{21}	2^{20}	2^{19}	2^{18}	2^{17}	2^{16}	
3	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	
4	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
5	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	number of days since 01.01.84
6	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	optional
msb									

Figure 222 – Encoding of Time of Day value

11.2.4.9 Encoding of a Time Difference value

- The encoding of a Time Difference value shall be primitive.
- The Length field shall indicate as a binary number the number of octets in the Time Difference value.
- The ContentsOctets shall be equal in value to the octets in the data value, as shown in Figure 223:

bits	8	7	6	5	4	3	2	1	
octets									
1	0	0	0	0	2^{27}	2^{26}	2^{25}	2^{24}	number of milliseconds
2	2^{23}	2^{22}	2^{21}	2^{20}	2^{19}	2^{18}	2^{17}	2^{16}	
3	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	
4	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
5	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	number of days
6	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	optional
msb									

Figure 223 – Encoding of Time Difference Value

11.2.4.10 Encoding of a Time value

- The encoding of a Time value shall be primitive.
- The Length field shall indicate as a binary number the number of octets in the Time value.
- The ContentsOctets shall be equal in value to the octets in the data value, as shown in Figure 224:

bits	8	7	6	5	4	3	2	1	
octets	SN	2^{62}	2^{61}	2^{60}	2^{59}	2^{58}	2^{57}	2^{56}	
1									
2	2^{55}	2^{54}	2^{53}	2^{52}	2^{51}	2^{50}	2^{49}	2^{48}	
3	2^{47}	2^{46}	2^{45}	2^{44}	2^{43}	2^{42}	2^{41}	2^{40}	
4	2^{39}	2^{38}	2^{37}	2^{36}	2^{35}	2^{34}	2^{33}	2^{32}	
5	2^{31}	2^{30}	2^{29}	2^{28}	2^{27}	2^{26}	2^{25}	2^{24}	
6	2^{23}	2^{22}	2^{21}	2^{20}	2^{19}	2^{18}	2^{17}	2^{16}	
7	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	signed integer
8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	of 8 octet length of 1/32ms unit
MSB									

Figure 224 – Encoding of Time Value

11.2.4.11 Encoding of a Null value

- a) The encoding of a NULL value shall be primitive.
- b) The ContentsOctet shall be omitted.

11.2.4.12 Encoding of an ANY value

The Data Type ANY contains one or more data elements of the Data Types which are chained together without any gap. The composition is known implicitly.

11.2.4.13 Encoding of Structured Types

When a structured type is encoded, the structure flag of the Identification Information shall be set to one. The Length field, whether or not it is extended, shall contain the number of components of the structured type being encoded.

11.2.4.14 Encoding of a SEQUENCE value

The SEQUENCE type is comparable to a record. It represents a collection of user data of the same or of different Data Types.

A SEQUENCE type value may contain a simple variable or a further structured variable as its components. If a SEQUENCE type contains another structured type value, it shall be counted as a single component even if it contains several components.

11.2.4.15 Encoding of a SEQUENCE OF value

The SEQUENCE OF type represents a succession of components. It is comparable to an array.

A SEQUENCE OF type value may contain one or more simple or constructed variables. If a SEQUENCE OF type contains another structured type value, it shall be counted as a single component even if it contains several components.

The encoding is as for the structure SEQUENCE. For the statement of the number of components the number of repetitions shall be taken into account.

11.2.4.16 Encoding of a CHOICE value

A CHOICE type represents a selection from a set of predefined values. The components of a CHOICE construct shall have different tags to allow proper identification. Instead of the CHOICE construct, the actually selected component is encoded. Only one component shall be encoded for a CHOICE.

11.2.4.17 Encoding of an ENUMERATED value

An ENUMERATED value shall be encoded as an Unsigned8 value.

11.2.4.18 Encoding of an Object-Definition value

The encoding of the individual object definitions onto the parameters "List Of Objects and Attributes" (data type Gn_ObjectDefinition) of the GetAttributePDUs is shown in this subclause.

The objectClass attribute is the identifier of the object and indicates the class to which this object belongs. The other object attributes are object specific and shall be encoded as a string of octets. In addition to the object attributes, the object class shall be transmitted in the GetAttributes service, except List Header, whose numeric ID is zero. An example of the structure of an Object Definition is shown in Figure 225.

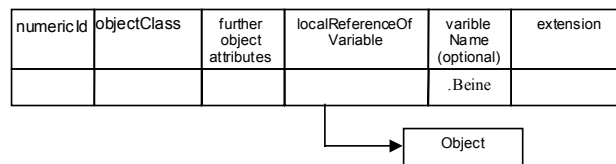


Figure 225 – Example for an Object Definition

11.3 Protocol Machine Overview

The FAL Protocol State Machine functionality is shown in 4.3.

Figure 226 shows the primitives exchanged between protocol machines.

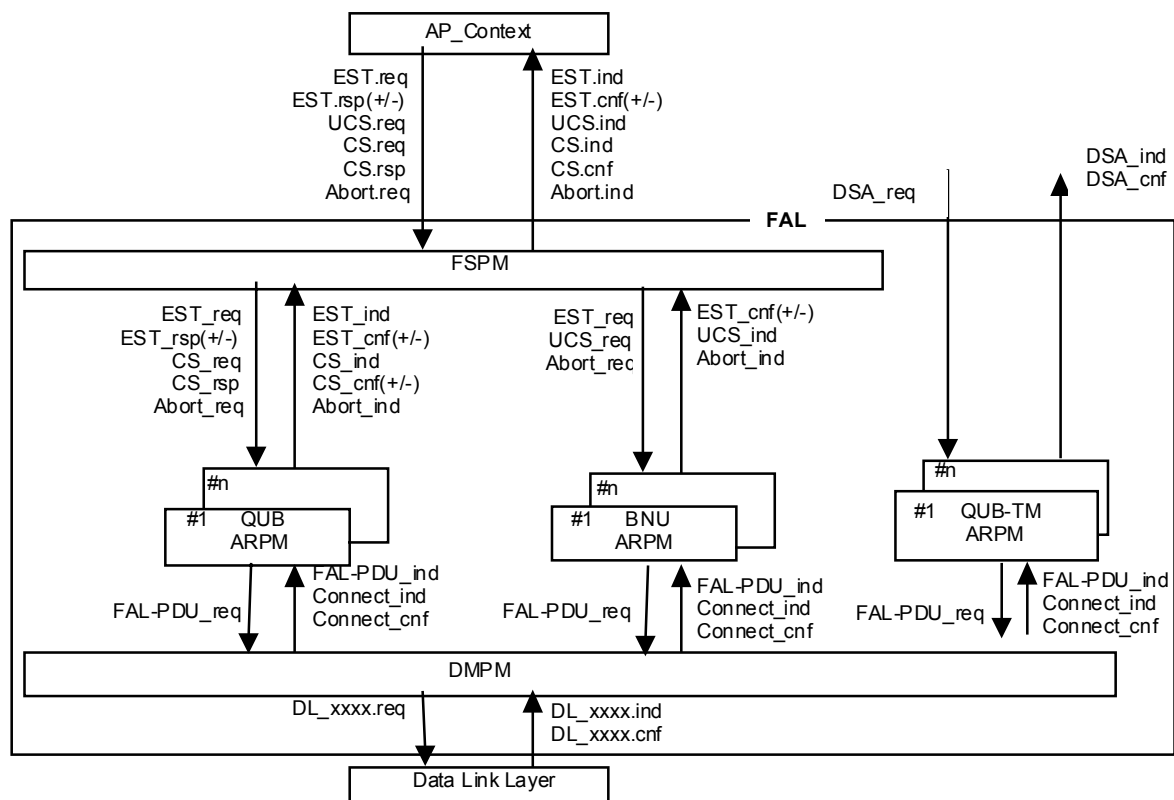


Figure 226 – Primitives Exchanged between Protocol Machines

11.4 AP-Context State Machine

The AP-Context State Machine is specified in 4.4.

11.5 FAL Service Protocol Machine (FSPM)

The FAL Service Protocol Machine is specified in 4.5.

The primitives

- FCMP.req
- GBM.req
- RW.req
- RR.req
- AR_XON_OFF.req

are not supported.

11.6 Application Relationship Protocol Machines (ARPMs)

11.6.1 Queued Usertriggered Bidirectional-Flow Control (QUB-FC) ARPM

11.6.1.1 QUB-FC Primitive Definitions

11.6.1.1.1 Primitives Exchanged between ARPM and FSPM

The primitives exchanged between ARPM and FSPM are specified in 4.6.5.1.1. The primitive

AR_XON_OFF.req

is not supported.

11.6.1.1.2 Parameters of FSPM/ARPM Primitives

The parameters used with the primitives exchanged between the FSPM and ARPM are specified in 4.6.5.2. The parameters

Duplicate_fal_sdu
Remote_dlsap_address
Status
Reported_status
Local_timeliness
Remote_timeliness

are not supported.

11.6.1.2 DLL Mapping of QUB-FC AREP Class

This subclause describes the mapping of the QUB-FC AREP class to the IEC 61158 Type-8 Data Link Layer. It does not redefine the DLSAP attributes or DLME attributes that are or will be defined in the Data Link Layer standard; rather, it defines how they are used by this AR class.

NOTE A means to configure and monitor the values of these attributes are not in the scope of this International Standard.

The DLL Mapping attributes and their permitted values and the DLL services used with the QUB-FC AREP class are defined in this subclause.

CLASS: Type-8 QubFC
PARENT CLASS: QueuedUser-TriggeredBidirectional-FlowControlAREP
ATTRIBUTES:

1	(m)	KeyAttribute: LocalDlcepAddress
2	(m)	Attribute: RemoteDlcepAddress
3	(m)	Attribute: QosParameterSet
3.1	(m)	Attribute: DllPriorityNegotiated
3.2	(m)	Attribute: MaxConfirmDelay
3.3	(m)	Attribute: MaxDlsduSizes
3.3.1	(m)	Attribute: MaxDlsduSizeFromRequestor
3.3.2	(m)	Attribute: MaxDlsduSizeFromResponder
3.3.3	(m)	Attribute: MaxDlsduSizeFromRequestorNegotiated
3.3.4	(m)	Attribute: MaxDlsduSizeFromResponderNegotiated
3.4	(m)	Attribute: MaxQueueDepth
3.4.1	(m)	Attribute: MaxSendingQueueDepth
3.4.2	(m)	Attribute: MaximimReceivingQueueDepth

DLL SERVICES:

1	(m)	OpsService: DL-Data
---	-----	---------------------

11.6.1.3 Attributes

11.6.1.3.1 LocalDlcepAddress

This attribute specifies the local DLCEP address and identifies the DLCEP.

The value of this attribute is used as the “DLCEP-address” parameter of the DLL.

11.6.1.3.2 RemoteDlcepAddress

This attribute specifies the remote DLCEP address and identifies the DLCEP.

11.6.1.3.3 QosParameterSet

The QosParameterSet attributes specify the DL quality of service that is used by this AREP. These attribute values may be negotiated with the remote AREP.

11.6.1.3.4 DllPriorityNegotiated

This attribute specifies the negotiated value of the DLL priority.

11.6.1.3.5 MaxConfirmDelay

This attribute specifies the maximum confirmation delay of certain DLL connection-oriented services.

11.6.1.3.6 MaxDisduSize

MaxDisduSizeFromRequestor

This attribute specifies the configured value of the maximum length of an FAL-PDU that can be sent from the AREP whose Initiator attribute has a value of “True” to the remote AREP.

This attribute supplies the value for the “Maximum DLSDU sizes from responder” parameter of the DLL.

MaxDisduSizeFromResponder

This attribute specifies the configured value of the maximum length of an FAL-PDU that can be sent from the AREP whose Initiator attribute has a value of “False” to the remote AREP.

This attribute supplies the value for the “Maximum DLSDU sizes from responder” parameter of the DLL.

MaxDisduSizeFromRequestorNegotiated

This attribute specifies the negotiated value of the maximum length of an FAL-PDU that can be sent from the AREP whose Initiator attribute has a value of “True” to the remote AREP.

MaxDisduSizeFromResponderNegotiated

This attribute specifies the negotiated value of the maximum length of an FAL-PDU that can be sent from the AREP whose Initiator attribute has a value of “False” to the remote AREP.

11.6.1.3.7 MaxQueueDepth

MaxSendingQueueDepth

This attribute specifies the maximum number of FAL-PDUs that can be queued for transmission.

This attribute supplies the value for the “Maximum queue depth” parameter of the DLL for Send queues.

MaxReceivingQueueDepth

This attribute specifies the maximum number of FAL-PDUs that can be queued at reception.

This attribute supplies the value for the “Maximum queue depth” parameter of the DLL for Receive queues.

11.6.1.4 DLL services

Refer to IEC 61158-3 Type-8 for DLL service descriptions.

11.6.1.5 QUB-FC ARPM State Machine

11.6.1.5.1 QUB-FC ARPM States

The ARPM states are specified in 4.6.5.4.1.

11.6.1.5.2 QUB-FC ARPM State Table

The state table of the QUB-FC is specified in 4.6.5.4.2.

11.6.1.5.3 Functions used by QUB-FC ARPM

The functions used by the ARPM are specified in 4.6.5.4.3.

11.6.2 Buffered Networkscheduled Unidirectional (BNU) ARPM

11.6.2.1 Primitive definitions

11.6.2.1.1 Primitives Exchanged between ARPM and FSPM

The primitives exchange between ARPM and FSPM is specified in 4.6.8.1.1. The primitives

FCMP_req

GBM_req

are not supported.

11.6.2.1.2 Parameters of FSPM/ARPM Primitives

The parameters used with the primitives exchanged between the FSPM and ARPM are specified in 4.6.8.1.2. The parameters

Duplicate_fal_sdu

Remote_dlsap_address

Status

Reported_status

Local_timeliness

Remote_timeliness

are not supported.

11.6.2.2 DLL Mapping of BNU AREP Class

This subclause describes the mapping of the BNU AREP class to the IEC 61158 Type-8 Data Link Layer. It does not redefine the DLSAP attributes or DLME attributes that are or will be defined in the Data Link Layer standard; rather, it defines how they are used by this AR class.

NOTE A means to configure and monitor the values of these attributes are not in the scope of this International Standard.

The DLL Mapping attributes and their permitted values and the DLL services used with the BNU AREP class are defined in this subclause.

CLASS:	Type-8 Bnu
PARENT CLASS:	BufferedNetworkScheduledUnidirectionalAREP
ATTRIBUTES:	
1	(m) KeyAttribute: PublisherDlcepAddress
2	(m) Attribute: QosParameterSet
3.1	(m) Attribute: DlcepClass (Publisher, Subscriber)
4.2	(m) Constraint: DlcepClass = Publisher
4.2.1	(m) Attribute: MaxDlsduSizeFromRequestor
4.3	(m) Constraint: DlcepClass = Subscriber
4.3.1	(m) Attribute: MaxDlsduSizeFromResponder
DLL SERVICES:	
1	(c) Constraint: DlcepClass (Publisher)
1.1	(m) OpsService: DL-Put
1.2	(o) OpsService: DL-Get
2	(c) Constraint: DlcepClass (Subscriber)
2.1	(o) OpsService: DL-Put
2.2	(m) OpsService: DL-Get
3	(m) OpsService: DL-Buffer-Received

11.6.2.3 Attributes

11.6.2.3.1 PublisherDlcepAddress

This attribute specifies the Publisher's DLCEP address and identifies the DLCEP.

The value of this attribute is used as the "DLCEP-address" parameter of the DLL.

11.6.2.3.2 QosParameterSet

11.6.2.3.2.1 DlcepClass

This attribute specifies the behavior of the DLCEP which is attached to the AREP.

This attribute supplies the value for the "DLCEP class" parameter of the DLL. The possible value of this attribute is either Publisher or Subscriber and corresponds to PUBLISHER and SUBSCRIBER defined by the DLL, respectively.

11.6.2.3.2.2 MaxDlsduSizeFromRequestor

This attribute is used if the Role attribute has a value of "PushPublisher" and specifies the maximum length of an FAL-PDU that can be sent from this AREP.

11.6.2.3.2.3 MaxDlsduSizeFromResponder

This attribute is used if the Role attribute has a value of "PushSubscriber" and specifies the maximum length of an FAL-PDU that can be received by this AREP.

11.6.2.4 DLL Services

Refer to IEC 61158-3 Type-8 for DLL service descriptions.

11.6.2.5 BNU AREP state machine

11.6.2.5.1 BNU ARPM States

The states are specified in 4.6.8.3.1.

11.6.2.5.2 BNU ARPM State Table

The state table of the BNU ARPM is specified in 4.6.8.3.2.

11.6.2.5.3 Functions used by BNU ARPM

The functions used by the BNU ARPM are specified in 4.6.8.3.3.

11.6.3 Queued Usertriggered Bidirectional – Transparent Mode (QUB-TM) ARPM

11.6.3.1 QUB-TM Primitive Definitions

11.6.3.1.1 Primitives Exchanged between ARPM and FAL

The primitives exchanged between ARPM and FAL are shown in Table 852 and Table 853

Table 852 – Primitives issued by FAL to ARPM

Primitive name	Source	Associated parameters	Functions
DSA_req	FAL	user_data	This is a primitive used to convey a Data-Send-Acknowledge request primitive from the FAL to the ARPM.

Table 853 – Primitives issued by ARPM to FAL

Primitive name	Source	Associated parameters	Functions
DSA_cnf(+)	ARPM	arep_id, user_data	This is a primitive used to convey a Data-Send-Acknowledge confirmation(+) primitive from the ARPM to the FAL.
DSA_cnf(-)	ARPM	arep_id, user_data	This is a primitive used to convey a Data-Send-Acknowledge confirmation(-) primitive from the ARPM to the FAL.
DSA_ind	ARPM	arep_id, user_data	This is a primitive used to convey a Data-Send-Acknowledge indication from the ARPM to the FAL.

11.6.3.1.2 Parameters of FAL/ARPM Primitives

The parameters used with the primitives exchanged between the FAL and the ARPM are described in Table 854.

Table 854 – Parameters used with Primitives Exchanged between FAL and ARPM

Parameter name	Description
Arep_id	This parameter is used to unambiguously identify an instance of the AREP that has issued a primitive. A means for such identification is not specified by this standard.
User_data	This parameter conveys FAL-User data.

11.6.3.2 DLL Mapping of QUB-TM AREP Class

This subclause describes the mapping of the QUB-TM AREP class to the IEC 61158 Type-8 Data Link Layer. It does not redefine the DLSAP attributes or DLME attributes that are or will be defined in the Data Link Layer standard; rather, it defines how they are used by this AR class.

NOTE A means to configure and monitor the values of these attributes are not in the scope of this International Standard.

The DLL Mapping attributes and their permitted values and the DLL services used with the QUB-TM AREP class are defined in this subclause.

CLASS: Type-8 QubTM

PARENT CLASS: QueuedUser-TriggeredBidirectional-FlowControlAREP

ATTRIBUTES:

- 1 (m) KeyAttribute: LocalDlcepAddress
- 2 (m) Attribute: RemoteDlcepAddress
- 3 (m) Attribute: QosParameterSet
- 3.3 (m) Attribute: MaxDlsduSizes
- 3.3.1 (m) Attribute: MaxDlsduSizeFromRequestor
- 3.3.2 (m) Attribute: MaxDlsduSizeFromResponder
- 3.3.3 (m) Attribute: MaxDlsduSizeFromRequestorNegotiated
- 3.3.4 (m) Attribute: MaxDlsduSizeFromResponderNegotiated
- 3.4 (m) Attribute: MaxQueueDepth
- 3.4.1 (m) Attribute: MaxSendingQueueDepth
- 3.4.2 (m) Attribute: MaximimReceivingQueueDepth

DLL SERVICES:

- 1 (m) OpsService: DL-Data

11.6.3.3 Attributes

11.6.3.3.1 LocalDlcepAddress

This attribute specifies the local DLCEP address and identifies the DLCEP.

The value of this attribute is used as the “DLCEP-address” parameter of the DLL.

11.6.3.3.2 RemoteDlcepAddress

This attribute specifies the remote DLCEP address and identifies the DLCEP.

11.6.3.3.3 QosParameterSet

11.6.3.3.3.1 General

The QosParameterSet attributes specify the DL quality of service that is used by this AREP. These attribute values may be negotiated with the remote AREP.

11.6.3.3.3.2 MaxDlsduSize

MaxDlsduSizeFromRequestor

This attribute specifies the configured value of the maximum length of an FAL-PDU that can be sent from the AREP whose Initiator attribute has a value of “True” to the remote AREP.

This attribute supplies the value for the “Maximum DLSDU sizes from requestor” parameter of the DLL.

MaxDlsduSizeFromResponder

This attribute specifies the configured value of the maximum length of an FAL-PDU that can be sent from the AREP whose Initiator attribute has a value of “False” to the remote AREP.

This attribute supplies the value for the “Maximum DLSDU sizes from responder” parameter of the DLL.

11.6.3.3.3.3 MaxQueueDepth

MaxSendingQueueDepth

This attribute specifies the maximum number of FAL-PDUs that can be queued for transmission.

This attribute supplies the value for the “Maximum queue depth” parameter of the DLL for Send queues.

MaxReceivingQueueDepth

This attribute specifies the maximum number of FAL-PDUs that can be queued at reception.

This attribute supplies the value for the “Maximum queue depth” parameter of the DLL for Receive queues.

11.6.3.4 DLL services

Refer to IEC 61158-3 Type-8 for DLL service descriptions.

11.6.3.5 QUB-TM AREP state machine

11.6.3.5.1 QUB-TM ARPM States

The defined states and their description of the QUB-TM are shown in Table 855 and Figure 227.

Table 855 – QUB-TM ARPM States

State name	Description
ACTIVE	The QUB-TM in the ACTIVE state is ready to transmit or receive primitives to or from the FAL and the ARPM.

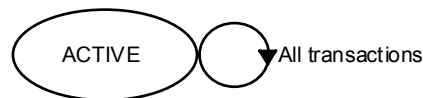


Figure 227 – State Transition Diagram of QUB-TM AREP

11.6.3.5.2 QUB-TM ARPM State Table

The sender transactions are shown in Table 856 and the receiver transactions are shown in Table 857.

Table 856 – QUB-TM State Table - Sender Transactions

#	Current state	Event or condition => action	Next state
S1	ACTIVE	DSA_req => FAL-PDU_req { dmpm_service_name := "DMPM_Data_req", arep_id := GetArepld(), dlsdu := BuildFAL-PDU (fal_pdu_name := " DataSendAcknowledge-PDU", fal_data := user_data; } }	ACTIVE

Table 857 – QUB-TM State Table - Receiver Transactions

#	Current state	Event or condition => action	Next state
S1	ACTIVE	FAL-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && FAL_pdu_type(fal_pdu) = " DataSendAcknowledge-PDU" => DSA_ind { arep_id := GetArepID(), user_data = fal_pdu; }	ACTIVE
S2	ACTIVE	DL_Data.cnf && dl_status = "success" => DSA_cnf(+) { arep_id := GetArepID(), }	ACTIVE
S3	ACTIVE	DL_Data.cnf && dl_status <> "success" => DSA_cnf(-) { arep_id := GetArepID(), reason := dl_status }	ACTIVE
NOTE In state S2 and S3 the Data Link Layer confirmation is used to build the DSA_cnf. The DataSendAcknowledge Service has no FAL response PDU.			

11.6.3.5.3 Functions used by QUB-TM ARPM

The functions used by the QUB-TM ARPM are show in Table 858 through Table 864.

Table 858 – Function GetArepId ()

Name	GetArepId ()	Used in	ARPM
Input		Output	
(none)		AREP Identifier	
Function	Returns a value that can unambiguously identify the current AREP.		

Table 859 – Function BuildFAL-PDU

Name	BuildFAL-PDU	Used in	ARPM
Input		Output	
fal_pdu_name, calling_dlcep_address, called_dlcep_address, fal_data, fal_id,		dlsdu	
Function	Builds an FAL-PDU out of the parameters given as input variables.		

Table 860 – Function FAL_Pdu_Type

Name	FAL_Pdu_Type	Used in	ARPM
Input		Output	
dls_user_data		One of the FAL-PDU types defined in the FAL-PDUs section.	
Function	This function decodes the FAL-PDU that is conveyed in the dls_user_data parameter and retrieves one of the FAL-PDU types.		

Table 861 – Function ResetCounters

Name	ResetCounters	Used in	ARPM
Input		Output	
Function	This function sets OSCC, UCC, OSCP to zero.		

Table 862 – Function IncrementCounter

Name	IncrementCounter	Used in	ARPM
Input		Output	
Function	This function increments the selected counter.		

Table 863 – Function DecrementCounter

Name	DecrementCounter	Used in	ARPM
Input		Output	
Function	This function decrements the selected counter.		

Table 864 – Function GetCounterValue

Name	GetCounterValue	Used in	ARPM
Input		Output	
Function	This function returns the current value of the selected counter.		

11.7 DLL Mapping Protocol Machine

11.7.1 Overview

The DLL Mapping Protocol Machine is common to all the AREP types. Only applicable primitives are different among different AREP types.

Remarks about dl identifiers:

The IEC 61158-3 Type-8 Data Link Layer standard defines two types of identifiers to distinguish each DL-primitive or to match one DL outgoing primitive with its mate incoming primitive. They are suffixed as dl-identifier or DLS-user-identifier. In a real implementation of an FAL-DL interface, these identifications may be achieved by means of a pointer to memory location or a return value of a function call, or something else. Since they are purely a local matter, it is not testable over the Fieldbus network. For this reason, they are not included as parameters of the DMPM primitives.

“dl-identifiers” or “dls-user-identifiers” are mandatory in the DL-services. The FAL assumes that the values of these parameters are provided from DLSAPs or DLCEPs by a local means.

A remark about DLS-user identification:

It is assumed that a connection between one ARPM instance and one DMPM instance is established locally other than a protocol means. Therefore, DLS-user identification parameters are not used in the DMPM primitives.

A remark about buffer or queue identifiers:

The Data Link Layer standard uses parameters to identify a buffer that are shared between the Data Link Layer and the DLS-user. Although they are useful to clarify the operations of the Data Link Layer, none of them affects protocol behavior of the FAL and DL. In a real implementation, these parameters are implementation dependent. Therefore, this standard does not include parameters that directly correspond to these buffer identifiers. A means for identifying the buffers between the FAL and the DL is a local matter.

A remark about initialization of the Data Link Layer:

The FAL assumes that the initialization procedures of the Data Link Layer have been executed prior to the operations of the FAL state machines. Therefore, DL-services to deal with such initialization procedures are not listed in IEC61158-3 Type-8 of this standard.

A remark about DLC establishment:

The FAL assumes that all DLCs have been established prior to the operation of the FAL state machine. Therefore, DL-services to deal with such establishment procedures are not listed.

11.7.2 Primitive definitions

11.7.2.1 Primitives Exchanged between DMPM and ARPM

Table 865 shows the primitives issued by ARPM to DMPM.

Table 865 – Primitives issued by ARPM to DMPM

Primitive names	Source	Associated parameters	Functions
FAL-PDU_req	ARPM	dmpm_service_name, arep_id, called_address ^{*)} , calling_address ^{*)} , responding_address ^{*)} , local_dlcep_address ^{*)} , reason, action_class ^{*)} , dl pdu	This primitive is used to request the DMPM to transfer an FAL-PDU, or to request an abort without transferring an FAL-PDU. It passes the FAL-PDU to the DMPM as a DLSDU. It also carries some of the Data Link Layer parameters that are referenced there.
NOTE These parameters are issued by the ARPM state machines but they are not used by the Data Link Layer.			

Table 866 shows the primitives issued by DMPM to ARPM

Table 866 – Primitives issued by DMPM to ARPM

Primitive names	Source	Associated parameters	Functions
FAL-PDU_ind	DMPM	dmpm_service_name, originator, reason, duplicate_dlsdu, calling_address, fal_pdu, local_dle_timeliness, remote_dle_timeliness	This primitive is used to pass an FAL-PDU received as a Data Link Layer service data unit to a designated ARPM. It also carries some of the Data Link Layer parameters that are referenced in the ARPM.
Connect_ind	DMPM	calling_dlcep_address, dlsdu_size_from_requestor, dlsdu_size_from_responder, dls_user_data	This primitive is used to convey a DL_Connect.ind primitive to the ARPM to process connection establishment. All the parameters that are associated with the DL_Connect.ind primitive are carried with this primitive.
Connect_cnf	DMPM	dll_priority, dlsdu_size_from_requestor, dlsdu_size_from_responder, dls_user_data	This primitive is used to convey a DL_Connect.cnf primitive to the ARPM. All the parameters that are associated with the DL_Connect.cnf are carried with this primitive.
ErrorToARPM	DMPM	originator, reason	This primitive is used to convey selected communication errors reported by the Type8 Data Link Layer to a designated ARPM.

11.7.2.2 Parameters of ARPM/DMPM primitives

The parameters used with the primitives exchanged between the ARPM and the Type-8 DMPM are described in Table 867.

Table 867 – Parameters used with Primitives Exchanged between ARPM and DMPM

Parameter name	Description
arep_id	This parameter carries a local identifier to specify the associated AR instance.
action_class	This parameter conveys the value of the dl_action_class parameter.
calling_address	This parameter conveys the value of the dl_calling_address.
calling_dlcep_address	This parameter conveys the value of the RequestingAREP parameter supplied with the EST_ReqPDU.
called_address	This parameter conveys the value of the dl_called_address parameter.
dmpm_service_name	This parameter conveys a Type8 Data Link Layer service name. Possible values are all the DL-X.yyy primitives defined in this section and are represented as DMPM_X.yyy.
local_dlcep_address	This parameter conveys the value of the dl_dlcep_address parameter.
duplicate_dlsdu	As duplicate_dlsdu detection is not supported by the Data Link Layer this attribute is always set to "False".
dll_priority	As no different priorities are supported by the Data Link Layer this attribute is always set to "normal".
dlsdu_size_from_requestor	This parameter conveys the value of the dl_dlsdu_size_from_requestor parameter.
dlsdu_size_from_responder	This parameter conveys the value of the dl_dlsdu_size_from_responder parameter.
dls_user_data	This parameter conveys the value of the dl_dls_user_data parameter.
dlsdu	This parameter conveys the value of the dl_dls_user_data parameter.
fal_pdu	This parameter conveys the value of the dl_dls_user_data parameter.
remote_dle_timeliness	As timeliness is not supported by the Data Link Layer this attribute is always set to "False".
local_dle_timeliness	As timeliness is not supported by the Data Link Layer this attribute is always set to "False".
originator	This parameter conveys the value of the dl_originator parameter.
reason	This parameter conveys the value of the dl_reason parameter.
responding_address	This parameter conveys the value of the dl_responding_address parameter.

11.7.2.3 Primitives exchanged between Data Link Layer and DMPM

NOTE The DLL primitives (see Table 868)and their parameters are defined in IEC 61158-3.

Table 868 – Primitives exchanged between Type8 Data Link Layer and DMPM

Primitive names	Source	Associated parameters
DL_Data.ind	Data Link Layer	dl_dls_user_data
DL_Data.cnf	Data Link Layer	dl_status
DL_Buffer_Received.ind	Data Link Layer	dl_status
DL_Get.cnf(+)	Data Link Layer	dl_dls_user_data
DL_Get.cnf(-)	Data Link Layer	dl_error_type
DL_Put.cnf(+)	Data Link Layer	(none)
DL_Put.cnf(-)	Data Link Layer	dl_error_type
DL_Data.req	DMPM	dl_dls_user_data
DL_Get.req	DMPM	(none)
DL_Put.req	DMPM	dl_dls_user_data

11.7.2.4 Parameters of DMPM/Data Link Layer primitives

The parameters used with the primitives exchanged between the DMPM and the Data Link Layer are defined in the Type 8 DLL Service Definition (IEC 61158-3). They are prefixed by “dl_” to indicate that they are used by the FAL.

11.7.3 DMPM state machine

11.7.3.1 DMPM States

The defined state of the DMPM together with its description listed in Table 869 and Figure 228 shows the corresponding state transition diagram.

Table 869 – Type-8 DMPM state descriptions

State name	Description
ACTIVE	The DMPM in the ACTIVE state is ready to transmit or receive primitives to or from the Data Link Layer and the ARPM.

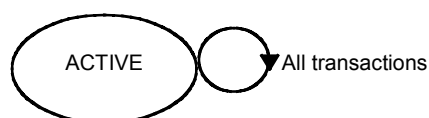


Figure 228 – State transition diagram of Type-8 DMPM

11.7.3.2 DMPM state table

NOTE 1 Although each primitive contains all the available parameters, only those applicable to particular ARPM are relevant.

NOTE 2 Parameters starting with a capital letter, “DlcepClass” for instance, refer to those defined in the attribute list of each ARPM. Therefore, they are not conveyed by the service primitives defined here.

Table 870 shows the Sender Transaction State Table:

Table 870 – Type-8 DMPM state table – sender transactions

#	Current state	Event or condition => action	Next state
S1a1	ACTIVE	FAL-PDU_req && DILinkStatus() = "Unlinked" && (DMPM_service_name = "DMPM_Connect_req" DMPM_service_name = "DMPM_Connect_rsp" DMPM_service_name = "DMPM_Data_req" DMPM_service_name = "DMPM_Put_req") => PickArep (arep_id), FAL-PDU_ind { DMPM_service_name := "DMPM_Disconnect_ind", originator := local_dls_provider, reason := "Unlinked", fal_pdu := "null" } }	ACTIVE
S1a2	ACTIVE	FAL-PDU_req && DILinkStatus() = "Unlinked" && DMPM_service_name = "DMPM_Disconnect_req" && (ArepRole(arep_id) = "Client" ArepRole(arep_id) = "Server" ArepRole(arep_id) = "Peer") => (no actions taken)	ACTIVE
S1b1	ACTIVE	FAL-PDU_req && DILinkStatus() = "Linked" && DMPM_service_name = "DMPM_Connect_req" && (ArepRole(arep_id) = "Client" ArepRole(arep_id) = "Peer") => PickArep (arep_id), DL_Data.req { dl_dls_user_data := dlsdu } }	ACTIVE
S1b2	ACTIVE	FAL-PDU_req && DILinkStatus() = "Linked" && DMPM_service_name = "DMPM_Connect_req" && ArepRole(arep_id) = "PushPublisher" => PickArep (arep_id), Connect_cnf { dlsdu_size_from_requestor := MaxDlsduSizeFromRequestor, dlsdu_size_from_responder := "null", dls_user_data := "null" } }	ACTIVE
S1b3	ACTIVE	FAL-PDU_req && DILinkStatus() = "Linked" && DMPM_service_name = "DMPM_Connect_req" && ArepRole(arep_id) = "PushSubscriber" => PickArep (arep_id), Connect_cnf { dlsdu_size_from_requestor := "null", dlsdu_size_from_responder := MaxDlsduSizeFromResponder, dls_user_data := "null" } }	ACTIVE

Table 870 (continued)

#	Current state	Event or condition => action	Next state
S2	ACTIVE	FAL-PDU_req && DLinkStatus() = "Linked" && DMPM_service_name = "DMPM_Connect_rsp" => PickArep (arep_id), DL_Data.req { dl_dls_user_data := dlsdu } FAL-PDU_ind { dmpm_service_name := "DMPM_Connection_Established_ind", fal_pdu := "null" }	ACTIVE
S3a	ACTIVE	FAL-PDU_req && DLinkStatus() = "Linked" && DMPM_service_name = "DMPM_Disconnect_req" && (ArepRole(arep_id) = "Client" ArepRole(arep_id) = "Server" ArepRole(arep_id) = "Peer" => PickArep (arep_id), DL_Data.req { dl_dls_user_data := dlsdu }	ACTIVE
S3b	ACTIVE	FAL-PDU_req && DMPM_service_name = "DMPM_Disconnect_req" && (ArepRole(arep_id) = "PushPublisher" ArepRole(arep_id) = "PushSubscriber") => (no actions taken)	ACTIVE
S4	ACTIVE	FAL-PDU_req && DLinkStatus() = "Linked" && DMPM_service_name = "DMPM_Data_req" => PickArep (arep_id), DL_Data.req { dl_dls_user_data := dlsdu }	ACTIVE
S6a	ACTIVE	FAL-PDU_req && DLinkStatus() = "Linked" && DMPM_service_name = "DMPM_Put_req" && ArepRole(arep_id) <> "PushPublisher" => PickArep (arep_id), DL_Put.req { dl_dls_user_data := dlsdu } DL_Put.cnf(+) -- This is an immediate response. => FAL-PDU_ind { DMPM_service_name := "DMPM_Put_cnf", reason := "success" } DL_Put.cnf(-) -- This is an immediate response. => FAL-PDU_ind { DMPM_service_name := "DMPM_Put_cnf", reason := dl_error_type } ErrorToARPM { originator := "local_dls_provider", reason := dl_error_type }	ACTIVE

Table 870 (continued)

#	Current state	Event or condition => action	Next state
S6b	ACTIVE	FAL-PDU_req && DILinkStatus() = "Linked" && DMPM_service_name = "DMPM_Put_req" && ArepRole(arep_id) = "PushPublisher" => PickArep (arep_id), DL_Put.req { dl_dls_user_data := RemoveUcsPduHeader(dlsdu) } DL_Put.cnf(+) -- This is an immediate response. => FAL-PDU_ind { DMPM_service_name := "DMPM_Put_cnf", reason := "success" } DL_Put.cnf(-) -- This is an immediate response. => FAL-PDU_ind { DMPM_service_name := "DMPM_Put_cnf", reason := dl_error_type } ErrorToARPM { originator := "local_dls_provider", reason := dl_error_type }	ACTIVE
S7a	ACTIVE	FAL-PDU_req && DILinkStatus() = "Linked" && DMPM_service_name = "DMPM_Get_req" && ArepRole(arep_id) <> "PushSubscriber" => PickArep (arep_id), DL_Get.req { } DL_Get.cnf(+) -- This is an immediate response. => FAL-PDU_ind { DMPM_service_name := "DMPM_Get_cnf", reason := "success", fal_pdu := dl_dls_user_data } DL_Get.cnf(-) -- This is an immediate response. => FAL-PDU_ind { DMPM_service_name := "DMPM_Get_cnf", reason := dl_error_type }	ACTIVE

Table 870 (continued)

#	Current state	Event or condition => action	Next state
S7b	ACTIVE	FAL-PDU_req && DILinkStatus() = "Linked" && DMPM_service_name = "DMPM_Get_req" && ArepRole(arep_id) = "PushSubscriber" => PickArep (arep_id), DL_Get.req { } DL_Get.cnf(+) -- This is an immediate response. => FAL-PDU_ind { DMPM_service_name := "DMPM_Get_cnf", reason := "success", fal_pdu := AddUcsPduHeader(dl_dls_user_data) } DL_Get.cnf(-) -- This is an immediate response. => FAL-PDU_ind { DMPM_service_name := "DMPM_Get_cnf", reason := dl_error_type }	ACTIVE
S8a	ACTIVE	FAL-PDU_req && DILinkStatus() = "Unlinked" && DMPM_service_name = "DMPM_Compel_req" => PickArep (arep_id), FAL-PDU_ind { DMPM_service_name := "DMPM_Compel_Service_cnf", reason := "failure-inappropriate request" }	ACTIVE
S8b	ACTIVE	FAL-PDU_req && DILinkStatus() = "Linked" && DMPM_service_name = "DMPM_Compel_req" => PickArep (arep_id), FAL-PDU_ind { DMPM_service_name := "DMPM_Compel_Service_cnf", reason := "failure-inappropriate request" }	ACTIVE

Table 871 shows the Receiver Transaction State Table:

Table 871 – Type-8 DMPM state table – receiver transactions

#	Current state	Event or condition => action	Next State
R15a	ACTIVE	DL_Data.ind && FindArep() <> "True" => (no actions taken)	ACTIVE
R16a	ACTIVE	DL_Data.ind && FindArep() = "True" && FalArHeader (dl_dls_user_data) = "Establish-Request-PDU" => Connect_ind { calling_dlcep_address := RemoteDlcepAddress, dlsdu_size_from_requestor := MaxDlsduSizeFromRequestor, dlsdu_size_from_responder := MaxDlsduSizeFromResponder, dls_user_data := dl_dls_user_data }	ACTIVE
R16b	ACTIVE	DL_Data.ind && FindArep() = "True" && FalArHeader (dl_dls_user_data) = "Establish-Response-PDU" => Connect_cnf { dll_priority := "normal", dlsdu_size_from_requestor := MaxDlsduSizeFromRequestor, dlsdu_size_from_responder := MaxDlsduSizeFromResponder, dls_user_data := dl_dls_user_data }	ACTIVE
R16c	ACTIVE	DL_Data.ind && FindArep() = "True" && (FalArHeader (dl_dls_user_data) = "Establish-Error-PDU" FalArHeader(dl_dls_user_data) = "Abort-PDU") => FAL-PDU_ind{ DMPM_service_name = "DMPM_Disconnect_ind", fal_pdu := LSDU }	ACTIVE
R16d	ACTIVE	DL_Data.ind && FindArep() = "True" && FalArHeader(dl_dls_user_data) <> "Establish-Request-PDU" && FalArHeader(dl_dls_user_data) <> "Establish-Response-PDU" && FalArHeader(dl_dls_user_data) <> "Establish-Error-PDU" && FalArHeader(dl_dls_user_data) <> "Abort-PDU" => FAL-PDU_ind { DMPM_service_name := "DMPM_Data_ind", fal_pdu := LSDU }	ACTIVE
R18	ACTIVE	DL_Data.cnf && dl_status = "OK" => (no actions taken)	ACTIVE
R19a	ACTIVE	DL_Data.cnf && dl_status <> "OK" && FindArep() = "False" => (no actions taken)	ACTIVE
R19b	ACTIVE	DL_Data.cnf && dl_status <> "OK" && FindArep() = "True" => FAL-PDU_ind { DMPM_service_name := "DMPM_Data_cnf", reason := dl_status } ErrorToARPM { originator := "local_dls_provider", reason := dl_status }	ACTIVE

#	Current state	Event or condition => action	Next State
R21	ACTIVE	DL_Buffer_Received.ind => --loop through all AREPs arep_id := SelectNextArep() if (ArepRole(arep_id) = "PushSubscriber") DL_Get.req { } DL_Get.cnf(+) FAL-PDU_ind { DMPM_service_name := "DMPM_Buffer_Received_ind", duplicate_dlsdu := "False", fal_pdu := AddUcsPduHeader(dl_dls_user_data), local_dle_timeliness := "False", remote_dle_timeliness := "False" } DL_Get.cnf(-) FAL-PDU_ind { DMPM_service_name := "DMPM_Get_cnf", reason := dl_error_type } ErrorToARPM { originator := "local_dls_provider", reason := dl_error_type } endif if (ArepRole(arep_id) = "PushPublisher") FAL-PDU_ind { DMPM_service_name := "DMPM_Buffer_Sent_ind" } endif	ACTIVE

11.7.3.3 Functions used by Type-8 DMPM

Table 872 through Table 879 show the functions used by the DMPM

Table 872 – Function PickArep

Name	PickArep	Used in	DMPM
Input		Output	
arep_id		(all the attributes of the specified AREP)	
Function	Selects the attributes for the AREP specified by the arep_id parameter. After this function is executed, the attributes of the selected AREP are available to the state machine.		

Table 873 – Function FindAREP

Name	FindAREP	Used in	DMPM
Input		Output	
(local mapping)		True False	
Function		This function identifies the AREP that shall be bound with an active DMPM. True means the AREP exists. If it does, this function also returns a means to send a DMPM primitive to that AREP.	

Table 874 – Function SelectNextArep

Name	SelectNextAREP	Used in	DMPM
Input		Output	
(none)		arep_id	
Function	Returns the arep_id parameter of the next existing AREP, starting with the first one.		

Table 875 – Function ArepRole

Name	ArepRole	Used in	DMPM
Input		Output	
arep_id		(the value of the Role attribute of the specified AREP)	
Function			
This function returns the Role attribute of the AREP specified by the arep_id parameter.			

Table 876 – Function FalArHeader

Name	FalArHeader	Used in	DMPM
Input		Output	
dl_dls_user_data		(the FAL header)	
Function	This function decodes the dlsdu conveyed by the dl_dls_user_data argument and returns the FAL-PDU type.		

Table 877 – Function AddUcsPduHeader

Name	AddUcsPduHeader	Used in	DMPM
Input		Output	
dl_dls_user_data		fal_pdu	
Function	Adds the UCS-PDU header to the dl_dls_user_data received from the DLE.		

Table 878 – Function RemoveUcsPduHeader

Name	FalArHeader	Used in	DMPM
Input		Output	
fal_pdu		dlsdu	
Function	Removes the UCS-PDU header from the FAL-PDU received from an AREP		

Table 879 – Function DILinkStatus

Name	DILinkStatus	Used in	DMPM
Input		Output	
(none)		Linked Unlinked	
Function	This function returns whether or not the local DIs-provider is synchronized at the moment.		

12 Type 9

12.1 FAL syntax description

12.1.1 FAL-AR header abstract syntax

12.1.1.1 Abstract syntax for Type 1 Data Link Layer

12.1.1.1.1 FAL-AR header

This subclause specifies the abstract and transfer syntax of the AR Header for use over the Type 1 Data Link Layer.

Bit 8 of the Header is a boolean parameter that indicates, when 1 (TRUE), that Type 9 services are being conveyed.

```
Type9ArHeader ::= Unsigned 8 {
  (0),      -- 0 0 0 0 0 0 0 0    ASC_ReqPDU          Revision1
  (1),      -- 0 0 0 0 0 0 0 1    ASC_RspPDU          Revision1
  (2),      -- 0 0 0 0 0 0 1 0    ASC_ErrPDU           Revision1
  (3),      -- 0 0 0 0 0 0 1 1    AbortPDU             Revision1
  (4),      -- 0 0 0 0 0 1 0 0    DTC_ReqPDU           Revision1
  (5),      -- 0 0 0 0 0 1 0 1    DTC_RspPDU           Revision1
  (6),      -- 0 0 0 0 0 1 1 0    DTU_PDU              Revision1
  (7..255)  -- Reserved for future use
}
```

12.1.1.1.2 Associate PDUs

```
ASC_ReqPDU ::= SEQUENCE {
  FalArHeader,
  RespondingAREP,
  RequestingAREP,
  UserData          -- Initiate-Request PDU
}

Associate-AffirmativePDU ::= SEQUENCE {
  FalArHeader,
  UserData          -- Initiate-Response PDU
}

Associate-NegativePDU ::= SEQUENCE {
  FalArHeader,
  PDUBody          -- Initiate-Error PDU
}
```

12.1.1.1.3 Abort PDU

```
Abort-PDU ::= SEQUENCE {
  FalArHeader,
  Identifier,
  ReasonCode,
  AdditionalDetail
}
```

12.1.1.1.4 Data Transfer Confirmed (DTC) PDUs

This PDU is used to convey the Confirmed Send service.

```
DTC-ReqPDU ::= SEQUENCE {
  FalArHeader,
  PDUBody          -- Confirmed Service Request PDU
}
```

```
DTC-RspPDU ::= SEQUENCE {
    FaArHeader,
    PDUBody
}
-- Confirmed Service Response or Error PDU
```

12.1.1.1.5 Data Transfer Unconfirmed (DTC) PDU

This PDU is used to convey the Unconfirmed Send service.

```
DTU_PDU ::= SEQUENCE {
    FaArHeader,
    PDUBody
}
-- Unconfirmed Service Request PDU
```

12.1.1.1.6 RespondingAREP

RespondingAREP ::= Unsigned32

12.1.1.1.7 RequestingAREP

RequestingAREP ::= Unsigned32

12.1.1.1.8 Identifier

Identifier ::= Unsigned8

12.1.1.1.9 ReasonCode

ReasonCode ::= Unsigned8

12.1.1.1.10 AdditionalDetail

AdditionalDetail ::= OctetString16

12.1.2 Abstract Syntax of PDUBody

12.1.2.1 General

The APDU consists of a fixed part of 3 octets that identifies the service primitive being conveyed, and a part of variable length that contains service specific information. Not all PDUs need a part with variable length. The abstract syntax of the PDUBody is described in the formal description language ASN.1.

12.1.2.2 Fixed Part

The fixed part consists of two identifiers separated by an invoke id. The first identifier identifies the type of APDU (e.g. Confirmed Request, Confirmed Response). The second identifier identifies the service. See Table 880.

Table 880 – Field with Fixed Format

First ID Info:	Invoke ID Second ID
1. Confirmed Request	Info: e.g. Read, (Integer8) Write, etc.
2. Confirmed Response	
3. Confirmed Error	
4. Unconfirmed PDU	
5. Reject PDU	
6. Initiate PDU	

The abstract syntax for the fixed part is shown below, followed by the abstract syntax for the elements of the fixed part.

```

FMSpdu ::= CHOICE {
    confirmed-RequestPDU          [1] IMPLICIT Confirmed-RequestPDU,
    confirmed-ResponsePDU        [2] IMPLICIT Confirmed-ResponsePDU,
    confirmed-ErrorPDU           [3] IMPLICIT Confirmed-ErrorPDU,
    unconfirmed-PDU              [4] IMPLICIT Unconfirmed-PDU,
    reject-PDU                   [5] IMPLICIT Reject-PDU,
    initiate-PDU                 [6] IMPLICIT Initiate-PDU
}

Confirmed-RequestPDU ::= SEQUENCE {
    invokeID                      InvokeID,
    confirmedServiceRequest       ConfirmedServiceRequest
}

Confirmed-ResponsePDU ::= SEQUENCE {
    invokeID                      InvokeID,
    confirmedServiceResponse      ConfirmedServiceResponse
}

Confirmed-ErrorPDU ::= SEQUENCE {
    invokeID                      InvokeID,
    serviceError                  ServiceError
}

Unconfirmed-PDU ::= SEQUENCE {
    invokeID                      InvokeID,
    unconfirmed                   UnconfirmedService           -- Dummy
}

Reject-PDU ::= SEQUENCE {
    invokeID                      InvokeID,
    reject                        Reject                       -- Dummy
}

Initiate-PDU ::= SEQUENCE {
    invokeID                      InvokeID,
    initiatepdu                   InitiatePDU                 -- Dummy
}

InvokeID ::= Integer8

```

12.1.2.3 ConfirmedServiceRequest

```
ConfirmedServiceRequest ::= CHOICE {
    status [0] IMPLICIT Status-Request,
    identify [1] IMPLICIT Identify-Request,
    read [2] IMPLICIT Read-Request,
    write [3] IMPLICIT Write-Request,
    getOD [4] IMPLICIT GetOD-Request,
    defineVariableList [7] IMPLICIT DefineVariableList-Request,
    deleteVariableList [8] IMPLICIT DeleteVariableList-Request,
    initiateDownloadSequence [9] IMPLICIT InitiateDownloadSequence-Request,
    downloadSegment [10] IMPLICIT DownloadSegment-Request,
    terminateDownloadSequence [11] IMPLICIT TerminateDownloadSequence-Request,
    initiateUploadSequence [12] IMPLICIT InitiateUploadSequence-Request,
    uploadSegment [13] IMPLICIT UploadSegment-Request,
    terminateUploadSequence [14] IMPLICIT TerminateUploadSequence-Request,
    requestDomainDownload [15] IMPLICIT RequestDomainDownload-Request,
    requestDomainUpload [16] IMPLICIT RequestDomainUpload-Request,
    createProgramInvocation [17] IMPLICIT CreateProgramInvocation-Request,
    deleteProgramInvocation [18] IMPLICIT DeleteProgramInvocation-Request,
    start [19] IMPLICIT Start-Request,
    stop [20] IMPLICIT Stop-Request,
    resume [21] IMPLICIT Resume-Request,
    reset [22] IMPLICIT Reset-Request,
    kill [23] IMPLICIT Kill-Request,
    alterEventConditionMonitoring [24] IMPLICIT AlterEventConditionMonitoring-Request,
    acknowledgeEventNotification [25] IMPLICIT AcknowledgeEventNotification-Request,
    initiatePutOD [28] IMPLICIT InitiatePutOD-Request,
    putOD [29] IMPLICIT PutOD-Request,
    terminatePutOD [30] IMPLICIT TerminatePutOD-Request,
    genericInitiateDownloadSequence [31] IMPLICIT GenericInitiateDownloadSequence-Request,
    genericDownloadSegment [32] IMPLICIT GenericDownloadSegment-Request,
    genericTerminateDownloadSequence [33] IMPLICIT GenericTerminateDownloadSequence-Request
}
```

12.1.2.4 ConfirmedServiceResponse

```
ConfirmedServiceResponse ::= CHOICE {
    status [0] IMPLICIT Status-Response,
    identify [1] IMPLICIT Identify-Response,
    read [2] IMPLICIT Read-Response,
    write [3] IMPLICIT Write-Response,
    getOD [4] IMPLICIT GetOD-Response,
    defineVariableList [7] IMPLICIT DefineVariableList-Response,
    deleteVariableList [8] IMPLICIT DeleteVariableList-Response,
    initiateDownloadSequence [9] IMPLICIT InitiateDownloadSequence-Response,
    downloadSegment [10] IMPLICIT DownloadSegment-Response,
    terminateDownloadSequence [11] IMPLICIT TerminateDownloadSequence-Response,
    initiateUploadSequence [12] IMPLICIT InitiateUploadSequence-Response,
    uploadSegment [13] IMPLICIT UploadSegment-Response,
    terminateUploadSequence [14] IMPLICIT TerminateUploadSequence-Response,
    requestDomainDownload [15] IMPLICIT RequestDomainDownload-Response,
    requestDomainUpload [16] IMPLICIT RequestDomainUpload-Response,
    createProgramInvocation [17] IMPLICIT CreateProgramInvocation-Response,
    deleteProgramInvocation [18] IMPLICIT DeleteProgramInvocation-Response,
    start [19] IMPLICIT Start-Response,
    stop [20] IMPLICIT Stop-Response,
    resume [21] IMPLICIT Resume-Response,
    reset [22] IMPLICIT Reset-Response,
    kill [23] IMPLICIT Kill-Response,
    alterEventConditionMonitoring [24] IMPLICIT AlterEventConditionMonitoring-Response,
    acknowledgeEventNotification [25] IMPLICIT AcknowledgeEventNotification-Response,
    initiatePutOD [28] IMPLICIT InitiatePutOD-Response,
    putOD [29] IMPLICIT PutOD-Response,
    terminatePutOD [30] IMPLICIT TerminatePutOD-Response,
    genericInitiateDownloadSequence [31] IMPLICIT GenericInitiateDownloadSequence-Response,
    genericDownloadSegment [32] IMPLICIT GenericDownloadSegment-Response,
    genericTerminateDownloadSequence [33] IMPLICIT GenericTerminateDownloadSequence-Response
}
```

12.1.2.5 ServiceError

12.1.2.5.1 Top Level Definition

```
ServiceError ::= CHOICE {
    status [0] IMPLICIT Error-Type,
    identify [1] IMPLICIT Error-Type,
    read [2] IMPLICIT Error-Type,
    write [3] IMPLICIT Error-Type,
    getOD [4] IMPLICIT Error-Type,
    defineVariableList [7] IMPLICIT Error-Type,
    deleteVariableList [8] IMPLICIT Error-Type,
    initiateDownloadSequence [9] IMPLICIT Error-Type,
    downloadSegment [10] IMPLICIT Error-Type,
    terminateDownloadSequence [11] IMPLICIT Error-Type,
    initiateuploadSequence [12] IMPLICIT Error-Type,
    uploadSegment [13] IMPLICIT Error-Type,
    terminateUploadSequence [14] IMPLICIT Error-Type,
    requestDomainDownload [15] IMPLICIT Error-Type,
    requestDomainUpload [16] IMPLICIT Error-Type,
    createProgramInvocation [17] IMPLICIT Error-Type,
    deleteProgramInvocation [18] IMPLICIT Error-Type,
    start [19] IMPLICIT PI-Error-Type,
    stop [20] IMPLICIT PI-Error-Type,
    resume [21] IMPLICIT PI-Error-Type,
    reset [22] IMPLICIT PI-Error-Type,
    kill [23] IMPLICIT Error-Type,
    alterEventConditionMonitoring [24] IMPLICIT Error-Type,
    acknowledgeEventNotification [25] IMPLICIT Error-Type,
    initiatePutOD [28] IMPLICIT Error-Type,
    putOD [29] IMPLICIT Error-Type,
    terminatePutOD [30] IMPLICIT OD-Error-Type,
    genericInitiateDownloadSequence [31] IMPLICIT Error-Type,
    genericDownloadSegment [32] IMPLICIT Error-Type,
    genericTerminateDownloadSequence [33] IMPLICIT Error-Type
}
```

12.1.2.5.2 Error Type

```
Error-Type ::= SEQUENCE {
    error-class [0] IMPLICIT Error-Class,
    additional-code [1] IMPLICIT Integer16 OPTIONAL,
    additional-description [2] IMPLICIT VISIBLE STRING OPTIONAL
}
```

12.1.2.5.3 PI Error Type

```
PI-Error-Type ::= SEQUENCE {
    error-class [0] IMPLICIT Error-Class,
    additional-code [1] IMPLICIT Integer16 OPTIONAL,
    additional-description [2] IMPLICIT VISIBLE STRING OPTIONAL,
    pi-state [3] IMPLICIT ProgramInvocationState
}
```

12.1.2.5.4 OD Error Type

```
ODErrorType ::= SEQUENCE {
    error-class [0] IMPLICIT Error-Class,
    additional-code [1] IMPLICIT Integer16 OPTIONAL,
    additional-description [2] IMPLICIT VISIBLE STRING OPTIONAL,
    index [3] IMPLICIT Index
}
```

12.1.2.5.5 Error Class

```

error-class ::= CHOICE {
    vfd-state [1] IMPLICIT Integer8 {
        other (0)
    },
    application-reference [2] IMPLICIT Integer8 {
        other (0),
        application-unreachable (1)
    },
    definition [3] IMPLICIT Integer8 {
        other (0),
        object-undefined (1),
        object-attributes-inconsistent (2),
        name-already-exists (3)
    },
    resource [4] IMPLICIT Integer8 {
        other (0),
        memory-unavailable (1)
    },
    service [5] IMPLICIT Integer8 {
        other (0),
        object-state-conflict (1),
        pdu-size (2),
        object-constraint-conflict (3),
        parameter-inconsistent (4),
        illegal-parameter (5)
    },
    access [6] IMPLICIT Integer8 {
        other (0),
        object-invalidated (1),
        hardware-fault (2),
        object-access-denied (3),
        invalid-address (4),
        object-attribute-inconsistent (5),
        object-access-unsupported (6),
        object-non-existent (7),
        type-conflict (8),
        named-access-unsupported (9),
        access-to-element-unsupported (10)
    },
    od [7] IMPLICIT Integer8 {
        other (0),
        name-length-overflow (1),
        od-overflow (2),
        od-write-protected (3),
        extension-length-overflow (4),
        od-description-length-overflow (5),
        operational-problem (6)
    },
    other [8] IMPLICIT Integer8 {
        other (0)
    }
}

```

12.1.2.5.6 Unconfirmed PDUs

```

UnconfirmedService ::= CHOICE {
    informationReport [0] IMPLICIT InformationReport,
    unsolicitedStatus [1] IMPLICIT UnsolicitedStatus,
    eventNotification [2] IMPLICIT EventNotification
}

```

12.1.2.5.7 Reject

```

Reject ::= [0] IMPLICIT SEQUENCE {
    original-invokeld
    reject-code
    other
    PDU-Size
}
[0] IMPLICIT Integer8,
[1] IMPLICIT Integer8 {
    (0),
    (5)
}

```

12.1.2.5.8 Initiate PDUs

```

InitiatePDU ::= CHOICE {
    init-request
    init-response
    init-error
}
[0] IMPLICIT Initiate-RequestPDU,
[1] IMPLICIT Initiate-ResponsePDU,
[2] IMPLICIT Initiate-ErrorPDU

```

12.1.2.5.9 General Substitutions

Index ::= Unsigned16

Name ::= VISIBLE STRING

Subindex ::= Unsigned8

Local-Address ::= Unsigned32

Length ::= Unsigned8

MoreFollows ::= BOOLEAN

Data ::= PACKED

```

Typedescription ::= SEQUENCE OF CHOICE {
    Simple
    array
    structure
}
[1] IMPLICIT PACKED,
    -- Octet1: Hioctet Data Type Index
    -- Octet2: Looctet Data Type Index
    -- Octet3: Length
[2] IMPLICIT PACKED,
    -- Octet1: Hioctet Data Type Index
    -- Octet2: Looctet Data Type Index
    -- Octet3: Length
    -- Octet4: Number Of Elements
[3] IMPLICIT PACKED
    -- Octet 1 : Hioctet Data Type Index (Element 1)
    -- Octet 2 : Looctet Data Type Index (Element 1)
    -- Octet 3 : Length (Element 1)
    -- Octet 3n-2 : Hioctet Data Type Index (Element n)
    -- Octet 3n-1 : Looctet Data Type Index (Element n)
    -- Octet 3n-0 : Length (Element n)

```

Integer8 ::= INTEGER -- 8 Bit Integer

Integer16 ::= INTEGER -- 16 Bit Integer

Integer32 ::= INTEGER -- 32 Bit Integer

Unsigned8 ::= UNSIGNED -- 8 Bit Unsigned

Unsigned16 ::= UNSIGNED -- 16 Bit Unsigned

Unsigned32 ::= UNSIGNED -- 32 Bit Unsigned

12.1.2.6 VFD Support

12.1.2.6.1 Status

Status-Request ::= NULL

```
Status-Response ::= SEQUENCE {
    logical-status          [0] IMPLICIT Unsigned8 {
        state-changes-allowed      (0),
        limited-services-permitted  (2),
        od-loading-non-interacting  (4),
        od-loading-interacting     (5)
    },
    physical-status        [1] IMPLICIT Unsigned8 {
        operational              (0),
        partially-operational    (1),
        inoperable              (2),
        needs-commissioning     (3)
    },
    local-detail           [2] IMPLICIT BIT STRING OPTIONAL
                           -- Length 24 Bit
}
```

12.1.2.6.2 Identify

Identify-Request ::= NULL

```
Identify-Response ::= SEQUENCE {
    vendor-name      [0] IMPLICIT VISIBLE STRING,
    model-name       [1] IMPLICIT VISIBLE STRING,
    revision         [2] IMPLICIT VISIBLE STRING
}
```

12.1.2.6.3 UnsolicitedStatus

UnsolicitedStatus ::= Status-Response

12.1.2.7 OD Management

12.1.2.7.1 Object Description and OD Description

Objectdescription ::= PACKED (see section "List of packed fields")

12.1.2.7.2 GetOD

```
GetOD-Request ::= SEQUENCE {
    all-attributes          [0] IMPLICIT BOOLEAN,
    access-specification CHOICE {
        index               [1] IMPLICIT Index,
        variable-name       [2] IMPLICIT Name,
        variable-list-name  [3] IMPLICIT Name,
        domain-name         [4] IMPLICIT Name,
        pi-name             [5] IMPLICIT Name,
        event-name          [6] IMPLICIT Name,
        startindex          [7] IMPLICIT Index
    }
}

GetOD-Response ::= SEQUENCE {
    list-of-objectdescription [0] IMPLICIT SEQUENCE OF {
        objectdescription    [0] IMPLICIT Objectdescription
    },
    more-follows             [1] IMPLICIT MoreFollows
}
```


12.1.2.7.3 InitiatePutOD

InitiatePutOD-Request ::= Consequence

```
Consequence ::= Integer8 {
    od-loading-non-interacting      (0),
    od-appending-interacting        (1),
    od-fresh-loading-interacting    (2)
}
```

InitiatePutOD-Response ::= NULL

12.1.2.7.4 PutOD

```
PutOD-Request ::= SEQUENCE OF {
    objectdescription [0] IMPLICIT Objectdescription
}
```

PutOD-Response ::= NULL

12.1.2.7.5 TerminatePutOD

TerminatePutOD-Request ::= NULL

TerminatePutOD-Response ::= NULL

12.1.2.8 Context Management**12.1.2.8.1 AccessControl**

```
Accesscontrol ::= BIT STRING {
    Password_Bit1      (7),
    Password_Bit2      (6),
    Password_Bit3      (5),
    Password_Bit4      (4),
    Password_Bit5      (3),
    Password_Bit6      (2),
    Password_Bit7      (1),
    Password_Bit8      (0),
    Access_Groups-1    (15),
    Access_Groups-2    (14),
    Access_Groups-3    (13),
    Access_Groups-4    (12),
    Access_Groups-5    (11),
    Access_Groups-6    (10),
    Access_Groups-7    (9),
    Access_Groups-8    (8)
}
-- The Password (Unsigned8) is encoded as a bit string.
```

12.1.2.8.2 Initiate

```
Initiate-RequestPDU ::= SEQUENCE {
    version-od-calling      [0] IMPLICIT Integer16,
    profile-number-calling  [1] IMPLICIT OCTET STRING,
    access-protection-supported-calling [2] IMPLICIT BOOLEAN,
    password_and_access-groups-calling [3] IMPLICIT Accesscontrol,
    max-fms-pdu-sending-calling [4] IMPLICIT Unsigned8,
    max-fms-pdu-receiving-calling [5] IMPLICIT Unsigned8,
    fms-features-supported-calling [6] IMPLICIT BIT STRING,
    fms-max-scc-calling      [7] IMPLICIT Unsigned8,
    fms-max-rcc-calling      [8] IMPLICIT Unsigned8
}
```

```
Initiate-ResponsePDU ::= SEQUENCE {
    version-od-called      [0] IMPLICIT Integer16,
    profile-number-called  [1] IMPLICIT OCTET STRING,
    access-protection-supported-called [2] IMPLICIT BOOLEAN,
    password_and_access-groups-called [3] IMPLICIT Accesscontrol,
    fms-max-scc-called     [4] IMPLICIT Unsigned8,
    fms-max-rcc-called     [5] IMPLICIT Unsigned8
}
```

```
Initiate-ErrorPDU ::= SEQUENCE {
    error-code                               [0] IMPLICIT Integer8 {
        other                               (0),
        max-fms-pdu-size-insufficient      (1),
        feature-not-supported              (2),
        version-od-incompatible            (3),
        user-initiate-denied               (4),
        password-error                     (5),
        profile-number-incompatible        (6)
    },
    max-fms-pdu-sending-called             [1] IMPLICIT Unsigned8,
    max-fms-pdu-receiving-called           [2] IMPLICIT Unsigned8,
    fms-features-supported-called          [3] IMPLICIT BIT STRING
}
```

12.1.2.9 Domain Management

12.1.2.9.1 Domain Object

```
Domain-State ::= Unsigned8 {
    Existent                               (1),
    Loading                                (2),
    Incomplete                             (3),
    Complete                               (4),
    Ready                                  (5),
    In-Use                                 (6)
}

Upload-State ::= Unsigned8 {
    Non-existent                           (0),
    Uploading                              (1),
    Uploaded                               (2)
}
```

12.1.2.9.2 GenericInitiateDownloadSequence

```
GenericInitiateDownloadSequence-Request ::= SEQUENCE {
    access-specification CHOICE {
        index                               [0] IMPLICIT Index,
        domain-name                         [1] IMPLICIT Name
    }
}
```

GenericInitiateDownloadSequence-Response ::= NULL

12.1.2.9.3 GenericDownloadSegment

```
GenericDownloadSegment-Request ::= SEQUENCE {
    access-specification CHOICE {
        index                               [0] IMPLICIT Index,
        domain-name                         [1] IMPLICIT Name
    },
    load-data                              [2] IMPLICIT OCTET STRING,
    more-follows                           [3] IMPLICIT MoreFollows
}
```

GenericDownloadSegment-Response ::= NULL

12.1.2.9.4 GenericTerminateDownloadSequence

```
GenericTerminateDownloadSequence-Request ::= SEQUENCE {
    access-specification CHOICE {
        index                               [0] IMPLICIT Index,
        domain-name                         [1] IMPLICIT Name
    }
}
```

GenericTerminateDownloadSequence-Response ::= final-result

12.1.2.9.5 InitiateDownloadSequence

```
InitiateDownloadSequence-Request ::= SEQUENCE {
    access-specification CHOICE {
        index [0] IMPLICIT Index,
        domain-name [1] IMPLICIT Name
    }
}
```

```
InitiateDownloadSequence-Response ::= NULL
```

12.1.2.9.6 DownloadSegment

```
DownloadSegment-Request ::= SEQUENCE {
    access-specification CHOICE {
        index [0] IMPLICIT Index,
        domain-name [1] IMPLICIT Name
    }
}
```

```
DownloadSegment-Response ::= SEQUENCE {
    load-data [0] IMPLICIT OCTET STRING,
    more-follows [1] IMPLICIT MoreFollows
}
```

12.1.2.9.7 TerminateDownloadSequence

```
TerminateDownloadSequence-Request ::= SEQUENCE {
    access-specification CHOICE {
        index [0] IMPLICIT Index,
        domain-name [1] IMPLICIT Name
    },
    final-result [2] IMPLICIT BOOLEAN
}
```

```
TerminateDownloadSequence-Response ::= NULL
```

12.1.2.9.8 InitiateUploadSequence

```
InitiateUploadSequence-Request ::= SEQUENCE {
    access-specification CHOICE {
        index [0] IMPLICIT Index,
        domain-name [1] IMPLICIT Name
    }
}
```

```
InitiateUploadSequence-Response ::= NULL
```

12.1.2.9.9 UploadSegment

```
UploadSegment-Request ::= SEQUENCE {
    access-specification CHOICE {
        index [0] IMPLICIT Index,
        domain-name [1] IMPLICIT Name
    }
}
```

```
UploadSegment-Response ::= SEQUENCE {
    load-data [0] IMPLICIT OCTET STRING,
    more-follows [1] IMPLICIT MoreFollows
}
```

12.1.2.9.10 TerminateUploadSequence

```
TerminateUploadSequence-Request ::= SEQUENCE {
    access-specification CHOICE {
        index [0] IMPLICIT Index,
        domain-name [1] IMPLICIT Name
    }
}
```

```
TerminateUploadSequence-Response ::= NULL
```

12.1.2.9.11 RequestDomainDownload

```
RequestDomainDownload-Request ::= SEQUENCE {
    access-specification CHOICE {
        index [0] IMPLICIT Index,
        domain-name [1] IMPLICIT Name
    },
    additionalInformation [2] IMPLICIT VISIBLE STRING OPTIONAL
}

RequestDomainDownload-Response ::= NULL
```

12.1.2.9.12 RequestDomainUpload

```
RequestDomainUpload-Request ::= SEQUENCE {
    access-specification CHOICE {
        index [0] IMPLICIT Index,
        domain-name [1] IMPLICIT Name
    },
    additionalInformation [2] IMPLICIT VISIBLE STRING OPTIONAL
}

RequestDomainUpload-Response ::= NULL
```

12.1.2.10 Program Invocation Management

12.1.2.10.1 ProgramInvocationState

```
ProgramInvocationState ::= Integer8 {
    non-existent (0), -- NON-EXISTENT
    unrunnable (1), -- UNRUNNABLE
    idle (2), -- IDLE
    running (3), -- RUNNING
    stopped (4), -- STOPPED
    starting (5), -- STARTING
    stopping (6), -- STOPPING
    resuming (7), -- RESUMING
    resetting (8) -- RESETING
}

PI-Access-Protection ::= BIT STRING {
    S (23),
    H (22),
    D (21),
    Sg (19),
    Hg (18),
    Dg (17),
    Sa (31),
    Ha (30),
    Da (29),
    Password_Bit1 (7),
    Password_Bit2 (6),
    Password_Bit3 (5),
    Password_Bit4 (4),
    Password_Bit5 (3),
    Password_Bit6 (2),
    Password_Bit7 (1),
    Password_Bit8 (0),
    Access_Groups-1 (15),
    Access_Groups-2 (14),
    Access_Groups-3 (13),
    Access_Groups-4 (12),
    Access_Groups-5 (11),
    Access_Groups-6 (10),
    Access_Groups-7 (9),
    Access_Groups-8 (8)
} -- The Password (unsigned8) is encoded as a bit string.
```

12.1.2.10.2 CreateProgramInvocation

```

CreateProgramInvocation-Request ::= SEQUENCE {
    listOfDomains [0] IMPLICIT SEQUENCE OF CHOICE {
        index [0] IMPLICIT Index,
        domain-name [1] IMPLICIT Name
    },
    access-protection [1] IMPLICIT PI-Access-Protection,
    pi-name [2] IMPLICIT Name OPTIONAL,
    extension [3] IMPLICIT PACKED OPTIONAL,
    reusable [4] IMPLICIT BOOLEAN
}

```

CreateProgramInvocation-Response ::= Index

12.1.2.10.3 DeleteProgramInvocation

```

DeleteProgramInvocation-Request ::= SEQUENCE {
    access-specification CHOICE {
        index [0] IMPLICIT Index,
        pi-name [1] IMPLICIT Name
    }
}

```

DeleteProgramInvocation-Response ::= NULL

12.1.2.10.4 Start

```

Start-Request ::= SEQUENCE {
    access-specification CHOICE {
        index [0] IMPLICIT Index,
        pi-name [1] IMPLICIT Name
    },
    execution-argument [2] IMPLICIT OCTET STRING OPTIONAL
}

```

Start-Response ::= NULL

12.1.2.10.5 Stop

```

Stop-Request ::= SEQUENCE {
    access-specification CHOICE {
        index [0] IMPLICIT Index,
        pi-name [1] IMPLICIT Name
    }
}

```

Stop-Response ::= NULL

12.1.2.10.6 Resume

```

Resume-Request ::= SEQUENCE {
    access-specification CHOICE {
        index [0] IMPLICIT Index,
        pi-name [1] IMPLICIT Name
    },
    execution-argument [2] IMPLICIT OCTET STRING OPTIONAL
}

```

Resume-Response ::= NULL

12.1.2.10.7 Reset

```

Reset-Request ::= SEQUENCE {
    access-specification CHOICE {
        index [0] IMPLICIT Index,
        pi-name [1] IMPLICIT Name
    }
}

```

Reset-Response ::= NULL

12.1.2.10.8 Kill

```
Kill-Request ::= SEQUENCE {
    access-specification CHOICE {
        index                [0] IMPLICIT Index,
        pi-name              [1] IMPLICIT Name
    }
}
```

Kill-Response ::= NULL

12.1.2.11 Variable Access

12.1.2.11.1 VariableListAccessProtection

```
Variable-List-Access-Protection ::= BIT STRING {
    R                (23),
    W                (22),
    D                (21),
    Rg               (19),
    Wg               (18),
    Dg               (17),
    Ra               (31),
    Wa               (30),
    Da               (29),
    Password_Bit1    (7),
    Password_Bit2    (6),
    Password_Bit3    (5),
    Password_Bit4    (4),
    Password_Bit5    (3),
    Password_Bit6    (2),
    Password_Bit7    (1),
    Password_Bit8    (0),
    Access_Groups-1  (15),
    Access_Groups-2  (14),
    Access_Groups-3  (13),
    Access_Groups-4  (12),
    Access_Groups-5  (11),
    Access_Groups-6  (10),
    Access_Groups-7  (9),
    Access_Groups-8  (8)
} -- The Password (unsigned8) is encoded as a bit string.
```

12.1.2.11.2 Read

```
Read-Request ::= SEQUENCE {
    access-specification CHOICE {
        index                [0] IMPLICIT Index,
        variable-name        [1] IMPLICIT Name,
        variable-list-name   [2] IMPLICIT Name
    },
    subindex                [3] IMPLICIT Subindex OPTIONAL
}
```

Read-Response ::= Data

12.1.2.11.3 Write

```
Write-Request ::= SEQUENCE {
    access-specification CHOICE {
        index                [0] IMPLICIT Index,
        variable-name        [1] IMPLICIT Name,
        variable-list-name   [2] IMPLICIT Name
    },
    subindex                [3] IMPLICIT Subindex OPTIONAL,
    data                    [4] IMPLICIT Data
}
```

Write-Response ::= NULL

12.1.2.11.4 DefineVariableList

```

DefineVariableList-Request ::= SEQUENCE {
    listOfVariables                               [0] IMPLICIT SEQUENCE OF CHOICE {
        index                                     [0] IMPLICIT Index,
        variable-name                             [1] IMPLICIT Name
    },
    access-protection                             [1] IMPLICIT Variable-List-Access-Protection,
    variable-list-name                             [2] IMPLICIT Name OPTIONAL,
    extension                                     [3] IMPLICIT PACKED OPTIONAL
}

```

```

DefineVariableList-Response ::= Index

```

12.1.2.11.5 DeleteVariableList

```

DeleteVariableList-Request ::= SEQUENCE {
    access-specification CHOICE {
        index                                     [0] IMPLICIT Index,
        variable-list-name                       [1] IMPLICIT Name
    }
}

```

```

DeleteVariableList-Response ::= NULL

```

12.1.2.11.6 InformationReport

```

InformationReport ::= SEQUENCE {
    access-specification CHOICE {
        index                                     [0] IMPLICIT Index,
        variable-name                             [1] IMPLICIT Name,
        variable-list-name                       [2] IMPLICIT Name
    },
    subindex                                     [3] IMPLICIT Subindex OPTIONAL,
    data                                         [4] IMPLICIT Data
}

```

12.1.2.12 Event Management**12.1.2.12.1 AlterEventConditionMonitoring**

```

AlterEventConditionMonitoring-Request ::= SEQUENCE {
    access-specification CHOICE {
        index                                     [0] IMPLICIT Index,
        event-name                             [1] IMPLICIT Name
    },
    enabled                                     [2] IMPLICIT BOOLEAN
}

```

```

AlterEventConditionMonitoring-Response ::= NULL

```

12.1.2.12.2 AcknowledgeEventNotification

```

AcknowledgeEventNotification-Request ::= SEQUENCE {
    access-specification CHOICE {
        index                                     [0] IMPLICIT Index,
        event-name                             [1] IMPLICIT Name
    },
    eventNumber                                 [2] IMPLICIT Unsigned8
}

```

```

AcknowledgeEventNotification-Response ::= NULL

```

12.1.2.12.3 EventNotification

```

EventNotification ::= SEQUENCE {
    access-specification CHOICE {
        index                                     [0] IMPLICIT Index,
        event-name                             [1] IMPLICIT Name
    },
    eventNumber                                 [2] IMPLICIT Unsigned8,
    eventData                                  [3] IMPLICIT Data OPTIONAL
}

```

12.1.2.13 List of packed fields

This subclause contains the mapping of the individual object descriptions onto the parameter "object description" (data type packed) of the services GetOD and PutOD. This mapping is shown graphically in Figure 229 through 239. The representation contains two lines. The names of the object attributes are in the upper line. The lower line contains the data types. If the length of the lines is insufficient, the representation is continued on the following lines.

The object code is the identifier of the object and indicates which class this object belongs to. The other object attributes are object specific and shall be coded corresponding to their data type. In addition to the object attributes the Object Code shall be transmitted in the services GetOD and PutOD. But in the OD Object Description no Object Code is necessary, because the Index in the structure of the OD Object Description has always the value 0.

The length of the entry of one object description is variable within the maximum PDU size.

The values used for Object Codes and Data Type Indexes are shown in 12.1.4.

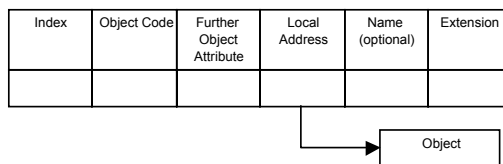


Figure 229 – Structure of an object description in the OD

Index	ROM/RAID Ref	Name Length	Access Properties Property	Version OD	Local Address OD OD-OD	ST-OD Length	Local Address ST-OD	File Index ST-OD	S-OD Length	Local Address S-OD	File Index DP-OD	DP-OD Length	Local Address DP-OD
Unsigned16	Boolean	Unsigned8	Boolean	Integer16	Unsigned32	Unsigned16	Unsigned32	Integer16	Integer16	Unsigned32	Integer16	Unsigned32	Integer16

Figure 230 – Structure of the OD object description

Index	Object Code	Max Octets	Password	Access Groups	Access Rights	Local Address	Domain State	Upload State	Counter	Domain Name	Extension	
Unsigned16	Integer8	Unsigned16	Unsigned8	Bit String 8bit	Bit String 16bit	Unsigned32	Unsigned8	Unsigned8	Integer8	Visible String	Length Unsigned8	Extended Data Octet String

Figure 231 – Structure of a domain entry in the S-OD

Index	Object Code	Number of Domains	Password	Access Groups	Access Rights	Deletable	Reusable	PI State	Domain Index (1)	Domain Index (n)	PI Name	Extension
Unsigned16	Integer8	Unsigned8	Unsigned8	Bit String 8bit	Bit String 16bit	Boolean	Boolean	Unsigned8	Unsigned16	Unsigned16	Visible String	Length Unsigned8 Extended Data Octet String

Figure 232 – Object description of program invocation in the DP-OD

Index	Object Code	Data Type Index	Length	Password	Access Groups	Access Rights	Local Address	Variable Name	Extension	
Unsigned16	Integer8	Unsigned16	Integer8	Unsigned8	Bit String 8bit	Bit String 16bit	Unsigned32	Visible String	Length Unsigned8	Extended Data Octet String

Figure 233 – Object description of simple variable in the S-OD

Index	Object Code	Data Type Index	Length	Number of Elements	Password	Access Groups	Access Rights	Local Address	Variable Name	Extension	
Unsigned16	Integer8	Unsigned16	Integer8	Unsigned8	Unsigned8	Bit String 8bit	Bit String 16bit	Unsigned32	Visible String	Length Unsigned8	Extended Data Octet String

Figure 234 – Object description of array in the S-OD

Index	Object Code	Data Type Index	Password	Access Groups	Access Rights	Record Name	Extension	Local Address (1)	...	Local Address (n)
Unsigned16	Integer8	Unsigned16	Unsigned8	Bit String 8bit	Bit String 16bit	Visible String	Octet String	Unsigned32		Unsigned32

Figure 235 – Object description of record in the S-OD

Index	Object Code	Number of Elements	Password	Access Groups	Access Rights	Deletable	Element Index (1)	...	Element Index (n)	Variable List Name	Extension	
Unsigned16	Integer8	Unsigned8	Unsigned8	Bit String 8bit	Bit String 16bit	Boolean	Unsigned16		Unsigned16	Visible String	Length Unsigned8	Extended Data Octet String

Figure 236 – Object description of variable list in the DV-OD

Index	Object Code	Description	
		Symbolic Name Length	Symbolic Name
Unsigned16	Integer8	Unsigned8	Visible String

Figure 237 – Object description of data type in the ST-OD

Index	Object Code	Number of Elements	Data Type Index (1)	Length (1)	...	Data Type Index (n)	Length (n)
Unsigned16	Integer8	Unsigned8	Unsigned16	Unsigned8		Unsigned16	Unsigned8

Figure 238 – Object description of data type structure description in the ST-OD

Index	Object Code	Index Event Data	Length	Password	Access Groups	Access Rights	Enabled	Event Name	Extension	
Unsigned16	Integer8	Unsigned16	Unsigned8	Unsigned8	Bit String 8bit	Bit String 16bit	Boolean	Visible String	Length Unsigned8	Extended Data Octet String

Figure 239 – Object description of event in the S-OD

12.1.3 Data Types

12.1.3.1 Notation for the Boolean Type

Boolean ::= BOOLEAN
 -- TRUE if the value is non-zero.
 -- FALSE if the value is zero.

12.1.3.2 Notation for the Integer Types

Integer ::= INTEGER -- any integer
 Integer8 ::= INTEGER (-128..+127) -- range $-2^7 \leq i \leq 2^7-1$
 Integer16 ::= INTEGER (-32768..+32767) -- range $-2^{15} \leq i \leq 2^{15}-1$
 Integer32 ::= INTEGER -- range $-2^{31} \leq i \leq 2^{31}-1$

12.1.3.3 Notation for the Unsigned Types

Unsigned ::= INTEGER -- any non-negative integer
 Unsigned8 ::= INTEGER (0..255) -- range $0 \leq i \leq 2^8-1$
 Unsigned16 ::= INTEGER (0..65535) -- range $0 \leq i \leq 2^{16}-1$
 Unsigned32 ::= INTEGER -- range $0 \leq i \leq 2^{32}-1$

12.1.3.4 Notation for the Floating Point Type

Floating32 ::= BIT STRING SIZE (4) -- IEC 60559 Single precision

12.1.3.5 Notation for the BitString Types

BitString ::= BIT STRING -- For generic use
 BitString8 ::= BIT STRING SIZE (8) -- Fixed eight bits bitstring
 BitString16 ::= BIT STRING SIZE (16) -- Fixed 16 bits bitstring
 BitString32 ::= BIT STRING SIZE (32) -- Fixed 32 two bits bitstring

12.1.3.6 Notation for the OctetString Types

OctetString ::= OCTET STRING -- For generic use
 OctetString2 ::= OCTET STRING SIZE (2) -- Fixed two-octet octet string
 OctetString4 ::= OCTET STRING SIZE (4) -- Fixed four-octet octet string
 OctetString6 ::= OCTET STRING SIZE (6) -- Fixed six-octet octet string
 OctetString7 ::= OCTET STRING SIZE (7) -- Fixed seven-octet octet string
 OctetString8 ::= OCTET STRING SIZE (8) -- Fixed eight-octet octet string
 OctetString16 ::= OCTET STRING SIZE (16) -- Fixed 16 octet octet string

12.1.3.7 Notation for VisibleString Types

VisibleString2 ::= VisibleString SIZE (2) -- Fixed two-octet visible string
 VisibleString4 ::= VisibleString SIZE (4) -- Fixed four-octet visible string
 VisibleString8 ::= VisibleString SIZE (8) -- Fixed eight-octet visible string
 VisibleString16 ::= VisibleString SIZE (16) -- Fixed 16 octet visible string

12.1.3.8 Notation for Date Type

BinaryDate ::= OctetString7

12.1.3.9 Notation for TimeOfDay indication Type

TimeOfDay ::= OctetString6

12.1.3.10 Notation for TimeOfDay without date indication Type

TimeOfDayWithoutDate ::= OctetString4

12.1.3.11 Notation for TimeDifference indication Type

TimeDifference ::= OctetString6

12.1.3.12 Notation for TimeDifference without date indication Type

TimeDifferenceWithoutDate ::= OctetString4

12.1.3.13 Notation for TimeValue Type

TimeValue ::= OctetString8

12.1.4 Codes

12.1.4.1 Object Codes

See Table 881.

Table 881 – Object codes

Object	Object code
Null Object	0
- reserved	1
Domain	2
Program Invocation	3
Event	4
Data Type	5
Data Type Structure Description	6
Simple Variable	7
Array	8
Record	9
Variable List	10

The Object Code is of Type Integer8.

12.1.4.2 Data Types Codes

Indices from 1 to 63 are reserved for standard data types. Following types are specified in this document. See Table 882.

Table 882 – Standard data types

Data type	Index	Number of octets
Boolean	1	1
Integer8	2	1
Integer16	3	2
Integer32	4	4
Unsigned8	5	1
Unsigned16	6	2
Unsigned32	7	4
Floating Point	8	4
Visible String	9	1, 2, 3, ...
Octet String	10	1, 2, 3, ...
Date	11	7
Time Of Day	12	4 or 6
Time Difference	13	4 or 6
Bit String	14	1, 2, 3, ...
Time Value	21	8

12.1.4.3 Reason Codes

12.1.4.3.1 General

This section specifies the values of the Reason parameter. See Table 883.

Table 883 – Reason codes

FAL reason parameter	Value
Invalid FAL-PDU	31 (Hex)
Remote Address Mismatch	32 (Hex)
Multiple Initiators	33 (Hex)
Invalid DL-Event	34 (Hex)
AREP Busy	35 (Hex)
AREP Not Found	36 (Hex)
Invalid AREP Type	37 (Hex)
DLCEP Not Found	38 (Hex)

12.2 Transfer Syntax

12.2.1 General

Additional information is to be added to the user data to allow an unique association of the data at the communication partner. The coding rules for the additional information are optimized to produce messages as short as possible in accordance with fieldbus requirements. The frequency of occurrence of special messages is taken into account.

The conditions in the fieldbus area are the following:

- short messages
- low number of different messages
- some messages such as Read and Write occur especially often

12.2.2 Coding Rules

The structuring of a FMS PDU is done either by inserting explicitly Identification Information or by implicit agreements.

Structure of a PDU is shown in Figure 239.

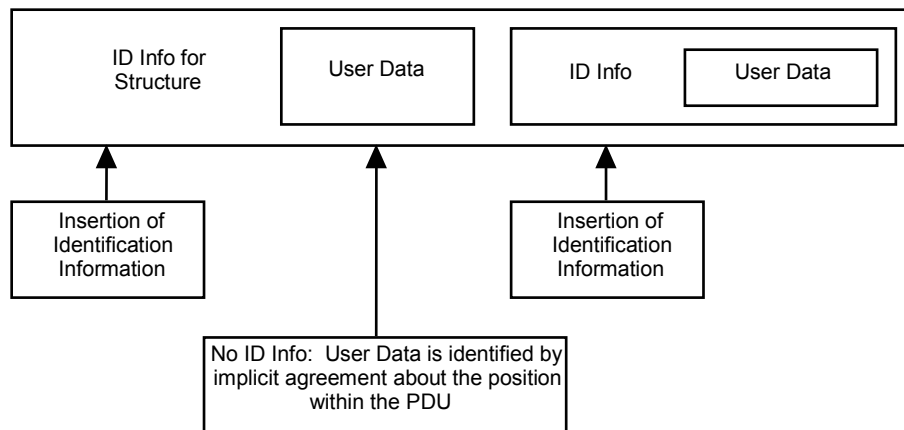


Figure 239 – Insertion of identification information in the FMS PDU

The Identification Information consists of P/C flag, tag and length. The structures and the user data of the PDU may be identified using this Identification Information.

The semantics of the user data are either known from the context or are universally known (context specific tags or universal tags). In the syntax description the context specific tags are enclosed in rectangular brackets. If the semantics of a parameter are implicitly known from the position in the PDU, no tag is used.

The following restrictions on the usage of implicitly known components (universal tags) shall be made:

- The length of the user data shall be fixed
- The component may not be OPTIONAL
- The component may not be inside a CHOICE construct

12.2.3 Structure of the Identification Information

12.2.3.1 General

The Identification Information (ID Info) consists of P/C flag, tag and length, as shown in Figure 241.

The P/C flag indicates if it is a simple component (primitive) or if it is a structured component (constructed, SEQUENCE, SEQUENCE OF).

P/C Flag =0 <=> simple component

P/C Flag =1 <=> structured component

The tag identifies the semantics of the component.

The length is the length of the component in octets if this component is a simple one or the number of contained components if this component is structured.

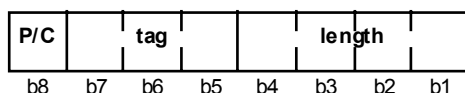
The whole ID Info is coded in one octet, if possible. The octet is divided into 3 parts of different lengths.

- P/C flag 1 bit
- tag 3 bits
- length 4 bits

If the space in the fields for the length or for the tag is not sufficient, an extension is used. For this all bits of the concerned field are set to one. The information is then encoded in the following octet. The range of the tag is 0 to 6 and the range of the length is 0 to 14 when no extension is used. These ranges are preferred for the most frequently occurring messages because they produce very short PDUs.

If an extension has to be used for both the tag and the length, the tag is encoded in the first subsequent octet and the length is encoded in the second subsequent octet.

a) Tag 0 to 6, Length 0 to 14



b) Tag 7 to 255, Length 0 to 14



c) Tag 0 to 6, Length 15 to 255



d) Tag 7 to 255, Length 15 to 255

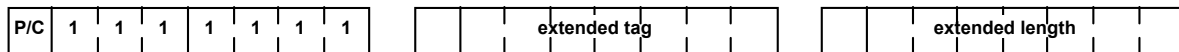


Figure 241 – Coding of the ID info with and without extension

12.2.3.2 Data Types

12.2.3.2.1 General

User data is always a simple (primitive) component. It is encoded as shown in Figure 242.



Figure 242 – Coding with identification

If the semantics of the user data are known implicitly from the position in the PDU and the length is fixed and implicitly known, then no Identification Information is added as shown in Figure 243.

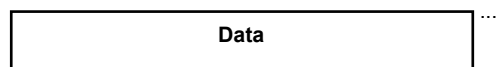


Figure 243 – Coding without identification

12.2.3.2.2 Coding for Boolean Type

Representation of the value true or false in one octet as shown in Figure 244 and 245.

Notation: Range of values:		Boolean											
Coding:		true or false false is represented by the value 0, true is represented by the value FF											
bits		8	7	6	5	4	3	2	1				
octet													
1		1	1	1	1	1	1	1	1				

Figure 244 – Representation of the value true

bits		8	7	6	5	4	3	2	1				
octet													
1		0	0	0	0	0	0	0	0				

Figure 245 – Representation of the value false

12.2.3.2.3 Coding for Integer Types

Integer values are signed quantities as shown in Figure 246.

Notation:		Integer8, Integer16, Integer32									
Range of Values:		Data Type			range of values					length	
		Integer8			-128 ≤ i ≤ 127					1 octet	
		Integer16			-32768 ≤ i ≤ 32767					2 octets	
		Integer32			-2 ³¹ ≤ i ≤ 2 ³¹ - 1					4 octets	
Coding:		In two's complement representation the MSB (Most Significant Bit) is the bit after the sign bit (SN) in the first octet. SN = 0: positive numbers and z SN = 1: negative numbers									
	bits	8	7	6	5	4	3	2	1		
octets	1	SN	2 ¹⁴	2 ¹³	2 ¹²	2 ¹¹	2 ¹⁰	2 ⁹	2 ⁸		
	2	2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰		

Figure 246 – Coding of data of data type Integer16

12.2.3.2.4 Coding for Unsigned Types

Unsigned Values are encoded as shown in Figure 247.

Notation:	Unsigned8, Unsigned16, Unsigned32									
Range of Values:	Data Type			range of values				length		
	Unsigned8			0 ≤ i ≤ 255				1 octet		
	Unsigned16			0 ≤ i ≤ 65535				2 octets		
	Unsigned32			0 ≤ i ≤ 4294967295				4 octets		
Coding	Binary									
bits	8	7	6	5	4	3	2	1		
octets										
1	2 ¹⁵	2 ¹⁴	2 ¹³	2 ¹²	2 ¹¹	2 ¹⁰	2 ⁹	2 ⁸		
2	2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰		

Figure 247 – Coding of data of data type Unsigned16

12.2.3.2.5 Coding for Floating Point Type

Floating Point values are encoded as shown in Figure 248.

Notation:		Floating-Point (4 octet)							
Range of Values:		see IEEE Std 754 Short Real Number (32 bits)							
Coding:		see IEEE Std 754 Short Real Number (32 bits)							
octets	bits	8	7	6	5	4	3	2	1 LSB
	1	Exponent (E)							
	2	SN	2^7	2^6	2^5	2^4	2^3	2^2	2^1
	3	(E)	Fraction (F)						
	4	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}	2^{-7}
		Fraction (F)							
		2^{-8}	2^{-9}	2^{-10}	2^{-11}	2^{-12}	2^{-13}	2^{-14}	2^{-15}
		Fraction (F)							
		2^{-16}	2^{-17}	2^{-18}	2^{-19}	2^{-20}	2^{-21}	2^{-22}	2^{-23}

SN: sign 0 = positive, 1 = negative

Figure 248 – Coding of data of data type Floating Point

12.2.3.2.6 Coding for Visible String Type

Visible String values are encoded as shown in Figure 249.

Notation:		Visible-String							
Range of Values:		see ISO 646 and ISO 2375: Defining registration							
		number 2 + SPACE							
Coding:		see ISO 646							
octets	bits	8	7	6	5	4	3	2	1
	1	first character							
	2	second character							
	·	etc.							
	n								

Figure 249 – Coding of data of data type Visible String

12.2.3.2.7 Coding for Octet String Type

Octet String values are encoded as shown in Figure 250.

Notation: Octet String
Coding: Binary

octets	bits	8	7	6	5	4	3	2	1
	1								
	·								
	n								

Figure 250 – Coding of data of data type Octet String

12.2.3.2.8 Coding for Date Type

The Data Type Date consists of a calendar date and a time as shown in Table 884 and Figure 251.

Notation: Date/Time
 Range of Values: ms to 99 years
 Coding: in 7 octets

Table 884 – Coding for Date Type

Parameter	Range of values	Meaning of the parameters
ms	0...59 999	milli-seconds
min	0...59	minutes
SU	0,1	0: standard time, 1: summer time
RSV		reserve
h	0...23	hours
d. of w.	1...7	day of week: 1 = Monday, 7 = Sunday
d. of m.	1...31	day of month
months	1...12	months
years	0...99	years (without the century)

bits	8	7	6	5	4	3	2	1	
octets									
1	2 ¹⁵	2 ¹⁴	2 ¹³	2 ¹²	2 ¹¹	2 ¹⁰	2 ⁹	2 ⁸	0...59 999 ms
2	2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	
3	RSV	RSV	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	0...59 min
4	SU	RSV	RSV	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	0...23 hours
5	day of week			day of month					1...7 d. of w.
	2 ²	2 ¹	2 ⁰	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	1...31 d. of m.
6	RSV	RSV	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	1...12 months
7	RSV	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	0 ... 99 years
msb									

Figure 251 – Coding of data of data type Date

12.2.3.2.9 Coding for Time Of Day Type

The Data Type Time Of Day consists of a time and an optional date.

The time is stated in milliseconds since midnight. At midnight the counting starts with the value zero.

The date is stated in days relatively to the first of January 1984. On the first of January 1984 the date starts with the value zero.

12.2.3.2.9.1 Notation: Time Of Day

Range of Values: $0 \leq i \leq (2^{28} - 1)$ ms
 $0 \leq i \leq (2^{16} - 1)$ days

12.2.3.2.9.2 Coding:

The time is represented as a 32 bit binary value. The first four (MSB) bits shall have the value zero.

The (optional) date is encoded as a 16 bit (2 octets) binary value.

The Time Of Day is a string of 4 or 6 octets as shown in Figure 252.

bits	8	7	6	5	4	3	2	1	
octets									number of milliseconds since midnight
1	0	0	0	0	2^{27}	2^{26}	2^{25}	2^{24}	
2	2^{23}	2^{22}	2^{21}	2^{20}	2^{19}	2^{18}	2^{17}	2^{16}	
3	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	
4	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	number of days since 01.01.84 optional
5	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	
6	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
msb									

Figure 252 – Coding of data of data type Time Of Day

12.2.3.2.10 Coding for Time Difference Type

The Data Type Time Difference consists of a time in milliseconds and an optional day count. The structure is equivalent to the structure of the Time Of Day but it states in this case a Time Difference.

12.2.3.2.10.1 Notation Time Difference

Range of Values: $0 \leq i \leq (2^{32} - 1)$ ms
 $0 \leq i \leq (2^{16} - 1)$ days

12.2.3.2.10.2 Coding:

The time is represented as a 32 bit binary value. The first four (MSB) bits shall have the value zero. The (optional) date is encoded as a 16 bit (2 octets) binary value. The Time Difference is a string of 4 or 6 octets as shown in Figure 253.

bits	8	7	6	5	4	3	2	1	
octets									number of milliseconds
1	0	0	0	0	2^{27}	2^{26}	2^{25}	2^{24}	
2	2^{23}	2^{22}	2^{21}	2^{20}	2^{19}	2^{18}	2^{17}	2^{16}	
3	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	
4	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	number of days optional
5	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	
6	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
msb									

Figure 253 – Coding of data of data type Time Difference

12.2.3.2.11 Coding for Bit String Type

Figure 254 shows the numbering scheme of the bits of the Data Type Bit String.

Only multiples of eight are legal values of the length (in bits) of the Bit String.

Notation: Bit String
Coding: Binary

bits	8	7	6	5	4	3	2	1
octets								
1	0	1	2	3	4	5	6	7
2	8	9	10	11	12	13	14	15
.	etc.							
n								

Figure 254 – Coding of data of data type Bit String

12.2.3.2.12 Coding for Time Value Type

This data type is used to represent date and time in the required precision for application clock synchronization. It is a 64-bit unsigned fixed-point number, which is the time in 1/32s of a millisecond. When SN bit is 1, the data is negative and in two's complement representation, while the data is positive when SN bit is 0. See Figure 255.

bits	8	7	6	5	4	3	2	1
octets	SN	2^{62}	2^{61}	2^{60}	2^{59}	2^{58}	2^{57}	2^{56}
1								
2	2^{55}	2^{54}	2^{53}	2^{52}	2^{51}	2^{50}	2^{49}	2^{48}
3	2^{47}	2^{46}	2^{45}	2^{44}	2^{43}	2^{42}	2^{41}	2^{40}
4	2^{39}	2^{38}	2^{37}	2^{36}	2^{35}	2^{34}	2^{33}	2^{32}
5	2^{31}	2^{30}	2^{29}	2^{28}	2^{27}	2^{26}	2^{25}	2^{24}
6	2^{23}	2^{22}	2^{21}	2^{20}	2^{19}	2^{18}	2^{17}	2^{16}
7	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8
8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
msb								

signed integer
of 8 octets length of
1/32ms unit

Figure 255 – Coding of data of data type Time Value

12.2.3.3 User Data Definitions

12.2.3.3.1 General

User data is always a simple (primitive) component. It is encoded as shown in Figure 256.



Figure 256 – Coding of data of user data definitions with identifier

If the semantics of the user data are known implicitly from the position in the PDU and the length is fixed and implicitly known, then no Identification Information is added as shown in Figure 257.

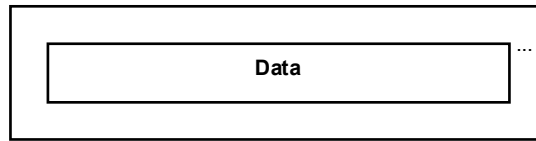


Figure 257 – Coding of data of user data definitions without identifier

Two special definitions are used to describe user data in the PDUBody definitions, Null and Packed. They are defined below.

12.2.3.3.2 Coding for Null

Null has the length zero. There are no subsequent (empty) octets.

12.2.3.3.3 Coding for Packed

Packed contains one or more data elements of the Data Types which are chained together without any gap. The composition is known by the user.

12.2.3.4 Coding of Structure Information

12.2.3.4.1 General

User data may be combined to structured (constructed) components.

The communication partner shall be able to identify these structures and the components of these structures. The P/C flag of the ID Info is 1.

12.2.3.4.2 SEQUENCE

The SEQUENCE structure is comparable with a record. It represents a collection of user data of the same or of different Data Types. Before the SEQUENCE structure there is an ID Info, which conveys the length not in octets but in number of components. The number of components is less than the total length in octets. In most cases an extension is not necessary due to this length encoding. Components should be encoded in the order of ANS.1. See Figure 258. Neither duplication nor deletion of an item is allowed.

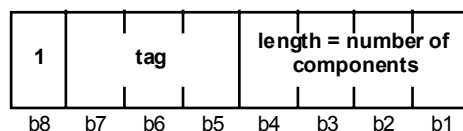


Figure 258 – Coding of ID info for a SEQUENCE

A structure may contain user data or further structures as components. Single components may be OPTIONAL, i.e. they may be omitted. In this case the ID Info is omitted too. A SEQUENCE shall be counted as a single component even if it contains several components.

EXAMPLE The hexadecimal notation is used for the following example of encoding. The upper case letter X is used as a fill-in for unknown values, such as the length of the single components or the tag of the structure. A lower case letter x represents user data.

Syntax Description	Code	comment
Person	[1] IMPLICIT SEQUENCE	{94 4 components
[0] Surname,	0X xx ...	tag 0
[1] First name,	1X xx ...	tag 1
[2] City,	2X xx ...	tag 2
[3] Street	3X xx ...	tag 3
}		

12.2.3.4.3 SEQUENCE OF

The SEQUENCE OF structure represents a succession of components. It is comparable with an array.

The structure may contain one or more components. The components may be user data or structures.

The coding is as for the structure SEQUENCE. For the statement of the number of components the number of repetitions shall be taken into account. The tags of the Syntax Description shall be used for the components of SEQUENCE OF. The same tags may be coded several times in succession.

EXAMPLE:

```
employeedata [2] IMPLICIT SEQUENCE OF {
  [0] Person
}
```

12.2.3.4.4 CHOICE

A CHOICE represents a selection from a set of predefined possibilities. The components of a CHOICE construct shall have different tags to allow proper identification. Instead of the CHOICE construct the actually selected component is encoded. Only one component should be encoded for a CHOICE. Its tag has a value specified in ASN.1.

EXAMPLE:

```
Data ::= CHOICE {
  [0] Employeedata,
  [1] Clientdata,
  [2] Supplierdata
}
```

12.3 FAL Protocol State Machines Structure

This field defines a single state machine, the the VCR Connection protocol machine. Its relationship with its user and with the underlying FSPM, as well as primitives exchanged among them, are depicted in Figure 259, Table 885 and Table 886.

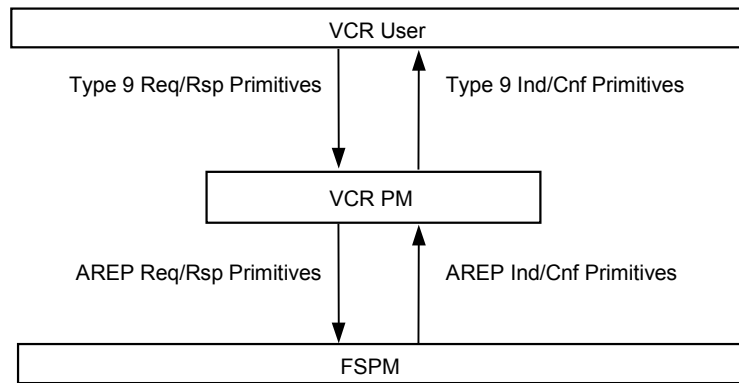


Figure 259 – Relationships among protocol machines and adjacent layers

The VCR Protocol Machine (VCR PM) is an VCR PM state machine that describes the operation of VCRs that are dynamically established by exchanging Initiate service APDUs. The VCR PM is responsible for the following activities:

1. to accept service request and response primitives from the VCR service user;
2. to accept service indication and confirmation primitives from the FSPM;
3. to deliver service indication and confirmation primitives to the VCR user.
4. to issue service request and response primitives to the FSPM based on the rules of its state machine.

The VCR PM may use any AREP defined by any Type, as long as its FSPM provides the services it requires. The FSPM services required by the VCR Context Management PM are specified in the tables below.

Table 885 – Primitives issued by VCR PM to FSPM

Primitive name	Source	Associated parameters	Functions
ASC.req	VCR PM	Arep_Id, Data, Remote_DLCEP_Address	This primitive is used to convey an Associate request primitive from the VCR PM to the FSPM.
ASC.rsp(+)	VCR PM	Arep_Id, Data	This primitive is used to convey an Associate response(+) primitive from the VCR PM to the FSPM.
ASC.rsp(-)	VCR PM	Arep_Id, Data	This primitive is used to convey an Associate response(-) primitive from the VCR PM to the FSPM.
Abort.req	VCR PM	Arep_Id, Identifier, Reason_Code, Additional_Detail	This primitive is used to convey an Abort request primitive from the VCR PM to the FSPM.
CS.req	VCR PM	Arep_Id, Data	This primitive is used to convey a Confirmed Send (CS) request primitive from the VCR PM to the FSPM.
CS.rsp	VCR PM	Arep_Id, Data	This primitive is used to convey a Confirmed Send (CS) response primitive from the VCR PM to the FSPM.
UCS.req	VCR PM	Arep_Id, Remote_DLSAP_Address, Data	This primitive is used to convey an Unconfirmed Send (UCS) request primitive from the VCR PM to the FSPM.
FCMP.req	VCR PM	Arep_Id	This primitive is used to convey an FAL-Compel (FCMP) request primitive from the VCR PM to the FSPM.
GBM.req	VCR PM	Arep_Id	This primitive is used to convey a Get-Buffered-Message (GBM) request primitive from the VCR PM to the FSPM.
AR_XON_OFF.req	VCR PM	Arep_Id, Remote_Dlsap_Address, Data	This primitive is used to perform flow control.

Table 886 – Primitives issued by FSPM to VCR PM

Primitive name	Source	Associated parameters	Functions
ASC.ind	FSPM	Arep_Id, Data	This primitive is used to convey an Associate indication primitive from the FSPM to the VCR PM.
ASC.cnf(+)	FSPM	Arep_Id, Data	This primitive is used to convey an Associate result(+) primitive from the FSPM to the VCR PM.
ASC.cnf(-)	FSPM	Arep_Id, Data	This primitive is used to convey an Associate result(-) primitive from the FSPM to the VCR PM.
Abort.ind	FSPM	Arep_Id, Locally_Generated, Identifier, Reason_Code, Additional_Detail	This primitive is used to convey an Abort indication primitive from the FSPM to the VCR PM.
CS.ind	FSPM	Arep_Id, Data	This primitive is used to convey a Confirmed Send (CS) indication primitive from the FSPM to the VCR PM.
CS.cnf	FSPM	Arep_Id, Data	This primitive is used to convey a Confirmed Send (CS) confirmation primitive from the FSPM to the VCR PM.
UCS.ind	FSPM	Arep_Id, Remote_DLSAP_Address, Duplicate_FAL-SDU, Data, Local_Timeliness, Remote_Timeliness	This primitive is used to convey an Unconfirmed Send (UCS) indication primitive from the FSPM to the VCR PM.
FCMP.cnf	FSPM	Arep_Id, Status	This primitive is used to convey an FAL-Compel (FCMP) confirmation primitive from the FSPM to the VCR PM.
GBM.cnf(+)	FSPM	Arep_Id, Duplicate_FAL-SDU, Data, Local_Timeliness, Remote_Timeliness	This primitive is used to convey a Get-Buffered-Message (GBM) positive confirmation primitive from the FSPM to the VCR PM.
GBM.cnf(-)	FSPM	Arep_Id	This primitive is used to convey a Get-Buffered-Message (GBM) negative confirmation primitive from the FSPM to the VCR PM.
FSTS.ind	FSPM	Arep_Id, Reported_Status	This primitive is used to convey an FAL-Status (FSTS) indication primitive from the FSPM to the VCR PM.

12.4 VCR PM State Machine

12.4.1 General

The VCR PM state machine describes the VCR endpoint behavior. For connectionless VCRs, the state machine begins in the OPEN state.

12.4.2 Primitive Definitions

12.4.2.1 Primitives Exchanged between FAL-User and VCR PM

See Table 887 and Table 888 for the primitives exchanged between the FAL-User and the VCR PM.

Table 887 – Primitives issued by FAL-User to VCR PM

Primitive name	Source	Associated parameters and functions
Terminate.req	FAL-User	Refer to FAL Service Definition (IEC 61158-5)
Initiate.req	FAL-User	
Initiate.rsp(+)	FAL-User	
Initiate.rsp(-)	FAL-User	
ConfirmedService.req	FAL-User	
ConfirmedService.rsp	FAL-User	
UnconfirmedService.req	FAL-User	

Table 888 – Primitives issued by VCR PM to FAL-User

Primitive name	Source	Associated parameters and functions
Terminate.ind	VCR PM	Refer to FAL Service Definition (IEC 61158-5)
Initiate.ind	VCR PM	
Initiate.cnf(+)	VCR PM	
Initiate.cnf(-)	VCR PM	
ConfirmedService.ind	VCR PM	
ConfirmedService.cnf	VCR PM	
UnconfirmedService.ind	VCR PM	

12.4.3 State Machine Description

The attributes used in this state machine as shown in Figure 260 and Table 889 are defined as elements of the AP attribute *List Of In-Use AR Endpoint Info*.

CONNECTION-NOT-ESTABLISHED

The connection is not established. Only the service primitives Initiate.req, ASC.ind, Abort.req and ABT.ind are allowed. All other services shall be rejected with the Abort service.

CONNECTION-ESTABLISHING (CALLING)

The local FMS user wishes to establish the connection. Only the service primitives ASC.cnf(+), ASC.cnf(-), Abort.req and ABT.ind are allowed. All other services shall be rejected with the Abort service. This state is abbreviated as CON-ESTABLISHING-CALLING.

CONNECTION-ESTABLISHING (CALLED)

The remote FMS user wishes to establish the connection. Only the service primitives Initiate.rsp(+), Initiate.rsp(-), Abort.req and ABT.ind are allowed. All other services shall be rejected with the Abort service. This state is abbreviated as CON-ESTABLISHING-CALLED.

CONNECTION-ESTABLISHED

The VCR is established. The service primitives Initiate.req, Initiate.rsp(+), Initiate.rsp(-) are not allowed and shall be rejected with the Abort service. The following actions shall be taken to reset a VCR (Reset VCR):

- Set attribute "Outstanding Services Counter Sending" and attribute "Outstanding Services Counter Receiving" of the VCR (dynamic part) to 0
- Set state of the VCR to "CONNECTION-NOT-ESTABLISHED"

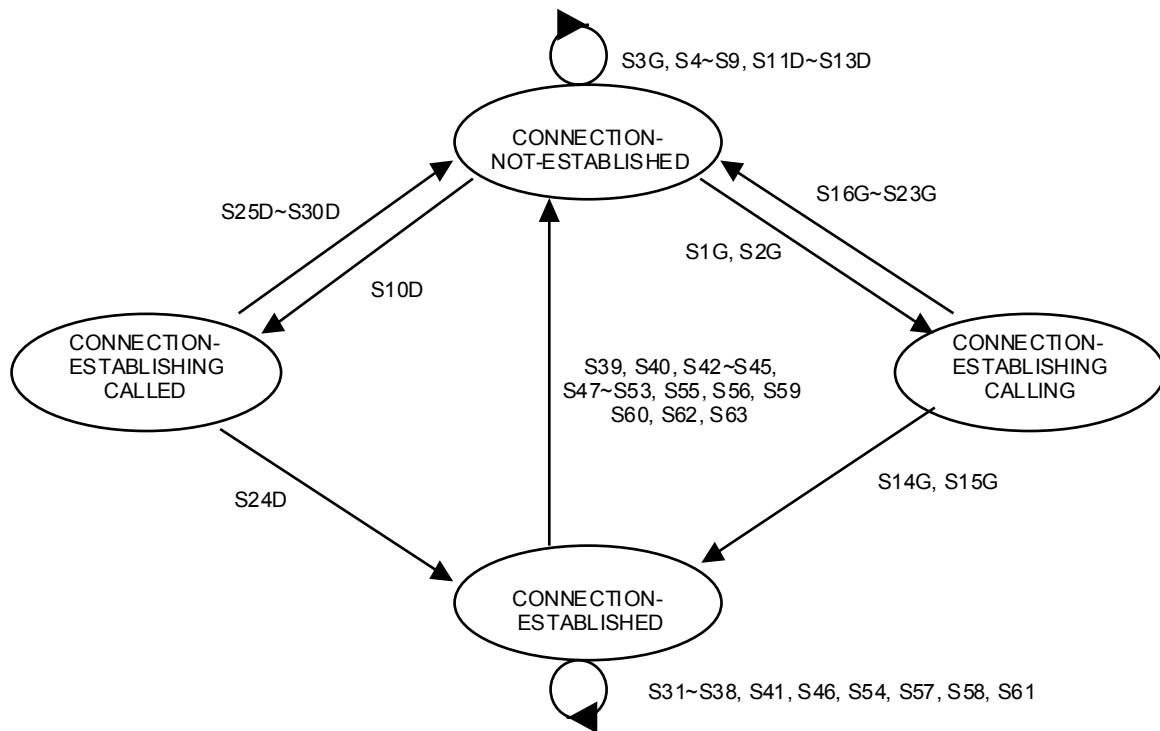


Figure 260 – VCR state machine

NOTE The transaction numbers with “G” suffix are for the calling protocol machine and those with “D” suffix are for the called protocol machine. The transactions without a suffix apply to both protocol machines, or those whose role is yet to be determined.

12.4.4 AP-VCR initiation state transitions

Table 889 – AP-VCR state machine transactions

#	Current state	Event or condition => action	Next state
S1G	CONNECTION- NOT- ESTABLISHED	Initiate.req && VCR is valid && VCR Type = QUB => ASC.req{ Data := Initiate-RequestPDU } }	CONNECTION – ESTABLISHING- CALLING
S2G	CONNECTION- NOT- ESTABLISHED	Initiate.req && VCR is valid && VCR Type <> QUB => ASC.req{ Data := NULL } }	CONNECTION – ESTABLISHING- CALLING
S3G	CONNECTION- NOT- ESTABLISHED	Initiate.req && VCR is not valid => Abort.ind{ AbortIdentifier := FMS, ReasonCode := VCR error } }	CONNECTION- NOT- ESTABLISHED
S4	CONNECTION- NOT- ESTABLISHED	Abort.req => (no actions taken)	CONNECTION- NOT- ESTABLISHED
S5	CONNECTION- NOT- ESTABLISHED	FMS service.primitive other than Initiate & Abort => Abort.ind{ AbortIdentifier := FMS, ReasonCode := User error } }	CONNECTION- NOT- ESTABLISHED
S6	CONNECTION- NOT- ESTABLISHED	ABT.ind => (no actions taken)	CONNECTION- NOT- ESTABLISHED
S7	CONNECTION- NOT- ESTABLISHED	Faulty or unknown FAS service primitive => ABT.req{ AbortIdentifier := FMS, ReasonCode := FAS error } }	CONNECTION- NOT- ESTABLISHED
S8	CONNECTION- NOT- ESTABLISHED	FAS service primitive other than ASC.ind and ABT.ind => ABT.req{ AbortIdentifier := FMS, ReasonCode := Connection state conflict FAS } }	CONNECTION- NOT- ESTABLISHED
S9	CONNECTION- NOT- ESTABLISHED	ASC.ind && Data = not allowed, unknown or faulty FMS PDU => ABT.req{ AbortIdentifier := FMS, ReasonCode := FMS PDU error } }	CONNECTION- NOT- ESTABLISHED
S10D	CONNECTION- NOT- ESTABLISHED	ASC.ind && Data Initiate-RequestPDU && VCR is valid && Max FMS PDU length test is positive && FmsFeaturesSupported test is positive => Initiate.ind{ } }	CONNECTION – ESTABLISHING- CALLED

Table 889 (continued)

#	Current state	Event or condition => action	Next state
S11D	CONNECTION- NOT- ESTABLISHED	ASC.ind && Data = Initiate-RequestPDU && VCR is invalid => ABT.req{ AbortIdentifier := FMS, ReasonCode := VCR error }	CONNECTION- NOT- ESTABLISHED
S12D	CONNECTION- NOT- ESTABLISHED	ASC.ind && Data = Initiate-RequestPDU && VCR is valid && Max FMS PDU length test is negative => ASC.rsp(-){ Data := Initiate-ErrorPDU{ ErrorCode := max-fms-pdu-size-insufficient } }	CONNECTION- NOT- ESTABLISHED
S13D	CONNECTION- NOT- ESTABLISHED	ASC.ind && Data = Initiate-RequestPDU && VCR is valid && Max FMS PDU length test is positive && FmsFeaturesSupported test is negative => ASC.rsp(-){ Data := Initiate-ErrorPDU{ ErrorCode := feature-not-supported } }	CONNECTION- NOT- ESTABLISHED
S14G	CONNECTION - ESTABLISHING -CALLING	ASC.cnf(+) && Data = Initiate-ResponsePDU && VCR Type = QUB => Initiate.cnf(+){ }	CONNECTION- ESTABLISHED
S15G	CONNECTION - ESTABLISHING -CALLING	ASC.cnf(+) && Data = NULL && VCR Type = QUU or BNU => Initiate.cnf(+){ }	CONNECTION- ESTABLISHED
S16G	CONNECTION - ESTABLISHING -CALLING	ASC.cnf(-) && Data = Initiate-ErrorPDU && VCR Type = QUB => Initiate.cnf(-){ ErrorCode := Error code in ASC.cnf }, reset VCR	CONNECTION- NOT- ESTABLISHED
S17G	CONNECTION - ESTABLISHING -CALLING	ASC.cnf(-) && Data = NULL && VCR Type = QUU or BNU => Initiate.cnf(-){ ErrorCode := Error code in ASC.cnf }, reset VCR	CONNECTION- NOT- ESTABLISHED
S18G	CONNECTION - ESTABLISHING -CALLING	ABT.ind => Abort.ind{ AbortIdentifier := AbortIdentifier of ABT.ind, ReasonCode := ReasonCode of ABT.ind }, reset VCR	CONNECTION- NOT- ESTABLISHED
S19G	CONNECTION - ESTABLISHING -CALLING	Abort.req => ABT.req{ AbortIdentifier := AbortIdentifier of Abort.req, ReasonCode := ReasonCode of Abort.req }, reset VCR	CONNECTION- NOT- ESTABLISHED

Table 889 (continued)

#	Current state	Event or condition => action	Next state
S20G	CONNECTION - ESTABLISHING -CALLING	Faulty or unknown FAS service primitive => ABT.req{ AbortIdentifier := FMS, ReasonCode := FAS error }, Abort.ind{ AbortIdentifier := FMS, ReasonCode := FAS error }, reset VCR	CONNECTION- NOT- ESTABLISHED
S21G	CONNECTION - ESTABLISHING -CALLING	FAS service primitive other than ASC.cnf and ABT.ind => ABT.req{ AbortIdentifier := FMS, ReasonCode := Connection state conflict FAS }, Abort.ind{ AbortIdentifier := FMS, ReasonCode := Connection state conflict FAS }, reset VCR	CONNECTION- NOT- ESTABLISHED
S22G	CONNECTION - ESTABLISHING -CALLING	ASC.cnf && Data = not allowed, unknown or faulty FMS PDU => ABT.req{ AbortIdentifier := FMS, ReasonCode := FMS PDU error }, Abort.ind{ AbortIdentifier := FMS, ReasonCode := FMS PDU error }, reset VCR	CONNECTION- NOT- ESTABLISHED
S23G	CONNECTION - ESTABLISHING -CALLING	FMS service.primitive other than Initiate.rsp and Abort.req => ABT.req{ AbortIdentifier := FMS, ReasonCode := User error }, Abort.ind{ AbortIdentifier := FMS, ReasonCode := User error }, reset VCR	CONNECTION- NOT- ESTABLISHED
S24D	CONNECTION - ESTABLISHING -CALLED	Initiate.rsp(+) => ASC.rsp(+){ Data = Initiate-ResponsePDU }	CONNECTION- ESTABLISHED
S25D	CONNECTION - ESTABLISHING -CALLED	Initiate.rsp(-) => ASC.rsp(-){ Data = Initiate-ErrorPDU{ ErrorCode = ErrorCode of Initiate.rsp } }, reset VCR	CONNECTION- NOT- ESTABLISHED
S26D	CONNECTION - ESTABLISHING -CALLED	Abort.req => ABT.req{ AbortIdentifier := AbortIdentifier of Abort.req, ReasonCode := ReasonCode of Abort.req }, reset VCR	CONNECTION- NOT- ESTABLISHED

Table 889 (continued)

#	Current state	Event or condition => action	Next state
S27D	CONNECTION - ESTABLISHING -CALLED	ABT.ind => Abort.ind{ AbortIdentifier := AbortIdentifier of ABT.ind, ReasonCode := ReasonCode of ABT.ind }, reset VCR	CONNECTION- NOT- ESTABLISHED
S28D	CONNECTION - ESTABLISHING -CALLED	Faulty or unknown FAS service primitive => ABT.req{ AbortIdentifier := FMS, ReasonCode := FAS error }, Abort.ind{ AbortIdentifier := FMS, ReasonCode := FAS error }, reset VCR	CONNECTION- NOT- ESTABLISHED
S29D	CONNECTION - ESTABLISHING -CALLED	FAS service primitive other ABT.ind => ABT.req{ AbortIdentifier := FMS, ReasonCode := Connection state conflict FAS }, Abort.ind{ AbortIdentifier := FMS, ReasonCode := Connection state conflict FAS }, reset VCR	CONNECTION- NOT- ESTABLISHED
S30D	CONNECTION - ESTABLISHING -CALLED	FMS service.primitive other than Initiate.rsp and Abort.req => ABT.req{ AbortIdentifier := FMS, ReasonCode := User error }, Abort.ind{ AbortIdentifier := FMS, ReasonCode := User error }, reset VCR	CONNECTION- NOT- ESTABLISHED
S31	CONNECTION- ESTABLISHED	ConfirmedService.req && OSCS < ActualMaxSCC && InvokeID not existent && PDU length ≤ Max FMS PDU sending && FmsFeaturesSupported test (client) positive => DTC.req{ Data := Confirmed-RequestPDU }, OSCS := OSCS + 1	CONNECTION- ESTABLISHED
S32	CONNECTION- ESTABLISHED	ConfirmedService.req && OSCS = ActualMaxSCC => Reject.ind{ DetectedHere := true, Original InvokeID := InvokeID in ConfirmedService.req, RejectPDUType := Confirmed-RequestPDU, RejectCode := Max-services-overflow }	CONNECTION- ESTABLISHED
S33	CONNECTION- ESTABLISHED	ConfirmedService.req && OSCS < ActualMaxSCC && InvokeID already existent => Reject.ind{ DetectedHere := true, Original InvokeID := InvokeID in ConfirmedService.req, RejectPDUType := Confirmed-RequestPDU, RejectCode := Invoke-ID-exists }	CONNECTION- ESTABLISHED

Table 889 (continued)

#	Current state	Event or condition => action	Next state
S34	CONNECTION-ESTABLISHED	ConfirmedService.req && OSCS < ActualMaxSCC && InvokeID not existent && PDU length > Max FMS PDU sending => Reject.ind{ DetectedHere := true, Original InvokeID := InvokeID in ConfirmedService.req, RejectPDUType := Confirmed-RequestPDU, RejectCode := PDU-size }	CONNECTION-ESTABLISHED
S35	CONNECTION-ESTABLISHED	ConfirmedService.req && OSCS < ActualMaxSCC && InvokeID not existent && PDU length ≤ Max FMS PDU sending && FmsFeaturesSupported test (client) negative => Reject.ind{ DetectedHere := true, Original InvokeID := InvokeID in ConfirmedService.req, RejectPDUType := Confirmed-RequestPDU, RejectCode := Feature-not-supported-connection-oriented }	CONNECTION-ESTABLISHED
S36	CONNECTION-ESTABLISHED	UnconfirmedService.req && PDU length ≤ Max FMS PDU sending && FmsFeaturesSupported test (client) positive => DTU.req{ Data := Unconfirmed-PDU }	CONNECTION-ESTABLISHED
S37	CONNECTION-ESTABLISHED	UnconfirmedService.req && PDU length > Max FMS PDU sending => Reject.ind{ DetectedHere := true, Original InvokeID := InvokeID in UnconfirmedService.req, RejectPDUType := Unconfirmed-PDU, RejectCode := PDU-size }	CONNECTION-ESTABLISHED
S38	CONNECTION-ESTABLISHED	UnconfirmedService.req && PDU length ≤ Max FMS PDU sending && FmsFeaturesSupported test (client) negative => Reject.ind{ DetectedHere := true, Original InvokeID := InvokeID in UnconfirmedService.req, RejectPDUType := Unconfirmed-PDU, RejectCode := Feature-not-supported-connection-oriented }	CONNECTION-ESTABLISHED
S39	CONNECTION-ESTABLISHED	Abort.req => ABT.req{ AbortIdentifier := AbortIdentifier of Abort.req, ReasonCode := ReasonCode of Abort.req }, reset VCR	CONNECTION-NOT-ESTABLISHED
S40	CONNECTION-ESTABLISHED	Faulty, unknown or not-allowed FMS service.primitive => ABT.req{ AbortIdentifier := FMS, ReasonCode := User error }, Abort.ind{ AbortIdentifier := FMS, ReasonCode := User error }, reset VCR	CONNECTION-NOT-ESTABLISHED

Table 889 (continued)

#	Current state	Event or condition => action	Next state
S41	CONNECTION-ESTABLISHED	DTC.ind && Data = Confirmed-ServicePDU && PDU length ≤ Max FMS PDU receiving && OSCR < ActualMaxRCC && InvokeID not existent && Features Supported test (server) positive => ConfirmedService.ind{ } OSCR := OSCR + 1	CONNECTION-ESTABLISHED
S42	CONNECTION-ESTABLISHED	DTC.ind && Data = Confirmed-ServicePDU && PDU length > Max FMS PDU receiving => ABT.req{ AbortIdentifier := FMS, ReasonCode := PDU-size }, Abort.ind{ AbortIdentifier := FMS, ReasonCode := PDU-size }, reset VCR	CONNECTION-NOT-ESTABLISHED
S43	CONNECTION-ESTABLISHED	DTC.ind && Data = Confirmed-ServicePDU && PDU length ≤ Max FMS PDU receiving && OSCR ≥ ActualMaxRCC => ABT.req{ AbortIdentifier := FMS, ReasonCode := Max-services-overflow }, Abort.ind{ AbortIdentifier := FMS, ReasonCode := Max-services-overflow }, reset VCR	CONNECTION-NOT-ESTABLISHED
S44	CONNECTION-ESTABLISHED	DTC.ind && Data = Confirmed-ServicePDU && PDU length ≤ Max FMS PDU receiving && OSCR < ActualMaxRCC && InvokeID already existent => ABT.req{ AbortIdentifier := FMS, ReasonCode := InvokeID-error-request }, Abort.ind{ AbortIdentifier := FMS, ReasonCode := InvokeID-error-request }, reset VCR	CONNECTION-NOT-ESTABLISHED
S45	CONNECTION-ESTABLISHED	DTC.ind && Data = Confirmed-ServicePDU && PDU length ≤ Max FMS PDU receiving && OSCR < ActualMaxRCC && InvokeID not existent && Features Supported test (server) negative => ABT.req{ AbortIdentifier := FMS, ReasonCode := Feature-not-supported }, Abort.ind{ AbortIdentifier := FMS, ReasonCode := Feature-not-supported }, reset VCR	CONNECTION-NOT-ESTABLISHED

Table 889 (continued)

#	Current state	Event or condition => action	Next state
S46	CONNECTION-ESTABLISHED	DTU.ind && Data = Unconfirmed-PDU && PDU length ≤ Max FMS PDU receiving && Features Supported test (server) positive => UnconfirmedService.ind{ }	CONNECTION-ESTABLISHED
S47	CONNECTION-ESTABLISHED	DTU.ind && Data = Unconfirmed-PDU && PDU length > Max FMS PDU receiving => ABT.req{ AbortIdentifier := FMS, ReasonCode := PDU-size }, Abort.ind{ AbortIdentifier := FMS, ReasonCode := PDU-size }, reset VCR	CONNECTION-NOT-ESTABLISHED
S48	CONNECTION-ESTABLISHED	DTU.ind && Data = Unconfirmed-PDU && PDU length ≤ Max FMS PDU receiving && Features Supported test (server) negative => ABT.req{ AbortIdentifier := FMS, ReasonCode := Feature-not-supported }, Abort.ind{ AbortIdentifier := FMS, ReasonCode := Feature-not-supported }, reset VCR	CONNECTION-NOT-ESTABLISHED
S49	CONNECTION-ESTABLISHED	ABT.ind => Abort.ind{ AbortIdentifier := AbortIdentifier of ABT.ind, ReasonCode := ReasonCode of ABT.ind }, reset VCR	CONNECTION-NOT-ESTABLISHED
S50	CONNECTION-ESTABLISHED	ASC.ind && Data = Initiate-RequestPDU => ABT.req{ AbortIdentifier := FMS, ReasonCode := Connection-state-conflict-FMS }, Abort.ind{ AbortIdentifier := FMS, ReasonCode := Connection-state-conflict-FMS }, reset VCR	CONNECTION-NOT-ESTABLISHED
S51	CONNECTION-ESTABLISHED	Faulty or unknown FAS service primitive => ABT.req{ AbortIdentifier := FMS, ReasonCode := FAS error }, Abort.ind{ AbortIdentifier := FMS, ReasonCode := FAS error }, reset VCR	CONNECTION-NOT-ESTABLISHED

Table 889 (continued)

#	Current state	Event or condition => action	Next state
S52	CONNECTION-ESTABLISHED	Not-allowed FAS service primitive => ABT.req{ AbortIdentifier := FMS, ReasonCode := Connection-state-conflict-FMS }, Abort.ind{ AbortIdentifier := FMS, ReasonCode := Connection-state-conflict-FMS }, reset VCR	CONNECTION-NOT-ESTABLISHED
S53	CONNECTION-ESTABLISHED	valid FAS Service primitive && Data = not-allowed, unknown or faulty FMS PDU => ABT.req{ AbortIdentifier := FMS, ReasonCode := FMS-PDU-error }, Abort.ind{ AbortIdentifier := FMS, ReasonCode := FMS-PDU-error }, reset VCR	CONNECTION-NOT-ESTABLISHED
S54	CONNECTION-ESTABLISHED	ConfirmedService.rsp && InvokeID (server) existent && Services in .rsp and .ind are identical && PDU length ≤ Max FMS PDU sending => DTC.rsp{ Data := Confirmed-ResponsePDU }, OSCR := OSCR - 1	CONNECTION-ESTABLISHED
S55	CONNECTION-ESTABLISHED	ConfirmedService.rsp && InvokeID (server) not existent => ABT.req{ AbortIdentifier := FMS, ReasonCode := InvokeID-error-response }, Abort.ind{ AbortIdentifier := FMS, ReasonCode := InvokeID-error-response }, reset VCR	CONNECTION-NOT-ESTABLISHED
S56	CONNECTION-ESTABLISHED	ConfirmedService.rsp && InvokeID (server) existent && Services in .rsp and .ind are not identical => ABT.req{ AbortIdentifier := FMS, ReasonCode := Service-error }, Abort.ind{ AbortIdentifier := FMS, ReasonCode := Service-error }, reset VCR	CONNECTION-NOT-ESTABLISHED
S57	CONNECTION-ESTABLISHED	ConfirmedService.rsp && InvokeID (server) existent && Services in .rsp and .ind are identical && PDU length > Max FMS PDU sending => DTC.rsp{ Data := Reject-PDU{ Original InvokeID := InvokeID in ConfirmedService.req, RejectCode := PDU-size } }, OSCR := OSCR - 1	CONNECTION-ESTABLISHED

Table 889 (continued)

#	Current state	Event or condition => action	Next state
S58	CONNECTION-ESTABLISHED	DTC.cnf && Data = Confirmed-ResponsePDU && PDU length ≤ Max FMS PDU receiving && InvokeID (client) existent && Services in .cnf and .req are identical => ConfirmedService.cnf{ }, OSCR := OSCR	CONNECTION-ESTABLISHED
S59	CONNECTION-ESTABLISHED	DTC.cnf && Data = Confirmed-ResponsePDU && PDU length > Max FMS PDU receiving => ABT.req{ AbortIdentifier := FMS, ReasonCode := PDU-size }, Abort.ind{ AbortIdentifier := FMS, ReasonCode := PDU-size }, reset VCR	CONNECTION-NOT-ESTABLISHED
S60	CONNECTION-ESTABLISHED	DTC.cnf && Data = Confirmed-ResponsePDU && PDU length ≤ Max FMS PDU receiving && InvokeID (client) not existent => ABT.req{ AbortIdentifier := FMS, ReasonCode := InvokeID-error-response }, Abort.ind{ AbortIdentifier := FMS, ReasonCode := InvokeID-error-response }, reset VCR	CONNECTION-NOT-ESTABLISHED
S61	CONNECTION-ESTABLISHED	DTC.cnf && Data = Reject-PDU && Original InvokeID (client) existent && RejectCode:= PDU-size => Reject.ind{ DetectedHere := false, Original InvokeID := InvokeID in Reject-PDU, RejectPDUType := Confirmed-Response-PDU, RejectCode := PDU-size }, OSCS := OSCS -1	CONNECTION-ESTABLISHED
S62	CONNECTION-ESTABLISHED	DTC.cnf && Data = Reject-PDU && Original InvokeID (client) not existent => ABT.req{ AbortIdentifier := FMS, ReasonCode := InvokeID-error-response }, Abort.ind{ AbortIdentifier := FMS, ReasonCode := InvokeID-error-response }, reset VCR	CONNECTION-NOT-ESTABLISHED

Table 889 (continued)

#	Current state	Event or condition => action	Next state
S63	CONNECTION-ESTABLISHED	DTC.cnf && Data = Reject-PDU && Original InvokeID (client) existent && RejectCode:<> PDU-size => ABT.req{ AbortIdentifier := FMS, ReasonCode := FMS-PDU-error }, Abort.ind{ AbortIdentifier := FMS, ReasonCode := FMS-PDU-error }, reset VCR	CONNECTION-NOT-ESTABLISHED

12.5 AREP State Machines

12.5.1 AREP Mapping to Data Link Layer

12.5.1.1 General

This section describes the mapping of the FAS to the Fieldbus Data Link Layer. It does not redefine the DLSAP attributes or DLME attributes that are or will be defined in the Data Link Layer specification; rather, it defines how they are used by each of the AR classes. A means to configure and monitor the values of these attributes is provided by Network Management.

The following class definitions describe the DLSAP attributes and the DLME attributes required to support each of the AREP classes.

NOTE Undefined attributes use the same definitions as those previously defined.

12.5.1.2 DLL Mapping of QUU AREP Class

12.5.1.2.1 QUU AREP Class Formal Model

The DLL Mapping attributes and their permitted values and the DLL services used with the QUU AREP class are defined in this subclause.

CLASS: QuuCI

PARENT CLASS: QueuedUser-TriggeredUnidirectionalAREP

ATTRIBUTES:

1. (m) KeyAttribute: LocalDisapAddress
2. (m) Attribute: RemoteDisapAddress
3. (m) Attribute: LocalDisapRole (Basic, Group)
4. (c) Constraint: Role = Source
- 4.1 (m) Attribute: DefaultQosAsSender
- 4.2 (m) Attribute: DIIPriority (Urgent, Normal, TimeAvailable)
- 4.3 (m) Attribute: MaxConfirmDelayOnUnitdata
- 4.4 (m) Attribute: DIpduAuthentication (Ordinary, Source, Maximal)
- 4.5 (m) Attribute: DISchedulingPolicy (Implicit)
- 4.6 (m) Attribute: ExplicitQueue (True, False)
5. (c) Constraint: ExplicitQueue = True
- 5.1 (m) Attribute: QueueBindings—either for a sender or a receiver
- 5.2 (m) Attribute: QueueIdentifier
- 5.3 (m) Attribute: MaxQueueDepth
- 5.4 (m) Attribute: MaxDisduSize

DLL SERVICES:

1. (m) OpsService: DL-Unitdata
2. (c) Constraint: ExplicitQueue = True
3. (m) OpsService: DL-Get

12.5.1.2.2 Attributes

12.5.1.2.2.1 LocalDisapAddress

This attribute specifies the DLSAP address to which this AREP is attached. This attribute is a DLSAP-address if the Role attribute has the value of Source, and either a group DL-address or a DLSAP-address if the Role attribute has the value of Sink.

This attribute supplies the value for the “DL(SAP)-address” parameter specified in the DLL.

This attribute contains the following three sub-attributes: Link Address, Node Address, and Selector.

12.5.1.2.2.2 RemoteDisapAddress

This attribute specifies the remote address to which FAS-PDUs are sent (for Source AREPs), or from which they are received (for Sink AREPs).

If the ConfigurationType attribute is Linked, the value of this attribute has been configured. If it is Free, the value of this attribute is provided with a service request.

This attribute contains the following three sub-attributes: Link Address, Node Address, and Selector.

12.5.1.2.2.3 LocalDisapRole

This attribute specifies the behavior of the local DLSAP to be used. If the Role is Source, this attribute has the value of Basic. If the Role is Sink, it has the value of either Basic or Group.

This attribute supplies the value for the “DL(SAP)-role” parameter specified by the DLL.

12.5.1.2.2.4 DefaultQosAsSender

The DefaultQosAsSender attributes specify the DLL quality of service that is used by the sending AREP. The receiving DLL shall support the quality of service specified by these attributes.

12.5.1.2.2.5 DllPriority

This attribute defines the DLL priority, and thus restricts the maximum length of an FAS-PDU, of the conveyance path of an AR.

This attribute supplies the value for the “DLL priority” parameter of the DLL. The values Urgent, Normal, and Time-Available correspond to URGENT, NORMAL, and TIME-AVAILABLE as defined in Part-3 and –4 of this International Standard, respectively.

NOTE It is not possible to use different priorities for each FAS-PDU sent from the same QUU AREP.

12.5.1.2.2.6 MaxConfirmDelayOnUnitdata

This attribute specifies the maximum confirmation delay for a local confirmation from a DL-Unitdata request primitive.

This attribute supplies the value for the “Max confirm delay on locally-confirmed DL-Unitdata” parameter of the DLL.

12.5.1.2.2.7 DlpduAuthentication

This attribute specifies the lower bound of the length of the DL-addresses to be used by the DLL.

This attribute supplies the value for the “DLPDU authentication” parameter of the DLL. The values Ordinary, Source, and Maximal correspond to ORDINARY, SOURCE, and MAXIMAL as defined in Part-3 and –4 of this International Standard, respectively.

12.5.1.2.2.8 DISchedulingPolicy

This attribute provides the guidance to the DLL on the scheduling needed by an AR. For this AREP, the DLL tries to transmit the FAS-PDU as soon as it is passed from the FAS.

This attribute supplies the value for DL-Scheduling-policy attribute. Only the value Implicit is used. It corresponds to IMPLICIT as defined in Part-3 and –4 of this International Standard.

12.5.1.2.2.9 ExplicitQueue

This attribute specifies, when True, that the characteristics of the associated sending and receiving queues are explicitly configured and managed through the Network Management. The value of False means that queues with implementation specific depth and length are provided by the DLL.

12.5.1.2.2.10 QueueBindings

The following attributes specify the explicit queue that is bound to this DLSAP. For a sender, the queue is for sending. For a receiver, it is for receiving.

12.5.1.2.2.11 QueueIdentifier

This attribute provides a local means to identify a queue that is associated with this DLSAP.

This attribute supplies the value for the “Buffer-or-queue-identifier” parameter defined in the DLL.

12.5.1.2.2.12 MaxQueueDepth

This attribute specifies the maximum number of FAS-PDUs that can be queued for sending or receiving.

This attribute supplies the value for the “Maximum queue depth” parameter of the DLL.

12.5.1.2.2.13 MaxDlsduSize

This attribute specifies the maximum length of an FAS-PDU that can be sent or received by the DLL.

This attribute supplies the value for the “Maximum DLSDU size” parameter of the DLL.

12.5.1.2.3 DLL Services

Refer to 4.7.4 for DLL service descriptions.

12.5.1.3 DLL Mapping of QUB AREP Class

12.5.1.3.1 QUB AREP Class Formal Model

The DLL Mapping attributes and their permitted values and the DLL services used with the QUB AREP class are defined in this subclause.

CLASS:		QubCo
PARENT CLASS:		QueuedUser-TriggeredBidirectionalAREP
ATTRIBUTES:		
1.	(m) KeyAttribute:	LocalDlcepAddress
2.	(m) Attribute:	RemoteDlcepAddress
3.	(m) KeyAttribute:	DlcepDlIdentifier
4.	(m) Attribute:	DisapRole (Basic)
5.	(m) Attribute:	QosParameterSet
5.1	(m) Attribute:	DlcepClass (Peer)
5.2	(m) Attribute:	DlcepDataDeliveryFeatures
5.2.1	(m) Attribute:	FromRequesterToResponder (Classical, Disordered)
5.2.2	(m) Attribute:	FromResponderToRequester (Classical, Disordered)
5.3	(m) Attribute:	Priority
5.3.1	(m) Attribute:	DlIPriority (Urgent, Normal, TimeAvailable)
5.3.2	(m) Attribute:	DlIPriorityNegotiated (Urgent, Normal, TimeAvailable)
5.4	(m) Attribute:	DlpduAuthentication (Ordinary, Source, Maximal)
5.5	(m) Attribute:	ResidualActivity
5.5.1	(m) Attribute:	ResidualActivityAsSender (True, False)
5.5.2	(m) Attribute:	ResidualActivityAsReceiver (True, False)
5.6	(m) Attribute:	MaxConfirmDelay
5.6.1	(m) Attribute:	MaxConfirmDelayOnDIConnect
5.6.2	(m) Attribute:	MaxConfirmDelayOnDIData
5.7	(m) Attribute:	DISchedulingPolicy (Implicit)
5.8	(m) Attribute:	ExplicitQueue (True, False)
5.9	(c) Constraint:	ExplicitQueue = True
5.9.1	(m) Attribute:	MaxDlsduSizes
5.9.2	(m) Attribute:	MaxDlsduSizeFromRequester
5.9.3	(m) Attribute:	MaxDlsduSizeFromResponder
5.9.4	(m) Attribute:	MaxDlsduSizeFromRequesterNegotiated
5.9.5	(m) Attribute:	MaxDlsduSizeFromResponderNegotiated
5.10	(m) Attribute:	QueueBindings
5.10.1	(m) Attribute:	SendingBufferOrQueueIdentifier
5.10.2	(m) Attribute:	ReceivingBufferOrQueueIdentifier
5.11	(m) Attribute:	MaxQueueDepth
5.11.1	(m) Attribute:	MaxSendingQueueDepth
5.11.2	(m) Attribute:	MaximimReceivingQueueDepth
DLL SERVICES:		
1.	(m) OpsService:	DL-Data
2.	(c) Constraint:	ExplicitQueue = True
2.1	(m) OpsService:	DL-Get
3.	(m) OpsService:	DL-Connect
4.	(m) OpsService:	DL-Connection-Established
5.	(m) OpsService:	DL-Disconnect

12.5.1.3.2 Attributes

12.5.1.3.2.1 LocalDlcepAddress

This attribute specifies the local DLCEP address and identifies the DLCEP.

The value of this attribute is used as the “DLCEP-address” parameter of the DLL.

NOTE The value of this attribute is also carried in the Associate Request PDU.

This attribute contains the following three sub-attributes: Link Address, Node Address, and Selector.

NOTE Since the local Link and Node addresses are set by the Network Management, only the Selector portion of the LocalDisapAddress attribute is a configurable attribute of the FAS.

12.5.1.3.2.2 RemoteDlcepAddress

This attribute specifies the remote DLCEP address and identifies the DLCEP.

This attribute supplies the value for called DLCEP-address of the DL-Connect service.

NOTE The value of this attribute is also carried in the header part of the Associate Request PDU.

This attribute contains the following three sub-attributes: Link Address, Node Address, and Selector.

12.5.1.3.2.3 DlcepDlIdentifier

The DlcepDlIdentifier attribute identifies the DLCEP.

This attribute supplies the value for the DLCEP DL-identifier parameter of the DL-Connect service. It is returned to the calling FAS by the DL-Connect request primitive, and by the DL-Connect indication primitive to the called FAS.

12.5.1.3.2.4 DisapRole

This attribute specifies the behavior of the DLSAP that is used by the AREP.

This attribute supplies the value for the “DL(SAP)-role” parameter. The possible value Basic corresponds to BASIC as defined in the Data Link Layer specification.

12.5.1.3.2.5 QosParameterSet

The QosParameterSet attributes specify the DL quality of service that is used by this AREP. These attribute values may be negotiated with the remote AREP.

12.5.1.3.2.6 DlcepClass

This attribute specifies the behavior of the DLCEP which is attached to the AREP.

This attribute supplies the value for the “DLCEP class” parameter of the DLL. The possible value of this attribute is Peer and corresponds to Peer defined by the DLL.

12.5.1.3.2.7 DlcepDataDeliveryFeatures

These two attributes specify data delivery features of the DLL.

The permitted values Classical and Disordered correspond, respectively to CLASSICAL and DISORDERED defined by the Data Link Layer specification.

NOTE The FromRequesterToResponder and FromResponderToRequester attributes shall have the same value.

12.5.1.3.2.8 FromRequesterToResponder

This attribute specifies the DLL data delivery feature of the DLPDUs sent from the AREP whose Initiator attribute has a value of “True” to the remote AREP. It supplies the value for the “DLCEP data delivery features from requester to responder(s)” parameter defined in the DLL.

12.5.1.3.2.9 FromResponderToRequester

This attribute specifies the DLL data delivery feature of the DLPDUs sent from the AREP whose Initiator attribute has a value of “False” to the remote AREP. It supplies the value for the “DLCEP data delivery features from responder(s) to requester” parameter defined in the DLL.

12.5.1.3.2.10 DllPriority

This attribute specifies the configured value of the DLL priority.

NOTE It is not possible to use different priorities for each FAS-PDU sent from the same QUB AREP. Also, it is not permitted to use different priorities for the send and receive conveyance paths.

12.5.1.3.2.11 DllPriorityNegotiated

This attribute specifies the negotiated value of the DLL priority.

12.5.1.3.2.12 DlpduAuthentication

This attribute specifies the lower bound of the length of DL-addressing to be used by the DLL.

This attribute supplies the value for the “DLPDU-authentication” parameter of the DLL. The permitted value Ordinary, Source, and Maximal correspond to ORDINARY, Source, and MAXIMAL, respectively, as defined in Part-3 and –4 of this International Standard.

12.5.1.3.2.13 ResidualActivityAsSender

This attribute specifies sender’s DLC residual activity. It supplies the value for the “Residual activity as sender” parameter defined in the DLL. The possible values are “True” and “False.”

12.5.1.3.2.14 ResidualActivityAsReceiver

This attribute specifies receiver’s DLC residual activity. It supplies the value for the “Residual activity as receiver” parameter defined in the DLL. The possible values are “True” and “False.”

12.5.1.3.2.15 MaxConfirmDelay

This attribute specifies the maximum confirmation delay of certain DLL connection-oriented services.

12.5.1.3.2.16 MaxConfirmDelayOnDIConnect

This attribute species the maximum confirmation delay for a confirmation from a DL-Connect service.

This attribute supplies the value for the “Maximum confirmation delay on DL-Connect, DL-Reset and DL-Subscriber-Query” parameter specified in the Data Link Layer specification.

12.5.1.3.2.17 MaxConfirmDelayOnDIData

This attribute species the maximum confirmation delay for a confirmation from a DL-Data service.

This attribute supplies the value for the “Maximum confirmation delay on DL-Data” parameter specified in Part-3 and –4 of this International Standard.

12.5.1.3.2.18 DISchedulingPolicy

This attribute provides the guidance to the DLL on the scheduling needed by an AR. For this AREP, the DLL tries to transmit the FAS-PDU as soon as possible.

This attribute supplies the value for the “DL-Scheduling-policy” parameter of the DLL. The permitted value Implicit corresponds to IMPLICIT defined in Part-3 and –4 of this International Standard.

12.5.1.3.2.19 ExplicitQueue

This attribute specifies, when True, that the characteristics of the associated sending and receiving queues are explicitly configured and managed through the Network Management. The value of False means that queues with implementation specific depth and length are provided by the DLL.

12.5.1.3.2.20 MaxDisduSizeFromRequester

This attribute specifies the configured value of the maximum length of an FAS-PDU that can be sent from the AREP whose Initiator attribute has a value of “True” to the remote AREP.

This attribute supplies the value for the “Maximum DLSDU sizes from requester” parameter of the DLL.

12.5.1.3.2.21 MaxDisduSizeFromResponder

This attribute specifies the configured value of the maximum length of an FAS-PDU that can be sent from the AREP whose Initiator attribute has a value of “False” to the remote AREP.

This attribute supplies the value for the “Maximum DLSDU sizes from responder” parameter of the DLL.

12.5.1.3.2.22 MaxDisduSizeFromRequesterNegotiated

This attribute specifies the negotiated value of the maximum length of an FAS-PDU that can be sent from the AREP whose Initiator attribute has a value of “True” to the remote AREP.

12.5.1.3.2.23 MaxDisduSizeFromResponderNegotiated

This attribute specifies the negotiated value of the maximum length of an FAS-PDU that can be sent from the AREP whose Initiator attribute has a value of “False” to the remote AREP.

12.5.1.3.2.24 SendingBufferOrQueueIdentifier

This attribute provides a local means to identify a queue that is used to store sending FAS-PDUs.

This attribute supplies the value for the “Buffer-and-queue bindings as sender” parameter of the DLL.

12.5.1.3.2.25 ReceivingBufferOrQueueIdentifier

This attribute provides a local means to identify a queue that is used to store receiving FAS-PDUs.

This attribute supplies the value for the “Buffer-and-queue bindings as receiver” parameter of the DLL.

12.5.1.3.2.26 MaxSendingQueueDepth

This attribute specifies the maximum number of FAS-PDUs that can be queued for transmission.

This attribute supplies the value for the “Maximum queue depth” parameter of the DLL for Send queues.

12.5.1.3.2.27 MaxReceivingQueueDepth

This attribute specifies the maximum number of FAS-PDUs that can be queued at reception.

This attribute supplies the value for the “Maximum queue depth” parameter of the DLL for Receive queues.

12.5.1.3.3 DLL Services

Refer to 4.7.4 for DLL service descriptions.

12.5.1.4 DLL Mapping of BNU AREP Class

12.5.1.4.1 BNU AREP Class Formal Model

The DLL Mapping attributes and their permitted values and the DLL services used with the BNU AREP class are defined in this subclause.

CLASS: BnuCo

PARENT CLASS: BufferedNetworkScheduledUnidirectionalAREP

ATTRIBUTES:

1. (m) KeyAttribute: PublisherDlcepAddress
2. (m) KeyAttribute: DlcepDIIdentifier
3. (m) Attribute: DisapRole (Basic)
4. (m) Attribute: QosParameterSet
- 4.1 (m) Attribute: DlcepClass (Publisher, Subscriber)
- 4.2 (m) Attribute: DIIPriority (Urgent, Normal, TimeAvailable)
- 4.3 (m) Attribute: DIpduAuthentication (Ordinary, Source)
- 4.4 (m) Attribute: ResidualActivity
- 4.4.1 (m) Attribute: AsSender (False)
- 4.5 (m) Attribute: MaxConfirmDelayOnDIConnect
- 4.6 (m) Attribute: DISchedulingPolicy (Explicit)
- 4.7 (m) Constraint: DlcepClass = Publisher
- 4.7.1 (m) Attribute: FromRequesterToResponder (Ordered, Unordered)
- 4.7.2 (m) Attribute: MaxDlsduSizeFromRequester
- 4.7.3 (m) Attribute: SendingBufferOrQueueIdentifier
- 4.7.4 (o) Attribute: SenderTimeliness
- 4.7.4.1 (o) Attribute: PublisherDITimelinessClass
(Residence, Update, Synchronous, None)
- 4.7.4.2 (o) Attribute: PublisherTimeWindowSize
- 4.7.4.3 (o) Constraint: PublisherDITimelinessClass == Update || Synchronous
- 4.7.4.3.1 (o) Attribute: PublisherSynchronizingDlcep
- 4.8 (m) Constraint: DlcepClass = Subscriber
- 4.8.1 (m) Attribute: FromResponderToRequester (Ordered, Unordered)
- 4.8.2 (m) Attribute: MaxDlsduSizeFromResponder
- 4.8.3 (m) Attribute: ReceivingBufferOrQueueIdentifier
- 4.8.4 (o) Attribute: ReceiverTimeliness
- 4.8.4.1 (o) Attribute: PublisherDITimelinessClass
(Residence, Update, Synchronous, None)
- 4.8.4.2 (o) Attribute: PublisherTimeWindowSize
- 4.8.4.3 (o) Attribute: SubscriberDITimelinessClass
(Residence, Update, Synchronous, None)
- 4.8.4.4 (o) Attribute: SubscriberTimeWindowSize
- 4.8.4.5 (o) Constraint: SubscriberDITimelinessClass == Update || Synchronous
- 4.8.4.5.1 (o) Attribute: SubscriberSynchronizingDlcep
5. (m) Attribute: LasScheduled (True, False)

DLL SERVICES:

1. (m) OpsService: DL-Put
2. (m) OpsService: DL-Get
3. (m) OpsService: DL-Connect
4. (m) OpsService: DL-Connection-Established
5. (m) OpsService: DL-Disconnect
6. (m) OpsService: DL-Buffer-Received
7. (m) OpsService: DL-Buffer-Sent
8. (m) Constraint: LasScheduled == False
9. (m) OpsService: DL-Compel-Service

12.5.1.4.2 Attributes

12.5.1.4.2.1 PublisherDlcepAddress

This attribute specifies the Publisher's DLCEP address and identifies the DLCEP.

The value of this attribute is used as the "DLCEP-address" parameter of the DLL.

This attribute contains the following three sub-attributes: Link Address, Node Address, and Selector.

NOTE Since the local Link and Node addresses are set by the Network Management, only the Selector portion of the LocalDlsapAddress attribute is a configurable attribute of the FAS.

12.5.1.4.2.2 DlcepDllIdentifier

The DlcepDllIdentifier attribute identifies the DLCEP.

This attribute supplies the value for the DLCEP DL-identifier parameter of the DL-Connect service. It is returned to the calling FAS by the DL-Connect request primitive, and by the DL-Connect indication primitive to the called FAS.

12.5.1.4.2.3 DlsapRole

This attribute specifies the behavior of the DLSAP that is used by the AREP.

This attribute supplies the value for the "DL(SAP)-role" parameter. The possible value Basic corresponds to BASIC as defined in the Data Link Layer specification.

12.5.1.4.2.4 QosParameterSet

The QosParameterSet attributes specify the DL quality of service that is used by this AREP. These attribute values may be negotiated with the remote AREP.

12.5.1.4.2.5 DlcepClass

This attribute specifies the behavior of the DLCEP which is attached to the AREP.

This attribute supplies the value for the "DLCEP class" parameter of the DLL. The possible value of this attribute is Peer and corresponds to Peer defined by the DLL.

12.5.1.4.2.6 DllPriority

This attribute specifies the configured value of the DLL priority.

NOTE It is not possible to use different priorities for each FAS-PDU sent from the same BNU AREP.

12.5.1.4.2.7 DlpduAuthentication

This attribute specifies the lower bound of the length of DL-addressing to be used by the DLL.

This attribute supplies the value for the "DLPDU-authentication" parameter of the DLL. The permitted value Ordinary, Source, and Maximal correspond to ORDINARY, Source, and MAXIMAL, respectively, as defined in Part-3 and –4 of this International Standard.

12.5.1.4.2.8 ResidualActivity

This attribute specifies receiver's DLC residual activity. It supplies the value for the "Residual activity as receiver" parameter defined in the DLL. The possible values are "True" and "False."

12.5.1.4.2.9 MaxConfirmDelayOnDIConnect

This attribute species the maximum confirmation delay for a confirmation from a DL-Connect service.

This attribute supplies the value for the “Maximum confirmation delay on DL-Connect, DL-Reset and DL-Subscriber-Query” parameter specified in the Data Link Layer specification.

12.5.1.4.2.10 DISchedulingPolicy

This attribute provides the guidance to the DLL on the scheduling needed by an AR. For this AREP, the DLL tries to transmit the FAS-PDU when it is instructed to do so by a stimulus from the network.

This attribute supplies the value for the “DL-Scheduling-policy” parameter of the DLL. The permitted value Explicit corresponds to EXPLICIT defined in IEC 61158-3 and IEC 61158-4.

12.5.1.4.2.11 FromRequesterToResponder

This attribute is used if the Role attribute has a value of “Publisher” and specifies the DLL data delivery feature of the AREP.

It supplies the value for the “DLCEP data delivery features from requester to responder(s)” parameter of the DLL. The possible values Ordered and Unordered correspond to ORDERED and UNORDERED defined in IEC 61158-3 and IEC 61158-4.

If the DuplicatePduDetectionSupported attribute is True, this attribute has the value of “Ordered.”

12.5.1.4.2.12 MaxDisduSizeFromRequester

This attribute is used if the Role attribute has a value of “Publisher” and specifies the maximum length of an FAS-PDU that can be sent from this AREP.

This attribute supplies the value for the “Maximum DLSDU sizes from requester” parameter of the DLL.

12.5.1.4.2.13 SendingBufferOrQueueIdentifier

This attribute provides a local means to identify a buffer that is used to store FAS-PDUs for sending. This attribute supplies the value for the “Buffer-and-queue bindings as sender” parameter of the DLL.

12.5.1.4.2.14 FromResponderToRequester

This attribute is used if the Role attribute has a value of “Subscriber” and specifies the DLL data delivery feature of the AREP.

It supplies the value for the “DLCEP data delivery features from responder(s) to requester” parameter of the DLL. The possible values Ordered and Unordered correspond to ORDERED and UNORDERED defined in the Data Link Layer specification.

If the DuplicatePduDetectionSupported attribute is True, this attribute has the value of “Ordered.”

12.5.1.4.2.15 MaxDisduSizeFromResponder

This attribute is used if the Role attribute has a value of “Subscriber” and specifies the maximum length of an FAS-PDU that can be received by this AREP.

This attribute supplies the value for the “Maximum DLSDU size from responder” parameter of the DLL.

12.5.1.4.2.16 ReceivingBufferOrQueueIdentifier

This attribute provides a local means to identify a buffer that is used to store FAS-PDUs for receiving. This attribute supplies the value for the “Buffer-and-queue bindings as receiver” parameter of the DLL.

12.5.1.4.2.17 SenderTimeliness**12.5.1.4.2.18 PublisherDITimelinessClass**

This optional attribute provides the timeliness class of a Publisher provided by the DLL. This attribute supplies the value for the “DL-timeliness-class” parameter of the DLL. The permitted values Residence, Update, Synchronous, and None correspond to RESIDENCE, UPDATE, SYNCHRONOUS, and NONE defined by the DLL.

12.5.1.4.2.19 PublisherTimeWindowSize

This optional attribute provides time window of a Publisher provided by the DLL. This attribute supplies the value for the “Time window size” parameter of the DLL.

12.5.1.4.2.20 PublisherSynchronizingDlcep

This optional attribute is present when the PublisherDITimelinessClass attribute has the value of Update or Synchronous and provides a DLCEP that is used to generate synchronizing events by the DLL. The FAS user may derive the AREP from which the synchronizing events are delivered by this attribute value. This attribute supplies the value for the “synchronizing DLCEP” parameter of the DLL.

12.5.1.4.2.21 ReceiverTimeliness**12.5.1.4.2.22 SubscriberDITimelinessClass**

This optional attribute provides the timeliness class of a Subscriber provided by the DLL. This attribute supplies the value for the “DL-timeliness-class” parameter of the DLL. The permitted values Residence, Update, Synchronous, and None correspond to RESIDENCE, UPDATE, SYNCHRONOUS, and NONE defined by the DLL.

12.5.1.4.2.23 SubscriberTimeWindowSize

This optional attribute provides time window of a Subscriber provided by the DLL. This attribute supplies the value for the “Time window size” parameter of the DLL.

12.5.1.4.2.24 SubscriberSynchronizingDlcep

This optional attribute is present when the SubscriberDITimelinessClass attribute has the value of Update or Synchronous and provides a DLCEP that is used to generate synchronizing events by the DLL. The FAS user may derive the AREP from which the synchronizing events are delivered by this attribute value. This attribute supplies the value for the “synchronizing DLCEP” parameter of the DLL.

12.5.1.4.2.25 LasScheduled

This attribute specifies whether this AREP is LAS-scheduled or not. The value of True means that this APRE is LAS-Scheduled. The value of False means that it is not LAS-Scheduled.

NOTE This attribute does not affect the operation of the FAS and the DLL. It is used by the FAS users only.

12.5.1.4.3 DLL services

Refer to to 4.7.4 for DLL service descriptions.

12.5.2 AREP Service Protocol Machine (FSPM)

12.5.2.1 General

FAS Service Protocol Machine is common to all the AREP types. Only applicable primitives are different among different AREP types. It has one state called "ACTIVE" as shown in Figure 261.

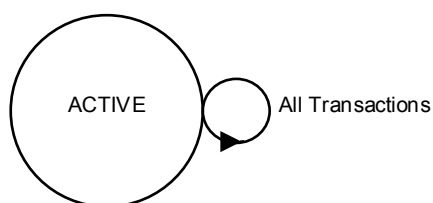


Figure 261 – State transition diagram of FSPM

12.5.2.2 FSPM state tables

Table 890 and Table 891 specify the FSPM protocol machine.

Table 890 – FSPM state table – sender transactions

#	Current state	Event or condition => action	Next state
S1	ACTIVE	ASC.req && SelectArep (Arep_Id) = "True" => ASC_req { user_data := Data, remote_dlcep_address := Remote_DLCEP_Address }	ACTIVE
S2	ACTIVE	ASC.rsp(+) && SelectArep (Arep_Id) = "True" => ASC_rsp(+) { user_data := Data }	ACTIVE
S3	ACTIVE	ASC.rsp(-) && SelectArep (Arep_Id) = "True" => ASC_rsp(-) { user_data := Data }	ACTIVE
S4	ACTIVE	DTU.req && SelectArep (Arep_Id) = "True" => DTU_req { remote_dlsap_address := Remote_DLSAP_Address, user_data := Data }	ACTIVE
S5	ACTIVE	DTC.req && SelectArep (Arep_Id) = "True" => DTC_req { user_data := Data }	ACTIVE
S6	ACTIVE	DTC.rsp && SelectArep (Arep_Id) = "True" => DTC_rsp { user_data := Data }	ACTIVE
S7	ACTIVE	Abort.req && SelectArep (Arep_Id) = "True" => Abort_req { identifier := Identifier, reason_code := Reason_Code, additional_detail := Additional_Detail }	ACTIVE
S8	ACTIVE	FCMP.req && SelectArep (Arep_Id) = "True" => FCMP_req { }	ACTIVE
S9	ACTIVE	GBM.req && SelectArep (Arep_Id) = "True" => GBM_req { }	ACTIVE
NOTE 1 A primitive generated in the FSPM sender state machine is sent to an appropriate ARPM that is selected by the FSPM using the SelectArep function. The Arep_Id parameter supplied by the FAS user is the argument of this function. NOTE 2 If the SelectArep function returns the value of False, it is a local matter to report such instance and the FSPM does not generate any primitives to the ARPM.			

Table 891 – FSPM state table – receiver transactions

#	Current state	Event or condition => action	Next state
R1	ACTIVE	ASC_ind => ASC.ind { Arep_Id := arep_id, Data := user_data }	ACTIVE
R2	ACTIVE	ASC_cnf(+) => ASC.cnf(+) { Arep_Id := arep_id, Data := user_data }	ACTIVE
R3	ACTIVE	ASC_cnf(-) => ASC.cnf(-) { Arep_Id := arep_id, Data := user_data }	ACTIVE
R4	ACTIVE	DTU_ind => DTU.ind { Arep_Id := arep_id, Remote_DLSAP_Address := remote_dlsap_address, Duplicate_FAS-SDU := duplicate_fas_sdu, Data := user_data, Local_Timeliness := local_timeliness, Remote_Timeliness := remote_timeliness }	ACTIVE
R5	ACTIVE	DTC_ind => DTC.ind { Arep_Id := arep_id, Data := user_data }	ACTIVE
R6	ACTIVE	DTC_cnf => DTC.cnf { Arep_Id := arep_id, Data := user_data }	ACTIVE
R7	ACTIVE	Abort_ind => Abort.ind { Arep_Id := arep_id, Locally_Generated := locally_generated, Identifier := identifier, Reason_Code := reason_code, Additional_Detail := additional_detail }	ACTIVE
R8	ACTIVE	FCMP_cnf => FCMP.cnf { Arep_Id := arep_id, Status := status }	ACTIVE
R9	ACTIVE	GBM_cnf(+) => GBM.cnf(+) { Arep_Id := arep_id, Duplicate_FAS-SDU := duplicate_fas_sdu, Data := user_data, Local_Timeliness := local_timeliness, Remote_Timeliness := remote_timeliness }	ACTIVE
R10	ACTIVE	GBM_cnf(-) => GBM.cnf(-) { Arep_Id := arep_id, }	ACTIVE

#	Current state	Event or condition => action	Next state
R11	ACTIVE	FSTS_ind => FSTS.ind { Arep_Id := arep_id, Reported_Status := reported_status }	ACTIVE

12.5.3 Application Relationship Protocol Machines (ARPMs)

12.5.3.1 AR Protocol Machine (ARPM) for QUU AREP

12.5.3.1.1 QUU ARPM states

The defined states and their descriptions of the QUU ARPM are shown in Table 892 and Figure 262.

Table 892 – QUU ARPM states

CLOSED	The AREP is defined, but not capable of sending or receiving FAS-PDUs. It may send or receive Associate service FAS-PDUs while in this state.
OPEN	The AREP is defined and capable of sending or receiving FAS-PDUs.

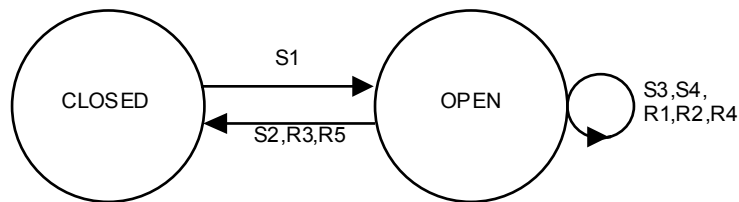


Figure 262 – State transition diagram of the QUU ARPM

12.5.3.1.2 QUU ARPM state table

Table 893 and Table 894 specify the QUU QRPM state machine.

Table 893 – QUU ARPM state table – sender transactions

#	Current state	Event or condition => action	Next state
S1	CLOSED	ASC_req => ASC_cnf(+) { arep_id := GetArepld (), user_data := "null" }	OPEN
S2	OPEN	Abort_req => (no actions taken)	CLOSED
S3	OPEN	DTU_req && ConfigurationType = "Linked" && Role = "Source" => FAS-PDU_req { dmpm_service_name := "DMPM_Unitdata_req", arep_id := GetArepld (), dlsdu := BuildFAS-PDU (fas_pdu_name := "DTU_PDU", fas_data := user_data) } }	OPEN
S4	OPEN	DTU_req && ConfigurationType = "Free" && Role = "Source" => RemoteDisapAddress := remote_disap_address, FAS-PDU_req { dmpm_service_name := "DMPM_Unitdata_req", arep_id := GetArepld (), dlsdu := BuildFAS-PDU (fas_pdu_name := "DTU_PDU", fas_data := user_data) } }	OPEN

Table 894 – QUU ARPM state table – receiver transactions

#	Current state	Event or condition => action	Next state
R1	OPEN	FAS-PDU_ind && dmpm_service_name = "DMPM_Unitdata_ind" && ConfigurationType = "Linked" && RemoteDisapAddress = calling_address && Role = "Sink" && FAS_Pdu_Type (fas_pdu) = "DTU_PDU" => DTU_ind { arep_id := GetArepld (), user_data := fas_pdu }	OPEN
R2	OPEN	FAS-PDU_ind && dmpm_service_name = "DMPM_Unitdata_ind" && ConfigurationType = "Free" && Role = "Sink" && FAS_Pdu_Type (fas_pdu) = "DTU_PDU" => DTU_ind { arep_id := GetArepld (), remote_disap_address := calling_address, user_data := fas_pdu }	OPEN
R3	OPEN	FAS-PDU_ind && dmpm_service_name = "DMPM_Unitdata_ind" && Role = "Sink" && FAS_Pdu_Type (fas_pdu) <> "DTU_PDU" => Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAS", reason_code := "Invalid FAS-PDU", additional_detail := "null" }	CLOSED
R4	OPEN	ErrorToARPM => (no actions taken)	OPEN
R5	OPEN	FAS-PDU_ind && Role = "Source" => Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAS", reason_code := "Invalid Event for Role", additional_detail := "null" }	CLOSED

12.5.3.2 AR Protocol Machine (ARPM) for QUB AREP

12.5.3.2.1 QUB ARPM states

The defined states and their descriptions of the QUB ARPM are shown in Table 895 and Figure 263.

Table 895 – QUB ARPM states

CLOSED	The AREP is defined, but not capable of sending or receiving FAS-PDUs. It may send or receive Associate service FAS-PDUs while in this state.
OPEN	The AREP is defined and capable of sending or receiving FAS-PDUs.
REQUESTING (REQ)	The AREP has sent an Associate Request FAS-PDU and is waiting for a response from the remote AREP.
RESPONDING (RSP)	The AREP has received an Associate Request FAS-PDU, delivered an Associate.indication primitive and is waiting for a response from its user.
REPLIED (REPL)	The Server AREP has issued an ASC_rsp(+) primitive and is waiting for receiving a "connection-established" indication from the DLL.
SAME	Indicates that the next state is the same as the current state.

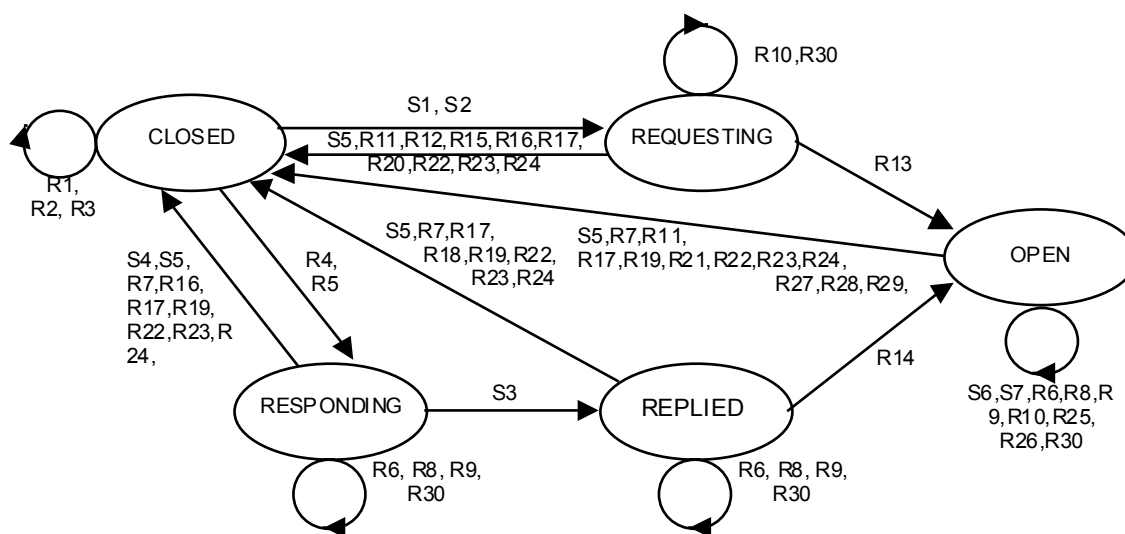
**Figure 263 – State transition diagram of QUB ARPM****12.5.3.2.2 QUB ARPM state table**

Table 896 and Table 897 specify the QUB ARPM state machine.

Table 896 – QUB ARPM state table – sender transactions

#	Current state	Event or condition => action	Next state
S1	CLOSED	<pre> ASC_req && Initiator = "True" && RemoteAddressConfigurationType := "Free" RemoteDlcepAddress := remote_dlcep_address, => FAS-PDU_req { dmpm_service_name := "DMPM_Connect_req", arep_id := GetArepld (), called_address := "default dlsap address", calling_address := "default dlsap address", local_dlcep_address := LocalDlcep, dlsdu := BuildFAS-PDU (fas_pdu_name := "ASC_ReqPDU", calling_dlcep_address := LocalDlcepAddress, called_dlcep_address := RemoteDlcepAddress, fas_data := user_data) } </pre>	REQ
S2	CLOSED	<pre> ASC_req && Initiator = "True" && RemoteAddressConfigurationType := "Linked" => FAS-PDU_req { dmpm_service_name := "DMPM_Connect_req", arep_id := GetArepld (), called_address := "default dlsap address", calling_address := "default dlsap address", local_dlcep_address := LocalDlcep, dlsdu := BuildFAS-PDU (fas_pdu_name := "ASC_ReqPDU", calling_dlcep_address := LocalDlcepAddress, called_dlcep_address := RemoteDlcepAddress, fas_data := user_data) } </pre>	REQ
S3	RSP	<pre> ASC_rsp(+) => FAS-PDU_req { dmpm_service_name := "DMPM_Connect_rsp", arep_id := GetArepld (), responding_address := "default dlsap address", local_dlcep_address := LocalDlcep, dlcep_dl_id := DlcepDlIdentifier, dlsdu := BuildFAS-PDU (fas_pdu_name := "ASC_RspPDU", fas_data := user_data) } </pre>	REPL
S4	RSP	<pre> ASC_rsp(-) => FAS-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := DlcepDlIdentifier, reason := "connection rejection–transient condition", dlsdu := BuildFAS-PDU (fas_pdu_name := "ASC_ErrPDU", fas_data := user_data) } </pre>	CLOSED
S5	NOT CLOSED	<pre> Abort_req => FAS-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := DlcepDlIdentifier, reason := "disconnection–normal condition", dlsdu := BuildFAS-PDU (fas_pdu_name := "Abort_PDU", fas_id := identifier, fas_reason_code := reason_code, fas_additional_detail := additional_detail) } </pre>	CLOSED

#	Current state	Event or condition => action	Next state
S6	OPEN	DTC_req && Role = "Client" "Peer" => FAS-PDU_req { dmpm_service_name := "DMPM_Data_req", arep_id := GetArepld (), dlcep_dl_id := DlcepDlIdentifier, dlsdu := BuildFAS-PDU (fas_pdu_name := "DTC_ReqPDU", fas_data := user_data) } 	OPEN
S7	OPEN	DTC_rsp && Role = "Server" "Peer" => FAS-PDU_req { dmpm_service_name := "DMPM_Data_req", arep_id := GetArepld (), dlcep_dl_id := DlcepDlIdentifier, dlsdu := BuildFAS-PDU (fas_pdu_name := "DTC_RspPDU", fas_data := user_data) } 	OPEN

Table 897 – QUB ARPM state table – receiver transactions

#	Current state	Event or condition => action	Next state
R1	CLOSED	Connect_ind && Initiator = "True" => <pre> FAS-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := dlcep_dl_id (from Connect_ind), reason := "Multiple Initiators", dlsdu := "null" } </pre>	CLOSED
R2	CLOSED	Connect_ind && Initiator = "False" && FAS_Pdu_Type (dls_user_data) <> "ASC_ReqPDU" => <pre> FAS-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := dlcep_dl_id (from Connect_ind), reason := "Invalid FAS-PDU", dlsdu := "null" } </pre>	CLOSED
R3	CLOSED	Connect_ind && Initiator = "False" && FAS_Pdu_Type (dls_user_data) = "ASC_ReqPDU" && RemoteAddressConfigurationType = "Linked" && RemoteDlcepAddress <> calling_dlcep_address => <pre> FAS-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := dlcep_dl_id (from Connect_ind), reason := "Remote Address Mismatch", dlsdu := "null" } </pre>	CLOSED
R4	CLOSED	Connect_ind && Initiator = "False" && FAS_Pdu_Type (dls_user_data) = "ASC_ReqPDU" && RemoteAddressConfigurationType = "Free" => <pre> RemoteDlcepAddress := calling_dlcep_address, DlcepDIIdentifier := dlcep_dl_id, MaxDlsduSizeFromRequesterNegotiated := dlsdu_size_from_requester, MaxDlsduSizeFromResponderNegotiated := dlsdu_size_from_responder, ASC_ind { arep_id := GetArepld (), user_data := dls_user_data } </pre>	RSP
R5	CLOSED	Connect_ind && Initiator = "False" && FAS_Pdu_Type (dls_user_data) = "ASC_ReqPDU" && RemoteAddressConfigurationType = "Linked" && RemoteDlcepAddress = calling_dlcep_address => <pre> DlcepDIIdentifier := dlcep_dl_id, MaxDlsduSizeFromRequesterNegotiated := dlsdu_size_from_requester, MaxDlsduSizeFromResponderNegotiated := dlsdu_size_from_responder, ASC_ind { arep_id := GetArepld (), user_data := dls_user_data } </pre>	RSP

Table 897 (continued)

#	Current state	Event or condition => action	Next state
R6	RSP REPL OPEN	Connect_ind && Initiator = "False" && RemoteAddressConfigurationType = "Linked" && RemoteDlcepAddress <> calling_dlcep_address => FAS-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := dlcep_dl_id (from Connect_ind), reason := "Remote Address Mismatch", dlsdu := "null" }	SAME
R7	RSP REPL OPEN	Connect_ind && Initiator = "False" && RemoteDlcepAddress = calling_dlcep_address => FAS-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := dlcep_dl_id (from Connect_ind), reason := "Invalid FAS-PDU", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAS", reason_code := "Invalid FAS-PDU" }	CLOSED
R8	RSP REPL OPEN	Connect_ind && Initiator = "False" && FAS_Pdu_Type (dls_user_data) = "ASC_ReqPDU" && RemoteAddressConfigurationType = "Free" && RemoteDlcepAddress <> calling_dlcep_address => FAS-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := dlcep_dl_id (from Connect_ind), reason := "AREP Busy", dlsdu := "null" }	SAME
R9	RSP REPL OPEN	Connect_ind && Initiator = "False" && FAS_Pdu_Type (dls_user_data) <> "ASC_ReqPDU" && RemoteAddressConfigurationType = "Free" && RemoteDlcepAddress <> calling_dlcep_address => FAS-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := dlcep_dl_id (from Connect_ind), reason := "Invalid FAS-PDU", dlsdu := "null" }	SAME
R10	REQ OPEN	Connect_ind && Initiator = "True" && RemoteDlcepAddress <> calling_dlcep_address => FAS-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := dlcep_dl_id (from Connect_ind), reason := "Multiple Initiators", dlsdu := "null" }	SAME

R11	REQ OPEN	<pre> Connect_ind && Initiator = "True" && RemoteDlcepAddress = calling_dlcep_address => FAS-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := dlcep_dl_id (from Connect_ind), reason := "Multiple Initiators", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAS", reason_code := "Multiple Initiators" } </pre>	CLOSED
R12	REQ	<pre> Connect_cnf && FAS_Pdu_Type (dls_user_data) <> "ASC_RspPDU" => FAS-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := DlcepDlIdentifier, reason := "Invalid FAS-PDU", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAS", reason_code := "Invalid FAS-PDU" } </pre>	CLOSED
R13	REQ	<pre> Connect_cnf && FAS_Pdu_Type (dls_user_data) = "ASC_RspPDU" => MaxDlsduSizeFromRequesterNegotiated := dlsdu_size_from_requester, MaxDlsduSizeFromResponderNegotiated := dlsdu_size_from_responder, DlIPriorityNegotiated := dll_priority, ASC_cnf(+) { arep_id := GetArepld (), user_data := dls_user_data } </pre>	OPEN
R14	REPL	<pre> FAS-PDU_ind && dmpm_service_name = "DMPM_Connection_Established_ind" => (no actions taken) </pre>	OPEN
R15	REQ	<pre> FAS-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && FAS_Pdu_Type (fas_pdu) = "ASC_ErrPDU" => ASC_cnf(-) { arep_id := GetArepld (), user_data := dls_user_data } </pre>	CLOSED
R16	REQ RSP	<pre> FAS-PDU_ind && dmpm_service_name <> "DMPM_Disconnect_ind" => FAS-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := DlcepDlIdentifier, reason := "Invalid DL-Event", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAS", reason_code := "Invalid DL-Event" } </pre>	CLOSED

R17	NOT CLOSED	<pre> FAS-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fas_pdu <> "null" && FAS_Pdu_Type (fas_pdu) = "Abort_PDU" => Abort_ind{ arep_id := GetArepld (), locally_generated := "False", identifier := AbortIdentifier (fas_pdu), reason_code := AbortReason (fas_pdu), additional_detail := AbortDetail (fas_pdu) } </pre>	CLOSED
R18	REPL	<pre> FAS-PDU_ind && ((dmpm_service_name <> "DMPM_Disconnect_ind") && (dmpm_service_name <> "DM_Connection_Established_ind")) => FAS-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := DlcepDlIdentifier, reason := "Invalid DL-Event", dlsdu := "null" }, Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "FAS", reason_code := "Invalid DL-Event", additional_detail := "null" } </pre>	CLOSED
R19	REPL RSP OPEN	<pre> FAS-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fas_pdu <> "null" && FAS_Pdu_Type (fas_pdu) <> "Abort_PDU" => Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "FAS", reason_code := "Invalid FAS-PDU", additional_detail := "null" } </pre>	CLOSED
R20	REQ	<pre> FAS-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fas_pdu <> "null" && ((FAS_Pdu_Type (fas_pdu) <> "Abort_PDU") && (FAS_Pdu_Type (fas_pdu) <> "ASC_ErrPDU")) => Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "FAS", reason_code := "Invalid FAS-PDU", additional_detail := "null" } </pre>	CLOSED
R21	OPEN	<pre> FAS-PDU_ind && ((dmpm_service_name <> "DMPM_Disconnect_ind") && (dmpm_service_name <> "DMPM_Data_ind")) => FAS-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := DlcepDlIdentifier, reason := "Invalid DL-Event", dlsdu := "null" }, Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "FAS", reason_code := "Invalid DL-Event", additional_detail := "null" } </pre>	CLOSED

R22	NOT CLOSED	<pre> FAS-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fas_pdu = "null" && originator = "remote_dls_provider" => Abort_ind{ arep_id := GetArepld (), locally_generated := "False", identifier := "Data Link Layer", reason_code := reason, additional_detail := "null" } </pre>	CLOSED
R23	NOT CLOSED	<pre> FAS-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fas_pdu = "null" && originator = "remote_dls_user" => Abort_ind{ arep_id := GetArepld (), locally_generated := "False", identifier := "FAS", reason_code := reason, additional_detail := "null" } </pre>	CLOSED
R24	NOT CLOSED	<pre> FAS-PDU_ind && dmpm_service_name = "DMPM_Disconnect_ind" && fas_pdu = "null" && originator = "local_dls_provider" => Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "Data Link Layer", reason_code := reason, additional_detail := "null" } </pre>	CLOSED
R25	OPEN	<pre> FAS-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Peer" "Server" && FAS_Pdu_Type (fas_pdu) = "DTC_ReqPDU" => DTC_ind { arep_id := GetArepld (), user_data := fas_pdu } </pre>	OPEN
R26	OPEN	<pre> FAS-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Client" "Peer" && FAS_Pdu_Type (fas_pdu) = "DTC_RspPDU" => DTC_cnf { arep_id := GetArepld (), user_data := fas_pdu } </pre>	OPEN
R27	OPEN	<pre> FAS-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Server" && FAS_Pdu_Type (fas_pdu) <> "DTC_ReqPDU" => FAS-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := DlcepDlIdentifier, reason := "Invalid FAS-PDU", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAS", reason_code := "Invalid FAS-PDU", additional_detail := "null" } </pre>	CLOSED

R28	OPEN	<pre> FAS-PDU_ind && dmpm_service_name = "DMPM_Data_ind" && Role = "Client" && FAS_Pdu_Type (fas_pdu) <> "DTC_RspPDU" => FAS-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := DlcepDlIdentifier, reason := "Invalid FAS-PDU", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAS", reason_code := "Invalid FAS-PDU", additional_detail := "null" } </pre>	CLOSED
R29	OPEN	<pre> FAS-PDU_ind && Role = "Peer" && dmpm_service_name = "DMPM_Data_ind" && ((FAS_Pdu_Type (fas_pdu) <> "DTC_ReqPDU") && (FAS_Pdu_Type (fas_pdu) <> "DTC_RspPDU")) => FAS-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := DlcepDlIdentifier, reason := "Invalid FAS-PDU", dlsdu := "null" }, Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAS", reason_code := "Invalid FAS-PDU", additional_detail := "null" } </pre>	CLOSED
R30	NOT CLOSED	<pre> ErrorToARPM => (no actions taken) </pre>	SAME
NOTE It is a local matter to report an error status to network management entities. The ARPM does not abort the existing connections. The FAS user may issue an Abort request to disconnect the current connection, depending on the type of status information conveyed by the ErrorToARPM primitive.			

12.5.3.3 AR Protocol Machine (ARPM) for BNU AREP

12.5.3.3.1 BNU ARPM states

The defined states and their descriptions of the BNU ARPM are shown in Table 898 and Figure 264.

Table 898 – BNU ARPM states

CLOSED	The AREP is defined, but not capable of sending or receiving FAS-PDUs.
REQUESTING	The AREP has issued an ASC_req and waiting for an ASC_cnf primitive.
OPEN	The AREP is defined and capable of sending or receiving FAS-PDUs.

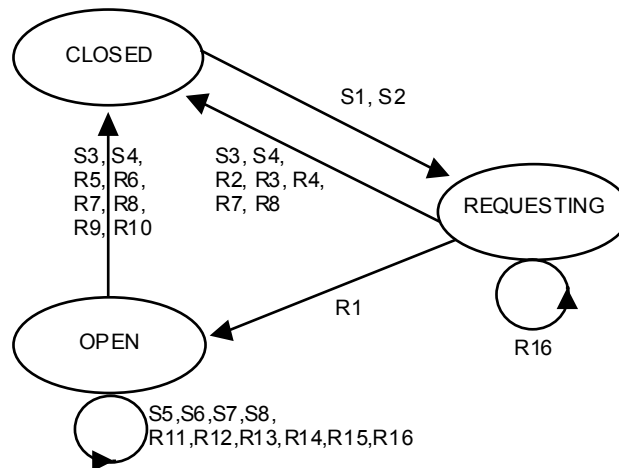


Figure 264 – State transition diagram of the BNU ARPM

12.5.3.3.2 BNU ARPM state table

Table 899 and Table 900 specify the BNU ARPM state machine.

Table 899 – BNU ARPM state table – sender transactions

#	Current state	Event or condition => action	Next state
S1	CLOSED	ASC_req && Role = "Publisher" => FAS-PDU_req { dmpm_service_name := "DMPM_Connect_req", arep_id := GetArepld (), called_address := "null", calling_address := "default dlsap address", local_dlcep_address := PublisherDlcepAddress, dlsdu := "null" }	REQ
S2	CLOSED	ASC_req && Role = "Subscriber" => FAS-PDU_req { dmpm_service_name := "DMPM_Connect_req", arep_id := GetArepld (), called_address := PublisherDlcepAddress, calling_address := "default dlsap address", local_dlcep_address := "default subscriber dlcep address", dlsdu := "null" }	REQ
S3	NOT CLOSED	Abort_req && Role = "Publisher" => FAS-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := DlcepDlIdentifier, reason := "disconnection-normal condition", dlsdu := BuildFAS-PDU (fas_pdu_name := "Abort_PDU", fas_id := identifier, fas_reason_code := reason_code, fas_additional_detail := additional_detail) }	CLOSED

#	Current state	Event or condition => action	Next state
S4	NOT CLOSED	Abort_req && Role = "Subscriber" => FAS-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := DlcepDlIdentifier, reason := "disconnection-normal condition", dlsdu := "null" }	CLOSED
S5	OPEN	DTU_req && Role = "Publisher" => FAS-PDU_req { dmpm_service_name := "DMPM_Put_req", arep_id := GetArepld (), dlsdu := BuildFAS-PDU (fas_pdu_name := "DTU_PDU", fas_sdu := user_data) }	OPEN
S6	OPEN	FCMP_req && Role = "Publisher" => FAS-PDU_req { dmpm_service_name := "DMPM_Compel_Service_req", arep_id := GetArepld (), action_class := "Local" }	OPEN
S7	OPEN	FCMP_req && Role = "Subscriber" => FAS-PDU_req { dmpm_service_name := "DMPM_Compel_Service_req", arep_id := GetArepld (), action_class := "Remote" }	OPEN
S8	OPEN	GBM_req && Role = "Subscriber" => FAS-PDU_req { dmpm_service_name := "DMPM_Get_req", arep_id := GetArepld () }	OPEN

Table 900 – BNU ARPM state table – receiver transactions

#	Current state	Event or condition => action	Next state
R1	REQ	Connect_cnf => ASC_cnf(+) { arep_id := GetArepld (), user_data := "null" }	OPEN
R2	REQ	FAS-PDU_ind && Role = "Subscriber" && dmpm_service_name = "DMPM_Disconnect_ind" && fas_pdu = "null" && originator = "local_dls_provider" => ASC_cnf(-) { arep_id := GetArepld (), user_data := "null" }	CLOSED
R3	REQ	FAS-PDU_ind && Role = "Subscriber" && dmpm_service_name = "DMPM_Disconnect_ind" && fas_pdu = "null" && originator = "remote_dls_provider" => ASC_cnf(-) { arep_id := GetArepld (), user_data := "null" }	CLOSED
R4	REQ	FAS-PDU_ind && Role = "Subscriber" && dmpm_service_name <> "DMPM_Disconnect_ind" => Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAS", reason_code := "Invalid DI Event", additional_detail := "null" }	CLOSED
R5	OPEN	FAS-PDU_ind && Role = "Subscriber" && dmpm_service_name = "DMPM_Disconnect_ind" && fas_pdu = "null" && originator = "local_dls_provider" => Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "Data Link Layer", reason_code := reason, additional_detail := "null" }	CLOSED
R6	OPEN	FAS-PDU_ind && Role = "Subscriber" && dmpm_service_name = "DMPM_Disconnect_ind" && dlsdu = "null" && originator = "remote_dls_provider" => Abort_ind { arep_id := GetArepld (), locally_generated := "False", identifier := "Data Link Layer", reason_code := reason, additional_detail := "null" }	CLOSED

Table 900 (continued)

#	Current state	Event or condition => action	Next state
R7	NOT CLOSED	FAS-PDU_ind && Role = "Subscriber" && dmpm_service_name = "DMPM_Disconnect_ind" && Fas_Pdu_Type (fas_pdu) = "Abort_PDU" => Abort_ind { arep_id := GetArepld (), locally_generated := "False", identifier := AbortIdentifier (fas_pdu), reason_code := AbortReason (fas_pdu), additional_detail := AbortDetail (fas_pdu) }	CLOSED
R8	NOT CLOSED	FAS-PDU_ind && Role = "Subscriber" && dmpm_service_name = "DMPM_Disconnect_ind" && fas_pdu <> "null" && Fas_Pdu_Type (fas_pdu) <> "Abort_PDU" => Abort_ind { arep_id := GetArepld (), locally_generated := "True", identifier := "FAS", reason_code := "Invalid FAS-PDU", additional_detail := "null" }	CLOSED
R9	OPEN	FAS-PDU_ind && Role = "Subscriber" && ((dmpm_service_name <> "DMPM_Buffer_Received_ind") && (dmpm_service_name <> "DMPM_Disconnect_ind") && (dmpm_service_name <> "DMPM_Compel_Service_cnf") && (dmpm_service_name <> "DMPM_Get_cnf") && (dmpm_service_name <> "DMPM_Buffer_Sent_ind")) => Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "FAS", reason_code := "Invalid DI Event", additional_detail := "null" }, FAS-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := DlcepDIIdentifier, reason := "Invalid DL-Event", dlsdu := "null" }	CLOSED
R10	OPEN	FAS-PDU_ind && Role = "Subscriber" && dmpm_service_name = "DMPM_Buffer_Received_ind" && Fas_pdu_type <> "DTU_PDU" => Abort_ind{ arep_id := GetArepld (), locally_generated := "True", identifier := "FAS", reason_code := "Invalid FAS-PDU", additional_detail := "null" }, FAS-PDU_req { dmpm_service_name := "DMPM_Disconnect_req", arep_id := GetArepld (), dlcep_dl_id := DlcepDIIdentifier, reason := "Invalid FAS-PDU", dlsdu := "null" }	CLOSED

Table 900 (continued)

#	Current state	Event or condition => action	Next state
R11	OPEN	FAS-PDU_ind && Role = "Subscriber" && dmpm_service_name = "DMPM_Buffer_Received_ind" && FAS_Pdu_Type (fas_pdu) = "DTU_PDU" => DTU_ind { arep_id := GetArepld (), duplicate_fas_sdu := duplicate_dlsdu, user_data := fas_pdu, local_timeliness := local_dle_timeliness, remote_timeliness := remote_dle_timeliness } 	OPEN
R12	OPEN	FAS-PDU_ind && dmpm_service_name = "DMPM_Compel_Service_cnf" => FCPM_cnf { status := dl_status } 	OPEN
R13	OPEN	FAS-PDU_ind && Role = "Subscriber" && dmpm_service_name = "DMPM_Get_cnf" && status = "success" => GBM_cnf(+) { arep_id := GetArepld (), duplicate_fas_sdu := duplicate_dlsdu, user_data := fas_pdu, local_timeliness := local_dle_timeliness, remote_timeliness := remote_dle_timeliness } 	OPEN
R14	OPEN	FAS-PDU_ind && Role = "Subscriber" && dmpm_service_name = "DMPM_Get_cnf" && status <> "success" => GBM_cnf(-) { arep_id := GetArepld () } 	OPEN
R15	OPEN	FAS-PDU_ind && Role = "Publisher" && dmpm_service_name = "DMPM_Buffer_Sent_ind" => FSTS_ind { arep_id := GetArepld (), reported_status := "Buffer-Sent" } 	OPEN
R16	NOT CLOSED	ErrorToARPM => (no actions taken) 	SAME
NOTE It is a local matter to report an error status to network management entities. The ARPM does not abort the existing connections. The FAS user may issue an Abort request to disconnect the current connection, depending on the type of status information conveyed by the ErrorToARPM primitive.			

12.5.4 DLL Mapping Protocol Machine (DMPM)

12.5.4.1 DMPM States

The defined states and their descriptions of the DMPM are shown in Table 901 and Figure 265.

Table 901 – DMPM state descriptions

State Name	Description
ACTIVE	The DMPM in the ACTIVE state is ready to transmit or receive primitives to or from the Data Like Layer and the ARPM.

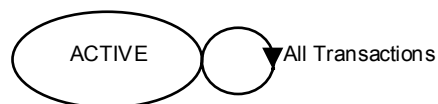


Figure 265 – State transition diagram of DMPM

12.5.4.2 DMPM state table

Table 902 and Table 903 specify the DMPM state machine.

Table 902 – DMPM state table – sender transactions

#	Current state	Event or condition => action	Next state
S1	ACTIVE	FAS-PDU_req && dmpm_service_name = "DMPM_Connect_req" => PickArep (arep_id), DL_Connect.req { calling_address := calling_address, dl_dlcep_address := local_dlcep_address, dl_dlcep_class := DlcepClass, dl_delivery_from_requester := FromRequesterToResponder, *1 dl_delivery_from_responder := FromResponderToRequester, *2 dl_dli_priority := DliPriority, dl_max_confirm_delay_on_connect := MaxConfirmDelayOnDIConnect, dl_max_confirm_delay_on_data := MaxConfirmDelayOnDIData, *1, *2 dl_dlpdu_authentication := DlpduAuthentication, dl_residual_activity_as_sender := ResidualActivityAsSender, *1 dl_residual_activity_as_receiver := ResidualActivityAsReceiver, *1, *2 dl_scheduling_policy := DISchedulingPolicy, dl_dlsdu_size_from_requester := MaxDlsduSizeFromRequester, *1 dl_dlsdu_size_from_responder := MaxDlsduSizeFromResponder, *2 dl_buffer_queue_binding_as_sender := SendingBufferOrQueueIdentifier, *1 dl_buffer_queue_binding_as_receiver := ReceivingBufferOrQueueIdentifier, *2 dl_dls_user_data := dlsdu, *1, *2 dl_sender_dl_timeliness_class := SenderDlTimelinessClass, *4 dl_sender_time_window_size := SenderTimeWindowSize, *4 dl_sender_synchronizing_dlcep := SenderSynchronizingDlcep, *4 dl_receiver_dl_timeliness_class := ReceiverDlTimelinessClass, *3 dl_receiver_time_window_size := ReceiverTimeWindowSize, *3 dl_receiver_synchronizing_dlcep := ReceiverSynchronizingDlcep *3 DlcepDIIdentifier := dl_dlcep_dl_id (from DL_Connect.req(out)) } *1: Not used with BNU AREPs whose role is Subscriber. *2: Not used with BNU AREPs whose role is Publisher. *3: Only used with BNU AREPs whose role is Subscriber. *4: Only used with BNU AREPs whose role is Publisher.	ACTIVE

Table 902 (continued)

#	Current state	Event or condition => action	Next state
S2	ACTIVE	<pre> FAS-PDU_req && dmpm_service_name = "DMPM_Connect_rsp" => PickArep (arep_id), DL_Connect.rsp { dl_dlcep_dl_id := dlcep_dl_id, dl_responding_address := responding_address, dl_dlcep_address := local_dlcep_address, dl_dlcep_class := DlcepClass, dl_delivery_from_requester := FromRequesterToResponder, dl_delivery_from_responder := FromResponderToRequester, dl_dll_priority := DIIPriority, dl_max_confirm_delay_on_connect := MaxConfirmDelayOnDIConnect, dl_max_confirm_delay_on_data := MaxConfirmDelayOnDIData, dl_dlpdu_authentication := DlpduAuthentication, dl_residual_activity_as_sender := ResidualActivityAsSender, dl_residual_activity_as_receiver := ResidualActivityAsReceiver, dl_scheduling_policy := DISchedulingPolicy, dl_dlsdu_size_from_requester := MaxDlsduSizeFromRequesterNegotiated, dl_dlsdu_size_from_responder := MaxDlsduSizeFromResponderNegotiated, dl_buffer_queue_binding_as_sender := SendingBufferOrQueueIdentifier, dl_buffer_queue_binding_as_receiver := ReceivingBufferOrQueueIdentifier, dl_dls_user_data := dlsdu } </pre>	ACTIVE
S3	ACTIVE	<pre> FAS-PDU_req && dmpm_service_name = "DMPM_Disconnect_req" => PickArep (arep_id), DL_Disconnect.req { dl_dlcep_dl_id := dlcep_dl_id, dl_reason := reason, dl_dls_user_data := dlsdu } </pre>	ACTIVE
S4	ACTIVE	<pre> FAS-PDU_req && dmpm_service_name = "DMPM_Data_req" => PickArep (arep_id), dmpm_request_id := SelectIdentifier(), DL_Data.req { dl_request_dls_user_id := dmpm_request_id, dl_dlcep_dl_id := dlcep_dl_id, dl_dls_user_data := dlsdu, } </pre>	ACTIVE
S5	ACTIVE	<pre> FAS-PDU_req && dmpm_service_name = "DMPM_Unitdata_req" => dmpm_request_id := SelectIdentifier(), PickArep (arep_id), DL_Unitdata.req { dl_request_dls_user_id := dmpm_request_id, dl_called_address := RemoteDisapAddress, dl_calling_address := LocalDisapAddress, dl_dll_priority := DIIPriority, dl_max_confirm_delay := MaxConfirmDelayOnUnitdata, dl_remote_dle_confirmed := "False", dl_dls_user_data := dlsdu } </pre>	ACTIVE

Table 902 (continued)

#	Current state	Event or condition => action	Next state
S6	ACTIVE	FAS-PDU_req && dmpm_service_name = "DMPM_Put_req" => PickArep (arep_id), DL_Put.in { dl_buffer_id := SendingBufferOrQueueIdentifier, dl_dls_user_data := dlsdu } DL_Put.out && dl_status = "success" (no actions taken) DL_Put.out && dl_status <> "success" ErrorToARPM { originator := "local_dls", reason := dl_status } }	ACTIVE
S7	ACTIVE	FAS-PDU_req && dmpm_service_name = "DMPM_Get_req" => PickArep (arep_id), DL_Get.in { dl_buffer_or_queue_id := ReceivingBufferOrQueueIdentifier } DL_Get.out && dl_reported_service_identification_class = "NONE" FAS-PDU_ind { dmpm_service_name := "DMPM_Get_cnf", status := dl_status, duplicate_dlsdu := dmpm_duplicate_dlsdu, fas_pdu := dl_dls_user_data, local_dle_timeliness := dl_local_dle_timeliness, remote_dle_timeliness := dl_sender_and_remote_dle_timeliness } }	ACTIVE
S8	ACTIVE	FAS-PDU_req && dmpm_service_name = "DMPM_Compel_req" => PickArep (arep_id), DL_Compel_Service.in { dl_action_class := action_class, dl_dlcep_dl_id := DlcepDllIdentifier } DL_Compel_Service.out FAS-PDU_ind { dmpm_service_name := "DMPM_Compel_Service_cnf", status := dl_status } }	ACTIVE

Table 903 – DMPM state table – receiver transactions

#	Current state	Event or condition => action	Next state
R1	ACTIVE	DL_Connect.ind && LocateQubArep (dl_dls_user_data) = "NOT FOUND" => DL_Disconnect.req { dl_dlcep_dl_id := dl_dlcep_dl_id (from indication primitive), dl_reason := "AREP Not Found", dl_dls_user_data := "null" }	ACTIVE
R2	ACTIVE	DL_Connect.ind && LocateQubArep (dl_dls_user_data) = "NOT QUB" => DL_Disconnect.req { dl_dlcep_dl_id := dl_dlcep_dl_id (from indication primitive), dl_reason := "Invalid AREP Type", dl_dls_user_data := "null" }	ACTIVE
R3	ACTIVE	DL_Connect.ind && LocateQubArep (dl_dls_user_data) = "True" => Connect_ind { dlcep_dl_id := dl_dlcep_dl_id, calling_dlcep_address := RequestingAREP, called_dlcep_address := RespondingAREP, called_address := dl_called_address, calling_address := dl_calling_address, dlcep_class := dl_dlcep_class, delivery_from_requester := dl_delivery_from_requester, delivery_from_responder := dl_delivery_from_responder, dll_priority := dl_dll_priority, max_confirm_delay_on_connect := dl_max_confirm_delay_on_connect, max_confirm_delay_on_data := dl_max_confirm_delay_on_data, dlsdu_size_from_requester := dl_dlsdu_size_from_requester, dlsdu_size_from_responder := dl_dlsdu_size_from_responder, dls_user_data := dl_dls_user_data }	ACTIVE
R4	ACTIVE	DL_Connect.cnf && FindAREP (dl_dlcep_dl_id) = "False" => DL_Disconnect.req { dl_dlcep_dl_id := dl_dlcep_dl_id (from confirmation primitive), dl_reason := "DLCEP Not Found", dl_dls_user_data := "null" }	ACTIVE
R5	ACTIVE	DL_Connect.cnf && FindAREP (dl_dlcep_dl_id) = "True" => Connect_cnf { responding_address := dl_responding_address, dlcep_class := dl_dlcep_class, delivery_from_requester := dl_delivery_from_requester, *1 delivery_from_responder := dl_delivery_from_responder, *2 dll_priority := dl_dll_priority, max_confirm_delay_on_connect := dl_max_confirm_delay_on_connect, max_confirm_delay_on_data := dl_max_confirm_delay_on_data, dlsdu_size_from_requester := dl_dlsdu_size_from_requester, *1 dlsdu_size_from_responder := dl_dlsdu_size_from_responder, *2 dls_user_data := dl_dls_user_data *1 } *1: Not used with BNU AREPs whose role is Subscriber. *2: Not used with BNU AREPs whose role is Publisher.	ACTIVE
R6	ACTIVE	DL_Connection_Established.ind && FindAREP (dl_dlcep_dl_id) = "False" => (no actions taken)	ACTIVE

Table 903 (continued)

#	Current state	Event or condition => action	Next state
R7	ACTIVE	DL_Connection_Established.ind && FindAREP (dl_dlcep_dl_id) = "True" => FAS-PDU_ind { dmpm_service_name := "DMPM_Connection_Established_ind", fas_pdu := "null" }	ACTIVE
R8	ACTIVE	DL_Disconnect.ind && FindAREP (dl_dlcep_dl_id) = "False" => (no actions taken)	ACTIVE
R9	ACTIVE	DL_Disconnect.ind && FindAREP (dl_dlcep_dl_id) = "True" => FAS-PDU_ind { dmpm_service_name := "DMPM_Disconnect_ind", originator := dl_originator, reason := dl_reason, fas_pdu := dl_dls_user_data }	ACTIVE
R10	ACTIVE	DL_Unitdata.ind && FindAREP (dl_called_address) = "False" => (no actions taken)	ACTIVE
R11	ACTIVE	DL_Unitdata.ind && FindAREP (dl_called_address) = "True" && ExplicitQueue = "False" => FAS-PDU_ind { dmpm_service_name := "DMPM_Unitdata_ind", calling_address := dl_calling_address, dll_priority := dl_dll_priority, fas_pdu := dl_dls_user_data }	ACTIVE
R12	ACTIVE	DL_Unitdata.ind && FindAREP (dl_called_address) = "True" && ExplicitQueue = "True" => DL_Get.in { dl_buffer_or_queue_dl_id := QueueIdentifier } DL_Get.out && dl_status = "success" && dl_reported_service_identification_class = "DL(SAP)-ADDRESS" && dl_called_dl(sap)_addresss = LocalDisapAddress FAS-PDU_ind { dmpm_service_name := "DMPM_Unitdata_ind", calling_address := dl_calling_address, dll_priority := dl_dll_priority, fas_pdu := dl_dls_user_data } DL_Get.out && dl_status <> "success" ErrorToARPM { originator := "local_dls", reason := dl_status }	ACTIVE
R13	ACTIVE	DL_Unitdata.cnf && dmpm_request_id = dl_request_dls_user_id && dl_status <> "success" => ErrorToARPM { originator := "local_dls", reason := dl_status }	ACTIVE

Table 903 (continued)

#	Current state	Event or condition => action	Next state
R14	ACTIVE	DL_Unitdata.cnf && dmpm_request_id = dl_request_dls_user_id && dl_status = "success" => (no actions taken)	ACTIVE
R15	ACTIVE	DL_Data.ind && FindAREP (dl_dlcep_dl_id) = "False" => (no actions taken)	ACTIVE
R16	ACTIVE	DL_Data.ind && FindAREP (dl_dlcep_dl_id) = "True" && ExplicitQueue = "False" => FAS-PDU_ind { dmpm_service_name := "DMPM_Data_ind", fas_pdu := dl_dls_user_data }	ACTIVE
R17	ACTIVE	DL_Data.ind && FindAREP (dl_dlcep_address) = "True" && ExplicitQueue = "True" => DL_Get.in { dl_buffer_or_queue_id := ReceivingBufferOrQueueIdentifier } DL_Get.out && dl_status = "success" && dl_reported_service_identification_class = "DLCEP" FAS-PDU_ind { dmpm_service_name := "DMPM_Data_ind", fas_pdu := dl_dls_user_data } DL_Get.out && dl_status <> "success" ErrorToARPM { originator := "local_dls", reason := dl_status }	ACTIVE
R18	ACTIVE	DL_Data.cnf && dmpm_request_id <> dl_request_dls_user_id => (no actions taken)	ACTIVE
R19	ACTIVE	DL_Data.cnf && dmpm_request_id = dl_request_dls_user_id && dl_status = "success" => (no actions taken)	ACTIVE
R20	ACTIVE	DL_Data.cnf && dmpm_request_id = dl_request_dls_user_id && dl_status <> "success" => ErrorToARPM { originator := "local_dls", reason := dl_status, dlsdu := "null" }	ACTIVE
R21	ACTIVE	DL_Buffer_Received.ind && FindAREP (dl_dlcep_dl_id) = "False" => (no actions taken)	ACTIVE

Table 903 (continued)

#	Current state	Event or condition => action	Next state
R22	ACTIVE	DL_Buffer_Received.ind && FindAREP (dl_dlcep_dl_id) = "True" => dmpm_duplicate_dlsdu := dl_duplicate_dlsdu, DL_Get.in { dl_buffer_or_queue_id := ReceivingBufferOrQueueIdentifier } DL_Get.out && dl_status = "success" && dl_reported_service_identification_class = "NONE" FAS-PDU_ind { dmpm_service_name := "DMPM_Buffer_Received_ind", duplicate_dlsdu := dmpm_duplicate_dlsdu, fas_pdu := dl_dls_user_data, local_dle_timeliness := dl_local_dle_timeliness, remote_dle_timeliness := dl_sender_and_remote_dle_timeliness } DL_Get.out && dl_status <> "success" ErrorToARPM { originator := "local_dls", reason := dl_status } 	ACTIVE
R23	ACTIVE	DL_Buffer_Sent.ind && FindAREP (dl_dlcep_dl_id) = "False" => (no actions taken) 	ACTIVE
R24	ACTIVE	DL_Buffer_Sent.ind && FindAREP (dl_dlcep_dl_id) = "True" => FAS-PDU_ind { dmpm_service_name := "DMPM_Buffer_Sent_ind" } 	ACTIVE

12.5.5 AREP state machine primitive definitions

12.5.5.1 Primitives exchanged between FAS user and FSPM

The primitives exchanged between the FAS user and the FSPM are specified in Table 904 and Table 905.

Table 904 – Primitives issued by FAS User to FSPM

Primitive name	Source	Associated parameters	Functions
ASC.req	FAS User	Arep_Id, Data, Remote_DLCEP_Address	This primitive is used to convey an Associate request primitive from the FAS user to the FSPM.
ASC.rsp(+)	FAS User	Arep_Id, Data	This primitive is used to convey an Associate response(+) primitive from the FAS user to the FSPM.
ASC.rsp(-)	FAS User	Arep_Id, Data	This primitive is used to convey an Associate primitive(-) primitive from the FAS user to the FSPM.
Abort.req	FAS User	Arep_Id, Identifier, Reason_Code, Additional_Detail	This primitive is used to convey an Abort request primitive from the FAS user to the FSPM.
DTC.req	FAS User	Arep_Id, Data	This primitive is used to convey a Data Transfer Confirmed (DTC) request primitive from the FAS user to the FSPM.
DTC.rsp	FAS User	Arep_Id, Data	This primitive is used to convey a Data Transfer Confirmed (DTC) response primitive from the FAS user to the FSPM.
DTU.req	FAS User	Arep_Id, Remote_DLSAP_Address, Data	This primitive is used to convey a Data Transfer Unconfirmed (DTU) request primitive from the FAS user to the FSPM.
FCMP.req	FAS User	Arep_Id	This primitive is used to convey an FAS-Compel (FCMP) request primitive from the FAS user to the FSPM.
GBM.req	FAS User	Arep_Id	This primitive is used to convey an Get-Buffered-Message (GBM) request primitive from the FAS user to the FSPM.

Table 905 – Primitives issued by FSPM to FAS User

Primitive name	Source	Associated parameters	Functions
ASC.ind	FSPM	Arep_Id, Data	This primitive is used to convey an Associate indication primitive from the FSPM to the FAS user.
ASC.cnf(+)	FSPM	Arep_Id, Data	This primitive is used to convey an Associate result(+) primitive from the FSPM to the FAS user.
ASC.cnf(-)	FSPM	Arep_Id, Data	This primitive is used to convey an Associate result(-) primitive from the FSPM to the FAS user.
Abort.ind	FSPM	Arep_Id, Locally_Generated, Identifier, Reason_Code, Additional_Detail	This primitive is used to convey an Abort indication primitive from the FSPM to the FAS user.
DTC.ind	FSPM	Arep_Id, Data	This primitive is used to convey a Data Transfer Confirmed (DTC) indication primitive from the FSPM to the FAS user.
DTC.cnf	FSPM	Arep_Id, Data	This primitive is used to convey a Data Transfer Confirmed (DTC) confirmation primitive from the FSPM to the FAS user.
DTU.ind	FSPM	Arep_Id, Remote_DLSAP_Address, Duplicate_FAS-SDU, Data, Local_Timeliness, Remote_Timeliness	This primitive is used to convey a Data Transfer Unconfirmed (DTU) indication primitive from the FSPM to the FAS user.
FCMP.cnf	FSPM	Arep_Id, Status	This primitive is used to convey an FAS-Compel (FCMP) confirmation primitive from the FSPM to the FAS user.
GBM.cnf(+)	FSPM	Arep_Id, Data, Local_Timeliness, Remote_Timeliness	This primitive is used to convey a Get-Buffered-Message (GBM) positive confirmation primitive from the FSPM to the FAS user.
GBM.cnf(-)	FSPM	Arep_Id	This primitive is used to convey a Get-Buffered-Message (GBM) negative confirmation primitive from the FSPM to the FAS user.
FSTS.ind	FSPM	Arep_Id, Reported_Status	This primitive is used to convey an FAS-Status (FSTS) indication primitive from the FSPM to the FAS user.

12.5.5.2 Parameters of FAS-user/FSPM primitives

All the parameters used in the primitives exchanged between the FAS-User and the FSPM are identical to those defined in the “Operational Services” section.

12.5.5.3 Primitives exchanged between ARPM and FSPM

The primitives exchanged between the ARPM and the FSPM are specified in Table 906 and Table 907.

Table 906 – Primitives issued by FSPM to ARPM

Primitive name	Source	Associated parameters	Functions
ASC_req	FSPM	user_data	This is an FAS internal primitive used to convey an Associate request primitive from the FSPM to the ARPM.
ASC_rsp(+)	FSPM	user_data	This is an FAS internal primitive used to convey an Associate response(+) primitive from the FSPM to the ARPM.
ASC_rsp(-)	FSPM	user_data	This is an FAS internal primitive used to convey an Associate response(-) primitive from the FSPM to the ARPM.
Abort_req	FSPM	identifier, reason_code, additional_detail	This is an FAS internal primitive used to convey an Abort request primitive from the FSPM to the ARPM.
DTC_req	FSPM	user_data	This is an FAS internal primitive used to convey a Data Transfer Confirmed (DTC) request primitive from the FSPM to the ARPM.
DTC_rsp	FSPM	user_data	This is an FAS internal primitive used to convey a Data Transfer Confirmed (DTC) response primitive from the FSPM to the ARPM.
DTU_req	FSPM	remote_dlsap_address, user_data	This is an FAS internal primitive used to convey a Data Transfer Unconfirmed (DTU) request primitive from the FSPM to the ARPM.
FCMP_req	FSPM	(none)	This is an FAS internal primitive used to convey a FAS-Compel (FCMP) request primitive from the FSPM to the ARPM.
GBM_req	FSPM	(none)	This is an FAS internal primitive used to convey a Get-Buffered-Message (GBM) request primitive from the FSPM to the ARPM.

Table 907 – Primitives issued by ARPM to FSPM

Primitive name	Source	Associated parameters	Functions
ASC_ind	ARPM	arep_id, user_data	This is an FAS internal primitive used to convey an Associate indication primitive from the ARPM to the FSPM.
ASC_cnf(+)	ARPM	arep_id, user_data	This is an FAS internal primitive used to convey an Associate response(+) primitive from the ARPM to the FSPM.
ASC_cnf(-)	ARPM	arep_id, user_data	This is an FAS internal primitive used to convey an Associate response(-) primitive from the ARPM to the FSPM.
Abort_ind	ARPM	arep_id, locally_generated, identifier, reason_code, additional_detail	This is an FAS internal primitive used to convey an Abort primitive from the ARPM to the FSPM.
DTC_ind	ARPM	arep_id, user_data	This is an FAS internal primitive used to convey a Data Transfer Confirmed (DTC) indication primitive from the ARPM to the FSPM.
DTC_cnf	ARPM	arep_id, user_data	This is an FAS internal primitive used to convey a Data Transfer Confirmed (DTC) confirmation primitive from the ARPM to the FSPM.
DTU_ind	ARPM	arep_id, remote_dlsap_address, duplicate_fas_sdu, user_data, local_timeliness, remote_timeliness	This is an FAS internal primitive used to convey a Data Transfer Unconfirmed (DTU) indication primitive from the ARPM to the FSPM.
FCMP_cnf	ARPM	arep_id, status	This is an FAS internal primitive used to convey a FAS-Compel (FCMP) confirmation primitive from the ARPM to the FSPM.
GBM_cnf(+)	ARPM	arep_id, duplicate_fas_sdu, user_data, local_timeliness, remote_timeliness	This is an FAS internal primitive used to convey a Get-Buffered-Message (GBM) positive confirmation primitive from the ARPM to the FSPM.
GBM_cnf(-)	ARPM	arep_id	This is an FAS internal primitive used to convey a Get-Buffered-Message (GBM) negative confirmation primitive from the ARPM to the FSPM.
FSTS_ind	ARPM	arep_id, reported_status	This is an FAS internal primitive used to convey a FAS-Status (FSTS) indication primitive from the ARPM to the FSPM.

12.5.5.4 Parameters of FSPM/ARPM primitives

The parameters used with the primitives exchanged between the FSPM and the ARPM are described in Table 908.

Table 908 – Parameters used with primitives exchanged between FSPM and ARPM

Parameter name	Description
arep_id	This parameter is used to unanimously identify an instance of the AREP that has issued a primitive. A means for such identification is not specified by this specification.
user_data	This parameter conveys FAS-User data.
locally_generated	This parameter conveys value that is used for the Locally_Generated parameter.
identifier	This parameter conveys value that is used for the Identifier parameter.
reason_code	This parameter conveys value that is used for the Reason_Code parameter.
additional_detail	This parameter conveys value that is used for the Additional_Detail parameter.
duplicate_fas_sdu	This parameter conveys value that is used for the Duplicate_FAS-SDU parameter.
remote_dlsap_address	This parameter conveys value that is used for the Remote_DLSAP_Address parameter.
status	This parameter conveys value that is used for the Status parameter.
reported_status	This parameter conveys a Data Link Layer event status.
local_timeliness	This parameter conveys value that is used for the Local_Timeliness parameter.
remote_timeliness	This parameter conveys value that is used for the Remote_Timeliness parameter.

12.5.5.5 Primitives exchanged between DMPM and ARPM

The primitives exchanged between the DMPM and the ARPM are specified in Table 909 and Table 910.

Table 909 – Primitives issued from ARPM to DMPM

Primitive names	Source	Associated parameters	Functions
FAS-PDU_req	ARPM	dmpm_service_name, arep_id, dlcep_dl_id, called_address, calling_address, responding_address, local_dlcep_address, reason, dlsdu	This primitive is used to request the DMPM to transfer an FAS-PDU, or to request an abort without transferring an FAS-PDU. It passes the FAS-PDU to the DMPM as a DLSDU. It also carries some of the Data Like Layer parameters that are referenced there.

Table 910 – Primitives issued by DMPM to ARPM

Primitive names	Source	Associated parameters	Functions
FAS-PDU_ind	DMPM	dmpm_service_name, originator, reason, duplicate_dlsdu, calling_address, dll_priority, fas_pdu, local_dle_timeliness, remote_dle_timeliness	This primitive is used to pass an FAS-PDU received as a Data Like Layer service data unit to a designated ARPM. It also carries some of the Data Like Layer parameters that are referenced in the ARPM.
ErrorToARPM	DMPM	originator, reason	This primitive is used to convey selected communication errors reported by the Data Like Layer to a designated ARPM.
Connect_ind	DMPM	dlcep_dl_id, calling_dlcep_address, called_dlcep_address, called_address, calling_address, dlcep_class, delivery_from_responder, delivery_from_requester, dll_priority, max_confirm_delay_on_connect, max_confirm_delay_on_data, dlsdu_size_from_requester, dlsdu_size_from_responder, dls_user_data	This primitive is used to convey a DL_Connect.ind primitive to the ARPM to process connection establishment. All the parameters that are associated with the DL_Connect.ind primitive are carried with this primitive.
Connect_cnf	DMPM	responding_address, dlcep_class, delivery_from_responder, delivery_from_requester, dll_priority, max_confirm_delay_on_connect, max_confirm_delay_on_data, dlsdu_size_from_requester, dlsdu_size_from_responder, dls_user_data	This primitive is used to convey a DL_Connect.cnf primitive to the ARPM. All the parameters that are associated with the DL_Connect.cnf are carried with this primitive.

12.5.5.6 Parameters of ARPM/DMPM primitives

The parameters used with the primitives exchanged between the ARPM and the DMPM are described in Table 911.

Table 911 – Parameters used with primitives exchanged between ARPM and DMPM

Parameter name	Description
dmpm_service_name	This parameter conveys a Data Like Layer primitive name. Possible values are all the DL-XXXX.yyy primitives defined in this section and are represented as DMPM_XXXX.yyy.
originator	This parameter conveys the value of the dl_originator parameter of the received primitive.
reason	This parameter conveys the value of the dl_reason parameter.
duplicate_dlsdu	This parameter conveys the value of the dl_duplicate_dlsdu parameter of the received primitive.
calling_address	This parameter conveys the value of the dl_calling_address parameter of the received primitive.
dll_priority	This parameter conveys the value of the dl_dll_priority parameter of the received primitive.
fas_pdu	This parameter conveys the value of the dl_dls_user_data parameter of the received primitive.
dlcep_dl_id	This parameter conveys the value of the dl_dlcep_dl_id parameter supplied with the Data Like Layer primitive.
calling_dlcep_address	This parameter conveys the value of the RequestingAREP parameter supplied with the ASC_ReqPDU.
called_dlcep_address	This parameter conveys the value of the RespondingAREP parameter supplied with the ASC_ReqPDU.
called_address	This parameter conveys the value of the dl_called_address parameter of the received primitive.
dlcep_class	This parameter conveys the value of the dl_dlcep_class parameter of the received primitive.
delivery_from_responder	This parameter conveys the value of the dl_delivery_from_responder parameter of the received primitive.
delivery_from_requester	This parameter conveys the value of the dl_delivery_from_requester parameter of the received primitive.
max_confirm_delay_on_connect	This parameter conveys the value of the dl_max_confirm_delay_on_connect parameter of the received primitive.
max_confirm_delay_on_data	This parameter conveys the value of the dl_max_confirm_delay_on_data parameter of the received primitive.
dlsdu_size_from_requester	This parameter conveys the value of the dl_dlsdu_size_from_requester parameter of the received primitive.
dlsdu_size_from_responder	This parameter conveys the value of the dl_dlsdu_size_from_responder parameter of the received primitive.
dls_user_data	This parameter conveys the value of the dl_dls_user_data parameter of the received primitive.
responding_address	This parameter conveys the value of the dl_responding_address parameter of the received primitive.

12.5.5.7 Primitives exchanged between Data Link Layer and DMPM

The primitives exchanged between the Data Link Layer and the DMPM are specified in Table 912.

NOTE The following primitives are defined in Part-3 and –4 of this International Standard.

Table 912 – Primitives exchanged between Data Link Layer and DMPM

Primitive names	Source	Associated parameters
DL_Connect.ind	Data Link Layer	dl_dlcep_dl_id, dl_called_address, dl_calling_address, dl_dlcep_class, dl_delivery_from_requester, dl_delivery_from_responder, dl_dll_priority, dl_max_confirm_delay_on_connect, dl_max_confirm_delay_on_data, dl_dlsdu_size_from_requester, dl_dlsdu_size_from_responder, dl_dls_user_data
DL_Connect.req(out)	Data Link Layer	dl_dlcep_dl_id
DL_Connect.cnf		dl_dlcep_dl_id, dl_responding_address, dl_dlcep_class, dl_delivery_from_requester, dl_delivery_from_responder, dl_dll_priority, dl_max_confirm_delay_on_connect, dl_max_confirm_delay_on_data, dl_dlsdu_size_from_requester, dl_dlsdu_size_from_responder, dl_sender_dl_timeliness_class, dl_receiver_dl_timeliness_class, dl_dls_user_data
DL_Connection_Established.ind	Data Link Layer	dl_dlcep_dl_id,
DL_Disconnect.ind	Data Link Layer	dl_dlcep_dl_id, dl_originator, dl_reason, dl_dls_user_data
DL_Data.ind	Data Link Layer	dl_dlcep_dl_id, dl_dls_user_data
DL_Data.cnf	Data Link Layer	dl_request_dls_user_id, dl_status
DL_Buffer_Received.ind	Data Link Layer	dl_dlcep_dl_id, dl_duplicate_dlsdu
DL_Unitdata.ind	Data Link Layer	dl_called_address, dl_calling_address, dl_dll_priority, dl_dls_user_data
DL_Unitdata.cnf	Data Link Layer	dl_request_dls_user_id, dl_status
DL_Buffer_Sent.ind	Data Link Layer	dl_dlcep_dl_id
DL_Get.out	Data Link Layer	dl_status, dl_reported_service_identification_class, dl_receiving_dlcep_address, dl_called_dl(sap)_address, dl_calling_dlsap_address, dl_dll_priority, dl_dls_user_data, dl_local_dle_timeliness, dl_sender_and_remote_dle_timeliness
DL_Put.out	Data Link Layer	dl_status
DL_Compel_Service.out	Data Link Layer	dl_status

Primitive names	Source	Associated parameters
DL_Connect.req(in)	DMPM	dl_called_address, dl_calling_address, dl_dlcep_address, dl_dlcep_class, dl_delivery_from_requester, dl_delivery_from_responder, dl_dll_priority, dl_max_confirm_delay_on_connect, dl_max_confirm_delay_on_data, dl_dlpdu_authentication, dl_scheduling_policy, dl_dlsdu_size_from_requester, dl_dlsdu_size_from_responder, dl_buffer_queue_binding_as_sender, dl_buffer_queue_binding_as_receiver, dl_sender_dl_timeliness_class, dl_sender_time_window_size, dl_sender_synchronizing_dlcep, dl_receiver_dl_timeliness_class, dl_receiver_time_window_size, dl_receiver_synchronizing_dlcep, dl_dls_user_data
DL_Connect.rsp	DMPM	dl_dlcep_dl_id, dl_responding_address, dl_dlcep_address, dl_dlcep_class, dl_delivery_from_requester, dl_delivery_from_responder, dl_dll_priority, dl_max_confirm_delay_on_connect, dl_max_confirm_delay_on_data, dl_dlpdu_authentication, dl_scheduling_policy, dl_dlsdu_size_from_requester, dl_dlsdu_size_from_responder, dl_buffer_queue_binding_as_sender, dl_buffer_queue_binding_as_receiver, dl_dls_user_data
DL_Unitdata.req	DMPM	dl_request_dls_user_id, dl_called_address, dl_calling_address, dl_dll_priority, dl_max_confirm_delay, dl_remote_dle_confirm, dl_dls_user_data
DL_Disconnect.req	DMPM	dl_dlcep_dl_id, dl_reason dl_dls_user_data
DL_Data.req	DMPM	dl_request_dls_user_id, dl_dlcep_dl_id, dl_dls_user_data
DL_Get.in	DMPM	dl_buffer_or_queue_id
DL_Put.in	DMPM	dl_buffer_id, dl_dls_user_data
DL_Compel_Service.in	DMPM	dl_action_class, dl_dlcep_dl_id

12.5.5.8 Parameters of DMPM/Data Link Layer primitives

The parameters used with the primitives exchanged between the DMPM and the Data Link Layer are identical to those defined in IEC 61158-3 and IEC 61158-4. They are prefixed by "dl_" to indicate that they are used by the FAS.

12.5.6 AREP state machine functions

12.5.6.1 General

This section defines internal functions that are used by the three protocol machines. See Table 913 through Table 923 for the definitions.

12.5.6.2 Functions used by FSPM

Table 913 – Function SelectArep()

Name	SelectArep	Used in	FSPM
Input		Output	
Arep_Id		True False	
Function	Looks for the AREP entry that is specified by the Arep_Id parameter. The Arep_Id parameter is provided with the FAS user service primitives.		

12.5.6.3 Functions used by ARPM

Table 914 – Function GetArepId()

Name	GetArepId ()	Used in	ARPM
Input		Output	
(none)		AREP Identifier	
Function	Returns a value that can unanimously identify the current AREP.		

Table 915 – Function BuildFAS-PDU

Name	BuildFAS-PDU	Used in	ARPM
Input		Output	
fas_pdu_name, calling_dlcep_address, called_dlcep_address, fas_data, fas_id, fas_reason_code, fas_additional_detail		dlsdu	
Function	Builds an FAS-PDU out of the parameters given as input variables.		

Table 916 – Function FAS_Pdu_Type

Name	FAS_Pdu_Type	Used in	ARPM
Input		Output	
dls_user_data		One of the FAS-PDU types defined in the FAS-PDUs section.	
Function	This function decodes the FAS-PDU that is conveyed in the dls_user_data parameter and retrieves one of the FAS-PDU types.		

Table 917 – Function AbortIdentifier

Name	AbortIdentifier	Used in	ARPM
Input		Output	
fas_pdu		The Identifier parameter of the Abort service.	
Function			
This function decodes the Abort_PDU that is conveyed in the fas_pdu parameter and extracts the Identifier parameter.			

Table 918 – Function AbortReason

Name	AbortReason	Used in	ARPM
Input		Output	
fas_pdu		The Reason Code parameter of the Abort service.	
Function			
This function decodes the Abort_PDU that is conveyed in the fas_pdu parameter and extracts the Reason Code parameter.			

Table 919 – Function AbortDetail

Name	AbortDetail	Used in	ARPM
Input		Output	
fas_pdu		The Additional Detail parameter of the Abort service.	
Function			
This function decodes the Abort_PDU that is conveyed in the fas_pdu parameter and extracts the Additional Detail parameter.			

12.5.6.4 Functions used by DMPM

Table 920 – Function PickArep

Name	PickArep	Used in	DMPM
Input		Output	
arep_id		(all the attributes of the specified AREP)	
Function	Selects the attributes for the AREP specified by the arep_id parameter. After this function is executed, the attributes of the selected AREP are available to the state machine.		

Table 921 – Function FindAREP

Name	FindAREP	Used in	DMPM
Input		Output	
dl_called_address dl_dlcep_dl_id		True False	
Function	Check if the AREP that is specified either by the dl_called_address or dl_dlcep_dl_id parameters exists or not. True means the AREP exists. If it does, this function also returns a means to send a DMPM primitive to that AREP.		

Table 922 – Function LocateQubArep

Name	LocateQubArep	Used in	DMPM
Input		Output	
dl_dls_user_data (of DL_Connect.ind)		True Not Found Not QUB	
		Access to attributes of the located AREP.	
Function	This function returns the value of True and a means to get access to attributes of the located AREP if all of the following conditions are met. Otherwise, it returns the value of either Not Found or Not QUB. a) Decodes the DLSDU that is conveyed by the dl_dls_user_data argument and checks if the FAS PDU type is "ASC_ReqPDU." b) Decodes the FAS-PDU and extracts the RequestingAREP and RespondingAREP parameters. c) Looks for the QUB AREP whose LocalDlcepAddress attribute value is equal to the RespondingAREP.		

Table 923 – Function SetIdentifier()

Name	SetIdentifier()	Used in	DMPM
Input		Output	
(none)		Data Like Layer identifier	
Function	This function picks an identifier that is used to uniquely identify Data Like Layer primitives.		

CBA_<SEV: S or E>_<Reason>.

Table 924 – Error Messages

MessageId	Code	MessageText
CBA_E_DEFECT	0x8004CB11L	Hardware defect detected, replacement needed.
CBA_E_INUSE	0x8004CB0EL	The destination specified in the item identifier is already connected to some other provider.
CBA_E_INVALIDCONNECTION	0x8004CB08L	It is not allowed to specify a connection from an identifier to itself.
CBA_E_INVALIDCOOKIE	0x8004CB09L	The Cookie value is not valid.
CBA_E_INVALIDENUMVALUE	0x8004CB04L	The enumeration value is invalid.
CBA_E_INVALIDEPSILON	0x8004CB06L	The Epsilon type or value is not valid.
CBA_E_INVALIDID	0x8004CB05L	The ConsumerID or ProviderID is not valid.
CBA_E_INVALIDSUBSTITUTE	0x8004CB07L	The Substitute type or value is not valid.
CBA_E_MALFORMED	0x8004CB00L	The identifier is malformed (too long, unallowed characters, no separation characters, syntactically invalid)
CBA_E_NONACCESSIBLE	0x8004CB10L	The item is not accessible.
CBA_E_NOTAPPLICABLE	0x8004CB0FL	The operation is currently not applicable.
CBA_E_PERSISTRUNNING	0x8004CB0DL	While the Save service is running, no changes are allowed to the configuration data base (try again in a few seconds).
CBA_E_QOSTYPEUNSUPPORTED	0x8004CB0BL	The QoS type is unsupported.
CBA_E_QOSVALUEUNSUPPORTED	0x8004CB0CL	The QoS value is unsupported.
CBA_E_TIMEVALUEUNSUPPORTED	0x8004CB0AL	The time value is unsupported.
CBA_E_TYPEMISMATCH	0x8004CB03L	The type specified does not match the expected type.
CBA_E_UNKNOWNMEMBER	0x8004CB02L	The member specified in the item identifier is not known.
CBA_E_UNKNOWNOBJECT	0x8004CB01L	The object specified in the item identifier is not known.
CBA_S_ESTABLISHING	0x0004CA01L	The connection is not yet established.
CBA_S_NOCONNECTION	0x0004CA02L	There is no connection from the provider to this specific consumer.
CBA_S_PERSISTPENDING	0x0004CA00L	The value requested is currently not persistent.
CBA_S_VALUEBUFFERED	0x0004CA03L	The value was only buffered and has no immediate effect on the process (e.g. device is in state CBAReady).
CBA_S_VALUEUNCERTAIN	0x0004CA04L	The value requested is currently uncertain.
DISP_E_BADINDEX	0x8002000BL	Invalid index.
DISP_E_BADPARAMCOUNT	0x8002000EL	The number of elements provided to DISPPARAMS is different from the number of arguments accepted by the method or property.
DISP_E_BADVARTYPE	0x80020008L	One of the arguments is not a valid variant type.
DISP_E_EXCEPTION	0x80020009L	The application needs to raise an exception. In this case, the structure passed in pExcepInfo should be filled in.
DISP_E_MEMBERNOTFOUND	0x80020003L	The requested member does not exist, or the call to Invoke tried to set the value of a read-only property.
DISP_E_NONAMEDARGS	0x80020007L	This implementation of IDispatch does not support named arguments
DISP_E_OVERFLOW	0x8002000AL	One of the arguments could not be coerced to the specified type.
DISP_E_PARAMNOTFOUND	0x80020004L	One of the parameter DISPIDs does not correspond to a parameter on the method. In this case, puArgErr should be set to the first argument that contains the error.
DISP_E_PARAMNOTOPTIONAL	0x8002000FL	A required parameter was omitted.
DISP_E_TYPEMISMATCH	0x80020005L	One or more of the arguments could not be coerced. The index within rgvarg of the first parameter with the incorrect type is returned

		in the puArgErr parameter.
DISP_E_UNKNOWNINTERFACE	0x80020001L	The interface identifier passed in riid is not IID_NULL.
DISP_E_UNKNOWNLCID	0x8002000CL	Unknown language.
DISP_E_UNKNOWNNAME	0x80020006L	Unknown name.
E_FAIL	0x80004005L	Unspecified error.
E_INVALIDARG	0x80070057L	One ore more arguments are invalid.
E_NOINTERFACE	0x80004002L	No such interface supported.
E_NOTIMPL	0x80004001L	The service is not implemented.
E_OUTOFMEMORY	0x8007000EL	Insufficient memory to complete the call.
RPC_E_INVALID_OBJECT	0x80010114L	The requested object does not exist.
RPC_E_INVALID_OID	0x80070777L	The specified object was not found or recognized.
RPC_E_INVALID_OXID	0x80070776L	The object exporter was not found.
RPC_E_INVALID_SET	0x80070778L	The object exporter set was not found.
RPC_E_VERSION_MISMATCH	0x80010110L	The ORPC version on the client and server machine does not match.
RPC_S_PROCNUM_OUT_OF_RANGE	0x000006D1L	The procedure number is out of range.
S_FALSE	0x00000001L	Success but with additional results, "function worked and the result is false".
S_OK	0x00000000L	Success, "everything worked".
TYPE_E_ELEMENTNOTFOUND	0x8002802BL	Element not found.

13.1.1.3 Interface Definitions for the FAL AE

13.1.1.3.1 Overview

The IDL description for the FAL AE, the abstract classes and the used type definitions are defined below. The types are composed of enumerations and structures of other types. These types are used by the FAL services defined in the interface description to store data into a classified and known container or to use quality codes in a readable form. In the class definitions are the interfaces summarized.

13.1.1.3.2 Coding of FAL ASE Data Types

13.1.1.3.2.1 VARTYPE

The allowed values of VARTYPE are shown in Table 925.

Table 925 – VARTYPE

Value	Code
VT_EMPTY	0x0000
VT_NULL	0x0001
VT_I2	0x0002
VT_I4	0x0003
VT_R4	0x0004
VT_R8	0x0005
VT_CY	0x0006
VT_DATE	0x0007
VT_BSTR	0x0008
VT_DISPATCH	0x0009
VT_ERROR	0x000A
VT_BOOL	0x000B
VT_UNKNOWN	0x000D
VT_I1	0x0010
VT_UI1	0x0011
VT_UI2	0x0012
VT_UI4	0x0013
VT_USERDEFINED	0x001D
VT_SAFEARRAY_I2	0x2002
VT_SAFEARRAY_I4	0x2003
VT_SAFEARRAY_R4	0x2004
VT_SAFEARRAY_R8	0x2005
VT_SAFEARRAY_CY	0x2006
VT_SAFEARRAY_DATE	0x2007
VT_SAFEARRAY_BSTR	0x2008
VT_SAFEARRAY_BOOL	0x200B
VT_SAFEARRAY_I1	0x2010
VT_SAFEARRAY_UI1	0x2011
VT_SAFEARRAY_UI2	0x2012
VT_SAFEARRAY_UI4	0x2013

13.1.1.3.2.2 ITEMQUALITYDEF

This data type contains the status information of the related data. It consists of three portions: Quality, Substatus and Limits. There are four states of quality (Bad, Uncertain, Good (Non Cascade) and Good (Cascade)), a set of sub-status values for each quality, and four states of the limits (OK, low limited (LL), high limited (HL) and constant (C)). The allowed values are shown in Table 926. The detailed encoding of the quality code is shown in Figure 267.

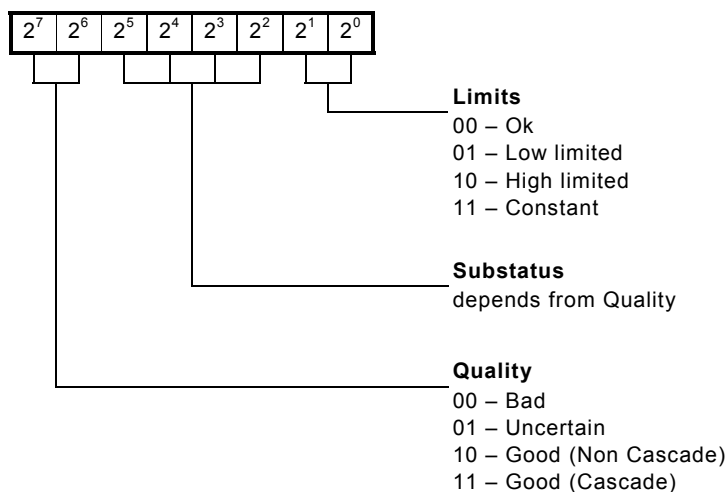
**Figure 267 – Coding scheme of ITEMQUALITYDEF**

Table 926 – ITEMQUALITYDEF values

Value	Code	Value	Code
BadNonSpecific	0x00	UncertainEngineeringUnitsExceededHL	0x56
BadNonSpecificLL	0x01	UncertainEngineeringUnitsExceededC	0x57
BadNonSpecificHL	0x02	UncertainSubNormal	0x58
BadNonSpecificC	0x03	UncertainSubNormalLL	0x59
BadConfigurationError	0x04	UncertainSubNormalHL	0x5A
BadConfigurationErrorLL	0x05	UncertainSubNormalC	0x5B
BadConfigurationErrorHL	0x06	UncertainConfigurationError	0x5C
BadConfigurationErrorC	0x07	UncertainConfigurationErrorLL	0x5D
BadNotConnected	0x08	UncertainConfigurationErrorHL	0x5E
BadNotConnectedLL	0x09	UncertainConfigurationErrorC	0x5F
BadNotConnectedHL	0x0A	UncertainSimulatedValue	0x60
BadNotConnectedC	0x0B	UncertainSimulatedValueLL	0x61
BadDeviceFailure	0x0C	UncertainSimulatedValueHL	0x62
BadDeviceFailureLL	0x0D	UncertainSimulatedValueC	0x63
BadDeviceFailureHL	0x0E	UncertainSensorCalibration	0x64
BadDeviceFailureC	0x0F	UncertainSensorCalibrationLL	0x65
BadSensorFailure	0x10	UncertainSensorCalibrationHL	0x66
BadSensorFailureLL	0x11	UncertainSensorCalibrationC	0x67
BadSensorFailureHL	0x12	GoodNonCascOk	0x80
BadSensorFailureC	0x13	GoodNonCascOkC	0x83
BadLastKnownValue	0x14	GoodNonCascActiveUpdateEvent	0x84
BadLastKnownValueLL	0x15	GoodNonCascActiveUpdateEventLL	0x85
BadLastKnownValueHL	0x16	GoodNonCascActiveUpdateEventHL	0x86
BadLastKnownValueC	0x17	GoodNonCascActiveUpdateEventC	0x87
BadCommFailure	0x18	GoodNonCascActiveAdvisoryAlarm	0x88
BadCommFailureLL	0x19	GoodNonCascActiveAdvisoryAlarmLL	0x89
BadCommFailureHL	0x1A	GoodNonCascActiveAdvisoryAlarmHL	0x8A
BadCommFailureC	0x1B	GoodNonCascActiveAdvisoryAlarmC	0x8B
BadOutOfService	0x1C	GoodNonCascActiveCriticalAlarm	0x8C
BadOutOfServiceLL	0x1D	GoodNonCascActiveCriticalAlarmLL	0x8D
BadOutOfServiceHL	0x1E	GoodNonCascActiveCriticalAlarmHL	0x8E
BadOutOfServiceC	0x1F	GoodNonCascActiveCriticalAlarmC	0x8F
UncertainNonSpecific	0x40	GoodNonCascUnackUpdateEvent	0x90
UncertainNonSpecificLL	0x41	GoodNonCascUnackUpdateEventLL	0x91
UncertainNonSpecificHL	0x42	GoodNonCascUnackUpdateEventHL	0x92
UncertainNonSpecificC	0x43	GoodNonCascUnackUpdateEventC	0x93
UncertainLastUsableValue	0x44	GoodNonCascUnackAdvisoryAlarm	0x94
UncertainLastUsableValueLL	0x45	GoodNonCascUnackAdvisoryAlarmLL	0x95
UncertainLastUsableValueHL	0x46	GoodNonCascUnackAdvisoryAlarmHL	0x96
UncertainLastUsableValueC	0x47	GoodNonCascUnackAdvisoryAlarmC	0x97
UncertainSubstituteSet	0x48	GoodNonCascUnackCriticalAlarm	0x98
UncertainSubstituteSetLL	0x49	GoodNonCascUnackCriticalAlarmLL	0x99
UncertainSubstituteSetHL	0x4A	GoodNonCascUnackCriticalAlarmHL	0x9A
UncertainSubstituteSetC	0x4B	GoodNonCascUnackCriticalAlarmC	0x9B
UncertainInitialValue	0x4C	GoodNonCascInitialFailSafe	0xA0
UncertainInitialValueLL	0x4D	GoodNonCascInitialFailSafeLL	0xA1
UncertainInitialValueHL	0x4E	GoodNonCascInitialFailSafeHL	0xA2
UncertainInitialValueC	0x4F	GoodNonCascInitialFailSafeC	0xA3
UncertainSensorNotAccurate	0x50	GoodNonCascMaintenanceRequired	0xA4
UncertainSensorNotAccurateLL	0x51	GoodNonCascMaintenanceRequiredLL	0xA5
UncertainSensorNotAccurateHL	0x52	GoodNonCascMaintenanceRequiredHL	0xA6
UncertainSensorNotAccurateC	0x53	GoodNonCascMaintenanceRequiredC	0xA7
UncertainEngineeringUnitsExceeded	0x54	GoodCascOk	0xC0
UncertainEngineeringUnitsExceededLL	0x55	GoodCascOkC	0xC3

Value	Code	Value	Code
GoodCascInitializationAcknowledge	0xC4	GoodCascDoNotSelect	0xD4
GoodCascInitializationAcknowledgeLL	0xC5	GoodCascDoNotSelectLL	0xD5
GoodCascInitializationAcknowledgeHL	0xC6	GoodCascDoNotSelectHL	0xD6
GoodCascInitializationAcknowledgeC	0xC7	GoodCascDoNotSelectC	0xD7
GoodCascInitializationRequest	0xC8	GoodCascLocalOverride	0xD8
GoodCascInitializationRequestLL	0xC9	GoodCascLocalOverrideLL	0xD9
GoodCascInitializationRequestHL	0xCA	GoodCascLocalOverrideHL	0xDA
GoodCascInitializationRequestC	0xCB	GoodCascLocalOverrideC	0xDB
GoodCascNotInvited	0xCC	GoodCascInitiateFailSafe	0xE0
GoodCascNotInvitedLL	0xCD	GoodCascInitiateFailSafeLL	0xE1
GoodCascNotInvitedHL	0xCE	GoodCascInitiateFailSafeHL	0xE2
GoodCascNotInvitedC	0xCF	GoodCascInitiateFailSafeC	0xE3

13.1.1.3.2.3 STATEDEF

The allowed values of STATEDEF are shown in Table 927.

Table 927 – STATEDEF values

Value	Code
CBANonExistent	0x00
CBAInitializing	0x01
CBAReady	0x02
CBAOperating	0x03
CBADefect	0x04

13.1.1.3.2.4 GROUPERRORDEF

The allowed values of GROUPERRORDEF are shown in Table 928.

Table 928 – GROUPERRORDEF values

Value	Code
CBANonAccessible	0x00
CBAOkay	0x01
CBAProblem	0x02
CBAUnknown	0x03

13.1.1.3.2.5 ACCESSRIGHTSDEF

The allowed values of ACCESSRIGHTSDEF are shown in Table 929.

Table 929 – ACCESSRIGHTSDEF values

Value	Code
CBANoAccess	0x00
CBAReadAccess	0x01
CBAReadAccess	0x02
CBAFullAccess	0x03

13.1.1.3.2.6 PERSISTDEF

The allowed values of PERSISTDEF are shown in Table 930.

Table 930 – PERSISTDEF values

Value	Code
CBAVolatile	0x00
CBAPendingPersistent	0x01
CBAPersistent	0x02

13.1.1.3.2.7 READITEMOUT

```
typedef struct READITEMOUT
{
    VARIANT          Value;           // value
    ITEMQUALITYDEF   QualityCode;     // Quality Code
    FILETIME         TimeStamp;       // Time stamp
    HRESULT          ErrorState;      // Error condition
} READITEMOUT, *pREADITEMOUT;
```

13.1.1.3.2.8 WRITEITEMIN

```
typedef struct WRITEITEMIN
{
    [string] LPWSTR   Item;           // Identifier
    VARIANT     Value;           // value
} WRITEITEMIN, *pWRITEITEMIN;
```

13.1.1.3.2.9 WRITEITEMQCDIN

```
typedef struct WRITEITEMQCDIN
{
    WRITEITEMIN   WriteItem;         // WRITEITEMIN structure
    ITEMQUALITYDEF QualityCode;     // Quality Code
    FILETIME      TimeStamp;        // Time stamp
} WRITEITEMQCDIN, *pWRITEITEMQCDIN;
```

13.1.1.3.2.10 ADDCONNECTIONIN

```
typedef struct ADDCONNECTIONIN
{
    [string] LPWSTR   ProviderItem;  // Identifier of connection source
    [string] LPWSTR   ConsumerItem;  // Identifier of connection target
    PERSISTDEF        Persistence;    // Required persistence
    VARIANT            SubstituteValue; // Substitute value
    VARIANT            Epsilon;       // Hysteresis as absolute value change
} ADDCONNECTIONIN, *pADDCONNECTIONIN;
```

13.1.1.3.2.11 ADDCONNECTIONOUT

```
typedef struct ADDCONNECTIONOUT
{
    DWORD          ConsumerID;        // ConsumerID
    WORD           Version;           // Version number of the connection
    HRESULT        ErrorState;        // Error condition
} ADDCONNECTIONOUT, *pADDCONNECTIONOUT;
```

13.1.1.3.2.12 GETIDOUT

```
typedef struct GETIDOUT
{
    DWORD          ConsumerID;        // ConsumerID
    BOOLEAN        State;             // Connection state (inactive = false,
active = true)
    WORD           Version;           // Version number
    HRESULT        ErrorState;        // Error condition
} GETIDOUT, *pGETIDOUT;
```

13.1.1.3.2.13 GETCONNECTIONOUT

```
typedef struct GETCONNECTIONOUT
{
    [string] LPWSTR      Provider;          // Identifier for the provider
    [string] LPWSTR      ProviderItem;      // Identifier of connection source
    [string] LPWSTR      ConsumerItem;      // Identifier of connection target
    VARIANT             SubstituteValue;    // Substitute value
    VARIANT             Epsilon;           // Hysteresis as absolute value change
    WORD                QoSType;           // QoS type
    WORD                QoSValue;          // QoS qualifier
    BOOLEAN             State;             // Connection state (inactive = false,
active = true)
    PERSISTDEF          Persistence;       // Persistence of the connection
    WORD                Version;           // Version number
    HRESULT             ErrorState;        // Error condition
} GETCONNECTIONOUT, *pGETCONNECTIONOUT;
```

13.1.1.3.2.14 CONNECTIN

```
typedef struct CONNECTIN
{
    [string] LPWSTR      ProviderItem;      // Identifier of connection source
    VARTYPE             DataType;           // Data type
    VARIANT             Epsilon;           // Hysteresis as absolute value change
    DWORD               ConsumerID;        // ConsumerID
} CONNECTIN, *pCONNECTIN;
```

13.1.1.3.2.15 CONNECTOUT

```
typedef struct CONNECTOUT
{
    DWORD               ProviderID;         // ProviderID
    HRESULT             ErrorState;         // Error condition
} CONNECTOUT, *pCONNECTOUT;
```

13.1.1.3.2.16 UUID

Predefined values are shown in Table 931.

Table 931 – UUID values

Value	Code
UUID_NULL	00000000-0000-0000-0000-000000000000
UUID_IUnknown	00000000-0000-0000-C000-000000000046
UUID_IDispatch	00020400-0000-0000-C000-000000000046
UUID_ICBAPhysicalDevice	CBA00001-6C97-11D1-8271-00A02442DF7D
UUID_ICBABrowse	CBA00002-6C97-11D1-8271-00A02442DF7D
UUID_ICBAPersist	CBA00005-6C97-11D1-8271-00A02442DF7D
UUID_ICBALogicalDevice	CBA00011-6C97-11D1-8271-00A02442DF7D
UUID_ICBAState	CBA00012-6C97-11D1-8271-00A02442DF7D
UUID_ICBATime	CBA00014-6C97-11D1-8271-00A02442DF7D
UUID_ICBAGroupError	CBA00015-6C97-11D1-8271-00A02442DF7D
UUID_ICBAAccoMgt	CBA00041-6C97-11D1-8271-00A02442DF7D
UUID_ICBAAccoServer	CBA00043-6C97-11D1-8271-00A02442DF7D
UUID_ICBAAccoCallback	CBA00042-6C97-11D1-8271-00A02442DF7D
UUID_ICBAAccoSync	CBA00044-6C97-11D1-8271-00A02442DF7D
UUID_ICBARTAuto	CBA00051-6C97-11D1-8271-00A02442DF7D
UUID_PhysicalDevice	CBA00000-6C97-11D1-8271-00A02442DF7D
UUID_LogicalDevice	CBA00010-6C97-11D1-8271-00A02442DF7D
UUID_ACCO	CBA00040-6C97-11D1-8271-00A02442DF7D
UUID_RTAuto	CBA00050-6C97-11D1-8271-00A02442DF7D

13.1.1.3.3 FAL ASE Class Interfaces

13.1.1.3.3.1 ICBAPhysicalDevice

```
[
    object,
    uuid(CBA00001-6C97-11D1-8271-00A02442DF7D),
    dual, oleautomation,
    helpstring(„ICBAPhysicalDevice Interface“),
    pointer_default(unique)
]
interface ICBAPhysicalDevice : IDispatch
{
    [propget, helpstring(„Producer/Manufacturer of the physical device“)]
    HRESULT Producer([out, retval] BSTR *pVal);

    [propget, helpstring(„Catalog Name of the physical device“)]
    HRESULT Product([out, retval] BSTR *pVal);

    [propget, helpstring(„SerialNo of the physical device“)]
    HRESULT SerialNo([out, retval] VARIANT *pVal);

    [propget, helpstring(„Production Date of the physical device“)]
    HRESULT ProductionDate([out, retval] DATE *pVal);

    [helpstring(„Revision major.minor number of physical device“)]
    HRESULT Revision([out] SHORT *pMajor, [out] SHORT *pMinor);

    [propget, helpstring(„Returns LogicalDevice by Name“)]
    HRESULT LogicalDevice([in] BSTR Name, [out, retval] ICBALogicalDevice **ppLDev);
};
```

13.1.1.3.3.2 ICBABrowse

```
[
    object,
    uuid(CBA00002-6C97-11D1-8271-00A02442DF7D),
    dual, oleautomation,
    helpstring(„ICBABrowse Interface“),
    pointer_default(unique)
]
```

```

interface ICBABrowse : IDispatch
{
    [propget, helpstring(„Returns number of items“)]
    HRESULT Count([out, retval] LONG *pVal);

    [helpstring(„Browse all available items“)]
    HRESULT BrowseItems(
        [in] LONG Offset,
        // Offset - !=0 if last call did not return all items
        [in] LONG MaxReturn,
        // At most return that many items
        [out] VARIANT *pItem,
        // SAFEARRAY(BSTR) - Item names
        [out, optional] VARIANT *pDataTypes,
        // SAFEARRAY(LONG) - VARTYPE
        [out, optional] VARIANT *pAccessRights);
    // SAFEARRAY(LONG) - ACCESSRIGHTSDEF
};

```

13.1.1.3.3.3 ICBAPersist

```

[
    object,
    uuid(CBA00005-6C97-11D1-8271-00A02442DF7D),
    dual, oleautomation,
    helpstring(„ICBAPersist Interface“),
    pointer_default(unique)
]
interface ICBAPersist : IDispatch
{
    [helpstring(„Save“)]
    HRESULT Save();
};

```

13.1.1.3.3.4 ICBAPhysicalDevicePC

```

[
    object,
    uuid(CBA00003-6C97-11D1-8271-00A02442DF7D),
    helpstring(„ICBAPhysicalDevicePC Interface“),
    pointer_default(unique)
]
interface ICBAPhysicalDevicePC : IUnknown
{
    [helpstring(„Add a Logical Device“)]
    HRESULT AddLogicalDevice([in] ICBALogicalDevice *pLDev, [out] LONG *pCookie);

    [helpstring(„Remove a Logical Device“)]
    HRESULT RemoveLogicalDevice([in] LONG Cookie);

    [helpstring(„Advise of the ICBAPhysicalDevicePCEvent“)]
    HRESULT AdvisePDevPC([in] ICBAPhysicalDevicePCEvent *pPhysicalDevicePCEvent,
    [out] LONG *pCookie);

    [helpstring(„Unadvise of the ICBAPhysicalDevicePCEvent“)]
    HRESULT UnadvisePDevPC([in] LONG Cookie);
};

```

13.1.1.3.3.5 ICBAPhysicalDevicePCEvent

```

[
    object,
    uuid(CBA00004-6C97-11D1-8271-00A02442DF7D),
    helpstring(„ICBAPhysicalDevicePCEvent Interface“),
    pointer_default(unique)
]
interface ICBAPhysicalDevicePCEvent : IUnknown
{
    [helpstring(„OnLogicalDeviceAdded“)]
    HRESULT OnLogicalDeviceAdded(
        [in] LONG Cookie,
        [in] ICBALogicalDevice *pLDev);
    [helpstring(„OnLogicalDeviceRemoved“)]
    HRESULT OnLogicalDeviceRemoved([in] LONG Cookie);
};

```

13.1.1.3.3.6 ICBALogicalDevice

```
[
    object,
    uuid(CBA00011-6C97-11D1-8271-00A02442DF7D),
    dual, oleautomation,
    helpstring(„ICBALogicalDevice Interface“),
    pointer_default(unique)
]
interface ICBALogicalDevice : IDispatch
{
    [propget, helpstring(„Name of the logical device“)]
    HRESULT Name([out, retval] BSTR *pVal);

    [propget, helpstring(„Producer/Manufacturer of the logical device“)]
    HRESULT Producer([out, retval] BSTR *pVal);

    [propget, helpstring(„Catalog Name of the logical device“)]
    HRESULT Product([out, retval] BSTR *pVal);

    [propget, helpstring(„SerialNo of the logical device“)]
    HRESULT SerialNo([out, retval] VARIANT *pVal);

    [propget, helpstring(„Production Date of the logical device“)]
    HRESULT ProductionDate([out, retval] DATE *pVal);

    [helpstring(„Revision major.minor number of logical device“)]
    HRESULT Revision([out] SHORT *pMajor, [out] SHORT *pMinor);

    [propget, helpstring(„Returns pointer to Active Control Connection Object“)]
    HRESULT ACCO([out, retval] IUnknown **ppACCO);

    [propget, helpstring(„Returns RT-Auto by Name“)]
    HRESULT RTAuto([in] BSTR Name, [out, retval] ICBARTAuto **ppAuto);
};
```

13.1.1.3.3.7 ICBASState

```
[
    object,
    uuid(CBA00012-6C97-11D1-8271-00A02442DF7D),
    dual, oleautomation,
    helpstring(„ICBASState Interface“),
    pointer_default(unique)
]
interface ICBASState : IDispatch
{
    [propget, helpstring(„State of the device“)]
    HRESULT State([out, retval] STATEDEF *pVal);

    [helpstring(„Activate device - transfer device to State Operating“)]
    HRESULT Activate();

    [helpstring(„Deactivate device - transfer device to State Ready“)]
    HRESULT Deactivate();

    [helpstring(„Reset device - transfer device via State Initializing to State Ready“)]
    HRESULT Reset();

    [helpstring(„Advise of the ICBASStateEvent“)]
    HRESULT AdviseState([in] ICBASStateEvent *pStateEvent, [out] LONG *pCookie);

    [helpstring(„Unadvise of the ICBASStateEvent“)]
    HRESULT UnadviseState([in] LONG Cookie);
};
```

13.1.1.3.3.8 ICBASStateEvent

```
[
    object,
    uuid(CBA00013-6C97-11D1-8271-00A02442DF7D),
    oleautomation,
    helpstring(„Event interface for States“),
    pointer_default(unique)
]
```

```
interface ICBAStateEvent : IUnknown
{
    [helpstring(„OnStateChanged“)]
    HRESULT OnStateChanged([in] STATEDEF NewState, [in] STATEDEF OldState);
};
```

13.1.1.3.3.9 ICBATime

```
[
    object,
    uuid(CBA00014-6C97-11D1-8271-00A02442DF7D),
    dual, oleautomation,
    helpstring(„ICBATime Interface“),
    pointer_default(unique)
]
interface ICBATime : IDispatch
{
    [propget, helpstring(„Get current Time“)]
    HRESULT Time([out, retval] DATE *pVal);

    [propput, helpstring(„Set Time“)]
    HRESULT Time([in] DATE NewVal);
};
```

13.1.1.3.3.10 ICBAGroupError

```
[
    object,
    uuid(CBA00015-6C97-11D1-8271-00A02442DF7D),
    dual, oleautomation,
    helpstring(„ICBAGroupError Interface“),
    pointer_default(unique)
]
interface ICBAGroupError : IDispatch
{
    [helpstring(„Get current error state“)]
    HRESULT GroupError([out] GROUPERRORDEF *pVal, [out] LONG *pMagicCookie);

    [helpstring(„Advise of the ICBAGroupErrorEvent“)]
    HRESULT AdviseGroupError([in] ICBAGroupErrorEvent *pGroupErrorEvent, [out] LONG *pCookie);

    [helpstring(„Unadvise of the ICBAGroupErrorEvent“)]
    HRESULT UnadviseGroupError([in] LONG Cookie);
};
```

13.1.1.3.3.11 ICBAGroupErrorEvent

```
[
    object,
    uuid(CBA00016-6C97-11D1-8271-00A02442DF7D),
    oleautomation,
    helpstring(„Event interface for Error States“),
    pointer_default(unique)
]
interface ICBAGroupErrorEvent : IUnknown
{
    [helpstring(„OnGroupErrorChanged“)]
    HRESULT OnGroupErrorChanged([in] GROUPERRORDEF NewState, [in] GROUPERRORDEF OldState);
};
```

13.1.1.3.3.12 ICBAAccoMgt

```
[
    object,
    uuid(CBA00041-6C97-11D1-8271-00A02442DF7D),
    helpstring(„ICBAAccoMgt Interface“),
    pointer_default(unique)
]
interface ICBAAccoMgt : IUnknown
{
    // Adding a block of connections at the consumer of the connection
    // ProviderItem[i] -> ConsumerItem[i] with QoS, data type and corresponding
    // ActiveState
    // The ConsumerID is subsequently reference to an connection
    //
```

```
[helpstring(„AddConnections“)]
HRESULT AddConnections(
    [in, string] LPWSTR Provider,
    // Identifier for the provider
    [in] WORD QoSType,
    // Type of the common Quality of Service
    [in] WORD QoSValue,
    //Additional information of the common Quality of Service
    [in] BOOLEAN State,
    // State (active=TRUE/inactive=FALSE)
    [in] DWORD Count,
    // Number of connections
    [in, size_is(Count)] ADDCONNECTIONIN *pAddConnectionIn,
    // Connection information
    [out, size_is(,Count)] ADDCONNECTIONOUT **ppAddConnectionOut);
    // Connection result

// Removing an connection block via ConsumerID
//
[helpstring(„RemoveConnections“)]
HRESULT RemoveConnections(
    [in] DWORD Count,
    // Number of connections
    [in, size_is(Count)] DWORD *pConsumerID,
    // ConsumerIDs
    [out, size_is(,Count)] HRESULT **ppError);
    // Connection-specific error

// Deleting all connections for which the device acts as a consumer.
//
[helpstring(„ClearConnections“)]
HRESULT ClearConnections();

// Activating or deactivating an connection block
// via ConsumerID (per connection)
//
[helpstring(„SetActivationState“)]
HRESULT SetActivationState(
    [in] BOOLEAN State,
    // State (active=TRUE/inactive=FALSE)
    [in] DWORD Count,
    // Number of connections
    [in, size_is(Count)] DWORD *pConsumerID,
    // ConsumerIDs
    [out, size_is(,Count)] HRESULT **ppError);
    // Connection-specific error

// Status information of the consumer
//
[helpstring(„GetInfo“)]
HRESULT GetInfo(
    [out] DWORD *pMax,
    // Maximum number of connections
    [out] DWORD *pCurCnt);
    // Currently used connections

// Reading the connection information
//
[helpstring(„GetIDs“)]
HRESULT GetIDs(
    [out] DWORD *pCount,
    // Number of connections
    [out, size_is(*pCount)] GETIDOUT **ppGetIDOut);
    // Connection properties

// Reading the connection information
//
[helpstring(„GetConnections“)]
HRESULT GetConnections(
    [in] DWORD Count,
    // Number of connections
    [in, size_is(Count)] DWORD *pConsumerID,
    // ConsumerIDs
    [out, size_is(,Count)] GETCONNECTIONOUT **ppGetConnectionOut);
    // Connection properties

// Verifying QoS values
//
```



```

[helpstring(„ReviseQoS“)]
    HRESULT ReviseQoS(
        [in, string] LPWSTR RTAuto,
        // Identifier of the RT-Auto
        [in] WORD QoSType,
        // Type of the Quality of Service
        [in] WORD QoSValue,
        // Additional information of the Quality of Service
        [out] WORD *pRevisedQoSValue);
    // Revised additional information

// Setting the PingFactor for the heartbeat monitoring of all providers
//
[propget, helpstring(„Actual Ping Factor“)]
    HRESULT PingFactor([out, retval] WORD *pVal);

[propput, helpstring(„Set Ping Factor“)]
    HRESULT PingFactor([in] WORD NewVal);

[propget, helpstring(„Returns Configuration Data Base cookie“)]
    HRESULT CDBCcookie([out, retval] LONG *pVal);
};

```

13.1.1.3.3.13 ICBAAccoCallback

```

[
    object,
    uuid(CBA00042-6C97-11D1-8271-00A02442DF7D),
    helpstring(„ICBAAccoCallback Interface“),
    pointer_default(unique)
]
interface ICBAAccoCallback : IUnknown
{
    // reception of data
    // Buffer structure:
    // header:
    // 00 UCHAR    version
    // 01 UCHAR    flags
    // 02 USHORT   cnt        -- number of records
    // 04 array of record
    //
    // each record:
    // 00 USHORT   len        -- length of record
    // 02 ULONG    ConsumerID -- consumer side of connection
    // 06 UCHAR    QualityCode -- OPC QualityCode
    // 07          value      -- according to data type
    // len-7       -- optional TimeStamp
    //
    // case ARRAY:
    // 07 USHORT   dim        -- array dimensions
    // 09 ULONG[dim] elem      -- number of elements in each dimension
    // 4*dim+09 etype[elem]    -- elements
    //
    // case BSTR:
    // 07 ULONG    strlen      -- string length (number of wide chars)
    // 11 USHORT[strlen] str   -- wide char string (not terminated with 0)
    //
    [helpstring(„OnDataChanged“)]
        HRESULT OnDataChanged(
            [in] long Length,
            // Buffer length
            [in, size_is(Length)] unsigned char * pBuffer);
    // Buffer
};

```

13.1.1.3.3.14 ICBAAccoServer

```

[
    object,
    uuid(CBA00043-6C97-11D1-8271-00A02442DF7D),
    helpstring(„ICBAAccoServer Interface“),
    pointer_default(unique)
]

```

```

interface ICBAAccoServer : IUnknown
{
    // Adding a block of provider ends of an connection to the provider
    // ProviderItem[i] mit QoS, data type and corresponding ActiveState. The ConsumerID
    // is included for each data item in the data transfer to the callback interface.
    // The ProviderID is subsequently the reference to the provider end.
    //
    [helpstring(„Connect“)]
    HRESULT Connect(
        [in, string] LPWSTR Consumer,
        // Identifier for the consumer
        [in] WORD QoSType,
        // Type of the common Quality of Service
        [in] WORD QoSValue,
        // Additional information of the common Quality of Service
        [in] BOOLEAN State,
        // State (active=TRUE/inactive=FALSE)
        [in] ICBAAccoCallback *pICBAAccoCallback,
        // Interface pointer to consumer's callback
        [in] DWORD Count,
        // Number of connections
        [in, size_is(Count)] CONNECTIN *pConnectIn,
        // Connection information
        [out] BOOLEAN *pFirstConnect,
        // First connect call
        [out, size_is(,Count)] CONNECTOUT **ppConnectOut);
    // Connection result

    // Removing a block of provider ends via ProviderID
    //
    [helpstring(„Disconnect“)]
    HRESULT Disconnect(
        [in]DWORD Count,
        // Number of connections
        [in, size_is(Count)] DWORD *pProviderID,
        // ProviderIDs
        [out, size_is(,Count)] HRESULT **ppError);
    // Connection-specific error

    // Clearing all connections that concern a given consumer
    //
    [helpstring(„DisconnectMe“)]
    HRESULT DisconnectMe(
        [in, string] LPWSTR Consumer);
    // Identifier for the consumer

    // Activating or deactivating an connection block via ProviderID
    // (per connection).
    //
    [helpstring(„SetActivation“)]
    HRESULT SetActivation(
        [in] BOOLEAN State,
        // State (active=TRUE/inactive=FALSE)
        [in] DWORD Count,
        // Number of connections
        [in, size_is(Count)] DWORD *pProviderID,
        // ProviderIDs
        [out, size_is(,Count)] HRESULT **ppError);
    // Connection-specific error

    // Heartbeat monitoring by the consumer
    //
    [helpstring(„Ping“)]
    HRESULT Ping([in, string] LPWSTR Consumer);
    // Identifier for the consumer
};

```

13.1.1.3.3.15 ICBAAccoSync

```

[
    object,
    uuid(CBA00044-6C97-11D1-8271-00A02442DF7D),
    helpstring(„ICBAAccoSync Interface“),
    pointer_default(unique)
]

```

```

interface ICBAAccoSync : IUnknown
{
    // Read items
    [helpstring("ReadItems")]
    HRESULT ReadItems(
        [in] DWORD Count,
        // Number of Items to read
        [in, size_is(Count), string] LPWSTR *pReadItem,
        // Items to read
        [out, size_is(,Count)] READITEMOUT **ppReadItemOut);
    // Item information

    // Write items
    [helpstring("WriteItems")]
    HRESULT WriteItems(
        [in] DWORD Count,
        // Number of Items to write
        [in, size_is(Count)] WRITEITEMIN *pWriteItemIn,
        // Items to write
        [out, size_is(,Count)] HRESULT **ppError);
    // Results

    // Write items with Quality Code and time stamps
    [helpstring("WriteItemsQCD")]
    HRESULT WriteItemsQCD(
        [in] DWORD Count,
        // Number of Items to write
        [in, size_is(Count)] WRITEITEMQCDIN *pWriteItemQcdIn,
        // Items to write
        [out, size_is(,Count)] HRESULT **ppError);
    // Results
};

```

13.1.1.3.3.16 ICBARTAuto

```

[
    object,
    uuid(CBA00051-6C97-11D1-8271-00A02442DF7D),
    dual, oleautomation,
    helpstring("ICBARTAuto Interface"),
    pointer_default(unique)
]
interface ICBARTAuto : IDispatch
{
    [propget, helpstring("Name of the runtime automation object")]
    HRESULT Name([out, retval] BSTR *pVal);

    [helpstring("Revision major.minor number of the runtime automation object")]
    HRESULT Revision([out] SHORT *pMajor, [out] SHORT *pMinor);
};

```

13.1.1.3.4 FAL ASE Classes

13.1.1.3.4.1 CBAPhysicalDevice

```

[
    uuid(CBA00000-6C97-11D1-8271-00A02442DF7D),
    helpstring("PhysicalDevice Class")
]
coclass CBAPhysicalDevice
{
    [default] interface ICBAPhysicalDevice;
    interface ICBABrowse;
    interface ICBAPersist;
    interface ICBAPhysicalDevicePC;
};

```

13.1.1.3.4.2 CBALogicalDevice

```

[
    uuid(CBA00010-6C97-11D1-8271-00A02442DF7D),
    noncreatable,
    helpstring("LogicalDevice Class")
]

```

```

coclass CBALogicalDevice
{
    [default] interface ICBALogicalDevice;
                interface ICBAState;
                interface ICBATime;
                interface ICBABrowse;
                interface ICBAGroupError;
                interface ICBAPersist;
};

13.1.1.3.4.3 CBAAcco

[
    uuid(CBA00040-6C97-11D1-8271-00A02442DF7D),
    noncreatable,
    helpstring(„Acco Class“)
]
coclass CBAAcco
{
    [default] interface ICBAAccoMgt;
                interface ICBAAccoServer;
                interface ICBAAccoCallback;
                interface ICBAAccoSync;
                interface ICBAGroupError;
};

13.1.1.3.4.4 CBARTAuto

[
    uuid(CBA00050-6C97-11D1-8271-00A02442DF7D),
    noncreatable,
    helpstring(„RT Auto Class“)
]
coclass CBARTAuto
{
    [default] interface ICBARTAuto;
                interface ICBABrowse;
};

13.1.1.3.4.5 CBASStateEvent

[
    uuid(CBA00060-6C97-11D1-8271-00A02442DF7D),
    noncreatable,
    helpstring(„State Event Class“)
]
coclass CBASStateEvent
{
    [default] interface ICBASStateEvent;
};

13.1.1.3.4.6 CBAGroupErrorEvent

[
    uuid(CBA00070-6C97-11D1-8271-00A02442DF7D),
    noncreatable,
    helpstring(„Group Error Event Class“)
]
coclass CBAGroupErrorEvent
{
    [default] interface ICBAGroupErrorEvent;
};

13.1.1.3.4.7 CBAPhysicalDevicePCEvent

[
    uuid(CBA00080-6C97-11D1-8271-00A02442DF7D),
    noncreatable,
    helpstring(„Physical Device PC Event Class“)
]
coclass CBAPhysicalDevicePCEvent
{
    [default] interface ICBAPhysicalDevicePCEvent;
};

```

13.2 Transfer Syntax

The transfer syntax is specified by the underlying ORPC wire protocol. The IDL defined by this specification shall be the basis for the APDU encoding and therefore the input for the ORPC model.

13.2.1 Serialization of Values and Parameter

Table 932 defines the serialization of the data types that are permitted for connections.

Table 932 – Data format for serialized connection data

Vartype	Data Type	Used Bytes in Data
VT_BOOL	2-byte signed int (True=-1, False=0)	2 bytes
VT_I1	char	1 byte
VT_UI1	unsigned char	1 byte
VT_I2	short	2 bytes
VT_UI2	unsigned short	2 bytes
VT_I4	long	4 bytes
VT_UI4	unsigned long	4 bytes
VT_R4	float	4 bytes
VT_R8	double	8 bytes
VT_CY	currency, 8-byte fixed point number	8 bytes
VT_DATE	date, 8-byte real	8 bytes
VT_BSTR	binary string	Byte count (VT_UI4) afterwards (byte count / 2) * 2-byte wide chars
VT_SAFEARRAY	multidimensional array	Array dimension (VT_UI2) for each dimension Number of elements (VT_UI4) Serialized elements of the basis data type according to their declaration. The array elements are (like in C/C++) serialized line by line. This means that the last index runs the fastest.
VT_USERDEFINED	structure	The components of the structure are packed in the sequence of their declaration (i.e. without padding bytes) and stored as defined above.

Since Length is an unsigned short, only a maximum amount of data can be transmitted in a connection. This restricts the following data types, if they are used within an connection:

- Strings can have up to 32758 characters,
- Arrays (one dimensional assumed) can have a maximum amount of data with 65515 Byte,
- and structures are limited to a maximum size of 65521 Byte.

13.3 FAL Protocol State Machines

The FAL protocol state machine structure is as defined in Figure 268. The general structure is according to the type 1 protocol machine model.

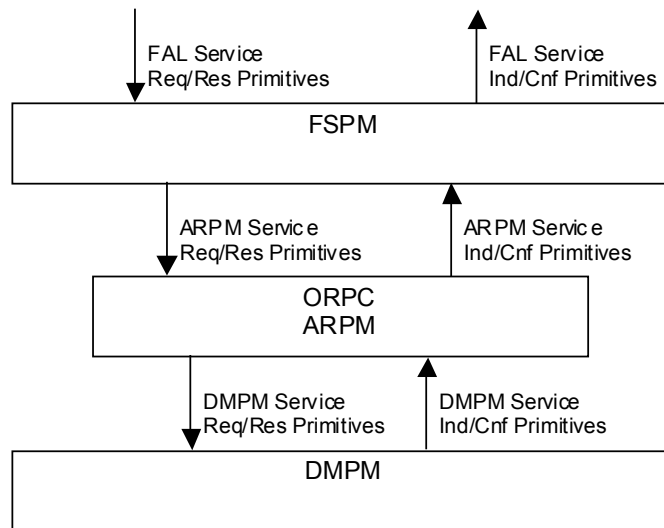


Figure 268 – Relationship among Protocol Machines

The behavior of the FAL is specified by three integrated protocol machines. The FSPM is the service interface between the FAL services that are part of the FAL Class specification and the particular AREP. Currently, the type 10 specifies exactly one AREP class and shall have only one instance of this class. Therefore, the common service attribute AREP or AREP_ID shall be omitted and an implicit AREP selection is used. The addressing of the FAL object interface instance is specified via the common class and service attribute Interface Pointer.

The FSPM is responsible for the following activities:

- To accept service primitives from the FAL service user and convert them into FAL internal primitives.
- To select the ORPC ARPM state machine based on the implicit addressing mechanism and send FAL internal primitives with unified service parameters to the ORPC ARPM.
- To accept FAL internal primitives from the ARPM and convert them into service primitives for the FAL service user.
- To deliver the FAL service primitives to the FAL user or object instances addressed via the service attribute Interface Pointer.

The ARPM specifies the conveyance type for the application relation and adds the reference to the appropriated IDL to support the data marshaling of the underlying ORPC model.

The DMPM specifies the mapping to an abstract ORPC model. The ORPC model defines therefore an abstract service interface.

13.4 AP Context State Machine

There is no AP Context State Machine defined for the FAL AE.

13.5 FAL Service Protocol Machines (FSPM)

13.5.1 Overview

This type specifies one FSPM state machine to transfer the FAL user service primitives into FAL internal service primitives and vice versa.

13.5.2 Primitive Definitions

13.5.2.1 Primitives exchanged between FSPM and FAL User

Table 933 shows the service primitives including their associated parameters issued by the FAL User and received by the FSPM.

Table 933 – Primitives issued by FAL User to FSPM

Primitive name	Source	Associated parameters	Functions
QueryInterface.req	FAL User	Interface Pointer, Interface ID	Refer to FAL Service Definition Type 10 Fieldbus IEC 61158-5
QueryInterface.res(+)	FAL User	Interface Pointer, hresult	
QueryInterface.res(-)	FAL User	Interface Pointer, hresult	
AddRef.req	FAL User	Interface Pointer	
AddRef.res	FAL User	Interface Pointer, Reference Count	
Release.req	FAL User	Interface Pointer	
Release.res(+)	FAL User	Interface Pointer, Reference Count	
Release.res(-)	FAL User	Interface Pointer, Reference Count	
GetTypeInfoCount.req	FAL User	Interface Pointer	
GetTypeInfoCount.res(+)	FAL User	Interface Pointer, hresult, pcTInfo	
GetTypeInfoCount.res(-)	FAL User	Interface Pointer, hresult	
GetTypeInfo.req	FAL User	Interface Pointer, iTInfo, lcId	
GetTypeInfo.res(+)	FAL User	Interface Pointer, hresult, ppTInfo	
GetTypeInfo.res(-)	FAL User	Interface Pointer, hresult	
GetIDsOfNames.req	FAL User	Interface Pointer, riid, rgszNames, cNames, lcId	
GetIDsOfNames.res(+)	FAL User	Interface Pointer, hresult, rgDispld	
GetIDsOfNames.res(-)	FAL User	Interface Pointer, hresult	
Invoke.req	FAL User	Interface Pointer, dispIdMember, riid, lcId, wFlags, pDispParams	
Invoke.res(+)	FAL User	Interface Pointer, hresult, pVarResult	
Invoke.res(-)	FAL User	Interface Pointer, hresult, pExcepInfo, puArgErr	
get_Producer.req	FAL User	Interface Pointer	
get_Producer.res(+)	FAL User	Interface Pointer, hresult, pVal	
get_Producer.res(-)	FAL User	Interface Pointer, hresult	
get_Product.req	FAL User	Interface Pointer	
get_Product.res(+)	FAL User	Interface Pointer, hresult, pVal	
get_Product.res(-)	FAL User	Interface Pointer, hresult	
get_SerialNo.req	FAL User	Interface Pointer	
get_SerialNo.res(+)	FAL User	Interface Pointer, hresult, pVal	
get_SerialNo.res(-)	FAL User	Interface Pointer, hresult	

Table 933(continued)

Primitive name	Source	Associated parameters	Functions
get_ProductionDate.req	FAL User	Interface Pointer	
get_ProductionDate.res(+)	FAL User	Interface Pointer, hresult, pVal	
get_ProductionDate.res(-)	FAL User	Interface Pointer, hresult	
Revision.req	FAL User	Interface Pointer	
Revision.res(+)	FAL User	Interface Pointer, hresult, pMajor, pMinor	
Revision.res(-)	FAL User	Interface Pointer, hresult	
get_LogicalDevice.req	FAL User	Interface Pointer	
get_LogicalDevice.res(+)	FAL User	Interface Pointer, hresult, ppLDev	
get_LogicalDevice.res(-)	FAL User	Interface Pointer, hresult	
get_Count.req	FAL User	Interface Pointer	
get_Count.res(+)	FAL User	Interface Pointer, hresult, pVal	
get_Count.res(-)	FAL User	Interface Pointer, hresult	
BrowseItems.req	FAL User	Interface Pointer, Offset, MaxReturn	
BrowseItems.res(+)	FAL User	Interface Pointer, hresult, pItem, pDataType, pAccessRight	
BrowseItems.res(-)	FAL User	Interface Pointer, hresult	
Save.req	FAL User	Interface Pointer	
Save.res(+)	FAL User	Interface Pointer, hresult	
Save.res(-)	FAL User	Interface Pointer, hresult	
get_Name.req	FAL User	Interface Pointer	
get_Name.res(+)	FAL User	Interface Pointer, hresult, pVal	
get_Name.res(-)	FAL User	Interface Pointer, hresult	
get_ACCO.req	FAL User	Interface Pointer	
get_ACCO.res(+)	FAL User	Interface Pointer, hresult, ppACCO	
get_ACCO.res(-)	FAL User	Interface Pointer, hresult	
get_RTAuto.req	FAL User	Interface Pointer, Name	
get_RTAuto.res(+)	FAL User	Interface Pointer, hresult, ppAuto	
get_RTAuto.res(-)	FAL User	Interface Pointer, hresult	
get_State.req	FAL User	Interface Pointer	
get_State.res(+)	FAL User	Interface Pointer, hresult, pVal	
get_State.res(-)	FAL User	Interface Pointer, hresult	
Activate.req	FAL User	Interface Pointer	
Activate.res(+)	FAL User	Interface Pointer, hresult	
Activate.res(-)	FAL User	Interface Pointer, hresult	
Deactivate.req	FAL User	Interface Pointer	
Deactivate.res(+)	FAL User	Interface Pointer, hresult	
Deactivate.res(-)	FAL User	Interface Pointer, hresult	
Reset.req	FAL User	Interface Pointer	
Reset.res(+)	FAL User	Interface Pointer, hresult	
Reset.res(-)	FAL User	Interface Pointer, hresult	

Table 933(continued)

Primitive name	Source	Associated parameters	Functions
AdviseState.req	FAL User	Interface Pointer, pStateEvent	
AdviseState.res(+)	FAL User	Interface Pointer, hresult, pCookie	
AdviseState.res(-)	FAL User	Interface Pointer, hresult	
UnadviseState.req	FAL User	Interface Pointer, Cookie	
UnadviseState.res(+)	FAL User	Interface Pointer, hresult	
UnadviseState.res(-)	FAL User	Interface Pointer, hresult	
get_Time.req	FAL User	Interface Pointer	
get_Time.res(+)	FAL User	Interface Pointer, hresult, pVal	
get_Time.res(-)	FAL User	Interface Pointer, hresult	
put_Time.req	FAL User	Interface Pointer, NewVal	
put_Time.res(+)	FAL User	Interface Pointer, hresult	
put_Time.res(-)	FAL User	Interface Pointer, hresult	
GroupError.req	FAL User	Interface Pointer	
GroupError.res(+)	FAL User	Interface Pointer, hresult, pVal, pMagicCookie	
GroupError.res(-)	FAL User	Interface Pointer, hresult	
AdviseGroupError.req	FAL User	Interface Pointer, pGroupErrorEvent	
AdviseGroupError.res(+)	FAL User	Interface Pointer, hresult, pCookie	
AdviseGroupError.res(-)	FAL User	Interface Pointer, hresult	
UnAdviseGroupError.req	FAL User	Interface Pointer, Cookie	
UnAdviseGroupError.res(+)	FAL User	Interface Pointer, hresult	
UnAdviseGroupError.res(-)	FAL User	Interface Pointer, hresult	
AddConnections.req	FAL User	Interface Pointer, Provider, QoSType, QoSValue, State, Count, List of pAddConnectionIn	
AddConnections.res(+)	FAL User	Interface Pointer, hresult, List of ppAddConnectionOut	
AddConnections.res(-)	FAL User	Interface Pointer, hresult	
RemoveConnections.req	FAL User	Interface Pointer, Count, List of pConsumerID	
RemoveConnections.res(+)	FAL User	Interface Pointer, hresult, List of ppError	
RemoveConnections.res(-)	FAL User	Interface Pointer, hresult, List of ppError	
ClearConnections.req	FAL User	Interface Pointer	
ClearConnections.res(+)	FAL User	Interface Pointer, hresult	
ClearConnections.res(-)	FAL User	Interface Pointer, hresult	
SetActivationState.req	FAL User	Interface Pointer, State, Count, List of pConsumerID	
SetActivationState.res(+)	FAL User	Interface Pointer, hresult, List of ppError	
SetActivationState.res(-)	FAL User	Interface Pointer, hresult	
GetInfo.req	FAL User	Interface Pointer	

Table 933(continued)

Primitive name	Source	Associated parameters	Functions
GetInfo.res(+)	FAL User	Interface Pointer, hresult, pMax, pCurCnt	
GetInfo.res(-)	FAL User	Interface Pointer, hresult	
GetIDs.req	FAL User	Interface Pointer	
GetIDs.res(+)	FAL User	Interface Pointer, hresult, pCount, List of ppGetIDOut	
GetIDs.res(-)	FAL User	Interface Pointer, hresult	
GetConnections.req	FAL User	Interface Pointer, Count, List of pConsumerID	
GetConnections.res(+)	FAL User	Interface Pointer, hresult, List of ppGetConnectionOut	
GetConnections.res(-)	FAL User	Interface Pointer, hresult	
ReviseQoS.req	FAL User	Interface Pointer, RTAuto, QoSType, QoSValue	
ReviseQoS.res(+)	FAL User	Interface Pointer, hresult, pRevisedQoSValue	
ReviseQoS.res(-)	FAL User	Interface Pointer, hresult	
get_PingFactor.req	FAL User	Interface Pointer	
get_PingFactor.res(+)	FAL User	Interface Pointer, hresult, pVal	
get_PingFactor.res(-)	FAL User	Interface Pointer, hresult	
put_PingFactor.req	FAL User	Interface Pointer, NewVal	
put_PingFactor.res(+)	FAL User	Interface Pointer, hresult	
put_PingFactor.res(-)	FAL User	Interface Pointer, hresult	
get_CDBCcookie.req	FAL User	Interface Pointer	
get_CDBCcookie.res(+)	FAL User	Interface Pointer, hresult, pVal	
get_CDBCcookie.res(-)	FAL User	Interface Pointer, hresult	
Connect.req	FAL User	Interface Pointer, Consumer, QoSType, QoSValue, State, pICBAAccoCallback, Count, List of pConnectIn	
Connect.res(+)	FAL User	Interface Pointer, hresult, pFirstConnect, List of ppConnectOut	
Connect.res(-)	FAL User	Interface Pointer, hresult	
Disconnect.req	FAL User	Interface Pointer, Count, List of pProviderID	
Disconnect.res(+)	FAL User	Interface Pointer, hresult, List of ppError	
Disconnect.res(-)	FAL User	Interface Pointer, hresult	
DisconnectMe.req	FAL User	Interface Pointer, Consumer	
DisconnectMe.res(+)	FAL User	Interface Pointer, hresult	
DisconnectMe.res(-)	FAL User	Interface Pointer, hresult	
SetActivation.req	FAL User	Interface Pointer, State, Count, List of pProviderID	
SetActivation.res(+)	FAL User	Interface Pointer, hresult, List of ppError	

Table 933(continued)

Primitive name	Source	Associated parameters	Functions
SetActivation.res(-)	FAL User	Interface Pointer, hresult	
Ping.req	FAL User	Interface Pointer, Consumer	
Ping.res(+)	FAL User	Interface Pointer, hresult	
Ping.res(-)	FAL User	Interface Pointer, hresult	
OnDataChanged.req	FAL User	Interface Pointer, Length, pBuffer	
OnDataChanged.res(+)	FAL User	Interface Pointer, hresult	
OnDataChanged.res(-)	FAL User	Interface Pointer, hresult	
ReadItems.req	FAL User	Interface Pointer, Count, List of pReadItem	
ReadItems.res(+)	FAL User	Interface Pointer, hresult, List of ppReadItemOut	
ReadItems.res(-)	FAL User	Interface Pointer, hresult	
WriteItems.req	FAL User	Interface Pointer, Count, List of pWriteItem	
WriteItems.res(+)	FAL User	Interface Pointer, hresult, List of ppError	
WriteItems.res(-)	FAL User	Interface Pointer, hresult	
WriteItemsQCD.req	FAL User	Interface Pointer, Count, List of pWriteItemQcdIn	
WriteItemsQCD.res(+)	FAL User	Interface Pointer, hresult, List of ppError	
WriteItemsQCD.res(-)	FAL User	Interface Pointer, hresult	
CoCreateInstance.req	FAL User	Host address, Class ID, Interface ID, Interface Pointer	
CoCreateInstance.res(+)	FAL User	hresult, Interface Pointer	
CoCreateInstance.res(-)	FAL User	hresult	
Call.req	FSPM	Interface Pointer, Service Name, List of Unified Service In-Parameter	
Call.res(+)	FSPM	Interface Pointer, Service Name, hresult, List of Unified Service Out-Parameter	
Call.res(-)	FSPM	Interface Pointer, Service Name, hresult	

Table 934 shows the service primitives including their associated parameters issued by the FSPM and received by the FAL User.

Table 934 – Primitives issued by FSPM to FAL User

Primitive name	Source	Associated parameters	Functions
QueryInterface.ind	FSPM	Interface Pointer, Interface ID	Refer to FAL Service Definition Type 10 Fieldbus IEC 61158-5
QueryInterface.cnf(+)	FSPM	Interface Pointer, hresult	
QueryInterface.cnf(-)	FSPM	Interface Pointer, hresult	
AddRef.ind	FSPM	Interface Pointer	
AddRef.cnf	FSPM	Interface Pointer, Reference Count	
Release.ind	FSPM	Interface Pointer	
Release.cnf(+)	FSPM	Interface Pointer, Reference Count	
Release.cnf(-)	FSPM	Interface Pointer, Reference Count	
GetTypeInfoCount.ind	FSPM	Interface Pointer	
GetTypeInfoCount.cnf(+)	FSPM	Interface Pointer, hresult, pcTInfo	
GetTypeInfoCount.cnf(-)	FSPM	Interface Pointer, hresult	
GetTypeInfo.ind	FSPM	Interface Pointer, iTInfo, lcId	
GetTypeInfo.cnf(+)	FSPM	Interface Pointer, hresult, ppTInfo	
GetTypeInfo.cnf(-)	FSPM	Interface Pointer, hresult	
GetIDsOfNames.ind	FSPM	Interface Pointer, riid, rgszNames, cNames, lcId	
GetIDsOfNames.cnf(+)	FSPM	Interface Pointer, hresult, rgDispId	
GetIDsOfNames.cnf(-)	FSPM	Interface Pointer, hresult	
Invoke.ind	FSPM	Interface Pointer, dispIdMember, riid, lcId, wFlags, pDispParams	
Invoke.cnf(+)	FSPM	Interface Pointer, hresult, pVarResult	
Invoke.cnf(-)	FSPM	Interface Pointer, hresult, pExcepInfo, puArgErr	
get_Producer.ind	FSPM	Interface Pointer	
get_Producer.cnf(+)	FSPM	Interface Pointer, hresult, pVal	
get_Producer.cnf(-)	FSPM	Interface Pointer, hresult	
get_Product.ind	FSPM	Interface Pointer	
get_Product.cnf(+)	FSPM	Interface Pointer, hresult, pVal	
get_Product.cnf(-)	FSPM	Interface Pointer, hresult	
get_SerialNo.ind	FSPM	Interface Pointer	
get_SerialNo.cnf(+)	FSPM	Interface Pointer, hresult, pVal	
get_SerialNo.cnf(-)	FSPM	Interface Pointer, hresult	
get_ProductionDate.ind	FSPM	Interface Pointer	
get_ProductionDate.cnf(+)	FSPM	Interface Pointer, hresult, pVal	
get_ProductionDate.cnf(-)	FSPM	Interface Pointer, hresult	
Revision.ind	FSPM	Interface Pointer	

Table 934 (continued)

Primitive name	Source	Associated parameters	Functions
Revision.cnf(+)	FSPM	Interface Pointer, hresult, pMajor, pMinor	
Revision.cnf(-)	FSPM	Interface Pointer, hresult	
get_LogicalDevice.ind	FSPM	Interface Pointer	
get_LogicalDevice.cnf(+)	FSPM	Interface Pointer, hresult, ppLDev	
get_LogicalDevice.cnf(-)	FSPM	Interface Pointer, hresult	
get_Count.ind	FSPM	Interface Pointer	
get_Count.cnf(+)	FSPM	Interface Pointer, hresult, pVal	
get_Count.cnf(-)	FSPM	Interface Pointer, hresult	
BrowseItems.ind	FSPM	Interface Pointer, Offset, MaxReturn	
BrowseItems.cnf(+)	FSPM	Interface Pointer, hresult, pItem, pDataType, pAccessRight	
BrowseItems.cnf(-)	FSPM	Interface Pointer, hresult	
Save.ind	FSPM	Interface Pointer	
Save.cnf(+)	FSPM	Interface Pointer, hresult	
Save.cnf(-)	FSPM	Interface Pointer, hresult	
get_Name.ind	FSPM	Interface Pointer	
get_Name.cnf(+)	FSPM	Interface Pointer, hresult, pVal	
get_Name.cnf(-)	FSPM	Interface Pointer, hresult	
get_ACCO.ind	FSPM	Interface Pointer	
get_ACCO.cnf(+)	FSPM	Interface Pointer, hresult, ppACCO	
get_ACCO.cnf(-)	FSPM	Interface Pointer, hresult	
get_RTAuto.ind	FSPM	Interface Pointer, Name	
get_RTAuto.cnf(+)	FSPM	Interface Pointer, hresult, ppAuto	
get_RTAuto.cnf(-)	FSPM	Interface Pointer, hresult	
get_State.ind	FSPM	Interface Pointer	
get_State.cnf(+)	FSPM	Interface Pointer, hresult, pVal	
get_State.cnf(-)	FSPM	Interface Pointer, hresult	
Activate.ind	FSPM	Interface Pointer	
Activate.cnf(+)	FSPM	Interface Pointer, hresult	
Activate.cnf(-)	FSPM	Interface Pointer, hresult	
Deactivate.ind	FSPM	Interface Pointer	
Deactivate.cnf(+)	FSPM	Interface Pointer, hresult	
Deactivate.cnf(-)	FSPM	Interface Pointer, hresult	
Reset.ind	FSPM	Interface Pointer	
Reset.cnf(+)	FSPM	Interface Pointer, hresult	
Reset.cnf(-)	FSPM	Interface Pointer, hresult	
AdviseState.ind	FSPM	Interface Pointer, pStateEvent	
AdviseState.cnf(+)	FSPM	Interface Pointer, hresult, pCookie	
AdviseState.cnf(-)	FSPM	Interface Pointer, hresult	
UnadviseState.ind	FSPM	Interface Pointer, Cookie	
UnadviseState.cnf(+)	FSPM	Interface Pointer, hresult	

Table 934 (continued)

Primitive name	Source	Associated parameters	Functions
UnadviseState.cnf(-)	FSPM	Interface Pointer, hresult	
get_Time.ind	FSPM	Interface Pointer	
get_Time.cnf(+)	FSPM	Interface Pointer, hresult, pVal	
get_Time.cnf(-)	FSPM	Interface Pointer, hresult	
put_Time.ind	FSPM	Interface Pointer, NewVal	
put_Time.cnf(+)	FSPM	Interface Pointer, hresult	
put_Time.cnf(-)	FSPM	Interface Pointer, hresult	
GroupError.ind	FSPM	Interface Pointer	
GroupError.cnf(+)	FSPM	Interface Pointer, hresult, pVal, pMagicCookie	
GroupError.cnf(-)	FSPM	Interface Pointer, hresult	
AdviseGroupError.ind	FSPM	Interface Pointer, pGroupErrorEvent	
AdviseGroupError.cnf(+)	FSPM	Interface Pointer, hresult, pCookie	
AdviseGroupError.cnf(-)	FSPM	Interface Pointer, hresult	
UnAdviseGroupError.ind	FSPM	Interface Pointer, Cookie	
UnAdviseGroupError.cnf(+)	FSPM	Interface Pointer, hresult	
UnAdviseGroupError.cnf(-)	FSPM	Interface Pointer, hresult	
AddConnections.ind	FSPM	Interface Pointer, Provider, QoSType, QoSValue, State, Count, List of pAddConnectionIn	
AddConnections.cnf(+)	FSPM	Interface Pointer, hresult, List of ppAddConnectionOut	
AddConnections.cnf(-)	FSPM	Interface Pointer, hresult	
RemoveConnections.ind	FSPM	Interface Pointer, Count, List of pConsumerID	
RemoveConnections.cnf(+)	FSPM	Interface Pointer, hresult, List of ppError	
RemoveConnections.cnf(-)	FSPM	Interface Pointer, hresult, List of ppError	
ClearConnections.ind	FSPM	Interface Pointer	
ClearConnections.cnf(+)	FSPM	Interface Pointer, hresult	
ClearConnections.cnf(-)	FSPM	Interface Pointer, hresult	
SetActivationState.ind	FSPM	Interface Pointer, State, Count, List of pConsumerID	
SetActivationState.cnf(+)	FSPM	Interface Pointer, hresult, List of ppError	
SetActivationState.cnf(-)	FSPM	Interface Pointer, hresult	
GetInfo.ind	FSPM	Interface Pointer	
GetInfo.cnf(+)	FSPM	Interface Pointer, hresult, pMax, pCurCnt	
GetInfo.cnf(-)	FSPM	Interface Pointer, hresult	
GetIDs.ind	FSPM	Interface Pointer	
GetIDs.cnf(+)	FSPM	Interface Pointer, hresult, pCount, List of ppGetIDOut	
GetIDs.cnf(-)	FSPM	Interface Pointer, hresult	

Table 934 (continued)

Primitive name	Source	Associated parameters	Functions
GetConnections.ind	FSPM	Interface Pointer, Count, List of pConsumerID	
GetConnections.cnf(+)	FSPM	Interface Pointer, hresult, List of ppGetConnectionOut	
GetConnections.cnf(-)	FSPM	Interface Pointer, hresult	
ReviseQoS.ind	FSPM	Interface Pointer, RTAuto, QoSType, QoSValue	
ReviseQoS.cnf(+)	FSPM	Interface Pointer, hresult, pRevisedQoSValue	
ReviseQoS.cnf(-)	FSPM	Interface Pointer, hresult	
get_PingFactor.ind	FSPM	Interface Pointer	
get_PingFactor.cnf(+)	FSPM	Interface Pointer, hresult, pVal	
get_PingFactor.cnf(-)	FSPM	Interface Pointer, hresult	
put_PingFactor.ind	FSPM	Interface Pointer, NewVal	
put_PingFactor.cnf(+)	FSPM	Interface Pointer, hresult	
put_PingFactor.cnf(-)	FSPM	Interface Pointer, hresult	
get_CDBCcookie.ind	FSPM	Interface Pointer	
get_CDBCcookie.cnf(+)	FSPM	Interface Pointer, hresult, pVal	
get_CDBCcookie.cnf(-)	FSPM	Interface Pointer, hresult	
Connect.ind	FSPM	Interface Pointer, Consumer, QoSType, QoSValue, State, pCBAAccoCallback, Count, List of pConnectIn	
Connect.cnf(+)	FSPM	Interface Pointer, hresult, pFirstConnect, List of ppConnectOut	
Connect.cnf(-)	FSPM	Interface Pointer, hresult	
Disconnect.ind	FSPM	Interface Pointer, Count, List of pProviderID	
Disconnect.cnf(+)	FSPM	Interface Pointer, hresult, List of ppError	
Disconnect.cnf(-)	FSPM	Interface Pointer, hresult	
DisconnectMe.ind	FSPM	Interface Pointer, Consumer	
DisconnectMe.cnf(+)	FSPM	Interface Pointer, hresult	
DisconnectMe.cnf(-)	FSPM	Interface Pointer, hresult	
SetActivation.ind	FSPM	Interface Pointer, State, Count, List of pProviderID	
SetActivation.cnf(+)	FSPM	Interface Pointer, hresult, List of ppError	
SetActivation.cnf(-)	FSPM	Interface Pointer, hresult	
Ping.ind	FSPM	Interface Pointer, Consumer	
Ping.cnf(+)	FSPM	Interface Pointer, hresult	
Ping.cnf(-)	FSPM	Interface Pointer, hresult	
OnDataChanged.ind	FSPM	Interface Pointer, Length, pBuffer	
OnDataChanged.cnf(+)	FSPM	Interface Pointer, hresult	

Table 934 (continued)

Primitive name	Source	Associated parameters	Functions
OnDataChanged.cnf(-)	FSPM	Interface Pointer, hresult	
ReadItems.ind	FSPM	Interface Pointer, Count, List of pReadItem	
ReadItems.cnf(+)	FSPM	Interface Pointer, hresult, List of ppReadItemOut	
ReadItems.cnf(-)	FSPM	Interface Pointer, hresult	
WriteItems.ind	FSPM	Interface Pointer, Count, List of pWriteItem	
WriteItems.cnf(+)	FSPM	Interface Pointer, hresult, List of ppError	
WriteItems.cnf(-)	FSPM	Interface Pointer, hresult	
WriteItemsQCD.ind	FSPM	Interface Pointer, Count, List of pWriteItemQcdIn	
WriteItemsQCD.cnf(+)	FSPM	Interface Pointer, hresult, List of ppError	
WriteItemsQCD.cnf(-)	FSPM	Interface Pointer, hresult	
CoCreateInstance.ind	FSPM	Host address, Class ID, Interface ID, Interface Pointer	
CoCreateInstance.cnf(+)	FSPM	hresult, Interface Pointer	
CoCreateInstance.cnf(-)	FSPM	hresult	
Call.ind	FSPM	Interface Pointer, Service Name, hresult	
Call.cnf(+)	FSPM	Interface Pointer, hresult, List of Unified Service Out-Parameter	
Call.cnf(-)	FSPM	Interface Pointer, hresult	

13.5.2.2 Parameters of FAL User/FSPM Primitives

The parameters used with the primitives exchanged between the FAL User and the FSPM are described in FAL Service Definition Type 10 Fieldbus IEC 61158-5.

13.5.2.3 FSPM States

The defined state of the FSPM together with the description is listed in Table 935.

Table 935 – FSPM state descriptions

State Name	Description
ACTIVE	The FSPM in the ACTIVE state is ready to transmit and receive service primitives to and from the ARPM and the FAL user.

The state transition diagram of the FSPM is shown in Figure 269.

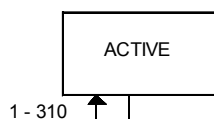


Figure 269 – State Transition Diagram of FSPM

13.5.2.4 FSPM State Table

The FSPM state transitions are specified in Table 936.

Table 936 – FSPM state table

#	Current State	Event /Condition =>Action	Next State
1	ACTIVE	QueryInterface.req(Interface Pointer, Interface ID) => Interface Pointer:= Interface Pointer Service Name:=QueryInterface List of Unified Service In-Parameter:=Interface ID ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
2	ACTIVE	QueryInterface.res(+)(Interface Pointer, hresult, ppvObject) => Interface Pointer:= Interface Pointer Service Name:=QueryInterface List of Unified Service Out-Parameter:=ppvObject hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
3	ACTIVE	QueryInterface.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=QueryInterface List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
4	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=QueryInterface => Interface Pointer:=Interface Pointer Interface ID:=List of Unified Service In-Parameter QueryInterface.ind(Interface Pointer, Interface ID)	ACTIVE
5	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=QueryInterface && hresult.S=Success => Interface Pointer:=Interface Pointer ppvObject:=List of Unified Service Out-Parameter hresult:=hresult QueryInterface.cnf(+)(Interface Pointer, hresult, ppvObject)	ACTIVE
6	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=QueryInterface && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult QueryInterface.cnf(-)(Interface Pointer, hresult)	ACTIVE
7	ACTIVE	AddRef.req(Interface Pointer) => Interface Pointer:= Interface Pointer Service Name:=AddRef List of Unified Service In-Parameter:="empty" ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
8	ACTIVE	AddRef.res(Interface Pointer, Reference Count) => Interface Pointer:= Interface Pointer Service Name:=AddRef List of Unified Service Out-Parameter:=Reference Count hresult:="empty" ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
9	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=AddRef => Interface Pointer:=Interface Pointer AddRef.ind(Interface Pointer)	ACTIVE
10	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=AddRef => Interface Pointer:=Interface Pointer Reference Count:=List of Unified Service Out-Parameter hresult:=hresult AddRef.cnf(Interface Pointer, Reference Count)	ACTIVE
11	ACTIVE	Release.req(Interface Pointer) => Interface Pointer:= Interface Pointer Service Name:=Release List of Unified Service In-Parameter:="empty" ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
12	ACTIVE	Release.res(Interface Pointer, Reference Count) => Interface Pointer:= Interface Pointer Service Name:=Release List of Unified Service Out-Parameter:=Reference Count hresult:="empty" ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
13	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=Release => Interface Pointer:=Interface Pointer Release.ind(Interface Pointer)	ACTIVE
14	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=Release => Interface Pointer:=Interface Pointer Reference Count:=List of Unified Service Out-Parameter hresult:=hresult Release.cnf(Interface Pointer, Reference Count)	ACTIVE
15	ACTIVE	GetTypeInfoCount.req(Interface Pointer) => Interface Pointer:= Interface Pointer Service Name:=GetTypeInfoCount List of Unified Service In-Parameter:="empty" ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
16	ACTIVE	GetTypeInfoCount.res(+)(Interface Pointer, hresult, pcTInfo) => Interface Pointer:= Interface Pointer Service Name:=GetTypeInfoCount List of Unified Service Out-Parameter:=pcTInfo hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
17	ACTIVE	GetTypeInfoCount.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=GetTypeInfoCount List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
18	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=GetTypeInfoCount => Interface Pointer:=Interface Pointer GetTypeInfoCount.ind(Interface Pointer)	ACTIVE
19	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=GetTypeInfoCount && hresult.S=Success => Interface Pointer:=Interface Pointer pcTInfo:=List of Unified Service Out-Parameter hresult:=hresult GetTypeInfoCount.cnf(+)(Interface Pointer, hresult, pcTInfo)	ACTIVE
20	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=GetTypeInfoCount && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult GetTypeInfoCount.cnf(-)(Interface Pointer, hresult)	ACTIVE
21	ACTIVE	GetTypeInfo.req(Interface Pointer, iTInfo, Icid) => Interface Pointer:= Interface Pointer Service Name:=GetTypeInfo List of Unified Service In-Parameter:=(iTInfo, Icid) ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
22	ACTIVE	GetTypeInfo.res(+)(Interface Pointer, hresult, ppTInfo) => Interface Pointer:= Interface Pointer Service Name:=GetTypeInfo List of Unified Service Out-Parameter:=ppTInfo hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
23	ACTIVE	GetTypeInfo.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=GetTypeInfo List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
24	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=GetTypeInfo => Interface Pointer:=Interface Pointer Icid:=List of Unified Service In-Parameter(Icid) iTInfo:=List of Unified Service In-Parameter(iTInfo) GetTypeInfo.ind(Interface Pointer, iTInfo, Icid)	ACTIVE
25	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=GetTypeInfo && hresult.S=Success => Interface Pointer:=Interface Pointer ppTInfo:=List of Unified Service Out-Parameter hresult:=hresult GetTypeInfo.cnf(+)(Interface Pointer, hresult, ppTInfo)	ACTIVE
26	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=GetTypeInfo && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult GetTypeInfo.cnf(-)(Interface Pointer, hresult)	ACTIVE
27	ACTIVE	GetIDsOfNames.req(Interface Pointer, riid, List of rgpszNames, cNames, Icid) => Interface Pointer:= Interface Pointer Service Name:=GetIDsOfNames List of Unified Service In-Parameter:=(riid, List of rgpszNames, cNames, Icid) ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
28	ACTIVE	GetIDsOfNames.res(+)(Interface Pointer, hresult, List of rgDispld) => Interface Pointer:= Interface Pointer Service Name:=GetIDsOfNames List of Unified Service Out-Parameter:=List of rgDispld hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
29	ACTIVE	GetIDsOfNames.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=GetIDsOfNames List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
30	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=GetIDsOfNames => Interface Pointer:=Interface Pointer riid:=List of Unified Service In-Parameter(riid) List of rgpszNames:=List of Unified Service In-Parameter(List of rgpszNames) cNames:=List of Unified Service In-Parameter(cNames) lcid:=List of Unified Service In-Parameter(lcid) GetIDsOfNames.ind(Interface Pointer, riid, List of rgpszNames, cNames, lcid)	ACTIVE
31	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=GetIDsOfNames && hresult.S=Success => Interface Pointer:=Interface Pointer List of rgDispId:=List of Unified Service Out-Parameter hresult:=hresult GetIDsOfNames.cnf(+)(Interface Pointer, hresult, List of rgDispId)	ACTIVE
32	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=GetIDsOfNames && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult GetIDsOfNames.cnf(-)(Interface Pointer, hresult)	ACTIVE
33	ACTIVE	Invoke.req(Interface Pointer, dispIdMember, riid, lcid, wFlags, pDispParams) => Interface Pointer:= Interface Pointer Service Name:=Invoke List of Unified Service In-Parameter:=(dispIdMember, riid, lcid, wFlags, pDispParams) ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
34	ACTIVE	Invoke.res(+)(Interface Pointer, hresult, pVarResult) => Interface Pointer:= Interface Pointer Service Name:=Invoke List of Unified Service Out-Parameter:=pVarResult hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
35	ACTIVE	Invoke.res(-)(Interface Pointer, hresult, pExcepInfo, puArgErr) => Interface Pointer:= Interface Pointer Service Name:=Invoke List of Unified Service Out-Parameter:=(pExcepInfo, puArgErr) hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
36	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=Invoke => Interface Pointer:=Interface Pointer riid:=List of Unified Service In-Parameter(riid) displdMember:=List of Unified Service In-Parameter(displdMember) Icid:=List of Unified Service In-Parameter(Icid) wFlags:=List of Unified Service In-Parameter(wFlags) pDispParams:=List of Unified Service In-Parameter(pDispParams) Invoke.ind(Interface Pointer, displdMember, riid, Icid, wFlags, pDispParams)	ACTIVE
37	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=Invoke && hresult.S=Success => Interface Pointer:=Interface Pointer pVarResult:=List of Unified Service Out-Parameter(pVarResult) hresult:=hresult Invoke.cnf(+)(Interface Pointer, hresult, pVarResult)	ACTIVE
38	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=Invoke && hresult.S<>DISP_E_EXCEPTION && hresult.S<>DISP_E_TYPERMISMATCH && hresult.S<>DISP_E_PARAMNOTFOUND => Interface Pointer:=Interface Pointer hresult:=hresult Invoke.cnf(-)(Interface Pointer, hresult)	ACTIVE
39	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=Invoke && (hresult.S=DISP_E_TYPERMISMATCH hresult.S=DISP_E_PARAMNOTFOUND) => Interface Pointer:=Interface Pointer puArgErr:=List of Unified Service Out-Parameter(puArgErr) hresult:=hresult Invoke.cnf(-)(Interface Pointer, hresult, puArgErr)	ACTIVE
40	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=Invoke && hresult.S=DISP_E_EXCEPTION => Interface Pointer:=Interface Pointer pExcepInfo:=List of Unified Service Out-Parameter(pExcepInfo) hresult:=hresult Invoke.cnf(-)(Interface Pointer, hresult, pExcepInfo)	ACTIVE
41	ACTIVE	get_Producer.req(Interface Pointer) => Interface Pointer:= Interface Pointer Service Name:=get_Producer List of Unified Service In-Parameter:="empty" ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
42	ACTIVE	get_Producer.res(+)(Interface Pointer, hresult, pVal) => Interface Pointer:= Interface Pointer Service Name:=get_Producer List of Unified Service Out-Parameter:=pVal hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
43	ACTIVE	get_Producer.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=get_Producer List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
44	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=get_Producer => Interface Pointer:=Interface Pointer get_Producer.ind(Interface Pointer)	ACTIVE
45	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=get_Producer && hresult.S=Success => Interface Pointer:=Interface Pointer pVal:=List of Unified Service Out-Parameter hresult:=hresult get_Producer.cnf(+)(Interface Pointer, hresult, pVal)	ACTIVE
46	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=get_Producer && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult get_Producer.cnf(-)(Interface Pointer, hresult)	ACTIVE
47	ACTIVE	get_Product.req(Interface Pointer) => Interface Pointer:= Interface Pointer Service Name:=get_Product List of Unified Service In-Parameter:="empty" ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
48	ACTIVE	get_Product.res(+)(Interface Pointer, hresult, pVal) => Interface Pointer:= Interface Pointer Service Name:=get_Product List of Unified Service Out-Parameter:=pVal hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
49	ACTIVE	get_Product.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=get_Product List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
50	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=get_Product => Interface Pointer:=Interface Pointer get_Product.ind(Interface Pointer)	ACTIVE
51	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=get_Product&& hresult.S=Success => Interface Pointer:=Interface Pointer pVal:=List of Unified Service Out-Parameter hresult:=hresult get_Product.cnf(+)(Interface Pointer, hresult, pVal)	ACTIVE
52	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=get_Product && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult get_Product.cnf(-)(Interface Pointer, hresult)	ACTIVE
53	ACTIVE	get_SerialNo.req(Interface Pointer) => Interface Pointer:= Interface Pointer Service Name:=get_SerialNo List of Unified Service In-Parameter:="empty" ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
54	ACTIVE	get_SerialNo.res(+)(Interface Pointer, hresult, pVal) => Interface Pointer:= Interface Pointer Service Name:=get_SerialNo List of Unified Service Out-Parameter:=pVal hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
55	ACTIVE	get_SerialNo.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=get_SerialNo List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
56	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=get_SerialNo => Interface Pointer:=Interface Pointer get_SerialNo.ind(Interface Pointer)	ACTIVE
57	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=get_SerialNo&& hresult.S=Success => Interface Pointer:=Interface Pointer pVal:=List of Unified Service Out-Parameter hresult:=hresult get_SerialNo.cnf(+)(Interface Pointer,hresult, pVal)	ACTIVE
58	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=get_SerialNo && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult get_SerialNo.cnf(-)(Interface Pointer, hresult)	ACTIVE
59	ACTIVE	get_ProductionDate.req(Interface Pointer) => Interface Pointer:= Interface Pointer Service Name:=get_Product List of Unified Service In-Parameter:="empty" ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
60	ACTIVE	get_ProductionDate.res(+)(Interface Pointer, hresult, pVal) => Interface Pointer:= Interface Pointer Service Name:=get_ProductionDate List of Unified Service Out-Parameter:=pVal hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
61	ACTIVE	get_ProductionDate.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=get_ProductionDate List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
62	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=get_ProductionDate => Interface Pointer:=Interface Pointer get_ProductionDate.ind(Interface Pointer)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
63	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=get_ProductionDate && hresult.S=Success => Interface Pointer:=Interface Pointer pVal:=List of Unified Service Out-Parameter hresult:=hresult get_ProductionDate.cnf(+)(Interface Pointer, hresult, pVal)	ACTIVE
64	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=get_ProductionDate && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult get_ProductionDate.cnf(-)(Interface Pointer, hresult)	ACTIVE
65	ACTIVE	Revision.req(Interface Pointer) => Interface Pointer:= Interface Pointer Service Name:=Revision List of Unified Service In-Parameter:="empty" ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
66	ACTIVE	Revision.res(+)(Interface Pointer, hresult, pMajor, pMinor) => Interface Pointer:= Interface Pointer Service Name:=Revision List of Unified Service Out-Parameter:=(pMajor, pMinor) hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
67	ACTIVE	Revision.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=Revision List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
68	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=Revision => Interface Pointer:=Interface Pointer Revision.ind(Interface Pointer)	ACTIVE
69	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=Revision && hresult.S=Success => Interface Pointer:=Interface Pointer pMajor:=List of Unified Service Out-Parameter(pMajor) pMinor:=List of Unified Service Out-Parameter(pMinor) hresult:=hresult Revision.cnf(+)(Interface Pointer, hresult, pMajor, pMinor)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
70	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=Revision && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult Revision.cnf(-)(Interface Pointer, hresult)	ACTIVE
71	ACTIVE	get_LogicalDevice.req(Interface Pointer, Name) => Interface Pointer:= Interface Pointer Service Name:=get_LogicalDevice List of Unified Service In-Parameter:=(Name) ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
72	ACTIVE	get_LogicalDevice.res(+)(Interface Pointer, hresult, ppLDev) => Interface Pointer:= Interface Pointer Service Name:=get_LogicalDevice List of Unified Service Out-Parameter:=ppLDev hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
73	ACTIVE	get_LogicalDevice.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=get_LogicalDevice List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
74	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=get_LogicalDevice => Interface Pointer:=Interface Pointer get_LogicalDevice.ind(Interface Pointer)	ACTIVE
75	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=get_LogicalDevice && hresult.S=Success => Interface Pointer:=Interface Pointer ppLDev:=List of Unified Service Out-Parameter hresult:=hresult get_LogicalDevice.cnf(+)(Interface Pointer, hresult, ppLDev)	ACTIVE
76	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=get_LogicalDevice && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult get_LogicalDevice.cnf(-)(Interface Pointer, hresult)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
77	ACTIVE	get_Count.req(Interface Pointer) => Interface Pointer:= Interface Pointer Service Name:=get_Count List of Unified Service In-Parameter:="empty" ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
78	ACTIVE	get_Count.res(+)(Interface Pointer, hresult, pVal) => Interface Pointer:= Interface Pointer Service Name:=get_Count List of Unified Service Out-Parameter:=pVal hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
79	ACTIVE	get_Count.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=get_Count List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
80	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=get_Count => Interface Pointer:=Interface Pointer get_Count.ind(Interface Pointer)	ACTIVE
81	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=get_Count && hresult.S=Success => Interface Pointer:=Interface Pointer pVal:=List of Unified Service Out-Parameter hresult:=hresult get_Count.cnf(+)(Interface Pointer, hresult, pVal)	ACTIVE
82	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=get_Count && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult get_Count.cnf(-)(Interface Pointer, hresult)	ACTIVE
83	ACTIVE	BrowseItems.req(Interface Pointer, Offset, MaxReturn) => Interface Pointer:= Interface Pointer Service Name:=BrowseItems List of Unified Service In-Parameter:=(Offset, MaxReturn) ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
84	ACTIVE	BrowseItems.res(+)(Interface Pointer, hresult, pltem, pDataType, pAccessRight) => Interface Pointer:= Interface Pointer Service Name:=BrowseItem List of Unified Service Out-Parameter:=(pltem, pDataType, pAccessRight) hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
85	ACTIVE	BrowseItems.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=BrowseItems List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
86	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=BrowseItems => Interface Pointer:=Interface Pointer Offset:=List of Unified Service In-Parameter(Offset) MaxReturn:=List of Unified Service In-Parameter(MaxReturn) BrowseItem.ind(Interface Pointer, Offset, MaxReturn)	ACTIVE
87	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=BrowseItem && hresult.S=Success => Interface Pointer:=Interface Pointer pltem:=List of Unified Service Out-Parameter(pltem) pDataType:=List of Unified Service Out-Parameter(pDatatype) pAccessRights:=List of Unified Service Out-Parameter(pAccessRights) hresult:=hresult BrowseItem.cnf(+)(Interface Pointer, hresult, pltem, pDataType, pAccessRight)	ACTIVE
88	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=BrowseItem && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult BrowseItem.cnf(-)(Interface Pointer, hresult)	ACTIVE
89	ACTIVE	Save.req(Interface Pointer) => Interface Pointer:= Interface Pointer Service Name:=Save List of Unified Service In-Parameter:="empty" ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
90	ACTIVE	Save.res(+)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=Save List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
91	ACTIVE	Save.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=Save List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
92	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=Save => Interface Pointer:=Interface Pointer Save.ind(Interface Pointer)	ACTIVE
93	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=Save && hresult.S=Success => Interface Pointer:=Interface Pointer hresult:=hresult Save.cnf(+)(Interface Pointer, hresult)	ACTIVE
94	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=Save && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult Save.cnf(-)(Interface Pointer, hresult)	ACTIVE
95	ACTIVE	get_Name.req(Interface Pointer) => Interface Pointer:= Interface Pointer Service Name:=get_Name List of Unified Service In-Parameter:="empty" ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
96	ACTIVE	get_Name.res(+)(Interface Pointer, hresult, pVal) => Interface Pointer:= Interface Pointer Service Name:=get_Name List of Unified Service Out-Parameter:=pVal hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
97	ACTIVE	get_Name.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=get_Name List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
98	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=get_Name => Interface Pointer:=Interface Pointer get_Name.ind(Interface Pointer)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
99	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=get_Name && hresult.S=Success => Interface Pointer:=Interface Pointer pVal:=List of Unified Service Out-Parameter hresult:=hresult get_Name.cnf(+)(Interface Pointer, hresult, pVal)	ACTIVE
100	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=get_Name && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult get_Name.cnf(-)(Interface Pointer, hresult)	ACTIVE
101	ACTIVE	get_ACCO.req(Interface Pointer) => Interface Pointer:= Interface Pointer Service Name:=get_ACCO List of Unified Service In-Parameter:="empty" ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
102	ACTIVE	get_ACCO.res(+)(Interface Pointer, hresult, ppACCO) => Interface Pointer:= Interface Pointer Service Name:=get_ACCO List of Unified Service Out-Parameter:=ppACCO hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
103	ACTIVE	get_ACCO.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=get_ACCO List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
104	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=get_ACCO => Interface Pointer:=Interface Pointer get_ACCO.ind(Interface Pointer)	ACTIVE
105	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=get_ACCO && hresult.S=Success => Interface Pointer:=Interface Pointer ppACCO:=List of Unified Service Out-Parameter hresult:=hresult get_ACCO.cnf(+)(Interface Pointer, hresult, ppACCO)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
106	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=get_ACCO && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult get_ACCO.cnf(-)(Interface Pointer, hresult)	ACTIVE
107	ACTIVE	get_RTAuto.req(Interface Pointer, Name) => Interface Pointer:= Interface Pointer Service Name:=get_RTAuto List of Unified Service In-Parameter:=Name ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
108	ACTIVE	get_RTAuto.res(+)(Interface Pointer, hresult, ppAuto) => Interface Pointer:= Interface Pointer Service Name:=get_RTAuto List of Unified Service Out-Parameter:=ppAuto hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
109	ACTIVE	get_RTAuto.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=get_RTAuto List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
110	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=get_RTAuto => Interface Pointer:=Interface Pointer Name:=List of Unified Service InParameter get_RTAuto.ind(Interface Pointer, Name)	ACTIVE
111	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=get_RTAuto && hresult.S=Success => Interface Pointer:=Interface Pointer ppAuto:=List of Unified Service Out-Parameter hresult:=hresult get_RTAuto.cnf(+)(Interface Pointer, hresult, ppAuto)	ACTIVE
112	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=get_RTAuto && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult get_RTAuto.cnf(-)(Interface Pointer, hresult)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
113	ACTIVE	get_State.req(Interface Pointer) => Interface Pointer:= Interface Pointer Service Name:=get_State List of Unified Service In-Parameter:="empty" ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
114	ACTIVE	get_State.res(+)(Interface Pointer, hresult, pVal) => Interface Pointer:= Interface Pointer Service Name:=get_State List of Unified Service Out-Parameter:=pVal hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
115	ACTIVE	get_State.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=get_State List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
116	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=get_State => Interface Pointer:=Interface Pointer get_State.ind(Interface Pointer)	ACTIVE
117	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=get_State && hresult.S=Success => Interface Pointer:=Interface Pointer pVal:=List of Unified Service Out-Parameter hresult:=hresult get_State.cnf(+)(Interface Pointer, hresult, pVal)	ACTIVE
118	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=get_State && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult get_State.cnf(-)(Interface Pointer, hresult)	ACTIVE
119	ACTIVE	Activate.req(Interface Pointer) => Interface Pointer:= Interface Pointer Service Name:=Activate List of Unified Service In-Parameter:="empty" ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
120	ACTIVE	Activate.res(+)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=Activate List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
121	ACTIVE	Activate.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=Activate List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
122	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=Activate => Interface Pointer:=Interface Pointer Activate.ind(Interface Pointer)	ACTIVE
123	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=Activate && hresult.S=Success => Interface Pointer:=Interface Pointer hresult:=hresult Activate.cnf(+)(Interface Pointer, hresult)	ACTIVE
124	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=Activate && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult Activate.cnf(-)(Interface Pointer, hresult)	ACTIVE
125	ACTIVE	Deactivate.req(Interface Pointer) => Interface Pointer:= Interface Pointer Service Name:=Deactivate List of Unified Service In-Parameter:="empty" ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
126	ACTIVE	Deactivate.res(+)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=Deactivate List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
127	ACTIVE	Deactivate.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=Deactivate List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
128	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=Deactivate => Interface Pointer:=Interface Pointer Deactivate.ind(Interface Pointer)	ACTIVE
129	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=Deactivate && hresult.S=Success => Interface Pointer:=Interface Pointer hresult:=hresult Deactivate.cnf(+)(Interface Pointer, hresult)	ACTIVE
130	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=Deactivate && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult Deactivate.cnf(-)(Interface Pointer, hresult)	ACTIVE
131	ACTIVE	Reset.req(Interface Pointer) => Interface Pointer:= Interface Pointer Service Name:=Reset List of Unified Service In-Parameter:="empty" ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
132	ACTIVE	Reset.res(+)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=Reset List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
133	ACTIVE	Reset.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=Reset List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
134	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=Reset => Interface Pointer:=Interface Pointer Reset.ind(Interface Pointer)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
135	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=Reset && hresult.S=Success => Interface Pointer:=Interface Pointer hresult:=hresult Reset.cnf(+)(Interface Pointer, hresult)	ACTIVE
136	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=Reset && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult Reset.cnf(-)(Interface Pointer, hresult)	ACTIVE
137	ACTIVE	AdviseState.req(Interface Pointer, pStateEvent) => Interface Pointer:= Interface Pointer Service Name:=AdviseState List of Unified Service In-Parameter:=pStateEvent ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
138	ACTIVE	AdviseState.res(+)(Interface Pointer, hresult, pCookie) => Interface Pointer:= Interface Pointer Service Name:=AdviseState List of Unified Service Out-Parameter:=pCookie hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
139	ACTIVE	AdviseState.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=AdviseState List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
140	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=AdviseState => Interface Pointer:=Interface Pointer pStateEvent:=List of Unified Service In-Parameter AdviseState.ind(Interface Pointer, pStateEvent)	ACTIVE
141	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=AdviseState && hresult.S=Success => Interface Pointer:=Interface Pointer pCookie:=List of Unified Service Out-Parameter hresult:=hresult AdviseState.cnf(+)(Interface Pointer, hresult, pCookie)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
142	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=AdviseState && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult AdviseState.cnf(-)(Interface Pointer, hresult)	ACTIVE
143	ACTIVE	UnadviseState.req(Interface Pointer, Cookie) => Interface Pointer:= Interface Pointer Service Name:=UnadviseState List of Unified Service In-Parameter:=Cookie ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
144	ACTIVE	UnadviseState.res(+)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=UnadviseState List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
145	ACTIVE	UnadviseState.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=UnadviseState List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
146	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=UnadviseState => Interface Pointer:=Interface Pointer Cookie:=List of Unified Service In-Parameter UnadviseState.ind(Interface Pointer, Cookie)	ACTIVE
147	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=UnadviseState && hresult.S=Success => Interface Pointer:=Interface Pointer hresult:=hresult UnadviseState.cnf(+)(Interface Pointer, hresult)	ACTIVE
148	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=UnadviseState && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult UnadviseState.cnf(-)(Interface Pointer, hresult)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
149	ACTIVE	get_Time.req(Interface Pointer) => Interface Pointer:= Interface Pointer Service Name:=get_Time List of Unified Service In-Parameter:="empty" ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
150	ACTIVE	get_Time.res(+)(Interface Pointer, hresult, pVal) => Interface Pointer:= Interface Pointer Service Name:=get_Time List of Unified Service Out-Parameter:=pVal hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
151	ACTIVE	get_Time.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=get_Time List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
152	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=get_Time => Interface Pointer:=Interface Pointer get_Time.ind(Interface Pointer)	ACTIVE
153	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=get_Time && hresult.S=Success => Interface Pointer:=Interface Pointer pVal:=List of Unified Service Out-Parameter hresult:=hresult get_Time.cnf(+)(Interface Pointer, hresult, pVal)	ACTIVE
154	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=get_Time && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult get_Time.cnf(-)(Interface Pointer, hresult)	ACTIVE
155	ACTIVE	put_Time.req(Interface Pointer, NewVal) => Interface Pointer:= Interface Pointer Service Name:=put_Time List of Unified Service In-Parameter:=NewVal ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
156	ACTIVE	put_Time.res(+)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=put_Time List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
157	ACTIVE	put_Time.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=put_Time List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
158	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=put_Time => Interface Pointer:=Interface Pointer NewVal:=List of Unified Service In-Parameter put_Time.ind(Interface Pointer, NewVal)	ACTIVE
159	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=put_Time && hresult.S=Success => Interface Pointer:=Interface Pointer hresult:=hresult put_Time.cnf(+)(Interface Pointer, hresult)	ACTIVE
160	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=put_Time && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult put_Time.cnf(-)(Interface Pointer, hresult)	ACTIVE
161	ACTIVE	GroupError.req(Interface Pointer) => Interface Pointer:= Interface Pointer Service Name:=GroupError List of Unified Service In-Parameter:="empty" ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
162	ACTIVE	GroupError.res(+)(Interface Pointer, hresult, pVal, pMagicCookie) => Interface Pointer:= Interface Pointer Service Name:=GroupError List of Unified Service Out-Parameter:=(pVal, pMagicCookie) hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
163	ACTIVE	GroupError.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=GroupError List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
164	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=GroupError => Interface Pointer:=Interface Pointer GroupError.ind(Interface Pointer)	ACTIVE
165	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=GroupError && hresult.S=Success => Interface Pointer:=Interface Pointer pVal:=List of Unified Service Out-Parameter(pVal) pMagicCookie:=List of Unified Service Out-Parameter(pMagicCookie) hresult:=hresult GroupError.cnf(+)(Interface Pointer, hresult, pVal, pMagicCookie)	ACTIVE
166	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=GroupError && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult GroupError.cnf(-)(Interface Pointer, hresult)	ACTIVE
167	ACTIVE	AdviseGroupError.req(Interface Pointer, pGroupErrorEvent) => Interface Pointer:= Interface Pointer Service Name:=AdviseGroupError List of Unified Service In-Parameter:=pGroupErrorEvent ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
168	ACTIVE	AdviseGroupError.res(+)(Interface Pointer, hresult, pCookie) => Interface Pointer:= Interface Pointer Service Name:=AdviseGroupError List of Unified Service Out-Parameter:=pCookie hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
169	ACTIVE	AdviseGroupError.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=AdviseGroupError List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
170	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=AdviseGroupError => Interface Pointer:=Interface Pointer pGroupErrorEvent:=List of Unified Service In-Parameter AdviseGroupError.ind(Interface Pointer, pGroupErrorEvent)	ACTIVE
171	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=AdviseGroupError && hresult.S=Success S_FALSE => Interface Pointer:=Interface Pointer pCookie:=List of Unified Service Out-Parameter hresult:=hresult AdviseGroupError.cnf(+)(Interface Pointer, hresult, pCookie)	ACTIVE
172	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=AdviseGroupError && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult AdviseGroupError.cnf(-)(Interface Pointer, hresult)	ACTIVE
173	ACTIVE	UnadviseGroupError.req(Interface Pointer, Cookie) => Interface Pointer:= Interface Pointer Service Name:=UnadviseGroupError List of Unified Service In-Parameter:=Cookie ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
174	ACTIVE	UnadviseGroupError.res(+)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=UnadviseGroupError List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
175	ACTIVE	UnadviseGroupError.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=UnadviseGroupError List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
176	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=UnadviseGroupError => Interface Pointer:=Interface Pointer Cookie:=List of Unified Service In-Parameter UnadviseGroupError.ind(Interface Pointer, Cookie)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
177	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=UnadviseGroupError && hresult.S=Success => Interface Pointer:=Interface Pointer hresult:=hresult UnadviseGroupError.cnf(+)(Interface Pointer, hresult)	ACTIVE
178	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=UnadviseGroupError && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult UnadviseGroupError.cnf(-)(Interface Pointer, hresult)	ACTIVE
179	ACTIVE	AddConnections.req(Interface Pointer, Provider, QoSType, QoSValue, State, Count, List of pAddConnectionIn) => Interface Pointer:= Interface Pointer Service Name:=AddConnections List of Unified Service In-Parameter:=(Provider, QoSType, QoSValue, State, Count, List of pAddConnectionIn) ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
180	ACTIVE	AddConnections.res(+)(Interface Pointer, hresult, List of ppAddConnectionOut) => Interface Pointer:= Interface Pointer Service Name:=AddConnections List of Unified Service Out-Parameter:=List of ppAddConnectionOut hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
181	ACTIVE	AddConnections.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=AddConnection List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
182	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=AddConnections => Interface Pointer:=Interface Pointer Provider:=List of Unified Service In-Parameter(Provider) QoSType:=List of Unified Service In-Parameter(QoSType) QoSValue:=List of Unified Service In-Parameter(QoSValue) State:=List of Unified Service In-Parameter(State) Count:=List of Unified Service In-Parameter(Count) pAddConnectionIn:=List of Unified Service In-Parameter(pAddConnectionIn) AddConnections.ind(Interface Pointer, Provider, QoSType, QoSValue, State, Count, List of pAddConnectionIn)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
183	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=AddConnections && hresult.S=Success => Interface Pointer:=Interface Pointer ppAddConnectionOut:=List of Unified Service Out-Parameter(ppAddConnectionOut) hresult:=hresult AddConnections.cnf(+)(Interface Pointer, hresult, List of ppAddConnectionOut)	ACTIVE
184	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=AddConnections && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult AddConnections.cnf(-)(Interface Pointer, hresult)	ACTIVE
185	ACTIVE	RemoveConnections.req(Interface Pointer, Count, List of pConsumerID) => Interface Pointer:= Interface Pointer Service Name:=RemoveConnections List of Unified Service In-Parameter:=(Count, List of pConsumerID) ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
186	ACTIVE	RemoveConnections.res(+)(Interface Pointer, hresult, List of ppError) => Interface Pointer:= Interface Pointer Service Name:=RemoveConnections List of Unified Service Out-Parameter:=List of ppError hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
187	ACTIVE	RemoveConnections.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=RemoveConnections List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
188	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=RemoveConnections => Interface Pointer:=Interface Pointer Count:=List of Unified Service In-Parameter(Count) pConsumerID:=List of Unified Service In-Parameter(List of pConsumerID) RemoveConnections.ind(Interface Pointer, Count, List of pConsumerID)	ACTIVE
189	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=RemoveConnections && hresult.S=Success => Interface Pointer:=Interface Pointer ppError:=List of Unified Service Out-Parameter(List of ppError) hresult:=hresult RemoveConnections.cnf(+)(Interface Pointer, hresult, List of ppError)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
190	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=RemoveConnections && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult RemoveConnections.cnf(-)(Interface Pointer, hresult)	ACTIVE
191	ACTIVE	ClearConnections.req(Interface Pointer) => Interface Pointer:= Interface Pointer Service Name:=ClearConnections List of Unified Service In-Parameter:="empty" ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
192	ACTIVE	ClearConnections.res(+)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=ClearConnections List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
193	ACTIVE	ClearConnections.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=ClearConnections List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
194	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=ClearConnections => Interface Pointer:=Interface Pointer ClearConnections.ind(Interface Pointer)	ACTIVE
195	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=ClearConnections && hresult.S=Success => Interface Pointer:=Interface Pointer hresult:=hresult ClearConnections.cnf(+)(Interface Pointer, hresult)	ACTIVE
196	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=ClearConnections && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult ClearConnections.cnf(-)(Interface Pointer, hresult)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
197	ACTIVE	SetActivationState.req(Interface Pointer, State, Count, List of pConsumerID) => Interface Pointer:= Interface Pointer Service Name:=SetActivationState List of Unified Service In-Parameter:=(State, Count, List of pConsumerID) ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
198	ACTIVE	SetActivationState.res(+)(Interface Pointer, hresult, List of ppError) => Interface Pointer:= Interface Pointer Service Name:=SetActivationState List of Unified Service Out-Parameter:=List of ppError hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
199	ACTIVE	SetActivationState.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=SetActivationState List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
200	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=SetActivationState => Interface Pointer:=Interface Pointer State:=List of Unified Service In-Parameter(State) Count:=List of Unified Service In-Parameter(Count) pConsumerID:=List of Unified Service In-Parameter(List of pConsumerID) SetActivationState.ind(Interface Pointer, State, Count, List of pConsumerID)	ACTIVE
201	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=SetActivationState && hresult.S=Success => Interface Pointer:=Interface Pointer ppError:=List of Unified Service Out-Parameter(List of ppError) hresult:=hresult SetActivationState.cnf(+)(Interface Pointer, hresult, List of ppError)	ACTIVE
202	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=AddConnections && hresult.S=Error => Interface Pointer:=Interface Pointer ppError:="undefined" hresult:=hresult SetActivationState.cnf(-)(Interface Pointer, hresult)	ACTIVE
203	ACTIVE	GetInfo.req(Interface Pointer) => Interface Pointer:= Interface Pointer Service Name:=GetInfo List of Unified Service In-Parameter:="empty" ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
204	ACTIVE	GetInfo.res(+)(Interface Pointer, hresult, pMax, pCurCnt) => Interface Pointer:= Interface Pointer Service Name:=GetInfo List of Unified Service Out-Parameter:=(pMax, pCurCnt) hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
205	ACTIVE	GetInfo.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=GetInfo List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
206	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=GetInfo => Interface Pointer:=Interface Pointer GetInfo.ind(Interface Pointer)	ACTIVE
207	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=GetInfo && hresult.S=Success => Interface Pointer:=Interface Pointer pMax:=List of Unified Service Out-Parameter(pMax) pCurCnt:=List of Unified Service Out-Parameter(pCurCnt) hresult:=hresult GetInfo.cnf(+)(Interface Pointer, hresult, pMax, pCurCnt)	ACTIVE
208	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=GetInfo && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult GetInfo.cnf(-)(Interface Pointer, hresult)	ACTIVE
209	ACTIVE	GetIDs.req(Interface Pointer) => Interface Pointer:= Interface Pointer Service Name:=GetIDs List of Unified Service In-Parameter:="empty" ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
210	ACTIVE	GetIDs.res(+)(Interface Pointer, hresult, pCount, List of ppGetIDOut) => Interface Pointer:= Interface Pointer Service Name:=GetIDs List of Unified Service Out-Parameter:=(pCount, List of ppGetIDOut) hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
211	ACTIVE	GetIDs.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=GetInfo List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
212	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=GetIDs => Interface Pointer:=Interface Pointer GetInfo.ind(Interface Pointer)	ACTIVE
213	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=GetIDs && hresult.S=Success => Interface Pointer:=Interface Pointer pCount:=List of Unified Service Out-Parameter(pCount) ppGetIDOut:=List of Unified Service Out-Parameter(List of ppGetIDOut) hresult:=hresult GetIDs.cnf(+)(Interface Pointer, hresult, pCount, List of ppGetIDOut)	ACTIVE
214	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=GetIDs && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult GetIDs.cnf(-)(Interface Pointer, hresult)	ACTIVE
215	ACTIVE	GetConnections.req(Interface Pointer, Count, List of pConsumerID) => Interface Pointer:= Interface Pointer Service Name:=GetConnections List of Unified Service In-Parameter:=(Count, List of pConsumerID) ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
216	ACTIVE	GetConnections.res(+)(Interface Pointer, hresult, List of ppGetConnectionOut) => Interface Pointer:= Interface Pointer Service Name:=GetConnections List of Unified Service Out-Parameter:=List of ppGetConnectionOut hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
217	ACTIVE	GetConnections.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=GetConnections List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
218	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=GetConnections => Interface Pointer:=Interface Pointer Count:=List of Unified Service In-Parameter(Count) pConsumerID:=List of Unified Service In-Parameter(List of pConsumerID) GetConnections.ind(Interface Pointer, Count, List of pConsumerID)	ACTIVE
219	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=GetConnections && hresult.S=Success => Interface Pointer:=Interface Pointer ppGetConnectionsOut:=List of Unified Service Out-Parameter(List of ppGetConnectionsOut) hresult:=hresult GetConnections.cnf(+)(Interface Pointer, hresult, List of ppGetConnectionsOut)	ACTIVE
220	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=GetConnections && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult GetConnections.cnf(-)(Interface Pointer, hresult)	ACTIVE
221	ACTIVE	ReviseQoS.req(Interface Pointer, RTAuto, QoSType, QoSValue) => Interface Pointer:= Interface Pointer Service Name:=ReviseQoS List of Unified Service In-Parameter:=(RTAuto, QoSType, QoSValue) ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
222	ACTIVE	ReviseQoS.res(+)(Interface Pointer, hresult, pRevisedQoSValue) => Interface Pointer:= Interface Pointer Service Name:=ReviseQoS List of Unified Service Out-Parameter:=ppRevisedQoSValue hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
223	ACTIVE	ReviseQoS.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=RevisedQoS List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
224	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=RevisedQoS => Interface Pointer:=Interface Pointer Count:=List of Unified Service In-Parameter(Count) pConsumerID:=List of Unified Service In-Parameter(pConsumerID) ReviseQoS.ind(Interface Pointer, RTAuto, QoSType, QoSValue)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
225	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=RevisedQoS && hresult.S=Success => Interface Pointer:=Interface Pointer pRevisedQoSValue:=List of Unified Service Out-Parameter(pRevisedQoSValue) hresult:=hresult RevisedQoS.cnf(+)(Interface Pointer, hresult, pRevisedQoSValue)	ACTIVE
226	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=RevisedQoS && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult RevisedQoS.cnf(-)(Interface Pointer, hresult)	ACTIVE
227	ACTIVE	get_PingFactor.req(Interface Pointer) => Interface Pointer:= Interface Pointer Service Name:=get_PingFactor List of Unified Service In-Parameter:="empty" ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
228	ACTIVE	get_PingFactor.res(+)(Interface Pointer, hresult, pVal) => Interface Pointer:= Interface Pointer Service Name:=get_PingFactor List of Unified Service Out-Parameter:=pVal hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
229	ACTIVE	get_PingFactor.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=get_PingFactor List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
230	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=get_PingFactor => Interface Pointer:=Interface Pointer get_PingFactor.ind(Interface Pointer)	ACTIVE
231	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=get_PingFactor && hresult.S=Success => Interface Pointer:=Interface Pointer pVal:=List of Unified Service Out-Parameter hresult:=hresult get_PingFactor.cnf(+)(Interface Pointer, hresult, pVal)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
232	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=get_PingFactor && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult get_PingFactor.cnf(-)(Interface Pointer, hresult)	ACTIVE
233	ACTIVE	put_PingFactor.req(Interface Pointer, NewVal) => Interface Pointer:= Interface Pointer Service Name:=put_PingFactor List of Unified Service In-Parameter:=NewVal ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
234	ACTIVE	put_PingFactor.res(+)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=put_PingFactor List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
235	ACTIVE	put_PingFactor.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=put_PingFactor List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
236	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=put_PingFactor => Interface Pointer:=Interface Pointer NewVal:=List of Unified Service In-Parameter put_PingFactor.ind(Interface Pointer, NewVal)	ACTIVE
237	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=put_PingFactor && hresult.S=Success => Interface Pointer:=Interface Pointer hresult:=hresult put_PingFactor.cnf(+)(Interface Pointer, hresult)	ACTIVE
238	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=put_PingFactor && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult put_PingFactor.cnf(-)(Interface Pointer, hresult)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
239	ACTIVE	get_CDBCcookie.req(Interface Pointer) => Interface Pointer:= Interface Pointer Service Name:=get_CDBCcookie List of Unified Service In-Parameter:="empty" ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
240	ACTIVE	get_CDBCcookie.res(+)(Interface Pointer, hresult, pVal) => Interface Pointer:= Interface Pointer Service Name:=get_CDBCcookie List of Unified Service Out-Parameter:=pVal hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
241	ACTIVE	get_CDBCcookie.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=get_CDBCcookie List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
242	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=get_CDBCcookie => Interface Pointer:=Interface Pointer get_CDBCcookie.ind(Interface Pointer)	ACTIVE
243	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=get_CDBCcookie && hresult.S=Success => Interface Pointer:=Interface Pointer pVal:=List of Unified Service Out-Parameter hresult:=hresult get_CDBCcookie.cnf(+)(Interface Pointer, hresult, pVal)	ACTIVE
244	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=get_CDBCcookie && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult get_CDBCcookie.cnf(-)(Interface Pointer, hresult)	ACTIVE
245	ACTIVE	Connect.req(Interface Pointer, Consumer, QoSType, QoSValue, State, pICBAccoCallBack, Count, List of pConnectIn) => Interface Pointer:= Interface Pointer Service Name:=Connect List of Unified Service In-Parameter:=(Consumer, QoSType, QoSValue, State, pICBAccoCallBack, Count, List of pConnectIn) ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
246	ACTIVE	Connect.res(+)(Interface Pointer, hresult, pFirstConnect, List of ppConnectOut) => Interface Pointer:= Interface Pointer Service Name:=Connect List of Unified Service Out-Parameter:=(pFirstConnect, List of ppConnectOut) hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
247	ACTIVE	Connect.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=Connect List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
248	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=Connect => Interface Pointer:=Interface Pointer Consumer:=List of Unified Service In-Parameter(Consumer) QoSType:=List of Unified Service In-Parameter(QoSType) QoSValue:=List of Unified Service In-Parameter(QoSValue) State:=List of Unified Service In-Parameter(State) pCBACallback:=List of Unified Service In-Parameter(pCBACallback) Count:=List of Unified Service In-Parameter(Count) List of pConnectIn:=List of Unified Service In-Parameter(List of pConnectIn) Connect.ind(Interface Pointer, Consumer, QoSType, QoSValue, State, pCBAAccoCallBack, Count, List of pConnectIn)	ACTIVE
249	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=Connect && hresult.S=Success => Interface Pointer:=Interface Pointer pFirstConnect:=List of Unified Service Out-Parameter(pFirstConnect) ppConnectOut:=List of Unified Service Out-Parameter(List of ppConnectOut) hresult:=hresult Connect.cnf(+)(Interface Pointer, hresult, pFirstConnect, List of ppConnectOut)	ACTIVE
250	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=Connect && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult Connect.cnf(-)(Interface Pointer, hresult)	ACTIVE
251	ACTIVE	Disconnect.req(Interface Pointer, Count, List of pProviderID) => Interface Pointer:= Interface Pointer Service Name:=Disconnect List of Unified Service In-Parameter:=(Count, List of pProviderID) ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
252	ACTIVE	Disconnect.res(+)(Interface Pointer, hresult, List of ppError) => Interface Pointer:= Interface Pointer Service Name:=Disconnect List of Unified Service Out-Parameter:=(List of ppError) hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
253	ACTIVE	Disconnect.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=Disconnect List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
254	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=Disconnect => Interface Pointer:=Interface Pointer Count:=List of Unified Service In-Parameter(Count) pProviderID:=List of Unified Service In-Parameter(List of pProviderID) Disconnect.ind(Interface Pointer, Count, List of pProviderID)	ACTIVE
255	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=Disconnect && hresult.S=Success => Interface Pointer:=Interface Pointer ppError:=List of Unified Service Out-Parameter(List of ppError) hresult:=hresult Disconnect.cnf(+)(Interface Pointer, hresult, List of ppError)	ACTIVE
256	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=Disconnect && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult Disconnect.cnf(-)(Interface Pointer, hresult)	ACTIVE
257	ACTIVE	DisconnectMe.req(Interface Pointer, Consumer) => Interface Pointer:= Interface Pointer Service Name:=DisconnectMe List of Unified Service In-Parameter:=Consumer ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
258	ACTIVE	DisconnectMe.res(+)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=DisconnectMe List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
259	ACTIVE	DisconnectMe.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=DisconnectMe List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
260	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=DisconnectMe => Interface Pointer:=Interface Pointer Consumer:=List of Unified Service In-Parameter(Consumer) DisconnectMe.ind(Interface Pointer, Consumer)	ACTIVE
261	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=DisconnectMe && hresult.S=Success => Interface Pointer:=Interface Pointer hresult:=hresult DisconnectMe.cnf(+)(Interface Pointer, hresult)	ACTIVE
262	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=DisconnectMe && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult DisconnectMe.cnf(-)(Interface Pointer, hresult)	ACTIVE
263	ACTIVE	SetActivation.req(Interface Pointer, State, Count, List of pProviderID) => Interface Pointer:= Interface Pointer Service Name:=SetActivation List of Unified Service In-Parameter:=(State, Count, List of pProviderID) ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
264	ACTIVE	SetActivation.res(+)(Interface Pointer, hresult, List of ppError) => Interface Pointer:= Interface Pointer Service Name:=SetActivation List of Unified Service Out-Parameter:=List of ppError hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
265	ACTIVE	SetActivation.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=SetActivation List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
266	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=SetActivation => Interface Pointer:=Interface Pointer State:=List of Unified Service In-Parameter(State) Count:=List of Unified Service In-Parameter(Count) pProviderID:=List of Unified Service In-Parameter(List of pProviderID) SetActivation.ind(Interface Pointer, State, Count, List of pProviderID)	ACTIVE
267	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=SetActivation && hresult.S=Success => Interface Pointer:=Interface Pointer ppError:=List of Unified Service Out-Parameter(ppError) hresult:=hresult SetActivation.cnf(+)(Interface Pointer, hresult, ppError)	ACTIVE
268	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=SetActivation && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult SetActivation.cnf(-)(Interface Pointer, hresult)	ACTIVE
269	ACTIVE	Ping.req(Interface Pointer, Consumer) => Interface Pointer:= Interface Pointer Service Name:=Ping List of Unified Service In-Parameter:=Consumer ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
270	ACTIVE	Ping.res(+)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=Ping List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
271	ACTIVE	Ping.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=Ping List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
272	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=Ping => Interface Pointer:=Interface Pointer Consumer:=List of Unified Service In-Parameter(Consumer) Ping.ind(Interface Pointer, Consumer)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
273	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=Ping && hresult.S=Success => Interface Pointer:=Interface Pointer hresult:=hresult Ping.cnf(+)(Interface Pointer, hresult)	ACTIVE
274	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=Ping && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult Ping.cnf(-)(Interface Pointer, hresult)	ACTIVE
275	ACTIVE	OnDataChanged.req(Interface Pointer, Length, pBuffer) => Interface Pointer:= Interface Pointer Service Name:=OnDataChanged List of Unified Service In-Parameter:=(Length, pBuffer) ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
276	ACTIVE	OnDataChanged.res(+)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=OnDataChanged List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
277	ACTIVE	OnDataChanged.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=OnDataChanged List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
278	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=OnDataChanged => Interface Pointer:=Interface Pointer Length:=List of Unified Service In-Parameter(Length) pBuffer:=List of Unified Service In-Parameter(pBuffer) OnDataChanged.ind(Interface Pointer, Length, pBuffer)	ACTIVE
279	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=OnDataChanged && hresult.S=Success => Interface Pointer:=Interface Pointer hresult:=hresult OnDataChanged.cnf(+)(Interface Pointer, hresult)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
280	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=OnDataChanged && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult OnDataChanged.cnf(-)(Interface Pointer, hresult)	ACTIVE
281	ACTIVE	ReadItems.req(Interface Pointer, Count, List of pReadItem) => Interface Pointer:= Interface Pointer Service Name:=ReadItems List of Unified Service In-Parameter:=(Count, List of pReadItem) ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
282	ACTIVE	ReadItems.res(+)(Interface Pointer, hresult, List of ppReadItemOut) => Interface Pointer:= Interface Pointer Service Name:=ReadItems List of Unified Service Out-Parameter:=List of ppReadItemOut hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
283	ACTIVE	ReadItems.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=ReadItems List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
284	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=ReadItems => Interface Pointer:=Interface Pointer Count:=List of Unified Service In-Parameter(Count) pReadItem:=List of Unified Service In-Parameter(List of pReadItem) ReadItems.ind(Interface Pointer, Count, pReadItem)	ACTIVE
285	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=ReadItems && hresult.S=Success => Interface Pointer:=Interface Pointer ppReadItemOut:=List of Unified Service Out-Parameter(ppReadItemOut) hresult:=hresult ReadItems.cnf(+)(Interface Pointer, hresult, List of ppReadItemOut)	ACTIVE
286	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=ReadItems && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult ReadItems.cnf(-)(Interface Pointer, hresult)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
287	ACTIVE	Writeltems.req(Interface Pointer, Count, List of pWriteltem) => Interface Pointer:= Interface Pointer Service Name:=Writeltems List of Unified Service In-Parameter:=(Count, List of pWriteltem) ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
288	ACTIVE	Writeltems.res(+)(Interface Pointer, hresult, List of ppError) => Interface Pointer:= Interface Pointer Service Name:=Writeltems List of Unified Service Out-Parameter:=List of ppError hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
289	ACTIVE	Writeltems.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=Writeltems List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
290	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=Writeltems => Interface Pointer:=Interface Pointer Count:=List of Unified Service In-Parameter(Count) pWriteltem:=List of Unified Service In-Parameter(List of pWriteltem) Writeltems.ind(Interface Pointer, Count, List of pWriteltem)	ACTIVE
291	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=Writeltems && hresult.S=Success => Interface Pointer:=Interface Pointer ppError:=List of Unified Service Out-Parameter(List of ppError) hresult:=hresult Writeltems.cnf(+)(Interface Pointer, hresult, List of ppError)	ACTIVE
292	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=Writeltems && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult Writeltems.cnf(-)(Interface Pointer, hresult)	ACTIVE
293	ACTIVE	WriteltemsQCD.req(Interface Pointer, Count, pReadItem, List of pWriteltemQCDIn) => Interface Pointer:= Interface Pointer Service Name:=WriteltemsQCD List of Unified Service In-Parameter:=(Count, List of pWriteltemQCDIn) ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
294	ACTIVE	WriteItemsQCD.res(+)(Interface Pointer, hresult, List of ppError) => Interface Pointer:= Interface Pointer Service Name:=WriteItemsQCD List of Unified Service Out-Parameter:=List of ppError hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
295	ACTIVE	WriteItemsQCD.res(-)(Interface Pointer, hresult) => Interface Pointer:= Interface Pointer Service Name:=ReadItemsQCD List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
296	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=WriteItemsQCD => Interface Pointer:=Interface Pointer Count:=List of Unified Service In-Parameter(Count) pWriteItemQCdIn:=List of Unified Service In-Parameter(List of pWriteItemQCdIn) WriteItemsQCD.ind(Interface Pointer, Count, List of pWriteItemQCdIn)	ACTIVE
297	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=WriteItemsQCD && hresult.S=Success => Interface Pointer:=Interface Pointer ppError:=List of Unified Service Out-Parameter(List of ppError) hresult:=hresult WriteItemsQCD.cnf(+)(Interface Pointer, hresult, List of ppError)	ACTIVE
298	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=WriteItemsQCD && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult WriteItemsQCD.cnf(-)(Interface Pointer, hresult)	ACTIVE
299	ACTIVE	Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter) => Interface Pointer:= Interface Pointer Service Name:=ARPM_Call List of Unified Service In-Parameter:=List of Unified Service In-Parameter ARPM_Call.req(Interface Pointer, Service Name, List of Unified Service In-Parameter)	ACTIVE
300	ACTIVE	Call.res(+)(Interface Pointer, Service Name, hresult, List of Unified Service Out-Parameter) => Interface Pointer:= Interface Pointer Service Name:=ARPM_Call List of Unified Service Out-Parameter:=List of Unified Service Out-Parameter hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
301	ACTIVE	Call.res(-)(Interface Pointer, Service Name, hresult) => Interface Pointer:= Interface Pointer Service Name:=ARPM_Call List of Unified Service Out-Parameter:="empty" hresult:=hresult ARPM_Call.res(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult)	ACTIVE
302	ACTIVE	ARPM_Call.ind(Interface Pointer, Service Name, List of Unified Service In-Parameter) /Service Name=ARPM_Call => Interface Pointer:=Interface Pointer Count:=List of Unified Service In-Parameter(Count) pWriteltemQCDIn:=List of Unified Service In-Parameter(pWriteltemQCDIn) Call.ind(Interface Pointer, List of Unified Service In-Parameter)	ACTIVE
303	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=ARPM_Call && hresult.S=Success => Interface Pointer:=Interface Pointer ppError:=List of Unified Service Out-Parameter(ppError) hresult:=hresult Call.cnf(+)(Interface Pointer, hresult, List of Unified Service Out-Parameter)	ACTIVE
304	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, List of Unified Service Out-Parameter, hresult) /Service Name=ARPM_Call && hresult.S=Error => Interface Pointer:=Interface Pointer hresult:=hresult Call.cnf(-)(Interface Pointer, hresult)	ACTIVE
305	ACTIVE	CoCreateInstance.req(Host address, Class ID, Interface ID, Interface Pointer) => Host address := Host address Interface Pointer:=Interface Pointer Class ID := Class ID Interface ID := Interface ID ARPM_CoCreateInstance.req(Host address, Class ID, Interface ID, Interface Pointer)	ACTIVE
306	ACTIVE	CoCreateInstance.res(+)(hresult, Interface Pointer) => Interface Pointer:= Interface Pointer hresult:=hresult ARPM_CoCreateInstance.res(+)(hresult, Interface Pointer)	ACTIVE
307	ACTIVE	CoCreateInstance.res(-)(hresult) => hresult:=hresult ARPM_CoCreateInstance.res(-)(hresult)	ACTIVE
308	ACTIVE	ARPM_CoCreateInstance.ind(Host address, Class ID, Interface ID, Interface Pointer) => Host address := Host address Interface Pointer:=Interface Pointer Class ID := Class ID Interface ID := Interface ID CoCreateInstance.ind(Host address, Class ID, Interface ID, Interface Pointer)	ACTIVE

Table 936 (continued)

#	Current State	Event /Condition =>Action	Next State
309	ACTIVE	ARPM_CoCreateInstance.cnf(+)(hresult, Interface Pointer) /hresult.S=Success => Interface Pointer:= Interface Pointer hresult:=hresult ARPM_CoCreateInstance.cnf(+)(hresult, Interface Pointer)	ACTIVE
310	ACTIVE	ARPM_CoCreateInstance.cnf(-)(hresult) /hresult.S=Error => hresult:=hresult ARPM_CoCreateInstance.cnf(-)(hresult)	ACTIVE

13.6 Application Relationship Protocol Machine (ARPM)

13.6.1 Overview

The type 10 specifies one ARPM class referred to as ORPC ARPM. Therefore, one instance of the state machine is specified and the service attribute AREP to address this state machine is omitted.

NOTE Additional ARPM machines can be added in future editions.

The class formal model is specified in IEC 61158-5 type 10.

13.6.2 Primitive Definitions

13.6.2.1 Primitives Exchanged between FSPM and ARPM

The internal service primitives issued by FSPM to ARPM are specified in Table 937.

Table 937 – Primitives issued by FSPM to ARPM

Primitive Names	Source	Associated Parameters	Functions
ARPM_Call.req	FSPM	Interface Pointer, Service Name, Unified Service In-Parameter	This primitive is used to request the ARPM to transfer a FAL service request.
ARPM_Call.cnf	FSPM	Interface Pointer, Service Name, Unified Service Out-Parameter, hresult	This primitive is used to request the ARPM to transfer a FAL service response.
ARPM_CoCreateInstance.req	FSPM	Host address, Class ID, Interface ID	This primitive is used to request the ARPM to transfer the FAL CoCreateInstance service request.
ARPM_CoCreateInstance.cnf	FSPM	Interface Pointer, hresult	This primitive is used to request the ARPM to transfer the FAL CoCreateInstance service response.

The internal service primitives issued by ARPM to FSPM are specified in Table 938.

Table 938 – Primitives issued by ARPM to FSPM

Primitive Names	Source	Associated Parameters	Functions
ARPM_Call.ind	ARPM	Interface Pointer, Service Name, Unified Service In-Parameter	This primitive is used to indicate the FSPM the reception of an ARPM Call.
ARPM_Call.cnf	ARPM	Interface Pointer, Service Name, Unified Service Out-Parameter, hresult	This primitive is used to indicate the FSPM the reception of an ARPM Call response.
ARPM_CoCreateInstance.ind	ARPM	Host address, Class ID, Interface ID	This primitive is used to indicate the FSPM the reception of an ARPM CoCreateInstance.
ARPM_CoCreateInstance.cnf	ARPM	Interface Pointer, hresult	This primitive is used to indicate the FSPM the reception of an ARPM CoCreateInstance response.

13.6.2.2 Parameters of FSPM/ARPM Primitives

The parameters used with the primitives exchanged between the FSPM and the ARPM are described in Table 939.

Table 939 – Parameters used with primitives exchanged between FSPM and ARPM

Parameter Name	Description
Interface Pointer	This parameter specifies the interface address that uniquely represents the associated interface instance.
Service Name	This parameter specifies the service name of the FAL service.
List of Unified Service In-Parameter	This parameter summarizes all FAL service parameters specified in the related IDL with the [in] attribute in one unified container.
List of Unified Service Out-Parameter	This parameter summarizes all FAL service parameters specified in the related IDL with the [out] attribute in one unified container.
Host address	This parameter contains the IP address or DNS name of the node hosting the object to create.
Class ID	This parameter contains the Class ID of the object to create according to its FAL class specification and IDL specification.
Interface ID	This parameter contains the Interface ID of the interface of the created object. It is set according to its FAL class specification and IDL specification.
hresult	This parameter contains the result code.

13.6.2.3 ARPM States

The defined state of the ARPM together with the description is listed in Table 940.

Table 940 – ARPM state descriptions

State Name	Description
ACTIVE	The ARPM in the ACTIVE state is ready to transmit and receive service primitives to and from the DMPM and the FSPM.

The state transition diagram of the ARPM is shown in Figure 270.

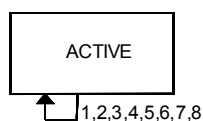


Figure 270 – State Transition Diagram of ARPM

13.6.2.4 ARPM State Table

The ARPM state transitions are specified in Table 941.

Table 941 – ARPM state table

#	Current State	Event /Condition =>Action	Next State
1	ACTIVE	ARPM_Call.req(Interface Pointer, Service Name, Unified Service In-Parameter) => pInterface:= Interface Pointer serviceName:=Service Name unifiedInParameter:=Unified Service In-Parameter referenceToIdl:=IDL Marshaling Reference DMPM_Call.req(pInterface, serviceName, unifiedInParameter, referenceToIdl)	ACTIVE
2	ACTIVE	ARPM_Call.cnf(Interface Pointer, Service Name, Unified Service Out-Parameter, hresult) => pInterface:= Interface Pointer serviceName:=Service Name unifiedOutParameter:=Unified Service Out-Parameter referenceToIdl:=IDL Marshaling Reference hresult:=hresult DMPM_Call.cnf(pInterface, serviceName, unifiedOutParameter, referenceToIdl, hresult)	ACTIVE
3	ACTIVE	ARPM_CoCreateInstance.req(Host address, Class ID, Interface ID) => hostAddress:=Host address classID:=Class ID interfaceID:=Interface ID DMPM_Create.req(hostAddress, classID, interfaceID)	ACTIVE
4	ACTIVE	ARPM_CoCreateInstance.cnf(Interface Pointer, hresult) => pInterface:= Interface Pointer hresult:=hresult DMPM_Create.cnf(pInterface, hresult)	ACTIVE
5	ACTIVE	DMPM_Call.ind(pInterface, serviceName, unifiedInParameter) => Interface Pointer:=pInterface Service Name:=serviceName Unified Service In-Parameter:=unifiedInParameter ARPM_Call.ind(Interface Pointer, Service Name, Unified Service In-Parameter)	ACTIVE
6	ACTIVE	DMPM_Call.cnf(pInterface, serviceName, unifiedOutParameter, hresult) => Interface Pointer:=pInterface Service Name:=serviceName Unified Service Out-Parameter:=unifiedOutParameter hresult:=hresult ARPM_Call.cnf(Interface Pointer, Service Name, Unified Service Out-Parameter, hresult)	ACTIVE
7	ACTIVE	DMPM_Create.ind(hostAddress, classID, interfaceID) => Host address:=hostAddress Class ID:=classID Interface ID:=interfaceID ARPM_CoCreateInstance.ind(Host address, Class ID, Interface ID)	ACTIVE
8	ACTIVE	DMPM_Create.cnf(pInterface, hresult) => Interface Pointer:=pInterface hresult:=hresult ARPM_CoCreateInstance.cnf(Interface Interface, hresult)	ACTIVE

13.7 DLL Mapping Protocol Machine (DMPM)

13.7.1 Overview

The DLL mapping is represented by the ORPC (object oriented remote procedure call) model.

The ORPC model defines an abstract service interface that permits the underlying layer to exist at the data link layer, network layer, or transport layer. It permits, for example, using existing implementations of an object oriented remote procedure.

13.7.2 Primitive Definitions

13.7.2.1 Primitives Exchanged between DMPM and ARPM

The internal service primitives issued by ARPM to DMPM are specified in Table 942.

Table 942 – Primitives issued by ARPM to DMPM

Primitive Names	Source	Associated Parameters	Functions
DMPM_Call.req	ARPM	pInterface, serviceName, unifiedInParameter, referenceToldI	This primitive is used to request the DMPM to transfer an ORPC call.
DMPM_Call.cnf	ARPM	pInterface, serviceName, unifiedOutParameter, referenceToldI hresult	This primitive is used to request the DMPM to transfer an ORPC response.
DMPM_Create.req	ARPM	hostAddress, classID, interfaceID	This primitive is used to request the DMPM to transfer an ORPC create.
DMPM_Create.cnf	ARPM	pInterface, hresult	This primitive is used to request the DMPM to transfer an ORPC create response.

The internal service primitives issued by DMPM to ARPM are specified in Table 943.

Table 943 – Primitives issued by DMPM to ARPM

Primitive Names	Source	Associated Parameters	Functions
DMPM_Call.ind	DMPM	pInterface, serviceName, unifiedInParameter	This primitive is used to indicate the ARPM the transfer of an ORPC call.
DMPM_Call.cnf	DMPM	pInterface, serviceName, unifiedOutParameter, hresult	This primitive is used to indicate the ARPM the transfer of an ORPC response.
DMPM_Create.ind	DMPM	hostAddress, classID, interfaceID	This primitive is used to indicate the DMPM the transfer of an ORPC create.
DMPM_Create.cnf	DMPM	pInterface, hresult	This primitive is used to indicate the DMPM the transfer of an ORPC create response.

13.7.2.2 Parameters of ARPM/DMPM Primitives

The parameters used with the primitives exchanged between the ARPM and the DMPM are described in Table 944.

Table 944 – Parameters used with primitives exchanged between ARPM and DMPM

Parameter Name	Description
pInterface	This parameter specifies the interface address that uniquely represents the associated interface instance.
serviceName	This parameter specifies the service name of the FAL service.
unifiedInParameter	This parameter summarizes all FAL service parameters specified in the related IDL with the [in] attribute in one unified container.
unifiedOutParameter	This parameter summarizes all FAL service parameters specified in the related IDL with the [out] attribute in one unified container.
referenceToIdl	This parameter contains a reference to the IDL specification that shall be used by the abstract ORPC for data marshaling of the unified in and out parameter.
hostAddress	This parameter contains the IP address or DNS name of the node hosting the object to create.
classID	This parameter contains the Class ID of the object to create according to its FAL class specification and IDL specification.
interfaceID	This parameter contains the Interface ID of the object to create according to its FAL class specification and IDL specification.
hresult	This parameter contains the result code.

13.7.2.3 Primitives Exchanged between the ORPC model and DMPM

The internal service primitives issued by DMPM to ORPC model are specified in Table 945.

Table 945 – Primitives issued by DMPM to ORPC model

Primitive Names	Source	Associated Parameters	Functions
ORPC_Send.req	DMPM	pInterface, serviceName, unifiedInParameter, referenceToIdl	This primitive is used to request the ORPC model to transfer an ORPC call.
ORPC_Send.cnf	DMPM	pInterface, serviceName, unifiedOutParameter, referenceToIdl, hresult	This primitive is used to request the ORPC model to transfer an ORPC response.
ORPC_Create.req	DMPM	hostAddress, classID, interfaceID	This primitive is used to request the ORPC model to transfer an ORPC call.
ORPC_Create.cnf	DMPM	pInterface, hresult	This primitive is used to request the ORPC model to transfer an ORPC response.

The internal service primitives issued by the ORPC model to DMPM are specified in Table 946.

Table 946 – Primitives issued by ORPC model to DMPM

Primitive Names	Source	Associated Parameters	Functions
ORPC_Send.ind	ORPC model	plInterface, serviceName, unifiedInParameter	This primitive is used to indicate the DMPM the transfer of an ORPC call.
ORPC_Send.cnf	ORPC model	plInterface, serviceName, unifiedOutParameter, hresult	This primitive is used to indicate the DMPM the transfer of an ORPC response.
ORPC_Create.ind	ORPC model	hostAddress, classID, interfaceID	This primitive is used to indicate the DMPM the transfer of an ORPC create.
ORPC_Create.cnf	ORPC model	plInterface, hresult	This primitive is used to indicate the DMPM the transfer of an ORPC create response.

13.7.2.4 Parameters of DMPM/ORPC Model Primitives

The parameters used with the primitives exchanged between the DMPM and the ORPC model are specified in Table 947.

Table 947 – Parameters used with primitives exchanged between DMPM and ORPC model

Parameter Name	Description
plInterface	This parameter specifies the interface address that uniquely represents the associated interface instance.
serviceName	This parameter specifies the service name of the FAL service.
unifiedInParameter	This parameter summarizes all FAL service parameters specified in the related IDL with the [in] attribute in one unified container.
unifiedOutParameter	This parameter summarizes all FAL service parameters specified in the related IDL with the [out] attribute in one unified container.
referenceToIdl	This parameter contains a reference to the IDL specification that shall be used by the abstract ORPC for data marshaling of the unified in and out parameter.
hostAddress	This parameter contains the IP address or DNS name of the node hosting the object to create.
classID	This parameter contains the Class ID of the object to create according to its FAL class specification and IDL specification.
interfaceID	This parameter contains the Interface ID of the object to create according to its FAL class specification and IDL specification.
hresult	This parameter contains the result code.

13.7.2.5 DMPM States

The defined state of the DMPM together with the description is listed in Table 948.

Table 948 – DMPM state descriptions

State Name	Description
ACTIVE	The DMPM in the ACTIVE state is ready to transmit and receive service primitives to and from the ORPC model and the ARPM.

The state transition diagram of the DMPM is shown in Figure 271.

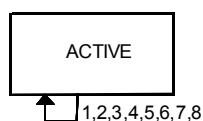


Figure 271 – State Transition Diagram of DMPM

13.7.2.6 DMPM State Table

The DMPM state transitions are specified in Table 949.

Table 949 – DMPM state table

#	Current State	Event /Condition =>Action	Next State
1	ACTIVE	DMPM_Call.req(pInterface, serviceName, unifiedInParameter, referenceToldl) => ORPC_Send.req(pInterface, serviceName, unifiedInParameter, referenceToldl)	ACTIVE
2	ACTIVE	DMPM_Call.cnf(pInterface, serviceName, unifiedOutParameter, referenceToldl, hresult) => ORPC_Send.cnf(pInterface, serviceName, unifiedOutParameter, referenceToldl, hresult)	ACTIVE
3	ACTIVE	DMPM_Create.req(hostAddress, classID, interfaceID) => ORPC_Create.req(hostAddress, classID, interfaceID)	ACTIVE
4	ACTIVE	DMPM_Create.cnf(pInterface, hresult) => ORPC_Create.cnf(pInterface, hresult)	ACTIVE
5	ACTIVE	ORPC_Send.ind(pInterface, serviceName, unifiedInParameter) => DMPM_Call.ind(pInterface, serviceName, unifiedInParameter)	ACTIVE
6	ACTIVE	ORPC_Send.cnf(pInterface, serviceName, unifiedOutParameter, hresult) => DMPM_Call.cnf(pInterface, serviceName, unifiedOutParameter, hresult)	ACTIVE
7	ACTIVE	ORPC_Create.ind(hostAddress, classID, interfaceID) => DMPM_Create.ind(hostAddress, classID, interfaceID)	ACTIVE
8	ACTIVE	ORPC_Create.cnf(pInterface, hresult) => DMPM_Create.cnf(pInterface, hresult)	ACTIVE

13.8 Protocol Options

Type 10 has no protocol options.



Standards Survey

The IEC would like to offer you the best quality standards possible. To make sure that we continue to meet your needs, your feedback is essential. Would you please take a minute to answer the questions overleaf and fax them to us at +41 22 919 03 00 or mail them to the address below. Thank you!

Customer Service Centre (CSC)

International Electrotechnical Commission

3, rue de Varembe
1211 Genève 20
Switzerland

or

Fax to: **IEC/CSC** at +41 22 919 03 00

Thank you for your contribution to the standards-making process.

A Prioritaire

Nicht frankieren
Ne pas affranchir



Non affrancare
No stamp required

RÉPONSE PAYÉE

SUISSE

Customer Service Centre (CSC)
International Electrotechnical Commission
3, rue de Varembe
1211 GENEVA 20
Switzerland



Q1 Please report on **ONE STANDARD** and **ONE STANDARD ONLY**. Enter the exact number of the standard: (e.g. 60601-1-1)

.....

Q2 Please tell us in what capacity(ies) you bought the standard (tick all that apply). I am the/a:

- purchasing agent ☐
- librarian ☐
- researcher ☐
- design engineer ☐
- safety engineer ☐
- testing engineer ☐
- marketing specialist ☐
- other

Q3 I work for/in/as a:
(tick all that apply)

- manufacturing ☐
- consultant ☐
- government ☐
- test/certification facility ☐
- public utility ☐
- education ☐
- military ☐
- other

Q4 This standard will be used for:
(tick all that apply)

- general reference ☐
- product research ☐
- product design/development ☐
- specifications ☐
- tenders ☐
- quality assessment ☐
- certification ☐
- technical documentation ☐
- thesis ☐
- manufacturing ☐
- other

Q5 This standard meets my needs:
(tick one)

- not at all ☐
- nearly ☐
- fairly well ☐
- exactly ☐

Q6 If you ticked NOT AT ALL in Question 5 the reason is: (tick all that apply)

- standard is out of date ☐
- standard is incomplete ☐
- standard is too academic ☐
- standard is too superficial ☐
- title is misleading ☐
- I made the wrong choice ☐
- other

Q7 Please assess the standard in the following categories, using the numbers:

- (1) unacceptable,
- (2) below average,
- (3) average,
- (4) above average,
- (5) exceptional,
- (6) not applicable

- timeliness
- quality of writing.....
- technical contents.....
- logic of arrangement of contents
- tables, charts, graphs, figures.....
- other

Q8 I read/use the: (tick one)

- French text only ☐
- English text only ☐
- both English and French texts ☐

Q9 Please share any comment on any aspect of the IEC that you would like us to know:

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....



ISBN 2-8318-6974-9



9 782831 869742

ICS 25.040; 35.100; 35.240.50
